Catálogo de ideias

Gatilho → ideia → técnica → código, para programação competitiva e ICPC

Partes: mapa rápido · frequências em CF · catálogo completo · índice da lib
Blocos marcados "adicionado … não revisado" foram gerados automaticamente e ainda não foram conferidos.

Sumário

Mapa rápido
- Reduções
- Limites escondidos
- Por tópico

Frequências em CF (A–D)
- Ranking geral
- O que tentar primeiro, por letra
- Por divisão
- Leituras
- Exemplos por técnica

Catálogo completo
- Como usar quando travar
- Leia as constraints
- Reformulações universais
- Cadeias de pulos (gatilho → observação → técnica)
- Arrays e guloso
- Busca binária e otimização
- DP
- Teoria dos números e divisores
- Combinatória, contagem e probabilidade
- Bits e XOR
- Strings
- Grafos
- Árvores
- Estruturas de dados e queries
- Construtivos e interativos
- Teoria dos jogos
- Geometria
- Aleatorização
- Álgebra, polinômios e FFT
- Matemática de olimpíada e math.stackexchange
- Otimização, programação linear e matroides
- Amostras de problemas e padrões de ICPC
- Quando nada funciona

Mapa rápido

Uma linha por salto: gatilho → técnica. O porquê está em `catalogo-completo.md`.

Reduções

Universais

- $\Sigma_i \Sigma_j$ difícil → troque a ordem dos fors; contribuição de cada elemento
- $\Sigma\, a_i \to \Sigma_t \#\{a_i \geq t\}$ (fatiar por camadas); $E[X] = \Sigma\, P(X \geq k)$
- "Pelo menos um" → complemento; vários defeitos → inclusão-exclusão
- Remoções → adições ao contrário (DSU)
- Min do max / max do min → binária na resposta
- "Exatamente k" com custo convexo → Aliens (λ)
- Operação reversível → forma canônica dos dois lados
- "Sempre existe" → pombal / extremo / invariante viram a construção
- Min difícil → dual (corte = fluxo, König, Dilworth)
- Atingíveis mudam de 2 em 2 → só mín, máx e paridade
- Operação em todos menos um → em todos + desfaz um

Arrays e guloso

- Soma = 0 → prefixos iguais; média ≥ k → subtraia k; mediana ≥ x → ±1
- Op. em intervalo → 2 pontos no array de diferenças
- Ordem ótima de tarefas → exchange argument (t/w, prazo); jobs com prazo e lucro → heap por prazo; código de Huffman → heap
- Estritamente crescente → $a_i - i$ não decrescente
- Minimizar $\Sigma|a_i - b_i|$ com b monótono → slope trick (heap)
- Escolher k não adjacentes → heap + lista ligada (arrependimento)
- Trocas em permutação → ciclos (n − #ciclos) / inversões (adjacentes)
- Pares i < j com condição → ordene + BIT; 3 dimensões → CDQ
- Cíclico → duplique; n ≤ 40 → meet in the middle
- Cobertura mínima por crescentes → maior não crescente (Dilworth)
- Intervalos ocupados → olhe as k + 1 lacunas
- Prefixo vira mín / sufixo vira máx → sobrevivem os recordes
- Equilibrar por transferências → Σ excessos
- $a_i + a_{i+1}$ constante → período 2
- $f(i) = \Sigma\, w \cdot |i - x|$ dado → 2ª diferença = 2w

Busca

- k-ésimo de conjunto implícito → binária + contar ≤ x
- Razão máxima Σa/Σb → binária em r ou Dinkelbach
- Primeiro índice com prefixo ≥ x → descida em BIT/segtree
- Primeiro instante em que conecta → Kruskal reconstruction ou binária paralela
- Binária sobre racionais / fração mais simples → Stern–Brocot

DP

- Valor grande, resposta pequena → troque estado e valor (dp[resposta] = melhor parâmetro)
- $dp[i] = \min_j (dp[j] + (S_i - S_j)^2)$ → CHT / Li Chao
- opt monótono (Monge) → D&C DP; intervalo → Knuth
- Transição linear com n enorme → matriz / Kitamasa; recorrência desconhecida → Berlekamp–Massey
- dp(n) polinomial em n → Lagrange com d+1 pontos
- Árvore + escolher k → knapsack $O(n^2)$ por tamanho ou ordem DFS pegar/pular
- Estado = profundidade em árvore → long-path decomposition
- Permutação com restrição de adjacência → connected components DP / insertion DP
- Esperança com ciclos → sistema linear ou $E[i] = a_i \cdot E[0] + b_i$
- Grid estreito → broken profile; com conectividade → plug DP; n ≤ 20 → máscara / SOS / IE + SOS
- Custos ≥ 0 com ciclos nos estados → Dijkstra em vez de DP
- Mochila com remoção ou itens com validade → dp inversa (modular) ou segtree no tempo

Números

- Pares com gcd = g, $\Sigma\, \gcd(i, j)$ → Möbius/φ + blocos de $\lfloor n/d \rfloor$
- Soma de multiplicativa até 10^{11} → Dirichlet sieve / Lucy / Min_25
- $\Sigma\, \lfloor (ai+b)/m \rfloor$, pontos sob reta → floor sum
- Congruências → CRT (não coprimo: g | diferença); a^x ≡ b → BSGS (se gcd(a,m) > 1, exBSGS); $x^2 \equiv a$ → Tonelli–Shanks
- a^b^c... mod m → cadeia φ(φ(·)) (não coprimos: use b mod φ(m) + φ(m)); $v_p(x^n \pm y^n)$ → LTE
- Muitas moedas, menor pequena → Dijkstra nos restos
- Precisa de primo novo → Bertrand (primo em (n, 2n])
- x·(10^d + 1) = x concatenado
- Trocas a distância a ou b → classes mod gcd (se n ≥ a + b − gcd)
- Mais próximo com dígitos permitidos → prefixo comum + desvio + extremos

Contagem

- "Exatamente k defeitos" → inversão binomial de "pelo menos k"
- Rotações/reflexões → Burnside
- Conexos rotulados → total − componente do vértice 1 (ln de EGF)
- Árvores rotuladas → Prüfer/Cayley; geradoras → Matrix-Tree; circuitos eulerianos → BEST
- Caminhos disjuntos em DAG/grid → LGV (determinante; exige fontes/sumidouros compatíveis, senão há sinais)
- Extensões lineares de floresta → n!/∏ sz
- Caminhos que não cruzam uma linha → reflexão / Catalan
- Tempo esperado até padrão → Σ q^k sobre as bordas, incluindo o padrão inteiro
- Subconjuntos com soma ≡ r (mod m) → ∏(1+x^{a_i}) mod ($x^m - 1$)

Bits

- Soma/contagem com AND/OR/XOR → bit a bit
- Max xor → trie; com intervalo/versões → trie persistente ou base com índice mais recente
- Max xor de passeio em grafo → px + base linear dos ciclos
- Operação a_i ^= a_j, lights out, paridades → Gauss em GF(2)
- Pares com a_i & a_j = 0 → SOS; convolução xor/and/or → Walsh / zeta

Strings

- Substring de s em t → s + '#' + t; rotação/potência → s + s
- Ocorrências de vários padrões → Aho-Corasick + soma na link tree
- Substrings de um padrão → intervalo no suffix array; LCP de sufixos → RMQ no lcp
- Suffix tree de s → SAM de rev(s)
- Partição em palíndromos → eertree + série link; palíndromos por posição → Manacher
- LCS de permutações → LIS; subsequência palindrômica → LCS(s, rev s)
- Casamento com erros/curinga → FFT por letra ou kangaroo (LCE)
- Período d → DSU em classes i mod d
- Árvore ↔ parênteses → subárvore = intervalo, LCA = RMQ do balanço
- Troca todas as ocorrências de um símbolo → mapeamento tem de ser função
- Palíndromo com mudanças pontuais → pares (i, n − 1 − i) independentes

Grafos

- Grafo direcionado → DAG (SCC); 2 opções por item + conflitos → 2-SAT
- Restrições x_j − x_i ≤ c → caminho mínimo com Bellman-Ford (ciclo negativo = inviável)
- Atribuir/emparelhar → matching/fluxo; lucro com dependências → corte mínimo (closure)
- Rótulo 0/1 com custo por par submodular → corte mínimo
- Bipartido: independente máximo = n − matching; cobertura de caminhos em DAG = n − matching
- Antichain máxima = cadeias mínimas (Dilworth) → matching no fecho transitivo
- Caminhos disjuntos em vértices → fluxo com v_in/v_out; em arestas → capacidade 1 na aresta; custo convexo → arestas unitárias marginais
- Matching geral (não bipartido) → Blossom; dominadores → Lengauer–Tarjan; árvore dinâmica → link-cut
- Corte mínimo global → Stoer–Wagner; com limites inferiores → circulação viável; interseção de 2 matroides → matroid intersection
- Carteiro chinês → Euler + matching nos vértices ímpares
- Minimax de caminho → MST + LCA / árvore de Kruskal
- Árvore + poucas arestas extras → Dijkstra dos extremos + LCA
- Orientar/decompor arestas, de Bruijn → circuito euleriano (vértice fantasma para ímpares)
- Vizinhos = intervalo de vértices → segtree de arestas; "todos os não visitados" → set/DSU
- Corte mínimo planar s–t → adicione a aresta st e ache o caminho mínimo entre as duas faces novas no dual (sem usar o dual de st)
- Grafo funcional → ciclos + árvores; k passos → binary lifting
- Reconstruir de alcançabilidade → redução transitiva (pai = ancestral de menor fecho)

Árvores

- Subárvore → intervalo (Euler tour); caminho → HLD ou prefixos da raiz + LCA (prefixos só para operações inversíveis)
- Todos os caminhos com condição → centroide; resposta para toda raiz → rerooting
- Update de vértice em caminho, leitura no fim → diff[u]++, diff[v]++, diff[lca]−−, diff[pai(lca)]−−; update de aresta → diff[lca] −= 2
- Soma de distâncias sobre pares de vértices → cada aresta contribui sz·(n − sz)
- Query com k vértices → virtual tree; cores na subárvore → small-to-large
- Isomorfismo → hash AHU a partir dos centros

Estruturas

- Queries sem updates → offline + sweep, Mo
- Arestas/itens com tempo de vida → segtree no tempo + rollback
- Janela com operação sem inverso → duas pilhas
- k-ésimo em intervalo → wavelet / persistente; mex → last[v] em segtree + descida
- Reta mais alta em x → Li Chao; no caminho da raiz → Li Chao com rollback

- Só insere + estrutura estática → Bentley–Saxe
- Estado de autômato com updates → segtree de funções

Geometria

- Manhattan → (x + y, x − y)
- Ponto acima/abaixo de retas → dualidade + CHT/hull
- Ordenar por ângulo → semiplano + cross; "contém a origem" → complemento por semiplano
- União de retângulos → sweep + segtree; retângulo → 4 prefixos + BIT
- Interseção de convexos → origem ∈ Minkowski(A, −B); LP 2D → semiplanos
- Espelhos/bilhar → refletir o mundo; bolas idênticas colidindo → atravessam
- Grade → diagonais i + j e i − j; camada = min(i, j, n − 1 − i, m − 1 − j)
- Cortes que separam origem e destino → cota = nº de cortes

Jogos

- Configuração simétrica → espelho / pareamento
- Duração fixa → paridade de lances
- Jogos independentes somados → xor dos Grundy
- Lance que o outro desfaz → ignore
- Ciclos com empate → BFS retrógrada
- Pontuação → dp de diferença (minimax)

Álgebra

- Pares com soma/diferença em valores pequenos → produto de polinômios (FFT/NTT)
- Total = exp(conexas) → ln/exp de série
- Σk^d e contagens polinomiais → Lagrange
- Igualdade de polinômios/matrizes → ponto aleatório (Schwartz–Zippel, Freivalds)
- Matching perfeito existe? → posto da matriz de Tutte aleatória

Limites escondidos

Raiz e log

- Σ tamanhos ≤ S → ≤ √(2S) tamanhos distintos; itens > √S são ≤ √S
- m arestas → ≤ 2√m vértices de grau ≥ √m; grau de saída ≤ √(2m) orientando por grau
- $\Sigma \lfloor n/i \rfloor \to 2\sqrt{n}$ valores; 1 + 2 + … + k ≤ n → k ≤ √(2n)
- gcd/OR/AND de subarray → ≤ log mudanças; x mod y → cai pela metade
- Caminho raiz–folha → ≤ log arestas leves; cadeia de divisibilidade ≤ $\log_2 N + 1$
- φ(φ(…m)) chega a 1 em ≤ $2 \log_2 m$ passos
- Bordas de string → O(log n) PAs; ocorrências de p em janela 2|p| → uma PA
- "Pelo menos metade" → aleatorizar (30–60 tentativas)

Números para decorar

- Divisores máx.: 10^6 → 240; 10^9 → 1344; 10^{12} → 6720; 10^{18} → 103680
- Primos distintos: 10^6 → 7; 10^9 → 9; 10^{18} → 15
- $\pi(10^6) = 78498$; $\pi(10^7) = 664579$; $\pi(10^9) \approx 5{,}08 \cdot 10^7$
- Maior gap de primos: < 10^9 → 282; < 10^{18} → < 1500
- 20! < 2^{63} < 21!; lcm(1..43) estoura 2^{63}; ~86 Fibonacci ≤ 10^{18}
- double guarda inteiros até 2^{53}; FFT double ok se $n \cdot coef^2 \lesssim 10^{14}$–$10^{15}$
- NTT em 998244353 → tamanho ≤ 2^{23}; módulo arbitrário → 3 NTTs + CRT
- Miller–Rabin 64 bits → 12 primeiras bases primas
- Pares com n = $2 \cdot 10^5$ → $4 \cdot 10^{10}$ (long long); a·d vs b·c com 10^9 → __int128

Complexidades-chave

- Subconjuntos 3^n: n ≤ 16–18; 2^n·n: n ≤ 20–22; meet in the middle: n ≤ 40
- Knapsack em árvore → $O(n^2)$, ou O(nK) com limite K
- Mo → n√q; com updates → n^(5/3)
- Triângulos e C4 → m√m; bitset → ÷64
- Hopcroft–Karp → E√V; Hungarian → n^3; Gauss → n^3 (GF(2): $n^3/64$)
- Berlekamp–Massey precisa de 2k termos; $\Sigma k^d \to d + 2$ pontos
- Interativo: ⌈log_b(#respostas)⌉ queries; 10^{18} → 60 perguntas binárias

Estrutura garantida

- Planar → m ≤ 3n − 6, degenerescência ≤ 5
- Sem triângulo → $m \le n^2/4$; sem C4 → m = O(n√n)
- Torneio → sempre tem caminho hamiltoniano
- Grau mínimo ≥ k → caminho de comprimento ≥ k; ≥ n/2 → ciclo hamiltoniano
- n + 1 prefixos mod n → bloco com soma ≡ 0; LIS · LDS ≥ n
- n > nº de bits → subconjunto de xor 0; base linear tem ≤ (nº de bits) vetores
- Centroide → componentes ≤ n/2; virtual tree → ≤ 2k − 1 vértices
- SAM → ≤ 2n − 1 estados; palíndromos distintos ≤ n; runs < n
- Dim. do espaço de ciclos → m − n + c; Euler: V − E + F = 1 + c

Por tópico

Arrays/guloso: ordenar · trás pra frente · two pointers · pilha monotônica · deque · heap com arrependimento · diferenças · mediana · contribuição · exchange argument · slope trick

Busca binária: na resposta · k-ésimo · razão · paralela · descida na segtree · ternária · Aliens · Dinkelbach

DP: prefixo · intervalo · subárvore · máscara · dígitos · trocar estado e valor · última operação · CHT · D&C · Knuth · matriz · SOS · bitset · rerooting · small-to-large · broken profile · long-path · connected components · Lagrange · BM

Números: √ divisores · múltiplos (harmônico) · fatorar e separar primos · Möbius · gcd(a, b − a) · CRT · Euclides estendido · crivo linear · blocos de ⌊n/d⌋ · floor sum · BSGS · Miller–Rabin/Pollard · LTE

Contagem: complemento · IE · bijeção · reflexão · contribuição · linearidade da esperança · Burnside · OEIS · inversão binomial · Prüfer · Matrix-Tree · LGV · EGF · tail sum

Bits: bit a bit · trie · base linear · prefix xor · submáscaras · SOS · Walsh · Gauss GF(2) · Gray · pigeonhole n > bits

Strings: hash · Z/KMP · Aho-Corasick · SA + LCP · SAM · trie · Manacher · eertree · frequências 26 · máscara de paridade · run-length · s + s · Fine–Wilf · Duval · next[i][c]

Grafos: grafo de estados · 0-1 BFS · super-fonte · SCC · 2-SAT · pontes · DSU · MST · bipartido · funcional + lifting · Euler · fluxo/corte · König/Hall/Dilworth · árvore de Kruskal · base de ciclos · degenerescência

Árvores: enraizar bem · rerooting · centroide · diâmetro · LCA · HLD · Euler tour · small-to-large · virtual tree · pesado/leve · long-path · diferenças em caminho · AHU

Estruturas: offline + sweep · Mo · segtree (o que o nó guarda?) · lazy · persistente · beats · sparse · ODT · √ decomposition · CDQ · DSU com info · segtree no tempo · duas pilhas · Li Chao · wavelet · treap implícito

Jogos: brute force pequeno · espelho · primeira escolha decide · roubo de estratégia · pares · paridade de lances · invariante · Grundy · Nim · BFS retrógrada · minimax em intervalo

Construtivos: impossível primeiro · n pequeno à mão · indução n → n + 2 · forma canônica · paridade/xadrez · pares · sentinela · Gray · de Bruijn · Havel–Hakimi · interativo: binária/centroide

Aleatório: sortear e verificar · hash de conjunto · xor hashing · shuffle · color coding · Schwartz–Zippel · Freivalds · annealing

Geometria: cross/dot · ordem por ângulo · hull · calipers · Minkowski · semiplanos · sweep · Pick · dualidade · closest pair · Welzl

Álgebra/otimização: FFT/NTT · série formal · Lagrange · BM/Kitamasa · Gauss · Kirchhoff · dualidade LP · matroide (guloso) · fluxo convexo · Hungarian

Travado: brute force n ≤ 15 · imprimir a escolha ótima · releia o enunciado · o que o autor quer cortar? · trocar a ordem dos fors · casos de borda · stress test

Frequências em CF (A–D)

Reclassificação manual dos 351 registros (A–D de Div. 2/3/4, 90 contests) num vocabulário fixo de 28 técnicas alinhado às seções do catálogo. Substitui as 43 "famílias" de `frequencias.md`, cujo campo `tecnica` era texto livre (347 valores distintos em 351). Cada problema recebe UMA técnica principal, a que destrava o problema. Dados em `catalogo/editoriais/classificacao.jsonl` (regenere com `python reclassifica.py` naquela pasta).

Como ler: a amostra é pequena (cerca de 85 problemas por letra). Diferenças de 1–2 pontos percentuais são ruído; confie na ordem do topo, não em células isoladas.

Ranking geral

#	Técnica	Total	%	A	B	C	D	Seção do catálogo
1	Simulação / implementação direta	30	9%	20	5	3	2	—
2	Guloso (local, mais restrito primeiro, da esquerda)	23	7%	2	5	13	3	Arrays e guloso
3	Só as frequências importam / multiconjunto	22	6%	4	9	5	4	Arrays e guloso
4	Prefixos / diferenças / sufixos	21	6%	—	5	7	9	Arrays e guloso
5	Extremo decide (máx/mín, pontas, caso extremo)	18	5%	8	7	2	1	Reformulações universais (Caso extremo)
6	Paridade / classes mod 2	18	5%	6	5	3	4	Reformulações universais (Invariante)
7	Fórmula fechada / telescopia / conta direta	17	5%	8	3	4	2	Matemática de olimpíada
8	Resto, divisibilidade, gcd/lcm	17	5%	4	6	7	—	Teoria dos números e divisores
9	Bits, xor, potências de 2, bit mais alto	16	5%	2	1	5	8	Bits e XOR
10	Resposta pequena / poucos candidatos / casos	16	5%	7	4	3	2	Construtivos e interativos
11	Construção (padrão, blocos, identidade)	15	4%	3	7	1	4	Construtivos e interativos
12	Run-length / blocos de iguais	12	3%	1	6	2	3	Strings, Arrays e guloso
13	Invariante ou monovariante (soma, mod k, posição relativa)	12	3%	6	—	1	5	Reformulações universais
14	Grade / geometria	11	3%	—	3	5	3	Geometria
15	Pombal / cota que a construção atinge	11	3%	5	4	1	1	Construtivos e interativos
16	DP	10	3%	—	1	5	4	DP
17	Primos e fatoração	9	3%	2	1	3	3	Teoria dos números e divisores
18	Jogos	9	3%	4	3	1	1	Teoria dos jogos
19	Grafos e árvores	9	3%	—	1	2	6	Grafos, Árvores
20	Trás pra frente / última operação / desfazer	8	2%	3	—	4	1	Reformulações universais (Inverta o tempo)
21	Ordenar + varredura / two pointers	8	2%	—	2	2	4	Arrays e guloso
22	Exchange argument / mediana / convexidade	8	2%	3	2	1	2	Arrays e guloso
23	Estrutura simples (pilha, heap, segtree)	7	2%	—	2	1	4	Estruturas de dados e queries
24	Período, ciclo, estabiliza	6	2%	—	5	—	1	Cadeias de pulos
25	Strings (palíndromo, subsequência, espelho)	5	1%	1	2	1	1	Strings
26	Busca binária / ternária na resposta	5	1%	—	—	3	2	Busca binária e otimização
27	Interativo / comunicação	4	1%	—	1	1	2	Construtivos e interativos
28	Contagem / contribuição	4	1%	—	1	2	1	Combinatória, contagem e probabilidade

O que tentar primeiro, por letra

Top 6 de cada letra, em % dos problemas daquela letra. É a ordem em que vale testar as ideias quando travar.

Letra	1º	2º	3º	4º	5º	6º
A (89)	Simulação 22%	Extremo decide 9%	Fórmula fechada 9%	Resposta pequena 8%	Invariante ou monovariante 7%	Paridade 7%
B (91)	Só as frequências importam 10%	Extremo decide 8%	Construção 8%	Run-length 7%	Resto, divisibilidade, gcd/lcm 7%	Paridade 5%
C (88)	Guloso 15%	Prefixos 8%	Resto, divisibilidade, gcd/lcm 8%	Grade 6%	Só as frequências importam 6%	Bits, xor, potências de 2, bit mais alto 6%
D (83)	Prefixos 11%	Bits, xor, potências de 2, bit mais alto 10%	Grafos e árvores 7%	Invariante ou monovariante 6%	Só as frequências importam 5%	Ordenar + varredura 5%

Por divisão

Técnica	Div 4	Div 3	Div 2
Simulação / implementação direta	24%	4%	1%
Guloso (local, mais restrito primeiro, da esquerda)	7%	8%	5%
Só as frequências importam / multiconjunto	11%	7%	3%
Prefixos / diferenças / sufixos	3%	8%	7%
Extremo decide (máx/mín, pontas, caso extremo)	6%	8%	3%
Paridade / classes mod 2	5%	7%	4%
Fórmula fechada / telescopia / conta direta	5%	8%	3%
Resto, divisibilidade, gcd/lcm	2%	6%	6%
Bits, xor, potências de 2, bit mais alto	3%	4%	6%
Resposta pequena / poucos candidatos / casos	1%	6%	6%
Construção (padrão, blocos, identidade)	4%	5%	4%
Run-length / blocos de iguais	4%	3%	3%
Invariante ou monovariante (soma, mod k, posição relativa)	—	6%	4%
Grade / geometria	11%	—	—
Pombal / cota que a construção atinge	—	—	7%
DP	—	—	7%
Primos e fatoração	2%	2%	3%
Jogos	1%	4%	3%
Grafos e árvores	—	2%	5%
Trás pra frente / última operação / desfazer	3%	1%	3%
Ordenar + varredura / two pointers	3%	4%	1%
Exchange argument / mediana / convexidade	1%	3%	3%
Estrutura simples (pilha, heap, segtree)	1%	3%	2%
Período, ciclo, estabiliza	1%	2%	2%
Strings (palíndromo, subsequência, espelho)	3%	—	1%
Busca binária / ternária na resposta	—	1%	3%
Interativo / comunicação	—	1%	2%
Contagem / contribuição	—	—	3%

(n: Div 4 = 101, Div 3 = 103, Div 2 = 147)

Leituras

- **A é implementação ou extremo.** Simulação direta é 22% da letra A (quase toda de Div 4) e praticamente some a partir de C. Em Div 2 ela é 1%.

- **Cada letra tem um campeão claro.** B: frequências/multiconjunto. C: guloso (15%, o maior percentual de qualquer célula fora da simulação em A). D: prefixos/diferenças e bits.
- **Pombal/cota e DP só aparecem em Div 2** nesta amostra (7% cada). Resto/gcd e bits já aparecem em Div 3 em taxa parecida. Paridade é transversal (4–7% em todas as divisões).
- **DP, grafos e busca binária só a partir de C.** Em A e B são praticamente zero: se uma A/B parece pedir DP, procure antes a observação.
- **Estruturas de dados são 2%.** Em A–D, se parecer precisar de segtree, quase sempre há uma observação que a dispensa.

Exemplos por técnica

- **Simulação / implementação direta** (30): CF 2094A, CF 2126A, CF 2132A, CF 2167A, CF 2179B, CF 2231C, CF 2233A, CF 2259A
- **Guloso (local, mais restrito primeiro, da esquerda)** (23): CF 2228C1, CF 2228C2, CF 2231B, CF 2232C1, CF 2252C, CF 2259C, CF 2264B, CF 2277A
- **Só as frequências importam / multiconjunto** (22): CF 2209D, CF 2227B, CF 2230C, CF 2242A, CF 2267B, CF 2275C, CF 2278A, CF 2278B
- **Prefixos / diferenças / sufixos** (21): CF 2248D, CF 2252D, CF 2254D, CF 2258B2, CF 2264D, CF 2266C, CF 2276C, CF 2277D2
- **Extremo decide (máx/mín, pontas, caso extremo)** (18): CF 2185B, CF 2200A, CF 2209B, CF 2218B, CF 2236A, CF 2250A, CF 2258A, CF 2284B
- **Paridade / classes mod 2** (18): CF 2236B, CF 2236D, CF 2244D, CF 2246A, CF 2254C1, CF 2267D, CF 2272B, CF 2280A
- **Fórmula fechada / telescopia / conta direta** (17): CF 2171B, CF 2184A, CF 2184C, CF 2226A, CF 2232D, CF 2257B, CF 2266A, CF 2283A
- **Resto, divisibilidade, gcd/lcm** (17): CF 2226B, CF 2228A, CF 2230B, CF 2238B, CF 2241A, CF 2244C, CF 2267C, CF 2283B
- **Bits, xor, potências de 2, bit mais alto** (16): CF 2234D, CF 2240A, CF 2240B, CF 2246D, CF 2247D1, CF 2260C, CF 2277D1, CF 2283C
- **Resposta pequena / poucos candidatos / casos** (16): CF 2234A, CF 2236C, CF 2247C, CF 2253B, CF 2260A, CF 2264A, CF 2272A, CF 2275A
- **Construção (padrão, blocos, identidade)** (15): CF 2218C, CF 2218D, CF 2231D, CF 2233B, CF 2234B, CF 2241B, CF 2246B, CF 2259D
- **Run-length / blocos de iguais** (12): CF 2220B, CF 2241C, CF 2242C, CF 2244A, CF 2250B, CF 2252B, CF 2254B, CF 2276B
- **Invariante ou monovariante (soma, mod k, posição relativa)** (12): CF 2210D, CF 2238A, CF 2247A, CF 2253D, CF 2256A, CF 2257A, CF 2266D, CF 2278C
- **Grade / geometria** (11): CF 1873C, CF 1926B, CF 1950B, CF 1971C, CF 1985D, CF 2009D, CF 2094C, CF 2148B
- **Pombal / cota que a construção atinge** (11): CF 2217B, CF 2217D, CF 2220A, CF 2247B, CF 2252A, CF 2263B, CF 2276A, CF 2277B
- **DP** (10): CF 2225C, CF 2230D, CF 2238C, CF 2242D, CF 2248C, CF 2258D, CF 2260D, CF 2274B
- **Primos e fatoração** (9): CF 2167D, CF 2195A, CF 2205B, CF 2227C, CF 2238D, CF 2253A, CF 2284C1, CF 2284C2
- **Jogos** (9): CF 2123A, CF 2123D, CF 2171C1, CF 2217A, CF 2228B, CF 2248A, CF 2263A, CF 2266B
- **Grafos e árvores** (9): CF 2195B, CF 2208D1, CF 2208D2, CF 2257C, CF 2272D, CF 2276D, CF 2277C, CF 2283D
- **Trás pra frente / última operação / desfazer** (8): CF 1703C, CF 1742C, CF 1915D, CF 2137A, CF 2205C, CF 2224A, CF 2253C, CF 2274A
- **Ordenar + varredura / two pointers** (8): CF 1791C, CF 1850D, CF 1999C, CF 2121D, CF 2149B, CF 2193D, CF 2248B, CF 2278D
- **Exchange argument / mediana / convexidade** (8): CF 2065D, CF 2149D, CF 2210B, CF 2224B, CF 2232A, CF 2254A, CF 2254C2, CF 2269A
- **Estrutura simples (pilha, heap, segtree)** (7): CF 2126D, CF 2185D, CF 2200C, CF 2205D, CF 2208B, CF 2247D2, CF 2275B
- **Período, ciclo, estabiliza** (6): CF 1692D, CF 2184B, CF 2256B, CF 2259B, CF 2260B, CF 2269B
- **Strings (palíndromo, subsequência, espelho)** (5): CF 1999D, CF 2044B, CF 2065B, CF 2233C, CF 2267A
- **Busca binária / ternária na resposta** (5): CF 2226C, CF 2232C2, CF 2257D, CF 2272C, CF 2275D
- **Interativo / comunicação** (4): CF 2162D, CF 2258C, CF 2280B, CF 2284D
- **Contagem / contribuição** (4): CF 2228D, CF 2246C, CF 2258B1, CF 2264C

Catálogo completo

> Aviso: os blocos 'adicionado em 2026-11-10 por agentes' estão marcados como 'revisado', mas a auditoria de 2026-11-11 encontrou erros neles; este catálogo já foi corrigido. Ver docs/AUDITORIA-2026-11-11.md.

Nov 10, 2026 · @Walfrido

Use como checklist de perguntas, não como lista de técnicas: cada linha é um gatilho no enunciado → uma pergunta para se fazer.

Como usar quando travar

Forçar ideia funciona quando é rápido e barato: gaste 1–2 minutos por seção relevante, testando cada pergunta contra um exemplo pequeno. Se nenhuma encaixa, volte a observar o problema em vez de forçar mais.

1. Releia o enunciado e as constraints. Anote o que é estranho (um limite pequeno demais, uma soma garantida, uma condição "distintos").
2. Resolva à mão 3–4 casos pequenos. Se possível, escreva um brute force e imprima respostas para n = 1..10.
3. Abra a seção das constraints (abaixo) e decida a complexidade-alvo.
4. Passe pelas Reformulações universais. Elas destravam mais do que qualquer técnica.
5. Só então abra a seção do tipo do problema e teste cada gatilho contra o seu exemplo.
6. Travou 10+ minutos sem progresso: troque de problema e volte depois.

Leia as constraints

O limite de n quase sempre entrega a complexidade-alvo; um limite estranho quase sempre entrega a técnica.

Limite	Complexidade-alvo	O que tentar
n ≤ 10–12	n!, n!·n (este só até n = 10)	Permutações, backtracking
n ≤ 18–22	2^n·n	Bitmask DP, SOS, brute force de subconjuntos
n ≤ 40	2^(n/2)	Meet in the middle
n ≤ 100–500	n^3	DP em intervalos, Floyd, DP com 3 índices
n ≤ 2000–5000	n^2	DP quadrática, todos os pares, O(n^2) com bitset
n ≤ 10^5	n·√n, n $\log^2$ n	Mo, sqrt decomposition, D&C, CDQ
n ≤ 2·10^5–10^6	n log n, n	Sort, segtree, two pointers, guloso, prefix sums
n, x ≤ 10^{18}	log, polylog	Fórmula, exponenciação de matriz, digit DP, binária na resposta; fatoração com Miller–Rabin + Pollard–Rho (√x só até 10^{12})
n ≤ 10^7	n (linear, sem log)	Crivo linear, prefix sums, two pointers; evite map/set; leitura rápida (fread)
n ≤ 10^8	n com constante minúscula	Só laço simples, crivo em bitset/segmentado; sem memória por elemento
n ≤ 10^{10}–10^{11}	n^(2/3), √n·log	Soma de funções multiplicativas: Dirichlet, Min_25, Lucy; contagem de primos
n ≤ 10^{12}	√n, n^(1/3)	Fatorar por tentativa até 10^6, depois Miller–Rabin; divisão em blocos
n ≤ 60 (ou 64)	operações em uint64, base linear, branching (2^(n/2)/3^(n/3) só até n ≈ 40–50)	Máscara cabe em palavra; base linear de 60 bits; bits de 10^{18}; cliques máximas por Bron–Kerbosch
n ≤ 30	2^(n/2)·n (2^n só n ≲ 20–22)	Subconjuntos com bit hacks; meet in the middle vira 2^15; DP de máscara sem log (só n ≲ 20–22)
n ≤ 50	2^(n/2), branching	Meet in the middle em 2^25 com sort; conjunto independente máximo por branching e poda
n ≤ 300	n^3 (n^4 só n ≲ 100)	Floyd, DP em intervalo, Hungarian O(n^3), matriz inversa; n^4 só com n ≲ 100 e operações triviais

Limite	Complexidade-alvo	O que tentar
$n \le 3 \cdot 10^4$–10^5 com TL 3 s	$n^3/64$ (fecho geral) ou $n \cdot m/64$ (DAG esparso); $n^2/64$ (mochila/LCS)	Bitset em fecho transitivo: geral só $n \lesssim 3000$–5000; em DAG esparso até $3 \cdot 10^4$–10^5; mochila booleana, LCS bit-parallel
$n \le 2000$, TL 1 s	n^2 simples	DP dp[i][j] com poucos acessos; n^2 log só com cuidado
$k \le 10$, n grande	$k \cdot n$, $2^k \cdot n$, $3^k \cdot n$	k no estado: Steiner (3^k), TSP parcial, DP sobre cores
$\min(n, m) \le 20$ (com $n \cdot m \le 10^5$) ou $n \le 5$ e m grande	profile DP, matriz	DP em perfil (broken profile) por coluna; transição linear → matriz $2^n \times 2^n$ e exponenciação (só $n \le 5$–8)
Resposta ≤ 20 ou "no máximo log"	iterar a resposta	Para cada candidato, checagem O(n log n); ou BFS em camadas até 20
Operações "no máximo 10^6" e $n \le 10^3$	$O(n^2)$ total; O(1) ou O(log n) por operação	Construtivo; a quantidade permite n^2 trocas simples (bubble sort)
T testes, n pequeno por teste	precompute global	Tabelas por tamanho (factoriais, Grundy, padrões) uma vez; não zere arrays de tamanho máximo a cada teste
Modulo $m \le 100$–1000	dp[i][r] em $m \cdot n$	Estado = resto; resposta por pigeonhole; ciclo de $x^k \bmod m$
Valores de peso ≤ W pequenos ($W \le 10$–100)	BFS com W filas, Dial	Caminho mínimo em $O(m + n \cdot W)$; expandir aresta de peso w em w arestas unitárias só se $m \cdot w$ couber

Sinais escondidos nas constraints:

- Soma de n sobre os testes ≤ $2 \cdot 10^5$ → pode ser O(n) por teste; sem essa garantia, pré-processe tudo antes.
- Valores $a_i \le 10^6$ (bem menores que 10^9) → itere sobre valores, crivo, contagem por valor, divisores de cada valor.
- $k \le 20$ ou "no máximo 3 cores" num problema grande → o k vai no estado ou na máscara.
- $n \cdot m \le 10^6$ com n e m livres → $\min(n, m) \le 1000$; troque linhas e colunas se preciso.
- "a_i distintos" ou "é uma permutação" → posição do valor v, ciclos, inversões (ver também "Todos distintos" abaixo).
- Grafo com $m = n - 1$ ou $m \le n + 20$ → árvore mais poucas arestas extras.
- Soma dos valores ≤ 10^5 (com $a_i \ge 0$ inteiros) → há no máximo $\sqrt{(2 \cdot 10^5)} \approx 450$ valores distintos.
- Queries Q grande e offline permitido → ordene queries (Mo, sweep, DSU offline).
- Resposta "mod 998244353" → NTT pode aparecer; "mod p" → contagem, quase nunca guloso.
- TL alto (4–5 s) e $n = 10^5$ → provavelmente √ ou bitset (ver o bullet de TL abaixo).
- Memória baixa (32–64 MB) → DP com rolling array, sem armazenar tudo.

adicionado em 2026-11-10 por agentes; revisado

- **Módulo pequeno ($m \le 100$–1000)** → estado = resto; ou pigeonhole (entre m+1 prefixos há dois iguais mod m ⇒ existe subarray com soma ≡ 0); ou $x \to x^k \bmod m$ com ciclo curto (ver também a linha "Modulo" da tabela).
  - *Gatilho:* "módulo m" pequeno, "soma divisível por m", "quantos restos", "ciclo de $x^k \bmod m$".
  - *Por que funciona:* só existem m restos, então m+1 grandezas forçam repetição; a sequência de restos é finita e vira pré-período + ciclo de comprimento ≤ m.
  - *Como fazer:* no DP, use estado r = valor mod m (dp[i][r]); para subarray com soma ≡ 0, guarde a primeira posição de cada prefixo mod m; para x^k, marque os restos visitados até repetir.
  - *Exemplo:* achar subarray não vazio com soma múltipla de m → dois prefixos com o mesmo resto; o trecho entre eles é a resposta.
  - *Armadilha:* o subarray vazio não vale, então compare ocorrências distintas; quando x não é invertível mod m, o ciclo tem pré-período e não começa em k = 0.
- **$k \le 5$ ou 6** → k! permutações cabem no estado; ou k distinguíveis sem custo; ou o problema difícil só aparece em $k \ge 3$.
  - *Gatilho:* k minúsculo explícito com n grande, "no máximo k cores", "k = 5".
  - *Por que funciona:* k! cresce devagar até 6 (720), e 2^k é ainda menor; enumerar ordens ou atribuições de k objetos é barato mesmo com n grande.
  - *Como fazer:* enumere as k! ordens ou as 2^k máscaras; se os k objetos são distinguíveis, use bitmask no estado; trate $k \le 2$ por fórmula e $k \ge 3$ pela enumeração geral.
  - *Exemplo:* escolher a ordem de k tarefas que minimiza o custo total, mantendo as demais fixas.
  - *Armadilha:* 6! = 720, mas 8! = 40320 já pesa; k é o número de objetos especiais, não o n do problema.

- **Queries ≤ 10 (ou muito poucas)** → pode fazer O(n log n) ou O(n√n) por query, sem estrutura; às vezes a ideia é rodar o algoritmo de uma query inteira cada vez.
  - *Gatilho:* Q ≤ 10 declarado, "poucas consultas", "cada consulta pode ser cara".
  - *Por que funciona:* o custo total é Q vezes o custo de uma consulta; com Q pequeno, o fator Q cabe no orçamento e não compensa construir estrutura sofisticada.
  - *Como fazer:* rode o algoritmo completo por consulta; ordene/processe a entrada uma vez só se isso baratear todas; evite generalizar para Q grande.
  - *Exemplo:* 5 consultas pedindo o k-ésimo menor de um subarray → copie e ordene cada subarray, O(Q·n log n).
  - *Armadilha:* Q ≤ 10 com n = 10^6 e TL curto ainda pode estourar; estime Q·n·log n antes de escolher.
- **Σ tamanhos limitada em vários objetos** (Σ|s|, Σk, Σ deg) → quase sempre a ideia é pesado/leve ou "tamanhos distintos ≤ √(2S)"; a soma limitada permite O(Σ·log) mas não O(n·Σ).
  - *Gatilho:* "a soma dos comprimentos é ≤ S", "soma dos graus = 2m", muitos objetos mas pouco conteúdo total.
  - *Por que funciona:* o custo somado é S, não n·S; poucos objetos podem ser grandes e muitos são pequenos, e o número de tamanhos distintos é O(√S) porque 1+2+…+k ≤ S.
  - *Como fazer:* escolha B ≈ √S; objetos grandes (tamanho ≥ B) são no máximo S/B; processe pequenos custando o tamanho real e grandes com estrutura, somando O(S log S).
  - *Exemplo:* strings com Σ|s| ≤ $2 \cdot 10^5$ → hashes e trie em O(Σ|s|), não O(n·max|s|).
  - *Armadilha:* cobrar O(n) por objeto mesmo quando ele é vazio; use o tamanho real e não o número de objetos.
- **Grid n × m com n ≤ 5–8 e m enorme** → DP em perfil por coluna e, se m ≥ 10^9, exponenciação de matriz da transição de colunas.
  - *Gatilho:* largura minúscula, comprimento gigante, "tabuleiro 5 × 10^9", cobertura por dominós.
  - *Por que funciona:* o estado de uma coluna cabe em n bits e a transição entre colunas é sempre a mesma; m só entra no número de aplicações.
  - *Como fazer:* estado = bitmask (ou lista de padrões válidos); monte T[estado][próximo] checando compatibilidade; resposta = T^m aplicada ao estado inicial; O(P^3 log m) com P padrões válidos.
  - *Exemplo:* dominós em tabuleiro 2 × m → Fibonacci; colorir 5 × 10^9 com restrições locais.
  - *Armadilha:* n ≤ 8 dá 2^n = 256, mas os padrões válidos P podem ser bem menos; não aloque 2^n × 2^n sem filtrar.
- **Pesos pequenos (w ≤ 10–100)** → Dial / BFS em camadas; "valor pequeno × peso grande" → troque o estado da mochila; grafo com pesos 1..W → distância em (vértice, tempo) só até n·W (ver também a linha "Dial" da tabela).
  - *Gatilho:* pesos inteiros pequenos, W ≤ 100, "custo entre 1 e 10".
  - *Por que funciona:* distâncias são inteiras e limitadas por n·W; uma fila de buckets indexada pela distância processa em ordem sem heap.
  - *Como fazer:* Dial com W filas circulares em O(m + n·W); BFS 0-1 se os pesos são 0/1; na mochila, se o valor é pequeno, use dp[valor] = menor peso.
  - *Exemplo:* caminho mínimo com pesos 1..10 em grafo de 10^5 arestas; mochila com valores ≤ 100.
  - *Armadilha:* W = 10^9 não é pequeno e n·W precisa caber; pesos negativos invalidam Dial.
- **"Todos distintos" / "permutação"** → o conjunto de valores é conhecido: soma, xor, produto são fixos; posição→valor é bijeção e ciclos aparecem; "distinto" também limita multiplicidade a 1 (hash por soma de aleatórios).
  - *Gatilho:* "permutação de 1..n", "todos os a_i são distintos", "cada valor aparece uma vez".
  - *Por que funciona:* o multiconjunto é fixo, então estatísticas globais (soma, xor, soma de quadrados) são conhecidas; a bijeção posição↔valor se decompõe em ciclos.
  - *Como fazer:* ache o que falta/repete por soma ou xor; siga os ciclos da permutação em O(n); para testar igualdade de multiconjuntos, compare Σ random[a_i] ou ordene.
  - *Exemplo:* número faltante em permutação 1..n → n(n+1)/2 − Σ a_i; trocas mínimas para ordenar → n − nº de ciclos.
  - *Armadilha:* hash por soma de aleatórios é probabilístico (há colisão); soma e xor não detectam todos os multiconjuntos, então use hash forte ou ordenação quando precisar de exatidão.
- **TL de 3 s ou mais** com n ≤ 10^5 → √ decomposition, bitset $n^2/64$, ou um log a mais que o ótimo; TL de 0,5–1 s com n = 10^6 → O(n log n) ainda passa; só O(n) para n ≥ 10^7 ou constante alta.
  - *Gatilho:* TL explícito no enunciado, n na fronteira de uma complexidade.
  - *Por que funciona:* o limite prático é ~10^8–10^9 operações simples por segundo; TL alto compra um fator √ ou n/64, TL baixo exige o ótimo.
  - *Como fazer:* estime operações = complexidade × constante; $n^2/64$ em bitset é ~$1{,}5 \cdot 10^8$ palavras para n = 10^5; n·√n ≈ $3 \cdot 10^7$ para n = 10^5.
  - *Exemplo:* n = 10^5 com TL 4 s → O(n√n) ou bitset passa; n = 10^6 com TL 1 s → O(n log n) com constante baixa.
  - *Armadilha:* "um log a mais" com map/set pode estourar; leitura e escrita também consomem o TL.

- **Resposta mod 2** → paridade: cancele em pares (involução), C(n,k) é ímpar ⇔ k ⊆ n em bits (Lucas), determinante mod 2 por Gauss em bitset; muitos problemas "conte as paridades" caem em xor.
  - *Gatilho:* "mod 2", "par ou ímpar", "o número de … é par?".
  - *Por que funciona:* sobre GF(2), soma é xor e sinais desaparecem; parear por involução elimina tudo exceto os pontos fixos.
  - *Como fazer:* reescreva a contagem como xor de indicadores; use Lucas (C(n,k) mod 2 = 1 sse k & ~n = 0); para sistemas lineares mod 2, elimine com bitset.
  - *Exemplo:* número de subconjuntos de tamanho k de n elementos é ímpar sse (k & n) == k.
  - *Armadilha:* "mod 2" não significa "metade dos casos"; não divida por 2 sem exibir a bijeção/involução.
- **"Imprima qualquer"** → existe um critério simples de existência; ache a condição de impossível e depois uma construção que a dispense; teste n pequeno com brute force que lista todas as soluções para ver o padrão.
  - *Gatilho:* "imprima qualquer solução válida", "se houver várias, qualquer uma".
  - *Por que funciona:* o juiz verifica só a validade, então não é preciso otimizar; a estrutura da impossibilidade costuma indicar a construção.
  - *Como fazer:* brute force para n pequeno listando todas as soluções; veja o que elas têm em comum; construa por indução ou ganancioso e verifique no final.
  - *Exemplo:* construir permutação com $|p_i - i| \le 1$ → troque pares consecutivos; "imprima qualquer subconjunto com soma S" → guloso com ajuste.
  - *Armadilha:* construir "quase sempre" e esquecer o caso impossível; sempre teste o menor n e os extremos.
- **"Se existir"** → a decisão pode ser mais fácil que a construção: faça primeiro o teste de existência (paridade, Hall, matching), depois reconstrua por backtrack do certificado.
  - *Gatilho:* "determine se é possível e, se for, construa".
  - *Por que funciona:* decidir pode ser polinomial (matching, fluxo, paridade) mesmo quando construir exige busca; o certificado do decisor guia a reconstrução.
  - *Como fazer:* rode o decisor; se der possível, use o certificado (matching, fluxo, parent) e desfaça as escolhas; se não, exiba o invariante que prova a impossibilidade.
  - *Exemplo:* emparelhamento perfeito → matching máximo + caminhos alternados; fluxo → decompor o fluxo em caminhos.
  - *Armadilha:* reconstruir sem usar o certificado e cair em busca exponencial; a decisão fácil não garante construção fácil por força bruta.
- **"Mostre que" / "prove" em problema de olimpíada** → o enunciado dá a resposta; procure o invariante, o extremo (maior, mais à esquerda) ou o pigeonhole que a prova usa; esse é o algoritmo.
  - *Gatilho:* "prove que é sempre possível", "mostre que existe", "demonstre que o mínimo é …".
  - *Por que funciona:* a prova construtiva é o algoritmo; o objeto extremo ou o invariante usado na prova é exatamente o que se deve computar.
  - *Como fazer:* escreva a prova mais simples; extraia o objeto (maior elemento, prefixo mínimo, par repetido) e calcule-o; se a prova é por indução, a recursão é o algoritmo.
  - *Exemplo:* "prove que dá para ordenar com ≤ n^2 trocas" → bubble sort; "prove que há dois prefixos iguais mod n" → guarde restos.
  - *Armadilha:* procurar técnica sofisticada quando a prova é elementar; leia o que o enunciado já afirma.
- **Entrada muito grande (10^6 números)** → leitura rápida vira parte do limite; saída com 10^6 linhas → '\n' em vez de endl, string buffer.
  - *Gatilho:* n ≥ 10^6 na entrada ou na saída.
  - *Por que funciona:* cin/cout sincronizados custam ~10× mais; endl faz flush a cada linha e transforma a saída em gargalo.
  - *Como fazer:* ios::sync_with_stdio(false); cin.tie(nullptr); ou fread/scanf; acumule a saída em string/ostringstream e dê um write; use '\n'.
  - *Exemplo:* ler 10^6 inteiros e somar; imprimir 10^6 respostas de uma vez.
  - *Armadilha:* esquecer o flush final ao usar buffer próprio; misturar cin dessincronizado com scanf.
- **"n ≤ $2 \cdot 10^5$ e $a_i \le 2 \cdot 10^5$"** → valores como índices: arrays por valor, crivo, harmônico Σ N/d; sem isso o limite estaria 10^9.
  - *Gatilho:* valor máximo da mesma ordem que n, "$a_i \le 2 \cdot 10^5$".
  - *Por que funciona:* o valor é um índice válido; iterar sobre valores ou múltiplos custa Σ N/d = O(N log N), não O(N·max).
  - *Como fazer:* cnt[valor]++; crivo para primos/divisores; para cada d, percorra 2d, 3d, …; complexidade O(N log N).
  - *Exemplo:* contar pares (i, j) com $a_i \mid a_j$ → percorra múltiplos ou divisores de cada valor.
  - *Armadilha:* $a_i \le 10^9$ não cabe em array; comprima se os valores forem esparsos.

- **Limite com forma $n \cdot k \le 10^6$, $n \cdot m \le 10^5$** → o produto é o tamanho do estado; DP dp[i][j] pode ser O(nk); sempre inverta o menor.
  - *Gatilho:* duas dimensões cujo produto é limitado, "$n \cdot m \le 10^5$".
  - *Por que funciona:* o custo é o produto, independentemente de qual dimensão é maior; a memória e a constante melhoram quando o menor índice fica por dentro.
  - *Como fazer:* dimensione o estado pelo produto; ponha a menor dimensão no índice interno; se preciso, transponha a entrada; use rolling array na dimensão externa.
  - *Exemplo:* mochila com $n \cdot W \le 10^6$; DP em grade $n \cdot m \le 10^5$.
  - *Armadilha:* alocar n × m quando só o produto cabe; declarar int em vez de short/char pode dobrar a memória.
- **Memória 16–64 MB** → rolling array, bitset, hash de estados, evitar vector de vector pequeno (overhead por vetor ≈ 24 B); árvore em lista de adjacência plana.
  - *Gatilho:* limite de memória explícito baixo.
  - *Por que funciona:* a memória é medida em bytes, não em células; overhead de containers e alinhamento domina quando há muitos vetores pequenos.
  - *Como fazer:* guarde só as duas últimas camadas; use bitset/uint64; adjacência em arrays (head, to, next); evite map e vector<vector>.
  - *Exemplo:* DP de 10^7 posições com int = 40 MB (já no limite); 10^6 vetores vazios = ~24 MB só de overhead.
  - *Armadilha:* vector<vector> com 10^6 linhas curtas desperdiça mais que a matriz; recarregar a memória pode custar mais que o tempo.
- **Queries online forçadas (resposta anterior no input)** → sem offline: persistente, sqrt, ou estrutura online (binary lifting, sparse).
  - *Gatilho:* a consulta depende da resposta anterior; não dá para reordenar.
  - *Por que funciona:* sweep/Mo exigem ver todas as consultas antes; online exige estrutura que responda em qualquer ordem.
  - *Como fazer:* segtree persistente para versões; sqrt decomposition com blocos; binary lifting/sparse table para consultas estáticas; cuidado com updates.
  - *Exemplo:* k-ésimo menor em prefixo com consultas online → segtree persistente; ancestral com saltos → binary lifting.
  - *Armadilha:* tentar ordenar consultas e quebrar a dependência; a resposta anterior muda o input, então a ordem é parte do problema.
- **Entrada com "garantido que resposta existe"** → não precisa tratar impossível; às vezes o enunciado garante um grafo conexo ou árvore, o que dispensa uma busca.
  - *Gatilho:* "é garantido que existe", "o grafo é conexo", "a entrada é uma árvore".
  - *Por que funciona:* a garantia remove casos de borda e permite assumir propriedades estruturais sem checá-las.
  - *Como fazer:* pule checagens de impossibilidade; use a propriedade (conexo → BFS alcança tudo; árvore → m = n−1); ainda assim valide o que for barato.
  - *Exemplo:* "garantido que há caminho entre 1 e n" → não precisa testar conectividade.
  - *Armadilha:* a garantia pode ser só de existência, não de unicidade; leia exatamente o que é garantido, e não confunda "existe" com "seu algoritmo encontra".
- **"Cada elemento aparece no máximo k vezes"** → somas sobre pares de mesma cor/valor limitadas a n·k; bruta de pares iguais é aceitável.
  - *Gatilho:* limite de multiplicidade, "cada valor aparece no máximo k vezes".
  - *Por que funciona:* o número de pares com mesmo valor é $\Sigma\, C(\text{freq}, 2) \le n \cdot (k-1)/2 = O(nk)$; para k constante, é linear.
  - *Como fazer:* agrupe por valor/cor; para cada grupo de tamanho f, processe os C(f, 2) pares; some contribuições.
  - *Exemplo:* contar pares iguais em array onde cada valor aparece ≤ 3 vezes → O(n).
  - *Armadilha:* k pode ser grande (k = n); aí a bruta é quadrática e você precisa de fórmula/frequência, não de pares.
- **Estrutura "quase árvore" (cacto, cada aresta em ≤ 1 ciclo, série-paralelo)** → treewidth ≤ 2, DP em decomposição; grau ≤ 3 sozinho não limita treewidth.
  - *Gatilho:* "cada aresta pertence a no máximo um ciclo", grafo com poucos ciclos, série-paralelo.
  - *Por que funciona:* treewidth 2 permite DP em uma árvore de blocos; o grafo vira uma árvore de componentes biconexas (block-cut tree).
  - *Como fazer:* decomponha em blocos biconexos; resolva cada ciclo/bloco e combine na block-cut tree; para série-paralelo, DP com dois terminais.
  - *Exemplo:* cacto com pesos → DP no block-cut tree; caminho mais longo em cactus.
  - *Armadilha:* grau máximo 3 não implica treewidth 2 (subdividir K4 dá grau 3 e treewidth 3); planar também não garante tw 2.

Reformulações universais

Estas servem para qualquer tipo de problema; teste todas antes de procurar técnica.

- **Fixe algo.** Fixe o máximo, o mínimo, o elemento do meio, a posição da resposta, o primeiro passo, o valor do gcd. Depois conte/otimize o resto.
  - *Gatilho:* a resposta depende de um elemento/posição especial, "conte os subarrays cujo máximo é ...", "o primeiro passo é ...".
  - *Por que funciona:* fixar remove uma dimensão da busca e transforma o resto em um problema mais simples, muitas vezes com fórmula fechada.
  - *Como fazer:* enumere o valor fixado (n opções); para cada um, conte em O(log n) ou O(1) com prefixos/BIT/monotonia; total O(n log n) ou O(n).
  - *Exemplo:* soma dos máximos de todos os subarrays → fixe o máximo e conte com anterior/próximo maior.
  - *Armadilha:* contar o mesmo objeto duas vezes quando o "fixado" não é único (máximo repetido); imponha desempate (estritamente maior à esquerda, ≥ à direita).
- **Contribuição.** "Soma sobre todos os subarrays/pares/caminhos de f" → troque a ordem da soma: quanto cada elemento/aresta contribui? (ex.: a_i é máximo em $(i - L)\cdot(R - i)$ subarrays, com L = anterior estritamente maior e R = próximo maior ou igual).
  - *Gatilho:* "soma sobre todos os subarrays/pares/caminhos de f", "contribuição total".
  - *Por que funciona:* soma é linear; contar quantas vezes cada elemento entra evita enumerar os objetos.
  - *Como fazer:* para cada i, calcule L_i e R_i com pilha monotônica; contribuição $a_i\cdot(i-L_i)\cdot(R_i-i)$; some em O(n).
  - *Exemplo:* soma dos máximos de todos os subarrays; soma de $|a_i - a_j|$ sobre pares → ordene e use prefixos.
  - *Armadilha:* L e R precisam de critérios diferentes (< de um lado, ≤ do outro) para valores repetidos contarem uma vez; para mínimo, inverta as comparações.
- **Complemento.** Contar "pelo menos um" ou "algum par ruim" → total menos o ruim; ruim com vários defeitos → inclusão-exclusão.
  - *Gatilho:* "pelo menos um", "existe algum", "nem todos".
  - *Por que funciona:* o complemento costuma ser mais restrito e fácil de caracterizar, e o total é conhecido.
  - *Como fazer:* calcule o total; caracterize o complemento (todos bons); subtraia; com vários defeitos, use $|\cup A_i| = \Sigma|A_i| - \Sigma|A_i\cap A_j| + \dots$.
  - *Exemplo:* subarrays com pelo menos um par igual = total − subarrays com todos distintos.
  - *Armadilha:* inclusão-exclusão explode em 2^k; use só com k pequeno ou interseções de forma simples.
- **Inverta o tempo.** Remoções → adições ao contrário (DSU offline). Processo que destrói → rode de trás pra frente.
  - *Gatilho:* operações de remoção/desligamento, processo que só piora.
  - *Por que funciona:* DSU só adiciona arestas; rodar ao contrário transforma remoção em adição, e o estado final é conhecido.
  - *Como fazer:* leia todas as operações; comece do estado final; processe de trás para frente adicionando o que foi removido; guarde respostas na ordem inversa.
  - *Exemplo:* arestas removidas ao longo do tempo → conectividade offline com DSU.
  - *Armadilha:* DSU não desfaz; se houver adição e remoção, use DSU com rollback (e não comprima caminho).
- **Inverta a pergunta.** "Mínimo k tal que dá" → binária em k + checagem (confirme que a factibilidade é monótona em k antes). "Qual valor é o maior possível" → "dá pra atingir ≥ x?".
  - *Gatilho:* "mínimo/máximo k", "maior valor possível", "menor capacidade".
  - *Por que funciona:* a checagem booleana costuma ser mais fácil que a otimização, e a monotonicidade garante que a binária acha o limiar.
  - *Como fazer:* prove monotonicidade; faça binária no inteiro ou no valor; checagem O(f); total O(f log V).
  - *Exemplo:* menor capacidade de caminhão para entregar em D dias → binária + guloso.
  - *Armadilha:* sem monotonicidade a binária mente; "maior x" pode não ser monótono (paridade, restrições não conexas).
- **Troque papéis.** Itere sobre valores em vez de índices; sobre a resposta em vez da entrada; sobre arestas em vez de vértices.
  - *Gatilho:* o laço natural é grande e o outro lado é pequeno ou tem estrutura.
  - *Por que funciona:* o custo total é o mesmo, mas o lado escolhido pode ter fórmula fechada, ordenação ou BIT.
  - *Como fazer:* escreva o brute force; troque o laço externo; use Σ sobre valores/arestas com contagem.
  - *Exemplo:* em vez de "para cada vértice, some arestas", "para cada aresta, some aos dois vértices" (grau).
  - *Armadilha:* trocar sem verificar que o outro lado é pequeno; às vezes ambos são grandes.

- **Invariante.** Operação que se repete → o que nunca muda? (paridade, soma mod k, xor total, número de inversões mod 2, gcd).
  - *Gatilho:* operações repetidas, "é possível chegar em ...", "prove que não dá".
  - *Por que funciona:* se um invariante difere entre inicial e final, é impossível; se for completo, também guia a construção.
  - *Como fazer:* calcule o invariante nas duas pontas; compare; para construir, preserve-o a cada passo.
  - *Exemplo:* trocar dois elementos preserva a soma, mas muda a paridade das inversões; soma mod k é invariante sob somas de k.
  - *Armadilha:* invariante necessário não é suficiente; pode haver mais de um, e todos precisam bater.
- **Monovariante.** Algo que sempre cresce/decresce → prova término e limita o número de passos.
  - *Gatilho:* processos iterativos, "prove que termina", "quantos passos no máximo".
  - *Por que funciona:* uma quantidade inteira limitada que muda em ≥ 1 a cada passo dá cota de passos.
  - *Como fazer:* identifique a medida (distância, energia, número de inversões); mostre que decresce; limite pelo valor inicial.
  - *Exemplo:* algoritmo de Euclides → o resto decresce; bubble sort → as inversões decrescem.
  - *Armadilha:* a medida pode ficar parada em passos neutros; exija decrescimento estrito por ciclo.
- **Caso extremo.** Olhe o maior elemento, a folha, o vértice de grau 1, a primeira/última posição. Quem é forçado?
  - *Gatilho:* problema em árvore/sequência/grafo, construção, "prove que existe".
  - *Por que funciona:* o extremo tem menos escolhas (folha tem grau 1, máximo não pode ser aumentado), então sua decisão é forçada.
  - *Como fazer:* processe/remova o extremo; reduza indutivamente; veja o que a obrigação impõe.
  - *Exemplo:* árvore → remova folhas (ordem topológica); coloração → folha copia a cor do pai.
  - *Armadilha:* "maior" pode não ser único; defina desempate; em ciclo não existe folha.
- **Simplifique.** Resolva com n = 2, com valores só 0/1, sem a query de update, com o array ordenado. Depois generalize.
  - *Gatilho:* enunciado complexo, sem ideia, muitas dimensões.
  - *Por que funciona:* a estrutura essencial aparece no caso mínimo; generalizar é adicionar camadas uma a uma.
  - *Como fazer:* resolva n = 2 / 0-1 / estático; anote a fórmula; reintroduza cada complicação e veja o que muda.
  - *Exemplo:* soma de subarrays com 0/1 → contar pares de prefixos; depois valores arbitrários.
  - *Armadilha:* generalizar cedo demais; o caso 0/1 pode ter propriedades (ordem) que não valem no geral.
- **Transforme o array.** Prefix sums, diferenças (a_i − a_{i−1}), a_i − i, a_i·i, troque sinal, tome log.
  - *Gatilho:* somas de intervalos, condições de monotonicidade, produtos.
  - *Por que funciona:* a transformação converte uma condição global em local ou constante.
  - *Como fazer:* some/derive; para soma de subarray use prefixo; para a_i − i, condições de "sem buracos"; log para produtos.
  - *Exemplo:* soma de subarray = P[r] − P[l−1]; "a_i − i constante" → progressão aritmética.
  - *Armadilha:* log de zero/negativo; diferenças perdem o valor absoluto.
- **Mude a condição.** "Soma = 0" → prefixos iguais. "Média ≥ k" → subtraia k e veja soma ≥ 0. "Mediana ≥ x" → vire ±1 (testa a mediana superior; para a média dos centrais com n par, ajuste o critério).
  - *Gatilho:* condições com média/mediana/soma zero.
  - *Por que funciona:* subtrair a referência transforma comparação em sinal; ±1 transforma mediana em soma.
  - *Como fazer:* subtraia k de cada a_i; procure subarray de soma ≥ 0 com prefixos; para mediana, a_i ≥ x → +1, senão −1, e procure soma > 0.
  - *Exemplo:* existe subarray com média ≥ k ⇔ max subarray de (a_i − k) ≥ 0.
  - *Armadilha:* "≥ 0" vs "> 0" muda o critério de mediana; para n par, mediana inferior e superior diferem.
- **Modele como grafo.** Restrições entre pares → arestas; estados → vértices; "trocar a_i por a_j" → ciclos da permutação.
  - *Gatilho:* relações binárias, dependências, movimentos.
  - *Por que funciona:* alcançabilidade, emparelhamento e caminho já têm algoritmos prontos.
  - *Como fazer:* defina vértices e arestas; escolha BFS/DFS/DSU/fluxo/matching; para permutação, siga os ciclos.
  - *Exemplo:* restrições de desigualdade → ordenação topológica; trocas → ciclos.
  - *Armadilha:* grafo implícito grande demais para materializar; gere arestas sob demanda.
- **Separe as dimensões.** Distância Manhattan → x e y independentes; rotacione 45° para Chebyshev (u = x + y, v = x − y).
  - *Gatilho:* distância de Manhattan, grades, pontos.
  - *Por que funciona:* |dx| + |dy| = max(|du|, |dv|) com u = x+y, v = x−y; as coordenadas ficam independentes.
  - *Como fazer:* transforme (x, y) → (x+y, x−y); distância Manhattan vira Chebyshev; processe x e y separadamente.
  - *Exemplo:* soma de |x_i − x_j| sobre pares → ordene cada coordenada; menor retângulo que cobre pontos.
  - *Armadilha:* a rotação muda a paridade (u, v têm a mesma paridade); ao voltar para (x, y), divida por 2.

- **Encontre o limite pequeno escondido.** "Cada valor dobra" → log passos; "gcd muda" → log mudanças; "divisores" → ≈ 240 para 10^6.
  - *Gatilho:* operações que crescem/decrescem rápido, "cada valor dobra", "gcd".
  - *Por que funciona:* medidas multiplicativas ou divisoras têm poucos estados; o número de mudanças é logarítmico.
  - *Como fazer:* prove a cota (valor dobra → ≤ log V passos); itere até estabilizar; use o número máximo de divisores (240 para 10^6).
  - *Exemplo:* processo "divide por 2" → O(log V) passos; gcd de pares → O(log max).
  - *Armadilha:* "dobra" com subtração não garante log; verifique a medida exata.

adicionado em 2026-11-10 por agentes; revisado

- **Troque a ordem dos fors (do pensamento).** Escreva o brute force como laços aninhados e pergunte: e se o laço de dentro virasse o de fora? "Para cada i, quais j contam?" vira "para cada j, quantos i contam?". O laço de fora deve ser aquele que deixa o de dentro virar fórmula fechada, prefixo, BIT ou contagem. Ex.: $\Sigma_{i\le n} \Sigma_{d|i} f(d) = \Sigma_d f(d)\cdot\lfloor n/d \rfloor$; Σ de max de todos os subarrays = $\Sigma_i a_i\cdot(i - L)\cdot(R - i)$, com L = anterior estritamente maior e R = próximo maior ou igual (sentinelas 0 e n+1).
  - *Gatilho:* brute force com dois laços, "para cada i, quais j contam?".
  - *Por que funciona:* inverter pode transformar o laço interno em fórmula fechada, prefixo, BIT ou contagem; o custo total é o mesmo, mas a estrutura aparece.
  - *Como fazer:* escreva o brute; troque externo/interno; procure contagem por prefixo/BIT; use $\Sigma_d f(d)\lfloor n/d \rfloor$ e a contribuição com L/R.
  - *Exemplo:* soma dos divisores de todos os números até n em O(n log n); soma dos máximos de subarrays em O(n).
  - *Armadilha:* trocar sem verificar que a contagem interna é O(1)/O(log n); se continuar varrendo, não adiantou.
- **Quais são os laços possíveis?** Índice · valor · bit · divisor/múltiplo · tempo · query · aresta · vértice · tamanho do bloco · resposta candidata. Liste todos os laços que o brute force poderia ter e teste cada um como o mais externo.
  - *Gatilho:* você não sabe qual dimensão enumerar.
  - *Por que funciona:* cada escolha de laço externo expõe uma estrutura diferente; listar todas evita fixar na primeira.
  - *Como fazer:* liste as dimensões; para cada uma, pergunte o que o laço interno vira; escolha a que dá O(1)/O(log).
  - *Exemplo:* valor como laço → crivo; bit como laço → digit DP/base linear; bloco → sqrt decomposition.
  - *Armadilha:* enumerar duas dimensões grandes; o produto precisa caber.
- **Fatiar por camadas.** Para a_i inteiro não negativo, $\Sigma a_i = \Sigma_{t\ge 1} \#\{i : a_i \ge t\}$. Histogramas, áreas, "soma de mínimos" e $E[X] = \Sigma_{k\ge 1} P(X \ge k)$ (X inteiro não negativo) ficam fáceis ao iterar sobre o nível t em vez do elemento i.
  - *Gatilho:* soma de valores, áreas, esperança de inteiro não negativo.
  - *Por que funciona:* contar por nível troca "somar valores" por "contar quantos ultrapassam cada nível"; a estrutura por nível costuma ser simples.
  - *Como fazer:* para cada t, conte elementos ≥ t (ordene/bitset/eventos); some; O(max A) ou O(n log n).
  - *Exemplo:* soma de mínimos de subarrays = Σ_t #subarrays com min ≥ t.
  - *Armadilha:* valores negativos quebram a identidade; com max A grande, use eventos em vez de iterar t.
- **Queries × elementos.** Em vez de "para cada query, percorra os elementos", faça "para cada elemento, atualize as queries que ele afeta" (offline, sweep, diferenças). Inverta quando o número de queries ou de elementos for o lado difícil.
  - *Gatilho:* muitas queries e muitos elementos, mas um dos lados permite ordenar.
  - *Por que funciona:* offline permite ordenar queries por limite e varrer os elementos uma vez; cada elemento afeta um intervalo de queries.
  - *Como fazer:* ordene queries por r; adicione elementos; responda com BIT/prefixo; use diferenças para updates em intervalo.
  - *Exemplo:* "quantos valores ≤ x no subarray [l, r]" → offline por r + BIT.
  - *Armadilha:* queries online (dependentes) impedem ordenar; updates não comutativos complicam o sweep.
- **Pares (i, j): fixe um lado.** Fixe j e conte i com a propriedade (BIT, map, two pointers). Se f(i, j) é separável, $\Sigma_{i,j} f(i)\cdot g(j) = (\Sigma f)(\Sigma g)$; se tem uma condição, ordene e vire prefixo.
  - *Gatilho:* somar sobre todos os pares com condição.
  - *Por que funciona:* fixar um lado elimina um grau de liberdade; a separabilidade reduz a produto de somas.
  - *Como fazer:* ordene por um critério; para cada j, conte i com BIT/map; se separável, multiplique somas; se a condição é de ordem, use two pointers.
  - *Exemplo:* contar pares com $a_i + a_j \le S$ → ordene + two pointers.
  - *Armadilha:* contar (i, j) e (j, i) duas vezes; restrinja i < j; condição não separável exige estrutura.

- **DP: ordem de preenchimento e eixo externo.** Itere por valor em vez de índice (dp[valor][...]), por comprimento em vez de início, ou por fim em vez de começo. Em knapsack, itens fora/capacidade dentro conta combinações (no 0/1, capacidade decrescente); capacidade fora/itens dentro conta sequências (ordem importa) no caso ilimitado, não no 0/1. Ordem trocada muda o que é contado.
  - *Gatilho:* DP com mais de uma ordem possível; knapsack contando combinações vs sequências.
  - *Por que funciona:* o eixo externo decide se a ordem dos itens importa; a mesma recorrência conta coisas diferentes.
  - *Como fazer:* itens fora/capacidade dentro (0/1: capacidade decrescente) = combinações; capacidade fora/itens dentro = sequências no ilimitado; dp por comprimento/fim quando a transição é local.
  - *Exemplo:* contar formas de somar S com moedas (ordem não importa) vs sequências (ordem importa).
  - *Armadilha:* 0/1 com capacidade crescente usa o item várias vezes; ilimitado com capacidade decrescente perde combinações.
- **Push vs pull.** Se "quem chega em v" é difícil, calcule "para quem u contribui" (e vice-versa). Também vale para BFS/DP em grafo e para somas sobre divisores/múltiplos.
  - *Gatilho:* transição de DP/grafo assimétrica, grau de entrada vs saída.
  - *Por que funciona:* push percorre arestas de saída, pull as de entrada; o custo total é o mesmo, mas a estrutura de dados difere.
  - *Como fazer:* escolha o lado com menos arestas ou atualização mais barata; para divisores, itere múltiplos (push) ou divisores (pull).
  - *Exemplo:* crivo de soma de divisores → para cada d, some aos múltiplos (push).
  - *Armadilha:* push em DAG exige ordem topológica; pull em grafo com ciclo precisa de fixpoint.
- **Agrupe por valor igual.** Troque "para cada elemento" por "para cada valor distinto × multiplicidade". Com $a_i \geq 0$ e soma limitada, o número de valores distintos é $O(\sqrt{\Sigma})$, o que limita as iterações a $\sqrt{}$.
  - *Gatilho:* muitos elementos repetidos, soma limitada.
  - *Por que funciona:* elementos iguais têm o mesmo comportamento; agrupar reduz n a #distintos; com soma S, #distintos ≤ $\sqrt{(2S)}$.
  - *Como fazer:* conte frequências; processe (valor, freq); contribuições em bloco.
  - *Exemplo:* contar pares iguais → Σ C(freq, 2).
  - *Armadilha:* valores negativos ou grandes quebram a cota $\sqrt{\Sigma}$; use hash e não arrays por valor.
- **Transposta.** Troque linhas por colunas e trabalhe com a menor dimensão ($n \cdot m \leq 10^6 \Rightarrow \min(n, m) \leq 1000$); vale também para matrizes de adjacência e tabelas de DP.
  - *Gatilho:* matriz grande com uma dimensão pequena.
  - *Por que funciona:* o custo depende de min(n, m) quando se escolhe o eixo certo; acessos contíguos melhoram a constante.
  - *Como fazer:* se n > m, transponha; no DP, troque a ordem dos índices; na adjacência, liste arestas.
  - *Exemplo:* $n \cdot m \leq 10^6$, min ≤ 1000 → $O(\min \cdot n \cdot m)$ cabe; DP com a menor dimensão por dentro.
  - *Armadilha:* transpor custa O(nm); não faça repetidamente; a alocação transposta pode estourar a memória.
- **∃ e ∀ trocados.** "Existe i tal que para todo j" × "para todo j existe i" são problemas diferentes; ao reescrever a condição, confira qual quantificador está dentro de qual. Min-max e max-min só coincidem sob dualidade (jogos, LP, fluxo).
  - *Gatilho:* enunciado com "para todo/existe".
  - *Por que funciona:* a ordem dos quantificadores muda o problema (∃∀ é mais forte); trocar sem perceber leva a algoritmo errado.
  - *Como fazer:* reescreva em prenex form; negue (¬∀ = ∃¬); use dualidade só com teorema.
  - *Exemplo:* "para todo x existe y com f(x) ≤ f(y)" (máximo global) vs "existe y que é ≥ todos".
  - *Armadilha:* min-max ≠ max-min em geral; jogos com estratégias mistas precisam de LP.
- **Soma por partes (Abel).** $\Sigma a_i \cdot b_i = A_n \cdot b_n + \Sigma_{i=1}^{n-1} A_i \cdot (b_i - b_{i+1})$, com $A_i = \Sigma_{j \leq i} a_j$ (b não precisa ser monótono). Troca "valor × peso" por "prefixos × saltos", mais o termo de borda.
  - *Gatilho:* soma de produtos $a_i \cdot b_i$, especialmente com prefixos.
  - *Por que funciona:* troca "valor × peso" por "prefixo × salto"; se b é monótono, os saltos têm sinal.
  - *Como fazer:* $A_i = \Sigma_{j \leq i} a_j$; aplique a fórmula; não esqueça o termo de borda $A_n \cdot b_n$.
  - *Exemplo:* $\Sigma i \cdot x_i$ com x ordenado; somas de $\lfloor n/i \rfloor$.
  - *Armadilha:* sinal dos saltos e o termo b_0; não confunda com a telescopia da seção seguinte.

adicionado em 2026-11-10 por agentes; revisado

- **Dualidade.** Um min vira um max do problema dual quando há teorema de igualdade: König (cobertura mínima = matching máximo), max-flow = min-cut, Dilworth (cadeias = antichain máxima), LP dual. Se o primal é difícil, o dual muitas vezes é um fluxo/greedy.
  - *Gatilho:* minimizar cobertura/corte; primal difícil.
  - *Por que funciona:* teoremas de igualdade transformam o problema no dual, que costuma ser fluxo ou guloso.
  - *Como fazer:* identifique o par primal-dual; resolva o lado fácil; reconstrua a solução ótima.
  - *Exemplo:* cobertura mínima de vértices em bipartido = matching máximo.
  - *Armadilha:* dualidade só vale sob as hipóteses (bipartido, LP integral); cobertura mínima em grafo geral é NP-difícil.
- **Min-max theorems.** König (bipartido), Hall (bipartido com $|L| = |R|$: existe matching perfeito $\Leftrightarrow \forall S \subseteq L, |N(S)| \geq |S|$), Dilworth/Mirsky, Menger (caminhos disjuntos = corte mínimo), Gallai (independente + cobertura = n), Erdős–Szekeres. "Cobrir com o menor número de X" pede o pior caso (antichain) como certificado.
  - *Gatilho:* "cobrir com o menor número de X", "caminhos disjuntos".
  - *Por que funciona:* cada teorema dá um certificado de optimalidade e um algoritmo (matching/fluxo).
  - *Como fazer:* reconheça a estrutura (bipartido → König/Hall; poset → Dilworth; grafo → Menger); calcule o lado algorítmico.
  - *Exemplo:* largura de poset = menor número de cadeias (Dilworth); matching perfeito via Hall.
  - *Armadilha:* aplicar König em grafo não bipartido; Hall exige $|L| = |R|$ para matching perfeito.
- **Relaxação.** Resolva o problema relaxado (fracionário, sem uma restrição) e prove que o ótimo relaxado é atingível; matrizes totalmente unimodulares (bipartido, intervalos consecutivos, fluxo) garantem que LP = inteiro.
  - *Gatilho:* restrição inteira difícil; LP parece resolver.
  - *Por que funciona:* se a matriz é TU, o ótimo do LP é inteiro; a relaxação dá o mesmo valor.
  - *Como fazer:* formule o LP; verifique TU (bipartido, intervalos, fluxo); resolva; derive o inteiro.
  - *Exemplo:* assignment → LP integral (matriz de permutação é TU).
  - *Armadilha:* nem toda relaxação é apertada; o gap de integralidade pode ser grande.
- **Forma paramétrica.** Se há uma restrição de quantidade ("exatamente k"), Lagrange/Aliens; se é "razão", binária em λ; se é "mínimo do máximo", binária na resposta; todas trocam a restrição por penalidade.
  - *Gatilho:* "exatamente k", "razão", "mínimo do máximo".
  - *Por que funciona:* trocar a restrição rígida por uma penalidade λ resolve a família e permite binária em λ.
  - *Como fazer:* Aliens (penalize a quantidade, ajuste λ, recupere k pela contagem); binária em λ para razão (Dinkelbach); binária na resposta para min-max.
  - *Exemplo:* caminho com exatamente k arestas e custo mínimo → Aliens.
  - *Armadilha:* Aliens exige convexidade da função custo(k); empates precisam de desempate por contagem.
- **Lifting de dimensão.** Acrescente uma dimensão (tempo, nível, estado extra) para tornar o problema local: grafo de estados (v, t), duplicar o grafo para "usei k bônus", DP em (posição, valor).
  - *Gatilho:* estado global (tempo, bônus, nível) difícil de acompanhar.
  - *Por que funciona:* acrescentar a dimensão torna a transição local e markoviana.
  - *Como fazer:* duplique estados (v, t) ou (v, used); BFS/DP no grafo expandido; cuidado com o tamanho.
  - *Exemplo:* caminho com até k arestas de peso 0 → grafo em camadas.
  - *Armadilha:* explosão de estados; use só dimensões limitadas.
- **Escala log.** Uma operação multiplicativa vira soma em log; ou processe bit a bit de cima para baixo (potências de 2), e binary lifting resume saltos 2^k.
  - *Gatilho:* multiplicações, potências, saltos grandes.
  - *Por que funciona:* log transforma produto em soma; binary lifting resume 2^k passos em O(log).
  - *Como fazer:* tome log (cuidado com precisão); processe bits de cima para baixo; up[k][v] para ancestrais.
  - *Exemplo:* k-ésimo ancestral; comparar produtos gigantes.
  - *Armadilha:* log de zero/negativo; precisão de double; use inteiros quando possível.
- **Amortização.** Se cada elemento entra e sai um número limitado de vezes (pilha monotônica, two pointers, potencial), o custo é O(n) total; DSU custa $O((n+m)\alpha(n))$ para n elementos e m operações; procure o potencial que só decresce.
  - *Gatilho:* operações repetidas com estruturas que consomem elementos.
  - *Por que funciona:* se cada elemento entra/sai limitadas vezes, o total é linear mesmo com operações caras isoladas.
  - *Como fazer:* argumente com potencial (pilha, two pointers); DSU $\alpha(n)$; escolha a medida que só decresce.
  - *Exemplo:* pilha monotônica O(n); DSU quase O(1) por operação.
  - *Armadilha:* pior caso por operação não é o total; não confunda amortizado com médio.

- **Extremal principle.** Considere o objeto extremo (maior elemento, par mais próximo, caminho mais longo, ciclo mais curto) e mostre que ele tem uma propriedade; frequentemente é a ordem do guloso ou o vértice a processar primeiro.
  - *Gatilho:* provar propriedade de um objeto ótimo.
  - *Por que funciona:* o extremo tem menos liberdade; qualquer violação pode ser trocada por algo melhor.
  - *Como fazer:* pegue o objeto extremo; derive contradição/troca; use para achar a ordem do guloso.
  - *Exemplo:* no caminho mais curto, subcaminhos são ótimos.
  - *Armadilha:* o extremo pode não ser único; defina tie-break.
- **Descida infinita / Vieta jumping.** Dada uma solução (a, b) de uma equação quadrática simétrica, (a, b') com b' = (coef) − b é outra; vá descendo até o caso mínimo. Em programação: operações que reduzem o par (Euclides) → o mínimo é caso base.
  - *Gatilho:* equação diofantina simétrica, "prove que não há solução".
  - *Por que funciona:* a transformação gera outra solução menor; infinitas descidas contradizem a boa-ordenação.
  - *Como fazer:* isole uma variável como raiz; use soma/produto das raízes; desça até o mínimo; no algoritmo, Euclides.
  - *Exemplo:* $a^2 + b^2 = k(ab + 1)$ → Vieta jumping.
  - *Armadilha:* precisa garantir que a nova solução é positiva e menor; nem toda troca desce.
- **Indução forte.** "Assuma que vale para todo tamanho menor" é um algoritmo recursivo: ache a parte menor (remover folha, maior elemento, par) e verifique que a construção estende; vale como prova e como implementação.
  - *Gatilho:* construção recursiva, "para todo n".
  - *Por que funciona:* assumir todos os menores permite remover uma parte e estender.
  - *Como fazer:* ache a parte removível (folha, maior, par); prove que a construção estende; implemente a recursão.
  - *Exemplo:* torres de Hanói; coloração de árvore.
  - *Armadilha:* a parte removida pode quebrar a propriedade; escolha a certa.
- **Pigeonhole.** Mais objetos que gavetas ⇒ repetição: prefixos mod n, estados repetidos (ciclo ≤ nº de estados), "n + 1 números em [1, 2n] contêm dois onde um divide o outro". Use para provar existência e para limitar passos.
  - *Gatilho:* "n+1 objetos em n gavetas", prefixos mod n, estados repetidos.
  - *Por que funciona:* mais objetos que gavetas força repetição; repetição = ciclo/colisão.
  - *Como fazer:* defina objetos e gavetas; use para existência e para limitar passos (ciclo ≤ #estados).
  - *Exemplo:* prefixos mod n → subarray divisível por n; n+1 em [1, 2n] → um divide o outro (maior potência de 2).
  - *Armadilha:* a garantia é de existência, não diz onde; para achar, use hash/visited.
- **Double counting.** Conte um mesmo conjunto de pares de duas formas e iguale: Σ grau = 2m, número de pares (elemento, conjunto), e $\Sigma_i f(i) = \Sigma_j g(j)$; frequentemente troca duas somas.
  - *Gatilho:* somas sobre pares/incidências.
  - *Por que funciona:* contar o mesmo conjunto por duas visões dá uma identidade; permite trocar soma difícil por fácil.
  - *Como fazer:* defina o conjunto de pares (elemento, conjunto); conte por linhas e colunas; iguale.
  - *Exemplo:* Σ grau = 2m; $\Sigma_i f(i) = \Sigma_j g(j)$.
  - *Armadilha:* contar cada objeto com multiplicidade errada; defina o conjunto com precisão.
- **Linearização.** Termo quadrático $\Sigma_{i<j} a_i \cdot a_j \rightarrow ((\Sigma a)^2 - \Sigma a^2)/2$; produto de dois indicadores → indicador da interseção ($I_A \cdot I_B = I_{A \cap B}$); esperança de produto de variáveis inteiras não negativas (dependentes) → ainda soma de pares de indicadores; max (inteiro não negativo) → soma de indicadores [max ≥ k].
  - *Gatilho:* termos quadráticos, produtos de indicadores, max.
  - *Por que funciona:* identidades algébricas transformam produto/quadrado em somas simples.
  - *Como fazer:* $\Sigma_{i<j} = ((\Sigma)^2 - \Sigma^2)/2$; $I_A \, I_B = I_{A \cap B}$; max = Σ_k [max ≥ k]; E[XY] via indicadores.
  - *Exemplo:* variância = $E[X^2] - E[X]^2$; contagem de pares em array.
  - *Armadilha:* a identidade de pares só vale sem i = j; subtraia os termos diagonais.
- **Soma de potências / polinômio em n.** Se uma contagem é polinomial em n de grau d (soma de potências, contagem em grade), calcule d+1 pontos por brute force e interpole por Lagrange.
  - *Gatilho:* contar em grade/estrutura regular; a resposta parece polinomial em n.
  - *Por que funciona:* polinômio de grau d é determinado por d+1 pontos.
  - *Como fazer:* brute force para n = 0..d; interpole por Lagrange ou diferenças finitas; avalie em n grande mod p.
  - *Exemplo:* soma de k-ésimas potências é polinômio de grau k+1.
  - *Armadilha:* o grau pode ser maior do que parece; use d+2 pontos para conferir.

- **Escolha de coordenadas.** Origem no ponto crítico, eixos rotacionados (x+y, x−y), posições como diferenças, tempo como terceira coordenada; muitas condições viram condições independentes por eixo.
  - *Gatilho:* geometria com distâncias/condições.
  - *Por que funciona:* coordenadas adequadas diagonalizam a condição (x+y, x−y; origem no centro).
  - *Como fazer:* rotacione/translated; separe eixos; escolha origem no ponto crítico.
  - *Exemplo:* Manhattan → Chebyshev; círculo centrado na origem.
  - *Armadilha:* rotação muda paridade/escala; converta de volta corretamente.
- **Fixe o par.** Fixe dois elementos (extremos, ou o par que realiza a resposta) e conte o resto livre. Quando o par é "o mais distante" (diâmetro), todas as outras posições ficam em uma região fácil.
  - *Gatilho:* a resposta é realizada por dois elementos (diâmetro, par ótimo).
  - *Por que funciona:* fixar o par extremo restringe os demais a uma região simples.
  - *Como fazer:* enumere o par (ou os extremos); conte o resto com prefixos/ordenação.
  - *Exemplo:* diâmetro de pontos → par mais distante; todos os outros ficam entre.
  - *Armadilha:* enumerar $O(n^2)$ pares; use a estrutura para reduzir.
- **O que acontece no limite?** Faça n → ∞ ou k grande para ver o regime assintótico (a resposta estabiliza, cresce linear, periódico). Ajuda a chutar a forma da fórmula e a ver estabilização (2–3 cópias).
  - *Gatilho:* fórmula fechada desconhecida; padrão em n.
  - *Por que funciona:* o regime assintótico revela a forma (linear, periódica, estabiliza).
  - *Como fazer:* calcule para n pequeno, veja diferenças/razões; teste periodicidade; chute a forma e prove.
  - *Exemplo:* número de peças com k cortes → quadrático; estabilização de autômato com 2–3 cópias.
  - *Armadilha:* padrão pequeno pode não continuar; prove ou use mais termos.
- **Homogeneização.** Igualdade com constante → subtrair (a_i − k) para virar soma 0; desigualdade → divide por escala para fixar o máximo em 1.
  - *Gatilho:* igualdade com constante, desigualdade com escala.
  - *Por que funciona:* subtrair/dividir normaliza e revela a estrutura de soma zero.
  - *Como fazer:* a_i − k; divida pelo máximo; converta "média = k" em soma 0.
  - *Exemplo:* média = k → Σ(a_i − k) = 0.
  - *Armadilha:* divisão inteira perde informação; use frações.
- **Normalização.** Reduza a uma forma canônica (ordenar, gcd = 1, rotação mínima, pilha com mod k, árvore enraizada no centro) para comparar objetos como iguais.
  - *Gatilho:* comparar objetos que podem ser iguais por simetria.
  - *Por que funciona:* a forma canônica elimina simetrias e permite hash/comparação.
  - *Como fazer:* ordene, divida por gcd, rotacione ao mínimo, enraíze no centro; use como chave.
  - *Exemplo:* forma canônica de árvore (AHU); string mínima por rotação.
  - *Armadilha:* a canonicalização precisa ser completa; senão há colisões/falsos negativos.
- **Parity argument.** Coloque 0/1 e procure o que muda de paridade a cada lance; a configuração final com paridade errada é impossível. Também conta grau ímpar em grafo, número de inversões de permutação, soma de cores num tabuleiro.
  - *Gatilho:* tabuleiro, trocas, movimentos.
  - *Por que funciona:* cada movimento muda a paridade de forma controlada; a paridade final é invariante.
  - *Como fazer:* pinte 0/1; calcule a mudança por lance; compare com o alvo.
  - *Exemplo:* 15-puzzle (paridade de inversões + posição do branco); grau ímpar.
  - *Armadilha:* pode haver mais de um invariante (posição do branco); use todos.
- **Coloring argument.** Pinte o tabuleiro (xadrez, 3 cores por (i + j) mod 3, linhas alternadas); cada peça cobre cores num certo número, e a contagem de cada cor é um invariante.
  - *Gatilho:* cobertura de tabuleiro por peças.
  - *Por que funciona:* cada peça cobre um número fixo de cada cor; a contagem de cores é invariante.
  - *Como fazer:* pinte com 2/3 cores; conte peças × cobertura; compare com o tabuleiro.
  - *Exemplo:* dominó cobre 1 preto + 1 branco; L-trominó cobre cores específicas.
  - *Armadilha:* a cor pode não ser suficiente; combine com paridade/área.

- **Grafo auxiliar.** Crie vértices que representam coisas que o enunciado não chama de vértice (valores, intervalos, restrições, classes mod m) e arestas que representam a operação; resultado em componentes, ciclos, caminho Euler.
  - *Gatilho:* relações que não são obviamente grafo.
  - *Por que funciona:* componentes/ciclos/caminho Euler respondem perguntas de consistência.
  - *Como fazer:* crie vértices para valores/intervalos/classes mod m; arestas para operações; analise.
  - *Exemplo:* restrições de diferença → Bellman-Ford; igualdades → DSU.
  - *Armadilha:* grafo implícito grande; construa sob demanda.
- **Reduza a contagem.** "Existe?" → "quantos?" e verifique ≠ 0 mod p (certificado por Schwartz–Zippel, determinante, matrix-tree); "uma solução" → "conte solução" em modelo pequeno mostra se é única.
  - *Gatilho:* "existe?" difícil; "quantos?" parece tratável.
  - *Por que funciona:* contar dá certificado (≠ 0) e às vezes é mais fácil (álgebra linear, determinante).
  - *Como fazer:* conte mod p via Schwartz–Zippel/determinante/matrix-tree; se ≠ 0, existe; para unicidade, conte em modelo pequeno.
  - *Exemplo:* matching perfeito via determinante de Tutte (probabilístico).
  - *Armadilha:* cancelamento mod p pode dar 0 falso; use p grande/aleatório.
- **Probabilistic method.** Para provar existência: aleatório tem esperança boa ⇒ existe um bom. Para construir: derandomize por esperança condicional (escolha cada bit mantendo a esperança ≥ alvo).
  - *Gatilho:* provar existência/construir objeto bom.
  - *Por que funciona:* esperança boa ⇒ algum objeto atinge a média.
  - *Como fazer:* sorteie; calcule E; derandomize por esperança condicional (fixe bits mantendo E ≥ alvo).
  - *Exemplo:* corte máximo ≥ m/2; coloração com poucos conflitos.
  - *Armadilha:* esperança boa não garante o objeto específico; a derandomização pode custar caro.
- **Reflexão (unfolding).** Trajetória que reflete em paredes = reta no espaço espelhado (bolas em tabuleiro, luz em espelhos); bolas idênticas colidindo = atravessam, só os nomes trocam.
  - *Gatilho:* reflexão em paredes, colisões.
  - *Por que funciona:* espelhar o espaço transforma trajetória refletida em reta.
  - *Como fazer:* desdobre o tabuleiro; calcule a reta; dobre de volta; bolas idênticas atravessam.
  - *Exemplo:* bilhar em retângulo; laser em espelhos.
  - *Armadilha:* o ponto imagem precisa respeitar a paridade do número de reflexões; bordas contam.
- **Cancelamento por involução.** Pareie objetos de sinais opostos (swap de um par) e o que sobra são os pontos fixos: soma alternada, determinantes, contagem mod 2.
  - *Gatilho:* soma alternada, determinante, contagem mod 2.
  - *Por que funciona:* pares de sinais opostos cancelam; sobra só o conjunto fixo.
  - *Como fazer:* defina a involução (trocar dois elementos); mostre que fora dos pontos fixos ela pareia; conte os fixos.
  - *Exemplo:* sinal do determinante; número de permutações com um par trocado.
  - *Armadilha:* a involução precisa não ter pontos fixos no conjunto cancelado; cuidado com sinais.
- **Limite inferior = construção.** Prove cota fácil (máximo de linhas, de passos) e tente construir exatamente esse valor; se não der, o gap mostra o invariante.
  - *Gatilho:* "mínimo número de operações", "prove que é ótimo".
  - *Por que funciona:* uma cota fácil (contagem/invariante) e uma construção que a atinge provam o ótimo.
  - *Como fazer:* ache a cota (máximo de linhas/passos/grau); construa exatamente; se não der, analise o gap.
  - *Exemplo:* ordenar por comparações ≥ log(n!) = Ω(n log n).
  - *Armadilha:* a cota pode não ser atingível; o gap é informação, não erro.
- **Telescopia e soma por partes (Abel).** $\Sigma\, a_i\cdot(b_i - b_{i-1}) = a_n b_n - a_1 b_0 - \Sigma_{i=1}^{n-1}(a_{i+1} - a_i)\cdot b_i$ (terminais incluídos); reduz soma de produtos a diferenças de somas parciais; também transforma $\Sigma\, \lfloor\cdot\rfloor$ e somas com mudança de limite em bloco.
  - *Gatilho:* somas de diferenças, $\lfloor\cdot\rfloor$, mudança de limite.
  - *Por que funciona:* soma de diferenças colapsa nos extremos; reduz produto a diferenças de prefixos.
  - *Como fazer:* escreva $a_i = A_i - A_{i-1}$; aplique; inclua os terminais.
  - *Exemplo:* $\Sigma\, (a_{i+1} - a_i) = a_{n+1} - a_1$.
  - *Armadilha:* sinais e índices; não esqueça o termo de borda.

- **Estrutura algébrica.** Operação associativa → segtree, doubling, matriz; com identidade e inversa → prefix; idempotente (min, gcd, or) → sparse table; comutativa → Mo e ordem livre.
  - *Gatilho:* operações de consulta/atualização; combinável.
  - *Por que funciona:* a álgebra da operação determina a estrutura (segtree, sparse, Mo, prefix).
  - *Como fazer:* associativa → segtree/doubling; identidade+inversa → prefix; idempotente → sparse; comutativa → Mo.
  - *Exemplo:* soma (prefix), min (sparse), multiplicação de matrizes (segtree).
  - *Armadilha:* sparse table não suporta update; prefix exige inversa.
- **Corte na árvore de decisão.** Para limites de queries/comparações use a árvore de decisão: k respostas possíveis exigem log_b k queries; o enunciado costuma dar exatamente esse limite.
  - *Gatilho:* limite de queries/comparações.
  - *Por que funciona:* k respostas possíveis exigem ≥ log_b k queries; o enunciado costuma dar exatamente esse limite.
  - *Como fazer:* conte folhas/respostas; compare com b^q; construa a árvore balanceada (binária em valor).
  - *Exemplo:* adivinhar número em [1, n] com perguntas sim/não → ⌈log2 n⌉.
  - *Armadilha:* respostas não uniformes reduzem a informação; use entropia.

adicionado em 2026-11-11 a partir dos editoriais de CF Div. 2/3/4, A–D; triado contra o catálogo

- **Valores atingíveis andam de 2 em 2.** Se cada operação muda a quantidade em ±2 (ou preserva a paridade) e dá para ir de um extremo ao outro passo a passo, os atingíveis são todos os de [mín, máx] com a paridade certa: calcule só mín, máx e paridade. Ex.: CF 2114B.
  - *Gatilho:* operações que somam/subtraem 2 ou preservam a paridade.
  - *Por que funciona:* a paridade é invariante; se o intervalo é conexo por passos de 2, todos os valores com a mesma paridade são atingíveis.
  - *Como fazer:* calcule mínimo, máximo e paridade; a resposta é (máx − mín)/2 + 1 se a paridade bate.
  - *Exemplo:* CF 2114B (original).
  - *Armadilha:* o intervalo pode ter buracos além da paridade; verifique a conectividade passo a passo.
- **Operação em todos menos um.** Aplicar uma involução (negar, inverter, complementar) em n − 1 de n itens = aplicar em todos e desfazer em um: escolha o melhor item para "devolver". Ex.: CF 2218B (negar n − 1 números).
  - *Gatilho:* "aplique uma operação em n − 1 itens", "escolha um para não operar".
  - *Por que funciona:* operar em n − 1 = operar em todos e desfazer em um; a composição da involução simplifica.
  - *Como fazer:* aplique a operação em todos; escolha o item cuja "desfeita" otimiza o objetivo; compare com os casos.
  - *Exemplo:* CF 2218B (negar n − 1 números) → negar todos e re-negar o escolhido.
  - *Armadilha:* a operação pode não ser involução (aplicar duas vezes não desfaz); verifique.

Cadeias de pulos (gatilho → observação → técnica)

Cada linha é um salto que você costuma dar em vários passos; a coluna "Aparece em" diz em quais seções a ideia também está repetida.

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Σ tamanhos ≤ S (strings, listas, pesos)	≤ √(2S) tamanhos distintos	k tamanhos distintos somam ao menos 1+2+…+k = k(k+1)/2	Agrupar por tamanho; laço externo sobre tamanhos distintos	Strings, DP, Arrays, Estruturas
Σ tamanhos ≤ S e itens "grandes"	Itens com tamanho > √S são ≤ √S	cada item maior que √S gasta mais que √S; no máximo √S deles	Pesado/leve: grandes um a um, pequenos em tabela	Strings, Grafos, Estruturas
Grafo com m arestas	≤ 2√m vértices com grau ≥ √m	graus ≥ √m somariam mais que 2m se houvesse mais de 2√m	Triângulos, queries por vértice em m√m	Grafos, Árvores
Valor que dobra / divide por 2	Só log passos	cada passo reduz o valor por um fator constante	Simule direto, ou binary lifting	Teoria dos números, Árvores
gcd / OR / AND de subarray	Muda ≤ log(a) vezes ao estender	cada mudança estrita remove um fator ou um bit	Para cada fim, lista de (valor, início)	Teoria dos números, Bits
x mod y repetido	x cai pela metade quando y ≤ x	se y ≤ x, x mod y < x/2; se y > x, nada muda	Pule elementos que não mudam (segtree com max)	Teoria dos números, Estruturas

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
$a_i \le 10^6$	Divisores ≤ 240, primos distintos ≤ 7	$2\cdot3\cdot5\cdot7\cdot11\cdot13\cdot17 \le 10^6$ e 720720 tem 240 divisores	Iterar divisores/múltiplos, crivo, Möbius	Teoria dos números, Combinatória
"Pelo menos metade dos elementos"	Sorteio acerta com prob. ≥ 1/2	cada sorteio acerta com probabilidade ≥ 1/2	Aleatorizar 30–60 vezes	Aleatorização, Teoria dos números
Configuração simétrica num jogo	Segundo jogador pode copiar	copiar devolve ao oponente a posição espelhada	Espelho; primeiro ganha se cria a simetria	Teoria dos jogos, Construtivos
Jogo com duração fixa	Só a paridade de lances importa	o total de lances é fixo; só o último decide	Contar lances totais	Teoria dos jogos
Operação reversível	Estados formam classes	reverter leva qualquer estado ao representante canônico	Forma canônica para os dois lados	Construtivos, Strings, Grafos
Operação em intervalo [l, r]	Vira 2 pontos no array de diferenças	só as bordas l e r+1 mudam no array de diferenças	Raciocinar nas diferenças; BIT	Arrays, Estruturas
"Soma = 0" / "balanceado"	Prefixos iguais	a soma do trecho é P[r] − P[l−1], que zera	Contar prefixos com map	Arrays, Strings, Bits
Mediana ≥ x	Troque por ±1	cada elemento ≥ x vira +1; mediana ≥ x ⇔ soma dos sinais positiva	Binária em x; P[r] − P[l−1] > 0 (mediana inferior) ou ≥ 0 (superior)	Busca binária, Arrays
Resposta não depende da raiz? Depende	Recalcule mudando 1 aresta	trocar a raiz por uma aresta muda só a contribuição dela	Rerooting	Árvores, DP
Caminhos em árvore com condição	Todo caminho passa por um centroide	na decomposição, o centroide que separa u e v está no caminho	Centroid decomposition	Árvores, Estruturas
Queries sem updates	Dá para ordenar	sem updates, a ordem das consultas é livre	Offline + sweep, Mo	Estruturas, Strings, Árvores
s repetida k vezes / infinita	Transiente pode ser O(|p|/|s|), não 2–3 cópias	cada cópia de s avança o comprimento de s; alinhar p exige proporcionalmente muitas	Calcule ⌈|p|/|s|⌉+1 cópias e extrapole	Strings, DP
n ≤ 20 em problema de conjunto	2^n cabe	$2^{20} \approx 10^6$ estados cabem	Máscara, SOS, meet in the middle	DP, Bits, Combinatória
Restrições "a ou b" em pares	Implicações booleanas	cada cláusula vira duas implicações	2-SAT	Grafos, Construtivos
Soma dupla / laços aninhados em $O(n^2)$ ou pior	Inverta qual índice é o externo	trocar a ordem de soma preserva o total	Contribuição, camadas, ou agrupar por valor/divisor	Arrays, Teoria dos números, Combinatória, DP
Σ sobre divisores ou múltiplos de cada i	Troque para Σ sobre d de (n/d termos)	d tem n/d múltiplos e $\Sigma_d\, n/d = n \log n$	Harmônica O(n log n), blocos de ⌊n/d⌋, Möbius	Teoria dos números, Combinatória
"Para cada query, percorra tudo"	Troque queries por elementos	acumular por elemento atende todas as queries de uma vez	Offline + sweep/diferenças/BIT	Estruturas, Arrays
Valor esperado de variável inteira não negativa	$E[X] = \Sigma_k P(X \ge k)$	cada unidade de X é contada enquanto X ≥ k	Calcule P(X ≥ k) para cada k	Combinatória
Tarefas com duração, peso ou prazo e "ordem das tarefas"	A ordem ótima é decidida por uma comparação de pares	trocar vizinhos invertidos não piora; sobra o comparador	Exchange argument: ordene por t/w (Smith) ou prazo (EDF)	Arrays, DP
"Max de k intervalos / k máquinas" ou "salas mínimas"	Máquina livre com maior fim ≤ início	usar a máquina de maior fim entre as livres preserva as demais	Multiset ordenado por fim	Arrays, Estruturas

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
"Pelo menos k", "exatamente k itens" com custo convexo	f(k) convexa em k (minimização)	λ penaliza a quantidade; com f convexa, cada λ tem um k ótimo	Aliens: penalidade λ, g(λ)=min_k[f(k)+λk], resposta g(λ)−λk; desempate consistente	Busca binária, DP
Razão Σa/Σb ou média máxima	max Σ(a − r·b) é monótona em r	max Σ(a−r·b) > 0 ⇔ existe conjunto com razão > r	Binária em r ou Dinkelbach	Busca binária, Arrays
n ≤ 40 e subconjuntos	2^20 por metade	2^20 por metade cobre 2^40 combinações	Meet in the middle + two pointers/ordenar	Arrays, Bits, Combinatória
Permutação e "operações de troca"	Ciclos = componentes	uma troca altera os ciclos em 1; p^k fatia c em gcd(c, k)	n − ciclos trocas; inversões para adjacentes; p^k quebra ciclos em gcd(c, k)	Arrays, Construtivos, Combinatória
Escolher itens não adjacentes de soma máxima	Escolha local pode ser desfeita	v_l+v_r−v_i é o ganho de trocar i pelas duas vizinhas	Heap + lista ligada com nó v_l + v_r − v_i	Arrays, DP
Tornar não decrescente com custo Σ|Δ|	f_i convexa em x	cada f_i é convexa; heaps guardam as inclinações	Slope trick com max-heap	Arrays, DP, Busca binária
Query "primeiro índice com prefixo ≥ x"	Soma de prefixo é monótona (exige a_i ≥ 0)	prefixos não decrescem quando a_i ≥ 0	Descida no BIT/segtree em O(log)	Busca binária, Estruturas
"Primeiro instante em que conecta/ocorre" para muitas queries	Resposta monótona no tempo	conectividade só cresce com o tempo	Binária paralela ou Kruskal reconstruction tree	Busca binária, Grafos, Árvores
Cobrir todas as strings/estados com uma sequência curta	Estados = vértices, símbolo = aresta	usar cada transição uma vez é um caminho euleriano	Ciclo Euleriano (de Bruijn)	Construtivos, Grafos, Strings
Todo número como soma de poucas peças	Representação em base b / Fibonacci	a base b escreve n com O(log_b n) dígitos	Construção por dígitos (binária, ternário balanceado)	Construtivos, Bits, Teoria dos números
Interativo com limite ≈ $\log_2$(possibilidades)	Cada query deve dividir ao meio	cada resposta binária divide as possibilidades por 2	Binária / centroide / descida na árvore	Construtivos, Árvores
Somar contribuições de pares com empates	Contagem dupla quando valores iguais	contar uma vez só exige critério estrito de um lado	Desempate assimétrico (estrito de um lado) na pilha monotônica	Arrays, Estruturas
Colocar itens com distância mínima / dividir recursos em blocos	check(d) é monótono e guloso	se cabe com d, também cabe com qualquer d' < d	Binária na resposta + guloso O(n)	Busca binária, Arrays
dp[i] = min_j (dp[j] + (S_i − S_j)²) ou produto cruzado	Abra o quadrado: termo S_i·S_j é reta em x = S_i	S_i·S_j é produto de variável por parâmetro: vira reta	CHT (deque se S monótono) / Li Chao / D&C se só Monge	DP, Estruturas
dp[k][i] com custo de segmento C(j, i) e k ≤ 100	opt monótono se C é Monge	Monge ⇒ opt(j, i) não decresce com i	D&C DP (O(kn log n)); Knuth em intervalo; se k grande, Aliens	DP, Busca binária
"Exatamente k" itens/segmentos e n grande	f(k) costuma ser convexa (minimização: fluxo, Monge, matroide)	f(k) é convexa nos casos usuais (Monge/fluxo)	Aliens/WQS: g(λ)=min_k[f(k)+λk] (penalidade λ) + DP de 1 camada; confira convexidade com brute force	DP, Busca binária
Mochila com Σ pesos ≤ S ou peso ≤ √S	≤ √(2S) pesos distintos	pesos distintos 1..k somam k(k+1)/2	Binary splitting/deque por peso distinto + bitset	DP, Números
Árvore + "escolha k vértices"	merge de filhos por tamanho é O(n^2) total	somar produtos de tamanhos de filhos dá O(n^2)	dp[v][k] com loops limitados por tamanho (ou ordem DFS pegar/pular em O(nk))	DP, Árvores
Árvore e estado é profundidade/altura (dist, nível)	Vetor por altura reaproveitado do filho mais profundo	só o filho mais profundo precisa de vetor novo	Long-path decomposition, O(n) total	DP, Árvores
Contagem de permutações com restrição entre adjacentes / soma de diferenças	Insira em ordem de valor, cuide das componentes	inserir do menor ao maior mantém componentes contíguas	Connected components DP, dp[i][c] (+ soma acumulada)	DP, Combinatória

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Permutação com sinais < > em posições	Posto relativo do último elemento	só o posto do último inserido decide as comparações	Insertion DP com prefix sums $O(n^2)$; ou EGF/ziguezague	DP, Combinatória
Valor pequeno ou resposta limitada (ovos, log passos, valor ≤ 10^5)	Estado e valor se trocam	estado e valor se trocam; o menor dos dois vira estado	dp[res] = melhor parâmetro (dp[ops][ovos] = alcance)	DP
Recorrência linear e n ≤ 10^{18}	Ordem k pequena	matriz ou Kitamasa calcula o termo n em tempo polinomial em k	Matriz / Kitamasa; sem fórmula: 2k termos + Berlekamp-Massey	DP, Combinatória
dp(n) parece polinômio em n	Diferenças finitas de ordem d+1 zeram	diferença de ordem d+1 de polinômio grau d é zero	d+1 valores brutos + Lagrange	DP, Combinatória
Esperança em estados com ciclos	Sistema linear (I − Q)E = 1	E = 1 + QE isola E num sistema linear	Gauss mod p / double (n ≤ 500); em cadeia: E[i] = a_i·E[0] + b_i	DP, Combinatória
Esperança de um tempo/contagem total	Linearidade: Σ P(evento k ainda ativo)	E[Σ 1_k] = Σ P(evento k)	Fórmula em vez de DP de estados	DP, Combinatória
Soma sobre subconjuntos / pares com AND = 0 / cobertura	Zeta e Möbius sobre máscaras	zeta soma sobre submáscaras; Möbius inverte	SOS (2^n n); k-colorações/cobertura por IE sobre SOS	DP, Bits, Combinatória
Partições de conjunto/union disjunta com peso	Primeiro bloco contém o menor bit	todo bloco tem um menor; fixe o que contém o bit mínimo	3^n submáscaras ou subset convolution 2^n n^2	DP, Bits
Partição de n em partes (distintas, ≤ k, ímpares)	Nº de partes distintas ≤ √(2n); conjugado troca "nº de partes" por "maior parte"	k partes distintas somam ao menos k(k+1)/2	dp[soma][nº de partes] O(n√n); pentagonal para p(n)	DP, Combinatória
Heap/lista ligada resolve k=1 e dp côncava em k (maximização)	Arrependimento dá resposta para todo k	v_l+v_r−v_i é o custo de trocar a escolha local	Escolha k não adjacentes por heap + lista, ou Aliens: bônus λ, g(λ)=max_k[f(k)+λk], resposta g(λ)−λk	DP, Arrays, Busca binária
Minimizar Σ|a_i − b_i| com b monótono/regra de vizinhança	f(x) convexa linear por partes	valor absoluto é convexo; slope trick mantém as quebras	Mesmo slope trick de "tornar não decrescente" acima, com heaps (O(n log n))	DP, Arrays
Grid com m ≤ 12–15 e vizinhança local	Perfil de m bits processado célula a célula	só a fronteira de m células interage com o futuro	Broken profile (plug DP; conectividade → parênteses em base 3/4)	DP
dp[i] depende de ciclo de volta / caminho com custo ≥ 0	Não é DAG por posição	estados viram grafo com pesos ≥ 0; Dijkstra resolve	Dijkstra/0-1 BFS sobre estados	DP, Grafos
Guloso com contraexemplo pequeno, "exatamente k" ou capacidade	Escolha local depende do futuro	se a escolha local depende do futuro, o estado o carrega	DP (mochila/estado extra); se côncava em k, guloso com arrependimento ou Aliens	DP, Arrays
Mochila com itens entrando e saindo no tempo	Cada item vive num intervalo	cada item vive num intervalo; segtree no tempo cobre todos	Segtree no tempo + dp copiada por nível (ou "undo" modular)	DP, Estruturas
Mochila com Σ valor pequeno e peso gigante	dp[v] = menor peso	trocar peso por valor reduz o estado a Σv	Troca estado e valor, O(n·Σv)	DP
Sequência cíclica / circular com custo de fechamento	Corte um ponto e feche	cortar num ponto transforma o ciclo em caminho	Duplicar, ou fixar estado inicial e rodar linear (|S| vezes), ou tr(M^n)	DP, Combinatória
dp[i][c] = custo + min c' ≠ c	Só o melhor e o segundo melhor importam	só o menor e o segundo menor podem ser o mínimo excluindo c	Top-2 por camada, O(nk)	DP
Contar objetos rotulados conexos/DAGs	Conexos: fixe o vértice 1 e sua componente; DAGs: IE sobre fontes/sorvedouros	fixar a componente do vértice 1 evita contagem dupla	Conexos: conn[n] = total − Σ_{k=1}^{n−1} C(n−1,k−1)·conn[k]·total_{n−k}; DAG: recorrência por fontes	DP, Combinatória, Grafos
a^b^c... mod m	Expoente enorme: reduza mod φ(m) somando φ(m)	o expoente se reduz mod φ(m), somando φ(m) quando é grande	Recursão pela cadeia φ(φ(·)) (≤ 2 $\log_2$ m níveis) com expoentes truncados	Combinatória

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Σ f(⌊n/i⌋) ou Σ sobre pares com gcd	⌊n/i⌋ tem 2√n valores; gcd → φ ou μ	⌊n/i⌋ assume 2√n valores	Blocos + prefixos de φ/μ; para n ≥ 10^{10} Dirichlet sieve O(n^{2/3})	Combinatória
Soma de função multiplicativa até n ≈ 10^{10}–10^{12}	Só 2√n valores ⌊n/i⌋ importam	só os 2√n quocientes ⌊n/i⌋ importam	Lucy → Min_25, ou powerful numbers, ou f = g∗h	Estruturas
Σ ⌊(a i + b)/m⌋ ou pontos de rede sob reta	Euclides dobra os parâmetros	Euclides reduz (a, m) como no gcd	Floor sum recursivo O(log)	Geometria
Equação em inteiros "sem solução"	Resíduos limitados (mod 3, 4, 7, 8, 9)	uma solução existiria em todo módulo; testar poucos elimina	Testar todos os resíduos pequenos; LTE para v_p	Construtivos
Pares com lcm ou gcd = g em array com a_i ≤ 10^6	Zeta sobre múltiplos/divisores	agrupar por g vira condição de múltiplos/divisores	GCD/LCM convolution + Möbius	Estruturas
"Quantos colares/giros/espelhos distintos"	Contagem por órbitas	Burnside: a média dos pontos fixos conta órbitas	Só giros: (1/n)Σ_{d|n} φ(d)k^{n/d}; com espelhos (diédrico): (1/(2n))[Σ_{d|n} φ(d)k^{n/d} + R], R = n·k^{(n+1)/2} (n ímpar) ou (n/2)(k^{n/2}+k^{n/2+1}) (n par)	Teoria dos números
"Conte estruturas conexas" (grafos, perms)	Conte todas e tire as com componente de vértice 1 de tamanho k	toda estrutura tem a componente do vértice 1	Recorrência c(n) = total − Σ C(n−1, k−1)c(k)·resto; ou log de EGF	Grafos
"Conte árvores/ligações entre componentes"	Árvores rotuladas = Prüfer	Prüfer é bijeção entre árvores e sequências	Fórmula n^{k−2}∏ s_i, ou Matrix-Tree se pesos	Grafos, Árvores
Contar circuitos eulerianos	Cada vértice ordena suas saídas	cada vértice ordena suas saídas de (deg−1)! formas	BEST: t_w·∏(deg−1)!, t_w por Matrix-Tree	Grafos
Caminhos não-cruzantes em grid	Troca de rotas nos cruzamentos	trocar sufixos nos cruzamentos cancela o sinal	LGV: det[C(·)]	Grafos
Sequência (ou DP) com n ≤ 10^{18} e ordem pequena	GF racional	GF racional ⇒ a sequência satisfaz recorrência linear	Mesma receita de "recorrência linear" acima: Berlekamp–Massey + Kitamasa	DP, Estruturas
Valor é polinômio em n	Diferenças finitas constantes	diferenças finitas de ordem d+1 anulam	Mesma interpolação de "dp(n) polinômio" acima: Lagrange com d+1 pontos	DP
E[tempo] em passeio ou processo com ganho linear	Existe potencial Φ com E[ΔΦ] = −1 (ou martingale)	Φ com E[ΔΦ] = −1 dá E[T] por optional stopping	Optional stopping: E[T] = Φ_0 − Φ_fim	Teoria dos jogos, Grafos
Probabilidade de "n eventos aleatórios sem coincidência"	Aniversário	P(sem colisão) ≈ e^(−n^2/(2N))	e^{−n^2/(2N)} ("sem coincidência"); 1 − e^{−n^2/(2N)} = colisão; 64 bits não basta para n ≈ 10^9	Aleatorização, Strings
"Existe construção com ≥ metade"	Média acima do limiar	média acima do limiar força algum objeto acima dele	Método probabilístico + esperanças condicionais	Construtivos, Aleatorização
C(n, k) mod m composto ou p^e	Fatore m; separe v_p(n!) e o fatorial sem p	fatore m, resolva cada p^e e junte por CRT	n!_p por blocos de p^e (Granville); o sinal (−1)^{⌊n/p⌋} só vale mod p; junte por CRT	Teoria dos números
Perm/estrutura com "pai < filho"	Cada subárvore tem mínimo na raiz com prob. 1/sz	a raiz é o mínimo da subárvore com prob. 1/sz	n!/∏ sz(v)	Árvores
Soma com |a_i − a_j| ou (a_i − a_j)^2	Ordenar / completar quadrado	ordenar conta quantos pares têm cada a_i como extremo	Σ a_(i)(2i−n−1) ou nΣa^2 − (Σa)^2	Arrays
n+1 prefixos em 2^k valores, com n+1 > 2^k (não "n > nº de bits")	Pigeonhole: dois prefixos xor iguais	há mais prefixos que valores; dois coincidem	Existe subarray/subconjunto de xor 0; limita o brute force	Bits, Combinatória
Dado s = a+b e x = a⊕b	s − x = 2(a&b)	s − x conta duas vezes os bits em comum	Checar paridade/subconjunto de bits; 2^popcount(x) soluções	Bits, Teoria dos números
Max/min xor de passeio em grafo	Passeio = caminho da árvore + ciclos fundamentais	todo ciclo é combinação dos ciclos fundamentais	Base linear dos ciclos + px	Bits, Grafos

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Max xor em [l, r] / com versões	Prefixos são versões	prefixos são versões; a trie persistente guarda cada uma	Trie persistente ou base com índice mais recente	Bits, Estruturas
Pares com a_i & a_j = 0 / "contém"	Complemento é submáscara	ser disjunto de a é ser submáscara do complemento de a	SOS sobre cnt e consulta em ~a	Bits, DP
"Soma sobre k passos com xor total x" / xor convolution	Caracteres do grupo Z_2^n	WHT diagonaliza a convolução em Z_2^n	Walsh–Hadamard, potência pontual, inversa	Bits, Combinatória
a_i + a_j com bit k: contar/xor das somas	Bit k só depende de a mod 2^(k+1)	só o resíduo mod 2^(k+1) afeta o bit k	Ordenar mod 2^(k+1) + two pointers por bit	Bits, Arrays
Distância de Hamming mínima/≤ d em 2^k máscaras	Hipercubo é grafo	virar um bit é andar numa aresta do hipercubo	BFS multi-fonte por flip de bit	Bits, Grafos
Operação a_i ← a_i⊕a_j	Span é invariante	operações elementares preservam o espaço gerado	Base linear, rank; resposta em função de n − rank	Bits, Construtivos
Contar ocorrências de vários padrões	Nó do AC sobe pelos links	cada ocorrência termina num nó; some a subárvore de links	Árvore de links + soma de subárvore (BIT/Euler)	Strings, Árvores, Estruturas
Ocorrências de p em texto de tamanho ≤ 2|p|	Se duas estão a distância < |p|, d é período	sobreposição a distância menor que o padrão força essa distância a ser período	Ocorrências formam PA; guardar (início, razão, qtd)	Strings
Bordas de s / períodos	Borda b ⇔ período n − b; Fine–Wilf	borda b ⇔ período n−b	Bordas ≥ n/2 em PA; O(log n) PAs; série link	Strings
Problema pede "ocorrências de t" e "substring t de s"	Estado do SAM = classe de endpos	estados do SAM são classes de endpos	cnt na subárvore da link tree; binary lifting por len	Strings, Árvores
Suffix tree de s	Link tree do SAM de rev(s)	a link tree do SAM de rev(s) é a suffix tree de s	Construa SAM de rev(s) em vez de Ukkonen	Strings
SA + LCP, "todos os pares de sufixos"	LCP(i, j) = min no intervalo do array lcp	LCP(i, j) é o mínimo do lcp no intervalo	Soma de mínimos de subarrays por pilha monotônica	Strings, Arrays
Palíndromos terminando/começando em cada posição	Manacher dá um raio por centro	Manacher dá raios; PAs atualizam todos de uma vez	Adicionar PAs por segunda diferença	Strings, Arrays
Partição em palíndromos / fatores	Palíndromos-sufixo em O(log n) PAs	palíndromos-sufixo formam O(log n) PAs	Série link na eertree, dp em O(n log n)	Strings, DP
String definida recursivamente (Fibonacci, SLP)	Só |p|−1 caracteres de borda importam	só a borda menor que o padrão cruza a concatenação	Recorrência cnt_n = cnt_{n−1} + cnt_{n−2} + cruzamento	Strings, DP
Rotações / "é potência"	Rotação de s é substring de s+s	rotação é substring de s+s; o deslocamento é o período	A menor ocorrência de s em s+s (p > 0) dá a raiz primitiva; qualquer p dá uma raiz de comprimento gcd(p, n)	Strings
Árvore de parênteses / árvore ↔ sequência	DFS escreve (ao entrar e) ao sair	subárvore é intervalo fechado; LCA é mínimo do balanço	Subárvore = intervalo, LCA = RMQ do balanço	Strings, Árvores
Substring válida de parênteses com updates	Válida ⇔ soma 0 e min prefixo ≥ 0	válida ⇔ soma 0 e nenhum prefixo negativo	Segtree com (soma, min prefixo)	Strings, Estruturas
LCS/LIS ou LCS com n·m/64 viável	Linha da DP é bitset monotônico	a linha da DP é monótona e cabe em bits	Fórmula V = (V + (V & M)) | (V & ~M)	Strings, Bits, DP
Mismatch ≤ k entre padrão e texto	Cada erro é um salto de LCE	cada erro salta direto para depois do LCE	Kangaroo O(k) por posição; ou FFT por letra	Strings
Operação "desfazer último caractere"	Amortização de KMP/Z quebra	KMP/Z amortiza supondo que o texto só cresce	Autômato completo ou pilha de estados	Strings
m ≤ n + k pequeno, queries de distância	O grafo é uma árvore + poucas arestas	uma árvore geradora usa n−1 das n+k arestas	Árvore geradora + Dijkstra/BFS dos ≤ 2k + 2 extremos, distância em árvore por LCA	Grafos, Árvores

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Distância máxima de cada vértice / "qual é o mais distante"	Sempre termina numa ponta do diâmetro	o mais distante de qualquer vértice é ponta do diâmetro	Duas BFS + max(d(v, a), d(v, b)); centro na metade	Árvores, Grafos
"Menor valor tal que u e v se conectam" / minimax	Caminho na MST	o gargalo é a aresta máxima da MST no caminho	MST + LCA (max) ou Kruskal reconstruction tree	Grafos, Árvores
Maximizar emparelhamento com conflitos/coberturas	Bipartido + König	König liga matching, cobertura e independente	Matching ⇒ cobertura mínima ⇒ conjunto independente máximo / antichain (Dilworth)	Grafos
Escolher conjunto com lucros e dependências ou custos de pares	Corte mínimo / closure	∞ força a closure; o corte paga o que sai	S/T com ∞ nas dependências; soma de lucros − corte	Grafos, Estruturas
Orientar arestas com grau ou paridade pedidos	Euler (vértice auxiliar para ímpares) ou fluxo	grau vira balanceamento entre entrada e saída	Circuito euleriano e orientação ao longo dele; senão Hall/fluxo	Grafos, Construtivos
Remover vértices/arestas em ordem e pedir conectividade/soma	Inverta o tempo	ao contrário, remoção vira inserção	Adicionar na ordem reversa com DSU (Floyd: adicionar vértice em $O(n^2)$)	Grafos, Estruturas
Soma de caminhos / contribuição por aresta	Aresta contribui sz·(n − sz)	cada aresta separa sz e n−sz vértices	Subárvore sizes; soma Σ w·sz·(n − sz)	Árvores, Combinatória
Vértices marcados com distância ao mais próximo	Centroid tree de altura log n	cada centroide guarda o melhor marcado do seu lado	best[c] por centroide + dist(v, c) por ancestral	Árvores, Estruturas
Update em caminho, query pontual/total	Diferenças em árvore	o caminho é decomposto em quatro atualizações pontuais	diff[u]++, diff[v]++, diff[lca]--, diff[pai(lca)]--; soma de subárvore	Árvores, Estruturas
Árvores a comparar sem raiz fixa	Hash canônico (AHU) nos centros	AHU identifica subárvores pelo multiconjunto de ids	Id por multiconjunto ordenado de filhos; compare nos 1–2 centros	Árvores
Nota/sequência de graus pedida	Erdős–Gallai / Havel–Hakimi ou Landau (torneio)	Erdős–Gallai testa; Havel–Hakimi constrói	Construção gulosa pelo maior grau	Grafos, Construtivos
Dependência por grupos de ancestral (primeiro ancestral com propriedade)	Monotonia ao subir	subir é monótono; binary lifting pula em O(log)	Binary lifting com busca em O(log); ou grafo de saltos	Árvores, Grafos
Remover vértice/aresta, "e se esta aresta sumisse"	Árvore de caminhos mínimos / DFS tree	arestas fora da árvore cobrem o caminho que substituem	Aresta fora da árvore cobre caminho (replacement path com DSU de pulo)	Grafos, Árvores
"Cada elemento tem 2 opções" com conflitos pareados	2-SAT	cada conflito pareado é uma cláusula 2-SAT	Cláusulas + SCC; cardinalidade ≤ 1 por variáveis de prefixo	Grafos
Soma de f(i) em intervalo com histórico de versões	Tempo como dimensão	versões são o eixo do tempo; o histórico acumula por versão	Segtree com soma histórica (acumulado ao longo das versões) ≠ persistente por versão (consulta por versão)	Estruturas
Pontos sob reta, "acima/abaixo" em muitas queries	Dualidade (a, b) ↔ y = ax − b	dualidade troca ponto por reta e consulta por envelope	CHT / Li Chao / hull	Geometria, Estruturas, DP
Retângulo/união/cobertura em plano	Sweep em x, eventos de abertura/fechamento	sweep em x; a segtree mantém o y coberto	Segtree em y com (cobertura, comprimento)	Geometria, Estruturas
Ordenar pontos por ângulo	Semiplano + cross (não atan2)	cross compara sem ambiguidade de atan2	Sweep angular com dois ponteiros	Geometria
Triângulos/conjuntos "contêm a origem"	Complemento: todos no mesmo semiplano	não conter ⇔ todos num mesmo semiplano	Ordenar por ângulo + C(k, 2)	Geometria, Combinatória
Manhattan com muitas queries	Rotacionar (x + y, x − y) ou 4 octantes	a rotação troca a métrica por Chebyshev	BIT/segtree por octante, MST Manhattan	Geometria, Estruturas, Grafos
Query em intervalo só com add (sem remoção)	Mo com rollback / duas pilhas	só adicionar permite desfazer em O(1)	Mo rollback, DSU rollback	Estruturas

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Fila/janela com operação sem inverso (gcd, matriz)	Duas pilhas com agregados	duas pilhas dão fila com agregados	Janela deslizante em O(1) amortizado	Estruturas, Arrays
Vetor xor máximo em subarray	Prefix linear basis com timestamp	base com timestamp guarda o que existe até r	Base linear persistente	Estruturas, Bits
"Primeiro i com condição" em estrutura de intervalo	Monotonia + agregado no nó	o agregado do nó diz de que lado está o primeiro	Descida na segtree/BIT em vez de binária+query	Estruturas, Busca binária
Conectividade com arestas ativas em intervalo de tempo	Insira aresta nos O(log) nós do tempo	cada aresta é inserida nos nós do seu intervalo	Segtree no tempo + DSU rollback	Estruturas, Grafos
Problema geométrico com n pequeno e n^2 eventos	Rotação de direção; ordem muda em pares	a ordem angular só muda quando dois pontos se alinham	Rotating sweep em $O(n^2 \log n)$	Geometria
Elemento/reta/ponto "comum a ≥ fração δ"	Amostrar e verificar	a maioria dos sorteios cai no conjunto comum	$1/\delta \cdot \log(1/\varepsilon)$ tentativas	Aleatorização, Geometria
Corte/conectividade em grafo com remoções	Valor aleatório em cada aresta, xor	hash xor identifica o corte pela soma de valores	Hash de corte por xor	Aleatorização, Grafos
Verificar identidade algébrica (matriz, polinômio)	Avaliar em ponto aleatório	polinômio não nulo zera em poucos pontos	Freivalds / Schwartz–Zippel	Aleatorização
Otimização sem estrutura clara, TL longo	Vizinhança + delta O(1)	aceitar pioras ocasionais escapa de ótimos locais	Simulated annealing / hill climbing	Aleatorização
Fila de n+1 itens e módulo n; "existe subarray com soma divisível"	n + 1 prefixos em n restos ⇒ dois iguais	n+1 prefixos em n restos ⇒ dois iguais	Pigeonhole direto; construção: guarde o primeiro índice de cada resto	Combinatória, Arrays, Construtivos
Sequência de n números distintos; pede sequência crescente ou decrescente longa	Erdős–Szekeres: LIS·LDS ≥ n	pares (LIS, LDS) são distintos e LIS·LDS ≥ n	Um dos dois ≥ √n; use o rótulo (LIS até i, LDS até i) como par único	Arrays, Combinatória
2n−1 inteiros (ou mais) e "n com soma divisível por n"	Erdős–Ginzburg–Ziv	entre 2n−1 inteiros sempre há n com soma ≡ 0	Existência garantida; construção por indução em fatores primos / brute	Combinatória, Construtivos
Ordem de aplicar operações em árvore/floresta com precedência pai → filho	Ordens topológicas = n!/Π tamanho(v)	cada subárvore escolhe a ordem dos seus sz vértices	Multiplique inversos modulares dos tamanhos de subárvore	Árvores, Combinatória
Contar árvores com graus dados	Código de Prüfer	no código de Prüfer, i aparece d_i−1 vezes	(n−2)!/Π(d_i−1)!; Cayley n^(n−2)	Árvores, Combinatória
Contar spanning trees de um grafo	Matrix-tree	o determinante do Laplaciano menor conta árvores geradoras	Determinante do Laplaciano (menor) por Gauss mod p primo (pivôs invertíveis); p composto → CRT/Bareiss; n ≤ 400	Grafos, Combinatória
Caminhos disjuntos em vértices, k pares (s_i, t_i) em DAG	Lema LGV	o determinante soma permutações com sinal; cruzamentos se cancelam	Det da matriz de contagem de caminhos; exige fontes/sumidouros compatíveis (ordem não cruzada), senão o determinante é assinado e pode zerar	Combinatória, Grafos
Domino/ladrilhos em tabuleiro n × m com n ≤ 10	Estado = perfil de bordas	só a última coluna processada importa	DP broken profile O(n·m·2^n); m enorme → matriz	DP, Combinatória
Soma Σ_i Σ_j f(gcd(i, j)) ou Σ gcd(i, n)	Agrupe por d = gcd	agrupar por d = gcd separa a soma	Σ_d f(d)·(contagem de pares com gcd d) via φ ou Möbius	Teoria dos números, Combinatória
C(n, k) mod 2, resposta mod 2	Lucas p = 2	Lucas p=2: ímpar ⇔ k ⊆ n	Ímpar ⇔ k & (n − k) = 0 ⇔ k ⊆ n; paridade de soma de binomiais por bits	Combinatória, Bits

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Xor de todos os subarrays/pares/subconjuntos	Cada valor aparece uma quantidade c de vezes	cada elemento aparece i(n−i+1) vezes; só a paridade importa	Só importa c mod 2: conte quais têm contagem ímpar (subarrays: i·(n − i + 1); pares não ordenados: n − 1; ordenados: 2(n − 1); subconjuntos: 2^{n − 1})	Bits, Arrays
"Maximizar xor de caminho" em grafo geral	Qualquer caminho = caminho da árvore + combinação de ciclos	os ciclos fundamentais geram todos os ciclos	Base linear dos ciclos fundamentais; guloso do bit alto	Bits, Grafos
Operação "a_i ^= a_j" repetida	Espaço gerado é invariante	operações elementares preservam o span	Gauss em GF(2) com bitset; posto, base canônica	Bits, Construtivos
Contagem polinomial em n de grau d pequeno	Resposta é polinômio de grau ≤ d	grau ≤ d ⇒ d+1 pontos determinam	Calcular d+1 valores por brute e interpolar (Lagrange); "n até 10^{18}"	Combinatória, DP
Recorrência linear desconhecida, termos pequenos fáceis	Se for linear de ordem d, 2d termos bastam	2d termos determinam a recorrência de ordem d	Berlekamp–Massey + Kitamasa/matriz para n enorme	DP, Combinatória, Teoria dos números
Σ_{i ≤ n} i^k, k até 10^3 e n até 10^{18}	Polinômio de grau k + 1 em n	somar polinômio de grau k dá grau k+1	Interpolação de Lagrange com k + 2 pontos, ou Bernoulli	Combinatória, Teoria dos números
Grau-sequência dada, "existe grafo simples"	Condição de realização	os teoremas de realização caracterizam exatamente essas sequências	Erdős–Gallai / Havel–Hakimi; torneio: Landau; matriz 0/1: Gale–Ryser	Construtivos, Grafos
Multigrafo bipartido k-regular	Hall ⇒ matching perfeito	k-regular ⇒ a vizinhança de S tem tamanho ao menos o de S; Hall se aplica	Decomponha em k matchings perfeitos; Δ-coloração de arestas (König)	Grafos, Construtivos
Torneio (um sentido por par) e "caminho passando por todos"	Rédei: todo torneio tem caminho hamiltoniano	Rédei: todo torneio tem caminho hamiltoniano	Inserção binária ou merge sort com comparador "u vence v"	Grafos, Construtivos
Grafo conexo com m par, "decomponha em caminhos de 2 arestas"	DFS pós-ordem	parear no filho deixa no máximo a aresta do pai	Pareie arestas incidentes no filho; sobra a aresta ao pai	Grafos, Construtivos
Orientar arestas para ficar fortemente conexo	Sem pontes (2-aresta-conexo) é condição necessária e suficiente	Robbins: 2-aresta-conexo ⇔ orientação fortemente conexa	DFS: árvore para baixo, retorno para cima	Grafos
k vértices de grau ímpar em um grafo conexo	Cobertura por max(1, k/2) trilhas (k = 0: 1, grafo euleriano)	cada trilha consome no máximo 2 ímpares; pareie-os	Ligar pares de ímpares com arestas virtuais e achar Euler	Grafos, Construtivos
Cobrir caminhos/DAG com menos caminhos	V − matching no grafo split	cada caminho economiza um matching	Disjunto em vértices; se podem se sobrepor, fecho transitivo antes	Grafos, Combinatória
Maior antichain / maior conjunto sem relação em poset	Dilworth	Dilworth: antichain máxima = cobertura mínima por cadeias	Tamanho = n − matching no fecho transitivo (ou min cadeias)	Grafos, Combinatória
Maior conjunto independente em grafo bipartido	König	König: α = n − ν	n − matching máximo; reconstrua pelo corte (alcançáveis no residual)	Grafos
Escolher intervalos com cada ponto coberto ≤ k vezes, max peso	Matriz de intervalos consecutivos é TU	a matriz de intervalos é totalmente unimodular	Fluxo de custo mínimo numa linha de pontos, capacidade k	Grafos, Arrays
Atribuir tarefas com janela [l_i, r_i] e capacidade 1 por slot	Hall só para intervalos contíguos	Hall para intervalos basta checar [L, R]	Checar #itens com janela ⊆ [L, R] ≤ R − L + 1; ou EDF com heap	Arrays, Grafos
Tarefas com prazo e lucro, 1 unidade de tempo	Matroide de agendamento	matroide de agendamento ⇒ guloso por lucro	Ordene por lucro e use DSU para o último slot livre	Arrays, Estruturas

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Rotular cada vértice com A/B, custo unário e custo se vizinhos diferem	Energia submodular	energia submodular ⇔ corte mínimo	Corte mínimo s-t (segmentação, "projeto" generalizado)	Grafos
Custo convexo por unidade numa aresta	Marginais crescentes	marginais crescentes viram arestas paralelas	Duplique a aresta em cópias de custo marginal, min cost flow	Grafos, DP
Restrição de razão (min Σa/Σb) num ciclo	Binária em λ	para λ fixo, o ciclo ótimo minimiza Σ(a−λb)	Karp só para média por aresta (Σa / número de arestas); com b arbitrário, binária em λ + Bellman-Ford com custo a − λb	Grafos, Busca binária
Vários agentes se espalham ao mesmo tempo, "quem chega antes"	Frente simultânea	BFS multi-fonte dá a origem mais próxima	BFS/Dijkstra multi-fonte guardando a origem (Voronoi em grafo)	Grafos
"Se removermos o vértice v, u ainda é alcançável?"	Pertinência na árvore dominadora	dominadores são exatamente os vértices necessários	Árvore dominadora (Lengauer–Tarjan) em grafo direcionado/DAG	Grafos
Restrições "maior aresta no caminho" / "caminho de gargalo"	MST contém o gargalo	na KRT, o LCA carrega o peso do gargalo	Árvore de reconstrução de Kruskal: LCA = peso do gargalo	Grafos, Árvores
Grafo com pesos de aresta pequenos W, distâncias grandes	Dial	distâncias inteiras ≤ nW cabem em buckets	Bucket queue por distância em O(m + nW); estados (v, tempo) até n·W são outra técnica (DP em camadas)	Grafos
Passos grande k em grafo, "caminho com exatamente k arestas"	Potência da matriz de adjacência	A^k conta caminhos de k arestas	Min-plus ou contagem, exponenciação por doubling	Grafos, DP
Terminais k ≤ 10 e custo mínimo para conectá-los	Árvore de Steiner	a árvore ótima se parte em v em dois subconjuntos de terminais	dp[mask][v] com 3^k·n + 2^k·m log	Grafos, DP
Grafo com m arestas e "triângulos / pequenos subgrafos"	Oriente de menor para maior grau	cada aresta vai para o vértice de maior grau; a saída é limitada	Grau de saída ≤ √(2m); triângulos em O(m√m)	Grafos, Estruturas
Árvore e random walk com esperança de passos	Tempo de atingimento por aresta	ao remover a aresta, o lado de u determina o tempo de retorno	E[T de u a v adjacentes] = 2·(arestas no lado de u) + 1; soma pelo caminho	Árvores, Combinatória
Formigas/bolas idênticas que colidem e voltam	Atravessam sem interagir	colisão equivale a atravessar trocando rótulos	Trajetória livre; a ordem relativa fica, só os rótulos trocam	Arrays, Construtivos
Jogador A e B alternam, últimos a mover perde	Misère	só o caso de todas as pilhas 1 inverte a paridade	Nim: igual ao normal exceto todas as pilhas ≤ 1; outros: brute força a diferença	Teoria dos jogos
Tokens independentes em DAG ou pilhas somadas	Sprague-Grundy	jogos independentes somam por xor	xor dos Grundy; token único sem soma → só vitória/derrota	Teoria dos jogos, Grafos
Jogo com lance que o oponente pode desfazer	Lance reversível exige definição precisa (posição ≤ original, jogo finito)	apagar o lance não é jogada válida; use as respostas	Não é simplesmente apagar o lance: substitua-o pelas opções da resposta	Teoria dos jogos
Jogo em tabuleiro retangular de alinhamento/conexão	Roubo de estratégia	se o 2º tivesse estratégia, o 1º a roubaria	1º nunca perde; Chomp, Hex: 1º ganha, a questão é construtivo ou espelho	Teoria dos jogos, Construtivos
Jogo "token em grafo não-direcionado, vértices não repetem"	Geography bipartido	o matching máximo decide a vitória	Ganha quem move de v ⇔ v está em todo matching máximo	Teoria dos jogos, Grafos
Jogo de Euclides ou subtração com números grandes	Quociente ≥ 2 dá escolha	quociente ≥ 2 deixa escolher a paridade	Simule Euclides e conte lances forçados por paridade	Teoria dos jogos, Teoria dos números

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Jogo de moedas (virar moedas)	Cada cara é jogo independente	cada moeda cara é um jogo independente	Soma: xor de g(i) por posição de cara (Turtles: g(i) = i; Ruler: maior potência de 2 em i)	Teoria dos jogos
Jogo de palavras (prefixo de dicionário) com k rodadas	Trie + (pode forçar ganhar, pode forçar perder)	cada posição é um nó da trie e o resultado só depende dele	Se ambos → 1º ganha; só ganhar → paridade de k; senão → 2º	Teoria dos jogos, Strings
Jogo com itens usados de 1..n e soma-alvo	Estado = máscara	popcount do subconjunto usado dá a vez	popcount dá a vez; dp[mask] com n ≤ 20	Teoria dos jogos, DP
Jogo infinito/ciclos, resultado "empate"	Retrógrada com contador de saídas	o grafo finito propaga valores a partir dos sorvedouros	Vitória se algum sucessor é derrota; derrota se todos são vitória; resto empate	Teoria dos jogos, Grafos
Grundy grande demais ou n 10^{18}	Eventual periodicidade (só garantida com conjunto de subtração finito)	conjunto finito de jogadas ⇒ sequência eventualmente periódica	Brute 10^4 valores é heurístico: ache período e pré-período, confirme e depois consulte por mod	Teoria dos jogos, Teoria dos números
Operação associativa sem inversa em janela deslizante	Duas pilhas	duas pilhas mantêm agregados de prefixo e sufixo	Fila a partir de duas pilhas guardando agregado; O(1) amortizado	Estruturas, Arrays
Operação associativa sem inversa, queries estáticas, n $5{\cdot}10^5$	Disjoint sparse table / sqrt tree	dois blocos pré-computados cobrem qualquer intervalo	Query O(1) com 2 acessos	Estruturas
Estrutura só suporta insert e rollback, mas dados têm remoção arbitrária	Segtree no tempo	cada item está ativo num intervalo de tempo	Offline: cada item vive num intervalo; DFS na segtree com rollback	Estruturas, Grafos
Elemento majoritário em subarray, sem estrutura pesada	Boyer–Moore não é associativo (o merge preserva só a maioria estrita)	o saldo cancela pares diferentes e preserva o majoritário	Segtree de (candidato, contagem); verificação final por posições + binária	Estruturas, Aleatorização
Transformações compostas x → a·x + b sobre intervalos	Monoide não comutativo	composição é associativa mas não comutativa	Segtree que guarda (a, b) respeitando a ordem de composição	Estruturas, Combinatória
Computar sobre pares com a_i = a_j em vez de todos	Limite de multiplicidade	$\Sigma\, c_v^2 \le n{\cdot}\max c$ limita os pares iguais	$\Sigma\, c_v^2 \le n{\cdot}\max c$; agrupa por valor; pesado/leve em c_v	Estruturas, Arrays
Pontos e retas em y = ax + b, "melhor reta num x"	Dualidade ponto-reta	dualidade transforma a consulta em máximo de retas	(a, b) viram pontos; envelope = fecho convexo ⇒ CHT / Li Chao	Geometria, DP
Restrições lineares em 2 variáveis, maximizar linear	Programação linear 2D	o ótimo de um PL ocorre em vértice	Interseção de semiplanos; ótimo em um vértice	Geometria
Minimize $\Sigma \lvert x_i - t_i \rvert$ com restrições x_i ≤ x_{i+1}	Slope trick	cada termo é convexo; heaps mantêm as quebras	Heap com pontos de quebra; ótimo O(n log n)	DP, Arrays
Custo convexo em cadeia de variáveis (Lipschitz, monotônico)	f convexa por partes	convexidade permite representar por inclinações	Slope trick: guarde as inclinações em heaps	DP, Estruturas
Mochila com Σ pesos pequena e muitos itens iguais	Pesos distintos ≤ √(2S)	pesos distintos ≤ √(2S)	Binary splitting ou deque por resto; bitset para viabilidade	DP
Partição em somas com n 10^5 e soma ≤ 10^5	Subset sum com multiplicidades	bitset processa a soma em blocos de 64 bits	bitset com divisão binária (n√n/64)	DP, Estruturas
Strings: bordas/períodos de prefixo	Lema de periodicidade	o lema de periodicidade agrupa as bordas em PAs	Bordas formam O(log n) progressões aritméticas; prefixo-função em O(n)	Strings
Contar padrões que casam com curinga	Vira convolução	casar posições vira convolução	FFT/NTT por caractere ou por soma de quadrados	Strings, Combinatória
Reflexões e espelhos (bolas, luz, bilhar)	Espelhe o mundo	espelhar o mundo desfaz o reflexo	Trajetória reta em grade replicada; resultado por gcd/mod	Geometria, Teoria dos números

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Problema de olimpíada "mostre que sempre há..."	Extremo/invariante/pigeonhole provam	extremo/invariante/pigeonhole dão existência	Transforme a prova em construção (indução forte + reduzir caso)	Construtivos, Combinatória
Soma de produtos A·B sobre pares de blocos	Divisão e conquista + FFT	A·B sobre pares de blocos é uma convolução	Convolução com D&C para recorrência tipo online (CDQ + FFT)	Combinatória, Estruturas
Árvore de decisão limitada (k perguntas, N respostas)	N ≤ b^k	b respostas por pergunta ⇒ b^k folhas	Limite inferior de queries é ⌈log_b N⌉; ache a pergunta que divide em b partes	Construtivos, Teoria dos jogos
Soma/convolução sobre pares com índices pequenos ($\leq 10^6$)	Vira produto de polinômios	produto de polinômios = convolução	NTT/FFT; mod arbitrário → 3 NTTs + CRT	Álgebra, Strings, Combinatória
Contagem de estruturas rotuladas conexas	Total = exp(conexas) em EGF	total = exp(conexas) em EGF	ln/exp de série formal (O(n log n))	Álgebra, Combinatória, Grafos
n até 10^{18} e recorrência linear de ordem k	Termo n = x^n mod polinômio característico	x^n mod polinômio característico dá o termo	Kitamasa/Bostan–Mori; BM para achar recorrência	Álgebra, DP, Combinatória
Resposta é polinomial em n (parece)	grau d ⇒ d+1 pontos bastam	d+1 pontos determinam grau d	Brute força d+2 valores, depois Lagrange	Álgebra, Combinatória, Olimpíada
$\Sigma_{k\leq n} k^d$ com n enorme	Polinômio de grau d + 1 em n	somar um polinômio de grau d eleva o grau em 1	Lagrange O(d) ou Bernoulli	Álgebra, Teoria dos números
Contar árvores geradoras / ciclos eulerianos	Determinante do Laplaciano	o Laplaciano codifica arestas; o determinante enumera árvores	Matrix-tree, BEST	Álgebra, Grafos, Combinatória
k caminhos disjuntos em DAG/grid	Sinal de permutação cancela cruzamentos	LGV: o determinante cancela cruzamentos	LGV (determinante) com fontes/sumidouros compatíveis; senão o determinante é assinado e pode zerar	Álgebra, Grafos
Sistema de equações xor	Linear sobre GF(2)	Gauss em GF(2); 2^(n−posto) soluções	Gauss com bitset, 2^{n − posto}	Álgebra, Bits
"Todo subconjunto" com n ≤ 20 e combinação OR/AND/XOR	Transformadas de conjuntos	as transformadas diagonalizam OR, AND e XOR	zeta/Möbius, subset convolution, Walsh	Álgebra, DP, Bits
Existência garantida por pombal / Erdős–Szekeres	Só precisa construir	a prova de existência vira construção	Prefixos mod n, LIS/LDS, EGZ	Olimpíada, Construtivos
Operação que sempre reduz uma grandeza inteira	Monovariante limita passos	monovariante inteiro limita os passos	Simule até estabilizar ou conte inversões	Olimpíada, Construtivos, Arrays
Grafo/instância sem $K_{2,2}$ ou sem triângulo (separar)	Sem $K_{2,2}$: $m = O(n^{3/2})$ e $\Sigma C(d_v,2) = O(n^2)$; sem triângulo: $m \leq n^2/4$	KST: sem C4 ⇒ $m = O(n^{3/2})$; Mantel: $m \leq n^2/4$	Pares de vizinhos em $O(n^2)$ só vale para $K_{2,2}$-free; triangle-free pode ter $\Sigma C(d_v,2) = \Theta(n^3)$ ($K_{n/2,n/2}$)	Olimpíada, Grafos
Grafo planar com n grande	m ≤ 3n − 6, degenerescência ≤ 5	Euler: m ≤ 3n−6; grau médio < 6	Orientação grau ≤ 5; dual para corte/fluxo	Olimpíada, Grafos
"Maximize/minimize" com certificado	Dual do LP	o dual do LP certifica o ótimo	Fluxo = corte, Hall/König, potenciais da Hungarian	Otimização, Grafos
Escolhas sem ciclo / ≤ c por cor / lin. independentes	Conjuntos formam matroide	matroide ⇒ guloso ótimo; 2 matroides ⇒ interseção	Guloso por peso (1 matroide) ou interseção de 2 matroides (polinomial; 3+ é NP-difícil)	Otimização, Grafos, Bits
f(k) com k itens parece convexa	Penalize cada item com λ	penalizar λ move o ótimo para o k desejado	Aliens/WQS + busca binária em λ	Otimização, DP, Busca binária
Custo convexo por unidade de fluxo	Divida em arestas unitárias crescentes	marginais crescentes = arestas unitárias em série	Min-cost flow com custos marginais	Otimização, Grafos

Vi no enunciado	Então	Por quê	Logo tente	Aparece em
Mover unidades em linha ou círculo	Custo = Σ |prefixo|	custo = soma dos fluxos prefixos; a mediana minimiza	Mediana dos prefixos (círculo)	Otimização, Arrays
Piso/teto com a·i + b e n ≈ 10^9	Reta com pontos de rede	a reta se dobra como no algoritmo de Euclides	Floor sum (Euclides)	Olimpíada, Teoria dos números
Sequência {kα}, "girar por α"	Equidistribuição / três intervalos	para α irracional, os múltiplos deixam no máximo três lacunas	Frações contínuas, Stern–Brocot	Olimpíada, Teoria dos números
Esperança de tempo em passeio linear	Existe potencial com Δ = −1	potencial com Δ = −1 e optional stopping	Martingal / optional stopping	Olimpíada, Combinatória
Grafo denso com Dirac/Ore	Hamiltoniano garantido	grau mínimo alto força hamiltoniano; Pósa constrói	Pósa/rotações para construir	Olimpíada, Grafos
Problema min-max com atribuição	Verificação monótona	viabilidade é monótona no limite	Binária + fluxo; checar capacidades	ICPC, Busca binária, Grafos
Alcançabilidade em DAG com n ~ $5 \cdot 10^4$	Fechamento transitivo	cada linha do fecho é um bitset; blocos cabem na palavra	SCC + bitset em blocos	ICPC, Grafos, Bits
Path em árvore, contagem por distância	Todo caminho passa por um centroide	todo caminho passa pelo centroide; os pares se contam por distância	Centroide + FFT (A^2 − Σ $filhos^2$)	ICPC, Árvores, Álgebra
DP com min sobre ancestrais e termo bilinear	Hull por caminho da raiz	cada ancestral é uma reta; hull no caminho	CHT/Li Chao persistente ou com rollback	ICPC, DP, Árvores
Queries sobre intervalo de arestas/tempo	Estrutura só com adição + desfazer	só adição permite desfazer ou guardar por tempo	Mo com rollback ou segtree no tempo + DSU	ICPC, Estruturas, Grafos
Substring/automaton em DP	DAG do SAM / AC	SAM/AC são DAGs; a DP percorre estados	DP no SAM, DP em AC + matriz/digit DP	ICPC, Strings, DP

Arrays e guloso

Primeira pergunta: a ordem dos elementos importa? Se não, ordene.

- **Ordenar** por valor, por fim de intervalo, por razão a/b, ou pela comparação de troca (a antes de b se a+b < b+a). Prove com exchange argument.
  - *Gatilho:* o enunciado pede uma ordem, um escalonamento, uma atribuição em sequência, ou o custo depende só de quais itens vêm antes de quais.
  - *Por que funciona:* se trocar dois itens vizinhos fora de ordem nunca piora, toda ordem pode ser "desinvertida" até a ordem proposta sem aumentar o custo; logo ela é ótima.
  - *Como fazer:* escreva o custo local dos dois itens em cada ordem, cancele o que é comum e obtenha o comparador. Depois ordene em O(n log n): por valor, por fim de intervalo, por razão ou por concatenação.
  - *Exemplo:* intervalos ordenados por fim; tarefas por t/w; strings por a+b < b+a.
  - *Armadilha:* provar a troca de um par não basta se o comparador não for transitivo; use produto cruzado em inteiros e desempate de forma consistente.
- **De trás pra frente.** O último elemento costuma ser forçado; decisões do fim não dependem do começo.
  - *Gatilho:* o último índice tem poucas opções, o estado final é conhecido, ou operações no fim mudam apenas o sufixo.
  - *Por que funciona:* ao fixar o último elemento, as restrições sobre o restante ficam menores; não há futuro depois dele para gerar dependência cíclica.
  - *Como fazer:* determine o que o último elemento é obrigado a ser, remova-o e repita, ou defina o DP do sufixo. Se a dependência for só para trás, o guloso do fim é O(n).
  - *Exemplo:* construir a menor sequência válida escolhendo o último caractere e retrocedendo; remover elementos do fim de uma pilha.
  - *Armadilha:* só funciona quando a decisão do fim não altera escolhas já feitas no começo; se alterar, use DP nos dois sentidos.

- **Two pointers / janela.** "Subarray com soma ≤ S", "no máximo k distintos" → se encolher a janela nunca piora, é janela deslizante.
  - *Gatilho:* "maior/menor subarray que satisfaz…", "no máximo k", e vale a monotonia: se [l, r] é válido, todo subintervalo também é.
  - *Por que funciona:* ao avançar r, o melhor l nunca precisa voltar; cada ponteiro percorre o array uma vez, total O(n).
  - *Como fazer:* adicione a[r], enquanto a janela for inválida remova a[l] e avance l; atualize a resposta com [l, r]. Para contar, some r − l + 1 (subarrays válidos que terminam em r).
  - *Exemplo:* subarrays com no máximo k valores distintos, usando um mapa de frequências; menor janela com soma ≥ S com valores positivos.
  - *Armadilha:* com negativos a soma deixa de ser monótona e a janela quebra; "exatamente k" obtém-se como f(≤ k) − f(≤ k − 1).
- **Prefix sums + map.** Contar subarrays com soma/xor = k → para soma, conte prefixos iguais a pref − k; para xor, prefixos iguais a pref ⊕ k.
  - *Gatilho:* "quantos subarrays têm soma/xor k", "subarray com soma zero", e os valores podem ser negativos.
  - *Por que funciona:* a soma de [l+1, r] é P[r] − P[l]; ela vale k sse P[l] = P[r] − k. Para xor, a operação inversa é o próprio xor, então P[l] = P[r] ⊕ k.
  - *Como fazer:* inicialize o mapa com {0: 1}; para cada prefixo, some a frequência da chave alvo e só depois insira o prefixo atual. O(n) esperado com hash.
  - *Exemplo:* [1, 2, 3] com k = 3 tem os subarrays [3] e [1, 2].
  - *Armadilha:* não use janela deslizante com negativos; limpe o mapa entre casos e use 64 bits para as somas.
- **Pilha monotônica.** Próximo maior/menor, maior retângulo, contribuição de cada elemento como máximo/mínimo.
  - *Gatilho:* "próximo maior/menor à esquerda/direita", "quanto cada elemento contribui", "maior retângulo no histograma".
  - *Por que funciona:* um elemento que é menor que um vizinho mais novo nunca mais será o próximo maior de ninguém; pode ser descartado. A pilha guarda só candidatos ainda úteis, em ordem monotônica.
  - *Como fazer:* para cada i, desempilhe enquanto o topo for pior que a[i]; o topo restante é o anterior maior/menor; empilhe i. Cada índice entra e sai uma vez: O(n).
  - *Exemplo:* para Σ max de todos os subarrays, com L_i e R_i os limites de domínio de a_i, a contribuição é a_i·(i − L_i)·(R_i − i).
  - *Armadilha:* com valores repetidos, escolha ">" de um lado e ">=" do outro; sem isso, subarrays com máximo repetido são contados duas vezes ou nenhuma.
- **Deque monotônico.** Máximo em janela deslizante; DP com transição de janela.
  - *Gatilho:* "máximo/mínimo de cada janela de tamanho k", ou dp[i] = a[i] + max(dp[j]) para j em uma janela de índices.
  - *Por que funciona:* só podem ser máximos os elementos que não têm à frente um valor maior dentro da janela; o deque mantém exatamente esses candidatos em ordem.
  - *Como fazer:* guarde índices em ordem decrescente de valor; ao inserir i, pop no fim enquanto a[back] ≤ a[i]; pop na frente enquanto o índice saiu da janela. O(n) amortizado.
  - *Exemplo:* máximo de todas as janelas de tamanho k; DP de partição com custo máximo na última janela.
  - *Armadilha:* remova o índice expirado antes de consultar; use ≤ ou < conforme queira manter a ocorrência mais antiga ou a mais nova.
- **Heap guloso.** Escolher o melhor disponível a cada passo; "arrependimento" (tire do heap e troque se vale a pena).
  - *Gatilho:* há um conjunto de candidatos que cresce com o tempo e é preciso escolher um agora, ou manter os k melhores de um fluxo.
  - *Por que funciona:* se a escolha local não bloqueia nenhuma escolha futura melhor, um argumento de troca mostra que pegar o melhor disponível é seguro; o heap só acelera essa escolha.
  - *Como fazer:* adicione candidatos ao heap, pop o melhor (ou o pior, se for para descartar) e, no arrependimento, reinsira a alternativa trocada. Custo O(n log n).
  - *Exemplo:* máximo de tarefas com lucro e prazo, guardando lucros num min-heap e removendo o menor quando o prazo aperta.
  - *Armadilha:* "melhor agora" sem propriedade de troca falha; o arrependimento precisa representar corretamente a opção de desfazer, não apenas reinserir o mesmo valor.

- **Intervalos.** Ordene por fim para máximo de disjuntos; por início para cobrir; sweep de eventos (+1 no início, −1 no fim) para sobreposição.
  - *Gatilho:* o problema envolve intervalos na reta, salas, cobertura, pontos ou quantidade máxima simultânea.
  - *Por que funciona:* terminar cedo deixa o máximo de espaço para os próximos; para cobrir, começar cedo permite alcançar mais longe; no sweep, cada evento muda o número de intervalos ativos.
  - *Como fazer:* para disjuntos, ordene por fim e aceite se l ≥ último fim (no modelo fechado, use l > se tocar não vale); para cobrir, escolha entre os que começam antes da cobertura atual o de maior fim; para sobreposição, processe eventos e mantenha o máximo.
  - *Exemplo:* máximo de intervalos disjuntos; mínimo de pontos para furar todos; número máximo de reuniões simultâneas.
  - *Armadilha:* decida se os intervalos são fechados ou semiabertos; empates no mesmo ponto precisam de uma ordem consistente.
- **Operação em subarray/prefixo.** Olhe o array de diferenças: operação em intervalo vira 2 mudanças pontuais.
  - *Gatilho:* "adicione x em [l, r]", "inverta um prefixo", ou muitas operações de intervalo e uma leitura no final.
  - *Por que funciona:* dentro do intervalo todas as diferenças entre vizinhos se cancelam; só a fronteira esquerda e a posição seguinte ao fim sentem a mudança.
  - *Como fazer:* d[l] += x, d[r+1] −= x; no final faça o prefixo de d. Com n + 2 posições, r+1 pode ser n+1 sem caso especial.
  - *Exemplo:* várias somas em intervalos e depois reconstruir o array; diferenças para contar cobertura.
  - *Armadilha:* use n + 2, não n; em 2D são quatro atualizações e um prefixo bidimensional.
- **Mediana.** Minimizar $\Sigma|a_i - x| \rightarrow x$ = mediana. Minimizar $\Sigma(a_i - x)^2 \rightarrow$ média.
  - *Gatilho:* escolher um ponto que minimize deslocamento total, ou uma soma de distâncias absolutas/quadrados.
  - *Por que funciona:* a soma de valores absolutos é convexa e qualquer mediana é minimizador (0 pertence ao subdiferencial); a soma de quadrados é convexa e tem mínimo onde a derivada é zero, em x = média.
  - *Como fazer:* para L1, ordene ou use nth_element e pegue a posição central; para L2, some e divida por n. Para mediana dinâmica, use dois heaps ou dois multisets.
  - *Exemplo:* ponto de encontro numa reta que minimiza a soma das distâncias; ajuste de uma constante a dados.
  - *Armadilha:* com n par, qualquer x entre as duas medianas é ótimo; a média pode não ser inteira; para L1 não use média.
- **Elementos ruins são poucos.** Se só importam os k maiores/menores, mantenha só eles.
  - *Gatilho:* k é pequeno e constante, ou a resposta depende apenas dos extremos, dos k maiores ou dos elementos que violam uma condição.
  - *Por que funciona:* elementos intermediários nunca entram na resposta; mantê-los só gasta memória e tempo. Um heap de tamanho k preserva só os candidatos relevantes.
  - *Como fazer:* percorra o array mantendo um heap de tamanho k (max-heap para os k menores, min-heap para os k maiores) ou use nth_element para separar em O(n) esperado.
  - *Exemplo:* soma dos k maiores; top-k frequências; detectar se há mais de k elementos fora do lugar.
  - *Armadilha:* se k é da ordem de n, ordenar é melhor; empates podem exigir contar ocorrências, não só valores.
- **Paridade/posição.** Swaps adjacentes → número de inversões; elementos em posições pares/ímpares separados.
  - *Gatilho:* trocas adjacentes, tabuleiro, posições alternadas, ou uma operação que preserva a paridade do índice.
  - *Por que funciona:* uma troca adjacente de elementos distintos muda o número de inversões em exatamente 1; por isso a paridade da permutação é invariante. Posições pares e ímpares só se misturam com um número par de trocas.
  - *Como fazer:* para ordenar com swaps adjacentes, conte inversões; para separar por posição, processe pares e ímpares como dois arrays independentes.
  - *Exemplo:* mínimo de swaps adjacentes para ordenar; verificar se uma permutação é par ou ímpar por n − #ciclos.
  - *Armadilha:* swaps de elementos iguais não mudam a permutação; não confunda número de swaps com número de inversões quando as trocas não são adjacentes.
- **Contar pares i < j com condição.** Ordene e use binária, BIT por valor, ou merge sort (inversões).
  - *Gatilho:* "conte pares i < j tais que a_i > a_j", "a_i + a_j ≤ S", ou qualquer relação entre dois elementos com ordem de índice.
  - *Por que funciona:* ao processar j da esquerda para a direita, os i já vistos são exatamente os i < j; uma estrutura ordenada conta quantos satisfazem a condição.
  - *Como fazer:* comprima os valores; para a_i > a_j, consulte quantos maiores já foram inseridos e insira a_j; para somas, ordene e use dois ponteiros ou binária. Inversões também saem de merge sort.
  - *Exemplo:* número de inversões de uma permutação; contar pares com soma ≤ S.
  - *Armadilha:* inclua a igualdade conforme o enunciado; a resposta pode chegar a n(n−1)/2 e precisa de 64 bits.

- **Guloso quebrou?** Gere um contraexemplo com brute force; quase sempre vira DP.
  - *Gatilho:* o guloso passa nos exemplos pequenos, mas você desconfia de uma escolha local que pode bloquear uma combinação melhor.
  - *Por que funciona:* um contraexemplo mínimo expõe exatamente qual estado o guloso ignora; esse estado vira a dimensão do DP ou o parâmetro do arrependimento.
  - *Como fazer:* escreva um brute force para n pequeno, gere todas as entradas ou aleatórias, compare com o guloso e reduza o primeiro contraexemplo. Depois defina o DP e teste contra o brute.
  - *Exemplo:* troco com moedas arbitrárias; escolher intervalos com pesos; agendar com duas restrições.
  - *Armadilha:* o brute force precisa cobrir o mesmo modelo (empates, limites, múltiplas cópias); um contraexemplo inválido leva a um DP desnecessário.

---

adicionado em 2026-11-10 por agentes; revisado

- **Scheduling: minimizar atraso máximo.** Jobs com duração e prazo, minimizar max(C_i − d_i) → ordene por prazo (EDF); exchange argument: trocar um par invertido nunca piora.
  - *Gatilho:* todos os jobs disponíveis no início, cada um com duração t_i e prazo d_i, e a resposta é o pior atraso C_i − d_i.
  - *Por que funciona:* se dois jobs consecutivos estão com prazos invertidos, colocar o de prazo menor primeiro não aumenta o atraso de nenhum dos dois e não altera o término dos jobs seguintes; repetir as trocas leva a EDF.
  - *Como fazer:* ordene por d_i crescente; acumule C e atualize ans = max(ans, C − d_i). O(n log n).
  - *Exemplo:* três jobs com prazos 4, 2, 6: EDF os executa na ordem dos prazos e minimiza o máximo de C_i − d_i.
  - *Armadilha:* "tardiness" costuma ser max(0, C_i − d_i), que é outro critério; release times e preempção mudam a regra. Não confunda prazo com duração.
- **Scheduling: minimizar Σ w_i·C_i.** Ordene por t_i/w_i crescente (regra de Smith); compare t_i·w_j < t_j·w_i para evitar divisão e double.
  - *Gatilho:* máquina única, jobs sem dependências, custo é a soma de w_i vezes o instante de término C_i.
  - *Por que funciona:* na troca de dois jobs vizinhos, os termos dos demais não mudam; i antes de j é melhor sse t_i·w_j < t_j·w_i. Isso é uma ordem fraca estrita e leva à regra de Smith.
  - *Como fazer:* ordene pelo comparador acima, execute na ordem e acumule C += t_i, ans += w_i·C. O(n log n).
  - *Exemplo:* jobs (t, w) = (2, 1) e (1, 1): o segundo vem antes, pois 1·1 < 2·1.
  - *Armadilha:* produto t·w pode estourar int; trate w = 0 sem divisão por zero; empates podem ser arbitrários.
- **Jobs com prazo e lucro, 1 unidade cada.** Ordene por prazo, empurre lucros num min-heap e, se o tamanho passar do prazo atual, remova o menor. O heap final é a resposta.
  - *Gatilho:* cada job ocupa exatamente uma unidade de tempo, tem prazo e lucro, e não se pode executar dois no mesmo instante.
  - *Por que funciona:* depois de processar os jobs com prazo ≤ d, só cabem d jobs; entre eles, vale a pena manter os d de maior lucro. Remover o menor preserva a melhor escolha para qualquer prefixo.
  - *Como fazer:* ordene por prazo crescente; insira o lucro no min-heap; se heap.size() > d_i, pop o mínimo. Ao final, o heap contém um subconjunto ótimo; some seus valores.
  - *Exemplo:* jobs (prazo 1, lucro 5), (prazo 1, lucro 3), (prazo 2, lucro 4): o heap fica com 5 e 4.
  - *Armadilha:* o prazo é o último instante permitido (modelo fechado); se d_i > n, pode limitar a n. O algoritmo escolhe um conjunto, e a ordem de execução pode ser obtida depois.
- **Intervalos: mínimo de pontos para furar todos.** Ordene por fim e coloque o ponto no fim do primeiro intervalo não furado. Mesmo problema que "máximo de disjuntos", em dupla.
  - *Gatilho:* escolher o menor número de pontos na reta de modo que todo intervalo contenha pelo menos um ponto.
  - *Por que funciona:* o intervalo que termina primeiro precisa ser furado; colocar o ponto no seu extremo direito é a posição que também pode furar o maior número de intervalos seguintes.
  - *Como fazer:* ordene por r crescente; mantenha o último ponto colocado; se l > último ponto, coloque um novo ponto em r e incremente a resposta. O(n log n).
  - *Exemplo:* [1,3], [2,5], [4,6]: o ponto 3 fura os dois primeiros e 6 fura o último.
  - *Armadilha:* para intervalos fechados, um ponto em r também cobre um intervalo que começa em r; use l > ponto para criar outro. Em semiabertos, a comparação muda.

- **Intervalos: mínimo de salas = máxima sobreposição.** Para atribuir de fato, ordene por início e use heap com os fins; reaproveite a sala cujo fim ≤ início.
  - *Gatilho:* alocar recursos para intervalos que não podem compartilhar horário, ou determinar quantas salas são necessárias.
  - *Por que funciona:* no instante de sobreposição máxima, todas essas reuniões precisam de salas distintas; o sweep mostra que esse limite é suficiente. O heap realiza a coloração do grafo de intervalos.
  - *Como fazer:* para o número, processe eventos e tome o máximo de ativos; para atribuir, ordene por início, guarde os fins num min-heap e reuse a sala de menor fim se fim ≤ início (semiaberto; no fechado, exija fim < início); senão abra outra.
  - *Exemplo:* [0,30], [5,10], [15,20] precisa de 2 salas; a terceira reunião pode reutilizar a sala da segunda.
  - *Armadilha:* use < para intervalos fechados e ≤ para semiabertos; a ordem dos eventos no mesmo ponto altera a sobreposição contada.
- **Intervalos com k máquinas, maximizar quantos atender.** Ordene por fim; para cada intervalo escolha a máquina livre com maior fim < início (intervalos fechados; use multiset + lower_bound e recue um; semiabertos usam ≤ e upper_bound); se não há, descarte.
  - *Gatilho:* há k máquinas idênticas e vários intervalos; cada máquina atende um intervalo por vez e quer-se maximizar o número atendido.
  - *Por que funciona:* processar pelo fim é o guloso clássico de intervalos; entre as máquinas livres, usar a de maior fim ainda compatível preserva as de fim menor para intervalos futuros.
  - *Como fazer:* ordene por r; mantenha um multiset dos fins das máquinas livres. Para intervalos fechados [l, r], ache o primeiro fim ≥ l, recue um e apague-o; se não existir, descarte. Para semiaberto, use upper_bound(l) e recue um.
  - *Exemplo:* k = 1 vira o máximo de intervalos disjuntos; k = n vira todos os intervalos compatíveis.
  - *Armadilha:* escolher a máquina de menor fim em vez da maior pode bloquear um intervalo futuro; mantenha as k máquinas no multiset, inclusive as ociosas, e trate empates de fim.
- **Cobrir [0, L] com o mínimo de intervalos.** Ordene por início; entre os que começam ≤ cobertura atual, pegue o que alcança mais longe; se nenhum avança, impossível.
  - *Gatilho:* cobertura de um segmento contínuo por intervalos, com ou sem sobreposição permitida, minimizando a quantidade usada.
  - *Por que funciona:* qualquer solução precisa cobrir o ponto "cobertura atual"; trocar o intervalo escolhido pelo que termina mais longe nunca piora o restante. É um guloso de alcance máximo.
  - *Como fazer:* ordene por l; mantenha um ponteiro e, enquanto l ≤ cobertura, atualize o maior r; quando o ponteiro parar, escolha esse maior r, conte um intervalo e repita. Se o máximo não passar da cobertura, não há solução.
  - *Exemplo:* [0,2], [0,5], [4,7] para cobrir [0,7]: escolha [0,5] e depois [4,7].
  - *Armadilha:* o alvo L é inclusivo; verifique se a cobertura chega a L, não só a L−1. Intervalos que começam antes de 0 podem ser usados.
- **Empates em sweep de eventos.** Intervalos fechados [l, r]: processe abertura antes de fechamento no mesmo ponto; semiabertos [l, r): fechamento antes. Errar isso muda a sobreposição máxima por 1.
  - *Gatilho:* há eventos de início e fim na mesma coordenada e a resposta é sobreposição máxima, cobertura ou contagem de ativos.
  - *Por que funciona:* no modelo fechado, o ponto final ainda pertence ao intervalo, então quem começa ali coexiste com quem termina ali; no semiaberto, o fim já saiu.
  - *Como fazer:* ordene por (coordenada, tipo), com tipo 0 para abertura e 1 para fechamento no fechado; inverta no semiaberto. Atualize a contagem na ordem escolhida.
  - *Exemplo:* [0,1] e [1,2] têm sobreposição 2 no fechado e 1 no semiaberto no ponto 1.
  - *Armadilha:* não misture as convenções no mesmo problema; em empates de vários eventos, agrupe por coordenada se for contar pontos distintos.
- **Pareamento na reta (pontos com pontos).** Minimizar $\Sigma|a_i - b_{\sigma(i)}|$ ou o $\max |a_i - b_{\sigma(i)}|$ → ordene os dois lados e case em ordem; vale pela desigualdade do rearranjo.
  - *Gatilho:* dois conjuntos de pontos na reta com o mesmo tamanho e o objetivo é casá-los um a um.
  - *Por que funciona:* se a ≤ b e c ≤ d, o pareamento cruzado (a,d) e (b,c) nunca é melhor que (a,c) e (b,d); desfazer cruzamentos não aumenta a soma nem o máximo. Repetindo, chega-se à ordem.
  - *Como fazer:* ordene A e B e case A[i] com B[i]. Para a soma, calcule $\Sigma|A[i]-B[i]|$; para o máximo, max dessas diferenças.
  - *Exemplo:* A = [1, 10], B = [2, 8]: casar 1–2 e 10–8 custa 1 + 2 = 3; cruzar custa 7 + 8 = 15.
  - *Armadilha:* os conjuntos precisam ter o mesmo tamanho; se um lado tem extras, o problema vira escolher um subconjunto e pode exigir DP ou guloso adicional.

- **Desigualdade do rearranjo.** Σ a_i·b_σ(i) é máximo com ambos ordenados do mesmo jeito e mínimo com ordens opostas. Serve para "atribuir pesos a posições/arestas por frequência de uso".
  - *Gatilho:* quer-se maximizar ou minimizar uma soma de produtos entre dois vetores, com liberdade para permutar um deles.
  - *Por que funciona:* trocar dois produtos desalinhados para a ordem monótona aumenta a soma; a prova é a mesma troca de vizinhos, usando (a_i − a_j)(b_i − b_j) ≥ 0.
  - *Como fazer:* ordene os dois vetores crescentemente e faça o produto escalar para o máximo; ordene um crescente e o outro decrescente para o mínimo.
  - *Exemplo:* pesos [1, 3] e valores [10, 20]: máximo 1·10 + 3·20 = 70; mínimo 1·20 + 3·10 = 50.
  - *Armadilha:* a regra vale para números reais, inclusive negativos, mas o produto pode estourar; empates permitem qualquer desempate.
- **Pontos × segmentos (atribuir cada ponto a um segmento que o contém).** Varra por x; heap dos segmentos abertos ordenado por fim; dê o ponto ao segmento que expira primeiro.
  - *Gatilho:* há pontos e segmentos na reta e cada segmento pode receber no máximo um ponto; quer-se maximizar os pontos atribuídos.
  - *Por que funciona:* entre os segmentos que contêm o ponto atual, o que termina antes tem menos chances futuras; usá-lo primeiro é um argumento de troca (EDF). Deixar o de fim maior preserva flexibilidade.
  - *Como fazer:* ordene pontos e segmentos por x; adicione ao min-heap os segmentos com l ≤ x, remova os que têm r < x, e se o heap não estiver vazio, atribua o ponto ao topo e pop. Para semiabertos, remova r ≤ x.
  - *Exemplo:* pontos 1 e 2; segmentos [0,1] e [0,2]: o ponto 1 deve ir para [0,1], deixando [0,2] para o ponto 2.
  - *Armadilha:* remova os expirados antes de atribuir; se um segmento puder receber vários pontos, o problema é outro e esse heap não se aplica.
- **Heap com arrependimento: comprar/vender ações.** O push duplo (vender e empurrar p duas vezes) superconta vendas sobrepostas: [0,1,4,7] retorna 10, mas o máximo é 7. Para o modelo "uma ação em mãos por vez", some as diferenças positivas adjacentes: Σ max(0, p_i − p_{i−1}).
  - *Gatilho:* há uma sequência de preços e se pode comprar e vender, mas há restrição de quantas ações se mantém ao mesmo tempo ou de número de transações.
  - *Por que funciona:* no modelo de uma ação por vez, cada subida entre dias consecutivos pode ser capturada; a soma das diferenças positivas é o lucro máximo. O push duplo representa vendas/desfazer de forma que reutiliza um preço antigo e conta lucros sobrepostos.
  - *Como fazer:* para uma ação por vez, percorra os preços e some max(0, p_i − p_{i−1}); para limites de transações, use DP ou o heap com a modelagem correta, não o push duplo ingênuo.
  - *Exemplo:* [0, 1, 4, 7]: 1 + 3 + 3 = 7; o push duplo devolve 10, que exigiria vender duas posições ao mesmo tempo.
  - *Armadilha:* taxas, short selling, limite de k transações e permissão de várias ações simultâneas mudam o modelo; valide a recorrência com brute force.
- **Escolher k elementos não adjacentes de soma máxima.** Lista ligada + heap; ao escolher i, substitua os vizinhos e i por um nó de valor v_l + v_r − v_i (arrependimento). Repita k vezes.
  - *Gatilho:* escolher exatamente k posições sem duas adjacentes, maximizando (ou minimizando) a soma dos valores.
  - *Por que funciona:* escolher i pode impedir a combinação melhor dos dois vizinhos; o nó fundido com valor v_l + v_r − v_i permite "desfazer" i e trocar por ambos, mantendo a contagem de escolhas.
  - *Como fazer:* use lista duplamente ligada e max-heap com posições; ao escolher o topo, some seu valor, crie um nó com v_l + v_r − v_i, ligue aos vizinhos externos, remova os três antigos e repita k vezes com remoção preguiçosa.
  - *Exemplo:* [4, 5, 4] com k = 1 escolhe 5; com k = 2, a melhor é 4 + 4 = 8, obtida ao desfazer a escolha do 5 pelo nó de valor 4 + 4 − 5 = 3.
  - *Armadilha:* se k > ⌈n/2⌉, não há solução; valores negativos podem tornar o nó fundido negativo; no círculo, os dois primeiros e os dois últimos são vizinhos.

- **Cortar em k segmentos minimizando/maximizando soma de custos que dependem dos cortes.** Se o custo é aditivo nos cortes, escolha os k − 1 melhores cortes com sort; vale para "soma de (último − primeiro) por bloco" → soma total menos os k − 1 maiores buracos.
  - *Gatilho:* particionar uma sequência em k blocos contíguos e o custo de cada bloco é uma soma de diferenças entre elementos consecutivos.
  - *Por que funciona:* o custo total de todos os blocos é o custo da sequência inteira menos o custo dos saltos que viram cortes; escolher cortes é escolher quais buracos remover.
  - *Como fazer:* calcule os n − 1 buracos; para minimizar a soma dos blocos, remova os k − 1 maiores; para maximizar, remova os k − 1 menores. O(n log n) ou O(n) com nth_element.
  - *Exemplo:* array [1, 5, 2, 8] com k = 2: o custo da sequência inteira é 8 − 1 = 7; os buracos são 4, −3 e 6. Cortar no buraco 6 deixa [1, 5, 2] e [8], com custo 1 + 0 = 1, que é 7 − 6.
  - *Armadilha:* a fórmula só vale se o custo for realmente aditivo nos cortes; k = 1 não tem cortes; custos que dependem do comprimento do bloco não se reduzem a escolher buracos.
- **Tornar array não decrescente com mínimo Σ|a_i − b_i| (slope trick).** Max-heap: empurre a_i; se top > a_i, ans += top − a_i, pop e empurre a_i de novo. Estritamente crescente: troque a_i por a_i − i.
  - *Gatilho:* ajustar um array para uma sequência monótona, pagando o deslocamento absoluto de cada posição.
  - *Por que funciona:* a função de custo ótima do prefixo é convexa e linear por partes; o max-heap guarda seus pontos de quebra. Quando a_i é menor que o maior ponto, mover esse ponto para a_i custa a diferença.
  - *Como fazer:* para b não decrescente, execute o algoritmo acima em O(n log n). Para b estritamente crescente, defina c_i = b_i − i; então c é não decrescente e o custo vira Σ|(a_i − i) − c_i|, aplicando o mesmo algoritmo a a_i − i.
  - *Exemplo:* [3, 1, 2] com b não decrescente: o custo mínimo é 2, com b = [1, 1, 2] ou [2, 2, 2].
  - *Armadilha:* a transformação estrita usa a_i − i (com base de índice consistente); o custo pode estourar int; slope trick exige convexidade, não vale para qualquer penalidade.
- **a_i − i.** Subsequência "a_j − a_i ≥ j − i" vira não decrescente em b_i = a_i − i; "|a_i − a_j| = |i − j|" vira iguais em a_i − i ou a_i + i (diagonais).
  - *Gatilho:* a condição mistura valor e índice, como "a_j − a_i ≥ j − i", ou pontos que devem estar na mesma diagonal.
  - *Por que funciona:* subtrair i dos dois lados elimina a diferença de índices; a condição vira b_j ≥ b_i. Para a igualdade absoluta, há duas diagonais: a_i − i constante (principal) ou a_i + i constante (secundária).
  - *Como fazer:* transforme cada elemento em b_i = a_i − i e resolva LIS/não decrescente; para diagonais, agrupe por a_i − i e por a_i + i.
  - *Exemplo:* [1, 3, 2, 4] vira b = [1, 2, 0, 1]; a LNDS de b tem tamanho 2 e corresponde às subsequências [1, 3] ou [2, 4].
  - *Armadilha:* use a mesma origem de índice em toda a transformação; para a condição estrita, b deve ser estritamente crescente. Nas diagonais, teste as duas famílias (a_i − i e a_i + i) separadamente.
- **Exatamente k distintos / soma exatamente k.** exato(k) = noMáximo(k) − noMáximo(k − 1) com janela deslizante; para soma com valores ≥ 0 vale o mesmo.
  - *Gatilho:* "exatamente k valores distintos" ou "soma exatamente k" em subarrays, e a versão "no máximo" é monótona.
  - *Por que funciona:* todo subarray com no máximo k distintos se divide entre os que têm no máximo k−1 e os que têm exatamente k; a diferença conta só os de exatamente k.
  - *Como fazer:* escreva uma função f(C) que conta subarrays com no máximo C usando janela e mapa de frequências; responda f(k) − f(k−1). Para soma, use o mesmo se os valores forem não negativos.
  - *Exemplo:* subarrays com exatamente 2 valores distintos; soma exatamente k com array de positivos.
  - *Armadilha:* com negativos, "soma ≤ k" não é monótona ao encolher; "exatamente k" com k = 0 precisa de base f(−1) = 0.
- **Janela com max − min ≤ k.** Dois deques (um de max, um de min); avance o início até a condição voltar. Cada elemento entra e sai uma vez.
  - *Gatilho:* contar ou achar o maior subarray cuja diferença entre o máximo e o mínimo é no máximo k.
  - *Por que funciona:* para um r fixo, se [l, r] é válido, qualquer l' > l também é; a validade é monótona no início. Os deques dão max e min em O(1) amortizado.
  - *Como fazer:* insira a[r] nos dois deques (max decrescente, min crescente); enquanto max − min > k, remova a[l] dos deques se ela for o topo e avance l; atualize a resposta.
  - *Exemplo:* [1, 5, 3, 2] com k = 2: as maiores janelas válidas têm tamanho 2, como [5, 3] e [3, 2]; a janela inteira tem diferença 4 e não vale.
  - *Armadilha:* remova índices expirados dos deques antes de consultar; para contar, some r − l + 1 a cada r.

- **Contribuição com empates na pilha monotônica.** Para Σ max de todos os subarrays, um lado usa ">" estrito e o outro ">=", senão subarrays com valores iguais são contados duas vezes ou nenhuma.
  - *Gatilho:* você calcula a soma das contribuições de cada elemento como máximo/mínimo de subarrays e há valores repetidos.
  - *Por que funciona:* cada subarray precisa eleger um único índice como representante do máximo; usar ">" de um lado e ">=" do outro define esse representante sem ambiguidade.
  - *Como fazer:* para máximo, ache L_i com o anterior estritamente maior e R_i com o próximo maior ou igual (ou vice-versa); contribua com a_i·(i − L_i)·(R_i − i). Para mínimo, inverta as comparações.
  - *Exemplo:* [2, 2]: cada subarray tem máximo 2; a soma é 2 + 2 + 2 = 6, e a convenção de empates deve contar exatamente três subarrays.
  - *Armadilha:* trocar os sinais de ambos os lados conta zero ou duas vezes; use sentinelas para não tratar a pilha vazia como valor válido.
- **Σ (max − min) de todos subarrays** = Σ max − Σ min, cada um com pilha monotônica; evite fazer os dois juntos numa estrutura só.
  - *Gatilho:* o enunciado pede a soma da diferença entre máximo e mínimo sobre todos os subarrays.
  - *Por que funciona:* a soma é linear; calcular separadamente a soma dos máximos e a soma dos mínimos não perde informação e cada parte tem uma pilha simples.
  - *Como fazer:* rode a contribuição de máximo e a de mínimo com as convenções de empate adequadas e subtraia. O(n) com duas passadas.
  - *Exemplo:* [1, 2, 3] tem ranges 0, 0, 0, 1, 1, 2, somando 4.
  - *Armadilha:* não tente manter max e min na mesma pilha; as comparações e os empates são diferentes. Use 64 bits.
- **Maior submatriz de 1s** → histograma por linha (altura acumulada) + maior retângulo com pilha; O(nm). A contagem de submatrizes de 1s usa a mesma pilha com DP de "largura × altura".
  - *Gatilho:* matriz binária e a pergunta é a maior área retangular de 1s ou quantas submatrizes só de 1s existem.
  - *Por que funciona:* em cada linha, a altura de 1s consecutivos por coluna transforma o problema num histograma; o maior retângulo do histograma é resolvido pela pilha monotônica. Para contar, cada subarray de colunas contribui com min(alturas), que é o número de escolhas de topo.
  - *Como fazer:* atualize h[j] = h[j] + 1 se a célula for 1, senão h[j] = 0; rode a pilha para a maior área. Para contar, some h[j]·(j − L_j)·(R_j − j) por linha.
  - *Exemplo:* uma linha 1 1 0 1 gera alturas [1,1,0,1]; o maior retângulo tem área 2.
  - *Armadilha:* zere a altura na célula 0; o número de submatrizes pode ser $O(n^2m^2)$ e precisa de 64 bits; não confunda área com número de submatrizes.
- **Árvore cartesiana** em O(n) com pilha: max de [l, r] é o LCA; recursão "divida no máximo" tem profundidade n mas pode ser iterada; serve para "subarrays em que o máximo é k-ésimo", small-to-large pelo lado menor.
  - *Gatilho:* consultas de máximo/mínimo em intervalos, divisão e conquista pelo máximo, ou contagem de subarrays com uma ordem de máximo.
  - *Por que funciona:* na árvore cartesiana, o maior elemento do intervalo é o ancestral comum de l e r; a subárvore de um nó delimita o intervalo em que ele pode ser o máximo, e os subarrays que contêm o nó dentro desse intervalo o têm como máximo.
  - *Como fazer:* construa com pilha em O(n), ligando cada novo nó ao último menor/maior; para RMQ, use LCA ou Euler tour. Para contar, processe o lado menor da subárvore e consulte o outro com estrutura adequada.
  - *Exemplo:* max de [2, 5, 1, 4] é 5; na árvore, 5 é o LCA dos índices do intervalo.
  - *Armadilha:* a árvore pode ser uma corrente de profundidade n; evite recursão sem aumentar a pilha. Defina uma convenção para valores iguais.
- **Menor subsequência lexicográfica de tamanho k / remover k dígitos.** Pilha: enquanto topo > novo e ainda sobram remoções (ou elementos suficientes), desempilhe. Com "cada caractere distinto uma vez": guarde última ocorrência e "já na pilha".
  - *Gatilho:* escolher uma subsequência de tamanho fixo lexicograficamente mínima, remover exatamente k dígitos, ou manter cada valor no máximo uma vez.
  - *Por que funciona:* um caractere maior no topo deve sair se um menor aparece depois e ainda há remoções permitidas; a pilha mantém a menor sequência possível para o prefixo processado.
  - *Como fazer:* para remover k, desempilhe enquanto k > 0 e topo > c[i], decrementando k; empilhe c[i]; no fim, remova o restante de k do final. Para tamanho k, acrescente a condição de que ainda há elementos suficientes para completar k.
  - *Exemplo:* "1432" removendo 2 dígitos: 1 entra, 4 sai ao ver 3, 3 sai ao ver 2, resultando em "12".
  - *Armadilha:* zero à esquerda pode ser permitido ou não; com "cada distinto uma vez", uma segunda ocorrência não pode simplesmente ser descartada se o caractere ainda não foi usado.

- **Circular com guloso (posto de gasolina).** Resposta = posição após o mínimo do prefixo de saldos; se a soma total < 0, impossível. Substitui busca binária ou simulação 2n.
  - *Gatilho:* percorrer um ciclo e em cada posto ganhar gasolina e gastar para o próximo; achar o posto inicial que permite completar a volta.
  - *Por que funciona:* se o total é não negativo, existe uma volta válida; começar depois do ponto de menor saldo acumulado garante que o saldo nunca fica negativo nessa volta.
  - *Como fazer:* calcule saldo += gas[i] − cost[i]; acompanhe o mínimo e o índice; se saldo final < 0, retorne −1; senão retorne (índice do mínimo + 1) mod n. Use a última ocorrência do mínimo em empates.
  - *Exemplo:* gas = [1,2], cost = [2,1]: total 0; o prefixo é [−1, 0], mínimo em 0, comece no posto 1.
  - *Armadilha:* a resposta é circular (0 após n−1); confira o total antes; o mínimo do prefixo não é o mínimo dos saldos individuais.
- **Mediana dinâmica** (janela deslizante) → dois multisets balanceados ou dois heaps com remoção preguiçosa; custo Σ|a_i − med| sai das somas de cada metade.
  - *Gatilho:* a mediana muda com inserções e remoções, ou se pede a soma das distâncias à mediana em cada janela.
  - *Por que funciona:* mantendo a metade inferior e a superior, a mediana é o maior da inferior (ou a média dos dois topos); as somas de cada metade dão o custo L1 sem percorrer a janela.
  - *Como fazer:* max-heap para a metade inferior, min-heap para a superior; rebalanceie para tamanhos ⌈n/2⌉ e ⌊n/2⌋; para remoções, use multiset ou heaps com mapa de deletados e limpeza preguiçosa dos topos.
  - *Exemplo:* janela [1, 9, 2, 8] com mediana entre 2 e 8; o custo é calculado pelas somas das metades.
  - *Armadilha:* remova elementos obsoletos antes de consultar; a mediana de tamanho par pode ser qualquer valor entre os dois centrais, e o custo L1 é o mesmo.
- **Tornar elementos consecutivos / formar progressão com +1.** Minimizar Σ|a_i − (x + i)| → mediana de b_i = a_i − i.
  - *Gatilho:* ajustar o array para uma progressão aritmética de razão 1, minimizando a soma dos deslocamentos absolutos.
  - *Por que funciona:* b_i = a_i − i transforma x + i em uma constante x; o problema vira escolher uma constante que minimize Σ|b_i − x|, cuja solução é a mediana de b.
  - *Como fazer:* calcule b_i = a_i − i, ache a mediana (ordenando ou nth_element) e some |b_i − mediana|. Para razão d, use b_i = a_i − d·i.
  - *Exemplo:* [3, 1, 2] com b = [3, 0, 0]; a mediana é 0 e o custo é 3, correspondendo ao alvo [0, 1, 2].
  - *Armadilha:* a origem do índice muda a progressão em uma constante; para d arbitrário, a mediana é de a_i − d·i, não de a_i.
- **Elemento > n/2 em O(1) de memória** → Boyer–Moore (candidato + contador); para > n/k mantenha k − 1 candidatos. Sempre confirme o candidato com uma segunda passada.
  - *Gatilho:* achar o elemento que aparece em mais da metade (ou mais de 1/k) das posições, com memória constante ou quase.
  - *Por que funciona:* cada par de elementos distintos pode ser cancelado sem alterar quem é maioria estrita; sobram apenas candidatos. Para > n/k, manter k−1 contadores garante que todo elemento frequente sobreviva.
  - *Como fazer:* Boyer–Moore: se contador 0, troque o candidato; senão some +1 se igual e −1 se diferente. Depois conte as ocorrências para confirmar. Para > n/k, use k−1 pares (candidato, contador) e uma fase de cancelamento.
  - *Exemplo:* [1, 2, 1, 1] termina com candidato 1; a segunda passada confirma 3 ocorrências.
  - *Armadilha:* o candidato não é garantido se não existe maioria; sempre verifique. A condição é estrita (> n/2, > n/k), não ≥.
- **Mex de prefixos é não decrescente; mex de todos os subarrays** → dois ponteiros sobre contagens, ou segtree sobre o último índice de cada valor.
  - *Gatilho:* o enunciado pede o menor inteiro não negativo ausente em prefixos, subarrays ou janelas.
  - *Por que funciona:* ao acrescentar um elemento, o conjunto só cresce, então o mex nunca diminui; para todos os subarrays, a janela permite manter as contagens e avançar o mex.
  - *Como fazer:* para prefixos, mantenha um vetor de presença e um ponteiro mex; ao inserir a[i], enquanto presente[mex] avance. Para subarrays, use dois ponteiros com contagens ou uma segtree que guarda o último índice de cada valor e encontra o primeiro cujo último índice é anterior a l.
  - *Exemplo:* [0, 1, 3] tem mex 2; ao adicionar 2, o mex sobe para 4.
  - *Armadilha:* o mex pode ser n + 1; ignore valores > n para não estourar o vetor; limpe as contagens entre casos.

- **Subarray com maioria / "mais x que y".** Troque por ±1 e use prefixos (mínimo de prefixo à esquerda, BIT se for contar). Maioria estrita → soma > 0.
  - *Gatilho:* contar ou achar subarrays em que um valor x aparece mais que y, ou em que x é maioria.
  - *Por que funciona:* com x = +1, y = −1 e os demais 0, a diferença de contagens é a soma do subarray. "Mais x que y" vira soma > 0, isto é, P[r] > P[l].
  - *Como fazer:* para existência, mantenha o mínimo de P à esquerda; para contar, use BIT/merge sort sobre os prefixos. Para maioria estrita de x no subarray, use a mesma redução.
  - *Exemplo:* [x, y, x] vira [1, −1, 1]; o subarray inteiro tem soma 1, então x aparece mais.
  - *Armadilha:* "≥" e ">" são problemas diferentes; valores diferentes de x e y contribuem com 0, não com −1; a contagem pode ser $O(n^2)$.
- **Subarrays com soma ≥ 0 com negativos.** Two pointers não vale; use prefixos + BIT/merge sort. Two pointers só serve quando estender monotonicamente muda a condição (valores ≥ 0).
  - *Gatilho:* contar subarrays com soma não negativa e o array tem valores negativos.
  - *Por que funciona:* a soma de [l+1, r] é P[r] − P[l]; a condição é P[l] ≤ P[r]. Cada par de prefixos com i < j satisfazendo isso é um subarray.
  - *Como fazer:* calcule os prefixos, comprima os valores e percorra da esquerda para a direita contando quantos prefixos anteriores são ≤ P[r] com BIT; ou conte com merge sort modificado. O(n log n).
  - *Exemplo:* [−1, 2, −1] tem quatro subarrays com soma ≥ 0: [2], [−1,2], [2,−1] e o array inteiro.
  - *Armadilha:* a igualdade conta (soma 0); use 64 bits; não tente encolher a janela, pois remover um negativo pode aumentar a soma.
- **Menor operações +1/−1 em subarray para zerar/igualar.** No array de diferenças d, cada operação muda dois pontos em ±1; resposta ≈ Σ max(0, d_i) = Σ|d_i|/2 (cuidado com as bordas de d_0 e d_{n+1}).
  - *Gatilho:* operações de +1 ou −1 em um intervalo contíguo, com objetivo de zerar o array ou deixar todos os elementos iguais.
  - *Por que funciona:* uma operação em [l, r] altera d[l] em +1 e d[r+1] em −1 (ou o contrário); dentro do intervalo as diferenças se cancelam. Fazer todos os d zerados é transportar unidades entre posições, e o custo mínimo é o total positivo.
  - *Como fazer:* para zerar, use d[1] = a[1], d[i] = a[i] − a[i−1], d[n+1] = −a[n]; a resposta é Σ max(0, d[i]), que também é Σ|d[i]|/2. Para apenas igualar a uma constante, só as diferenças internas precisam zerar; as bordas funcionam como sumidouro e o valor pode ser menor.
  - *Exemplo:* [1, 2, 3] tem d = [1, 1, 1, −3], soma positiva 3; são necessárias 3 operações para zerar, mas apenas 2 para igualar em [2, 2, 2].
  - *Armadilha:* a fórmula de zerar inclui as duas bordas; para igualar, o alvo é livre e a resposta muda. Use 64 bits e não esqueça d[n+1].
- **Prefixo 2D e diferenças 2D.** Soma de retângulo = P[r2][c2] − P[r1−1][c2] − P[r2][c1−1] + P[r1−1][c1−1]; adicionar retângulo = 4 pontos em D e depois prefixo 2D. Muitas adições + 1 leitura final → offline barato.
  - *Gatilho:* somas em submatrizes, atualizações de retângulos ou muitas consultas 2D sobre uma matriz.
  - *Por que funciona:* o prefixo 2D armazena a soma de [1..i]×[1..j]; a inclusão-exclusão isola o retângulo. No array de diferenças 2D, uma adição constante em um retângulo só altera os quatro cantos.
  - *Como fazer:* construa P[i][j] = A[i][j] + P[i−1][j] + P[i][j−1] − P[i−1][j−1]. Para atualizar [r1..r2]×[c1..c2] com x, faça D[r1][c1] += x, D[r1][c2+1] −= x, D[r2+1][c1] −= x, D[r2+1][c2+1] += x; no fim, prefixe D.
  - *Exemplo:* a soma de uma submatriz 3×3 sai de quatro consultas a P; duas atualizações de retângulo podem ser acumuladas antes de reconstruir.
  - *Armadilha:* use índices 1-based, dimensões n+2 por m+2 e 64 bits; a ordem das quatro atualizações não importa, mas os sinais sim.
- **Maior soma de submatriz.** Fixe duas linhas, comprima colunas por prefixo e rode Kadane: $O(\min(n, m)^2 \cdot \max(n, m))$.
  - *Gatilho:* achar a submatriz retangular de soma máxima, com valores possivelmente negativos.
  - *Por que funciona:* fixando as linhas superior e inferior, cada coluna vira a soma do segmento vertical; o problema 2D se reduz a maior subarray 1D, resolvido por Kadane.
  - *Como fazer:* para cada par de linhas, acumule a soma por coluna e rode Kadane (atual = max(x, atual + x), best = max(best, atual)). Transponha se necessário para fixar o menor lado.
  - *Exemplo:* uma matriz com uma coluna negativa entre duas positivas pode ter a melhor submatriz excluindo a coluna negativa.
  - *Armadilha:* Kadane com reset em 0 permite submatriz vazia; se o retângulo precisa ser não vazio, inicialize best com −∞ e trate todos os negativos.

- **Transponha** quando $n \cdot m \le 10^6$ mas $n \gg m$: itere sobre o lado pequeno e deixe o loop interno no grande (cache e complexidade).
  - *Gatilho:* a matriz é muito mais larga que alta, ou o algoritmo tem custo quadrático em uma dimensão e linear na outra.
  - *Por que funciona:* trocar linhas por colunas preserva a resposta e permite escolher o lado menor para o loop externo; o acesso sequencial ao lado grande melhora a localidade de cache.
  - *Como fazer:* se n > m, transponha a matriz (ou troque os índices no acesso) e rode o algoritmo com as dimensões invertidas; ao final, converta as coordenadas se precisar da orientação original.
  - *Exemplo:* 500000 × 2 com um algoritmo $O(n^2 m)$ fica $O(4 \cdot 500000)$ após transpor, em vez de $O(500000^2 \cdot 2)$.
  - *Armadilha:* transpor custa O(nm) e memória extra; para matrizes quase quadradas, não há ganho; lembre de inverter as coordenadas na resposta.
- **Rotações/reflexões de grid**: 8 simetrias em quadrado; para "mesmo padrão a menos de rotação" tome a forma canônica (mínima entre as 4/8).
  - *Gatilho:* dois padrões são equivalentes sob rotação/reflexão, ou o grid pode ser transformado para uma orientação padrão.
  - *Por que funciona:* as simetrias de um quadrado formam o grupo diedral D4, com 8 transformações; escolher a menor representação lexicográfica dá um identificador único para a órbita.
  - *Como fazer:* gere as 4 rotações (e as reflexões, se permitidas), serialize cada uma e pegue a mínima; compare ou armazene em hash. Para retângulos não quadrados, só valem as transformações que preservam as dimensões.
  - *Exemplo:* um L e suas rotações/reflexões recebem a mesma forma canônica.
  - *Armadilha:* rotações de 90° só existem em grid quadrado; com simetria própria, formas duplicadas não podem ser contadas duas vezes.
- **Meet in the middle (n ≤ 40 com 2 escolhas; n ≈ 24–26 com 3).** Gere as somas das duas metades, ordene e use dois ponteiros; contagem de subconjuntos com soma ≤ S, ou xor/max mod M, cabem aqui. 3^(n/2) quando cada item tem 3 escolhas.
  - *Gatilho:* n é pequeno demais para 2^n completo, mas grande para exponencial simples, e a resposta combina escolhas das duas metades.
  - *Por que funciona:* toda solução se decompõe em uma escolha na primeira metade e outra na segunda; gerar 2^(n/2) de cada lado e combinar é muito menor que 2^n.
  - *Como fazer:* enumere todas as somas de cada metade, ordene uma delas e, para cada soma da outra, use binária/dois ponteiros para achar o complemento. Para contagem, conte multiplicidades; para xor, use hash; para max mod M, ordene e varra.
  - *Exemplo:* n = 40, S = 100: 2^20 ≈ 10^6 somas por lado, combinadas em O(2^20 log 2^20).
  - *Armadilha:* 2^20 já usa memória considerável; xor não é ordenável, então use hash; não confunda "soma ≤ S" com "soma = S" na contagem de pares.
- **4-sum / soma de quatro (n ≤ 2000).** Armazene todas as somas de pares em hash/sorted e procure o complemento: $O(n^2)$; com três listas A + B + C ≥ x faça n^2 + two pointers.
  - *Gatilho:* quatro índices/valores cuja soma é um alvo, ou quatro listas com um alvo de soma.
  - *Por que funciona:* uma soma de quatro pode ser vista como duas somas de dois; armazenar todas as somas de pares permite procurar o complemento em tempo esperado O(1) ou por dois ponteiros após ordenar.
  - *Como fazer:* para quatro listas, gere as somas A+B e C+D, ordene e conte pares complementares. Para uma única lista com índices distintos, guarde também os índices e valide que não se repetem. Para A+B+C ≥ x com três listas, ordene B e C e, para cada a, use dois ponteiros em O(n).
  - *Exemplo:* A = [1,2], B = [3,4], C = [5], D = [0]: as somas de pares permitem achar 1+4+5+0 = 10.
  - *Armadilha:* em uma única lista, dois pares podem compartilhar índices; um hash sem validação conta soluções inválidas. Somas podem estourar int.
- **Permutação é um conjunto de ciclos.** Trocas arbitrárias mínimas = n − #ciclos; trocas adjacentes mínimas = #inversões; paridade da permutação = (n − #ciclos) mod 2.
  - *Gatilho:* a entrada é uma permutação e o problema envolve reordenar, trocar posições ou paridade.
  - *Por que funciona:* um ciclo de comprimento c pode ser resolvido com c−1 trocas, e ciclos são independentes; logo o mínimo é n − #ciclos. Cada troca adjacente muda a paridade, e o número de inversões mede exatamente essas trocas.
  - *Como fazer:* decompose em ciclos com um vetor de visitados; conte #ciclos; para swaps adjacentes, conte inversões com BIT/merge sort. A paridade sai de (n − #ciclos) mod 2.
  - *Exemplo:* [2, 3, 1] é um ciclo (1→2→3→1, na convenção 1-based), então mínimo de trocas arbitrárias = 3 − 1 = 2; há 2 inversões.
  - *Armadilha:* swaps de elementos iguais não contam; use a convenção 0-based ou 1-based de forma consistente; "troca arbitrária" e "adjacente" são métricas diferentes.

- **Potência de permutação.** p^k quebra um ciclo de tamanho c em gcd(c, k) ciclos de tamanho c/gcd(c, k); ciclo de c e gcd(c,k)=1 vira um ciclo só. Raiz quadrada: ciclos ímpares podem ficar sozinhos, ciclos pares só existem aos pares do mesmo tamanho.
  - *Gatilho:* aplicar uma permutação k vezes, ou decidir se ela é uma potência/raiz de outra permutação.
  - *Por que funciona:* dentro de um ciclo, cada passo de p^k anda k posições; o ciclo se divide em gcd(c, k) órbitas de tamanho c/gcd(c, k). Na raiz quadrada, um ciclo ímpar de q pode vir de um ciclo de mesmo tamanho em p; dois ciclos ímpares de mesmo tamanho também podem vir de um ciclo dobrado; ciclos pares precisam ser combinados dois a dois.
  - *Como fazer:* decomponha em ciclos, calcule g = gcd(c, k mod c) e gere os g ciclos avançando k. Para raiz quadrada, conte ciclos de cada comprimento e exija quantidade par para comprimentos pares.
  - *Exemplo:* um ciclo de tamanho 6 com k = 4 tem gcd(6,4) = 2 e vira dois ciclos de tamanho 3.
  - *Armadilha:* use k mod c para evitar estouro; a orientação do ciclo importa; a condição de raiz quadrada é por comprimento par, não apenas "n par".
- **Mínimo de trocas adjacentes para transformar string/array A em B.** Mapeie a i-ésima ocorrência de cada valor de A para a i-ésima de B (preserva ordem relativa) e conte inversões da permutação resultante.
  - *Gatilho:* A e B são anagramas ou multiconjuntos iguais e só se permite trocar elementos vizinhos.
  - *Por que funciona:* em uma solução ótima, ocorrências iguais não precisam se cruzar; casar a i-ésima de A com a i-ésima de B preserva a ordem relativa e minimiza o número de inversões.
  - *Como fazer:* guarde filas de posições de cada valor em B; para cada valor de A, pegue a próxima posição correspondente e forme a permutação alvo; conte inversões com BIT ou merge sort.
  - *Exemplo:* A = "aab" e B = "aba": a permutação alvo é [1, 3, 2], com uma inversão.
  - *Armadilha:* se as contagens de valores diferem, é impossível; a resposta pode chegar a n(n−1)/2 e precisa de 64 bits; não case a primeira ocorrência arbitrária em B sem preservar a ordem.
- **"Mova elemento para o início/fim" para ordenar.** Os que ficam parados são os maiores valores consecutivos já em ordem de posição (só início), ou um bloco contíguo de valores em ordem (início e fim); resposta = n − tamanho do maior bloco.
  - *Gatilho:* cada operação remove um elemento e o coloca no início ou no fim; quer-se o mínimo de operações para ordenar.
  - *Por que funciona:* elementos nunca movidos mantêm a ordem relativa e precisam formar um bloco contíguo da ordem final; todos os outros podem ser colocados ao redor desse bloco. Logo, maximizar o bloco intocado minimiza as operações.
  - *Como fazer:* ordene os valores e encontre o maior bloco contíguo (por valor) que aparece na mesma ordem no array original; para apenas mover ao início, restrinja a um sufixo da ordem ordenada. Resposta = n − tamanho do bloco.
  - *Exemplo:* [2, 1, 3]: o bloco [2, 3] já está em ordem; mover 1 para o início resolve em 1 operação.
  - *Armadilha:* com duplicatas, o bloco é por ocorrências, não apenas por valores distintos; mover só para o início permite menos elementos parados que mover para as duas pontas.
- **LIS.** O(n log n) com vetor de caudas: lower_bound para estritamente crescente, upper_bound para não decrescente (trocar errado é bug clássico). Reconstrução precisa de pais por posição.
  - *Gatilho:* maior subsequência que respeita uma ordem crescente/não decrescente, ou uma condição transformável nisso.
  - *Por que funciona:* tails[len] guarda o menor final possível de uma subsequência de comprimento len; finais menores deixam mais espaço para continuar. A busca binária encontra onde encaixar o novo valor.
  - *Como fazer:* para cada a[i], ache a primeira posição com tails[pos] ≥ a[i] (lower_bound) para estritamente crescente, ou > a[i] (upper_bound) para não decrescente; substitua. Guarde o pai para reconstruir a sequência.
  - *Exemplo:* [10, 9, 2, 5, 3, 7] tem LIS 2, 3, 7, de comprimento 3.
  - *Armadilha:* tails não é a LIS; para reconstruir, salve o índice anterior. lower/upper bound trocados contam duplicatas como se fossem estritas.
- **Dilworth em arrays.** Mínimo de subsequências estritamente crescentes que cobrem o array = tamanho da maior subsequência não crescente; o guloso "anexe x à subsequência cujo último elemento é o maior < x; se não houver, abra uma nova" (multiset) constrói a partição.
  - *Gatilho:* particionar todos os elementos em o mínimo de sequências crescentes, ou dualmente cobrir com decrescentes.
  - *Por que funciona:* é o teorema de Dilworth/Mirsky para a ordem parcial de índices e valores; o guloso de paciência mantém os finais das pilhas e usa o maior final compatível (< x).
  - *Como fazer:* mantenha um multiset dos últimos elementos das subsequências; para x, ache o maior elemento < x, substitua-o por x; se não existir, insira uma nova subsequência. O número final de pilhas é a resposta.
  - *Exemplo:* [3, 1, 2]: 3 abre uma pilha, 1 abre outra, 2 entra na pilha de 1; são 2 subsequências crescentes, e a maior não crescente tem tamanho 2.
  - *Armadilha:* "estritamente crescente" usa <; para não decrescente, o dual muda e a busca é ≤. O multiset guarda finais, não as sequências completas.

- **LIS com restrições extras (k-LIS, LIS em subárvore, LIS pesada)** → dp[v] = max sobre valores < v indexado por valor comprimido num BIT de max; peso entra como soma.
  - *Gatilho:* a subsequência tem pesos, restrições de valores, ou precisa ser calculada dentro de subárvores/estruturas.
  - *Por que funciona:* para LIS pesada, o DP ainda é "melhor soma terminando em v"; um BIT de máximo sobre valores comprimidos responde ao melhor prefixo < v. Restrições adicionais viram dimensões, rollback ou processamento por subárvore.
  - *Como fazer:* comprima os valores; processe na ordem de índice; consulte BIT.query(v−1), calcule dp = consulta + peso, e atualize BIT em v. Para subárvore, faça DFS com BIT e rollback ou merge das estruturas; k-LIS normalmente exige mais uma dimensão para as violações permitidas.
  - *Exemplo:* LIS pesada de [2, 1, 3] com pesos [5, 1, 1]: a melhor é 2, 3, com peso 6.
  - *Armadilha:* um único BIT global não resolve sozinho LIS em subárvore nem k-LIS; use strict < conforme o enunciado e 64 bits para pesos.
- **Permutação com k inversões exatas.** Insira os valores do menor ao maior; ao inserir o i-ésimo você escolhe quantas inversões novas (0..i−1), então qualquer k ≤ n(n−1)/2 é construível por guloso.
  - *Gatilho:* construir uma permutação de 1..n com exatamente k inversões, ou decidir se isso é possível.
  - *Por que funciona:* ao inserir o novo maior valor entre os i−1 já colocados, colocá-lo x posições à direita do fim cria exatamente x inversões com os menores; as faixas 0..i−1 permitem atingir qualquer total até n(n−1)/2.
  - *Como fazer:* comece vazio; para i = 1..n, escolha x = min(k, i−1), insira i na posição x contada a partir da direita e faça k −= x. Se k > 0 no fim, não existe.
  - *Exemplo:* n = 3, k = 2: insira 1, depois 2 à esquerda (1 inversão), depois 3 uma posição à esquerda do fim, obtendo [2, 3, 1].
  - *Armadilha:* n(n−1)/2 pode estourar int; a posição é contada a partir da direita; a construção é para permutações distintas de 1..n.
- **Inversões de a + b ou "inversões após operação"**: separe em pares que mudam e pares que não; swap de i, j com valores distintos muda inversões por um número ímpar (paridade sempre inverte); trocar iguais não muda nada.
  - *Gatilho:* o problema soma inversões de dois arrays, ou pergunta como uma troca/operação altera a contagem de inversões.
  - *Por que funciona:* a soma de inversões separa contribuições de pares; uma troca de dois elementos distintos é uma transposição, que inverte a paridade da permutação. O número de inversões pode mudar por um valor ímpar, não necessariamente 1.
  - *Como fazer:* conte as inversões de cada parte separadamente; para simular uma troca, recalcule apenas os pares que envolvem i ou j e ajuste a paridade. Para pares iguais, a troca é invisível.
  - *Exemplo:* trocar dois elementos adjacentes fora de ordem reduz as inversões em 1; trocar elementos distantes pode mudar por 3, 5, etc.
  - *Armadilha:* saber que a paridade muda não dá o novo número; não conte pares (i, j) duas vezes ao separar as contribuições.
- **Cuidado com overflow** em contagens de pares (n^2 com $n = 2 \cdot 10^5 = 4 \cdot 10^{10}$), somas de prefixos ($2 \cdot 10^5 \times 10^9$) e produtos cruzados (a·d vs b·c com valores 10^9 → __int128).
  - *Gatilho:* n ou os valores são grandes e a resposta envolve produtos, somas acumuladas ou contagens de pares.
  - *Por que funciona:* int só garante cerca de $2 \cdot 10^9$; n^2, prefixos e produtos frequentemente passam disso. O sinal errado ou o wrap silencioso produz WA sem aviso.
  - *Como fazer:* use long long para contagens e somas; antes de multiplicar, faça o cast de pelo menos um fator; para produtos de dois valores 10^9 que serão multiplicados de novo, use __int128. Confira o limite superior da resposta.
  - *Exemplo:* $n = 2 \cdot 10^5$: $n(n-1)/2 \approx 2 \cdot 10^{10}$; prefixos de $2 \cdot 10^5$ valores 10^9 chegam a $2 \cdot 10^{14}$.
  - *Armadilha:* `int ans = n * (n - 1) / 2` estoura antes da divisão; compare produtos cruzados em __int128, não em double.
- **Comparador do sort deve ser ordem fraca estrita.** `<=` ou comparador não transitivo (ex.: com razão em double) causa UB/crash; use cross-multiplication em inteiros e desempate por índice.
  - *Gatilho:* você escreve um comparador customizado para ordenar por razão, por concatenação ou por múltiplos critérios.
  - *Por que funciona:* o sort exige uma relação assimétrica e transitiva; `<=` viola a irreflexividade e pode fazer o algoritmo acessar fora dos limites. Double pode quebrar a transitividade por arredondamento.
  - *Como fazer:* implemente `a antes de b` com `<` estrito; para razões com denominadores positivos, compare a.x·b.y < b.x·a.y; para empates, use o índice ou outro critério determinístico. Use __int128 se o produto estourar.
  - *Exemplo:* ordenar frações por a/b crescente com a·d < c·b para denominadores positivos, não com (double)a/b.
  - *Armadilha:* um comparador que retorna true para iguais pode causar crash; não use `<=`, normalize o sinal de denominadores negativos, não misture double com inteiros e não dependa de ordem não especificada.

- **Off-by-one em intervalos.** Padronize [l, r) em código e converta na entrada; prefixos P[0] = 0 com tamanho n + 1; diferenças com tamanho n + 2.
  - *Gatilho:* o problema mistura intervalos fechados, prefixos e diferenças, ou há muitos casos de borda com l = r.
  - *Por que funciona:* o intervalo semiaberto [l, r) tem comprimento r − l e não precisa de +1 em cada operação; prefixos com P[0] = 0 eliminam o caso especial do início.
  - *Como fazer:* converta [l, r] da entrada para [l, r+1) no código; use P de tamanho n+1 e d de tamanho n+2; ao acessar r, garanta que está dentro do array.
  - *Exemplo:* [2, 5] fechado vira [2, 6) e tem comprimento 4; a diferença atualiza d[2] e d[6].
  - *Armadilha:* converter de volta para fechado só na saída; intervalos vazios têm l = r; não use r+1 sem o sentinela.
- **Sentinelas** (±∞, ou um elemento extra no fim) removem casos de borda em pilha monotônica, sweep e merge de intervalos.
  - *Gatilho:* a pilha pode ficar vazia, o sweep começa/termina no limite, ou o merge precisa de um fim comum.
  - *Por que funciona:* um valor sentinela garante que sempre existe um topo ou um próximo evento, eliminando `if empty` repetidos; no fim, ele força o desempilhamento de todos os candidatos.
  - *Como fazer:* adicione um valor menor/maior que todos os reais nas extremidades, ou processe um evento extra em +∞; escolha o sentinela fora do domínio para não competir com valores reais.
  - *Exemplo:* em maior retângulo no histograma, uma barra de altura 0 no fim desempilha todas as alturas positivas.
  - *Armadilha:* o sentinela não pode ser contado como elemento real; com empates, ele pode alterar a convenção de máximo/mínimo; ajuste o índice final.
- **Guloso com duas chaves (vantagem a_i − b_i).** Escolher para cada item o lado A ou B com cotas → comece com todos em B e escolha os k com maior ganho a_i − b_i; com cotas mínimas e máximas, ordene por ganho e corte nas cotas.
  - *Gatilho:* cada item pode ir para um de dois grupos, com limites de quantidade em cada grupo, e o objetivo é maximizar a soma de valores.
  - *Por que funciona:* partir de todos em B dá uma base; trocar o item i para A ganha a_i − b_i. Com uma cota simples, escolher os k maiores ganhos é ótimo por troca; com cotas mínimas/máximas, a escolha é um corte ordenado sujeito aos limites.
  - *Como fazer:* calcule os ganhos, ordene decrescentemente e escolha os k melhores respeitando as cotas; para cotas [L_A, U_A], selecione os maiores ganhos dentro do intervalo permitido, completando o mínimo com os melhores restantes.
  - *Exemplo:* n = 3, k = 1, ganhos 5, −2, 3: escolha o item de ganho 5 para A e deixe os outros em B.
  - *Armadilha:* cotas acopladas (o mínimo de A força o máximo de B) podem invalidar o corte simples; ganhos negativos ainda podem ser obrigatórios para cumprir o mínimo.
- **Dois recursos por item e orçamento (comprar com dinheiro e cupons).** Guloso por custo marginal com dois heaps, ou DP com troca de estado; teste o guloso contra brute force antes de confiar.
  - *Gatilho:* cada item custa dinheiro ou um cupom (com desconto), há orçamento e quantidade limitada de cupons, e quer-se maximizar o número/valor comprado.
  - *Por que funciona:* o custo efetivo de um item depende de qual recurso é usado; se os recursos são independentes, escolher o menor custo marginal funciona, mas a troca de um item pode mudar o custo marginal de outro, então a prova exige cuidado.
  - *Como fazer:* modele o estado (dinheiro gasto, cupons usados) e faça DP se os orçamentos forem pequenos; para o guloso, mantenha heaps de custo com dinheiro e com cupom, compre o menor marginal e permita arrependimento. Valide com brute force em n pequeno.
  - *Exemplo:* item custa 10 ou 3 com cupom, e outro custa 6 nos dois modos; com orçamento 9 e um cupom, usar o cupom no primeiro compra os dois por 3 + 6 = 9, enquanto usá-lo no segundo não cabe no orçamento.
  - *Armadilha:* dois heaps sem arrependimento podem falhar quando o uso de um cupom muda o conjunto elegível; não confie no guloso sem contraexemplo/estresse.
- **Ordenar por comparação de concatenação.** Menor/maior número formado por concatenar strings: ordene por a+b < b+a; prova por transitividade da comparação numérica.
  - *Gatilho:* juntar todos os números/strings em uma única sequência para formar o menor ou o maior valor lexicográfico/numérico.
  - *Por que funciona:* para duas strings, a deve vir antes de b sse a+b < b+a; essa relação é transitiva e define uma ordem ótima. Trocar quaisquer vizinhos fora dessa ordem não melhora o resultado.
  - *Como fazer:* ordene com o comparador a+b < b+a (ou > para o maior); concatene. Em C++, cuidado com o custo de criar strings no comparador, mas para n pequeno é aceitável.
  - *Exemplo:* 9 e 34: "349" < "934", então 34 vem antes de 9 no menor número.
  - *Armadilha:* comparar numericamente a·10^len(b)+b pode estourar; todos os zeros devem produzir "0", não "000"; o comparador precisa ser transitivo.

- **Pilha para colisões e cancelamentos** (robôs que se anulam, parênteses reduzidos, apagar pares adjacentes): o resultado final é canônico e independe da ordem.
  - *Gatilho:* elementos adjacentes se cancelam ou se combinam, e o processo se repete até não haver mais pares.
  - *Por que funciona:* o cancelamento é local e confluente; processar da esquerda para a direita com uma pilha aplica todas as reduções sem precisar reiniciar. O resultado é a forma normal.
  - *Como fazer:* para cada novo elemento, compare com o topo e desempilhe enquanto a regra de colisão se aplicar; senão empilhe. No fim, a pilha é o resultado.
  - *Exemplo:* parênteses "(()" reduzem para "("; backspaces "ab#c" viram "ac"; robôs que se anulam deixam só os sem par.
  - *Armadilha:* pode haver reação em cadeia após um pop; use while, não if. A ordem da regra de colisão importa quando os dois lados são diferentes.
- **Sparse table / pré-cômputo antes do guloso**: se o guloso consulta max/min de intervalo, não reordene o array, use RMQ O(1) para manter n log n.
  - *Gatilho:* um guloso precisa repetidamente do máximo/mínimo de um intervalo do array original, e a ordem posicional é importante.
  - *Por que funciona:* ordenar destruiria os índices; a sparse table responde RMQ idempotente em O(1) após O(n log n), preservando a ordem original para o guloso.
  - *Como fazer:* pré-calcule st[k][i] para intervalos de tamanho 2^k; consulte com k = $\lfloor \log_2(\text{len}) \rfloor$ e combine st[k][l] com st[k][r−2^k+1]. Use para max, min, gcd, OR.
  - *Exemplo:* em um jogo de saltos, consultar o maior alcance em [i, i+k] sem alterar o array.
  - *Armadilha:* a sparse table usa O(n log n) de memória; para somas não idempotentes, use prefixo ou segtree; recalcule os logs por caso.

adicionado em 2026-11-11 a partir dos editoriais de CF Div. 2/3/4, A–D; triado contra o catálogo

- **Intervalos ocupados → olhe as lacunas.** Com k intervalos disjuntos ordenados, o espaço livre está nos k + 1 buracos: antes do primeiro, entre consecutivos e depois do último. Ex.: CF 1999C.
  - *Gatilho:* há k compromissos ocupados e se pergunta se sobra um intervalo livre de tamanho mínimo, ou onde está o maior buraco.
  - *Por que funciona:* intervalos disjuntos ordenados dividem a reta em exatamente k + 1 regiões livres; qualquer tempo livre está em uma delas, e os buracos podem ser medidos pelos extremos dos intervalos.
  - *Como fazer:* ordene os intervalos por início; verifique o buraco antes do primeiro, os buracos entre r_i e l_{i+1}, e o buraco depois do último. Se algum tiver comprimento ≥ s, a resposta é possível; as fórmulas usam l_i − r_{i−1} (ou −1 no modelo fechado).
  - *Exemplo:* CF 1999C: compromissos [2,3] e [6,7], com dia útil até 10; os buracos são [1,1], [4,5] e [8,10], de comprimentos 1, 2 e 3.
  - *Armadilha:* o modelo fechado muda o comprimento do buraco em 1; os intervalos podem vir desordenados; não esqueça os buracos nas duas pontas.
- **Substituir prefixo pelo mínimo / sufixo pelo máximo.** Um elemento pode sobreviver sse é mínimo do seu prefixo ou máximo do seu sufixo (um "recorde"); marque com prefix min e suffix max. Ex.: CF 2241D.
  - *Gatilho:* operações que substituem um prefixo inteiro pelo seu mínimo (ou um sufixo pelo seu máximo), e a pergunta é quais posições ainda podem permanecer com o valor original.
  - *Por que funciona:* uma operação de prefixo só pode sobrescrever uma posição com um valor menor ou igual vindo da esquerda; para sobreviver, o elemento precisa ser um recorde mínimo do prefixo. Analogamente, um recorde máximo do sufixo sobrevive às operações pela direita.
  - *Como fazer:* calcule prefMin[i] = min(a[0..i]) e sufMax[i] = max(a[i..n−1]); marque como possível toda posição em que a[i] = prefMin[i] ou a[i] = sufMax[i]. Faça as operações de trás para frente se precisar reconstruir.
  - *Exemplo:* CF 2241D; em [3, 1, 2], a posição do 1 é recorde mínimo e a do 2 é recorde máximo do sufixo.
  - *Armadilha:* igualdades contam como recorde; a ordem das operações de prefixo e sufixo pode remover um candidato que parecia sobreviver, então valide a construção.

- **Equilibrar por transferências unitárias.** Se cada passo tira 1 de quem está acima do alvo e dá a quem está abaixo, o nº de passos é Σ max(0, a_i − alvo). Ex.: CF 2131A.
  - *Gatilho:* há um alvo de nível para todos e cada operação move exatamente uma unidade de uma posição acima para uma abaixo.
  - *Por que funciona:* o excesso total acima do alvo é igual ao déficit total abaixo, se a soma é compatível; cada transferência unitária corrige uma unidade de excesso e uma de déficit, então o total de excesso é o número mínimo de passos.
  - *Como fazer:* calcule alvo (frequentemente a média, um valor dado ou o mínimo/máximo viável); some max(0, a_i − alvo) ou, equivalentemente, max(0, alvo − a_i). Verifique se a soma total fecha.
  - *Exemplo:* CF 2131A; [1, 3, 5] com alvo 3 exige 2 transferências de 5 para 1, cada uma movendo uma unidade; o excesso total é 2 e o déficit total é 2.
  - *Armadilha:* se a soma não for múltipla de n ou o alvo não for inteiro, pode ser impossível; se as transferências só podem ocorrer entre vizinhos, o custo é outro (distância).
- **a_i + a_{i+1} constante ⇒ período 2.** Subtraindo duas equações vizinhas, a_{i+2} = a_i: o array é determinado pelos dois primeiros valores. Vale para qualquer relação em que o termo do meio se cancela. Ex.: CF 2256B.
  - *Gatilho:* o enunciado impõe a_i + a_{i+1} = c para todo i, ou uma relação local que envolve três termos e cancela o do meio.
  - *Por que funciona:* (a_{i+1} + a_{i+2}) − (a_i + a_{i+1}) = 0, logo a_{i+2} = a_i. O array alterna entre dois valores, e os dois primeiros determinam todo o resto.
  - *Como fazer:* verifique se a relação vale para o primeiro par; depois propague a_2 = c − a_1, a_3 = a_1, a_4 = a_2, etc., ou compare diretamente a_i com a_{i mod 2}.
  - *Exemplo:* CF 2256B; com c = 5 e a_1 = 3, o array é 3, 2, 3, 2, …
  - *Armadilha:* a relação pode ser modular ou valer só a partir de um índice; use a_0/a_1 de forma consistente e não assuma que c é conhecido sem calculá-lo.
- **Recuperar pesos de Σ w_j·\|i − x_j\|.** Com f(i) dado em todos os inteiros, a segunda diferença f(i+1) − 2f(i) + f(i−1) = 2·(peso total em x = i); as pontas saem por substituição. Ex.: CF 2254D.
  - *Gatilho:* você recebe os valores de uma função que é soma de distâncias absolutas ponderadas e precisa reconstruir os pesos ou as posições.
  - *Por que funciona:* cada termo w·\|i − x\| é linear fora de x e tem uma "quina" em x; a segunda diferença discreta é 0 fora dos pontos e vale 2w exatamente em x.
  - *Como fazer:* calcule g(i) = f(i+1) − 2f(i) + f(i−1); então W_i = g(i)/2 é o peso total em i. Se houver um peso por posição, isso recupera todos; use dois valores de f para determinar a constante e a inclinação, se necessário.
  - *Exemplo:* CF 2254D; um único peso 3 em x = 2 produz segunda diferença 6 apenas em i = 2.
  - *Armadilha:* pesos múltiplos na mesma coordenada só aparecem somados; a segunda diferença precisa de f nos dois vizinhos; use 64 bits e trate o domínio finito com cuidado.

Repetidas de outras seções:

- **Σ tamanhos ≤ S** → ≤ √(2S) tamanhos distintos; agrupe por tamanho.
- **Soma/balanço = 0** → prefixos iguais (também em strings de parênteses e xor).
- **gcd/OR/AND de subarray** → ≤ log mudanças por fim fixo.
- **Jogo em array** → pontas, paridade de posições, DP em intervalo (ver Teoria dos jogos).

Busca binária e otimização

Gatilho principal: "minimizar o máximo" ou "maximizar o mínimo" → binária na resposta.

- **Binária na resposta** sempre que a checagem "dá com x?" é monótona e mais fácil que achar o ótimo.
  - *Gatilho:* "minimizar o máximo", "maximizar o mínimo", "menor X tal que é possível", "maior X que ainda atende"; a resposta é um número (capacidade, tempo, distância) e existe um predicado de decisão.
  - *Por que funciona:* se check(x) vale para todo x ≥ X (ou para todo x ≤ X), os valores viáveis formam um intervalo com uma única fronteira; essa fronteira é o ótimo. Decidir costuma ser estritamente mais fácil que otimizar.
  - *Como fazer:* prove a monotonicidade, escolha [lo, hi] com lo viável e hi inviável (ou o contrário) e binária; O(log(hi − lo) · custo(check)). Para reais, use 100 iterações fixas.
  - *Exemplo:* comprimento máximo de tábua para cortar k pedaços — check(c) = Σ⌊L_i/c⌋ ≥ k é decrescente em c.
  - *Armadilha:* check que exige valor exato ("existe com exatamente x") não é monótono; hi precisa ser comprovadamente inviável, não apenas "grande".

- **k-ésimo menor** de um conjunto implícito (somas de pares, frações, produtos de tabela) → binária em x e conte quantos ≤ x.
  - *Gatilho:* "k-ésimo menor/maior", "mediana", conjunto gerado por combinação (todas as somas de pares, todos os produtos, todas as frações), grande demais para materializar.
  - *Por que funciona:* C(x) = #{elementos ≤ x} é não decrescente; o k-ésimo menor é o menor x com C(x) ≥ k (contando multiplicidades). A contagem costuma ser O(n) ou O(n log n).
  - *Como fazer:* binária em [min, max] dos valores possíveis; no check conte em O(n); total O(custo · log V). Para k-ésimo maior, conte ≥ x ou troque k por N − k + 1.
  - *Exemplo:* k-ésimo menor de todas as n^2 somas de dois arrays.
  - *Armadilha:* duplicatas contam por multiplicidade (defina se o k-ésimo é distinto); valores negativos deslocam o intervalo; se os valores são frações, a binária em real não devolve o valor exato — reconstrua.
- **Média/razão máxima** (Σa/Σb) → binária em r e verifique se max Σ(a − r·b) ≥ 0.
  - *Gatilho:* "maximizar a média", "densidade", "taxa Σa/Σb" sobre subarrays, subconjuntos, caminhos ou pares.
  - *Por que funciona:* existe elemento com Σa/Σb ≥ r sse max(Σa − rΣb) ≥ 0; o conjunto viável de r é um intervalo (−∞, r*], então o check é monótono. A função r ↦ max(Σa − rΣb) é convexa e cruza zero em r*.
  - *Como fazer:* binária em r; no check resolva a maximização linear de Σ(a − r·b): Kadane para subarray, ordenação/guloso para subconjunto, DP para caminho. O(log precisão × custo).
  - *Exemplo:* subarray de densidade máxima — Kadane em c_i = a_i − r·b_i.
  - *Armadilha:* o conjunto vazio dá 0 e pode dominar o máximo — exija não vazio; b_i = 0; precisão de double; para valor exato use Dinkelbach.
- **Binária paralela.** Várias queries cada uma com sua binária e uma estrutura cara para simular → processe todas juntas por nível.
  - *Gatilho:* q consultas independentes, cada uma com sua binária, e o check de cada uma usa os mesmos dados/eventos (conectividade, ordenação, sweep).
  - *Por que funciona:* em cada nível, as consultas compartilham o mesmo mid ou podem ser agrupadas; uma única passada responde todas, em vez de q passadas. Total O((n + q) log V · custo amortizado).
  - *Como fazer:* guarde lo_q, hi_q por consulta; a cada nível agrupe por mid, processe os eventos em ordem crescente, responda cada consulta e atualize lo/hi; repita ⌈log V⌉ níveis.
  - *Exemplo:* q pares (u, v) com arestas adicionadas no tempo — instante em que ficam conectados (ver Binária sobre o tempo com DSU).
  - *Armadilha:* se cada consulta exige estrutura própria ou rollback entre mids, a economia some; a ordem dos eventos precisa ser respeitada.
- **Binária na segtree** (descer na árvore) em vez de binária com query: tira um log.
  - *Gatilho:* "menor prefixo com soma ≥ x", "primeira posição com valor ≥ x", e a estrutura já mantém somas/máximos.
  - *Por que funciona:* a árvore resume cada intervalo pelo agregado; descer da raiz comparando o filho esquerdo localiza a fronteira em O(log n), sem query + binária (O($\log^2$)).
  - *Como fazer:* no nó, se o agregado do filho esquerdo satisfaz o critério, desça à esquerda; senão ajuste o alvo (subtraia a soma da esquerda) e desça à direita. Para "primeiro ≥ x", use o máximo.
  - *Exemplo:* menor i com prefixo ≥ x em um BIT (descida por potências de 2); primeiro elemento > x à direita de uma posição.
  - *Armadilha:* exige agregado decomponível e prefixo monótono — com valores negativos "menor prefixo com soma ≥ x" não é monótono e a descida não vale; BIT só com frequências ≥ 0.
- **Função unimodal** → ternária; inteiros → binária na diferença f(x+1) − f(x).
  - *Gatilho:* "minimizar a maior distância", "ponto que minimiza o custo", função de uma variável com um único vale/pico (contínua ou inteira).
  - *Por que funciona:* com m1 < m2, se f(m1) < f(m2) o mínimo não está à direita de m2 (para um vale); simetricamente do outro lado. Cada passo descarta cerca de 1/3 do intervalo.
  - *Como fazer:* m1 = l + (r − l)/3, m2 = r − (r − l)/3; atualize e pare com r − l pequeno (nos inteiros, ≤ 3 e teste os candidatos). Alternativa inteira: binária em f(m) vs f(m+1).
  - *Exemplo:* posição de uma antena que minimiza a maior distância aos clientes.
  - *Armadilha:* dois vales locais (não unimodal) ou platôs; com doubles use iterações fixas; a binária na diferença exige ausência de platôs fora do ótimo.

- **Função convexa na quantidade** (custo com exatamente k itens) → Aliens trick / lambda optimization.
  - *Gatilho:* DP com "exatamente k itens/blocos/segmentos", k até n, e o custo como função de k é convexo (ou côncavo).
  - *Por que funciona:* cobrando λ por item, resolve-se a versão sem restrição de k; o número ótimo de itens cai monotonicamente com λ, e a envoltória de f recupera f(K).
  - *Como fazer:* no DP, cada transição que compra um item soma λ e conta +1; guarde também o k ótimo; binária em λ até k = K; resposta = val(λ) ± λK conforme a convenção (ver Aliens com empates). O(log V · DP sem k).
  - *Exemplo:* escolher exatamente k subarrays disjuntos de soma máxima; particionar em k blocos.
  - *Armadilha:* f precisa ser convexa/côncava globalmente; λ grande pode estourar 64 bits; empates exigem desempate consistente.
- **Binária + guloso**, **binária + DP**, **binária + fluxo/matching** são combinações comuns: a checagem quase nunca é o próprio problema.
  - *Gatilho:* o enunciado pede um ótimo sob um recurso (capacidade, tempo, número de máquinas) e a decisão "dá com recurso x?" tem algoritmo conhecido.
  - *Por que funciona:* a decisão substitui a otimização por um predicado monótono; a binária converte "minimize o recurso" em log V decisões.
  - *Como fazer:* escreva check(x) com o algoritmo mais simples que decide corretamente — guloso para intervalos/agendamento, DP para partição, max-flow/matching para capacidades — e binária em x; O(log V · custo(check)).
  - *Exemplo:* k máquinas com makespan ≤ T (binária em T + agendamento); cobertura com raio r (binária + guloso).
  - *Armadilha:* usar guloso aproximado no check; se a decisão já é NP-difícil, a formulação está errada — procure outra estrutura.

adicionado em 2026-11-10 por agentes; revisado

- **Binária em reais.** Use um número fixo de iterações (100) em vez de `hi - lo > eps`, que pode não terminar com doubles grandes; imprima com 9+ casas e confira se o erro exigido é relativo ou absoluto.
  - *Gatilho:* a resposta é tempo, distância, raio, probabilidade ou razão; o enunciado pede erro absoluto ou relativo de 1e−6 a 1e−9.
  - *Por que funciona:* 100 bisseções reduzem o intervalo por 2^{100}, muito além da precisão de double (~15–16 dígitos); o número fixo de passos evita o loop que não termina quando hi − lo fica menor que o ulp.
  - *Como fazer:* `for (int it = 0; it < 100; it++) { double mid = (lo + hi)/2; if (ok(mid)) lo = mid; else hi = mid; }` e devolva lo; imprima com `setprecision(10)` ou o exigido. Para erro relativo, compare |ans − real|/max(1, real).
  - *Exemplo:* menor raio de círculo que cobre todos os pontos; instante de encontro de dois móveis.
  - *Armadilha:* `while (hi - lo > 1e-9)` com valores ~10^{18} nunca para (ulp ≫ 1e−9); float tem precisão insuficiente; cuidado com o sinal e com o intervalo inicial.
- **Binária inteira sem loop infinito.** Defina o invariante (lo = ok, hi = ruim) e use mid = lo + (hi − lo)/2 com a atualização consistente; para "maior x ok" use mid = (lo + hi + 1)/2. Negativos: `/2` arredonda para 0, não para baixo.
  - *Gatilho:* binária em inteiros, limites grandes ou negativos, "maior x que satisfaz" / "menor x que satisfaz".
  - *Por que funciona:* com lo viável, hi inviável e mid estritamente entre eles, o intervalo encolhe a cada passo; quando hi = lo + 1, lo é a resposta. A atualização precisa preservar o invariante.
  - *Como fazer:* "maior ok": lo = ok, hi = ruim; `while (hi - lo > 1) { mid = lo + (hi - lo)/2; ok(mid) ? lo = mid : hi = mid; }`. "Menor ok": lo = ruim, hi = ok; `if (ok(mid)) hi = mid; else lo = mid;`. Alternativa inclusiva: `mid = lo + (hi - lo + 1)/2` com `lo = mid`.
  - *Exemplo:* menor capacidade que atende à demanda; primeiro índice com a_i ≥ x.
  - *Armadilha:* hi que não é inviável gera resposta errada; (lo + hi)/2 estoura int se a soma passa de $2^{31}-1$; em negativos a divisão trunca para zero (−3/2 = −1), então `lo + (hi - lo)/2` é mais seguro (e depende da linguagem: em Python `//` arredonda para baixo).
- **Limite superior desconhecido.** Busca exponencial (1, 2, 4, ...) até o check falhar e depois binária no último intervalo; útil em interativos com n grande e valores 10^{18}.
  - *Gatilho:* não há limite superior confiável (a resposta pode chegar a 10^{18}) ou o problema é interativo e o máximo não é dado.
  - *Por que funciona:* dobrar hi até check falhar garante que a resposta está em (hi/2, hi]; uma binária normal resolve esse intervalo. O custo é O(log resposta), independente do valor.
  - *Como fazer:* `hi = 1; while (ok(hi)) hi *= 2;` e depois binária em [hi/2, hi]. Se a resposta pode ser 0, comece com lo = 0 viável.
  - *Exemplo:* menor tempo para produzir N peças com taxa desconhecida; achar limite superior em problema interativo.
  - *Armadilha:* `hi *= 2` estoura (pare em ~2^{60} ou use `__int128`); ok(hi) precisa fazer sentido em todo o domínio; com resposta 0 o primeiro hi já pode ser inviável.

- **mid*mid ou mid*k em 10^{18}.** Faça o check em __int128 ou compare com divisão (mid ≤ lim / k) para não estourar.
  - *Gatilho:* o check compara mid^2, mid·k, mid^3 ou outro produto de valores até 10^{18}, e o produto pode passar de $2^{63}-1$.
  - *Por que funciona:* reescrever a comparação como divisão preserva a ordem sem calcular o produto; `__int128` (ou equivalente) calcula o produto exato.
  - *Como fazer:* troque `mid * k <= lim` por `mid <= lim / k` (k > 0); para quadrados, `mid <= lim / mid`; ou use `(__int128)mid * k <= lim`. Em somas, cheque o overflow antes de somar.
  - *Exemplo:* verificar se $mid^2 \le n$ com $n \le 10^{18}$ → `mid <= n / mid`.
  - *Armadilha:* divisão por zero; com negativos a divisão trunca e a equivalência pode falhar (trate sinais antes); `mid * mid` em int estoura já perto de 46 341.
- **Binária em array bitônico ou rotacionado.** Compare a[m] com a[m+1] (pico) ou com a[0] (rotação); platôs de valores iguais quebram a monotonicidade e forçam O(n) no pior caso.
  - *Gatilho:* o array cresce e depois decresce (ou o contrário), ou é ordenado e rotacionado; busca do pico, do mínimo ou de um valor.
  - *Por que funciona:* no bitônico, a[m] < a[m+1] indica subida (pico à direita) e a[m] > a[m+1] indica descida (pico em m ou à esquerda); no rotacionado, comparar a[m] com um extremo identifica a metade ordenada e os extremos permitem descartar.
  - *Como fazer:* pico: `while (lo < hi) { m = (lo + hi)/2; if (a[m] < a[m+1]) lo = m + 1; else hi = m; }`. Rotacionado: ache a rotação e faça duas binárias, ou uma binária decidindo a metade ordenada.
  - *Exemplo:* mínimo em array ordenado rotacionado; pico de uma sequência unimodal.
  - *Armadilha:* com duplicatas, a[m] == a[m+1] (ou a[m] == a[hi]) não decide o lado e o pior caso vira O(n); valide m+1 < n.
- **Dinkelbach para razão máxima Σa/Σb.** Itere r ← (Σa)/(Σb) do conjunto ótimo para o r atual; converge superlinearmente em poucas iterações e dá o valor exato, sem binária em real.
  - *Gatilho:* maximizar Σa/Σb sobre uma família discreta (subarray, subconjunto, caminho) e querer o valor exato, não uma aproximação de binária.
  - *Por que funciona:* se r não é ótimo, o maximizador de Σa − rΣb tem razão > r; atualizar r para essa razão aumenta o objetivo estritamente. No ótimo o máximo vale 0. Para família finita termina, com convergência superlinear.
  - *Como fazer:* r = 0 (ou razão viável); repita { ache S maximizando Σa − rΣb; se valor ≤ 0 pare; r = Σa(S)/Σb(S); }; devolva r. Cada iteração custa o mesmo que o check da binária.
  - *Exemplo:* subarray de densidade máxima (Kadane a cada iteração); escala ótima de uma mistura.
  - *Armadilha:* Σb = 0; iterações em double exigem tolerância na parada; desempate inconsistente pode ciclar; o valor final é razão de inteiros, mas não o reconstrua por divisão inteira.
- **k-ésimo de A[i] + B[j].** Binária em x, conte pares ≤ x com dois ponteiros nos dois arrays ordenados; para k-ésimo de A[i]·B[j] com sinais, divida em quatro casos.
  - *Gatilho:* k-ésimo menor/maior de todas as somas (ou produtos) A[i] + B[j], com n·m grande demais para materializar.
  - *Por que funciona:* C(x) = #{(i,j): A[i] + B[j] ≤ x} é monótona em x; com os arrays ordenados, ao crescer i o limite x − A[i] decresce, então o número de j válidos só diminui — dois ponteiros contam em O(n + m).
  - *Como fazer:* ordene A e B; binária em x entre A[0] + B[0] e A[n−1] + B[m−1]; no check, i crescente e j decrescente: enquanto A[i] + B[j] > x, j−−; some j + 1; pare ao atingir k. Para produtos, separe por sinais em quatro quadrantes e aplique a contagem com a ordem adequada.
  - *Exemplo:* k-ésimo menor de A[i] + B[j] com $n = m = 10^5$.
  - *Armadilha:* overflow em A[i] + B[j] e em x − A[i] (use 64 bits); produtos com zero e sinais mistos exigem os quadrantes separados.
- **k-ésimo em matriz com linhas e colunas ordenadas / tabela de multiplicação n × m.** Binária em valor + "escada" O(n + m) ou soma min(m, ⌊x / i⌋) por linha.
  - *Gatilho:* matriz n × m com linhas e colunas ordenadas, ou tabela i·j; k-ésimo menor sem construir as n·m entradas.
  - *Por que funciona:* a contagem de entradas ≤ x é monótona em x; na matriz ordenada a escada a partir do canto superior direito conta em O(n + m); na tabela i·j a linha i contribui com min(m, ⌊x/i⌋) valores.
  - *Como fazer:* binária em x; check por escada (se a entrada ≤ x, conte e desça; senão vá à esquerda) ou $\Sigma_i \min(m, \lfloor x/i \rfloor)$; O((n+m) log V) ou O(n log V).
  - *Exemplo:* k-ésimo menor produto i·j com 1 ≤ i ≤ n e 1 ≤ j ≤ m.
  - *Armadilha:* multiplicação que estoura em ⌊x/i⌋ e nos limites; repetições contam por multiplicidade (não é k-ésimo distinto); a escada exige as duas ordenações.

- **k-ésima menor fração p/q com q ≤ n.** Binária em real x, conte frações ≤ x com dois ponteiros e guarde a maior fração ≤ x vista; se as frações são reduzidas/distintas, a contagem por piso repete representações — use Farey/Stern–Brocot (ou Möbius) para o rank exato, pois só a maior ≤ x não basta.
  - *Gatilho:* "k-ésima menor fração p/q", "denominador ≤ n", sequência de Farey, conjunto implícito de frações.
  - *Por que funciona:* a contagem de frações ≤ x é monótona; para cada q há ⌊x·q⌋ valores de p (respeitando o intervalo), então o check é O(n). O problema é o rank: representações não reduzidas contam a mesma fração várias vezes.
  - *Como fazer:* binária em x real contando com um ponteiro p que cresce com q; guarde a maior fração ≤ x para reconstruir. Para rank de frações distintas/reduzidas, use Farey/Stern–Brocot para achar a k-ésima diretamente, ou Möbius para contar irredutíveis (Σ_d μ(d)·(contagem dos múltiplos)).
  - *Exemplo:* k-ésima fração da sequência de Farey de ordem n (1 ≤ p < q ≤ n).
  - *Armadilha:* contar 2/4 e 1/2 como coisas diferentes muda o rank; double pode não distinguir frações próximas com n grande; só a maior ≤ x não corrige um rank errado.
- **Mediana/k-ésimo de dois arrays ordenados** → binária na partição (quantos tirar do primeiro) em O(log min(n, m)), comparando as bordas cruzadas.
  - *Gatilho:* mediana ou k-ésimo de dois arrays já ordenados, com exigência de O(log).
  - *Por que funciona:* uma partição válida toma i elementos do primeiro e j = k − i do segundo, com todos os da esquerda ≤ todos os da direita; as condições envolvem só as quatro bordas (A[i−1], A[i], B[j−1], B[j]) e são monótonas em i.
  - *Como fazer:* binária em i ∈ [max(0, k−m), min(k, n)]; j = k − i; se A[i−1] > B[j], diminua i; se B[j−1] > A[i], aumente i; pare na partição válida. Para mediana, k = (n+m+1)/2 (e também (n+m)/2 se o total é par).
  - *Exemplo:* mediana de dois arrays ordenados de tamanhos diferentes.
  - *Armadilha:* usar −∞/+∞ como sentinelas para bordas fora do array; arrays vazios; total par exige a média dos dois centrais.
- **Binária em estruturas.** BIT (só com frequências não negativas): descida por potências de 2 acha o menor prefixo com soma ≥ x em O(log); segtree: descida pelo nó em vez de binária + query; sparse table: saltos 2^k decrescentes; árvore: binary lifting para "primeiro ancestral que viola".
  - *Gatilho:* "primeiro prefixo com soma ≥ x", "primeiro ancestral que viola", "primeira posição com máximo ≥ x"; a estrutura já existe.
  - *Por que funciona:* cada estrutura localiza a fronteira descendo por seus níveis, sem binária externa + query; BIT de soma, segtree, sparse table e binary lifting permitem uma descida O(log).
  - *Como fazer:* BIT: desça por potências de 2 (se pos + bit ≤ n e soma + tree[pos+bit] < x, avance); segtree: desça comparando o filho esquerdo; sparse table: salte 2^k decrescente mantendo o agregado; árvore: binary lifting de 2^k para achar o primeiro ancestral que viola.
  - *Exemplo:* menor i com prefixo ≥ x; primeiro ancestral com peso > W; primeira posição à direita com valor ≥ x.
  - *Armadilha:* BIT só com frequências não negativas (prefixo monótono); sparse table exige predicado monotônico no intervalo; na árvore, cuidado com a raiz e com a direção de "primeiro".
- **Binária sobre o tempo com DSU** ("primeiro instante em que u e v conectam"): Kruskal reconstruction tree, ou binária paralela com um DSU percorrido uma vez por nível.
  - *Gatilho:* arestas/operações adicionadas ao longo do tempo; "primeiro instante em que u e v estão conectados", "quando o grafo fica conexo".
  - *Por que funciona:* a conectividade é monótona no tempo (só adiciona); existe um instante de ativação por par. A Kruskal reconstruction tree codifica todos os instantes em uma árvore; a binária paralela faz uma passada por nível com rollback.
  - *Como fazer:* KRT: rode Kruskal; cada união cria um nó com peso = tempo, e o instante de u, v é o peso do LCA. Alternativa: binária paralela com DSU rollback (sem compressão de caminho) processando eventos por nível.
  - *Exemplo:* rede que cresce; consultas "em que momento a e b se conectam?".
  - *Armadilha:* com remoções a monotonia some (use DSU rollback + D&C no tempo); o DSU da binária paralela não pode usar compressão de caminho, senão o rollback é inviável.
- **Binária paralela (variante D&C).** Em vez de rodar a estrutura log vezes, resolva(lo, hi, queries): aplique os eventos de [lo, mid], decida cada query para esquerda/direita, desfaça (rollback) e recurse; $O((n + q) \log^2 n)$, pois o DSU rollback sem compressão de caminho paga O(log n) por operação.
  - *Gatilho:* mesma situação da binária paralela, mas com q grande e a estrutura (DSU) precisando ser desfeita entre testes.
  - *Por que funciona:* cada evento é aplicado O(log n) vezes (uma por nível da recursão) e cada query é testada O(log n) vezes; com rollback sem compressão, cada operação custa O(log n), total $O((n + q) \log^2 n)$.
  - *Como fazer:* solve(lo, hi, queries) com o DSU no estado "eventos < lo aplicados"; aplique [lo, mid]; teste e separe cada query; desfaça [lo, mid] e recurse. Mantenha uma pilha de alterações para o rollback.
  - *Exemplo:* primeiro instante de conexão para q pares; "quando cada aresta entra na MST".
  - *Armadilha:* esquecer de desfazer antes de recursar; usar compressão de caminho (impossível desfazer); o custo $O(\log^2)$ só vale com união por tamanho.

- **Monotonicidade precisa ser provada.** Se check(x) = "existe solução com valor x" e esticar x pode piorar (ex.: tamanho exato em vez de ≤), a binária falha; reformule como ≥ ou ≤.
  - *Gatilho:* você quer binária, mas o predicado é "existe solução com exatamente x", não "com valor ≤ x"/"≥ x".
  - *Por que funciona:* a binária exige que {x : check(x)} seja um prefixo ou sufixo; "exatamente x" pode valer em x, falhar em x+1 e voltar em x+2, então não há fronteira única.
  - *Como fazer:* reformule para "existe com valor ≤ x" (crescente) ou "≥ x" (decrescente); se o problema pede exatamente k, tente Aliens (se convexo) ou DP com k no estado; teste a monotonicidade com força bruta em casos pequenos.
  - *Exemplo:* "maior subarray com soma exatamente S" não é binária; "maior subarray com soma ≤ S" é.
  - *Armadilha:* monotonicidade local não implica global; check que falha por empate ou precisão cria falsos "não" e quebra a binária.
- **Check mais barato que n log.** Pré-ordene fora da binária; um check O(n) + binária bate sort dentro do check. Dois ponteiros costuma tirar a binária interna ("menor janela" não precisa de binária + prefixo).
  - *Gatilho:* a solução óbvia é binária + sort/estrutura dentro do check, e o tempo está apertado.
  - *Por que funciona:* o fator log da binária multiplica o custo do check; pré-processar fora dela (ordenar uma vez, prefixos) reduz o check a O(n) e o total a O(n log V). Muitas vezes dois ponteiros eliminam a binária interna.
  - *Como fazer:* ordene/compresse uma vez; no check use dois ponteiros, prefixos ou fila monotônica; se o check busca em outro array, tente varrer os dois em ordem.
  - *Exemplo:* "menor janela com soma ≥ S" com positivos → dois ponteiros O(n), sem binária; se precisa de binária no tamanho, check O(n).
  - *Armadilha:* otimizar o check mas manter sort dentro; dois ponteiros só valem com monotonicidade (positivos); não confunda o log da binária com o log do sort.
- **Maximizar a menor distância** (colocar k itens numa reta, "vacas agressivas") → binária em d + guloso que coloca o próximo ponto o mais cedo possível.
  - *Gatilho:* "colocar k itens", "maior distância mínima entre escolhidos", vacas agressivas, postos ao longo de uma reta.
  - *Por que funciona:* se é possível com distância d, também é com d' ≤ d (monotonia); para d fixo, o guloso que escolhe o próximo ponto o mais cedo possível maximiza quantos cabem.
  - *Como fazer:* ordene as posições; binária em d; check: pegue o primeiro ponto e, para cada seguinte, se pos − último ≥ d, escolha e conte; viável se contou ≥ k. O(n log V).
  - *Exemplo:* vacas agressivas (USACO): maximizar a menor distância entre k vacas em estábulos.
  - *Armadilha:* o guloso precisa escolher o mais cedo possível; d = 0 e k = 1; posições duplicadas.
- **Partição em k blocos minimizando o maior soma** → binária em S + guloso que fecha o bloco quando estoura; é o padrão de "minimizar o máximo" com 2 níveis.
  - *Gatilho:* "dividir o array em k partes contíguas minimizando a maior soma" (ou o maior custo), "pintar cercas com k pintores".
  - *Por que funciona:* para um limite S, o guloso que fecha o bloco quando o próximo elemento estoura usa o mínimo de blocos possível; se esse mínimo ≤ k, dá para dividir em exatamente k (com valores não negativos, abrindo blocos extras). A viabilidade é monótona em S.
  - *Como fazer:* binária em S ∈ [max a_i, Σ a_i]; check: percorra somando; se soma + a_i > S, feche o bloco e comece outro; viável se blocos ≤ k. O(n log Σa).
  - *Exemplo:* "Capacity To Ship Packages Within D Days" (k dias); livros para k copistas.
  - *Armadilha:* com negativos o guloso falha; "no máximo k" vs "exatamente k" (com não negativos, ≤ k basta); limite inferior max a_i.
- **Gargalo (bottleneck).** Caminho que minimiza a maior aresta = caminho na MST; binária + DSU/BFS funciona mas é mais lenta; "min cap de fluxo ≥ F" → binária na capacidade + max-flow.
  - *Gatilho:* "minimizar a maior aresta do caminho", "capacidade mínima para ir de A a B", "min cap de fluxo ≥ F".
  - *Por que funciona:* em grafo não direcionado, o caminho minimax entre u e v é o caminho na MST; e "existe caminho com arestas ≤ C" é monótono em C, então cabe binária.
  - *Como fazer:* MST (Kruskal) e responda o máximo no caminho com binary lifting; ou binária em C + DSU/BFS usando só arestas ≤ C. Para "max-flow ≥ F com capacidade mínima", binária na capacidade e rode max-flow.
  - *Exemplo:* menor capacidade de caminhão para atravessar a cidade; gargalo em rede.
  - *Armadilha:* a MST resolve o minimax não direcionado (em direcionado, não); binária + BFS é O(m log C) e pode perder para a MST; max-flow por binária é caro — confira as constraints.

- **Aliens com empates.** Se há vários k ótimos para o mesmo λ, escolha sistematicamente o menor (ou o maior) número de itens e compare com K; fixe a convenção: com val(λ) = max_k [f(k) − λ·k] a resposta é val(λ) + λ·K, e com val(λ) = min_k [f(k) + λ·k] é val(λ) − λ·K. λ inteiro basta quando custos são inteiros.
  - *Gatilho:* Aliens trick e mais de um k ótimo para o mesmo λ; você precisa de exatamente K.
  - *Por que funciona:* a função penalizada tem vários maximizadores; escolher sempre o menor (ou sempre o maior) k torna a resposta de k monótona em λ e permite a binária; no ponto de ótimo vale f(K) = val(λ) ± λK.
  - *Como fazer:* no DP guarde (valor, k) e, no empate, prefira o menor k (ou o maior); binária em λ procurando K; aplique a convenção do sinal: max com −λk → val + λK; min com +λk → val − λK. λ inteiro basta se os custos são inteiros.
  - *Exemplo:* exatamente K subarrays com Aliens; MST com exatamente K arestas de uma cor.
  - *Armadilha:* misturar as convenções de sinal; desempate inconsistente faz a binária oscilar; confira se K é alcançável.
- **Aliens clássicos.** Escolher exatamente k subarrays disjuntos de soma máxima; MST com exatamente k arestas vermelhas (somar λ às vermelhas); fatiar o array em k blocos com custo quadrangular; "exatamente k trocas/compras" em ações.
  - *Gatilho:* reconhecer um "exatamente k" com custo aditivo e f convexa/côncava.
  - *Por que funciona:* são aplicações diretas do Aliens trick (ver Aliens com empates): cada item/unidade recebe λ, o DP sem k é simples e a convexidade vem da estrutura.
  - *Como fazer:* escolha a formulação: k subarrays disjuntos de soma máxima (penalize o início de cada subarray); MST com k arestas vermelhas (some λ às vermelhas); k blocos com custo quadrangular (penalize cada corte); k transações em ações (penalize cada compra).
  - *Exemplo:* "exatamente k segmentos de soma máxima" com DP O(n) para cada λ.
  - *Armadilha:* a convexidade é pré-requisito — confira com força bruta (ver Teste de convexidade); o DP sem k precisa ser barato.
- **Teste de convexidade para Aliens.** Calcule f(k) por brute force para k pequeno e verifique f(k+1) − f(k) monótona; se for côncava/convexa só por trechos, Aliens falha.
  - *Gatilho:* você suspeita que Aliens se aplica, mas não tem prova da convexidade de f(k).
  - *Por que funciona:* Aliens exige diferenças f(k+1) − f(k) monótonas (f convexa para minimização, côncava para maximização); se elas sobem e descem, a envoltória não representa f(K).
  - *Como fazer:* brute force para n pequeno e todos os k; calcule as diferenças; verifique se são não decrescentes (convexa) ou não crescentes (côncava); se só um trecho é monótono, Aliens falha ali.
  - *Exemplo:* DP de partição com custo w(l, r); teste f(k) para n = 10 e k = 1..n.
  - *Armadilha:* testar com uma instância só; a convexidade pode valer para uns custos e falhar para outros; empates mascaram a monotonicidade — olhe as diferenças, não f.
- **Ternária em inteiros.** Platôs que não estão no ótimo quebram a ternária; use binária em f(m) vs f(m+1) se estritamente unimodal, ou golden section com 1 avaliação por passo.
  - *Gatilho:* função unimodal em domínio inteiro, avaliação cara, e você quer o mínimo/máximo.
  - *Por que funciona:* com m1 < m2, comparar f(m1) e f(m2) descarta um lado; em inteiros, a binária em f(m) vs f(m+1) acha a fronteira onde a função para de subir, com uma avaliação nova por passo.
  - *Como fazer:* `while (lo < hi) { m = (lo + hi)/2; if (f(m) < f(m+1)) lo = m + 1; else hi = m; }` devolve lo; para ternária clássica em inteiros, pare com r − l ≤ 3 e teste os candidatos; golden section reaproveita uma avaliação por passo.
  - *Exemplo:* minimizar o custo de uma posição inteira (soma de distâncias).
  - *Armadilha:* platôs fora do ótimo quebram a ternária; a binária na diferença exige f estritamente unimodal; valide m+1 ≤ n.
- **Convexidade fecha por soma e max.** Soma de convexas é convexa (Σ|x − a_i| convexa), max de convexas é convexa, composição afim também; isso justifica ternária em "minimizar a maior distância" e em "ponto ótimo de custo agregado".
  - *Gatilho:* provar convexidade para usar ternária, Aliens ou slope trick.
  - *Por que funciona:* soma de convexas é convexa, max de convexas é convexa e composição com função afim preserva convexidade; logo custos agregados e "pior caso" de convexas continuam convexos.
  - *Como fazer:* decomponha o custo em parcelas convexas (Σ|x − a_i|, distâncias, termos lineares) e no máximo delas; se as operações são soma/max/afim, o total é convexo. Em duas variáveis, confira a Hessiana PSD.
  - *Exemplo:* minimizar a maior distância a um conjunto de pontos (max de convexas); Σ|x − a_i| (mediana).
  - *Armadilha:* min de convexas não é convexa em geral; produto de convexas não é convexo; subtração ou composição não afim quebram a preservação.

- **Ternária aninhada** em duas variáveis (f conjuntamente convexa — Hessiana PSD; convexidade separada em x e em y não garante que g(x) = min_y f(x, y) seja convexa/unimodal): $O(\log^2 \cdot \text{custo})$, ou resolva um eixo analiticamente.
  - *Gatilho:* f(x, y) com duas variáveis contínuas/inteiras, avaliação cara, e você quer o mínimo.
  - *Por que funciona:* se f é conjuntamente convexa, g(x) = min_y f(x, y) é convexa; então uma ternária/binária externa sobre g e outra interna sobre y resolvem. Convexidade separada em x e em y não basta (ex.: f(x, y) = x·y).
  - *Como fazer:* para cada x avaliado, rode a ternária interna em y (ou resolva y analiticamente); custo $O(\log^2 \cdot \text{custo})$ com iterações fixas. Verifique a Hessiana PSD no domínio.
  - *Exemplo:* ponto que minimiza a maior distância a pontos, com duas coordenadas; ajuste de dois parâmetros.
  - *Armadilha:* f separadamente convexa mas não conjuntamente (Hessiana não PSD); g pode não ser unimodal; $\log^2$ de avaliações caras pode estourar o tempo.
- **Convex hull trick por binária.** Consultas arbitrárias numa envoltória de retas em ordem de inclinação → lower_bound por interseção; consultas monótonas → ponteiro; sem ordem → Li Chao.
  - *Gatilho:* consultas de mínimo/máximo de retas y = m x + b; retas adicionadas em ordem de inclinação e consultas possivelmente arbitrárias.
  - *Por que funciona:* a envoltória tem breakpoints (interseções) em ordem crescente; guardá-los permite achar a reta ótima por lower_bound em O(log n), em vez do ponteiro O(1) amortizado.
  - *Como fazer:* mantenha a envoltória com os breakpoints; para consulta x, lower_bound no primeiro breakpoint ≥ x; consultas monótonas → ponteiro que só anda; retas fora de ordem → Li Chao (O(log C)).
  - *Exemplo:* dp[i] = min_j (m_j·x_i + b_j) com x_i arbitrário.
  - *Armadilha:* inclinações iguais (mantenha só a de melhor intercepto); overflow nas interseções (produto cruzado ou `__int128`); a ordem dos breakpoints precisa ser mantida na inserção.
- **Sqrt inteiro e raiz k-ésima** → calcule com sqrtl e corrija ±1 em um while, ou binária inteira; double perde precisão acima de 2^{53}.
  - *Gatilho:* parte inteira de √n ou raiz k-ésima exata, com n até 10^{18}.
  - *Por que funciona:* sqrtl/powl dão aproximação muito próxima; corrigir com while para cima/para baixo elimina o erro de arredondamento. Alternativamente, a binária inteira é exata e O(log n).
  - *Como fazer:* `r = (long long)sqrtl((long double)n); while ((__int128)(r+1)*(r+1) <= n) r++; while ((__int128)r*r > n) r--;` para raiz quadrada; para raiz k-ésima, binária em [0, n] (ou powl + ajuste). Cheque com `__int128`.
  - *Exemplo:* verificar se n é quadrado perfeito; menor r com r^k ≥ n.
  - *Armadilha:* double perde precisão acima de 2^{53}; o ajuste precisa de `__int128`; para k par e n negativo não há raiz real.
- **Binária em ângulo / raio / tempo contínuo** quando o check é "círculo de raio r cobre tudo" ou "distância no tempo t": o check geométrico costuma ser sweep angular ou interseção de intervalos.
  - *Gatilho:* o parâmetro é um ângulo, raio ou instante, e o check pergunta se uma família de formas/alcances cobre um conjunto ou se dois móveis se encontram.
  - *Por que funciona:* a viabilidade é monótona no parâmetro (raio maior cobre mais, tempo maior aproxima); o check geométrico tem estrutura de intervalos, então é mais barato que o problema de otimização.
  - *Como fazer:* binária no parâmetro (100 iterações se real); no check, reduza a intervalos de ângulo/tempo e teste cobertura por sweep ou interseção de intervalos; O(log × (n log n)).
  - *Exemplo:* menor raio de cobertura de pontos por um círculo; instante em que dois polígonos se tocam.
  - *Armadilha:* precisão e casos de tangência; intervalos que dão a volta em 2π (quebre em dois); comparar doubles com eps no check.
- **Slope trick** (função convexa linear por partes mantida em dois heaps) para dp[i][x] = min convexa em x com transições "min de janela" e "+ |x − a_i|".
  - *Gatilho:* DP em que o estado é uma posição x e o custo é convexo por partes, com transições que somam |x − a_i|, deslocam a janela ou tomam mínimo em um intervalo.
  - *Por que funciona:* somar |x − a| desloca as inclinações em uma unidade; mínimo em janela achata a função; uma função convexa linear por partes fica determinada pelas quebras, que dois heaps (esquerdo/direito) mantêm com deslocamentos preguiçosos.
  - *Como fazer:* mantenha as quebras à esquerda e à direita; adicionar |x − a| insere a duas vezes e move a quebra central; "min de janela" desloca/clampa os heaps com offsets; leia o mínimo no topo; O(n log n).
  - *Exemplo:* dp de posicionamento com custo |x − a_i| acumulado; "Slope Trick" clássico de CF.
  - *Armadilha:* deslocamentos preguiçosos precisam entrar na comparação dos heaps; a função deixa de ser convexa se houver max ou termos não convexos; use 64 bits.

- **Busca em f contínua sem monotonicidade (Bolzano).** Se f(l) e f(r) têm sinais opostos existe raiz; binária acha uma raiz mesmo sem ser monótona.
  - *Gatilho:* você precisa de uma raiz/zero de f contínua e só sabe avaliar f, sem derivada e sem monotonicidade.
  - *Por que funciona:* pelo teorema do valor intermediário, f contínua com sinais opostos nos extremos se anula em algum ponto; a bisseção mantém a troca de sinal e converge para uma raiz.
  - *Como fazer:* escolha l, r com f(l)·f(r) < 0; repita 100 vezes: mid = (l+r)/2; se f(l)·f(mid) ≤ 0, r = mid; senão l = mid; devolva mid. O(log precisão).
  - *Exemplo:* achar o ponto de equilíbrio de uma função de custo contínua; raiz de um polinômio em intervalo conhecido.
  - *Armadilha:* f precisa ser contínua (degraus quebram o teorema); se f(l) e f(r) têm o mesmo sinal pode haver raiz par; precisão e múltiplas raízes.

DP

Pergunta-chave: o que eu preciso lembrar do passado para decidir o futuro? Isso é o estado.

Escolhendo o estado:

- **Prefixo** dp[i]; **intervalo** dp[l][r] quando juntar/remover pedaços; **subárvore** dp[v]; **máscara** dp[mask] com n ≤ 20; **dígitos** dp[pos][tight][...] para contar até 10^{18}.
  - *Gatilho:* o enunciado descreve uma sequência (prefixo), pedaços que se combinam ou são removidos (intervalo), uma árvore (subárvore), um conjunto pequeno (máscara) ou um limite numérico gigante (dígitos).
  - *Por que funciona:* o estado é a menor informação do passado que resume tudo o que o futuro pode consultar; cada tipo de estado corresponde a uma fronteira natural do problema.
  - *Como fazer:* escolha o formato pela forma do dado:
    - prefixo — dp[i] = resposta para os i primeiros; transição de i−1 ou de um j < i.
    - intervalo — dp[l][r] = resposta para o segmento; transição quebrando em k ($O(n^3)$) ou com opt monotônico.
    - subárvore — dp[v] combina os filhos; resposta em dp[raiz] após uma DFS.
    - máscara — dp[mask] com 2^n estados; n ≤ 20–22 e transição O(1) ou O(n) por máscara.
    - dígitos — dp[pos][tight][...] da esquerda para a direita; N ≤ 10^{18} tem 19 dígitos.
  - *Exemplo:* LIS por prefixo; multiplicação de matrizes por intervalo; subárvore numa árvore; TSP por máscara; "conte números ≤ N com..." por dígitos.
  - *Armadilha:* máscara com n = 25 já são 33M estados; tight mal definido no digit DP conta números a mais, e prefixo/intervalo não se substituem sem mudar a transição.
- **Troque estado e valor.** dp[i][valor grande] = possível? → dp[i][resposta pequena] = menor valor. (Mochila com peso 10^9 e valor 10^3: dp[valor] = menor peso.)
  - *Gatilho:* um dos eixos da DP é enorme (10^9) e o outro é pequeno (10^3); a pergunta é "é possível?" ou "qual o menor/máximo custo?".
  - *Por que funciona:* para cada valor v basta o menor peso capaz de atingi-lo; estados com o mesmo valor e peso maior são dominados e nunca ajudam.
  - *Como fazer:* dp[v] = menor peso para obter valor v; para cada item, percorra v de Σv até val_i e faça dp[v] = min(dp[v], dp[v − val_i] + peso_i); resposta = maior v com dp[v] ≤ W. O(n·Σv).
  - *Exemplo:* mochila com W = 10^9 e valores ≤ 10^3; "mínimo de itens para somar exatamente S" com S grande e valores pequenos.
  - *Armadilha:* inicialize dp[0] = 0 e o resto com ∞; com os dois eixos grandes a troca não resolve (o problema é NP-difícil na forma geral).
- **Modele pela última operação**, não pela primeira (você tende a usar a primeira: vale testar a outra).
  - *Gatilho:* há uma sequência de operações que consome/remove/altera elementos; a primeira operação deixa um problema irregular, a última deixa lados independentes.
  - *Por que funciona:* condicionar à última operação separa o problema em subproblemas disjuntos já resolvidos; condicionar à primeira costuma deixar um "resto" que não é um intervalo ou estado limpo.
  - *Como fazer:* enumere qual elemento/operação foi o último; escreva a recorrência sobre os pedaços que sobram; se sobrar um intervalo, é DP de intervalo $O(n^3)$ (ou $O(n^2)$ com Knuth).
  - *Exemplo:* estourar balões (o último a estourar separa esquerda e direita); remover caixas; construir BST ótima escolhendo a última raiz.
  - *Armadilha:* "último" e "primeiro" dão a mesma resposta quando a operação é comutativa, mas a complexidade e o estado mudam; teste as duas formulações antes de codar.

- **Estado com "o que ainda está aberto".** Parênteses abertos, grupos não fechados, componentes conectadas (connected components DP para permutações).
  - *Gatilho:* o objeto é construído da esquerda para a direita e o que importa é quantos/quais itens ficaram pendentes: parênteses sem fechar, blocos abertos, componentes ainda não ligadas.
  - *Por que funciona:* o que já foi fechado não interage mais com o futuro; só as pontas abertas podem receber o próximo elemento, então elas formam um resumo suficiente.
  - *Como fazer:* guarde a quantidade (e às vezes o tipo) de itens abertos; cada novo elemento abre (+1), fecha (−1) ou liga dois (−1), multiplicando pelas escolhas de ponta.
  - *Exemplo:* contar sequências balanceadas de parênteses com dp[i][abertos]; connected components DP para permutações com restrição entre vizinhos.
  - *Armadilha:* num objeto linear (não circular) as duas pontas globais são especiais; sem flags de "ponta já usada" a contagem dobra ou perde casos.
- **Conte objetos ruins com DP** quando inclusão-exclusão tem defeitos que conflitam.
  - *Gatilho:* "pelo menos um", "nenhum", "não todos"; os eventos proibidos se sobrepõem e a interseção de vários é difícil de contar.
  - *Por que funciona:* em vez de somar interseções com sinais, a DP percorre a construção e carrega no estado se o defeito já ocorreu; cada objeto é contado uma vez.
  - *Como fazer:* conte o complemento (objetos que violam) com um estado que detecta a violação e subtraia do total; se a violação for "bateu no padrão", use autômato.
  - *Exemplo:* contar strings de comprimento n que contêm pelo menos um de vários padrões → total − DP no Aho-Corasick que evita todos; contar números ≤ N com o dígito 7.
  - *Armadilha:* a DP de "ruins" precisa ser determinística; se o mesmo objeto puder ser reconhecido por dois caminhos, ela conta duas vezes e a inclusão-exclusão é a ferramenta certa.
- **Probabilidade/esperança** → linearidade; dp de trás pra frente a partir do estado final.
  - *Gatilho:* processo aleatório com estados pequenos, "valor esperado", "probabilidade de terminar em...", ou soma de contribuições de muitos eventos.
  - *Por que funciona:* E[X] = Σ P(evento) para indicadores, e a esperança condicional satisfaz E[s] = custo + Σ p·E[próximo], que se resolve de trás para frente quando o grafo de estados é acíclico.
  - *Como fazer:* defina E[estado] e escreva a equação; se o processo é um DAG, processe na ordem topológica inversa; se há ciclos, resolva o sistema linear ou elimine a variável (ver Esperança com ciclos).
  - *Exemplo:* nº esperado de inversões numa permutação aleatória = n(n−1)/4, por linearidade sobre os pares; passeio absorvente com E[v] = 1 + média dos vizinhos.
  - *Armadilha:* não multiplique esperanças de etapas dependentes; e não use DP de valor quando a pergunta é a distribuição ("probabilidade de terminar com x").

Otimizando a transição:

- dp[i] = min_j (dp[j] + b_j·a_i) → **CHT / Li Chao**.
  - *Gatilho:* a transição tem um termo cruzado entre um parâmetro do estado anterior (b_j) e um valor do estado atual (a_i), e o resto separa em "só i" e "só j".
  - *Por que funciona:* cada j define uma reta y = b_j·x + dp[j]; o mínimo em x = a_i é uma consulta ao envelope inferior dessas retas.
  - *Como fazer:* se as inclinações b_j entram monotonicamente e as consultas a_i também, use deque (O(n)); caso contrário Li Chao sobre as coordenadas (O(n log C)) ou CHT dinâmico com set.
  - *Exemplo:* dp[i] = min_j dp[j] + (S_i − S_j)², que expande para retas de inclinação −2S_j e intercepto dp[j] + S_j².
  - *Armadilha:* inclinações iguais: mantenha só a de menor intercepto; cuidado com overflow no produto b·x (use __int128 ou long double) e com a ordem de inserção/consulta.
- opt(i) monótono em dp[k][i] = min_j dp[k−1][j] + cost(j, i) → **D&C DP**; em intervalos, com quadrangle inequality e w monótono → **Knuth** (não misture com o D&C).
  - *Gatilho:* DP em camadas (k fixo por camada) cuja transição olha todos os j < i, ou DP de intervalo com custo w(l, r).
  - *Por que funciona:* se a matriz de custos é Monge (desigualdade do quadrângulo mais monotonia), o argmin opt[i] é não decrescente; isso limita a busca de cada estado.
  - *Como fazer:* D&C DP: resolve(l, r, optl, optr) calcula mid e recursa em [l, mid−1] com optl..opt(mid) e em [mid+1, r] com opt(mid)..optr; O(n log n) por camada. Knuth: opt[l][r−1] ≤ opt[l][r] ≤ opt[l+1][r], O(n^2) total.
  - *Exemplo:* particionar array em k segmentos com custo de segmento; BST ótima e merge ótimo de arquivos adjacentes (Knuth).
  - *Armadilha:* D&C exige C(j, i) em O(1) e opt monotônico na camada; Knuth é para intervalo e não se aplica a dp[k][i] em camadas; empates tratados de forma inconsistente quebram a monotonia.

- Transição sobre um intervalo de j → **prefix min / deque / segtree** sobre dp.
  - *Gatilho:* dp[i] = min/max sobre j ∈ [L(i), R(i)] de dp[j] + termo, ou atualizações de faixa antes de consultar.
  - *Por que funciona:* a transição é uma consulta de intervalo sobre valores já calculados; a estrutura escolhida responde essa consulta sem varrer o intervalo.
  - *Como fazer:* se o intervalo desliza (L e R não decrescentes), deque monotônica O(n); se os limites são arbitrários, segtree/sparse table O(n log n); se há soma de constante por faixa, lazy.
  - *Exemplo:* sapo que pula até k posições e minimiza custo → deque de mínimos em janela; dp[i] = a_i + max_{j∈[i−k,i−1]} dp[j].
  - *Armadilha:* a deque exige que o elemento saia pela esquerda e os novos entrem pela direita na mesma ordem; para máximo com negativos, não inicialize com 0.
- Transição linear com n até 10^{18} → **exponenciação de matriz** (também em semi-anel min-plus).
  - *Gatilho:* o número de passos é gigante (10^{18}) e o estado é fixo e pequeno (k ≤ ~100, muitas vezes ≤ 5); a transição é linear/constante.
  - *Por que funciona:* se o vetor de estados vira M·v a cada passo, então v_n = M^n·v_0; compor a matriz é associativo e a exponenciação por quadrados faz log n composições.
  - *Como fazer:* monte M (k×k), calcule M^n em O(k^3 log n) e aplique em v_0; no min-plus troque (+, ×) por (min, +) e use +∞ como zero aditivo.
  - *Exemplo:* Fibonacci; contar caminhos de comprimento L num grafo pequeno; recorrência de ordem k.
  - *Armadilha:* a identidade min-plus é 0 na diagonal e +∞ fora; a ordem M^n·v importa; módulo composto não impede a exponenciação (só usa soma/produto).
- Somas sobre subconjuntos/supermáscaras → **SOS DP**.
  - *Gatilho:* precisa de f[mask] = Σ g[sub] para toda submáscara (ou superset), n ≤ 20–22, e o número de pares (mask, sub) seria 3^n.
  - *Por que funciona:* a soma sobre subconjuntos é a transformada zeta do reticulado booleano; cada bit pode ser processado independentemente, um nível por bit.
  - *Como fazer:* para cada bit b, para cada mask com b ligado: f[mask] += f[mask ^ (1<<b)]; O(n·2^n). Para supersets, itere nas máscaras com b desligado ou use complemento.
  - *Exemplo:* contar pares (i, j) com a_i & a_j = 0; soma de valores por submáscara; convolução de OR/AND.
  - *Armadilha:* a versão in-place precisa do laço de bits por fora e da ordem correta; inclua a máscara vazia se a definição pedir, e module as somas.
- Mochila só de booleanos → **bitset** (÷64). Itens com soma total S → só √(2S) pesos distintos.
  - *Gatilho:* a DP é de viabilidade (true/false) com capacidade até ~10^5–10^6 e n grande; ou há muitos itens repetidos.
  - *Por que funciona:* um bitset representa W+1 booleanos em W/64 palavras, e "dp | dp << w" faz todas as transições do item em O(W/64).
  - *Como fazer:* dp |= dp << w para cada item; para itens limitados, faça binary splitting; se Σ pesos ≤ S, há no máximo √(2S) pesos distintos (1+2+…+k ≤ S) e cada grupo pode ser tratado de uma vez.
  - *Exemplo:* subset sum com W = 10^5 e n = 10^3; valores alcançáveis com um conjunto de moedas.
  - *Armadilha:* shift além de W deve ser mascarado/truncado; bitset de tamanho dinâmico custa alocação; para contar modos o bitset não serve (é booleano).
- Estados pouco alcançáveis → **map/memo** em vez de array.
  - *Gatilho:* o estado formal tem muitos parâmetros (produto enorme), mas a recursão só visita uma fração pequena; ou os índices são valores grandes.
  - *Por que funciona:* a memoização guarda apenas o que é chamado; o custo passa a ser proporcional ao número de estados alcançáveis, não ao produto das dimensões.
  - *Como fazer:* escreva a recursão, guarde em unordered_map (ou map ordenado) com a chave codificada; estime o número de estados com um contador antes de confiar.
  - *Exemplo:* digit DP com restrições estranhas; jogos com estado (posição, máscara, turno) em que só alguns são visitados.
  - *Armadilha:* unordered_map tem fator constante alto e é atacável por anti-hash; profundidade de recursão > 10^5 estoura a pilha; limpe o memo entre casos de teste.

- "Exatamente k" com custo convexo em k → **Aliens trick**.
  - *Gatilho:* é preciso exatamente k grupos/segmentos/itens, k é grande, e a DP sem a restrição é fácil; o custo ótimo é convexo em k.
  - *Por que funciona:* cobrar λ por grupo transforma a restrição em penalidade; o número ótimo de grupos cai monotonicamente com λ, então dá para binariar λ até obter k.
  - *Como fazer:* calcule dp sem limite de grupos com custo + λ por grupo e registre a quantidade usada; binárie λ; resposta = dp_λ − λ·k. O(log C) execuções da DP.
  - *Exemplo:* particionar array em exatamente k segmentos minimizando a soma dos quadrados dos custos.
  - *Armadilha:* empates no número de grupos exigem desempate consistente (min ou max de grupos); λ inteiro pode pular k — guarde o melhor candidato dos dois lados.
- DP em árvore que você precisa para toda raiz → **rerooting**.
  - *Gatilho:* "para cada vértice como raiz", "escolha a raiz que minimiza/maximiza...", e a resposta de uma raiz pode ser obtida da vizinha.
  - *Por que funciona:* uma primeira DFS calcula a contribuição "para baixo"; uma segunda propaga a contribuição "de fora" pelo pai, então cada aresta é processada O(1) vezes.
  - *Como fazer:* down[v] = combine(down[filhos]); up[c] = combine(up[v], down[irmãos de c]); resposta[v] = combine(down[v], up[v]); O(n) se combine for O(1).
  - *Exemplo:* soma das distâncias de cada vértice a todos os outros; diâmetro por vértice; contagem de caminhos.
  - *Armadilha:* para operações não invertíveis use prefix/suffix dos filhos (ver Rerooting com operação não invertível); a DFS recursiva pode estourar com n = $2 \cdot 10^5$.
- Merge de DPs de filhos com tamanho → **small-to-large** ou soma $O(n^2)$ total pelo argumento de tamanhos.
  - *Gatilho:* cada nó tem uma estrutura (mapa, conjunto, vetor de DP) e é preciso fundir com os filhos; o tamanho total pode ser grande.
  - *Por que funciona:* se você sempre funde o menor no maior, cada elemento muda de estrutura O(log n) vezes; na versão de knapsack, cada par de vértices é contado uma vez no LCA, somando $O(n^2)$.
  - *Como fazer:* escolha o filho de maior estrutura como base e insira os demais; ou limite os laços de merge por min(sz, K) (ver Árvore knapsack).
  - *Exemplo:* cores distintas na subárvore; knapsack em árvore $O(n^2)$; DSU on tree.
  - *Armadilha:* fundir por valor em vez de por tamanho volta a $O(n^2)$ no pior caso; ao trocar containers, mantenha os ponteiros/iteradores válidos.

---

adicionado em 2026-11-10 por agentes; revisado

- **Digit DP, forma geral.** Conte f(N) = #{x ≤ N com propriedade} com estado (pos, tight, started, resto da propriedade); responda [L, R] como f(R) − f(L−1). "started" evita contar zeros à esquerda quando o dígito 0 importa (ex.: dígitos distintos, soma de dígitos com 0 proibido).
  - *Gatilho:* "quantos números ≤ N satisfazem...", N até 10^{18}, e a propriedade é local (dígitos, soma, resto, padrão).
  - *Por que funciona:* o prefixo já escolhido só influencia o futuro por tight (se ainda está colado em N), started (se já saiu dos zeros à esquerda) e pelo resto da propriedade; é um resumo suficiente.
  - *Como fazer:* dp[pos][tight][started][...] da esquerda para a direita; em cada posição teste dígitos 0..(tight ? d_N : 9); tight' = tight && (d = d_N); responda f(R) − f(L−1). O(nº de dígitos · estados).
  - *Exemplo:* contar números ≤ 10^{18} com todos os dígitos distintos; soma dos dígitos ≡ 0 mod 7.
  - *Armadilha:* sem started, o zero e os zeros à esquerda são contados como dígitos; trate L = 0 (f(−1) = 0) e note que tight só cai quando o dígito escolhido é menor que o de N.
- **Digit DP com soma/produto sobre os números.** Guarde no estado a dupla (quantidade, soma dos valores) e combine: soma' = soma_filho + dígito·10^pos·qtd_filho. Serve para "soma de todos os x ≤ N com propriedade".
  - *Gatilho:* a resposta é a soma (ou a soma de quadrados) de todos os números válidos, não só a contagem.
  - *Por que funciona:* cada número da subárvore de um prefixo compartilha o dígito já fixado; o dígito d na posição pos contribui d·10^pos para cada um dos qtd_filho números.
  - *Como fazer:* a DP devolve (qtd, soma); ao escolher o dígito d, qtd' = qtd_filho e soma' = soma_filho + d·10^pos·qtd_filho; some sobre os dígitos. Para Σx^2, guarde também a soma dos quadrados e use $(x + c)^2 = x^2 + 2cx + c^2$.
  - *Exemplo:* soma de todos os números ≤ N que não contêm o dígito 4; soma dos dígitos de todos os números de [L, R].
  - *Armadilha:* pos deve ser o expoente correto (unidades = 0); se pedirem módulo, aplique em qtd e soma, mas a combinação precisa das duas grandezas módulo M.

- **Digit DP em duas ou mais cotas (a ≤ A, b ≤ B, a op b).** Um flag tight por número; para AND/XOR/OR de pares, estado (bit, tightA, tightB) e 3 flags de 2 estados cada; "a xor b ≤ K" tem terceiro flag.
  - *Gatilho:* dois números são escolhidos simultaneamente com limites independentes e uma relação entre eles (xor, and, or, soma, comparação).
  - *Por que funciona:* processando os bits do mais significativo para o menos, cada cota vira um flag "ainda igual ao limite", e a relação entre a e b é decidida bit a bit.
  - *Como fazer:* estado (bit, tightA, tightB) e, se houver cota K sobre a op b, tightC; 2 flags = 4 casos, 3 flags = 8; transicione para cada par de bits (0/0, 0/1, 1/0, 1/1). O(bits·8·4).
  - *Exemplo:* contar pares (a, b) com a ≤ A, b ≤ B e a xor b ≤ K; pares com a + b ≤ K (carry também entra no estado).
  - *Armadilha:* o tight de cada número é independente; esquecer um flag conta valores acima da cota, e a comparação de a op b ≤ K também precisa do seu próprio tight (não basta olhar o bit atual).
- **Digit DP em base b com restrição de gcd/mod grande.** Se mod é grande (10^9) não cabe no estado; troque por contagem de múltiplos por fórmula ou use meet in the middle nos dígitos (metade alta × metade baixa).
  - *Gatilho:* a propriedade é "x ≡ 0 mod m" ou "gcd(x, m) = 1" com m ~ 10^9, e o estado do resto teria 10^9 valores.
  - *Por que funciona:* ou o resto é determinado por fórmula fechada (contar múltiplos num intervalo), ou o número é dividido em duas metades cujos restos se combinam por uma equação modular.
  - *Como fazer:* para múltiplos, conte ⌊N/m⌋ diretamente; para meet in the middle, enumere os valores da metade baixa e da metade alta, guarde restos em hash e combine; alternativamente explore o período de x mod m na base b.
  - *Exemplo:* contar x ≤ N com x ≡ 0 mod 999983; contar x com gcd(x, m) = 1 para m = 2·3·5·7·11·13 (inclusão-exclusão nos primos).
  - *Armadilha:* m grande e composto: gcd não é só "resto ≠ 0"; o meet in the middle precisa de memória b^{k/2}, viável só com k pequeno.
- **Digit DP + autômato.** "Números ≤ N sem substring proibida / com padrão" → estado do Aho-Corasick/KMP no lugar do resto; mesmo molde de contar strings.
  - *Gatilho:* a propriedade é sobre a representação decimal (ou binária) conter ou evitar um conjunto de padrões.
  - *Por que funciona:* o autômato resume todo o prefixo pelo nó atual; a transição por dígito é pronta e o número de nós é Σ|padrões|.
  - *Como fazer:* construa o Aho-Corasick dos padrões, marque nós terminais (e os que levam a terminal via fail) e use dp[pos][tight][nó]; a transição segue o dígito escolhido.
  - *Exemplo:* contar números ≤ N que não contêm "13" nem "666"; contar os que contêm "007".
  - *Armadilha:* leading zeros: com started = false o zero não deve avançar no autômato; padrões sobrepostos exigem tratar o fail corretamente.
- **K-ésimo número com propriedade** → faça a digit DP de contagem e desça dígito a dígito escolhendo o menor prefixo cuja contagem acumulada passa de k (contagem + construção).
  - *Gatilho:* "qual é o k-ésimo número que satisfaz...", k até 10^{18}, e a contagem de números ≤ X é fácil por digit DP.
  - *Por que funciona:* a ordem numérica é lexicográfica nos dígitos; fixado um prefixo, todos os números com prefixo menor vêm antes, então a contagem por prefixo localiza o k-ésimo.
  - *Como fazer:* para cada posição, teste dígitos em ordem crescente; conte quantos números válidos completam aquele prefixo; se a contagem < k, subtraia e passe ao próximo dígito; senão fixe o dígito e avance. O(10·dígitos·custo de uma contagem).
  - *Exemplo:* k-ésimo número sem o dígito 7; k-ésimo palíndromo.
  - *Armadilha:* a contagem de completações deve incluir o tight do prefixo já fixado; se k for maior que o total, não existe resposta; cuidado com overflow ao acumular contagens.
- **Broken profile (plug DP simples).** Grid n × m com m ≤ ~12–15: dp[célula][máscara de m bits] processando célula a célula, em vez de linha a linha; transição O(1) por célula em vez de O(2^m) por linha. Ladrilhos de dominó/L, independentes em grade, colorir sem vizinhos iguais.
  - *Gatilho:* tabuleiro estreito (m ≤ 12–15) e largo, cobertura por peças que ocupam poucas células, ou restrições que só olham a fronteira.
  - *Por que funciona:* a máscara guarda apenas o estado da fronteira entre células processadas e não processadas; o que está atrás da fronteira não importa mais.
  - *Como fazer:* percorra as células em ordem (linha a linha); a máscara tem m bits, um por coluna, indicando se a célula da fronteira já foi ocupada; ao processar (i, j), olhe os bits j e j−1 para decidir a peça; O(n·m·2^m).
  - *Exemplo:* contar ladrilhamentos de um grid com dominós; preencher com peças em L; colorir com 3 cores sem iguais adjacentes.
  - *Armadilha:* a ordem dos bits e a passagem de uma linha para outra precisam ser consistentes; use m = min(n, m) (transponha o grid) para reduzir a máscara.

- **Plug DP com conectividade.** Para caminho/ciclo hamiltoniano em grid ou "um único ciclo": estado = emparelhamento de plugs (parênteses) na fronteira, codificado em base 3 ou 4 (0 = sem plug, 1 = abre, 2 = fecha). Estados alcançáveis são poucos (Motzkin): guarde em hash map.
  - *Gatilho:* grid pequeno e a resposta exige uma única componente conectada (ciclo hamiltoniano, caminho que cobre tudo, loop único).
  - *Por que funciona:* a fronteira só pode conter plugs emparelhados (cada componente aberta tem duas pontas); a codificação de parênteses garante que os pares não se cruzam.
  - *Como fazer:* codifique cada plug em 2 bits (base 4) ou num dígito base 3; transicione célula a célula atualizando os dois plugs da célula; ao fechar um par, verifique se é o último e se não fecha ciclo prematuro; guarde só máscaras alcançáveis em unordered_map.
  - *Exemplo:* contar ciclos hamiltonianos em grid; preencher grid com um único loop; caminho hamiltoniano.
  - *Armadilha:* base 3 não cabe em bits — use base 4 com 2 bits por plug; fechar um ciclo antes de cobrir todas as células é inválido; o número de estados Motzkin cresce, então m ≤ ~12.
- **Máscara por linha com compatibilidade.** dp[linha][máscara] com máscaras válidas pré-filtradas (sem bits vizinhos): só Fib(m) máscaras; transição vira soma sobre máscaras compatíveis (pré-calcule listas ou use SOS no complemento).
  - *Gatilho:* grid largo e baixo, ou uma linha de estados com restrição local entre colunas vizinhas; a máscara de uma linha inteira é o estado.
  - *Por que funciona:* máscaras válidas são poucas (sem bits adjacentes cresce como Fibonacci) e a compatibilidade entre linhas é uma condição local que pode ser pré-calculada.
  - *Como fazer:* gere as máscaras válidas; para cada par compatível (a & b == 0, e às vezes (a & (b<<1)) == 0), some dp[linha−1][a] em dp[linha][b]; O(linhas·V^2) com V ≈ F_{m+2}, ou O(linhas·V·m) com SOS no complemento.
  - *Exemplo:* contar formas de colorir um grid 2 × n sem células adjacentes iguais; dominós em grid com estado por linha.
  - *Armadilha:* "sem bits vizinhos" é só uma das restrições — deslocamentos diagonais exigem checar (a<<1)&b e (a>>1)&b; a contagem de máscaras é F_{m+2}, não 2^m.
- **Subconjuntos de máscara, 3^n.** dp[mask] = melhor partição de mask em grupos: itere submáscaras (3^n). Para n até 16–18 cabe; se o grupo contém sempre o menor bit de mask, corta um fator constante (fixa o menor elemento).
  - *Gatilho:* é preciso particionar um conjunto pequeno em grupos e o custo de um grupo é independente dos demais.
  - *Por que funciona:* toda partição tem um grupo que contém o menor elemento ainda não usado; enumerar esse grupo (submáscara) e recursar no complemento cobre cada partição exatamente uma vez.
  - *Como fazer:* para mask, seja b o menor bit; para cada submáscara s de mask que contém b, dp[mask] = min(dp[mask], custo(s) + dp[mask ^ s]); o laço é `for (s = mask; s; s = (s−1) & mask)`. O(3^n/2).
  - *Exemplo:* particionar n ≤ 16 pessoas em grupos com custo por grupo; colorir grafo pequeno; dividir array em grupos com n pequeno.
  - *Armadilha:* sem fixar o menor bit você conta cada partição muitas vezes e ainda paga 3^n; cuidado com a máscara vazia e com o custo de grupo pré-calculado à parte.
- **TSP / hamiltoniano contando.** dp[mask][último] em 2^n·n^2; para contar ciclos hamiltonianos com memória polinomial, IE sobre subconjuntos de vértices com contagem de passeios fechados em 2^n·n^3 — inviável em n = 25; use n ≲ 20 ou variante O(2^n·n^2).
  - *Gatilho:* visitar todos os vértices exatamente uma vez, menor caminho/ciclo, ou contar ciclos hamiltonianos.
  - *Por que funciona:* a máscara de visitados mais o último vértice resumem o caminho; qualquer caminho que termina no mesmo (mask, último) tem o mesmo futuro.
  - *Como fazer:* dp[mask][v] = melhor/contagem para terminar em v; transicione para u ∉ mask com dp[mask|1<<u][u]; resposta no ciclo fechando em 0; O(2^n·n^2). Para contar ciclos com memória polinomial, use inclusão-exclusão: (1/n)·Σ_S (−1)^{n−|S|}·W(S), onde W(S) é o nº de passeios fechados de comprimento n em G[S].
  - *Exemplo:* TSP com n = 18; contar ciclos hamiltonianos em grafo de 20 vértices.
  - *Armadilha:* a IE é 2^n·n^3 e só vale a pena com memória apertada; divida por n (e por 2 se o ciclo não for orientado); no TSP, inicialize dp[1<<0][0] e não feche o ciclo antes de visitar todos.
- **Bitmask DP sobre posição + máscara de últimos k elementos.** Quando a restrição envolve só vizinhança de comprimento k (ex.: sem k iguais seguidos, k ≤ 10–20) → a máscara é a janela, deslizante.
  - *Gatilho:* a validade de colocar um novo elemento depende apenas dos últimos k colocados, e k é pequeno.
  - *Por que funciona:* o passado além da janela não pode mais influenciar; a máscara dos últimos k é o resumo exato do contexto.
  - *Como fazer:* dp[i][mask] = nº/valor para os i primeiros com janela mask; ao adicionar x, nova mask = ((mask << 1) | x) & ((1<<k)−1); descarte transições que violem a restrição. O(n·2^k).
  - *Exemplo:* contar strings binárias sem k zeros seguidos; colorir sequência sem k iguais; sequências com soma dos últimos k limitada.
  - *Armadilha:* k = 20 já são 1M estados por posição; com alfabeto maior que 2, codifique símbolos em mais bits; não esqueça a janela inicial incompleta.

- **Permutações como máscara de "já usados" com custo por posição** (alocação, atribuição com n ≤ 20): dp[mask] = custo mínimo para colocar os |mask| primeiros itens; a posição é popcount(mask), não precisa ir no estado.
  - *Gatilho:* é preciso ordenar/atribuir n itens a n posições com custo que depende do par (item, posição), n ≤ 20.
  - *Por que funciona:* depois de escolher o conjunto de itens usados, a posição atual é exatamente |mask|; o custo de colocar o próximo item depende só dele e dessa posição.
  - *Como fazer:* dp[mask] = min over i ∉ mask de dp[mask ^ (1<<i)] + custo[i][popcount(mask)]; O(2^n·n); guarde o pai para reconstruir.
  - *Exemplo:* atribuição de tarefas com n ≤ 20; menor custo de ordenar n peças; contar permutações com custo ≤ C.
  - *Armadilha:* calcule popcount uma vez por máscara (não dentro do laço); para contar, troque min por soma; 2^20 × 8 bytes = 8 MB, e 2^24 já são 128 MB.
- **Árvore knapsack $O(n^2)$.** dp[v][k] = melhor com k vértices da subárvore; ao fundir filho c, limite os loops por min(k, sz): a soma total é $O(n^2)$ (cada par de vértices é "visto" uma vez no LCA). Vale também para "k folhas / k arestas / k componentes".
  - *Gatilho:* escolher um conjunto de k vértices/folhas/arestas numa árvore com dependência ou custo aditivo.
  - *Por que funciona:* o merge de dois blocos de tamanhos a e b custa a·b; cada par de vértices é contado uma única vez, no LCA onde os dois blocos se encontram, então a soma é $O(n^2)$.
  - *Como fazer:* dp[v][0] = base; para cada filho c, combine dp[v][i] com dp[c][j] para i+j ≤ sz; limite os laços por sz[v] já acumulado e sz[c]. $O(n^2)$.
  - *Exemplo:* escolher k vértices conectados contendo a raiz; k folhas; k arestas num subgrafo.
  - *Armadilha:* inicialize o array com −∞/∞ corretamente e não deixe "buracos"; se k é limitado por K, use a versão com corte (ver Árvore knapsack com limite K).
- **Árvore knapsack com limite K.** Cortar os índices em min(sz, K) dá O(nK) total. Truque de ordem DFS (dp sobre a ordem dfs: pegar v implica ir para tin+1, pular implica ir para tout) deixa a DP linear em n·K sem merge.
  - *Gatilho:* o mesmo problema, mas k ≤ K com K pequeno (≤ 500) e n grande; ou é preciso evitar o merge $O(nK^2)$.
  - *Por que funciona:* cortar em K reduz cada merge a no máximo K^2, e a soma amortizada é O(nK); a ordem DFS transforma a dependência em intervalos contíguos.
  - *Como fazer:* no merge, limite i ≤ min(K, sz acumulado) e j ≤ min(K, sz do filho); na ordem DFS, dp[i][j] com escolher i → i+1 e pular → tout[i]; O(nK).
  - *Exemplo:* escolher até K vértices numa árvore de $2 \cdot 10^5$ nós; orçamento K numa hierarquia.
  - *Armadilha:* o corte sem limitar os dois laços ainda pode dar $O(nK^2)$; na ordem DFS, confirme que pular a subárvore inteira é válido para a dependência (pai escolhido ou não).
- **DP sobre ordem DFS (pegar/pular subárvore).** Para "escolher conjunto fechado para cima / dependência pai→filho com orçamento": dp[i][j] sobre a posição i na ordem de Euler; escolher i desce para i+1, não escolher salta para tout[i]. Vira knapsack linear.
  - *Gatilho:* a escolha de um vértice só é permitida se o pai foi escolhido (ou o contrário), e há um orçamento de quantos escolher.
  - *Por que funciona:* na ordem de Euler, a subárvore é um intervalo [tin, tout]; pular um vértice inválido pula a subárvore inteira, então o estado não precisa saber quais ancestrais foram escolhidos.
  - *Como fazer:* dp[i][j] = melhor usando as posições i..n−1 da ordem com j escolhas; se escolher o vértice i, vá para i+1; se pular, vá para tout[i]; O(nK) (ver Árvore knapsack com limite K).
  - *Exemplo:* selecionar projetos com pré-requisitos em árvore e orçamento K; máximo de vértices com peso e dependência.
  - *Armadilha:* a dependência precisa ser "fechada para cima" (se escolhe o filho, escolhe o pai); para dependência invertida, inverta a árvore ou o sentido da ordem.
- **DP em árvore com estado "o que sobe".** O que a subárvore precisa comunicar ao pai (cor da raiz, se a raiz está coberta, comprimento da cadeia aberta). Cobertura de vértices/dominating set: 3 estados (escolhido, dominado por filho, ainda precisa de pai).
  - *Gatilho:* a resposta local depende de uma informação sobre a raiz da subárvore que o pai precisa conhecer (cobertura, cor, cadeia aberta).
  - *Por que funciona:* o pai só interage com a subárvore pela raiz; condensar a subárvore em poucos estados de fronteira é suficiente e permite combinar filhos independentemente.
  - *Como fazer:* defina os estados e combine os filhos por produto cartesiano, com transições que validam a relação pai-filho; a resposta é o melhor estado da raiz. O(n·estados^2).
  - *Exemplo:* vertex cover em árvore (3 estados); dominating set (escolhido / coberto por filho / precisa do pai); maior caminho que sobe.
  - *Armadilha:* os estados precisam ser mutuamente exclusivos e exaustivos; no dominating set, "coberto por filho" não pode ser confundido com "coberto por pai", e a raiz não tem pai.

- **Rerooting com operação não invertível.** Calcule para cada vértice prefix e suffix dos resultados dos filhos; up[c] = combine(up[v], prefix[i−1], suffix[i+1]). Com max/min/gcd é o único jeito sem divisão.
  - *Gatilho:* rerooting com uma operação que não tem inversa (max, min, gcd, OR, AND), ou com muitas contribuições de filhos.
  - *Por que funciona:* para excluir o filho c do agregado do pai sem "desfazer" a operação, basta combinar o que vem de fora com todos os irmãos, pré-computados em prefix/suffix.
  - *Como fazer:* para cada v, calcule pre[i] = combine dos filhos 1..i e suf[i] = combine dos filhos i..m; então up[c_i] = combine(up[v], pre[i−1], suf[i+1]); O(grau) por vértice, O(n) total.
  - *Exemplo:* para cada raiz, o maior valor de um caminho que desce; gcd dos valores fora da subárvore; OR de bits fora da subárvore.
  - *Armadilha:* a identidade da operação (0 para soma, −∞ para max, 0 para gcd) deve ser usada nos prefix/suffix vazios; não aplique a identidade de forma que contamine o resultado.
- **Rerooting com contribuição.** Resposta para toda raiz de Σ dist, Σ f(dist): mantenha cnt e soma por subárvore e mova a raiz da aresta (v→c) atualizando ±sz[c]; quando f não é linear, guarde os momentos de f (centroide é alternativa, não obrigação).
  - *Gatilho:* a resposta para cada raiz é uma soma de uma função da distância a todos os vértices, e a mudança de raiz entre vizinhos é simples.
  - *Por que funciona:* ao mover a raiz de v para c, os sz[c] vértices da subárvore de c ficam uma unidade mais perto e os outros n−sz[c] ficam uma unidade mais longe; para f(d) = d, isso é ans[c] = ans[v] + n − 2·sz[c].
  - *Como fazer:* calcule sz e a soma das distâncias para a raiz 0; propague pela DFS com a fórmula; para f(d) = d^2, mantenha também a soma das distâncias (segundo momento) e atualize com $(d\pm1)^2$; O(n) por momento.
  - *Exemplo:* soma das distâncias de cada vértice; soma dos quadrados das distâncias; vértice que minimiza a soma (centroide/1-mediana).
  - *Armadilha:* "momentos" só fecham a conta se f for polinomial de grau fixo; para f arbitrária é preciso contar vértices por distância (O(n^2)) ou usar outra estrutura; use long long nas somas.
- **DP em DAG de estados implícito.** Se o grafo de estados é um DAG (tempo/posição cresce), dp com memo recursiva evita ordenar; verifique profundidade da recursão e converta para iterativa se > 10^5.
  - *Gatilho:* os estados se organizam por tempo/posição e as transições só vão para estados "maiores", mas o grafo não está materializado.
  - *Por que funciona:* em um DAG, a memoização calcula cada estado uma vez e a ordem topológica é implícita na recursão; não é preciso construir arestas.
  - *Como fazer:* escreva solve(estado) com memo; se a profundidade puder passar de 10^5, ordene os estados por tempo/posição e faça um laço iterativo; O(nº de estados · transições).
  - *Exemplo:* contar caminhos num grid com obstáculos; DP em (posição, tempo) com transições para frente; jogos de soma zero em DAG.
  - *Armadilha:* se houver ciclo, a memo recursiva entra em loop (não é DAG); a pilha do sistema pode estourar bem antes de 10^5 em alguns juízes.
- **DP em intervalo clássica O(n^3).** dp[l][r] = min_k dp[l][k] + dp[k+1][r] + custo(l, r,): multiplicação de matrizes, estourar balões (escolha o ÚLTIMO a estourar, não o primeiro), triangulação de polígono; remover caixas precisa de dp[l][r][k] (tipicamente O(n^4)), não cabe em 2D.
  - *Gatilho:* a resposta para um segmento combina a resposta de dois subsegmentos mais um custo que depende do segmento inteiro.
  - *Por que funciona:* qualquer solução do segmento tem uma "última divisão" k; os dois lados são independentes e já ótimos, e o custo de juntar é conhecido.
  - *Como fazer:* para comprimentos crescentes, dp[l][r] = min_k dp[l][k] + dp[k+1][r] + custo(l, r); O(n^3). Se o custo é a soma, prefix sums dão O(1); se a remoção deixa "caixas iguais" acumuladas, adicione a dimensão k.
  - *Exemplo:* multiplicação de matrizes; balões (o último a estourar separa); triangulação de polígono; remover caixas (3D).
  - *Armadilha:* a ordem dos laços (comprimento por fora) é obrigatória; em balões, o custo do último depende dos vizinhos que restam, então use a formulação de "último a estourar" e não a de primeiro.

- **Intervalo: escolha o último elemento a ser tratado.** Quando remover no meio muda vizinhos, pense em qual elemento é removido por último no intervalo: então os dois lados ficam independentes. Balões, cortar vara, merge de pilhas.
  - *Gatilho:* operações que removem/consomem um elemento e fazem os vizinhos se tornarem adjacentes.
  - *Por que funciona:* o último elemento tratado é o único que "vê" os dois lados; antes dele, esquerda e direita evoluem sem se tocar, então são subproblemas independentes.
  - *Como fazer:* enumere o último k; o custo de tratá-lo usa os limites do intervalo (ou os vizinhos que sobraram), e os subproblemas são [l, k−1] e [k+1, r]. $O(n^3)$, ou menos com otimização.
  - *Exemplo:* burst balloons; remover caixas; cortar uma vara com custo; juntar pilhas.
  - *Armadilha:* o custo do último deve ser calculado com os elementos que ainda existem nas bordas (não com os já removidos); inicialize os intervalos vazios como 0.
- **Intervalo com string: dp[l][r] para palíndromos, edição, "strange printer".** Se s[l] = s[r] une os extremos (dp[l+1][r−1] + c); impressora estranha: dp[i][i] = 1, dp[l][r] = dp[l][r−1] + 1 e, para k com s[k] = s[r], dp[l][r] = min(dp[l][r], dp[l][k] + dp[k+1][r−1]).
  - *Gatilho:* a string é processada por intervalos e a decisão depende de os extremos serem iguais ou não.
  - *Por que funciona:* se os extremos coincidem, eles podem ser "casados" e o miolo resolve o resto; se não, um dos extremos paga um custo e reduz o intervalo.
  - *Como fazer:* dp[l][r] para comprimentos crescentes; caso s[l] = s[r], combine com dp[l+1][r−1]; sempre considere dp[l+1][r]+1 e dp[l][r−1]+1; na impressora estranha, use também os k com o mesmo caractere. $O(n^3)$.
  - *Exemplo:* menor número de inserções para virar palíndromo; edit distance; strange printer (imprimir faixas iguais).
  - *Armadilha:* o caso s[l] = s[r] não é só dp[l+1][r−1]+1 — na impressora estranha, unir com um k intermediário pode custar menos; não esqueça o caso base de um caractere.
- **Intervalo com n grande mas estrutura de pilha.** "Remover pares adjacentes casáveis" pode virar DP $O(n^2)$ se a forma canônica é única (parênteses), ou O(n) guloso com pilha. Teste se algum guloso funciona antes.
  - *Gatilho:* remoções de pares adjacentes que se casam (parênteses, letras iguais, blocos), com n grande demais para $O(n^2)$.
  - *Por que funciona:* se a redução é confluente (a forma normal é única), uma pilha simula as remoções e o resultado independe da ordem; caso contrário, é preciso DP de intervalo.
  - *Como fazer:* empilhe e desempilhe quando o topo casar com o atual; se o problema pede o número mínimo/máximo de remoções e a ordem importa, use dp[l][r] com a condição de casamento. O(n) ou $O(n^2)$.
  - *Exemplo:* remover "()" repetidamente (pilha); validar parênteses; reduzir "ab" e "ba".
  - *Armadilha:* a pilha gulosa só dá o resultado certo se a forma canônica for única; teste casos pequenos em que duas ordens dão resultados diferentes.
- **Intervalo circular.** Duplique o array (tamanho 2n) e calcule dp[l][r] com r − l < n; a resposta é o melhor dp[i][i+n−1]. Para otimização em círculos de remoção ("matar vizinhos"), fixe um elemento como ponto de corte.
  - *Gatilho:* o array é circular (o último é vizinho do primeiro) e a DP é de intervalo ou de remoção de vizinhos.
  - *Por que funciona:* duplicar transforma qualquer janela circular de n elementos numa janela linear de tamanho n; fixar um elemento como corte elimina a dependência entre as pontas.
  - *Como fazer:* copie a para a[i+n]; calcule dp[l][r] só para r−l < n; resposta = max/min sobre i de dp[i][i+n−1]; para remoções, rode a DP linear fixando cada possível sobrevivente/primeiro removido. $O(n^3)$ no caso intervalar.
  - *Exemplo:* maior soma de subarray circular; remover vizinhos num círculo com pontos; colorir um anel.
  - *Armadilha:* a janela de tamanho n pode ser contada duas vezes se você não restringir a posição inicial; em remoções, fixar o corte certo evita perder a solução ótima.
- **Knuth (intervalo).** opt[l][r−1] ≤ opt[l][r] ≤ opt[l+1][r] dá $O(n^2)$ total para dp[l][r] = min_k dp[l][k] + dp[k+1][r] + w(l, r) com w satisfazendo desigualdade do quadrângulo e monotonia. Árvore de busca binária ótima, merge ótimo de arquivos adjacentes.
  - *Gatilho:* DP de intervalo $O(n^3)$ com custo w(l, r) aditivo e a suspeita de que o ponto de corte ótimo se move monotonicamente.
  - *Por que funciona:* a desigualdade do quadrângulo e a monotonia de w implicam que opt[l][r] está entre opt[l][r−1] e opt[l+1][r]; o total de candidatos por intervalo fica O(1) amortizado.
  - *Como fazer:* calcule por comprimento crescente; para dp[l][r], itere k de opt[l][r−1] a opt[l+1][r] e guarde o argmin; $O(n^2)$ total (a soma dos intervalos de busca é $O(n^2)$).
  - *Exemplo:* BST ótima com probabilidades; merge ótimo de arquivos adjacentes (custo = soma); triangulação com custo aditivo.
  - *Armadilha:* Knuth exige w monótona e quadrangular (teste com brute force); não confunda com D&C DP, que é para dp em camadas e usa opt monotônico entre i, não a janela opt[l][r−1]..opt[l+1][r].

- **Hu-Tucker / Garsia-Wachs** quando o problema é "juntar adjacentes com custo = soma" e n grande: O(n log n), não precisa Knuth. Se pode juntar qualquer par, é Huffman (heap).
  - *Gatilho:* fundir itens adjacentes (preservando a ordem) com custo igual à soma dos pesos, n até 10^5–10^6.
  - *Por que funciona:* é o problema da árvore alfabética ótima; Garsia-Wachs/Hu-Tucker constroem a árvore em O(n log n), enquanto a DP de intervalo é $O(n^2)$ mesmo com Knuth.
  - *Como fazer:* use a fila de Garsia-Wachs (combina o par localmente ótimo e trata o peso combinado com regras de compatibilidade) ou a implementação de Hu-Tucker com duas pilhas; some os custos de cada merge.
  - *Exemplo:* codificação alfabética ótima (Huffman com ordem fixa); juntar pedaços de madeira adjacentes com custo do pedaço.
  - *Armadilha:* se a ordem não importa, use Huffman com heap (mais simples); a implementação de GW é delicada — valide contra a DP $O(n^2)$ em casos pequenos.
- **D&C DP.** dp[k][i] = min_{j<i} dp[k−1][j] + C(j, i) com opt monótono: resolva a camada recursivamente (mid → opt, divida) em O(n log n) por camada; custo C(j, i) precisa ser O(1) via prefix sums (ou Mo-like movendo ponteiros em O(n log n) total).
  - *Gatilho:* DP em camadas (k grupos/etapas) com transição sobre todos os j < i e a matriz C Monge.
  - *Por que funciona:* opt[i] é não decrescente em i; ao calcular mid, o argmin fica entre optl e optr, e as duas metades herdam intervalos menores de busca.
  - *Como fazer:* solve(l, r, optl, optr): mid = (l+r)/2; varra j de optl a min(optr, mid−1), guarde o melhor; recursa em [l, mid−1] com optl..opt(mid) e em [mid+1, r] com opt(mid)..optr. O(n log n) por camada, O(k n log n) no total.
  - *Exemplo:* dividir array em k segmentos minimizando a soma dos custos; agrupar postos com custo de distância.
  - *Armadilha:* o custo C(j, i) precisa ser O(1) (prefix sums) ou os ponteiros precisam ser amortizados; se opt não for monotônico, o resultado fica errado sem aviso.
- **Custo C(j, i) = soma de pares dentro do segmento** (ex.: nº de pares iguais, inversões no bloco) → D&C DP + ponteiros que se movem como Mo (adicionar/remover nas pontas); complexidade O(k·n log n) com ponteiros amortizados.
  - *Gatilho:* o custo de um segmento conta pares internos (iguais, inversões, contribuições aos pares), difícil de prefixar diretamente.
  - *Por que funciona:* a D&C DP fixa a ordem das consultas de C; mover os ponteiros entre uma consulta e a próxima custa proporcional à distância, e a soma das distâncias por nível é O(n).
  - *Como fazer:* mantenha L, R e a contagem atual; ajuste L/R com add/remove O(1) (frequência e total de pares); calcule C(j, i) na ordem da recursão D&C. O(k·n log n).
  - *Exemplo:* particionar array em k segmentos minimizando a soma do nº de pares iguais em cada segmento; custo = inversões do bloco.
  - *Armadilha:* a ordem de visita da D&C precisa ser respeitada para os ponteiros amortizarem; add/remove têm de ser realmente O(1) (use frequências, não estruturas log).
- **Desigualdade do quadrângulo (Monge) como teste.** C(a, c) + C(b, d) ≤ C(a, d) + C(b, c) para a ≤ b ≤ c ≤ d → opt monótono (D&C, Knuth) e função convexa em k (Aliens). Verifique com brute force em valores pequenos antes de apostar.
  - *Gatilho:* você quer usar D&C DP, Knuth ou Aliens e precisa saber se a matriz de custos tem a propriedade necessária.
  - *Por que funciona:* a desigualdade do quadrângulo (Monge) é a condição suficiente para opt monotônico; a convexidade em k vem da mesma estrutura.
  - *Como fazer:* gere os custos para n pequeno e teste todos os quadruplos a ≤ b ≤ c ≤ d; se falhar em algum, a otimização não é válida. Cuidado com o sentido da desigualdade (Monge vs anti-Monge).
  - *Exemplo:* custo soma de quadrados de segmento é Monge; custo com "penalidade por máximo" pode não ser; teste antes.
  - *Armadilha:* a desigualdade sozinha não basta para Knuth — w também precisa ser monótona; empates podem exigir escolher o menor/maior opt para preservar a monotonia.
- **SMAWK / matriz totalmente monótona.** Mínimo por linha de matriz Monge em O(n) (sem log do D&C); raro, mas é a saída quando D&C dá TLE por log extra ou em min-plus de convexa × Monge.
  - *Gatilho:* a matriz de custos é Monge/totalmente monótona e o log do D&C ainda pesa (n ~ 10^6), ou o problema pede O(n).
  - *Por que funciona:* em matriz totalmente monótona, o argmin de cada linha é não decrescente; SMAWK elimina linhas/colunas por comparação e acha todos os mínimos em O(linhas + colunas).
  - *Como fazer:* implemente as rotinas reduce/recursão de SMAWK (ou a versão "online" para DP 1D/1D); precisa de acesso O(1) a C(i, j). O(n).
  - *Exemplo:* DP 1D/1D com custo Monge e n grande; min-plus de uma função convexa com matriz Monge.
  - *Armadilha:* SMAWK exige a matriz totalmente monótona (mais forte que Monge em alguns casos); a implementação é curta mas fácil de errar nos índices — valide contra $O(n^2)$ em casos pequenos.

- **Otimização 1D/1D (dp[i] = min_{j<i} dp[j] + C(j, i)) com Monge.** Quando dp[j] depende de dp anteriores da mesma camada, D&C simples não vale: use pilha de candidatos + busca binária (Knuth online) ou CDQ + D&C em O(n $\log^2$ n).
  - *Gatilho:* a transição de dp[i] usa dp[j] da mesma camada (não uma camada anterior), então não há como dividir por camadas.
  - *Por que funciona:* com C Monge, o melhor j para i é não decrescente; mantém-se uma pilha de intervalos onde cada candidato j é ótimo, e a troca de candidato é achada por busca binária.
  - *Como fazer:* empilhe pares (j, início do intervalo em que j é ótimo); para cada i, consulte o topo e ajuste; ao inserir i, binárie o ponto onde ele supera o topo. O(n log n). Alternativa: CDQ divide, aplica contribuições da metade esquerda na direita e resolve; O(n $\log^2$ n).
  - *Exemplo:* particionar uma sequência com custo w(j, i) Monge e sem limite de segmentos; dp[i] = min_j dp[j] + |S_i − S_j| com restrições.
  - *Armadilha:* a pilha online exige C Monge e consultas em ordem crescente de i; para custos que não são Monge, nem a pilha nem o CDQ corrigem.
- **CHT clássico.** dp[i] = min_j (m_j·x_i + b_j): inclinações monótonas e consultas monótonas → deque O(n); caso contrário Li Chao (O(n log n)) ou CHT dinâmico com set. Aparece em "custo = (soma)^2 + C" após expandir (S_i − S_j)^2.
  - *Gatilho:* a recorrência tem um termo m_j·x_i e as retas são inseridas/consultadas ao longo do processamento.
  - *Por que funciona:* o mínimo de retas é o envelope inferior; com inclinações e consultas monotônicas, o ponteiro de melhor reta só anda para frente.
  - *Como fazer:* deque de retas; ao inserir, remova as que ficam acima da interseção; ao consultar x crescente, avance enquanto a próxima for melhor. O(n). Se as ordens não são monotônicas, Li Chao sobre o domínio de x.
  - *Exemplo:* dp[i] = min_j dp[j] + (S_i − S_j)^2 + C; dp[i] = min_j (a_j·i + b_j).
  - *Armadilha:* compare interseções com produto cruzado (não double); retire retas de inclinação igual mantendo a de menor intercepto; a ordem de inserção e consulta precisa ser verificada.
- **Reconhecer CHT por expansão.** Custo de segmento (S_i − S_j)^2 ou (a_i − a_j)·b_i etc.: abra o produto; termos só de i saem do min, termos só de j viram intercepto, o termo cruzado S_i·S_j vira reta de inclinação S_j avaliada em S_i.
  - *Gatilho:* o custo de transição é um produto de duas quantidades que dependem de i e de j, ou uma distância ao quadrado.
  - *Por que funciona:* expandir separa a expressão em f(i) + [g(j) + h(i)·k(j)]; o que depende só de i é constante no min, e o resto é uma reta em h(i).
  - *Como fazer:* escreva o custo, expanda, agrupe: dp[i] = f(i) + min_j (m_j·x_i + b_j) com m_j = −2S_j, b_j = dp[j] + S_j^2, x_i = S_i. Depois aplique CHT/Li Chao.
  - *Exemplo:* particionar array com custo (soma do segmento)^2; custo (a_i − a_j)·b_i com a e b conhecidos; produto de prefixos.
  - *Armadilha:* sinais e fatores 2 são os erros mais comuns; confira a expansão com um caso numérico pequeno antes de codar a CHT.
- **Li Chao em árvore / persistente.** DP em árvore com dp[v] = min_{u ancestral} (dp[u] + b_u·a_v): Li Chao com rollback (desfaz inserção ao sair do vértice) em O(n log n); ou small-to-large mesclando Li Chaos.
  - *Gatilho:* a transição considera apenas ancestrais (ou um caminho), e o estado de consulta é uma reta avaliada no vértice.
  - *Por que funciona:* na DFS, os ancestrais de v são exatamente as retas ativas na pilha; um Li Chao com rollback mantém o conjunto sem reconstruir.
  - *Como fazer:* insira a reta de v ao entrar, consulte para os filhos, desfaça ao sair (guarde as alterações); O(n log C). Alternativa: mescle as Li Chaos dos filhos com small-to-large, O(n $\log^2$ n).
  - *Exemplo:* caminho da raiz até v com custo de soma/quadrado; dp[v] = min_u dp[u] + (dist(v) − dist(u))^2.
  - *Armadilha:* o rollback deve desfazer todas as mudanças feitas na inserção (inclusive as que substituíram nós); a recursão profunda e o domínio de x precisam ser discretizados.
- **Divide e conquista CDQ para DP online.** dp[i] depende de dp[j] com j < i por uma estrutura que só aceita ordem offline: resolve(l, mid), aplique contribuição de [l, mid) em [mid, r), resolve(mid, r). Convolução online, CHT com consultas fora de ordem.
  - *Gatilho:* a DP é 1D e cada dp[i] depende de todos os j < i, mas a estrutura auxiliar (BIT, CHT, NTT) não suporta inserções e consultas intercaladas.
  - *Por que funciona:* ao resolver(l, r), os dp da metade esquerda já estão prontos; aplicá-los à metade direita antes de recursar simula a ordem online, e cada par (j, i) é considerado uma vez por nível.
  - *Como fazer:* cdq(l, r): se l == r, finalize dp[l]; senão mid, cdq(l, mid), aplique as contribuições de [l, mid) sobre [mid, r) (ordenando por j/i se preciso), cdq(mid, r). O(n $\log^2$ n) com uma estrutura log no passo de aplicação.
  - *Exemplo:* dp[i] = Σ_{j<i} dp[j]·f[i−j] (convolução online); CHT com consultas em ordem arbitrária; contar pares com BIT.
  - *Armadilha:* a aplicação cruzada precisa usar apenas dp já finalizados; ordem de recursão errada propaga valores incompletos; com NTT, cuidado com o tamanho e com o módulo.

- **Convolução online (relaxada).** dp[i] = Σ_{j<i} dp[j]·f[i−j] (f conhecida): CDQ com NTT, O(n $\log^2$ n). Contagem de partições ordenadas, sequências com blocos, "composições com pesos".
  - *Gatilho:* a recorrência é uma convolução de dp com uma sequência conhecida, mas dp[i] depende de todos os anteriores.
  - *Por que funciona:* o CDQ transforma a convolução online em várias convoluções offline; cada nível faz uma NTT da metade esquerda com f e soma na direita.
  - *Como fazer:* cdq(l, r); após resolver a esquerda, faça NTT de dp[l..mid] e de f[1..r−l], multiplique e some em dp[mid+1..r]; depois resolva a direita. O(n $\log^2$ n).
  - *Exemplo:* número de composições de n com partes de um conjunto com pesos; contar sequências cujo custo é a soma das partes; partições ordenadas.
  - *Armadilha:* o índice da convolução é i−j (não i+j); o truncamento em mid+1..r precisa descartar termos com j ≥ i; use um módulo NTT-friendly.
- **Sequência P-recursiva / equação diferencial do GF.** Se dp tem recorrência de coeficientes polinomiais, calcule n termos em O(n) mesmo sem fechado (ex.: involuções, números de Motzkin, Bell via outra recorrência); ache a recorrência por chute e ajuste (guess P-recursive) com brute force pequeno.
  - *Gatilho:* a sequência cresce de forma complicada, mas parece satisfazer uma recorrência linear com coeficientes polinomiais em n.
  - *Por que funciona:* recorrências P-recursivas têm ordem e grau fixos; cada termo é combinação de poucos anteriores com coeficientes calculáveis em O(1), então n termos custam O(n).
  - *Como fazer:* calcule termos pequenos por brute force; use um "guess" (Berlekamp–Massey não se aplica a coeficientes variáveis; use ajuste linear nos coeficientes) para achar a recorrência; depois itere. Para EGFs, derive a equação diferencial e extraia os coeficientes.
  - *Exemplo:* involuções (a_n = a_{n−1} + (n−1)a_{n−2}); Motzkin; Bell pelo triângulo de Bell (ver Armadilha).
  - *Armadilha:* Bell não é P-recursiva; para ela use a Bell triangle O(n^2) ou a fórmula com números de Stirling. Confirme a recorrência com vários termos antes de confiar.
- **Recorrência linear de ordem pequena e n enorme.** Exponencie matriz O(k^3 log n); com k ~ 100–2000 e módulo primo use Kitamasa/Bostan–Mori em O(k log k log n) ou O(k^2 log n). Se só tem a sequência, Berlekamp–Massey recupera a recorrência com 2k termos.
  - *Gatilho:* a resposta é o n-ésimo termo de uma recorrência linear de ordem k, com n até 10^{18}.
  - *Por que funciona:* a recorrência define uma transformação linear de dimensão k; potências dessa transformação dão o termo n, e o polinômio característico permite reduzir x^n módulo ele.
  - *Como fazer:* matriz O(k^3 log n) para k pequeno; Kitamasa O(k^2 log n) (ou Bostan–Mori O(k log k log n)) para k médio; sem a recorrência explícita, rode BM com 2k termos e depois Kitamasa.
  - *Exemplo:* Fibonacci com n = 10^{18}; recorrência de ordem 1000 com n = 10^{18} e módulo 10^9+7.
  - *Armadilha:* BM precisa de módulo primo e de 2k termos válidos; com k ~ 2000, O(k^2 log n) já é ~$2\cdot10^8$, prefira Bostan–Mori; cuidado com coeficientes negativos.
- **Berlekamp–Massey como detetive.** Calcule 2k termos de uma DP por brute force e rode BM mod p: se a ordem achada é pequena (≤ ~50), a sequência é linear recorrente e n grande sai por Kitamasa. Funciona para contagem em grid de largura fixa (transfer matrix).
  - *Gatilho:* você suspeita que a resposta é linearmente recorrente (transfer matrix, autômato, grid de largura fixa), mas não quer derivar a recorrência à mão.
  - *Por que funciona:* toda sequência gerada por uma matriz k×k satisfaz uma recorrência de ordem ≤ k; BM recupera a menor recorrência a partir de 2k termos.
  - *Como fazer:* implemente a DP brute force para n = 1..2k; rode BM mod p; se a ordem for pequena e estável ao aumentar o número de termos, aplique Kitamasa para o n desejado; confirme com mais termos.
  - *Exemplo:* contar caminhos/independent sets em grid de largura fixa; número de strings aceitas por um autômato; sequências de uma transfer matrix.
  - *Armadilha:* BM sobre corpo finito pode achar ordem baixa por coincidência com poucos termos; use p grande e 2k+alguns; a sequência precisa ser gerada pela mesma regra (sem casos base diferentes).

- **Contagem de caminhos de comprimento L em grafo pequeno, L enorme** → potência de matriz de adjacência; para resposta "soma sobre todos comprimentos até L" aumente a matriz com estado acumulador (n+1) ou use bloco [[A, I],[0, I]].
  - *Gatilho:* contar caminhos de exatamente/até L arestas, L até 10^{18}, grafo com n pequeno.
  - *Por que funciona:* (A^L)[i][j] conta caminhos de comprimento L; o bloco [[A, I],[0, I]] acumula as potências e dá a soma até L.
  - *Como fazer:* exponencie a matriz em $O(n^3 \log L)$; para a soma, use a matriz 2n×2n [[A, I], [0, I]] ou aumente o estado com um acumulador; para "até L" inclua a identidade no início.
  - *Exemplo:* nº de caminhos de comprimento 10^{18} entre dois vértices; soma de caminhos de todos os comprimentos ≤ L.
  - *Armadilha:* a orientação da matriz (linha/coluna) muda a ordem da multiplicação; "até L" vs "exatamente L" difere por um termo; módulo em cada soma.
- **Min-plus matrix power.** Menor custo de exatamente L arestas em grafo com n ≤ ~100 e L grande → exponenciação em semi-anel (min, +); matriz com custos 0..c pequenos pode ser compactada expandindo vértices em cadeias.
  - *Gatilho:* menor/maior custo de um caminho com exatamente L arestas, L enorme, n pequeno.
  - *Por que funciona:* no semi-anel (min, +), o produto de matrizes corresponde a concatenar caminhos e escolher o mais barato; a potência L dá o custo mínimo com L arestas.
  - *Como fazer:* substitua + por min e × por +; identidade = 0 na diagonal e +∞ fora; exponencie em $O(n^3 \log L)$. Para custos pequenos, divida cada aresta de custo c em c arestas unitárias e use matriz booleana.
  - *Exemplo:* menor custo com exatamente k arestas; maior valor com exatamente L passos; fecho transitivo com custos.
  - *Armadilha:* a identidade min-plus não é a matriz identidade usual; "exatamente L" não permite ficar parado (a menos que haja aresta de custo 0); overflow ao somar +∞.
- **Distribuição estacionária / absorvente.** Probabilidade de terminar em cada estado absorvente: sistema (I − Q)x = r por Gauss mod p ou em double; n ≤ ~500. Se o grafo é quase DAG (poucos ciclos), elimine variáveis só nos ciclos.
  - *Gatilho:* cadeia de Markov com estados absorventes e a pergunta é a probabilidade de absorção em cada um.
  - *Por que funciona:* as probabilidades de absorção satisfazem x = r + Qx, onde Q são as transições entre estados transientes e r as transições diretas para o absorvente desejado; logo (I−Q)x = r.
  - *Como fazer:* monte Q e r; resolva o sistema por Gauss $O(n^3)$ (mod p com inverso, ou double com pivô); para vários absorventes, resolva uma vez por coluna de R. $O(n^3)$ é viável até n ~ 500.
  - *Exemplo:* passeio aleatório num grid com bordas absorventes; probabilidade de terminar no estado A antes de B.
  - *Armadilha:* Q precisa ser a submatriz dos transientes (não inclua os absorventes); em double, use pivoteamento e tolerância; em mod p, o determinante pode ser 0 se p dividir I−Q.
- **Esperança com ciclos nos estados.** E[v] = 1 + Σ p·E[w] com possibilidade de voltar: se cada estado só volta para si mesmo, E[v] = (1 + Σ_{w≠v} p E[w])/(1 − p_vv). Se volta para estados anteriores numa cadeia, escreva E[i] = a_i·E[0] + b_i e feche E[0] por equação linear (variável simbólica).
  - *Gatilho:* o processo tem transições de volta (ciclos), então a DP de trás para frente não se aplica diretamente.
  - *Por que funciona:* a equação de Bellman é linear; isolar o termo de E[v] elimina auto-laços, e numa cadeia a solução é uma família afim em E[0] que se fecha no fim.
  - *Como fazer:* para auto-laço, resolva algebricamente; para cadeia, propague E[i] = a_i·E[0] + b_i da esquerda para a direita e use a condição de contorno para achar E[0]; caso geral, sistema linear ou eliminação.
  - *Exemplo:* passeio aleatório com chance de voltar; fila com chegadas e serviço; jogo com "volta ao início".
  - *Armadilha:* p_vv = 1 (estado absorvente) divide por zero; a ordem das equações importa — em grafo geral, resolva o sistema, não tente propagar.
- **Esperança por linearidade em vez de DP.** Antes de montar DP de esperança, tente somar probabilidades de indicadores (E[#inversões], E[#componentes], E[tempo] = Σ P(passo k ainda ativo)). Muitos problemas que parecem DP de estados viram fórmula O(n).
  - *Gatilho:* a resposta é o valor esperado de uma contagem (quantos pares, componentes, passos), não de um estado final.
  - *Por que funciona:* E[ΣX_i] = ΣE[X_i] sem exigir independência; cada indicador costuma ser fácil de calcular sozinho.
  - *Como fazer:* escreva a quantidade como soma de indicadores; calcule P(X_i = 1) individualmente (simetria, posição, tempo); some. Para tempo, use E[T] = Σ_{k≥0} P(T > k).
  - *Exemplo:* nº esperado de inversões numa permutação aleatória = C(n,2)/2; nº esperado de pontos fixos = 1; tempo esperado até um evento = soma de probabilidades de ainda não ter ocorrido.
  - *Armadilha:* não confunda E[X·Y] com E[X]E[Y] quando há dependência; a linearidade vale para somas, não para produtos/razões.

- **Passeio aleatório em linha/árvore.** Tempo esperado de atingir vértice em árvore: E(v→pai) = 2·sz − 1 (arestas na subárvore + 1); equações de diferença em cadeia resolvem com telescópio sem Gauss. Cover time/hitting time: use resistência elétrica (hitting em grafo: E_ij + E_ji = 2m·R_ij).
  - *Gatilho:* passeio aleatório em grafo com estrutura (linha, árvore) e pergunta de tempo de batida/cobertura.
  - *Por que funciona:* a identidade do tempo de comutação (commute time) liga tempos de batida a resistências efetivas; em árvore, a estrutura recursiva dá uma fórmula fechada para subir.
  - *Como fazer:* em árvore, calcule sz e use E(v→pai) = 2·sz[v] − 1; para ir de u a v, suba até o LCA e desça (a descida é diferente); em linha, resolva a recorrência de diferenças; para tempos de batida gerais, calcule R por Laplaciano ou por série/paralelo em árvore.
  - *Exemplo:* tempo esperado para sair de uma árvore pela raiz; tempo de batida entre dois vértices de uma linha; comutador num ciclo.
  - *Armadilha:* a fórmula 2·sz−1 conta vértices (sz = tamanho da subárvore), não arestas; a descida não é simplesmente o negativo da subida; em grafo geral, R_ij exige resolver sistemas.
- **Probabilidade de vitória com estado pequeno + soma de 1 − p.** Para esperança de "repita até sucesso" com custo por tentativa, E = custo/p_sucesso; com várias etapas independentes, soma de E por etapa (linearidade, não multiplique).
  - *Gatilho:* o processo é uma sequência de tentativas independentes até sucesso, ou etapas com probabilidades conhecidas.
  - *Por que funciona:* o número de tentativas é geométrico: E[T] = 1/p; e a esperança do tempo total é a soma das esperanças das etapas, mesmo que as etapas tenham distribuições diferentes.
  - *Como fazer:* identifique p de sucesso por tentativa e o custo; E = custo/p; se há etapas em série, some os E parciais; para "probabilidade de vitória" com estados pequenos, monte o sistema/P(ganhar) e resolva.
  - *Exemplo:* tentar uma senha com probabilidade p até acertar; coletar itens em etapas; jogo com várias rodadas independentes.
  - *Armadilha:* multiplicar esperanças só é válido para tempos independentes e quando se quer o produto dos tempos, não a soma; p = 0 significa E infinito.
- **Coupon collector generalizado.** E[passos para ver todos de n] = n·H_n; para completar o conjunto (tipos distintos) ou um subconjunto fixo: dp[j] = esperança dado j já coletados, E[j] = (n/(n−j)) + E[j+1], de trás pra frente; para k cópias de cada tipo o estado é o vetor de contagens, não só j.
  - *Gatilho:* "quantas tentativas até ver todos os tipos", "até completar a coleção", com sorteio uniforme ou probabilidades.
  - *Por que funciona:* com j tipos coletados, a chance de um novo é (n−j)/n, então o tempo até o próximo novo é geométrico de média n/(n−j); somando, vem n·H_n.
  - *Como fazer:* dp de trás para frente em j: E[n] = 0, E[j] = n/(n−j) + E[j+1]; para k cópias de cada tipo, use o vetor de contagens (estado multidimensional) ou inclusão-exclusão/DP sobre subconjuntos.
  - *Exemplo:* completar álbum de figurinhas com n figurinhas; ver todos os n resultados de um dado; coletar 2 cópias de cada.
  - *Armadilha:* com probabilidades desiguais, n/(n−j) não vale; use DP sobre subconjuntos (n ≤ 20) ou a fórmula integrada; o estado vetor de contagens explode se k e n crescem.
- **Contar sem esperança: dp[estado] como distribuição, não valor.** "Probabilidade de terminar com valor = x" → dp[i][x] com transições por probabilidade (mod p usa inverso); só vira esperança se só o valor médio interessa (reduz uma dimensão).
  - *Gatilho:* a pergunta é a probabilidade de um resultado específico, ou a distribuição do resultado, não apenas a média.
  - *Por que funciona:* a distribuição evolui linearmente: cada transição espalha a massa por vários valores; guardar a massa por (estado, valor) é suficiente.
  - *Como fazer:* dp[pos][x] += dp[anterior][y]·p; se o valor pode ser grande, comprima ou mude para "probabilidade de ≥ x"; em mod p, multiplique pelo inverso do denominador.
  - *Exemplo:* probabilidade de soma final = S num jogo de dados; distribuição do número de caras; probabilidade de acabar com saldo x.
  - *Armadilha:* a dimensão "valor" pode ser tão grande quanto n·max; se só E[X] interessa, some x·dp e descarte a coluna; em mod p, não use divisão por zero.
- **DP em autômato.** Contar/otimizar strings de comprimento L que evitam/contêm padrões: dp[pos][estado do Aho-Corasick] com L ≤ 10^3 e Σ|padrões| ≤ 10^3; para L enorme use matriz (estados) com potência.
  - *Gatilho:* o objeto é uma string de comprimento fixo e a propriedade é sobre os padrões que aparecem nela.
  - *Por que funciona:* o Aho-Corasick reconhece todos os padrões em tempo linear e o nó atual resume o sufixo relevante; a DP conta as extensões válidas.
  - *Como fazer:* construa o autômato (com fail e marcação de terminais); dp[i][nó] = nº de strings de comprimento i que terminam no nó sem violar; transicione por caractere. O(L·Σ|padrões|·alfabeto); para L grande, matriz de transição e potência.
  - *Exemplo:* contar strings de comprimento 1000 que não contêm nenhum de k padrões; contar as que contêm pelo menos um (complemento).
  - *Armadilha:* um nó pode ser terminal por causa do fail (padrão que termina num sufixo); se o alfabeto é grande, a matriz fica estados×alfabeto e a potência precisa incluir todos os símbolos.

- **DP no autômato do KMP (um padrão).** dp[i][j] = nº de strings de comprimento i com borda j; ou "mínimo de trocas para não conter p": construa tabela de transições aut[j][c] em O(|p|·26) e use dp[i][j].
  - *Gatilho:* há um único padrão p e a propriedade é evitá-lo/contê-lo; o estado natural é o maior prefixo de p que é sufixo da string atual.
  - *Por que funciona:* o KMP mantém exatamente essa borda; o autômato dá a próxima borda para cada caractere em O(1).
  - *Como fazer:* pré-calcule aut[j][c] (usando o fail); dp[i][j] transiciona para aut[j][c]; se o destino é o estado de aceitação, descarte ou marque. O(L·|p|).
  - *Exemplo:* contar strings binárias de comprimento n sem "101"; mínimo de caracteres a trocar para não conter p (DP com custo).
  - *Armadilha:* o estado de aceitação (j = |p|) não deve ser usado como estado normal; ao calcular a borda após match, volte pelo fail; com custo de troca, o caractere original entra no custo.
- **LCS entre uma string e conjunto de padrões / subsequência com Aho** → estado duplo (posição, nó do autômato) quando padrões são pequenos; Σ pequeno do alfabeto mantém a tabela manejável.
  - *Gatilho:* a sequência procurada precisa casar com um entre vários padrões (ou evitá-los) e o comprimento é grande.
  - *Por que funciona:* o nó do autômato resume qual prefixo de qual padrão está ativo; junto com a posição na string, é suficiente para decidir a próxima extensão.
  - *Como fazer:* dp[i][nó] = melhor comprimento/contagem; ao ler o caractere s[i], avance no autômato e atualize; o número de estados é |s|·Σ|padrões|. Se a pergunta é LCS com um conjunto, adapte a transição de match.
  - *Exemplo:* maior subsequência de s que é prefixo de algum padrão; contar subsequências que formam um dos padrões; LCS com dicionário pequeno.
  - *Armadilha:* o autômato precisa marcar padrões que são sufixos de outros (fail); a dimensão |s|·estados pode ser grande — estime antes.
- **Contar ocorrências de t como subsequência de s** (ou número de pares (i < j) de padrão fixo) → para cada caractere de s, percorra j de |t| até 1 e faça dp[j] += dp[j−1] se s_i = t_j (j decrescente evita reusar o mesmo caractere); O(|s|·|t|) com 1 array.
  - *Gatilho:* contar quantas vezes um padrão aparece como subsequência (não necessariamente contígua), ou pares ordenados com uma relação.
  - *Por que funciona:* dp[j] conta as formas de casar o prefixo t[1..j]; ao encontrar s_i = t_j, cada casamento de t[1..j−1] ganha uma extensão; percorrer j decrescente impede usar s_i duas vezes.
  - *Como fazer:* dp[0] = 1; para cada caractere de s, para j de |t| até 1: se s_i = t_j, dp[j] += dp[j−1]; resposta dp[|t|]. O(|s|·|t|).
  - *Exemplo:* nº de subsequências iguais a "abc" em s; contar pares (i < j) com a_i < a_j (versão de 2 estados).
  - *Armadilha:* j crescente reutiliza o mesmo caractere e conta a mais; posições diferentes com o mesmo caractere contam como ocorrências distintas (é o esperado); módulo se pedido.
- **Menor supersequência comum / LCS reconstruída** → tabela dp[i][j] completa (Hirschberg se memória O(min) com reconstrução).
  - *Gatilho:* é preciso reconstruir a LCS (não só o comprimento) ou montar a menor string que contém as duas como subsequência.
  - *Por que funciona:* a LCS dá o máximo de caracteres que podem ser compartilhados; a menor supersequência tem comprimento |s| + |t| − LCS.
  - *Como fazer:* preencha dp[i][j]; reconstrua de trás para frente seguindo match (diagonal) ou o maior vizinho; para supersequência, emita o caractere compartilhado uma vez e os não compartilhados em ordem. Hirschberg divide e conquista para O(min) de memória.
  - *Exemplo:* imprimir a LCS; menor string que contém s e t; diff entre arquivos.
  - *Armadilha:* empates na reconstrução geram LCS diferentes (todas válidas, mas escolha consistente); a supersequência não é única; Hirschberg precisa de cuidado na reconstrução.
- **LCS com permutações → LIS.** Se as duas sequências são permutações (ou cada valor aparece uma vez), LCS = LIS da sequência de posições, O(n log n). Valores repetidos: mapeie ocorrências em ordem decrescente de posição para manter a corretude.
  - *Gatilho:* as duas sequências são permutações de 1..n (ou têm valores únicos), e se quer a LCS.
  - *Por que funciona:* uma subsequência comum corresponde a uma subsequência de posições crescentes na segunda sequência; maximizar o comprimento é LIS.
  - *Como fazer:* mapeie cada valor de a para sua posição em b; calcule LIS estrita dessa sequência (lower_bound) em O(n log n); com repetições, empilhe as posições em ordem decrescente para não casar duas iguais fora de ordem.
  - *Exemplo:* LCS de duas permutações; similaridade de rankings; LCS quando um dos arrays é 1..n.
  - *Armadilha:* usar LIS não-estrita com valores repetidos conta duas cópias do mesmo valor; a ordem decrescente no mapeamento é o que evita isso.

- **LCS bitset.** Para |s|, |t| até ~10^5 com alfabeto limitado: dp por linha em bitset, atualização x = row | match; row = x & (x ^ (x − ((row << 1) | 1))); O(n^2/64). Também edit distance bitset (Myers) para distância pequena.
  - *Gatilho:* as duas strings são grandes (10^4–10^5) e a LCS exata O(n^2) não passa, mas O(n^2/64) passa.
  - *Por que funciona:* para cada caractere de t, o bitset de matches permite atualizar a linha inteira da DP com operações de palavra (shift, subtração, and) em vez de célula a célula.
  - *Como fazer:* pré-calcule S[c] com os bits das posições de c em t; para cada caractere de s: x = row | S[c]; row = x & (x ^ (x − ((row << 1) | 1))); resposta = popcount(row). O(|s|·|t|/64).
  - *Exemplo:* LCS de strings de 10^5 com alfabeto pequeno; edit distance com k pequeno via Myers.
  - *Armadilha:* a fórmula é sobre bits e a subtração pode pedir emprestado além do tamanho — use bitset com bits suficientes; o resultado é o popcount, não o valor.
- **LCS/edit distance com distância ≤ k pequena.** Só a faixa diagonal |i − j| ≤ k importa → O(nk); ou Landau–Vishkin com LCP via hash em O(n + k^2).
  - *Gatilho:* as strings são parecidas (distância de edição ≤ k) e k é pequeno, mas n é grande.
  - *Por que funciona:* se a distância é ≤ k, nenhum alinhamento ótimo se afasta mais de k da diagonal; e o algoritmo de Landau–Vishkin salta blocos iguais com LCP.
  - *Como fazer:* DP na banda |i−j| ≤ k, O(nk); ou LV: para cada número de edições e, cada diagonal d ∈ [−e, e], calcule o ponto mais avançado via LCP e transicione com as três operações. Com LCP O(1), O(n + k^2); com hash + busca binária, O(n + k^2 log n).
  - *Exemplo:* verificar se a distância de edição entre duas strings de 10^5 é ≤ 5; busca aproximada.
  - *Armadilha:* a banda precisa incluir a diagonal correta no início/fim; k ≥ n degenera para O(n^2); a complexidade do LV depende de o LCP ser O(1) — com hash é O(log n) por consulta.
- **LIS com contagem.** Para contar LIS use BIT sobre valores guardando (comprimento, quantidade) com merge por max e soma nos empates; ou segtree com o mesmo par.
  - *Gatilho:* a pergunta é quantas LIS existem (ou quantas de comprimento máximo), não só o comprimento.
  - *Por que funciona:* o par (melhor comprimento, nº de formas) é um monóide: ao combinar dois candidatos, fica o maior comprimento e, em empate, somam-se as contagens.
  - *Como fazer:* processe da esquerda para a direita; consulte o BIT no prefixo a_i−1 (LIS estrita), obtenha (len, cnt), atualize o ponto a_i com (len+1, cnt); no fim, agregue o máximo global. O(n log V).
  - *Exemplo:* contar LIS de um array; contar subsequências crescentes de comprimento máximo; versão com módulo.
  - *Armadilha:* valores repetidos exigem consultar a_i−1 (estrito) e não a_i; conte cada LIS uma vez (a agregação no final, não durante); use módulo e cuidado com contagens grandes.
- **LIS em O(n log n) com reconstrução.** Guarde para cada posição o índice anterior e o índice da menor cauda de cada comprimento. "Mínimo de remoções para ficar ordenado" = n − LIS (não-decrescente: use upper_bound).
  - *Gatilho:* é preciso não só o comprimento da LIS, mas a sequência ou o número de remoções.
  - *Por que funciona:* tails[len] guarda o menor valor final possível de uma subsequência de comprimento len; substituir por um valor menor nunca piora as extensões futuras.
  - *Como fazer:* para cada a_i, ache a posição p = lower_bound(tails, a_i); guarde prev[i] = tails_idx[p−1]; atualize tails[p] = a_i; reconstrua a partir do índice da cauda final. O(n log n).
  - *Exemplo:* imprimir uma LIS; mínimo de elementos a remover para ordenar; LIS não-decrescente com upper_bound.
  - *Armadilha:* lower_bound dá LIS estrita; upper_bound dá não-decrescente; guarde os índices (não os valores) para reconstruir; tails não é a LIS em si.
- **Cobertura mínima por cadeias (Dilworth).** Mínimo de subsequências crescentes disjuntas cobrindo tudo = tamanho da maior decrescente (não-estrita); mesmo em O(n log n) com patience sorting. Aparece como "quantas pilhas/trilhos".
  - *Gatilho:* é preciso dividir a sequência em k subsequências monotônicas (crescentes) e minimizar k, ou o problema pergunta "quantos trilhos/pilhas".
  - *Por que funciona:* pelo teorema de Dilworth, a cobertura mínima por cadeias é o tamanho da maior anticadeia; aqui a anticadeia é a maior subsequência não-crescente.
  - *Como fazer:* calcule a maior subsequência não-crescente em O(n log n) (ou rode patience sorting e conte as pilhas); o número de pilhas é a resposta.
  - *Exemplo:* mínimo de subsequências crescentes para cobrir um array; número de pilhas em patience sorting; largura de uma poset 2D.
  - *Armadilha:* o tipo de monotonicidade (estrito vs não-estrito) muda a anticadeia; escolha lower/upper_bound de acordo; Dilworth vale para ordens parciais — confirme que a relação é de ordem.

- **LIS em k trocas / com updates.** Valores pequenos (≤ 1000) → dp[i][último] com BIT; "LIS com alterações pontuais" é difícil; pense em offline com CDQ (LIS 2D) em vez de segtree dinâmica.
  - *Gatilho:* a LIS precisa ser recalculada após alterações, ou há um limite de k mudanças/versões.
  - *Por que funciona:* com valores pequenos, o estado "último valor" cabe e o BIT acelera; para updates, o problema é 3D/offline e o CDQ processa por tempo.
  - *Como fazer:* para valores ≤ V, dp[i][v] com BIT de máximo; para updates, trate cada consulta como (tempo, posição, valor) e use CDQ 3D ou segtree de segment trees; para k pequeno, adicione a dimensão de trocas.
  - *Exemplo:* LIS com até k alterações de valor; LIS após remoções; consultas de LIS em subarrays.
  - *Armadilha:* a LIS com um update já é complicada — não tente adaptar a segtree de LIS padrão sem pensar na ordem; valide a modelagem 3D antes.
- **Maior subsequência com soma de pesos (LIS ponderada).** dp[i] = w_i + max_{j<i, a_j<a_i} dp[j] → BIT de max por valor; mesma ideia para "subsequência com |a_i − a_j| ≤ d" (segtree de max sobre valores no intervalo [a_i − d, a_i + d]).
  - *Gatilho:* cada elemento tem peso e a subsequência precisa ser crescente (ou com diferença limitada), maximizando a soma dos pesos.
  - *Por que funciona:* a transição é um máximo sobre um intervalo de valores; uma estrutura de max por valor responde em O(log V).
  - *Como fazer:* processe em ordem; consulte o BIT/segtree no intervalo permitido de valores, some w_i e atualize no valor a_i. O(n log V).
  - *Exemplo:* LIS com pesos; subsequência com diferença ≤ d entre consecutivos; maior soma de subsequência crescente.
  - *Armadilha:* atualize depois de consultar (para não usar o próprio i); para |a_i−a_j| ≤ d, o intervalo inclui os dois lados; comprima valores se V é grande.
- **Subsequência com restrição de vizinhos (|a_i − a_j| ≤ d, j anterior)** → dp sobre valores com segtree de max: dp[a_i] = max_{[a_i−d, a_i+d]} + 1. O(n log V).
  - *Gatilho:* a condição entre elementos consecutivos da subsequência é uma diferença limitada, não uma ordem estrita.
  - *Por que funciona:* o estado é apenas o último valor; o melhor anterior é um máximo de intervalo sobre os valores já vistos.
  - *Como fazer:* segtree de máximo indexada pelo valor; para cada a_i, consulte [a_i−d, a_i+d], some 1 e atualize a posição a_i; processe da esquerda para a direita (mesma máquina da LIS ponderada). O(n log V).
  - *Exemplo:* maior subsequência com saltos de até d; sequência de temperaturas com variação limitada.
  - *Armadilha:* j tem de ser anterior (processe em ordem); o intervalo é fechado dos dois lados; se a_i pode repetir, o update no mesmo ponto deve usar o maior valor.
- **Mochila 0/1 O(nW) com 1 array.** Itere W de cima para baixo para 0/1; de baixo para cima para ilimitada. Quase todo bug de mochila é a ordem.
  - *Gatilho:* mochila clássica com capacidade W e itens usáveis uma vez (0/1) ou ilimitadas.
  - *Por que funciona:* com um array 1D, percorrer W de cima para baixo garante que dp[x−w] ainda seja da camada anterior (item usado no máximo uma vez); de baixo para cima, dp[x−w] já é da camada atual (reuso).
  - *Como fazer:* dp[0] = 0, resto −∞ (ou 0 para "até W"); para cada item, para x de W até w (0/1) ou de w até W (ilimitada): dp[x] = max(dp[x], dp[x−w] + v). O(nW).
  - *Exemplo:* mochila 0/1; troco com moedas ilimitadas; contar modos (com a ordem dos laços correta — ver Moedas: ordem importa ou não?).
  - *Armadilha:* a ordem errada transforma 0/1 em ilimitada (ou vice-versa) silenciosamente; inicialize com −∞ se o problema pede "exatamente W", e 0 se pede "até W".
- **Mochila limitada por binary splitting.** Quantidade c_i → itens de 1, 2, 4, ..., resto: O(nW log c). Com Σ c_i·w_i ≤ S ainda dá √S pesos distintos e cada um pode ser dividido ao mesmo tempo.
  - *Gatilho:* cada item tem quantidade limitada c_i e a mochila é 0/1 (não dá para tratar como ilimitada).
  - *Por que funciona:* qualquer quantidade 0..c pode ser escrita como soma de potências de 2 mais o resto; dividir em itens 0/1 preserva todas as quantidades possíveis.
  - *Como fazer:* decomponha c em 1, 2, 4, ..., c − (2^k − 1); crie um item 0/1 para cada parte com peso e valor multiplicados; rode a mochila 0/1. O(W·Σlog c).
  - *Exemplo:* mochila com 10^5 itens de quantidade limitada; moedas com limite de cada denominação.
  - *Armadilha:* o resto não pode repetir uma potência (ex.: c = 6 → 1, 2, 3); c = 0 não gera item; a decomposição não serve para "exatamente c" sem cuidado.

- **Mochila limitada com fila monótona.** Para cada resíduo r mod w_i, dp'[r + t·w] = max_{t−c ≤ s ≤ t} dp[r + s·w] + (t − s)·v: deque sobre dp[s] − s·v; O(nW) sem log.
  - *Gatilho:* mochila limitada com W e n grandes, onde binary splitting O(nW log c) ainda pesa.
  - *Por que funciona:* para cada resíduo, a transição vira um máximo de janela deslizante de tamanho c sobre a sequência indexada por t; a deque mantém o máximo em O(1) amortizado.
  - *Como fazer:* para cada item, para cada r em 0..w−1, percorra t = 0,1,2,...; mantenha a deque de índices s com valor dp[r+s·w] − s·v; descarte s < t−c; dp'[r+t·w] = v·t + máximo da deque. O(W) por item.
  - *Exemplo:* mochila com quantidades limitadas e W = 10^5; troco com limite de moedas.
  - *Armadilha:* o índice da deque é s (não t) e o valor guardado é dp[s] − s·v; use −∞ para estados inalcançáveis e cuidado com o sinal de v.
- **Mochila limitada só de viabilidade (subset sum com multiplicidade).** Para cada peso w com cnt c: dp'[x] = true se dp[x] ou dp'[x − w] com usos < c; guarde used[x] mínimo usado do peso atual. O(nW) simples sem deque.
  - *Gatilho:* só interessa se a soma é possível (booleano), com quantidades limitadas de cada peso.
  - *Por que funciona:* percorrendo x crescente, used[x−w] diz quantas cópias do peso atual já foram usadas para chegar a x−w; se < c, pode-se usar mais uma.
  - *Como fazer:* para cada peso w com limite c: used[0..W] = 0; para x de w até W: se !dp[x] e dp[x−w] e used[x−w] < c, então dp[x] = true, used[x] = used[x−w] + 1. O(nW).
  - *Exemplo:* subset sum com várias cópias de cada moeda; alcançar um peso com estoque limitado.
  - *Armadilha:* used precisa ser reiniciado por peso; usar dp[x−w] da camada anterior em vez de dp' conta a mais; sem a checagem de usos, vira ilimitada.
- **Subset sum com bitset.** dp |= dp << w; O(nW/64). Com Σ pesos ≤ S e pesos distintos ≤ √(2S): agrupe e use binary splitting + bitset (S√S log S/64).
  - *Gatilho:* subset sum booleano com W até ~10^5–10^6 e n grande, ou muitos itens repetidos.
  - *Por que funciona:* o deslocamento do bitset aplica o item a todos os estados de uma vez; e com poucos pesos distintos, agrupar reduz o número de operações.
  - *Como fazer:* bitset dp com dp[0] = 1; para cada item, dp |= dp << w; para quantidades, binary splitting; se Σ pesos ≤ S, há ≤ √(2S) pesos distintos e cada grupo pode ser processado em bloco. O(nW/64).
  - *Exemplo:* alcançar soma S com n = 10^5 itens; partição de um conjunto em duas somas iguais.
  - *Armadilha:* truncar o bitset em W evita shifts gigantes; para contar modos, bitset não serve; com itens de peso 0, o shift não muda nada (trate à parte).
- **Subset sum com pesos pequenos e soma grande (Pisinger / balanced).** Soma de n itens com pesos ≤ C: existe solução com complexidade O(nC) independente de W (algoritmo de Pisinger: comece de uma solução gulosa e vá e volta no máximo ±C).
  - *Gatilho:* os pesos são pequenos (≤ C ~ 10^3) mas a soma total/W é enorme (10^9), e a DP O(nW) não cabe.
  - *Por que funciona:* o algoritmo balanceado mantém a diferença para uma solução gulosa limitada por C; os estados são as diferenças pequenas, não a soma.
  - *Como fazer:* ordene/processe os itens, mantenha uma solução gulosa e uma tabela de diferenças em [−C, C]; atualize ao adicionar cada item; no fim, recupere a solução. O(nC) de tempo.
  - *Exemplo:* subset sum com n = 10^3, pesos ≤ 10^3 e W = 10^9; knapsack com pesos pequenos e capacidade gigante.
  - *Armadilha:* a implementação é não trivial (é fácil perder a reconstrução); valide contra a DP O(nW) em casos pequenos; o limite é o maior peso, não a capacidade.
- **Subset sum modular / pigeonhole.** n itens com n ≥ m: algum subconjunto soma 0 mod m (prefixos); existência garantida → o problema é construir, não decidir (Erdős–Ginzburg–Ziv: entre 2m − 1 inteiros existem m com soma ≡ 0 mod m).
  - *Gatilho:* o enunciado pede um subconjunto com soma ≡ 0 mod m e há itens suficientes; ou a existência é garantida por contagem.
  - *Por que funciona:* os prefixos P_0..P_n tomam valores mod m; se n ≥ m, dois coincidem e a diferença é um subconjunto com soma ≡ 0; o EGZ garante m elementos especificamente.
  - *Como fazer:* calcule prefixos mod m até achar repetição; o subconjunto é o intervalo entre os dois; para EGZ, há algoritmos construtivos (ou força bruta para m pequeno). O(n) para o caso dos prefixos.
  - *Exemplo:* achar subconjunto com soma divisível por m; EGZ com m pequeno; "sempre existe" em problemas de construção.
  - *Armadilha:* o subconjunto dos prefixos é contíguo na ordem dada (o que basta para subconjunto); se o problema exige exatamente m elementos, o argumento dos prefixos não serve — é EGZ, bem mais difícil.

- **Mochila com valor pequeno e peso gigante.** dp[v] = menor peso para obter valor v; se Σ v ≤ 10^5 roda em n·Σv mesmo com W = 10^9. Já está na seção base, mas vale também para "contar # de itens" como valor.
  - *Gatilho:* os valores são pequenos (Σv ≤ 10^5) e os pesos/capacidade são grandes; a pergunta é maximizar valor.
  - *Por que funciona:* para cada valor v, só o menor peso importa; estados com o mesmo valor e peso maior são dominados.
  - *Como fazer:* dp[0] = 0, resto ∞; para cada item, para v de Σv até val_i: dp[v] = min(dp[v], dp[v−val_i] + peso_i); resposta = maior v com dp[v] ≤ W. O(n·Σv).
  - *Exemplo:* mochila com W = 10^9 e valores ≤ 10^3; "máximo de itens" tratando cada item como valor 1.
  - *Armadilha:* não confunda com a versão de contagem; inicialize ∞ e use long long para pesos; se Σv ainda é grande, o problema pode exigir outra técnica.
- **Mochila onde itens têm peso 1, 2, 3 só (ou poucos tipos).** Ordene cada tipo por valor decrescente, prefix sums por tipo, enumere quantidades de dois tipos e ache o terceiro por busca (função côncava em quantidade) → O(W^2 log W): enumerar dois tipos já é Θ(W^2).
  - *Gatilho:* poucos tipos de item (pesos 1, 2, 3, ...) e capacidade W grande; os valores dentro de cada tipo são arbitrários.
  - *Por que funciona:* dentro de um tipo, pegar os q itens de maior valor é ótimo; a soma é uma função côncava de q, então o melhor q do terceiro tipo é achado por busca ternária/binária.
  - *Como fazer:* ordene cada tipo por valor decrescente e faça prefix sums; para q1 = 0..W/w1 e q2 = 0..W/w2, o espaço restante determina q3 ≤ (W − q1w1 − q2w2)/w3; a função valor(q3) é côncava — ache o pico por busca. O(W^2 log W).
  - *Exemplo:* mochila com moedas de 1, 2 e 3; comprar itens de três categorias com orçamento.
  - *Armadilha:* é preciso garantir a ordem decrescente dentro do tipo; a busca do terceiro só vale se a função for côncava (diferenças não crescentes); a enumeração de dois tipos já é O(W^2) — não use com W = 10^5.
- **Valores côncavos por grupo → merge guloso.** Mochila onde cada grupo tem ganho marginal decrescente (função côncava de qtd) → heap/ordenar todos os incrementos marginais; custo unitário igual vira guloso.
  - *Gatilho:* há grupos de itens e, dentro de cada grupo, o k-ésimo item acrescenta cada vez menos valor.
  - *Por que funciona:* a soma de funções côncavas é côncava; o ótimo com orçamento é pegar os maiores incrementos marginais, que já estão em ordem decrescente dentro de cada grupo.
  - *Como fazer:* gere todos os incrementos marginais de todos os grupos; ordene (ou use heap k-way) e pegue os melhores até o orçamento. O(total de itens log).
  - *Exemplo:* comprar até W unidades com desconto progressivo por grupo; investir em projetos com retornos decrescentes.
  - *Armadilha:* se os custos unitários diferem entre grupos, ordenar por valor absoluto não é ótimo — precisa de razão valor/custo ou de outra técnica; a concavidade tem de ser discreta (diferenças não crescentes).
- **Mochila min-plus convexa.** Mesclar duas funções convexas (min-plus convolução) = mesclar suas diferenças ordenadas em O(n); arrays convexos em cada filho de árvore (ex.: knapsack com itens de custo crescente) somam por merge de listas de inclinações.
  - *Gatilho:* as DPs parciais são funções convexas (custos marginais crescentes) e é preciso combiná-las.
  - *Por que funciona:* a min-plus convolução de convexas tem como diferenças a união ordenada das diferenças das duas; portanto o merge é linear, não quadrático.
  - *Como fazer:* represente cada função pelas suas diferenças (inclinações) em ordem; para combinar, faça merge das duas listas ordenadas; O(n+m). Na árvore, combine os filhos assim em vez do produto cartesiano.
  - *Exemplo:* knapsack em árvore com custos marginais crescentes; alocar recursos entre projetos convexos; merge de listas de custos.
  - *Armadilha:* a representação só vale se as diferenças forem não decrescentes; se uma função não é convexa, o merge linear não dá o resultado; cuidado com o valor inicial em 0.
- **Mochila de "mochila de √" (pesos distintos).** Σ pesos ≤ S → ≤ √(2S) pesos distintos; combinado com cnt_i·w_i ≤ S dá total O(S√S) em vez de nS. Também: itens com w ≤ √ resolvidos por DP e itens com w > √ são ≤ √ no total usados.
  - *Gatilho:* há muitos itens, mas a soma dos pesos é limitada por S (ou a capacidade é S) e os pesos são inteiros positivos.
  - *Por que funciona:* pesos distintos 1, 2, ..., k somam k(k+1)/2, então k ≤ √(2S); e itens de peso > √S não podem ser usados mais que √S vezes.
  - *Como fazer:* agrupe por peso e trate cada grupo com binary splitting (ou conte quantas cópias cabem); para os grandes, itere sobre a quantidade usada (≤ √S). O(S√S).
  - *Exemplo:* contar subconjuntos com soma ≤ S quando Σ pesos ≤ S; mochila com muitos itens pequenos.
  - *Armadilha:* o argumento exige pesos positivos; se houver peso 0, trate separado; √(2S) é o número de pesos distintos, não de itens.

- **Mochila com contagem (número de modos)** → dp[x] += dp[x − w]; para "tamanho da lista com restrição de ordem de escolha" cuidado: ordem importa → itere W por fora (composições) vs itens por fora (combinações).
  - *Gatilho:* a pergunta é "de quantas maneiras" atingir a soma, não o menor/máximo custo.
  - *Por que funciona:* a soma de contagens é linear; a ordem dos laços decide se duas ordens do mesmo conjunto contam uma ou duas vezes.
  - *Como fazer:* dp[0] = 1; itens por fora e x crescente → combinações (cada item pode repetir); x por fora e itens por dentro → composições (ordem importa); 0/1 → x decrescente. O(nW).
  - *Exemplo:* número de combinações de moedas para um troco; número de composições de n com partes de um conjunto.
  - *Armadilha:* o mesmo problema com "ordem importa" e "não importa" tem respostas diferentes; escolha os laços pelo enunciado, não por hábito; módulo em cada soma.
- **Moedas: ordem importa ou não?** Combinações (multiconjuntos) → itens por fora, valor por dentro. Sequências ordenadas (composições) → valor por fora, itens por dentro. A troca de laços é o erro mais comum e muda a resposta.
  - *Gatilho:* problema de troco/partição com moedas e a pergunta é o número de formas.
  - *Por que funciona:* com itens por fora, cada moeda é considerada uma vez e a ordem de escolha não é registrada; com valor por fora, cada posição escolhe uma moeda e a ordem é registrada.
  - *Como fazer:* decida pelo enunciado: "de quantas maneiras" com moedas indistinguíveis → itens por fora; "sequências de moedas" → valor por fora; 0/1 → valor decrescente. O(nW).
  - *Exemplo:* troco com moedas (combinações) vs número de sequências de lançamentos que somam n (composições).
  - *Armadilha:* trocar os laços é o bug clássico; para moedas ilimitadas, valor crescente dentro do laço de item; para 0/1, decrescente.
- **DP em partições de n (√n).** Partição de n em partes distintas tem ≤ √(2n) partes; partição com partes ≤ k: dp[i][j] = dp[i][j−1] + dp[i−j][j]; O(n√n) via "partes ≤ √n (mochila) + número de partes ≤ √n (conjugada)": p(n) = DP de partes pequenas + DP de poucas partes grandes.
  - *Gatilho:* contar partições (não composições) de n, com restrições no tamanho/número de partes, n até 10^5.
  - *Por que funciona:* partes distintas limitam o número de partes; a recorrência dp[i][j] separa "não usa j" de "usa pelo menos um j". Para n grande, corte em B = √n: partes ≤ B por mochila, partes > B têm no máximo n/B < B partes.
  - *Como fazer:* dp[i][j] para partes ≤ j, O(n·B); para as partes > B, subtraia B de cada parte (cada uma fica ≥ 1) e conte partições em exatamente k partes pela recorrência q(i,k) = q(i−1,k−1) + q(i−k,k); some os dois casos. O(n√n).
  - *Exemplo:* contar partições de n com partes ≤ k; partições em partes distintas; número de partições p(n) para n até 10^5.
  - *Armadilha:* as duas metades precisam ser disjuntas (todas as partes ≤ B vs todas > B) — "poucas partes" e "partes pequenas" se sobrepõem e contariam duas vezes; a conjugação troca "k partes" por "maior parte k", não elimina a necessidade do corte.
- **Pentagonal number theorem.** Contar partições p(n) para n até 10^5–10^6 em O(n√n) via p(n) = Σ ±p(n − k(3k∓1)/2); para só p(n) de um n isolado use Hardy–Ramanujan–Rademacher se n gigantesco (raro).
  - *Gatilho:* é preciso p(n) para todo n até N (ou um único p(n) grande) e N é grande demais para a DP $O(n^2)$ de partições.
  - *Por que funciona:* o teorema pentagonal dá uma recorrência com O(√n) termos por n, com sinais alternados; a soma dos termos é O(n√n) para todos os n.
  - *Como fazer:* p[0] = 1; para n de 1..N, p[n] = Σ_k (−1)^{k−1}(p[n − k(3k−1)/2] + p[n − k(3k+1)/2]), parando quando o índice é negativo. O(N√N).
  - *Exemplo:* p(10^5) mod 10^9+7; número de partições de n para todos os n até 10^6.
  - *Armadilha:* os dois termos por k e o sinal alternado são fáceis de errar; para um n isolado gigante, Rademacher é exato mas precisa de raízes da unidade/float — na prática, use a recorrência.
- **Contar partições com partes distintas = partes ímpares** (Euler) — troca de modelo pode dar DP mais simples; também "diagrama de Young transposto" troca "k partes" por "maior parte = k".
  - *Gatilho:* o enunciado restringe partes distintas ou partes ímpares, ou o número de partes/maior parte, e a DP direta é inconveniente.
  - *Por que funciona:* a bijeção de Euler (conjugar o diagrama de Young) preserva o número de partições; o diagrama transposto troca o número de partes pela maior parte.
  - *Como fazer:* escolha o modelo mais fácil: partes distintas (mochila 0/1 sobre 1..n) ou partes ímpares (mochila com ímpares); para "k partes", conjugue para "maior parte k" e use a DP de partes ≤ k. O(n√n) com o corte.
  - *Exemplo:* partições em partes distintas; partições em partes ímpares; partições com no máximo k partes.
  - *Armadilha:* a bijeção é entre partições, não entre partições com partes limitadas arbitrárias; confirme que a restrição é preservada pela conjugação.

- **Insertion DP (permutações).** Construa a permutação inserindo n elementos em ordem de valor/tempo; dp[i][j] = nº de arranjos dos i primeiros elementos em que o i-ésimo elemento ficou na posição relativa j. Para contar permutações com restrição tipo "a_i < a_{i+1}" (sinais <,>) use prefix sums em j ($O(n^2)$).
  - *Gatilho:* contar permutações com uma restrição local entre posições consecutivas (sinais <, >, diferença), construindo da esquerda para a direita.
  - *Por que funciona:* ao inserir o novo maior elemento, a posição relativa do último determina quantas extensões são válidas; o estado (i, posto do último) é suficiente.
  - *Como fazer:* dp[i][j] com transição somando um prefixo ou sufixo de j conforme o sinal exigido; use prefix sums para $O(n^2)$ total; para "diferença ≤ d", adicione a diferença ao estado ou use estruturas por valor.
  - *Exemplo:* contar permutações alternadas (up-down); permutações com a_i < a_{i+1} em posições pares; contagem por padrão de sinais.
  - *Armadilha:* o posto é relativo aos i elementos, não o valor absoluto; a ordem de inserção (maior para menor ou menor para maior) muda a transição — escolha uma e seja consistente.
- **Connected components DP (permutações / sequências com condição entre vizinhos).** Insira elementos em ordem crescente; estado dp[i][c] = nº de componentes (blocos) após i elementos; cada novo elemento cria componente nova (c+1−gap), cola em uma ponta (2c) ou une duas (c−1). Contagem de permutações com |p_i − p_{i+1}| restrito, soma de diferenças adjacentes fixa (adicionar ao estado a soma acumulada = 2·c·δ).
  - *Gatilho:* a restrição envolve diferenças entre valores adjacentes na permutação, e os valores são processados em ordem crescente.
  - *Por que funciona:* os blocos são trechos já formados; o novo valor só pode abrir um bloco, encostar numa das pontas ou ligar dois blocos, e o número de escolhas depende só de c.
  - *Como fazer:* dp[i][c] com as três transições (c+1, 2c, c−1 multiplicadas pelas escolhas); para somas de diferenças, adicione (2c − pontas fechadas)·δ ao custo ao passar de a_i para a_{i+1}; no fim, c = 1. $O(n^2)$.
  - *Exemplo:* contar permutações com |p_i − p_{i+1}| ≤ d; soma total das diferenças adjacentes ≤ K; permutações com padrão de subida/descida.
  - *Armadilha:* numa permutação linear, as duas pontas globais não recebem vizinho de fora — carregue flags de ponta já fechada, senão a contagem de "2c" superestima; em círculo, as pontas se ligam no fim.
- **Soma de |p_i − p_{i+1}| por componentes.** Ordene valores crescentes, a cada passo o custo acumulado é (nº de componentes abertas·2)·(a_{i+1} − a_i); dp[i][c][soma] com soma ≤ K pequeno. Problemas "contar arranjos com soma total de diferenças ≤ K".
  - *Gatilho:* a resposta envolve a soma das diferenças absolutas entre elementos consecutivos da permutação, com K pequeno.
  - *Por que funciona:* entre os valores a_i e a_{i+1}, cada aresta que atravessa esse "corte" de valor contribui δ; o número de arestas abertas é 2c (menos as pontas globais já fechadas).
  - *Como fazer:* processe valores em ordem; ao passar de a_i para a_{i+1}, some (2c − pontas)·δ ao custo acumulado; dp[i][c][s] com s ≤ K; ao final, c = 1 e s é a soma. $O(n^2K)$.
  - *Exemplo:* contar permutações de 1..n com Σ|p_i−p_{i+1}| ≤ K; arranjos com pouca variação total.
  - *Armadilha:* o fator 2 e as pontas globais são a fonte dos erros; a soma é acumulada, não recalculada; K pequeno é essencial (o estado s é o gargalo).
- **Contagem de permutações por sinais (zigzag/up-down).** dp[i][j] = nº de permutações de i com último elemento ocupando posto j; transição por prefix sum. Para n 10^5 use EGF: sec + tan (números de Euler ziguezague) ou divisão de polinômios.
  - *Gatilho:* contar permutações com padrão de desigualdades entre posições consecutivas (alternadas, todas "<", todas ">").
  - *Por que funciona:* o posto do último elemento é suficiente; a transição é uma soma de prefixo/sufixo dos postos permitidos pelo sinal.
  - *Como fazer:* dp[i][j], transição por prefix sums $O(n^2)$; para n grande e padrão alternado, use a EGF das permutações alternadas (tan x + sec x) e extraia o coeficiente por série de potências O(n log n).
  - *Exemplo:* contar permutações up-down de comprimento n; números de Euler ziguezague; padrões de sinais arbitrários.
  - *Armadilha:* o padrão começar com "<" ou ">" muda a paridade/EGF (tan vs sec); a convenção do posto (1..i) precisa ser consistente; para n = 10^5, a resposta é mod p.

- **Contar permutações com k inversões.** dp[i][k] = Σ_{t=0}^{i−1} dp[i−1][k−t] → prefix sum, O(n·K); para K ≤ n(n−1)/2 grande e n pequeno é o produto Π(1 + x + … + x^{i−1}) — use polinômio.
  - *Gatilho:* contar permutações de n com exatamente (ou até) k inversões.
  - *Por que funciona:* inserir o maior elemento na posição t cria t novas inversões; a soma sobre t é uma janela de tamanho i, que vira prefix sum.
  - *Como fazer:* dp[i][k] com prefix sums em k (ou janela deslizante), O(nK); para K grande e n moderado, multiplique os polinômios ∏(1+x+…+x^{i−1}) com NTT ou use a fórmula de Mahonian. O(nK) ou O(n^2 log n).
  - *Exemplo:* número de permutações com 10 inversões; distribuição de inversões (Mahonian); permutações com no máximo k inversões.
  - *Armadilha:* k pode ser até n(n−1)/2 — estime a memória; a janela tem tamanho i (não i−1) e precisa do caso base dp[0][0]=1; módulo em cada soma.
- **Mínimo de adjacentes trocados / swap distance** = inversões; para "transforme a em b com swaps" mapeie a para posições em b e conte inversões (BIT). DP só se houver restrições extras.
  - *Gatilho:* "mínimo de trocas adjacentes para transformar uma permutação na outra", ou "distância de swaps".
  - *Por que funciona:* cada troca adjacente corrige exatamente uma inversão; o número mínimo é o número de pares fora de ordem, que é a distância.
  - *Como fazer:* mapeie cada elemento de a para sua posição em b (para duplicatas, case a i-ésima ocorrência com a i-ésima); conte inversões da sequência de posições com BIT/merge sort O(n log n).
  - *Exemplo:* mínimo de swaps adjacentes para ordenar; transformar uma string na outra com trocas adjacentes; distância de Kendall.
  - *Armadilha:* com elementos repetidos, o mapeamento precisa ser estável (i-ésima ocorrência), senão a contagem não é mínima; trocas não adjacentes mudam o problema.
- **DP em permutação circular.** Fixe o elemento 1 (ou o menor) numa posição para quebrar a simetria; contar n arranjos circulares = (n−1)! × linear; para DP circular sobre a escolha de itens (nenhum vizinho adjacente), rode duas vezes: primeiro e último excluídos de um deles.
  - *Gatilho:* o objeto é um círculo (rotação identifica arranjos) ou a restrição liga o último ao primeiro.
  - *Por que funciona:* fixar um elemento elimina as n rotações equivalentes; e condicionar a escolha do primeiro elemento transforma o ciclo numa linha com casos de borda.
  - *Como fazer:* fixe o elemento 1 na posição 0 e conte arranjos lineares dos demais ((n−1)!); para seleção sem vizinhos, rode a DP linear duas vezes: uma com o primeiro excluído, outra com o primeiro incluído e o último forçado excluído; some. O(n).
  - *Exemplo:* contar permutações circulares; colorir um colar; escolher vértices não adjacentes num ciclo.
  - *Armadilha:* rodar "primeiro excluído" e "último excluído" sem forçar o outro conta o caso ambos excluídos duas vezes; cuidado com n = 1 e n = 2 no ciclo.
- **Escolher k itens sem dois adjacentes (círculo/linha).** dp[i][j][pegou i-1?] em O(nk); com n grande e k pequeno há também o guloso heap com lista ligada (escolha o melhor, substitua por vizinhos − atual) — o "arrependimento" certo para max soma sem adjacentes com k.
  - *Gatilho:* "escolha exatamente k itens", "nenhum par adjacente", em fila ou em círculo, com k pequeno ou moderado.
  - *Por que funciona:* para decidir o item i, o único histórico que importa é se i−1 foi escolhido; no círculo, também é preciso lembrar se o primeiro foi escolhido, pois ele é vizinho do último.
  - *Como fazer:* na linha, dp[i][j][b] = melhor valor usando i primeiros, j escolhidos e b indicando se i foi escolhido; transicione para i+1 escolhendo ou não, e role a primeira dimensão. No círculo, fixe o estado do primeiro item e exija compatibilidade no fim; complexidade O(nk).
  - *Exemplo:* máxima soma de k elementos não adjacentes; versão de custo mínimo é análoga.
  - *Armadilha:* no círculo não basta proibir "primeiro e último escolhidos" no fim sem fixar o primeiro durante toda a DP; e "no máximo k" não é o mesmo que "exatamente k" quando há valores negativos.
  - *Guloso com arrependimento:* escolha o maior valor, some-o e substitua o bloco (vizinho esquerdo, escolhido, vizinho direito) por um único nó de valor L + R − x, religando as pontas; repetir k vezes. O nó negativo representa desfazer a escolha e pegar os dois vizinhos; com heap e lista ligada duplamente encadeada custa O((n + k) log n).
  - *Armadilha do guloso:* use sentinelas para as pontas e trate o ciclo quebrando-o ou usando lista circular; a factibilidade exige k ≤ ⌈n/2⌉ na linha (e k ≤ ⌊n/2⌋ no ciclo ímpar/par conforme o caso).

- **Heap + lista ligada vs DP.** "Escolha k não-adjacentes" onde dp é côncava em k → o guloso com arrependimento dá a resposta exata para cada k; use também como alternativa ao Aliens.
  - *Gatilho:* a resposta f(k) para "exatamente k não adjacentes" precisa ser obtida para um k só, e k é pequeno diante de n.
  - *Por que funciona:* a sequência f(k) é côncava nesse problema; a contração L + R − x é exatamente o "arrependimento" que corrige uma escolha local por duas escolhas vizinhas, e repetir k vezes simula o ótimo global.
  - *Como fazer:* insira todos os valores em max-heap com ponteiros para antecessor e sucessor; em cada passo retire o máximo, acumule-o, crie o nó L + R − x no lugar do trio e atualize os ponteiros. Alternativamente, binária em λ e DP com penalidade λ por item (Aliens).
  - *Exemplo:* [5, 100, 5, 100, 5] com k = 3: pega 100, depois 100, e o nó de arrependimento −185 fecha o total 15, que é o ótimo (5 + 5 + 5).
  - *Armadilha:* a concavidade é uma propriedade do problema, não de qualquer DP; confirme com força bruta em instâncias pequenas antes de trocar a DP O(nk) pelo guloso. Em ciclo, um único nó restante pode exigir cuidado com o fechamento.
- **Aliens / WQS (lambda).** "Exatamente k" com f(k) convexa: busque λ tal que o ótimo da DP com penalidade λ por item use k itens; resposta = g(λ) − λ·k, com g(λ) = min_k [f(k) + λ·k]. Cuidado com empates: guarde o MENOR (ou maior) número de itens entre ótimos para a binária ser coerente; λ inteiro basta quando f é inteira.
  - *Gatilho:* "exatamente k" com k grande, e a DP sem a restrição de quantidade é muito mais barata; a função ótima f(k) é convexa.
  - *Por que funciona:* a penalidade λ transforma a restrição de cardinalidade em preço; se f é convexa, os minimizadores de f(k) + λk varrem todos os k quando λ varia (dualidade de Lagrange exata nesse caso discreto).
  - *Como fazer:* para um λ, rode a DP guardando (valor penalizado, quantidade de itens); desempate sempre pelo menor número de itens (ou sempre pelo maior). Busque o maior λ cuja DP ainda usa ≥ k itens (ou o intervalo adequado), recupere g(λ) e faça g(λ) − λk.
  - *Exemplo:* escolher exatamente k caminhos disjuntos de custo mínimo: cobre um custo extra λ por caminho e resolve a versão sem restrição com fluxo/DP.
  - *Armadilha:* trocar o sinal da penalidade; usar desempates inconsistentes e a binária oscilar; assumir convexidade sem prova; λ fora do intervalo dos custos marginais; e contar itens de forma diferente entre a DP e a resposta.
- **Aliens: como reconhecer convexidade.** Se existe fluxo de custo mínimo de fundo (k unidades de fluxo), ou matroide/guloso com arrependimento para custo mínimo (máximo em matroide é côncavo, não convexo), f é convexa; se k é "número de segmentos" com custo Monge, também. Sem prova, valide com brute force imprimindo f(k) para k pequeno e checando segundas diferenças.
  - *Gatilho:* você quer aplicar Aliens, mas não há uma prova direta de que f(k) é convexa.
  - *Por que funciona:* fluxo de custo mínimo tem custo convexo no valor do fluxo; em matroides, o peso máximo é côncavo na cardinalidade, então a versão de minimização com sinal trocado tem a convexidade adequada; custos Monge também produzem essa estrutura.
  - *Como fazer:* calcule f(k) por força bruta para k = 0..K em instâncias pequenas e verifique f(k+1) − f(k) não decrescente (convexa) ou não crescente (côncava, dependendo do sinal); só então aplique a binária em λ.
  - *Exemplo:* k caminhos disjuntos com custo mínimo → convexa; máximo peso de conjunto independente de tamanho k em um matroide → côncava.
  - *Armadilha:* convexidade em cada k separadamente não garante convexidade conjunta; máximos em matroide são côncavos, não convexos; e uma checagem só nos primeiros k pode não revelar uma quebra posterior.
- **Aliens aninhado (duas restrições k_1, k_2).** Dois λ em binária dupla, $O(\log^2 \cdot DP)$; só funciona se f é convexa em ambos jointly (ex.: certos problemas de emparelhamento); desconfie e teste.
  - *Gatilho:* duas contagens exatas, "exatamente k_1 de um tipo e k_2 de outro", e a DP sem contagens é barata.
  - *Por que funciona:* com duas penalidades, a DP resolve $\min f(k_1,k_2) + \lambda_1 k_1 + \lambda_2 k_2$; a dualidade recupera o ponto desejado apenas se a função for conjuntamente convexa na região relevante.
  - *Como fazer:* faça binária em λ_1 e, dentro dela, binária em λ_2; em cada DP guarde o par de contagens e use regras de desempate fixas. Complexidade $O(\log^2 R \cdot$ custo da DP$)$.
  - *Exemplo:* emparelhamento com exatamente k_1 arestas de um grupo e k_2 de outro, quando a estrutura de custos é convexa.
  - *Armadilha:* convexidade em cada coordenada não implica convexidade conjunta; a região pode ter "buracos" ou pontos não suportados. Valide com brute force 2D e não use sem necessidade.

- **Slope trick: reconhecer.** dp[i][x] = min_y dp[i−1][y] + |x − a_i| com restrição em y−x (monotonia, |x−y| ≤ d) e dp[0] convexa → dp[i] continua convexa linear por partes; guarde só os pontos de quebra e um valor mínimo.
  - *Gatilho:* o estado é um valor inteiro x possivelmente enorme; a transição soma |x − a_i| ou impõe x ≥ y, x ≤ y, ou uma janela de deslocamento.
  - *Por que funciona:* somar |x − a| e tomar ínfimo sobre y preservam convexidade; a função continua linear por partes e só muda nos pontos de quebra, que podem ser mantidos em heaps.
  - *Como fazer:* represente a derivada por dois multiconjuntos: heap esquerdo com as quebras onde a inclinação sobe de −1 para 0 e heap direito com as quebras onde sobe de 0 para +1; guarde o mínimo. Inserir |x − a| move uma quebra entre os heaps; restrições monotônicas descartam um lado.
  - *Exemplo:* mínimo Σ |x_i − a_i| com x não decrescente (slope trick clássico), ou custo de ajustar alturas com |x − y| ≤ d.
  - *Armadilha:* a função inicial precisa ser convexa; |x − y| ≤ d não é a transição simples de |x − a| — é uma janela, e exige mínimo deslizante/offsets, não apenas dois heaps comuns. Coordenadas reais exigem multiplicar por denominadores ou usar frações.
- **Slope trick com translação.** "Janela deslizante" desloca os heaps esquerdo e direito por offsets lazy diferentes (addL, addR); inserção de |x − a| move um ponto entre heaps. Custo total O(n log n).
  - *Gatilho:* há um deslocamento permitido entre estados (x − y em um intervalo, ou x ≥ y + c) e os valores crescem muito.
  - *Por que funciona:* deslocar a variável x apenas translada todos os pontos de quebra; em vez de alterar cada elemento dos heaps, guarda-se um offset global para cada lado.
  - *Como fazer:* mantenha addL e addR; ao comparar ou inserir, use o valor real (valor guardado + offset). Para impor L ≤ x − y ≤ R, descarte quebras fora da janela e ajuste os offsets; cada quebra entra e sai no máximo uma vez.
  - *Exemplo:* sequência com |x_i − x_{i−1}| ≤ d e custo |x_i − a_i|; ou "mínimo custo para tornar a sequência não decrescente com folga".
  - *Armadilha:* comparar valores crus com offsets diferentes; esquecer de compensar o offset ao remover do heap; usar dois heaps quando a janela é assimétrica e exige tratamento separado dos dois lados.
- **SOS DP.** f[mask] = Σ_{sub ⊆ mask} a[sub] em O(2^n·n); também superset e "subconjuntos com um bit por vez" (zeta, e Möbius para inverter). Para contar pares (i, j) com a_i & a_j = 0: SOS no complemento.
  - *Gatilho:* n ≤ 20–24, somas/contagens sobre todos os subconjuntos de cada máscara, ou condições do tipo "todos os bits de um estão contidos nos do outro".
  - *Por que funciona:* o zeta transform aplica cada bit uma vez; ao processar o bit b, toda máscara com b recebe o valor da máscara sem b, acumulando exatamente todos os subconjuntos. A transformada de Möbius desfaz a soma.
  - *Como fazer:* copie f = a; para cada bit b, para cada mask com b, faça f[mask] += f[mask ^ (1<<b)]. Para superset, itere as máscaras sem b e some f[mask | bit]. Para contar pares com a_i & a_j = 0, faça cnt[mask] = frequência e zeta; para cada i consulte cnt[full ^ a_i] e ajuste a contagem ordenada/do próprio elemento conforme o caso.
  - *Exemplo:* f[S] = soma dos a[sub] para sub ⊆ S; ou número de pares cuja AND é zero via complemento.
  - *Armadilha:* 2^n memória/tempo; não confunda subconjunto com submáscara própria; na contagem de pares, a_i = 0 ou i = j pode ser contado indevidamente — defina se o par é ordenado e subtraia o caso diagonal.
- **SOS com min/max e "dois melhores".** Para "melhor par dentro do submask" guarde os 2 melhores com índices distintos por máscara; mescla de top-2 em cada passo.
  - *Gatilho:* quer o melhor par (ou a maior soma de dois elementos distintos) cujos índices/máscaras estão contidos em S, e não apenas o melhor elemento.
  - *Por que funciona:* para maximizar a soma de dois elementos distintos dentro de um conjunto, basta conhecer os dois maiores valores com índices diferentes; a união de dois subconjuntos só pode produzir o top-2 a partir dos top-2 de cada lado.
  - *Como fazer:* guarde (maior valor, índice) e (segundo maior, índice diferente) por máscara; no SOS, ao unir mask com mask^bit, mescle os dois pares e mantenha os dois melhores com índices distintos. Para mínimo, análogo.
  - *Exemplo:* máximo a_i + a_j com i, j ⊆ S e i ≠ j; ou melhor par de folhas em uma máscara.
  - *Armadilha:* pegar o mesmo índice duas vezes; empates com índices iguais; guardar só o melhor por máscara perde o segundo candidato necessário para formar o par.

- **Convolução OR/AND/XOR por SOS e Walsh-Hadamard.** (a ⊛_OR b)[m] = Möbius(zeta(a)·zeta(b)); XOR → WHT (2^n n). Contagem de subconjuntos com OR/xor fixo = potência (exponencial por FWHT).
  - *Gatilho:* combinar dois arrays indexados por máscara usando união (OR), interseção (AND) ou diferença simétrica (XOR), com n ≤ ~20.
  - *Por que funciona:* o zeta transform diagonaliza a convolução de OR (e o zeta de superset diagonaliza AND); para XOR, a transformada de Walsh-Hadamard diagonaliza a convolução, então basta multiplicar ponto a ponto.
  - *Como fazer:* OR: calcule za = zeta(a), zb = zeta(b), faça o produto ponto a ponto e aplique Möbius. AND: use zeta de superset e Möbius correspondente. XOR: aplique WHT em a e b, multiplique e aplique a WHT inversa dividindo por 2^n. Para k cópias, eleve as transformadas à potência k.
  - *Exemplo:* contar subconjuntos cujo OR é U, ou sequências de k máscaras com XOR fixo; OR de todos os subconjuntos de um conjunto.
  - *Armadilha:* esquecer a divisão por 2^n na WHT inversa; usar Möbius de subconjunto com AND; a "potência" de OR também funciona via zeta, mas a interpretação de subconjuntos versus tuplas precisa ser conferida.
- **Subset convolution** (união disjunta) em O(2^n n^2): ranked zeta — um polinômio por popcount. Aparece em "particione conjunto em k partes com função de peso multiplicativa" (exp de conjunto).
  - *Gatilho:* a convolução só deve contar pares (S, T) com S ∩ T = ∅ e S ∪ T = U; união com sobreposição deve valer zero.
  - *Por que funciona:* separar por |S| torna a disjunção detectável: se |S| + |T| = |U| e S ∪ T = U, então S e T são disjuntos. O ranked zeta transforma união em produto por grau.
  - *Como fazer:* para cada rank k, defina f_k[mask] = f[mask] se popcount(mask) = k, senão 0; aplique zeta em cada f_k. Para cada mask, calcule h_k[mask] = Σ_{i+j=k} f_i[mask]·g_j[mask]; aplique Möbius em cada h_k; a resposta em mask é h_{popcount(mask)}[mask].
  - *Exemplo:* contar partições de um conjunto em blocos com peso g[bloco], via exponencial de conjunto sob subset convolution.
  - *Armadilha:* aplicar Möbius antes de separar ranks; usar união comum no lugar da disjunta; o fator n^2 e a memória O(2^n n) tornam n > 20 inviável.
- **DP de contagem de partições de conjunto em máscara.** dp[mask] = Σ_{sub ∋ menor bit de mask} g[sub]·dp[mask \ sub], em 3^n; com subset convolution + exp de GF por conjuntos em 2^n n^2.
  - *Gatilho:* contar partições de um conjunto em blocos, cada bloco com peso g, e n é pequeno (até ~20).
  - *Por que funciona:* escolher o bloco que contém o menor elemento ainda não usado remove a simetria de ordem dos blocos: cada partição é gerada exatamente uma vez.
  - *Como fazer:* dp[0] = 1; para mask ≠ 0, seja b o menor bit; dp[mask] = Σ_{sub ⊆ mask, b ∈ sub} g[sub]·dp[mask \ sub]. A enumeração direta é O(3^n); para acelerar, use subset convolution e a exponencial de série formal de conjuntos em O(2^n n^2).
  - *Exemplo:* número de partições em blocos independentes de um grafo, ou partições com blocos de tamanho permitido.
  - *Armadilha:* escolher um subconjunto sem fixar o menor bit conta a mesma partição várias vezes; g[sub] precisa ser multiplicativo por bloco; cuidado com g[conjunto vazio].
- **Meet in the middle em DP/knapsack.** n ≤ 40 com valores grandes: divida em metades, enumere 2^{n/2} somas, ordene e use two pointers; para "soma ≤ W maximizando valor" com valor também, mantenha pareto (valor crescente com peso crescente) e busque binário.
  - *Gatilho:* n está entre ~25 e 45, os pesos/valores são grandes demais para DP por capacidade, e 2^n não passa.
  - *Por que funciona:* toda solução é a união de uma escolha da primeira metade com uma da segunda; enumerar as duas metades custa 2^{n/2} e a combinação pode ser feita de forma ordenada.
  - *Como fazer:* gere todos os subconjuntos de cada metade com (peso, valor); para peso apenas, ordene um lado e use two pointers. Para peso e valor, remova estados dominados (peso maior com valor menor) e, para cada estado da primeira metade, busque no vetor de prefixos máximos da segunda o maior valor com peso ≤ W − peso1.
  - *Exemplo:* mochila 0/1 com n = 40 e W = 10^15; ou contar/otimizar somas de subconjuntos.
  - *Armadilha:* 2^{n/2} memória ainda pode estourar (use geração iterativa); no pareto, ordenar por peso e guardar prefixo máximo de valor; não aplique two pointers a valor arbitrário sem essa monotonicidade.

- **Reduza o estado por monotonia ou dominância.** Se dp[i][j] é monótono em j (mais j nunca pior), guarde só a fronteira (j mínimo por valor / estados Pareto); a DP vira "lista de pares não dominados" e para de estourar.
  - *Gatilho:* muitos valores de j são inviáveis, mas a factibilidade é monótona; ou há dois critérios e um estado domina outro.
  - *Por que funciona:* se $j \le j'$ e dp[i][j] é pelo menos tão bom, o estado j' nunca será necessário para nenhuma transição futura que só piore com j; estados dominados podem ser descartados.
  - *Como fazer:* mantenha a fronteira em ordem (por exemplo, menor j para cada valor, com valores estritamente melhores) e insira novos estados removendo os dominados. Se a fronteira é grande, combine com busca binária ou uma estrutura de consulta.
  - *Exemplo:* dp[j] = menor custo para obter valor j; guardar apenas o menor custo para cada valor e remover custos maiores com valores menores.
  - *Armadilha:* a dominância precisa valer para todas as transições futuras; se uma restrição futura depender de j de forma não monótona, o descarte é inválido. Em empates, preserve o estado que ainda pode ser estendido.
- **Troca de dimensão por valor da DP (generalizada).** Se n é pequeno mas as coordenadas enormes, ou se o valor da resposta é pequeno ($\le \log$ ou $\le \sqrt{}$): indexe dp[resposta] = melhor parâmetro. Ex.: dp[k] = menor posição final com k escolhidos (LIS é isso).
  - *Gatilho:* a resposta ou um dos parâmetros tem faixa pequena, enquanto a coordenada original é enorme; ou n é pequeno e o valor é limitado.
  - *Por que funciona:* inverter estado e valor só é válido quando, para cada valor de resposta, basta guardar o melhor parâmetro possível; a viabilidade futura é monótona nesse parâmetro.
  - *Como fazer:* defina dp[r] = menor/maior parâmetro que realiza resposta r; atualize com a transição original e retire os valores impossíveis. Se a resposta é $\le \sqrt{n}$ ou $\le \log n$, o número de estados cai de n para essa faixa.
  - *Exemplo:* LIS indexada pelo comprimento: dp[k] = menor valor final (cauda) de uma subsequência crescente de tamanho k, com busca binária; a posição final mínima sozinha não é suficiente.
  - *Armadilha:* no exemplo do LIS, guardar a menor posição final em vez da menor cauda perde informação (uma posição menor pode ter cauda enorme); confira qual parâmetro realmente determina a extensibilidade antes de trocar as dimensões.
- **Resposta limitada por log ou √.** Se o valor ótimo é provadamente $\le O(\log n)$ ou $\le O(\sqrt{n})$ (ex.: ovos/andares — com 2 ovos é $\Theta(\sqrt{n})$, não log, e com e ovos é $\Theta(n^{1/e})$ — e menor número de operações de dobra), faça dp[j][ops] = alcance máximo com ops tentativas — troca estado e valor; ovos: $f(t, e) = f(t-1, e-1) + f(t-1, e) + 1$.
  - *Gatilho:* o número de operações/tentativas é a resposta e é pequeno; em vez de indexar pelos andares/posições, dá para indexar pelas tentativas.
  - *Por que funciona:* com t tentativas e e ovos, o alcance máximo é determinado por quantos andares ainda podem ser distinguidos: o teste no andar x separa "quebrou" ($e-1$ ovos, $t-1$ testes abaixo) de "não quebrou" (e ovos, $t-1$ testes acima), daí $f(t,e) = f(t-1,e-1) + f(t-1,e) + 1$.
  - *Como fazer:* calcule f(t,e) para t crescente até $f(t,e) \ge n$; a resposta é o menor t. Com 2 ovos, $f(t,2) = t(t+1)/2$, logo $\Theta(\sqrt{n})$; com e ovos, o crescimento é $\Theta(t^e)$, logo $\Theta(n^{1/e})$.
  - *Exemplo:* menor número de testes no pior caso para descobrir o andar crítico com 2 ovos é $\lceil(\sqrt{(8n+1)}-1)/2\rceil$; com e ovos, use a tabela de alcance. O mesmo padrão vale para variantes de "menor número de operações" em que cada dobra amplia o alcance.
  - *Armadilha:* 2 ovos NÃO dão $\Theta(\log n)$ (isso exigiria ovos ilimitados); a fórmula f(t,e) pressupõe monotonia: até o andar crítico o ovo não quebra e acima dele quebra. Use base $f(0,e)=0$, $f(t,0)=0$ e cuidado com overflow ao calcular t^e.
- **DP de pior caso (minimax) com resposta log.** "Mínimo de tentativas no pior caso" → inverta: quantos casos distinguíveis com t tentativas; $O(t \cdot k)$ em vez de $O(n^2 k)$.
  - *Gatilho:* "minimize o número de tentativas no pior caso", com poucas tentativas possíveis e muitos estados.
  - *Por que funciona:* uma estratégia de t tentativas é uma árvore de decisão de altura t; o número de folhas/casos distinguíveis é limitado por quanto cada teste pode separar, então contar o alcance máximo evita varrer todos os estados.
  - *Como fazer:* defina reach[t] = número máximo de casos que t testes conseguem distinguir; aplique a recorrência do problema e pare quando $reach[t] \ge n$. Complexidade $O(t \cdot k \log)$ em vez de $O(n^2 k)$.
  - *Exemplo:* ovos e andares com e ovos; adivinhação com respostas de k valores; "número mínimo de perguntas binárias no pior caso".
  - *Armadilha:* não confunda pior caso com caso médio; a árvore de decisão precisa ter respostas possíveis bem definidas e a recorrência deve contar todos os ramos, não apenas um.

- **Meio-DP: balanceie dimensões.** Se dp[i][j] tem n ≤ 10^5 e k ≤ 20, a transição k·n·log pode entrar; se k ≤ ~√n e n grande, trocar quem vai no estado (segmentos × posição) por dp[k][i] ou o contrário muda log.
  - *Gatilho:* a DP tem duas dimensões de tamanhos muito diferentes e a transição custa proporcional a uma delas; o limite está apertado.
  - *Por que funciona:* o custo é o produto das dimensões e do custo de transição; escolher qual dimensão fica no estado e qual fica no laço muda constantes, localidade de cache e o log de estruturas auxiliares.
  - *Como fazer:* estime n·k·T; se k é pequeno, dp[k][i] com laço interno em k; se n é pequeno, dp[i][k]; se ambos médios, role a maior dimensão e processe a menor em blocos. Reordene os laços para acessar memória contígua.
  - *Exemplo:* partição em k segmentos com n = 10^5 e k = 20; ou DP de caminhos com número de usos ≤ k.
  - *Armadilha:* trocar as dimensões sem ajustar a ordem topológica; acessos não contíguos podem ser mais lentos que o ganho assintótico; e k ≤ √n não significa que O(nk) caiba se a constante for alta.
- **DP sobre pequena dimensão: n·m ≤ 10^6.** Itere pela menor dimensão no estado de máscara/profundidade e pela maior no laço; se m ≤ 20, máscara na menor; se ambos ≤ 10^3, DP O(nm) rolando linha.
  - *Gatilho:* grade ou tabela com n·m ≤ 10^6, mas uma das dimensões é pequena o suficiente para máscara/estado.
  - *Por que funciona:* o número total de estados é o produto; indexar a dimensão pequena no estado permite transições por máscara, enquanto a dimensão grande pode ser processada em sequência.
  - *Como fazer:* se m ≤ 20, use dp[mask] por linha e transicione com deslocamentos de bits; se a dimensão pequena for profundidade, use-a no estado e a maior no laço; se n,m ≤ 10^3, use dp[j] e role a linha; sempre itere a dimensão grande por fora quando isso melhora a localidade.
  - *Exemplo:* colorir grade n × m com m ≤ 12; caminhos em grade com perfil; programação em tabuleiro.
  - *Armadilha:* m ≤ 20 dá 2^m, não m; n·m ≤ 10^6 mas 2^m pode ser 10^6 e ainda viável, porém 3^m não; cuidado com a memória de todas as linhas.
- **Redução de memória: rolling array.** dp[i] só olha dp[i−1] → 2 linhas; só olha dp[i−1][j] e dp[i−1][j−1] → 1 linha iterando j decrescente; Hirschberg para reconstruir caminho em O(min) memória (divide pela linha do meio e guarda onde cruza).
  - *Gatilho:* a DP tem muitas camadas e a memória O(nm) estoura, mas cada camada só depende da anterior.
  - *Por que funciona:* estados antigos que não serão mais consultados podem ser sobrescritos; a direção da iteração preserva os valores ainda necessários.
  - *Como fazer:* use duas linhas e troque-as; para dependências (i−1,j) e (i−1,j−1), uma linha com j decrescente basta. Para reconstruir o caminho, use divisão e conquista de Hirschberg: calcule a linha do meio para frente e para trás e guarde o ponto de cruzamento.
  - *Exemplo:* LCS e edit distance em O(min(n,m)) de memória; mochila 0/1 com uma linha e j decrescente.
  - *Armadilha:* iterar j crescente na mochila 0/1 reutiliza o item; guardar uma linha só quando a transição olha duas linhas; Hirschberg troca memória por recomputação e precisa de cuidado com empates na reconstrução.
- **Reconstrução do caminho com memória apertada.** Guarde só um bit "de onde veio" por estado (bitset), ou refaça a DP para trás comparando valores (re-derive a escolha), ou use D&C de Hirschberg; evite guardar ponteiros de 32 bits em dp $10^4 \times 10^4$.
  - *Gatilho:* a resposta exige a sequência de decisões e a tabela completa de ponteiros não cabe.
  - *Por que funciona:* a escolha em cada estado é uma função dos valores vizinhos; um bit por estado ou uma segunda passada reproduz a mesma decisão sem armazenar um ponteiro largo.
  - *Como fazer:* se há duas transições, empacote 1 bit por estado em um bitset; se há muitas, reconstrua de trás para frente recalculando o valor ótimo local; para LCS/edit distance, Hirschberg acha o ponto de corte e recursa.
  - *Exemplo:* reconstruir um LCS de $10^4 \times 10^4$; caminho mínimo em grade; sequência de escolhas de uma mochila.
  - *Armadilha:* a regra de desempate na reconstrução tem de ser idêntica à da DP, senão o valor recalculado não fecha; Hirschberg pode ter complexidade constante maior e não serve para qualquer transição com lookahead.
- **Memória de DP em árvore.** dp[v][·] pode ser liberado após o merge ao pai (vector move); com knapsack O(n^2) e n=5000 a memória é O(n) simultâneo se processar filhos grandes primeiro (heavy-first) — ou use ordem DFS com alocação por tamanho.
  - *Gatilho:* DP de árvore com vetor por vértice e n grande; guardar todos os vetores ao mesmo tempo estoura a memória.
  - *Por que funciona:* depois que o filho é mesclado no pai, seu vetor não é mais necessário; se os filhos são processados em uma ordem que mantém poucos vetores grandes vivos, o pico cai.
  - *Como fazer:* faça pós-ordem, mescle cada filho no pai e libere o vetor do filho (std::move/clear); processe primeiro o filho de maior subárvore para que o vetor grande do pai cresça cedo e os pequenos sejam absorvidos. Alternativamente, aloque por tamanho em uma arena.
  - *Exemplo:* knapsack em árvore com n = 5000; contagem de matchings por tamanho em subárvores.
  - *Armadilha:* liberar o filho antes de todas as transições que o usam; a análise O(n) de pico depende da ordem e do custo de merge; recursão profunda pode estourar a pilha.

- **DP em DAG vira caminho mínimo/longo.** Se estados são vértices e transições arestas com custo, dp em ordem topológica é caminho mais curto/longo; com ciclos de custo ≥ 0 use Dijkstra sobre estados em vez de DP por camada.
  - *Gatilho:* o grafo de estados é acíclico (ou quase) e cada transição tem um custo; a DP é uma relaxação de arestas.
  - *Por que funciona:* em ordem topológica, quando um vértice é processado, todos os caminhos que chegam nele já foram considerados; para caminho mais curto com pesos não negativos, Dijkstra generaliza quando há ciclos.
  - *Como fazer:* construa o grafo de estados, calcule a ordem topológica e relaxe as arestas; para contar caminhos, some as contagens na mesma ordem. Se houver ciclos de custo ≥ 0, troque por Dijkstra (ou 0-1 BFS se os pesos forem 0/1).
  - *Exemplo:* caminho mais longo em DAG; número de caminhos em um DAG; DP de mochila com dependências acíclicas.
  - *Armadilha:* caminho mais longo em grafo com ciclos é NP-hard; uma DP por camadas só é válida se as camadas realmente não tiverem arestas de volta; pesos negativos exigem Bellman-Ford/SPFA, não Dijkstra.
- **DP que vira caminho mínimo (e vice-versa).** dp[i] = min_{j: condição} dp[j] + w e as dependências formam ciclos (ex.: pode ir "para trás") → DP em ordem não funciona, use Dijkstra/0-1 BFS sobre i; teste: a transição tem custo ≥ 0 e o grafo não é DAG por posição?
  - *Gatilho:* a recorrência parece DP, mas uma transição pode voltar no índice (i → j < i e depois avançar de novo), ou o grafo de estados tem ciclos.
  - *Por que funciona:* a recorrência é a equação de Bellman de um caminho mínimo no grafo de estados; com custos não negativos, Dijkstra resolve mesmo com ciclos, sem precisar de ordem topológica.
  - *Como fazer:* crie um vértice por estado, uma aresta i → j com o custo da transição e rode Dijkstra a partir da origem; se os custos forem 0/1, use 0-1 BFS; se houver custo negativo, use Bellman-Ford ou reformule.
  - *Exemplo:* dp[i] = min over j de dp[j] + |a_i − a_j| com transições que podem andar para os dois lados; menor custo de transformar uma string com operações reversíveis.
  - *Armadilha:* assumir que "índice maior" define uma ordem topológica quando há transições para trás; usar Dijkstra com aresta negativa; e não confundir o grafo de estados com o grafo de entrada.
- **Caminho mínimo em grafo de estados pequenos com k grande.** dist[v][estado] com estado = (mod k, paridade, nº usos); "k-ésima operação" muda o custo ciclicamente → Dijkstra em v × (tempo mod T) com T pequeno (ex.: semáforos).
  - *Gatilho:* a operação/k-ésima etapa depende do tempo ou de uma fase periódica, e k é enorme, mas o período T é pequeno.
  - *Por que funciona:* o futuro só depende de (vértice, fase) quando os custos são periódicos; o produto cartesiano tem tamanho V·T e pode ser explorado por Dijkstra.
  - *Como fazer:* expanda cada vértice em T cópias (fase 0..T−1); de (v, t) para (u, (t+1) mod T) com o custo da aresta naquela fase; rode Dijkstra e leia o estado final na fase correta. Se a resposta é "após k passos", use caminhada mínima de comprimento k.
  - *Exemplo:* semáforos com ciclo de T segundos; robô que só pode andar em certos horários; custo de pedágio que varia por paridade.
  - *Armadilha:* T grande torna V·T inviável; fases inalcançáveis; "exatamente k" não é o mesmo que "caminho mínimo" — para contagem ou comprimento exato use matriz exponencial ou DP de fase.
- **Guloso vira DP: sinais.** (1) exchange argument falha em empate de custos, (2) o melhor local depende de escolhas futuras (mochila), (3) restrição de capacidade ou contagem exata, (4) pequenos casos mostram escolhas diferentes com mesma entrada local. Sinal forte: n ≤ 5000 ou "exatamente k".
  - *Gatilho:* você tentou ordenar por um critério, mas a ordem não determina a resposta; há capacidade, "exatamente k" ou escolhas locais que dependem do que vem depois.
  - *Por que funciona:* quando o problema tem subestrutura ótima e sobreposição de subproblemas, a decisão local não é segura; a DP enumera as alternativas relevantes e combina subproblemas.
  - *Como fazer:* procure um contraexemplo para o guloso; identifique o estado (posição, capacidade, contagem) e a transição; estime O(n·capacidade) ou O(nk). Se n ≤ 5000 e a capacidade é pequena, DP é o caminho.
  - *Exemplo:* mochila 0/1 versus mochila fracionária; agendamento com prazos e pesos; escolher k itens com restrição.
  - *Armadilha:* "n ≤ 5000" não é prova de DP se a capacidade for 10^9; empates no exchange argument podem ser tratados com regras de desempate, não necessariamente com DP; e uma DP mal dimensionada pode ser pior que um guloso correto.

- **DP vira guloso/estrutura: sinais.** dp[i] = melhor sobre prefixo e a escolha ótima parece sempre "a mais recente barata" → pilha monotônica; dp côncava/convexa em k → heap de arrependimento ou Aliens; dp[i] = max_{j<i} f(j) + g(i) → prefixo.
  - *Gatilho:* a recorrência tem uma forma especial (máximo sobre prefixo, dependência de k com concavidade, ou candidatos que entram e saem de forma monotônica).
  - *Por que funciona:* a estrutura da transição permite manter apenas os candidatos que ainda podem ser ótimos; os demais são dominados e podem ser descartados.
  - *Como fazer:* para max_{j<i} f(j)+g(i), mantenha o prefixo máximo de f; para "último barato", use pilha monotônica; para concavidade em k, use heap de arrependimento ou Aliens. Verifique a condição de monotonicidade antes de remover candidatos.
  - *Exemplo:* máximo subarray com prefixo mínimo; maior retângulo em histograma com pilha; k segmentos com custo convexo via Aliens.
  - *Armadilha:* a pilha monotônica exige que o candidato descartado nunca volte a ser ótimo; com valores negativos ou restrições extras isso pode falhar; "parece guloso" não substitui uma prova de troca.
- **Inverta quem é o índice (dp sobre valores em vez de posições).** Se $a_i \le 2 \cdot 10^5$ e n grande, dp[v] = melhor sequência terminando no valor v (com BIT/segtree) evita estado de posição; "transição por divisores/múltiplos" → harmônico em v.
  - *Gatilho:* os valores são pequenos (ou compressíveis) e n é grande; a transição depende do valor anterior, não da posição.
  - *Por que funciona:* o valor é a única informação necessária para estender a sequência; indexar por valor reduz o estado de n posições para V valores e permite consultas de prefixo/sufixo.
  - *Como fazer:* comprima os valores; dp[v] = melhor resposta terminando em v; use BIT/segtree para consultar máximo em um intervalo de valores. Se a transição é "d dividir v" ou "v múltiplo de d", itere múltiplos em O(V log V) ou divisores em O(n√V).
  - *Exemplo:* LIS com valores até $2 \cdot 10^5$ e $n = 10^6$; subsequência com diferença ≤ D; cadeia de divisibilidade.
  - *Armadilha:* valores repetidos e a regra de estritamente crescente versus não decrescente; compressão sem preservar a ordem; V grande torna o crivo/loop de valores pior que a DP por posição.
- **DP com transição por divisores.** dp[x] = Σ_{d | x, d < x} dp[d] ou max sobre múltiplos: itere crivo em N log N; para subsequência com "cada elemento divide o seguinte" (maior cadeia de divisibilidade), use o crivo de múltiplos em O(V log V) (ou enumere divisores em O(n√V)).
  - *Gatilho:* a transição é "d divide x", "x é múltiplo de d" ou "cada elemento divide o próximo".
  - *Por que funciona:* o crivo visita cada múltiplo uma vez; a soma Σ_{x≤V} V/x é O(V log V), e a ordem crescente de x garante que dp[d] já está pronto quando x é processado.
  - *Como fazer:* para dp[x] somando divisores, inicialize e itere d de 1 a V, propagando dp[d] para 2d, 3d, ...; para máximo, faça dp[múltiplo] = max(dp[múltiplo], dp[d] + ...). Para uma lista esparsa de n valores, enumere os divisores de cada valor em O(√V) ou ordene e use múltiplos.
  - *Exemplo:* maior cadeia em que cada elemento divide o seguinte; soma de dp sobre divisores; contar sequências com divisibilidade.
  - *Armadilha:* "divide" estrito versus não estrito (empates e duplicatas); processar x antes de todos os divisores; se "subsequência" exigir a ordem original, o crivo por valor sozinho não basta (acrescente a posição); V até 10^7 com O(V log V) pode ser pesado, e para n pequeno enumerar divisores é melhor.
- **DP por intervalo de valores (compactada).** Coordenadas enormes mas poucos pontos relevantes: comprima para os pontos de quebra (os próprios a_i; se a restrição envolve distância d, ótimos podem ser $a_i \pm k \cdot d$, até $O(n^2)$ candidatos — use slope trick se necessário) e rode dp[i][j] só sobre eles.
  - *Gatilho:* x pode ser 10^18, mas a função objetivo só muda de comportamento em torno dos a_i ou de deslocamentos por d.
  - *Por que funciona:* entre pontos de quebra, a DP é linear/constante; o ótimo pode ser movido até uma extremidade sem piorar, então basta testar coordenadas candidatas.
  - *Como fazer:* colete os a_i e, se houver restrição de distância, os valores $a_i \pm k \cdot d$ que podem ser ótimos; ordene e remova duplicatas; rode a DP indexada pela posição na lista comprimida. Se o número de candidatos for $O(n^2)$, considere slope trick em vez de materializar todos.
  - *Exemplo:* escolher alturas inteiras com custo $|x - a_i|$ e restrições de diferença; compressão de coordenadas em DP de intervalos.
  - *Armadilha:* comprimir só os a_i quando o ótimo pode estar em $a_i \pm d$; esquecer fronteiras e empates; $O(n^2)$ candidatos podem ser grandes demais — nesse caso, slope trick ou programação linear inteira estruturada.

- **DP com restrição de distância em ordem (dist-k).** "Escolha subconjunto com índices a mais de k de distância" → dp[i] = max(dp[i−1], dp[i−k−1] + v_i) com prefix max, não precisa de estrutura.
  - *Gatilho:* os itens escolhidos precisam ter índices com diferença maior que k (ou pelo menos k), e a ordem é a da sequência.
  - *Por que funciona:* se i é escolhido, o último escolhido pode ser qualquer j ≤ i−k−1; o melhor desses é exatamente o prefixo máximo até i−k−1.
  - *Como fazer:* dp[i] = max(dp[i−1], v_i + best[i−k−1]), onde best[p] = max(dp[0..p]); atualize best com dp[i]. Para "pelo menos k", o índice é i−k; para "mais de k", i−k−1. O(n).
  - *Exemplo:* máxima soma de subconjunto com distância mínima k entre índices; escolher casas com pelo menos k terrenos entre elas.
  - *Armadilha:* confundir "distância ≥ k" com "> k"; usar 0-indexação e errar o deslocamento; se a restrição for sobre valores em vez de índices, a recorrência muda.
- **DP com troca de estado: dp[i] relativo a "melhor até aqui".** Guarde dp[i] = melhor terminando exatamente em i e g[i] = max_{j ≤ i} dp[j]; separar "terminando em i" de "acumulado" simplifica transições com janelas.
  - *Gatilho:* a transição precisa do melhor estado em um prefixo ou janela, mas a resposta também distingue "terminar em i" de "já ter terminado antes".
  - *Por que funciona:* g[i] é um prefixo máximo que pode ser consultado em O(1); manter dp[i] separado preserva a informação de que a sequência termina exatamente em i, que é necessária para estender ou impor distância.
  - *Como fazer:* calcule f[i] terminando em i, depois g[i] = max(g[i−1], f[i]); para uma janela [i−L, i−1], consulte g[i−L]. Se a resposta é "melhor em qualquer lugar", use g[n]; se exige terminar em i, use f[i].
  - *Exemplo:* máxima soma de subarray (f[i] = max(a_i, f[i−1]+a_i), g é o máximo global); subconjunto com distância mínima usando g[i−k−1].
  - *Armadilha:* usar g onde a transição exige terminar exatamente em i; ao rolar a memória, g antigo pode ser sobrescrito antes da consulta; para reconstruir o caminho, g sozinho não guarda a decisão.
- **DP em jogos.** Vitória/derrota: dp[estado] = existe movimento para estado perdedor; com pontuação: dp[l][r] = melhor diferença = max(a_l − dp[l+1][r], a_r − dp[l][r−1]); jogos que mudam quem joga → dp[estado][vez] explícito, ou negamax.
  - *Gatilho:* dois jogadores alternam movimentos, ambos ótimos, e o enunciado pede vitória/derrota, empate ou diferença de pontuação.
  - *Por que funciona:* em um jogo de soma zero, o valor para o jogador da vez é o negativo do valor que o adversário consegue a partir do próximo estado; a recursão minimax captura isso.
  - *Como fazer:* para vitória/derrota, dp[s] = OR sobre movimentos de NOT dp[s']; para pontuação, dp[s] = max sobre movimentos de (ganho imediato − dp[s']); para jogos com turno, inclua a vez no estado. Se houver ciclos, marque empates ou use atrator.
  - *Exemplo:* pegar moedas das pontas de um vetor: dp[l][r] = max(a_l − dp[l+1][r], a_r − dp[l][r−1]); jogo da velha com estado e turno.
  - *Armadilha:* alternar o sinal duas vezes; esquecer estados de empate em jogos cíclicos; a hipótese de que o adversário também joga ótimo é essencial; para pontuação total, não confunda "diferença" com "total do primeiro jogador".
- **Jogo com pilha de moedas / passar vez / limite de lances por turno.** dp[i][m] = melhor diferença com m = máximo que pode pegar agora (clássico "pegar até 2M pedras"); estado = (restante, limite).
  - *Gatilho:* o jogador pode pegar até um limite que cresce com o que o adversário pegou, ou pode passar a vez, e o estado inclui quanto ainda resta.
  - *Por que funciona:* o limite m resume todas as regras de quanto é permitido pegar; o restante i e o limite determinam completamente as opções futuras, então o valor minimax só depende de (i, m).
  - *Como fazer:* com prefixo de pedras, dp[i][m] = max_{1≤x≤min(2m,i)} (S(i, i+x) − dp[i+x][max(m,x)]), onde S é a soma das pedras pegas; base dp[0][m]=0. Se "passar" for permitido, inclua a transição de passe com cuidado para não criar ciclo.
  - *Exemplo:* Stone Game II, em que cada jogador pode pegar 1..2M pedras e M vira max(M, x); "pegar até o dobro do último lance".
  - *Armadilha:* trocar "até 2m" por "exatamente 2m"; o limite pode crescer até i; a transição de passe pode gerar empate/ciclo e precisa de regra explícita; use prefixos para não somar o intervalo a cada transição.

- **Jogo com estratégias probabilísticas / mistas.** Quando os dois escolhem simultaneamente, resolva o jogo de soma zero por LP/minimax em matriz pequena (2×2 fórmula fechada); raro, mas é onde DP "normal" falha.
  - *Gatilho:* os jogadores escolhem ao mesmo tempo (não observam a escolha do outro) e o pagamento é uma matriz; nenhuma estratégia pura é claramente ótima.
  - *Por que funciona:* pelo teorema minimax, todo jogo finito de soma zero tem um valor em estratégias mistas; o problema vira um LP: o jogador linha maximiza o pior pagamento esperado.
  - *Como fazer:* para matriz pequena, resolva o LP ou, no caso 2×2, iguale os pagamentos esperados das duas colunas para achar a probabilidade p da linha; para matrizes maiores, use programação linear ou iteração de fictícios. Verifique se o valor está no envelope das estratégias puras.
  - *Exemplo:* par ou ímpar (matching pennies); pedra-papel-tesoura; escolher rota secreta contra um adversário que também escolhe.
  - *Armadilha:* estratégia pura pode não existir; a fórmula 2×2 divide por zero no caso degenerado (por exemplo, com dominância) — trate-o separadamente; precisão de ponto flutuante em LP; e não aplique isso quando os jogadores observam a jogada anterior (aí é jogo sequencial/DP).
- **DP em ciclo / circular com custo de fechamento.** "Costurar" início e fim: fixe o valor/estado do primeiro elemento e rode a DP linear exigindo estado final compatível (|S| execuções, ou matriz de transição e traço: Σ diagonal de produto de matrizes).
  - *Gatilho:* os elementos formam um ciclo e a restrição de fechamento liga o último ao primeiro (coloração de ciclo, caminho circular, particionamento circular).
  - *Por que funciona:* quebrar o ciclo em uma linha deixa apenas a dependência entre o primeiro e o último; fixar o estado inicial remove essa dependência e a DP linear verifica a compatibilidade no fim.
  - *Como fazer:* para cada estado s do primeiro elemento, rode a DP linear com dp[1][s]=0 e, ao final, aceite apenas estados finais compatíveis com s; o custo é |S| vezes a DP linear. Se a transição é homogênea, use matriz de transição e calcule o traço de M^n.
  - *Exemplo:* colorir um ciclo com k cores sem cores iguais adjacentes; escolher casas em um círculo sem adjacentes; TSP em anel com estado pequeno.
  - *Armadilha:* esquecer a aresta de fechamento; contar a mesma solução uma vez para cada rotação (se o problema não distingue rotações, divida ou use Burnside); o número de estados iniciais pode ser grande.
- **Traço de produto de matrizes para contagem circular.** Número de colorações de ciclo de n vértices com regra de vizinhança M (matriz k×k) = tr(M^n); para problemas "string cíclica sem padrão" calcule traço do autômato.
  - *Gatilho:* contagem de sequências cíclicas com uma regra local de adjacência, com n possivelmente grande e k (estados) pequeno.
  - *Por que funciona:* (M^n)[i][i] conta passeios fechados de comprimento n que começam e terminam no estado i; a soma das entradas diagonais é exatamente escolher o estado inicial e fechar o ciclo.
  - *Como fazer:* monte M[i][j] = 1 se a transição do estado i para j é permitida; calcule M^n por exponenciação rápida e some a diagonal. Para autômatos de padrões, os estados são os nós do KMP/Aho-Corasick.
  - *Exemplo:* número de colorações próprias de um ciclo com k cores é tr(M^n) para M = J − I; strings cíclicas que evitam um padrão.
  - *Armadilha:* usar M^{n−1} em vez de M^n (o ciclo tem n arestas); n=1 e n=2 têm interpretações especiais (laço e aresta dupla); em módulo, não divida por n sem justificar; matrizes direcionadas exigem transição transposta conforme a convenção.
- **DP com bitset (otimização).** dp[i] |= dp[j] para j ∈ janela: troque laços internos por operações de 64 bits; para alcançabilidade em DAG/transitivo (n ≤ $5 \cdot 10^4$ em ~n·m/64) mantenha bitset de alcançáveis por vértice em ordem reversa topológica.
  - *Gatilho:* a DP é booleana (alcançável/possível) e as transições são uniões de conjuntos de estados, com n até dezenas de milhares.
  - *Por que funciona:* cada palavra de 64 bits processa 64 estados de uma vez; a união de conjuntos vira OR bit a bit, reduzindo o fator 64.
  - *Como fazer:* represente o conjunto de estados alcançáveis de i como bitset; para cada transição i → j, faça reach[i] |= reach[j] na ordem topológica reversa (para descendentes) ou na ordem direta (para ancestrais). Para uma janela de j's, combine os bitsets com OR; se a janela for grande, use segment tree de bitsets ou uma sparse table (OR é idempotente).
  - *Exemplo:* fecho transitivo de um DAG; alcançabilidade em autômato; subconjuntos de somas possíveis.
  - *Armadilha:* memória $O(n^2/64)$ ainda pode ser grande; a ordem topológica é obrigatória para não usar informação incompleta; em linguagens sem bitset nativo, use arrays de uint64_t e cuidado com bits fora do intervalo.

- **Bitset em DP de caminhos em grade com "alcançável?"** b = reach[i−1] & livre[i]; repita b |= (b << 1) & livre[i] até estabilizar; reach[i] = b, cada passada em O(m/64); problemas de "existe caminho/escada" com m ≤ 10^5.
  - *Gatilho:* grade larga (m grande) e baixa (n pequeno), com obstáculos, e a pergunta é apenas se existe caminho com movimentos monotônicos.
  - *Por que funciona:* cada linha é um bitset de colunas; deslocar para a esquerda/direita propaga alcançabilidade horizontalmente, e o AND com livre remove células bloqueadas. Repetir até o ponto fixo fecha o fecho horizontal da linha.
  - *Como fazer:* b = reach[i−1] & livre[i] (vindo de cima); repita b |= (b << 1) & livre[i] (e (b >> 1) se andar para os dois lados) até b não mudar; reach[i] = b. Cada passada custa O(m/64); no pior caso são até m passadas por linha, então estime o total.
  - *Exemplo:* existe caminho de cima para baixo em uma grade com células livres; escada com movimentos direita/baixo; conectividade em faixa estreita.
  - *Armadilha:* se os movimentos não são monotônicos, precisa dos dois deslocamentos e o ponto fixo pode exigir mais passadas; máscara de bits para não sair da largura; a presença de caminho não conta caminhos — para contagem, bitset não basta.
- **Contagem de caminhos em grade com obstáculos (n pequeno, obstáculos poucos).** Ordene obstáculos e dp[k] = caminhos até k sem passar por outros = C(até k) − Σ dp[j]·C(j → k): O(K^2) em vez de O(nm) quando grade enorme.
  - *Gatilho:* grade enorme, poucos obstáculos (K pequeno), e quer contar caminhos monotônicos do canto superior esquerdo ao inferior direito evitando os obstáculos.
  - *Por que funciona:* todo caminho que chega ao obstáculo k ou não passou por nenhum obstáculo anterior (contado por dp[k]) ou passou pelo primeiro obstáculo j; subtrair dp[j]·C(j→k) elimina os caminhos ruins sem contar duas vezes, se os obstáculos são processados em ordem topológica.
  - *Como fazer:* inclua a origem e o destino como pontos especiais; ordene por (linha, coluna); para cada k, total = C((r_k−r_0)+(c_k−c_0), r_k−r_0) e subtraia Σ_{j<k, j alcança k} dp[j]·C((r_k−r_j)+(c_k−c_j), r_k−r_j); use fatoriais e inversos mod p.
  - *Exemplo:* contar caminhos em uma grade 10^9 × 10^9 com 2000 células bloqueadas; contar caminhos que evitam uma lista de pontos.
  - *Armadilha:* obstáculos fora da ordem parcial não se alcançam; duplicatas precisam ser removidas; o destino pode coincidir com um obstáculo; a fórmula modular exige fatoriais até a maior distância, não até n·m.
- **Lagrange / interpolação em DP.** Se dp(n) é polinômio em n de grau d (ex.: soma de potências, contagem de caminhos com passos limitados, soma por camada de uma DP polinomial), calcule d+1 valores por brute force e interpole (Lagrange O(d) com fatoriais para pontos consecutivos).
  - *Gatilho:* n é enorme, mas a resposta é um polinômio de grau pequeno em n, e você consegue calcular os primeiros valores com uma DP simples.
  - *Por que funciona:* um polinômio de grau d fica determinado por d+1 pontos; se os pontos são consecutivos, a interpolação de Lagrange pode ser avaliada em O(d) com fatoriais e produtos prefixo/sufixo.
  - *Como fazer:* determine ou estime o grau d; calcule f(0),...,f(d) com a DP; avalie em n usando L(n) = Σ y_i · Π_{j≠i}(n−j)/(i−j), com denominadores fatoriais e sinal. Para n ≤ d, basta ler a tabela.
  - *Exemplo:* Σ_{i=1}^{n} i^k é polinômio de grau k+1; número de caminhos com um número fixo de passos em uma direção; soma de uma DP cujo estado é camada e cujo valor é polinomial.
  - *Armadilha:* o grau pode ser maior do que o esperado; a fórmula só vale depois de um prefixo inicial (algumas recorrências são quase-polinomiais por partes); em módulo, os pontos precisam ser distintos e d < p; não confunda polinômio em n com exponencial em n.
- **Detectar polinômio em n.** Diferenças finitas da sequência: se as diferenças de ordem d+1 zeram, é polinômio de grau d → Lagrange. Aparece em "número de formas de colorir/distribuir com k cores" (polinômio cromático) e somas Σ i^k.
  - *Gatilho:* você tem uma sequência f(0), f(1), ... e suspeita que ela seja polinomial, mas não sabe o grau.
  - *Por que funciona:* a (d+1)-ésima diferença finita de um polinômio de grau d é identicamente zero; reciprocamente, diferenças de ordem d+1 nulas em pontos suficientes indicam grau ≤ d.
  - *Como fazer:* monte a tabela de diferenças; se a coluna de ordem d+1 zera, interpole com grau d. Se as diferenças não zeram mas têm período, trate como quase-polinômio; se obedecem a uma recorrência linear, use Berlekamp–Massey.
  - *Exemplo:* número de k-colorações próprias de um grafo fixo é polinômio em k (polinômio cromático); Σ_{i=1}^n i^3 é grau 4; contagem de colorações de uma grade com largura fixa.
  - *Armadilha:* uma sequência pode coincidir com um polinômio nos primeiros termos e depois desviar; use mais termos que o grau; quase-polinômios têm período e não são um único polinômio; coeficientes racionais exigem aritmética modular ou frações.

- **Estado cresce linearmente mas n é enorme.** Se dp[n][j] for polinomial em n para cada j fixo, ou a sequência em n se comporta como quasi-polinômio de período fixo, calcule primeiros termos e extrapole (BM, Lagrange ou período).
  - *Gatilho:* a DP tem uma dimensão n gigante e outra pequena, e o valor para j fixo parece seguir um padrão em n.
  - *Por que funciona:* para cada j fixo, a recorrência pode ter posto finito ou grau fixo; a sequência é gerada por uma recorrência linear ou por um polinômio/quase-polinômio, e os primeiros termos determinam o resto.
  - *Como fazer:* calcule termos para n = 0..N com N algumas dezenas; teste diferenças finitas (Lagrange), detecte período (quase-polinômio) ou use Berlekamp–Massey para achar a recorrência linear e avance com Kitamasa/matriz. Confirme com termos extras.
  - *Exemplo:* contar tilings de uma faixa 2×n por recorrência de Fibonacci; número de colorações de uma faixa com largura fixa; somas de potências por camada.
  - *Armadilha:* BM precisa de pelo menos 2L termos para uma recorrência de ordem L; o período do quase-polinômio pode ser o lcm de vários; extrapolar com termos insuficientes parece funcionar e falha depois.
- **Partições ordenadas/composições com restrição "partes em [a,b]".** dp[n] = Σ_{k=a}^{b} dp[n−k] → prefix sums O(n); com restrição em quantas partes também use GF 1D (coeficiente por polinômio com potência) e binomial.
  - *Gatilho:* contar formas de escrever n como soma ordenada de partes em um intervalo, possivelmente com um número fixo de partes.
  - *Por que funciona:* a ordem das partes é dada pela posição na composição; escolher a última parte k reduz o problema a n−k, e a soma de uma janela de dp é uma diferença de prefixos.
  - *Como fazer:* dp[0]=1; dp[n] = P[n−a] − P[n−b−1], onde P é o prefixo de dp (com índices negativos valendo 0). Para exatamente r partes, o número é o coeficiente de x^n em (x^a + ... + x^b)^r, que pode ser calculado por inclusão-exclusão/binomial ou DP em r.
  - *Exemplo:* composições de n com partes 1..3; composições em exatamente k partes positivas (estrelas e barras).
  - *Armadilha:* composições ordenadas não são partições (a ordem importa); a=0 permite partes vazias e muda a contagem; a janela precisa ser truncada em n; para exatamente k partes, o grau e o coeficiente inicial importam.
- **Contagem de sequências com soma de elementos fixa e limites superiores** → IE sobre limites estourados (estrelas e barras com C(n − Σ(b_i+1) + k − 1, k−1)), em O(2^k) ou DP sobre IE em O(kn) se limites iguais.
  - *Gatilho:* contar soluções não negativas x_1+...+x_k = n com 0 ≤ x_i ≤ b_i (ou versão com limites inferiores).
  - *Por que funciona:* sem os limites superiores, estrelas e barras dá C(n+k−1,k−1); para cada subconjunto T de variáveis que "estouram", substitua x_i por y_i + (b_i+1), reduzindo a soma; a inclusão-exclusão alterna os sinais.
  - *Como fazer:* some sobre T ⊆ {1..k}: (−1)^{|T|} C(n − Σ_{i∈T}(b_i+1) + k − 1, k−1), com binomial 0 se o topo for negativo ou menor que k−1. Se todos os b_i são iguais, a DP de IE em O(kn) evita 2^k.
  - *Exemplo:* número de soluções de x_1+x_2+x_3=10 com x_i ≤ 4; somas de dados com faces limitadas.
  - *Armadilha:* sinal trocado; limites inferiores exigem outra substituição; o binomial precisa ser zero quando a soma restante é negativa; use inteiros de 64 bits ou módulo, não double.
- **DP + IE: contar objetos sem padrão proibido.** Calcule f[S] = # que contêm pelo menos esses defeitos e combine com sinais; se defeitos formam estrutura de segmentos, a IE vira DP linear dp[i] = soma com sinal sobre último defeito (muito comum em "sem k iguais consecutivos").
  - *Gatilho:* contar sequências/estruturas que evitam um conjunto de padrões ou blocos proibidos, e as interseções de defeitos são fáceis de contar.
  - *Por que funciona:* a inclusão-exclusão sobre o conjunto de defeitos presentes converte "nenhum defeito" em uma soma alternada; se os defeitos se organizam em segmentos, a soma pode ser agregada por um DP em vez de 2^m subconjuntos.
  - *Como fazer:* defina f[S] = número de objetos que contêm todos os defeitos de S; some (−1)^{|S|} f[S]. Para defeitos lineares, defina dp[i] = soma com sinal dos objetos cujo último defeito termina em i e transicione a partir do defeito anterior; use prefixos para O(n).
  - *Exemplo:* strings sem k caracteres iguais consecutivos; sequências sem um conjunto de substrings proibidas; colorações sem blocos monocromáticos.
  - *Armadilha:* as interseções de defeitos podem ser incompatíveis (f[S]=0) e a estrutura de segmentos precisa ser realmente linear; não confunda "pelo menos um" com "exatamente"; o sinal e o caso vazio precisam ser testados.

- **Inclusão-exclusão por segmentos vs. DP de complemento.** Para "nenhum bloco de k seguidos iguais": dp[i] = (m−1)·(dp[i−1] + dp[i−2] ...) ou conte o complemento com defeito marcado por IE linear; escolha o que dá O(n).
  - *Gatilho:* colorações/sequências sobre um alfabeto de tamanho m que não podem ter k símbolos iguais consecutivos.
  - *Por que funciona:* classificar pela posição do último bloco proibido dá uma recorrência de janela; alternativamente, a IE sobre o bloco escolhido como "primeiro defeito" produz a mesma recorrência com sinais.
  - *Como fazer:* para i ≥ k, dp[i] = (m−1)(dp[i−1]+...+dp[i−k+1]), pois a última posição escolhe um símbolo diferente do anterior e fecha uma sequência de comprimento de bloco < k; para i < k, dp[i]=m^i. Use prefix sums para O(n). A versão IE substitui o fator (m−1) por sinais e evita casos base separados.
  - *Exemplo:* colorir uma fita de n posições com 3 cores sem 3 iguais consecutivas; strings binárias sem 4 zeros seguidos.
  - *Armadilha:* aplicar a recorrência para i < k sem o caso base m^i; k=1 (nenhuma sequência não vazia); m=1 com n ≥ k; a janela de tamanho k−1 precisa ser somada corretamente.
- **DP sobre soma de dígitos/valores pequenos com truque de balanço.** Sequências com soma prefixos limitada (caminhos que não cruzam): use reflexão para fechada; se a barreira é móvel ou várias, DP O(n·alt) com limite de altura (altura ≤ √n ou ≤ K).
  - *Gatilho:* cada passo soma +1/−1 (ou um valor pequeno), e a soma prefixada não pode cruzar uma barreira; a altura máxima é limitada.
  - *Por que funciona:* o princípio da reflexão conta caminhos que tocam uma barreira refletindo o trecho após o primeiro toque; com barreiras móveis ou múltiplas, a reflexão simples não fecha e a DP por altura modela diretamente a restrição.
  - *Como fazer:* para uma barreira fixa, conte todos os caminhos e subtraia os refletidos (Catalan/ballot); para barreira móvel ou várias, dp[i][h] com h na faixa permitida, transição para h±1, e corte quando h sai do limite. A altura máxima pode ser √n em problemas de soma 1+...+k.
  - *Exemplo:* parênteses balanceados (nunca negativos); caminhos de ±1 que ficam entre 0 e K; passeios com soma de prefixos ≤ S.
  - *Armadilha:* a reflexão exige passos simétricos e barreira reta; com barreira móvel, reflita a barreira, não o caminho; altitudes negativas/limites inclusivos; a altura pode ser O(n), não √n, se os passos não forem distintos.
- **Altura máxima do passeio limitada por √ / estados "|x| ≤ √n".** Em problemas de soma 1+2+...+k ≤ n o nº de passos é ≤ √(2n): dp[i][j] com i = soma e j ≤ √n = último valor. Também em partições em partes distintas.
  - *Gatilho:* os passos/partes são positivos e distintos (ou crescentes), de modo que a soma cresce pelo menos 1+2+...+k.
  - *Por que funciona:* se há k partes positivas distintas, a menor soma possível é k(k+1)/2; logo k ≤ √(2n), limitando a dimensão do último valor.
  - *Como fazer:* dp[s][v] = número/valor ótimo para soma s terminando no valor v, com v ≤ √(2s); transicione para valores maiores (partes distintas) ou some prefixos. O total é O(n√n).
  - *Exemplo:* partições de n em partes distintas; sequências estritamente crescentes de inteiros positivos com soma ≤ n; o número de partes distintas de uma partição é O(√n).
  - *Armadilha:* se as partes podem repetir, o número de partes não é limitado por √n; "distintas" e "não decrescentes" mudam a transição; a cota é assintótica, use um limite seguro e não estoure o vetor.
- **DP com somas de 1+...+k ≤ n (estado "último valor")** → dp[soma][último] com último ≤ √(2n); transição por prefix sum sobre último, O(n√n).
  - *Gatilho:* contar/otimizar partições ou sequências crescentes em que o custo mínimo cresce com o número de partes.
  - *Por que funciona:* o último valor limita o próximo (se as partes são distintas) e o número de valores possíveis é O(√n); a soma sobre últimos menores pode ser acumulada em prefixos.
  - *Como fazer:* dp[s][v] = Σ_{u<v} dp[s−v][u] (partes distintas) ou Σ_{u≤v} (não decrescentes), calculado com prefixo sobre u; responda somando dp[n][v]. Para otimização, troque a soma por min/max.
  - *Exemplo:* número de partições de n em partes distintas; número de composições crescentes; menor número de partes com soma n e valores distintos.
  - *Armadilha:* a ordem das partes é fixa (crescente) para evitar contagem dupla; o índice do último valor precisa ir até √(2n); a soma dos prefixos pode ser atualizada na ordem certa; para partes repetidas, o limite √n desaparece.

- **Pareto / "listas não dominadas" para DP bicritério.** dp[v] = conjunto de pares (custo, tempo) não dominados; se o tamanho ficar ≤ ~√ ou log, é viável; caso contrário fixe um critério por binária (resposta) e DP no outro.
  - *Gatilho:* há dois recursos/custos e nenhum pode ser reduzido a um único escalar; a resposta é "existe com custo ≤ C e tempo ≤ T".
  - *Por que funciona:* um par (c1,t1) domina (c2,t2) se c1≤c2 e t1≤t2 (com pelo menos uma estrita); o dominado nunca será melhor em nenhuma combinação futura monotônica.
  - *Como fazer:* mantenha a fronteira ordenada por custo crescente e tempo decrescente; ao inserir um par, remova os dominados e descarte o novo se for dominado. Combine fronteiras por merge. Se a fronteira explodir, binária em um critério e DP no outro.
  - *Exemplo:* caminho com orçamento e prazo; mochila com dois pesos; escolher projetos com custo e duração.
  - *Armadilha:* a fronteira pode ser exponencial; a dominância só vale se as transições futuras são monotônicas nos dois critérios; empates precisam de uma regra consistente; binária no critério errado pode perder a factibilidade.
- **Binária na resposta + DP monótona.** "Menor X tal que existe divisão em ≤ k partes com custo ≤ X" → checagem greedy O(n) ou DP em O(n log n); costuma ser mais rápida e simples que DP de camada k.
  - *Gatilho:* a resposta é um limiar mínimo/máximo e a factibilidade "custo ≤ X" é monótona em X.
  - *Por que funciona:* se uma solução é viável com X, também é com X' > X; a binária reduz o problema a decidir factibilidade para um X fixo, que muitas vezes é guloso ou uma DP de uma camada.
  - *Como fazer:* defina check(X) = é possível dividir/atribuir com custo máximo ≤ X usando ≤ k partes; implemente check em O(n) (guloso) ou O(n log n) (DP/estrutura); busque o menor X no intervalo [0, soma] ou [max, soma].
  - *Exemplo:* dividir array em ≤ k subarrays com maior soma ≤ X (guloso); alocar tarefas em k máquinas com carga ≤ X; menor capacidade de caminhão.
  - *Armadilha:* com números negativos o guloso de "fechar a parte quando estourar" não é válido; a factibilidade precisa ser realmente monótona; cuidado com "≤ k" versus "exatamente k" e com o limite inferior da binária.
- **Dividir em k segmentos minimizando o maior (ou soma).** Max: binária + guloso (exige a_i ≥ 0; com negativos use DP); soma de custos convexos: Aliens ou D&C; "partições de array em k blocos custo soma-quadrado" → CHT/D&C/Knuth conforme formato.
  - *Gatilho:* particionar uma sequência em k blocos contíguos, minimizando o maior bloco, a soma dos custos ou uma soma de quadrados.
  - *Por que funciona:* para o máximo, a factibilidade é monótona e o guloso ótimo com não negativos; para custos aditivos, a DP de partição tem a forma dp[t][i] = min_j dp[t−1][j] + C(j+1,i), otimizável conforme a quadrangle/concavidade.
  - *Como fazer:* máximo: binária em X e conte blocos gulosamente, O(n log S); soma de custos arbitrários: dp O(kn^2), ou Aliens se a função em k for convexa; custo quadrático: CHT/D&C/Knuth se as condições valerem.
  - *Exemplo:* Split Array Largest Sum; partitionar em k partes com soma dos quadrados mínima; agrupar arquivos em k discos.
  - *Armadilha:* negativos quebram o guloso do máximo; a DP de k camadas pode ser O(kn^2); D&C optimization exige opt monotônica e custo quadrangle; Knuth exige condições mais fortes; "k blocos" não vazios.
- **DP com técnica "pular estados inúteis" (map/memo).** Se n é 10^{18} e a DP só visita O(log^2) estados distintos (por divisões ⌊n/2⌋, ⌊n/3⌋), memoize em map: a recursão "min operações para reduzir n a 0 por /2, /3, −1" visita Θ(n) estados com a transição −1 literal; subtraia de uma vez até o múltiplo (n%2, n%3) para ficar O(log^2).
  - *Gatilho:* n enorme, operações de divisão por 2/3 e subtração, e a recursão ingênua de −1 teria n estados.
  - *Por que funciona:* os quocientes ⌊n/2^a3^b⌋ são poucos (O(log^2 n)); em vez de simular cada −1, pule direto para o múltiplo de 2 ou 3 e pague o resto de uma vez.
  - *Como fazer:* f(n) = 0 para n=0 (ou 1, conforme a definição); f(n) = min(1 + n%2 + f(n/2), 1 + n%3 + f(n/3)) quando se pode subtrair até o múltiplo; memoize em unordered_map. Se +1 também for permitido, inclua as opções de arredondar para cima.
  - *Exemplo:* reduzir n a 1 com /2, /3 e −1; menor número de operações para atingir 0; variantes com +1.
  - *Armadilha:* a transição −1 literal é O(n); o caso base e a direção do arredondamento mudam a fórmula; para n=0 ou 1, evite recursão infinita; o número de estados O(log^2) depende de pular direto, não de memoizar a recursão ingênua.

- **Floor-DP: estados ⌊n/k⌋.** Somas multiplicativas (Dirichlet) dp[⌊n/k⌋] com 2√n estados (Min_25, Du Jiao): tabelas indexadas por valores de ⌊n/i⌋ em O(n^{3/4}) ou O(n^{2/3}). Problemas "soma de f sobre 1..n" com f multiplicativa e n até 10^{10}–10^{12}.
  - *Gatilho:* somar uma função multiplicativa até n enorme, e a recorrência envolve somas em blocos de valores ⌊n/i⌋.
  - *Por que funciona:* ⌊n/i⌋ assume apenas O(√n) valores distintos; agrupar intervalos com o mesmo quociente permite somar blocos inteiros e reduzir a complexidade.
  - *Como fazer:* indexe os valores v = ⌊n/i⌋ (os ≤√n e os quocientes grandes); use a recorrência de Du Jiao para S(n)=Σ f(i) com prefixo das convoluções, ou Min_25 para funções multiplicativas; memoize por v. Complexidade típica O(n^{3/4}) ou O(n^{2/3}) conforme a implementação.
  - *Exemplo:* Σ μ(i), Σ φ(i), Σ d(i) até 10^{12}; soma de funções multiplicativas em problemas de contagem.
  - *Armadilha:* distinguir os dois índices (i pequeno e n/i pequeno); a memoização precisa da chave v, não i; as constantes e o pré-crivo afetam o expoente; não use para f não multiplicativa sem adaptação.
- **Crivo Dirichlet / DP sobre primos.** dp sobre primos em ordem (soma de f(p^k) com mínimo p do fator): Min_25 em duas fases; "contar números ≤ N cujo menor fator primo é ≥ p" é o estado S(n, j).
  - *Gatilho:* somar/contar f(n) por fatoração, com f multiplicativa e N grande (10^{10}–10^{12}), especialmente quando f depende do menor fator primo.
  - *Por que funciona:* processar primos em ordem garante que cada número seja decomposto uma vez; o estado S(n,j) restringe os fatores a primos ≥ p_j, e a transição separa p_j^k do resto.
  - *Como fazer:* na fase 1, calcule a soma sobre primos ≤ N (crivo até √N e crivo de mínimos); na fase 2, recorra S(n,j) = S(n,j+1) + Σ_{k≥1, p_j^k≤n} f(p_j^k)·(1 + S(n/p_j^k, j+1)), com bases para $p_j^2 > n$. Min_25 usa duas tabelas (g e S).
  - *Exemplo:* contar primos ≤ N; somar φ(n) ou μ(n) até N; contar números com menor fator primo em um intervalo.
  - *Armadilha:* confundir "primos ≤ n" com "menor fator ≥ p_j"; $p_j^2 > n$ encerra a recursão; o expoente k e as potências podem estourar; a fase de primos precisa de todos os primos até √N.
- **DP sobre fatoração (números altamente compostos).** Número com ≤ 10^18 tem ≤ 15 primos distintos e expoentes decrescentes: DFS gerando "números altamente compostos" com expoentes não-crescentes é muito pequeno (~10^5) e resolve "maior número de divisores ≤ N".
  - *Gatilho:* maximizar o número de divisores (ou minimizar o número para um dado número de divisores) com N até 10^18.
  - *Por que funciona:* se um número tem expoentes fora de ordem, trocar expoentes entre primos menores e maiores não aumenta o produto mas não diminui o número de divisores; para o representante mínimo, os expoentes podem ser tomados não crescentes. O produto dos 15 primeiros primos já é ~6·10^17, então há no máximo 15 fatores distintos.
  - *Como fazer:* liste os primeiros ~16 primos; faça DFS com (índice do primo, expoente máximo permitido, valor, número de divisores), exigindo que o próximo expoente seja ≤ o atual; pode também podar pelo limite N. Guarde o melhor (maior d(n), e o menor n em empate).
  - *Exemplo:* maior número de divisores ≤ 10^18; menor n com exatamente 1000 divisores; números altamente compostos.
  - *Armadilha:* a ordem não-crescente vale para o representante mínimo, não para todo número; use o limite de 15 primos com margem; overflow no produto; em empates, o problema pode pedir o menor n; a poda precisa considerar o número de divisores restante.
- **Divisão em blocos (√) para DP de transição lenta.** dp[i] depende de dp[i − k·b] para b na lista: para saltos pequenos (≤ √n) mantenha tabela por resíduo; para grandes percorra diretamente. Mochila "a cada segundo você anda p_i": heavy/light por p_i.
  - *Gatilho:* a transição soma/otimiza sobre múltiplos de vários saltos b, e fazer todos os múltiplos para todo i é $O(n^2)$ ou pior.
  - *Por que funciona:* para b pequeno, os índices i−k·b compartilham o mesmo resíduo módulo b; um acumulador por (b, resíduo) dá a soma de todos os múltiplos anteriores em O(1) amortizado. Para b grande, o número de múltiplos é pequeno.
  - *Como fazer:* escolha B ≈ √n; para b < B, mantenha acc[b][r] = Σ dp[i−k·b] com i ≡ r (mod b) e atualize após calcular dp[i]; para b ≥ B, itere k = 1,2,... enquanto i−k·b ≥ 0. Total O(n√n) (vezes o número de b's, se aplicável).
  - *Exemplo:* contar formas de subir com saltos variáveis; mochila com itens que andam p_i por segundo; DP de partições com partes de tamanhos variados.
  - *Armadilha:* b=0 não tem múltiplos positivos; a ordem de atualização do acumulador (antes/depois de dp[i]) evita contar o próprio i; para max-plus, o acumulador não pode ser "removido" se a janela for limitada; o threshold B precisa equilibrar memória $O(B^2)$ e tempo.

- **Dp[i] += dp[i − b_i·k] (salto b_i, qualquer múltiplo).** b_i ≥ √n → pule direto (≤ √n saltos); b_i < √n → arrays extras por (b, i mod b) com atualização em O(1) amortizada; total O(n√n).
  - *Gatilho:* para cada posição i há um salto b_i e a transição soma contribuições de todos os múltiplos anteriores desse salto.
  - *Por que funciona:* o conjunto {i−b, i−2b, ...} é uma progressão aritmética; para b pequeno, um acumulador por resíduo resume a progressão; para b grande, a progressão é curta.
  - *Como fazer:* escolha B≈√n; se b_i < B, dp[i] += acc[b_i][i mod b_i] e depois acc[b_i][i mod b_i] += dp[i]; se b_i ≥ B, some dp[i−k·b_i] diretamente para k≥1. Atualize o acumulador na ordem que não inclua dp[i] antes da leitura.
  - *Exemplo:* contar sequências de saltos em que de i se pode ir a i+b_i·k; DP de partições com partes múltiplas de b_i.
  - *Armadilha:* b_i=0; k deve começar em 1 (não conte dp[i] nele mesmo); se b_i muda com i, o acumulador por b continua válido, mas a ordem de atualização precisa ser cuidadosa; em max-plus, a soma vira max e a "remoção" não é possível.
- **Small-to-large em DP de árvore.** Quando dp[v] é uma estrutura (map/set) e o merge é inserir elementos da menor na maior: O(n log n) (ou $\log^2$); para dp em que índice é profundidade, use "long-path decomposition" (DP de profundidade em O(n) com array deslocado: o filho mais profundo reaproveita o array do pai).
  - *Gatilho:* cada vértice tem um map/set de estados e o merge de filhos é uma união/inserção; ou o estado é indexado por profundidade.
  - *Por que funciona:* inserir a estrutura menor na maior garante que cada elemento mude de estrutura O(log n) vezes; para profundidade, o filho mais alto pode compartilhar o array do pai com um deslocamento, e os demais são mesclados.
  - *Como fazer:* escolha o filho com maior estrutura, faça swap do map/set para o pai e insira os elementos dos outros filhos; para profundidade, use long-path: pai e filho pesado compartilham o vetor com offset +1, e os filhos leves são copiados. O(n log n) para map; O(n) para long-path com operações O(1).
  - *Exemplo:* contar cores distintas na subárvore; maior profundidade com propriedade; número de vértices a distância d.
  - *Armadilha:* fazer swap sem preservar o ponteiro usado pelo pai; mapas com muitas cópias; a análise O(n log n) não vale se o merge não for por inserção unitária; long-path não é o mesmo que DSU on tree quando a operação de merge é mais cara.
- **Long-path decomposition (DP por profundidade).** dp[v][d] = f(subárvore, profundidade d): o filho de maior altura herda o vetor (ponteiro + 1) e os outros são mesclados elemento a elemento; custo total O(n), em vez de O(n log n) do small-to-large. Para "contar vértices a distância d" ou "pares com dist = k".
  - *Gatilho:* o estado é indexado pela profundidade relativa e a resposta depende de distâncias dentro da subárvore.
  - *Por que funciona:* cada vetor de profundidade é usado por exatamente um caminho pesado; ao dar ao filho mais alto o mesmo buffer deslocado, não se copia o vetor. Cada vértice é inserido uma vez, e os filhos leves são mesclados em tempo proporcional ao seu tamanho.
  - *Como fazer:* calcule a altura de cada subárvore, escolha o filho mais alto, aloque um buffer de tamanho altura(v) e passe buffer+1 para esse filho; para os outros, mescle posição a posição. Some as contribuições de pares durante o merge para não contar duas vezes.
  - *Exemplo:* contar pares de vértices a distância exatamente k; somar valores por profundidade; maior caminho que passa por v.
  - *Armadilha:* o buffer precisa ter espaço para a altura e o offset; se a operação de merge não for O(1) por posição, o total pode voltar a $O(n^2)$; a contribuição de pares deve ser computada antes de sobrescrever o vetor.
- **DP de profundidade com soma ≤ ... na árvore.** Quando a restrição é sobre profundidade/distância em subárvore, não use knapsack em tamanho (n^2) — use long-path (altura) onde o estado tem tamanho = altura, não = nº de vértices.
  - *Gatilho:* a DP de árvore precisa de "até d níveis" ou "distância ≤ d", e fazer knapsack pelo tamanho da subárvore dá $O(n^2)$.
  - *Por que funciona:* o número de profundidades distintas é a altura, que pode ser muito menor que o tamanho; a estrutura de long-path compartilha os vetores e o custo total fica O(n) para operações de profundidade.
  - *Como fazer:* defina dp[v][d] para profundidade relativa até altura(v); herde o vetor do filho mais alto com offset e mescle os outros; se a restrição é distância entre vértices, combine pares de filhos antes de mesclar.
  - *Exemplo:* contar vértices a distância ≤ K de cada nó; soma de valores por nível; diâmetro com restrição de altura.
  - *Armadilha:* K pode ser maior que a altura e não precisa de estado até K; usar tamanho da subárvore em vez de altura reintroduz $O(n^2)$; a resposta para "distância exata" precisa de um vetor de tamanho K+1, não só altura.

- **Contagem de matchings / independent sets em árvore com polinômio.** dp[v] = polinômio em x (tamanho do conjunto); merge = multiplicação de polinômios por NTT com D&C por HLD ($O(n \log^2 n)$): "contar conjuntos independentes de tamanho k".
  - *Gatilho:* contar conjuntos independentes/matchings por cardinalidade em uma árvore e n é grande (10^5), com todos os k pedidos.
  - *Por que funciona:* a DP de árvore combina filhos por convolução; o produto de polinômios representa a soma das cardinalidades. HLD e divisão e conquista mantêm o custo total em $O(n \log^2 n)$ via NTT.
  - *Como fazer:* dp[v][0/1] = polinômios para v não escolhido/escolhido; para cada filho, multiplique os polinômios correspondentes e some; use NTT para multiplicações grandes e uma ordem de merge balanceada (D&C/HLD) para evitar $O(n^2)$.
  - *Exemplo:* contar conjuntos independentes de tamanho k em uma árvore; contar matchings de tamanho k; distribuir k marcadores com restrições locais.
  - *Armadilha:* o produto de muitos polinômios precisa ser balanceado; NTT exige módulo com raiz primitiva e tamanho suficiente; a resposta pode ter grau até n; a poda por k pode reduzir o custo.
- **DP em árvore com polinômio e HLD + D&C (matriz).** Updates pontuais em dp de árvore (ex.: conjunto independente máximo com mudanças de peso): HLD + segtree de matrizes 2×2 (semi-anel max-plus) → $O(\log^2)$ por update.
  - *Gatilho:* DP de árvore com atualizações de peso em vértices e consultas do valor ótimo global, sem reconstruir a árvore.
  - *Por que funciona:* a DP de uma cadeia pesada é uma composição de transições lineares em max-plus; a contribuição dos filhos leves entra como constante; uma segtree sobre a cadeia multiplica matrizes em O(log n), e HLD sobe O(log n) cadeias.
  - *Como fazer:* para cada vértice, matriz 2×2 com estados escolhido/não escolhido; inclua nos coeficientes a soma das contribuições dos filhos leves; segtree na cadeia; update no vértice, recompute a cadeia e propague a nova contribuição para o pai da cabeça; $O(\log^2 n)$.
  - *Exemplo:* maximum weight independent set em árvore com mudanças de peso; soma de DP de matching com updates.
  - *Armadilha:* a orientação das matrizes e a ordem do produto (não comutativa); incluir duas vezes os filhos leves; atualizar o pai da cabeça na ordem certa; max-plus não tem inverso, então não tente "desfazer" uma contribuição.
- **Segtree de matrizes para DP dinâmica.** dp[i] = M_i · dp[i−1]; update pontual em M_i e consulta de intervalo = produto de matrizes na segtree (max-plus ou mod p). Resolve "DP com alteração de parâmetros" em $O(k^3 \log n)$ por update.
  - *Gatilho:* a DP é uma recorrência linear de dimensão k e os coeficientes mudam pontualmente ou por intervalo.
  - *Por que funciona:* a composição de transições lineares é associativa; a segtree guarda o produto de um intervalo e responde a composição em O(log n) produtos de k×k.
  - *Como fazer:* construa as matrizes M_i na ordem dos índices; cada nó guarda o produto do intervalo na ordem correta; update recalcula O(log n) nós; consulta multiplica os produtos dos nós da esquerda para a direita. Cada multiplicação custa $O(k^3)$.
  - *Exemplo:* Fibonacci com coeficientes alterados; DP de autômato com transições que mudam; multiplicação de matrizes em cadeia.
  - *Armadilha:* a ordem do produto é da esquerda para a direita (M_r···M_l), não o contrário; a identidade deve ser o elemento neutro; max-plus não tem inverso; para k grande, $O(k^3 \log n)$ pode ser pesado.
- **DP com remoção de elemento (undo).** DP "sem o elemento x" para todo x: prefixo × sufixo (combina dp do prefixo e do sufixo) em vez de refazer; para mochila contável com remoção use a operação inversa (dp[x] −= dp[x−w], válida na contagem polinomial/modular).
  - *Gatilho:* você precisa da resposta para cada remoção individual de um item/elemento, e refazer a DP n vezes é caro.
  - *Por que funciona:* a DP sem o elemento x é a composição da DP dos elementos antes de x com a dos elementos depois de x; para contagem de subconjuntos, a contribuição de um item é o fator (1+x^w), que pode ser desfeito por uma recorrência inversa.
  - *Como fazer:* calcule prefix[i] e suffix[i] (ou a resposta combinada) e combine para cada x; para mochila de contagem, remova o item com for y crescente: dp'[y] = dp[y] − dp'[y−w] (equivalente a dp[y] −= dp[y−w] atualizado em ordem crescente); para max-plus, use prefixo×sufixo ou D&C.
  - *Exemplo:* para cada item, o número de subconjuntos/melhores valores sem ele; "qual item posso remover sem perder a solução".
  - *Armadilha:* a inversa só vale para contagem (ou estruturas com inverso); em max-plus não há subtração; a ordem da recorrência inversa é crescente; itens com peso 0 precisam de tratamento especial.

- **Mochila com "remover item" (undo) / adicionar online.** Contagem modular: remover = inverso da multiplicação por (1 + x^w): dp'[x] = dp[x] − dp'[x−w], O(W). Para max-plus sem inverso use D&C no tempo (segtree offline: adiciona nos nós, desfaz ao subir) em O(nW log).
  - *Gatilho:* há inserções/remoções dinâmicas de itens e consultas da mochila, ou a resposta para cada item removido.
  - *Por que funciona:* no anel dos polinômios truncados, multiplicar por (1+x^w) é invertível pela série 1−x^w+x^{2w}−...; a recorrência inversa reconstrói a DP sem o item em O(W). Com max-plus, a operação não tem inverso, mas uma segtree sobre o tempo adiciona o item aos nós que cobrem sua vida e faz DFS com rollback.
  - *Como fazer:* contagem: para remover w, percorra x de w a W e faça dp[x] = dp[x] − dp[x−w] (com dp[x−w] já atualizado). Max-plus/offline: para cada item com intervalo [l,r], insira nos nós da segtree de tempo; na DFS, copie/atualize a DP ao entrar e restaure ao sair; responda nas folhas.
  - *Exemplo:* para cada item, o número de subconjuntos com soma W; mochila com itens que entram e saem; consultas em tempos diferentes.
  - *Armadilha:* a inversa modular não é divisão por zero aqui (é subtração), mas a ordem crescente é obrigatória; em max-plus, rollback não pode ser feito por "desfazer max" — copie a DP ou use uma pilha de alterações; o custo O(nW log T) pode ser grande.
- **Mochila offline com itens com intervalo de vida.** Cada item vive em [l, r] de tempo → segtree sobre o tempo, cada item entra em O(log) nós; DFS copiando a dp (memória log·W); resposta por tempo na folha.
  - *Gatilho:* itens estão ativos apenas durante intervalos de tempo e você quer a melhor mochila em cada instante.
  - *Por que funciona:* a segtree de intervalos decompõe cada vida em O(log T) nós disjuntos; uma DFS da raiz às folhas aplica exatamente os itens ativos naquele tempo, e a DP é compartilhada ao longo do caminho.
  - *Como fazer:* construa uma segtree sobre os tempos; para cada item [l,r], adicione-o aos nós que cobrem o intervalo; faça DFS com uma cópia da DP (ou rollback) e processe os itens do nó; na folha, a DP contém todos os itens ativos naquele tempo. Memória O(W log T) se copiar por nível; O(W) com rollback explícito.
  - *Exemplo:* mochila com promoções válidas em intervalos; itens que aparecem e desaparecem; consultas por dia.
  - *Armadilha:* intervalos inclusivos/exclusivos; a cópia da DP em cada nível precisa ser restaurada corretamente; itens duplicados no mesmo nó; o custo O(nW log T) pode ser alto, mas é independente do número de consultas.
- **Tempo como dimensão: itens com validade (D&C offline).** "Em cada instante qual a melhor mochila dos itens ativos" → a mesma segtree-no-tempo (DSU rollback também usa isso); sem rollback seja explícito com cópia da dp por nível.
  - *Gatilho:* a resposta depende do tempo e os itens têm validade; as atualizações não precisam ser online.
  - *Por que funciona:* a técnica de divisão e conquista sobre o tempo transforma "item ativo em um intervalo" em "item aplicado em O(log T) nós"; a DFS mantém a DP do caminho e responde nas folhas.
  - *Como fazer:* segtree/D&C sobre [1,T]; distribua cada item nos nós; na recursão, aplique os itens do nó, chame os filhos e desfaça (ou copie a DP); nas folhas, registre a resposta. Para DSU com rollback, a ideia é a mesma, mas a estrutura precisa suportar desfazer.
  - *Exemplo:* orçamento com ofertas que valem por períodos; planejamento com recursos que expiram; consultas de mochila por data.
  - *Armadilha:* sem rollback, a cópia da DP a cada nível custa memória; a ordem de aplicação dos itens não importa para a mochila, mas importa para a restauração; não confunda "tempo" com a dimensão de capacidade.
- **DP em subsequências com dois índices crescentes (i < j, a_i < a_j, b_i < b_j).** LIS 2D → CDQ + BIT em O(n $\log^2$ n); com LIS ponderada, dp precisa ser calculada em ordem (CDQ com ordem de dp).
  - *Gatilho:* escolher uma subsequência que cresce em duas coordenadas simultaneamente (ou três, com o índice), e n é grande para O(n^2).
  - *Por que funciona:* CDQ divide pelo índice i, resolve a metade esquerda, usa os pontos já calculados para atualizar a direita por ordem de a e BIT em b, e resolve a direita; cada par é considerado uma vez.
  - *Como fazer:* cdq(l,r): cdq(l,mid); ordene os pontos de [l,mid] e [mid+1,r] por a; para cada ponto da direita em ordem de a, insira no BIT os da esquerda com a menor e consulte o máximo com b menor; atualize dp; cdq(mid+1,r). Para LIS 2D, dp[j] = 1 + max dp[i] com i<j, a_i<a_j, b_i<b_j.
  - *Exemplo:* LIS 2D (caixas que cabem umas nas outras); cadeia 3D; agendamento com duas dimensões.
  - *Armadilha:* desigualdades estritas exigem consultar b−1 e ordenar com desempate correto; processar a metade direita antes de a esquerda estar completa; CDQ com dp dependente de ordem exige que a esquerda já esteja resolvida; coordenadas comprimidas.

- **Conta de caminhos em DAG modulo p com n grande e k pequeno.** Contar caminhos com ≤ k arestas "de cor especial": dp[v][j] em O((n+m)k) ou matriz; se k ≤ 20 e cores, máscara.
  - *Gatilho:* o caminho é válido se usa no máximo k arestas de um tipo especial (ou até k no total), e k é pequeno.
  - *Por que funciona:* o número de arestas especiais usadas é um estado pequeno; processar o DAG em ordem topológica permite somar os caminhos sem contar permutações.
  - *Como fazer:* dp[v][j] = número de caminhos até v com j arestas especiais; para cada aresta u→v, dp[v][j + especial] += dp[u][j]; responda Σ_{j≤k} dp[v][j]. Se as cores formam uma máscara pequena, use dp[v][mask]. O((n+m)k).
  - *Exemplo:* caminhos com no máximo 3 arestas vermelhas; rotas com até k pedágios; contar caminhos que usam um conjunto de cores.
  - *Armadilha:* "≤ k" versus "exatamente k"; somar no destino e não no meio; k pode ser maior que o caminho máximo; se o grafo tem ciclos, a DP topológica não vale; módulo p.
- **Número de caminhos / maior caminho em DAG de intervalos (arestas para intervalo).** dp ordenada topológica com segtree/prefix sum sobre o intervalo de destinos; se destinos formam intervalo contíguo, dp[i] += prefix.
  - *Gatilho:* de um vértice saem arestas para todos os vértices de um intervalo (ou para um intervalo de valores), e o DAG é ordenado por índice.
  - *Por que funciona:* a transição é uma soma/máximo sobre um intervalo de dp; se os intervalos são processados em ordem topológica, um prefixo ou uma segtree responde em O(log n) por aresta.
  - *Como fazer:* ordene os vértices topologicamente; para cada u, consulte a soma/máximo de dp no intervalo [L_u,R_u] com segtree/prefixo e adicione a dp dos destinos; se as arestas apontam para trás, adapte a direção. Para contagem, use diferença/segment tree com lazy.
  - *Exemplo:* caminhos em um DAG onde cada nó aponta para um intervalo de índices maiores; contagem de sequências com transições em intervalos.
  - *Armadilha:* a ordem topológica precisa ser compatível com os índices; intervalos inclusivos; para contagem, uma atualização em range e consulta pontual pode ser mais eficiente que consulta em range; módulo p.
- **DP em string com "transição por todos os palíndromos/períodos".** Menor partição em palíndromos: eertree com séries de links (diff, slink) mantém dp em O(n log n) (já citado em strings); partição em periódicos: usa Z/KMP por início.
  - *Gatilho:* particionar uma string em blocos que são palíndromos, periódicos ou obedecem a uma propriedade de borda, e n é grande.
  - *Por que funciona:* o eertree lista todos os palíndromos que terminam em cada posição; as séries de links agrupam palíndromos com a mesma diferença de comprimento, permitindo atualizar a DP em O(log n) por posição em vez de $O(n^2)$.
  - *Como fazer:* para palíndromos, construa o eertree e mantenha dp[i] = menor custo para o prefixo i; para cada posição, percorra os links de sufixo agrupados por série e use o mínimo da série para atualizar. Para períodos, calcule Z/KMP a partir de cada início ou use bordas.
  - *Exemplo:* palindromic factorization mínima; particionar em blocos periódicos; contar partições com palíndromos.
  - *Armadilha:* percorrer todos os links de sufixo ainda pode ser $O(n^2)$ se não agrupar por série; a definição de período (borda) e o custo por bloco; eertree para palíndromos distintos versus todas as ocorrências.
- **Soma sobre subsets com restrição de popcount ou peso (DP em bits).** dp[bit][carry] para somas de pares com xor/and/or: processe bit a bit com carry (0/1) e contagem; "pares (i, j) com a_i + b_j ≥ K" em bits do mais alto ao mais baixo.
  - *Gatilho:* a condição envolve bits de dois números, soma com carry, popcount ou uma comparação com K.
  - *Por que funciona:* a soma binária é local: o bit resultante depende dos bits dos operandos e do carry; comparar com K pode ser feito mantendo um estado de "já é maior" ou processando do bit mais alto.
  - *Como fazer:* para somas, processe do LSB com dp[bit][carry]; para comparação com K, processe do MSB com um estado tight/greater, ou conte pares com soma < K e subtraia do total. Para popcount, use a contagem de bits no estado ou uma transformada.
  - *Exemplo:* contar pares (x,y) com x+y = z e x&y=0; contar pares com soma ≥ K; somas de subconjuntos com restrição de popcount.
  - *Armadilha:* não misture carry (que vem do LSB) com tight (que vem do MSB) no mesmo passo sem justificar; limites x≤A, y≤B exigem estados de comparação; use inteiros de 64 bits para as contagens.

- **DP sobre bits com carry.** Contar (x, y) com x + y = z, x & y = 0, ou x ≤ A, y ≤ B: dp[bit][carry][borrowA][borrowB] do bit menos significativo (limites via borrow), ou dp[bit][tightA][tightB] do mais significativo (sem carry) — não misture carry (LSB) com tight (MSB); mais simples que fórmulas.
  - *Gatilho:* contagem de pares de inteiros com restrições bit a bit, especialmente soma, AND/XOR e limites superiores.
  - *Por que funciona:* cada bit de x e y tem poucas escolhas; o carry resume a soma dos bits anteriores, e o borrow resume a comparação x ≤ A quando processada do LSB. O estado é O(1) por bit.
  - *Como fazer:* para x+y=z, processe do LSB com carry ∈ {0,1}, escolhendo bits x_b,y_b tais que x_b+y_b+carry ≡ z_b (mod 2); se x&y=0, rejeite x_b=y_b=1. Para x≤A, processe a subtração A−x do LSB e exija borrow final 0; para y≤B, idem. Se comparar do MSB, use tight, mas então a soma precisa de carry do lado inferior — não misture as duas direções.
  - *Exemplo:* contar pares (x,y) com x+y=z e x&y=0; contar x≤A, y≤B com x+y≤K; contar soluções de uma equação bit a bit.
  - *Armadilha:* a opção "MSB com tight sem carry" não fecha a equação x+y=z, porque o bit atual depende do carry dos bits menos significativos; use carry+borrow no LSB ou reformule. Não misture as direções; cuidado com bits além do maior limite.
- **Árvore de recursão de "estados equivalentes" (memo sobre estruturas).** Em jogos/contagem de configurações simétricas, normalize o estado (ordene multiconjunto, rotacione) antes de memoizar: reduz estados exponencialmente (partições, necklaces).
  - *Gatilho:* o estado é uma configuração com simetrias (pilhas, multiconjuntos, tabuleiro rotacionável) e a recursão visita muitas permutações equivalentes.
  - *Por que funciona:* se duas configurações são equivalentes por uma simetria do problema, elas têm o mesmo valor; canonicalizar antes de memoizar faz todas caírem na mesma chave.
  - *Como fazer:* escolha uma forma canônica (ordene o vetor de pilhas, rotacione a string para a menor, reflita se permitido); use essa forma como chave em um map/unordered_map. Para contagem, multiplique pelo tamanho da órbita ou use Burnside quando precisar contar configurações distintas.
  - *Exemplo:* jogo de Nim com pilhas (ordem irrelevante); partições de n; colares/braceletes; tabuleiro com rotações.
  - *Armadilha:* normalizar demais (identificar estados que não são equivalentes) ou de menos; a contagem de órbitas não é só dividir pelo tamanho do grupo quando há estabilizadores; o custo de canonicalizar a cada chamada pode dominar.
- **Estado = multiconjunto ordenado (partições).** n ≤ 40–60 com configuração "pilhas de pedras" ou "grupos": estados são partições (≤ 10^6 para n ~ 60); memo em map com vetor ordenado.
  - *Gatilho:* o estado é um conjunto de pilhas/grupos sem ordem entre eles, e n é moderado (até ~60).
  - *Por que funciona:* ordenar as pilhas remove permutações equivalentes; o número de partições de n cresce como p(n), que para n≈60 é ~10^6, viável com memoização.
  - *Como fazer:* represente o estado como vetor não crescente de tamanhos; gere transições escolhendo uma pilha/grupo e alterando; ordene o resultado antes de memoizar; use map, valor> ou hash.
  - *Exemplo:* jogo de dividir pilhas; contar partições de n; agrupar bolas em grupos não rotulados.
  - *Armadilha:* p(60) ≈ 966467, mas constantes de map e cópia de vetor pesam; não confunda partições (ordem irrelevante) com composições; a memoização precisa incluir todos os parâmetros do estado.
- **DP "dos dois lados" (prefixo e sufixo) para escolher um ponto de corte.** Resposta = max sobre i de pre[i] + suf[i+1]; com 3 cortes, mantenha estado "quantos cortes já fiz" (0..3) numa única passada.
  - *Gatilho:* a resposta é o melhor par/tripla de partes separadas por cortes, e cada lado pode ser resumido por uma DP de prefixo/sufixo.
  - *Por que funciona:* um corte separa o problema em dois independentes; o melhor valor é a combinação dos ótimos de cada lado, e para vários cortes o estado "quantos já fiz" evita recomputar.
  - *Como fazer:* calcule pre[i] (melhor no prefixo até i) e suf[i] (melhor no sufixo de i em diante); combine no corte. Para 3 partes, dp[i][c] = melhor com c cortes usando o prefixo i, transicionando de dp[j][c−1] + custo(j+1,i).
  - *Exemplo:* dividir array em 2 subarrays para maximizar a soma dos máximos; escolher dois pontos de corte; particionar em 3 segmentos.
  - *Armadilha:* sobreposição/ordem dos cortes; "até k" versus "exatamente k"; um dos lados pode ser vazio; para 3 cortes, a DP O(kn^2) pode precisar de otimização.

- **Máquina de estados de poucas fases.** Problemas "no máximo k transações / 3 fases (subir-descer-subir)" → dp[i][fase] com fase ≤ k; compre-venda de ações com cooldown/taxa são autômatos pequenos, escreva o diagrama de estados antes de codar.
  - *Gatilho:* o problema tem poucos modos (comprado/vendido, subindo/descendo, cooldown, número de transações) e as transições dependem do modo.
  - *Por que funciona:* o futuro só depende da posição i e da fase; a DP sobre o autômato enumera todas as transições válidas sem carregar histórico.
  - *Como fazer:* desenhe o diagrama (estados e arestas com custo/ganho); defina dp[i][fase] e transicione por "não fazer nada" ou "mudar de fase"; use -∞ para fases impossíveis. Para k transações, fase = (número de transações, está comprado?).
  - *Exemplo:* melhor momento para comprar/vender ações com no máximo k transações e cooldown; subir/descer/subir em uma sequência; máquina de senha.
  - *Armadilha:* a ordem das transições no mesmo dia (comprar e vender no mesmo instante); cooldown de um dia; k pode ser maior que n/2 (aí o limite deixa de importar); inicialize fases inviáveis com -∞, não 0.
- **Programação dinâmica sobre "tempo" quando custo é por tempo de espera.** Fila/escalonamento com custo Σ w_i·C_i: ordene por razão (exchange argument); se há prazos e lucros → dp[t] com mochila sobre tempo ou guloso com heap (job sequencing) — diferencie.
  - *Gatilho:* o custo é ponderado pelo tempo de conclusão/espera, ou há prazos e lucros por tarefa.
  - *Por que funciona:* para Σ w_i C_i sem prazos, trocar duas tarefas adjacentes mostra que a ordem ótima é por p_i/w_i crescente; com prazos e lucros, a decisão de aceitar tarefas vira uma mochila no tempo ou um guloso com heap.
  - *Como fazer:* sem prazos: ordene por p_i/w_i (Smith); com prazos e pesos/lucros: dp[t] = melhor lucro com tempo t, ou ordene por prazo e mantenha um heap das tarefas aceitas, removendo a de maior duração quando estourar (para maximizar número). Diferencie "minimizar atraso" de "maximizar lucro".
  - *Exemplo:* sequenciar tarefas para minimizar Σ w_i C_i; job sequencing with deadlines; mochila de tarefas com duração e lucro.
  - *Armadilha:* a regra p_i/w_i não vale com prazos; Moore–Hodgson só minimiza o número de atrasadas sem pesos; com pesos o problema é NP-hard fraco e a DP é pseudo-polinomial no tempo; empates e prazos inclusivos.
- **Escalonamento com prazos e penalidade (1 || Σ w_j U_j).** Guloso com heap: ordene por prazo, insira, se estourou remova o de maior p_j — Moore–Hodgson só vale sem pesos (minimizar nº de atrasados); com pesos é NP-hard fraco e vira mochila (DP em tempo).
  - *Gatilho:* uma máquina, tarefas com duração, prazo e peso; o objetivo é minimizar a soma dos pesos das tarefas atrasadas.
  - *Por que funciona:* sem pesos, remover a tarefa mais longa quando o prazo estoura preserva a factibilidade e minimiza o número de atrasadas (Moore–Hodgson). Com pesos, a troca não é tão simples; o problema é fracamente NP-hard e admite DP pelo tempo total.
  - *Como fazer:* sem pesos: ordene por prazo, acumule durações e use max-heap; se a soma exceder o prazo, remova a maior duração. Com pesos: dp[t] = menor peso de atraso para um subconjunto de tarefas processadas até o tempo t, ou use a formulação de mochila; O(n·Σp) pseudo-polinomial.
  - *Exemplo:* minimizar número de tarefas atrasadas (Moore–Hodgson); minimizar Σ w_j U_j com prazos e pesos.
  - *Armadilha:* usar Moore–Hodgson com pesos (não é ótimo); tarefas com prazo menor que a duração; o tempo total pode ser grande; "atrasada" é C_j > d_j, não ≥; a ordem das tarefas aceitas no heap não é a ordem final.
- **Maximizar nº de intervalos disjuntos com pesos.** Ordene por fim; dp[i] = max(dp[i−1], w_i + dp[último j com fim ≤ início_i]) com binária; para k máquinas/intervalos sobrepostos até k → fluxo de custo mínimo (não é matroide).
  - *Gatilho:* escolher intervalos que não se sobrepõem, com peso, e a ordem natural é por término; ou permitir até k sobreposições.
  - *Por que funciona:* ordenar por fim dá subestrutura ótima: ou o intervalo i não é escolhido, ou é escolhido e o próximo compatível é o último com fim ≤ início_i; a busca binária acha esse índice.
  - *Como fazer:* ordene por fim; dp[i] = max(dp[i−1], w_i + dp[p(i)]), onde p(i) é o maior j < i com fim_j ≤ início_i; O(n log n). Para k máquinas, modele como fluxo de custo mínimo no tempo (capacidade k) ou DP com estado de k.
  - *Exemplo:* weighted interval scheduling; selecionar palestras com horários e lucros; alocar k salas para intervalos.
  - *Armadilha:* intervalos que se tocam (fim = início) são compatíveis; ordenar por início em vez de fim quebra a recorrência; para k sobreposições, o guloso de escolher o de maior peso não é ótimo e o problema não é matroide.

- **Cobertura por k máquinas (intervalo, "no máximo k simultâneos").** Fluxo de custo mínimo em linha do tempo com arestas de capacidade k e arestas de intervalo; é DP "não óbvia" que vira fluxo; guloso com heap vale se pesos iguais.
  - *Gatilho:* escolher intervalos com pesos respeitando "no máximo k intervalos ativos ao mesmo tempo" (ou k máquinas), e a resposta é maximizar peso/cobertura.
  - *Por que funciona:* a linha do tempo com capacidade k é uma rede: arestas de tempo com capacidade k e custo 0 representam máquinas ociosas; uma aresta de intervalo desvia uma unidade e cobra/seleciona o intervalo; o fluxo de custo mínimo escolhe a combinação ótima.
  - *Como fazer:* comprima os extremos dos intervalos; crie arestas t→t+1 com capacidade k e custo 0, e para cada intervalo [l,r] uma aresta l→r com capacidade 1 e custo −w (ou +custo); envie fluxo e interprete as arestas usadas. Para pesos iguais, um guloso com heap por instante pode bastar.
  - *Exemplo:* agendar k salas para maximizar lucro; cobertura de pontos por até k intervalos; selecionar tarefas com k máquinas.
  - *Armadilha:* a orientação e a capacidade das arestas de tempo; intervalos com extremos iguais; custos negativos exigem o algoritmo certo; fluxo não é necessário se k=1 (basta weighted interval scheduling).
- **Min-cost flow como DP.** Problemas de escolher conjuntos com restrição "no máximo k cobrindo cada ponto" ou "caminhos disjuntos em DAG com k": DP em n·k não funciona, mas fluxo de custo mínimo em DAG linear resolve; reconhecer: pesos de intervalos e restrição de cobertura.
  - *Gatilho:* há uma restrição de capacidade em cada ponto do tempo/linha e escolhas com peso, mas o estado DP local não captura a interação global.
  - *Por que funciona:* a rede modela a capacidade compartilhada como fluxo; a integralidade da matriz de incidência garante solução inteira, e o custo mínimo escolhe as arestas de intervalo.
  - *Como fazer:* monte o grafo temporal com capacidade k, arestas de intervalo com capacidade 1 e custo, e rode min-cost flow; para caminhos disjuntos em DAG, capacidade 1 nos vértices/arestas e custo negativo. Recupere as arestas usadas.
  - *Exemplo:* k caminhos disjuntos de custo mínimo; cobertura de pontos por intervalos com limite k; seleção de tarefas em k máquinas.
  - *Armadilha:* a rede precisa ser acíclica ou usar o algoritmo com potenciais; capacidades e custos negativos; o valor do fluxo deve ser exatamente k; não confunda "no máximo k" com "exatamente k" (adicione fonte/coletor ou arestas de desperdício).
- **Estados "último que mudou" para custo de troca.** Custo de trocar de tipo (ex.: máquinas/cores) → dp[i][último] com último podendo ser comprimido: guarde só o melhor e o segundo melhor de cada grupo (top-2 em dp[i−1]) para transição O(1) por tipo.
  - *Gatilho:* pintar/escalonar uma sequência com custo por escolher um tipo diferente do anterior, e o número de tipos é grande.
  - *Por que funciona:* a transição "melhor anterior com tipo diferente de c" só precisa do melhor global e, se o melhor for justamente c, do segundo melhor; não é necessário varrer todos os tipos.
  - *Como fazer:* guarde para cada posição o melhor valor e o tipo que o atingiu, e o segundo melhor com tipo diferente; para cada cor c, use o melhor se tipo ≠ c, senão o segundo. Atualize os top-2 após processar a posição. O total é O(n·k), com O(1) por cor depois dos top-2.
  - *Exemplo:* pintar casas com custo por cor e multa por mudar de cor; escalonar tarefas sem repetir o tipo anterior; colorir com custo de troca.
  - *Armadilha:* empates entre o melhor e o segundo com o mesmo tipo; atualizar o top-2 antes de terminar a linha; se o custo de troca depende do par (não só de "diferente"), o top-2 não basta.
- **Top-2 trick em transição "diferente do anterior".** dp[i][c] = custo(i, c) + min_{c' ≠ c} dp[i−1][c']: mantenha mínimo e segundo mínimo de dp[i−1] (com cor), O(n·k) em vez de O(n·k²). Pintar casas, escalonar tarefas sem repetir.
  - *Gatilho:* a transição exclui exatamente o mesmo tipo/cor do estado anterior.
  - *Por que funciona:* o mínimo com c' ≠ c é o mínimo global, a menos que o mínimo global seja c; nesse caso é o segundo mínimo. Guardar os dois resolve em O(1).
  - *Como fazer:* para cada i, calcule o mínimo e o segundo mínimo de dp[i−1] com as cores correspondentes; para cada c, escolha o mínimo se a cor dele ≠ c, senão o segundo; some custo(i,c). Atualize os dois menores para a próxima linha.
  - *Exemplo:* pintar n casas com k cores e custo de troca; tarefas que não podem repetir a categoria anterior; DP de cores com penalidade.
  - *Armadilha:* o segundo mínimo precisa ter cor diferente do primeiro; se todos os dp[i−1][c'] são iguais, o segundo é o mesmo valor com outra cor; não use apenas valores sem a cor associada.

- **DP em "duas pessoas andando" (caminhos ida-e-volta).** Dois caminhantes simultâneos com passo t: dp[t][x1][x2] com y = t − x; 2n × n × n em vez de caminhos 4D. Cereja/ida e volta em grid; se se cruzam conta só uma vez.
  - *Gatilho:* dois caminhos simultâneos em uma grade, geralmente ida e volta, e o custo/pontuação é coletado ao passar pelas células.
  - *Por que funciona:* no passo t, cada caminhante está em uma célula (x, t−x); a posição y é determinada por t e x, então o estado 4D (x1,y1,x2,y2) colapsa para (t,x1,x2).
  - *Como fazer:* dp[t][x1][x2] = melhor pontuação com os dois em (x1,t−x1) e (x2,t−x2); transicione cada um para x+1 ou y+1; se as células coincidem, conte a cereja uma vez. Responda no passo 2(n−1) com ambos no destino.
  - *Exemplo:* Cherry Pickup (ida e volta em uma grade com obstáculos); dois caminhos disjuntos máximos; coleta de moedas por dois agentes.
  - *Armadilha:* contar a mesma célula duas vezes quando os caminhantes se encontram; passos fora da grade; a ordem dos caminhantes é simétrica (pode fixar x1 ≤ x2 para economizar); obstáculos bloqueiam os dois.
- **DP com dois ponteiros simultâneos em duas strings/arrays (intercalar).** Intercalação de duas sequências com restrição: dp[i][j] = pode formar o prefixo i+j da terceira com i da primeira e j da segunda; O(nm); troque por bitset se n,m ~ 10^4.
  - *Gatilho:* construir/intercalar uma terceira sequência a partir de duas, ou alinhar duas strings com um terceiro parâmetro.
  - *Por que funciona:* o prefixo consumido de cada string determina o prefixo da terceira; a decisão local é pegar o próximo caractere de uma das duas, e a sobreposição de subproblemas é (i,j).
  - *Como fazer:* dp[i][j] booleano; dp[0][0]=true; dp[i][j] = (dp[i−1][j] e s1[i]=t[i+j]) ou (dp[i][j−1] e s2[j]=t[i+j]); para contagem, some. Para n,m grandes, represente a linha por bitset e use deslocamentos.
  - *Exemplo:* interleaving string; merge de duas sequências; alinhamento com uma terceira string.
  - *Armadilha:* a ordem dos caracteres dentro de cada string é preservada; o comprimento de t precisa ser i+j; para contagem, bitset booleano não conta; a linha de base e os índices 1-based.
- **Contagem de caminhos Dyck com altura limitada** (parênteses balanceados com profundidade ≤ h) → dp[i][altura]; para n enorme use matriz h×h ou transfer; também fórmula por reflexão alternada (IE sobre reflexões múltiplas).
  - *Gatilho:* caminhos de ±1 que começam e terminam em 0 e nunca passam de uma altura máxima h.
  - *Por que funciona:* a altura é um estado suficiente; a restrição é local (subir só se altura < h, descer só se altura > 0), e a matriz de transferência codifica as transições entre alturas.
  - *Como fazer:* dp[posição][altura], transição para altura±1 dentro de [0,h]; para n grande, matriz (h+1)×(h+1) e exponenciação. Para a fórmula fechada, use reflexão repetida/IE sobre as barreiras y=−1 e y=h+1.
  - *Exemplo:* número de sequências de parênteses de n pares com profundidade ≤ h; caminhos de Dyck confinados; colisões de passeios.
  - *Armadilha:* h=0 só permite a sequência vazia; a paridade de n; a fórmula de reflexão alternada tem soma infinita truncada; a matriz precisa da condição de contorno correta.
- **Contar sequências que evitam padrão consecutivo por GF.** "Sem k iguais seguidos" para n até 10^{18} → recorrência linear de ordem k−1 (ou k·alfabeto) → Kitamasa/matriz.
  - *Gatilho:* n enorme, alfabeto pequeno e a restrição é local (não pode haver k símbolos iguais consecutivos).
  - *Por que funciona:* o número de sequências válidas de comprimento n satisfaz uma recorrência linear de ordem k−1; a matriz companheira ou Kitamasa calcula o n-ésimo termo em $O(k^3 \log n)$ ou $O(k^2 \log n)$.
  - *Como fazer:* derive a recorrência por último bloco ou use autômato de estados (comprimento do bloco atual); monte a matriz de transição e exponencie. Para k fixo, Kitamasa/Berlekamp–Massey com os primeiros termos também funciona.
  - *Exemplo:* strings binárias sem 3 zeros seguidos; colorações de fita sem k iguais; sequências sobre alfabeto m.
  - *Armadilha:* a ordem da recorrência é k−1, não k; os casos base para n<k; o alfabeto multiplica o número de estados; Kitamasa precisa de módulo e da recorrência correta.
- **Contar árvores, florestas ou estruturas rotuladas por DP em tamanho.** dp[n] = Σ binomial·dp[i]·dp[n−1−i] (EGF); sequências rotuladas por tamanho do primeiro bloco: dp[n] = Σ C(n−1, k−1)·g[k]·dp[n−k] — a escolha do menor rótulo no primeiro bloco evita contar ordem duas vezes.
  - *Gatilho:* estruturas rotuladas (árvores, florestas, partições) e a contagem depende do tamanho dos componentes.
  - *Por que funciona:* em estruturas rotuladas, escolher o conjunto de rótulos de cada componente introduz binomiais; fixar o menor rótulo no primeiro componente elimina a permutação dos componentes.
  - *Como fazer:* para árvores binárias rotuladas, dp[n] = Σ_{i=0}^{n−1} C(n−1,i)·dp[i]·dp[n−1−i]; para partições/sequências de blocos, dp[n] = Σ_k C(n−1,k−1)·g[k]·dp[n−k], com dp[0]=1. Use EGFs e convolução quando n é grande.
  - *Exemplo:* número de árvores binárias rotuladas; partições de um conjunto em blocos com peso; florestas de árvores rotuladas.
  - *Armadilha:* esquecer o binomial ou o fator (n−1); contar a ordem dos blocos duas vezes; a estrutura vazia; para árvores não binárias, o número de filhos muda a EGF.

- **Estruturas rotuladas: fixe o menor rótulo.** Para contar partições/componentes de conjunto, sempre inclua o elemento 1 no primeiro bloco (C(n−1, k−1)); é o segredo da recorrência de Bell, grafos conexos rotulados (conexos = total − não conexos fixando o componente do 1).
  - *Gatilho:* contagem de objetos rotulados decompostos em componentes/blocos não ordenados.
  - *Por que funciona:* cada objeto tem um componente que contém o rótulo 1; escolher os outros k−1 rótulos desse componente e contar o resto elimina a simetria de ordem dos componentes.
  - *Como fazer:* Bell: B[n] = Σ_{k=1}^n C(n−1,k−1)·B[n−k]; para grafos conexos, total = 2^{C(n,2)}, e subtraia os desconexos escolhendo o componente do vértice 1: conn[n] = total − Σ C(n−1,k−1)·conn[k]·2^{C(n−k,2)}.
  - *Exemplo:* números de Bell; grafos conexos rotulados; partições em componentes conexos.
  - *Armadilha:* fixar o rótulo 1 no primeiro bloco, não um rótulo qualquer; o componente do 1 pode ter qualquer tamanho; a contagem do resto depende de ser arbitrário (2^{C(n−k,2)}); bases n=1.
- **Grafos conexos rotulados.** conn[n] = 2^{C(n,2)} − Σ_{k<n} C(n−1, k−1)·conn[k]·2^{C(n−k,2)}; mesmo molde (total − componente do vértice 1) para torneios fortemente conexos, DAGs rotulados (IE com sinais alternados sobre fontes).
  - *Gatilho:* contar grafos conexos/fortemente conexos/acíclicos com rótulos distintos.
  - *Por que funciona:* todo grafo tem um componente conexo contendo o vértice 1; separar esse componente e contar o resto (arbitrário ou conexo) evita dupla contagem. Para DAGs, a IE sobre o conjunto de fontes controla a condição de aciclicidade.
  - *Como fazer:* conexos: total 2^{C(n,2)} menos a soma sobre o tamanho k do componente do vértice 1; torneios fortemente conexos: total de torneios menos os que têm um conjunto fonte próprio, com sinais; DAGs: escolha o conjunto de fontes e use (−1)^{k+1}.
  - *Exemplo:* número de grafos conexos rotulados; torneios fortemente conexos; DAGs rotulados.
  - *Armadilha:* o componente do vértice 1 não é necessariamente uma fonte; para torneios, a estrutura de fontes é mais sutil; os sinais da IE; use módulo e potências de 2 pré-calculadas.
- **DAGs rotulados (contagem).** dag[n] = Σ_{k≥1} (−1)^{k+1} C(n, k)·2^{k(n−k)}·dag[n−k] (IE sobre o conjunto de fontes); n até 5000 com NTT online.
  - *Gatilho:* contar DAGs rotulados com n vértices, possivelmente módulo um primo.
  - *Por que funciona:* todo DAG não vazio tem um conjunto não vazio de fontes; escolher k fontes, ligar arbitrariamente delas para o resto (2^{k(n−k)}) e contar o DAG restante; a IE alterna para que cada DAG seja contado exatamente uma vez pelo seu conjunto exato de fontes.
  - *Como fazer:* pré-calcule potências de 2 e binomiais; dp[0]=1; dp[n] = Σ_{k=1}^n (−1)^{k+1} C(n,k)·2^{k(n−k)}·dp[n−k]. Para n até 5000, O(n^2) pode passar; para maior, use NTT/CDQ online.
  - *Exemplo:* número de DAGs rotulados com n vértices; contar posets? (não é o mesmo que DAG); relações de ordem.
  - *Armadilha:* a fórmula conta DAGs, não posets (DAGs com diferentes transitividades podem dar a mesma ordem); o sinal e o caso n=0; 2^{k(n−k)} deve ser calculado com expoente inteiro; NTT online exige cuidado com a ordem das convoluções.
- **DP de contagem de matrizes/tabelas 0/1 com somas de linha e coluna.** Ordene colunas por soma e processe linha a linha com estado = nº de colunas ainda precisando de 1,2,... (quando as somas ≤ 2): dp[i][c1][c2] com contagem por escolhas.
  - *Gatilho:* contar matrizes 0/1 com somas de linha e coluna prescritas, especialmente quando as somas são pequenas (≤ 2).
  - *Por que funciona:* processar as linhas em ordem e manter quantas colunas ainda precisam de 1 ou 2 uns resume o efeito das escolhas anteriores; as colunas são simétricas, então só as contagens importam.
  - *Como fazer:* ordene as colunas por soma; para cada linha com soma r, escolha quantas colunas do grupo "precisa 1" e "precisa 2" recebem 1, atualize as contagens e multiplique pelas combinações; dp[i][c1][c2]. Se as somas são ≤ 2, o estado é O(n^2) e as transições O(1) ou O(n).
  - *Exemplo:* matrizes com somas de linha e coluna 0/1; tabelas de incidência com graus; contagem de bipartites com graus pequenos.
  - *Armadilha:* a ordem das colunas e a simetria precisam ser justificadas; não confunda existência (Gale–Ryser) com contagem; as combinações e o módulo; somas de linha > 2 exigem mais dimensões.
- **Gale–Ryser / contagem por graus.** Existência de matriz 0/1 com somas dadas → conjugada majora (verificação O(n)); contagem exata é difícil, mas com somas ≤ 2 o DP em (c1, c2) funciona.
  - *Gatilho:* decidir se existe uma matriz 0/1 (ou bipartite) com sequências de graus/somas prescritas; ou contar quando os graus são pequenos.
  - *Por que funciona:* o teorema de Gale–Ryser diz que, após ordenar as somas, a sequência de colunas é dominada pela conjugada da sequência de linhas; a verificação é uma cadeia de majoração.
  - *Como fazer:* ordene as somas; calcule a conjugada (nº de linhas com soma ≥ k); verifique Σ_{i≤k} c_i ≤ Σ_i min(r_i,k) para todo k (e igualdade total). Para contagem com somas ≤ 2, use o DP de estados (c1,c2).
  - *Exemplo:* existe matriz 0/1 com somas de linha e coluna dadas; graus de um grafo bipartite; matriz de incidência.
  - *Armadilha:* a condição é de majoração, não de igualdade termo a termo; ordenar as duas sequências; a contagem exata é #P-difícil em geral; somas ≤ 2 ainda exigem o DP, não basta Gale–Ryser.

- **Coloração de grafo / cobertura de conjuntos por DP em subconjuntos.** Número cromático: dp[S] = 1 + min sobre independentes I ⊆ S contendo menor bit; ou contagem de k-colorações por IE sobre arestas: $\Sigma_{F\subseteq E} (-1)^{|F|} k^{c(F)}$ com c(F) = nº de componentes de (V, F), $O(2^m)$; a soma $\Sigma_{S} (-1)^{n-|S|} I(S)^k$ conta coberturas com sobreposição, não colorações.
  - *Gatilho:* n pequeno (≤ 20) e o problema é colorir/particionar em conjuntos independentes ou cobrir com conjuntos.
  - *Por que funciona:* uma coloração é uma partição em conjuntos independentes; fixar o conjunto independente que contém o menor vértice evita contar permutações das cores. A IE sobre arestas remove as arestas monocromáticas.
  - *Como fazer:* pré-calcule independentes; dp[S] = 1 + $\min_{I\subseteq S, I \text{ independente, contém lsb}(S)}$ dp[S\I]; $O(3^n)$. Para contar k-colorações, use a IE sobre arestas ou a potência de cobertura com SOS; para k fixo, use inclusão-exclusão de partições.
  - *Exemplo:* número cromático de um grafo com n≤20; contar k-colorações próprias; cobertura por conjuntos independentes.
  - *Armadilha:* a IE sobre arestas tem 2^m, inviável se m é grande; a soma de $I(S)^k$ conta coberturas com sobreposição, não colorações; a escolha do menor bit é para partições, não para coberturas; módulo.
- **Contagem de subconjuntos independentes por SOS.** I(S) = nº de independentes ⊆ S: calcule ind[S] e zeta; cobre problemas de "k-coloração", "cobertura por k conjuntos", "partição em k independentes".
  - *Gatilho:* você precisa do número de subconjuntos independentes contidos em cada S, ou de potências disso para cobertura/coloração.
  - *Por que funciona:* se ind[T] = 1 quando T é independente, o zeta transform de ind dá $I(S)=\Sigma_{T\subseteq S} ind[T]$, exatamente o número de subconjuntos independentes de S.
  - *Como fazer:* marque ind[T] para todos os T independentes (teste $O(2^n n)$ ou recorrência), aplique SOS/zeta; use $I(S)^k$ para contar k-uplas de independentes contidos em S e combine com IE para cobertura/partição.
  - *Exemplo:* contar coberturas de U por k conjuntos independentes; número de k-colorações com IE; partição em k independentes.
  - *Armadilha:* $I(S)^k$ conta k-uplas ordenadas com sobreposição, não partições; para colorações, a IE e a distinção das cores mudam a fórmula; o zeta precisa ser sobre subconjuntos, não sobre máscaras arbitrárias.
- **DP de "nº de cobertura por k conjuntos" via IE + SOS.** Cobrir U com k conjuntos de uma família F: $\Sigma_{T \subseteq U} (-1)^{|U\setminus T|} (|\{f \in F: f \subseteq T\}|)^k$; $O(2^n n)$ após zeta de F. Dá cobertura exata contável e, com cuidado, existência (módulo primo grande).
  - *Gatilho:* contar/decidir se U pode ser coberto por exatamente k conjuntos de uma família, com n pequeno.
  - *Por que funciona:* a IE sobre os elementos faltantes: para cada T, contamos as k-uplas de conjuntos inteiramente contidos em T; a alternância remove as que deixam algum elemento de fora.
  - *Como fazer:* marque cnt[mask] = 1 se mask ∈ F; aplique zeta para obter $z[mask] = |\{f\in F: f\subseteq mask\}|$; para cada T, calcule $(-1)^{n-|T|}\cdot z[T]^k$ e some. Para existência, use um primo grande e teste se o resultado é diferente de zero (com risco de cancelamento).
  - *Exemplo:* cobertura de um universo por k conjuntos; número de maneiras de cobrir todos os elementos; set cover com k conjuntos.
  - *Armadilha:* a soma conta coberturas com sobreposição (conjuntos podem se sobrepor), não partições; a IE usa complemento de T; o resultado pode ser múltiplo do primo e zerar falsamente; k=0 e U vazio.
- **Steiner tree DP.** dp[mask][v] = menor árvore conectando terminais de mask com raiz v: merge dp[sub][v] + dp[mask\sub][v] e relaxamento por Dijkstra em cada mask; $O(3^k n + 2^k m \log)$ com k ≤ 10 terminais.
  - *Gatilho:* conectar um conjunto pequeno de terminais em um grafo com pesos, permitindo vértices de Steiner.
  - *Por que funciona:* a árvore ótima pode ser enraizada em v e dividida em duas subárvores que compartilham v; a DP combina esses dois conjuntos de terminais. Depois, mover a raiz por uma aresta é uma relaxação de caminho mínimo.
  - *Como fazer:* $dp[mask][v] = \min_{sub\subset mask} dp[sub][v]+dp[mask\setminus sub][v]$; depois rode Dijkstra (ou SPFA) para cada mask sobre o grafo para permitir que a raiz ande; inicialize dp[1<<i][terminal_i]=0. Responda min_v dp[full][v].
  - *Exemplo:* menor rede conectando cidades; Steiner tree em grafos com k≤10 terminais; árvore de conexão de componentes.
  - *Armadilha:* a ordem das máscaras (submask antes de mask); o merge pode contar a mesma árvore duas vezes se sub e complemento não forem tratados com uma regra (não afeta o mínimo); Dijkstra com arestas negativas não vale; k grande torna 3^k inviável.

- **Caminho simples longo por DP sobre subconjuntos de vértices.** n ≤ 20 hamilton; com n maior e caminho curto (k ≤ 10): color coding + dp[mask][v] em 2^k·m·e^k.
  - *Gatilho:* encontrar um caminho simples de comprimento k (ou ciclo) em um grafo, com k pequeno; ou Hamiltonian path com n≤20.
  - *Por que funciona:* para n≤20, a máscara de vértices visitados e o último vértice determinam o caminho; para k pequeno, color coding atribui cores aleatórias e procura caminhos coloridos, que são fáceis por DP.
  - *Como fazer:* DP clássica: dp[mask][v] = existe caminho que visita mask e termina em v; transicione para vizinhos não visitados; O(2^n n^2). Para k pequeno, repita colorações aleatórias de k cores, dp[mask][v] sobre cores distintas; probabilidade de um caminho ficar colorido é k!/k^k, então O(e^k) repetições.
  - *Exemplo:* Hamiltonian path com n≤20; caminho simples de tamanho 10; detecting a path of length k.
  - *Armadilha:* caminho simples não pode repetir vértices; color coding é Monte Carlo (ou use splitters para determinístico); a DP de cores exige que todas as cores do caminho sejam distintas; k=1 e grafos com laços.
- **Cobertura/dominação em grade larga.** Dominating set / vertex cover / independent set em grafo de treewidth pequena: DP sobre decomposição em árvore (largura ≤ ~10–15) com estados 3^w para dominating set e 2^w para vertex cover/independent set; em grade n × m com n ≤ 12 vira broken profile.
  - *Gatilho:* grafo com estrutura de largura pequena (grade fina, árvore, série-paralelo) e problema de cobertura/dominação/independente.
  - *Por que funciona:* na decomposição em árvore, separar o grafo por um separador de tamanho w permite resumir a interação com o resto por um estado por vértice do separador; a DP combina os sacos.
  - *Como fazer:* para vertex cover/independente, cada vértice tem 2 estados (dentro/fora); para dominating set, 3 (escolhido, coberto por vizinho, ainda não coberto); faça DP nos sacos com introdução, esquecimento e junção. Em grade fina, use perfil coluna a coluna (broken profile) com máscara de n células.
  - *Exemplo:* minimum dominating set em grade 12×m; maximum independent set em árvore de treewidth pequena; vertex cover em grafo série-paralelo.
  - *Armadilha:* o número de estados é 3^w ou 2^w, não 2^{2w}; a ordem de introdução/esquecimento; estados inválidos no esquecimento; a largura pode ser maior que o esperado; broken profile precisa da máscara da fronteira, não da linha inteira.
- **Grafo com treewidth/pathwidth pequena.** Se o enunciado diz "grafo de grade com 5 linhas", "série-paralelo", "cacto": cacto → DP em ciclos (ciclo como nó); série-paralelo → DP por redução (série/paralelo) com tabela por (estado das duas pontas).
  - *Gatilho:* o enunciado fornece uma estrutura de largura pequena (poucas linhas, série-paralelo, cacto, árvore de ciclos).
  - *Por que funciona:* a estrutura permite decompor o grafo em partes com poucos vértices de fronteira; o estado da DP é o comportamento nas pontas/fronteira, e a composição é associativa.
  - *Como fazer:* cacto: faça DFS, trate cada ciclo ao sair do vértice mais alto e combine com a árvore; série-paralelo: reduza arestas em série/paralelo mantendo uma tabela de estados das duas pontas; grade fina: perfil.
  - *Exemplo:* coloração/independente em cacto; caminho em grafo série-paralelo; dominância em grade de 5 linhas.
  - *Armadilha:* identificar corretamente ciclos que compartilham no máximo um vértice; a ordem de processamento dos ciclos; a tabela de estados pode crescer com o número de pontas; um grafo "quase" série-paralelo pode ter K4 e não ser.
- **Cacto / grafo com ciclos que não compartilham aresta.** DP na árvore dfs: cada ciclo é tratado ao sair do vértice mais alto; estados (usado, não usado) por vértice do ciclo, depois mini-DP circular por ciclo (cut no ponto de articulação).
  - *Gatilho:* o grafo é um cacto (cada aresta pertence a no máximo um ciclo simples) e o problema é de DP local (independente, dominante, coloração).
  - *Por que funciona:* contrair cada ciclo a um bloco ligado por articulações transforma o cacto em uma árvore de blocos; a DP resolve cada ciclo dado o estado do ponto de articulação e combina com os filhos.
  - *Como fazer:* faça DFS, marque o vértice mais alto (mais próximo da raiz) de cada ciclo; ao retornar por uma aresta de ciclo, colete os vértices do ciclo e rode uma DP circular com o estado do ponto de corte fixo; combine os resultados na árvore.
  - *Exemplo:* maximum independent set em cacto; coloração de cacto; dominância em cacto.
  - *Armadilha:* ciclos podem compartilhar vértices (articulações), não arestas; a orientação do ciclo e o corte; o estado do ponto de articulação precisa ser consistente entre o ciclo e a árvore; arestas múltiplas/laços.

- **DP sobre "decomposição em cadeias" (Dilworth + largura pequena).** Se a largura de um poset é pequena (k ≤ 10), DP com estado = antichain / frente de cadeias (índices por cadeia) tem O(n^k) estados; usa-se para "scheduling com precedência e poucas máquinas".
  - *Gatilho:* um poset com largura pequena, ou agendamento com precedências e poucas máquinas, e a resposta depende do progresso em cada cadeia.
  - *Por que funciona:* pelo teorema de Dilworth, o poset se particiona em k cadeias; o estado é o índice de progresso em cada cadeia, e a ordem dentro de cada cadeia é fixa. Isso resume todos os ideais já processados.
  - *Como fazer:* decomponha em k cadeias, defina dp[i1][...][ik] = melhor valor com os primeiros i_j elementos de cada cadeia; transicione avançando uma cadeia se os predecessores já foram processados; O(∏ tamanhos) ≤ O(n^k).
  - *Exemplo:* scheduling com precedência e k máquinas; selecionar um ideal de peso máximo em um poset de largura pequena; ordenar tarefas em k filas.
  - *Armadilha:* a decomposição em cadeias precisa respeitar o poset; o estado pode ser grande mesmo com k pequeno; a ordem topológica das cadeias; não confunda largura com número de elementos.
- **Número de ideais (down-sets) pequeno.** Em poset/árvore, estados "conjunto fechado para baixo" crescem, mas para cadeias paralelas k são n^k: DP sobre vetores de progresso por cadeia.
  - *Gatilho:* contar ideais/filtros ou escolher um down-set em um poset com estrutura de poucas cadeias.
  - *Por que funciona:* um ideal é determinado por quanto se avançou em cada cadeia, desde que a escolha seja fechada para baixo; para cadeias paralelas, qualquer vetor de progresso é um ideal, e o número é ∏(n_i+1).
  - *Como fazer:* decomponha em cadeias; dp[v_1][...][v_k] = número/valor de ideais com progresso v; transicione avançando uma cadeia; se há relações entre cadeias, rejeite estados que violem predecessores. Para contagem, some todos os estados válidos.
  - *Exemplo:* número de ideais de um poset de largura k; down-sets em uma grade; contar filtros.
  - *Armadilha:* n^k pode ser enorme mesmo para k=5 e n=100; as relações entre cadeias não podem ser ignoradas; a árvore não é o mesmo que cadeias paralelas; a contagem de ideais é #P-difícil em geral.
- **DP monotônica em ordem topológica + corte mínimo.** Escolher subconjunto fechado de máximo peso (closure) é fluxo, não DP; se o fecho é em árvore (pai exige filho) é DP de subárvore com knapsack — reconheça pela estrutura do poset (árvore = DP, DAG geral = corte).
  - *Gatilho:* escolher um conjunto fechado para baixo (se escolher um vértice, escolher seus predecessores) com peso máximo, em um DAG/poset.
  - *Por que funciona:* o problema de maximum weight closure é redutível a corte mínimo (fonte para pesos positivos, pesos negativos para o sorvedouro, arestas de precedência infinitas); se o poset é uma árvore, a DP de subárvore evita o fluxo.
  - *Como fazer:* geral: construa a rede de corte mínimo, com arestas de capacidade infinita para as implicações, e recupere o conjunto dos vértices do lado da fonte. Árvore: dp[v][0/1] = melhor na subárvore com v fora/dentro, exigindo que o pai dentro implique filho dentro; combine por soma.
  - *Exemplo:* maximum weight closure de um DAG; escolher projetos com pré-requisitos; maximum weight ideal em um poset.
  - *Armadilha:* a direção das arestas de implicação; pesos negativos; um DAG geral não é uma árvore e a DP de subárvore não vale; o corte mínimo pode escolher um conjunto vazio se todos os pesos forem negativos.

Repetidas de outras seções:

- **Mochila com Σ pesos ≤ S** → ≤ √(2S) pesos distintos; agrupe e use mochila limitada (binário ou deque).
- **DP sobre string com padrões** → DP no autômato de Aho-Corasick ou de KMP.
- **Jogo com pontuação** → DP minimax (melhor diferença para quem joga).
- **s repetida k vezes** → DP em 2–3 cópias e extrapole, ou matriz.
- **Aliens** para "exatamente k" com f convexa; confirme convexidade por brute force.
- **LIS por BIT** quando o DP indexa por valor e precisa de peso ou restrição extra.

Teoria dos números e divisores

Os limites pequenos escondidos resolvem metade dos problemas: até 10^9 há no máximo 1344 divisores e 9 primos distintos; até 10^{18}, 15 primos distintos.

- **Divisores:** itere até √x; para todos os x ≤ N, itere múltiplos (N log N harmônico).
  - *Gatilho:* "liste os divisores de x" (um único x) ou "conte/some algo sobre os divisores de todos os x ≤ N".
  - *Por que funciona:* todo divisor d ≤ √x tem par x/d ≥ √x; enumerar d até ⌊√x⌋ cobre cada par uma vez. Para todos os x, o número de pares (x, múltiplo) é Σ_{x≤N} N/x = N·H_N.
  - *Como fazer:* para um x, loop d = 1..⌊√x⌋: se d | x, adicione d e, se d ≠ x/d, x/d (O(√x)). Para todos, loop d = 1..N e para m = d, 2d, 3d, ... ≤ N processe m: O(N log N). Em crivo, comece o loop interno em d^2.
  - *Exemplo:* contar divisores de todos os números até N: τ[m]++ para cada múltiplo; somar divisores: σ[m] += d.
  - *Armadilha:* d·d pode estourar int para x ~ 10^9 (use long long ou d ≤ x/d); não conte o par duas vezes quando x é quadrado.
- **gcd de prefixo/subarray** muda no máximo log(a) vezes → para cada fim, só log valores distintos de gcd.
  - *Gatilho:* "conte subarrays com gcd ...", "mantenha o gcd de todos os subarrays que terminam em i".
  - *Por que funciona:* se g = gcd(S) e um novo elemento entra, o novo gcd divide g; se diminui, cai para um divisor próprio, que é ≤ g/2. Logo há no máximo $\log_2$(max a) valores distintos.
  - *Como fazer:* mantenha pares (gcd, contagem) para os subarrays que terminam em i; calcule gcd(g, a_i) para cada par antigo, funda os iguais e acrescente (a_i, 1). Total O(n log A), com vetor de tamanho ≤ log A.
  - *Exemplo:* contar subarrays com gcd = 1 somando, a cada i, as contagens cujo gcd atual é 1.
  - *Armadilha:* o número de valores distintos é log do valor, não log n; use long long se a_i chega a 10^{18}. Zere/funda os contadores com cuidado.
- **gcd(a, b) = gcd(a, b − a)** → gcd de um array somado com x vira gcd(a_1 + x, diferenças).
  - *Gatilho:* "adicione x a todos os elementos", "gcd do array depois de somar x", "torne o array constante".
  - *Por que funciona:* somar o mesmo x a todos preserva todas as diferenças; pela versão iterada de gcd(a,b)=gcd(a,b−a), gcd(a_1+x, a_2+x, ..., a_n+x) = gcd(a_1+x, a_2−a_1, ..., a_n−a_{n−1}).
  - *Como fazer:* pré-calcule g = gcd de todas as diferenças |a_{i+1}−a_i| (g = 0 se n = 1); a resposta para x é gcd(a_1+x, g). Para contar x, use inclusão-exclusão sobre os primos de g.
  - *Exemplo:* contar x ∈ [1, m] com gcd(a_i+x) = 1 → contar x com gcd(a_1+x, g) = 1.
  - *Armadilha:* use diferenças absolutas e trate n = 1 (g = 0, gcd(a_1+x, 0) = a_1+x); valores negativos exigem abs.
- **lcm cresce rápido**: passa de 10^{18} em poucos passos; corte quando estourar.
  - *Gatilho:* acumular lcm de muitos números ou de subconjuntos, com limite 10^{18}.
  - *Por que funciona:* lcm(a,b) = (a/gcd(a,b))·b e o produto dos primos pequenos cresce exponencialmente: lcm(1..42) ≈ 2,2·10^{17} e lcm(1..43) ≈ 9,4·10^{18}.
  - *Como fazer:* antes de multiplicar, teste a/g > LIM/b por divisão inteira; se sim, estourou. Use __int128 ou trunque em LIM+1 quando só precisa saber se passou.
  - *Exemplo:* lcm(1..42) ainda cabe em 2^{63}; lcm(1..43) já não cabe.
  - *Armadilha:* multiplicar em long long antes de checar é UB; compare por divisão. lcm de dois coprimos grandes estoura já no primeiro passo.
- **Fatore em primos** e trate cada primo independente (expoentes como vetor; gcd = min, lcm = max, produto = soma).
  - *Gatilho:* operações de gcd/lcm/produto sobre muitos números, ou condições que valem por primo.
  - *Por que funciona:* pela fatoração única, cada primo age independentemente; gcd toma o menor expoente, lcm o maior e o produto soma expoentes.
  - *Como fazer:* fatore cada número (SPF, crivo ou Pollard), guarde o vetor/mapa de expoentes e combine coordenada a coordenada; reconstrua multiplicando p^e.
  - *Exemplo:* para que o lcm dos dados seja múltiplo de N = ∏ p^e, algum número precisa ter expoente ≥ e em cada p.
  - *Armadilha:* primos ausentes têm expoente 0; inclua-os na comparação. A reconstrução pode estourar — aplique módulo ou cap.
- **Quadrado perfeito / produto quadrado** → vetor de paridades dos expoentes, xor sobre GF(2) (base linear).
  - *Gatilho:* "produto é quadrado perfeito", "todos os expoentes são pares", "escolha um subconjunto cujo produto é quadrado".
  - *Por que funciona:* um inteiro é quadrado sse todos os expoentes são pares; o produto de dois números é quadrado sse seus vetores de paridade são iguais, isto é, xor zero em GF(2)^π.
  - *Como fazer:* fatore e represente cada número pela máscara de paridades; use hash de 64 bits por primo ou bitset exato. Para dependência linear, use base linear sobre GF(2).
  - *Exemplo:* há subconjunto não vazio com produto quadrado sse os vetores são linearmente dependentes; nesse caso há 2^{n−rank}−1 subconjuntos.
  - *Armadilha:* só a paridade importa — não guarde expoentes. Hash de 64 bits pode colidir; use dois hashes ou representação exata.

- **Contar pares com gcd = g** → conte "múltiplos de d" e faça Möbius/inclusão-exclusão de cima para baixo.
  - *Gatilho:* "quantos pares (i, j) com gcd(i, j) = g", "coprimos", com valores até N.
  - *Por que funciona:* se cnt[d] = # elementos divisíveis por d, os pares com ambos divisíveis por d são C(cnt[d],2) (não ordenados) ou cnt[d]^2 (ordenados); isso conta gcd múltiplo de d. Subtrair os múltiplos maiores isola gcd = d.
  - *Como fazer:* calcule cnt[d] por crivo de múltiplos; para d de N até 1, f[d] = C(cnt[d],2) − Σ_{k≥2} f[kd]. Equivalente: Σ_{d} μ(d)·C(cnt[dg],2).
  - *Exemplo:* pares coprimos em [1,N] = Σ μ(d)⌊N/d⌋^{2} (ordenados) ou (Σ μ(d)⌊N/d⌋^{2} + 1)/2 (não ordenados).
  - *Armadilha:* "de cima para baixo" exige d decrescente; para ordenados use cnt^2, não C(cnt,2). Sinal de μ trocado inverte a resposta.
- **Σ ⌊n/i⌋** → só 2√n valores distintos (divisão em blocos).
  - *Gatilho:* somas com ⌊n/i⌋, ⌊n/i^2⌋, contagens por quociente; n até 10^{12} ou mais.
  - *Por que funciona:* para i ≤ √n há no máximo √n valores; para i > √n, ⌊n/i⌋ < √n, então também há no máximo √n valores.
  - *Como fazer:* para l = 1; l ≤ n; l = r+1: q = ⌊n/l⌋; r = ⌊n/q⌋; o bloco [l,r] tem quociente q; some q·(r−l+1). O(√n).
  - *Exemplo:* Σ_{i≤n}⌊n/i⌋ = 2Σ_{i≤√n}⌊n/i⌋ − ⌊√n⌋^{2} (contagem de pares a·b ≤ n).
  - *Armadilha:* r = n/q na divisão inteira; pare quando q = 0. Para dois limites, o bloco termina em min(⌊n/⌊n/l⌋⌋, ⌊m/⌊m/l⌋⌋).
- **x mod y** com x/2 < y ≤ x é x − y; mod reduz x pelo menos à metade quando y ≤ x.
  - *Gatilho:* analisar a complexidade de Euclides, exponenciação ou qualquer loop com `%`; "o valor cai rápido".
  - *Por que funciona:* se y > x/2, o quociente é 1 e x mod y = x−y < x/2; se y ≤ x/2, x mod y < y ≤ x/2.
  - *Como fazer:* use o fato para provar O(log x) iterações do Euclides e de loops `x %= y`; evite `%` quando y > x (o resto já é x).
  - *Exemplo:* a sequência de restos de Euclides cai pelo menos à metade a cada dois passos.
  - *Armadilha:* a queda vale para y ≤ x; se y > x, nada cai. Em C++, `%` com negativos tem o sinal do dividendo — normalize antes.
- **a^b mod m** com b enorme → Euler: reduza o expoente mod φ(m) (cuidado com gcd ≠ 1).
  - *Gatilho:* expoente com centenas de dígitos, torre de potências, "b é dado como string".
  - *Por que funciona:* para gcd(a,m)=1, a^{φ(m)}≡1, logo a^b≡a^{b mod φ(m)}. Em geral, a^b ≡ a^{(b mod φ(m))+φ(m)} (mod m) quando b ≥ φ(m).
  - *Como fazer:* fatore m para obter φ(m); leia b como string e reduza mod φ(m) detectando se b ≥ φ(m); use exponenciação rápida O(log φ). Para torre, recurse na cadeia de φ.
  - *Exemplo:* 2^{10^100} mod 13: φ(13)=12 e 10^100≡4 (mod 12), então 2^4=16≡3.
  - *Armadilha:* com gcd(a,m)≠1 e b < φ(m), não some φ(m) cegamente; compare b com φ(m) usando valor truncado. m=1 dá resposta 0.
- **Equação linear ax + by = c** → Euclides estendido; sistema de congruências → CRT.
  - *Gatilho:* "existem inteiros x, y", "conte soluções inteiras", "x ≡ a_i (mod m_i)".
  - *Por que funciona:* por Bézout, ax+by=c tem solução sse gcd(a,b) | c; o Euclides estendido dá uma solução particular e o espaço de soluções é uma reta discreta. O CRT combina congruências via Bézout.
  - *Como fazer:* obtenha g=gcd(a,b) e (x_0,y_0) com ax_0+by_0=g; se c%g≠0, não há solução; senão multiplique por c/g. Todas: x=x_0+(b/g)k, y=y_0−(a/g)k. Para CRT, combine aos pares com o algoritmo de módulos não coprimos.
  - *Exemplo:* 6x+10y=4 → g=2 | 4; x=4, y=−2; geral x=4+5k, y=−2−3k.
  - *Armadilha:* sinais e overflow ao multiplicar por c/g; normalize x_0 se pedirem contagem. O CRT exige a checagem de compatibilidade quando os módulos não são coprimos.
- **Primos até 10^7** → crivo linear guardando o menor fator primo.
  - *Gatilho:* fatorar muitos números ≤ 10^7, ou gerar todos os primos até 10^7.
  - *Por que funciona:* cada composto é marcado uma única vez pelo seu menor fator; o crivo linear é O(N) e o SPF permite fatorar em O(log n).
  - *Como fazer:* lp[1]=1; para i=2..N, se lp[i]==0 então lp[i]=i e primes.push(i); para cada p em primes com p ≤ lp[i] e i·p ≤ N, lp[i·p]=p. Fatoração: enquanto n>1, p=lp[n], conte o expoente dividindo.
  - *Exemplo:* 12 → lp[12]=2, 12/2=6; lp[6]=2, 6/2=3; lp[3]=3 → 2^2·3.
  - *Armadilha:* pare o loop interno quando p > lp[i] (ou i·p > N); i·p pode estourar int. lp de 10^7 ints ≈ 40 MB; para 10^8 use crivo ímpar/bit.

- **Paridade de números**, **resíduos mod 2, 3, 4** costumam bastar em construtivos.
  - *Gatilho:* construção com soma/produto/adjacência sem estrutura adicional; "prove que é impossível" ou "construa".
  - *Por que funciona:* muitas condições (soma par, alternância, cor de tabuleiro) são invariantes módulo pequeno; um invariante incompatível prova impossibilidade, e em geral a construção existe quando ele bate.
  - *Como fazer:* calcule o invariante (paridade da soma, cor de xadrez, mod 3/4) no alvo e no estado inicial; se diferirem, impossível. Para construir, ajuste blocos de tamanho 2/3/4 que mudam o invariante na direção certa.
  - *Exemplo:* dominó em tabuleiro cobre cores alternadas; coloração mod 3 para restrições de adjacência.
  - *Armadilha:* compatibilidade não garante construção — teste casos pequenos e a paridade do número de operações. Condição necessária não é suficiente.
- **Fatorar até 10^{18}** → Miller–Rabin + Pollard rho.
  - *Gatilho:* fatorar números de 64 bits, muitos casos, sem crivo até 10^9.
  - *Por que funciona:* Miller–Rabin testa primalidade em O(log^3 n); Pollard rho acha um fator não trivial em ~O(n^{1/4}) esperado pelo paradoxo do aniversário.
  - *Como fazer:* tire primos pequenos por divisão; teste MR; se composto, rode Brent até achar d; recursão em d e n/d. Use __int128 para mulmod e gcd.
  - *Exemplo:* fatorar um n = p·q com p, q ~ 10^9 leva ~3·10^4 iterações esperadas.
  - *Armadilha:* Pollard pode ciclar; mude a constante e reinicie. Nunca rode rho em primo — MR antes. Trate n=1, primos pequenos e potências de primo.

---

adicionado em 2026-11-10 por agentes; revisado

- **Crivo linear para funções multiplicativas.** Guarde o menor fator lp[i]; ao gerar i·p: φ(ip) = φ(i)·p se p | i, senão φ(i)·(p−1); μ(ip) = 0 se p | i, senão −μ(i). Para σ, τ guarde também a parte da maior potência de p em i.
  - *Gatilho:* computar φ, μ, τ ou σ para todo i ≤ N em O(N).
  - *Por que funciona:* cada i·p é gerado uma vez com p = lp(i·p); as fórmulas decorrem da multiplicatividade e do expoente de p em i.
  - *Como fazer:* inicialize φ[1]=μ[1]=1, τ[1]=1. Se p | i: φ[ip]=φ[i]·p, μ[ip]=0. Senão: φ[ip]=φ[i]·(p−1), μ[ip]=−μ[i]. Para τ, guarde cnt[i] = expoente de lp[i] em i; se p | i, τ[ip]=τ[i]/(cnt[i]+1)·(cnt[i]+2) e cnt[ip]=cnt[i]+1; senão τ[ip]=2τ[i] e cnt[ip]=1.
  - *Exemplo:* N=10: φ = 1,1,2,2,4,2,6,4,6,4; μ = 1,−1,−1,0,−1,1,−1,0,0,1.
  - *Armadilha:* a condição das fórmulas é p | i (p = lp[i]), não p ≤ lp[i]. Para σ, guarde a soma da maior potência p^e e o fator correspondente.
- **Crivo segmentado.** Primos de [L, R] com R ≤ 10^{12} e R−L ≤ 10^7: crive com primos ≤ √R; para fatorar/φ de todo o intervalo, divida cada rem[x] pelos primos e, se sobrar > 1, o resto é um primo.
  - *Gatilho:* intervalo [L,R] grande em valor e curto em comprimento; "primos entre L e R", "φ(n) para n no intervalo".
  - *Por que funciona:* todo composto em [L,R] tem fator ≤ √R; marcar os múltiplos de cada primo base cobre todos.
  - *Como fazer:* gere primos até √R; isPrime[0..R−L]; para cada p, comece em max(p^2, ⌈L/p⌉·p) e marque de p em p. Custo O((R−L) log log R + √R). Para fatorar, inicialize rem[i]=L+i e divida pelos primos.
  - *Exemplo:* primos em [10^{12}, 10^{12}+10^6] usando primos até 10^6.
  - *Armadilha:* L=0 ou 1 exige marcar 0 e 1; use long long em L+i e no início. Calcule ⌈L/p⌉·p sem estourar.
- **Miller–Rabin determinístico.** Com as 12 primeiras bases primas (2…37) vale para n < 3,3·10^{24}, logo para todo 64 bits; para n < 2^{32} bastam 2, 7, 61. Precisa de mulmod com __int128.
  - *Gatilho:* testar primalidade de n até 64 bits muitas vezes.
  - *Por que funciona:* para cada base a, se a cadeia de quadrados a partir de a^d não atinge ±1, n é composto; o conjunto de bases dado não tem pseudoprimo forte na faixa de 64 bits.
  - *Como fazer:* escreva n−1=d·2^s com d ímpar; para cada base a, calcule x=a^d mod n; se x==1 ou x==n−1 continue; eleve ao quadrado s−1 vezes; se chegar a n−1 continue; senão é composto. Descarte n<2 e pares pequenos.
  - *Exemplo:* n=2^{61}−1 é primo; MR confirma com poucas bases.
  - *Armadilha:* reduza a base mod n e descarte a≡0; a·a em long long estoura — use __int128. Sempre teste antes de chamar Pollard.

- **Pollard rho (Brent).** Fatora n ≤ 10^{18} em ~n^{1/4} passos; tire primos pequenos por divisão, teste primalidade antes de cada recursão, acumule produtos para gcd em lote. Recursão: se achou d, fatore d e n/d.
  - *Gatilho:* fatoração de 64 bits, muitos números, depois de remover primos pequenos.
  - *Por que funciona:* a sequência x_{i+1}=x_i^2+c mod n colide módulo um fator p em ~√p passos; gcd(|x_i−x_j|, n) revela p. Brent acelera com saltos em potências de 2.
  - *Como fazer:* se n%2==0, retorne 2; escolha c aleatório; x=y=2, q=1; dobre o passo, atualize x=f(x), y=f(f(y)); acumule q=q·|x−y| mod n e calcule gcd em lote; se gcd>1, refine. Recursão em d e n/d.
  - *Exemplo:* n=p·q com p,q ~10^9 → ~3·10^4 iterações esperadas.
  - *Armadilha:* se gcd=n, o lote juntou fatores demais — volte ao passo a passo. MR antes de rho; c=0 ou c=−2 podem falhar. Use mulmod __int128.
- **Fatorar muitos números ≤ 10^7.** Crivo de menor fator primo + loop de divisões = O(log) por número; n ≤ 10^{12} → divisão por primos ≤ 10^6 (78498 primos) e o resto é 1 ou primo (para n ≤ 10^{18} o resto é 1, primo, p^2 ou p·q).
  - *Gatilho:* entrada com muitos n (10^5–10^6) e limite 10^7, 10^{12} ou 10^{18}.
  - *Por que funciona:* com SPF até 10^7, cada divisão remove um fator e reduz o número; para 10^{12}, após dividir por todos os primos ≤10^6, o resto não pode ter dois fatores >10^6.
  - *Como fazer:* pré-processe SPF até 10^7; para cada n, divida por lp[n] contando expoentes. Para 10^{12} sem SPF, itere primos até p ≤ n/p; se sobrar n>1, é primo. Para 10^{18}, após primos ≤10^6, o resto é 1, primo, p^2 ou p·q: teste MR; se composto, verifique se é quadrado perfeito (p^2); senão é p·q.
  - *Exemplo:* 10^5 números ≤10^7 com SPF custam ~10^5·log n divisões.
  - *Armadilha:* p·p estoura int perto de 10^9; use p ≤ n/p. Não esqueça o resto >1 como fator primo.
- **Soma sobre divisores via crivo de primos (Dirichlet prefix sum).** Para cada primo p, para i = 1..N/p crescente: F[i·p] += F[i] dá F'[n] = Σ_{d|n} f(d) em O(N log log N). Sobre múltiplos: i decrescente, F[i] += F[i·p]. Inverter = subtrair na ordem reversa (é o Möbius).
  - *Gatilho:* precisa de zeta/Möbius sobre divisores ou múltiplos para todo n ≤ N (N ~ 10^7).
  - *Por que funciona:* cada primo é processado uma vez; a ordem crescente propaga as contribuições por todos os expoentes, e o produto sobre os primos gera exatamente os divisores.
  - *Como fazer:* copie f para F. Divisores: para cada p e i crescente, F[i·p]+=F[i]. Múltiplos: para cada p e i decrescente, F[i]+=F[i·p]. Para inverter, percorra os primos na ordem inversa e faça i decrescente subtraindo.
  - *Exemplo:* f=1 → F[n]=τ(n); f=φ → F[n]=n; f=μ → F=ε.
  - *Armadilha:* a ordem crescente/decrescente é essencial, e a inversão também inverte a ordem dos primos. Use long long (σ pode passar de 32 bits).
- **φ e somas com φ.** Σ_{d|n} φ(d) = n; Σ_{i≤n, gcd(i,n)=1} i = n·φ(n)/2 (n > 1); Σ_{i=1}^n gcd(i, n) = Σ_{d|n} d·φ(n/d); φ(n) ≥ √(n/2) e φ(n) é par para n ≥ 3.
  - *Gatilho:* somas com coprimos, gcd(i,n), ou provas de complexidade que precisam de lower bound para φ.
  - *Por que funciona:* os i com gcd(i,n)=d são coprimos com n/d; contar esses i dá φ(n/d), e cada um contribui d. A identidade Σφ(d)=n vem de contar as frações 1/n..n/n por denominador reduzido; na soma dos coprimos, i e n−i se cancelam.
  - *Como fazer:* fatore n e aplique as fórmulas; para somas em intervalo, agrupe por d|n. Use φ(n) ≥ √(n/2) como lower bound.
  - *Exemplo:* n=12: Σφ(d)=12; Σ_{i=1}^{12}gcd(i,12)=40 = 1·4+2·2+3·2+4·1+6·1+12·1.
  - *Armadilha:* a fórmula da soma dos coprimos vale para n>1 (n=1 dá 0, não 1/2); Σφ(d)=n é sobre divisores, não sobre d≤n.
- **Σ_{i,j≤n} gcd(i, j) = Σ_d φ(d)·⌊n/d⌋².** Vem de gcd = Σ_{d|gcd} φ(d). Com Möbius: pares coprimos = Σ μ(d)⌊n/d⌋² = 2Φ(n) − 1 (Φ = soma de φ).
  - *Gatilho:* somas de gcd sobre todos os pares até n; contagem de coprimos.
  - *Por que funciona:* escreva gcd(i,j)=Σ_{d|i,d|j}φ(d); trocando a ordem, cada d contribui φ(d) para cada par de múltiplos de d, que são ⌊n/d⌋².
  - *Como fazer:* pré-calcule φ até n e some φ(d)⌊n/d⌋²; O(n), ou O(√n) com prefixos de φ e blocos. Para coprimos, troque φ por μ.
  - *Exemplo:* n=2: Σgcd = 1+1+1+2=5; Σφ(d)⌊2/d⌋² = φ(1)·4+φ(2)·1 = 5. Pares coprimos = 3 = 2(1+1)−1.
  - *Armadilha:* a soma é sobre pares ordenados (i,j), incluindo i=j; para não ordenados, ajuste. O resultado cresce ~n^2 log n — use long long.

- **Möbius com dois limites.** #{(i≤n, j≤m): gcd = 1} = Σ μ(d)⌊n/d⌋⌊m/d⌋; agrupe d em blocos onde ambos ⌊n/d⌋ e ⌊m/d⌋ são constantes: O(√n + √m) com prefixos de μ.
  - *Gatilho:* contar pares coprimos com limites diferentes, ou somas com dois parâmetros.
  - *Por que funciona:* o indicador de coprimalidade é Σ_{d|gcd(i,j)}μ(d); para cada d há ⌊n/d⌋⌊m/d⌋ pares.
  - *Como fazer:* prefixo de μ até max(n,m); para l=1..min(n,m), qn=⌊n/l⌋, qm=⌊m/l⌋, r=min(⌊n/qn⌋,⌊m/qm⌋); some (M[r] −M[l−1])·qn·qm e pule para r+1.
  - *Exemplo:* n=2, m=3: coprimos = 5; Σ μ(d)⌊2/d⌋⌊3/d⌋ = 6−1 = 5.
  - *Armadilha:* o bloco termina no mínimo dos dois quocientes; usar só n quebra. M[0]=0 e μ(1)=1. Se n=0 ou m=0, resposta 0.
- **Pares coprimos ponderados.** Σ_{i,j≤n, gcd=1} i·j = Σ_d μ(d)d²·S(⌊n/d⌋)², S(m) = m(m+1)/2; e Σ_{i,j≤n} lcm(i, j) = Σ_g g·C(⌊n/g⌋), C(m) = Σ_d μ(d)d²S(⌊m/d⌋)².
  - *Gatilho:* somas de produtos ou de lcm sobre pares, com n até 10^6–10^7.
  - *Por que funciona:* Möbius força d|i,j; escrevendo i=d·i', j=d·j', o peso i·j vira d²·i'j'. Para lcm, use lcm(i,j)=ij/gcd(i,j) e agrupe por g=gcd.
  - *Como fazer:* pré-calcule μ e S; para a primeira soma, itere d ou agrupe por quociente com prefixo de μ(d)d². Para lcm, calcule C(m) nos m=⌊n/g⌋ e some g·C(⌊n/g⌋).
  - *Exemplo:* n=2: coprimos ponderados = 1+2+2 = 5; fórmula: d=1 → S(2)²=9; d=2 → −4·S(1)²=−4; total 5.
  - *Armadilha:* S(m)² e d² estouram rápido; reduza módulo se pedirem. C(m) já é a soma ponderada dos pares coprimos ≤ m.
- **k-tuplas com gcd 1.** #{tuplas em [1, n]^k com gcd = 1} = Σ_d μ(d)⌊n/d⌋^k. Com gcd exatamente g: use n/g.
  - *Gatilho:* contar k-uplas (k fixo, às vezes 2 ou 3) com gcd 1 ou gcd g.
  - *Por que funciona:* o indicador de gcd 1 por Möbius; para cada d, todas as coordenadas são múltiplas de d, dando ⌊n/d⌋^k tuplas.
  - *Como fazer:* pré-calcule μ até n e some μ(d)⌊n/d⌋^k; agrupe por quociente para O(√n) se k grande, ou faça O(n) direto. Para gcd=g, substitua n por ⌊n/g⌋.
  - *Exemplo:* n=2, k=2: tuplas com gcd 1 = 3; fórmula: d=1 → 4; d=2 → −1; total 3.
  - *Armadilha:* a potência estoura; use módulo/bigint. "gcd exatamente g" não é "todas as coordenadas múltiplas de g" — aplique Möbius em n/g.
- **Inversão de Möbius (as duas formas).** g(n) = Σ_{d|n} f(d) ⇔ f(n) = Σ_{d|n} μ(n/d)g(d); G(n) = Σ_{n|m} F(m) ⇔ F(n) = Σ_{n|m} μ(m/n)G(m). "Soma sobre múltiplos" é o caso que aparece em contagem de gcd.
  - *Gatilho:* você calculou a soma sobre divisores/múltiplos e precisa recuperar o termo individual (ex.: contou múltiplos, quer gcd exato).
  - *Por que funciona:* μ é a inversa de Dirichlet de 1 (1∗μ=ε); convoluir com 1 é invertido convoluindo com μ, tanto na ordem de divisores quanto na de múltiplos.
  - *Como fazer:* forma de divisores: f[n]=Σ_{d|n}μ(n/d)g[d]. Forma de múltiplos: processe n decrescente e F[n]=Σ_{k≥1}μ(k)G[nk]. O(N log N) direto ou O(N log log N) com crivo de primos.
  - *Exemplo:* g(n)=n (soma de φ nos divisores) → f(n)=φ(n) via Σ μ(n/d)d.
  - *Armadilha:* o argumento de μ é n/d (ou m/n), não d; a forma de múltiplos soma sobre múltiplos de n, não sobre divisores.
- **Convolução de Dirichlet: identidades úteis.** 1∗μ = ε, 1∗φ = id, μ∗id = φ, 1∗1 = τ, 1∗id = σ, id_k∗μ = J_k. Se f = g∗h e você sabe os prefixos de g e h, o prefixo de f sai por Σ_{d≤n} h(d)·G(⌊n/d⌋).
  - *Gatilho:* manipular somas multiplicativas, simplificar expressões com gcd/φ/μ, ou calcular prefixos de convoluções.
  - *Por que funciona:* (g∗h)(n)=Σ_{d|n}g(d)h(n/d); as identidades vêm de contar divisores ou de Σ_{d|n}φ(d)=n. No prefixo, troca-se a ordem: Σ_{n≤x}Σ_{d|n} = Σ_{d≤x}h(d)Σ_{m≤x/d}g(m).
  - *Como fazer:* substitua a função problemática pela convolução e troque a ordem das somas; para prefixos, S_f(x)=Σ_{d≤x}h(d)G(⌊x/d⌋), agrupando quocientes.
  - *Exemplo:* Σ_{n≤x}σ(n)=Σ_{d≤x}d⌊x/d⌋ (1∗id); Σφ(n)=Σ μ(d)S(⌊x/d⌋).
  - *Armadilha:* convolução é comutativa, mas a ordem escolhida muda a complexidade; cuidado com id_k(n)=n^k versus 1. ε(1)=1 e ε(n)=0 para n>1.

- **Hipérbole de Dirichlet.** $\Sigma_{ab\le n}\, F(a)G(b)$ em $O(\sqrt{n})$: $\Sigma_{a\le A}\, f(a)G(n/a) + \Sigma_{b\le B}\, g(b)F(n/b) - F(A)G(B)$ com $A\cdot B \le n < (A+1)(B+1)$. Dá $\Sigma\, \tau(i) = 2\Sigma_{i\le\sqrt{n}}\lfloor n/i\rfloor - \lfloor\sqrt{n}\rfloor^2$.
  - *Gatilho:* somar f(a)g(b) sobre pares com ab≤n, ou prefixos de convolução, quando n é grande (10^{10}–10^{12}).
  - *Por que funciona:* o conjunto ab≤n é coberto por {a≤A} ∪ {b≤B} quando (A+1)(B+1)>n; a interseção {a≤A,b≤B} é contada duas vezes e é subtraída.
  - *Como fazer:* escolha $A=B=\lfloor\sqrt{n}\rfloor$; calcule os prefixos F,G; some $f(a)G(\lfloor n/a\rfloor)$ para a≤A, $g(b)F(\lfloor n/b\rfloor)$ para b≤B e subtraia F(A)G(B). $O(\sqrt{n})$ avaliações.
  - *Exemplo:* f=g=1, F(a)=a, G(b)=b → $\Sigma_{ab\le n}1 = 2\Sigma_{a\le\sqrt{n}}\lfloor n/a\rfloor - \lfloor\sqrt{n}\rfloor^2 = \Sigma_{i\le n}\tau(i)$.
  - *Armadilha:* a fórmula é para Σ f(a)g(b) sobre ab≤n; os prefixos F,G aparecem apenas no lado direito, não como fatores no lado esquerdo. Use divisão inteira e long long em F(A)G(B).
- **Prefixo de φ ou μ até 10^{10}–10^{11} (sieve de Dirichlet).** $M(n) = 1 - \Sigma_{d\ge2}\, M(\lfloor n/d\rfloor)$ e $\Phi(n) = n(n+1)/2 - \Sigma_{d\ge2}\, \Phi(\lfloor n/d\rfloor)$; crive até $n^{2/3}$ e memorize o resto por $\lfloor n/i\rfloor$: $O(n^{2/3})$.
  - *Gatilho:* precisa de $\Sigma_{i\le n}\mu(i)$ (Mertens) ou $\Sigma\varphi(i)$ para n até 10^{10}–10^{11}.
  - *Por que funciona:* $\Sigma_{d\le n}M(\lfloor n/d\rfloor)=1$ (de $1*\mu=\varepsilon$ somado) e $\Sigma_{d\le n}\Phi(\lfloor n/d\rfloor)=n(n+1)/2$ (de $1*\varphi=\mathrm{id}$ somado); isolar o termo d=1 dá a recursão.
  - *Como fazer:* pré-calcule M,Φ até $K=n^{2/3}$; memoize por $\lfloor n/i\rfloor$; na recursão, agrupe d em blocos de quociente constante: $\Sigma (r-l+1)\cdot S(\lfloor n/l\rfloor)$.
  - *Exemplo:* M(10) = −1 (soma de μ(1)..μ(10)).
  - *Armadilha:* a recursão soma d=2..n; sem memoização por quociente volta a O(n). $O(n^{2/3})$ exige o crivo até $n^{2/3}$, não até n.
- **Truque geral do prefixo.** Para f difícil, ache g com prefixo fácil e f∗g fácil: $(\mathrm{id}^k\cdot\varphi)*\mathrm{id}^k = \mathrm{id}^{k+1}$; $(\mathrm{id}\cdot\mu)*\mathrm{id} = \varepsilon$. Então $S_f(n) = [S_{f*g}(n) - \Sigma_{d\ge2}\, g(d)S_f(\lfloor n/d\rfloor)]/g(1)$.
  - *Gatilho:* prefixo de uma função multiplicativa f sem fórmula direta, mas com uma convolução conhecida.
  - *Por que funciona:* somando $(f*g)(n)=\Sigma_{d\mid n}g(d)f(n/d)$ em n≤x e trocando a ordem, $S_{f*g}(x)=\Sigma_{d\le x}g(d)S_f(\lfloor x/d\rfloor)$; isole d=1.
  - *Como fazer:* escolha g com S_{f*g} e S_g fáceis; calcule S_f recursivamente com memoização nos $\lfloor x/d\rfloor$ e blocos de quociente; divida por g(1).
  - *Exemplo:* f=φ, g=1 → f∗g=id e $S_{\mathrm{id}}=n(n+1)/2$, recuperando Φ; f=id·μ, g=id → f∗g=ε.
  - *Armadilha:* g(1) precisa ser invertível; se g(1)=0, não serve. Memoize por quociente, senão a recursão não fecha em $O(n^{2/3})$.
- **Lucy_Hedgehog.** Contagem de primos ≤ n e Σp, Σp^2 em $O(n^{3/4})$ sobre os $2\sqrt{n}$ valores $\lfloor n/i\rfloor$: $S(v, p) = S(v, p-1) - f(p)[S(\lfloor v/p\rfloor, p-1) - S(p-1, p-1)]$ para $v \ge p^2$; f completamente multiplicativa (p^k).
  - *Gatilho:* contar primos ou somar p^k sobre primos até n com n até 10^{10}–10^{11}.
  - *Por que funciona:* é um crivo sobre os valores $\lfloor n/i\rfloor$; ao processar o primo p, remove os compostos cujo menor fator é p, subtraindo contribuições de $S(\lfloor v/p\rfloor)$ que ainda incluem primos < p.
  - *Como fazer:* inicialize $S(v)=\Sigma_{2\le i\le v}f(i)$ pela fórmula fechada; para cada primo $p\le\sqrt{n}$, para cada $v\ge p^2$ (valores decrescentes), faça $S(v) \mathrel{-}= f(p)\cdot(S(\lfloor v/p\rfloor) - S(p-1))$. Ao final $S(n)=\Sigma_{p\le n}f(p)$.
  - *Exemplo:* f(i)=1 → π(n); f(i)=i → Σp.
  - *Armadilha:* só atualize $v \ge p^2$; use a indexação lo/hi para achar $\lfloor v/p\rfloor$; f precisa ser completamente multiplicativa (p^k), não uma função qualquer.
- **Min_25.** Fase 1 = Lucy, exige f(p) polinomial em p (soma de f sobre primos); fase 2 soma qualquer f multiplicativa com $f(p^e)$ calculável, por DFS/DP sobre $\lfloor n/i\rfloor$; n até 10^{10}–10^{12}.
  - *Gatilho:* somar f(n) para n até 10^{10}–10^{12}, com f multiplicativa e f(p) polinomial.
  - *Por que funciona:* decompõe a soma em primos (fase 1, Lucy) e em compostos por recursão sobre o menor primo e seu expoente; cada estado é um quociente $\lfloor n/i\rfloor$.
  - *Como fazer:* a fase 1 calcula $G_k(v)=\Sigma_{p\le v}p^k$ para cada monômio de f(p); a fase 2 define S(v,j) para primos ≥ p_j e recursa escolhendo p_j^e e o próximo primo.
  - *Exemplo:* $\Sigma_{i\le n}\varphi(i)$ com $n=10^{10}$; f(p)=p−1 é polinomial.
  - *Armadilha:* a fase 1 só funciona se f(p) for combinação de potências; a fase 2 exige $f(p^e)$ direto. Índices de primos e memoização por (v,j) são a parte delicada.

- **Powerful numbers.** Se f multiplicativa e existe g multiplicativa com f(p) = g(p) e prefixo de g fácil, então f = g∗h com h só em números "powerful" (todo expoente ≥ 2, são O(√n)): Σf = Σ_{m powerful} h(m)·G(⌊n/m⌋).
  - *Gatilho:* somar f multiplicativa até n grande quando f(p) tem forma simples mas f(p^e) não.
  - *Por que funciona:* h=f∗g^{−1}; como f(p)=g(p), h(p)=0 para todo primo, então h só é não nulo em números com todos os expoentes ≥2 — os powerful, que são O(√n).
  - *Como fazer:* escolha g com prefixo G fácil (ex.: g=1, G(x)=x); gere os powerful por DFS sobre p^e (e≥2) e calcule h(m) multiplicativamente a partir de h(p^e)=f(p^e)−Σ_{i<e}h(p^i)g(p^{e−i}); some h(m)G(⌊n/m⌋).
  - *Exemplo:* f=φ: f(p)=p−1; escolha g=id, G(x)=x(x+1)/2; h só em powerful.
  - *Armadilha:* h(p^e) precisa ser invertido corretamente na potência; limite $p^2 \le n$ na DFS. O número de powerful é ~√n, mas a DFS ainda percorre primos pequenos.
- **Floor sum (Euclid-like).** Σ_{i=0}^{n−1} ⌊(a·i + b)/m⌋ em O(log m); trocando papéis (a ≥ m → reduza; senão troque i e o valor). Variantes com Σ i·⌊·⌋ e Σ $\lfloor\cdot\rfloor^2$ também são recursivas.
  - *Gatilho:* somas de pisos de progressão aritmética, contagens de pontos de rede, somas de `(a·i+b) mod m`.
  - *Por que funciona:* a região sob a reta y=(a·i+b)/m pode ser contada por colunas ou por linhas; a troca de papéis reduz (a,m) como no Euclides, daí O(log m).
  - *Como fazer:* reduza a e b módulo m; se a·n+b<m, pare; conte as colunas cheias e chame a recursão com os papéis trocados, como na implementação padrão de floor_sum. Use __int128 quando a·n estourar.
  - *Exemplo:* Σ_{i=0}^{3}⌊(3i+2)/5⌋ = 0+1+1+2 = 4.
  - *Armadilha:* a fórmula recursiva é fácil de errar em sinais; teste contra força bruta. Inclua i=0 e i=n−1 corretamente.
- **Onde floor sum aparece.** Pontos de rede sob uma reta; Σ (a·i + b) mod m = Σ(a·i + b) − m·floorsum; contar (x, y) com a·x + b·y ≤ n; mínimo de (a·i + b) mod m para i ≤ n (versão Euclid-like).
  - *Gatilho:* contagem de pontos inteiros, somas de restos de PA, minimização de resto.
  - *Por que funciona:* (a·i+b) mod m = a·i+b − m⌊(a·i+b)/m⌋; a soma dos restos vira soma aritmética menos m·floor_sum. Contar pontos sob reta é somar pisos.
  - *Como fazer:* identifique a soma de pisos e aplique floor_sum; para pontos de rede, some sobre x o número de y válidos. Para o mínimo do resto, use a versão Euclid-like que recursa com (m,a).
  - *Exemplo:* contar (x,y)≥0 com 2x+3y≤n → Σ_{x=0}^{⌊n/2⌋}(⌊(n−2x)/3⌋+1).
  - *Armadilha:* limites fechados/abertos mudam o número de termos; restos com negativos precisam normalização.
- **Blocos de ⌊n/i⌋: propriedades.** ⌊⌊n/a⌋/b⌋ = ⌊n/(ab)⌋, então o conjunto de valores é fechado; ⌈n/i⌉ = ⌊(n−1)/i⌋ + 1. Indexe DPs por valor v ≤ √n em lo[v] e por n/v em hi[n/v].
  - *Gatilho:* implementar crivo de Dirichlet, Lucy ou Min_25; somas por quociente.
  - *Por que funciona:* a identidade de pisos garante que ⌊n/(ab)⌋ também é da forma ⌊n/k⌋; logo o conjunto {⌊n/i⌋} tem ≤2√n elementos. A divisão por n/v indexa o complemento.
  - *Como fazer:* colete v=1..⌊√n⌋ em lo[v] e os valores >√n em hi[n/v]; mapeie x para lo[x] se x≤√n, senão hi[n/x]. Percorra na ordem desejada usando essa indexação.
  - *Exemplo:* n=10, √n=3: lo[1]=1, lo[2]=2, lo[3]=3; hi[1]=10, hi[2]=5.
  - *Armadilha:* calcule √n com sqrtl e ajuste; lo e hi podem compartilhar o valor √n — evite contar duas vezes. ⌈n/i⌉ exige n≥1.
- **Somas clássicas por blocos.** Σ_{i≤n} σ(i) = Σ d⌊n/d⌋; Σ_{k≤n} (n mod k) = n^2 − Σ k⌊n/k⌋; Σ_{i≤n} τ(i) ≈ n ln n + (2γ−1)n.
  - *Gatilho:* somatórios de funções divisoras até n grande (10^9–10^{12}).
  - *Por que funciona:* trocando a ordem, a soma de σ(i) conta cada d em todos os múltiplos ≤n (⌊n/d⌋ vezes); n mod k = n − k⌊n/k⌋; τ tem a mesma troca de ordem.
  - *Como fazer:* use blocos de ⌊n/d⌋ para O(√n): Σ d⌊n/d⌋ = Σ_{bloco}(Σ_{d=l}^r d)·q. Para Σ(n mod k), calcule n^2 − Σ k⌊n/k⌋.
  - *Exemplo:* n=5: Σσ=1+3+4+7+6=21 = 5+4+3+4+5.
  - *Armadilha:* n^2 estoura 64 bits quando n > √(2^{63}−1) ≈ 3,04·10^9; use __int128. A assíntota de τ não é exata para n pequeno.
- **CRT com módulos não coprimos.** x ≡ a_1 (m_1), x ≡ a_2 (m_2) tem solução sse g = gcd | a_2 − a_1; então x = a_1 + m_1·[(a_2−a_1)/g · inv(m_1/g mod m_2/g) mod (m_2/g)], novo módulo lcm. Detecte lcm > 10^{18} (use __int128).
  - *Gatilho:* combinar congruências com módulos arbitrários, não necessariamente primos entre si.
  - *Por que funciona:* a congruência módulo lcm é equivalente ao par; dividindo por g, a compatibilidade é g | (a_2−a_1) e a equação reduzida tem inverso bem definido.
  - *Como fazer:* combine aos pares; calcule o inverso de m_1/g módulo m_2/g via Euclides estendido; atualize a e M=lcm; se M estourar o limite, pare. Verifique a condição antes de inverter.
  - *Exemplo:* x≡1 (4), x≡3 (6): g=2 | 2; x=1+4·[1·inv(2 mod 3)] = 1+4·2 = 9 (mod 12).
  - *Armadilha:* o inverso é de m_1/g módulo m_2/g, não de m_1 módulo m_2; se g∤diferença não há solução. Normalize o resultado para [0,M).

- **Garner.** Reconstrói um número grande (ou mod M qualquer) a partir de resíduos em módulos primos entre si por base mista, sem bigint: usado em NTT com 3 primos para resposta mod arbitrário.
  - *Gatilho:* precisa do valor exato (ou mod M) de uma soma/produto grande, mas só tem resíduos em primos pequenos.
  - *Por que funciona:* o isomorfismo CRT diz que os resíduos determinam o número módulo o produto; Garner reconstrói dígito a dígito na base mista, mantendo tudo em 64 bits quando possível.
  - *Como fazer:* com módulos $m_1..m_k$, calcule $x = x_1 + m_1t_2 + m_1m_2t_3 + \ldots$; cada t_i é obtido subtraindo as contribuições anteriores e multiplicando pelo inverso de $m_1\ldots m_{i-1}$ mod m_i. Para resposta mod M, reduza cada coeficiente.
  - *Exemplo:* NTT com 998244353, 1004535809 e 469762049 para resposta mod 10^9+7.
  - *Armadilha:* o produto dos módulos precisa superar o valor verdadeiro para reconstrução exata; caso contrário trabalhe mod M desde o início. Inversos exigem módulos coprimos.
- **Euclides estendido: todas as soluções.** $ax + by = c$ tem solução sse $g \mid c$; $x = x_0 + k\cdot b/g$, $y = y_0 - k\cdot a/g$. Contar soluções com $x \in [x_1, x_2]$, $y \in [y_1, y_2]$ = interseção de dois intervalos de k (cuidado com floor/ceil de negativos).
  - *Gatilho:* contar soluções inteiras de uma equação diofantina em caixas.
  - *Por que funciona:* o conjunto de soluções é uma reta discreta parametrizada por k; as restrições em x e y viram intervalos em k, e a contagem é o tamanho da interseção.
  - *Como fazer:* obtenha (x_0,y_0) por Euclides estendido; se $c\%g\neq0$, zero. Para x: $k \geq \lceil (x_1-x_0)/(b/g) \rceil$ e $k \leq \lfloor (x_2-x_0)/(b/g) \rfloor$ (inverta se $b/g<0$); idem para y; intersecte e conte max(0, hi−lo+1).
  - *Exemplo:* $3x+5y=8$, $x,y\geq0$: $x=1+5k$, $y=1-3k$; só $k=0 \to (1,1)$.
  - *Armadilha:* divisão com negativos em C++ trunca para zero, não para $-\infty$; implemente floor_div/ceil_div. b/g negativo inverte as desigualdades.
- **Inverso em lote.** Produtos de prefixo, um único inverso do total, volta multiplicando: O(n + log). Para 1..n: $inv[i] = -(p/i)\cdot inv[p \bmod i]$; para fatoriais: invfact[n] = fact[n]^{−1} e desce multiplicando por i.
  - *Gatilho:* precisa de n inversos módulo p (p primo), ou de invfact para combinações.
  - *Por que funciona:* com $prefix[n]=a_1\ldots a_n$, $inv(prefix[n])$ dá o inverso de todos ao voltar: $inv(a_i)=prefix[i-1]\cdot inv(prefix[i])$. A recorrência de inv[i] vem de $p = \lfloor p/i \rfloor \cdot i + (p \bmod i)$.
  - *Como fazer:* batch: prefixos, um pow(prefix[n], p−2), desça. Linear: $inv[1]=1$; $inv[i]=(p-(p/i)\cdot inv[p\%i]\%p)\%p$. Fatoriais: $invfact[n]=pow(fact[n],p-2)$ e desça $invfact[i-1]=invfact[i]\cdot i$.
  - *Exemplo:* p=7: inv[2]=4, inv[3]=5, inv[4]=2, inv[5]=3, inv[6]=6.
  - *Armadilha:* a fórmula linear exige p primo e $i<p$; se $i\geq p$, o inverso não existe. O produto intermediário pode ser p antes da redução.
- **Ordem multiplicativa.** ord_n(a) só existe com gcd(a, n) = 1 (senão há pré-período); ord_n(a) | φ(n) (ou λ(n)); calcule começando em φ e, para cada primo q de φ, divida por q enquanto a^{x/q} ≡ 1. Com gcd(a, n) = 1, todo ciclo de a^k mod n tem período ord.
  - *Gatilho:* período de potências mod n, menor k com $a^k\equiv1$, ou reduzir expoentes em relação a ord.
  - *Por que funciona:* o grupo das unidades mod n tem ordem φ(n); por Lagrange, a ordem de a divide φ(n). O teste de divisão remove fatores primos desnecessários.
  - *Como fazer:* fatore φ(n); x=φ(n); para cada primo q|φ(n), enquanto x%q==0 e powmod(a,x/q,n)==1, x/=q. Custo O(ω(φ)·log φ·log n).
  - *Exemplo:* n=7, a=2: $\varphi=6$; $2^3=8\equiv1 \to x=3$; q=3, $2^1\neq1 \to ord=3$.
  - *Armadilha:* se $a\equiv1$, ord=1; para n=1 convencione ord=1. Se gcd≠1, a sequência tem pré-período e o período não é a ordem.
- **Raiz primitiva.** Existe sse n = 1, 2, 4, p^k, 2p^k com p primo ímpar; g é raiz mod p sse g^{(p−1)/q} ≢ 1 para todo primo q | p−1; a menor costuma ser pequena (tente 2, 3, 5...); há φ(p−1) raízes.
  - *Gatilho:* precisa de um gerador do grupo multiplicativo mod n, ou de logaritmo discreto.
  - *Por que funciona:* $(\mathbb{Z}/n\mathbb{Z})^\times$ é cíclico exatamente nesses casos; g é gerador sse sua ordem é φ(n), o que se testa dividindo φ pelos fatores primos.
  - *Como fazer:* fatore p−1; teste g=2,3,5,... com powmod(g,(p−1)/q,p)≠1 para cada q. Achado g, as raízes são g^k com gcd(k,p−1)=1 (φ(p−1) delas). Para p^k e 2p^k, aplique o mesmo critério com φ(p^k).
  - *Exemplo:* p=7: g=2 falha ($2^3\equiv1$); g=3: $3^2\neq1$ e $3^3\neq1 \to$ raiz; raízes 3 e $3^5=5$.
  - *Armadilha:* o expoente é (p−1)/q, não p−1; testar g^{p−1}=1 é necessário mas não suficiente. Para n=2p^k, ajuste a paridade.

- **Logaritmo/índice via raiz primitiva.** Mult. mod p vira soma mod p−1: x^k ≡ a ⇔ k·ind(x) ≡ ind(a); para a ≢ 0, a é potência k-ésima sse a^{(p−1)/gcd(k, p−1)} ≡ 1, e então há gcd(k, p−1) raízes (a ≡ 0 tem só x ≡ 0).
  - *Gatilho:* resolver x^k≡a mod p, comparar potências, ou transformar produto em soma.
  - *Por que funciona:* com raiz primitiva g, todo a≠0 é g^e; a multiplicação vira soma de expoentes mod p−1, e x^k=g^{k·ind(x)}, então a congruência é linear em ind(x).
  - *Como fazer:* ache g; calcule ind(a) por BSGS ou tabela; resolva k·y≡ind(a) (mod p−1) por Euclides; as soluções y dão x=g^y.
  - *Exemplo:* p=7, g=3, a=6=3^3 → ind=3; x^2≡6 pede 2y≡3 (mod 6), sem solução; de fato 6 não é QR mod 7.
  - *Armadilha:* ind(a) é módulo p−1, não p; a=0 é caso separado. O número de raízes é gcd(k,p−1), não k.
- **BSGS.** a^x ≡ b (mod m) em O(√m) com hash: x = i·s − j, s ≈ √m. Para gcd(a, m) ≠ 1 (exBSGS): enquanto g = gcd(a, m) > 1, se g ł b não há solução; faça b ← (b/g)·inv(a/g mod m/g) (mod m/g), m ← m/g e conte o expoente deslocado, até ficar coprimo.
  - *Gatilho:* resolver exponencial modular com m até 10^9–10^{12} e base arbitrária.
  - *Por que funciona:* escrevendo x=i·s−j com 0≤j<s, a^{is}≡b·a^j; guardar b·a^j em hash e testar a^{is} cobre x em O(√m) posições.
  - *Como fazer:* s=⌈√m⌉; tabela de b·a^j (j=0..s−1); para i=0..s, calcule a^{is} e procure na tabela; resposta x=is−j. Para o menor x, guarde o maior j por valor. exBSGS: remova fatores comuns contando cnt, resolva o caso coprimo e some cnt.
  - *Exemplo:* 2^x≡5 (mod 13): s=4; tabela b·2^j = {5,10,7,1}; i=3 dá 2^{12}=1, casando j=3 → x=9.
  - *Armadilha:* use o maior j para o menor x; no exBSGS teste também x<cnt. m=1 → x=0; a e b devem ser reduzidos mod m.
- **Raiz quadrada mod p.** Critério de Euler a^{(p−1)/2}; p ≡ 3 (mod 4) → a^{(p+1)/4}; geral → Tonelli–Shanks ou Cipolla. Mod p^k: Hensel; mod composto: CRT; com n ímpar, gcd(a, n) = 1 e a QR em cada p | n, 2^ω soluções (casos 2^k e p | a têm contagem especial).
  - *Gatilho:* resolver x^2≡a mod m, ou precisa de uma raiz para fatorar/contar.
  - *Por que funciona:* o critério de Euler decide se a é resíduo quadrático; para p≡3 mod4 a fórmula fecha; Tonelli–Shanks acha uma raiz no caso geral. Hensel levanta a solução para p^k.
  - *Como fazer:* calcule a^{(p−1)/2}; se ≠1, não há raiz (a≠0). p≡3 mod4: x=a^{(p+1)/4}; senão Tonelli–Shanks. Para m composto, resolva em cada potência de primo e combine por CRT.
  - *Exemplo:* p=11, a=5: 5^5≡1 → QR; 11≡3 mod4, x=5^3=125≡4 e 4^2≡5.
  - *Armadilha:* a=0 tem só x=0 (o critério de Euler dá 0, não 1); p=2 é caso à parte. Hensel falha quando a≡0 mod p — trate separadamente.
- **Legendre/Jacobi.** Para a ímpar, (a/p)(p/a) = (−1)^{(p−1)(a−1)/4}; (2/p) = (−1)^{(p^2−1)/8}; Jacobi em O(log) estilo Euclides, sem fatorar. x^2 ≡ −1 (mod p) tem solução sse p = 2 ou p ≡ 1 (mod 4).
  - *Gatilho:* decidir se a é resíduo quadrático sem fatorar, ou computar símbolos de Legendre/Jacobi.
  - *Por que funciona:* a reciprocidade quadrática relaciona (a/p) e (p/a), reduzindo o numerador como no Euclides; as fórmulas para 2 e −1 completam a recursão.
  - *Como fazer:* para (a/n): reduza a mod n; pare em 0 ou 1; fatore potências de 2 com a fórmula (2/n); use reciprocidade e troque o sinal; continue com (n mod a)/a.
  - *Exemplo:* (5/11): sinal +; (11/5)=(1/5)=1 → QR. (3/7): sinal −; (7/3)=(1/3)=1 → −1.
  - *Armadilha:* Jacobi=1 não garante QR mod n composto; só Legendre com p primo decide. A fórmula de (2/p) usa (p^2−1)/8.
- **Soma de dois/três/quatro quadrados.** n é soma de 2 quadrados sse todo primo ≡ 3 (mod 4) tem expoente par ($r_2(n) = 4(d_1 - d_3)$); soma de 3 quadrados sse n ≠ 4^a(8b+7); 4 quadrados sempre (Lagrange).
  - *Gatilho:* representar n como soma de quadrados, contar representações, ou provar impossibilidade.
  - *Por que funciona:* nos inteiros de Gauss, primos ≡3 mod4 são inertes e exigem expoente par; o teorema dos três quadrados é a caracterização clássica; r_2 conta divisores ≡1 e ≡3 mod4.
  - *Como fazer:* para 2 quadrados, fatore n e verifique os expoentes dos primos 3 mod4; para contar, use $r_2(n)=4(d_1-d_3)$. Para 3, teste n=4^a(8b+7). Para 4, use a existência de Lagrange.
  - *Exemplo:* 9=3^2 → 9=3^2+0^2; 21=3·7 (expoentes ímpares) → não é soma de 2 quadrados.
  - *Armadilha:* 0^2 é permitido; r_2 conta representações com sinal e ordem. A condição de 3 quadrados não constrói a representação.

- **Resíduos que provam impossibilidade.** Quadrados mod 8 ∈ {0, 1, 4}; mod 3 ∈ {0, 1}; cubos mod 9 ∈ {0, 1, 8}; quartas potências mod 16 ∈ {0, 1}. Também: $x^2 \equiv 1 \pmod n$ tem 2^ω soluções para n ímpar e 4 para 2^k, k ≥ 3.
  - *Gatilho:* provar que uma equação diofantina não tem solução; construção modular.
  - *Por que funciona:* uma igualdade nos inteiros implica a mesma congruência em qualquer módulo; se os conjuntos de resíduos dos dois lados não se intersectam, não há solução.
  - *Como fazer:* escolha um módulo pequeno (8, 9, 16, 3, 4) que separe os resíduos; calcule os conjuntos dos dois lados. Para $x^2 \equiv 1$, conte via CRT: 2 soluções por primo ímpar e 4 para 2^k (k≥3).
  - *Exemplo:* $x^2+y^2=4z+3$ é impossível mod 4 (quadrados 0,1; soma 0,1,2); $x^2 \equiv -1 \pmod p$ exige p=2 ou p≡1 mod4.
  - *Armadilha:* ausência de obstáculo modular não prova existência; use módulos compostos via CRT para refinar.
- **LTE (Lifting the Exponent).** p ímpar, p | x−y, p ł xy: $v_p(x^n - y^n) = v_p(x-y) + v_p(n)$; n ímpar, p | x+y, p ł xy: $v_p(x^n + y^n) = v_p(x+y) + v_p(n)$; p = 2, n par, x, y ímpares: $v_2(x^n - y^n) = v_2(x-y) + v_2(x+y) + v_2(n) - 1$.
  - *Gatilho:* valoração p-ádica de diferenças/somas de potências, divisibilidade por potências de primos.
  - *Por que funciona:* fatorando x^n-y^n e controlando a valoração de (x−y) e da soma geométrica; o termo variável vem de n.
  - *Como fazer:* verifique as hipóteses (p | x±y, p ł xy, paridade de n); aplique a fórmula. Para p=2 e n ímpar, $v_2(x^n-y^n)=v_2(x-y)$.
  - *Exemplo:* $v_3(4^6-1)$: 4−1=3 e 3ł4; fórmula $v_3(3)+v_3(6)=1+1=2$ (4095=$3^2 \cdot 5 \cdot 7 \cdot 13$).
  - *Armadilha:* as hipóteses fazem parte do teorema; sem p | x−y ou p | x+y a fórmula não vale. p=2 tem fórmula própria e exige n par.
- **Legendre/Kummer em p-ádico.** $v_p(n!) = \Sigma \lfloor n/p^i \rfloor = (n - s_p(n))/(p-1)$; $v_p(C(n, k)) = (s_p(k) + s_p(n-k) - s_p(n))/(p-1)$ = número de vai-uns de k + (n−k) na base p. Zeros finais de n! na base b = $\min_p \lfloor v_p(n!)/e_p(b) \rfloor$.
  - *Gatilho:* divisibilidade de fatorial/binomial por primos, zeros à direita em base b, ou n! mod p^k.
  - *Por que funciona:* Legendre conta múltiplos de p, p^2, ...; a forma com soma de dígitos vem de n=Σd_i p^i. Kummer interpreta os vai-uns da soma k+(n−k) na base p.
  - *Como fazer:* para $v_p(n!)$, itere n/=p somando; ou use (n−s_p(n))/(p−1). Para binomial, use a diferença de somas de dígitos. Para base b, fatore b=∏p^{e_p} e tome o mínimo.
  - *Exemplo:* $v_2(10!)=5+2+1=8$ e $v_5(10!)=2$; 10!=3628800 tem 2 zeros na base 10.
  - *Armadilha:* Kummer usa s_p(k) e s_p(n−k), não s_p(n)+...; a divisão por p−1 é inteira. Zeros na base b exigem todos os primos de b, não só o maior.
- **Wilson e consequências.** $(p-1)! \equiv -1$; $(n-1)! \equiv 0 \pmod n$ para composto n > 4; (p−1−k)!·k! ≡ (−1)^{k+1} (mod p), útil para n! mod p com n perto de p.
  - *Gatilho:* calcular fatorial mod p quando n está perto de p, ou usar a caracterização de primalidade.
  - *Por que funciona:* no grupo multiplicativo mod p, o produto de todos os elementos emparelha inversos e sobra só −1; para composto, o fatorial contém um fator próprio de n.
  - *Como fazer:* se n≥p, n!≡0; se p−n é pequeno, use (p−1)! = n!·∏_{i=n+1}^{p−1}i ≡ −1, logo n! ≡ −1/∏... . A identidade com k permite trocar k! por inverso.
  - *Exemplo:* p=7, k=2: 4!·2!=24·2=48≡6≡$(-1)^3$.
  - *Armadilha:* Wilson é O(p), inútil para primalidade em p grande; n=4 é a exceção composta (3!≡2≠0).
- **n! mod p com parte livre de p.** n! = p^{⌊n/p⌋}·⌊n/p⌋!·∏_{płi≤n} i, e o último produto ≡ (−1)^{⌊n/p⌋}(n mod p)! (mod p): recursão em $\log_p n$ com tabela de fatoriais até p. Base de Lucas generalizado e de C(n, k) mod p^e.
  - *Gatilho:* n! mod p (ou p^e) com n grande e p pequeno; coeficientes binomiais módulo potência de primo.
  - *Por que funciona:* separa os múltiplos de p (que viram p^{⌊n/p⌋}·⌊n/p⌋!) do produto dos não múltiplos; este último é periódico mod p com sinal (−1) por bloco.
  - *Como fazer:* pré-calcule fact[i] para i<p; f(n): se n==0 retorne 1; retorne f(n/p)·(−1)^{n/p}·fact[n%p] mod p. O expoente de p é $v_p(n!)$. Para p^e, use o produto dos não múltiplos módulo p^e.
  - *Exemplo:* 12!: $v_5=2$ e a parte livre de 5 é f(12)=f(2)·$(-1)^2$·2! = 2·2 = 4 (mod 5).
  - *Armadilha:* n! mod p é 0 para n≥p — o que se calcula é a parte livre de p. Trate p^{v} separadamente mod p^e; o sinal depende de ⌊n/p⌋.

- **Torre a^b^c... mod m.** a^e ≡ a^{(e mod φ(m)) + φ(m)} (mod m) quando e ≥ φ(m) (vale para qualquer gcd); recurse com φ(m), φ(φ(m))... (cadeia chega a 1 em ≤ 2·log₂ m passos); compare expoentes truncados (cap em 2^{60}) para saber se e ≥ φ.
  - *Gatilho:* torre de potências com expoentes enormes, "a^b^c mod m".
  - *Por que funciona:* a versão estendida de Euler vale para qualquer base quando o expoente é grande; a cadeia de φ decresce e o caso base é φ(1)=1. Truncar o expoente permite comparar com φ(m) sem bigint.
  - *Como fazer:* avalie a torre de cima para baixo (associativa à direita). Para a^E mod m: se m==1, retorne 0; calcule e = valor da torre abaixo truncado em CAP; se e < φ(m), use pow(a,e,m); senão use pow(a, e%φ(m)+φ(m), m), recursando para e com módulo φ(m).
  - *Exemplo:* 2^{2^{100}} mod 7: φ(7)=6; o expoente real é ≥6 e 2^100≡4 (mod 6), então 2^{4+6}=2^{10}≡2.
  - *Armadilha:* não some φ(m) quando o expoente é pequeno; a torre associa à direita. m=1 → 0.
- **λ de Carmichael.** Para gcd(a, n) = 1, a^k mod n só depende de k mod λ(n), e λ(n) | φ(n) pode ser bem menor (λ(2^k) = 2^{k−2} para k ≥ 3).
  - *Gatilho:* período de potências mod n, reduzir expoentes, ou quando φ(n) não é o menor expoente universal.
  - *Por que funciona:* λ(n) é o expoente do grupo (Z/nZ)^×, o menor m com a^m≡1 para todo a coprimo; é o lcm dos λ das potências de primos.
  - *Como fazer:* fatore n; λ(p^k)=φ(p^k) para p ímpar; λ(2)=1, λ(4)=2, λ(2^k)=2^{k−2} (k≥3); λ(n)=lcm dos λ(p_i^{e_i}). Para reduzir a^e, use e mod λ(n).
  - *Exemplo:* n=8: λ=2; 3^2=9≡1, embora φ(8)=4.
  - *Armadilha:* λ vale para todo a coprimo, mas a ordem de um a específico pode ser menor (divide λ). Com gcd(a,n)≠1, use a versão estendida com φ.
- **Zeckendorf.** Todo inteiro positivo é soma única de Fibonacci não consecutivos (guloso do maior); há ~86 Fibonacci ≤ 10^{18}. Strings binárias de comprimento n sem dois 1 adjacentes = F_{n+2}.
  - *Gatilho:* representar inteiros em base Fibonacci, contar strings sem 1s adjacentes, ou jogos/guloso com Fibonacci.
  - *Por que funciona:* a sequência de Fibonacci é completa e a condição de não consecutivos elimina ambiguidades (F_{k+1}=F_k+F_{k−1}); o guloso do maior sempre funciona.
  - *Como fazer:* pré-calcule F até o limite; pegue o maior F_k≤n, subtraia e continue com o maior F_j≤resto e j≤k−2. Para contar strings, DP com estados terminando em 0/1, resultando em F_{n+2}.
  - *Exemplo:* 100 = 89+8+3 = F_{11}+F_6+F_4 (índices não consecutivos).
  - *Armadilha:* use F_2=1, F_3=2, ... sem duplicar o 1; o guloso precisa pular o Fibonacci consecutivo.
- **Fibonacci: identidades.** gcd(F_a, F_b) = F_{gcd(a, b)}; doubling F_{2k} = F_k(2F_{k+1} − F_k), F_{2k+1} = F_k^2 + F_{k+1}^2; período de Pisano π(m) ≤ 6m; Cassini F_{n−1}F_{n+1} − F_n^2 = (−1)^n.
  - *Gatilho:* Fibonacci com índice gigante, gcd de Fibonacci, período mod m.
  - *Por que funciona:* as identidades vêm da forma matricial [[1,1],[1,0]]^n e da adição de ângulos; o doubling calcula F_n em O(log n). O período mod m é finito porque o par (F_n,F_{n+1}) tem m^2 estados, e 6m é o bound de Pisano.
  - *Como fazer:* para F_n mod m, use fast doubling com as fórmulas, reduzindo mod m; para gcd, calcule F_gcd. Para o período, itere até (0,1) reaparecer.
  - *Exemplo:* F_{10}=55, F_{15}=610, gcd=5=F_5; o período de Pisano mod 10 é 60.
  - *Armadilha:* o fast doubling precisa de F_k e F_{k+1} juntos; 6m é cota superior, não o período exato.
- **gcd(x^a − 1, x^b − 1) = x^{gcd(a, b)} − 1** (vale para 2^a−1). Análogo: gcd(a^n − b^n, a^m − b^m) = a^{gcd}− b^{gcd} se gcd(a, b) = 1.
  - *Gatilho:* gcd de números da forma x^k−1, ou de repunits em bases diferentes.
  - *Por que funciona:* o Euclides nos expoentes espelha o dos números: x^a−1 mod (x^b−1) = x^{a mod b}−1, então o gcd é x^{gcd(a,b)}−1. Para bases a,b coprimas, vale o mesmo com a^n−b^n.
  - *Como fazer:* reduza ao gcd dos expoentes; para a^n−b^n, verifique gcd(a,b)=1, calcule g=gcd(n,m) e a^g−b^g (bigint ou módulo).
  - *Exemplo:* gcd(2^{10}−1,2^{15}−1)=2^5−1=31; gcd(3^6−2^6,3^9−2^9)=3^3−2^3=19.
  - *Armadilha:* gcd(a,b)=1 é necessário no caso de bases diferentes; sem ela o gcd pode ter fatores extras. Reduza os expoentes antes de exponenciar.

- **Frações contínuas.** [a_0; a_1, ...] sai do algoritmo de Euclides em O(log); convergentes p_k = a_k p_{k−1} + p_{k−2} (idem q_k) são as melhores aproximações; com semi-convergentes acha-se a melhor fração com denominador ≤ N.
  - *Gatilho:* aproximar racionais, achar denominador pequeno, resolver Pell, ou reconstruir uma fração a partir de resíduos.
  - *Por que funciona:* cada convergente é a melhor aproximação com denominador do seu tamanho; os coeficientes vêm da divisão euclidiana.
  - *Como fazer:* itere a divisão euclidiana de numerador/denominador guardando a_i; mantenha p_{−1}=1, $p_0=a_0$, q_{−1}=0, q_0=1; atualize p_k=a_k p_{k−1}+p_{k−2}, q_k=a_k q_{k−1}+q_{k−2}. Para denominador ≤N, use semi-convergentes.
  - *Exemplo:* 355/113 = [3;7,16], uma excelente aproximação de π.
  - *Armadilha:* a_0 pode ser negativo; a convenção de piso muda a expansão. Os convergentes estouram — use __int128 ou pare.
- **Fração mais simples num intervalo.** A menor fração (menor denominador) em [l, r] (ou em (l, r)) é achada descendo a árvore de Stern–Brocot em paralelo para l e r até os caminhos divergirem: O(log) com saltos em blocos.
  - *Gatilho:* "ache a fração de menor denominador entre l e r", com l,r racionais.
  - *Por que funciona:* a árvore de Stern–Brocot contém todos os racionais positivos em ordem; descer em paralelo mantém o intervalo que contém a resposta; ao divergir, o primeiro nó entre os dois é a fração mais simples.
  - *Como fazer:* represente l=a/b, r=c/d; no nó p/q, compare com l e r usando produto cruzado; se ambos estão à direita, vá à direita; se ambos à esquerda, à esquerda; ao divergir, retorne a mediana. Pule blocos repetidos calculando o número de passos por divisão.
  - *Exemplo:* a menor fração em (1/3, 2/3) é 1/2; em (2/5, 3/5) também 1/2.
  - *Armadilha:* intervalos abertos vs fechados mudam a resposta nos extremos; normalize para [0,1) ou use a árvore com 1/1. Inteiros grandes exigem ceil/floor corretos.
- **Stern–Brocot.** Cada racional positivo aparece uma vez; o caminho L/R em run-length é a fração contínua. Busca binária sobre racionais (achar a/b com predicado monótono) = descer com saltos exponenciais: O($\log^2$) ou O(log).
  - *Gatilho:* busca sobre racionais, aproximação ótima, ou representar racionais por caminhos.
  - *Por que funciona:* a árvore é uma árvore binária de busca ordenada; o caminho até p/q corresponde à expansão em fração contínua, e o run-length dá os coeficientes.
  - *Como fazer:* desça comparando p/q com o alvo por produto cruzado; para um predicado monótono em a/b, use o caminho e pule blocos com saltos exponenciais (ou divisão) para O(log). Mantenha os limites e a mediana.
  - *Exemplo:* 3/5 = [0;1,1,2], com caminho LRL na árvore.
  - *Armadilha:* comparar frações com produto cruzado em 64 bits pode estourar; use __int128. A árvore cobre positivos; separe o sinal.
- **Farey.** Consecutivos a/b < c/d em F_n satisfazem bc − ad = 1; próximo termo: (kc − a)/(kd − b) com k = ⌊(n + b)/d⌋; |F_n| = 1 + Σ_{i≤n} φ(i). Contar frações irredutíveis de denominador ≤ n = Σφ.
  - *Gatilho:* gerar frações com denominador ≤n em ordem, contar frações irredutíveis, ou achar vizinhos.
  - *Por que funciona:* bc−ad=1 garante que não há outra fração com denominador ≤n entre elas; a fórmula do próximo termo mantém o determinante 1.
  - *Como fazer:* comece 0/1, 1/n; para avançar de a/b, c/d, k=⌊(n+b)/d⌋, próximo=(kc−a)/(kd−b); pare em 1/1. Para contar irredutíveis, Σ_{i=1}^n φ(i).
  - *Exemplo:* F_5 tem 0/1,1/5,1/4,1/3,2/5,1/2,3/5,2/3,3/4,4/5,1/1; 2/5 e 1/2: 5·1−2·2=1.
  - *Armadilha:* |F_n| inclui 0/1 e 1/1; a soma Σφ conta as frações de (0,1), então |F_n|=1+Σφ. O próximo termo exige a/b e c/d consecutivos.
- **Moeda / Frobenius.** Para a, b coprimos o maior não representável é ab − a − b e há (a−1)(b−1)/2 não representáveis. Com muitas moedas e a menor pequena: Dijkstra nos resíduos mod a_1 (dist[r] = menor soma ≡ r).
  - *Gatilho:* "maior valor que não pode ser formado", "quantos valores não representáveis", moedas com limites.
  - *Por que funciona:* no caso de duas coprimas, os não representáveis se emparelham x ↔ ab−a−b−x, dando a contagem; com várias moedas, o menor representável em cada resíduo mod a_1 determina todos os valores grandes.
  - *Como fazer:* para 2 moedas, use as fórmulas. Para muitas, rode Dijkstra nos resíduos 0..a_1−1 com arestas r→(r+a_i) mod a_1 de peso a_i; dist[r] é o menor valor ≡r; depois conte os não representáveis por resíduo.
  - *Exemplo:* moedas 3,5: maior não representável 15−3−5=7; não representáveis 1,2,4,7 (4 = (2·4)/2).
  - *Armadilha:* a fórmula de duas moedas exige coprimalidade; com gcd>1, só múltiplos do gcd são representáveis. Use long long nos pesos.

- **Pell $x^2 - Dy^2 = 1$.** Solução fundamental vem do período da fração contínua de √D; as demais são potências de $(x_1 + y_1\sqrt{D})$, logo matriz 2×2 elevada ao k. Soluções fundamentais podem ter ~√D dígitos: use bigint ou só mod.
  - *Gatilho:* resolver $x^2-Dy^2=1$, achar a menor solução, ou gerar todas.
  - *Por que funciona:* a unidade fundamental de Z[√D] tem norma 1 e é dada pelo período da fração contínua de √D; multiplicar unidades soma expoentes, daí a matriz.
  - *Como fazer:* expanda √D em fração contínua até o período; a convergente no fim do período dá (x_1,y_1) (se o período for ímpar, use o dobro). Gere (x_k,y_k) por recorrência ou eleve $[[x_1,Dy_1],[y_1,x_1]]$^k.
  - *Exemplo:* D=2: √2=[1;2,2,...], fundamental (3,2), pois 9−2·4=1.
  - *Armadilha:* D quadrado perfeito não tem solução não trivial; período ímpar exige ajuste. x,y crescem exponencialmente — bigint ou módulo.
- **Triplas pitagóricas.** Primitivas: $(m^2-n^2, 2mn, m^2+n^2)$ com m > n, gcd = 1, paridades opostas; ao menos um cateto é múltiplo de 3, de 4, e um dos lados de 5.
  - *Gatilho:* gerar triplas, contar triplas com limite, ou usar as propriedades de divisibilidade.
  - *Por que funciona:* a parametrização vem da fatoração nos inteiros de Gauss; coprimalidade e paridades opostas garantem que a tripla é primitiva e única.
  - *Como fazer:* para limite L, itere m de 2 até √L, n<m com gcd=1 e paridades opostas; calcule a,b,c e os múltiplos k. Valide com as propriedades mod 3,4,5.
  - *Exemplo:* m=2,n=1 → (3,4,5); m=3,n=2 → (5,12,13).
  - *Armadilha:* sem gcd=1 e paridades opostas a tripla não é primitiva; a ordem dos catetos pode variar. m^2+n^2 pode estourar.
- **Base fatorial (Lehmer).** Permutação ↔ número < n! com dígitos d_i ∈ [0, i−1] (número de menores à direita); k-ésima permutação e posto com BIT em O(n log n); n! passa de 2^{63} em n = 21.
  - *Gatilho:* enumerar/ordenar permutações, achar a k-ésima, ou o posto de uma permutação.
  - *Por que funciona:* o código de Lehmer conta, para cada posição, quantos elementos menores ainda não foram usados; esses dígitos são independentes e formam a representação fatorial do posto.
  - *Como fazer:* posto: percorra da esquerda para a direita, conte menores à direita não usados com BIT e some d_i·(n−i)!. k-ésima: para cada i, d=k/(n−i)!, escolha o (d+1)-ésimo menor disponível e k%=. O(n log n).
  - *Exemplo:* [2,3,1]: d=[1,1,0] → posto=1·2!+1·1!+0=3 (0-indexado).
  - *Armadilha:* indexação e ordem dos pesos mudam o resultado; o dígito conta menores à direita na ordem atual, não no array original. n=21 estoura 64 bits.
- **σ(n) mod p e séries geométricas.** σ(n) = ∏ (p^{e+1} − 1)/(p − 1); se p−1 ≡ 0 mod o módulo, use Σ_{i≤e} p^i por divisão e conquista (dobrar) sem inverso.
  - *Gatilho:* calcular σ(n) ou produtos de séries geométricas módulo M onde p−1 pode não ser invertível.
  - *Por que funciona:* σ é multiplicativa e cada fator é uma série geométrica; a fórmula fechada divide por p−1, válida só se p−1 for invertível mod M.
  - *Como fazer:* fatore n; para cada p^e, calcule Σ_{i=0}^e p^i mod M por exponenciação binária/dobrar (ou use inverso quando possível); multiplique. Se M é primo e p≠1 mod M, use (p^{e+1}−1)·inv(p−1).
  - *Exemplo:* $\sigma(12)=\sigma(2^2)\sigma(3)=7\cdot4=28\equiv3 \pmod 5$.
  - *Armadilha:* a série dobrada precisa de pares (soma, potência); não use a fórmula com inverso quando p≡1 mod M.
- **Funções multiplicativas.** f fica determinada pelos valores em potências de primos; a soma Σ_{d|n} f(d) de multiplicativa é multiplicativa; para provar multiplicatividade verifique f(mn) = f(m)f(n) com gcd = 1.
  - *Gatilho:* definir/calcular funções aritméticas, ou provar que uma fórmula é multiplicativa.
  - *Por que funciona:* pela fatoração única, n=∏p_i^{e_i}; se f é multiplicativa, f(n)=∏f(p_i^{e_i}). A convolução com 1 preserva a multiplicatividade.
  - *Como fazer:* fatore n e multiplique os valores nas potências de primos; para a soma sobre divisores, use a convolução. Para provar, use a bijeção entre divisores de mn e pares (d_1,d_2) quando gcd(m,n)=1.
  - *Exemplo:* τ(n)=∏(e_i+1), σ(n)=∏(p^{e+1}−1)/(p−1); ambas são multiplicativas.
  - *Armadilha:* multiplicativa exige gcd(m,n)=1; completamente multiplicativa é mais forte (f(p^e)=f(p)^e). Soma sobre divisores de uma função arbitrária não é multiplicativa.

- **Divisores: fórmulas prontas.** $\Sigma_{d|n} d = \prod(p^{e+1} - 1)/(p - 1)$; produto dos divisores = $n^{\tau(n)/2}$; $\tau(n)$ é ímpar sse n é quadrado (luzes/portas alternadas); #pares ordenados com lcm(a, b) = n é $\tau(n^2)$; com gcd = 1 e ab = n é 2^{ω}.
  - *Gatilho:* problemas de divisores, soma/produto deles, contagem de pares com lcm/gcd.
  - *Por que funciona:* os divisores são combinações independentes dos expoentes 0..e; a soma fatora, o produto emparelha d com n/d, e τ ímpar só ocorre quando há ponto fixo (n quadrado). Pares com lcm n têm expoentes $\max(a_i,b_i)=e_i$, dando $2e_i+1$ escolhas cada $\rightarrow \tau(n^2)$.
  - *Como fazer:* fatore n; use as fórmulas. Para pares com lcm n, calcule $\tau(n^2)$; para coprimos com ab=n, distribua cada primo entre a e b (2^{ω}).
  - *Exemplo:* $n=12=2^2\cdot3$: $\sigma=7\cdot4=28$; $\tau=6$; produto$=12^3=1728$; pares ordenados com lcm 12 = $\tau(144)=15$.
  - *Armadilha:* 2^{ω} conta pares ordenados; para não ordenados divida por 2 (cuidado com n=1). O produto é $n^{\tau/2}$; use a potência, não a raiz.
- **Tamanho de divisores (máximos).** $n \le 10^3$: 32; 10^4: 64; 10^5: 128; 10^6: 240; 10^9: 1344; 10^{12}: 6720; 10^{15}: 26880; 10^{18}: 103680. $\Sigma_{i\le N} \tau(i) \approx N \ln N$.
  - *Gatilho:* dimensionar loops sobre divisores, estimar complexidade, ou buscar números com muitos divisores.
  - *Por que funciona:* $\tau(n)$ é maximizado por números com primos pequenos e expoentes não crescentes; os valores tabelados são os recordes por faixa.
  - *Como fazer:* use a tabela para justificar $O(\tau(n))$ por número; para gerar todos os divisores, $O(\tau(n))$ após fatorar; para somas, use $N \ln N$.
  - *Exemplo:* um loop sobre os divisores de $n\le10^9$ roda no máximo 1344 vezes.
  - *Armadilha:* a tabela é o máximo, não a média (~ln n). Para 10^{18}, 103680 é raro — não use como pior caso por elemento em n grande.
- **Números altamente compostos.** Têm expoentes não crescentes nos primos 2, 3, 5, ...; gere por DFS (primo k ≤ 15 primos para 10^{18}) para achar o maior $\tau \le N$ ou o menor n com $\tau(n) \ge k$.
  - *Gatilho:* "menor número com pelo menos k divisores", "máximo $\tau(n)$ para $n\le N$".
  - *Por que funciona:* se um expoente é maior que o anterior, trocar os primos diminui n sem mudar τ; logo a forma ótima é não crescente. A DFS enumera essas formas.
  - *Como fazer:* DFS(pos, valor, τ, último expoente): para e=1..último, se $valor\cdot p^e\le N$, recursa; atualize a melhor resposta quando τ bater o recorde (maior τ) ou quando valor for menor com $\tau\ge k$.
  - *Exemplo:* $45360=2^4\cdot3^4\cdot5\cdot7$ tem $\tau=5\cdot5\cdot2\cdot2=100$ divisores.
  - *Armadilha:* para "menor n com $\tau\ge k$", compare valores, não só τ; um estado com mesmo τ e valor maior é dominado. Cuidado com overflow ao multiplicar p^e.
- **Divisibilidade encadeada é curta.** Cadeia $a_1 \mid a_2 \mid \ldots$ com $a_i \ne a_{i+1}$ em [1, N] tem $\le \log_2 N + 1$ elementos; DP sobre cadeias de divisores é $O(N \log N \log N)$.
  - *Gatilho:* "maior cadeia de divisores", "sequência onde cada termo divide o próximo".
  - *Por que funciona:* cada passo multiplica por um inteiro ≥2, então o valor pelo menos dobra; o comprimento é $\le\log_2N+1$.
  - *Como fazer:* $dp[m]=1+\max(dp[d])$ para d|m, d<m; processe m crescente e atualize os múltiplos, ou calcule por divisores. O número de pares (d,m) com d|m é $\Sigma\tau \approx N \log N$.
  - *Exemplo:* N=8: 1|2|4|8 tem 4 elementos; $\log_2 8+1=4$.
  - *Armadilha:* a definição de comprimento (nós vs arestas) muda o +1; se a cadeia puder repetir, o bound não vale.
- **Antichain de divisibilidade.** Entre 1..2n, qualquer n+1 números contêm um par a | b (agrupe pela parte ímpar); então a maior antichain é n.
  - *Gatilho:* "conjunto sem divisibilidade", "maior subconjunto onde nenhum divide outro".
  - *Por que funciona:* cada número se escreve $2^k\cdot m$ com m ímpar; há n ímpares em [1,2n], então n+1 números forçam dois com o mesmo m ímpar, e o de menor expoente de 2 divide o outro.
  - *Como fazer:* para provar o limite, agrupe pela parte ímpar; para exibir a antichain, use {n+1,...,2n} (o dobro do menor já passa de 2n).
  - *Exemplo:* [1,4]: {3,4} é antichain de tamanho 2=n.
  - *Armadilha:* o argumento é para [1,2n]; para [1,N] geral, a maior antichain é $\{\lfloor N/2\rfloor+1,\ldots,N\}$, tamanho $\lceil N/2\rceil$. O invariante é a parte ímpar.

- **lcm de 1..n.** = $\prod p^{\lfloor \log_p n \rfloor}$, $\ln \operatorname{lcm}(1..n) \approx n$; $\operatorname{lcm}(1..41) \approx 2{,}2 \cdot 10^{17}$ e $\operatorname{lcm}(1..43) \approx 9{,}4 \cdot 10^{18}$ já estoura 2^{63}.
  - *Gatilho:* período comum, múltiplo comum mínimo de 1..n, ou estimar crescimento.
  - *Por que funciona:* a maior potência de cada primo ≤n é o que entra no lcm; o log do produto é a função de Chebyshev ψ(n), que é ~n.
  - *Como fazer:* para n pequeno, multiplique $p^{\lfloor \log_p n \rfloor}$ para todos os primos ≤n; para n grande, use ln lcm≈n. Para comparar, use logaritmos ou bigint.
  - *Exemplo:* $\operatorname{lcm}(1..10) = 2520 = 2^3 \cdot 3^2 \cdot 5 \cdot 7$.
  - *Armadilha:* $\operatorname{lcm}(1..43) > 2^{63}$; use bigint/__int128 para o valor. ln lcm≈n é assintótico.
- **Primos: densidade e gaps.** $\pi(10^6) = 78498$, $\pi(10^7) = 664579$, $\pi(10^9) = 50847534$; maior gap $< 10^9$ é 282, $< 10^{18}$ é < 1500. $\Sigma_{p \le n} 1/p \approx \ln \ln n$, então iterar múltiplos só de primos custa n ln ln n.
  - *Gatilho:* dimensionar crivos, estimar número de primos, ou usar bounds de gaps.
  - *Por que funciona:* o teorema dos números primos dá π(x)~x/ln x; os gaps máximos tabelados são fatos computacionais; a soma dos recíprocos e o custo do crivo vêm da densidade.
  - *Como fazer:* use π(x)≈x/ln x para estimativas e os valores tabelados para limites; aloque ~n/ln n posições para primos.
  - *Exemplo:* até 10^7 há 664579 primos, então um array de primos precisa desse tamanho.
  - *Armadilha:* $\pi(10^9) = 50847534$ não é exatamente 50 milhões; use os valores para não subestimar arrays. O gap máximo cresce, mas devagar.
- **Contar coprimos com m.** $\#\{x \le n : \gcd(x, m) = 1\} = \Sigma_{d \mid \operatorname{rad}(m)} \mu(d) \lfloor n/d \rfloor$ (2^ω termos, ≤ 512 para $m \le 10^9$); o k-ésimo coprimo = busca binária + essa contagem.
  - *Gatilho:* contar/achar números coprimos com m em um intervalo, com m até 10^9.
  - *Por que funciona:* inclusão-exclusão sobre os primos distintos de m; só os divisores livres de quadrado importam, e há 2^ω deles. A contagem é monótona em n, permitindo busca binária.
  - *Como fazer:* fatore m e liste os primos distintos; gere todos os produtos d com sinal μ(d) (subconjunto); conte $\Sigma\, \mu(d) \lfloor n/d \rfloor$. Para o k-ésimo, busque o menor n com contagem ≥k.
  - *Exemplo:* m=12, primos {2,3}: #{x≤10: gcd(x,12)=1} = 10−5−3+1 = 3 (1,5,7).
  - *Armadilha:* use rad(m) (primos distintos), não todos os divisores; 2^ω pode ser 512. A busca binária precisa de limite superior (k·m serve).
- **Livres de quadrado e potências perfeitas.** #squarefree ≤ n = $\Sigma_{d \le \sqrt{n}} \mu(d) \lfloor n/d^2 \rfloor$; #potências perfeitas em [2, n] = $-\Sigma_{k \ge 2} \mu(k) (\lfloor n^{1/k} \rfloor - 1)$. Raiz inteira: use sqrtl e ajuste ±1.
  - *Gatilho:* contar números sem quadrado, potências perfeitas, ou usar μ para exclusão.
  - *Por que funciona:* um número é divisível por d^2 sse $d^2 \mid n$; a inclusão-exclusão com μ conta os livres de quadrado. Para potências perfeitas, μ evita contagens duplas.
  - *Como fazer:* pré-calcule μ até √n; some $\mu(d) \lfloor n/d^2 \rfloor$. Para potências, para k=2..$\log_2 n$, calcule a raiz k-ésima inteira e some $-\mu(k)(r-1)$. Ajuste a raiz com multiplicação.
  - *Exemplo:* n=10: squarefree = 10−2−1=7 (exclui 4,8,9); potências perfeitas = 3 (4,8,9).
  - *Armadilha:* a raiz k-ésima com double erra perto de potências exatas; ajuste com while. Para k grande, r=1 e o termo é 0; não conte 1.
- **Ponto de rede no círculo (Gauss).** $\#\{(x, y) : x^2 + y^2 \le r^2\} = \Sigma_x (2\lfloor \sqrt{r^2 - x^2} \rfloor + 1)$ em O(r); segmento de (0,0) a (a, b) contém $\gcd(|a|, |b|) + 1$ pontos de rede.
  - *Gatilho:* contar pontos inteiros em um círculo, ou pontos em um segmento de reta.
  - *Por que funciona:* para cada x, os y válidos vão de $-\sqrt{r^2-x^2}$ a $+\sqrt{(\ldots)}$, dando $2\lfloor \cdot \rfloor + 1$; no segmento, os pontos são múltiplos do vetor primitivo (a/g,b/g), e há g+1 deles.
  - *Como fazer:* itere x de −r a r, calcule y_max com sqrtl e ajuste; O(r). Para o segmento, g=gcd(|a|,|b|), conte k=0..g.
  - *Exemplo:* r=1: 5 pontos: (0,0),(±1,0),(0,±1).
  - *Armadilha:* o ajuste da raiz é essencial perto do círculo; r^2 pode estourar int. Para o segmento, gcd(0,0)=0 dá 1 ponto (a origem).

- **Inversa de φ.** φ(n) = k: enumere divisores d de k com d+1 primo e faça DFS multiplicando; φ(n) = k tem poucas soluções e n ≤ k·∏ p/(p−1).
  - *Gatilho:* achar n com φ(n)=k, ou contar tais n.
  - *Por que funciona:* se p|n, então p−1 | φ(n)=k; logo os primos possíveis são d+1 com d|k. A DFS escolhe expoentes e primos, dividindo k por p−1 e p^{e−1}.
  - *Como fazer:* para cada divisor d de k com d+1 primo, tente p=d+1; ao incluir p^e, o φ contribui (p−1)p^{e−1}; divida k e recursa; pare quando k=1 (o fator 2 pode ser incluído uma vez sem mudar k).
  - *Exemplo:* φ(n)=4: n=5,8,10,12.
  - *Armadilha:* o fator 2 é especial (p−1=1); trate a inclusão de 2 separadamente. Use set para evitar duplicatas.
- **Dízima e expansões.** Período de 1/n = ord_{n'}(10) com n' = n sem fatores 2 e 5; pré-período = max(v_2, v_5); primeiros dígitos de a^b: frac(b·$\log_{10}$ a); últimos k dígitos: mod 10^k.
  - *Gatilho:* dígitos de dízimas, primeiros/últimos dígitos de potências.
  - *Por que funciona:* multiplicar por 10^k desloca a vírgula; o período é a menor k com 10^k≡1 mod n' (ordem multiplicativa). Para a^b, $\log_{10}$ dá a parte inteira (número de dígitos) e a fracionária (mantissa).
  - *Como fazer:* fatore n para separar 2 e 5; calcule ord_{n'}(10) dividindo φ(n') pelos fatores primos. Para primeiros dígitos, calcule b·$\log_{10}$a, pegue frac e 10^frac; para últimos, powmod(a,b,10^k).
  - *Exemplo:* 1/6=0,1(6): n'=3, $\text{ord}_3(10)$=1; pré-período max(1,0)=1. 1/7 tem período 6.
  - *Armadilha:* n'=1 (decimal finito) não tem ordem definida; período 0. Para primeiros dígitos, double pode errar — use long double. Últimos k dígitos com zeros à esquerda exigem formatação.
- **Convolução de gcd/lcm.** h[g] = Σ_{gcd(i, j) = g} f[i]·f'[j]: zeta sobre múltiplos F[d] = Σ_{d|i} f[i], multiplique ponto a ponto, Möbius de volta, tudo em O(N log log N). lcm: zeta sobre divisores.
  - *Gatilho:* somas sobre pares com gcd/lcm exato, para valores até N (10^6–10^7).
  - *Por que funciona:* a zeta sobre múltiplos transforma gcd em produto ponto a ponto: F[d]F'[d] soma sobre pares ambos múltiplos de d, isto é, gcd múltiplo de d; a inversão de Möbius isola gcd=d.
  - *Como fazer:* calcule F[d]=Σ_{d|i}f[i] e F'[d] idem; H[d]=F[d]F'[d]; inverta por Möbius (subtraia múltiplos) para h[g]. O(N log log N). Para lcm, use zeta sobre divisores e o mesmo esquema.
  - *Exemplo:* f=f'=1 em [1,N]: h[g]=#pares com gcd=g; H[d]=$\lfloor N/d \rfloor^2$.
  - *Armadilha:* use long long nos produtos; a inversão desfaz cada primo na ordem contrária à da ida (para múltiplos e para divisores). No caso lcm, o zeta é sobre divisores.
- **Sequências que recorrem mod m.** x_{n+1} = F(x_n) mod m tem período ≤ m (Floyd/Brent acham ciclo); recorrências lineares de ordem k mod m têm período ≤ m^k (m^2 é o caso k = 2).
  - *Gatilho:* achar ciclo em recorrência modular, ou detectar repetição em simulação.
  - *Por que funciona:* o estado é finito (m valores, ou m^k para vetor de ordem k); a sequência repete um estado e daí é periódica. Floyd/Brent acham o ciclo em O(μ+λ) sem memória.
  - *Como fazer:* Floyd: tartaruga=f(1), lebre=f(f(1)); até igualar; depois ache o início e o comprimento. Brent usa potências de 2. Para recorrência linear, use o vetor de k estados.
  - *Exemplo:* x_{n+1}=x_n^2 mod 10: 2→4→6→6 (ciclo 6).
  - *Armadilha:* o período pode ser menor que m e pode haver pré-período. Para recorrências não invertíveis, o ciclo pode não incluir o início.
- **Reduza expoentes por p-ádico.** Para "n mod p^k" de produtos, separe a parte p^v e a unidade; para x^n com n gigante e gcd ≠ 1 trate v_p(x)·n separado do resto coprimo.
  - *Gatilho:* produtos/potências módulo p^k com fatores divisíveis por p, onde Euler simples não vale.
  - *Por que funciona:* todo inteiro x se escreve p^v·u com u coprimo de p; a valoração soma (v·n) e a unidade segue a exponenciação modular no grupo das unidades. Se v·n ≥ k, o resultado é 0 mod p^k.
  - *Como fazer:* fatore a base em p^v·u; calcule v_total=v·n (cap em k); se v_total≥k, resposta 0; senão calcule u^n mod p^{k−v_total} e multiplique por p^{v_total}. Para produtos, some valorações e multiplique unidades.
  - *Exemplo:* 12^{100} mod 2^{10}: 12=2^2·3, v_total=200≥10 → 0.
  - *Armadilha:* não use φ(p^k) na parte divisível por p; a valoração cresce linearmente com n. Para p=2, a ordem das unidades é λ(2^k), não φ.

- **mulmod 64 bits.** __int128, ou mod $2^{61}-1$ com shifts, ou Montgomery; long double só com cuidado. Para módulo fixo primo ~10^9 evite % em laço quente (Barrett).
  - *Gatilho:* multiplicar dois números $<2^{64}$ módulo m sem estourar, em código quente.
  - *Por que funciona:* __int128 tem 128 bits e cobre o produto; para $2^{61}-1$, a redução usa $2^{61}\equiv 1$; Montgomery/Barrett substituem a divisão por multiplicações e shifts.
  - *Como fazer:* use `(unsigned __int128)a*b%m` como referência; para $2^{61}-1$, dobre a parte alta e some; para módulo fixo, pré-calcule a constante de Barrett. Meça antes de otimizar.
  - *Exemplo:* $m=2^{61}-1$: $a\cdot b = hi\cdot 2^{61}+lo \equiv hi+lo$ (repita se necessário).
  - *Armadilha:* `long double` não é portátil e perde precisão; `__int128` não existe no MSVC. Montgomery exige m ímpar; Barrett exige o shift correto.

---

adicionado em 2026-11-11 a partir dos editoriais de CF Div. 2/3/4, A–D; triado contra o catálogo

- **Precisa de um primo novo → Bertrand.** Para n ≥ 1 existe primo em (n, 2n]: um primo maior que todos os valores é coprimo com todos e cabe em 2·máx. Ex.: CF 2284C1.
  - *Gatilho:* a construção precisa de um número/primo que não divida nenhum valor dado, ou de um módulo novo.
  - *Por que funciona:* Bertrand garante um primo entre n e 2n; escolhendo n = máximo dos valores, o primo é maior que todos e, portanto, coprimo com todos.
  - *Como fazer:* calcule M = máximo; procure o primeiro primo em (M, 2M] por teste de primalidade (ou pré-compute primos); use-o como módulo/base.
  - *Exemplo:* valores até 10: o primo 11 em (10,20] serve como módulo, pois nenhum valor é múltiplo de 11.
  - *Armadilha:* o primo pode passar do limite de saída — verifique se 2M cabe no tipo/limite. Para n=1, o primo é 2.
- **x·(10^d + 1) = x concatenado com x** (d = nº de dígitos de x; ex.: 123·1001 = 123123). Gatilho: construir múltiplo com poucos dígitos distintos ou com padrão repetido. Ex.: CF 2241B.
  - *Gatilho:* precisa de um múltiplo de x com representação repetida, ou de um número com poucos dígitos distintos.
  - *Por que funciona:* deslocar x por d casas e somar com x alinha os dois blocos, produzindo xx; o fator é 10^d+1.
  - *Como fazer:* calcule $d=\lfloor \log_{10} x \rfloor+1$; o número é x·(10^d+1). Para repetir k vezes, use x·(10^{dk}−1)/(10^d−1). Use bigint/string se necessário.
  - *Exemplo:* 123·1001=123123; 7·11=77 (d=1, 10+1=11).
  - *Armadilha:* zeros à esquerda quebram a concatenação; x não pode ter leading zeros. O produto cresce rápido.
- **Trocas a distância a ou b.** Se n ≥ a + b − gcd(a, b), as componentes das posições 0..n−1 são exatamente as classes mod gcd(a, b) (mesmo argumento de Fine–Wilf); para n menor, faça DSU. Ex.: CF 2244C.
  - *Gatilho:* trocar elementos a distâncias fixas a ou b, alcançabilidade de permutações, ou componentes de um grafo de passos.
  - *Por que funciona:* o grafo em Z com passos a e b tem componentes = classes mod g=gcd(a,b); em [0,n−1], se o intervalo é longo o suficiente, todas as arestas necessárias cabem e as componentes coincidem com as classes.
  - *Como fazer:* se n ≥ a+b−g, responda pelas classes mod g (conte elementos de cada); senão, construa DSU com n nós e arestas i↔i+a, i↔i+b quando dentro do intervalo.
  - *Exemplo:* a=2,b=3,g=1: n≥4 → tudo conectado; n=3 → {0,2} e {1}.
  - *Armadilha:* o limiar é a+b−g, não a+b; usar n menor sem DSU erra. Para g>1, conte cada classe mod g separadamente.
- **Inteiro mais próximo usando só dígitos permitidos.** Mantenha um prefixo comum com o alvo, desvie no primeiro dígito para o permitido logo acima (resto com o menor dígito permitido) ou logo abaixo (resto com o maior); teste também comprimentos ±1. São O(len) candidatos. Ex.: CF 2228C1.
  - *Gatilho:* achar o inteiro mais próximo de N usando apenas um conjunto de dígitos permitidos, com comprimento livre.
  - *Por que funciona:* qualquer número ótimo coincide com N até certo ponto e diverge em um dígito; para minimizar a distância, o primeiro dígito diferente deve ser o mais próximo possível (logo acima ou abaixo), e o resto extremo minimiza o valor naquela direção.
  - *Como fazer:* para cada posição e cada direção, se existe dígito permitido acima/abaixo do dígito de N, forme o candidato com prefixo igual, o dígito escolhido e o resto preenchido com min/max permitido; teste também comprimentos ±1. Tome o de menor |candidato−N|.
  - *Exemplo:* N=123, dígitos {1,4}: candidatos como 114 e 144; o mais próximo é 114 (distância 9).
  - *Armadilha:* zeros à esquerda não contam como dígitos, mas 0 pode ser permitido; lidere com o menor dígito não nulo. Comprimentos diferentes podem vencer.

Repetidas de outras seções:

- **"gcd de pelo menos metade"** → sorteie um elemento e teste seus divisores (aleatorização).
- **Jogo com números** → divisores, potências de 2, n mod (k+1); calcule pequenos e ache o padrão.
- **a_i ≤ 10^6** → itere valores/múltiplos em vez de índices (também em arrays).

- **Contar árvores com restrição de grau (Prüfer)** → multinomial; **Cayley** n^{n−2}; e Matrix-tree para contagem geral (determinante mod p).

Combinatória, contagem e probabilidade

Antes de qualquer fórmula: conte à mão n = 1..6 e jogue a sequência na OEIS.

- **Inclusão-exclusão:** escreva as duas linhas (quais são os defeitos; quanto vale N(S)). Se N(S) só depende de |S|, a soma vira Σ (−1)^k C(m, k) f(k).
  - *Gatilho:* "pelo menos", "nenhum", "todos", "união de eventos", ou contar o complemento de várias condições que podem falhar juntas.
  - *Por que funciona:* para um objeto com t defeitos, ele contribui Σ_{k=0}^{t} (−1)^k C(t, k) = [t = 0]; logo só os objetos sem defeito sobrevivem à soma com sinais.
  - *Como fazer:* defina o universo e os defeitos A_1..A_m; N(S) = #objetos que satisfazem todos os defeitos de S. A resposta é Σ_{S⊆[m]} (−1)^{|S|}N(S); se N(S) = f(|S|), vira Σ_{k=0}^{m} (−1)^k C(m, k)f(k). Custo: m avaliações de f.
  - *Exemplo:* desarranjos: N(S) = (n−|S|)! para fixar um conjunto S de pontos fixos, então D_n = Σ_k (−1)^k C(n, k)(n−k)!.
  - *Armadilha:* incluir S = ∅ com sinal +; não confundir "exatamente k" com "pelo menos k"; o número de defeitos considerados é m, não n.
- **Condição boa se separa por elemento** → é só multiplicar, sem IE.
  - *Gatilho:* a condição é uma conjunção de restrições independentes, uma por posição, vértice ou item.
  - *Por que funciona:* se as escolhas de cada elemento são independentes e nenhum recurso é compartilhado, o número total é o produto das contagens locais.
  - *Como fazer:* conte as opções permitidas para cada elemento e multiplique; se houver restrição global (soma, número de cores, colisões), o produto não se aplica.
  - *Exemplo:* strings de comprimento n em que a posição i aceita c_i letras: Π c_i.
  - *Armadilha:* condições como "pelo menos uma" ou "todas distintas" acoplam os elementos e pedem IE, DP ou bijeção.
- **Bijeção:** caminhos em grid, parênteses, sequências → Catalan, C(n+m, n), estrelas e barras C(n+k−1, k−1).
  - *Gatilho:* dois conjuntos parecem diferentes, mas têm a mesma estrutura recursiva ou a mesma codificação; o enunciado pede uma contagem com aparência incomum.
  - *Por que funciona:* uma bijeção preserva cardinalidade; em vez de contar o objeto difícil, conta-se o objeto codificado, que costuma ser uma escolha de posições ou um caminho.
  - *Como fazer:* exiba o mapa e a inversa. Exemplos: parênteses balanceados ↔ caminhos de Dyck ↔ árvores binárias (Catalan C_n); caminhos de (0,0) a (n,m) com passos E/N ↔ palavras com n E e m N (C(n+m, n)); composições fracas de n em k partes ↔ escolha de k−1 barras entre n+k−1 símbolos (C(n+k−1, k−1)).
  - *Exemplo:* número de caminhos monotônicos de (0,0) a (3,2) é C(5,3) = 10.
  - *Armadilha:* verificar injetividade e sobrejetividade, inclusive as convenções de borda; Catalan C_n costuma contar objetos de tamanho n, mas alguns textos usam n+1.
- **Reflexão:** caminhos que não cruzam uma linha → total menos os refletidos.
  - *Gatilho:* "não cruza", "fica abaixo de", "nunca passa de", ballot, parênteses balanceados.
  - *Por que funciona:* o primeiro instante em que o caminho cruza a barreira permite refletir o prefixo; isso dá uma bijeção entre caminhos ruins e caminhos irrestritos até um destino refletido.
  - *Como fazer:* conte todos os caminhos e subtraia os que tocam a barreira, refletindo o destino ou o prefixo. Para (0,0)→(n,m) com m≤n que não passa de y = x, o total é C(n+m, n) e os ruins são C(n+m, n+1); se m>n, a resposta é 0.
  - *Exemplo:* caminhos de (0,0) a (2,1) que nunca ficam acima de y = x: C(3,2) − C(3,3) = 3 − 1 = 2 (EEN e ENE).
  - *Armadilha:* o deslocamento da barreira muda o índice refletido; "tocar" e "cruzar" podem ter fórmulas diferentes; confira se o destino já está do lado proibido.
- **Contagem por contribuição:** Σ sobre subconjuntos de max → ordene e conte 2^(i) para cada a_i ser o máximo.
  - *Gatilho:* soma de máximos (ou mínimos) sobre todos os subconjuntos, ou soma de uma estatística que pode ser atribuída a um elemento.
  - *Por que funciona:* cada subconjunto não vazio tem um único máximo após desempatar por índice; a soma total é a soma de a_i vezes o número de subconjuntos em que i é o representante máximo.
  - *Como fazer:* ordene a em ordem crescente com índice 0..n−1. Para a_i ser o máximo, inclua i, exclua i+1..n−1 e escolha qualquer subconjunto de 0..i−1: 2^i subconjuntos. Some a_i·2^i; para mínimo, use 2^{n−1−i}.
  - *Exemplo:* a = [1,2,3]: 1·1 + 2·2 + 3·4 = 17, que é a soma dos máximos dos 7 subconjuntos não vazios.
  - *Armadilha:* com valores repetidos, fixe um critério de desempate (por exemplo, maior índice) para não contar o mesmo subconjunto duas vezes; a fórmula com 1-indexação usa 2^{i−1}.

- **Esperança = soma de probabilidades** de eventos indicadores (linearidade); funciona mesmo com dependência.
  - *Gatilho:* esperança de uma contagem (número de sucessos, pares, valores distintos, comparações) que pode ser escrita como soma de indicadores.
  - *Por que funciona:* E[Σ I_e] = Σ E[I_e] = Σ P(e); a linearidade não exige independência entre os eventos.
  - *Como fazer:* escolha indicadores cuja probabilidade seja fácil; some. Para valores distintos em n sorteios de N: I_x = "x aparece", P = 1 − (1 − 1/N)^n.
  - *Exemplo:* número esperado de pontos fixos de uma permutação aleatória: Σ_i P(i fixo) = n·(1/n) = 1.
  - *Armadilha:* linearidade vale para somas, não para produtos; E[XY] = E[X]E[Y] só com independência (ou após tratar a covariância).
- **E[max]** → Σ P(max ≥ k). **E[número de passos]** → Σ P(ainda não terminou após k).
  - *Gatilho:* variável inteira não negativa, especialmente máximo de várias variáveis ou tempo de absorção.
  - *Por que funciona:* X = Σ_{k≥1} [X ≥ k] para X inteiro não negativo; tomando esperança, E[X] = Σ_{k≥1} P(X ≥ k).
  - *Como fazer:* para max de X_1..X_n, P(max ≥ k) = 1 − P(todos < k); para tempo T, use P(T > k) ou P(T ≥ k) conforme a convenção. Troque a ordem da soma quando o evento for mais fácil de contar.
  - *Exemplo:* máximo de k dados de m faces: E = Σ_{j=1}^{m} (1 − ((j−1)/m)^k).
  - *Armadilha:* ≥ versus > desloca o índice; a fórmula pressupõe valores inteiros não negativos; para E[X^2] use Σ (2k−1)P(X ≥ k).
- **Simetria:** "probabilidade de i vir antes de j" = 1/2; "i é o máximo entre k" = 1/k.
  - *Gatilho:* itens iid ou exchangeáveis e a resposta depende só da ordem relativa, não dos valores.
  - *Por que funciona:* todas as ordens relativas são igualmente prováveis; cada item tem a mesma chance de ocupar cada posição de destaque.
  - *Como fazer:* ignore os valores e trabalhe com a permutação aleatória. P(i antes de j) = 1/2; entre um conjunto de k itens, P(i é o máximo) = 1/k; para "i é o mínimo" idem.
  - *Exemplo:* em quicksort, o pivô é o primeiro entre os elementos do subarray; a probabilidade de comparar i e j é 2/(j−i+1).
  - *Armadilha:* distribuições não idênticas, empates ou informação condicional quebram a simetria; às vezes é preciso condicionar ao conjunto de candidatos.
- **Burnside** quando conta "a menos de rotação/simetria".
  - *Gatilho:* colares, braceletes, colorações de tabuleiro, estruturas não rotuladas, "duas configurações são iguais se...".
  - *Por que funciona:* o lema de Burnside diz que o número de órbitas é a média dos pontos fixos: (1/|G|)Σ_{g∈G}|Fix(g)|.
  - *Como fazer:* identifique o grupo G, conte para cada g as colorações invariantes (k^{#ciclos}) e tire a média. Com contagens de cores fixas, use o índice de ciclos (Pólya).
  - *Exemplo:* colares de n contas com k cores sob rotação: (1/n)Σ_{r=0}^{n−1} k^{gcd(n,r)}.
  - *Armadilha:* usar o grupo errado (cíclico versus diédrico); a identidade também entra na média; para contagens prescritas, o polinômio de ciclos é necessário.
- **Funções geradoras** quando somas de partes têm restrições; produto de polinômios → NTT.
  - *Gatilho:* contar por tamanho, soma ou número de peças; partes independentes; recorrências de convolução.
  - *Por que funciona:* o coeficiente de x^n em um produto acumula as somas dos tamanhos; OGF trata objetos não rotulados e EGF trata objetos rotulados (com divisão por fatorial).
  - *Como fazer:* crie um fator por parte permitida; multiplique os polinômios (NTT se o grau for grande) e leia o coeficiente. Para partes em S: 1/(1 − Σ_{s∈S} x^s).
  - *Exemplo:* partições com partes 1, 2 e 5: coeficiente de x^n em $1/((1-x)(1-x^2)(1-x^5))$.
  - *Armadilha:* não misturar OGF e EGF; truncar no grau máximo; lembrar que o produto de EGFs é convolução binomial, não comum.
- **Lucas** para C(n, k) mod p pequeno; **Kummer** para a potência de p em C(n, k).
  - *Gatilho:* n enorme, p primo pequeno; binomial módulo p; valuation p-ádica; coeficiente binomial par ou ímpar.
  - *Por que funciona:* Lucas escreve n e k na base p e reduz a produto de binomiais de dígitos; Kummer conta os carries ao somar k e n−k na base p.
  - *Como fazer:* Lucas: C(n,k) ≡ Π C(n_i, k_i) (mod p), com C(n_i,k_i)=0 se k_i>n_i. Kummer: v_p(C(n,k)) = número de carries em k + (n−k) na base p. Custo O(log_p n).
  - *Exemplo:* C(n,k) é ímpar (p=2) quando k & (n−k) = 0, equivalente a k ser submáscara de n.
  - *Armadilha:* Lucas exige p primo; Kummer conta carries, não só dígitos diferentes; para p composto use fatoração ou métodos específicos.

- **Fatorial até 10^6** pré-computado com inverso; C(n, k) com n até 10^{18} e k pequeno → produto direto.
  - *Gatilho:* muitas consultas de binomial; n grande, k pequeno; módulo primo.
  - *Por que funciona:* C(n,k) = n(n−1)…(n−k+1)/k!; o denominador tem inverso quando k < p.
  - *Como fazer:* pré-compute fat[i] e invfat[i] até o maior n pequeno. Para n gigante e k pequeno, multiplique os k fatores em O(k) e use invfat[k]. Se k ≥ p ou n ≥ p, aplique Lucas.
  - *Exemplo:* C(10^{18}, 3) mod 998244353: produto de três termos vezes inverso de 6.
  - *Armadilha:* o denominador é 0 mod p quando k ≥ p; n pode ser maior que p sem que k seja; use Lucas nesses casos.
- **Matriz de transição + exponenciação** para contar caminhos de comprimento k.
  - *Gatilho:* contar sequências de exatamente k passos em um grafo, autômato ou cadeia de Markov; k gigante.
  - *Por que funciona:* (A^k)_{ij} é o número de passeios de comprimento k de i a j; a multiplicação de matrizes concatena os passos.
  - *Como fazer:* monte a matriz de adjacência/transição; exponencie por quadrados em O(n^3 log k); se só há um vetor inicial, faça vetor × matriz. Grafos esparsos ou recorrência de ordem pequena podem usar Kitamasa.
  - *Exemplo:* strings de comprimento k sobre {0,1} sem "11": matriz do autômato com estados "termina em 0" e "termina em 1".
  - *Armadilha:* a potência conta passeios (vértices podem repetir), não caminhos simples; conferir orientação da matriz e se k=0 conta o caminho vazio.
- **Stirling, partições, desarranjos** aparecem disfarçados: teste a sequência pequena.
  - *Gatilho:* contagem de partições de conjuntos, permutações com restrições de ciclos, blocos não vazios, "sem ponto fixo".
  - *Por que funciona:* essas famílias têm termos iniciais e recorrências reconhecíveis; a sequência pequena revela a estrutura antes de tentar uma fórmula fechada.
  - *Como fazer:* conte à mão n = 0..6, procure na OEIS e compare com S(n,k), c(n,k), Bell, p(n) e D_n; depois use a recorrência ou a GF correspondente.
  - *Exemplo:* partitionar n rótulos em blocos não vazios → Bell; permutações com k ciclos → Stirling de 1ª espécie; permutações sem ponto fixo → desarranjos.
  - *Armadilha:* rotulado versus não rotulado; convenções para n=0; a sequência pode estar deslocada por um índice.

adicionado em 2026-11-10 por agentes; revisado

- **"Exatamente k" a partir de "pelo menos k".** Se g(k) = Σ_{j≥k} C(j, k)f(j) então f(k) = Σ_{j≥k} (−1)^{j−k}C(j, k)g(j) (inversão binomial): conte quantos têm ≥ k defeitos marcados e inverta.
  - *Gatilho:* você sabe contar objetos com pelo menos k marcas/defeitos, mas o enunciado pede exatamente k.
  - *Por que funciona:* é a inversão da matriz triangular g(k) = Σ_j C(j,k)f(j); a soma alternada isola f(k).
  - *Como fazer:* calcule g(k) para todo k (por DP ou IE); aplique f(k) = Σ_{j≥k} (−1)^{j−k} C(j,k) g(j). Custo O(m^2) se todos os k forem pedidos.
  - *Exemplo:* se g(k) conta strings com pelo menos k posições marcadas, a inversão dá a distribuição exata do número de marcas.
  - *Armadilha:* a relação exige que g(k) realmente agregue C(j,k) objetos com j marcas; não é a mesma inversão de "≤ k"; sinais e limite superior importam.
- **IE de surjeções.** #sobrejeções [n] → [k] = Σ_j (−1)^j C(k, j)(k−j)^n = k!·S(n, k); desarranjos = Σ (−1)^j n!/j!.
  - *Gatilho:* funções sobrejetoras, distribuir n bolas rotuladas em k caixas não vazias, permutações sem ponto fixo.
  - *Por que funciona:* comece com todas as k^n funções e remova as que evitam cada um dos k valores; IE sobre os valores faltantes.
  - *Como fazer:* Surj(n,k) = Σ_{j=0}^{k} (−1)^j C(k,j)(k−j)^n = k!S(n,k). Desarranjos: use N(S)=(n−|S|)! e some com sinal.
  - *Exemplo:* n=3, k=2: $2^3 - 2\cdot 1^3 = 6$ sobrejeções.
  - *Armadilha:* conferir se as caixas são rotuladas; k>n dá 0; para k=0 e n=0 a convenção precisa ser tratada à parte.
- **IE com defeitos estruturados.** Se N(S) só depende da estrutura (blocos, intervalos), faça DP que soma sinais ao decidir se cada defeito entra; vira O(n^2) em vez de 2^n. n ≤ 20 → máscara.
  - *Gatilho:* os defeitos não são independentes, mas N(S) depende apenas de quantos/ quais blocos foram escolhidos, e o número de defeitos é grande.
  - *Por que funciona:* a soma de IE é uma soma com sinais sobre escolhas locais; se o efeito de escolher um defeito pode ser resumido por um estado pequeno, um DP calcula a soma sem enumerar todos os subconjuntos.
  - *Como fazer:* defina o estado (posição, número de defeitos escolhidos, estrutura acumulada); ao processar cada defeito, ramifique entre incluí-lo (sinal −) e não incluí-lo. Para m ≤ 20, máscara com popcount e sinal.
  - *Exemplo:* proibir que certos intervalos contenham um padrão; o DP varre os intervalos e mantém a última posição coberta.
  - *Armadilha:* o sinal é (−1)^{|S|}, não (−1)^{número de objetos}; estados que não capturam toda a dependência geram contagem errada.

- **Min–max (IE de máximos).** max(S) = Σ_{∅≠T⊆S} (−1)^{|T|+1} min(T); vale para esperança também: E[max] = Σ (−1)^{|T|+1}E[min T]. Útil quando min é fácil (mínimo de exponenciais/prob. por estado é soma).
  - *Gatilho:* esperança ou soma do máximo de variáveis, e o mínimo de qualquer subconjunto é fácil de calcular.
  - *Por que funciona:* a identidade combina funções de mínimo para produzir o máximo; para cada elemento, os coeficientes cancelam tudo exceto o maior.
  - *Como fazer:* aplique a identidade a E[max] e agrupe por |T| se E[min T] depender só do tamanho; para exponenciais independentes, E[min_{i∈T} X_i] = 1/Σ_{i∈T} λ_i.
  - *Exemplo:* tempo até todos os coupons: E[max] = Σ_{∅≠S} (−1)^{|S|+1}/p(S), onde p(S)=Σ_{i∈S}p_i.
  - *Armadilha:* o número de subconjuntos é 2^n; só use quando houver simetria ou agrupamento; a identidade é para máximos de números, não para eventos arbitrários.
- **Reflexão e ballot.** Caminhos de (0,0) a (n, m) tocando y = x + c (c > 0) = C(n+m, n+c) (reflita o destino), válida para m ≤ n+c; se m > n+c todos os caminhos tocam a reta (total C(n+m, n)) e a fórmula superconta. Ballot: A sempre estritamente à frente com a > b votos = (a−b)/(a+b)·C(a+b, a); permitindo empate (exige a ≥ b) = (a−b+1)/(a+1)·C(a+b, b).
  - *Gatilho:* contagem de caminhos que tocam/cruzam uma diagonal; eleições com A sempre à frente; filas com serviço nunca negativo.
  - *Por que funciona:* refletir o destino (ou o prefixo até o primeiro toque) transforma o evento "toca a reta" em caminhos irrestritos até um ponto espelhado; o ballot é a versão "nunca atrás".
  - *Como fazer:* para tocar y=x+c, conte C(n+m,n+c) quando o destino está do lado permitido; se o destino já ultrapassou a reta, todos os caminhos tocam e use o total. Ballot estrito: (a−b)/(a+b)·C(a+b,a); com empate permitido: (a−b+1)/(a+1)·C(a+b,b).
  - *Exemplo:* a=3, b=1, A sempre à frente: (2/4)·C(4,3)=2; permitindo empate: (3/4)·C(4,1)=3.
  - *Armadilha:* trocar os índices ao refletir; a fórmula de tocar não é a de nunca tocar; "estritamente à frente" e "nunca atrás" diferem por 1 no numerador.
- **Faixa 0 ≤ y ≤ h.** Número de caminhos que não saem da faixa = soma alternada de reflexões Σ_k [C(n, ·) − C(n, ·)] com O(n/h) termos; para h pequeno use DP O(n·h).
  - *Gatilho:* caminhada com dois limites (faixa horizontal), passos ±1 ou caminhos em grid confinados; h pequeno ou n grande.
  - *Por que funciona:* o método das imagens reflete a barreira superior e a inferior repetidamente; os termos se cancelam e sobram só as imagens compatíveis com os dois limites.
  - *Como fazer:* use imagens y → 2k(h+2) ± y e some com sinais alternados; há O(n/h) imagens relevantes. Para h pequeno, DP[posição][altura] em O(n·h).
  - *Exemplo:* caminhos de ±1 de 0 a 0 em n passos que nunca saem de [0,h]; para h=1 sobram apenas as alternâncias.
  - *Armadilha:* os deslocamentos das imagens dependem de a barreira ser aberta/fechada; para caminhos E/N monotônicos a restrição inferior é automática e a fórmula muda.
- **Cycle lemma (Raney).** Entre as n rotações de uma sequência de passos com soma +1, exatamente uma tem todos os prefixos positivos. Dá Catalan, ballot e árvores k-árias: 1/((m−1)n+1)·C(mn, n) (Fuss–Catalan).
  - *Gatilho:* provar que uma classe de caminhos tem contagem 1/n do total; sequências com soma +1 e prefixos positivos.
  - *Por que funciona:* o primeiro mínimo do passeio marca a rotação privilegiada; as n rotações são distintas quando o período é n, e exatamente uma tem todos os prefixos estritamente positivos.
  - *Como fazer:* normalize a sequência para soma +1; conte todas as rotações e divida por n; para árvores m-árias, use a codificação de Łukasiewicz (ou uma rotação com soma +1) e a mesma contagem de rotações, que dá 1/((m−1)n+1)C(mn,n).
  - *Exemplo:* Dyck paths: caminhos de ±1 com n subidas e n descidas, soma 0; adicione um passo +1 e use o cycle lemma para obter C_n.
  - *Armadilha:* a sequência precisa ter soma +1 e período n; no caso ±1, com soma k>0 há k rotações boas, não uma; "positivo" versus "não negativo" muda a contagem.
- **Catalan: outros disfarces.** Triangulações de (n+2)-gono, árvores binárias de n nós, partições não-cruzadas, emparelhamentos sem cruzar de 2n pontos, permutações que evitam 231 (ou qualquer padrão de tamanho 3), SYT 2×n. Narayana N(n, k) = (1/n)C(n, k)C(n, k−1) conta Dyck com k picos.
  - *Gatilho:* estruturas recursivas com uma divisão em duas partes, não-cruzamento ou padrão proibido de tamanho 3.
  - *Por que funciona:* todas essas famílias têm a decomposição recursiva de Catalan, C_n = Σ_{i=0}^{n−1} C_i C_{n−1−i}; os padrões de tamanho 3 são todos Wilf-equivalentes.
  - *Como fazer:* reconheça a recorrência ou a GF C(x)=1+xC(x)^2; para estatísticas por picos, use Narayana. SYT de forma (n,n) e partições não-cruzadas também dão C_n.
  - *Exemplo:* n=3: 5 triangulações de um pentágono, 5 árvores binárias e 5 partições não-cruzadas de [3].
  - *Armadilha:* o deslocamento do índice (n+2 gono ↔ C_n); "evitar 231" é C_n, mas evitar dois padrões pode dar outra sequência; Narayana conta picos, não vales.

- **Catalan: variantes de passo.** Com passo plano: Motzkin (1, 1, 2, 4, 9, 21, ...); com diagonais em grid: Delannoy D(m, n) = Σ_k C(m, k)C(n, k)2^k; Chung–Feller: o número de passos acima do eixo em caminhos ±1 de 2n passos que voltam a 0 é uniforme (cada valor tem C_n caminhos).
  - *Gatilho:* caminhos com três tipos de passo, diagonais ou uma estatística (tempo acima do eixo) que parece depender da forma.
  - *Por que funciona:* Motzkin adiciona um passo horizontal à recursão de Catalan; Delannoy escolhe k diagonais entre os passos; Chung–Feller dá uma bijeção entre caminhos com diferentes tempos acima do eixo.
  - *Como fazer:* Motzkin: M_n = M_{n−1} + Σ_{i=0}^{n−2}M_iM_{n−2−i}; Delannoy pela fórmula ou DP D(i,j)=D(i−1,j)+D(i,j−1)+D(i−1,j−1); Chung–Feller: conte por rotação/cycle lemma.
  - *Exemplo:* Motzkin M_4=9; Delannoy D(2,2)=13; caminhos de 6 passos que voltam a 0 têm 2k passos acima do eixo e, para cada k=0,1,2,3, exatamente C_3=5 caminhos.
  - *Armadilha:* Motzkin começa em M_0=1; Delannoy com diagonais ponderadas muda a fórmula; "passos acima do eixo" pode contar passos, não tempo.
- **Identidades de binomiais.** Vandermonde Σ_k C(m, k)C(n, r−k) = C(m+n, r); Σ C(n, k)^2 = C(2n, n); hockey stick Σ_{i=r}^{n} C(i, r) = C(n+1, r+1); Σ_{k=0}^{n} C(k, a)C(n−k, b) = C(n+1, a+b+1).
  - *Gatilho:* soma de produtos de binomiais, especialmente com índices que somam uma constante; coeficientes de (1+x)^m(1+x)^n.
  - *Por que funciona:* Vandermonde é o coeficiente de x^r no produto; a soma de quadrados é o caso m=n=r; hockey stick é a soma telescópica de C(i,r) ao longo de uma coluna.
  - *Como fazer:* escolha a identidade pela forma dos índices; para somas com parâmetro livre, compare coeficientes ou use a recorrência de Pascal. Todas valem com C(a,b)=0 fora de 0≤b≤a.
  - *Exemplo:* Σ_k C(3,k)C(2,2−k)=C(5,2)=10; Σ_k C(3,k)^2=1+9+9+1=20=C(6,3).
  - *Armadilha:* os limites inferiores e superiores; a identidade com C(k,a)C(n−k,b) tem n+1 no lado direito, não n; não confunda com Vandermonde direto.
- **Binomiais: somas ponderadas.** Σ k·C(n, k) = n·2^{n−1}; Σ k^{2}C(n, k) = n(n+1)2^{n−2}; Σ_k C(n, k)C(k, m) = C(n, m)2^{n−m}; k·C(n, k) = n·C(n−1, k−1); Σ_{k≤m}(−1)^k C(n, k) = (−1)^m C(n−1, m).
  - *Gatilho:* soma com fator k, k^2 ou C(k,m); contar pares (conjunto, elemento destacado).
  - *Por que funciona:* kC(n,k)=nC(n−1,k−1) remove um elemento destacado; somas de k^2 usam duas escolhas ou derivação da GF; a soma alternada é uma aplicação repetida de Pascal.
  - *Como fazer:* reduza k·C(n,k) e k(k−1)·C(n,k) a binomiais de n−1 e n−2; use (1+x)^n e derivadas para as ponderações; para C(n,k)C(k,m), escolha m elementos e depois os k−m restantes.
  - *Exemplo:* Σ kC(5,k)=5·2^4=80; Σ C(5,k)C(k,2)=C(5,2)2^3=80.
  - *Armadilha:* k^2 = k(k−1)+k, não k·k; a soma alternada truncada em m tem sinal (−1)^m e binomial n−1; confira m=0.
- **Somas parciais de binomiais em O(1) por passo.** S(n, m) = Σ_{i≤m} C(n, i) satisfaz S(n+1, m) = 2S(n, m) − C(n, m): ao deslizar (n, m) numa escada mantenha S sem recomputar.
  - *Gatilho:* precisa de muitas somas parciais de uma linha de Pascal ao longo de um caminho em que n e m andam juntos.
  - *Por que funciona:* C(n+1,i)=C(n,i)+C(n,i−1); somando até m, os dois termos viram 2S(n,m) menos a borda C(n,m).
  - *Como fazer:* mantenha S e o binomial atual; ao aumentar n, atualize S pela fórmula e o binomial por Pascal. Cada passo O(1).
  - *Exemplo:* S(5,2)=1+5+10=16; S(6,2)=2·16−C(5,2)=32−10=22=1+6+15.
  - *Armadilha:* a fórmula vale para a soma até m incluindo i=m; se a soma for i<m, ajuste a borda; módulo primo e divisões por i ao atualizar o binomial.
- **Caminhos em grid com obstáculos.** Ordene os k pontos proibidos; f(i) = caminhos(start, i) − Σ_{j antes de i} f(j)·caminhos(j, i): O(k^2) com fatoriais; boundary em escada → Σ de C(x+y, x) por hockey stick.
  - *Gatilho:* contar caminhos monotônicos que evitam pontos/obstáculos; k pequeno (até alguns milhares) e grid grande.
  - *Por que funciona:* classifique cada caminho ruim pelo primeiro obstáculo que ele visita; f(j) conta caminhos que evitam os anteriores e chegam a j, e o resto é livre.
  - *Como fazer:* ordene os obstáculos por x+y; f(i)=C(x_i+y_i,x_i) − Σ_{j<i} f(j)·C((x_i−x_j)+(y_i−y_j), x_i−x_j). Se a fronteira for uma escada, some binomiais com hockey stick.
  - *Exemplo:* de (0,0) a (2,2) evitando (1,1): total C(4,2)=6; caminhos que passam por (1,1): C(2,1)C(2,1)=4; restam 2.
  - *Armadilha:* ordenar apenas por x ou apenas por y; pontos fora do retângulo; obstáculos repetidos ou na borda; f(i) deve ser "primeiro obstáculo", não "qualquer obstáculo".

- **Estrelas e barras com limites.** x_i ≤ u: IE Σ_j (−1)^j C(k, j)·C(n − j(u+1) + k − 1, k − 1); x_i ≥ l_i: desloque; $x_1 < x_2 < \ldots < x_k$: subtraia i de x_i; multiconjuntos de tamanho k de n tipos = C(n+k−1, k).
  - *Gatilho:* distribuir n itens idênticos em k caixas com cotas, ou escolher multiconjuntos com repetição.
  - *Por que funciona:* sem cotas, estrelas e barras conta C(n+k−1,k−1); a cota superior é tratada por IE sobre as caixas que a violam (reserve u+1 em cada uma).
  - *Como fazer:* x_i≥l_i: substitua y_i=x_i−l_i e reduza n. x_i≤u: IE com j caixas violadas. x_1<…<x_k: y_i=x_i−i torna a sequência não decrescente; conte com barras.
  - *Exemplo:* x_1+x_2+x_3=5, 0≤x_i≤2: total C(7,2)=21; violações: 3·C(4,2)=18; duas violações: 3·C(1,2)=0; resposta 3.
  - *Armadilha:* a cota superior reserva u+1, não u; a ordem dos sinais de IE; para estritamente crescente, o deslocamento é i (1-indexado); limites inferiores diferentes exigem deslocamentos individuais.
- **Escolher sem vizinhos.** k posições de n em linha sem duas adjacentes = C(n−k+1, k); em círculo = n/(n−k)·C(n−k, k); com distância ≥ d entre escolhidos: C(n − (k−1)(d−1), k).
  - *Gatilho:* selecionar posições independentes em uma fila, ciclo ou grade 1D, com proibição de adjacência ou distância mínima.
  - *Por que funciona:* contraia cada posição escolhida junto com o espaço obrigatório à sua direita; isso transforma a restrição em uma escolha sem restrição em um universo menor.
  - *Como fazer:* linha: escolha k entre n−k+1 (cada escolhido "consome" um vizinho). Círculo: fixe se a posição 1 é escolhida ou use a fórmula n/(n−k)C(n−k,k). Distância d: substitua n por n−(k−1)(d−1).
  - *Exemplo:* n=5, k=2, sem adjacentes na linha: C(4,2)=6; no ciclo: 5/(3)·C(3,2)=5.
  - *Armadilha:* no ciclo, k=n ou k>n/2 pode zerar; a fórmula com distância usa d−1, não d; para grades 2D o problema é diferente (transferência).
- **Composições e partições.** Composições de n = 2^{n−1}; com partes em S → GF 1/(1 − Σ_{s∈S} x^s) (recorrência linear). Partições p(n) por pentágonos de Euler: p(n) = Σ_{k≥1} (−1)^{k+1}[p(n − k(3k−1)/2) + p(n − k(3k+1)/2)], O(n√n).
  - *Gatilho:* escrever n como soma ordenada (composição) ou não ordenada (partição), com ou sem restrição nas partes.
  - *Por que funciona:* composições escolhem os cortes entre n−1 espaços; partições são coeficientes do produto 1/∏(1−x^s); os pentagonais vêm da identidade de Euler para o produto.
  - *Como fazer:* composições com partes em S: dp[n]=Σ_{s∈S}dp[n−s], GF 1/(1−Σx^s). Partições: use a recorrência pentagonal com p(0)=1 e p(negativo)=0.
  - *Exemplo:* n=4: composições 2^3=8; partições p(4)=5 (4, 3+1, 2+2, 2+1+1, 1+1+1+1).
  - *Armadilha:* composições são ordenadas, partições não; a recorrência pentagonal usa dois termos por k e sinais alternados; para partes restritas, a GF muda.
- **Partições: dualidades e DP √.** Conjugação: partições com maior parte k ↔ exatamente k partes; distintas = ímpares (Euler). Partições de n em k partes distintas: q(n, k) = q(n−k, k) + q(n−k, k−1), só k ≤ √(2n) → O(n√n). Partes ≤ √n por DP, as partes > √n são em número < √n e se contam por conjugação.
  - *Gatilho:* partições com restrição no número de partes, no tamanho da maior parte ou em partes distintas.
  - *Por que funciona:* o diagrama de Ferrers transposto troca linhas por colunas; para partes distintas, remova 1 de cada uma das k partes e separe o caso em que a menor parte era 1.
  - *Como fazer:* use q(n,k)=q(n−k,k)+q(n−k,k−1), com q(0,0)=1; como k(k+1)/2≤n, só k≤√(2n). Para partes grandes, conjugue e conte as pequenas.
  - *Exemplo:* n=5, k=2 distintas: 4+1 e 3+2, q(5,2)=2.
  - *Armadilha:* a recorrência é para partes distintas; para partes quaisquer use outra; a cota k≤√(2n) vem da soma mínima 1+…+k; convenções de q(0,0).
- **Stirling 2ª espécie.** S(n, k) = (1/k!)·Σ_j (−1)^{k−j}C(k, j)j^n (um valor em O(k log n)); linha inteira por NTT; x^n = Σ_k S(n, k)·x^{\underline{k}}; Σ_i C(m, i)i^n = Σ_k S(n, k)m^{\underline{k}}2^{m−k}.
  - *Gatilho:* partitionar n elementos rotulados em k blocos não vazios; expressar potências em fatoriais descendentes.
  - *Por que funciona:* IE sobre quais dos k blocos ficam vazios; a identidade x^n = Σ S(n,k)x^{\underline{k}} converte potências em fatoriais descendentes.
  - *Como fazer:* um valor por IE em O(k log n); a linha inteira por convolução/NTT; para Σ C(m,i)i^n, troque a ordem e use Σ_i C(m,i)i^{\underline{k}} = m^{\underline{k}}2^{m−k}.
  - *Exemplo:* S(3,2)=3: {1}{23}, {2}{13}, {3}{12}.
  - *Armadilha:* S(n,0)=0 para n>0 e S(0,0)=1; a fórmula de IE tem sinal (−1)^{k−j}; não confundir com Stirling de 1ª espécie.

- **Momentos via Stirling.** $X^n = \Sigma_k S(n, k)X^{\underline{k}}$, e $E[X^{\underline{k}}] = E[\#$ k-tuplas ordenadas de indicadores$]$: calcula $E[X^n]$ (n pequeno) contando k-tuplas.
  - *Gatilho:* esperança de X^n para X que é soma de indicadores (sucessos, comparações, colisões), com n pequeno.
  - *Por que funciona:* $X^{\underline{k}}$ conta k-uplas ordenadas de indicadores distintos; sua esperança é a soma das probabilidades de todas as k-uplas serem 1, que muitas vezes é fácil.
  - *Como fazer:* escreva $E[X^n] = \Sigma_k S(n,k) E[X^{\underline{k}}]$; para cada k, conte as k-uplas ordenadas de eventos que ocorrem e multiplique pelo número de Stirling.
  - *Exemplo:* X = número de pontos fixos; $E[X^2] = S(2,1)E[X] + S(2,2)E[X(X-1)] = 1\cdot1 + 1\cdot1 = 2$ (para $n \geq 2$).
  - *Armadilha:* $X^{\underline{k}}$ é ordenado e sem repetição; para $k>n$ o termo é 0; não confundir com X^k.
- **Bell.** $B_{n+1} = \Sigma_k C(n, k)B_k$; $B_n = \Sigma_k S(n, k)$; triângulo de Bell em $O(n^2)$; Dobinski $B_n = (1/e)\Sigma\, k^n/k!$. Partições de conjunto com restrição de blocos → EGF $\exp(\Sigma\, x^s/s!)$.
  - *Gatilho:* contar todas as partições de um conjunto, sem fixar o número de blocos; blocos com tamanhos permitidos.
  - *Por que funciona:* escolha o bloco que contém o elemento n+1 (ou 1) e parta o resto; somando os tamanhos possíveis obtém-se a recorrência de Bell.
  - *Como fazer:* $B_{n+1}=\Sigma_{k=0}^{n}C(n,k)B_k$; ou some a linha de Stirling. Com blocos de tamanhos em S, use EGF $\exp(\Sigma_{s\in S}x^s/s!)$.
  - *Exemplo:* $B_3=5$: {123}, {1}{23}, {2}{13}, {3}{12}, {1}{2}{3}.
  - *Armadilha:* Bell conta todos os blocos não vazios; a EGF com exp(Σ) é para estruturas rotuladas; restrições de tamanho mudam a série.
- **Stirling 1ª espécie.** $c(n, k)$ = # permutações com k ciclos; $x(x+1)\ldots(x+n-1) = \Sigma\, c(n, k)x^k$; linha por divisão e conquista (ou duplicar com $x \to x + n/2$) + NTT; $c(n, k) = (n-1)c(n-1, k) + c(n-1, k-1)$; $E[\#\text{ciclos}] = H_n$.
  - *Gatilho:* permutações com número de ciclos, produtos de fatores lineares, ou coeficientes de $x^{\overline{n}}$.
  - *Por que funciona:* inserir o elemento n em uma permutação de n−1: ou forma um ciclo novo (c(n−1,k−1)), ou entra em um dos n−1 lugares (n−1)c(n−1,k).
  - *Como fazer:* use a recorrência $O(n^2)$; para uma linha inteira, multiplique os fatores (x+i) por divisão e conquista com NTT. E[#ciclos]=H_n por linearidade.
  - *Exemplo:* n=3: c(3,1)=2 (3-ciclos), c(3,2)=3 (uma transposição e um fixo), c(3,3)=1.
  - *Armadilha:* é a versão sem sinal; a versão com sinal usa $x(x-1)\ldots$; c(n,0)=0 para n>0; a recorrência usa n−1, não n.
- **Desarranjos e permutações especiais.** $D_n = (n-1)(D_{n-1} + D_{n-2}) = nD_{n-1} + (-1)^n \approx n!/e$; com exatamente k pontos fixos: $C(n, k)D_{n-k}$; involuções $I_n = I_{n-1} + (n-1)I_{n-2}$; perms com $x^k = id$: EGF $\exp(\Sigma_{d|k} x^d/d)$.
  - *Gatilho:* permutações sem ponto fixo, com número prescrito de fixos, involuções ou ordem que divide k.
  - *Por que funciona:* no desarranjo, o elemento 1 vai para j e há dois casos (j vai para 1 ou não); involuções: 1 fixo ou trocado com um de n−1; $x^k=id$ significa ciclos de comprimento divisor de k.
  - *Como fazer:* D_n pela recorrência ou IE; com k fixos, escolha os fixos e desarranje o resto; involuções pela recorrência; para ordem | k, use a EGF $\exp(\Sigma_{d|k}x^d/d)$.
  - *Exemplo:* $D_4=9$; permutações de 4 com exatamente 2 fixos: $C(4,2)D_2=6\cdot1=6$.
  - *Armadilha:* "sem ponto fixo" não é "sem ciclo"; a recorrência de D_n tem duas formas equivalentes; a EGF de ordem divide k não é exp(x^k/k) apenas.
- **Ciclo do elemento é uniforme.** Em permutação aleatória, o comprimento do ciclo que contém um elemento fixo é uniforme em 1..n; E[pontos fixos] = 1; P(1 e 2 no mesmo ciclo) = 1/2; E[# recordes] = E[# ciclos] = H_n.
  - *Gatilho:* perguntas sobre o ciclo de um elemento específico, pontos fixos, recordes ou número de ciclos em permutação aleatória.
  - *Por que funciona:* a estrutura de ciclos é exchangeável; o ciclo do elemento 1 tem tamanho uniforme; os indicadores de recorde são independentes com P(i recorde)=1/i.
  - *Como fazer:* para o ciclo de 1, escolha os outros elementos do ciclo e ordene-os: P(tamanho=k)=1/n. Para recordes, use linearidade; para ciclos, E=H_n.
  - *Exemplo:* n=4: P(1 está em um ciclo de tamanho 3)=1/4; E[#ciclos]=1+1/2+1/3+1/4=25/12.
  - *Armadilha:* "uniforme em 1..n" é para um elemento fixo, não para o tamanho de um ciclo escolhido ao acaso; recordes e ciclos têm a mesma esperança, não a mesma distribuição.

- **Inversões.** Mahoniano: GF $\prod_{i=1}^{n} (1 + x + \dots + x^{i-1})$; perms com k inversões em O(n·k) por prefix sum no DP (adicione 0..i−1 inversões ao inserir i); a tabela de inversões ($d_i \in [0, i-1]$) é livre, então restrições por elemento viram produtos.
  - *Gatilho:* número de inversões, estatística de Mahonian, permutações com restrições no número de inversões por elemento.
  - *Por que funciona:* inserir o elemento i na permutação de i−1 cria entre 0 e i−1 inversões novas; a tabela d_i é independente e determina a permutação.
  - *Como fazer:* DP com prefix sums: $dp[i][j] = \sum_{t=0}^{i-1} dp[i-1][j-t]$, otimizado para O(nk). Para restrições por elemento, multiplique as escolhas permitidas de d_i.
  - *Exemplo:* n=3: distribuição 1,2,2,1 para 0,1,2,3 inversões; total 6.
  - *Armadilha:* a inversão é par (ip_j); a tabela d_i é livre, mas o número total é a soma; não confundir com número de descidas.
- **Descidas e Eulerianos.** $A(n, k) = (k+1)A(n-1, k) + (n-k)A(n-1, k-1) = \sum_j (-1)^j C(n+1, j)(k+1-j)^n$; Worpitzky $x^n = \sum_k A(n, k)C(x + k, n)$. Perm com assinatura <><> dada: DP em $O(n^2)$ pelo posto relativo do último elemento + prefix sums.
  - *Gatilho:* permutações com número de descidas, padrões de subida/descida, ou expressões de potências em binomiais.
  - *Por que funciona:* inserir n em uma permutação de n−1: se entra em uma descida existente, o número de descidas não muda; se entra em uma subida ou nas pontas, muda. A fórmula por IE e Worpitzky vêm da contagem por descidas.
  - *Como fazer:* use a recorrência de Eulerian ou a fórmula fechada; para uma assinatura de desigualdades, DP[i][j] com j = posto do último elemento e prefix sums.
  - *Exemplo:* n=3: A(3,0)=1, A(3,1)=4, A(3,2)=1; 1+4+1=6.
  - *Armadilha:* descidas vs subidas (k+1 vs n−k); a fórmula fechada tem n+1 no binomial; para assinaturas, o posto relativo, não o valor absoluto.
- **Zigue-zague.** Alternantes: EGF sec x + tan x; triângulo de Entringer/Seidel $O(n^2)$.
  - *Gatilho:* permutações alternantes (up-down ou down-up), números de Euler zigzag.
  - *Por que funciona:* a EGF de permutações alternantes é sec x + tan x; o triângulo de Entringer conta alternâncias por último elemento.
  - *Como fazer:* use a recorrência de Entringer/Seidel $E(n,k)=E(n,k-1)+E(n-1,n-k)$ para $O(n^2)$, ou extraia coeficientes de sec/tan com convoluções.
  - *Exemplo:* n=3: 2 permutações alternantes (132 e 231 para up-down); n=4: 5.
  - *Armadilha:* "alternante" pode começar subindo ou descendo; sec x conta as de comprimento par e tan x as de comprimento ímpar; os números de Euler não são os Eulerianos.
- **Permutações indecomponíveis / conexas.** $n! = \sum_j I(j)(n-j)! \rightarrow I(n) = n! - \sum_{j<n} I(j)(n-j)!$; (1, 1, 3, 13, 71...). Mesma ideia: contar a estrutura "de um bloco" tirando de todas a decomposição.
  - *Gatilho:* permutações que não se decompõem em blocos iniciais independentes (o prefixo de tamanho j não é fechado), ou estruturas conexas.
  - *Por que funciona:* toda permutação tem um primeiro bloco indecomponível; escolha seu tamanho j, a permutação indecomponível desse bloco e qualquer permutação do resto.
  - *Como fazer:* $I(n)=n!-\sum_{j=1}^{n-1} I(j)(n-j)!$, com I(1)=1; equivalentemente, use a GF $1-1/P(x)$, onde $P(x)=\sum n!x^n$.
  - *Exemplo:* n=4: 24 − [1·6 + 2·2 + 3·1] = 24 − (6+2+3)=13.
  - *Armadilha:* "indecomponível" depende da definição de bloco inicial; a soma exclui j=n; a sequência começa em n=1.
- **Ordens lineares em floresta.** # rotulações com pai < filho (ou extensões lineares de floresta) = $n!/\prod sz(v)$; equivale a "v é o mínimo da sua subárvore" com prob. 1/sz(v), independentes. Para DAG geral n ≤ 20: DP em máscara.
  - *Gatilho:* contar ordens topológicas de uma árvore/floresta com rótulos, ou probabilidade de uma ordem aleatória respeitar pai antes de filho.
  - *Por que funciona:* em uma ordem uniforme, a probabilidade de cada v ser o mínimo da sua subárvore é 1/sz(v); os eventos são independentes, então o produto dá a probabilidade e n! vezes isso dá a contagem.
  - *Como fazer:* calcule sz(v); resposta $n!/\prod sz(v)$. Para um DAG geral com n≤20, $DP[mask]=\sum_{v \in mask, pred(v) \subseteq mask\{v\}} DP[mask\{v\}]$.
  - *Exemplo:* cadeia de 3 nós: 3!/(3·2·1)=1 ordem válida (raiz, filho, neto).
  - *Armadilha:* a fórmula é para floresta (cada nó tem no máximo um pai); em DAG geral não vale; a raiz tem sz=n; a profundidade não entra.

- **Árvore cartesiana / recorrência pelo máximo.** f(n) = Σ_i C(n−1, i−1)f(i−1)f(n−i) (convolução binomial) → EGF com EDO; árvores binárias crescentes = n!.
  - *Gatilho:* estruturas definidas pelo máximo/mínimo, que divide o conjunto em esquerda e direita; árvore cartesiana.
  - *Por que funciona:* o máximo (ou mínimo) é a raiz; escolha quais i−1 elementos vão para a esquerda e recursão independente nos dois lados.
  - *Como fazer:* use a recorrência com binomial; para EGF, a convolução binomial vira produto, e a recorrência vira uma EDO. Árvores binárias crescentes: a EGF T satisfaz T'=(1+T)^2, T(0)=0, logo T=x/(1−x) e a contagem é n!.
  - *Exemplo:* árvore cartesiana de uma permutação: a raiz é o máximo; a permutação é reconstruída pela árvore.
  - *Armadilha:* o binomial C(n−1,i−1) escolhe os rótulos da esquerda; sem ele, a recorrência é para estruturas não rotuladas; a EGF usa n!, não OGF.
- **Intercalações.** Mesclar sequências preservando ordem interna: multinomial n!/∏ n_i!; k-ésima string lexicográfica com multiconjunto: conte completamentos por multinomial a cada escolha.
  - *Gatilho:* contar intercalações de várias sequências, anagramas ou a k-ésima string lexicográfica de um multiconjunto.
  - *Por que funciona:* escolha as posições de cada sequência no total; a ordem interna é fixa, então o número é multinomial. Para a k-ésima, escolha o próximo caractere e conte quantas strings começam com cada opção.
  - *Como fazer:* n!/∏n_i!; para lexicográfico, em cada passo teste os caracteres em ordem e subtraia do k o bloco de completamentos (multinomial do resto).
  - *Exemplo:* intercalar "ab" e "cd": 4!/(2!2!)=6; anagramas de "AAB": 3!/(2!1!)=3.
  - *Armadilha:* se as sequências têm caracteres iguais, a fórmula de multinomial ainda vale para posições, mas strings idênticas podem colidir; no lexicográfico, use o multiconjunto restante, não posições.
- **Hook length e RSK.** # SYT de forma λ = n!/∏ hooks; RSK dá perm ↔ (P, Q) mesma forma, então n! = Σ f_λ^2; LIS = λ_1, LDS = # linhas; # involuções = Σ f_λ. Perms com LIS ≤ 2 = Catalan.
  - *Gatilho:* tableaux, formas de Young, LIS/LDS de permutações, involuções, ou contagem de permutações por forma.
  - *Por que funciona:* a regra dos ganchos conta SYT; RSK é uma bijeção entre permutações e pares de SYT da mesma forma; propriedades de subseqüências viram a forma.
  - *Como fazer:* f_λ=n!/∏hooks; n!=Σf_λ^2; LIS=λ_1 e LDS=λ'_1 (número de linhas); involuções correspondem a P=Q, então Σf_λ.
  - *Exemplo:* λ=(2,1): f=2; n=3: 6=1^2+2^2+1^2; LIS≤2 conta 5 permutações (Catalan C_3).
  - *Armadilha:* convenção de linhas/colunas para LDS; a forma de P e Q é a mesma, mas os rótulos diferem; "LIS≤2" é Catalan, "LDS≤2" também por simetria.
- **Burnside em grid.** Rotações 90°/180° de n×n: ciclos = $n^2/4$ (ou $(n^2-1)/4 + 1$) e $n^2/2$ (ou $(n^2+1)/2$); Fix = k^{#ciclos}. Para cores com contagem fixa use índice de ciclos (Pólya) com GF.
  - *Gatilho:* colorir tabuleiro n×n a menos de rotação (e talvez reflexão), com k cores ou contagens prescritas.
  - *Por que funciona:* uma coloração fixa por uma rotação é constante em cada ciclo de posições; há k escolhas por ciclo. A média de Burnside sobre o grupo dá o número de órbitas.
  - *Como fazer:* conte ciclos: 90° tem $n^2/4$ se n par e $(n^2+3)/4$ se n ímpar; 180° tem $n^2/2$ se n par e $(n^2+1)/2$ se n ímpar. Some Fix sobre as rotações e divida por |G| (4 ou 2, ou 8 com reflexões).
  - *Exemplo:* n=2, k=2, rotações de 90°: ciclos 1, Fix=2; identidade Fix=16; 180° Fix=4; média (16+2+4+2)/4=6.
  - *Armadilha:* incluir a identidade na média; para 90° em n ímpar o centro é um ciclo de tamanho 1; com contagem fixa de cores, k^{ciclos} não basta, use o índice de ciclos.
- **Contagem por simetria/estabilizador.** #classes de estruturas rotuladas = #rotuladas·ā/n!, com ā = média de |Aut| sobre as estruturas rotuladas (Burnside: (1/n!)Σ_π |Fix(π)|); orbit–stabilizer: |órbita| = |G|/|Stab|. Isomorfismo de árvores com raiz: AHU (hash do multiconjunto ordenado dos filhos); sem raiz: centroides.
  - *Gatilho:* contar estruturas não rotuladas a partir de estruturas rotuladas; decidir isomorfismo de árvores; órbitas de uma ação de grupo.
  - *Por que funciona:* cada classe com grupo de automorfismos de tamanho a contribui n!/a estruturas rotuladas; invertendo, #classes = L·média(a)/n!. Orbit–stabilizer dá o tamanho da órbita.
  - *Como fazer:* Burnside: (1/n!)Σ_π|Fix(π)|; ou calcule a média de |Aut|. Para árvores, AHU canonicaliza filhos recursivamente; sem raiz, ache um ou dois centroides e compare.
  - *Exemplo:* árvores com raiz em n rótulos: n^{n−1} rotuladas; sem raiz, Cayley n^{n−2}; a razão é n (escolha da raiz), como esperado.
  - *Armadilha:* |Aut| conta automorfismos, não o tamanho da órbita; para árvores sem raiz, usar um centroide errado pode falhar; o hash precisa ser determinístico e sem colisões práticas.

- **Strings primitivas e Lyndon.** #strings de comprimento n sem período próprio = $\Sigma_{d|n} \mu(n/d)k^d$; #colares primitivos (Lyndon) = (1/n)·esse valor; mínimo período exato d: Möbius sobre divisores.
  - *Gatilho:* strings aperiódicas, colares primitivos, período mínimo, palavras de Lyndon.
  - *Por que funciona:* toda string tem um período mínimo d que divide n; a inversão de Möbius remove as strings com período próprio. Cada colar primitivo tem n rotações distintas e um representante de Lyndon.
  - *Como fazer:* primitivas(n)=$\Sigma_{d|n}\mu(n/d)k^d$; Lyndon=primitivas(n)/n; período mínimo exatamente d: aplique Möbius sobre os divisores de d.
  - *Exemplo:* n=2, k=2: primitivas=$\mu(2)\cdot2+\mu(1)\cdot4=2$ ("01" e "10"); Lyndon=1 ("01" é o representante e "10" é sua rotação).
  - *Armadilha:* "colar" e "bracelete" diferem por reflexão; Lyndon é uma palavra, colar primitivo é uma classe; $\mu(1)=1$ e $\mu(n)=0$ se n tem quadrado.
- **Prüfer.** Sequência de $n-2 \leftrightarrow$ árvore rotulada; vértice v aparece deg(v)−1 vezes. # árvores com graus d_i = $(n-2)!/\prod(d_i-1)!$; com vértice v de grau d: $C(n-2, d-1)(n-1)^{n-d-1}$; Cayley n^{n-2}.
  - *Gatilho:* árvores rotuladas com restrições de grau, contagem de árvores, ou construção de uma árvore a partir de uma sequência.
  - *Por que funciona:* a sequência de Prüfer tem n−2 posições e cada ocorrência de v corresponde a uma aresta incidente; a correspondência é uma bijeção.
  - *Como fazer:* para graus d_i, distribua as ocorrências: $(n-2)!/\prod(d_i-1)!$; para o grau de um vértice específico, escolha suas d−1 posições e preencha as outras com os n−1 rótulos restantes. Cayley: n^{n-2}.
  - *Exemplo:* n=4, graus (2,2,1,1): $(2)!/(1!1!0!0!)=2$ árvores; são os dois caminhos com os vértices de grau 2 no meio.
  - *Armadilha:* a sequência tem n−2, não n; a soma dos graus é 2(n−1); para n=1,2 as convenções são casos especiais.
- **Contar árvores que conectam componentes.** Com k componentes de tamanhos s_i, o # de formas de ligá-las em árvore é $n^{k-2}\prod s_i$; florestas com k raízes especificadas: $k\cdot n^{n-k-1}$; $K_{a,b}$ tem $a^{b-1}b^{a-1}$ geradoras.
  - *Gatilho:* conectar blocos/componentes já formados com arestas, contar árvores geradoras de um grafo completo multipartido, florestas com raízes fixas.
  - *Por que funciona:* contraia cada componente a um supervértice; pelo teorema de Cayley há k^{k-2} árvores de supervértices, e cada aresta entre componentes i,j pode escolher $s_i\cdot s_j$ extremos. O produto final simplifica para $n^{k-2}\prod s_i$.
  - *Como fazer:* use a fórmula para componentes; para florestas com k raízes especificadas, $k\cdot n^{n-k-1}$; para $K_{a,b}$, $a^{b-1}b^{a-1}$.
  - *Exemplo:* dois componentes de tamanhos 2 e 3: n=5, k=2, fórmula $5^0\cdot6=6$ arestas possíveis (2·3).
  - *Armadilha:* a fórmula é para ligar componentes em uma árvore, não para contar todas as árvores dentro delas; as raízes precisam ser especificadas; $K_{a,b}$ é um caso particular.
- **Matrix-Tree.** # árvores geradoras = qualquer cofator de L = D − A (multigrafo ok); com pesos multiplicativos some pesos; dirigido: Laplaciano de grau de entrada → arborescências com raiz escolhida. det mod p em $O(n^3)$, n ≤ ~400; módulo composto: eliminação tipo Euclides.
  - *Gatilho:* contar árvores geradoras de um grafo (com pesos ou multigrafo), arborescências em grafo dirigido.
  - *Por que funciona:* o teorema de Kirchhoff identifica qualquer cofator do Laplaciano com o número de árvores geradoras; pesos viram produtos ao longo da árvore.
  - *Como fazer:* monte L=D−A, apague a linha/coluna de um vértice e calcule o determinante. Para dirigido, use o Laplaciano de grau de entrada e apague a raiz desejada (arborescências saindo da raiz). Determinante por eliminação $O(n^3)$.
  - *Exemplo:* triângulo K_3: L tem cofator 3, que é o número de árvores geradoras.
  - *Armadilha:* apagar a linha e a coluna do mesmo vértice; em multigrafo, A_{ij} é a multiplicidade; módulo composto não permite inverso, use eliminação sem divisão ou Euclides.
- **BEST theorem.** Em grafo dirigido euleriano (conexo nos vértices de grau > 0), # circuitos eulerianos = $t_w(G)\cdot\prod_{v: deg(v)>0}$ (deg(v) − 1)! (t_w = # arborescências com raiz w, qualquer w), contando circuitos como ciclos sem ponto inicial; fixando a aresta inicial o valor é o mesmo.
  - *Gatilho:* contar circuitos/tours eulerianos em grafo dirigido, especialmente com muitas arestas e validação de Euler.
  - *Por que funciona:* escolha uma arborescência para cada vértice (exceto a raiz) que fixa a última saída; as demais arestas podem ser ordenadas livremente, dando os fatoriais.
  - *Como fazer:* verifique que o grafo é euleriano (grau de entrada = saída e fortemente conexo nos vértices ativos); calcule t_w por Matrix-Tree; multiplique por $\prod(deg(v)-1)!$.
  - *Exemplo:* ciclo dirigido de 3 vértices: $t_w=1$, produto 1, um circuito euleriano (a menos de rotação).
  - *Armadilha:* a fórmula é para circuitos (fechados); para caminhos eulerianos com início/fim diferentes, adapte; t_w é o mesmo para todo w em grafo euleriano; ignore vértices isolados.

- **Grafos conexos e DAGs rotulados.** Conexos: c(n) = 2^{C(n,2)} − Σ_{k<n} C(n−1, k−1)c(k)2^{C(n−k,2)} (fixe o vértice 1); DAGs: a(n) = Σ_{k≥1} (−1)^{k+1}C(n, k)2^{k(n−k)}a(n−k) (IE sobre fontes). Em geral: exp/log de EGF.
  - *Gatilho:* contar grafos rotulados conexos, DAGs rotulados, ou estruturas conexas a partir de todas as estruturas.
  - *Por que funciona:* todo grafo tem uma componente conexa contendo o vértice 1; escolha seu tamanho e subtraia. Para DAGs, IE sobre o conjunto de fontes.
  - *Como fazer:* conexos pela recorrência com C(n−1,k−1); DAGs pela soma alternada com 2^{k(n−k)} arestas entre fontes e o resto. Em geral, se A(x) é a EGF de todas e C(x) a de conexas, A=exp(C).
  - *Exemplo:* n=3: grafos totais 2^3=8; conexos = 8 − [c(1)2^{1} + C(2,1)c(2)2^0] = 8 − [1·2 + 2·1·1] = 4.
  - *Armadilha:* a componente que contém o vértice 1 é única; o exp/log é para EGFs; DAGs não podem ter ciclos, e a IE sobre fontes evita contar duas vezes.
- **Contagens clássicas rotuladas.** Árvores com raiz n^{n-1}; florestas com raiz $(n+1)^{n-1}$; funções n→n com ciclos = n^n; emparelhamentos perfeitos (2n−1)!!; grafos 2^{C(n,2)}; injeções n^{\underline{k}}.
  - *Gatilho:* contagens rotuladas básicas que aparecem como subproblema ou caso base.
  - *Por que funciona:* cada fórmula vem de uma escolha independente de arestas, raízes ou imagens; Cayley e o princípio de multiplicação.
  - *Como fazer:* árvores com raiz: escolha a raiz (n) e a árvore (n^{n−2}) → n^{n−1}; florestas com raiz: (n+1)^{n−1}; funções: n escolhas por elemento; emparelhamentos: (2n−1)(2n−3)…1; injeções: n(n−1)…(n−k+1).
  - *Exemplo:* n=3: árvores com raiz 3^2=9; funções 3^3=27; emparelhamentos perfeitos de 4 elementos 3.
  - *Armadilha:* "floresta com raiz" não é "árvore com raiz"; (2n−1)!! conta emparelhamentos não ordenados; injeções exigem k≤n.
- **Funções geradoras: dicionário.** OGF: produto = escolhas independentes; EGF: produto = convolução binomial (rotulado); 1/(1−f) = sequências, exp(f) = conjuntos de componentes, ln(1/(1−x)) = ciclos; árvores com raiz T = x·e^T → $[x^n]T = n^{n-1}/n!$.
  - *Gatilho:* traduzir operações combinatórias (sequência, conjunto, ciclo, árvore) em operações algébricas sobre séries.
  - *Por que funciona:* OGF soma coeficientes por tamanho; EGF divide por n! para compensar rótulos. Produto = par ordenado; 1/(1−f) = sequência; exp = conjunto não ordenado; log = componente conexa.
  - *Como fazer:* escolha OGF ou EGF; escreva a equação estrutural e resolva para a série. Ex.: T=x e^T; use Lagrange para extrair coeficientes.
  - *Exemplo:* T=x e^T dá T=x+x^2+3x^3/2!+… e [x^n]T=n^{n−1}/n!.
  - *Armadilha:* OGF de sequências é 1/(1−f), mas EGF de sequências também é 1/(1−f) apenas se f(0)=0; conjunto é exp, mas "conjunto de tamanho exato k" é f^k/k!.
- **Lagrange (inversão).** Se F = x·φ(F) então $[x^n]F = (1/n)[x^{n-1}]φ(x)^n$; $[x^n]F^k = (k/n)[x^{n-k}]φ^n$. Dá Catalan, árvores k-árias, contagem de árvores por grau.
  - *Gatilho:* série definida implicitamente por F=xφ(F), comum em árvores e caminhos; precisa de um coeficiente específico.
  - *Por que funciona:* a fórmula de inversão de Lagrange-Bürmann extrai coeficientes de potências de F a partir de potências de φ, evitando resolver a equação.
  - *Como fazer:* identifique φ; para [x^n]F use (1/n)[x^{n−1}]φ^n; para F^k use (k/n)[x^{n−k}]φ^n. Calcule a potência por convolução/NTT se necessário.
  - *Exemplo:* para Catalan, F=C−1 satisfaz F=x$(1+F)^2$; então φ(F)=$(1+F)^2$ e [x^n]F=(1/n)x^{n−1}^{2n}=C_n.
  - *Armadilha:* a fórmula exige F(0)=0 e φ(0)≠0; o expoente de φ é n, não n−1; para F^k, k≤n.
- **Strings sem padrão (GF por autocorrelação).** Alfabeto k, padrão de tamanho m com c(x) = Σ x^j sobre os períodos j (j = 0 e os j com borda m−j): GF = c(x)/(x^m + (1−kx)c(x)); ex.: "11" binário → Fibonacci.
  - *Gatilho:* contar strings de comprimento n que evitam um padrão fixo, com n grande e alfabeto uniforme.
  - *Por que funciona:* a autocorrelação do padrão mede os auto-alinhamentos possíveis; a GF racional codifica a primeira ocorrência e as sobreposições.
  - *Como fazer:* calcule c(x) (coeficiente 1 para j=0 e para cada borda de tamanho m−j); monte c(x)/(x^m+(1−kx)c(x)); extraia coeficientes por recorrência linear ou expansão.
  - *Exemplo:* padrão "11", k=2, c=1+x: GF=(1+x)/(1−x−x^2), que gera 2,3,5,8,… para n=1,2,3,4.
  - *Armadilha:* a fórmula é para alfabeto uniforme e padrão fixo; c(x) não é 1; para vários padrões simultâneos, use autômato e matriz de transição.

- **Tempo esperado até um padrão.** Moedas justas: E = Σ_{k: prefixo k = sufixo k} 2^k (HH: 6, HT: 4); alfabeto q: Σ q^k sobre as bordas (incluindo a string toda).
  - *Gatilho:* tempo esperado até a primeira ocorrência de um padrão em sorteios iid; "quanto tempo até sair HH".
  - *Por que funciona:* as sobreposições do padrão criam "falsos começos"; a soma sobre bordas (prefixo = sufixo) pondera cada sobreposição pelo custo de recomeçar.
  - *Como fazer:* liste os k tais que o prefixo de tamanho k é igual ao sufixo de tamanho k, incluindo k=m; some q^k para alfabeto uniforme de tamanho q (ou o inverso das probabilidades no caso viciado).
  - *Exemplo:* HH: k=1,2 → 2+4=6; HT: só k=2 → 4; HHH: 2+4+8=14.
  - *Armadilha:* incluir a string toda; para padrões com borda vazia apenas o termo q^m aparece; com moeda viciada, o peso não é q^k.
- **Recorrência linear gigante.** Sequência de ordem d: Kitamasa/Bostan–Mori O(d log d log n); não conhece a recorrência → Berlekamp–Massey com 2d termos (DP com matriz de transição ≤ 50 estados).
  - *Gatilho:* termo n de uma sequência que satisfaz recorrência linear de ordem pequena, com n gigante; ou descobrir a recorrência a partir dos primeiros termos.
  - *Por que funciona:* a recorrência define uma matriz companheira; a exponenciação calcula o n-ésimo termo sem iterar. Berlekamp–Massey recupera a recorrência mínima de 2d termos.
  - *Como fazer:* se a recorrência é conhecida, use Kitamasa (O(d^2 log n)) ou Bostan–Mori (O(d log d log n)); se não, gere 2d termos por DP e rode Berlekamp–Massey.
  - *Exemplo:* Fibonacci (d=2) com n=10^{18}; ou contar caminhos em um autômato de 50 estados.
  - *Armadilha:* a recorrência precisa ser linear e homogênea (ou com termo forçado); BM exige corpo (módulo primo); termos insuficientes dão recorrência falsa.
- **Sequências P-recursivas.** Catalan, Motzkin, Delannoy satisfazem recorrências com coeficientes polinomiais: ache por ajuste, calcule n até 10^7 em O(n) com inversos pré-calculados.
  - *Gatilho:* sequência combinatória clássica com recorrência de ordem 1 ou 2 e coeficientes polinomiais em n; n grande (10^7), mas não astronômico.
  - *Por que funciona:* a razão entre termos consecutivos é uma função racional de n; multiplicando, obtém-se uma recorrência P(n)a_n=Q(n)a_{n−1}+… que pode ser iterada.
  - *Como fazer:* encontre a razão a_n/a_{n−1} (por ajuste ou tabela), converta em recorrência, pré-compute inversos de 1..N e itere em O(n).
  - *Exemplo:* Catalan: C_n = C_{n−1}·2(2n−1)/(n+1); Motzkin tem recorrência de ordem 2 com coeficientes lineares.
  - *Armadilha:* a recorrência pode ter divisão por zero mod p quando n+1≡0; para n≥p, use outro método ou Lucas; ordem 1 nem sempre é suficiente.
- **Polinômio em n.** Se f(n) é polinômio de grau d (confira diferenças finitas), f(N) para N gigante = Lagrange com d+1 pontos em O(d); Σ_{i<n} i^k é grau k+1; Σ_{i<n} i^k r^i = r^nP(n) − P(0), grau P = k, para r ≠ 1 (em r = 1, Faulhaber de grau k+1).
  - *Gatilho:* a resposta é um polinômio em n ou uma soma de potências; n é enorme e d é pequeno.
  - *Por que funciona:* um polinômio de grau d fica determinado por d+1 pontos; a interpolação de Lagrange avalia em O(d) se os pontos são consecutivos. A soma de potências é polinomial de grau k+1.
  - *Como fazer:* calcule f(0..d) por força bruta/DP; use prefixos e fatoriais para interpolar em N; para Σ i^k r^i, use a forma r^nP(n) −P(0) e ajuste P, ou Faulhaber quando r=1.
  - *Exemplo:* Σ_{i=0}^{n−1}i = n(n−1)/2; Σ i^2 = n(n−1)(2n−1)/6.
  - *Armadilha:* confirme o grau por diferenças finitas; a interpolação exige d+1 pontos distintos; em r=1 a forma com P(n) colapsa e o grau sobe.
- **Filtro de raízes da unidade.** Σ_{k ≡ r (m)} C(n, k) = (1/m)Σ_j ω^{−jr}(1+ω^j)^n; versão discreta: $(1+x)^n$ mod (x^m − 1) por exponenciação rápida de polinômio cíclico (subconjuntos com soma ≡ r mod m).
  - *Gatilho:* soma de coeficientes binomiais em uma classe de congruência; contar subconjuntos por cardinalidade módulo m.
  - *Por que funciona:* a média com raízes m-ésimas da unidade isola os expoentes k ≡ r; a versão polinomial reduz (1+x)^n módulo x^m−1, acumulando os expoentes.
  - *Como fazer:* calcule (1+x)^n mod (x^m−1) por exponenciação de polinômios cíclicos em O(m^2 log n) ou O(m log m log n) com NTT; o coeficiente de x^r é a soma desejada. Para pesos w_i, use ∏(1+x^{w_i}) mod x^m−1.
  - *Exemplo:* n=4, m=2, r=0: C(4,0)+C(4,2)+C(4,4)=1+6+1=8; a média com ω=−1 dá 8.
  - *Armadilha:* a fórmula com (1+x)^n conta por cardinalidade, não pela soma dos valores; para soma de um subconjunto de {1..n}, o produto é ∏(1+x^i); a raiz precisa ser primitiva e o corpo precisa conter ω.

- **Strings com contagem par de cada letra.** Alfabeto k, comprimento n: $2^{-k}\Sigma_j C(k, j)(k - 2j)^n$ (EGF cosh^k).
  - *Gatilho:* strings em que cada letra aparece um número par de vezes (ou par/ímpar com sinal), com n grande.
  - *Por que funciona:* o filtro de paridade por letra usa ±1; a soma com C(k,j) escolhe quais letras recebem sinal negativo, e (k−2j)^n conta as strings com esses pesos.
  - *Como fazer:* use 2^{−k}Σ_{j=0}^{k}C(k,j)(k−2j)^n; equivalentemente, o coeficiente de x^n/n! em cosh(x)^k.
  - *Exemplo:* k=2, n=2: (1/4)[2^2+0+$(-2)^2$]=2 ("aa","bb").
  - *Armadilha:* a fórmula é para todas as k letras com paridade par; para exigir apenas algumas, ajuste o número de letras filtradas; n ímpar pode dar 0.
- **Possibilidade de grau.** Sequência é graphical (Erdős–Gallai): soma par e $\Sigma_{i\le k} d_i \le k(k-1) + \Sigma_{i>k} \min(d_i, k)$ para cada k (ordenada decrescente); constrói por Havel–Hakimi. Bipartido: Gale–Ryser. Torneio: Landau.
  - *Gatilho:* decidir se uma sequência de graus é realizável por um grafo simples, bipartido ou torneio; construir o grafo.
  - *Por que funciona:* Erdős–Gallai é uma condição necessária e suficiente via cotas de arestas entre os k maiores e o resto; Havel–Hakimi remove o maior grau e repete.
  - *Como fazer:* ordene decrescente; verifique soma par, $0\le d_i\le n-1$ e as n desigualdades de EG. Para construir, use Havel–Hakimi (ou Gale–Ryser no bipartido, Landau no torneio).
  - *Exemplo:* (3,3,2,2) soma 10 par; EG com k=1: 3≤0+min(3,1)+min(2,1)+min(2,1)=3; com k=2: 6≤2+min(2,2)+min(2,2)=6; a sequência é graphical (K_4 menos uma aresta).
  - *Armadilha:* a desigualdade vale para todo k, não só k=1; ordenar decrescente; soma ímpar elimina; no bipartido, as duas partes precisam ter somas iguais.
- **Torneios: triângulos.** # 3-ciclos = $C(n, 3) - \Sigma_v C(\text{out}_v, 2)$; em grafo, # caminhos de comprimento 2 = Σ C(deg, 2) e # 4-ciclos = $(1/2)\Sigma_{u<w} C(\text{cnt}(u, w), 2)$ com cnt = vizinhos comuns.
  - *Gatilho:* contar triângulos ou 4-ciclos em grafo/torneio, muitas vezes com n grande e m moderado.
  - *Por que funciona:* em um torneio, cada tripla é ou transitiva (tem uma fonte com duas saídas) ou um 3-ciclo; contar as transitivas por vértice é fácil. Para 4-ciclos, cada ciclo tem duas diagonais.
  - *Como fazer:* torneio: C(n,3)−Σ C(out_v,2), O(n+m). Grafo: caminhos de comprimento 2 = Σ C(deg(v),2); 4-ciclos = $(1/2)\Sigma_{u<w}C(\text{cnt}(u,w),2)$, com cnt por interseção de listas ou bitsets.
  - *Exemplo:* torneio de 4 vértices com graus de saída 2,2,1,1: C(4,3)=4 triplas e Σ C(out_v,2)=1+1+0+0=2, logo há 2 triângulos cíclicos.
  - *Armadilha:* no torneio, out_v é a saída total, não a saída dentro da tripla; no grafo, cada 4-ciclo é contado duas vezes (as duas diagonais); para multigrafos, ajuste.
- **Torres e posições proibidas.** Permutações evitando posições proibidas = $\Sigma_k (-1)^k r_k (n-k)!$ (r_k = # modos de k torres não atacantes nas proibidas); em tabuleiro de Young com colunas ordenadas crescente: dp[i][j] = dp[i−1][j] + dp[i−1][j−1]·(h_i − (j−1)).
  - *Gatilho:* permutações com posições proibidas, tabuleiro de Ferrers/Young, ou contagem de rooks.
  - *Por que funciona:* IE sobre as posições proibidas ocupadas; r_k é o número de escolhas de k posições proibidas sem duas na mesma linha/coluna. No tabuleiro de Young, o DP conta r_k.
  - *Como fazer:* calcule r_k (DP de rooks ou força bruta para tabuleiro pequeno); resposta $\Sigma_k(-1)^k r_k(n-k)!$. No Young com alturas h_i não decrescentes, dp[i][j]=dp[i−1][j]+dp[i−1][j−1].
  - *Exemplo:* n=3, proibido (1,1): $r_0=1$, $r_1=1$; resposta $3!-1\cdot 2!=4$.
  - *Armadilha:* r_k conta torres não atacantes, não escolhas arbitrárias; a ordem das colunas do Young importa; o sinal é (−1)^k.
- **Permanente e determinante.** Permanente 0/1 (= # emparelhamentos perfeitos bipartidos) por Ryser $O(2^n \cdot n)$ com Gray code, $n \le 25$; permanente mod 2 = det mod 2; planar → Kasteleyn (Pfaffian/det). Rank da matriz de Tutte com valores aleatórios = 2·(emparelhamento máximo) em grafo geral.
  - *Gatilho:* contar emparelhamentos perfeitos em grafo bipartido, permanent de matriz 0/1, ou emparelhamento máximo em grafo geral.
  - *Por que funciona:* o permanente expande a soma sobre permutações; Ryser usa IE sobre colunas. Mod 2, sinal e permanente coincidem. Em grafo planar, Kasteleyn orienta as arestas para transformar o permanente em determinante.
  - *Como fazer:* Ryser: $\text{per}(A)=\Sigma_{S\subseteq[n]}(-1)^{n-|S|}\prod_i \Sigma_{j\in S}a_{ij}$, com Gray code para atualizar produtos; para Tutte, substitua arestas por variáveis aleatórias e calcule o posto.
  - *Exemplo:* matriz identidade 3×3: permanente 1; matriz all-ones: permanente 6.
  - *Armadilha:* Ryser é O(2^n n), não O(n!); o sinal é (−1)^{n−|S|}; Kasteleyn exige planaridade e orientação correta; o posto da Tutte é 2·matching, não o matching.

- **Determinantes úteis.** Lema do determinante de matriz: $\det(A + uv^T) = \det(A)(1 + v^T A^{-1} u)$; Vandermonde $\prod(x_j - x_i)$; polinômio característico por Hessenberg $O(n^3)$; tridiagonal → recorrência de 3 termos.
  - *Gatilho:* determinante de matriz com atualização de posto 1, Vandermonde, ou matriz esparsa com estrutura.
  - *Por que funciona:* o lema de Sherman–Morrison evita recalcular o determinante; Vandermonde é o produto das diferenças; Hessenberg e tridiagonal permitem recorrências.
  - *Como fazer:* para posto 1, calcule $\det(A)$ e $v^T A^{-1} u$; para Vandermonde, $\prod_{i<j}(x_j - x_i)$; para tridiagonal, use $D_n = a_n D_{n-1} - b_{n-1} c_{n-1} D_{n-2}$.
  - *Exemplo:* $\det(I+uv^T) = 1 + v^T u$; matriz 2×2 tridiagonal tem determinante $a_1 a_2 - bc$.
  - *Armadilha:* o lema exige A invertível (ou use a versão polinomial); a ordem dos índices no Vandermonde muda o sinal; Hessenberg não é triangular.
- **LGV.** Famílias de caminhos disjuntos em DAG (planaridade não é necessária) com fontes/destinos compatíveis = $\det[e(a_i, b_j)]$; com $e = C(\cdot)$ em grid; permutações não-identidade entram com sinal, então precisa garantir que só a identidade é possível sem interseção. Aplica a ladrilhos de losangos e partições planas.
  - *Gatilho:* contar famílias de caminhos vértice-disjuntos com fontes e destinos fixos, ou tilings que se codificam assim.
  - *Por que funciona:* o determinante expande as permutações com sinal; caminhos que se cruzam cancelam aos pares, sobrando apenas as famílias disjuntas.
  - *Como fazer:* defina a matriz $E_{ij} = \#$caminhos de a_i a b_j; calcule $\det(E)$. Em grid, use binomiais. Verifique que permutações não-identidade não podem ser disjuntas; caso contrário, o determinante conta com sinal e não é a resposta.
  - *Exemplo:* duas fontes e dois destinos em um grid sem interseção; o determinante é $e(a1,b1)e(a2,b2) - e(a1,b2)e(a2,b1)$.
  - *Armadilha:* LGV é para caminhos vértice-disjuntos (ou aresta-disjuntos com adaptação); a planaridade não é necessária, mas a compatibilidade de fontes/destinos é; sinais de permutações.
- **Ladrilhos de dominó.** Largura pequena → DP de perfil (broken profile) ou matriz de transição + exponenciação; planar grande → Kasteleyn.
  - *Gatilho:* contar ladrilhamentos por dominós de uma região; largura pequena e comprimento grande, ou região planar geral.
  - *Por que funciona:* o perfil registra quais células da fronteira já estão ocupadas; a transição é local. Em regiões planares, Kasteleyn reduz a um determinante/Pfaffian.
  - *Como fazer:* para largura $w \leq \sim 10$, DP de perfil $O(n \cdot 2^w)$ ou matriz de transição 2^w e exponenciação; para grid grande, use Kasteleyn/Pfaffian.
  - *Exemplo:* tabuleiro 2×n: Fibonacci; 3×n: 0 para n ímpar e recorrência linear de ordem 4 (ou de ordem 2 nos n pares); grid n×n: Kasteleyn.
  - *Armadilha:* o perfil precisa incluir a célula atual e a fronteira; a paridade da área é necessária; Kasteleyn não é trivial para regiões com buracos.
- **Zeta/Möbius em subconjuntos.** $\hat{f}(S) = \Sigma_{T \subseteq S} f(T)$ em $O(2^n \cdot n)$, inversa por subtração; convolução de subconjuntos $O(2^n n^2)$ com posto (popcount); OR/AND por zeta; XOR por FWHT. Posets gerais: μ de cadeia = diferença, de divisores = $\mu(n/d)$, de partições = $(-1)^{n-1}(n-1)!$.
  - *Gatilho:* somas sobre subconjuntos, convolução de subconjuntos, ou inversão de Möbius em um poset.
  - *Por que funciona:* a transformada de zeta acumula f(T) em todos os subconjuntos; a inversa remove com o sinal de Möbius. A convolução de subconjuntos usa o posto para evitar sobreposição.
  - *Como fazer:* para cada bit, para cada máscara com o bit, faça f[mask]+=f[mask^(1<<bit)]; a inversa subtrai. Convolução: agrupe por popcount, faça zeta, multiplique e inverta. XOR: FWHT.
  - *Exemplo:* $f(T)=1$ para todo T: $\hat{f}(S) = 2^{|S|}$; a inversa recupera 1.
  - *Armadilha:* a transformada OR acumula subconjuntos; AND acumula superconjuntos; a convolução de subconjuntos não é o produto ponto a ponto; μ de partições é para o poset de partições, não de subconjuntos.
- **Contagem dupla clássica.** $\Sigma_v \deg(v) = 2m$; $\Sigma_{S \subseteq [n]} |S| = n \cdot 2^{n-1}$; Σ de max (ou min) sobre subconjuntos: ordene crescente; max usa $a_{(i)} \cdot 2^{i-1}$, min usa $a_{(i)} \cdot 2^{n-i}$; "contar pares (elemento, conjunto)" pelos dois lados.
  - *Gatilho:* uma soma pode ser contada por elementos ou por objetos; graus, pares, subconjuntos com estatística.
  - *Por que funciona:* contar o mesmo conjunto de pares de duas maneiras dá uma identidade; para max/min, cada subconjunto contribui uma vez.
  - *Como fazer:* identifique os dois lados (elemento, objeto); para max, ordene e use 2^{i-1} (1-indexado); para min, 2^{n-i}; some.
  - *Exemplo:* $\Sigma_v \deg(v) = 2m$; $\Sigma_S |S| = n 2^{n-1}$; max de [1,2,3] sobre subconjuntos = 17.
  - *Armadilha:* desempate de max/min; a contagem dupla precisa do mesmo conjunto de pares; não confundir soma de valores com número de objetos.

- **Somatórios: Abel e inversão de ordem.** $\Sigma a_i b_i = A_n b_n - \Sigma A_i(b_{i+1} - b_i)$; $\Sigma_{i\le n}\Sigma_{j\le i} = \Sigma_{j\le n}\Sigma_{i=j}^{n}$; condição $i\cdot j \le n$ troca para somar sobre $\lfloor n/i \rfloor$ valores.
  - *Gatilho:* soma com produto de sequências, limites triangulares, ou restrição $i\cdot j\le n$; quer reduzir de $O(n^2)$ para $O(n\sqrt{n})$ ou $O(n \log n)$.
  - *Por que funciona:* Abel é soma por partes (discrete integration by parts); inverter a ordem troca o domínio de soma; a hipérbole $i\cdot j\le n$ tem $O(n \log n)$ pares e pode ser agrupada por $\lfloor n/i \rfloor$.
  - *Como fazer:* Abel: acumule $A_i=\Sigma_{k\le i}a_k$ e escreva a soma; para $i\cdot j\le n$, para cada i some $j=1..\lfloor n/i \rfloor$, ou agrupe intervalos de i com o mesmo quociente.
  - *Exemplo:* $\Sigma_{i=1}^{n}\Sigma_{j=1}^{i} = \Sigma_{j=1}^{n}(n-j+1)=n(n+1)/2$.
  - *Armadilha:* os índices de Abel ($A_n b_n$, não A_{n-1}); a condição $i\cdot j\le n$ é simétrica, mas o agrupamento por quociente exige cuidado com blocos; somas com divisão usam floor.
- **Completar o quadrado em somas.** $\Sigma_{i<j}(a_i - a_j)^2 = n\Sigma a^2 - (\Sigma a)^2$; $\Sigma_{i<j} a_i a_j = ((\Sigma a)^2 - \Sigma a^2)/2$; $\Sigma_{i<j}|a_i - a_j| = \Sigma a_{(i)}(2i - n - 1)$ com a ordenado. Soma sobre pares vira $O(n)$ com prefixos.
  - *Gatilho:* soma sobre todos os pares de uma função de diferença ou produto; n grande e $O(n^2)$ inviável.
  - *Por que funciona:* expandir o quadrado separa termos individuais dos produtos cruzados; para valor absoluto, ordenar remove o sinal e cada elemento contribui conforme quantos são maiores/menores.
  - *Como fazer:* use as identidades para quadrados e produtos; para $|a_i-a_j|$, ordene e use coeficientes $(2i-n-1)$ (1-indexado) ou prefixos para variantes com pesos.
  - *Exemplo:* $a=[1,2,3]$: $\Sigma_{i<j}(a_i-a_j)^2=1+4+1=6$; $3\cdot14-36=6$.
  - *Armadilha:* a identidade do valor absoluto exige ordenação; para pares com restrição ($i<j$ e i,j em conjuntos diferentes), adapte; o coeficiente é $2i-n-1$, não $2i-n$.
- **Telescópica.** $1/(i(i+1)) = 1/i - 1/(i+1)$; $1/(i(i+k)) = (1/k)(1/i - 1/(i+k))$; $\Sigma i\cdot i! = (n+1)! - 1$; $\Sigma C(i, k) =$ hockey; produtos $\prod(1 + 1/i) = n+1$. Se uma soma tem termo "$f(i+1) - f(i)$", ache f.
  - *Gatilho:* soma de frações racionais, fatoriais, binomiais ou produtos que se cancelam em cadeia.
  - *Por que funciona:* escrever o termo como diferença de dois termos consecutivos faz todos os intermediários cancelarem; produtos telescópicos cancelam numeradores e denominadores.
  - *Como fazer:* decomponha em frações parciais, procure $f(i+1)-f(i)$, ou reescreva fatoriais/binomiais com Pascal; some os extremos.
  - *Exemplo:* $\Sigma_{i=1}^{n} 1/(i(i+1)) = 1 - 1/(n+1)$; $\Sigma_{i=1}^{n} i\cdot i! = (n+1)!-1$.
  - *Armadilha:* o último termo não cancela; para $1/(i(i+k))$ o fator $1/k$; produtos telescópicos podem exigir log ou índice deslocado.
- **Pombos.** Prefixos mod n: qualquer n inteiros têm bloco consecutivo com soma múltipla de n; Erdős–Ginzburg–Ziv: $2n-1$ inteiros têm n com soma $\equiv 0 \pmod n$; Erdős–Szekeres: n^2+1 distintos têm monótona de tamanho $n+1$ (LIS·LDS $\ge n^2+1$); Dirichlet $|\alpha - p/q| < 1/(qN)$.
  - *Gatilho:* garantir a existência de um bloco, subsequência ou aproximação sem construir; "prove que existe".
  - *Por que funciona:* mais objetos que caixas força uma colisão; prefixos iguais mod n dão bloco com soma múltipla; LIS/LDS dão cotas por posições; Dirichlet usa $q\le N$.
  - *Como fazer:* escolha as caixas (resíduos, pares (LIS,LDS), denominadores); aplique pigeonhole; para EGZ, use a versão clássica ou indução.
  - *Exemplo:* 5 inteiros têm um bloco com soma múltipla de 5; 3 inteiros têm 2 com soma par ($n=2$, $2n-1=3$).
  - *Armadilha:* "n inteiros" inclui o prefixo vazio para o bloco; EGZ pede $2n-1$, não n; Erdős–Szekeres é para distintos; Dirichlet precisa de $q\le N$.
- **Dilworth e Mirsky.** Menor cobertura por cadeias = maior antichain (via matching bipartido); menor cobertura por antichains = maior cadeia; Greene–Kleitman para uniões de k cadeias.
  - *Gatilho:* decompor um poset em cadeias/antichains, ou provar cotas de largura/altura.
  - *Por que funciona:* Dilworth e Mirsky são duais; a versão algorítmica usa matching bipartido para a cobertura mínima por cadeias. Greene–Kleitman generaliza para k cadeias.
  - *Como fazer:* para cobertura por cadeias, monte o grafo bipartido ($x_L \to y_R$ se $x<y$) e calcule o matching máximo; cobertura mínima = n − matching. Para antichains, use a altura.
  - *Exemplo:* poset $1<2$, $1<3$: largura 2 (antichain $\{2,3\}$), cobertura mínima por cadeias 2.
  - *Armadilha:* Dilworth é para cadeias, Mirsky para antichains; a cobertura é por cadeias disjuntas; Greene–Kleitman conta a união de k cadeias, não a cobertura.

- **Linearidade: truques de indicadores.** E[# valores distintos em n sorteios de N] = $N(1 - (1 - 1/N)^n)$; E[comparações do quicksort] = $\Sigma_{i<j}\ 2/(j-i+1)$; E[# operações em "remova subárvore de vértice aleatório"] = $\Sigma_v\ 1/\text{depth}(v)$ (v é escolhido antes dos ancestrais com prob. 1/depth).
  - *Gatilho:* esperança de uma contagem em processos aleatórios, sorteios, ordenação ou remoção de subárvores.
  - *Por que funciona:* escreva a contagem como soma de indicadores; a esperança vira soma de probabilidades, mesmo com dependência. Para a subárvore, v é removido se for escolhido antes de todos os seus ancestrais.
  - *Como fazer:* defina um indicador por valor, par ou vértice; calcule a probabilidade individual. Quicksort: i e j são comparados se um deles for o primeiro pivô entre i..j, prob. $2/(j-i+1)$.
  - *Exemplo:* N=365, n=23: valores distintos esperados = $365(1-(1-1/365)^{23}) \approx 22{,}3$; quicksort: $\Sigma_{i<j}2/(j-i+1)$.
  - *Armadilha:* a independência não é necessária para a esperança, mas é para variância; a profundidade da raiz deve ser 1; o indicador de "v removido" é escolhido antes dos ancestrais, não antes dos descendentes.
- **Troca de ordem/tail sum.** $E[X] = \Sigma_{k\geq 1} P(X \geq k)$; $E[X^2] = \Sigma_{i,j}E[I_i I_j]$; E[máx de k dados de m faces] = $\Sigma_{j=1}^{m} (1 - ((j-1)/m)^k)$; máx de n uniformes em [0, 1] tem média $n/(n+1)$ e o j-ésimo menor $j/(n+1)$.
  - *Gatilho:* esperança de variável inteira, máximo de variáveis iid, estatísticas de ordem, ou soma de indicadores dupla.
  - *Por que funciona:* tail sum expressa X como soma de indicadores $[X\geq k]$; para X^2, $X^2=\Sigma_{i,j}I_iI_j$; para máximo de dados, $P(\max\geq j)=1-P(\text{todos}<j)$.
  - *Como fazer:* troque soma e esperança; para máx de dados, some j=1..m; para uniformes, use a distribuição Beta da j-ésima ordem (média $j/(n+1)$).
  - *Exemplo:* um dado de 6 faces: $E[\max]=\Sigma_{j=1}^6(1-((j-1)/6)^1)=3{,}5$; dois dados: 4,472….
  - *Armadilha:* ≥ vs >; para X^2 a soma dupla inclui i=j; a média do máximo de n uniformes é $n/(n+1)$, que também é $1-1/(n+1)$; não confunda com a média do mínimo, $1/(n+1)$.
- **Exchangeabilidade.** Sem reposição, P(k-ésima retirada ser vermelha) = r/N independente de k; recordes em permutação aleatória: P(i é recorde) = 1/i, independentes; gaps de n pontos em [0, 1] são intercambiáveis (cada um E = $1/(n+1)$).
  - *Gatilho:* amostragem sem reposição, ordem de retirada, recordes, gaps uniformes.
  - *Por que funciona:* a sequência é exchangeável: qualquer permutação das posições tem a mesma distribuição; logo a k-ésima retirada tem a mesma marginal que a primeira. Recordes têm indicadores independentes.
  - *Como fazer:* para sem reposição, ignore a ordem e use a marginal r/N; para recordes, use linearidade e P(i recorde)=1/i; para gaps, use Dirichlet(1,…,1).
  - *Exemplo:* 3 vermelhas, 2 azuis: P(2ª é vermelha)=3/5; gaps de 2 pontos em [0,1] têm média 1/3 cada.
  - *Armadilha:* exchangeabilidade não implica independência; recordes são independentes, mas os valores não; "sem reposição" muda a distribuição conjunta, não a marginal.
- **Coupon collector.** E = $n\cdot H_n$; para colecionar k de n: $\Sigma_{i<k}\ n/(n-i)$; probabilidades desiguais p_i: $\Sigma_{\emptyset\neq S}(-1)^{|S|+1}/p(S)$ (min–max); variância = $E[T^2] - E[T]^2$ com $E[T^2] = \Sigma_{\emptyset\neq S}(-1)^{|S|+1}(2-p(S))/p(S)^2$.
  - *Gatilho:* tempo para coletar todos (ou k) coupons, com probabilidades iguais ou desiguais.
  - *Por que funciona:* no caso uniforme, depois de i coupons, a prob. de um novo é $(n-i)/n$, então o tempo é soma de geométricas. No caso geral, T=max T_i e min–max reduz a mínimos de geométricas.
  - *Como fazer:* uniforme: nH_n; k de n: $\Sigma_{i=0}^{k-1}n/(n-i)$; desiguais: use a soma de min–max sobre subconjuntos; para variância, use a mesma identidade para T^2.
  - *Exemplo:* n=3: E=3(1+1/2+1/3)=5,5; coletar 2 de 3: 3/3+3/2=2,5.
  - *Armadilha:* a soma para k de n começa em i=0; p(S) é a soma das probabilidades; a fórmula de $E[T^2]$ não é o quadrado de E[T]; 2^n subconjuntos.
- **Geométrica.** E[tempo até k sucessos] = k/p; P(primeiro sucesso em passo par) = $(1-p)/(2-p)$; E[min de exponenciais] = $1/\Sigma\lambda_i$ e quem vence é i com prob. $\lambda_i/\Sigma\lambda$.
  - *Gatilho:* tentativas independentes até sucesso, tempo de espera, mínimo de exponenciais, corrida entre processos.
  - *Por que funciona:* a geométrica é sem memória; k sucessos é soma de k geométricas; o mínimo de exponenciais independentes é exponencial com taxa soma, e a corrida é vencida proporcionalmente à taxa.
  - *Como fazer:* E[k sucessos]=k/p; para paridade, some a série ou use a equação $P = (1-p)(1-P)$; para exponenciais, use $1/\Sigma\lambda$ e $\lambda_i/\Sigma\lambda$.
  - *Exemplo:* p=1/2: E[2 sucessos]=4; P(primeiro sucesso em passo par)=1/3; duas exponenciais λ=1,2: E[min]=1/3, P(primeira vence)=1/3.
  - *Armadilha:* "passo par" depende de começar em 1; a geométrica conta tentativas, não falhas; o mínimo de exponenciais não é independente de quem vence.

- **Cadeia de Markov absorvente.** E[tempos] = $(I - Q)^{-1} \cdot 1$ (Gauss, n ≤ ~500, em mod p com inversos). Estrutura linear/árvore: escreva $E_v = \alpha_v \cdot E_{pai} + \beta_v$ e resolva sem Gauss; ciclo: duas incógnitas.
  - *Gatilho:* tempo esperado até absorção em cadeia de Markov; estados não absorventes e transições com probabilidades.
  - *Por que funciona:* condicionando no primeiro passo, $E_v = 1 + \Sigma_w P_{vw} E_w$; em forma matricial, $(I-Q)E=1$. Em árvore, a dependência é só do pai, permitindo substituição.
  - *Como fazer:* monte Q (transições entre não absorventes); resolva o sistema por Gauss $O(n^3)$ ou, se a estrutura for linear/árvore, use $E_v=\alpha_v E_{pai}+\beta_v$ e resolva de baixo para cima. Em ciclo, duas incógnitas.
  - *Exemplo:* passeio absorvente em caminho 0-1-2 com absorção em 0 e 2: $E_1=1$.
  - *Armadilha:* o termo +1 por passo; Q não inclui estados absorventes; módulo p exige que $I-Q$ seja invertível; em árvore, a raiz precisa de condição de contorno.
- **Ruína do jogador e martingais.** Passeio justo ±1 de i até {0, N}: P(chegar a N) = i/N, E[T] = i(N−i); com p ≠ q: $((q/p)^i - 1)/((q/p)^N - 1)$. Optional stopping: ache Φ com $E[\Phi_{t+1} \mid \Phi_t] = \Phi_t$ (ex. $S_t^2 - t$) ou $E[\Delta\Phi] = -1$ (então $E[T] = \Phi_0 - \Phi_{fim}$).
  - *Gatilho:* passeio aleatório com barreiras, probabilidade de atingir um lado, tempo esperado de absorção, ou um martingal adequado.
  - *Por que funciona:* S_t é martingal no caso justo; $S_t^2 - t$ também; com $p \neq q$, $(q/p)^{S_t}$ é martingal. Optional stopping iguala as esperanças nos tempos de parada.
  - *Como fazer:* caso justo: $P=i/N$, $E[T]=i(N-i)$; viciado: use a razão (q/p) e resolva; para E[T], ache Φ com drift −1 e use $E[T]=\Phi_0 - E[\Phi_T]$.
  - *Exemplo:* i=2, N=5: P(chegar a 5)=2/5; E[T]=2·3=6.
  - *Armadilha:* optional stopping exige condições (tempo de parada limitado ou integrável); no caso viciado, a fórmula de P vale para p≠q; não confundir "chegar a N" com "chegar a 0".
- **Passeio aleatório em grafo.** Distribuição estacionária $\pi_v = deg(v)/2m$, retorno esperado 2m/deg(v); em árvore, E[passos u → v adjacente] = 2·|E(componente de u após cortar uv)| + 1; tempo de ida e volta = 2m·R_eff.
  - *Gatilho:* passeio aleatório não dirigido em grafo, tempo de retorno, hitting time em árvore, resistência efetiva.
  - *Por que funciona:* a cadeia é reversível com $\pi_v \propto deg(v)$; o tempo de retorno é $1/\pi_v$. Em árvore, cortar a aresta separa o componente de u e o passeio precisa atravessá-lo.
  - *Como fazer:* use $\pi_v=deg(v)/2m$ e retorno 2m/deg(v); para árvore, calcule o número de arestas do componente de u após remover uv e use 2|E|+1; para ida e volta, 2m·R_eff.
  - *Exemplo:* folha de uma estrela com n vértices: retorno esperado 2(n−1)/1; hitting da folha ao centro =1.
  - *Armadilha:* a fórmula da árvore é para u→v adjacente; o componente é o de u após cortar uv; R_eff é a resistência efetiva, não o hitting time.
- **Wald.** $E[S_T] = E[T] \cdot E[X]$ para T tempo de parada (soma de iid); P(vencer duelos) ou "primeiro a atingir" costuma sair de um argumento de simetria.
  - *Gatilho:* soma aleatória com número aleatório de termos, tempo de parada, ou jogo com incrementos iid.
  - *Por que funciona:* a equação de Wald vale sob condições de integrabilidade; ela transforma $E[S_T]$ em $E[T]E[X]$, permitindo resolver para E[T] quando $E[S_T]$ é conhecido por outro argumento.
  - *Como fazer:* identifique X_i iid, T tempo de parada e S_T; use $E[S_T]=E[T]E[X]$. Em jogos simétricos, use a probabilidade de vitória e o valor final.
  - *Exemplo:* passeio justo com incrementos ±1 e T até {0,N}: $E[S_T]=0$, mas $E[X]=0$, então Wald não dá E[T]; use S_t^2-t.
  - *Armadilha:* Wald não vale para qualquer T (precisa de condições); se E[X]=0, a equação é trivial e não determina E[T]; use um martingal de segunda ordem.
- **Método probabilístico e derandomização.** Atribuição aleatória garante ≥ esperança: MAX-CUT ≥ m/2, 7/8 das cláusulas 3-SAT; construtivo: método das esperanças condicionais (decida cada variável para não diminuir E). Lovász Local Lemma: $e \cdot p \cdot (d+1) \leq 1$.
  - *Gatilho:* provar existência de um objeto bom, ou construir uma atribuição/estratégia; cotas de satisfação.
  - *Por que funciona:* se a esperança de uma variável aleatória é M, existe um resultado com valor ≥M; as esperanças condicionais permitem fixar variáveis uma a uma mantendo a cota. LLL garante que todos os eventos ruins são evitados se as dependências são limitadas.
  - *Como fazer:* defina a variável aleatória (corte, cláusulas satisfeitas); calcule E; para construir, fixe cada variável no valor que não reduz E. Para LLL, verifique $e \cdot p \cdot (d+1) \leq 1$.
  - *Exemplo:* MAX-CUT: aresta é cortada com prob. 1/2, E=m/2; 3-SAT: cada cláusula é satisfeita com prob. 7/8, E=7m/8.
  - *Armadilha:* a esperança garante existência, não uma distribuição uniforme; no método condicional, a escolha precisa preservar a cota; LLL tem condições de dependência e p<1.

- **Aniversário.** P(colisão) ≈ 1 − e^{−n^2/(2N)}: hash de 64 bits com 10^6 itens quase nunca colide, de 32 bits colide; use também para estimar quantas tentativas aleatórias bastam.
  - *Gatilho:* colisões de hash, pares iguais em sorteios, estimar o número de tentativas para repetir um valor.
  - *Por que funciona:* o número de pares é n(n−1)/2; cada par colide com prob. 1/N; para n≪√N, P(colisão)≈n^2/(2N), e a aproximação exponencial é 1−e^{−n^2/(2N)}.
  - *Como fazer:* estime n ≈ √(2N ln(1/(1−P))); para 64 bits e 10^6 itens, n^2/(2N)≈2,7·10^{-8}; para 32 bits, ≈116, colisão quase certa.
  - *Exemplo:* 32 bits, 10^6 hashes: P≈1−e^{−116}≈1; 64 bits: P≈2,7·10^{-8}.
  - *Armadilha:* a aproximação vale para n≪√N; colisões de hash não são independentes se o hash é determinístico; para estimar tentativas, use o mesmo cálculo com P desejado.
- **Esperança como fração mod p.** Calcule E = P/Q com inversos de inteiros pequenos (inv[1..n]); nunca use double se a saída é mod 998244353; DP "esperado do futuro" a partir do estado final.
  - *Gatilho:* a resposta é uma probabilidade/esperança módulo primo; denominadores são produtos de inteiros pequenos.
  - *Por que funciona:* toda probabilidade racional pode ser representada como P·Q^{-1} mod p; a linearidade da esperança vale no corpo finito, desde que os denominadores não sejam múltiplos de p.
  - *Como fazer:* pré-compute inv[1..n] por inv[i]=−(p/i)·inv[p%i] mod p; faça a DP de trás para frente (estado final tem E=0) e multiplique pelas transições; nunca converta para double.
  - *Exemplo:* moeda justa: E = 1/2 = (p+1)/2 mod p; DP de absorção com coeficientes 1/2.
  - *Armadilha:* denominadores podem ser 0 mod p se n≥p; "esperado do futuro" precisa da condição de contorno correta; não use Fermat se o denominador não é inversível.
- **Secretário.** Recuse os primeiros n/e e escolha o primeiro melhor que todos: sucesso ≈ 1/e.
  - *Gatilho:* escolher online o melhor de uma sequência aleatória, sem saber o futuro, com uma única escolha.
  - *Por que funciona:* a estratégia de limiar ótima rejeita uma fração inicial e depois escolhe o primeiro recorde; a probabilidade de sucesso tende a 1/e quando n→∞.
  - *Como fazer:* rejeite os primeiros ⌊n/e⌋ candidatos; depois escolha o primeiro que supere todos os vistos. Para n pequeno, teste o limiar ótimo por DP.
  - *Exemplo:* n=3: rejeite 1, escolha o próximo recorde; sucesso 1/2 (a aproximação 1/e é assintótica).
  - *Armadilha:* a aproximação é assintótica; a ordem aleatória é essencial; se os valores têm distribuição conhecida, a estratégia muda.

Repetidas de outras seções:

- **Subsequências distintas** → dp[i] = 2·dp[i−1] − dp[última ocorrência − 1] (strings).
- **Parênteses/caminhos que não cruzam** → Catalan e reflexão (strings, grid).
- **Contar pares com gcd/condição de divisor** → Möbius de cima para baixo (teoria dos números).
- **n ≤ 20** → somar sobre subconjuntos com SOS; IE sobre máscaras.
- **Sequências com mínimo de prefixo ≥ −k** = C(2n, n) − C(2n, n+k+1) (reflexão); nº de palavras de Lyndon = (1/n) Σ μ(d) k^(n/d).
- **LGV (Lindström–Gessel–Viennot)** → número de famílias de caminhos disjuntos em vértices em DAG = det de matriz de caminhos (quando só a permutação identidade é possível para caminhos disjuntos).
- **Contar caminhos/ciclos de comprimento k** → potência da matriz de adjacência; ciclos de comprimento 3/4 por orientação em m√m.

Bits e XOR

Quase sempre: trate cada bit separadamente, do mais significativo para o menos.

- **Bit a bit:** soma de AND/OR/XOR sobre pares → conte, para cada bit, quantos têm 1 e multiplique por 2^b.
  - *Gatilho:* "soma de a_i & a_j", "soma de a_i | a_j", "soma de a_i ⊕ a_j" sobre todos os pares, com n grande e valores de até 30 bits.
  - *Por que funciona:* cada bit contribui independentemente; o total é Σ_b 2^b · (nº de pares cujo resultado tem 1 no bit b), e esse número depende só dos bits b dos elementos.
  - *Como fazer:* com c_1 elementos tendo 1 no bit b e c_0 = n − c_1, use XOR: c_1·c_0; AND: C(c_1,2); OR: C(n,2) − C(c_0,2). Some 2^b vezes cada contagem para b = 0..29; O(30n).
  - *Exemplo:* [1,2,3]: XOR = 2 (bit 0) + 4 (bit 1) = 6; AND = 1 + 2 = 3; OR = 3 + 6 = 9.
  - *Armadilha:* C(n,2) conta pares i < j; se o enunciado pedir pares ordenados (i ≠ j), dobre as contagens. Use 64 bits na soma.

- **Maximizar XOR** → guloso do bit alto ao baixo; trie binária; base linear (eliminação de Gauss em GF(2)).
  - *Gatilho:* "maior x possível", "maximize a_i⊕x", "subconjunto com xor máximo", com valores de até 30 ou 60 bits.
  - *Por que funciona:* o bit b vale mais que a soma de todos os menores (2^b > Σ_{j<b} 2^j); por isso fixar um bit alto nunca é prejudicado por escolhas nos bits baixos.
  - *Como fazer:* com trie, em cada bit prefira o filho oposto ao bit de x; com base linear, insira os vetores e percorra os pivôs do alto ao baixo fazendo ans = max(ans, ans⊕basis[b]); o guloso de prefixo serve quando o conjunto de candidatos é mantido explicitamente.
  - *Exemplo:* S={1,2,3}, x=2: 2⊕1=3, 2⊕2=0, 2⊕3=1 → máximo 3. Base {3,5}: span {0,3,5,6}, máximo 6.
  - *Armadilha:* "maior xor entre dois elementos" não é escolher os dois maiores; use trie. Em C++, 1<<31 estoura int.
- **XOR de subarray** → prefix xor; xor de caminho em árvore → xor das raízes.
  - *Gatilho:* "xor de a_l..a_r", "subarray com xor k", "caminho com xor k".
  - *Por que funciona:* xor é a própria inversa; com px[i] = a_1⊕...⊕a_i, o xor de l..r é px[r]⊕px[l−1]. Na árvore, px[v] é o xor da raiz até v e o trecho comum raiz–lca aparece duas vezes, cancelando.
  - *Como fazer:* para contar subarrays com xor k, percorra r e some as ocorrências de px[r]⊕k num mapa, inserindo px[0]=0 antes; na árvore, DFS acumulando px e use px[u]⊕px[v] (arestas) ou px[u]⊕px[v]⊕a[lca] (vértices).
  - *Exemplo:* a=[1,2,3] → px=[0,1,3,0]; xor(2..3)=px[3]⊕px[1]=0⊕1=1, que é 2⊕3.
  - *Armadilha:* esquecer px[0]=0 perde os subarrays que começam em 1; com valores nos vértices o lca não cancela (ver **Xor de caminho em árvore**).
- **OR/AND de subarray** muda no máximo 30 vezes ao estender o fim (igual ao gcd).
  - *Gatilho:* contar subarrays por valor de OR/AND, "quantos ORs distintos", n grande e valores de 30 bits.
  - *Por que funciona:* ao acrescentar um elemento, OR só ganha bits e AND só perde; cada mudança liga ou desliga pelo menos um dos ≤30 bits, então há poucos valores distintos por ponto final.
  - *Como fazer:* para cada r, mantenha a lista de pares (valor, menor l) dos sufixos que terminam em r; gere a nova lista aplicando OR/AND com a[r] à anterior e mesclando valores iguais. O(30n).
  - *Exemplo:* OR dos sufixos de [1,2,4] terminando em 4: 4, 6, 7 (três valores); AND dos sufixos de [7,3]: 3, 3&7=3 (um valor).
  - *Armadilha:* o "30" é o nº de bits, não uma constante universal (para 64 bits, até 64 mudanças); a lista guarda valores distintos, mas cada valor representa vários inícios, então conte por diferença de inícios.
- **a + b = (a xor b) + 2(a and b)**; a xor b ≤ a + b.
  - *Gatilho:* relações entre soma e xor, "a+b = a⊕b", sistemas com carry, contar pares que satisfazem igualdades.
  - *Por que funciona:* por bit, a soma sem carry é a⊕b e o and marca as posições em que há carry; esse carry vale 2^{b+1}, ou seja, o dobro na posição seguinte, e a identidade vale globalmente.
  - *Como fazer:* isole a&b = (a+b − (a⊕b))/2; a igualdade a+b = a⊕b vale exatamente quando a&b = 0, isto é, nenhuma posição tem 1 nos dois.
  - *Exemplo:* a=5 (101), b=3 (011): a⊕b=6, a&b=1 → 6 + 2 = 8 = 5+3.
  - *Armadilha:* a fórmula é sempre exata; a identidade equivalente é a+b = (a|b) + (a&b). Não conclua "sem carry" sem verificar a&b = 0.
- **x xor y = 0 ⇔ x = y**: transforma pares em contagem de iguais.
  - *Gatilho:* "pares com xor 0", "subarray com xor 0", agrupar valores iguais.
  - *Por que funciona:* xor é cancelativo; x⊕y=0 implica x=y, e valores iguais sempre dão xor 0.
  - *Como fazer:* conte frequências num mapa; o nº de pares é Σ C(f,2). Para subarrays com xor 0, conte prefixos iguais (ver **XOR de subarray**).
  - *Exemplo:* [1,1,2] tem um par com xor 0 (as duas cópias de 1).
  - *Armadilha:* para xor k ≠ 0 a contagem é de px[r]⊕k, não de iguais.
- **Submáscaras** → enumere com s = (s − 1) & m (total 3^n); somas sobre subconjuntos → SOS.
  - *Gatilho:* "para toda submáscara", "soma sobre subconjuntos", n até ~18–20 e 3^n viável.
  - *Por que funciona:* (s−1)&m é a maior submáscara de m menor que s; a sequência visita todas em ordem decrescente e termina em 0. Sobre todas as máscaras há Σ 2^popcount(m) = 3^n pares (m,s) com s⊆m.
  - *Como fazer:* for (s=m;;s=(s-1)&m){ ...; if(s==0) break; }; para somas sobre submáscaras de todas as máscaras use SOS/zeta O(2^n·n), não 3^n.
  - *Exemplo:* m=101_2=5 → 5, 4, 1, 0.
  - *Armadilha:* 3^20 ≈ 3,5·10^9, então n≤18–19 na prática; não use `while (s < m)` como condição, pois pularia a própria submáscara s=m.

- **Construtivo com XOR:** xor de 0..n tem padrão de período 4; use números grandes (2^29) para "ajustar" o xor final.
  - *Gatilho:* construir array ou permutação com xor alvo, "xor de todos os elementos = x", n fixo.
  - *Por que funciona:* F(n)=0⊕...⊕n depende só de n mod 4 (n, 1, n+1, 0); um número grande isolado altera apenas bits altos e corrige o xor sem mexer nos valores pequenos já escolhidos.
  - *Como fazer:* calcule o xor atual; para trocar de atual para alvo, substitua um elemento y por y⊕(atual⊕alvo); escolha y grande para o resultado continuar dentro do limite e distinto dos demais.
  - *Exemplo:* 0,1,2,3 tem xor 0; para xor alvo 5, troque 3 por 3⊕5=6 (se o limite permitir), ficando 0,1,2,6 com xor 5.
  - *Armadilha:* o ajuste pode colidir com um valor existente ou estourar o limite; em permutação, verifique unicidade e intervalo.
- **Paridade de popcount** e **número de 1s** podem ser o invariante.
  - *Gatilho:* operações que ligam/desligam bits, "é possível transformar", estados classificados por paridade.
  - *Por que funciona:* operações elementares mudam o popcount em ±1 ou preservam sua paridade; quando a paridade (ou o popcount total) é invariante, ela separa classes de estados inalcançáveis entre si.
  - *Como fazer:* calcule popcount(x) mod 2 antes e depois; use como chave/hash ou para podar busca. Em GF(2), procure invariantes lineares: paridade de cada bit ou Σ de bits com pesos.
  - *Exemplo:* x&(x−1) remove exatamente um 1; se o alvo é 0, o nº de passos é pelo menos popcount(x) quando cada passo remove no máximo um 1.
  - *Armadilha:* a paridade de popcount não é invariante para qualquer operação (x⊕=1 muda conforme o bit); confira a operação antes de usar.

adicionado em 2026-11-10 por agentes; revisado

- **Trie xor com contagem por nó.** "Quantos y têm x⊕y < k": desça pelos bits de k; quando o bit de k é 1, some o cnt do filho que dá xor 0 naquele bit e siga pelo que dá 1; quando é 0, siga pelo que dá 0. Com cnt também se faz delete (conjunto dinâmico).
  - *Gatilho:* "quantos y em S com x⊕y < k", contagem por limiar, inserções e remoções.
  - *Por que funciona:* a trie ordena os valores por bit; em cada nível, todos os y que dão xor 0 naquele bit formam um filho inteiro. Se o bit de k é 1, esses y já são menores que k naquele bit e podem ser somados de uma vez.
  - *Como fazer:* desça do bit mais alto; se k_b=1, some cnt[filho que dá xor 0] e siga pelo filho que dá xor 1; se k_b=0, siga só pelo que dá xor 0. O(bits).
  - *Exemplo:* S={0,1,2,3}, x=1, k=3: bit 1 de k soma o ramo de y com bit 1 = 0 ({0,1}, cnt 2); bit 0 de k soma y com bit 0 = 1 ({3}, cnt 1); total 3, que são y=0,1,3 (x⊕y = 1,0,2).
  - *Armadilha:* para "≤ k" conte também o próprio x se ele estiver em S (ou use k+1); se k tem bits acima da trie, todos os y contam; no delete, decremente cnt e não deixe nó órfão atrapalhar.
- **Trie xor persistente / diferença de versões.** Max x⊕a_i com i ∈ [l, r]: guarde cnt por nó em root[i]; só entram ramos com cnt[root[r]] − cnt[root[l−1]] > 0.
  - *Gatilho:* queries de max xor em subarray [l,r], sem updates, com n e q grandes.
  - *Por que funciona:* root[i] é a trie dos a_1..a_i; a diferença de cnt entre as versões r e l−1 informa se existe valor do intervalo naquele ramo, então o guloso por bit pode escolher o ramo oposto a x apenas quando ele tem elemento no intervalo.
  - *Como fazer:* construa uma versão por inserção (cada uma cria O(bits) nós novos); na query, em cada bit prefira o filho oposto se cnt[r]−cnt[l−1] > 0, senão o mesmo. O(bits) por query e O(n·bits) de memória.
  - *Exemplo:* a=[1,3], x=2, intervalo [2,2]: só existe 3; a trie persistente rejeita o ramo que daria 2⊕1=3 e devolve 2⊕3=1.
  - *Armadilha:* root[i] é "primeiros i" e a subtração é com root[l−1]; memória ≈ n·bits nós, não n.
- **Max xor de caminho / de subarray** = max de pref[i]⊕pref[j]; insira os prefixos na trie conforme varre (subarray) ou todos os px[v] da raiz (caminho; o LCA se cancela).
  - *Gatilho:* "maior xor de subarray", "maior xor entre dois nós de um caminho".
  - *Por que funciona:* xor de subarray = px[r]⊕px[l−1]; xor de caminho com valores nas arestas = px[u]⊕px[v], pois o trecho raiz–lca aparece nos dois prefixos e cancela. Os dois problemas viram "max xor entre dois valores de um conjunto".
  - *Como fazer:* subarray: insira px[0..r−1] na trie e consulte px[r]; caminho: insira todos os px[v] e pegue o melhor par. O(n·bits).
  - *Exemplo:* a=[1,2,3] → px=[0,1,3,0]; o maior par é px[0]⊕px[2]=0⊕3=3, que corresponde ao subarray 1..2 (1⊕2=3).
  - *Armadilha:* com valores nos vértices o lca NÃO cancela; inclua a[lca] (ver **Xor de caminho em árvore**). Não emparelhe px[r] consigo mesmo se o subarray vazio não for permitido.

- **k-ésimo menor a_i⊕x** → desça na trie com cnt (se o ramo "xor 0" tem cnt ≥ k vá nele, senão subtraia). Mesma ideia dá o rank de um valor.
  - *Gatilho:* "k-ésimo menor a_i⊕x", "qual o rank de y", consultas de ordem em conjunto dinâmico.
  - *Por que funciona:* no bit b, todos os valores do ramo que dá xor 0 são menores que todos os do ramo que dá xor 1; cnt diz o tamanho de cada grupo, então o k-ésimo está inteiro em um dos dois.
  - *Como fazer:* se cnt[ramo xor 0] ≥ k, desça por ele; senão faça k −= cnt e desça pelo outro. Para o rank de y, acumule os cnt dos ramos que ficam à esquerda a cada nível. O(bits).
  - *Exemplo:* S={0,1,2,3}, x=0, k=2: bit 1, ramo 0 tem {0,1} (cnt 2 ≥ 2), desce; bit 0, ramo 0 tem {0} (cnt 1 < 2), k=1 e vai ao ramo 1 → 1.
  - *Armadilha:* k é 1-indexado; com duplicatas cnt conta ocorrências; se k > |S| não existe resposta.
- **Mex de {a_i⊕x}** → trie sobre os distintos; um ramo "cheio" tem exatamente 2^b folhas; desça pelo ramo cujo xor é 0 se ele NÃO estiver cheio; se estiver cheio, o bit do mex é 1 e siga pelo outro ramo.
  - *Gatilho:* "mex de {a_i⊕x}", "menor inteiro não representado", conjunto transformado por xor.
  - *Por que funciona:* um nó no nível do bit b (com b bits abaixo) cobre 2^b valores possíveis; se o ramo que dá xor 0 está cheio, todos os valores com aquele prefixo existem, então o mex tem bit 1 ali; se não está cheio, o mex ainda está naquele ramo com bit 0.
  - *Como fazer:* desça da raiz; em cada bit olhe o filho que corresponde a xor 0 e compare cnt com 2^b; se cnt < 2^b, bit do mex = 0 e desça nele; se cnt = 2^b, bit = 1 e desça no outro. O(bits).
  - *Exemplo:* S={0,1,2}, x=1 → valores {1,0,3}; o ramo de bit 1 com xor 0 contém {0,1} (cheio), então o mex tem bit 1; no bit 0 o ramo desejado está vazio → mex = 10_2 = 2.
  - *Armadilha:* "cheio" é cnt == 2^b, não cnt > 0; use valores distintos; se o universo tem k bits, o mex pode ser 2^k.
- **Ordem sob xor.** Comparar a⊕x com b⊕y é decidido pelo bit mais alto onde (a⊕x) e (b⊕y) diferem; para x fixo, o bit mais alto onde a e b diferem decide, e x só inverte a ordem se tiver 1 ali.
  - *Gatilho:* comparar valores depois de aplicar xor, "x inverte a ordem de a e b", ordenação por xor.
  - *Por que funciona:* o bit mais alto onde dois números diferem determina qual é maior; para o mesmo x, (a⊕x) e (b⊕x) diferem exatamente nas posições em que a e b diferem, e o bit de x naquela posição troca os lados.
  - *Como fazer:* ache p = msb(a⊕b); se o bit p de x é 0, a ordem é a<b; se é 1, é invertida; para x e y diferentes, use msb((a⊕x)⊕(b⊕y)). O(1) com __builtin_clz.
  - *Exemplo:* a=4 (100), b=6 (110), x=3 (011): a⊕x=7, b⊕x=5, logo a>b; msb(a⊕b)=1 e x tem 1 ali, então a ordem original a<b foi invertida.
  - *Armadilha:* use o bit mais alto, não o mais baixo; a regra simplificada só vale com o mesmo x dos dois lados.
- **Min xor entre dois elementos** → o par ótimo é adjacente no array ordenado. Max xor global → trie.
  - *Gatilho:* "par com menor xor", "maior xor entre dois elementos", n até 2·10^5 e valores de 30 bits.
  - *Por que funciona:* no array ordenado, se a<b<c, seja p o bit mais alto onde a e c diferem; a tem 0 e c tem 1 em p. Se b tem 0 em p, então a⊕b < a⊕c; se b tem 1, então b⊕c < a⊕c. Repetindo, algum par adjacente realiza o mínimo. Para o máximo, pares não adjacentes podem ganhar, então a ordem não basta.
  - *Como fazer:* min: ordene e calcule a_i⊕a_{i+1}, O(n log n). Max: trie, inserindo os valores e consultando o melhor para cada um, O(n·bits).
  - *Exemplo:* [1,5,4] ordenado [1,4,5]: 1⊕4=5, 4⊕5=1 → min 1; max 1⊕4=5.
  - *Armadilha:* o min não é "os dois menores" e o max não é "os dois maiores"; duplicatas dão min 0, e i=j deve ser proibido se o enunciado exigir dois elementos distintos.
- **Min do max a_i⊕x sobre x** → f(nó, b): se só um filho, f(filho); se dois, 2^b + min(f(esq), f(dir)). Recursão pela trie, O(n·bits).
  - *Gatilho:* "minimize max_i (a_i⊕x)" escolhendo x, "menor raio xor" que cobre todos os valores.
  - *Por que funciona:* se todos os valores têm o mesmo bit, escolha x_b igual a esse bit e o xor zera ali; se há os dois filhos, qualquer escolha deixa um lado com 1 naquele bit, contribuindo 2^b, e o pior caso vem do melhor dos dois lados.
  - *Como fazer:* f(nó,b) = 0 numa folha; um filho → f(filho); dois filhos → 2^b + min(f(esq), f(dir)). O(n·bits).
  - *Exemplo:* {0,3}: raiz com dois filhos → 2^1 + min(0,0) = 2; x=1 ou x=2 dá max 2, e não existe x com max < 2.
  - *Armadilha:* o 2^b entra uma vez por nível em que os dois ramos existem no caminho escolhido; não é 2^b + f(esq) + f(dir).

- **Xor-MST (n números, peso = xor)** → divida por bit alto (ou trie): MST de cada metade + uma aresta mínima de xor entre as metades (achada por trie da outra metade). Clássico: CF 888G.
  - *Gatilho:* "MST com peso a_i⊕a_j", n até 2·10^5, valores de 30 bits.
  - *Por que funciona:* no bit mais alto, arestas dentro de cada metade pesam < 2^b e arestas entre metades pesam ≥ 2^b; qualquer MST resolve primeiro cada metade e usa exatamente uma aresta cruzando o corte, a mais barata.
  - *Como fazer:* divida recursivamente pelo bit; some MST(esq) + MST(dir) + min xor entre um valor de esq e um de dir; para a aresta mínima, insira um lado na trie e consulte cada valor do outro. O(n·bits·log V).
  - *Exemplo:* {0,1,2,3}: custos internos 1 (0-1) e 1 (2-3); aresta mínima entre {0,1} e {2,3} é 2 → total 4.
  - *Armadilha:* não faça dois loops para a aresta cruzada; use trie. Duplicatas dão aresta 0 e podem unir componentes.
- **Update xor em todos os elementos / em intervalo** → trie com tag lazy de xor (troca filhos por bit) ou segtree com contagem de bits e flip por bit.
  - *Gatilho:* "xor em todos os elementos", "xor em intervalo", consultas de max/contagem depois dos updates.
  - *Por que funciona:* aplicar xor em todos os valores é uma permutação que, na trie, troca os filhos em cada nível onde o bit do xor é 1; guardar a tag evita descer. Em intervalo, a posição importa e a trie sozinha não é indexada por índice.
  - *Como fazer:* trie global: guarde tag no nó e, ao descer, se o bit correspondente da tag é 1, troque os filhos e propague. Para intervalo, use segtree com cnt[b] de uns por nó e lazy xor: flip troca cnt[b] por len−cnt[b]. O(bits) ou O(log n·bits).
  - *Exemplo:* S={0,1,2,3}, xor 1 → {1,0,3,2}; a trie com tag 1 faz a consulta de max xor sem reconstruir o conjunto.
  - *Armadilha:* a tag da trie serve para xor global; para intervalo, uma trie não indexada por posição não basta, e o flip por bit precisa do tamanho do nó.
- **Range xor update + range xor query** → x só contribui na interseção se o tamanho for ímpar; use duas BITs separadas por paridade do índice.
  - *Gatilho:* updates "a_i ^= x para l≤i≤r" e queries "xor de a_l..a_r".
  - *Por que funciona:* o xor de um intervalo é o xor dos updates que cobrem cada posição; um update (L,R,x) contribui x para [l,r] exatamente quando |[L,R]∩[l,r]| é ímpar. Com d_i = a_i⊕a_{i−1}, o prefixo P(r)=a_1⊕...⊕a_r é o xor dos d_j com j≤r e j≡r (mod 2).
  - *Como fazer:* no update [L,R]^=x faça d_L^=x e d_{R+1}^=x; guarde d por paridade em duas BITs de xor (uma para índices pares, outra para ímpares); P(r) = BIT[r mod 2].prefixo(r); a query é P(r)⊕P(l−1). O(log n).
  - *Exemplo:* n=5, update [2,4]^=1: a=[0,1,1,1,0]; query [2,4] = P(4)⊕P(1) = 1⊕0 = 1.
  - *Armadilha:* P(r) usa a paridade de r, não de l; se R+1=n+1, ignore ou dimensione n+2; aqui é xor/paridade, não soma.
- **Base linear (Gauss em GF(2)).** Insira vetores; rank r ≤ nº de bits. Nº de xors distintos de subconjuntos = 2^r, cada valor alcançável aparece 2^(n−r) vezes; x é alcançável ⇔ reduz a 0.
  - *Gatilho:* "xor de subconjunto", "max xor", "x é representável", n e bits até 30/60.
  - *Por que funciona:* os vetores geram um subespaço de GF(2)^B de dimensão r; a aplicação de GF(2)^n (escolhas) no span tem núcleo de tamanho 2^{n−r}, então cada valor do span tem exatamente 2^{n−r} subconjuntos.
  - *Como fazer:* insira com pivô no bit mais alto: reduza o vetor pelos pivôs existentes e, se sobrar ≠0, guarde em basis[msb]; rank = nº de pivôs. Para max xor, percorra os pivôs do alto ao baixo e faça ans = max(ans, ans⊕basis[b]). O(n·B).
  - *Exemplo:* {3,5}: pivôs 5 (bit 2) e 3 (bit 1); span {0,3,5,6}, 2^2 = 4 valores, cada um aparece uma vez (n=r=2); máximo 6.
  - *Armadilha:* para k-ésimo menor é preciso reduzir a base; em 64 bits use 1LL<<b; testar se x é alcançável é reduzir x, não comparar com o máximo.
- **k-ésimo menor xor de subconjunto** → base reduzida (forma escalonada reduzida); os bits de k−1 escolhem quais vetores (do menor pivô ao maior) entram.
  - *Gatilho:* "k-ésimo menor xor entre subconjuntos", ordenar o span, "quantos valores menores que X".
  - *Por que funciona:* na base reduzida, cada pivô aparece em um único vetor; escolher o vetor de um pivô não altera os pivôs maiores, então a ordem numérica dos xors é a ordem binária das escolhas, com o menor pivô no bit menos significativo.
  - *Como fazer:* reduza a base eliminando cada pivô dos outros vetores; ordene os vetores por pivô crescente; o bit i de k−1 diz se o vetor de i-ésimo menor pivô entra. k=1 → subconjunto vazio (0). O(r·B).
  - *Exemplo:* base {3 (011), 5 (101)}: span ordenado {0,3,5,6}; k=3 → $k-1=10_2$ → escolhe o vetor de maior pivô (5) → 5.
  - *Armadilha:* sem a forma reduzida os bits de k não correspondem à ordem; se o problema conta só subconjuntos não vazios, desloque k em 1.

- **Base com prefixo (queries [l, r] de max xor de subconjunto)** → mantenha, ao varrer r, a base com o índice mais recente em cada pivô (troca ao inserir se o novo é mais recente); query usa só vetores com índice ≥ l.
  - *Gatilho:* queries offline [l,r] de max xor de subconjunto, sem updates, n e q grandes.
  - *Por que funciona:* guardar em cada pivô o vetor de maior índice garante que, ao filtrar índices ≥ l, a base ainda gera todos os xors representáveis com elementos do intervalo; o vetor antigo continua sendo inserido após a troca.
  - *Como fazer:* varra r de 1..n inserindo (a_r, r) e mantendo pos[pivô] máxima (se o novo tem índice maior, troque e continue inserindo o antigo); para query, percorra os pivôs e use só os que têm pos ≥ l no guloso de máximo. O((n+q)·B).
  - *Exemplo:* a=[1,2,4], query [2,3], l=2: a base do prefixo 3 tem os vetores 1,2,4 com posições; filtrando pos≥2 sobram 2 e 4, cujo máximo xor é 6.
  - *Armadilha:* a base muda com l; não use a base global. A troca deve preservar o vetor antigo, senão o span do prefixo se perde.
- **Base com remoções** → segtree no tempo (cada vetor vive num intervalo) + cópia da base (≤ 30 inteiros) a cada nível da recursão, em vez de rollback.
  - *Gatilho:* vetores entram e saem ao longo do tempo, queries de max xor em cada instante, operações offline.
  - *Por que funciona:* cada vetor tem um intervalo de vida; a segtree sobre o tempo decompõe esse intervalo em O(log q) nós. A base é minúscula, então copiá-la na descida é barato e evita o rollback complicado das inserções.
  - *Como fazer:* adicione cada vetor aos nós que cobrem [entrada, saída]; faça DFS com uma cópia local da base, insira os vetores do nó e responda nas folhas. O(n·B·log q + q·B).
  - *Exemplo:* operações "adicione x" e "remova x" com query de max xor após cada uma; cada x vive entre a inserção e a remoção.
  - *Armadilha:* rollback da base não é trivial (uma inserção pode alterar vários pivôs); copie. Não confunda a segtree do tempo com uma segtree de posições.
- **n > nº de bits (ou n > 2^k com valores < 2^k)** → pigeonhole: existe subconjunto/subarray de xor 0 (n+1 prefixos em 2^k valores têm repetição). Use para provar que a resposta existe ou limitar o brute force.
  - *Gatilho:* n grande comparado ao nº de bits, "prove que existe subconjunto/subarray com xor 0", limites que parecem impossíveis.
  - *Por que funciona:* n vetores em GF(2)^B são linearmente dependentes se n > B, dando uma combinação não trivial igual a 0. Para subarray, n+1 prefixos em 2^k valores: se n+1 > 2^k, dois são iguais e o trecho entre eles tem xor 0.
  - *Como fazer:* use o argumento para garantir existência ou para limitar uma busca; para achar de fato, hash dos prefixos (subarray) ou base linear (subconjunto). Subconjunto arbitrário ≠ subarray contíguo.
  - *Exemplo:* k=3 bits e n=9 valores → 10 prefixos em 8 valores → repetição → subarray de xor 0. Com 4 vetores de 3 bits, há dependência linear.
  - *Armadilha:* a condição do subarray é n+1 > 2^k, não n > k; a do subconjunto é n > B; não troque as duas nem conclua que o subarray é o conjunto inteiro.
- **Operação "a_i ← a_i⊕a_j"** → o span (espaço gerado) é invariante; classes de equivalência = mesmo span; o mínimo de passos para a forma canônica NÃO depende só de n − rank: depende de quais vetores são redundantes e de suas posições.
  - *Gatilho:* "podemos fazer a_i ^= a_j", "tornar todos iguais/zero", minimizar o nº de operações.
  - *Por que funciona:* a operação é uma operação de linha elementar em GF(2); o espaço gerado pelas linhas não muda. Duas configurações com o mesmo span são equivalentes, mas o custo até uma forma canônica depende de onde estão os vetores redundantes.
  - *Como fazer:* use Gauss para identificar pivôs e vetores redundantes; para eliminar um vetor, combine com os pivôs na ordem certa. n−rank conta redundâncias, mas não determina o custo; trate caso a caso os vetores que já são combinação dos outros e sua posição.
  - *Exemplo:* rank 1, n=2: [a,0] já está canônico (0 passos), enquanto [a,a] precisa de 1 operação; os dois têm n−rank=1.
  - *Armadilha:* a operação não troca posições (uma troca custa 3 xors); n−rank é só uma contagem de redundantes, não uma fórmula do mínimo.
- **Soma de xor de todos os subconjuntos** = (OR de todos) · 2^(n−1) (cada bit presente em algum elemento aparece em metade dos subconjuntos).
  - *Gatilho:* "soma dos xors de todos os subconjuntos", "xor de todos os subconjuntos não vazios".
  - *Por que funciona:* para um bit b presente no OR, fixe um elemento w que tem esse bit; para cada escolha dos outros n−1 elementos, os dois subconjuntos (com e sem w) diferem no bit b do xor, então exatamente um dos dois tem o bit 1. Se nenhum elemento tem o bit, ele nunca aparece.
  - *Como fazer:* calcule OR de todos os a_i; a resposta é OR · 2^{n−1}. O subconjunto vazio contribui 0, então "não vazio" dá o mesmo. O(n).
  - *Exemplo:* [1,2]: OR=3, 2^{1}=2 → 6; subconjuntos 0,1,2,3 têm xors 0,1,2,3, soma 6.
  - *Armadilha:* o fator é 2^{n−1}, não 2^n; se o problema pede subconjuntos de tamanho fixo, a fórmula não vale.

- **Xor de todos os subarrays** → a_i (0-indexado) está em (i+1)(n−i) subarrays e entra se esse número é ímpar, ou seja, i par e n ímpar; se n é par, o resultado é 0.
  - *Gatilho:* "xor de todos os subarrays", "cada elemento aparece quantas vezes", soma/xor sobre todos os subarrays.
  - *Por que funciona:* a_i aparece em (i+1) escolhas de início e (n−i) de fim; no xor total, só importam as ocorrências ímpares, pois pares se cancelam.
  - *Como fazer:* (i+1)(n−i) é ímpar ⇔ i par e n ímpar; se n é par, todos cancelam e a resposta é 0; se n é ímpar, faça xor dos a_i com i par. O(n).
  - *Exemplo:* [1,2,3], n=3: i=0 e i=2 entram → 1⊕3=2; conferindo os seis subarrays, o xor total é 2.
  - *Armadilha:* o índice é 0-based; se n for par, não há exceção; não confunda com a soma dos subarrays.
- **Soma de a_i⊕a_j, a_i&a_j, a_i|a_j sobre pares** → por bit: $c_1 \cdot c_0$, $C(c_1,2)$, $C(n,2) - C(c_0,2)$, vezes 2^b.
  - *Gatilho:* somas sobre todos os pares de xor, and ou or.
  - *Por que funciona:* cada operação é independente por bit; xor tem 1 quando os bits diferem, and quando ambos são 1, or quando pelo menos um é 1.
  - *Como fazer:* conte c_1 e c_0 por bit; para pares não ordenados use xor c_1c_0, and $C(c_1,2)$, or $C(n,2)-C(c_0,2)$; para ordenados dobre (and vira $c_1(c_1-1)$, or $n(n-1)-c_0(c_0-1)$); multiplique por 2^b. O(30n).
  - *Exemplo:* [1,2,3]: xor 6, and 3, or 9; por bit, xor = 2·1 + 2·2 = 6.
  - *Armadilha:* "pares" pode incluir i=j; nesse caso and/or ganham a diagonal Σa_i; confirme se o enunciado pede i<j.
- **Max AND de um par** → guloso do bit alto: teste cand | 2^b e conte elementos que contêm cand; se ≥ 2, fixe o bit.
  - *Gatilho:* "maior a_i & a_j", "existe par com AND contendo X".
  - *Por que funciona:* se pelo menos dois elementos contêm todos os bits de cand, existe um par cujo AND contém cand; como bits altos dominam os baixos, fixar de cima para baixo é ótimo.
  - *Como fazer:* cand=0; para b do alto ao baixo, t=cand|2^b; conte x com (x&t)==t; se cnt≥2, cand=t. Resposta cand. O(30n).
  - *Exemplo:* [3,5,6]: bit 2, t=4 tem 5 e 6 (2) → cand=4; bit 1, t=6 só 6 (1) → não; bit 0, t=5 só 5 (1) → não; resposta 4 (5&6=4).
  - *Armadilha:* conte (x&t)==t, não x≥t; para AND de um subconjunto de tamanho k, a condição cnt≥k muda a contagem.
- **Pares com a_i + a_j em que o bit k é 1 (ou xor das somas)** → reduza cada valor mod $2^{(k+1)}$, ordene e conte somas ∈ $[2^k, 2^{(k+1)}) \cup [2^{(k+1)}+2^k, 2^{(k+2)})$ com two pointers. Xor de todas as somas = xor dos bits obtidos (CF 1322B).
  - *Gatilho:* "quantos pares têm bit k =1 em a_i+a_j", "xor de todas as somas de pares", CF 1322B.
  - *Por que funciona:* o bit k da soma depende só dos bits 0..k de cada parcela; reduzir mod 2^{k+1} preserva esses bits. Com os valores reduzidos, a soma fica em $[0,2^{k+2})$ e o bit k é 1 em duas faixas.
  - *Como fazer:* para cada b, r_i = a_i mod 2^{b+1}; ordene; conte pares com soma em $[2^b,2^{b+1})$ ou $[2^{b+1}+2^b,2^{b+2})$ com two pointers/binária. Para o xor das somas, basta a paridade da contagem por bit. O(30·n log n).
  - *Exemplo:* a=[1,2]: única soma 3 (11_2); b=0 conta 1 par, b=1 conta 1 par, xor = 3.
  - *Armadilha:* a segunda faixa termina em 2^{k+2} (não 2^{k+1}); use 64 bits para k=30; não conte i=j se o problema pede i<j.
- **a + b = (a⊕b) + 2(a&b) invertido.** Dados s = a+b e x = a⊕b: precisa s ≥ x, s − x par, e (s−x)/2 & x = 0; então o nº de pares (a, b) ordenados é $2^{popcount(x)}$. Também a+b = a⊕b ⇔ a&b = 0, e n+x = n⊕x ⇔ x ⊆ ~n.
  - *Gatilho:* dados a+b e a⊕b (ou a&b), "quantos pares", "existe par", reconstruir a e b.
  - *Por que funciona:* a&b = (s−x)/2; posições com x=1 pertencem a exatamente um dos dois, posições com and=1 pertencem aos dois, e não pode haver 1 nos dois ao mesmo tempo. Cada bit de x escolhe qual elemento o recebe.
  - *Como fazer:* verifique s≥x, (s−x) par e (((s−x)/2) & x) == 0; então há $2^{popcount(x)}$ pares ordenados (metade se o problema pede a<b). a+b=a⊕b ⇔ a&b=0; n+x=n⊕x ⇔ (x&n)==0 (x⊆~n).
  - *Exemplo:* s=8, x=6: (8−6)/2=1, 1&6=0 → válido; popcount(6)=2 → pares (1,7),(3,5),(5,3),(7,1).
  - *Armadilha:* $2^{popcount(x)}$ conta pares ordenados; se x=0, há um par a=b, então dividir por 2 exige cuidado. s−x precisa ser par.
- **Identidades úteis:** a+b = (a|b) + (a&b); a⊕b = (a|b) − (a&b); a|b ≥ max(a, b); a&b ≤ min(a, b).
  - *Gatilho:* simplificar expressões com |, &, ⊕, +, trocar uma operação por outra.
  - *Por que funciona:* por bit valem as identidades booleanas: a|b = (a⊕b) + (a&b), pois as posições com 1 nos dois aparecem no or e no and; as desigualdades seguem de or adicionar bits e and removê-los.
  - *Como fazer:* use para eliminar operações: a|b = (a&b) + (a⊕b); a&b = a+b − (a|b); a⊕b = (a|b) − (a&b); a+b = (a⊕b) + 2(a&b). O(1).
  - *Exemplo:* a=5, b=3: a|b=7, a&b=1 → 7+1=8=a+b e 7−1=6=a⊕b.
  - *Armadilha:* as desigualdades não são igualdades; a&b = a+b−(a|b) é exato, mas confira a ordem das parcelas.

- **AND é não-crescente, OR não-decrescente ao estender.** Contar subarrays com AND/OR ≥ k, = k → para cada fim, ≤ 30 pares (valor, menor início); ou two pointers com contador cnt[b] por bit (janela com OR: bit ligado ⇔ cnt[b] > 0).
  - *Gatilho:* contar subarrays por valor de AND/OR, "≥ k", "= k", janela deslizante.
  - *Por que funciona:* ao estender o subarray, AND só perde bits (valor não cresce) e OR só ganha bits (não decresce); para cada fim, os valores distintos formam no máximo 31 blocos de inícios (30 mudanças).
  - *Como fazer:* mantenha a lista de (valor, menor l) por fim, atualize com a[r] e mescle iguais; conte por diferença de inícios. Para OR ≥ k, use two pointers com cnt[b] e bits ligados = cnt[b]>0; para AND, inverta a direção da monotonia. O(30n).
  - *Exemplo:* [7,3]: OR por fim: {7}, depois 7|3=7; AND: {7}, depois 3 e 3&7=3.
  - *Armadilha:* "= k" = f(≥k) − f(≥k+1) quando o valor é monotônico no sentido certo; para AND, a janela que cresce faz o valor cair, então não use a mesma fórmula sem inverter. A contagem "≤30 pares" supõe que o primeiro valor já tem bits; se o sufixo inicial é 0, podem existir 31 valores.
- **Janela deslizante com AND/OR/gcd (não invertíveis)** → duas pilhas (fila com agregado), ou contador por bit; ambos O(n·bits) ou O(n).
  - *Gatilho:* "maior janela com AND ≥ k", "subarray com OR ≤ k", agregados que não têm inversa.
  - *Por que funciona:* a fila com duas pilhas mantém o agregado de cada pilha; remover da frente só exige transferir a pilha de entrada para a de saída (invertendo a ordem) e recalcular os agregados. Cada elemento é transferido uma vez.
  - *Como fazer:* pilha in e pilha out, cada uma com (valor, agregado); push O(1), pop amortizado O(1), query = agg(in) op agg(out). Alternativa para OR/AND: cnt[b] por bit na janela, O(n·bits).
  - *Exemplo:* janela com OR ≤ 7 em [1,2,4,8]: a maior janela sem o 8 tem comprimento 3 (OR 7).
  - *Armadilha:* isso vale para agregados associativos não invertíveis, mas não para remover um elemento do meio; com soma e negativos a monotonia da janela quebra.
- **Simulação de soma/carry.** "Quantos pares (a, b) ≤ A, B com a+b = ... / a&b = 0 / a⊕b ≤ K": digit DP bit a bit, estado [bit][tightA][tightB][carry].
  - *Gatilho:* contar pares com restrições bit a bit e limites A, B, "a+b", "a&b=0", "a⊕b≤K".
  - *Por que funciona:* as restrições são locais por bit e os limites se resumem a "o prefixo já é menor" (tight); o carry da soma é a única dependência entre bits vizinhos, então o estado é pequeno e cobre todas as combinações.
  - *Como fazer:* dp[bit][tightA][tightB][carry]; escolha os bits a_b,b_b respeitando os limites; para a&b=0 proíba (1,1); para xor compare com K; para soma propague o carry. O(bits·8·4).
  - *Exemplo:* contar pares com a≤A, b≤B e a+b=N; no bit b, o carry entra no estado e a comparação com N é feita bit a bit.
  - *Armadilha:* tight precisa considerar o limite de cada variável separadamente; carry é 0/1 na base 2; inicialize tight=1 e responda no estado final.
- **Intervalo [0, A) = ≤ log A blocos alinhados** (prefixo fixo + sufixo livre). Contar x ≤ A com propriedade bit a bit, ou xor de dois intervalos, vira combinação de blocos.
  - *Gatilho:* contar x em [0,A), xor dos números de um intervalo, propriedade bit a bit com limite.
  - *Por que funciona:* percorrendo os bits 1 de A, cada 1 na posição b gera um bloco com esse bit 0 e os b bits abaixo livres; esses blocos são disjuntos e cobrem [0,A). Em cada bloco, os bits livres são independentes.
  - *Como fazer:* para A=13 (1101): blocos [0,8), [8,12), [12,13). Para cada 1 na posição b, some a contribuição do bloco de tamanho 2^b; para xor, use o padrão periódico dentro do bloco. O(log A).
  - *Exemplo:* A=13 cobre [0,8) ∪ [8,12) ∪ [12,13) = [0,13).
  - *Armadilha:* é [0,A), não [0,A]; para ≤A use A+1; o bloco tem 2^b valores, e o prefixo fixo é o de A acima do bit b.
- **Soma dos popcounts de 0..n** → por bit b: ⌊(n+1)/2^(b+1)⌋·2^b + max(0, (n+1) mod 2^(b+1) − 2^b).
  - *Gatilho:* "soma de popcount de 0 a n", "quantos bits 1 no total", contagem de uns por posição.
  - *Por que funciona:* no bit b o padrão é 2^b zeros seguidos de 2^b uns, repetido a cada 2^{b+1}; os ciclos completos contribuem 2^b cada e o resto contribui só a parte que passa de 2^b.
  - *Como fazer:* para b=0..⌊$\log_2$ n⌋, ciclos=⌊(n+1)/2^{b+1}⌋, resto=(n+1) mod 2^{b+1}; some ciclos·2^b + max(0, resto−2^b). O(log n).
  - *Exemplo:* n=5: popcounts 0,1,1,2,1,2 somam 7; b=0 dá 3, b=1 dá 2, b=2 dá 2.
  - *Armadilha:* use n+1 (números de 0 a n); o max(0,·) é obrigatório; em 64 bits use 1LL.

- **Contar x ≤ n com popcount = k** → percorra os bits 1 de n: ao passar por um 1 na posição b, some C(b, k − usados); ao final, some 1 se popcount(n) = k (o próprio n).
  - *Gatilho:* "quantos números ≤ n têm exatamente k bits 1", combinatória com limite superior.
  - *Por que funciona:* quando n tem 1 na posição b, os números com o mesmo prefixo acima e 0 nessa posição são todos menores que n; os b bits abaixo são livres, então basta escolher onde ficam os k−usados uns restantes.
  - *Como fazer:* usados=0; para b do alto ao baixo, se o bit b de n é 1, some C(b, k−usados) e usados++; no fim, se usados==k some 1 (o próprio n). O(bits).
  - *Exemplo:* n=5 (101), k=2: no bit 2, C(2,2)=1 (o 3=011); no bit 0, C(0,1)=0; final +1 (o 5=101) → 2 números.
  - *Armadilha:* C(b, ·) com argumento inválido é 0; b é 0-indexado e há b bits abaixo; o +1 final é o que inclui o próprio n.
- **n como soma de exatamente k potências de 2** ⇔ popcount(n) ≤ k ≤ n (quebre uma potência 2^b em duas 2^(b−1) para aumentar). Mínimo de termos com ±2^i = peso da NAF (forma não adjacente).
  - *Gatilho:* "n como soma de exatamente k potências de 2", "mínimo de termos com ±2^i", representações binárias alternativas.
  - *Por que funciona:* a binária dá popcount(n) termos; quebrar 2^b em 2^{b−1}+2^{b−1} aumenta o nº de termos em 1 e preserva a soma, então todos os valores entre popcount(n) e n são atingíveis. Com sinais, a NAF minimiza o peso.
  - *Como fazer:* verifique popcount(n) ≤ k ≤ n; para o mínimo com ±2^i, calcule a NAF (dígitos −1,0,1 sem dois não nulos adjacentes) e conte os não nulos. O(bits).
  - *Exemplo:* n=6 (110): k=2 → 4+2; k=3 → 4+1+1; k=4 → 2+2+1+1; máximo k=6 (seis 1s).
  - *Armadilha:* "potências" normalmente permite repetição; se exigir potências distintas, só a representação binária vale. k>n é impossível.
- **Gosper's hack.** Próxima máscara com o mesmo popcount: c = x&−x; r = x+c; nxt = (((r⊕x)>>2)/c)|r. Enumera C(n, k) máscaras em ordem crescente.
  - *Gatilho:* enumerar todas as máscaras de n bits com exatamente k bits 1, ou a próxima máscara com mesmo popcount.
  - *Por que funciona:* c isola o 1 menos significativo; x+c propaga o carry e transforma o bloco 011...1 em 100...0; a expressão recoloca os uns restantes o mais à direita possível, gerando a menor máscara seguinte com o mesmo popcount.
  - *Como fazer:* c=x&−x; r=x+c; nxt=(((r⊕x)>>2)/c)|r. Comece em (1<<k)−1 e pare quando passar de n bits. O(1) por máscara.
  - *Exemplo:* x=0b01110 (14) → c=2, r=16, nxt=0b10011 (19), que é a próxima com três bits 1.
  - *Armadilha:* x=0 divide por zero; a fórmula supõe k≥1; em 64 bits use unsigned/1ULL para não estourar.
- **Contador binário** → custo amortizado O(1) por incremento (potencial = nº de 1s); vale só para incrementos puros, não para mistura ±1.
  - *Gatilho:* analisar o custo total de uma sequência de incrementos, "por que O(1) amortizado".
  - *Por que funciona:* o incremento desliga t bits 1 e liga o próximo 0; com potencial Φ=popcount, o custo real 1+t é compensado pela queda de Φ, dando amortizado 2.
  - *Como fazer:* use Φ=popcount e prove custo + ΔΦ ≤ 2; só vale para incrementos puros. Com incrementos e decrementos misturados, alternar 2^k e 2^k−1 custa Θ(k) por operação.
  - *Exemplo:* 0111→1000 custa 4, Φ cai de 3 para 1, amortizado 4−2=2.
  - *Armadilha:* não use esse argumento para operações arbitrárias; o pior caso por incremento continua O(bits).
- **Xor de 0..n** → n mod 4 = 0: n; 1: 1; 2: n+1; 3: 0. Xor de a..b = F(b) ⊕ F(a−1).
  - *Gatilho:* "xor de 1 a n", "xor de a até b", padrão periódico de xor.
  - *Por que funciona:* (4k)⊕(4k+1)⊕(4k+2)⊕(4k+3)=0, então os blocos completos de 4 se cancelam e sobra apenas o resto n mod 4.
  - *Como fazer:* F(n) conforme a tabela (n≥0); xor(a..b)=F(b)⊕F(a−1) com F(−1)=0. O(1).
  - *Exemplo:* xor 1..5 = F(5)⊕F(0) = 1⊕0 = 1; direto: 1⊕2⊕3=0, ⊕4=4, ⊕5=1.
  - *Armadilha:* a tabela é para 0..n; para a..b use a diferença; F não é definida para n negativo.
- **Paridade de C(n, k)** → ímpar ⇔ k & (n−k) = 0 ⇔ k ⊆ n (Lucas mod 2). Nº de ímpares na linha n = 2^popcount(n).
  - *Gatilho:* "C(n,k) é par ou ímpar", "quantos ímpares na linha n", padrão de Sierpinski.
  - *Por que funciona:* por Lucas mod 2, C(n,k) ≡ Π C(n_b,k_b); o fator é 0 quando k_b=1 e n_b=0, e 1 caso contrário. Logo o produto é 1 exatamente quando k é submáscara de n.
  - *Como fazer:* teste (k & ~n) == 0 (ou (k & (n−k)) == 0); o nº de ímpares é 2^popcount(n). O(1).
  - *Exemplo:* n=5 (101): k=1 (001) é submáscara → C(5,1)=5 ímpar; k=2 (010) não é → C(5,2)=10 par.
  - *Armadilha:* ~n precisa ser mascarado ao nº de bits; use unsigned em n grande; k&(n−k)=0 é equivalente.

- **Nim com limite / generalizações.** Tirar até k de uma pilha: Grundy = a mod (k+1). Moore's Nim_k (tirar de até k pilhas): escreva em binário e some cada bit mod (k+1); perde quem tem tudo 0.
  - *Gatilho:* "tirar até k de uma pilha", "Moore's Nim", "tirar de até k pilhas", variantes de Nim.
  - *Por que funciona:* numa pilha, o jogo de subtração 1..k tem Grundy a mod (k+1), e pilhas independentes se combinam por xor. Em Moore's Nim_k a jogada atinge até k pilhas, o que não é soma disjunta; o critério de Moore generaliza a paridade para cada coluna binária módulo k+1.
  - *Como fazer:* pilhas independentes: xor dos (a_i mod (k+1)). Moore: para cada bit, some os bits das pilhas e verifique se todos dão 0 mod (k+1); se sim, é posição perdedora para quem vai jogar. O(n·bits).
  - *Exemplo:* k=2, pilha 5 → Grundy 5 mod 3=2; pilhas 5 e 5 → 2⊕2=0, posição P. Em Moore com três pilhas de 1 e k=2, cada coluna soma 3≡0 mod 3 → P.
  - *Armadilha:* Moore's Nim_k não é coberto por Sprague-Grundy simples; use o critério das colunas. O "tudo 0" significa todas as somas de coluna ≡0 mod (k+1), não que as pilhas estão vazias.
- **Ordem de Gray.** g(i) = i⊕(i>>1); de i−1 para i muda o bit ctz(i). Percorre os 2^n subconjuntos alterando 1 elemento por passo → soma/xor/estado atualizado em O(1) por subconjunto (enumeração de subset sums sem recomputar).
  - *Gatilho:* enumerar subconjuntos/máscaras alterando um bit por vez, percorrer o hipercubo, manter um agregado incremental.
  - *Por que funciona:* g(i) e g(i−1) diferem em exatamente um bit, o bit ctz(i) (o bit que vira 1 ao incrementar i); assim cada passo flipa um único elemento do conjunto.
  - *Como fazer:* para i=0..2^n−1, m=g(i); se i>0, flip o bit ctz(i) no estado (some ou remova o elemento correspondente); processe m. Atualize soma/xor em O(1). O(2^n).
  - *Exemplo:* n=2: 00, 01, 11, 10; as transições flipam os bits 0, 1, 0.
  - *Armadilha:* ctz(i) é o bit alterado de i−1 para i; 2^n estoura para n≥63; o índice é 0-based.
- **Lowbit.** x & −x isola o menor bit; x & (x−1) remove-o; itere os bits ligados em O(popcount). Fenwick: na atualização, o pai de i é i + lowbit(i); na consulta de prefixo, o próximo bloco é i − lowbit(i).
  - *Gatilho:* iterar só os bits 1, isolar o menor bit, Fenwick, operações de conjunto.
  - *Por que funciona:* em complemento de dois, −x tem os bits acima do lowbit invertidos; x&−x deixa só o menor 1. x−1 inverte esse bit e zera os de baixo, então x&(x−1) remove exatamente esse 1.
  - *Como fazer:* while(x){ b=x&−x; ...; x^=b; } percorre os uns; no Fenwick, update com i += i&−i e query com i −= i&−i. O(popcount) e O(log n).
  - *Exemplo:* x=12 (1100): lowbit=4; x&(x−1)=8 (1000).
  - *Armadilha:* x=0 dá lowbit 0 e pode causar loop infinito no Fenwick; use unsigned/long long para evitar overflow em −x.
- **Troca de dois bits** i, j: x ⊕= (((x>>i) ⊕ (x>>j)) & 1) · ((1<<i) | (1<<j)).
  - *Gatilho:* trocar dois bits sem branch, permutar bits em máscaras.
  - *Por que funciona:* ((x>>i)⊕(x>>j))&1 é 1 exatamente quando os bits diferem; nesse caso, xorar a máscara com os dois bits troca os valores, e quando são iguais a operação não faz nada.
  - *Como fazer:* use a expressão (o fator 0/1 multiplica a máscara) ou um if; O(1).
  - *Exemplo:* x=5 (101), i=0, j=1: os bits diferem → x⊕3 = 6 (110).
  - *Armadilha:* i=j dá diferença 0 e é inofensivo; use 1LL para bits ≥31; mantenha a multiplicação (ou use if), não troque por soma.
- **Zeta/Möbius sobre subconjuntos (SOS) como ferramenta de contagem.** Pares com a_i & a_j = 0 → para cada a_i, quantos b ⊆ ~a_i (soma sobre submáscaras do cnt). Möbius (inversa) recupera "exatamente" a partir de "⊆".
  - *Gatilho:* "pares com a_i&a_j=0", "quantos b são submáscara de ~a_i", contagens sobre subconjuntos.
  - *Por que funciona:* a zeta de submáscaras transforma cnt[m] em F[m]=Σ_{s⊆m} cnt[s]; então F[~a_i] conta os valores que não compartilham bits com a_i. A Möbius é a inversa e desfaz a acumulação.
  - *Como fazer:* para cada bit b, para cada m com o bit b, F[m]+=F[m^(1<<b)] (bit externo); a Möbius subtrai na mesma ordem. Para pares, some F[full^a_i], subtraia os casos a_i=0 e divida por 2. O(n·2^n).
  - *Exemplo:* cnt em {0,1,2}; para a=1, ~a mascarado é 2 (10); F[2]=cnt[0]+cnt[2]=2, os dois valores com a&b=0.
  - *Armadilha:* ~a deve ser mascarado ao universo de n bits; i=j conta quando a_i=0; a ordem dos laços é essencial.

- **SOS guardando top-2.** "Max/min de a_j sobre j com a_j ⊇ máscara (ou ⊆)" e "dois melhores índices distintos" → SOS com 2 melhores por máscara (ex.: max a_i | (a_j & a_k) com i < j < k, CF 1208F).
  - *Gatilho:* consultas que precisam de dois elementos distintos com uma relação de máscara, "max a_i | (a_j & a_k)", CF 1208F.
  - *Por que funciona:* guardar os dois melhores valores (com índices distintos) por máscara permite responder quando o melhor índice não serve; a transição SOS combina os top-2 dos dois lados e mantém os dois melhores.
  - *Como fazer:* inicialize best[m] com os valores/índices que têm máscara m; para cada bit, mescle best[m] com best[m|bit] mantendo dois índices diferentes (para consultas ⊆, use best[m^bit]); consulta O(1). O(n·2^n).
  - *Exemplo:* CF 1208F: para cada i, procure a melhor máscara complementar a a_i que tenha dois índices >i; a resposta é a_i | máscara.
  - *Armadilha:* os dois índices precisam ser distintos e, no CF 1208F, maiores que i; guardar só o máximo perde a resposta.
- **SOS sobre superconjuntos** para AND ("existe elemento que contém m?"), sobre subconjuntos para OR; a ordem dos laços (bit externo) é essencial.
  - *Gatilho:* "existe a_j que contém m" (AND), "existe a_j contido em m" (OR), contagens por máscara.
  - *Por que funciona:* a zeta de submáscaras acumula tudo que está contido em m; a de superconjuntos acumula tudo que contém m. São transformações duais, e cada uma diagonaliza a operação correspondente.
  - *Como fazer:* OR: F[m]=Σ_{s⊆m} cnt[s] (para cada bit, m que tem o bit recebe F[m^bit]); AND: G[m]=Σ_{s⊇m} cnt[s] (para cada bit, m que não tem o bit recebe G[m|bit]). O(n·2^n).
  - *Exemplo:* saber se existe a_j com (a_j&m)==m é G[m]>0; com (a_j|m)==m é F[m]>0.
  - *Armadilha:* trocar a direção do laço quebra a transformação; não use a mesma tabela para OR e AND sem inverter a relação.
- **Convoluções AND/OR/XOR.** OR: zeta de subconjuntos, produto pontual, Möbius. AND: o mesmo com superconjuntos. XOR: Walsh–Hadamard.
  - *Gatilho:* "convolução de OR/AND/XOR de dois arrays", combinar contagens por máscara, contar pares com a_i op a_j = m.
  - *Por que funciona:* a zeta diagonaliza OR e AND, transformando convolução em produto pontual; a WHT faz o mesmo para XOR. A inversa correspondente volta ao domínio original.
  - *Como fazer:* OR: zeta sub, C=F·G ponto a ponto, Möbius sub. AND: zeta super, produto, Möbius super. XOR: WHT, produto, WHT dividida por 2^n. O(n·2^n).
  - *Exemplo:* contar pares (i,j) com a_i|a_j=m é a convolução OR das frequências dos valores.
  - *Armadilha:* para XOR há divisão por 2^n (ou inv(2)^n mod p); para OR/AND não há divisão. Em mod p, confira a invertibilidade.
- **Walsh–Hadamard (xor convolution).** Transformada em O(n log n) com +/−; a inversa é a mesma transformada dividida por 2^k (mod p: multiplique por inv(2)^k). Em Z ou com mod p o resultado é exato.
  - *Gatilho:* convolução XOR, "soma de produtos com xor = x", contar pares com xor dado.
  - *Por que funciona:* a WHT muda para a base dos caracteres ±1, onde χ_s(a⊕b)=χ_s(a)χ_s(b); por isso a convolução XOR vira multiplicação ponto a ponto.
  - *Como fazer:* para cada bit, para cada par (i, i|bit): u=a[i], v=a[i|bit]; a[i]=u+v; a[i|bit]=u−v. Inversa: mesma transformada e divida por 2^k. O(k·2^k).
  - *Exemplo:* [1,0,0,0] → WHT [1,1,1,1]; ao quadrado ponto a ponto continua [1,1,1,1]; a inversa /4 volta a [1,0,0,0].
  - *Armadilha:* os valores crescem por 2^k antes da inversa (estouro em int); em mod p, p=2 não tem inv(2); não esqueça a divisão final.
- **Potência/caminhos no hipercubo.** Contar sequências de k passos com xor total x (cada passo ∈ conjunto S) → WHT de [S], eleve ponto a ponto a k, WHT inversa. Também "nº de maneiras de escolher k elementos (com ordem) com xor = x".
  - *Gatilho:* "nº de sequências de k passos com xor x", "caminhos de comprimento k no hipercubo", escolhas ordenadas com xor alvo.
  - *Por que funciona:* cada passo é uma convolução XOR com o vetor de frequências de S; k passos são a potência k da convolução, e a WHT transforma convolução em produto, então basta elevar cada coordenada a k.
  - *Como fazer:* F=WHT(freq de S); eleve cada F[i] a k; aplique a WHT inversa (divida por 2^bits). O(bits·2^bits + 2^bits·log k).
  - *Exemplo:* S={0,1}, k=2: sequências com xor 0 são (0,0),(1,1) → 2; com xor 1 são (0,1),(1,0) → 2.
  - *Armadilha:* a potência conta sequências com repetição e ordem; para escolher elementos distintos a fórmula não vale. Reduza mod p para não estourar.

- **DP de submáscaras 3^n.** dp[m] = f(dp[s], dp[m\s]): fixe lowbit(m) ∈ s para não contar partições ordenadas duas vezes (÷2 em constante). Para partições em blocos válidos, use bloco que contém o menor elemento.
  - *Gatilho:* "particionar o conjunto", "dp sobre submáscaras", n≤18–20.
  - *Por que funciona:* enumerar todas as submáscaras de todas as máscaras custa Σ 2^popcount(m)=3^n. Fixar o bit menos significativo em um dos lados elimina a simetria entre s e m\s.
  - *Como fazer:* para m, b=lowbit(m); para s submáscara de m com b∈s, dp[m]=op(dp[s], dp[m\s]); o outro lado nunca contém b, então cada partição não ordenada aparece uma vez. O(3^n).
  - *Exemplo:* m={1,2} (11_2), b=01_2: s∈{01_2,11_2}; s=01_2 dá {1}|{2}; s=11_2 dá {1,2}|∅; a opção s={2} é pulada e não duplica.
  - *Armadilha:* sem fixar o lowbit você conta cada partição duas vezes; 3^n não cabe para n=25; para partições ordenadas não fixe o bit.
- **Meet in the middle em xor.** n ≤ 40 e "subconjunto com xor = X / maior xor ≤ K": 2^20 de cada metade em dois arrays ordenados/hash; para "max xor" combine com trie. Se n grande e valores pequenos, prefira base linear.
  - *Gatilho:* n≤40, "subconjunto com xor = X", "maior xor ≤ K", 2^n inviável mas 2^{n/2} viável.
  - *Por que funciona:* divida em duas metades de ≤20; os xors de subconjuntos de cada metade são 2^{20}; combinar uma lista com a outra resolve a restrição sem enumerar 2^n.
  - *Como fazer:* gere os 2^{n/2} xors de cada lado; para xor=X, ordene/hash um lado e procure X⊕v no outro; para max xor ≤K ou max global, use trie. O(2^{n/2}·n).
  - *Exemplo:* n=40, X=0: conte pares de xors iguais entre as metades (inclui o subconjunto vazio dos dois lados).
  - *Armadilha:* 2^20 listas cabem, 2^40 não; se os valores são pequenos e n grande, a base linear é melhor; cuidado com o subconjunto vazio.
- **Distância de Hamming pequena (≤ d, com ≤ 20 bits):** BFS multi-fonte no hipercubo (vizinho = flip de 1 bit) dá, para cada máscara, a distância ao conjunto mais próximo em O(2^k·k).
  - *Gatilho:* "distância de Hamming ≤ d", "máscara mais próxima", k≤20 bits.
  - *Por que funciona:* o hipercubo tem 2^k vértices e arestas de flip de 1 bit; uma BFS multi-fonte a partir de todo o conjunto dado calcula a distância mínima a qualquer elemento.
  - *Como fazer:* fila com todos os elementos (dist 0); para cada u e cada bit b, v=u^(1<<b); se não visitado, dist[v]=dist[u]+1 e enfileire. O(2^k·k).
  - *Exemplo:* conjunto {000,111}, k=3: a máscara 001 está a distância 1 (de 000).
  - *Armadilha:* 2^k de memória/tempo (k=25 já é 33M); para d pequeno, enumerar a bola de raio d pode ser melhor que a BFS completa.
- **Soma de distâncias de Hamming** entre n strings binárias → por coluna $c_0 \cdot c_1$. Mediana/centro de Hamming: bit a bit pela maioria.
  - *Gatilho:* "soma das distâncias de Hamming entre todos os pares", "mediana de Hamming", escolher uma string que minimiza a soma.
  - *Por que funciona:* a distância é a soma das diferenças por coluna; em cada coluna, cada par com bits diferentes contribui 1, e há $c_0 \cdot c_1$ desses pares. Para minimizar a soma a um ponto, cada bit é independente e a maioria minimiza.
  - *Como fazer:* para cada coluna conte c_1 e c_0; some $c_1 \cdot c_0$; para a mediana escolha o bit majoritário (empate arbitrário). O(n·k).
  - *Exemplo:* 00, 01, 11: coluna 0 tem $c_0=2, c_1=1 \rightarrow 2$; coluna 1 tem $c_0=1, c_1=2 \rightarrow 2$; total 4.
  - *Armadilha:* pares ordenados dobram a soma; para mediana ponderada a maioria simples não vale.
- **Xor de caminho em árvore.** px[v] = xor da raiz até v; xor(u, v) = px[u]⊕px[v] para valores em arestas; para valores em vértices, inclua a[lca].
  - *Gatilho:* "xor no caminho u–v", "caminho com xor k", consultas de caminho em árvore.
  - *Por que funciona:* px[u]⊕px[v] cancela o trecho raiz–lca, que aparece nos dois. Com valores nas arestas, o lca não entra; com valores nos vértices, ele é cancelado e precisa ser incluído de volta.
  - *Como fazer:* DFS/BFS calculando px; resposta px[u]⊕px[v] (arestas) ou px[u]⊕px[v]⊕val[lca] (vértices); LCA por binlifting O(log n) ou Euler+RMQ.
  - *Exemplo:* arestas 1-2 (5) e 2-3 (3): px[2]=5, px[3]=6; xor(2,3)=6⊕5=3.
  - *Armadilha:* escolher a raiz e esquecer de incluir seu valor; para vértices, esquecer o lca; para arestas, lembre que px[v] já inclui a aresta que entra em v (não some o pai de novo).

- **Contar caminhos com xor = k em árvore** → pares de px com px[u]⊕px[v] = k (contagem por mapa); com valores em vértices, o a[lca] extra exige small-to-large ou centroide.
  - *Gatilho:* "quantos caminhos têm xor k", "pares de nós com xor de caminho k".
  - *Por que funciona:* com valores em arestas, xor(u,v)=px[u]⊕px[v]; contar pares com xor k é contar, para cada v, quantos px[u] anteriores são px[v]⊕k. Com valores em vértices, o xor é px[u]⊕px[v]⊕a[lca], que depende do lca.
  - *Como fazer:* arestas: DFS com mapa de frequências; para cada v conte freq[px[v]⊕k] e depois insira px[v]; O(n) esperado. Vértices: small-to-large nos filhos (ajustando o valor do lca) ou centroide, O(n $\log^2$ n) ou O(n log n).
  - *Exemplo:* caminho 1-2-3 com arestas 5,3 e k=3: px=[0,5,6]; o par (2,3) dá 5⊕6=3, um caminho.
  - *Armadilha:* não conte u=v se o caminho vazio não vale; para vértices, o ajuste do lca no small-to-large é o erro clássico.
- **Ciclo base em grafo (xor de ciclos).** Faça DFS, px[v] pela árvore; cada aresta fora da árvore (u, v, w) gera ciclo c = px[u]⊕px[v]⊕w; insira todos na base linear. Xor de qualquer passeio u→v = px[u]⊕px[v]⊕(elemento do span); mínimo/máximo = reduza pela base. Dimensão do espaço de ciclos = m − n + (componentes).
  - *Gatilho:* "xor de ciclos", "passeio com xor mínimo/máximo", "ciclo fundamental", grafos com pesos xor.
  - *Por que funciona:* numa árvore DFS, todo ciclo é combinação dos ciclos fundamentais (uma aresta não-árvore mais o caminho na árvore); o espaço de ciclos tem dimensão m−n+c. Qualquer passeio difere de um caminho simples por uma combinação de ciclos.
  - *Como fazer:* DFS, px[v]; para cada aresta não-árvore (u,v,w), c=px[u]⊕px[v]⊕w, insira na base; para query, reduza px[u]⊕px[v] pela base para obter max/min. O((n+m)·B).
  - *Exemplo:* triângulo com pesos 1 (1-2), 2 (2-3) e 4 (1-3): ciclo xor=1⊕2⊕4=7; base {7}; o caminho 1→2 tem xor 1 e o passeio 1→3→2 tem 4⊕2=6=1⊕7.
  - *Armadilha:* inclua cada aresta não-árvore uma vez (paralelas e self-loops contam); componentes separados não têm caminho entre si; use 64 bits.
- **Aresta a aresta em GF(2).** "Escolher arestas com paridade de grau dada em cada vértice" = sistema linear em GF(2) (T-join); numa árvore a solução é única, de baixo para cima.
  - *Gatilho:* "escolher arestas com paridade de grau prescrita", "T-join", "cada vértice com grau par/ímpar".
  - *Por que funciona:* cada aresta é uma variável 0/1; a paridade do grau em v é a soma das variáveis das arestas incidentes mod 2, um sistema linear em GF(2). Numa árvore a matriz de incidência tem posto n−1, então, se a soma das paridades for par, a solução é única.
  - *Como fazer:* na árvore, processe de baixo para cima: a aresta para o pai é escolhida para corrigir a paridade do filho; na raiz, verifique consistência. Em grafo geral, Gauss com bitset O(n^3/64) ou T-join. O(n) na árvore.
  - *Exemplo:* caminho 1–2–3 com paridades (1,1,0): a aresta 2–3 não entra, a 1–2 entra; a solução é única.
  - *Armadilha:* a soma das paridades pedidas tem de ser par (handshake); em grafo com ciclos pode haver 2^{m−rank} soluções; "grau par em todos" é o espaço de ciclos.
- **Operação "inverter bits de um conjunto" (lights out).** Cada operação é um vetor em GF(2)^n; resolva por Gauss com bitset O(n^3/64); o nº de soluções é 2^(n−rank) ou 0.
  - *Gatilho:* "inverter bits de um conjunto", "cada operação afeta vizinhos", "nº de soluções", lights out.
  - *Por que funciona:* cada operação é um vetor de GF(2)^n e o efeito das escolhas é a soma (xor) dos vetores; resolver o alvo é resolver um sistema linear. Se consistente, o espaço de soluções tem dimensão n−rank.
  - *Como fazer:* monte a matriz n×n (coluna j = efeito da operação j, linha = posição), faça Gauss com bitset O(n^3/64); se inconsistente, 0 soluções; senão 2^{n−rank}. Para o mínimo de operações, não basta o rank: continue a base e minimize o peso.
  - *Exemplo:* n=2 com duas operações idênticas que flipam os dois bits: rank 1, n−rank=1 → 2 soluções para cada alvo alcançável (nenhuma ou ambas).
  - *Armadilha:* o nº de soluções é 2^{n−rank}, não rank; "mínimo de operações" é outra otimização e pode exigir busca nos graus de liberdade.
- **Bitset shift-and.** Padrão curto/média em texto, ou matching com classes de caracteres: mantenha o bitset de prefixos do padrão casados, D = ((D<<1)|1) & M[c]; O(n·m/w). Com k erros, k+1 bitsets.
  - *Gatilho:* busca de padrão em texto, "ocorrências com k erros", alfabeto/classes de caracteres, m até algumas centenas.
  - *Por que funciona:* D marca os prefixos do padrão que casam com o sufixo atual do texto; deslocar e fazer AND com a máscara do caractere avança o casamento em paralelo para todos os prefixos.
  - *Como fazer:* pré-calcule M[c] com bit j=1 se pattern[j]==c (ou pertence à classe); D=((D<<1)|1)&M[c]; houve ocorrência se o bit m−1 de D está ligado. O(n·m/w). Com k erros, mantenha k+1 bitsets e combine inserção, deleção e substituição.
  - *Exemplo:* padrão "aba" no texto "ababa": ocorrências nas posições 0 e 2.
  - *Armadilha:* a ordem dos bits (bit 0 = primeiro caractere) e o |1 são essenciais; para k erros, a recorrência é a de Myers, não uma simples união de bitsets.

- **Bitset: BFS/DFS em grafo denso** → unvisited &= ~adj[v] e extraia com _Find_first/_Find_next: cada vértice sai uma vez; $O(n^2/w)$ total.
  - *Gatilho:* grafo denso, BFS/DFS visitando vizinhos, n até ~5000 e $m \sim n^2$.
  - *Por que funciona:* a lista de não visitados é um bitset; expandir v remove todos os vizinhos ainda não visitados de uma vez com uma operação de palavra, em vez de iterar as arestas uma a uma. Cada vértice sai do conjunto uma única vez.
  - *Como fazer:* fila; para cada v, x=unvisited&adj[v]; extraia cada bit de x com _Find_first/_Find_next, marque, enfileire e faça unvisited&=~x. $O(n^2/w + n)$.
  - *Exemplo:* grafo completo: a partir da fonte, todos os outros são descobertos numa única operação de bitset.
  - *Armadilha:* `~adj[v]` tem bits além de n, mascare; _Find_first só em bitset não vazio; para grafo esparso, lista de adjacência é melhor.
- **Fecho transitivo/Floyd booleano com bitset** → para k, para i: if reach[i][k] reach[i] |= reach[k]; $O(n^3/w)$. Produto de matrizes booleanas e "vizinhos comuns" = popcount(adj[u] & adj[v]).
  - *Gatilho:* alcançabilidade em grafo denso, "vizinhos comuns", n≤2000–5000.
  - *Por que funciona:* se i alcança k, então i alcança tudo que k alcança; com linhas em bitset, o OR de uma linha inteira custa n/w palavras. A ordem k externo garante que reach[k] já está fechado.
  - *Como fazer:* para k, para i, if reach[i][k] reach[i]|=reach[k]; $O(n^3/w)$. Para vizinhos comuns, popcount(adj[u]&adj[v]); para produto booleano, a mesma ideia.
  - *Exemplo:* n=3, arestas 1→2 e 2→3: em k=2, reach[1]|=reach[2] faz 1 alcançar 3.
  - *Armadilha:* a ordem k externo é obrigatória; bitset de n bits, não confunda alcançabilidade com distância; com pesos, use Floyd min.
- **Bitset knapsack e variantes.** dp |= dp << w; reconstrução guardando por item ou usando divide e conquista. Para "soma alcançável com k itens" use dp[k] bitset (k·S/w).
  - *Gatilho:* "soma alcançável", n·S grande mas S/w viável, subset sum booleano.
  - *Por que funciona:* dp é um bitset de somas; incluir um item de peso w desloca todas as somas alcançáveis em paralelo, e o OR preserva as antigas. Cada item é usado no máximo uma vez porque o deslocamento é calculado a partir do dp antigo.
  - *Como fazer:* dp[0]=1; para cada w, dp |= dp<<w; reconstrução com snapshots por item ou divide e conquista; para exatamente k itens, dp[k] |= dp[k−1]<<w em k decrescente. O(n·S/w) no caso booleano; O(n·k·S/w) tempo e O(k·S/w) memória para exatamente k itens.
  - *Exemplo:* pesos 2,3: após 2, dp={0,2}; após 3, dp={0,2,3,5}.
  - *Armadilha:* descarte shifts além de S; bitset só dá alcançabilidade, não o número de modos; múltiplas cópias exigem desdobrar ou splitting binário.
- **LCS/LIS com bitset:** ver Strings (LCS em O(nm/w)).
  - *Gatilho:* LCS de strings longas com n·m grande, mas n·m/w viável.
  - *Por que funciona:* a DP de LCS pode ser codificada por diferenças de bits e atualizada com deslocamentos e operações de bitset (algoritmo de Hyyrö), reduzindo o fator de w; a LIS tem algoritmo próprio O(n log n).
  - *Como fazer:* a fórmula e a implementação estão em Strings; para LIS use paciência (lower_bound), não bitset. LCS O(nm/w), LIS O(n log n).
  - *Exemplo:* LCS("abc","ac") = 2 ("ac").
  - *Armadilha:* o bitset de LCS não reconstrói a subsequência trivialmente; LIS não precisa de bitset e a versão por LCS ordenado é mais lenta.

Repetidas de outras seções:

- **Nim e Sprague-Grundy** → xor dos Grundy (teoria dos jogos).
- **Anagrama de palíndromo** → máscara de paridades de 26 bits; prefixos com máscara igual ou a 1 bit (strings).
- **Hash de conjunto por xor** de valores aleatórios (aleatorização).
- **C(n, k) ímpar** ⇔ k é submáscara de n (Lucas p=2); # ímpares na linha n = 2^{popcount(n)}.
- **Convolução XOR/OR/AND** → FWHT / zeta em subconjuntos; subset convolution $O(2^n n^2)$.

Strings

Primeiro teste: hashing resolve? Se sim, é o caminho mais curto até um AC.

Comparação e busca:

- **Comparar substrings** → hash polinomial (dois módulos ou mod $2^{61} - 1$); LCP de dois sufixos → binária + hash.
  - *Gatilho:* muitas perguntas de igualdade ou de ordem entre trechos arbitrários, sem construir estrutura pesada; "são iguais?", "qual o primeiro caractere diferente?".
  - *Por que funciona:* o hash é uma impressão digital do trecho; com base aleatória e módulo grande, dois trechos diferentes colidem com probabilidade desprezível. Para o LCP, o predicado "os primeiros L caracteres coincidem" é monotônico em L, então cabe binária.
  - *Como fazer:* com H[i+1] = (H[i]·B + s[i]) mod p, o hash de [l, r) é H[r] − H[l]·B^(r−l); compare em O(1). Para LCP, binária em L testando hash de [a, a+L) contra [b, b+L): O(log n) por consulta. Use dois módulos ou p = $2^{61} - 1$.
  - *Exemplo:* comparar s[a..b) com s[c..d) em O(1); achar o LCP de dois sufixos com binária + hash.
  - *Armadilha:* base e módulo fixos permitem testes anti-hash; sorteie a base. Cuidado com potências pré-computadas até o tamanho máximo e com o caso L = 0.
- **Menor/maior lexicográfico** → compare via LCP (hash + binária) em O(log n); guloso caractere a caractere do início.
  - *Gatilho:* "ordem lexicográfica", "menor rotação", "menor sufixo", comparar muitos pares de strings/substrings.
  - *Por que funciona:* o LCP dá a primeira posição em que os trechos diferem; a ordem é decidida por esse caractere. Se um é prefixo do outro, o mais curto é o menor. O guloso funciona porque a ordem lexicográfica é local: escolher o menor caractere possível no início nunca pode ser pior.
  - *Como fazer:* ache l = LCP(a, b); se l ≥ min(tamanhos), compare os tamanhos; senão compare s[a+l] com s[b+l]. Para construir a menor string entre candidatas, teste caractere a caractere do início, mantendo só as candidatas compatíveis.
  - *Exemplo:* ordenar substrings de mesmo tamanho; achar a menor rotação de uma string comparando candidatas.
  - *Armadilha:* comparar hashes diretamente sem achar o primeiro ponto de diferença; no empate total, o prefixo mais curto vem primeiro.
- **Padrão em texto** → KMP/Z-function; muitos padrões → Aho-Corasick (+ DP no autômato para "string que não contém nenhum padrão").
  - *Gatilho:* "contém", "conta ocorrências", "busca de padrão", "string que evita um conjunto de padrões".
  - *Por que funciona:* KMP e Z reaproveitam o casamento já feito e não retrocedem no texto; Aho-Corasick generaliza a ideia para vários padrões, com um autômato cujos estados são prefixos da trie e cujos links de falha dizem o maior sufixo que ainda é prefixo de algum padrão.
  - *Como fazer:* KMP/Z em O(n + m) para um padrão. Para vários, insira-os na trie, construa links de falha em BFS e percorra o texto em O(|T| + Σ|p| + ocorrências). Para DP, estado = nó do autômato e marque nós terminais (e seus descendentes de link) como proibidos.
  - *Exemplo:* contar ocorrências de "aba" em "abacaba"; contar strings binárias de tamanho L que não contêm "101".
  - *Armadilha:* em AC, não basta olhar o nó atual: um padrão pode terminar em um ancestral de link; propague "ruim" na BFS ou use dict links.
- **Padrão com coringa '?'** → FFT: Σ $(a - b)^2 \cdot a \cdot b = 0$ por posição.
  - *Gatilho:* matching de padrão e texto em que '?' casa com qualquer caractere.
  - *Por que funciona:* codifique '?' como 0 e as letras como inteiros positivos. No alinhamento, cada par contribui com $a \cdot b \cdot (a-b)^2$; a parcela é 0 exatamente quando os dois são iguais ou quando um deles é '?'. A soma é 0 se todas as posições casam.
  - *Como fazer:* expanda a soma em três correlações: $\Sigma a^3 b - 2\Sigma a^2 b^2 + \Sigma a b^3$. Cada uma vira convolução entre o padrão invertido e o texto; use FFT/NTT. O(n log n) por convolução.
  - *Exemplo:* achar as posições de "ab?d" em um texto.
  - *Armadilha:* '?' precisa ser 0 dos dois lados; use valores aleatórios/módulo grande para evitar cancelamento e dois módulos se houver risco de erro de FFT.
- **Período/borda** → função prefixo: menor período p = n − π[n−1]; é período inteiro se p divide n.
  - *Gatilho:* "período", "borda", "prefixo que também é sufixo", "a string é uma repetição".
  - *Por que funciona:* π[i] é o maior prefixo próprio do prefixo s[0..i] que também é sufixo. Se o prefixo de tamanho n−p é borda, então deslocar p posições casa a string consigo mesma; p é período. Se p divide n, a string é potência de um bloco de tamanho p.
  - *Como fazer:* calcule π em O(n); p = n − π[n−1]. Teste p | n para período inteiro. Todas as bordas saem da cadeia π[n−1], π[π[n−1]−1], ….
  - *Exemplo:* "abcabcabc" tem π[8] = 6, p = 3, e 3 divide 9; logo é $(abc)^3$.
  - *Armadilha:* p pode ser período sem dividir n (período parcial); "período inteiro" exige divisibilidade. Borda de tamanho b corresponde a período n − b.

- **Concatenar com separador** (s + '#' + t) para transformar "busca de s em t" em Z/prefixo de uma string só.
  - *Gatilho:* buscar um padrão em um texto, comparar prefixos, evitar tratar separadamente o começo do texto.
  - *Por que funciona:* um separador que não aparece em s nem em t impede que um casamento atravesse a fronteira; assim, as ocorrências de s em t aparecem como valores altos de Z ou como bordas da string concatenada.
  - *Como fazer:* monte u = s + '#' + t; calcule Z. Para cada i > |s|, se z[i] ≥ |s|, há ocorrência começando em i − |s| − 1. Com prefix function, use π no fim de s#t. O(|s| + |t|).
  - *Exemplo:* achar "aba" em "abacaba" com "aba#abacaba".
  - *Armadilha:* o separador precisa ser um símbolo fora do alfabeto; se ele aparecer nas strings, cria casamentos falsos.

Estruturas:

- **Substrings distintas, k-ésima substring, ocorrências** → suffix array + LCP ou suffix automaton.
  - *Gatilho:* "quantas substrings distintas", "k-ésima substring", "quantas ocorrências deste padrão".
  - *Por que funciona:* no suffix array, sufixos com prefixo comum ficam adjacentes; o LCP mede o que já foi contado. No SAM, cada estado representa uma classe de substrings com o mesmo conjunto de posições finais, e o link tree organiza as ocorrências.
  - *Como fazer:* SA + LCP: distintas = n(n+1)/2 − Σ lcp; k-ésima desce pelos sufixos somando n − sa[i] − lcp[i] novas. SAM: distintas = Σ(len[v] − len[link[v]]); ocorrências = soma de cnt na link tree.
  - *Exemplo:* "ababa": o SA agrupa "a", "aba", "ababa", "ba", "baba".
  - *Armadilha:* convenção lcp[0] = 0; não confunda k-ésima com multiplicidade com k-ésima distinta; separadores mudam a contagem.
- **LCP de dois sufixos quaisquer** → SA + sparse table no array LCP.
  - *Gatilho:* muitas consultas de LCP depois de construir o suffix array.
  - *Por que funciona:* o LCP de dois sufixos é o mínimo dos LCPs entre ranks consecutivos no intervalo entre eles; isso é uma consulta de RMQ.
  - *Como fazer:* construa SA e o array lcp (Kasai), em que lcp[i] = LCP(sa[i−1], sa[i]). Sparse table ou segtree sobre lcp; consulta O(1) ou O(log n).
  - *Exemplo:* LCP(sa[2], sa[5]) = min(lcp[3], lcp[4], lcp[5]).
  - *Armadilha:* off-by-one entre ranks e índices; mantenha rank[pos] para achar a posição de um sufixo no SA.
- **Maior substring comum de k strings** → SA da concatenação + janela deslizante sobre o LCP; ou SAM.
  - *Gatilho:* "maior substring comum a todas", "comum a pelo menos k strings".
  - *Por que funciona:* uma substring comum a todas é prefixo comum de sufixos escolhidos, um de cada string. No SA da concatenação, esses sufixos aparecem juntos em uma janela que contém todas as cores; o mínimo LCP da janela é o tamanho do prefixo comum.
  - *Como fazer:* concatene com separadores distintos; faça SA + LCP; janela deslizante mantendo a contagem de cores e o mínimo LCP (deque/multiset). Maximize o mínimo. Alternativa: SAM de uma string e processe as outras.
  - *Exemplo:* três strings "abac", "baba", "caba": a maior comum é "aba".
  - *Armadilha:* os separadores devem ser únicos e não podem entrar no LCP; a janela precisa de pelo menos um sufixo de cada string.
- **Prefixos de várias strings** → trie; contar pares com LCP ≥ k → profundidade na trie.
  - *Gatilho:* "prefixo comum", "LCP ≥ k", muitas strings e contagem de pares.
  - *Por que funciona:* a trie compartilha prefixos; o LCP de duas strings é a profundidade do LCA das folhas. Dois prefixos de tamanho k são iguais exatamente quando as strings caem no mesmo nó de profundidade k.
  - *Como fazer:* insira todas as strings; para cada nó de profundidade k, conte quantas strings passam por ele e some C(cnt, 2). O(total de caracteres).
  - *Exemplo:* contar pares de strings com prefixo comum de tamanho ≥ 2.
  - *Armadilha:* conte no nó de profundidade exatamente k para não duplicar; strings iguais e nós terminais precisam ser tratados.

- **Palíndromos** → Manacher; todos distintos/contar → eertree; "menor número de palíndromos" → DP + eertree (séries em log).
  - *Gatilho:* "palíndromo", "conta palíndromos", "palíndromos distintos", "partição mínima em palíndromos".
  - *Por que funciona:* Manacher calcula o maior raio de cada centro em O(n), e todos os palíndromos daquele centro são os raios menores. O eertree armazena cada palíndromo distinto uma vez e seus suffix links; as séries de links agrupam sufixos palindrômicos em progressões aritméticas.
  - *Como fazer:* Manacher para contagem/cobertura; eertree para distintos e ocorrências; DP de partição com série link em O(n log n).
  - *Exemplo:* contar substrings palindrômicas de "ababa"; listar palíndromos distintos; menor partição de "abacaba".
  - *Armadilha:* separe centros ímpares e pares; no eertree, as raízes têm comprimentos −1 e 0; no DP com série link, o tratamento de diff e link de série é fácil de errar.

Contagem e limites escondidos:

- **N strings com Σ|s| ≤ S** → no máximo √(2S) tamanhos distintos. Agrupe por tamanho; para cada tamanho faça um hash set. (Também vale em DP, arrays e queries: mesmo truque.)
  - *Gatilho:* muitas strings, mas a soma dos comprimentos é limitada; "agrupe por tamanho", "quantos tamanhos diferentes existem?".
  - *Por que funciona:* se há d tamanhos distintos positivos, a soma mínima possível é 1 + 2 + … + d = d(d+1)/2 ≤ S; logo d ≤ √(2S).
  - *Como fazer:* agrupe por comprimento; para cada comprimento presente, use um hash set ou ordenação. Consultas percorrem só os comprimentos existentes.
  - *Exemplo:* contar strings iguais em um conjunto com $\Sigma|s| \le 10^5$; DP de dicionário testando só tamanhos de palavras.
  - *Armadilha:* o limite é no número de comprimentos distintos, não no número de strings; strings vazias e colisões de hash precisam de cuidado.
- **Σ|s| ≤ S e uma string "curta"/"longa"** → curtas (≤ √S) por brute force, longas são ≤ √S strings.
  - *Gatilho:* limite total S e uma escolha de threshold; "muitas strings curtas" versus "poucas strings longas".
  - *Por que funciona:* com B = √S, strings de comprimento > B são no máximo S/B = √S; as curtas têm comprimento ≤ B e podem ser processadas quadraticamente ou por força bruta.
  - *Como fazer:* escolha B = √S; processe curtas por força bruta e longas separadamente (hash de substrings ou comparação direta). A complexidade total fica O(S√S) ou melhor.
  - *Exemplo:* contar pares de strings com uma substring comum de tamanho limite; processar consultas de dicionário.
  - *Armadilha:* ajuste o threshold ao custo real; não conte o mesmo par nas duas categorias.
- **Uma string de tamanho n** → só n substrings distintas por tamanho; O(n) palíndromos distintos; O(n) bordas.
  - *Gatilho:* "para cada tamanho", "quantos palíndromos distintos", "quantas bordas".
  - *Por que funciona:* para um comprimento L fixo há n − L + 1 substrings, no máximo n; palíndromos distintos são no máximo n (um novo por posição final); as bordas formam uma cadeia via π, também no máximo n.
  - *Como fazer:* itere por comprimento quando o número de comprimentos for pequeno; use eertree para palíndromos distintos; use a cadeia de π para bordas.
  - *Exemplo:* n = 5 tem no máximo 5 substrings de cada comprimento; "aaaaa" tem 5 palíndromos distintos.
  - *Armadilha:* o total de substrings distintas é $O(n^2)$, não O(n); o limite por tamanho não se soma ingenuamente.
- **Aho-Corasick com padrões de Σ ≤ S** → cada posição do texto casa com ≤ √(2S) tamanhos distintos de padrão.
  - *Gatilho:* muitos padrões com soma de comprimentos limitada; contar quais ocorrem em cada posição.
  - *Por que funciona:* os comprimentos de padrões que terminam em uma posição são distintos; como há no máximo √(2S) comprimentos distintos no conjunto, há no máximo um padrão de cada comprimento terminando ali.
  - *Como fazer:* construa o AC; em cada posição, siga os dict links e enumere os padrões terminais. Total O(|T|√S + ocorrências), ou use a link tree para contagens agregadas.
  - *Exemplo:* padrões de comprimentos 1, 2, 3, … com soma limitada; contar ocorrências por posição.
  - *Armadilha:* vários padrões podem ter o mesmo comprimento; para contagem de ocorrências, use a link tree em vez de enumerar todos.

- **Alfabeto pequeno (26)** → 26 no estado, frequências, máscara de paridades (anagrama de palíndromo = ≤ 1 bit ligado).
  - *Gatilho:* letras minúsculas, anagramas, palíndromo por permutação, DP com contagens.
  - *Por que funciona:* o alfabeto constante permite guardar frequências ou uma máscara de paridade em tempo O(1) por caractere; a paridade das contagens determina se uma permutação é palíndromo.
  - *Como fazer:* use vetor de 26 contagens ou máscara de 26 bits; para palíndromo por permutação, conte bits ligados e aceite ≤ 1. Em DP, o estado pode ser a máscara.
  - *Exemplo:* verificar se uma substring pode ser reorganizada em palíndromo.
  - *Armadilha:* o alfabeto pode ser maior que 26; não assuma minúsculas sem ler a entrada; máscaras de 26 bits cabem em int.
- **Anagramas** → ordene a string ou compare vetores de frequência; hash = Σ rand[c].
  - *Gatilho:* "anagrama", "mesmas letras", "permutação", "grupos de anagramas".
  - *Por que funciona:* duas strings são anagramas exatamente quando têm o mesmo vetor de frequências. Ordenar produz a mesma forma canônica; um hash comutativo também, com colisão aleatória.
  - *Como fazer:* ordenação O(n log n); vetor de contagens O(n + σ); hash Σ rand[c] (com multiplicidade) para comparar em O(1) e atualizar em janela deslizante.
  - *Exemplo:* agrupar ["eat", "tea", "ate"]; achar todos os anagramas de p em s.
  - *Armadilha:* hash de soma é comutativo, mas tem colisões; para igualdade exata, use contagens; ordenar perde as posições originais.
- **Contar subsequências** → DP com next[i][c]; subsequências distintas → dp[i] = 2·dp[i−1] − dp[última ocorrência − 1].
  - *Gatilho:* "subsequência", "quantas subsequências distintas", "conta formas".
  - *Por que funciona:* toda subsequência do prefixo i ou não usa s[i] (dp[i−1]) ou usa (outras dp[i−1]); as que usam s[i] e já tinham sido contadas terminando na ocorrência anterior desse caractere são descontadas.
  - *Como fazer:* dp[0] = 1 (vazia); dp[i] = 2·dp[i−1] − dp[last[c]−1], com last[c] a última posição de s[i] antes de i; atualize last. O(n) com alfabeto constante.
  - *Exemplo:* "aaa" tem 4 subsequências distintas: "", "a", "aa", "aaa".
  - *Armadilha:* incluir ou não a vazia; índice da última ocorrência; módulo e valores negativos.

Transformações:

- **Rotação/cíclico** → duplique a string (s + s); mínima rotação → Booth ou SA de s + s.
  - *Gatilho:* "rotação", "deslocamento cíclico", "menor rotação", "string circular".
  - *Por que funciona:* toda rotação de s é uma substring de comprimento n de s + s; comparar rotações reduz-se a comparar essas substrings. Booth acha a mínima em O(n) sem construir SA.
  - *Como fazer:* duplique; para a mínima, use Booth O(n) ou SA de s + s e escolha o sufixo de posição < n com menor rank. Para comparar rotações, use LCP.
  - *Exemplo:* rotações de "baca": "baca", "acab", "caba", "abac"; a menor é "abac".
  - *Armadilha:* só as substrings de comprimento n são rotações; cuidado com empates e com o dobro de memória.
- **Strings binárias e operações** → run-length (blocos iguais), número de trocas 01/10, balanço prefixo (+1/−1).
  - *Gatilho:* string de 0/1, flips, blocos, alternância, balanceamento.
  - *Por que funciona:* run-length comprime blocos; transições 01/10 contam mudanças; o balanço prefixo transforma 0/1 em +1/−1 e revela desequilíbrios.
  - *Como fazer:* RLE para blocos; conte transições; prefixo +1/−1 para balanço. Com updates, segtree guarda soma, prefixo mínimo e transições.
  - *Exemplo:* mínimo de flips para tornar a string alternada; maior bloco após k trocas.
  - *Armadilha:* primeiro e último bloco; comprimento par/ímpar muda a alternância possível.
- **Parênteses** → prefixo ≥ 0 e total 0; mínimo de prefixo diz quantos consertar; Catalan para contar.
  - *Gatilho:* "parênteses", "balanceado", "sequência válida", "mínimo de inserções".
  - *Por que funciona:* uma sequência é válida se nenhum prefixo fica negativo e o total é zero. O mínimo prefixo mede o déficit máximo; inserções ou remoções podem corrigi-lo.
  - *Como fazer:* contador +1/−1; guarde o mínimo. Inserções mínimas = (−min) aberturas + (total − min) fechamentos. Para contar sequências válidas, Catalan.
  - *Exemplo:* "())(" tem min −1 e total 0; precisa de 2 inserções.
  - *Armadilha:* com vários tipos de parênteses, o mínimo prefixo não basta; use pilha.

- **Operação "apague dois iguais adjacentes"** → pilha; resultado é canônico (invariante).
  - *Gatilho:* redução de string, "remova pares adjacentes iguais", "forma final".
  - *Por que funciona:* a reescrita aa → ε é confluente; a pilha produz a forma normal, que é única e independente da ordem das remoções.
  - *Como fazer:* percorra a string; se o topo da pilha é igual ao caractere, desempilhe; senão empilhe. O(n).
  - *Exemplo:* "abba" → pilha: a, ab, a (remove b), vazio.
  - *Armadilha:* a operação é só para iguais adjacentes; remover um par em qualquer lugar é outro problema.
- **Reverter, ler de trás pra frente** → strings invertidas, palíndromo = s igual ao reverso (hash nos dois sentidos).
  - *Gatilho:* "palíndromo", "inverter", "ler de trás para frente", "comparar com o reverso".
  - *Por que funciona:* um palíndromo é exatamente uma string igual ao seu reverso; com hash da string e do reverso, a comparação de qualquer substring vira O(1).
  - *Como fazer:* construa rev(s) e os dois prefixos de hash; para [l, r), compare o hash direto com o hash do intervalo correspondente em rev. Com updates, segtree/BIT guardando os dois sentidos.
  - *Exemplo:* verificar se s[l..r) é palíndromo em O(1).
  - *Armadilha:* o mapeamento de índices no reverso é n−1−i; trate centros ímpares e pares.
- **s se repete k vezes / s infinita** → não basta 2 ou 3 cópias em geral; para matching de um padrão p use ⌈|p|/|s|⌉+1 cópias e extrapole.
  - *Gatilho:* string periódica infinita, "s repetida", buscar padrão em s^∞.
  - *Por que funciona:* uma ocorrência de p pode começar no fim de uma cópia e atravessar várias outras; o número de cópias precisa cobrir |p| caracteres a partir de qualquer posição.
  - *Como fazer:* use ⌈|p|/|s|⌉ + 1 cópias (ou trabalhe módulo |s| com prefixo/sufixo); rode KMP/Z no texto repetido e mapeie as posições de volta.
  - *Exemplo:* s = "ab", p = "aba": "ababab" contém "aba".
  - *Armadilha:* duas ou três cópias falham para padrões longos; alinhe corretamente o início da ocorrência módulo |s|.
- **Jogo com strings** → ver Teoria dos jogos (quem escolhe a primeira letra geralmente decide).
  - *Gatilho:* jogo em que jogadores escolhem letras, formam strings ou movem em um autômato.
  - *Por que funciona:* muitas vezes o primeiro caractere fixa a classe da string ou a estratégia; o jogo reduz a paridade, escolha de primeira letra ou teoria de jogos imparciais.
  - *Como fazer:* modele o estado (string/autômato/turno) e aplique Sprague-Grundy, minimax ou argumento de estratégia; veja a seção Teoria dos jogos.
  - *Exemplo:* primeiro jogador escolhe a primeira letra e força um prefixo.
  - *Armadilha:* nem todo jogo de strings é trivial; confira se a escolha da primeira letra realmente determina o vencedor.

adicionado em 2026-11-10 por agentes; revisado

- **Hash mod 2^61−1.** Base aleatória em [2, p−2], mulmod via __int128; colisão ≈ n/2^61 por comparação. É o padrão seguro; mod 2^64 (overflow) é quebrável.
  - *Gatilho:* precisa de hash confiável contra testes adversariais; strings longas e muitas comparações.
  - *Por que funciona:* p = $2^{61} - 1$ é primo grande; com base sorteada, a probabilidade de colisão entre duas strings de comprimento n é da ordem de n/p. O produto cabe em __int128.
  - *Como fazer:* escolha B aleatório em [2, p−2]; calcule (a·b) mod p com __int128 ou redução especial de Mersenne; pré-compute potências. Dois módulos se quiser margem extra.
  - *Exemplo:* hash de "abacaba" e comparação de substrings.
  - *Armadilha:* mod 2^{64} por overflow é quebrável por anti-hash; base fixa também; cuidado com B = 0, 1 ou p−1.
- **Anti-hash por Thue–Morse.** Para hash com overflow em 2^64 (qualquer base), a string de Thue–Morse de ordem ≥ 11 e seu complemento colidem. Contra mod conhecido fixo, ataques de aniversário/reticulado; a defesa é base aleatória + mod grande (ou dois mods).
  - *Gatilho:* juiz com testes anti-hash, submissões hackeadas, mod 2^{64}.
  - *Por que funciona:* a sequência de Thue–Morse tem cancelamentos sistemáticos; com comprimento 2^{11} e seu complemento, a diferença de hash é múltipla de 2^{64} para qualquer base. Com módulo fixo conhecido, ataques de reticulado encontram colisões.
  - *Como fazer:* não use hash de overflow; use base aleatória por execução e módulo grande ($2^{61} - 1$) ou dois módulos. Se precisar de 2^{64}, reforce com randomização e verificação.
  - *Exemplo:* teste anti-hash clássico com Thue–Morse.
  - *Armadilha:* dois módulos pequenos também podem ser atacados se conhecidos; sorteie a base.

- **Concatenação e remoção por hash.** H(AB) = H(A) + B^|A|·H(B) (com H = Σ s_i B^i); prefixo/sufixo se subtrai. Permite treap/rope com hash para strings com inserção/remoção no meio.
  - *Gatilho:* string dinâmica, editor com inserção/remoção, comparar versões, rope/treap.
  - *Por que funciona:* o hash polinomial é composável: o hash da concatenação é uma combinação linear dos hashes das partes, deslocados pelas potências da base. Assim, uma treap pode manter o hash da subárvore.
  - *Como fazer:* cada nó guarda tamanho e hash; merge concatena com H(A) + B^|A|·H(B); split preserva. Para substring, subtraia o prefixo multiplicado pela potência adequada.
  - *Exemplo:* inserir/remover no meio e comparar duas versões da string.
  - *Armadilha:* orientação do hash (baixa para alta potência); potências até o tamanho máximo; inverso modular só se a base for invertível.
- **Hash de string e do reverso** → palíndromo de substring em O(1); com updates de caractere, BIT/segtree guardando os dois hashes.
  - *Gatilho:* consultas de palíndromo em substrings, com ou sem updates.
  - *Por que funciona:* comparar o hash direto de [l, r) com o hash do reverso correspondente decide igualdade ao reverso; com estrutura de atualização, os dois hashes são mantidos em O(log n).
  - *Como fazer:* pré-compute hash direto e reverso; consulta O(1). Com updates, segtree/BIT guarda os dois sentidos e recalcula no ponto; para achar o maior palíndromo, binária no raio.
  - *Exemplo:* verificar se s[l..r) é palíndromo após trocar um caractere.
  - *Armadilha:* o intervalo no reverso é [n−r, n−l); erros de off-by-one; colisões de hash.
- **Hash 2D.** H[i][j] com duas bases (uma por eixo); hash de submatriz em O(1); busca de padrão 2D e "submatrizes iguais" por tamanho fixo com set de hashes.
  - *Gatilho:* grade, matriz, busca de padrão 2D, comparar submatrizes.
  - *Por que funciona:* o hash polinomial em duas variáveis é composável nos dois eixos; o hash de uma submatriz sai por inclusão-exclusão, como no caso 1D.
  - *Como fazer:* H[i][j] = valor da submatriz [0,i)×[0,j); combine bases Bx e By; para [r1,r2)×[c1,c2), use inclusão-exclusão com as potências. Busca de padrão: compare o hash do padrão com cada posição.
  - *Exemplo:* achar um padrão 3×3 em uma grade.
  - *Armadilha:* colisões; orientação das bases; memória O(nm); não confunda os eixos.
- **Hash de árvore enraizada (isomorfismo).** id(v) = id do multiconjunto ordenado de ids dos filhos (AHU com mapa), ou hash H(v) = 1 + Σ f(H(filho)) com f pseudoaleatório. Sem raiz: enraíze nos 1–2 centroides e compare ambos.
  - *Gatilho:* isomorfismo de árvores, comparar subárvores, hash de estrutura.
  - *Por que funciona:* a estrutura de uma árvore enraizada é determinada recursivamente pelos multiconjuntos de filhos; ordenar os ids ou usar um hash comutativo captura essa informação. Para árvores sem raiz, o centro (1 ou 2 vértices) é invariante.
  - *Como fazer:* AHU: calcule ids dos filhos, ordene, mapeie o vetor para um id canônico com mapa; ou hash H(v) = 1 + Σ f(H(filho)). Sem raiz, ache os centroides e compare as formas enraizadas.
  - *Exemplo:* comparar duas árvores com 10^5 nós.
  - *Armadilha:* colisões no hash aleatório; a árvore pode ter um ou dois centroides, então compare as duas formas quando houver dois; recursão profunda.
- **Comparação lexicográfica por hash.** LCP por binária em O(log n); ordenar n sufixos/substrings com esse comparador em O(n $\log^2$ n). Com SA + RMQ vira O(1) por comparação.
  - *Gatilho:* ordenar sufixos ou muitas substrings, comparar pares repetidamente.
  - *Por que funciona:* o comparador usa LCP para pular o prefixo igual e decide no primeiro caractere diferente; com hash, o LCP custa O(log n). Com SA + RMQ, o LCP custa O(1).
  - *Como fazer:* implemente lcp(a,b) por binária + hash; compare s[a+l] e s[b+l] ou os tamanhos; ordene. Para O(1), construa SA e sparse table no lcp.
  - *Exemplo:* ordenar todos os sufixos de "banana".
  - *Armadilha:* o comparador precisa ser transitivo; empates de prefixo; colisões de hash.

- **Hash de multiconjunto de letras** (anagrama) = Σ rand[c]; janela deslizante de tamanho |p| em O(1) por passo. Alternativa: manter cnt e um contador de letras "erradas".
  - *Gatilho:* achar anagramas/permutações de um padrão em janelas, comparar multiconjuntos.
  - *Por que funciona:* a soma de valores aleatórios por letra é comutativa e depende só das frequências; ao deslizar a janela, remover e adicionar uma letra atualiza o hash em O(1).
  - *Como fazer:* atribua rand[c] a cada letra; hash da janela = Σ rand[c]; compare com o hash do padrão. Alternativa exata: mantenha cnt[26] e o número de letras com contagem diferente do padrão.
  - *Exemplo:* todas as posições de anagramas de "abc" em "abcbac".
  - *Armadilha:* colisões de soma; mantenha multiplicidades; sorteie os valores por execução.
- **Z-function para contar prefixos.** Ocorrências de s[0..L) em s = #{i : z[i] ≥ L} + 1; com array de diferenças, conta para todos os L em O(n). Soma de z = Σ LCP(s, sufixo).
  - *Gatilho:* contar ocorrências de cada prefixo, somar LCPs com sufixos.
  - *Por que funciona:* z[i] é o LCP entre s e o sufixo i; se z[i] ≥ L, o prefixo de tamanho L ocorre em i. O próprio prefixo ocorre na posição 0, daí o +1.
  - *Como fazer:* calcule z; para cada i ≥ 1, faça add +1 no intervalo [1, z[i]] de um array de diferenças; a soma de prefixos dá as ocorrências de cada L. O(n).
  - *Exemplo:* "aaaa": z = [4,3,2,1]; ocorrências do prefixo de tamanho 1 = 4.
  - *Armadilha:* z[0] costuma ser n (ou 0); não conte i = 0 duas vezes; L ≤ n.
- **Prefix function: contar ocorrências de cada prefixo.** cnt[π[i]]++ para todo i, depois propague de trás pra frente cnt[π[i−1]] += cnt[i]; cnt[k]+1 = nº de ocorrências do prefixo de tamanho k.
  - *Gatilho:* contar ocorrências de todos os prefixos, analisar bordas.
  - *Por que funciona:* cada ocorrência de um prefixo como sufixo de um prefixo maior é registrada por π; propagar os contadores pela cadeia de falhas soma todas as ocorrências.
  - *Como fazer:* π em O(n); cnt[π[i]]++ para i ≥ 1; para i = n−1 até 1, cnt[π[i−1]] += cnt[i]; ocorrências do prefixo k = cnt[k] + 1 (o próprio prefixo).
  - *Exemplo:* "ababa": prefixo "aba" ocorre 2 vezes.
  - *Armadilha:* índices de π e cnt; incluir o prefixo inteiro; módulo se contar em excesso.
- **Autômato KMP.** aut[0][c] = (p[0] == c ? 1 : 0); para i > 0, aut[i][c] = i+1 se p[i] = c, senão aut[π[i−1]][c]; O(n·Σ). Serve para DP "string de tamanho L contendo/evitando p" e (com matriz) para L enorme.
  - *Gatilho:* DP sobre strings que contêm/evitam um padrão, L grande, autômato de estados.
  - *Por que funciona:* o estado é o tamanho do maior prefixo de p que é sufixo do texto lido; a transição é determinística e O(1) por caractere. Com matriz, a contagem de caminhos de tamanho L é exponenciação.
  - *Como fazer:* construa aut[i][c] pela recorrência; marque estados terminais; dp[0][0] = 1; para L enorme, matriz S×S e exponenciação O(S^3 log L).
  - *Exemplo:* contar strings binárias de tamanho L que não contêm "101".
  - *Armadilha:* estados terminais precisam ser absorventes ou marcados; a recorrência usa π[i−1], não π[i]; alfabeto efetivo.
- **KMP e rollback.** A amortização de π quebra se você desfaz caracteres; use o autômato completo, ou guarde a pilha de estados, ou faça DFS numa trie de strings com π por nó.
  - *Gatilho:* operações de adicionar/remover caractere, DFS em trie de strings, rollback.
  - *Por que funciona:* a análise amortizada do KMP supõe que o texto só cresce; ao desfazer, o número de fallbacks pode explodir. O autômato completo ou uma pilha de estados restaura o custo.
  - *Como fazer:* use aut[i][c]; ou guarde em pilha o estado antes de cada caractere e restaure no rollback; em DFS, calcule π por nó a partir do pai.
  - *Exemplo:* DFS em uma trie de strings mantendo o estado KMP.
  - *Armadilha:* apenas decrementar o índice não restaura o estado; não reuse a amortização do KMP linear.

- **Todas as bordas** = cadeia π[n−1], π[π[n−1]−1], ...; todos os períodos = n − borda. Existe borda de tamanho b ⇔ período n − b; s é potência ⇔ p = n − π[n−1] divide n e p < n.
  - *Gatilho:* listar bordas/períodos, decidir se a string é potência.
  - *Por que funciona:* toda borda é obtida encadeando a maior borda com a maior borda dela; cada borda b corresponde ao período n − b. A menor potência é o menor período que divide n.
  - *Como fazer:* calcule π; percorra b = π[n−1], depois π[b−1], até 0; para cada b, período n − b. Para potência, p = n − π[n−1], exija p | n e p < n.
  - *Exemplo:* "abcab" tem borda "ab" (b = 2) e período 3.
  - *Armadilha:* a borda de tamanho n não é própria; p = n significa que não há período próprio; não confunda período parcial com potência.
- **Lema de periodicidade (Fine–Wilf).** Se p e q são períodos e p + q − gcd(p, q) ≤ n, então gcd(p, q) é período. Consequência: as bordas de tamanho ≥ n/2 formam uma PA; todas as bordas se agrupam em O(log n) PAs.
  - *Gatilho:* dois ou mais períodos, bordas grandes, estrutura de repetições.
  - *Por que funciona:* períodos pequenos e próximos se reforçam; o gcd também é período. Bordas grandes correspondem a períodos pequenos, e a condição de Fine–Wilf se aplica, gerando progressões aritméticas.
  - *Como fazer:* aplique o lema para combinar períodos; para listar bordas, agrupe as que têm a mesma diferença em PA. O(log n) grupos.
  - *Exemplo:* períodos 4 e 6 em uma string de comprimento 10: gcd 2 é período.
  - *Armadilha:* a condição é p + q − gcd ≤ n; sem ela, o gcd pode não ser período; períodos parciais também contam.
- **Ocorrências de p em janela de tamanho ≤ 2|p|** formam uma PA (ou são ≤ 2): guarde (início, razão, quantidade) em O(1). Duas ocorrências a distância d < |p| fazem de d um período de p.
  - *Gatilho:* muitas ocorrências sobrepostas, processar ocorrências por janela, construir autômatos compactos.
  - *Por que funciona:* se duas ocorrências estão a distância d < |p|, elas se sobrepõem e d é período de p; as ocorrências se alinham em PA. Em uma janela de tamanho ≤ 2|p|, se não há sobreposição, há no máximo duas.
  - *Como fazer:* agrupe ocorrências em PA (início, razão, quantidade); para uma janela, corte a PA e devolva o intervalo. O(1) por janela.
  - *Exemplo:* p = "aba" em "ababa": ocorrências em 0 e 2, PA de razão 2.
  - *Armadilha:* a PA pode ter quantidade 1; janelas maiores que 2|p| podem conter mais de duas ocorrências sem sobreposição.
- **Runs theorem.** O nº de runs (repetições maximais com expoente ≥ 2) é < n; a soma dos expoentes é ≤ 3n; calcule todas via fatores de Lyndon + LCP/LCE em O(n log n). Número de quadrados distintos numa string é < 2n.
  - *Gatilho:* repetições maximais, runs, quadrados, análise de periodicidade.
  - *Por que funciona:* runs são os blocos periódicos maximais; o teorema limita sua quantidade e a soma dos expoentes. A fatoração de Lyndon + LCE permite encontrá-los.
  - *Como fazer:* calcule a fatoração de Lyndon e use LCP/LCE para estender cada run; ou use um algoritmo de runs O(n). Para quadrados, use runs ou Main–Lorentz.
  - *Exemplo:* "aaaa" tem um run de expoente 4; "ababab" tem um run de expoente 3.
  - *Armadilha:* a implementação é complexa; quadrados distintos não são o mesmo que ocorrências de quadrados.
- **Quadrados (xx) / tandem repeats** → Main–Lorentz O(n log n) por D&C, ou via runs; "existe substring xx" em O(n).
  - *Gatilho:* "substring xx", tandem repeat, repetição dupla, quadrados.
  - *Por que funciona:* Main–Lorentz divide a string e trata quadrados que cruzam o meio usando LCE; runs também representam todos os quadrados.
  - *Como fazer:* D&C com LCE/LCP para achar quadrados que cruzam o ponto médio; ou calcule runs. Para existência, O(n) com runs.
  - *Exemplo:* "abab" contém "ab"+"ab".
  - *Armadilha:* quadrados sobrepostos; distintos versus ocorrências; LCE precisa de estrutura O(1).
- **Comutação.** xy = yx ⇔ x e y são potências da mesma palavra (Lyndon–Schützenberger); s^a = t^b ⇔ s·t = t·s. Para a raiz primitiva de s: p0 = n − π[n−1]; se p0 | n, a raiz é s[0..p0); senão a raiz é a própria s (s é primitiva).
  - *Gatilho:* xy = yx, potências, raiz primitiva, comparar strings periódicas.
  - *Por que funciona:* se duas palavras comutam, ambas são potências de uma palavra comum; a raiz primitiva é o menor bloco que gera a string.
  - *Como fazer:* calcule p0 = n − π[n−1]; se p0 | n, raiz = s[0..p0); senão s é primitiva. Para comparar s^a e t^b, compare as raízes primitivas e os expoentes.
  - *Exemplo:* "abab" e "ab" comutam; raiz "ab".
  - *Armadilha:* p0 pode não dividir n (aí s é primitiva); não use apenas o menor período parcial.

- **Rotações e potências por s+s.** t é rotação de s ⇔ t ocorre em s+s (|t| = |s|). Menor período cíclico (raiz primitiva) = primeira posição i > 0 onde s ocorre em s+s.
  - *Gatilho:* rotações, período cíclico, comparar strings circulares.
  - *Por que funciona:* as rotações são as substrings de comprimento n de s+s; a primeira recorrência de s após a posição 0 dá o menor deslocamento cíclico que fixa a string.
  - *Como fazer:* monte s+s; use KMP/Z para achar ocorrências de s; a primeira posição i > 0 (i ≤ n) é o período cíclico. Para rotação de t, verifique |t| = |s| e ocorrência.
  - *Exemplo:* s = "abab" ocorre em s+s nas posições 0, 2, 4; período cíclico 2.
  - *Armadilha:* só considere i ≤ n; uma ocorrência em posição > n é repetição; não confunda com período parcial.
- **Aho-Corasick: link tree.** Contar quantas vezes cada padrão ocorre no texto: marque cnt[nó] nos nós visitados e some a subárvore na árvore de links (processe em ordem inversa de BFS). O(|T| + Σ|p|).
  - *Gatilho:* contar ocorrências de cada padrão em um texto, muitos padrões.
  - *Por que funciona:* cada visita a um nó representa um prefixo do texto; os padrões que terminam ali são os terminais na cadeia de suffix links, isto é, a subárvore do nó na link tree.
  - *Como fazer:* construa AC; percorra o texto e cnt[nó]++; processe os nós em ordem inversa de BFS, somando cnt[v] ao link[v]; ao final, cnt[nó terminal] é o número de ocorrências.
  - *Exemplo:* padrões "a", "ab" em "abab": "a" ocorre 2, "ab" ocorre 2.
  - *Armadilha:* não marque o nó errado; a ordem reversa de BFS é essencial; padrões duplicados contam separadamente.
- **AC em DP.** Marque nó como ruim se ele ou algum ancestral de link é terminal (propague na BFS). dp[i][nó] conta strings de tamanho i sem padrão; para i enorme → matriz (nós × nós) e exponenciação, $O(S^3 \log L)$.
  - *Gatilho:* contar strings de tamanho L que evitam/contêm padrões; L até 10^{18}.
  - *Por que funciona:* o autômato AC reconhece se um padrão terminou; a DP conta caminhos no grafo de estados. Para L grande, a recorrência é linear e vira exponenciação de matriz.
  - *Como fazer:* propague bad[v] |= bad[link[v]] na BFS; dp[0][raiz] = 1; transições para nós não ruins; matriz S×S e potência. $O(S^3 \log L)$.
  - *Exemplo:* contar strings binárias de tamanho L sem "00" nem "111".
  - *Armadilha:* estados ruins não podem ser usados; a raiz pode ser terminal se houver padrão vazio; dimensão da matriz.
- **AC com ganhos.** val[v] += val[link[v]] na BFS; max de soma de valores de padrões presentes numa string de tamanho L = dp[i][nó] com transições pelo autômato.
  - *Gatilho:* cada padrão tem um valor; maximizar a soma dos valores dos padrões que ocorrem na string.
  - *Por que funciona:* o valor de um estado é a soma dos valores dos padrões que terminam naquele nó ou em ancestrais de link; propagar na BFS deixa val pronto. A DP escolhe as transições que maximizam.
  - *Como fazer:* val[terminal] += peso; na BFS, val[v] += val[link[v]]; dp[i][v] = max(dp[i−1][u] + val[v]) sobre transições u→v.
  - *Exemplo:* padrões com pesos 3 e 5; escolher uma string de tamanho L que os contenha.
  - *Armadilha:* um padrão pode ocorrer várias vezes e contar várias vezes; se o problema conta só presença, use máscara/estado diferente.
- **Dict link (exit link).** Para listar só os padrões que terminam numa posição, pule pelo próximo terminal via link: O(nº de ocorrências) em vez de O(profundidade).
  - *Gatilho:* listar todos os padrões que terminam em cada posição, muitos nós não terminais.
  - *Por que funciona:* o dict link aponta para o terminal mais próximo na cadeia de falhas; assim, pulamos diretamente de um padrão encontrado para o próximo, sem percorrer links não terminais.
  - *Como fazer:* na BFS, dict[v] = (terminal[link[v]] ? link[v] : dict[link[v]]); ao processar uma posição, siga dict a partir do nó atual.
  - *Exemplo:* padrões "a", "ba", "cba" no texto "cba".
  - *Armadilha:* o nó atual também pode ser terminal; padrões duplicados; não confunda dict com link de falha.
- **AC com padrões inseridos dinamicamente** → técnica logarítmica: mantenha autômatos de tamanhos potências de 2 e junte como binário (Bentley–Saxe); consulta soma sobre O(log) autômatos.
  - *Gatilho:* inserções de padrões online, consultas intercaladas, sem rebuild total.
  - *Por que funciona:* como um contador binário, mantenha no máximo um AC de cada tamanho potência de 2; ao inserir, junte os de mesmo tamanho. Cada padrão é reconstruído O(log S) vezes.
  - *Como fazer:* insira o novo padrão em um AC de tamanho 1; enquanto houver dois do mesmo tamanho, junte; consulta percorre todos os ACs, $O(\log S \cdot |T|)$ ou agrega.
  - *Exemplo:* dicionário online com inserções e buscas.
  - *Armadilha:* o custo amortizado é $O(\log^2 S)$ por inserção; a consulta precisa somar resultados sem duplicar.

- **Menor string que contém todos os padrões** (poucos padrões, ≤ 10–15): remova padrões que são substring de outros, calcule overlaps (KMP/Z) e faça DP[máscara][último] (superstring mínima = TSP); alternativa BFS em (nó AC, máscara).
  - *Gatilho:* "menor string que contém todos os padrões", poucos padrões, superstring.
  - *Por que funciona:* padrões que são substring de outros são redundantes; a ordem dos restantes determina a string, e o custo de concatenar dois é o overlap máximo. Isso é um TSP sobre máscaras.
  - *Como fazer:* remova redundantes; calcule overlap[i][j] (maior sufixo de i que é prefixo de j) via KMP/Z; dp[mask][i] = menor comprimento terminando em i; reconstrua. Alternativa: BFS em (nó AC, mask).
  - *Exemplo:* padrões "abc", "bcd", "cde": a menor é "abcde".
  - *Armadilha:* overlaps nos dois sentidos; padrões iguais; a BFS pode explodir se os padrões forem muitos.
- **Ocorrências de padrão i em prefixo s[1..r], por queries.** Ordene por r, percorra s no AC fazendo +1 no tin do nó (BIT sobre Euler da link tree); resposta = soma da subárvore do nó do padrão. Intervalo [l, r]: diferença de dois prefixos (r e l+|p|−2).
  - *Gatilho:* muitas queries "quantas vezes o padrão p ocorre no prefixo r" ou em um intervalo.
  - *Por que funciona:* ao percorrer o texto, cada prefixo marca o nó AC correspondente; a soma na subárvore do nó do padrão conta quantos prefixos têm p como sufixo, isto é, ocorrências terminando até r.
  - *Como fazer:* achata a link tree por Euler (tin/tout); offline, ordene queries por r; adicione +1 no tin do nó atual; resposta = soma(tin..tout). Para [l,r], subtraia as queries em r e l+|p|−2.
  - *Exemplo:* s = "ababa", p = "aba", r = 5: 2 ocorrências.
  - *Armadilha:* o limite inferior é l+|p|−2 (1-indexado); não esqueça de ajustar para 0-index; padrões duplicados.
- **SA: substrings distintas** = n(n+1)/2 − Σ lcp. **k-ésima substring distinta:** o sufixo i do SA contribui n − sa[i] − lcp[i] novas; desça somando.
  - *Gatilho:* contar substrings distintas, achar a k-ésima em ordem lexicográfica.
  - *Por que funciona:* cada sufixo contribui com todos os seus prefixos, menos os que já foram contados pelo sufixo anterior no SA (lcp). Somando, cada substring distinta é contada uma vez.
  - *Como fazer:* SA + LCP; total = n(n+1)/2 − Σ lcp. Para k-ésima, percorra o SA na ordem e subtraia a contribuição de cada sufixo até achar o grupo; dentro do sufixo, pegue o prefixo de tamanho restante.
  - *Exemplo:* "banana": 15 − 6 = 9 substrings distintas.
  - *Armadilha:* lcp[0] = 0; k 1-indexado; com multiplicidade, a fórmula muda.
- **SA: maior substring que ocorre ≥ k vezes** → máximo, sobre janelas de k−1 LCPs consecutivos, do mínimo da janela (deque monotônico). k = 2: max lcp.
  - *Gatilho:* "substring que ocorre pelo menos k vezes", repetição frequente.
  - *Por que funciona:* quaisquer k sufixos consecutivos no SA têm um prefixo comum de tamanho igual ao mínimo dos k−1 LCPs entre eles; o máximo sobre as janelas é a resposta.
  - *Como fazer:* para cada janela de k−1 posições do array lcp, mantenha o mínimo com deque monotônico; pegue o máximo. O(n).
  - *Exemplo:* "banana", k = 2: "ana" ocorre 2 vezes.
  - *Armadilha:* k = 1: a resposta é n; a janela tem k−1 LCPs; não confunda com substrings distintas.
- **SA: menor substring única** (ocorre uma vez) começando em i → max(lcp[i], lcp[i+1]) + 1, se ≤ n − sa[i].
  - *Gatilho:* menor substring que ocorre só uma vez, unicidade.
  - *Por que funciona:* um prefixo do sufixo i é único quando seu comprimento supera o LCP com os dois vizinhos no SA; o menor único tem max(lcp[i], lcp[i+1]) + 1.
  - *Como fazer:* calcule lcp; para cada i, t = max(lcp[i], lcp[i+1]) + 1; se t ≤ n − sa[i], é uma candidata; pegue o mínimo global (ou por posição).
  - *Exemplo:* "ababa": o menor único é "abab" ou "baba".
  - *Armadilha:* lcp[0] e lcp[n] valem 0; o sufixo pode ser curto demais; não confunda com menor substring que ocorre exatamente k vezes.
- **SA: soma de LCP sobre todos os pares de sufixos** = soma de min de todos subarrays do array lcp → pilha monotônica. Contar substrings comuns a duas strings: o mesmo, separando origens (conta(A#B) − conta(A) − conta(B)).
  - *Gatilho:* soma de LCPs, substrings comuns, contar pares.
  - *Por que funciona:* o LCP de um par é o mínimo no intervalo de lcp entre eles; somar todos os pares é somar o mínimo de todos os subarrays. Para duas strings, as substrings comuns são as distintas da concatenação menos as de cada uma.
  - *Como fazer:* pilha monotônica para soma de mínimos de subarrays; para duas strings, use a fórmula com separador e a contagem de distintas.
  - *Exemplo:* "abab" e "baba": conta(A#B) − conta(A) − conta(B).
  - *Armadilha:* separador único; substrings que atravessam a fronteira; lcp inicial.

- **SA: maior substring repetida sem sobreposição** → binária em L: agrupe sufixos consecutivos com lcp ≥ L; vale se max(sa) − min(sa) ≥ L no grupo.
  - *Gatilho:* "repetida sem sobreposição", "duas ocorrências disjuntas".
  - *Por que funciona:* para um comprimento L, sufixos com LCP ≥ L compartilham um prefixo de tamanho L; se as posições inicial e final do grupo distam ≥ L, há duas ocorrências disjuntas.
  - *Como fazer:* binária em L; para cada grupo maximal de lcp ≥ L, guarde min e max de sa; se max − min ≥ L, L é viável. O(n log n).
  - *Exemplo:* "ababab": "aba" ocorre em 0 e 3, sem sobreposição; "abab" ocorre em 0 e 2, sobreposto.
  - *Armadilha:* max − min ≥ L é a condição de não sobreposição; grupos com um sufixo só não valem.
- **SA de múltiplas strings.** Concatene com separadores distintos (ou use um só e limite o LCP pelo tamanho de cada string); substring comum a todas as k → menor janela do SA com todas as cores, valor = min lcp da janela, maximizar.
  - *Gatilho:* substring comum a k strings, "common substring".
  - *Por que funciona:* no SA da concatenação, uma janela que contém sufixos de todas as strings tem um prefixo comum de tamanho igual ao mínimo LCP da janela; a menor janela com todas as cores maximiza esse mínimo.
  - *Como fazer:* concatene com separadores únicos; SA + LCP; janela deslizante com contagem de cores e mínimo LCP (multiset/deque); maximize o mínimo.
  - *Exemplo:* k = 3 strings "abac", "baca", "caba": "aba" é comum.
  - *Armadilha:* separadores distintos; LCP não pode atravessar fronteiras; janela precisa de todas as cores.
- **SA: comparar substrings s[a..b) vs s[c..d)** em O(1): l = LCP(a, c) por RMQ; se l ≥ min(tamanhos) compare tamanhos, senão compare s[a+l] com s[c+l].
  - *Gatilho:* muitas comparações de substrings, ordenação, hashing evitado.
  - *Por que funciona:* o LCP diz quantos caracteres iniciais coincidem; se ele cobre o menor dos dois, o mais curto é prefixo e é menor; senão o primeiro caractere diferente decide.
  - *Como fazer:* SA + sparse table no lcp; l = LCP(a,c); se l ≥ min(b−a, d−c), compare comprimentos; senão compare s[a+l] e s[c+l].
  - *Exemplo:* comparar "ana" e "anaba".
  - *Armadilha:* índices de LCP por rank; substrings sobrepostas; lcp além do fim.
- **SA: ocorrências de p em s[l..r].** Intervalo [lo, hi] do SA por binária; conte sa[i] ∈ [l, r−|p|+1] com wavelet/segtree persistente ou BIT offline.
  - *Gatilho:* consultas de ocorrências de um padrão dentro de um intervalo do texto.
  - *Por que funciona:* todas as ocorrências de p são sufixos em um intervalo contíguo do SA; dentro dele, basta contar as posições iniciais que cabem em [l, r−|p|+1].
  - *Como fazer:* ache [lo, hi] por binária com comparação de substrings; conte sa[i] no intervalo de posições com wavelet tree, segtree persistente ou BIT offline. O(log n) por consulta.
  - *Exemplo:* s = "ababa", p = "aba", [0,5): ocorrências em 0 e 2.
  - *Armadilha:* o limite superior é r−|p|+1; 0-index; o intervalo do SA pode ser vazio.
- **BWT.** BWT[i] = s[sa[i]−1] (com $); invertível por LF-mapping; agrupa letras iguais (compressão, run-length). FM-index: busca de padrão em O(|p|) por backward search com C[] e rank.
  - *Gatilho:* compressão, indexação de texto, busca de padrão com pouca memória.
  - *Por que funciona:* a BWT é uma permutação reversível que tende a agrupar caracteres iguais; o LF-mapping reconstrói a string. O FM-index usa a BWT para buscar padrões de trás para frente.
  - *Como fazer:* construa SA; BWT[i] = s[sa[i]−1] (ou $ se sa[i]=0); LF(i) = C[BWT[i]] + rank(BWT[i], i); para buscar p, faça backward search mantendo [lo,hi).
  - *Exemplo:* "banana" com sentinela.
  - *Armadilha:* o sentinela precisa ser único e menor; rank é sobre a BWT; não confunda com SA.
- **SAM: contagem de ocorrências.** cnt = 1 nos estados não-clone (prefixos), 0 nos clones; soma na subárvore da link tree. Substrings distintas = Σ (len[v] − len[link[v]]); online: ao adicionar um caractere, ans += len[last] − len[link[last]].
  - *Gatilho:* ocorrências de substrings, substrings distintas, construir SAM online.
  - *Por que funciona:* cada estado representa substrings com o mesmo endpos; os não-clones correspondem a prefixos e recebem cnt 1; somar na link tree conta quantos prefixos terminam naquele estado. As substrings distintas são os intervalos de comprimentos de cada estado.
  - *Como fazer:* durante a construção, cnt[cur] = 1, cnt[clone] = 0; ordene por len e some cnt[v] ao link[v]; distintas = Σ(len[v] −len[link[v]]); online, ans += len[last]−len[link[last]].
  - *Exemplo:* "ababa": o estado de "aba" tem cnt 2.
  - *Armadilha:* clones têm cnt 0 inicialmente; a soma precisa ser em ordem decrescente de len; múltiplas strings exigem reiniciar.

- **SAM: maximizar |t|·ocorrências(t)** → max sobre estados de cnt[v]·len[v]. Maior substring com ≥ k ocorrências: max len[v] com cnt[v] ≥ k.
  - *Gatilho:* maximizar comprimento vezes frequência, substring mais frequente com limite.
  - *Por que funciona:* todas as substrings representadas por um estado têm o mesmo conjunto de ocorrências; a mais longa é a melhor para |t|·occ. Para ≥ k, basta olhar estados com cnt ≥ k.
  - *Como fazer:* calcule cnt na link tree; para cada estado, avalie cnt[v]·len[v] e guarde o máximo; para ≥ k, pegue max len[v] com cnt[v] ≥ k.
  - *Exemplo:* "ababa": "aba" tem 2 ocorrências e comprimento 3.
  - *Armadilha:* usar len do estado, não o comprimento de uma substring curta; cnt de clones; k maior que o número de ocorrências.
- **SAM: substring comum de duas strings (ou k).** SAM de A; percorra B mantendo (estado, comprimento) com saltos de link ao falhar; para k strings, para cada string calcule best[v] = maior match no estado e propague para o link com best[link[v]] = max(best[link[v]], min(best[v], len[link[v]])) em ordem decrescente de len; após propagar cada string, atualize o mínimo global por estado (o menor best entre as strings).
  - *Gatilho:* maior substring comum de duas ou k strings.
  - *Por que funciona:* ao percorrer B no SAM de A, mantemos o maior sufixo de B que é substring de A; para k strings, cada estado guarda o maior match comum a todas as strings processadas até agora, propagando para os links.
  - *Como fazer:* construa SAM de A; para cada string B, percorra mantendo (v, l), saltando por links quando não há transição; best[v] = max(best[v], l); propague em ordem decrescente de len com min(best[v], len[link[v]]); atualize o mínimo entre strings.
  - *Exemplo:* A = "ababc", B = "babca": a comum é "babc".
  - *Armadilha:* reiniciar estado/comprimento a cada string; a propagação usa min com len do link; atualizar o global só depois de propagar.
- **SAM: link tree de s = suffix tree de reverse(s).** Logo, suffix tree de s = SAM de rev(s). Tamanho ≤ 2n−1 estados, ≤ 3n−4 transições.
  - *Gatilho:* precisa de suffix tree, ocorrências por nó, LCA de folhas.
  - *Por que funciona:* a árvore de suffix links do SAM de s codifica os sufixos de s e corresponde à suffix tree de rev(s); as folhas representam posições e o LCA dá LCP.
  - *Como fazer:* construa o SAM de rev(s) para obter a suffix tree de s; use a link tree para ocorrências, LCA e caminhadas. Limites: ≤ 2n−1 estados e ≤ 3n−4 transições.
  - *Exemplo:* construir a suffix tree de s via SAM de rev(s).
  - *Armadilha:* nós implícitos e folhas; a correspondência é a menos de compressão de caminhos; o alfabeto afeta as transições.
- **SAM: substring t[l..r] → estado.** Pegue o estado do prefixo t[1..r] e suba pela link tree (binary lifting) até o ancestral de menor profundidade com len ≥ r−l+1; ali estão cnt e endpos.
  - *Gatilho:* consultar ocorrências/endpos de uma substring arbitrária, many queries.
  - *Por que funciona:* o estado do prefixo r representa todos os sufixos que terminam em r; subir pelos links até o estado cujo len ainda cobre o comprimento da substring dá a classe de endpos correta.
  - *Como fazer:* construa SAM e binary lifting na link tree; para [l,r], L = r−l+1; suba de last[r] para o ancestral mais alto com len ≥ L; o estado resultante tem cnt/endpos.
  - *Exemplo:* "ababa", substring [1,3] (1-based) = "aba": estado com cnt 2.
  - *Armadilha:* o estado do prefixo é o last após r caracteres; suba até len ≥ L, não até len = L; índices 1-based.
- **SAM: k-ésima substring lexicográfica** → dp[v] = nº de caminhos a partir de v (distintas) ou Σ cnt (com multiplicidade); desça pelas transições em ordem de letra.
  - *Gatilho:* k-ésima substring em ordem lexicográfica, com ou sem multiplicidade.
  - *Por que funciona:* o SAM é um DAG de substrings; o número de caminhos a partir de um estado conta as substrings distintas que continuam dali; com cnt, conta ocorrências. Descer em ordem de letra respeita a ordem lexicográfica.
  - *Como fazer:* processe os estados em ordem decrescente de len. Para substrings não vazias, use f[v] = Σ_c (1 + f[go[v][c]]); com multiplicidade, o grupo da letra c vale cnt[go] + f[go]. Escolha o grupo que contém k, subtraia os grupos anteriores e emita c; no caso distinto, pare se k = 1, senão k-- e desça; no caso com multiplicidade, pare se k ≤ cnt[go], senão k -= cnt[go] e desça.
  - *Exemplo:* "ababa", k = 3 → "aba".
  - *Armadilha:* dp em ordem topológica; k 1-indexado; a fórmula 1 + Σ inclui a vazia e precisa de ajuste; multiplicidade muda a base.

- **SAM generalizado (várias strings / trie).** Reinicie last = root a cada string e trate o caso em que a transição já existe (evite clone duplicado); ou construa por BFS na trie. Substrings distintas do conjunto = Σ (len − len[link]).
  - *Gatilho:* SAM de um conjunto de strings, trie, dicionário.
  - *Por que funciona:* o SAM generalizado representa todas as substrings do conjunto; construir a partir da trie preserva os prefixos compartilhados e evita duplicatas.
  - *Como fazer:* construa a trie das strings e faça BFS estendendo o SAM; ou reinicie last por string com tratamento de transição existente. Distintas = Σ(len[v]−len[link[v]]).
  - *Exemplo:* conjunto {"ab", "ba"}.
  - *Armadilha:* reiniciar last sem tratar transição existente cria clones/estados duplicados; separadores não são necessários na trie.
- **Menor string que NÃO é substring** (alfabeto k): no SAM dp[v] = 1 + min_c dp[go[v][c]] com dp = 0 se go não existe (o menor é 1 se falta uma letra). A lexicográfica mínima segue do dp.
  - *Gatilho:* menor string ausente, shortest absent substring.
  - *Por que funciona:* se falta uma transição, a string de uma letra já não é substring; senão, qualquer string é formada por uma letra mais uma string ausente a partir do próximo estado.
  - *Como fazer:* no SAM, defina dp[missing] = 0; dp[v] = 1 + min_c dp[go[v][c]]; reconstrua escolhendo a menor letra que atinge o mínimo.
  - *Exemplo:* alfabeto {0,1}, s = "001": as substrings de tamanho 1 são "0" e "1"; as de tamanho 2 são "00" e "01"; logo "10" é a menor string ausente.
  - *Armadilha:* base dp=0 para transição ausente; desempate lexicográfico; alfabeto completo.
- **Menor string que não é subsequência.** dp[i] = 1 + min_c dp[next[i][c]+1] (next = n se não existe, dp[n+1] = 0); reconstrua pegando a letra de menor dp (menor lexicográfica entre as de mesmo valor).
  - *Gatilho:* shortest absent subsequence, menor string que não é subsequência.
  - *Por que funciona:* a partir da posição i, escolher a letra c pula para a próxima ocorrência; se não existe, a string de uma letra já não é subsequência. A DP mede o menor comprimento restante.
  - *Como fazer:* calcule next[i][c] para i = 0..n; dp[n+1] = 0; dp[i] = 1 + min_c dp[next[i][c]+1]; reconstrua escolhendo a menor letra que minimiza.
  - *Exemplo:* s = "abc": "d" não é subsequência; se o alfabeto é {a,b,c}, "aa" não é.
  - *Armadilha:* next = n quando não existe; dp[n+1] = 0; desempate lexicográfico; índices.
- **Suffix tree.** LCA das folhas = LCP; substring de tamanho ≥ L mais frequente, "ocorrências por nó" por small-to-large ou Euler tour; é a estrutura certa quando o problema fala de "árvore de sufixos" ou mistura strings com DP em árvore.
  - *Gatilho:* suffix tree explícita, LCP por LCA, ocorrências por nó, DP em árvore de sufixos.
  - *Por que funciona:* as folhas representam sufixos; o caminho até uma folha é o sufixo; o LCA de duas folhas tem profundidade igual ao LCP. A subárvore de um nó contém as ocorrências da substring correspondente.
  - *Como fazer:* construa a suffix tree (ou via SAM de rev(s)); faça DFS/Euler; LCA para LCP; small-to-large para ocorrências; DP nos nós para padrões.
  - *Exemplo:* maior substring que ocorre pelo menos k vezes.
  - *Armadilha:* nós implícitos; memória; o alfabeto e o sentinela; LCA de folhas, não de nós internos quaisquer.
- **Manacher + diferenças.** Cada centro com raio r contribui, para o nº de palíndromos que cobrem cada posição, com uma função triangular; some tudo com array de segunda diferença e integre duas vezes. Para palíndromos que terminam/começam em cada índice, use range add (1ª diferença): cada centro soma +1 no intervalo correspondente. Dá "nº de palíndromos que cobrem" (2ª diferença) ou "que terminam/começam" (1ª diferença) em O(n).
  - *Gatilho:* contar quantos palíndromos cobrem/terminam/começam em cada posição.
  - *Por que funciona:* um centro com raio r gera palíndromos de raios 1..r; a quantidade que cobre uma posição cresce e decresce linearmente, então sua segunda diferença é constante por trecho. Para términos/começos, cada palíndromo contribui em um único índice, e os índices formam um intervalo.
  - *Como fazer:* Manacher para os raios; para cobertura, some a função triangular com segunda diferença e integre duas vezes; para términos/começos, range add +1 no intervalo e prefixe. O(n).
  - *Exemplo:* "ababa": o palíndromo "aba" cobre as posições 0..2.
  - *Armadilha:* centros ímpares e pares; limites do intervalo; a segunda diferença precisa de duas integrações.

- **Pares de palíndromos adjacentes / s = P_1P_2...** → conte palíndromos que terminam em i (L[i]) e começam em i+1 (R[i+1]) e some L·R; fatoração em k palíndromos com k ≤ 3 tem caracterização direta.
  - *Gatilho:* dividir a string em dois palíndromos, contar pares adjacentes, fatoração em poucos palíndromos.
  - *Por que funciona:* uma divisão em dois palíndromos é determinada pelo ponto de corte i: o primeiro termina em i e o segundo começa em i+1; as escolhas são independentes, então o número de pares é L[i]·R[i+1].
  - *Como fazer:* calcule L[i] (palíndromos que terminam em i) e R[i] (que começam em i) com Manacher/eertree; some L[i]·R[i+1]. Para k ≤ 3, use as caracterizações diretas.
  - *Exemplo:* "abaaba" = "aba"+"aba"; o corte depois do primeiro "aba" conta em L[2]·R[3].
  - *Armadilha:* os dois lados do corte precisam ser não vazios; conte cada par uma vez; k=1 e k=2 se sobrepõem.
- **Eertree: ocorrências.** cnt[v] += cnt[filho em link] em ordem de criação invertida dá nº de ocorrências de cada palíndromo distinto; ≤ n palíndromos distintos. Rollback do último caractere por pilha de `last`.
  - *Gatilho:* ocorrências de palíndromos distintos, palíndromos online com rollback.
  - *Por que funciona:* cada vez que um palíndromo é criado, ele ocorre na posição atual; os suffix links apontam para sufixos palindrômicos, então somar os cnt na ordem inversa de criação conta todas as ocorrências.
  - *Como fazer:* construa o eertree; cnt[v] = 1 ao criar; processe os nós em ordem inversa de criação e some cnt[v] ao link[v]; para rollback, guarde last em pilha.
  - *Exemplo:* "ababa": "aba" ocorre 2 vezes.
  - *Armadilha:* as raízes não contam; a ordem de criação é topológica para os links; rollback precisa restaurar last e, se necessário, remover nós.
- **Fatoração em palíndromos (série link).** diff[v] = len[v] − len[link[v]]; palíndromos-sufixos com mesmo diff formam PA, só O(log n) por posição → dp de partição mínima/contagem em O(n log n). Partição em palíndromos de tamanho par: intercale s_1 s_n s_2 s_{n-1}... e conte partições em palíndromos de tamanho par da nova string (CF 932G).
  - *Gatilho:* partição mínima/contagem em palíndromos, palíndromos pares.
  - *Por que funciona:* os sufixos palindrômicos de uma posição se agrupam por diff em progressões aritméticas; a série link permite pular o grupo e atualizar a DP em O(log n) por posição.
  - *Como fazer:* eertree com série links; dp[i] = min/count sobre grupos; para palíndromos pares, transforme a string intercalando as metades e conte partições em palíndromos de tamanho par.
  - *Exemplo:* "abba" tem a partição "abba" (1 palíndromo); "a"+"bb"+"a" usa 3. Na versão de tamanho par (CF 932G), "abba" conta 1.
  - *Armadilha:* a transformação de CF 932G exige comprimento par; o diff e o link de série; índices da DP.
- **Menor palíndromo adicionando na frente/no fim.** Na frente: n − (maior prefixo palíndromo), e o maior prefixo palíndromo = π de s#rev(s). No fim: simétrico (maior sufixo palíndromo).
  - *Gatilho:* mínimo de caracteres para tornar a string palíndromo adicionando em uma das pontas.
  - *Por que funciona:* se o maior prefixo palíndromo tem tamanho L, o restante é o sufixo que precisa ser espelhado na frente; adiciona-se n−L caracteres. O KMP em s#rev(s) acha esse prefixo.
  - *Como fazer:* calcule π de s + '#' + rev(s); o último valor é o maior prefixo palíndromo. Para o fim, use rev(s) + '#' + s ou o maior sufixo palíndromo.
  - *Exemplo:* "abac": maior prefixo palíndromo "aba" (3); adiciona 1 caractere na frente: "cabac".
  - *Armadilha:* frente usa prefixo, fim usa sufixo; o separador não pode aparecer; não confunda com o maior palíndromo interno.
- **Lyndon / Duval.** Toda string se fatora de modo único em palavras de Lyndon não-crescentes, em O(n) (Duval). Lyndon = estritamente menor que todos os seus sufixos próprios (e que todas as rotações).
  - *Gatilho:* fatoração de Lyndon, menor rotação, palavras primitivas.
  - *Por que funciona:* a fatoração de Lyndon é única quando as palavras são ordenadas de forma não-crescente; o algoritmo de Duval a constrói em O(n) com dois ponteiros.
  - *Como fazer:* Duval: i = 0; j = i+1; k = i; compare s[k] com s[j]; avance ou reinicie; ao final, s[i..j) é um fator. Repita.
  - *Exemplo:* "banana" → "b" "an" "an" "a" (fatores de Lyndon em ordem não-crescente).
  - *Armadilha:* a ordem é não-crescente; Lyndon é estritamente menor que sufixos/rotações; não confunda com fatoração Lyndon crescente.

- **Rotação mínima** → início do último fator de Duval de s+s que começa antes de n; ou Booth; ou SA de s+s. Canonização de colar/rotação: hash da rotação mínima.
  - *Gatilho:* menor rotação, canonicalização de strings circulares, colares.
  - *Por que funciona:* a fatoração de Lyndon de s+s codifica a ordem das rotações; o último fator que começa antes de n marca a rotação mínima. Booth faz o mesmo em O(n).
  - *Como fazer:* Duval em s+s; guarde o início do último fator com início < n; ou Booth; ou SA de s+s. Para canonicalizar, use o hash da rotação mínima.
  - *Exemplo:* "baca" → rotação mínima "abac".
  - *Armadilha:* o último fator precisa começar antes de n; empates; o dobro da string.
- **Nº de palavras de Lyndon de tamanho n em k letras** = (1/n) Σ_{d|n} μ(d)·k^(n/d). Colares: (1/n) Σ φ(d)·k^(n/d) (Burnside).
  - *Gatilho:* contar palavras primitivas, colares, necklaces.
  - *Por que funciona:* a fórmula de Möbius conta as palavras primitivas; Burnside conta as órbitas sob rotação, com φ(d) para rotações de ordem d.
  - *Como fazer:* pré-compute μ e φ; para cada divisor d de n, some μ(d)·k^(n/d) e divida por n; para colares, troque μ por φ.
  - *Exemplo:* n = 3, k = 2: Lyndon = (1/3)(8 − 2) = 2; colares = (1/3)(8 + 2·2) = 4.
  - *Armadilha:* divisores de n; módulo primo e inverso de n; μ e φ; não confunda colar (rotação) com pulseira (rotação + reflexão).
- **Pulseira / "string reversível".** Se s e rev(s) são equivalentes, canon = min(rotmin(s), rotmin(rev s)); com operação reversível, compare formas canônicas dos dois lados.
  - *Gatilho:* bracelets, equivalência por rotação e reflexão, strings reversíveis.
  - *Por que funciona:* a forma canônica de uma pulseira é o menor entre a rotação mínima da string e a rotação mínima do reverso; isso cobre rotações e reflexões.
  - *Como fazer:* calcule rotmin(s) e rotmin(rev(s)); pegue o menor; compare formas canônicas para decidir equivalência.
  - *Exemplo:* "abc" e "cba" são equivalentes como pulseira (reflexão), mas não por rotação; ambos têm forma canônica "abc".
  - *Armadilha:* não inclua reflexão se o problema só permite rotação; a comparação deve ser das formas canônicas completas.
- **Strings de Fibonacci (S_n = S_{n−1}S_{n−2}) e SLPs.** Ocorrências cnt_n = cnt_{n−1} + cnt_{n−2} + cruzamentos; só os |p|−1 caracteres de borda importam, e as bordas estabilizam depois de poucos passos. Vale para qualquer string definida por concatenações recursivas.
  - *Gatilho:* string definida recursivamente, SLP, ocorrências de padrão em string gigante.
  - *Por que funciona:* as ocorrências internas vêm das duas partes; as que cruzam a fronteira dependem só dos |p|−1 caracteres de cada lado. Em Fibonacci, as bordas estabilizam, então os cruzamentos podem ser calculados por recorrência.
  - *Como fazer:* guarde prefixo/sufixo de tamanho |p|−1 e a contagem; combine na concatenação; para SLP, faça DP nos nós. Para Fibonacci, use a recorrência e as bordas estáveis.
  - *Exemplo:* contar ocorrências de "aba" em S_10.
  - *Armadilha:* cruzamentos podem contar múltiplas vezes; o padrão pode ser maior que a string; estabilização exige verificação.
- **Thue–Morse.** t_n = popcount(n) mod 2; livre de sobreposições (sem cubos nem xyxyx). Aparece como anti-hash, em construções "sem repetição" e em partição de {0..2^k−1} em dois conjuntos com mesmas somas de potências até k−1 (Prouhet).
  - *Gatilho:* anti-hash, sequências sem repetição, partições de Prouhet.
  - *Por que funciona:* o bit de paridade de popcount gera uma sequência com cancelamentos e sem cubos/overlaps; a partição por paridade de popcount dá somas iguais de potências.
  - *Como fazer:* gere t_n = popcount(n) & 1; use como teste anti-hash ou construção; para Prouhet, separe os índices por paridade.
  - *Exemplo:* t = 0,1,1,0,1,0,0,1,….
  - *Armadilha:* a indexação começa em 0; "livre de sobreposições" é mais forte que livre de cubos; comprimento do teste anti-hash.
- **de Bruijn B(k, n).** Sequência cíclica de tamanho k^n com toda string de tamanho n exatamente uma vez; construa por ciclo euleriano nos (n−1)-gramas ou concatenando palavras de Lyndon de comprimento divisor de n em ordem lexicográfica. Cadeado/senha mínima, construtivos.
  - *Gatilho:* menor sequência que contém todos os n-gramas, cadeado, senha, construções cíclicas.
  - *Por que funciona:* os (n−1)-gramas são vértices de um grafo cujas arestas são os n-gramas; um ciclo euleriano percorre cada n-grama exatamente uma vez. A construção por Lyndon (FKM) gera o mesmo.
  - *Como fazer:* Hierholzer no grafo de de Bruijn; ou concatene palavras de Lyndon cujo comprimento divide n em ordem lexicográfica.
  - *Exemplo:* B(2,3) = 00010111 (cíclico).
  - *Armadilha:* a sequência é cíclica; o tamanho é k^n; para n=1, todas as letras; não confunda com sequência linear.

- **Trie + small-to-large / merge de tries.** Mesclar duas tries recursivamente (juntando filhos de mesma letra) custa O(total de nós) amortizado; serve para DP sobre árvore onde cada nó tem um conjunto de strings/sufixos.
  - *Gatilho:* DP em árvore com conjuntos de strings, merge de tries, ocorrências em subárvores.
  - *Por que funciona:* tries compartilham prefixos; ao mesclar, filhos com a mesma letra se fundem e os nós menores são absorvidos, permitindo small-to-large.
  - *Como fazer:* merge(a,b): se um é nulo, retorne o outro; para cada letra, merge recursivo; atualize informações do nó; sempre mescle o menor no maior quando possível.
  - *Exemplo:* para cada nó de uma árvore, manter a trie dos sufixos da subárvore.
  - *Armadilha:* a complexidade amortizada é O(N log N) no caso geral, não necessariamente O(N); destructive merge; mapas de filhos.
- **Trie persistente / de versões.** "Versão v + caractere c" cria um nó na trie global (árvore de prefixos); LCP = profundidade do LCA, hash do prefixo = hash do pai + c, e comparações de versões via binary lifting. Processar offline por DFS com rollback.
  - *Gatilho:* versões de strings, append/rollback, LCP entre versões, persistência.
  - *Por que funciona:* cada versão é um nó na trie de prefixos; o LCA de duas versões é o maior prefixo comum. O hash do prefixo é o hash do pai mais o caractere; binary lifting dá LCA.
  - *Como fazer:* crie um nó por versão; pai = versão anterior; profundidade = LCP; hash = hash(pai)·B + c; binary lifting para LCA. Alternativa offline: DFS com rollback.
  - *Exemplo:* histórico de edições de uma string com append.
  - *Armadilha:* memória O(n log n) se binary lifting; o hash precisa da potência; versões que não são extensões simples exigem outra estrutura.
- **Ordenar strings por trie** em O(Σ|s|) (DFS em ordem de letra); LCP de duas strings ordenadas = min dos LCP adjacentes (RMQ).
  - *Gatilho:* ordenar muitas strings, LCP entre strings ordenadas.
  - *Por que funciona:* a DFS da trie em ordem alfabética visita as strings em ordem lexicográfica; o LCP de duas strings é o mínimo dos LCPs adjacentes entre elas, como no SA.
  - *Como fazer:* insira na trie; DFS em ordem de letra; calcule LCPs adjacentes; RMQ para consultas.
  - *Exemplo:* ordenar ["banana", "band", "ban"].
  - *Armadilha:* strings que são prefixo de outras; nós terminais; duplicatas.
- **Edit distance com limite k.** Banda $|i - j| \le k \rightarrow O(nk)$; Landau–Vishkin com LCE em O(1) → $O(n + k^2)$ para distância ≤ k.
  - *Gatilho:* edit distance pequena, aproximação, k limitado.
  - *Por que funciona:* se a distância é ≤ k, nenhuma célula fora da banda $|i-j| \le k$ pode ser usada; a banda reduz a DP. Landau–Vishkin salta trechos iguais com LCE.
  - *Como fazer:* DP em banda O(nk); ou Landau–Vishkin: para cada diagonal e número de edições, use LCE para avançar; $O(n + k^2)$.
  - *Exemplo:* editar "kitten" para "sitting" com k = 3.
  - *Armadilha:* a banda precisa de bordas; transposição é outra métrica; LCE O(1) requer hash/SA.
- **Kangaroo jumps.** Posições onde p casa em t com ≤ k erros → em cada deslocamento, k+1 saltos de LCE (hash+binária ou SA+RMQ): O(n·k) total.
  - *Gatilho:* matching aproximado com poucos mismatches, busca com ≤ k erros.
  - *Por que funciona:* depois de um erro, o próximo trecho igual pode ser pulado com LCE; com no máximo k erros, há no máximo k+1 saltos por alinhamento.
  - *Como fazer:* para cada deslocamento, use LCE para avançar; conte erros; pare após k+1 erros. O(nk) com LCE O(1).
  - *Exemplo:* achar p em t com no máximo 1 mismatch.
  - *Armadilha:* LCE precisa limitar ao tamanho restante; inserções/deleções não são cobertas; wildcards.
- **Matching de Hamming por FFT.** Nº de coincidências por deslocamento = Σ_c conv(1[t=c], 1[p reverso = c]) (σ convoluções para alfabeto de tamanho σ); uma única convolução complexa com raízes de unidade ω^c não recupera a contagem para alfabeto > 2. Com '?' use $\Sigma (a-b)^2 \cdot a \cdot b$ e valores aleatórios mod p para evitar falso zero.
  - *Gatilho:* contar coincidências/mismatches por alinhamento, pattern matching com wildcards.
  - *Por que funciona:* para cada letra, a convolução entre as posições do texto e do padrão invertido conta as coincidências daquela letra; somando as σ letras, tem-se o total. Uma única convolução com raízes de unidade não separa as letras para σ > 2.
  - *Como fazer:* faça σ convoluções (FFT/NTT) de indicadores; some. Para wildcards, use o polinômio $\Sigma(a-b)^2ab$, que é zero nos matches, com valores aleatórios mod p.
  - *Exemplo:* achar todas as posições de p em t com até 1 mismatch.
  - *Armadilha:* σ pode ser grande; precisão de FFT; falso zero por cancelamento; '?' deve ser 0.

- **LCS com bitset** em O(nm/w): V = tudo 1; para cada caractere c de t, M = máscara de ocorrências de c em s, V = (V + (V & M)) | (V & ~M); LCS = nº de zeros em V (nos primeiros |s| bits).
  - *Gatilho:* LCS com n, m grandes e alfabeto pequeno, bitset.
  - *Por que funciona:* o bitset codifica a linha da DP de LCS; a atualização combina máscara de ocorrências e carry para simular a recorrência em paralelo, processando w células por operação.
  - *Como fazer:* V com |s| bits em 1; para cada c de t, M = máscara de posições de c em s; V = (V + (V & M)) | (V & ~M); ao final, conte zeros nos |s| primeiros bits. O(nm/w).
  - *Exemplo:* LCS("abc", "ac") = 1; V termina com um zero.
  - *Armadilha:* ordem dos bits; mascarar além de |s|; w = 64; a fórmula usa complemento dentro da máscara.
- **LCS ↔ LIS.** Para permutações (ou strings com poucas ocorrências por letra), LCS = LIS em O(n log n) (Hunt–Szymanski, O(r log n)); LPS (subsequência palindrômica) = LCS(s, rev s); inserções mínimas p/ palíndromo = n − LPS.
  - *Gatilho:* LCS de permutações, poucas ocorrências por letra, LPS.
  - *Por que funciona:* em uma permutação, a ordem relativa vira uma sequência de posições; uma subsequência comum é uma subsequência crescente dessas posições. Para LPS, a simetria reduz ao LCS com o reverso.
  - *Como fazer:* mapeie cada caractere de s para suas posições; percorra t e faça LIS nas posições (Hunt–Szymanski); para LPS, LCS(s, rev s). Inserções mínimas = n − LPS.
  - *Exemplo:* s = "abc", t = "bac": LCS = 2 ("ac" ou "bc").
  - *Armadilha:* LIS estritamente crescente; duplicatas; LPS não é o mesmo que maior substring palindrômica.
- **Subsequência como gatilho.** "t é subsequência de s?" com muitas queries → next[i][c] ou listas de ocorrências + binária; contar quantas de várias strings t são subsequências de s → listas de espera por próxima letra, O(|s| + Σ|t|).
  - *Gatilho:* muitas consultas de subsequência, "is subsequence", contar strings que são subsequências.
  - *Por que funciona:* next[i][c] dá a próxima ocorrência de c a partir de i, permitindo avançar em O(1) por caractere; para muitas t, processe todas em paralelo com listas de espera.
  - *Como fazer:* pré-compute next[i][c] de trás para frente; para cada t, avance; ou mantenha, para cada letra, a lista de strings que esperam aquela letra e processe s uma vez.
  - *Exemplo:* s = "abc", t = "ac" é subsequência; "ca" não.
  - *Armadilha:* alfabeto; índices 1-based; strings vazias; listas de espera precisam avançar corretamente.
- **k-ésima subsequência distinta lexicográfica** → cnt[i] = 1 + Σ_c cnt[next[i][c]+1] (next = n se não existe; cnt[n] = 1, cnt[n+1] = 0) e desça escolhendo letras em ordem.
  - *Gatilho:* k-ésima subsequência distinta em ordem lexicográfica.
  - *Por que funciona:* cnt[i] conta as subsequências distintas (incluindo a vazia) que começam na posição i; escolher a menor letra e pular para sua próxima ocorrência enumera em ordem lexicográfica sem duplicatas.
  - *Como fazer:* calcule next; dp de trás para frente; para k, percorra letras em ordem, subtraia cnt[next+1] até achar o grupo; reconstrua.
  - *Exemplo:* s = "abc": subsequências distintas: "", "a", "ab", "abc", "ac", "b", "bc", "c".
  - *Armadilha:* a fórmula inclui a vazia; se o problema pede a k-ésima não vazia, use k+1 ou subtraia a vazia; overflow; next = n e cnt[n+1] = 0.
- **Maior borda que também aparece no meio** (prefix = sufixo = substring interior) → entre as bordas de π, a maior com ocorrência interna (use max de z/π no meio) (CF 126B).
  - *Gatilho:* prefixo que é sufixo e aparece no meio, "password" (CF 126B).
  - *Por que funciona:* as bordas candidatas são a cadeia de π; para cada uma, é preciso verificar se ela ocorre em uma posição interna, o que pode ser feito com Z/π.
  - *Como fazer:* calcule π e a cadeia de bordas; verifique ocorrência interna com Z (z[i] ≥ b) ou com uma segunda passada de π; pegue a maior que ocorre. O(n).
  - *Exemplo:* "fixprefixsuffix": "fix" é prefixo, sufixo e aparece no meio.
  - *Armadilha:* a ocorrência interna não pode ser o próprio prefixo/sufixo; a borda de tamanho n não vale; índices.

- **Strings k-periódicas / período d.** s tem período d ⇔ s[i] = s[i+d]; classes i mod d via DSU. Menor nº de mudanças para período d = n − Σ_classes(max freq). Com palíndromo + período, DSU em i ~ n−1−i e i ~ i+d.
  - *Gatilho:* impor período d, minimizar mudanças, combinar com palíndromo.
  - *Por que funciona:* um período d agrupa posições congruentes módulo d; todas precisam ter o mesmo caractere. O custo mínimo do grupo é o tamanho menos a frequência máxima. Restrições extras viram uniões no DSU.
  - *Como fazer:* DSU para i ~ i+d (e i ~ n−1−i, se palíndromo); para cada componente, conte frequências e escolha a mais comum; custo = n − Σ máximos.
  - *Exemplo:* "abac" com período 2: classes {0,2} e {1,3}; a classe {0,2} já é "a","a", mas {1,3} é "b","c", então o custo é 1.
  - *Armadilha:* d > n; múltiplas restrições; o centro do palíndromo; componentes vazios.
- **Anagramas e multiconjuntos.** Nº de permutações distintas = n!/Π c_i!; ranking lexicográfico de permutação de multiconjunto → soma sobre letras menores com BIT de contagens; "próxima permutação" em O(n).
  - *Gatilho:* permutações de multiconjunto, ranking, próxima permutação.
  - *Por que funciona:* o número de permutações é multinomial; para o ranking, para cada posição conte as permutações que começam com uma letra menor, usando fatorial e frequências. A próxima permutação usa o próximo arranjo lexicográfico.
  - *Como fazer:* pré-compute fatoriais; para o rank, para cada posição, some sobre letras menores: (n−i−1)!/Π c! com a letra decrementada; use BIT para as contagens. Próxima permutação: ache o maior i com s[i] < s[i+1], troque com o sucessor e inverta o sufixo.
  - *Exemplo:* "aab" tem 3 permutações: "aab", "aba", "baa".
  - *Armadilha:* módulo; divisão por fatoriais; letras repetidas; rank 0 ou 1-indexado.
- **Parênteses: balanço e mínimo de prefixo.** Válida ⇔ min prefixo ≥ 0 e total 0. Inserções mínimas = (−min) aberturas + (total − min) fechamentos; maior subsequência válida = 2·(nº de pares casados pelo guloso com contador/pilha), que pode ser menor que 2·min(#(, #)).
  - *Gatilho:* validade, mínimo de inserções, maior subsequência válida.
  - *Por que funciona:* o prefixo mínimo mede o déficit máximo de aberturas; o total mede o excesso final. Inserir aberturas antes do déficit e fechamentos no fim corrige. Para subsequência, o guloso casa cada fechamento com uma abertura disponível.
  - *Como fazer:* contador +1/−1 e mínimo; inserções = −min + (total−min); pares casados com contador/pilha; resposta 2·pares.
  - *Exemplo:* "())(" tem min −1, total 0, inserções 2; maior subsequência válida é "()" (2).
  - *Armadilha:* min positivo e total negativo; vários tipos de parênteses; subsequência não é substring.
- **Parênteses em segtree.** Nó guarda (soma, min prefixo); merge = (sL+sR, min(mL, sL+mR)). Substring é válida ⇔ soma 0 e min ≥ 0. Updates por troca de caractere.
  - *Gatilho:* consultas/atualizações em intervalos de parênteses.
  - *Por que funciona:* a soma e o mínimo prefixo formam um monóide; a concatenação soma os totais e o mínimo é o mínimo do lado esquerdo ou o total esquerdo mais o mínimo do direito.
  - *Como fazer:* segtree com (sum, minpref); merge; consulta de intervalo; update de folha. Para validade, verifique sum=0 e min≥0.
  - *Exemplo:* "()((": soma 1, min 0; não é válida no total.
  - *Armadilha:* identidade (0, 0); min prefixo do intervalo vazio; atualização de caractere.
- **Contar sequências com mínimo de prefixo ≥ −k** (2n passos, terminando em 0) = C(2n, n) − C(2n, n+k+1) (reflexão). k = 0: Catalan.
  - *Gatilho:* caminhos com barreira, sequências balanceadas com limite, Catalan generalizado.
  - *Por que funciona:* pelo princípio da reflexão, os caminhos que violam a barreira são refletidos e contados por um binomial; subtrair do total dá os válidos.
  - *Como fazer:* total C(2n,n); subtraia C(2n, n+k+1); para k=0, C(2n,n)−C(2n,n+1) = Catalan.
  - *Exemplo:* n=2, k=1: 6 − 1 = 5 caminhos.
  - *Armadilha:* a barreira é ≥ −k; k pode ser maior que n; módulo; passos exatamente 2n.

- **Árvore ↔ Dyck.** DFS que escreve (ao entrar e) ao sair: subárvore = intervalo balanceado, LCA = posição de mínimo do balanço (RMQ), profundidade = balanço. Contar árvores com n nós = Catalan(n−1).
  - *Gatilho:* representar árvore por parênteses, LCA via RMQ, contagem de árvores.
  - *Por que funciona:* a DFS produz uma sequência balanceada; a subárvore é um intervalo balanceado; o LCA de dois nós é o nó de menor profundidade entre suas ocorrências, que corresponde ao mínimo do balanço.
  - *Como fazer:* faça a DFS e registre entrada/saída; use RMQ sobre o balanço para LCA; conte árvores ordenadas com Catalan(n−1).
  - *Exemplo:* uma folha é "()"; um nó com dois filhos é "(())()" ou "()(())".
  - *Armadilha:* árvores ordenadas versus não ordenadas; a posição de mínimo é do intervalo entre as primeiras ocorrências; nós implícitos.
- **Parênteses de vários tipos** → pilha; troca mínima / "casamento" quando só existe um tipo ou tipos de mesma classe → guloso com contador.
  - *Gatilho:* parênteses de vários tipos, colchetes, chaves, trocas.
  - *Por que funciona:* a pilha casa cada fechamento com a abertura mais recente do tipo correto; com um só tipo, um contador basta. Trocas mínimas podem ser reduzidas a casamentos gulosos quando os tipos são da mesma classe.
  - *Como fazer:* pilha para validade; para trocas, analise os desemparelhamentos e case com contador; se os tipos são permutáveis, trate como um só.
  - *Exemplo:* "([)]" é inválido; "([]{})" é válido.
  - *Armadilha:* tipos diferentes não são intercambiáveis; a pilha precisa do tipo; trocas podem criar novos casamentos.
- **Shunting-yard.** Operadores com precedência e associatividade (^ à direita); menos unário como token distinto com precedência alta; avalie com pilha de operandos. Para gramática recursiva, descent: expr → termo (±) → fator (×, /) → átomo/parênteses.
  - *Gatilho:* avaliar expressões, parser de operadores, precedência.
  - *Por que funciona:* a shunting-yard converte para RPN respeitando precedência e associatividade; a pilha de operandos avalia. O descent recursivo implementa a mesma gramática.
  - *Como fazer:* leia tokens; operandos vão para a saída; operadores desempilham os de precedência maior (ou igual, se à esquerda); parênteses controlam; avalie a RPN. Descent: expr, termo, fator.
  - *Exemplo:* "2+3*4" → 14; "2^3^2" → 512 (à direita).
  - *Armadilha:* menos unário precisa de token distinto; a precedência dele em relação a ^ varia; associatividade à direita; parênteses.
- **Parsing com módulo.** Ao avaliar expressões mod p, divisão = inverso modular; números grandes lidos como string e reduzidos mod p na hora; avalie com pilhas em O(n) sem recursão profunda.
  - *Gatilho:* expressão enorme, números gigantes, módulo p.
  - *Por que funciona:* as operações de soma, subtração e multiplicação comutam com a redução módulo p; a divisão vira multiplicação pelo inverso, quando existe. Ler como string evita overflow.
  - *Como fazer:* reduza cada número mod p durante a leitura; avalie com pilhas; para divisão, use inverso modular (p primo e denominador não múltiplo de p). O(n).
  - *Exemplo:* "2^100 mod 1000000007" via exponenciação rápida.
  - *Armadilha:* inverso só existe se gcd(denominador, p)=1; p não primo; precedência; números negativos.
- **Regex/glob simples.** '.' e '' → *DP O(nm); glob com '?' e* '' → guloso com retorno à última '*' (O(nm) no pior caso); NFA de Thompson simulado com bitset.
  - *Gatilho:* matching de padrões com '.', '*', '?', glob, regex simples.
  - *Por que funciona:* a DP testa se o prefixo do texto casa com o prefixo do padrão; o guloso de glob adia a decisão do '*' e volta a ele quando necessário; a NFA simula todos os estados ativos em paralelo.
  - *Como fazer:* DP[i][j] para regex com '.' e ''*; glob: dois ponteiros, guarde a última posição de* '' e o texto correspondente; NFA com bitset para muitos estados.
  - *Exemplo:* glob "a*b?*" *casa* "*axby*"; *regex* "*a.*b" casa "axxxb".
  - *Armadilha:* o '' *de regex e o* '' de glob têm semânticas diferentes; matches vazios; pior caso do guloso.

- **DP de dicionário.** dp[i] = nº de modos de dividir s[0..i) em palavras do dicionário: use trie parado em profundidade maxlen (O(n·maxlen)), ou ≤ √(2S) tamanhos distintos com hash set por tamanho, ou AC com dict link.
  - *Gatilho:* word break, contar segmentações, dicionário.
  - *Por que funciona:* a DP sobre prefixos testa todos os sufixos que são palavras; a trie/hash/AC acelera a verificação. O número de tamanhos distintos é limitado pela soma dos comprimentos.
  - *Como fazer:* dp[0]=1; para i, teste palavras que terminam em i usando trie (até maxlen), hash por tamanho, ou AC com dict link; dp[i] += dp[i−len].
  - *Exemplo:* s = "leetcode", dict {"leet","code"}: 1 modo.
  - *Armadilha:* palavras vazias; duplicatas; maxlen; contar modos versus decidir existência.
- **LZ78 / LZ77.** LZ78 = percorrer trie criando um nó por fator (≤ O(n/log n) fatores); LZ77 por SA/SAM. O nº de fatores é medida de compressibilidade e aparece como limite de complexidade em estruturas sobre strings repetitivas.
  - *Gatilho:* compressão, strings repetitivas, número de fatores, LZ.
  - *Por que funciona:* no LZ78, cada fator é um nó da trie (prefixo anterior + caractere); a soma dos comprimentos dos fatores é o total, e a trie limita o número de fatores. No LZ77, as referências podem ser encontradas com SA/SAM.
  - *Como fazer:* LZ78: trie de fatores; para cada posição, desça o maior prefixo existente e crie o próximo nó; LZ77: SA/SAM para achar o maior match anterior.
  - *Exemplo:* o LZ78 de "abab" tem os fatores "a", "b", "ab".
  - *Armadilha:* a definição de fator varia; o limite O(n/log n) usa alfabeto constante; LZ77 e LZ78 não são equivalentes.
- **Construção lexicográfica mínima.** Escolha letra a letra e teste viabilidade com um dp pronto (SAM/AC/next[i][c]); o desempate é local, o teste é global.
  - *Gatilho:* construir a menor string que satisfaz uma propriedade, com restrições de autômato/ocorrências.
  - *Por que funciona:* a ordem lexicográfica é decidida pela primeira posição em que as strings diferem; se houver alguma conclusão viável com a menor letra, ela é ótima. O dp responde à viabilidade do sufixo.
  - *Como fazer:* pré-compute o dp de viabilidade; para cada posição, teste as letras em ordem e escolha a primeira cuja transição leva a um estado viável; continue.
  - *Exemplo:* menor string de tamanho L que evita padrões; menor string que contém todos os padrões.
  - *Armadilha:* o teste precisa considerar o estado atual; empates; o dp pode ser de contagem, não só booleano.
- **Strings com operações e rollback** → pilha de estados (hash, nó do AC, nó da trie) em vez de recomputar; versionamento por árvore de prefixos.
  - *Gatilho:* adicionar/remover caracteres, DFS, undo, versões.
  - *Por que funciona:* o estado (hash, nó do autômato, nó da trie) é atualizado incrementalmente; guardar o estado anterior em uma pilha permite restaurar em O(1).
  - *Como fazer:* antes de cada operação, empilhe o estado; ao desfazer, desempilhe; para versões persistentes, use uma trie de prefixos.
  - *Exemplo:* DFS em uma trie de strings mantendo o nó do AC e o hash.
  - *Armadilha:* restaurar todos os componentes do estado; não usar a amortização do KMP; memória da pilha.
- **Palíndromo em substring com updates** → segtree com (hash direto, hash do reverso); para contagem de palíndromos/qualquer centro, Manacher não serve com updates: use hash + binária no raio.
  - *Gatilho:* atualizações de caractere e consultas de palíndromo.
  - *Por que funciona:* com os dois hashes, a igualdade com o reverso é O(1); para achar o maior raio em um centro, o predicado "é palíndromo de raio r" é monotônico, então cabe binária.
  - *Como fazer:* segtree com hash direto e reverso; update de ponto; para consulta, binária no raio testando os hashes; $O(\log^2 n)$ por consulta.
  - *Exemplo:* trocar um caractere e verificar se uma substring é palíndromo.
  - *Armadilha:* Manacher é offline; colisões de hash; centros ímpares/pares; o intervalo do reverso.

adicionado em 2026-11-11 a partir dos editoriais de CF Div. 2/3/4, A–D; triado contra o catálogo

- **Operação troca todas as ocorrências de um símbolo.** Dá para chegar ao alvo sse o mapeamento s_i → t_i é função (mesmo símbolo sempre vai no mesmo alvo). Ex.: CF 1807C.
  - *Gatilho:* operação de substituir todas as ocorrências de um símbolo, transformar s em t, consistência de mapeamento.
  - *Por que funciona:* a operação não distingue ocorrências do mesmo símbolo; se duas posições com o mesmo caractere precisam terminar com caracteres diferentes, é impossível. Se o mapeamento é consistente, dá para aplicá-lo.
  - *Como fazer:* verifique para cada caractere de s se todas as posições correspondentes em t têm o mesmo caractere; se sim, aplique as substituições (cuidando de ciclos com um símbolo temporário).
  - *Exemplo:* CF 1807C: mapear cada letra para 0/1 de forma consistente e alternada.
  - *Armadilha:* a condição de função é necessária; com ciclos e alfabeto sem símbolo temporário, pode não ser suficiente; a ordem das substituições importa.
- **Palíndromo com mudanças pontuais.** Cada par (i, n − 1 − i) é independente: custo total = Σ custo mínimo de igualar o par; o centro (n ímpar) é livre. Ex.: CF 2227D.
  - *Gatilho:* tornar a string palíndromo com mudanças de caractere, custo mínimo.
  - *Por que funciona:* para ser palíndromo, cada par simétrico precisa ter caracteres iguais; os pares não interagem, então o custo é a soma dos custos independentes. O centro não tem par.
  - *Como fazer:* para cada par, calcule o custo mínimo de igualar os dois caracteres (segundo a função de custo do problema); some; ignore o centro se n for ímpar.
  - *Exemplo:* CF 2227D: pares independentes.
  - *Armadilha:* o custo pode não ser a diferença simples; pares iguais custam 0; o centro pode ter custo se o problema exigir algo específico.

Repetidas de outras seções:

- **Tempo esperado até padrão** → Σ q^k sobre bordas do padrão (ABRACADABRA = $26^{11} + 26^4 + 26$).
- **Strings primitivas / períodos exatos** → Möbius sobre divisores de n: Σ_{d|n} μ(n/d)k^d.

Grafos

Pergunte: quais são os vértices de verdade? Muitas vezes o grafo útil é de estados, não o do enunciado.

- **Grafo de estados:** (vértice, quantos usos de algo, paridade, tempo mod k) → BFS/Dijkstra em camadas.
  - *Gatilho:* "no máximo k usos", "paridade", "tempo módulo k", "estado depois de uma sequência de ações"; a resposta não depende só do vértice atual.
  - *Por que funciona:* a coordenada extra é toda a memória necessária para decidir a próxima transição; o produto V × K é um grafo comum e caminhos nele correspondem a sequências válidas no original.
  - *Como fazer:* crie um vértice por par (v, j); para cada aresta u → v e cada j, ligue (u, j) a (v, j') conforme a transição; BFS sem pesos, Dijkstra com pesos; O(K(n + m)) ou O(Km log(Kn)).
  - *Exemplo:* "caminho com paridade par de arestas" = BFS em (v, bit); "usar no máximo k teleportes" = (v, j).
  - *Armadilha:* indexe o composto (v·K + j) e inclua a coordenada no estado inicial e no critério de parada; se K é grande, o produto estoura.
- **Pesos 0/1** → 0-1 BFS com deque. Pesos pequenos → BFS com várias filas.
  - *Gatilho:* arestas de custo 0 ou 1, "mínimo de trocas", "custo de virar/inverter".
  - *Por que funciona:* com dois custos a deque mantém a fronteira ordenada por distância: custo 0 entra na frente, custo 1 atrás; é um Dijkstra com fila especializada.
  - *Como fazer:* ao relaxar (u, v, w), faça push_front(v) se w = 0 e push_back(v) se w = 1; descarte se d > dist[u]; O(n + m). Para pesos 0..C use C + 1 filas (Dial), O(m + nC).
  - *Exemplo:* grid com setas: custo 0 seguir a seta, 1 mudar de direção.
  - *Armadilha:* BFS comum não ordena custos diferentes; com pesos arbitrários a deque não vale; marque visitado ao extrair.
- **Várias fontes** → super-fonte ligada a todas com custo 0; caminho de volta → grafo reverso.
  - *Gatilho:* "o mais próximo de qualquer fonte", "todos os bombeiros ao mesmo tempo", "custo de escolher uma origem".
  - *Por que funciona:* todo caminho que sai da super-fonte começa em uma fonte e paga 0 por isso, então dist(s, v) = min sobre fontes; no reverso, distâncias até t viram distâncias a partir de t.
  - *Como fazer:* inicialize todas as fontes com dist 0 na fila (equivalente à super-fonte); para múltiplos alvos, rode a partir do alvo no grafo com as arestas invertidas; O(n + m) ou O(m log n).
  - *Exemplo:* BFS multi-fonte do fogo em grid; Dijkstra partindo de várias cidades.
  - *Armadilha:* no reverso inverta todas as arestas, não só a ordem de relaxamento; com pesos, o peso é preservado; fontes duplicadas não somam custo.

- **Arestas de um vértice para um intervalo** → segtree de arestas (grafo sobre os nós da segtree).
  - *Gatilho:* "u se conecta a todos os vértices de [l, r]", "aresta para um intervalo de posições/valores", "pular para qualquer j na faixa".
  - *Por que funciona:* a segtree tem O(n) nós e decompõe [l, r] em O(log n) nós; arestas internas de custo 0 propagam o custo, então o grafo expandido tem O((n + m) log n) arestas.
  - *Como fazer:* segtree com pai → filho de custo 0 para arestas de u para o intervalo; para arestas do intervalo para u use a segtree reversa (filho → pai) ou duas segtrees; ligue u aos O(log n) nós que cobrem [l, r] com o peso; rode Dijkstra/BFS no grafo expandido.
  - *Exemplo:* "de i posso ir a qualquer j em [i − a_i, i + a_i] com custo 1" → segtree + BFS.
  - *Armadilha:* a aresta u → nó da segtree custa w e as internas custam 0; ligar u a todas as folhas do intervalo é O(n) por aresta; memória O((n + m) log n).
- **Restrições x_j − x_i ≤ c** → diferença de potenciais, Bellman-Ford.
  - *Gatilho:* sistema de desigualdades com diferenças, "x_j ≤ x_i + c", "horários com folga mínima".
  - *Por que funciona:* cada desigualdade vira uma aresta i → j de peso c; um ciclo negativo é exatamente uma combinação que implica x_i < x_i; sem ciclo, as distâncias de uma super-fonte satisfazem todas as restrições.
  - *Como fazer:* super-fonte ligada a todos com custo 0; Bellman-Ford/SPFA; viável sse não há ciclo negativo; solução x_v = dist(v); O(nm).
  - *Exemplo:* x_2 − x_1 ≤ 3, x_3 − x_2 ≤ −5, x_1 − x_3 ≤ 1 forma ciclo de peso −1 e é inviável.
  - *Armadilha:* o sinal é o erro clássico: x_j − x_i ≤ c é aresta i → j com peso c; x_j − x_i ≥ c vira j → i com −c; igualdade = duas desigualdades; em inteiros, "x_i < x_j" vira x_i − x_j ≤ −1.
- **Restrições booleanas de 2 variáveis** → 2-SAT (SCC).
  - *Gatilho:* "cada cláusula tem no máximo 2 literais", "se A então B", "não pode A e B juntos", "escolher uma de duas opções".
  - *Por que funciona:* (a ∨ b) ≡ (¬a → b) e (¬b → a); uma atribuição satisfaz tudo sse nenhuma variável e sua negação caem na mesma SCC, pois SCCs são fechadas sob implicação.
  - *Como fazer:* 2n vértices (x e ¬x); cada cláusula gera duas implicações; Tarjan/Kosaraju; se comp(x) = comp(¬x), insatisfatível; senão, com Tarjan (ids em ordem topológica reversa), x é verdadeiro sse comp(x) < comp(¬x); O(n + m).
  - *Exemplo:* (x ∨ y) ∧ (¬x ∨ y) ∧ (x ∨ ¬y) força y = verdadeiro.
  - *Armadilha:* adicionar só uma implicação; esquecer cláusulas unitárias (a ∨ a); a regra de atribuição se inverte entre Tarjan e Kosaraju.
- **Direcionado com ciclos** → condense em SCC e vire DAG; no DAG, DP em ordem topológica.
  - *Gatilho:* DP em grafo com ciclos, "caminho que pode repetir vértices", "maior soma ao longo de um caminho".
  - *Por que funciona:* dentro de uma SCC todos se alcançam, então o valor agregado da componente pode ser calculado uma vez; a condensação é um DAG e a ordem topológica dá a ordem de cálculo.
  - *Como fazer:* Tarjan/Kosaraju; construa o DAG com uma aresta por par de componentes (deduplicate); DP na ordem topológica; O(n + m).
  - *Exemplo:* soma dos valores de uma SCC e depois longest path no DAG.
  - *Armadilha:* condensar perde a multiplicidade de caminhos dentro da SCC; se o problema conta caminhos, não basta condensar; arestas duplicadas no DAG não podem contar duas vezes.
- **Pontes e articulações** → árvore de componentes 2-aresta-conexas / block-cut tree.
  - *Gatilho:* "estrada cuja remoção desconecta", "cidade de corte", "componentes 2-aresta-conexas".
  - *Por que funciona:* na DFS, low[child] > tin[u] significa que a aresta é a única ligação com a subárvore (ponte); low[child] ≥ tin[u] marca u como articulação; contrair as componentes 2-aresta-conexas deixa uma árvore.
  - *Como fazer:* DFS com tin/low; pontes e articulações em O(n + m); monte a árvore de pontes ou o block-cut tree (nós-bloco ligados às articulações).
  - *Exemplo:* "quantas estradas são críticas" = pontes; "quais cidades derrubam a rede" = articulações.
  - *Armadilha:* em multigrafo pule a aresta pelo id, não pelo vértice pai; a raiz é articulação só com ≥ 2 filhos; recursão profunda pode estourar a pilha.

- **"Adicione arestas" ou "conectividade"** → DSU; com remoções → offline ao contrário ou DSU com rollback + D&C no tempo.
  - *Gatilho:* "adicione arestas para conectar", "quantas componentes", "arestas ativas/removidas ao longo do tempo".
  - *Por que funciona:* DSU mantém componentes sob união em O(α); adicionar arestas só une; processar remoções ao contrário transforma remoção em adição; rollback + D&C cobre intervalos de vida.
  - *Como fazer:* DSU com path compression e union by size; offline reversed; DSU rollback (sem path compression) + segtree de tempo para inserções e remoções; O((n + m) α) ou O((n + m) log m α).
  - *Exemplo:* "após cada aresta adicionada, diga se o grafo é conexo"; "arestas ativas em [l, r]".
  - *Armadilha:* rollback exige desfazer na ordem inversa e não pode comprimir caminhos; para remoção de vértice, remova também suas arestas.
- **MST:** Kruskal; "maior aresta mínima no caminho" → MST + LCA ou árvore de Kruskal.
  - *Gatilho:* "conectar todos com custo mínimo", "rede", "custo de ligar", "menor gargalo entre dois pontos".
  - *Por que funciona:* pela propriedade do corte, a aresta mais leve que cruza um corte é segura; Kruskal aceita exatamente essas arestas, e a troca nunca piora.
  - *Como fazer:* ordene as arestas por peso, DSU, aceite se une componentes; O(m log m). Para minimax, o maior peso no caminho da MST é o menor gargalo possível; responda com LCA que guarda o máximo ou com a árvore de Kruskal.
  - *Exemplo:* "estradas para conectar cidades"; "qual o caminhão mais leve que passa de A a B".
  - *Armadilha:* pesos negativos não quebram Kruskal; empates podem gerar várias MSTs; para minimax não é preciso Floyd–Warshall.
- **Bipartido / 2-coloração** → paridade dos ciclos; ciclo ímpar = não bipartido.
  - *Gatilho:* "dividir em dois grupos sem conflito", "x e y não podem ficar juntos", "checar paridade".
  - *Por que funciona:* um grafo é bipartido sse não tem ciclo ímpar; a BFS alterna cores e uma aresta entre vértices de mesma cor fecha um ciclo ímpar.
  - *Como fazer:* BFS/DFS colorindo com 1 − cor; se um vizinho já tem a mesma cor, não é bipartido; O(n + m); para relações online de igual/diferente use DSU com paridade.
  - *Exemplo:* 2-coloração de grid; conflitos entre pessoas.
  - *Armadilha:* rode de todos os não visitados em grafo desconexo; laços tornam impossível; duas arestas paralelas entre dois vértices não formam ciclo ímpar.
- **Emparelhamento, cobertura, "atribuir"** → fluxo/matching; cortes mínimos para "escolher conjunto com dependências" (projeto de seleção / closure).
  - *Gatilho:* "cada tarefa a uma máquina", "maximizar pares", "cobrir com o mínimo", "projetos com pré-requisitos".
  - *Por que funciona:* König iguala matching máximo e cobertura mínima no bipartido; fluxo modela capacidades; corte mínimo resolve seleção com dependências porque a dependência vira capacidade infinita.
  - *Como fazer:* rede S → L cap 1, L → R cap 1, R → T cap 1; maxflow = matching. Para closure: lucros positivos de S, custos para T, dependências com ∞; resposta = Σ lucros − corte mínimo. O(maxflow).
  - *Exemplo:* "n alunos, m vagas"; "escolher projetos com pré-requisitos".
  - *Armadilha:* matching geral precisa de Blossom; a dependência deve apontar na direção "se escolho, preciso"; custos negativos viram arestas para T.
- **Funcional (cada vértice uma saída)** → ciclos com árvores penduradas; k passos → binary lifting.
  - *Gatilho:* "cada posição aponta para outra", "sucessor", "permutação", "k-ésimo passo".
  - *Por que funciona:* com outdeg = 1 cada componente tem exatamente um ciclo e árvores que apontam para ele; seguir ponteiros é determinístico, então k passos podem ser acelerados por potências de dois.
  - *Como fazer:* detecte ciclos por cores ou tortoise–hare; calcule pré-período e período; up[k][v] para o 2^k-ésimo sucessor; para somas, guarde soma junto; O(n log K).
  - *Exemplo:* "onde estou após 10^18 saltos"; "quantos vértices chegam a cada ciclo".
  - *Armadilha:* não confunda pré-período com período; k pode precisar de 64 bits; se algum vértice tem outdeg = 0 a estrutura não é funcional.

- **Grafo denso ou complemento** → BFS no complemento com conjunto de não visitados.
  - *Gatilho:* "grafo completo exceto m arestas", "n grande e m pequeno", "não existe aresta entre".
  - *Por que funciona:* o complemento pode ter $O(n^2)$ arestas, mas o BFS não precisa enumerá-las: cada vértice sai do conjunto de não visitados uma vez e os vizinhos originais são pulados.
  - *Como fazer:* mantenha um set/lista de não visitados; para cada u, marque seus vizinhos originais e reconstrua a lista removendo os que não são vizinhos de u (esses entram na fila); O(n + m) esperado com hash/set.
  - *Exemplo:* componentes conexas do grafo complemento.
  - *Armadilha:* não materialize a matriz do complemento; percorrer a lista inteira sem remover pode dar $O(n^2)$; use marcação por timestamp.
- **Euleriano** → graus pares; caminho usando cada aresta uma vez → Hierholzer.
  - *Gatilho:* "usar cada aresta exatamente uma vez", "circuito", "desenhar sem levantar a caneta".
  - *Por que funciona:* cada passagem por um vértice usa duas arestas, então todos os graus são pares (circuito) ou exatamente dois são ímpares (trilha); em componente conexa, essa condição é suficiente.
  - *Como fazer:* confira graus e conectividade dos vértices com aresta; Hierholzer iterativo empilhando arestas e concatenando no retorno; O(n + m).
  - *Exemplo:* "roteiro que percorre cada rua uma vez"; construção de de Bruijn.
  - *Armadilha:* vértices isolados não contam para a conectividade; no direcionado a condição é in = out; em multigrafo remova arestas por id.
- **Grau ≤ 2** → caminhos e ciclos; grau alto com m pequeno → no máximo $2\sqrt{m}$ vértices com grau ≥ $\sqrt{m}$ (contar triângulos em $m\sqrt{m}$).
  - *Gatilho:* "cada vértice tem grau no máximo 2", "muitos vértices e poucas arestas", "contar triângulos".
  - *Por que funciona:* grau ≤ 2 faz cada componente ser caminho ou ciclo; em geral, há no máximo $2m/\sqrt{m} = 2\sqrt{m}$ vértices com grau ≥ $\sqrt{m}$, então enumerar só esses e seus vizinhos custa $O(m\sqrt{m})$.
  - *Como fazer:* para grau ≤ 2, siga a partir de grau 1 ou percorra ciclos; para triângulos, oriente arestas do menor grau para o maior e verifique arestas entre as saídas; $O(m\sqrt{m})$.
  - *Exemplo:* classificar componentes como caminhos/ciclos; contar triângulos em grafo esparso.
  - *Armadilha:* a cota $2\sqrt{m}$ é assintótica; sem orientação, enumerar pares de vizinhos custa $\Sigma \deg^2$ e pode ser $O(n^3)$.

adicionado em 2026-11-10 por agentes; revisado

- **Dijkstra com pesos negativos só nas fontes/arestas de saída.** Se só há pesos negativos em arestas que saem de uma super-fonte, some constantes e rode Dijkstra; em geral, potenciais h(v) (Johnson) tornam $w'(u,v) = w + h(u) - h(v) \geq 0$ e dão todos-os-pares em $n\cdot(m \log n)$ com um Bellman-Ford inicial.
  - *Gatilho:* pesos negativos apenas nas arestas que saem de uma fonte/super-fonte, "bônus inicial", "desconto por escolher a origem", ou todos-os-pares com algumas arestas negativas.
  - *Por que funciona:* todo caminho a partir da super-fonte usa exatamente uma aresta negativa; somar a mesma constante C a todas elas desloca todas as distâncias pelo mesmo C, sem mudar a ordem. Com potenciais, $w'(u,v) = w + h(u) - h(v) \geq 0$ pela desigualdade triangular $h(v) \leq h(u) + w$.
  - *Como fazer:* no caso da super-fonte, inicialize dist[v] = w(s,v), coloque todos na fila e rode Dijkstra (o deslocamento é implícito). Para Johnson, um Bellman-Ford inicial dá h(v) = dist(s,v); rode n Dijkstras com pesos reduzidos e desfaça o potencial; $O(nm + n\cdot m \log n)$.
  - *Exemplo:* super-fonte com arestas de custo $-c_i$ seguidas de arestas normais; todos-os-pares em grafo com poucas arestas negativas.
  - *Armadilha:* se houver peso negativo em aresta que não sai da mesma fonte, o deslocamento constante não existe; Johnson exige que o Bellman-Ford inicial não encontre ciclo negativo; use long long.
- **Dijkstra lazy vs. decrease-key.** Com $m \approx n^2$ (grafo denso) use Dijkstra $O(n^2)$ com vetor, sem heap; com m pequeno, heap com pares e ignore entradas velhas (d > dist[v]).
  - *Gatilho:* escolher a implementação pelo tamanho de m; "n ≤ 5000 e grafo completo".
  - *Por que funciona:* no denso, achar o mínimo entre os não visitados custa O(n) por extração e $O(n^2)$ no total, melhor que O(m log n); no esparso, o heap evita varrer todos os vértices e entradas velhas são descartadas.
  - *Como fazer:* denso: vetor de visitados, para n iterações ache o não visitado de menor dist em O(n) e relaxe seus vizinhos; $O(n^2)$. Esparso: priority_queue de pares; ao extrair (d, u), ignore se d > dist[u]; O(m log n).
  - *Exemplo:* n = 5000 com $m \approx 25\cdot10^6$ favorece $O(n^2)$; $m \approx n \log n$ favorece heap.
  - *Armadilha:* com heap, não marque visitado antes de checar d > dist[u] (poderia fixar uma distância velha); a versão $O(n^2)$ não precisa de decrease-key.

- **Dijkstra com custo "máximo" ou "bottleneck".** Minimizar o máximo de arestas no caminho (ou maximizar o mínimo) → Dijkstra com combinação max/min no lugar de soma (funciona porque é monotônica); equivale a caminho na MST (minimax).
  - *Gatilho:* "minimizar o maior peso do caminho", "maximizar a menor capacidade", "caminhão que passa com o menor peso máximo".
  - *Por que funciona:* max e min são monotônicos, então o rótulo "melhor gargalo até v" só melhora e a extração gulosa é ótima; é um Dijkstra num semianel (min, max) ou (max, min).
  - *Como fazer:* dist[v] = menor possível do máximo de aresta; relaxe dist[v] = min(dist[v], max(dist[u], w)); heap; O(m log n). Para maximizar o mínimo, use max(dist[v], min(dist[u], w)).
  - *Exemplo:* "qual o caminhão mais leve que consegue ir de A a B" → minimax.
  - *Armadilha:* não use soma; o critério não é aditivo. Se o problema mistura soma e gargalo, o estado precisa carregar as duas coordenadas.
- **Dijkstra com custo multiplicativo.** Maximizar produto de probabilidades → Dijkstra com −log, ou direto com max-heap (probabilidades ≤ 1 mantêm a monotonicidade).
  - *Gatilho:* "maximizar a probabilidade de sucesso", "produto de fatores", "confiabilidade de uma rota".
  - *Por que funciona:* log é monótono e transforma produto em soma; com $0 < p \le 1$, $-\log p \ge 0$, então o Dijkstra usual vale. Alternativamente, um max-heap com produtos funciona porque multiplicar por $p \le 1$ só diminui.
  - *Como fazer:* aresta com peso −log(p); rode Dijkstra e exponencie no fim; ou guarde prod[v] num max-heap e relaxe prod[v] = max(prod[v], prod[u]·p); O(m log n).
  - *Exemplo:* rota mais confiável numa rede com probabilidades por trecho.
  - *Armadilha:* p = 0 vira log infinito; trate separadamente. Em double, compare com folga ou use a versão multiplicativa para evitar erro.
- **Dijkstra em grafo implícito.** Estado = (posição, algo) sem listar arestas: gere vizinhos na hora; se o grafo é denso e implícito (ex.: i → j com custo |a_i − a_j|), só os vizinhos na ordem por valor importam, ou use segtree/set para pegar o próximo não visitado.
  - *Gatilho:* "de i posso ir a qualquer j com custo f(i,j)", "grafo completo implícito", "não liste as arestas".
  - *Por que funciona:* no exemplo do custo |a_i − a_j|, ir de i a j passando pelos valores intermediários ordenados tem o mesmo custo (telescopa), então as arestas para o anterior e o próximo na ordem bastam; em geral, só os candidatos que podem ser ótimos precisam ser gerados.
  - *Como fazer:* ordene por a_i e ligue cada i ao anterior e ao próximo; rode Dijkstra; O(n log n). Se a condição é "qualquer não visitado que satisfaz P", use set ou segtree para extrair em O(log n).
  - *Exemplo:* mover-se entre pontos da reta com custo |a_i − a_j|.
  - *Armadilha:* gerar todos os pares é $O(n^2)$; a justificativa de "só vizinhos" depende da desigualdade triangular, não vale para custo arbitrário.
- **Dijkstra com "k arestas grátis" / "k cupons".** Estado (v, j) com j ≤ k → n·k vértices; se o enunciado diz k pequeno é isto; se k grande e custo convexo em k, Aliens.
  - *Gatilho:* "k cupons", "k arestas grátis", "no máximo k vezes", "k teleportes".
  - *Por que funciona:* o número de cupons usados é a única memória que falta; (v, j) é estado suficiente, e usar um cupom é uma transição de custo 0 que avança j.
  - *Como fazer:* camadas (v, j), j = 0..k; aresta normal (u, j) → (v, j) com peso w; cupom (u, j) → (v, j + 1) com peso 0; Dijkstra em n(k + 1) vértices; O(km log(kn)). Com k grande e f(k) convexa, penalize cada cupom com λ e faça binária em λ (Aliens).
  - *Exemplo:* "menor tempo usando no máximo k teleportes".
  - *Armadilha:* "exatamente k" ≠ "no máximo k"; se f(k) não for convexa, Aliens falha; k grande sem convexidade pode exigir outra modelagem.
- **Dijkstra com tempo de espera / janelas.** Chegar mais cedo nunca piora (propriedade FIFO) → Dijkstra por tempo de chegada; sem FIFO, o problema fica NP-difícil em geral.
  - *Gatilho:* "ônibus partem em horários", "posso esperar", "janela [a, b]", "próxima partida".
  - *Por que funciona:* se chegar mais cedo nunca leva a chegar mais tarde (f(t) = t + espera é não decrescente), o menor tempo de chegada domina os outros e a extração gulosa é correta.
  - *Como fazer:* dist = menor instante de chegada; ao relaxar uma aresta com horários, ache por busca binária a próxima partida ≥ dist[u], some a duração; heap por tempo; O(m log n log H).
  - *Exemplo:* "menor hora de chegada usando trens com horários".
  - *Armadilha:* sem FIFO (chegar antes pode implicar espera maior) o problema é NP-difícil; "no máximo k esperas" acrescenta uma camada ao estado.

- **Dijkstra com contagem de caminhos mínimos.** Mantenha cnt[v]; ao relaxar com igualdade some cnt[u]; para "arestas que pertencem a algum caminho mínimo" use d_s[u] + w + d_t[v] = d(s,t).
  - *Gatilho:* "quantos caminhos mínimos", "arestas que estão em algum caminho mínimo", "rotas ótimas".
  - *Por que funciona:* no DAG das arestas que satisfazem d[u] + w = d[v], caminhos mínimos são caminhos comuns; cnt[v] soma as contribuições dos predecessores nessa ordem. A igualdade d_s[u] + w + d_t[v] = d(s,t) caracteriza aresta em algum caminho mínimo.
  - *Como fazer:* Dijkstra guardando cnt; se d[u] + w < d[v], cnt[v] = cnt[u]; se for igual, cnt[v] += cnt[u]. Para arestas, rode Dijkstra de s e de t no reverso e teste a igualdade em cada aresta; O(m log n).
  - *Exemplo:* contar caminhos mínimos em grade; achar as arestas críticas de todos os caminhos mínimos.
  - *Armadilha:* com pesos 0, a ordem de processamento importa (ciclos de custo 0); processe por ordem topológica das distâncias ou condense; use módulo se pedido.
- **Segundo menor caminho / k-ésimo caminho (walks).** Dijkstra deixando cada vértice ser extraído até k vezes (walks, não simples); para caminhos simples use Yen (k·n Dijkstras) ou Eppstein para walks em O(m + n log n + k log k).
  - *Gatilho:* "segundo menor caminho", "k-ésimo caminho", "rota alternativa".
  - *Por que funciona:* permitir que cada vértice seja extraído k vezes enumera os k menores rótulos de passeio, podendo repetir vértices; para caminhos simples é preciso excluir repetições, o que Yen faz removendo arestas/vértices.
  - *Como fazer:* heap com (dist, v); conte quantas vezes cada v foi extraído; a j-ésima extração de v é o j-ésimo menor passeio; O(km log(kn)). Yen: k − 1 iterações, cada uma O(n(m + n log n)). Eppstein: O(m + n log n + k log k).
  - *Exemplo:* "2º menor caminho de 1 a n" quando pode repetir vértices.
  - *Armadilha:* "caminho" no enunciado pode significar simples; a versão de k extrações conta passeios; para k grande, Eppstein e cuidado com memória.
- **Caminho mínimo com exatamente k arestas** → matriz min-plus elevada a k (O(n^3 log k)) ou DP por camadas em O(k·m).
  - *Gatilho:* "exatamente k arestas", "k passos", "matriz de adjacência".
  - *Por que funciona:* (A^k)_{ij} conta passeios de i a j com k arestas; no semianel min-plus, a potência dá o menor custo com exatamente k arestas, e a associatividade permite exponenciação.
  - *Como fazer:* matriz n×n com min-plus; potência binária O(n^3 log k); ou DP camada por camada O(k·m) se k é pequeno. Para "no máximo k", inclua a opção de parar (diagonal 0) ou some as potências.
  - *Exemplo:* "caminho de 1 a n com exatamente k arestas".
  - *Armadilha:* min-plus não tem inverso; a identidade é 0 na diagonal e ∞ fora; não confunda com a matriz booleana (alcançabilidade).
- **Caminho mínimo com paridade/cores.** Estado (v, paridade do número de arestas/uma cor) dobra o grafo; "caminho de comprimento ímpar" é BFS em (v, bit).
  - *Gatilho:* "comprimento par/ímpar", "alternar cores", "número de arestas módulo 2".
  - *Por que funciona:* a paridade é uma coordenação finita; duplicar o grafo em (v, 0) e (v, 1) com arestas que invertem o bit transforma o critério em alcançabilidade.
  - *Como fazer:* 2n vértices; cada aresta u → v vira (u, b) → (v, 1 − b); BFS de (s, 0); "caminho ímpar" = alcançar (t, 1); O(n + m). Para c cores, use c camadas.
  - *Exemplo:* "existe passeio de comprimento ímpar entre u e v?" equivale a não ser bipartido no componente.
  - *Armadilha:* passeio ≠ caminho simples; com pesos, o estado dobra o grafo e o custo; a paridade do número de arestas não é a paridade da soma dos pesos.
- **Caminho de mínimo custo que alterna tipos de aresta ou limita trocas** → estado (v, último tipo); se há "no máximo k trocas", camadas.
  - *Gatilho:* "alternar entre dois tipos de aresta", "no máximo k trocas de tipo", "voos e trens", "baldeações".
  - *Por que funciona:* o último tipo usado é a memória mínima para decidir se a próxima transição é permitida; com limite de trocas, o número de trocas é uma camada adicional.
  - *Como fazer:* estado (v, tipo), ou (v, tipo, j) para ≤ k trocas; mesma aresta custa 0 trocas, tipo diferente custa 1; 0-1 BFS ou Dijkstra; O((n·T·k + m·k) log).
  - *Exemplo:* "mínimo de baldeações" com tipo = linha.
  - *Armadilha:* o estado inicial pode precisar de "sem tipo" ou de inicializar todos os tipos com 0; contar a primeira escolha como troca é erro comum.

- **0-1 BFS com custo de "virar".** Grid com setas: custo 0 seguir a seta, 1 mudar → 0-1 BFS; também para "mínimo de arestas para inverter" e "mínimo de mudanças de direção" (estado (célula, direção)).
  - *Gatilho:* grid com setas, "custo 0 seguir, 1 mudar", "mínimo de inversões", "mínimo de curvas".
  - *Por que funciona:* os custos são 0/1, então a deque ordena; para curvas, a direção atual é parte do estado, senão não se sabe se virar custa.
  - *Como fazer:* estado (célula, direção); seguir na mesma direção custa 0, mudar custa 1; 0-1 BFS com deque; O(4nm). Para "inverter setas", estado só (célula), custo 0 na seta e 1 nas outras três.
  - *Exemplo:* "girar setas para ir de A a B com o mínimo de mudanças".
  - *Armadilha:* BFS comum não lida com custos diferentes; sem a direção no estado, o algoritmo subestima as curvas.
- **BFS multi-fonte em grid com "tempo de propagação".** Fogo que se espalha + pessoa que foge: BFS do fogo dá t_fogo[v]; pessoa só pode entrar em v se chega estritamente antes. Para o último vértice, relaxe a condição (≤).
  - *Gatilho:* "fogo se espalha", "tempo de propagação", "fugir antes do fogo", "inundação".
  - *Por que funciona:* o tempo mínimo do fogo em cada célula é uma BFS multi-fonte independente; a condição de segurança é local: a pessoa pode estar em v no instante t sse t < t_fogo[v].
  - *Como fazer:* BFS do fogo para t_fogo[v]; BFS/Dijkstra da pessoa com a regra dist[u] + 1 < t_fogo[v] para entrar; se o enunciado permitir escapar no mesmo instante, use ≤ apenas na célula final; O(nm).
  - *Exemplo:* pessoa em grid com várias fontes de fogo.
  - *Armadilha:* a igualdade no destino depende do enunciado; verifique a célula inicial; se a pessoa pode esperar, o estado muda.
- **Bellman-Ford em n passos detecta ciclo negativo.** Se a n-ésima iteração ainda relaxa, há ciclo negativo alcançável; para saber quais vértices têm dist = −∞, propague dos relaxados na última rodada.
  - *Gatilho:* "detectar ciclo negativo", "distância −∞", "custo que melhora indefinidamente".
  - *Por que funciona:* sem ciclo negativo, caminhos mínimos simples têm no máximo n − 1 arestas; se a n-ésima rodada ainda relaxa, há ciclo negativo alcançável, e todos os vértices alcançáveis a partir dele têm dist = −∞.
  - *Como fazer:* n − 1 rodadas de relaxamento; guarde os vértices relaxados na rodada n; BFS/DFS no grafo a partir deles (ou mais n rodadas) para marcar −∞; O(nm).
  - *Exemplo:* arbitragem de moedas com log de câmbio.
  - *Armadilha:* o ciclo precisa ser alcançável da fonte; para todos os pares, use super-fonte com arestas 0; cuidado com INF + peso negativo (overflow).
- **Ciclo negativo explícito.** Guarde pai ao relaxar; vértice relaxado na rodada n → ande n passos pelos pais para garantir cair dentro do ciclo, depois volte até repetir.
  - *Gatilho:* "imprima o ciclo negativo", "sequência de arestas que melhora o custo".
  - *Por que funciona:* se v é relaxado na rodada n, o caminho de pais até v tem mais de n vértices, logo repete; andar n passos para trás garante entrar no ciclo, e continuar até repetir o vértice fecha o ciclo.
  - *Como fazer:* pai[v] = u ao relaxar; ache x relaxado na rodada n; recue n vezes; colete até voltar a x; inverta a ordem; O(nm).
  - *Exemplo:* detectar e imprimir um ciclo de custo negativo.
  - *Armadilha:* recuar menos de n passos pode parar antes do ciclo; o ciclo de 2 vértices precisa das duas arestas; inverta a ordem ao imprimir.
- **SPFA** é só heurística (pior caso Θ(nm)); em problemas com anti-SPFA use Dijkstra com potenciais ou Bellman-Ford camadas.
  - *Gatilho:* "Bellman-Ford mais rápido na prática", "fila de relaxamento".
  - *Por que funciona:* SPFA só melhora constantes na prática; não tem garantia melhor que O(nm) e existem grafos anti-SPFA construídos para forçar o pior caso.
  - *Como fazer:* fila de vértices com dist atualizada; conte relaxamentos por vértice (> n sugere ciclo negativo); se os pesos permitem, prefira Dijkstra; com negativos, Bellman-Ford ou potenciais.
  - *Exemplo:* arbitragem pequena; verificação rápida de ciclo negativo.
  - *Armadilha:* pior caso Θ(nm); anti-SPFA é comum em juízes; não use como substituto de Dijkstra.

- **Floyd-Warshall como "fecho" de qualquer semianel.** min-plus (menor caminho), or-and (fecho transitivo), max-min (gargalo), max-× (confiabilidade); a ordem dos laços k externo é essencial.
  - *Gatilho:* "todos os pares", "fecho transitivo", "composição de relações", "gargalo entre todos".
  - *Por que funciona:* o laço k externo fixa os intermediários permitidos em {1..k}; a relaxação d[i][j] = d[i][j] ⊕ d[i][k] ⊗ d[k][j] é a composição do semianel, e por indução cobre todos os caminhos.
  - *Como fazer:* três laços com k externo; $O(n^3)$. Instancie ⊕/⊗: min/+ para caminho, or/and para alcançabilidade, max/min para gargalo, max/× para confiabilidade.
  - *Exemplo:* fecho transitivo booleano; menor gargalo entre todos os pares.
  - *Armadilha:* k tem de ser o laço mais externo; inicialize a diagonal com a identidade do semianel (0 ou 1, conforme); ∞/0 adequados.
- **Floyd com adição incremental de vértices.** Adicionar vértice k e atualizar todas as distâncias em $O(n^2)$; "remova vértices numa ordem, soma das distâncias" → inverta e adicione ($O(n^3)$ total).
  - *Gatilho:* "remova vértices em ordem", "some as distâncias após cada remoção", "adicione vértices um a um".
  - *Por que funciona:* ao adicionar o vértice k, o novo menor caminho entre i e j ou é o antigo ou passa por k; a atualização d[i][j] = min(d[i][j], d[i][k] + d[k][j]) para todos os pares é $O(n^2)$.
  - *Como fazer:* inverta a ordem de remoção; mantenha a matriz; ao adicionar k, calcule as distâncias via k para todos os pares ativos e some; $O(n^3)$ total.
  - *Exemplo:* "após remover cada vértice, some as distâncias entre os restantes".
  - *Armadilha:* vértices ainda não adicionados não podem servir de intermediários; some apenas pares ativos; a ordem inversa é obrigatória.
- **Floyd com ciclo mínimo.** Ao processar k, antes de relaxar pares via k, o menor ciclo cujo maior índice é k é min(d[i][j] + w(j,k) + w(k,i)); gira ciclo mínimo (peso) em $O(n^3)$.
  - *Gatilho:* "menor ciclo", "ciclo de peso mínimo", "girth ponderado".
  - *Por que funciona:* antes de usar k como intermediário, d[i][j] só usa vértices < k; então d[i][j] + w(j,k) + w(k,i) é o menor ciclo cujo maior índice é k, e todo ciclo tem um maior índice.
  - *Como fazer:* para k = 1..n: primeiro atualize a resposta com d[i][j] + w[j][k] + w[k][i] para i, j < k; depois relaxe d via k; $O(n^3)$.
  - *Exemplo:* menor ciclo em grafo direcionado ponderado.
  - *Armadilha:* responder antes de relaxar é essencial; arestas paralelas e laços precisam de tratamento; no não direcionado, cuidado para não usar a mesma aresta duas vezes.
- **Ciclo mínimo em grafo não pesado (girth).** BFS de cada vértice e, para toda aresta não-da-árvore (u,v), candidato d[u] + d[v] + 1; O(nm). Girth ≥ 3 e m > n·√n garante ciclo de comprimento ≤ 4 (Kővári–Sós–Turán: sem C4 ⇒ m = O(n^{3/2})).
  - *Gatilho:* "menor ciclo", "girth", "grafo sem C4", "ciclo curto garantido".
  - *Por que funciona:* numa BFS de raiz r, toda aresta não-árvore (u, v) fecha um ciclo de comprimento d[u] + d[v] + 1; o mínimo sobre todas as raízes e arestas é o girth. Pelo KST, um grafo sem C4 tem O(n^{3/2}) arestas.
  - *Como fazer:* BFS de cada vértice; para cada aresta não-árvore, candidato d[u] + d[v] + 1; O(nm). Para detectar C4, BFS de profundidade 2 ou bitsets.
  - *Exemplo:* achar o menor ciclo em grafo esparso não pesado.
  - *Armadilha:* arestas de árvore não contam; arestas paralelas formam ciclo de comprimento 2; a cota m > n√n é assintótica (constantes para n pequeno).
- **Ciclo mínimo em grafo grande esparso com girth pequeno.** Se m ≥ n^{1+1/k} há ciclo de comprimento ≤ 2k (Bondy–Simonovits): BFS limitada a profundidade k acha.
  - *Gatilho:* "m grande mas grafo esparso", "garantir ciclo curto", "girth pequeno", "achar C4/C6".
  - *Por que funciona:* Bondy–Simonovits diz que um grafo sem C_{2k} tem O(k·n^{1+1/k}) arestas; com arestas suficientes existe ciclo de comprimento ≤ 2k, e uma BFS limitada à profundidade k o encontra.
  - *Como fazer:* BFS de cada vértice até profundidade k; uma aresta não-árvore vista dentro da profundidade fecha ciclo de comprimento ≤ 2k; O(n·m) no pior caso, viável para k pequeno e grafo esparso.
  - *Exemplo:* achar C4 ou C6 num grafo com densidade acima do limiar.
  - *Armadilha:* a forma exata do teorema tem constante que depende de k (ex(n, C_{2k}) = O(k·n^{1+1/k})); a BFS precisa limitar a profundidade, senão perde o ganho.

- **Ciclo de comprimento ímpar** existe ⇔ não bipartido; ache por BFS: aresta entre dois vértices de mesma paridade de nível fecha ciclo ímpar via LCA na árvore BFS.
  - *Gatilho:* "ciclo ímpar", "não bipartido", "conflito de paridade".
  - *Por que funciona:* na BFS, uma aresta entre vértices de mesmo nível (mesma paridade) fecha um ciclo de comprimento ímpar; se não existe nenhuma, a própria BFS é uma 2-coloração e o grafo é bipartido.
  - *Como fazer:* BFS com níveis; para cada aresta não-árvore, se nível[u] ≡ nível[v] (mod 2), o ciclo é ímpar; para reconstruir, suba ao LCA na árvore BFS; O(n + m).
  - *Exemplo:* detectar se um grafo é bipartido e exibir o ciclo ímpar.
  - *Armadilha:* a aresta pode ligar níveis que diferem de 1; a paridade é o que importa; no direcionado, ignore a direção para ciclo não direcionado.
- **Ciclo par em grafo com min grau ≥ 3** sempre existe; todo grafo com grau mínimo ≥ k tem caminho de comprimento ≥ k e ciclo de comprimento ≥ k + 1 (DFS: último vértice da trilha maximal tem todos vizinhos na trilha).
  - *Gatilho:* "grau mínimo ≥ 3", "garantir ciclo par", "caminho longo garantido".
  - *Por que funciona:* no caminho maximal v_0..v_l, o último vértice tem todos os vizinhos no caminho; com grau ≥ 3, há três vizinhos, e dois deles têm a mesma paridade de distância, cuja diferença fecha um ciclo par. Com grau ≥ k, há pelo menos k vizinhos e um ciclo de comprimento ≥ k + 1.
  - *Como fazer:* pegue um caminho mais longo (DFS); analise as posições dos vizinhos do extremo; para o ciclo par, escolha dois vizinhos de mesma paridade; O(n + m).
  - *Exemplo:* provar que todo grafo cúbico tem ciclo par.
  - *Armadilha:* o argumento exige caminho maximal e grafo simples; laços e multiedges mudam a contagem de grau.
- **Base de ciclos (espaço de ciclos GF(2)).** Dimensão m − n + c; cada aresta fora da árvore geradora dá um ciclo fundamental; "xor máximo de ciclo/caminho" → base linear dos xors dos ciclos fundamentais + xor na árvore.
  - *Gatilho:* "xor de ciclos", "combinar ciclos", "espaço vetorial sobre GF(2)".
  - *Por que funciona:* o espaço de ciclos tem dimensão m − n + c; cada aresta fora de uma floresta geradora define um ciclo fundamental, e todo ciclo é xor deles; uma base linear resume todos os xors possíveis.
  - *Como fazer:* ache uma spanning forest; para cada aresta não-árvore, calcule o xor do caminho na árvore mais a aresta; insira na base linear; O((m − n + c)·B), B = número de bits.
  - *Exemplo:* "xor máximo de um ciclo" → maximize sobre a base.
  - *Armadilha:* use floresta para grafo desconexo; a base é sobre GF(2) (xor), não sobre inteiros; para caminhos s→t inclua o xor do caminho na árvore.
- **Xor mínimo/máximo de passeio s→t em grafo qualquer** = xor da árvore s→t xor algum elemento do span dos ciclos fundamentais.
  - *Gatilho:* "xor de um passeio s→t", "maximizar/minimizar xor", "caminho com xor".
  - *Por que funciona:* qualquer passeio s→t difere do caminho da árvore por um conjunto de ciclos; o xor de um ciclo é combinação dos fundamentais, então o conjunto de xors possíveis é x_0 ⊕ span.
  - *Como fazer:* base linear dos ciclos fundamentais; x_0 = xor do caminho s→t na árvore; maximize/minimize x_0 ⊕ span por eliminação gaussiana; O(B·dim).
  - *Exemplo:* "maior xor de um caminho de 1 a n" com pesos nas arestas.
  - *Armadilha:* o passeio pode repetir arestas; se o enunciado pede caminho simples, o conjunto não é o span; a base só vale para passeios.
- **Quantas arestas remover para virar árvore/floresta** = m − n + c; arestas que fecham ciclo no Kruskal são as redundantes.
  - *Gatilho:* "quantas arestas remover para virar árvore", "arestas redundantes", "deixar acíclico".
  - *Por que funciona:* uma floresta com n vértices e c componentes tem n − c arestas; remover m − (n − c) é necessário e suficiente; as arestas rejeitadas pelo Kruskal são exatamente as que fecham ciclo.
  - *Como fazer:* rode Kruskal; toda aresta cujos extremos já estão na mesma componente é redundante; conte m − n + c; O(m α).
  - *Exemplo:* "mínimo de estradas a fechar para não haver ciclo".
  - *Armadilha:* c é o número de componentes do grafo original; remover vértices muda a fórmula; arestas paralelas contam como ciclos de comprimento 2.

- **Ciclo único (m = n numa componente)** → pseudofloresta: ache o ciclo com DSU ou removendo folhas por grau; árvores penduradas viram DP.
  - *Gatilho:* "m = n", "uma componente com exatamente um ciclo", "pseudofloresta".
  - *Por que funciona:* numa componente conexa com n vértices e n arestas o posto do ciclo é 1, então há exatamente um ciclo; o resto são árvores penduradas.
  - *Como fazer:* DSU para achar a aresta que fecha o ciclo; ou remova folhas (grau 1) iterativamente; o núcleo restante é o ciclo; faça DP nas árvores penduradas.
  - *Exemplo:* "cada componente tem no máximo um ciclo".
  - *Armadilha:* m = n por componente, não no grafo todo; se m > n há mais de um ciclo; use grau, não só a contagem.
- **Remover folhas iterativamente (peeling por grau 1)** → acha o ciclo único, o centro da árvore e vértices que não participam de ciclos; com grau ≤ k, k-core.
  - *Gatilho:* "remover folhas", "centro da árvore", "vértices em ciclos", "k-core".
  - *Por que funciona:* remover grau 1 repetidamente elimina exatamente as árvores penduradas; o que sobra é o 2-core (os ciclos); na árvore, o último vértice (ou aresta) é o centro.
  - *Como fazer:* fila de grau 1; remova e decremente vizinhos, guardando a ordem; O(n + m). Para k-core, remova grau < k (não ≤ k); O(n + m).
  - *Exemplo:* achar o centro de uma árvore; achar os vértices que pertencem a algum ciclo.
  - *Armadilha:* no k-core o critério é grau < k; usar ≤ k remove o próprio k-core; laços contam duas vezes no grau.
- **k-core / degenerescência.** Remova repetidamente o vértice de menor grau (fila de buckets, O(m)); a ordem dá orientação onde cada vértice tem ≤ d arestas de saída (d = degenerescência ≤ √(2m)); contagem de triângulos/cliques de tamanho fixo em O(m·d) (contar todas as cliques é #P-difícil).
  - *Gatilho:* "cada subgrafo tem vértice de grau baixo", "orientar para limitar grau de saída", "contar triângulos".
  - *Por que funciona:* a ordem de remoção do menor grau define a degenerescência d; orientando cada aresta do removido para o que fica, cada vértice tem ≤ d saídas; d ≤ √(2m) porque o d-core tem ≥ d + 1 vértices e ≥ d(d + 1)/2 arestas.
  - *Como fazer:* bucket queue por grau; remova o mínimo e decremente vizinhos; O(n + m). Para triângulos, oriente por (grau, id) e verifique pares de saídas; O(m·d) = O(m√m).
  - *Exemplo:* contagem de triângulos e de cliques de tamanho fixo.
  - *Armadilha:* d é a degenerescência, não o grau máximo; a orientação por degenerescência difere da orientação por grau; contar todas as cliques é #P-difícil.
- **Arboricidade ≤ √(2m)** e m ≤ a·(n − 1): orientar arestas do menor para o maior grau limita grau de saída a √(2m) → triângulos, C4 em O(m√m).
  - *Gatilho:* "grafo esparso por densidade", "orientar arestas", "m ≤ a(n − 1)", "contar C4".
  - *Por que funciona:* a arboricidade a = max_S ⌈m_S/(|S| − 1)⌉; orientar do menor para o maior grau limita a saída a O(√m), pois um vértice com muitas saídas teria muitos vizinhos de grau maior e a densidade estouraria.
  - *Como fazer:* ordene por grau (desempate por id); oriente cada aresta para o de maior grau; para triângulos/C4, use essa orientação e enumere caminhos de 2 arestas; O(m√m).
  - *Exemplo:* contar C4 em O(m√m).
  - *Armadilha:* a cota é assintótica; desempate por id para ordem total; a orientação por grau não é a de degenerescência, mas ambas dão O(√m).
- **Contar triângulos** → oriente cada aresta do menor (grau, id) para o maior; para cada u marque saídas e percorra saídas das saídas: O(m√m); com bitset, $O(n^3/64)$ ou O(n·m/64).
  - *Gatilho:* "contar triângulos", "cliques de tamanho 3".
  - *Por que funciona:* com a orientação por (grau, id), cada triângulo é contado uma vez no seu vértice de menor ordem, e o número total de caminhos de 2 arestas na orientação é O(m√m).
  - *Como fazer:* para cada u, marque os vizinhos de saída; para cada v de saída, percorra as saídas de v e conte as marcadas; desmarque; O(m√m). Bitset: $O(n^3/64)$ ou O(n·m/64).
  - *Exemplo:* número de triângulos em grafo esparso.
  - *Armadilha:* sem orientação o custo é $\Sigma \deg^2$; marque/desmarque em O(1) para não zerar o vetor inteiro; bitset troca tempo por memória.

- **Contar ciclos de comprimento 4** → para cada u, para cada v alcançável em 2 passos com ordem (grau, id) maior que u, some C(cnt[v], 2); O(m√m).
  - *Gatilho:* "contar ciclos de comprimento 4", "C4", "pares com dois vizinhos comuns".
  - *Por que funciona:* um C4 tem um vértice de menor ordem; contando pares de caminhos de 2 passos que chegam ao mesmo vértice, cada C4 é contado exatamente uma vez nesse vértice.
  - *Como fazer:* oriente por (grau, id); para cada u, para cada v alcançável em 2 passos com ordem > u, cnt[v]++; depois some C(cnt[v], 2); O(m√m).
  - *Exemplo:* contar C4 em grafo esparso.
  - *Armadilha:* zere cnt apenas para os vértices tocados; sem a ordem, cada ciclo é contado várias vezes; divida pelas simetrias se usar outro método.
- **Contar caminhos de comprimento 2 e ciclos pelo traço** → $tr(A^3)/6$ = triângulos; $tr(A^4)$ contém C4 mais termos de grau; matriz de adjacência elevada a k conta walks de comprimento k.
  - *Gatilho:* "contar walks", "traço da matriz", "potências da adjacência".
  - *Por que funciona:* (A^k)_{ij} conta passeios de comprimento k; $tr(A^3)$ = 6·triângulos, pois cada triângulo tem 6 ordens; $tr(A^4)$ inclui 8·C4 mais termos de grau.
  - *Como fazer:* multiplicação de matrizes $O(n^3 \log k)$; para k = 3, 4 use as fórmulas diretas; $tr(A^3)/6$ e $tr(A^4)$ com correções; $O(n^3)$.
  - *Exemplo:* contar triângulos por álgebra linear.
  - *Armadilha:* walk ≠ ciclo simples; $tr(A^4)$ não é só C4; para grafos grandes, prefira o método combinatório O(m√m).
- **Subgrafo induzido / conjunto independente máximo em grafo pequeno (n ≤ 40)** → meet in the middle ou Bron–Kerbosch com pivô (≈ 3^{n/3}); n ≤ 50 clique máxima → branch and bound com bitset.
  - *Gatilho:* "n ≤ 40", "conjunto independente máximo", "clique máxima", "subgrafo induzido".
  - *Por que funciona:* meet in the middle divide os vértices e combina soluções compatíveis; Bron–Kerbosch com pivô enumera cliques maximais em O(3^{n/3}), e bitsets tornam a poda barata.
  - *Como fazer:* n ≤ 40: divida em duas metades, para cada máscara da primeira guarde o melhor independente compatível; n ≤ 50 para clique: branch and bound com bitsets e poda por grau; O(3^{n/3}).
  - *Exemplo:* "escolher o máximo de vértices sem arestas entre eles" com n = 40.
  - *Armadilha:* memória 2^{n/2}; para n = 50, 2^25 máscaras é viável mas o pareamento precisa de ordenação; use bitset para vizinhança.
- **Coloração de grafos: grau máximo Δ ⇒ guloso usa ≤ Δ + 1 cores; Brooks: Δ cores se é conexo, Δ ≥ 3, e não é clique nem ciclo ímpar; Vizing: arestas em Δ ou Δ + 1; bipartido: arestas em exatamente Δ cores (König).**
  - *Gatilho:* "colorir vértices/arestas", "poucas cores", "conflitos", "agenda sem choques".
  - *Por que funciona:* o guloso nunca usa mais que Δ + 1 porque cada vértice tem no máximo Δ vizinhos; Brooks e Vizing refinam a cota; em bipartido, König garante Δ cores para arestas.
  - *Como fazer:*
    - Guloso: ordene os vértices (ex.: por grau decrescente) e atribua a menor cor livre; O(n + m), ≤ Δ + 1 cores.
    - Brooks: se o grafo é conexo, Δ ≥ 3 e não é clique nem ciclo ímpar, existe coloração com Δ cores.
    - Vizing: coloração de arestas de grafo simples usa Δ ou Δ + 1 cores; decidir qual é NP-difícil.
    - König (bipartido): coloração de arestas em exatamente Δ cores, construtiva por caminhos alternados.
  - *Exemplo:* escalonar jogos de um torneio (coloração de arestas de K_n).
  - *Armadilha:* Δ + 1 é cota do guloso, não o ótimo; Brooks exclui clique e ciclo ímpar; Vizing não dá Δ sempre.
- **Coloração de arestas de bipartido em Δ cores** → construtivo: para cada aresta (u,v) escolha cor livre a em u e b em v e troque ao longo do caminho alternado a/b (O(nm)); também por decomposição euleriana em potências de 2.
  - *Gatilho:* "colorir arestas de bipartido com Δ cores", "horários sem conflito", "rodadas de um torneio bipartido".
  - *Por que funciona:* König garante Δ cores; o algoritmo de troca ao longo do caminho alternado corrige o conflito sem aumentar Δ, e cada troca preserva a propriedade de coloração.
  - *Como fazer:* para cada aresta (u, v), ache cores livres a em u e b em v; se a = b, pinte; senão troque as cores a/b ao longo do caminho alternado que começa em u; O(nm). Alternativa: regularize e decomponha por circuitos eulerianos em matchings.
  - *Exemplo:* escalonar jogos de um torneio bipartido em Δ rodadas.
  - *Armadilha:* o caminho alternado pode terminar em v; implemente com cuidado; Δ é o máximo grau, não o número de arestas.

- **Grafo de intervalos** (vértice = intervalo, aresta = interseção): é cordal e perfeito; cor mínima = clique máxima = máximo de sobreposição (sweep); MIS = guloso por menor fim; cobertura de cliques = guloso.
  - *Gatilho:* "vértice = intervalo", "interseção", "reservar salas", "aulas com horários".
  - *Por que funciona:* interval graphs são cordais e perfeitos, então $\chi = \omega$; a clique máxima é o máximo de intervalos sobrepostos num ponto, e o MIS segue da troca "terminar antes".
  - *Como fazer:* sweep com eventos de início/fim para o máximo de sobreposição; para MIS, ordene por fim e pegue o que começa após o último fim; $O(n \log n)$. Cobertura de cliques = MIS, e o MIS é guloso por menor fim (dualidade em grafos perfeitos).
  - *Exemplo:* "mínimo de salas para todas as aulas" = máximo de sobreposição.
  - *Armadilha:* tocar no endpoint (fim = início) pode ou não contar como interseção; defina fechado/aberto; ordenar por início para salas e por fim para MIS.
- **Grafo cordal** (sem ciclo induzido ≥ 4) → ordem de eliminação perfeita por MCS/LexBFS; coloração gulosa na ordem reversa é ótima; clique máxima = max(|vizinhos posteriores| + 1).
  - *Gatilho:* "sem ciclo induzido ≥ 4", "ordem de eliminação perfeita", "grafo cordal".
  - *Por que funciona:* em uma PEO, os vizinhos posteriores de cada vértice formam clique; colorir na ordem reversa usa exatamente ω cores; MCS/LexBFS acham uma PEO em tempo linear.
  - *Como fazer:* rode MCS/LexBFS; se algum vértice tem vizinhos posteriores não adjacentes, não é cordal; coloração gulosa na ordem reversa; clique máxima = max(|posteriores| + 1); $O(n + m)$.
  - *Exemplo:* reconhecer grafos de intervalos (cordais com propriedade extra).
  - *Armadilha:* PEO não é ordem topológica; a coloração ótima é na ordem reversa; verificar a propriedade é parte do algoritmo.
- **Grafos de permutação** (aresta i–j se $i < j$ e $p_i > p_j$): conjunto independente máximo = LIS; clique máxima = subsequência decrescente máxima; cor mínima = LDS (por Dilworth/Mirsky).
  - *Gatilho:* "permutação", "inversões", "aresta se $i < j$ e $p_i > p_j$".
  - *Por que funciona:* uma clique é uma subsequência decrescente; um independente é uma subsequência crescente; por Dilworth, colorir é particionar em subsequências crescentes, e o mínimo é o comprimento da maior decrescente.
  - *Como fazer:* LIS/LDS com paciência em $O(n \log n)$; MIS = LIS; clique máxima = LDS; cor mínima = LDS.
  - *Exemplo:* "máximo de pares invertidos dois a dois".
  - *Armadilha:* a convenção da aresta pode ser $p_i < p_j$; troque LIS por LDS se o enunciado inverter; subsequência ≠ subarray.
- **Dilworth.** Antichain máxima = mínimo de cadeias que cobrem o poset = $n -$ matching máximo no fecho transitivo (cobertura de caminhos com vértices repetíveis); para vértices disjuntos num DAG, $n -$ matching direto nas arestas.
  - *Gatilho:* "antichain máxima", "cobrir com o mínimo de cadeias", "poset", "elementos incomparáveis".
  - *Por que funciona:* Dilworth iguala a largura ao mínimo de cadeias; via matching no grafo de comparabilidade, a cobertura mínima de caminhos é $n - \nu$; cada caminho é uma cadeia.
  - *Como fazer:* feche transitivamente; bipartido com cópias L/R; aresta $i_L \to j_R$ se $i < j$; matching máximo ν; resposta $n - \nu$; construtivo seguindo os pares; $O(n^3)$ ou Hopcroft–Karp.
  - *Exemplo:* "mínimo de sequências para cobrir todos os elementos".
  - *Armadilha:* para cobertura por caminhos com vértices disjuntos no DAG original, use as arestas diretas; para cadeias (comparabilidade), use o fecho; não misture.
- **Mirsky (dual).** Cadeia mais longa = mínimo de antichains para particionar = nível do vértice no DAG (profundidade máxima); em problema "particione em o mínimo de grupos sem relações" é o comprimento da maior cadeia.
  - *Gatilho:* "particionar em grupos sem relações", "cadeia mais longa", "dual de Dilworth".
  - *Por que funciona:* Mirsky: o comprimento da maior cadeia é o mínimo de antichains para particionar; no DAG, é a profundidade (maior caminho a partir da fonte), e os níveis formam antichains.
  - *Como fazer:* DP de maior caminho no DAG; nível[v] = 1 + max nível dos predecessores; os níveis são os grupos; $O(n + m)$.
  - *Exemplo:* "mínimo de grupos tal que nenhum par do grupo é comparável".
  - *Armadilha:* é o dual de Dilworth; se o problema pede cadeias, use Dilworth; uma aresta entre vértices do mesmo nível indica que a orientação/ordem está errada.

- **Antichain máxima a partir de König.** Dado o matching máximo no grafo bipartido (L, R) do fecho, tome a cobertura mínima C; antichain = vértices i com nem i_L nem i_R em C.
  - *Gatilho:* "construir a antichain máxima", "certificado", "não só o tamanho".
  - *Por que funciona:* por König, a cobertura mínima C do bipartido do fecho tem tamanho ν; os vértices com ambas as cópias fora de C formam uma antichain de tamanho n − ν.
  - *Como fazer:* matching máximo; BFS alternado de L não casados; C = (L não visitados) ∪ (R visitados); antichain = {i : i_L ∉ C e i_R ∉ C}; O(n + m).
  - *Exemplo:* obter a antichain, não só o seu tamanho.
  - *Armadilha:* a construção depende da convenção do BFS; inverter L/R muda a fórmula; use o fecho transitivo.
- **Cobertura mínima de caminhos em DAG** (disjuntos em vértices) → n − matching bipartido (out-copy, in-copy); com vértices podendo ser compartilhados, tome fecho transitivo antes.
  - *Gatilho:* "cobrir todos os vértices com o mínimo de caminhos", "vértices disjuntos", "DAG".
  - *Por que funciona:* cada caminho escolhe no máximo um sucessor por vértice; o matching seleciona esses sucessores, e o número de caminhos é n menos o número de pares escolhidos.
  - *Como fazer:* bipartido com L = out-copy, R = in-copy; aresta u_L → v_R para cada aresta u → v; matching máximo ν; resposta n − ν; reconstrua seguindo os pares; $O(n^3)$ ou $O(m\sqrt{n})$.
  - *Exemplo:* "mínimo de caminhos para visitar todos os vértices de um DAG".
  - *Armadilha:* se vértices podem ser compartilhados, feche transitivamente antes; caminhos disjuntos em arestas é outro problema; não confunda com repetição de vértices.
- **König.** Matching máximo = cobertura mínima de vértices (bipartido); conjunto independente máximo = n − matching. Construção: BFS alternado a partir de L não casados; cobertura = (L não visitados) ∪ (R visitados).
  - *Gatilho:* "matching máximo = cobertura mínima", "bipartido", "conjunto independente".
  - *Por que funciona:* König: ν = τ no bipartido; o BFS alternado separa os vértices alcançáveis por caminhos alternados e a união indicada cobre todas as arestas.
  - *Como fazer:* matching máximo; BFS alternado a partir de L não casados (arestas não-matching L → R, matching R → L); C = (L não visitados) ∪ (R visitados); MIS = V \ C; O(n + m).
  - *Exemplo:* "mínimo de vértices para cobrir todas as arestas".
  - *Armadilha:* a fórmula troca conforme o lado inicial do BFS; para pesos, König não vale (é vertex cover ponderado); matching geral não é König.
- **Hall.** Existe matching saturando L ⇔ $|N(S)| \geq |S|$ $\forall S \subseteq L$; deficiência $\max_S(|S| - |N(S)|) = |L|$ − matching máximo. Aparece em "existe distribuição válida?" com restrições de vizinhança.
  - *Gatilho:* "existe distribuição válida", "cada grupo precisa de tantas opções", "restrições de vizinhança", "todos conseguem vaga?".
  - *Por que funciona:* Hall diz que saturar L exige e basta que todo subconjunto S tenha vizinhança pelo menos do tamanho de S; a deficiência máxima é exatamente |L| − ν, então o matching certifica a resposta.
  - *Como fazer:* na prática rode matching e compare |L| − ν; para checar Hall diretamente, só com n pequeno (2^n subconjuntos); com capacidades > 1, generalize as multiplicidades; $O(m\sqrt{n})$.
  - *Exemplo:* "n alunos, cada um aceita um conjunto de vagas; todos conseguem?".
  - *Armadilha:* Hall satura L, não garante matching perfeito dos dois lados; a deficiência é $\max_S(|S| - |N(S)|)$; capacidades exigem fluxo, não o Hall básico.
- **Kuhn com heurística.** Ordem aleatória das arestas, inicialização gulosa e reset de visited só quando o DFS acha caminho aumentante: praticamente $O(m\sqrt{n})$; para $n \leq 10^5$ prefira Hopcroft–Karp $O(m\sqrt{n})$.
  - *Gatilho:* "matching bipartido", "$n \leq 10^5$", "atribuir".
  - *Por que funciona:* cada DFS procura caminho aumentante; com ordem aleatória e matching guloso inicial, poucos augmentos ocorrem na prática; Hopcroft–Karp tem a garantia $O(m\sqrt{n})$.
  - *Como fazer:* Kuhn: para cada u, DFS com visited; use timestamp para limpar em O(1) (a variante do bullet só reinicia o visited após um augmento bem-sucedido); embaralhe arestas e faça matching guloso inicial; Hopcroft–Karp: BFS de camadas + DFS de bloqueio; $O(m\sqrt{n})$.
  - *Exemplo:* atribuir tarefas a máquinas.
  - *Armadilha:* a heurística não garante $O(m\sqrt{n})$ no pior caso; resetar visited a cada DFS é O(n) por vértice; em grafos densos, use bitset.

- **Matching bipartido em grafo denso com bitset.** Kuhn onde "próximo vizinho não visitado" usa bitset de não visitados: $O(n^3/64)$.
  - *Gatilho:* "bipartido denso", "n ≤ 2000–5000", "muitas arestas".
  - *Por que funciona:* o conjunto de não visitados cabe em bitset; achar o próximo candidato é uma operação de 64 bits, então o DFS inteiro fica O(n/64) por vértice.
  - *Como fazer:* adj[u] como bitset; no DFS, candidatos = adj[u] & ~visited; pegue o próximo bit; visited |= bit; $O(n^3/64)$, memória $O(n^2/64)$.
  - *Exemplo:* matching em grafo quase completo.
  - *Armadilha:* n = 5000 dá ~3 MB de bitsets; zere visited por DFS com timestamp; bitset de adjacência precisa ser construído com cuidado.
- **Matching em grafo bipartido com vizinhança = intervalo** (cada L conecta a [l_i, r_i] de R) → guloso: varra R em ordem, heap por menor r_i; também para "emparelhar tarefas/janelas".
  - *Gatilho:* "cada tarefa aceita uma janela [l_i, r_i]", "emparelhar com recursos", "agendar".
  - *Por que funciona:* entre as tarefas disponíveis, a de menor r é a mais urgente; deixá-la para depois nunca ajuda (argumento de troca).
  - *Como fazer:* ordene os recursos R crescentes; adicione ao heap as tarefas com l_i ≤ R; escolha a de menor r_i; se r_i < R, falhou; O((n + m) log n).
  - *Exemplo:* "máximo de tarefas agendadas em máquinas".
  - *Armadilha:* ordenar por l e escolher a de menor r não é o mesmo que ordenar por r; use heap; intervalos fechados vs abertos.
- **Matching máximo ponderado em bipartido (atribuição)** → Hungarian $O(n^3)$; "maximize" → negar custos; para custo mínimo com k pares exatos use mincost flow com k augmentações (cada uma é caminho mínimo, custo convexo em k).
  - *Gatilho:* "atribuir com custo", "maximizar lucro", "cada um a um", "k pares".
  - *Por que funciona:* Hungarian resolve o LP de atribuição em $O(n^3)$ via potenciais; mincost flow generaliza para capacidades e número exato de pares, e os custos marginais são não decrescentes.
  - *Como fazer:* matriz de custos; Hungarian com potenciais; maximize → negue os custos; k pares exatos → mincost flow com k unidades; $O(n^3)$ / $O(k \cdot m \log n)$.
  - *Exemplo:* "n tarefas, n máquinas, custo c_ij".
  - *Armadilha:* matriz não quadrada → dummies de custo 0; custos negativos são ok no Hungarian; para k exato, não pare antes de k augmentações.
- **Casamento estável (Gale–Shapley).** Propostos pelo lado que propõe → ótimo para quem propõe, pior possível para o outro lado; dá solução única do lado dos pedintes mesmo com empates quebrados em ordem fixa.
  - *Gatilho:* "cada um tem preferências", "casamento estável", "sem par que prefira trocar".
  - *Por que funciona:* Gale–Shapley termina com matching estável; o lado que propõe recebe o melhor par estável possível, e o outro recebe o pior; o ótimo dos proponentes é único.
  - *Como fazer:* fila de proponentes; cada um propõe na ordem de preferência; se o receptor está livre ou prefere o novo, troca e o antigo volta à fila; $O(n^2)$.
  - *Exemplo:* residência médica.
  - *Armadilha:* empates exigem desempate fixo; a unicidade é do ótimo dos proponentes, não do matching em si; o lado que recebe não é ótimo.
- **Matching geral (não bipartido)** → Blossom $O(n^3)$ / Edmonds; quando o grafo é uma árvore, guloso de folhas; tamanho do máximo pela fórmula de Tutte–Berge ($n - 2\nu = \max_S(\text{odd}(G - S) - |S|)$).
  - *Gatilho:* "matching em grafo geral", "não bipartido", "flores", "pares disjuntos".
  - *Por que funciona:* Blossom trata ciclos ímpares contraindo-os; em árvore, o guloso de folhas é ótimo; Tutte–Berge dá o tamanho máximo sem construir o matching.
  - *Como fazer:* Edmonds/Blossom $O(n^3)$; em árvore, emparelhe folhas com o pai; fórmula $n - 2\nu = \max_S(\text{odd}(G - S) - |S|)$.
  - *Exemplo:* "máximo de pares disjuntos" em grafo qualquer.
  - *Armadilha:* Kuhn/Hopcroft–Karp não valem fora de bipartido; Blossom é difícil de implementar; para árvore, use o guloso.

- **Pareamento de arestas em caminhos de comprimento 2** → um grafo conexo com m arestas pode ser particionado em ⌊m/2⌋ caminhos de comprimento 2 (DFS pós-ordem emparelhando arestas pendentes no vértice); se m é par, tudo.
  - *Gatilho:* "particionar arestas em caminhos de comprimento 2", "parear arestas", "P2".
  - *Por que funciona:* na DFS pós-ordem, ao subir, emparelham-se as arestas pendentes no vértice; cada par forma um P2; sobra no máximo uma aresta, que sobe para o pai.
  - *Como fazer:* DFS; em cada vértice, emparelhe as arestas de filhos ainda não usadas; se sobrar uma, passe-a para o pai; O(n + m). Resultado: ⌊m/2⌋ P2 e, se m é ímpar, uma aresta solta.
  - *Exemplo:* decompor um grafo conexo em P2.
  - *Armadilha:* a implementação gulosa precisa emparelhar no vértice certo; arestas de árvore e de retorno contam igual; com m ímpar não há cobertura completa por P2.
- **Orientação de arestas com grau de saída/entrada limitado** → fluxo (aresta → vértice com capacidade) ou Hall; "orientar para que todo grau de saída seja par/dado" → Euler após adicionar vértice auxiliar.
  - *Gatilho:* "orientar arestas com grau de saída ≤ k", "balancear entrada/saída", "grau par".
  - *Por que funciona:* a orientação é uma escolha binária por aresta; fluxo modela a capacidade de cada vértice; Hall dá a condição de viabilidade; para paridade, o vértice auxiliar fecha os graus ímpares.
  - *Como fazer:* rede com aresta → vértice e capacidades; para grau de saída par, ligue os ímpares a um vértice auxiliar e oriente um circuito euleriano; O(maxflow).
  - *Exemplo:* "orientar para que cada vértice tenha no máximo k saídas".
  - *Armadilha:* limitar entrada e saída ao mesmo tempo é mais restritivo; verifique a soma dos graus; viabilidade é fluxo, não guloso.
- **Orientar grafo 2-aresta-conexo para ficar fortemente conexo** (Robbins): orientação da DFS (árvore para baixo, retorno para cima); ponte ⇒ impossível.
  - *Gatilho:* "orientar grafo para ficar fortemente conexo", "sem pontes", "mão única mantendo alcançabilidade".
  - *Por que funciona:* Robbins: um grafo conexo tem orientação fortemente conexa sse é 2-aresta-conexo; a DFS orienta arestas de árvore para baixo e de retorno para cima, e todo vértice alcança a raiz e é alcançado por ela.
  - *Como fazer:* verifique pontes; DFS; oriente arestas de árvore u → v e arestas de retorno v → u (para o ancestral); O(n + m).
  - *Exemplo:* "transformar ruas em mão única mantendo alcançabilidade".
  - *Armadilha:* a orientação da DFS só funciona sem pontes; arestas paralelas contam como 2-aresta-conexão; laços não ajudam.
- **Circulação com demandas** → fonte S → v com cap d_v para excedentes, v → T para déficits; viável ⇔ fluxo máximo satura tudo. Limite inferior l na aresta: demanda l, capacidade c − l.
  - *Gatilho:* "demandas por vértice", "fluxo com oferta e procura", "balanço", "limites inferiores".
  - *Por que funciona:* demandas viram arestas de S/T; a viabilidade é um fluxo que satura as demandas; um limite inferior l pode ser subtraído do fluxo, virando demanda e reduzindo a capacidade para c − l.
  - *Como fazer:* excedente d → S → v cap d; déficit → v → T cap |d|; maxflow; viável sse satura; para lower l, ajuste demandas e capacidade c − l; O(maxflow).
  - *Exemplo:* "distribuir água com oferta e demanda".
  - *Armadilha:* o sinal da demanda é fácil de inverter; a soma das demandas deve ser 0; limites inferiores mudam o valor do fluxo.
- **Fluxo com limites inferiores, fluxo máximo s→t:** adicione aresta t→s de capacidade ∞, teste viabilidade, depois rode maxflow s→t no resíduo (sem a aresta t→s) e some o fluxo inicial (valor em t→s).
  - *Gatilho:* "fluxo máximo com capacidade mínima em cada aresta", "lower bounds".
  - *Por que funciona:* a aresta t → s de capacidade ∞ transforma o fluxo s → t em circulação; após viabilizar, o fluxo em t → s é o valor inicial, e o residual sem essa aresta permite aumentar.
  - *Como fazer:* adicione t → s ∞; rode a viabilidade com S/T; se viável, guarde f = fluxo em t → s; remova t → s; rode maxflow s → t no residual; resposta f + aumento; O(maxflow).
  - *Exemplo:* "fluxo com mínimo de envio por tubo".
  - *Armadilha:* não esqueça de somar o fluxo inicial; remover t → s é essencial para não reaproveitar o ciclo; some as demandas dos limites inferiores corretamente.

- **Menger.** Máximo de caminhos disjuntos em arestas (vértices) = corte mínimo de arestas (vértices); disjuntos em vértices → divida v em v_in → v_out de capacidade 1 para v ∉ {s, t} (s e t com capacidade ∞).
  - *Gatilho:* "caminhos disjuntos", "corte mínimo", "vértices disjuntos", "quantos caminhos independentes".
  - *Por que funciona:* Menger iguala o máximo de caminhos aresta-disjuntos ao corte mínimo de arestas; para vértices, a divisão v_in → v_out de capacidade 1 transforma "usar o vértice" em "usar a aresta".
  - *Como fazer:* arestas cap 1; maxflow = número de caminhos; para vértices, v_in → v_out cap 1 para v ∉ {s, t}, s/t ∞; arestas originais u_out → v_in; O(maxflow).
  - *Exemplo:* "quantos caminhos independentes de A a B".
  - *Armadilha:* s e t não podem ser divididos com cap 1; no direcionado, cap 1 na aresta; reconstrua os caminhos pelos fluxos.
- **k caminhos disjuntos de custo mínimo** → mincost flow com k unidades (capacidade 1); "ir e voltar" visitando cada ponto uma vez = 2 caminhos disjuntos.
  - *Gatilho:* "k caminhos disjuntos de custo mínimo", "ir e voltar", "rota de ida e volta".
  - *Por que funciona:* mincost flow com k unidades e capacidade 1 nas arestas escolhe k caminhos disjuntos; o custo é aditivo e os caminhos mínimos sucessivos mantêm a otimalidade.
  - *Como fazer:* rede com cap 1 e custo w; envie k unidades por caminhos mínimos sucessivos; O(k·m log n). "Ida e volta" = 2 caminhos disjuntos (ou s → t e t → s no direcionado).
  - *Exemplo:* "dois caminhos disjuntos de s a t minimizando a soma".
  - *Armadilha:* cap 1 em cada aresta, não em cada vértice; se os caminhos não podem compartilhar vértices, divida os vértices também.
- **Corte mínimo em grid** (pintar célula preto/branco com custo de vizinhos diferentes, pixels, segmentação) → cada célula é vértice, penalidade de vizinhança como arestas bidirecionais; células fixas ligadas a S/T.
  - *Gatilho:* "pintar células preto/branco", "segmentação de imagem", "custo de vizinhos diferentes".
  - *Por que funciona:* cada célula é uma variável binária; o custo de aresta entre vizinhos diferentes é submodular e vira aresta de corte; células fixas viram ligações de capacidade infinita a S/T.
  - *Como fazer:* S = preto, T = branco; aresta entre vizinhos com capacidade c (nos dois sentidos); célula fixa preta: S → v ∞; branca: v → T ∞; mincut; O(maxflow).
  - *Exemplo:* "separar objeto do fundo com penalidade de fronteira".
  - *Armadilha:* a capacidade da aresta de vizinhança é o custo de cortar; custo assimétrico exige duas direções; o corte não é o mesmo que colorir com guloso.
- **Seleção com custo de par "se escolher a e b juntos"** (projetos, lucro − custo) → closure: lucro positivo de S, custo para T, dependência com ∞; resposta = soma lucros − corte mínimo.
  - *Gatilho:* "projetos com lucro e custo", "dependências", "se escolher A precisa de B".
  - *Por que funciona:* seleção fechada (se v então pré-requisitos) é um corte: lucro positivo de S, custo para T, dependência ∞; o corte paga exatamente o lucro perdido.
  - *Como fazer:* S → v cap lucro (se > 0); v → T cap custo (se < 0); dependência u → v ∞; resposta = Σ lucros − mincut; O(maxflow).
  - *Exemplo:* "máximo lucro escolhendo projetos com pré-requisitos".
  - *Armadilha:* dependência na direção errada muda o problema; custos negativos viram arestas para T; some só os lucros positivos.
- **Seleção com bônus de pares (x_i, x_j diferentes custa c)** → corte mínimo onde variável é 0/1 = lado de S/T; custos tipo c(x_i ≠ x_j) são submodulares e se resolvem; c(x_i = x_j) em geral não (a menos que bipartido: inverta um lado).
  - *Gatilho:* "bônus se escolher os dois", "custo se diferentes", "x_i ≠ x_j".
  - *Por que funciona:* funções submodulares de duas variáveis são representáveis por corte; c·[x_i ≠ x_j] é submodular e vira aresta entre i e j; c·[x_i = x_j] é supermodular, mas em grafo bipartido inverter um lado a transforma em desigualdade.
  - *Como fazer:* variável 0/1 = lado de S/T; termo c·[x_i ≠ x_j] = aresta de capacidade c entre i e j; termo de igualdade: se o grafo é bipartido, inverta a variável de um lado; O(maxflow).
  - *Exemplo:* "economia de escala ao escolher ambos".
  - *Armadilha:* termos de igualdade em grafo não bipartido não são representáveis; não force; para bônus "se ambos", reescreva como custo penalizado.

- **Atribuição com custo submodular em 3+ variáveis** → ainda dá corte mínimo se a função for submodular e grafo-representável (decomponível em termos; ex.: "ao menos um"); grafo com vértice auxiliar para cada termo.
  - *Gatilho:* "custo de combinações de 3 variáveis", "ao menos um", "termos de ordem alta".
  - *Por que funciona:* nem toda função submodular de ordem ≥ 3 é representável por grafo, mas termos decomponíveis (por exemplo, recompensa por "ao menos um") podem ser modelados com vértices auxiliares.
  - *Como fazer:* decomponha o termo em uma soma de termos representáveis; para cada termo crie um nó auxiliar e arestas de capacidade; verifique a tabela-verdade; O(maxflow).
  - *Exemplo:* "penalidade se nenhuma das 3 opções for escolhida".
  - *Armadilha:* submodularidade é necessária mas não suficiente para representação por grafo; teste os 2^3 valores; não invente arestas sem verificar.
- **Máximo closure / densidade máxima (Goldberg).** Subgrafo de densidade máxima m_S/|S| → binária em g + corte mínimo (m_S − g|S| ≥ 0); arestas viram vértices ou nó por vértice.
  - *Gatilho:* "subgrafo mais denso", "densidade máxima", "escolher subconjunto maximizando arestas/vértices".
  - *Por que funciona:* para um chute g, existe S com m_S − g|S| ≥ 0 sse a densidade máxima é pelo menos g; esse teste é um problema de corte mínimo (Goldberg).
  - *Como fazer:* binária em g; construa a rede de Goldberg (nós para arestas ou por vértice) e calcule o corte mínimo; se o corte indicar lucro ≥ 0, aumente g; precisão $1/n^2$ ou busca racional; O(log(1/ε)·maxflow).
  - *Exemplo:* "achar o subgrafo com maior razão arestas/vértices".
  - *Armadilha:* a rede é específica; a resposta é uma razão, não um conjunto; cuidado com precisão e com o caso de subgrafo vazio.
- **Fluxo máximo em unidade (capacidades 1)** → Dinic é O(min(V^{2/3}, E^{1/2})·E); em bipartido com capacidade 1 é O(E√V) (igual a Hopcroft–Karp).
  - *Gatilho:* "capacidades unitárias", "matching", "fluxo em grafo esparso".
  - *Por que funciona:* com cap 1, cada fase do Dinic aumenta o caminho mínimo e o número de fases é O(√E); no bipartido, é O(√V), igual ao Hopcroft–Karp.
  - *Como fazer:* Dinic com BFS de níveis e DFS de bloqueio; limite O(min(V^{2/3}, E^{1/2})·E); bipartido unitário O(E√V).
  - *Exemplo:* matching máximo em grafo bipartido grande.
  - *Armadilha:* o ganho só vale com capacidades unitárias; com capacidades grandes o limite é $O(V^2E)$; não confunda com o caso geral.
- **Escolha em dois lados com capacidades (ex.: n alunos, m clubes, cada um ≤ k)** → S → alunos (1) → clubes (cap) → T; restrições de "no máximo/ao menos" por grupo dão camadas extras de vértices.
  - *Gatilho:* "alunos e clubes", "cada clube aceita k", "cotas", "grupos".
  - *Por que funciona:* fluxo com capacidades modela cotas; restrições por grupo viram camadas de vértices com capacidade; o maxflow decide a alocação.
  - *Como fazer:* S → aluno cap 1; aluno → clube cap 1; clube → T cap k; para cotas de grupo, insira nó de grupo com capacidade; maxflow; O(maxflow).
  - *Exemplo:* "alocar alunos em clubes com capacidade".
  - *Armadilha:* "ao menos" não é capacidade simples; use limites inferiores ou complemento; some as capacidades para checar viabilidade.
- **Min-cost flow com capacidades pequenas e custo ≥ 0** → SPFA/Dijkstra com potenciais; com fluxo F pequeno, F augmentações; custos negativos sem ciclo negativo: Bellman-Ford inicial.
  - *Gatilho:* "custo mínimo com fluxo", "capacidades pequenas", "k augmentações".
  - *Por que funciona:* successive shortest paths mantém otimalidade se não há ciclo negativo; com custos ≥ 0, Dijkstra com potenciais; com negativos, um Bellman-Ford inicial calcula os potenciais.
  - *Como fazer:* h(v) = dist inicial; pesos reduzidos w' = w + h(u) − h(v) ≥ 0; Dijkstra por augmentação; atualize h; O(F·m log n).
  - *Exemplo:* "transporte com custo por unidade".
  - *Armadilha:* custos negativos sem Bellman-Ford inicial quebram Dijkstra; ciclo negativo torna o problema ilimitado; cuidado com capacidades 0/∞.

- **Mincost flow com custo convexo por aresta** → divida a aresta em k cópias de capacidade 1 com custos crescentes (ou cap/custo por segmento linear).
  - *Gatilho:* "custo crescente por unidade", "desconto por volume", "custo convexo".
  - *Por que funciona:* uma aresta com custo convexo f(x) equivale a k arestas de capacidade 1 com custos marginais f(i) − f(i − 1), que são não decrescentes; o fluxo usa as mais baratas primeiro.
  - *Como fazer:* expanda cada aresta em k cópias cap 1 com custos marginais; rode mincost flow; O(k·m log n). Para custo linear por faixas, idem.
  - *Exemplo:* "produzir x unidades com custo quadrático".
  - *Armadilha:* se o custo não é convexo, as marginais não são ordenadas e a expansão não vale; a capacidade total é k; cuidado com custo fixo.
- **Dualidade fluxo/corte para "mínimo de remoção".** Remover o menor número de vértices/arestas para desconectar s de t = corte mínimo; com pesos nos vértices, divida v em in/out.
  - *Gatilho:* "remover o mínimo para desconectar", "corte", "vértices com peso", "isolar A de B".
  - *Por que funciona:* max-flow min-cut; para vértices, dividir v em v_in → v_out com capacidade = peso transforma "remover vértice" em "cortar aresta".
  - *Como fazer:* arestas originais cap ∞; v_in → v_out cap w_v; maxflow de s_out a t_in; o corte mínimo dá o conjunto; O(maxflow).
  - *Exemplo:* "mínimo de cidades a destruir para isolar A de B".
  - *Armadilha:* s e t não podem ser removidos (cap ∞); no não direcionado, cada aresta vira duas direções; corte de vértices ≠ corte de arestas.
- **2-SAT: contagem.** SCC dá uma atribuição; "quantas variáveis forçadas" = x forçado a falso se x → ¬x, a verdadeiro se ¬x → x (ambos ⇒ insatisfatível); "menor lexicográfica" = tente x=0 e propague (O(n·m) ou com SCC tricks); "contar soluções" é #P em geral (só vale para casos especiais).
  - *Gatilho:* "quantas atribuições", "variável forçada", "lexicograficamente menor", "contar soluções".
  - *Por que funciona:* SCC dá uma atribuição válida; x é forçado a falso sse x → ¬x (a implicação obriga ¬x), e a verdadeiro sse ¬x → x; ambos os sentidos tornam insatisfatível. Contar soluções é #P em geral.
  - *Como fazer:* Tarjan; para cada x, teste alcançabilidade x → ¬x ou ¬x → x (ou use as SCCs); lexicográfica: tente x = 0, propague, refaça; O(n·m) ou SCC incremental.
  - *Exemplo:* "menor atribuição lexicográfica que satisfaz".
  - *Armadilha:* a ordem de Tarjan não dá diretamente "forçado"; use alcançabilidade; contar soluções só em casos especiais (ex.: componentes independentes).
- **2-SAT com cardinalidade "no máximo uma de k"** → prefix variables: p_i = x_1 ∨ ... ∨ x_i, cláusulas x_i → p_i, p_i → p_{i+1}, p_{i} → ¬x_{i+1}, usando O(k) cláusulas em vez de $O(k^2)$.
  - *Gatilho:* "no máximo uma de k", "no máximo k verdadeiros", "cardinalidade" junto com restrições 2-SAT.
  - *Por que funciona:* p_i significa "algum de x_1..x_i é verdadeiro"; as cláusulas forçam a monotonicidade de p e impedem dois x verdadeiros, com O(k) cláusulas em vez de $O(k^2)$ pares.
  - *Como fazer:* crie p_1..p_k; cláusulas (¬x_i ∨ p_i), (¬p_{i−1} ∨ p_i) e (¬p_i ∨ ¬x_{i+1}); para "no máximo k" generalize com um contador; O(k) variáveis e cláusulas.
  - *Exemplo:* "escolher no máximo um horário por dia".
  - *Armadilha:* a codificação muda para "exatamente um"; p_i precisa da base correta; não use $O(k^2)$ cláusulas pairwise se k é grande.
- **2-SAT com escolha de 2 opções por item** (pintar/escolher uma de duas posições/horários) e conflitos entre pares → vértice por opção; conflito (a,b) vira a → ¬b e b → ¬a.
  - *Gatilho:* "cada item tem 2 opções", "conflitos entre pares", "escolher uma de duas posições".
  - *Por que funciona:* cada opção é um literal; um conflito (a, b) significa que não podem ser escolhidos juntos, então a → ¬b e b → ¬a; o 2-SAT resolve todas as escolhas simultaneamente.
  - *Como fazer:* duas variáveis por item (opção 0/1); para cada conflito adicione as duas implicações; para "exatamente uma opção", acrescente (a ∨ b) e (¬a ∨ ¬b); SCC; O(n + m).
  - *Exemplo:* "cada pessoa escolhe manhã ou tarde; alguns pares são incompatíveis".
  - *Armadilha:* "exatamente uma opção" precisa das duas cláusulas; conflito entre opções do mesmo item vira cláusula unitária; não confunda opção com variável.

- **2-SAT online/binária**: "maior t tal que as primeiras t restrições são satisfatíveis" (ou o menor t insatisfatível) → binária em t ou adicionar incremental com verificação.
  - *Gatilho:* "maior prefixo de restrições satisfazível", "adicionar restrições uma a uma".
  - *Por que funciona:* adicionar cláusulas só restringe; a satisfatibilidade é monótona (verdadeira até um ponto, falsa depois), então a busca binária em t é válida.
  - *Como fazer:* binária em t; para cada teste construa o 2-SAT com as t primeiras e rode SCC; O(log n·(n + m)). Incremental: adicione e verifique as SCCs afetadas.
  - *Exemplo:* "a partir de que restrição fica impossível?".
  - *Armadilha:* a monotonicidade vale para adição, não para remoção; reconstruir o grafo a cada teste custa O(n + m); para muitas queries, SCC incremental.
- **SCC: ordem topológica de Tarjan** devolve componentes em ordem topológica reversa; condensação com fontes/sumidouros: mínimo de arestas para tornar fortemente conexo = max(#fontes, #sumidouros) (se há > 1 componente).
  - *Gatilho:* "condensar SCC", "ordem topológica", "tornar fortemente conexo", "fontes e sumidouros".
  - *Por que funciona:* Tarjan devolve as SCCs na ordem reversa da topológica (sorvedouros primeiro); a condensação é um DAG; para torná-lo fortemente conexo, cada fonte precisa de uma entrada e cada sorvedouro de uma saída, e max(fontes, sumidouros) basta.
  - *Como fazer:* Tarjan/Kosaraju; conte fontes e sorvedouros na condensação; se > 1 componente, resposta max(fontes, sumidouros); O(n + m).
  - *Exemplo:* "mínimo de arestas para o grafo virar fortemente conexo".
  - *Armadilha:* se já é fortemente conexo, a resposta é 0; a ordem de Kosaraju é a oposta de Tarjan; ignore arestas dentro da mesma SCC.
- **Alcançabilidade em DAG com n ≈ $5 \cdot 10^4$ e bitset** → fecho transitivo em O(n·m/64) ou, com queries, bitsets em blocos de 64 fontes por vez para caber na memória.
  - *Gatilho:* "alcançabilidade em DAG", "n ≈ $5 \cdot 10^4$", "fecho transitivo", "quem alcança quem".
  - *Por que funciona:* processando em ordem topológica reversa, reach[v] = {v} ∪ OR reach[u] dos sucessores; bitsets fazem o OR em n/64 palavras.
  - *Como fazer:* bitset por vértice; ordem topológica reversa; OR dos sucessores; O(n·m/64) tempo e O(n^2/64) memória; com queries, processe 64 fontes por bloco.
  - *Exemplo:* "quantos vértices cada um alcança".
  - *Armadilha:* n = $5 \cdot 10^4$ dá ~39M palavras (~300 MB) se todos os bitsets; use blocos ou compressão; memória é o gargalo.
- **Contar pares (u, v) alcançáveis em DAG grande** → estimativa probabilística (min de hashes aleatórios por vértice, k rodadas) ou bitset por blocos; exato é difícil.
  - *Gatilho:* "contar pares alcançáveis", "DAG grande", "estimativa".
  - *Por que funciona:* min-hash estima a união/interseção de conjuntos de alcançabilidade: em cada rodada, o mínimo hash propagado identifica um elemento, e a fração de acertos estima o tamanho.
  - *Como fazer:* k rodadas com hash aleatório; para cada vértice, propague o mínimo na ordem topológica; estime a união; O(k(n + m)); bitsets por blocos para exato.
  - *Exemplo:* "número de pares (u, v) com caminho".
  - *Armadilha:* o erro cai com ~1/√k; hashes independentes; o mínimo deve ser propagado, não somado; para resposta exata, bitsets.
- **Dominator tree** (Lengauer–Tarjan / Cooper–Harvey): v é dominado por d se todo caminho s→v passa por d; em DAG, idom(v) = LCA dos predecessores na própria árvore, construindo em ordem topológica.
  - *Gatilho:* "todo caminho de s a v passa por d", "dominação", "vértices inevitáveis".
  - *Por que funciona:* em DAG, o dominador imediato de v é o LCA dos dominadores imediatos dos predecessores, que já foram calculados em ordem topológica; em geral, Lengauer–Tarjan mantém a árvore com union-find.
  - *Como fazer:* DAG: processe em ordem topológica; idom(v) = LCA dos predecessores na árvore de dominadores (binary lifting); geral: Lengauer–Tarjan O(m α) ou Cooper–Harvey iterativo.
  - *Exemplo:* "quais vértices são inevitáveis para chegar a t".
  - *Armadilha:* dominador é direcionado, diferente de articulação; a raiz domina todos; em DAG, os predecessores precisam ter idom calculado.

- **Condensação de componentes por pontes.** Contraia 2-aresta-conexas → árvore; "mínimo de arestas para tornar 2-aresta-conexo" = ⌈folhas/2⌉ (árvore de pontes com ≥ 2 vértices).
  - *Gatilho:* "contrair componentes 2-aresta-conexas", "árvore de pontes", "eliminar pontes".
  - *Por que funciona:* contrair cada componente 2-aresta-conexa deixa uma árvore cujas arestas são as pontes; adicionar arestas para eliminar pontes é conectar folhas, e cada aresta nova resolve duas folhas.
  - *Como fazer:* ache pontes (low > tin); contraia com DSU/DFS; monte a árvore; resposta ⌈folhas/2⌉ se a árvore tem ≥ 2 vértices; O(n + m).
  - *Exemplo:* "mínimo de estradas novas para não haver ponte".
  - *Armadilha:* árvore de 1 vértice não precisa de arestas; folhas = grau 1; grafo desconexo soma por componente.
- **Block-cut tree.** BCCs são vértices-bloco ligados aos cortes que pertencem a eles; caminho entre u e v passa por articulações da árvore; "u e v separados se remover w" = w é vértice de corte no caminho do block-cut tree.
  - *Gatilho:* "articulações", "BCC", "caminho entre u e v", "separados ao remover w".
  - *Por que funciona:* cada BCC vira um nó-bloco ligado às articulações que pertencem a ela; o caminho no block-cut tree representa exatamente por quais blocos e articulações se passa.
  - *Como fazer:* Tarjan de BCCs com pilha de arestas; construa a árvore; use LCA para caminhos; "separados ao remover w" = w é articulação no caminho; O(n + m).
  - *Exemplo:* "quantas articulações separam u e v".
  - *Armadilha:* vértice isolado ou aresta sem ciclo também é bloco; o número de nós pode ser O(n + m); BCC de arestas ≠ BCC de vértices.
- **Contar pares desconectados ao remover vértice v** → tamanhos das componentes pendentes na DFS (low[child] ≥ tin[v]) e o resto n − 1 − Σ; pares desconectados = C(n−1, 2) − Σ C(sz_i, 2).
  - *Gatilho:* "quantos pares ficam desconectados ao remover v", "impacto de cada vértice".
  - *Por que funciona:* os filhos com low[child] ≥ tin[v] definem componentes pendentes de tamanhos sz_i; o resto é n − 1 − Σsz_i; pares em componentes diferentes ficam separados, então subtraímos os pares internos de C(n−1, 2).
  - *Como fazer:* DFS com low e sz; para cada v, colete os sz_i; resposta C(n−1, 2) − Σ C(sz_i, 2); O(n + m).
  - *Exemplo:* "qual vértice, ao falhar, desconecta mais pares".
  - *Armadilha:* inclua o "resto" como uma componente; a raiz não tem resto; use long long (pares ~n^2); o vértice removido não conta.
- **Articulação ≠ ponte para multiarestas.** Em ponte com arestas paralelas, compare por id da aresta, não por vértice pai; ao testar articulação, a raiz é articulação só se tiver ≥ 2 filhos na DFS.
  - *Gatilho:* "multigrafo", "arestas paralelas", "pontes", "vértice de corte".
  - *Por que funciona:* uma aresta paralela é um caminho alternativo, então nenhuma das duas é ponte; comparar pelo id evita pular as duas ao mesmo tempo; a raiz só separa componentes se tiver dois ramos independentes.
  - *Como fazer:* na DFS, pule apenas a aresta de id igual ao pai; use low[child] > tin[v] para ponte e ≥ para articulação; raiz com ≥ 2 filhos; O(n + m).
  - *Exemplo:* dois vértices com duas arestas paralelas: nenhuma é ponte.
  - *Armadilha:* pular pelo vértice pai remove as paralelas e cria ponte falsa; laços não são pontes; cuidado com ids.
- **Pontes dinâmicas (só adições de arestas)** → mantenha a árvore de 2-aresta-conexas com DSU: nova aresta (u,v) contrai o caminho da árvore; amortizado O(n α) + travessia.
  - *Gatilho:* "arestas só são adicionadas", "pontes online", "componentes 2-aresta-conexas".
  - *Por que funciona:* a árvore de componentes 2-aresta-conexas só contrai; adicionar (u, v) elimina todas as pontes do caminho entre u e v, e cada aresta é contraída uma vez.
  - *Como fazer:* DSU para componentes; ao adicionar, suba de u e v até o LCA na árvore (ou use DSU de "próximo não contraído") e contraia; O(n α + m α) amortizado.
  - *Exemplo:* "após cada adição, quantas pontes existem".
  - *Armadilha:* só adições; remoções exigem outra estrutura; a contração deve atualizar a árvore; use LCA ou caminhada com DSU.

- **Euler em não direcionado:** todos graus pares (circuito) ou exatamente 2 ímpares (trilha, comece num); grafo (ignorando isolados) conexo. Para completar com o mínimo de arestas, pareie os ímpares (menor emparelhamento: Chinese postman, via Floyd + emparelhamento perfeito mínimo em ≤ 20 ímpares com bitmask).
  - *Gatilho:* "usar cada aresta uma vez", "circuito", "trilha", "desenhar sem levantar a caneta".
  - *Por que funciona:* cada passagem por um vértice usa duas arestas, então os graus são pares, exceto os extremos da trilha; em componente conexa, essa condição é suficiente; para o Chinese postman, duplicar arestas pareia os ímpares.
  - *Como fazer:* conte graus e verifique conectividade; Hierholzer iterativo; O(n + m). Para o Chinese postman, Floyd + matching perfeito mínimo nos ímpares (bitmask se ≤ 20).
  - *Exemplo:* "percurso que passa por todas as ruas"; "menor caminhada que cobre todas as arestas".
  - *Armadilha:* ignore vértices isolados na conectividade; multigrafo conta arestas; a trilha começa num ímpar, o circuito em qualquer vértice.
- **Euler em direcionado:** in = out em todo vértice (circuito); trilha com um vértice out − in = 1 (início) e um in − out = 1 (fim); conectividade fraca basta quando os graus fecham.
  - *Gatilho:* "cada aresta direcionada uma vez", "circuito dirigido", "trilha dirigida".
  - *Por que funciona:* cada aresta entra e sai, então in = out (circuito) ou há um desequilíbrio de +1 e −1 (trilha); a conectividade fraca basta quando os graus fecham.
  - *Como fazer:* conte in/out; verifique as diferenças; Hierholzer no direcionado; O(n + m); para trilha, comece no vértice com out − in = 1.
  - *Exemplo:* "roteiro dirigido que usa cada voo uma vez".
  - *Armadilha:* conectividade forte não é necessária; verifique que todos os vértices com aresta estão na mesma componente fraca; não inverta início/fim.
- **Sequência de de Bruijn** B(k, n) = circuito euleriano no grafo de vértices k^{n−1}; é cíclica de comprimento k^n; a string linear que contém todas as strings de comprimento n tem k^n + n − 1 caracteres; "senha" → de Bruijn.
  - *Gatilho:* "todas as strings de comprimento n", "senha que contém todas", "circuito euleriano".
  - *Por que funciona:* no grafo de de Bruijn, vértices são (n − 1)-uplas e arestas são n-uplas; um circuito euleriano visita cada aresta (cada string) uma vez; o circuito tem k^n arestas.
  - *Como fazer:* construa o grafo implicitamente; Hierholzer; para a string linear, acrescente os n − 1 primeiros caracteres ao final; comprimento k^n + n − 1; O(k^n).
  - *Exemplo:* "menor string que contém todos os códigos de 4 dígitos".
  - *Armadilha:* o circuito é cíclico; a versão linear precisa do sufixo; para k = 2, cuidado com a ordem dos bits; memória O(k^n).
- **Cobertura de arestas por mínimo de trilhas** = max(1, #vértices de grau ímpar / 2) por componente conexa com arestas; "desenhar sem levantar a caneta" = isto.
  - *Gatilho:* "mínimo de trilhas para cobrir arestas", "desenhar sem levantar a caneta".
  - *Por que funciona:* cada trilha tem no máximo dois extremos ímpares; um componente com 2k ímpares precisa de pelo menos k trilhas, e a construção com vértice auxiliar mostra que k bastam.
  - *Como fazer:* conte ímpares por componente; resposta max(1, ímpares/2) para componente com arestas; some por componente; O(n + m).
  - *Exemplo:* "mínimo de traços para desenhar um desenho".
  - *Armadilha:* componentes sem arestas não contam; se não há ímpares, 1 circuito; grafos desconexos somam por componente.
- **Decomposição de grafo em caminhos de comprimento par/ímpar** → Euler após adicionar vértice auxiliar ligado aos de grau ímpar, depois remova-o para cortar o circuito em trilhas.
  - *Gatilho:* "decompor em caminhos de comprimento par/ímpar", "trilhas", "vértice auxiliar".
  - *Por que funciona:* adicionar um vértice auxiliar ligado a todos os ímpares torna todos os graus pares; um circuito euleriano, ao remover o auxiliar, se parte em trilhas entre os vértices ímpares.
  - *Como fazer:* adicione x ligado a cada vértice ímpar; Hierholzer; remova x e corte o circuito nas ocorrências de x; as trilhas têm extremos ímpares; O(n + m).
  - *Exemplo:* decompor um grafo em trilhas mínimas.
  - *Armadilha:* a construção controla os extremos (ímpares), não o comprimento par/ímpar das trilhas; para comprimento prescrito, o problema é outro; se não há ímpares, o circuito inteiro é uma trilha.

- **Hamilton em n ≤ 20** → dp[mask][v] em O(2^n·n²) com bitmask; para contar ciclos hamiltonianos fixe o vértice 0; para n ≤ 25–30 e grafo esparso, meet in the middle.
  - *Gatilho:* "n ≤ 20", "visitar todos uma vez", "ciclo hamiltoniano", "caixeiro viajante".
  - *Por que funciona:* dp[mask][v] guarda o melhor custo para visitar exatamente mask e terminar em v; a transição acrescenta um vértice não visitado adjacente, cobrindo todos os caminhos simples.
  - *Como fazer:* dp[mask][v]; inicialize dp[1<<v][v]; transições para u ∉ mask adjacente; O(2^n·n²); para contar ciclos, fixe o vértice 0 e feche nele.
  - *Exemplo:* "caixeiro viajante com n = 18".
  - *Armadilha:* memória 2^n·n; para n = 20, ~20M estados; não conte o mesmo ciclo n vezes (fixe o início); meet in the middle para n até 30 esparso.
- **Hamilton em torneio:** todo torneio tem caminho hamiltoniano (inserção por busca binária/merge sort com comparação "u→v"); torneio fortemente conexo tem ciclo hamiltoniano (Camion).
  - *Gatilho:* "torneio", "caminho hamiltoniano", "fortemente conexo".
  - *Por que funciona:* em um torneio, ao inserir um vértice num caminho hamiltoniano existente, sempre há uma posição válida (as comparações mudam de sentido uma vez); se o torneio é fortemente conexo, Camion garante o ciclo.
  - *Como fazer:* insira cada vértice por busca binária/merge sort na ordem do caminho (comparação u → v); para o ciclo, ache um ciclo e insira os demais; O(n log n) comparações ou O(n²).
  - *Exemplo:* "ordenar jogadores de forma que cada um venceu o próximo".
  - *Armadilha:* torneio não é transitivo; a inserção precisa achar a posição, não ordenar; ciclo hamiltoniano exige forte conexão.
- **Torneio — score sequences (Landau).** Sequência de pontuações s_1 ≤ … ≤ s_n é válida ⇔ Σ_{i≤k} s_i ≥ C(k, 2) ∀k com igualdade em k = n; reconstrução gulosa (maior pontuação ganha dos de menor).
  - *Gatilho:* "sequência de pontuações de torneio", "é possível?", "reconstruir o torneio".
  - *Por que funciona:* Landau: ordenada, as somas parciais ≥ C(k, 2) e a igualdade em n são necessárias e suficientes; a construção gulosa realiza exatamente essas pontuações.
  - *Como fazer:* ordene; verifique as somas parciais; reconstrução: o de maior pontuação vence os de menor até completar; O(n²) ou O(n log n).
  - *Exemplo:* "dadas as vitórias, existe torneio?".
  - *Armadilha:* a ordem importa (crescente); a igualdade em n é obrigatória; pontuações fora de [0, n−1] invalidam.
- **Torneio: número de triângulos cíclicos** = C(n,3) − Σ C(w_i, 2) (w_i = vitórias); rei do torneio = vértice com distância ≤ 2 a todos (vértice de maior pontuação serve).
  - *Gatilho:* "triângulos cíclicos", "rei", "distância ≤ 2".
  - *Por que funciona:* cada tripla é transitiva ou cíclica; uma tripla é transitiva sse tem um vértice que vence os outros dois, contado por C(w_i, 2); o vértice de maior pontuação é rei.
  - *Como fazer:* conte w_i; cíclicos = C(n,3) − Σ C(w_i, 2); para rei, pegue o de maior w e verifique distância ≤ 2; O(n²) para verificar.
  - *Exemplo:* "quantos triângulos cíclicos tem o torneio".
  - *Armadilha:* C(w_i, 2) conta triplas com i como fonte; subtraia de C(n,3); rei não é necessariamente único.
- **Grafo funcional (outdeg = 1).** Componente = 1 ciclo + árvores; detecte ciclo com cores/tortoise–hare; k-ésimo sucessor → binary lifting ou "índice na rota + tamanho do ciclo"; contar quantos entram num ciclo → peeling por grau de entrada.
  - *Gatilho:* "cada vértice tem exatamente uma saída", "sucessor", "funcional".
  - *Por que funciona:* outdeg = 1 faz cada componente ter um ciclo com árvores apontando para ele; seguir é determinístico, então binary lifting acelera; peeling por indegree remove quem não está no ciclo.
  - *Como fazer:* detecte ciclos por cores ou tortoise–hare; calcule entrada no ciclo e posição; up[k][v]; para somas, sum[k][v]; O(n log K); peeling por indegree para contar entradas.
  - *Exemplo:* "onde chego após k saltos"; "quantos nós chegam a cada ciclo".
  - *Armadilha:* distinguir ciclo de árvore; k pode ser 10^18; o peeling remove quem não está no ciclo, não quem entra nele.

- **Permutação como grafo funcional** → ciclos disjuntos; "transformar p em q com mínimo de swaps" = n − #ciclos de $p^{-1}q$; p^k mantém ciclos de comprimento L em gcd(L, k) ciclos de L/gcd.
  - *Gatilho:* "permutação", "swaps mínimos", "p^k", "ciclos".
  - *Por que funciona:* uma permutação é um grafo funcional com todos os vértices em ciclos; trocar dois elementos em ciclos diferentes funde os ciclos, então o mínimo de swaps para p → q é n − #ciclos($p^{-1}q$).
  - *Como fazer:* decomponha $p^{-1}q$ em ciclos; resposta n − c; p^k: cada ciclo de comprimento L vira gcd(L, k) ciclos de L/gcd(L, k); O(n).
  - *Exemplo:* "mínimo de swaps para ordenar uma permutação".
  - *Armadilha:* a composição $p^{-1}q$ depende da convenção (aplicar p depois q); para ordenar, q = identidade e a resposta é n − #ciclos(p); k pode ser grande.
- **Grafos funcionais com pesos** → soma de k passos por binary lifting guardando soma; "primeira repetição / pré-período" = distância até o ciclo + posição.
  - *Gatilho:* "soma de k passos", "primeira repetição", "pré-período".
  - *Por que funciona:* binary lifting guarda destino e soma; a estrutura ciclo + árvores decompõe o tempo em pré-período e período, permitindo calcular a primeira repetição.
  - *Como fazer:* up[k][v], sum[k][v]; para k passos, decompose k em bits e acumule; primeira repetição = distância até o ciclo + tamanho do ciclo; O(n log K).
  - *Exemplo:* "soma dos valores visitados em k saltos".
  - *Armadilha:* overflow em k e nas somas; a soma do ciclo pode ser multiplicada; se k < pré-período, não use o ciclo.
- **Grafo bipartido por DSU com paridade** → relação "x e y diferentes/iguais" mantida com paridade ao pai; conflito = ciclo ímpar; serve para "adicionar arestas online e dizer se ainda é bipartido".
  - *Gatilho:* "x e y iguais/diferentes", "adicionar relações online", "bipartido".
  - *Por que funciona:* a paridade até o pai codifica a relação com a raiz; unir duas componentes com paridade consistente preserva a interpretação; uma contradição é exatamente um ciclo ímpar.
  - *Como fazer:* DSU com par[v] = paridade de v até parent; find comprime acumulando par; union ajusta a paridade; conflito se a relação contradiz; O(α).
  - *Exemplo:* "adicionar arestas e dizer se o grafo continua bipartido".
  - *Armadilha:* a paridade é relativa; ao comprimir, acumule o xor; com rollback, não comprima; "iguais" = paridade 0.
- **Bipartido completo vs. complemento:** se o complemento é bipartido (ou tem poucas arestas), BFS com conjunto de não visitados; componentes do complemento com m' arestas ausentes em O(n + m').
  - *Gatilho:* "complemento é bipartido", "poucas arestas ausentes".
  - *Por que funciona:* se o complemento tem poucas arestas, construí-lo e colori-lo é barato; caso contrário, o BFS com conjunto de não visitados no original evita enumerar $O(n^2)$ arestas.
  - *Como fazer:* se m' (arestas do complemento) é pequeno, construa-o e 2-colora; senão, BFS com set de não visitados no original; O(n + m') ou O(n + m).
  - *Exemplo:* "o grafo complemento é bipartido?".
  - *Armadilha:* o complemento de um grafo esparso é denso; não materialize; a bipartição do complemento é diferente da do original.
- **Complemento: clique/independente.** Conjunto independente máximo no complemento = clique; em grafo com m pequeno, o complemento é denso, ordem de grau e Turán dizem que existe clique ou independente grande (Ramsey: n ≥ C(2k, k) garante clique ou independente de tamanho k).
  - *Gatilho:* "clique no complemento", "independente", "Ramsey", "grafo esparso".
  - *Por que funciona:* independente no complemento = clique no original; com m pequeno o complemento é denso, e Turán/Ramsey garantem estruturas grandes; a cota de Ramsey é uma garantia existencial.
  - *Como fazer:* se m é pequeno, procure clique no original por branch and bound; se n ≥ C(2k, k), há clique ou independente de tamanho k; para independente no complemento, use o original.
  - *Exemplo:* "achar um independente grande no complemento".
  - *Armadilha:* a cota de Ramsey é enorme; para n pequeno, enumere; não confunda clique no complemento com clique no original.

- **Fecho transitivo/BFS com bitset.** "Alcançável em ≤ 2 passos": BFS onde o próximo frontier = OR das linhas dos vértices da fronteira e conjunto de não visitados mascara; $O(n^2/64)$ por camada total.
  - *Gatilho:* "alcançável em ≤ 2 passos", "fecho com bitset", "BFS em camadas".
  - *Por que funciona:* a fronteira é um bitset; o próximo conjunto é o OR das linhas de adjacência dos vértices da fronteira, menos os já visitados; cada linha tem n bits.
  - *Como fazer:* visited e frontier como bitsets; para cada camada, next = OR adj[u] para u ∈ frontier; next &= ~visited; visite; $O(n^2/64)$ no total.
  - *Exemplo:* "vértices a distância exatamente 2".
  - *Armadilha:* o OR custa O(n/64) por vértice da fronteira; para "≤ 2", pare após 2 camadas; não confunda com o fecho completo.
- **Contar componentes conexas após remover cada aresta/vértice (offline)** → segtree no tempo + DSU rollback (já conhecido); se só queries do tipo "subgrafo induzido por [l, r]" use DSU com ordenação por r e pesos de aresta como tempo de vida (MST por tempo).
  - *Gatilho:* "após remover cada aresta/vértice", "componentes ao longo do tempo", "queries offline".
  - *Por que funciona:* DSU só adiciona; remoções viram adições na ordem inversa; para intervalos de vida, DSU rollback + segtree no tempo cobre cada aresta nos nós certos.
  - *Como fazer:* offline reversed para remoções; para arestas ativas em [l, r], segtree de tempo com DSU rollback; O((n + m) log m α).
  - *Exemplo:* "número de componentes após cada remoção".
  - *Armadilha:* rollback não pode usar path compression; desfaça na ordem inversa; para "subgrafo induzido [l, r]", outra técnica (MST por tempo).
- **Subgrafo induzido por índices [l, r] com queries de componentes** → árvore geradora de máxima "l" para cada r (LCT) ou Mo com DSU rollback (Mo com rollback: ordena por bloco de l, r crescente).
  - *Gatilho:* "queries em subgrafo induzido por [l, r]", "componentes", "intervalo de vértices".
  - *Por que funciona:* para cada r, mantém-se uma árvore geradora máxima em "l" (cada aresta tem l como peso); responder componentes de [l, r] é consultar arestas com peso ≥ l; Mo com rollback faz o mesmo offline.
  - *Como fazer:* LCT ou DSU com rollback; Mo com rollback ordenando por bloco de l e r crescente; O((n + q)√n α) ou O(n log n).
  - *Exemplo:* "quantas componentes no subgrafo dos vértices l..r".
  - *Armadilha:* a árvore é sobre arestas com ambos endpoints no intervalo; a "l" máxima não é a MST usual; Mo com rollback não pode comprimir.
- **MST como "caminho minimax".** Menor valor M tal que u e v se conectam usando arestas ≤ M = maior aresta no caminho da MST; árvore de reconstrução de Kruskal (Kruskal reconstruction tree): cada junção cria nó com peso da aresta, então minimax(u,v) = peso do LCA e "componentes alcançáveis com peso ≤ x" = subárvore.
  - *Gatilho:* "menor M tal que u e v se conectam com arestas ≤ M", "gargalo", "primeiro instante de conexão".
  - *Por que funciona:* na MST, o caminho entre u e v minimiza a maior aresta; a Kruskal reconstruction tree registra o peso de cada união, e o LCA é exatamente o instante em que u e v passam a estar conectados.
  - *Como fazer:* Kruskal; para cada união crie um nó com o peso da aresta; minimax(u, v) = peso do LCA; componentes com arestas ≤ x = subárvore do ancestral mais alto com peso ≤ x; O((n + m) log n).
  - *Exemplo:* "a partir de que peso u e v ficam conectados".
  - *Armadilha:* o LCA na KRT é um nó interno, não um vértice original; folhas são os vértices; empates de peso mudam a forma da árvore, mas não o minimax.
- **Kruskal reconstruction tree + k-ésimo.** Em queries "k-ésimo maior vértice alcançável de v usando arestas ≤ x", suba ao ancestral mais alto com peso ≤ x (binary lifting) e consulte a k-ésima folha da subárvore pelo Euler tour (persistente/wavelet).
  - *Gatilho:* "k-ésimo maior vértice alcançável com arestas ≤ x".
  - *Por que funciona:* subir ao ancestral mais alto com peso ≤ x dá a subárvore que é exatamente o componente de v; a k-ésima folha por valor é uma consulta de ordem em intervalo do Euler tour.
  - *Como fazer:* KRT + binary lifting; Euler tour das folhas; persistent segment tree ou wavelet tree para k-ésimo na subárvore; O(log n) por query.
  - *Exemplo:* "consulta de k-ésimo maior em componente com limite de peso".
  - *Armadilha:* a ordem das folhas no Euler deve ser a dos vértices; o k-ésimo é por valor, não por posição; cuidado com empates de peso na subida.

- **Segundo MST** → para cada aresta fora da MST, troque pela maior aresta do caminho na MST (max por binary lifting); o menor aumento dá o 2º melhor. "MST sem uma aresta" é outra query: se a aresta está na MST, use a aresta de reposição que cruza o corte; se não está, a MST não muda.
  - *Gatilho:* "segundo melhor MST", "MST sem uma aresta", "custo do 2º MST".
  - *Por que funciona:* trocar uma aresta não-MST pela maior do caminho na MST é o menor aumento possível para incluir essa aresta; a segunda melhor é o mínimo dessas trocas.
  - *Como fazer:* máximo no caminho por binary lifting; para cada aresta fora da MST, candidato = custo + w − max; o mínimo é o 2º MST; O(m log n). "MST sem aresta": reposição que cruza o corte.
  - *Exemplo:* "qual o custo do segundo MST".
  - *Armadilha:* se a aresta tem o mesmo peso da máxima, o aumento é 0 (MST alternativo); se a aresta não está na MST, removê-la não muda nada; use long long.
- **MST com aresta forçada/proibida** → forçada: se e ∉ T, custo MST + w(e) − max do caminho (se e ∈ T, o custo não muda); ela pertence a alguma MST ⇔ w(e) = max do caminho na MST; a todas ⇔ é ponte no subgrafo das arestas com peso ≤ w(e).
  - *Gatilho:* "MST que contém/não contém uma aresta", "custo com obrigação".
  - *Por que funciona:* forçar e ∉ T equivale a trocar a maior aresta do caminho por e; e pertence a alguma MST sse w(e) = max do caminho (senão, trocar melhoraria); a todas sse é ponte no subgrafo de arestas de peso ≤ w(e).
  - *Como fazer:* calcule a MST; para forçada, some w(e) − max se e ∉ T; para "alguma", compare; para "todas", rode Kruskal só com arestas < w(e), contraia e teste se e é ponte no multigrafo das arestas de peso w(e); O(m log n).
  - *Exemplo:* "custo da MST obrigando a estrada e".
  - *Armadilha:* "proibida" é refazer a MST sem e (ou usar reposição); empates afetam "alguma/todas"; não confunda com MST única.
- **Aresta está em alguma/toda MST** → agrupe arestas por peso; para cada grupo, com os componentes do DSU antes do grupo, a aresta é "em toda MST" se é ponte no multigrafo reduzido e "em alguma" se une componentes distintos.
  - *Gatilho:* "essa aresta pode estar na MST?", "todas as MSTs".
  - *Por que funciona:* processando por grupos de peso, os componentes formados por pesos menores são fixos; uma aresta que une componentes distintos está em alguma MST, e se é ponte no multigrafo do grupo, está em todas.
  - *Como fazer:* Kruskal por grupos; para cada grupo, contraia os componentes atuais; monte o multigrafo; pontes = em todas; arestas que unem componentes = em alguma; O(m log m).
  - *Exemplo:* "classificar arestas em nunca/alguma/todas".
  - *Armadilha:* processar todas as arestas do grupo antes de unir; aresta interna ao componente não está em nenhuma; pontes no multigrafo contam paralelas.
- **MST único?** Sim ⇔ nenhuma aresta fora da árvore tem peso = maior do caminho. Contar MSTs → matrix-tree por classe de peso (determinante) em cada grupo de peso igual.
  - *Gatilho:* "MST é única?", "contar MSTs".
  - *Por que funciona:* unicidade sse nenhuma aresta fora da MST tem peso igual ao máximo do caminho; para contar, contraia os componentes de pesos menores e conte as árvores geradoras em cada grupo de peso com matrix-tree.
  - *Como fazer:* compare pesos; para contar, por grupo de peso, contraia componentes menores, monte o multigrafo e aplique matrix-tree; o produto dos determinantes é o total; $O(n^3 \log)$.
  - *Exemplo:* "quantas MSTs tem o grafo".
  - *Armadilha:* contar sem contrair conta árvores erradas; use módulo/primos; pesos iguais podem gerar ciclos no grupo.
- **Matrix-tree (Kirchhoff).** Número de árvores geradoras = det de qualquer cofator da Laplaciana; com pesos, soma dos produtos; arestas "exatamente k" → polinômio em x; Cayley: n^{n−2}.
  - *Gatilho:* "contar árvores geradoras", "soma de produtos", "exatamente k arestas de um tipo".
  - *Por que funciona:* Kirchhoff: qualquer cofator da Laplaciana conta árvores geradoras; com pesos, soma os produtos dos pesos; Cayley é o caso do grafo completo.
  - *Como fazer:* monte L (grau na diagonal, −w nas arestas); apague uma linha/coluna; determinante $O(n^3)$; para "exatamente k", use polinômio em x ou avalie em pontos e interpole.
  - *Exemplo:* "número de árvores geradoras de K_n" = n^{n−2}.
  - *Armadilha:* precisão com doubles; use módulo/frações; multigrafo soma pesos; laços não contam.

- **Boruvka** → O(m log n), útil quando as arestas são geradas implicitamente (MST de pontos com custo de par, MST com k "melhor aresta por componente" via estrutura); também serve para MST em O(n log n) com consulta de "vizinho mais próximo fora da componente".
  - *Gatilho:* "MST com arestas implícitas", "vizinho mais próximo por componente".
  - *Por que funciona:* em cada rodada, cada componente escolhe sua aresta mais leve para fora; essas arestas são seguras, e o número de componentes ao menos dobra.
  - *Como fazer:* DSU; para cada componente, ache a melhor aresta; una; O(m log n); com arestas implícitas, consulta de vizinho mais próximo por estrutura.
  - *Exemplo:* "MST de pontos com custo de par".
  - *Armadilha:* escolher a mesma aresta dos dois lados é ok; empates podem criar ciclos, mas a união é segura; sem estrutura, só vale com m explícito.
- **MST de pontos no plano** → arestas só entre vizinhos de Delaunay (O(n)); Manhattan: para cada ponto, 4 octantes → vizinho mais próximo em cada (BIT) → 4n arestas candidatas e Kruskal.
  - *Gatilho:* "MST de pontos no plano", "distância euclidiana/Manhattan".
  - *Por que funciona:* a MST euclidiana usa apenas arestas de Delaunay (O(n)); para Manhattan, basta o vizinho mais próximo em cada um dos 8 octantes (4 varreduras por simetria).
  - *Como fazer:* Delaunay O(n log n) e Kruskal; Manhattan: para cada ponto, 4 transformações, BIT para o vizinho mais próximo em cada octante; até 4n arestas; O(n log n).
  - *Exemplo:* "conectar cidades com custo de distância".
  - *Armadilha:* Delaunay com pontos colineares/duplicados; Manhattan precisa das 4 orientações; a MST euclidiana não usa todos os pares.
- **MST com custo |a_i − a_j| ou a_i xor a_j** → xor: Boruvka + trie binária (ou divida & conquiste em bit alto); |a_i − a_j|: arestas entre vizinhos ordenados.
  - *Gatilho:* "custo |a_i − a_j|", "custo a_i xor a_j".
  - *Por que funciona:* para |diff|, a métrica da reta faz os vizinhos ordenados bastarem; para xor, o bit mais alto separa o conjunto e a MST se decompõe recursivamente (ou Boruvka + trie).
  - *Como fazer:* |diff|: ordene e use arestas consecutivas; O(n log n). Xor: trie binária com Boruvka O(n log A) ou D&C no bit mais alto: MST de cada metade + aresta mínima entre metades.
  - *Exemplo:* "MST de números com peso xor".
  - *Armadilha:* xor não é métrica; não use só os vizinhos ordenados; no D&C, a aresta entre metades é o mínimo xor entre os dois conjuntos (trie).
- **MST com custo a_i + a_j (ou min/max(a_i, a_j))** → ligue todos ao menor a_i (para a_i + a_j) ou analise por ordenação; para gcd(a_i, a_j): arestas via múltiplos com custo d (crivo + Kruskal só entre pares compartilhando divisor).
  - *Gatilho:* "custo soma", "custo min/max", "custo gcd".
  - *Por que funciona:* para a_i + a_j, a estrela no menor a_i é ótima (ligar qualquer outro par custa mais); para min/max, ordenar resolve; para gcd, os candidatos vêm de divisores comuns.
  - *Como fazer:* soma: escolha o mínimo e ligue todos; O(n). Min/max: ordene e conecte consecutivos; O(n log n). Gcd: para cada divisor d, considere os múltiplos de d e gere arestas candidatas; Kruskal; a direção do custo é decisiva.
  - *Exemplo:* "MST com peso gcd(a_i, a_j)".
  - *Armadilha:* no caso gcd, o custo da aresta é gcd(a_i, a_j), não o divisor d; gerar arestas entre múltiplos de d com custo d subestima o peso e pode dar a MST errada.
- **Árvore geradora com grau restrito** → em geral NP-difícil; com restrição em um único vértice (grau exato k) resolve-se via MST por multiplicador (Lagrange/Aliens nas arestas desse vértice).
  - *Gatilho:* "árvore geradora com grau limitado", "grau exato k".
  - *Por que funciona:* restrições de grau em vários vértices tornam o problema NP-difícil; com um único vértice, o custo ótimo como função do grau é convexo, então um multiplicador λ resolve (ou matroid intersection).
  - *Como fazer:* binária no multiplicador λ somado às arestas do vértice; rode Kruskal; ajuste até o grau k; O(log·m log m); a sequência de custos é convexa.
  - *Exemplo:* "MST onde o vértice 1 tem grau exatamente k".
  - *Armadilha:* restrições em vários vértices = NP-difícil; a convexidade vale para um vértice; empates podem exigir desempate para atingir k exato.

- **MST com arestas "cor": exatamente k brancas** → Aliens/binária numa penalidade somada às arestas brancas; pela matroid, o custo é convexo em k.
  - *Gatilho:* "exatamente k arestas brancas", "arestas coloridas".
  - *Por que funciona:* somar λ às arestas brancas e rodar Kruskal dá, para cada λ, uma MST com certo número de brancas; a função custo(k) é convexa (matroid), então Aliens acha o k pedido.
  - *Como fazer:* binária em λ inteiro; conte brancas; ajuste; resposta = custo − λ·k; O(log C·m log m); desempate para maximizar/minimizar brancas.
  - *Exemplo:* "MST com exatamente k estradas de pedágio".
  - *Armadilha:* sem desempate consistente, a binária pode oscilar; λ negativo é permitido; a convexidade é discreta.
- **Árvore geradora de custo "máximo mínimo dos buracos" / bottleneck** → MST já é ótima para minimizar o maior peso (basta o peso da maior aresta da MST).
  - *Gatilho:* "minimizar o maior peso", "bottleneck", "menor largura para conectar".
  - *Por que funciona:* a MST minimiza a maior aresta entre todas as árvores geradoras (propriedade do corte); qualquer árvore tem que cruzar o mesmo corte.
  - *Como fazer:* rode Kruskal e pare quando conectar; a última aresta aceita é o gargalo; O(m log m); não precisa da árvore inteira.
  - *Exemplo:* "menor largura de estrada para conectar todos".
  - *Armadilha:* a MST também minimiza o gargalo, mas não necessariamente a soma; para maximizar o mínimo, use maximum spanning tree.
- **MST dinâmico offline** (arestas entram/saem, query MST) → D&C no tempo com contração: em cada nível reduza o grafo às arestas que importam (O(k) por nível) → $O(m \log^2 n)$.
  - *Gatilho:* "arestas entram e saem", "MST após cada operação", "offline".
  - *Por que funciona:* D&C no tempo; em cada nível, arestas que nunca saem podem ser contraídas e arestas muito pesadas descartadas, reduzindo o grafo a O(k) por nível.
  - *Como fazer:* segtree de tempo; em cada nó, contraia arestas garantidas e descarte as que não podem estar na MST; recursão; $O(m \log^2 n)$.
  - *Exemplo:* "MST com arestas ativas por intervalos".
  - *Armadilha:* as regras de contração/descarte precisam do contexto; DSU rollback; implementação delicada.
- **Grafo de intervalos/arestas implícitas por faixas** → arestas u→[l, r] por segtree/sparse table de vértices; Dijkstra/BFS usando "próximo não visitado" por DSU ou set removendo ao visitar (cada vértice sai uma vez).
  - *Gatilho:* "u se liga a um intervalo", "arestas por faixa", "pular para qualquer j na faixa".
  - *Por que funciona:* a segtree representa o intervalo com O(log n) nós e arestas internas de custo 0 propagam o custo; o grafo expandido tem O((n + m) log n) arestas, e cada vértice é visitado uma vez.
  - *Como fazer:* segtree (ou duas, para as duas direções) com arestas internas de custo 0; ligue u aos O(log n) nós que cobrem [l, r] com o peso; Dijkstra/BFS; para "próximo não visitado", DSU ou set.
  - *Exemplo:* "de i posso pular para qualquer j em [l_i, r_i] com custo 1".
  - *Armadilha:* a direção das arestas da segtree; não ligue u a todas as folhas; a memória do grafo expandido.
- **BFS onde vizinhos são "todos não visitados com condição"** → mantenha set/DSU de não visitados e extraia quem satisfaz; custo amortizado O((n + m) log) mesmo em grafo denso implícito.
  - *Gatilho:* "vizinhos são todos os não visitados que satisfazem P", "grafo denso implícito".
  - *Por que funciona:* cada vértice sai do conjunto de não visitados uma vez; a extração é amortizada; o custo é o número de remoções mais as consultas à estrutura.
  - *Como fazer:* set/DSU de não visitados; ao processar u, extraia todos os v que satisfazem P (set) ou use "próximo não visitado" (DSU); O((n + m) log n) ou O(n α).
  - *Exemplo:* BFS no complemento; vizinhos por diferença ≤ d.
  - *Armadilha:* a condição P precisa ser verificável por vértice; se depender da aresta, o conjunto não basta; não recoloque visitados.

- **Planar duality.** Corte mínimo s–t em planar = caminho mínimo no dual (com ciclo/arestas de s a t); "número de faces" = m − n + 1 + c (Euler: V − E + F = 1 + c); em grid, componentes e buracos viram faces (4-vs-8 conectividade complementar).
  - *Gatilho:* "grafo planar", "corte mínimo", "faces", "separar regiões no grid".
  - *Por que funciona:* em planar, cada corte s–t corresponde a um ciclo no dual que separa as faces de s e t; esse ciclo vira caminho ao cortar o dual ao longo de uma aresta entre as faces; Euler dá o número de faces.
  - *Como fazer:* construa o dual (faces como vértices, arestas cruzando arestas); se s e t estão na mesma face, adicione uma aresta e procure o ciclo separador; Dijkstra no dual; O(n log n). F = m − n + 1 + c.
  - *Exemplo:* "cerca mínima para separar duas regiões".
  - *Armadilha:* s e t em faces diferentes mudam a construção; arestas de fronteira; no grid, 4-conectividade para uma cor e 8 para a complementar.
- **Difference constraints com sistema grande** → grafo de $x_j - x_i \le c$; viável ⇔ sem ciclo negativo; solução mínima/máxima: distâncias de super-fonte; igualdade vira duas desigualdades; "$x_i < x_j$" com inteiros vira $x_i - x_j \le -1$.
  - *Gatilho:* "$x_j - x_i \le c$", "sistema de desigualdades", "viabilidade", "horários com folgas".
  - *Por que funciona:* cada desigualdade é uma aresta i → j de peso c; viável sse não há ciclo negativo; as distâncias de uma super-fonte satisfazem todas e dão a solução extrema; igualdade é duas implicações.
  - *Como fazer:* Bellman-Ford/SPFA com super-fonte 0; se ciclo negativo, inviável; x_v = dist(v); para $x_i < x_j$ inteiro use $x_i - x_j \le -1$; O(nm).
  - *Exemplo:* "atribuir horários com folgas mínimas".
  - *Armadilha:* o sinal; desigualdades estritas em reais não viram ≤ −1; ciclo negativo = inconsistência; para maximizar, inverta as desigualdades.
- **Difference constraints + minimizar Σ ou max** → dual é fluxo de custo mínimo (LP dual); max $x_n - x_1$ = distância mais curta de 1 a n.
  - *Gatilho:* "minimizar soma/máximo sujeito a diferenças", "LP dual", "maior diferença possível".
  - *Por que funciona:* o dual do sistema de diferenças é um fluxo de custo mínimo; a maior diferença $x_n - x_1$ é limitada pelo caminho mais curto de 1 a n no grafo das desigualdades.
  - *Como fazer:* para max $x_n - x_1$, Bellman-Ford de 1 a n; para somas, formule o dual como mincost flow; O(nm) / O(flow).
  - *Exemplo:* "maior diferença possível entre dois eventos".
  - *Armadilha:* a direção do caminho; a solução pode ser ilimitada se não houver caminho; o dual exige a formulação correta.
- **Contagem de componentes conexas de grafo aleatório/denso** → n ≤ 5000 com m grande: BFS com bitset de não visitados $O(n^2/64)$.
  - *Gatilho:* "n ≤ 5000, m grande", "componentes conexas", "grafo denso".
  - *Por que funciona:* o BFS com bitset testa a vizinhança de um vértice em blocos de 64 bits; cada vértice é removido do conjunto de não visitados uma vez.
  - *Como fazer:* adj como bitsets; BFS: para cada u, next = adj[u] & ~visited; $O(n^2/64)$ tempo, $O(n^2/64)$ memória.
  - *Exemplo:* "contar componentes de um grafo quase completo".
  - *Armadilha:* n = 5000 dá ~3 MB de bitsets; a leitura de m arestas domina; construa os bitsets uma vez.
- **Grafos com vértice removido e queries "n − 1 vezes"** → ordem inversa + DSU, ou percorra por centroides; para "tudo exceto um" use prefixos/sufixos de estruturas (divide and conquer com rollback).
  - *Gatilho:* "remover cada vértice", "n − 1 consultas", "tudo exceto um".
  - *Por que funciona:* processar remoções ao contrário vira adições; para "tudo exceto um", prefixos e sufixos combinam os dois lados; D&C com rollback cobre os intervalos.
  - *Como fazer:* reverse + DSU; ou D&C com rollback; para intervalos, segtree de tempo; O((n + m) log n α).
  - *Exemplo:* "componentes após remover cada vértice".
  - *Armadilha:* remover um vértice remove suas arestas; no reverse, adicione o vértice e suas arestas; rollback sem path compression.

- **Distância em grafo com poucas arestas extras (m ≤ n + k)** → árvore geradora + ≤ 2(k + 1) vértices especiais (extremos das extras): BFS/Dijkstra de cada especial e distância em árvore por LCA; query = min(d_árvore, min_s d(u,s) + d(s,v)).
  - *Gatilho:* "m ≤ n + k", "quase árvore", "distância entre pares", "poucas arestas extras".
  - *Por que funciona:* uma árvore geradora tem n − 1 arestas; as no máximo k + 1 extras tocam no máximo 2(k + 1) vértices especiais; um caminho ótimo usa especiais no máximo uma vez.
  - *Como fazer:* BFS/Dijkstra de cada especial; distância na árvore por LCA; query = min(d_árvore(u,v), min_s d(u,s) + d(s,v)); O(k(n + m) + qk).
  - *Exemplo:* "distância em grafo com poucas arestas extras".
  - *Armadilha:* os especiais incluem os endpoints das extras; o caminho pode usar mais de um especial; conte k + 1 extras, não k; fixe a árvore.
- **Grafo quase-árvore com consultas de distância** → idem; ou HLD/Euler para a árvore e 2(k + 1) Dijkstras.
  - *Gatilho:* "grafo quase-árvore", "consultas de distância".
  - *Por que funciona:* a árvore responde distâncias em O(log n) com HLD/LCA, e os especiais cobrem as arestas extras.
  - *Como fazer:* árvore + HLD/LCA; 2(k + 1) Dijkstras; query min; O((n + m) log n + qk).
  - *Exemplo:* "distância com uma aresta extra".
  - *Armadilha:* o número de especiais dobra com as extras; se k é grande, não escala; pesos negativos quebram Dijkstra.
- **Orientar para minimizar o maior grau de saída (pseudoarboricidade)** → binária no limite + fluxo, ou Hakimi; o mínimo possível é ⌈max_S m_S/|S|⌉.
  - *Gatilho:* "minimizar o maior grau de saída", "orientar arestas", "pseudoarboricidade".
  - *Por que funciona:* a orientação com grau de saída ≤ k existe sse m_S ≤ k|S| para todo S (Hakimi); o menor k é ⌈max_S m_S/|S|⌉, e o fluxo testa a viabilidade.
  - *Como fazer:* binária em k; rede com aresta → vértice cap 1 e vértice → T cap k; viável sse o fluxo satura; ou Hakimi; O(log n·maxflow).
  - *Exemplo:* "orientar para que cada vértice tenha no máximo 2 saídas".
  - *Armadilha:* pseudoarboricidade ≠ arboricidade (|S| vs |S| − 1); a cota é um máximo sobre subgrafos; o fluxo precisa de todas as arestas.
- **Árvores geradoras e "mínimo de arestas para conectar"** → c − 1 arestas ligando componentes; ligar com custo mínimo = MST sobre componentes contraídos.
  - *Gatilho:* "mínimo de arestas para conectar componentes", "custo de ligar".
  - *Por que funciona:* contrair componentes e conectar com c − 1 arestas é necessário e suficiente; o custo mínimo é a MST no grafo contraído.
  - *Como fazer:* DSU para achar componentes; contraia; MST nas arestas entre componentes; O(m log m).
  - *Exemplo:* "ligar ilhas com pontes de custo".
  - *Armadilha:* arestas internas não ajudam; se o grafo já é conexo, 0; c − 1 é o número de arestas, não o custo.
- **Grafos de "bola" / vizinhança em hipercubo e xor** → vértices = valores, aresta se diferem por 1 bit: BFS com 20·2^20; "vizinhos a distância de Hamming ≤ d" → BFS multi-fonte sobre máscaras.
  - *Gatilho:* "vértices = máscaras", "diferem por 1 bit", "distância de Hamming".
  - *Por que funciona:* o hipercubo de dimensão d tem 2^d vértices e d·2^{d−1} arestas; a BFS explora o flip de cada bit, e a distância de Hamming é o número de flips.
  - *Como fazer:* BFS multi-fonte; para cada máscara, vizinhos = mask xor (1 << b); O(d·2^d); para "≤ d bits", bolas ou BFS multi-fonte.
  - *Exemplo:* "menor número de flips para chegar a uma máscara".
  - *Armadilha:* 2^20 é viável, 2^25 não; distância de Hamming é número de bits, não valor; não confunda com xor de pesos.
- **Caminho com "no máximo k vezes" um tipo de aresta** → mais eficiente: BFS em camadas por k (O(k(n + m))); para k ≈ log, estado (v, j) cabe.
  - *Gatilho:* "no máximo k vezes um tipo de aresta", "usar k vezes".
  - *Por que funciona:* o número de usos é uma coordenação; camadas (v, j) registram quantas vezes o tipo especial foi usado; a transição especial avança j.
  - *Como fazer:* camadas j = 0..k; aresta do tipo especial avança j; as demais mantêm; O(k(n + m)); se k ≈ log, o estado (v, j) é pequeno.
  - *Exemplo:* "no máximo k voos".
  - *Armadilha:* k grande estoura; "exatamente k" ≠ "no máximo k"; se o custo é convexo, Aliens.

- **Grafo de duas camadas (ônibus/linhas)** → vértices de linha e de parada, entrada com custo de embarque, saída custo 0; reduz arestas de $O(L^2)$ a $O(L)$.
  - *Gatilho:* "linhas de ônibus", "baldeações", "embarque".
  - *Por que funciona:* modelar linhas como vértices evita arestas entre todas as paradas de todas as linhas; parada → linha paga o embarque, linha → parada custa 0.
  - *Como fazer:* vértices parada e linha; parada → linha custa embarque, linha → parada custa 0; para cada linha, ligue às paradas dela; O(total de incidências parada-linha).
  - *Exemplo:* "mínimo de baldeações".
  - *Armadilha:* o número de arestas é O(total de incidências), não O(L); a direção do custo; não crie arestas linha-linha.
- **Teorema de Ore/Dirac para Hamilton** → grau mínimo ≥ n/2 garante ciclo hamiltoniano; construtivos com "grafo denso" costumam ter isto: a construção é por inserção de vértices e rotação de caminho (Pósa).
  - *Gatilho:* "grafo denso", "ciclo hamiltoniano garantido", "grau mínimo".
  - *Por que funciona:* Dirac: $\delta \geq n/2$ ($n \geq 3$) implica Hamiltoniano; Ore generaliza para pares não adjacentes com graus somando ≥ n; a construção usa inserção e rotações de Pósa.
  - *Como fazer:* comece com um ciclo e insira vértices usando a condição de grau; rotações de Pósa para fechar o ciclo; $O(n^2)$.
  - *Exemplo:* "grafo com grau mínimo n/2 tem ciclo hamiltoniano".
  - *Armadilha:* as condições são suficientes, não necessárias; $n \geq 3$; não é um algoritmo de decisão para grafos arbitrários (Hamilton é NP-completo).

adicionado em 2026-11-11 a partir dos editoriais de CF Div. 2/3/4, A–D; triado contra o catálogo

- **Reconstruir o grafo a partir da alcançabilidade.** Aresta direta u → v sse u alcança v e não existe w intermediário (redução transitiva, $O(n^3/64)$ com bitset). Numa árvore enraizada, o pai de v é, entre os que alcançam v, o de menor conjunto alcançável. Ex.: CF 2208D1.
  - *Gatilho:* "dado o fecho/alcançabilidade, reconstrua as arestas", "árvore enraizada a partir de alcançabilidade", "redução transitiva".
  - *Por que funciona:* a aresta direta u → v existe sse u alcança v e nenhum w intermediário alcança v; remover as arestas redundantes é a redução transitiva. Numa árvore, o pai é o alcançador de v com menor conjunto alcançável (o mais próximo).
  - *Como fazer:* bitsets de alcançabilidade; para cada par (u, v) com u alcança v, verifique se existe $w \neq u, v$ com u alcança w e w alcança v; $O(n^3/64)$ ou $O(n \cdot m/64)$; na árvore, compare os conjuntos/ordem.
  - *Exemplo:* CF 2208D1.
  - *Armadilha:* em DAG a redução transitiva é única; em grafo com ciclos, condense primeiro; a ordem dos bitsets importa.

Repetidas de outras seções:

- **Jogo em grafo com empate** → BFS retrógrada (teoria dos jogos).
- **Strings que viram umas nas outras por operação** → vértices = strings, BFS; operação reversível → componentes.
- **Operação reversível** → componentes conexas = classes de equivalência (construtivos).
- **Euler com vértice fantasma** orienta arestas com $|in - out| \leq 1$ e constrói de Bruijn.
- **Kruskal reconstruction tree** responde "maior aresta mínima no caminho" e "primeiro instante de conexão".
- **Coloração de arestas de bipartido** com Δ cores; K_n em $n - 1$ rodadas (n par) pelo método do círculo.
- **BEST** → circuitos eulerianos dirigidos = $t_w \cdot \prod(deg-1)!$.
- **Graphical sequence** → Erdős–Gallai/Havel–Hakimi; torneios: Landau.
- **Torneio: 3-ciclos** → $C(n,3) - \Sigma\, C(out_v, 2)$.
- **Número de 4-ciclos** → $\Sigma_{u<w}\, C(cnt(u,w), 2)/2$ com vizinhos comuns.
- **Ciclo base (xor de ciclos)** → px pela árvore geradora; cada aresta fora vira um ciclo; base linear dá max/min xor de qualquer passeio; dimensão $m - n + c$.
- **Sistema de paridades em arestas (T-join)** → GF(2); em árvore a solução é única, de baixo para cima.

Árvores

Primeira pergunta: a resposta depende da raiz? Se sim, pense em rerooting ou centroide.

- **Enraíze** em 1 (ou numa folha, ou no centroide, ou no centro do diâmetro) e faça DP de subárvore.
  - *Gatilho:* "para cada vértice", "subárvore", "caminho que passa por", árvore dada sem raiz.
  - *Por que funciona:* enraizar orienta cada aresta pai→filho e transforma a árvore num DAG; toda grandeza de subárvore sai em pós-ordem, e todo caminho u–v se decompõe nos ramos u→lca e v→lca.
  - *Como fazer:* uma DFS/BFS calcula pai[], ordem e profundidade; processe a ordem ao contrário combinando os filhos. O(n) tempo e memória.
  - *Exemplo:* sz[v] = 1 + Σ sz[filho]; altura[v] = 1 + max altura[filho].
  - *Armadilha:* recursão em n = $2{\cdot}10^5$ estoura a pilha em C++; use DFS iterativa ou aumente a pilha. A raiz 1 é arbitrária para DP de subárvore; só escolha folha/centroide/centro quando isso simplificar o problema.
- **Para toda raiz** → rerooting (prefixo/sufixo dos filhos se a operação não é invertível).
  - *Gatilho:* "calcule X para cada vértice supondo que ele é a raiz", "soma das distâncias de cada v", "melhor raiz".
  - *Por que funciona:* down[v] (subárvore) e up[v] (o resto visto do pai) bastam; ao trocar a raiz de p para c, só o ramo de c muda de lado, então a resposta se atualiza localmente.
  - *Como fazer:* 1ª DFS calcula down; 2ª DFS leva up e calcula, para cada filho c, a combinação de up[p] com down dos irmãos. Se a operação é invertível, "total − contribuição de c"; se não, prefixo e sufixo dos filhos. O(n) no total.
  - *Exemplo:* Σ_u d(v,u): res[c] = res[p] + n − 2·sz[c] com arestas unitárias.
  - *Armadilha:* max/min/gcd não são invertíveis; use prefixo/sufixo ou guarde os dois maiores valores. Cuidado com a identidade da operação (0 para soma, −∞ para max) e com o caso de um único filho.
- **Todos os pares de caminhos** → contribuição por aresta: sz·(n − sz); ou centroide.
  - *Gatilho:* "soma das distâncias entre todos os pares", "quantos caminhos usam cada aresta", "valor esperado da distância".
  - *Por que funciona:* um caminho u–v usa a aresta e = (pai, filho) exatamente quando u e v estão em lados opostos do corte; há sz·(n − sz) pares assim, e cada par é contado uma vez por aresta do seu caminho.
  - *Como fazer:* uma DFS calcula sz; some w_e·sz_e·(n − sz_e) para obter a soma das distâncias. Para contar caminhos que passam por cada aresta, o mesmo produto. O(n).
  - *Exemplo:* árvore com 4 vértices em estrela, todas as arestas peso 1: cada aresta contribui 1·3 = 3, soma das distâncias = 9? Na estrela K1,3 há 3 pares a distância 2 e 3 pares a distância 1: 3·2 + 3·1 = 9.
  - *Armadilha:* a fórmula é por aresta quando o peso está na aresta; para valores em vértices a contribuição é outra. Para caminhos com condição (distância ≤ K) a contribuição por aresta não decide nada; use centroide.
- **Caminhos com condição (comprimento k, soma ≤ x)** → centroid decomposition; ou small-to-large de maps.
  - *Gatilho:* "conte pares (u,v) com distância ≤ K", "caminho de comprimento exatamente k", n ≤ $2{\cdot}10^5$ e sem updates.
  - *Por que funciona:* todo caminho passa por exatamente um centroide — o primeiro removido que separa suas pontas; contar em cada centroide os caminhos que o atravessam cobre cada par uma única vez.
  - *Como fazer:* decomponha recursivamente (O(n log n)); em cada centroide colete as distâncias de cada subárvore filha, combine com as distâncias já processadas (ordene + two pointers ou hash map) e remova o centroide. Para "≤ K", conte todos e subtraia os pares da mesma subárvore.
  - *Exemplo:* contar pares a distância exatamente k; contar pares com soma de valores ≤ K.
  - *Armadilha:* contar (u,u) ou contar o mesmo par em dois níveis; sempre subtraia a contribuição da mesma subárvore. A decomposição recursiva pode ter profundidade O(log n), mas a DFS de coleta pode ser profunda: use iterativa.
- **Diâmetro** → duas BFS (sem pesos) ou dois DFS/Dijkstra (pesos ≥ 0); distância máxima de cada vértice = até uma das pontas do diâmetro.
  - *Gatilho:* "maior distância entre dois vértices", "caminho mais longo", "quão longe cada vértice está".
  - *Por que funciona:* a partir de qualquer vértice s, um vértice a mais distante é ponta de algum diâmetro; repetindo de a, chega-se à outra ponta b. Para qualquer v, a distância máxima é max(d(v,a), d(v,b)).
  - *Como fazer:* BFS/DFS de 1 → a; de a → b e distâncias; D = d(a,b). Com pesos ≥ 0 troque BFS por Dijkstra; com pesos unitários, duas BFS são O(n).
  - *Exemplo:* achar as duas pontas e responder max(d(v,a), d(v,b)) para todo v em O(n).
  - *Armadilha:* com pesos negativos o two-sweep falha (use DP de diâmetro); BFS só vale sem pesos; em grafo com ciclos, "diâmetro" já não é o mesmo problema.

- **Queries de caminho** → LCA (binary lifting/Euler + sparse); com updates → HLD + segtree.
  - *Gatilho:* muitas perguntas (u,v) sobre soma/max/min/contagem no caminho, árvore fixa.
  - *Por que funciona:* caminho u–v = u→lca mais lca→v; com LCA em O(log) ou O(1), cada consulta se reduz a subir os dois lados e combinar.
  - *Como fazer:* pré-calcule profundidade e up[k][v] (ou Euler + sparse table); agregue de u e v até o LCA. Com updates, HLD quebra o caminho em O(log n) intervalos contíguos e a segtree responde cada um em O(log n).
  - *Exemplo:* soma de valores no caminho; máximo; k-ésimo vértice do caminho; contar arestas com peso > x (com segtree ordenada).
  - *Armadilha:* operação não comutativa exige tratar o lado v invertido; em HLD, o intervalo da cadeia pode estar em ordem crescente ou decrescente conforme se sobe.
- **Queries de subárvore** → Euler tour (tin/tout) vira intervalo de array.
  - *Gatilho:* "quantos vértices na subárvore", "soma da subárvore", updates que afetam uma subárvore inteira.
  - *Por que funciona:* a DFS visita todos os descendentes de v de forma contígua antes de sair; logo [tin[v], tout[v]) é exatamente a subárvore.
  - *Como fazer:* registre tin na entrada e tout na saída; mapeie valor para a posição tin; use BIT/segtree para ponto e intervalo. O(log n) por query.
  - *Exemplo:* soma de valores na subárvore com point updates; contar valores ≤ x na subárvore (offline, ordenando por valor).
  - *Armadilha:* usar tout inclusivo num lugar e exclusivo noutro; a subárvore inclui v; updates de caminho não são resolvidos só com Euler tour (use HLD ou diferença em árvore).
- **Contar cores/valores distintos em subárvore** → small-to-large (DSU on tree / sack).
  - *Gatilho:* "quantos valores distintos na subárvore de cada v", "cor mais frequente na subárvore", sem updates.
  - *Por que funciona:* mantenha a estrutura do filho pesado e insira os vértices dos filhos leves; cada vértice é inserido O(log n) vezes porque cada vez que é reinserido o tamanho do conjunto pelo menos dobra.
  - *Como fazer:* DFS com sz; escolha o filho de maior sz; processe os leves com keep = false, o pesado com keep = true, adicione as subárvores leves e responda em v. O(n log n) com contadores em vetor; O(n $\log^2$ n) com set/map.
  - *Exemplo:* número de cores distintas por subárvore; moda por subárvore.
  - *Armadilha:* limpar a estrutura só quando keep = false; o contador global e o máximo corrente precisam ser restaurados; para "k-ésimo" é preciso estrutura ordenada, não só frequências.
- **Folhas e graus:** número de folhas, vértices de grau ≥ 3, cadeias de grau 2 costumam ser o que importa.
  - *Gatilho:* árvore sem raiz, "folhas", "grau", "comprimir caminhos", "esqueleto".
  - *Por que funciona:* a forma da árvore é determinada pelas folhas e pelos vértices de grau ≥ 3; cadeias de grau 2 podem ser comprimidas numa aresta de peso igual à soma, preservando distâncias.
  - *Como fazer:* conte graus; pode folhas iterativamente (fila) para achar o centro/centroide; comprima cadeias para construir a árvore de esqueleto. O(n).
  - *Exemplo:* menor número de caminhos para cobrir todas as arestas = ⌈folhas/2⌉; diâmetro termina em folhas.
  - *Armadilha:* n = 1 e n = 2 são casos especiais; ao comprimir, guarde os extremos para não perder LCA; folhas de grau 0 não existem em árvore com n > 1.
- **Remover vértice/aresta** → componentes = subárvores dos filhos + o resto (n − sz).
  - *Gatilho:* "remova v e conte componentes", "corte uma aresta", "quais pares se separam".
  - *Por que funciona:* ao remover v, cada filho fica numa componente de tamanho sz[filho] e o resto (acima de v) forma uma componente de tamanho n − sz[v]; para uma aresta pai–filho, os lados são sz[filho] e n − sz[filho].
  - *Como fazer:* uma DFS calcula sz; para vértice, liste sz dos filhos e n − sz[v]; para aresta, os dois lados. O(n).
  - *Exemplo:* contar pares (u,w) separados por uma aresta = sz·(n − sz).
  - *Armadilha:* em grafo com ciclos a fórmula não vale; em árvore, remover v produz grau(v) componentes, não sz[v]; cuidado com o componente "acima" quando v é a raiz.
- **Pesado/leve:** caminho raiz-folha cruza ≤ log arestas leves; ≤ 2√n vértices com grau ≥ √n.
  - *Gatilho:* provar complexidade de HLD/saltos; enumerar vértices de grau alto; "quantas cadeias".
  - *Por que funciona:* ao subir por uma aresta leve, o tamanho da subárvore pelo menos dobra, então há no máximo $\lfloor \log_2 n \rfloor$ arestas leves. Por handshake, Σ grau = 2(n−1), então há no máximo 2(n−1)/√n < 2√n vértices com grau ≥ √n.
  - *Como fazer:* defina filho pesado como o de maior sz; numere o pesado primeiro (HLD); use a cota para limitar saltos e para listar vértices de grau alto.
  - *Exemplo:* LCA por HLD sobe O(log n) cadeias; enumerar os O(√n) vértices de grau ≥ √n para processar consultas.
  - *Armadilha:* a cota é de arestas leves, não de arestas totais; "grau ≥ √n" dá 2√n vértices, mas "grau ≥ k" dá 2n/k; não confunda com a cota de cadeias.

- **Paridade de profundidade** → árvore é bipartida; jogos em árvore costumam depender disso.
  - *Gatilho:* "caminho par/ímpar", "alternar", "jogo de mover peça", "2-coloração".
  - *Por que funciona:* toda aresta liga profundidades de paridades opostas, então depth mod 2 é uma 2-coloração válida; o comprimento de u–v é d[u] + d[v] − 2·d[lca], cuja paridade é (d[u] + d[v]) mod 2.
  - *Como fazer:* pinte por depth & 1; em jogos, conte quantos movimentos cada jogador consegue forçar e quem controla a paridade; para caminhos, use a fórmula acima.
  - *Exemplo:* jogo em que uma peça alterna entre vértices adjacentes; colorir um tabuleiro em forma de árvore.
  - *Armadilha:* trocar a raiz troca todas as cores, mas não a bipartição; com pesos nas arestas, a paridade da distância não é só a paridade da profundidade.
- **Virtual tree** quando cada query toca k vértices e Σk é pequeno.
  - *Gatilho:* "cada consulta dá um conjunto de k vértices", $\Sigma k \le 2 \cdot 10^5$, queries de caminho/ancestralidade entre eles.
  - *Por que funciona:* só os k vértices e os LCAs entre consecutivos na ordem de tin importam; a árvore virtual tem O(k) vértices e preserva todas as distâncias entre os especiais.
  - *Como fazer:* ordene os k vértices por tin; acrescente o LCA de cada par consecutivo (e o primeiro com o último, se necessário); ordene tudo por tin e conecte com uma pilha. Arestas virtuais têm peso = distância original. O(k log k) por query.
  - *Exemplo:* soma das distâncias entre todos os pares de um conjunto; menor subárvore que contém os k vértices; DP de cortar especiais.
  - *Armadilha:* incluir a raiz quando ela não é especial (só se o DP precisar); limpar os arrays usados sem varrer O(n); usar LCA O(1) para não pagar mais um log.
- **Floresta/grafo quase árvore** → ache o ciclo único (m = n) e trate as árvores penduradas.
  - *Gatilho:* "grafo conexo com n vértices e n arestas", "existe exatamente um ciclo", "duas rotas entre pares".
  - *Por que funciona:* removendo uma aresta do ciclo único o grafo vira árvore; todo vértice está numa árvore pendurada a um vértice do ciclo, e qualquer caminho usa no máximo um arco do ciclo.
  - *Como fazer:* ache o ciclo com DFS (primeira aresta de retorno) ou DSU (primeira aresta que fecha ciclo); marque os vértices do ciclo e suas árvores; para distância, combine o caminho dentro da árvore com o menor dos dois arcos do ciclo.
  - *Exemplo:* distância em grafo unicyclic; contar caminhos que passam pelo ciclo.
  - *Armadilha:* se m > n há mais de um ciclo independente; autoloops e arestas múltiplas quebram a detecção; o "ciclo" achado pela DFS pode incluir cauda se você não usar a pilha corretamente.

adicionado em 2026-11-10 por agentes; revisado

- **Euler tour de aresta (2n − 2 posições) ou de vértice (2n − 1); eventos tin/tout = 2n.** Registre o vértice na entrada e saída; subárvore = intervalo; caminho u–v em Mo ou em BIT de ±: guarde +x em tin e −x em tout para "soma de caminho raiz–v".
  - *Gatilho:* queries offline de caminho, "soma da raiz até v", "subárvore é intervalo", Mo em árvore.
  - *Por que funciona:* a DFS entra e sai de cada vértice; com +a[v] na entrada e −a[v] na saída, o prefixo em tin[v] contém exatamente os ancestrais de v (os ramos já fechados se cancelam), ou seja, a soma raiz–v. A subárvore, por sua vez, é o intervalo entre entrada e saída.
  - *Como fazer:* guarde tin/tout; para soma raiz–v use um BIT com +a[v] em tin[v] e −a[v] em tout[v] e consulte o prefixo em tin[v]; para Mo em caminho, use a Euler de vértices de 2n − 1 posições e o truque de alternar ocorrências (nós com ocorrência ímpar entram, e o LCA é tratado à parte).
  - *Exemplo:* soma no caminho u–v = pre[u] + pre[v] − 2·pre[lca] + a[lca], com pre = prefixo da raiz.
  - *Armadilha:* são três Euler tours diferentes (vértices 2n − 1, arestas 2n − 2, eventos 2n); usar a errada para Mo ou para subárvore dá resposta errada. No BIT de ±, a subárvore não é a soma do intervalo (ela dá zero); a soma de subárvore usa só +a em tin.
- **LCA com sparse table sobre ordem de DFS (n posições, sem 2n).** Para u ≠ v com tin[u] < tin[v], LCA = pai do vértice de menor profundidade em (tin[u], tin[v]]; meia memória e O(1).
  - *Gatilho:* muitas queries de LCA, memória apertada, árvore estática.
  - *Por que funciona:* no preorder, todos os vértices entre u e v pertencem à subárvore do LCA; o de menor profundidade nesse intervalo é filho do LCA no ramo que leva a v (ou o próprio v, se u é ancestral), então seu pai é o LCA.
  - *Como fazer:* faça uma DFS e guarde preorder[t] = vértice, tin e profundidade; construa sparse table de argmin sobre profundidade; para u ≠ v, seja l = min(tin[u], tin[v]) + 1 e r = max(tin[u], tin[v]); LCA = pai[argmin(l, r)]. O(n log n) memória, O(1) por query.
  - *Exemplo:* u ancestral de v: o mínimo em (tin[u], tin[v]] é um filho de u (ou v), e o pai é u.
  - *Armadilha:* incluir tin[u] no intervalo faz o mínimo ser u quando u é ancestral, e pai[u] dá errado; use o intervalo aberto à esquerda. Para u = v, responda u diretamente.

- **LCA offline (Tarjan)** → DFS + DSU: ao sair de v, una v ao pai; para query (u, v) com u já visitado, LCA = find(u). Útil quando memória é restrita.
  - *Gatilho:* q queries de LCA conhecidas de antemão, memória curta, sem updates.
  - *Por que funciona:* quando a DFS termina v, toda a subárvore de v já foi unida a v; se u está em outra subárvore já visitada, o representante de u é o ancestral comum mais baixo que ainda está ativo, isto é, o LCA.
  - *Como fazer:* DFS com visited; ao sair de v, union(v, pai[v]); para cada query (v, u) pendente em v, se visited[u], responda find(u). O((n + q)·α(n)).
  - *Exemplo:* $2 \cdot 10^5$ queries de LCA com memória de alguns MB.
  - *Armadilha:* responder a mesma query nos dois sentidos (armazene nos dois e responda uma vez); unir v ao pai antes de terminar os filhos quebra; a DSU precisa de compressão de caminho.
- **LCA em árvores dinâmicas (adicionar folha online)** → binary lifting preenchido na criação (O(log) por folha) ou skew-binary jump pointers (O(1) memória extra por vértice, O(log) por query).
  - *Gatilho:* a árvore cresce com folhas novas e as queries chegam intercaladas.
  - *Por que funciona:* a tabela de saltos de um vértice novo depende só do pai; preencher up[k][v] = up[k−1][up[k−1][v]] na criação mantém todas as queries antigas válidas.
  - *Como fazer:* ao criar v com pai p, defina up[0][v] = p e calcule up[k][v] para k = 1..⌊log n⌋; profundidade[v] = profundidade[p] + 1; a query é a usual. Skew-binary mantém O(1) ponteiros por vértice e O(log) por salto.
  - *Exemplo:* inserir folhas e perguntar LCA/ancestralidade online.
  - *Armadilha:* arrays estáticos dimensionados para o máximo de vértices; recalcular a tabela inteira a cada inserção é O(n log n); a altura pode crescer e o log máximo precisa acompanhar.
- **k-ésimo ancestral** → binary lifting O(log); O(1) com ladder decomposition + binary lifting (long-path); offline: DFS com pilha de caminho atual (ancestral à profundidade d − k = stack[d − k]).
  - *Gatilho:* "suba k arestas", "k-ésimo ancestral", "nível d", saltos repetidos.
  - *Por que funciona:* k em binário se decompõe em potências de 2; cada bit ligado é um salto de 2^i. Offline, a pilha da DFS contém exatamente o caminho raiz–v, então o ancestral está no índice d − k.
  - *Como fazer:* up[i][v]; para i de 0 a log, se k tem bit i, v = up[i][v]; O(log). Ladder decomposition + binary lifting responde em O(1) com O(n) memória. Offline, responda no tin de v usando stack[d − k].
  - *Exemplo:* k = 0 → o próprio v; k = d[v] → a raiz.
  - *Armadilha:* k > profundidade → resposta inexistente (−1/0); a pilha offline só existe enquanto a DFS está dentro da subárvore de v; não confunda k em arestas com k em vértices.
- **Distância em árvore** = d[u] + d[v] − 2·d[lca]; **k-ésimo vértice no caminho u–v** = sobe de u se k ≤ d[u] − d[lca], senão sobe de v o complemento.
  - *Gatilho:* "distância entre u e v", "vértice na posição k do caminho", "quantos vértices no caminho".
  - *Por que funciona:* o caminho se divide no LCA; a distância é a soma dos dois trechos, e o k-ésimo vértice está no trecho u→lca ou no trecho v→lca.
  - *Como fazer:* D = dep[u] + dep[v] − 2·dep[l]; se k ≤ dep[u] − dep[l], suba k de u; senão, a posição a partir de v é D − k, então suba D − k de v. O(log) com binary lifting.
  - *Exemplo:* k = 0 → u; k = D → v; k = dep[u] − dep[l] → lca.
  - *Armadilha:* off-by-one entre "k arestas" e "k-ésimo vértice"; quando k cai exatamente no LCA, subir de u ou de v deve dar o mesmo nó; não suba além da raiz.
- **Soma/xor/qualquer grupo abeliano em caminho** → prefixo da raiz (valores em vértices): f(u) + f(v) − 2·f(lca) + a[lca]; para xor, f(lca) aparece duas vezes e cancela: pre[u] ⊕ pre[v] ⊕ a[lca]. Com valores em arestas não há o termo a[lca].
  - *Gatilho:* muitas queries de soma/xor/produto (com inverso) no caminho, árvore estática.
  - *Por que funciona:* f(v) é a agregação raiz–v; f(u) + f(v) conta o caminho u–v duas vezes os trechos comuns até o LCA e uma vez os trechos exclusivos; subtrair 2·f(lca) remove a duplicação, e o valor do LCA precisa ser recolocado (ou não, para xor, pois aparece duas vezes).
  - *Como fazer:* pré-calcule pre[v] em O(n); cada query em O(1) com LCA. Para arestas, guarde o valor da aresta no filho e use pre[u] + pre[v] − 2·pre[lca].
  - *Exemplo:* xor de vértices no caminho = pre[u] ⊕ pre[v] ⊕ a[lca]; soma de arestas = pre[u] + pre[v] − 2·pre[lca].
  - *Armadilha:* a operação precisa ser grupo abeliano com inverso; max/min não têm inverso; com valores em arestas, o termo do LCA não entra; com valores em vértices, entra.

- **Max/min em caminho sem updates** → binary lifting guardando (pai 2^k, max); ou HLD + sparse table nas cadeias; ou Kruskal reconstruction tree quando o peso está nas arestas e query é minimax.
  - *Gatilho:* "maior/menor peso no caminho", "minimizar o máximo", muitas queries sem updates.
  - *Por que funciona:* max é associativo e idempotente, então pode ser acumulado nos saltos; a KRT ordena as arestas por peso e transforma "minimax entre u e v" no valor do LCA na árvore de união.
  - *Como fazer:* up[k][v] e mx[k][v] = max(mx[k−1][v], mx[k−1][up[k−1][v]]); suba u e v equalizando profundidades e depois juntos, combinando os máximos. KRT: ordene as arestas, crie um nó novo por união com o peso; o minimax é o valor do LCA. O(log) por query.
  - *Exemplo:* "peso máximo na rota" em uma árvore; "qual o menor gargalo para ir de u a v".
  - *Armadilha:* max não é invertível, então não tente "total − filho"; na KRT, o valor do LCA é o minimax, não a soma; arestas com pesos repetidos exigem desempate consistente.
- **Árvore com updates em aresta** → coloque o peso da aresta no filho; update em subárvore/caminho como Euler tour/HLD.
  - *Gatilho:* updates de peso de aresta e queries de caminho/subárvore.
  - *Por que funciona:* cada aresta tem um filho único; guardar o peso em val[filho] converte update de aresta em update de vértice, e o caminho u–v usa exatamente os vértices do caminho menos o LCA.
  - *Como fazer:* val[v] = peso(pai[v], v); update da aresta (pai[v], v) = point update em val[v]; query de caminho = agregação de val nos vértices de u a v, excluindo o LCA. Com HLD, cuidado com a posição do topo da cadeia.
  - *Exemplo:* soma de pesos de arestas no caminho; maior aresta no caminho com updates.
  - *Armadilha:* incluir a aresta do LCA para o pai; a raiz não tem aresta; em HLD, ao subir uma cadeia, o intervalo começa em pos[top] + 1 para não incluir a aresta do topo para o pai.
- **HLD com subárvore também por segtree** → numere filho pesado primeiro; subárvore = [pos[v], pos[v] + sz[v] − 1], cadeia = intervalo contíguo; ambos funcionam no mesmo array.
  - *Gatilho:* precisa de queries de caminho e de subárvore com updates no mesmo problema.
  - *Por que funciona:* a DFS2 que entra primeiro no filho pesado numera cada cadeia contiguamente e, ao mesmo tempo, mantém a subárvore contígua; as duas propriedades coexistem no mesmo pos[].
  - *Como fazer:* DFS1 calcula sz e heavy; DFS2 atribui pos em pré-ordem entrando no pesado primeiro; segtree sobre pos; subárvore = [pos[v], pos[v] + sz[v] − 1]; caminho = O(log n) intervalos [pos[top], pos[v]].
  - *Exemplo:* soma de subárvore e soma de caminho com point updates.
  - *Armadilha:* a subárvore só é contígua se o pesado for visitado primeiro; a cadeia é [pos[top], pos[v]], não [pos[v], pos[top]]; o topo da cadeia pertence a ela.
- **HLD: query não comutativa** → mantenha lado esquerdo e direito separados (u→lca e v→lca invertido) e combine na ordem do caminho.
  - *Gatilho:* concatenação de strings, composição de matrizes/funções, hash direcional no caminho.
  - *Por que funciona:* subir de u produz segmentos na ordem u→lca; subir de v produz segmentos na ordem v→lca, que é o inverso do que o caminho pede. Guardar os dois lados e inverter o lado v no final restaura a ordem correta.
  - *Como fazer:* acumule L (da ponta u para o LCA, na ordem) e R (da ponta v para o LCA); a cada cadeia, combine o segmento na orientação certa; ao final, resposta = L + inverso(R). Para matrizes, multiplique na ordem correspondente.
  - *Exemplo:* hash de string no caminho; composição de permutações; soma com sinal dependente da direção.
  - *Armadilha:* inverter também a ordem dentro de cada segmento da cadeia; usar a identidade errada de um dos lados; o LCA não deve ser contado duas vezes.
- **HLD offline com 2 Fenwicks para "adicione x no caminho, some caminho"** → diferenças no array HLD; por contribuição do vértice × tamanho: Σ_{v em caminho} a_v = prefixos da raiz.
  - *Gatilho:* updates de "soma x no caminho" e queries de "soma no caminho", sem exigir online.
  - *Por que funciona:* cada caminho se decompõe em O(log n) intervalos de cadeias; em cada intervalo, range add + range sum é resolvido por dois BITs (o primeiro guarda a diferença, o segundo corrige o peso linear).
  - *Como fazer:* para cada cadeia, BIT1 e BIT2 para range add/range sum; add(l, r, x): B1.add(l, x), B1.add(r+1, −x), B2.add(l, x·(l−1)), B2.add(r+1, −x·r); soma até i = B1.sum(i)·i − B2.sum(i). O($\log^2 n$) por operação de caminho.
  - *Exemplo:* somar 5 em u–v e depois perguntar a soma em u–v.
  - *Armadilha:* não confundir com point update + path sum, que se resolve com prefixo da raiz; range add + range sum exige os dois BITs. A formulação por "contribuição do vértice × tamanho" só vale numa versão offline específica e não substitui os dois BITs no caso geral.

- **Link-cut tree** quando a árvore muda (link/cut) e ainda há queries de caminho; `access/makeroot/splay` → O(log n) amortizado; também mantém MST incremental (só inserções; MST totalmente dinâmico exige D&C/ETT) com arestas ordenadas por peso (max no caminho), "o grafo é floresta?", e subárvore virtual (info de subárvore).
  - *Gatilho:* link/cut de arestas, queries de caminho intercaladas com mudanças, MST que só recebe arestas.
  - *Por que funciona:* cada árvore é representada por caminhos preferidos em splay trees; `access(v)` expõe o caminho raiz–v como uma splay, e as rotações mantêm os agregados (soma, max) em O(log n) amortizado.
  - *Como fazer:* implemente `access`, `makeroot` (inverte o caminho), `findroot`, `link` e `cut`; para query de caminho, `makeroot(u)`, `access(v)` e leia o agregado da splay de v. Para MST incremental, guarde a aresta de peso máximo no caminho: se a nova aresta (u,v,w) fecha ciclo e o máximo > w, corte o máximo e ligue a nova.
  - *Exemplo:* conectar/desconectar componentes e perguntar soma ou máximo no caminho; manter MST com inserções de arestas.
  - *Armadilha:* constantes altas e implementação delicada; LCT não dá subárvore comum de graça (precisa de "virtual subtree" e propagação de contagem); MST totalmente dinâmico (com deleções arbitrárias) não é só LCT.
- **Top tree / Euler tour tree** → estruturas para subárvore + dinâmico; ETT (árvore de Euler em balanced BST) deixa link/cut e consulta de componente/soma em O(log n); top tree é o caso geral (diâmetro dinâmico, centro dinâmico).
  - *Gatilho:* precisa de link/cut e também de informação de subárvore ou de componente; diâmetro/centro mudam com updates.
  - *Por que funciona:* a ETT guarda a sequência de entrada/saída da Euler tour numa BST balanceada; cortar uma aresta é dividir a sequência, e ligar é concatenar, mantendo agregados por componente. A top tree agrupa clusters e permite agregados de caminho e de subárvore ao mesmo tempo.
  - *Como fazer:* ETT com um nó por aresta (ou por ocorrência), split/merge para link/cut, agregados de soma/contagem; top tree com clusters e `expose`/`merge` para consultas. O(log n) amortizado.
  - *Exemplo:* ligar duas árvores e consultar a soma de um componente; manter o diâmetro de uma floresta dinâmica.
  - *Armadilha:* ETT responde bem componente, mas caminho arbitrário é mais natural em LCT; top tree é complexa e raramente necessária; a memória da ETT é O(n) mas com constante alta.
- **Diâmetro: propriedades.** Todos os diâmetros se cruzam no centro (1 ou 2 vértices); a distância máxima de v = max(d(v, a), d(v, b)) com (a, b) diâmetro; ao unir duas árvores por uma aresta (u, v), o novo diâmetro tem pontas entre os 4 candidatos {a1, b1, a2, b2}; vale também para união de conjuntos (distância máxima mergeable).
  - *Gatilho:* unir árvores/componentes, responder distância máxima de cada vértice, manter diâmetro com DSU.
  - *Por que funciona:* todo diâmetro passa pelo centro da árvore; a partir de v, o caminho mais longo termina numa das duas pontas de um diâmetro fixo. Ao unir por (u,v), qualquer caminho novo cruza essa aresta, então suas pontas são pontas antigas dos dois lados.
  - *Como fazer:* guarde (a, b, D) por componente; ao unir, teste D1, D2 e as quatro distâncias cruzadas d(a1,a2), d(a1,b2), d(b1,a2), d(b1,b2) (cada uma via LCA); pegue o máximo e as pontas correspondentes. Para distância máxima de cada v, use max(d(v,a), d(v,b)).
  - *Exemplo:* DSU que une componentes e responde o diâmetro do componente; "qual a maior distância depois de ligar as duas árvores".
  - *Armadilha:* você precisa das distâncias entre as quatro pontas, não só de D1 e D2; o centro pode ser uma aresta (dois centros); a propriedade vale para árvores, não para grafos com ciclos.
- **Two-sweep não vale com pesos negativos** (use DP); com pesos ≥ 0, dois DFS/Dijkstra valem (BFS só sem pesos); para "diâmetro após remover vértice", use prefixo/sufixo ou rerooting guardando os 2–3 maiores filhos.
  - *Gatilho:* arestas com pesos possivelmente negativos; "diâmetro se removermos v"; "maior caminho" em árvore com pesos.
  - *Por que funciona:* com peso negativo, o vértice mais distante de s pode não ser ponta de diâmetro, então o two-sweep escolhe a ponta errada. O DP de diâmetro (maior e segundo maior ramo) não depende de sinal.
  - *Como fazer:* down[v] = max(0, max over filhos (w + down[c])); em cada v, o melhor caminho que passa por v é a soma dos dois maiores (w + down[c]); o diâmetro é o máximo disso (ou 0, se todos negativos). Para remover v, combine prefixo/sufixo das contribuições dos filhos ou faça rerooting guardando os 2–3 maiores.
  - *Exemplo:* árvore com uma aresta de peso −10: o diâmetro pode ser 0 (um único vértice); dois DFS dariam um caminho negativo.
  - *Armadilha:* dois DFS só valem com pesos ≥ 0; BFS só sem pesos; ao remover um vértice, não recalcule de zero para cada um.

- **Raio = ⌈diâmetro/2⌉** (só com arestas unitárias; com pesos, raio ≥ diam/2 e não há fórmula fechada); centro minimiza a excentricidade; ao adicionar uma folha por aresta de peso w, o diâmetro cresce no máximo w e atualiza-se comparando distâncias às pontas atuais.
  - *Gatilho:* "raio", "centro", "excentricidade", "adicionar folhas e manter o diâmetro".
  - *Por que funciona:* com arestas unitárias, o meio do diâmetro é um vértice ou aresta e a maior distância a partir dele é ⌈D/2⌉. Com pesos, o ponto que minimiza a excentricidade pode cair no interior de uma aresta; restrito a vértices, o raio só pode ser ≥ D/2. Ao ligar uma folha x por peso w, qualquer distância nova é dist(x,y) ≤ dist(origem,y) + w ≤ D + w.
  - *Como fazer:* ache o diâmetro (a,b); suba ⌈D/2⌉ para o centro unitário. Para pesos, caminhe a partir de a até a metade do diâmetro para achar o centro geométrico; se só vértices valem, teste os dois extremos da aresta central. Para folha nova, compare D com d(x,a) e d(x,b).
  - *Exemplo:* D = 5 unitário → raio 3; aresta de peso 10 com D = 10 e centro no meio da aresta → nenhum vértice tem excentricidade 5.
  - *Armadilha:* com pesos, raio ≠ ⌈D/2⌉; "centro" (minimiza o máximo) ≠ "centroide" (minimiza a soma); a atualização da folha precisa das distâncias às pontas atuais, não só de D.
- **Distâncias a k (todos os vértices com dist exata k)** → rerooting guardando contagens por profundidade; para k fixo, DP com dp[v][d] só até d ≤ k: O(nk) tempo/memória (O(n) só com long-path decomposition ou k constante).
  - *Gatilho:* "quantos vértices a distância exatamente k de v", "pares a distância k", k pequeno ou fixo.
  - *Por que funciona:* dp[v][d] conta descendentes de v a distância d; para incluir o resto, o rerooting combina a contribuição do pai deslocada de 1. Para um k fixo, nunca é preciso olhar d > k.
  - *Como fazer:* dp[v][0] = 1; dp[v][d] = Σ dp[filho][d−1]; uma segunda DFS passa a contribuição de fora (up[v][d]) e a resposta é dp[v][k] + up[v][k]. Trunque em k. O(nk).
  - *Exemplo:* contar pares (u,w) com distância exatamente k combinando, no LCA, os ramos de dois filhos.
  - *Armadilha:* O(nk) memória estoura para k grande; para todos os d de uma vez, a saída é $O(n^2)$; long-path decomposition resolve as contagens de subárvore em O(n), mas não todas as distâncias de todos os v em O(n); não confunda "exatamente k" com "≤ k".
- **dp[v][d] "profundidade" com merge O(n)** → long-path decomposition: herde o vetor do filho de maior altura e some os outros elementos em O(altura); resolve "contar vértices na subárvore com dist exata d" e "profundidade mais comum" em O(n).
  - *Gatilho:* vetores indexados por profundidade na subárvore, "cor mais comum em cada nível", "contagem por distância".
  - *Por que funciona:* o vetor do filho mais alto é reutilizado com um offset; os vetores dos outros filhos são menores e são somados ao herdado; a soma dos tamanhos dos vetores pequenos ao longo da árvore é O(n) (soma geométrica por long-path).
  - *Como fazer:* escolha o filho de maior altura; herde o ponteiro do vetor dele deslocando o offset em 1; some os vetores dos demais; atualize a resposta global (máximo de frequência, etc.). O(n) tempo e memória.
  - *Exemplo:* para cada v, a profundidade (relativa a v) mais comum na subárvore.
  - *Armadilha:* não copie o vetor do filho pesado (seria $O(n^2)$); controle o offset (índices podem ficar negativos se usar um vetor global); a altura é a chave, não sz.
- **Small-to-large por cadeia pesada (DSU on tree/sack)**: mantenha a estrutura do filho pesado, adicione todos os vértices das subárvores leves, limpe só nos leves; O(n log n) com contadores em vetor.
  - *Gatilho:* "para cada subárvore, o valor mais frequente/distinto", sem updates.
  - *Por que funciona:* o filho pesado é mantido; cada vértice de subárvore leve é adicionado de novo, mas o tamanho da subárvore em que ele está pelo menos dobra a cada vez, então ele é reinserido O(log n) vezes.
  - *Como fazer:* DFS: processe os filhos leves com keep = false, o pesado com keep = true; adicione as subárvores leves ao contador; responda em v; se keep = false, remova a subárvore inteira. O(n log n).
  - *Exemplo:* cor dominante por subárvore; número de cores distintas.
  - *Armadilha:* limpar só quando keep = false, e limpar a subárvore inteira (não só os leves); o máximo corrente precisa ser recalculado ao remover; usar vetor de frequência, não map, para manter O(1).

- **Small-to-large com sets e queries "k-ésimo na subárvore" ou "primeiro ≥ x"** → merge de sets/mapas; ou merge de segtrees dinâmicas (O(n log n) total) com consulta na raiz.
  - *Gatilho:* "k-ésimo menor valor na subárvore", "menor valor ≥ x", merge de conjuntos por subárvore.
  - *Por que funciona:* cada elemento muda de conjunto O(log n) vezes no merge pequeno→grande, dando O(n $\log^2$ n) com set; a segtree dinâmica merge em O(n log V) total e responde k-ésimo em O(log V).
  - *Como fazer:* um set/mapa por vértice; merge do menor para o maior; para k-ésimo, use PBDS/ordered set ou segtree de contagem; para "primeiro ≥ x", lower_bound no set ou descida na segtree. Alternativa: segtree dinâmica por valor e merge(a,b).
  - *Exemplo:* para cada v, o k-ésimo menor valor da subárvore; o menor valor ≥ x.
  - *Armadilha:* std::set não tem k-ésimo em O(log n) (use PBDS/`order_of_key`); o merge de sets pequenos é O(n $\log^2$ n), não O(n log n); a segtree dinâmica gasta O(n log V) memória.
- **Merge de segtree dinâmica** para "para cada vértice, o mais frequente / soma de conjunto" em subárvore: cada operação cria O(log) nós e o merge custa O(total de nós) amortizado.
  - *Gatilho:* agregações por subárvore sobre valores (frequência, soma, distintos) com n e V grandes.
  - *Por que funciona:* cada valor inserido cria O(log V) nós; o merge de duas segtrees visita só os nós sobrepostos e nunca cria nós novos, então o custo total é proporcional ao número de nós criados, O(n log V).
  - *Como fazer:* segtree de contagem por valor; `merge(a,b)`: se a ou b é nulo, retorne o outro; senão some as contagens e recursão nos filhos; a resposta de v é a raiz após mergir os filhos. O(n log V).
  - *Exemplo:* moda por subárvore; soma de valores distintos; contar valores em faixa por subárvore.
  - *Armadilha:* memória O(n log V) (não recicle nós entre testes); o merge destrói uma das árvores; para moda, guarde (freq, valor) e desempate.
- **Centroide: propriedades.** Todo componente ao remover o centroide tem ≤ n/2 vértices; existem 1 ou 2 centroides (2 ⇔ aresta que divide n/2 e n/2); o centroide minimiza Σ dist (mediana da árvore): ele é a "mediana ponderada".
  - *Gatilho:* "vértice cuja remoção deixa componentes balanceados", "mediana da árvore", decomposição por centroide.
  - *Por que funciona:* se um componente tivesse > n/2, descer para o vizinho desse componente diminuiria o maior componente; no ponto de equilíbrio todos têm ≤ n/2. O gradiente de Σ dist ao mover de v para um vizinho é n − 2·sz(componente), que é ≥ 0 no centroide, então ele minimiza a soma.
  - *Como fazer:* calcule sz; desça a partir da raiz enquanto existir filho com sz > n/2; o vértice atual é centroide. Se uma aresta divide n/2 e n/2, os dois extremos são centroides. O(n).
  - *Exemplo:* estrela K1,4: o centro é o centroide; caminho de 4 vértices: os dois vértices do meio são centroides.
  - *Armadilha:* "centro" (meio do diâmetro, minimiza o máximo) ≠ "centroide" (minimiza a soma); com pesos nos vértices, o centroide ponderado é outro; a cota ≤ n/2 é do componente após remover, não da subárvore enraizada.
- **Árvore de centroides (centroid tree)** → altura ≤ log n; para query de distância ao vértice marcado mais próximo, guarde best[c] e, para cada ancestral c do vértice v, dist(v, c) + best[c]; pré-calcule dist(v, c) por nível (O(n log n) memória).
  - *Gatilho:* marcar/desmarcar vértices e perguntar a distância ao marcado mais próximo.
  - *Por que funciona:* cada componente é dividido por um centroide em partes ≤ n/2, então a profundidade da centroid tree é $\lfloor \log_2 n \rfloor$. Todo caminho de v a um marcado passa por algum ancestral de v na centroid tree, e o mínimo sobre os ancestrais captura o mais próximo.
  - *Como fazer:* decomponha recursivamente; para cada centroide c e cada vértice v do componente, guarde dist(v,c) no nível; best[c] = min dist(c, marcado) no componente de c (atualize ao marcar, subindo os ancestrais); query: min sobre ancestrais c de v de dist(v,c) + best[c].
  - *Exemplo:* inserir pontos e consultar o mais próximo; desmarcar exige multiset ou recomputação.
  - *Armadilha:* memória O(n log n) para as distâncias; atualizar best ao desmarcar (use multiset/heap preguiçosa); dist(v,c) é a distância na árvore original, não o nível na centroid tree.
- **Centroid tree com soma de distâncias de v a todos os marcados** → em cada centroide c guarde (cnt, sum de dist ao c) e (cnt, sum de dist ao pai de c na centroid tree) para excluir o subcentroide; query em O(log n).
  - *Gatilho:* "soma das distâncias de v a todos os ativos", com marcação/desmarcação.
  - *Por que funciona:* a soma em c conta todos os marcados do componente; os que estão no mesmo ramo de v foram contados no nível de c e serão contados de novo no nível seguinte, então é preciso subtrair a contribuição desse ramo.
  - *Como fazer:* em c guarde cnt[c], sum[c] (distâncias ao c) e, para o ramo do filho, cnt_sub, sum_sub; query: Σ sobre ancestrais c de (sum[c] + cnt[c]·dist(v,c)) − (sum_sub[prev] + cnt_sub[prev]·dist(v,c)), onde prev é o ancestral anterior. Atualização sobe os ancestrais. O(log n).
  - *Exemplo:* soma das distâncias a todos os vértices marcados.
  - *Armadilha:* o termo de exclusão é do ramo que contém v, não do pai; dist(v,c) precisa ser exata; ao desmarcar, subtraia nos dois acumuladores.

- **Centroid tree: caminho u–v passa por LCA na centroid tree** (o ancestral comum mais baixo) → distância(u, v) = dist(u, c) + dist(v, c) para c = LCA centroide (exato: c sempre está no caminho u–v).
  - *Gatilho:* queries de distância entre pares usando a centroid tree; "o LCA centroid está no caminho".
  - *Por que funciona:* o primeiro centroide que separa u e v (o LCA na centroid tree) foi removido quando u e v caíram em componentes diferentes; como ele estava no componente que continha ambos, está no caminho original entre eles.
  - *Como fazer:* pré-calcule dist(v,c) para cada ancestral c; c = LCA(u,v) na centroid tree (subindo por pai); resposta = dist(u,c) + dist(v,c). O(log n).
  - *Exemplo:* consultas de distância sem binary lifting; validar um LCA centroid.
  - *Armadilha:* não é o LCA da árvore original; se u e v estão no mesmo ramo em todos os níveis, o LCA centroid é mais fundo; a igualdade dist(u,v) = dist(u,c)+dist(v,c) só vale para esse c.
- **Contar caminhos com soma de peso ≤ K ou comprimento = k via centroide** → para cada centroide, ordene as distâncias e use two pointers; subtraia o par da mesma subárvore filha (ou faça por subárvore com BIT). Com comprimento exato k e k pequeno, use vetor de contagem por profundidade e processe filhos do menor para o maior.
  - *Gatilho:* "conte pares com distância ≤ K", "caminho de comprimento exatamente k", sem updates.
  - *Por que funciona:* todo caminho é contado no primeiro centroide que separa suas pontas; para pares em ramos diferentes, a distância é a soma das distâncias ao centroide; pares no mesmo ramo são contados (e subtraídos) para não duplicar.
  - *Como fazer:* para cada centroide, colete as distâncias por filho; para "≤ K", junte todas, ordene e use two pointers, subtraindo os pares de cada filho; para "= k", use um vetor de contagem de profundidades até k e combine cada filho com os anteriores, processando filhos do menor para o maior. O(n $\log^2$ n) com ordenação por centroide (O(n log n) se as distâncias são limitadas e usam counting); o vetor de contagem é limitado pela profundidade máxima do componente e por k.
  - *Exemplo:* contar pares a distância ≤ K; contar caminhos de comprimento exatamente k.
  - *Armadilha:* não conte (u,u); subtraia a mesma subárvore com as mesmas distâncias; K negativo não tem pares; no caso "= k", limpe o vetor de contagem entre centroides sem varrer O(n).
- **Contar caminhos com condição "produto/mod"** → centroide + hash map: guarde o valor prefixo de v até c e o sufixo para combinar (inverso modular) em O(n log n).
  - *Gatilho:* "produto dos valores no caminho ≡ x (mod p)", "conte caminhos com produto".
  - *Por que funciona:* o produto é um grupo quando os valores são invertíveis; no centroide c, defina pref(v) = produto dos valores de v até c, excluindo c; o produto do caminho u–v é pref(u)·pref(v)·a[c], então para cada pref(u) procura-se o alvo multiplicado pelo inverso.
  - *Como fazer:* em cada centroide, colete pref(v) = produto de v até c, excluindo c (pref(c) = 1); para cada v, procure no hash map a frequência de alvo·inv(pref(v)·a[c]); insira os do mesmo filho depois de consultar, ou subtraia por filho. O(n log n).
  - *Exemplo:* contar caminhos com produto ≡ 1 mod 10^9+7.
  - *Armadilha:* se algum valor for 0 ou não coprimo com o módulo, não existe inverso; trate zeros e fatores separadamente; o produto pode ser calculado com multiplicação modular em long long.
- **Contribuição por aresta** → número de caminhos através da aresta = sz·(n − sz); soma de distâncias entre todos os pares = Σ_e w_e·sz_e·(n − sz_e); valor esperado de distância em árvore aleatória idem.
  - *Gatilho:* "soma de todas as distâncias", "número de caminhos que usam a aresta", "esperança da distância".
  - *Por que funciona:* a aresta separa a árvore em dois lados de tamanhos sz e n − sz; um caminho a usa exatamente quando as pontas estão em lados opostos, e há sz·(n − sz) pares assim.
  - *Como fazer:* uma DFS calcula sz; some w·sz·(n − sz); para a esperança entre um par uniforme, divida por C(n,2); se o sorteio for ordenado com repetição, divida por n^2. O(n).
  - *Exemplo:* árvore com 4 vértices em estrela e pesos 1: cada aresta contribui 3, total 9.
  - *Armadilha:* a fórmula é para pesos em arestas; para valores em vértices, a contribuição é por vértice; para caminhos com restrição (≤ K), a contribuição por aresta não decide a condição.
- **Caminhos que passam por um conjunto de vértices (queries com k vértices)** → árvore virtual (auxiliary tree) ordenando por tin, adicionando LCAs de adjacentes, ligando por pilha; Σk ≤ $2 \cdot 10^5$ → O(Σk log).
  - *Gatilho:* cada query recebe k vértices e pergunta algo sobre a menor subárvore que os contém ou sobre caminhos entre eles.
  - *Por que funciona:* os únicos vértices que podem ser ramificação são os k especiais e os LCAs de pares consecutivos na ordem de tin; todos os outros têm grau 2 e podem ser comprimidos numa aresta ponderada.
  - *Como fazer:* ordene por tin; insira o LCA de cada par consecutivo; ordene de novo e conecte com pilha (ou use a pilha diretamente); o peso da aresta virtual é a distância original. O(k log k) por query, O(Σk log) no total.
  - *Exemplo:* soma das distâncias entre todos os pares de um conjunto; menor subárvore que contém todos; contar caminhos que passam por pelo menos um especial.
  - *Armadilha:* incluir a raiz só se necessário; limpar os vetores usados sem varrer O(n); usar LCA O(1) para não pagar mais um log; o conjunto pode ter duplicatas.

- **Virtual tree + DP** → dp[v] só sobre vértices relevantes, com arestas ponderadas pela distância original; tipicamente "mínimo de cortes/custo para separar os especiais da raiz": defina dp[v] = custo de separar v do pai; se v é especial é obrigatório cortar acima dele (dp[v] = menor peso no caminho pai→v, não min com os filhos); senão dp[v] = min(menor peso no caminho pai→v, Σ dp filhos).
  - *Gatilho:* "cortar arestas para isolar todos os vértices marcados da raiz", "custo mínimo de separar".
  - *Por que funciona:* se v é especial, cortar abaixo de v não o separa do pai; é preciso cortar em algum lugar do caminho v→pai, e o mais barato é o menor peso desse caminho. Se v não é especial, ou se corta acima dele, ou se cortam todos os ramos com especiais.
  - *Como fazer:* construa a árvore virtual; pré-calcule o menor peso no caminho da raiz a cada vértice (ou acumule ao descer); pós-ordem: se v é especial, dp[v] = min_path(v); senão dp[v] = min(min_path(v), Σ dp[filhos]); a resposta na raiz é Σ dp[filhos] (a raiz não tem aresta para o pai). O(k log k).
  - *Exemplo:* cortar o mínimo de peso para separar todas as folhas marcadas da raiz.
  - *Armadilha:* se v é especial, não use min com os filhos (cortar abaixo não resolve); o menor peso precisa ser acumulado ao descer, não só o peso da aresta virtual; a raiz especial não tem pai (corte os filhos).
- **Rerooting com monoide não invertível** → prefixo/sufixo de filhos; com invertível, "total − contribuição"; para max, guarde os dois melhores filhos; para min/máx de duas coisas, combine por top-2.
  - *Gatilho:* "para cada raiz", operação max/min/gcd/AND/OR, sem inverso.
  - *Por que funciona:* para calcular a resposta do filho c, o pai precisa da combinação de todos os outros filhos; prefixo e sufixo respondem isso em O(1) por filho depois de O(grau) de pré-cálculo.
  - *Como fazer:* no pai, monte pref[i] e suf[i] das contribuições dos filhos; para o filho i, combine up[pai], pref[i−1] e suf[i+1]; para max, basta guardar o maior e o segundo maior. O(n) total.
  - *Exemplo:* diâmetro por raiz; maior soma de caminho que passa por cada vértice.
  - *Armadilha:* usar "total − filho" para max/gcd dá errado; cuidado com a identidade (0 para soma, −∞ para max, 0 para gcd); o caso de nenhum filho precisa de valor neutro.
- **Rerooting em árvore com estado "removi uma aresta"** → para cada aresta, resultado da componente de cima e de baixo: down[v] e up[v] e depois combine; para "melhor aresta para cortar" itere e combine as duas metades.
  - *Gatilho:* "para cada aresta, o valor nas duas componentes", "qual aresta cortar para maximizar/equilibrar".
  - *Por que funciona:* remover a aresta pai–v separa exatamente a subárvore de v (down) do resto (up); down e up são computáveis com duas DFS.
  - *Como fazer:* down em pós-ordem; up em pré-ordem combinando o pai e os irmãos; para cada aresta, combine down[v] com up[v] e avalie a resposta. O(n).
  - *Exemplo:* melhor aresta para cortar de modo que as somas das duas partes sejam mais equilibradas; maior produto dos tamanhos.
  - *Armadilha:* up[v] precisa incluir o pai e todos os irmãos, não só o avô; não confunda remover aresta com remover vértice; a resposta da aresta é uma combinação das duas metades, não só down.
- **Árvore → intervalos.** Ordem de DFS de subárvores gera intervalos aninhados (laminar); duas subárvores ou são disjuntas ou uma contém a outra, então qualquer família laminar de intervalos é uma árvore (use para contar/conectar com "pilha de intervalos").
  - *Gatilho:* "subárvores", "intervalos aninhados", "família laminar", "interseção de subárvores".
  - *Por que funciona:* a DFS atribui tin crescente e tout; se as subárvores se intersectam, o primeiro vértice visitado de uma está dentro da outra, logo uma está contida na outra.
  - *Como fazer:* tin/tout; u é ancestral de v se tin[u] ≤ tin[v] < tout[u]; para uma família laminar, ordene por início e use pilha; para contar interseções, use a estrutura de inclusão. O(n log n).
  - *Exemplo:* verificar se duas subárvores se intersectam; construir a árvore de inclusão de intervalos.
  - *Armadilha:* a ordem dos filhos é arbitrária, então os intervalos não são únicos; use < tout (não ≤); intervalos de subárvores de uma mesma árvore nunca se cruzam parcialmente.
- **Família laminar de conjuntos** → árvore de inclusão (pilha após ordenar por início e fim decrescente); contar pares de subconjunto = DP na árvore.
  - *Gatilho:* conjuntos aninhados, "quantos pares A ⊆ B", "árvore de inclusão".
  - *Por que funciona:* ordenar por início crescente e, em empate, fim decrescente faz com que todo conjunto que contém o atual já esteja na pilha; o topo é o menor conjunto que o contém.
  - *Como fazer:* ordene; percorra com pilha; cada conjunto é filho do topo que o contém; construa a árvore e faça DP (subtree size, contagem de pares). O(m log m).
  - *Exemplo:* dado um conjunto de intervalos laminares, contar pares (A,B) com A ⊆ B.
  - *Armadilha:* conjuntos iguais (duplicatas) precisam de desempate; a ordem de fim decrescente é essencial; intervalos que se cruzam parcialmente não formam família laminar.

- **Árvore cartesiana + small-to-large** → "para cada elemento como máximo, percorra o lado menor": total O(n log n) (usado em contar subarrays com condição em max e a resposta se parte).
  - *Gatilho:* "para cada a[i], conte subarrays em que ele é o máximo/mínimo", "soma dos máximos".
  - *Por que funciona:* a árvore cartesiana tem o máximo na raiz e as subárvores são os intervalos em que ele é máximo; cada subarray é contado no LCA; percorrer o lado menor e consultar o maior dá O(n log n).
  - *Como fazer:* construa a cartesiana com pilha O(n); para cada nó, percorra os elementos da subárvore menor e conte/consulte no lado maior (BIT, map); total O(n log n).
  - *Exemplo:* contar subarrays em que a[i] é o máximo e a soma é ≤ S; soma dos máximos de todos os subarrays.
  - *Armadilha:* elementos repetidos exigem desempate consistente (< de um lado, ≤ do outro); a cartesiana usa a comparação, não a posição; a resposta pode precisar de long long.
- **Árvore de Huffman / merge de menores** → heap: junte os dois menores; custo total = Σ pesos × profundidade; com pesos já ordenados faça em O(n) com duas filas. Variantes com "k-ária": adicione 0s para (n − 1) mod (k − 1) = 0.
  - *Gatilho:* "juntar arquivos", "codificação", "custo de merge", "menor custo total".
  - *Por que funciona:* juntar os dois menores é ótimo por exchange: em qualquer árvore ótima, os dois menores podem ser irmãos mais profundos; trocá-los para essa posição não aumenta o custo.
  - *Como fazer:* heap com os pesos; retire dois, some, acumule o custo e reinsira a soma; O(n log n). Se os pesos já estão ordenados, use duas filas (original e somas) e faça O(n). Para k-ário, complete com zeros até (n−1) mod (k−1) = 0.
  - *Exemplo:* codificação de Huffman; fundir k listas minimizando o custo total.
  - *Armadilha:* o custo é a soma de todos os merges, não só o último; no k-ário, sem os zeros a última fusão pode ter menos de k filhos e o guloso falha; use long long.
- **Árvore binária de busca com inserção aleatória** → altura esperada O(log n); treap: prioridade aleatória faz a forma ser a mesma de uma árvore cartesiana por prioridade, e alturas são independentes da ordem de inserção.
  - *Gatilho:* "inserção aleatória", "altura esperada", "treap", "árvore balanceada probabilística".
  - *Por que funciona:* com prioridades aleatórias, a raiz é o elemento de menor prioridade; recursivamente, a forma é a cartesiana das prioridades, e a altura esperada é O(log n) (≈ 4,3·ln n), independentemente da ordem de inserção.
  - *Como fazer:* treap com split/merge e prioridade aleatória de 64 bits; inserção/remoção em O(log n) esperado; ou insira os elementos em ordem aleatória numa BST comum.
  - *Exemplo:* BST com inserções aleatórias; treap para conjunto ordenado com split por valor.
  - *Armadilha:* a altura esperada é O(log n), mas o pior caso é O(n); use gerador aleatório de 64 bits, não `rand()`; a prioridade precisa ser única (ou desempate por chave).
- **Contar árvores com restrição de grau** → fórmula multinomial de Prüfer + GF; para "árvores com a aresta (u,v) obrigatória": 2·n^{n−3}.
  - *Gatilho:* "graus prescritos", "quantas árvores rotuladas", "aresta obrigatória", "florestas".
  - *Por que funciona:* a sequência de Prüfer tem n − 2 posições e o vértice i aparece d_i − 1 vezes; o número de sequências com essas multiplicidades é o multinomial (n−2)!/∏(d_i−1)!. A aresta obrigatória aparece em uma fração 2/n das árvores de Cayley, dando 2·n^{n−3}.
  - *Como fazer:* verifique Σd_i = 2(n−1); aplique a fórmula; para restrições por classe, use GF exponencial; para aresta obrigatória, contraia u e v e multiplique por 2 (ou use a fórmula direta). O(n) para avaliar.
  - *Exemplo:* n = 4, graus (1,1,2,2) → 2!/(0!0!1!1!) = 2 árvores; aresta (1,2) em n = 4 → 2·4 = 8 das 16 árvores.
  - *Armadilha:* Σd_i precisa ser 2(n−1); n = 1 e n = 2 são casos especiais; a fórmula da aresta obrigatória vale para n ≥ 3; "enraizada" multiplica por n.
- **Hash de árvore enraizada (AHU).** Canônico = id do multiset ordenado de ids dos filhos (map global de vetores → id); O(n log n); isomorfismo de árvores sem raiz → compare os hashes enraizados nos 1–2 centros; ou hash aleatório f(v) = 1 + Σ f(filho)·rnd[altura] (hash por altura).
  - *Gatilho:* "duas árvores têm a mesma forma", "isomorfismo", "subárvores iguais".
  - *Por que funciona:* o id canônico do multiconjunto ordenado dos filhos é um invariante completo por indução: duas árvores têm o mesmo id se, e só se, têm a mesma forma enraizada. Sem raiz, o centro (1 ou 2 vértices) é preservado por isomorfismo.
  - *Como fazer:* pós-ordem; ordene os ids dos filhos; procure o vetor no map global e atribua um id novo se não existir; O(n log n). Sem raiz, ache os 1–2 centros e compare os ids (ou o multiconjunto dos dois). O hash aleatório é O(n) e probabilístico.
  - *Exemplo:* comparar duas árvores de $2 \cdot 10^5$ vértices; contar formas distintas de subárvores.
  - *Armadilha:* ordenar os filhos é obrigatório; sem raiz, comparar nos dois centros e aceitar se qualquer um bater; o hash aleatório pode colidir e depende de 64 bits; alturas iguais com estruturas diferentes precisam de randomização independente.

- **Subárvores isomórficas dentro da mesma árvore** → mesmo id canônico; contar classes de simetria de uma árvore = ids distintos; número de automorfismos = ∏ (grupos de filhos isomorfos)! · ∏ automorfismos dos filhos.
  - *Gatilho:* "quantas subárvores têm a mesma forma", "simetrias", "automorfismos".
  - *Por que funciona:* o id AHU identifica a forma; filhos com o mesmo id podem ser permutados livremente, gerando (multiplicidade)! automorfismos, e cada filho contribui com seus próprios automorfismos.
  - *Como fazer:* calcule ids; conte multiplicidades dos ids dos filhos; aut(v) = ∏ (mult_id!) · ∏ aut(filho); classes = número de ids distintos; se pedirem módulo, aplique. O(n log n).
  - *Exemplo:* uma estrela com 4 folhas tem 4! = 24 automorfismos; uma cadeia tem 2 (identidade e reflexão, se não enraizada).
  - *Armadilha:* o produto é sobre grupos de filhos isomorfos, não sobre pares; automorfismos de árvore não enraizada incluem a escolha do centro; use módulo quando a resposta for grande.
- **DP em árvore: dp[v][0/1]** → cobertura de vértices mínima, conjunto independente máximo (soma ponderada), dominating set mínimo (3 estados: escolhido, dominado por filho, ainda não dominado); em árvore também é guloso: folhas pelo pai.
  - *Gatilho:* "escolha um conjunto de vértices que cobre...", "máximo independente", "dominar todos".
  - *Por que funciona:* a decisão em v só interage com o pai e os filhos; estados locais resumem a condição pendente, e a árvore permite combiná-los em pós-ordem.
  - *Como fazer:* vertex cover: dp[v][0] = Σ dp[c][1]; dp[v][1] = 1 + Σ min(dp[c][0], dp[c][1]). Independente: dp[v][0] = Σ max(dp[c][0], dp[c][1]); dp[v][1] = w[v] + Σ dp[c][0]. Dominating: 3 estados (escolhido, coberto por filho, não coberto). Guloso para vertex cover: processe folhas e tome o pai.
  - *Exemplo:* cobertura mínima de vértices; conjunto independente máximo ponderado; dominating set mínimo.
  - *Armadilha:* dominating set não cabe em 2 estados (precisa saber se v já está coberto por um filho); o guloso folha-pai vale para vertex cover, não para independente/dominating; a raiz tem condições de contorno diferentes.
- **Matching máximo em árvore** → guloso de baixo para cima: se filho livre e v livre, case (v, filho); equivale a König (cobertura mínima de vértices = matching máximo em bipartido/árvore).
  - *Gatilho:* "máximo de pares disjuntos", "emparelhamento", "cobertura mínima".
  - *Por que funciona:* em pós-ordem, se um filho está livre e v está livre, casar é seguro: qualquer solução ótima pode ser trocada para incluir essa aresta sem perder cardinalidade. König garante que o matching máximo é dual da cobertura mínima em bipartido.
  - *Como fazer:* DFS pós-ordem; se v não está casado e tem filho não casado, case v com um deles e marque ambos; senão deixe v livre; conte os pares. O(n).
  - *Exemplo:* máximo de pares de vértices adjacentes disjuntos em uma árvore.
  - *Armadilha:* o guloso é para matching máximo (não ponderado); em matching ponderado use DP; König vale em bipartido (toda árvore é bipartida), mas não em grafo geral.
- **Mochila em árvore (tree knapsack).** dp[v][j] com soma de tamanhos limitando j: merge O(sz_a·sz_b) soma total O(n·K) se j ≤ K (truncando em K); para k exato de "k vértices conectados", dp[v][k] com ordem DFS (o "dp de ordem de DFS": pule a subárvore inteira em O(1) por tin).
  - *Gatilho:* "escolha k vértices conectados", "custo/peso por tamanho", "mochila na subárvore".
  - *Por que funciona:* combinar dois filhos é convolução dos vetores de tamanhos; o custo sz_a·sz_b somado sobre a árvore é $O(n^2)$ sem truncamento, mas com j ≤ K cada merge custa no máximo K^2 e o total é O(nK). A ordem de DFS permite pular uma subárvore inteira em O(1).
  - *Como fazer:* dp[v][0] = 0; para cada filho, combine dp[v][j] com dp[filho][t] limitando j + t ≤ K; resposta na raiz. Para "conectados contendo v", inicialize dp[v][1] com o valor de v. Alternativa por ordem de DFS: dp[i][j] escolhe i (vai para i+1) ou pula a subárvore (vai para i+sz). O(nK).
  - *Exemplo:* máximo peso de k vértices conectados; escolher k vértices numa subárvore com soma de valores.
  - *Armadilha:* O(nK) memória (pode ser necessário rolar os filhos); sem truncar em K vira $O(n^2)$; o merge precisa limitar pelo sz do filho, não só por K; "conectados" exige incluir o pai.
- **Tree knapsack com DFS order** → dp[i][j] sobre ordem de DFS: escolher o vértice i (vai para i + 1) ou pular sua subárvore (vai para i + sz): O(n·K) sem merge de vetores.
  - *Gatilho:* "k vértices conectados contendo a raiz", "escolha um conjunto fechado para cima".
  - *Por que funciona:* na ordem de DFS, a subárvore de i é o bloco [i, i + sz[i]); se não escolher i, nenhum descendente pode ser escolhido (conjunto fechado para cima), então basta pular o bloco inteiro em O(1).
  - *Como fazer:* dp[i][j] = melhor com j escolhidos processando de i até o fim; transição: escolher i → dp[i+1][j−1] + valor[i], ou pular → dp[i+sz[i]][j]; responda dp[1][K] ou dp[0][K]. O(nK) tempo, O(nK) ou O(K) memória com rolagem.
  - *Exemplo:* escolher k vértices conectados que contenham a raiz, maximizando a soma.
  - *Armadilha:* só vale para conjuntos fechados para cima (se escolhe um vértice, escolhe o pai); para conjuntos arbitrários use tree knapsack com merge; a ordem precisa ter a subárvore contígua.

- **Merge de dp côncavas em árvore** → dp[v][j] côncava em j (ex.: escolher k vértices com custo) → guarde a lista de incrementos e junte por heap/merge small-to-large; "corte k arestas / selecione k caminhos disjuntos" caem em Aliens ou slope trick.
  - *Gatilho:* "escolha k itens", "custo marginal decrescente", "k caminhos disjuntos", dp por quantidade com concavidade.
  - *Por que funciona:* se dp[v][j] é côncava, ela é determinada pelos incrementos dp[v][j] − dp[v][j−1], que são não crescentes; combinar filhos é juntar listas de incrementos, e o merge small-to-large/heap é eficiente.
  - *Como fazer:* guarde a lista de deltas de cada subárvore; no merge, junte as listas (heap ou small-to-large); para "exatamente k", pegue os k maiores incrementos. Alternativa: penalize cada item com λ e faça binary search em λ (Aliens), recuperando a contagem de itens.
  - *Exemplo:* escolher k caminhos disjuntos com custo mínimo; cortar k arestas.
  - *Armadilha:* a concavidade precisa ser provada (não vale para qualquer DP); Aliens exige custo inteiro e convexidade/concavidade; é preciso recuperar a solução com a contagem de itens usados, não só o valor penalizado.
- **Cobrir árvore com k caminhos máximo de vértices distintos** → guloso por cadeias de altura longa (long-path decomposition): ordene comprimentos das cadeias e some as k maiores.
  - *Gatilho:* "cubra o máximo de vértices com k caminhos", "caminhos disjuntos", "selecione k caminhos".
  - *Por que funciona:* a long-path decomposition particiona os vértices em cadeias descendentes disjuntas; pegar as k cadeias mais longas é ótimo por exchange: qualquer caminho que desça por uma cadeia pode ser trocado pelo sufixo mais longo sem piorar.
  - *Como fazer:* decomponha por altura (filho de maior altura); cada vértice pertence a uma cadeia; ordene os comprimentos e some os k maiores. O(n log n) (ou O(n) com counting/ordenação).
  - *Exemplo:* k = 2 numa árvore com cadeias de comprimentos 5, 4, 2 → cobertura 9.
  - *Armadilha:* a decomposição é em caminhos descendentes (não podem subir); se o problema exigir caminhos disjuntos em arestas, a resposta pode diferir; k maior que o número de cadeias → todos os vértices.
- **Escolher k folhas para maximizar união de caminhos até a raiz** → mesma long-path decomposition: as k cadeias mais longas (peso nas arestas), por exchange argument.
  - *Gatilho:* "escolha k folhas", "união dos caminhos folha–raiz", "máximo de vértices/arestas cobertos".
  - *Por que funciona:* cada folha adiciona à união a parte do seu caminho ainda não coberta; a decomposição em cadeias separa essas partes, e o ganho de escolher uma folha é o comprimento da sua cadeia; escolher as k maiores maximiza a soma.
  - *Como fazer:* long-path decomposition com pesos; ordene os comprimentos das cadeias; some os k maiores; as folhas correspondentes são as pontas das cadeias escolhidas. O(n log n).
  - *Exemplo:* k = 2 folhas que maximizam a união dos caminhos até a raiz.
  - *Armadilha:* caminhos compartilham prefixos, então o ganho não é a distância folha–raiz inteira; a soma das cadeias já desconta o compartilhado; se as folhas precisam ser distintas e k > número de folhas, resposta é a árvore inteira.
- **Dominating set de raio r em árvore** → guloso: processe de baixo para cima, o vértice mais profundo não coberto escolhe seu ancestral a r níveis acima; O(n) com dp (dist ao mais perto coberto/não coberto).
  - *Gatilho:* "todo vértice a distância ≤ r de um centro", "cubra a árvore com raio r", r fixo.
  - *Por que funciona:* o vértice não coberto mais profundo precisa de um centro a distância ≤ r; escolher o ancestral r níveis acima cobre o máximo possível (o centro mais alto que ainda o cobre), sem prejudicar os demais.
  - *Como fazer:* pós-ordem guardando d[v] = distância ao centro mais próximo na subárvore e u[v] = distância ao vértice não coberto mais profundo; se u[v] + d[v] ≤ r, o ramo já está coberto; se u[v] == r, coloque um centro em v (ou no ancestral r acima, conforme a implementação) e atualize. O(n).
  - *Exemplo:* mínimo de centros de raio r para dominar uma árvore.
  - *Armadilha:* r = 0 é caso especial; o guloso do "ancestral a r" precisa tratar a raiz (centro na raiz); a distância usada é em arestas, não em vértices.
- **Facility location (p-mediana/p-centro) em árvore** → p-centro: binária no raio + guloso de baixo para cima; p-mediana: DP $O(n^2p)$ ou O(np) com Aliens.
  - *Gatilho:* "instale p centros", "minimize a distância máxima", "minimize a soma das distâncias".
  - *Por que funciona:* no p-centro, "existe solução com raio ≤ R?" é monotônica em R e o guloso de dominância com raio R decide em O(n); no p-mediana, a árvore permite DP por subárvore e centro designado.
  - *Como fazer:* p-centro: binary search em R, rodando o guloso de raio R; p-mediana: dp[v][j][c] (subárvore, centros usados, centro que serve v) em $O(n^2p)$, ou penalidade por centro + binary search (Aliens), que troca o fator p por um log do custo. $O(n^2p)$ é a referência segura.
  - *Exemplo:* p hospitais minimizando a distância máxima (p-centro) ou a soma (p-mediana).
  - *Armadilha:* p-centro e p-mediana são objetivos diferentes; o guloso do p-centro depende de R e não serve para soma; a implementação do Aliens exige recuperar a contagem de centros, e a complexidade não é simplesmente O(np).

- **Cortar a árvore em componentes com condição (tamanho ≥ k, soma ≤ S)** → guloso pós-ordem: acumule sz, se ≥ k corte a aresta e zere; para soma máxima por componente + "minimizar # de cortes", binária no limite e guloso (combine os menores filhos primeiro).
  - *Gatilho:* "divida em componentes de tamanho pelo menos k", "cada componente com soma ≤ S", "minimize o número de cortes".
  - *Por que funciona:* para tamanho ≥ k, acumular na pós-ordem e cortar assim que atinge k é ótimo: cortar antes só produziria componentes menores, e cortar depois desperdiça vértices que poderiam formar outra componente.
  - *Como fazer:* DFS pós-ordem; acumule sz; se sz ≥ k, conte a componente, corte a aresta e devolva 0 ao pai; no fim, o resto pode formar uma componente extra se ≥ k. Para soma ≤ S, junte os filhos menores primeiro e corte quando a soma passar de S; para minimizar cortes, binary search no limite.
  - *Exemplo:* máximo de componentes de tamanho ≥ k; partir uma árvore em componentes com soma ≤ S.
  - *Armadilha:* "exatamente k" é diferente de "≥ k"; o resto na raiz pode ou não contar; na versão de soma, a ordem dos filhos (menores primeiro) é essencial para o guloso.
- **Dividir a árvore em componentes de tamanho exatamente k/igual** → só cortar arestas com sz múltiplo de k; isso é necessário, não suficiente (ex.: estrela K1,5, n=6, k=2). Se a partição existe (k | n), os cortes são exatamente os v ≠ raiz com sz_v ≡ 0 mod k, e há n/k − 1 cortes.
  - *Gatilho:* "divida em componentes de tamanho exatamente k", "k divide n".
  - *Por que funciona:* um corte separa uma subárvore de tamanho sz; se a partição existe, essa subárvore é união de componentes de tamanho k, logo sz é múltiplo de k. Reciprocamente, cortar em todos esses vértices produz componentes cujos tamanhos são diferenças de múltiplos de k; se há n/k − 1 cortes, há n/k componentes, e cada um só pode ter tamanho k.
  - *Como fazer:* calcule sz; conte os v ≠ raiz com sz_v % k == 0; se k | n e a contagem é n/k − 1, a partição existe; corte essas arestas. O(n).
  - *Exemplo:* estrela K1,5 com n = 6, k = 2: as folhas têm sz = 1 e o centro sz = 6; nenhum vértice não-raiz tem sz par, contagem 0 ≠ 2, então é impossível.
  - *Armadilha:* sz múltiplo de k é necessário mas não suficiente sozinho (a estrela mostra isso); a contagem precisa ser exatamente n/k − 1; a raiz nunca é cortada; k = 1 é trivial.
- **Remover aresta e comparar partes** → sz_v e n − sz_v; somas de xor/soma de subárvore equilibram: "remover duas arestas para xors iguais" → procure três partes com xor igual: se total ≠ 0, ou dois vértices incomparáveis com xor = total, ou um ancestral com xor 0 e um descendente com xor = total; se total = 0, dois vértices não-raiz com xor 0.
  - *Gatilho:* "corte duas arestas e compare as três partes", "xors iguais", "partes balanceadas".
  - *Por que funciona:* com duas arestas, as três partes têm xors A, B, C com A⊕B⊕C = total; se A = B = C = X, então total = X. Se total ≠ 0, X = total; se total = 0, X = 0. As posições relativas das arestas (ancestral/descendente ou incomparáveis) determinam quais subárvores combinar.
  - *Como fazer:* pré-calcule xor de subárvore; se total ≠ 0, procure (a) dois vértices incomparáveis com xor total, ou (b) um ancestral com xor 0 e um descendente com xor total; se total = 0, procure dois vértices não-raiz distintos com xor 0 (ancestral/descendente também funciona). O(n) com dfs e conjuntos.
  - *Exemplo:* três componentes com o mesmo xor; cortar duas arestas para equilibrar.
  - *Armadilha:* as arestas precisam ser distintas e gerar três partes não vazias; o caso ancestral/descendente tem ordem (xor 0 acima, total abaixo); não confunda xor de vértices com xor de arestas.
- **Ordenação topológica de árvore (ordem pai antes do filho) contada** → n!/∏ sz_v; "heap ordenado" idem; número de ordens de DFS = ∏ (filhos)! ; "permutações onde cada elemento aparece depois do pai" = n!/∏ sz.
  - *Gatilho:* "quantas ordens respeitam pai antes de filho", "ordens de DFS", "heap".
  - *Por que funciona:* o número de linear extensions de uma árvore enraizada é n!/∏ sz_v: escolha a raiz, depois intercale as ordens das subárvores; por indução, o denominador acumula os tamanhos. Ordens de DFS são mais restritas: cada subárvore é um bloco, então só a ordem dos filhos é livre, dando ∏(filhos)!.
  - *Como fazer:* calcule sz; resposta n!/∏sz (divisão exata); para DFS, ∏ (número de filhos)!; para "pai antes de filho" é o mesmo que linear extensions. O(n).
  - *Exemplo:* cadeia de 3 vértices: 3!/(3·2·1) = 1; estrela com 3 folhas: 4!/(4·1·1·1) = 6.
  - *Armadilha:* o denominador é o produto de sz de todos os vértices (inclusive folhas, que contribuem 1); ordens de DFS não permitem intercalar subárvores; a resposta pode precisar de módulo/primos.

- **Ordem de DFS livre (escolha da ordem dos filhos).** Dá intervalos de subárvores contíguos; para maximizar/minimizar algo (soma |pos_u − pos_v|, tempo de percurso) ordene filhos por uma chave com exchange argument (ex.: sz/custo).
  - *Gatilho:* "escolha a ordem dos filhos", "minimize a soma das distâncias na ordem", "tempo de percurso".
  - *Por que funciona:* a ordem dos filhos só troca blocos contíguos; trocar dois filhos adjacentes a, b altera o custo por um termo que depende de sz_a, sz_b e dos custos internos; o sinal da diferença dá um comparador.
  - *Como fazer:* escreva o custo de a antes de b e de b antes de a, cancele os termos comuns e ordene pela chave resultante (por exemplo, sz crescente ou custo/sz). O(Σ deg log deg).
  - *Exemplo:* minimizar Σ|pos_u − pos_v| sobre arestas; ordenar filhos por tamanho para reduzir o tempo de percurso.
  - *Armadilha:* o comparador precisa ser transitivo; a chave pode depender do custo acumulado (use exchange argument com cuidado); a ordem dos filhos não afeta a subárvore, só a posição.
- **Passeio de Euler e distância percorrida** → andar visitando todos e voltar = 2·Σ w; sem voltar = 2·Σ w − maior distância (do começo ao fim, diâmetro/distância do ponto de partida); ao visitar k vértices especiais, só vale a árvore virtual.
  - *Gatilho:* "visite todos os vértices/arestas", "volte ou não ao início", "menor passeio".
  - *Por que funciona:* cada aresta precisa ser atravessada pelo menos duas vezes (ida e volta) para visitar os dois lados; se o passeio não volta, as arestas do caminho entre início e fim são percorridas uma vez, economizando a distância entre eles. O máximo economizado é o diâmetro (extremos livres).
  - *Como fazer:* 2Σw se precisa voltar; 2Σw − diâmetro se pode terminar em qualquer lugar; se o início é fixo, 2Σw − max_v dist(início,v). Para k especiais, construa a árvore virtual e use 2·peso − diâmetro dela. O(n).
  - *Exemplo:* inspecionar todas as arestas de uma árvore e parar em qualquer lugar; visitar k cidades marcadas.
  - *Armadilha:* se o início e o fim são fixos, subtraia dist(início,fim), não o diâmetro; com pesos, use distâncias ponderadas; a árvore virtual só vale para os especiais (não para todos os vértices).
- **Árvore de pontes (2-edge-connected components)** → compressão por DSU nas arestas não-ponte; queries "adicione aresta (u, v): quantas pontes restam" = contrai caminho; "mínimo de arestas para 2-aresta-conexo" = ⌈L/2⌉.
  - *Gatilho:* "pontes", "componentes 2-aresta-conexos", "adicionar arestas para não ter pontes".
  - *Por que funciona:* contrair todas as arestas que não são pontes deixa uma árvore cujas arestas são exatamente as pontes; adicionar uma aresta (u,v) cria um ciclo e transforma em não-pontes todo o caminho entre as componentes de u e v.
  - *Como fazer:* Tarjan de pontes (low); DSU para unir as não-pontes; construa a árvore de pontes; para cada aresta nova, contraia o caminho (DSU de pulo). Para o mínimo de arestas, conte as folhas L da árvore de pontes: resposta ⌈L/2⌉. O((n+m) α).
  - *Exemplo:* quantas pontes restam após adicionar arestas; mínimo de arestas para tornar o grafo 2-aresta-conexo.
  - *Armadilha:* L é o número de folhas da árvore de pontes (grau 1), não de folhas da árvore original; grafo desconexo exige somar por componente (e conectar componentes primeiro); autoloops não são pontes.
- **DSU com peso (potencial) em árvore/grafo** → guarde diferença/xor/soma ao pai com compressão: relações "a_u − a_v = w" consistentes; contradição = ciclo; também resolve "pesos entre componentes" online em α(n).
  - *Gatilho:* restrições de diferença entre variáveis, "a_u − a_v = w", "xor de u a v = x", consistência.
  - *Por que funciona:* se cada nó guarda o potencial relativo ao pai, a compressão acumula os potenciais e find(u) devolve a raiz e o potencial de u; duas restrições que dão potenciais diferentes para o mesmo par são contraditórias.
  - *Como fazer:* find(x) retorna (raiz, pot[x] relativo à raiz); union(u,v,w) com a_u − a_v = w ajusta o potencial da raiz unida; para xor, troque soma por ⊕; para consistência, compare pot[u] ⊕ pot[v] com w. O(α(n)).
  - *Exemplo:* atribuir valores a vértices com restrições de diferença; detectar ciclo ímpar numa bipartição (xor de cores).
  - *Armadilha:* a diferença é directional (a_u − a_v, não |a_u − a_v|); a compressão precisa acumular o potencial na ordem certa; xor não é subtração.
- **DSU com merge de listas pequenas e queries "quem está no mesmo componente em tempo t"** → small-to-large guardando tempos; ou DSU sem compressão (por rank) + tempo de união: caminho até a raiz tem o máximo/minimax de tempos (árvore de Kruskal implícita).
  - *Gatilho:* consultas offline de conectividade com tempo, "quando u e v se conectaram".
  - *Por que funciona:* cada união tem um tempo; no grafo de uniões, o tempo em que dois vértices passam a estar conectados é o máximo dos tempos no caminho entre eles na árvore de união. Guardar listas de tempos por componente também responde consultas offline com binary search.
  - *Como fazer:* opção 1: DSU small-to-large com um vetor de (tempo, membro) e binary search; opção 2: DSU sem compressão, união por rank, guarde o tempo em cada aresta e responda max no caminho por binary lifting; opção 3: árvore de Kruskal. O(log n) por query.
  - *Exemplo:* "em que momento u e v ficaram conectados"; consultas de conectividade em um grafo que só recebe arestas.
  - *Armadilha:* compressão de caminho destrói a árvore de união (use rank sem compressão ou guarde a árvore separada); o tempo da união é o da aresta, não o da consulta; empates de tempo.

- **Árvore de tempo de união (DSU sem compressão, união por tamanho)** → altura O(log n); para cada aresta guarde o tempo; "u e v conectados no tempo t?" = max dos tempos no caminho na árvore DSU ≤ t.
  - *Gatilho:* conectividade offline com tempo, Kruskal, "a partir de quando".
  - *Por que funciona:* unir por tamanho sem compressão mantém altura O(log n); o tempo de conexão de u e v é o maior tempo de união no caminho entre eles, pois é a última aresta que faltava.
  - *Como fazer:* construa a árvore de união (cada união vira uma aresta pai–filho com o tempo); para query (u,v,t), suba os dois até a raiz calculando o máximo; ou use LCA com max. O(log n) por query.
  - *Exemplo:* "em que tempo u e v se conectam" em Kruskal; componentes em um grafo que cresce.
  - *Armadilha:* não use compressão de caminho na DSU que constrói a árvore; a árvore tem n−1 uniões; tempos iguais exigem desempate consistente.
- **Queries offline em árvore: "quantos vértices na subárvore de v com valor ≤ x"** → Euler tour + BIT offline ordenado por valor; para "k-ésimo" persistente/wavelet; com updates em vértices → BIT de BIT/CDQ.
  - *Gatilho:* queries de subárvore com filtro de valor, offline, sem updates.
  - *Por que funciona:* a subárvore vira um intervalo [tin, tout); ordenar as queries por x e inserir no BIT os vértices com valor ≤ x transforma a pergunta em uma soma de intervalo.
  - *Como fazer:* ordene vértices por valor e queries por x; insira tin[v] no BIT conforme avança x; resposta = sum(tout−1) − sum(tin−1). Para "k-ésimo", use merge sort tree/wavelet tree/persistente; com updates, BIT de BIT ou CDQ. O((n+q) log n).
  - *Exemplo:* contar vértices com valor ≤ x na subárvore; k-ésimo menor valor na subárvore.
  - *Armadilha:* queries online não cabem no offline ordenado (use wavelet/merge sort tree); empates de valor precisam ser inseridos antes de responder ≤ x; a subárvore inclui v.
- **Queries "v está na subárvore de u?"** → tin[u] ≤ tin[v] < tout[u] em O(1); "u é ancestral de v em árvore dinâmica" idem com ETT.
  - *Gatilho:* checar ancestralidade, filtrar vértices por subárvore, "está contido".
  - *Por que funciona:* a DFS entra em u antes de todos os seus descendentes e só sai depois de todos; logo o intervalo [tin[u], tout[u]) é exatamente a subárvore.
  - *Como fazer:* guarde tin/tout; a checagem é O(1). Em árvore dinâmica, use ETT para manter a ordem e comparar posições; para LCA, combine com binary lifting. O(1) estático.
  - *Exemplo:* verificar se v está na subárvore de u; ordenar vértices por tin para processar subárvores.
  - *Armadilha:* usar ≤ tout em vez de <; u = v é verdadeiro; em árvore dinâmica, a ETT precisa de link/cut para manter as posições.
- **Árvore enraizada e "reduzir o conjunto de vértices a ancestrais minimais/maximais"** → ordene por tin e use pilha; "remova vértices que têm ancestral no conjunto" → varra por tin mantendo o último intervalo ativo.
  - *Gatilho:* dado um conjunto de vértices, remover descendentes de outros do conjunto; achar os ancestrais minimais.
  - *Por que funciona:* se u é ancestral de v, tin[u] < tin[v] < tout[u]; ordenando por tin e mantendo o último intervalo ativo, todo vértice dentro dele é descendente e pode ser descartado.
  - *Como fazer:* ordene por tin; percorra mantendo (last_tout); se tin[v] < last_tout, v é descendente de um mantido, descarte; senão mantenha e atualize last_tout = tout[v]. Para ancestrais maximais, faça o simétrico (ordem reversa). O(k log k).
  - *Exemplo:* reduzir uma lista de vértices aos que não têm ancestral na lista; construir a árvore virtual.
  - *Armadilha:* ordenar só por tin não basta se você não mantém o tout do último mantido; um vértice pode estar dentro de um intervalo e não ser descendente? Em DFS, não; use < tout.
- **Conjunto máximo de caminhos disjuntos em vértices com LCAs** → ordene por profundidade do LCA decrescente; aceite se nenhum vértice está marcado (BIT em Euler tour: "soma na subárvore de u" ou "marca no caminho"); um caminho com LCA mais fundo "bloqueia" menos.
  - *Gatilho:* "escolha o máximo de caminhos vértice-disjuntos" de um conjunto dado.
  - *Por que funciona:* um caminho com LCA mais fundo está contido numa região menor; processá-lo primeiro é seguro por exchange: se uma solução ótima usa um caminho mais raso no lugar, a troca não reduz a cardinalidade.
  - *Como fazer:* ordene os caminhos pela profundidade do LCA (decrescente); para cada um, verifique se algum vértice do caminho já está marcado (BIT/HLD: soma no caminho ou consulta pontual); se livre, marque o caminho (range add) e conte. O((n+q) log n).
  - *Exemplo:* máximo de caminhos disjuntos em vértices numa árvore com q caminhos dados.
  - *Armadilha:* disjunto em vértices ≠ disjunto em arestas; o LCA conta como vértice do caminho; a marcação de caminho precisa de HLD ou diferença em árvore + verificação.

- **Cobertura de arestas da árvore por caminhos dados** (escolher o mínimo de caminhos para cobrir) → guloso: pegue a aresta mais funda não coberta e escolha o caminho cobrindo-a que sobe mais alto; DSU de "pulo" para cobrir.
  - *Gatilho:* "cubra todas as arestas com o mínimo de caminhos dados", "cada aresta em pelo menos um caminho".
  - *Por que funciona:* a aresta não coberta mais funda precisa ser coberta por algum caminho; entre os que a contêm, o que sobe mais alto cobre o maior número de arestas ancestrais, então é a escolha dominante.
  - *Como fazer:* pós-ordem para achar a aresta mais funda não coberta; escolha o caminho que a contém com o topo mais alto; marque as arestas cobertas com DSU de pulo (cada aresta é pintada uma vez). O((n+q) α).
  - *Exemplo:* cobrir uma árvore com o mínimo de caminhos de um conjunto.
  - *Armadilha:* o caminho precisa conter a aresta (não só cruzar); caminhos podem compartilhar vértices; o DSU de pulo precisa lidar com o LCA e com arestas já cobertas.
- **Contar árvores rotuladas/enraizadas** → Cayley: n^{n−2} sem raiz, n^{n−1} enraizada, florestas enraizadas (n + 1)^{n−1}; árvores binárias não rotuladas = Catalan; ordenadas com n nós = Catalan(n − 1).
  - *Gatilho:* "quantas árvores com n vértices rotulados", "florestas", "formas não rotuladas".
  - *Por que funciona:* Cayley conta árvores rotuladas via Prüfer; enraizar multiplica por n; a floresta enraizada usa a fórmula de Cayley generalizada. Formas não rotuladas são contadas por Catalan (árvores binárias/ordenadas).
  - *Como fazer:* n^{n−2} sem raiz; n^{n−1} com raiz; (n+1)^{n−1} para florestas enraizadas; Catalan(n) para árvores binárias com n nós; Catalan(n−1) para árvores planas/ordenadas com n nós (e para binárias cheias com n folhas). O(log n) por potência modular.
  - *Exemplo:* n = 4 rotuladas → 16; florestas enraizadas com 3 vértices → $4^2 = 16$.
  - *Armadilha:* "enraizada" e "com raiz fixa" são contagens diferentes; Catalan tem vários índices (n nós, n folhas, n arestas); a fórmula de Cayley vale para n ≥ 1 com convenções.
- **Número de subárvores conexas (subconjuntos conexos) de uma árvore** → dp[v] = ∏ (1 + dp[filho]) e soma sobre v; para contar por tamanho k, tree knapsack.
  - *Gatilho:* "quantos subconjuntos conexos", "quantas subárvores", "componentes conexos".
  - *Por que funciona:* todo subconjunto conexo tem um vértice de menor profundidade único v; uma vez incluído v, cada ramo de filho pode ser incluído de forma independente (0 ou uma subárvore conexa que contém o filho), dando o produto.
  - *Como fazer:* dp[v] = ∏(1 + dp[filho]) (a subárvore conexa que contém v e desce); some dp[v] sobre todos os v; para contar por tamanho, faça tree knapsack com a mesma recorrência. O(n).
  - *Exemplo:* uma cadeia de 3 vértices tem 6 subconjuntos conexos; uma estrela com 3 folhas tem 4 + 3 = 7? Conte: {centro}, {centro,folha1}, {centro,folha2}, {centro,folha3}, {centro,f1,f2}, {centro,f1,f3}, {centro,f2,f3}, {centro,f1,f2,f3} = 8, mais as 3 folhas sozinhas? Cada folha sozinha é conexa: total 3 + 8 = 11. A fórmula: folhas dp=1; centro dp = (1+1)^3 = 8; soma = 8 + 3 = 11.
  - *Armadilha:* "subárvore" aqui é subconjunto conexo, não subárvore enraizada; a soma sobre v conta cada conjunto uma vez pelo topo; use módulo e long long.
- **Probabilidade/esperança em árvore** → linearidade por vértice/aresta: E[tamanho do componente] = Σ P(u conectado a v); "sorteio de ordem aleatória" = ancestrais em ordem de remoção (v é "raiz" com prob. 1/(depth+1)): E[altura de treap/depth de nó] ≈ 2 ln n.
  - *Gatilho:* "esperança", "probabilidade de conexão", "ordem aleatória de remoção/inserção".
  - *Por que funciona:* a esperança é linear mesmo com dependência; a probabilidade de v ser o primeiro removido entre seus ancestrais é 1/(depth+1), o que dá a probabilidade de v ser raiz do seu componente.
  - *Como fazer:* some sobre arestas/vértices as probabilidades locais; para ordem aleatória, use 1/(depth+1); para a profundidade esperada em uma BST aleatória, E[depth] = 2(H_n − 1) ≈ 2 ln n. O(n).
  - *Exemplo:* tamanho esperado do componente de v após remoções aleatórias; profundidade esperada de um nó em treap.
  - *Armadilha:* os eventos não são independentes, mas a linearidade da esperança não exige independência; "≈ 2 ln n" é esperado, não garantido; a base do log é e.
- **Subárvore com tamanho ≥ n/2 única, "heavy"** → o centroide é o vértice de menor profundidade cujo filho maior tem sz ≤ n/2 — desça sempre pelo filho com sz > n/2.
  - *Gatilho:* "ache o centroide", "subárvore majoritária", "vértice balanceado".
  - *Por que funciona:* no máximo um filho pode ter sz > n/2; enquanto existir, o centroide está nessa subárvore; ao descer, o maior componente diminui, e o primeiro vértice sem filho > n/2 é o centroide.
  - *Como fazer:* calcule sz; a partir da raiz, enquanto houver filho com sz > n/2, desça para ele; o vértice atual é centroide. O(n).
  - *Exemplo:* caminho de 4 vértices: desça do 1 para o 2 (sz 3 > 2), depois para o 3 (sz 2 não > 2) → centroide 2; o 3 também é centroide (dois).
  - *Armadilha:* a condição é > n/2 (não ≥); se o vértice atual é a raiz, não há componente acima; o centroide pode não ser único.

- **Aresta de custo c e "pintar caminhos"** → coloração de caminho em árvore offline: "último a pintar" → processe em ordem inversa com DSU pulando arestas já pintadas (O(n α)).
  - *Gatilho:* "pinte o caminho", "qual a cor final", "o último update vence".
  - *Por que funciona:* processando de trás para frente, a primeira vez que uma aresta é encontrada é o último update que a afetou; o DSU "próximo não pintado" pula as arestas já decididas, então cada aresta é visitada uma vez.
  - *Como fazer:* inverta a ordem dos updates; para cada caminho, pinte as arestas ainda não pintadas com DSU de pulo (find retorna o próximo ancestral não pintado); pare no LCA. O((n+q) α).
  - *Exemplo:* cor final de cada aresta após uma sequência de pinturas.
  - *Armadilha:* updates em vértices vs arestas; o DSU precisa pular para o pai; caminhos podem repetir; a ordem inversa é essencial.
- **Atualizar caminho + ler ponto, sem HLD** → cada update (u, v, x): diff[u] += x, diff[v] += x, diff[lca] −= x, diff[pai(lca)] −= x; final: soma de subárvore = valor do vértice. Para arestas, use só diff[lca] −= 2x.
  - *Gatilho:* "some x no caminho" e "qual o valor de um vértice", updates e queries intercalados, sem HLD.
  - *Por que funciona:* a soma de subárvore de diff em v conta quantos caminhos têm exatamente uma ponta na subárvore de v; os ajustes no LCA e no pai do LCA cancelam os caminhos que apenas passam por cima, deixando o valor correto em cada vértice.
  - *Como fazer:* para cada update de vértices, faça quatro point updates em diff; para consultar o valor de v, use Euler tour + BIT e calcule a soma de subárvore (intervalo [tin, tout)). Para arestas, guarde o valor no filho e use diff[u] += x, diff[v] += x, diff[lca] −= 2x. O(log n) por operação.
  - *Exemplo:* adicionar 1 em todos os vértices do caminho u–v e perguntar quantos caminhos cobrem v.
  - *Armadilha:* o termo pai(lca) é obrigatório para vértices; para arestas, não use pai(lca) e subtraia 2x no LCA; a soma é sempre de subárvore, não pontual no array de diff.
- **Quantos caminhos cobrem cada aresta/vértice (offline)** → mesma diferença em árvore com soma de subárvore; "arestas cobertas por exatamente/pelo menos k caminhos" = após somar.
  - *Gatilho:* "quantos caminhos passam por cada aresta", "arestas com cobertura exatamente k".
  - *Por que funciona:* após aplicar todas as diferenças, a soma de subárvore em v é o número de caminhos que usam a aresta pai–v (ou que contêm v, na versão de vértices); a contagem é local.
  - *Como fazer:* acumule as diferenças de todos os caminhos; faça uma DFS somando de baixo para cima; para arestas, o valor em v é a cobertura de (pai[v], v); para vértices, use a versão com pai(lca). Depois, conte/compare com k. O(n + q).
  - *Exemplo:* achar arestas cobertas por todos os q caminhos; arestas com cobertura 0 (não usadas).
  - *Armadilha:* para vértices e arestas as fórmulas diferem; caminhos com u = v cobrem o vértice, não arestas; a raiz não tem aresta pai.
- **Mediana de árvore (vértice que minimiza Σ dist)** → centroide; **centro** minimiza max dist (meio do diâmetro). Se pesos nos vértices, centroide ponderado.
  - *Gatilho:* "escolha um vértice que minimize a soma/máximo das distâncias", "depósito", "antena".
  - *Por que funciona:* mover de v para um vizinho do componente de tamanho t muda Σ dist em n − 2t; no centroide todos os t ≤ n/2, então nenhum movimento melhora. O centro do diâmetro equilibra os dois lados mais distantes e minimiza o máximo.
  - *Como fazer:* mediana: ache o centroide (desça pelo filho com sz > n/2); centro: ache o diâmetro e suba metade; com pesos nos vértices, use a condição de peso ≤ W/2. O(n).
  - *Exemplo:* um único depósito que minimiza a soma das distâncias (centroide) ou a pior entrega (centro).
  - *Armadilha:* mediana (soma) ≠ centro (máximo); com pesos nos vértices, o centroide muda; o centro pode ser o interior de uma aresta.
- **Árvore com "todo vértice dentro de dist d de um escolhido"** → p-centro com raio dado = guloso de baixo para cima; com k escolhidos e minimizar o raio → binária no raio.
  - *Gatilho:* "cubra todos os vértices com raio d", "mínimo de centros", "k centros minimizando o raio".
  - *Por que funciona:* com raio fixo, o vértice não coberto mais profundo força um centro no seu ancestral a d; essa escolha é a que cobre mais, e o guloso é ótimo. A factibilidade é monotônica no raio, permitindo binary search.
  - *Como fazer:* para raio d, pós-ordem guardando a distância ao centro mais próximo e ao não coberto mais fundo; quando o não coberto atinge d, coloque um centro no ancestral d acima (ou no vértice atual, conforme a implementação). Para k centros, binary search no raio. O(n log D).
  - *Exemplo:* cobrir uma árvore com k centros de raio mínimo.
  - *Armadilha:* o guloso precisa dos dois valores por vértice; d = 0 é caso especial; o centro pode ser colocado na raiz quando o não coberto está a menos de d dela.

- **Soma de distâncias de todos a todos por vértice (Σ_u d(v, u))** → rerooting: res[child] = res[parent] + n − 2·sz[child] (arestas de peso 1); com pesos: res[child] = res[parent] + w·(n − 2sz).
  - *Gatilho:* "para cada vértice, a soma das distâncias a todos os outros", "grau de centralidade".
  - *Por que funciona:* ao mudar a raiz de p para c pela aresta de peso w, os sz[c] vértices da subárvore de c ficam w mais perto e os n − sz[c] restantes ficam w mais longe; a variação é w·(n − 2sz[c]).
  - *Como fazer:* calcule res[raiz] = Σ d(raiz,u) na primeira DFS; na segunda, aplique a fórmula para cada filho; O(n).
  - *Exemplo:* soma das distâncias de cada vértice em uma árvore com pesos.
  - *Armadilha:* use long long; com pesos, multiplique a variação por w; a fórmula é para arestas, não para valores em vértices.
- **Árvore com valores e "ganhos de remoção" (somas máximas de componentes conectados)** → dp[v] = a_v + Σ max(0, dp[filho]); resposta max_v dp[v]; rerooting para "máximo componente contendo v".
  - *Gatilho:* "máxima soma de um componente conectado", "remova arestas e ganhe", "subárvore de soma máxima".
  - *Por que funciona:* para o componente que contém v e desce, incluir o ramo de um filho só vale se dp[filho] > 0; os ramos são independentes, então a soma das partes positivas é ótima.
  - *Como fazer:* pós-ordem com dp[v] = a[v] + Σ max(0, dp[filho]); resposta = max dp[v]; para "contendo v" faça rerooting combinando a contribuição de fora. O(n).
  - *Exemplo:* subárvore de soma máxima (não necessariamente enraizada); maior ganho ao escolher um componente conexo.
  - *Armadilha:* se todos os valores forem negativos, a resposta é o maior a[v] (componente de um vértice), não 0; o componente precisa ser conectado; no rerooting, use max(0, contribuição) dos irmãos.
- **Árvore ordenada ↔ binária (filho esquerdo, irmão direito)** → bijeção; reduz queries de ordem entre irmãos e "DP sobre lista de filhos" a DP binária; também "árvores com n nós" ↔ Catalan.
  - *Gatilho:* árvore com muitos filhos, ordem dos filhos importa, DP que percorre a lista de filhos.
  - *Por que funciona:* cada nó aponta para o primeiro filho (esquerda) e para o próximo irmão (direita); isso preserva a ordem e transforma grau arbitrário em grau ≤ 2, sem perder informação.
  - *Como fazer:* represente left[v] = primeiro filho, right[v] = próximo irmão; a DFS binária percorre a mesma ordem; use para DP de listas de filhos e para contar formas (Catalan). O(n).
  - *Exemplo:* converter uma árvore ordenada em binária; percorrer irmãos como uma lista ligada.
  - *Armadilha:* a raiz não tem irmão; a ordem dos filhos é preservada (não é uma BST); "árvores binárias" aqui são ordenadas, não de busca.
- **Árvore de segmentos implícita sobre os filhos** → vértices com grau enorme e queries por intervalo de filhos: crie "árvore de prefixos" dos filhos (cadeia auxiliar) para reduzir arestas de $O(deg^2)$ para O(deg) em grafos derivados.
  - *Gatilho:* grau muito alto, "cada filho se conecta a um intervalo de irmãos", construir grafo auxiliar.
  - *Por que funciona:* uma cadeia de nós auxiliares representando prefixos/sufixos dos filhos permite que uma conexão com um intervalo seja feita em O(1) arestas por extremo, em vez de O(deg) por filho.
  - *Como fazer:* crie nós auxiliares para prefixos dos filhos; conecte o filho i ao nó de prefixo adequado; use uma segtree implícita para queries de intervalo. O(deg) arestas auxiliares por vértice.
  - *Exemplo:* derivar um grafo onde cada filho se conecta a todos os anteriores ($O(deg^2)$ → O(deg)).
  - *Armadilha:* os nós auxiliares não são vértices reais (não contam na resposta); memória O(deg) extra; os índices precisam ser mapeados.
- **Árvore geradora como estrutura "ancestral–descendente" (DFS tree).** Em grafo não direcionado, toda aresta fora da árvore DFS liga ancestral–descendente (sem cross edges): usado para pontes, ciclos (cada aresta de retorno fecha um ciclo fundamental), caminhos longos (≥ k se grau ≥ k) e "cobrir com √n caminhos/ciclos".
  - *Gatilho:* grafo não direcionado, pontes, ciclos, cobertura por caminhos, "não há arestas cruzadas".
  - *Por que funciona:* na DFS não direcionada, quando se examina uma aresta para um vértice já visitado, esse vértice está na pilha da DFS (ancestral) ou já terminou; se tivesse terminado, a aresta teria sido explorada antes e ele não seria uma nova raiz — logo não há cross edges.
  - *Como fazer:* DFS com tin e low; arestas de árvore, de retorno; low[v] = min(tin[v], low[filho], tin[ancestral]); use para pontes, componentes 2-aresta-conexos e ciclos fundamentais. Grafo com grau mínimo ≥ k tem caminho com ≥ k+1 vértices. O(n+m).
  - *Exemplo:* achar pontes; cada aresta de retorno define um ciclo fundamental; construir a árvore de pontes.
  - *Armadilha:* em grafo direcionado há cross edges e a propriedade falha; arestas múltiplas exigem identificar a aresta de retorno pelo id; DFS iterativa precisa simular a pilha.

- **DFS tree e coloração/independent set grande** → ou há caminho com ≥ √n vértices (profundidade da DFS), ou alguma camada grande (independência por níveis): "ache caminho de tamanho ≥ √n ou conjunto independente de tamanho ≥ √n" (par de níveis).
  - *Gatilho:* "caminho longo ou independente grande", cotas de √n, problemas de separação.
  - *Por que funciona:* se a profundidade da DFS é ≥ √n, o caminho raiz–folha tem ≥ √n vértices; caso contrário, há menos de √n níveis, e por pigeonhole alguma camada tem ≥ √n vértices. Em uma árvore, vértices da mesma profundidade são independentes.
  - *Como fazer:* rode DFS/BFS; se a profundidade ≥ √n, reconstrua o caminho pai; senão, pegue a camada com mais vértices (BFS) e devolva como independente. O(n).
  - *Exemplo:* achar caminho de √n vértices ou independente de √n em uma árvore.
  - *Armadilha:* a camada é independente em árvore; em grafo geral, o subgrafo induzido de uma camada BFS pode ter arestas (mas arestas dentro da camada criam ciclos ímpares, o que também ajuda); a cota é assintótica, arredonde.
- **BFS tree e dual:** em BFS cada aresta liga níveis consecutivos ou iguais; ciclo ímpar ⇔ aresta entre vértices do mesmo nível; ciclo mais curto através de v = min sobre arestas (u, w) com lca(u, w) = v de d(u) + d(w) + 1 − 2·d(v) (d(u) + d(w) + 1 sozinho é passeio fechado).
  - *Gatilho:* ciclo ímpar, ciclo mais curto, bipartição, "menor ciclo passando por v".
  - *Por que funciona:* em BFS, uma aresta não pode ligar níveis que diferem de mais de 1; se liga o mesmo nível, o caminho na árvore mais a aresta forma um ciclo de comprimento 2d+1, ímpar. Para o ciclo mais curto por v, remova v: todo ciclo simples por v é formado por dois vizinhos a, b de v mais um caminho a–b em G−v, então o comprimento é dist_{G−v}(a,b) + 2.
  - *Como fazer:* BFS a partir da raiz para níveis; para ciclo ímpar, procure aresta (u,w) com d[u] = d[w]. Para o ciclo mais curto por v, remova v e calcule a menor distância entre dois vizinhos de v em G−v (BFS de cada vizinho, ou BFS multi-fonte marcando a origem), somando 2; alternativamente, enraíze a BFS em v e minimize d(u) + d(w) + 1 sobre arestas com lca(u,w) = v (os dois caminhos da árvore divergem em v). O(deg(v)·(n+m)) na primeira versão.
  - *Exemplo:* detectar se um grafo é bipartido; achar o menor ciclo que passa por um vértice.
  - *Armadilha:* d(u)+d(w)+1 sozinho pode ser um passeio que repete arestas se u e w não estiverem no mesmo nível ou se o caminho não for simples; a fórmula com lca(u,w)=v só é exata quando a árvore está enraizada em v (ou quando os dois caminhos realmente divergem em v), e não cobre ciclos por v com LCA acima de v; para o ciclo mais curto por v, remova v e combine dois vizinhos (ou enraíze a BFS em v).
- **Árvore e 2-coloração por profundidade** → qualquer árvore é bipartida; grafo = árvore + arestas extras: cada aresta extra decide a paridade; caminhos de comprimento par/ímpar entre u e v dependem de uma aresta extra de ciclo ímpar.
  - *Gatilho:* "grafo é uma árvore mais poucas arestas", "caminho par/ímpar", "bipartido".
  - *Por que funciona:* a árvore tem uma 2-coloração única a menos de troca global; cada aresta extra conecta duas cores e, se conectar a mesma, cria um ciclo ímpar. Com um ciclo ímpar alcançável, é possível alternar a paridade do caminho.
  - *Como fazer:* pinte a árvore por profundidade; para cada aresta extra, verifique a paridade das cores; para caminhos, use a paridade base da árvore e veja se um ciclo ímpar no caminho permite trocar. O(n+m).
  - *Exemplo:* grafo com uma aresta extra; verificar se é bipartido; contar caminhos de comprimento par.
  - *Armadilha:* a paridade do caminho não é só (d[u]+d[v]) mod 2 quando há arestas extras; um ciclo ímpar permite ambas as paridades; arestas extras podem ser várias.
- **Árvores de menor caminho (SPT) e restrições adicionais** → "remova uma aresta e recalcule distâncias": usar a árvore de caminhos mínimos de s, e para a aresta (u, v) da SPT usar candidatos de arestas fora da árvore (substituição, "replacement path"); método clássico: Dijkstra com "lowest cross edge" ou LCT + sweep por distância.
  - *Gatilho:* "remova uma aresta e recalcule as distâncias a partir de s", "replacement paths", pesos não negativos.
  - *Por que funciona:* a SPT contém um caminho mínimo de s a cada vértice; remover uma aresta da SPT afeta apenas as distâncias na subárvore abaixo dela; o novo caminho mínimo usa uma aresta que cruza o corte, e o melhor candidato é a de menor d[u] + w + d[v].
  - *Como fazer:* construa a SPT de s; para cada aresta da SPT, os candidatos são arestas não-árvore que cruzam o corte da subárvore; processe por varredura (sweep) por distância com LCT, ou use "lowest cross edge" com Dijkstra modificado. O(m log n).
  - *Exemplo:* distâncias após remover uma aresta da SPT; "replacement path" clássico.
  - *Armadilha:* se a aresta removida não está na SPT, as distâncias não mudam; pesos negativos quebram Dijkstra; a aresta de substituição precisa cruzar o corte, não apenas tocar a subárvore.

- **Replacement path em árvore geradora mínima (substituir aresta removida)** → varra arestas fora da árvore em ordem crescente de peso e pinte o caminho ainda não pintado com DSU "pular caminhos pintados": cada aresta da MST recebe a menor aresta fora que a cobre.
  - *Gatilho:* "remova uma aresta da MST", "qual o menor peso para reconectar", "sensibilidade da MST".
  - *Por que funciona:* a menor aresta fora da árvore que cruza o corte de uma aresta da MST é a substituta ótima; varrer em ordem crescente garante que a primeira aresta que cobre uma aresta da MST é a menor possível.
  - *Como fazer:* construa a MST; ordene as arestas não-árvore por peso; para cada uma, pinte com DSU de pulo as arestas ainda não pintadas do caminho entre suas pontas; a aresta pintada recebe o peso atual. O(m α).
  - *Exemplo:* MST após remover uma aresta; aresta de substituição para cada aresta da MST.
  - *Armadilha:* se a aresta removida não está na MST, nada muda; arestas de peso igual exigem desempate consistente; o DSU de pulo precisa de LCA (ou HLD) para percorrer o caminho.
- **Árvore binária cheia/heap implícito** → filhos 2i, 2i + 1; LCA(u, v) = desloque o maior até igualar; profundidade = $\lfloor \log_2 i \rfloor$; subárvore em nível d é intervalo [i·2^k, (i + 1)·2^k).
  - *Gatilho:* árvore binária completa armazenada em array (heap), índices 2i, "sem ponteiros".
  - *Por que funciona:* o pai de i é $\lfloor i/2 \rfloor$; subir um nível é dividir por 2; o LCA é o primeiro ancestral comum, obtido igualando as profundidades e subindo os dois.
  - *Como fazer:* LCA: enquanto a ≠ b, se a > b, a = $\lfloor a/2 \rfloor$; senão b = $\lfloor b/2 \rfloor$; O(log n). Profundidade = $\lfloor \log_2 i \rfloor$; a subárvore de i no nível relativo k é [i·2^k, (i+1)·2^k) (half-open). O(log n).
  - *Exemplo:* LCA(5,7): 5→2→1, 7→3→1, LCA = 1; subárvore de 2 no nível 1 é [4,6) = {4,5}.
  - *Armadilha:* índices 1-based; overflow ao multiplicar i·2^k; a subárvore é por nível, não por DFS; a árvore precisa ser completa (ou o intervalo precisa respeitar n).
- **Trie binária como árvore de altura 30** → mesmas técnicas: DP em subárvore, small-to-large de tries (merge de tries é O(total de nós)), xor máximo = descer pelo bit oposto, e "mínimo xor MST" por D&C nos bits.
  - *Gatilho:* "xor máximo", "pares com xor mínimo", "trie de bits", valores até 2^{30}.
  - *Por que funciona:* a trie é uma árvore binária de altura 30 (bits do mais significativo ao menos); a subárvore de um nó é o conjunto de números com aquele prefixo; o merge de duas tries visita só nós sobrepostos, custando O(nós) total.
  - *Como fazer:* insira os números; para xor máximo com x, desça pelo bit oposto quando existir; para o mínimo xor entre dois conjuntos, desça pelo mesmo bit e, se não existir, pelo oposto; para a MST de xor, faça D&C por bit (Boruvka: o menor xor entre subárvores irmãs). O(n·30).
  - *Exemplo:* máximo xor de um par; MST com pesos xor.
  - *Armadilha:* processe bits altos primeiro; use 64 bits se os valores passarem de 2^{30}; o merge de tries é O(total de nós) mas a memória pode estourar; valores repetidos.
- **Árvore de sufixos / links** → árvore de links do SAM e de falhas do Aho-Corasick: operações de "subárvore = todas as strings que têm este sufixo/padrão" são Euler tour; propagar contagens por DP de baixo para cima.
  - *Gatilho:* "quantas ocorrências de cada substring", "todos os padrões que são sufixos", "links de falha".
  - *Por que funciona:* no SAM, o link de sufixo aponta para o estado do maior sufixo próprio; a árvore de links tem como subárvore de um estado todos os estados que representam strings que terminam com aquele sufixo. No Aho-Corasick, a árvore de falhas dá o análogo para padrões.
  - *Como fazer:* construa o SAM (ou o autômato Aho-Corasick); monte a árvore de links; faça Euler tour para subárvore; propague as contagens de ocorrências de baixo para cima (cada estado soma aos filhos e ao link). O(n·σ) para construir, O(n) para propagar.
  - *Exemplo:* contar ocorrências de cada substring; achar o padrão mais frequente.
  - *Armadilha:* a raiz representa a string vazia; as arestas da árvore de links não são caracteres; o SAM tem até 2n−1 estados; a contagem de um estado precisa somar as ocorrências dos estados que o têm como sufixo.
- **Binary lifting guardando "primeiro ancestral com propriedade"** → monotonia por caminho (propriedade só piora/melhora subindo) → busca binária por lifting em O(log); cabe em "número de saltos para chegar à raiz" (greedy pulos).
  - *Gatilho:* "primeiro ancestral com valor ≥ x", "subir até a raiz com orçamento", propriedade monótona ao longo do caminho.
  - *Por que funciona:* se a propriedade é monótona (uma vez verdadeira, continua verdadeira ao subir, ou vice-versa), o conjunto de ancestrais que a satisfazem é um sufixo do caminho; o lifting binário encontra a fronteira em O(log) saltos.
  - *Como fazer:* para k de $\lfloor \log n \rfloor$ a 0, se up[k][v] existe e satisfaz a propriedade, v = up[k][v]; ao final, v é o ancestral mais alto que satisfaz (ou o pai é o primeiro que não satisfaz). O(log n).
  - *Exemplo:* primeiro ancestral com profundidade ≤ d; primeiro ancestral com soma acumulada ≤ L; primeiro com valor mínimo ≤ x.
  - *Armadilha:* a propriedade precisa ser monótona; se não for, o lifting não vale; cuidado com o vértice atual (pode já satisfazer); trate a raiz e o "não existe".

- **Pulos gulosos em árvore** (de v sobe até o ancestral mais alto com custo ≤ L, repete) → defina próximo[v] = esse ancestral e conte saltos com binary lifting sobre próximo (grafo funcional/árvore de saltos); mesma ideia de "sparse table de saltos" em array.
  - *Gatilho:* "suba o máximo possível com orçamento L por salto", "número de saltos até a raiz", "saltos gulosos".
  - *Por que funciona:* cada salto é o mais alto possível dentro do orçamento; a escolha gulosa é ótima porque qualquer salto menor não alcança mais longe e só aumenta o número de saltos. A função próximo[v] define uma árvore funcional, e contar saltos é binary lifting sobre ela.
  - *Como fazer:* calcule próximo[v] com binary lifting e prefixos de custo (ou uma DFS com dois ponteiros); construa up_next[k][v]; para contar saltos até a raiz, desça os bits de ⌊log n⌋ a 0. O(n log n) de pré-processamento e O(log n) por consulta.
  - *Exemplo:* subir uma árvore com custo máximo L por salto; número de saltos para chegar à raiz.
  - *Armadilha:* se próximo[v] = v (nem o pai cabe no orçamento), a função tem laço; use um sentinela para a raiz; o custo pode ser acumulado (use prefixos) e não apenas o peso da aresta; a altura da árvore de saltos pode ser O(n).

Repetidas de outras seções:

- **Jogo em árvore** → paridade da profundidade, centro, folhas; Green Hackenbush (teoria dos jogos).
- **Trie é uma árvore** → LCP de duas strings = profundidade do LCA na trie (strings).
- **Árvore de sufixos / links do SAM** formam árvore → DP de subárvore conta ocorrências (strings).
- **Prüfer / Cayley** → graus e nós rotulados: $(n-2)!/\prod(d_i-1)!$; ligar k componentes: $n^{k-2}\prod s_i$.
- **Rotulações pai < filho** → $n!/\prod sz(v)$.
- **Hitting time numa árvore (passeio aleatório)** → $E[u \to v \text{ adjacente}] = 2\cdot|E(\text{lado de } u)| + 1$.
- **Isomorfismo de árvores** → AHU (hash do multiconjunto ordenado dos filhos); sem raiz: centroide(s).
- **Hash de árvore enraizada** → $H(v) = 1 + \Sigma f(H(\text{filho}))$ com f pseudoaleatório (ou AHU com mapa de multiconjuntos); sem raiz, use os 1–2 centroides.
- **Árvore ↔ parênteses (Dyck)** → subárvore = intervalo balanceado; LCA = mínimo do balanço; contar árvores = Catalan.
- **Xor de caminho** → $px[u] \oplus px[v]$ (arestas) ou $\oplus a[lca]$ (vértices).

Estruturas de dados e queries

Perguntas: dá para responder offline? Dá para ordenar as queries? A operação é associativa?

- **Offline + ordenar** queries por r (ou por valor) e varrer com BIT/segtree: resolve "distintos em [l, r]", "elementos ≤ x em [l, r]".
  - *Gatilho:* todas as queries conhecidas antes de responder, sem updates, e a resposta é uma contagem/estatística de um intervalo.
  - *Por que funciona:* com r fixo, para "distintos" só a última ocorrência de cada valor importa; para "≤ x", inserir os elementos em ordem de valor transforma a condição em "já está na estrutura".
  - *Como fazer:* distintos: sweep em r, BIT com 1 na última ocorrência, remova a anterior (prev) e some em [l, r]. "≤ x": ordene as queries por x, insira as posições dos $a_i \le x$ e conte em [l, r]. O((n + q) log n).
  - *Exemplo:* SPOJ DQUERY (distintos em [l, r]); SPOJ KQUERY (contar > k, ordenando por k).
  - *Armadilha:* são dois ordenamentos diferentes (por r e por x); não misture. Com updates, o sweep por r deixa de valer (persistente ou BIT de BITs).
- **Mo** quando add/remove é O(1) e não há updates; com updates → Mo com tempo $(n^{5/3})$.
  - *Gatilho:* muitas queries de intervalo sobre array estático e dá para manter o estado adicionando/removendo uma ponta.
  - *Por que funciona:* reordenar as queries não muda as respostas; com a ordem certa, o total de movimentos de L e R cai de O(nq) para O((n + q)√n).
  - *Como fazer:* bloco $B \approx n/\sqrt{q}$ ($\sqrt{n}$ se $q \approx n$); ordene por $(\lfloor l/B \rfloor, r)$ alternando o sentido de r a cada bloco; mova L e R ajustando o estado. Com updates: terceira coordenada t e $B = n^{2/3}$.
  - *Exemplo:* contar valores distintos em [l, r] com vetor de frequências.
  - *Armadilha:* remover tem de desfazer exatamente o add; sem inverso (max, gcd), use Mo com rollback. Ajuste B ao número de queries, não a n.
- **Segtree:** pense em qual informação do nó permite juntar dois filhos (max prefixo, max sufixo, soma, resposta). Esse é o problema inteiro.
  - *Gatilho:* updates pontuais e queries de intervalo cuja resposta pode ser montada de pedaços.
  - *Por que funciona:* a junção precisa ser associativa; a árvore guarda o resumo de cada bloco e a query junta O(log n) resumos na ordem.
  - *Como fazer:* defina o nó e merge(a, b); build O(n); update e query O(log n). Memória 2n na iterativa, 4n na recursiva.
  - *Exemplo:* nó (soma, melhor prefixo, melhor sufixo, melhor subarray).
  - *Armadilha:* se o resumo não basta para juntar (ex.: moda), a segtree simples não resolve; a ordem do merge importa.

- **Lazy** para update em intervalo; **segtree beats** para chmin/chmax; **persistente** para versões ou "k-ésimo em [l, r]".
  - *Gatilho:* (i) update e query em intervalo; (ii) o update é chmin/chmax; (iii) precisa de estados passados ou de k-ésimo num intervalo.
  - *Por que funciona:* lazy adia o update nos filhos até precisar; beats só desce onde o chmin realmente altera o máximo; persistente copia apenas O(log n) nós por versão.
  - *Como fazer:* lazy: o nó guarda o resumo já com a tag e o push aplica aos filhos. Beats: (max, 2º max, cnt do max, soma). Persistente: cada update cria novos nós no caminho; versões por prefixo.
  - *Exemplo:* range add + range sum; chmin + soma; k-ésimo em [l, r].
  - *Armadilha:* a ordem de composição das tags importa; beats com add precisa também de min/2º min; persistente com lazy usa marcação permanente (sem push-down).
- **a_i ← a_i mod x, a_i ← √a_i, a_i ← gcd** → cada elemento muda poucas vezes; pule os que não mudam. A poda por max no nó vale para mod/√; para gcd guarde o lcm do nó e pule quando lcm | x (senão O(nq)).
  - *Gatilho:* updates que encolhem cada valor (mod, raiz, gcd) e queries de soma/max.
  - *Por que funciona:* quando mod muda o valor, o resultado é < a_i/2; √ reduz o expoente à metade; um gcd próprio é ≤ a_i/2. Cada posição muda O(log A) vezes (√ muda O(log log A)).
  - *Como fazer:* segtree com max no nó para mod/√ e lcm para gcd; se o nó não pode mudar (max < x, max ≤ 1, lcm | x), pare; senão desça até as folhas e atualize. Total O((n + q) log A log n) amortizado.
  - *Exemplo:* CF 438D "The Child and Sequence" (mod + soma).
  - *Armadilha:* sem a poda o pior caso é O(nq); para gcd, podar por max não serve, pois o gcd pode cair sem o max mudar.
- **Sparse table** para RMQ estático em O(1).
  - *Gatilho:* array estático e operação idempotente (min, max, gcd, and, or).
  - *Por que funciona:* dois blocos de 2^k cobrem o intervalo e podem se sobrepor sem alterar o resultado idempotente.
  - *Como fazer:* st[k][i] resume 2^k elementos; k = ⌊log(r − l + 1)⌋; resposta = op(st[k][l], st[k][r − 2^k + 1]). Build O(n log n).
  - *Exemplo:* RMQ de mínimo; gcd de intervalo.
  - *Armadilha:* não vale para soma (a sobreposição conta duas vezes); nesse caso use disjoint sparse table.
- **Set de intervalos** (Chtholly / ODT) para "atribua valor em [l, r]".
  - *Gatilho:* updates de atribuição em intervalo e queries simples (soma, contagem, k-ésimo) sobre pedaços constantes.
  - *Por que funciona:* cada assign funde os pedaços tocados; se as atribuições são aleatórias, o número de intervalos se mantém pequeno.
  - *Como fazer:* map de início → (fim, valor); split(pos) corta o intervalo; apague o miolo de [l, r] e insira um intervalo novo. Esperado O((n + q) log n).
  - *Exemplo:* "atribua x em [l, r]" seguido de "some [l, r]".
  - *Armadilha:* dados adversariais derrubam para O(nq); não é estrutura de propósito geral.
- **Sqrt decomposition** quando a operação não é associativa ou é estranha; blocos de √n e reconstrução periódica.
  - *Gatilho:* operação difícil de juntar (moda, contagem por valor) ou custos assimétricos entre update e query.
  - *Por que funciona:* um bloco inteiro é respondido por um resumo local; só as duas pontas custam O(B).
  - *Como fazer:* B ≈ √n; guarde por bloco um resumo (vetor ordenado, contagens, bitset); query = pontas na força + blocos inteiros; update reconstrói só o bloco.
  - *Exemplo:* contar ≤ x em [l, r] com vetor ordenado por bloco: O(√n log n).
  - *Armadilha:* o B ótimo depende da operação; se a reconstrução é cara, a conta muda.
- **Pesado/leve por frequência:** valores que aparecem > √n vezes são poucos → trate separado.
  - *Gatilho:* a resposta depende da frequência dos valores num intervalo (moda, "aparece ≥ k vezes").
  - *Por que funciona:* há no máximo n/√n = √n valores que aparecem mais de √n vezes.
  - *Como fazer:* pesados: prefixo de frequências por valor (O(n√n) de tempo/memória); leves: vetor de posições + binária; combine as respostas.
  - *Exemplo:* existe valor com frequência > √n em [l, r]?
  - *Armadilha:* o limiar é sobre n, não sobre q; com updates a lista de pesados muda.

- **CDQ / D&C no tempo** para dominância 3D ou queries com updates offline.
  - *Gatilho:* contribuições entre pares com i < j e duas condições, ou updates e queries que só dependem da ordem temporal.
  - *Por que funciona:* dividir no tempo garante que a metade esquerda só contribui para a direita; dentro do nível, uma dimensão é ordenada e a outra vira BIT.
  - *Como fazer:* cdq(l, r); cdq(l, m); contribua de [l, m] para [m+1, r] ordenando pela 2ª coordenada e usando BIT na 3ª; cdq(m+1, r). O(n $\log^2$ n).
  - *Exemplo:* dominância 3D; DP com condição 2D.
  - *Armadilha:* desempate (update antes de query); desfaça o BIT ao fim de cada nível.
- **Hash de multiconjunto** (soma de valores aleatórios) para "[l, r] é permutação" ou "conjuntos iguais".
  - *Gatilho:* comparar conjuntos/multiconjuntos de intervalos, "é permutação de", "as mesmas cartas".
  - *Por que funciona:* some um valor aleatório r_v por ocorrência; a soma não depende da ordem e duas multisets diferentes colidem com probabilidade baixa.
  - *Como fazer:* h[i] = h[i−1] + r[a_i]; hash de [l, r] = h[r] − h[l−1]; compare com o hash do alvo. Use r_v aleatório de 64 bits, fixo por valor.
  - *Exemplo:* [l, r] é permutação de 1..k ⇔ h[r] − h[l−1] = Σ_{v=1..k} r_v.
  - *Armadilha:* use soma (não xor) para multiconjunto; o alvo precisa da multiplicidade certa.
- **DSU com informação extra** (paridade, tamanho, distância ao pai) para relações entre pares.
  - *Gatilho:* equações entre variáveis (x_a − x_b = w), bipartição, somas por componente.
  - *Por que funciona:* a floresta guarda a diferença de cada nó até o pai; find acumula até a raiz e a diferença entre dois nós é d(a) − d(b).
  - *Como fazer:* pot[x] = diferença até o pai; find comprime somando pot; union(a, b, w) checa consistência e liga as raízes ajustando a diferença. Quase O(α).
  - *Exemplo:* verificar se um grafo é bipartido online; equações de soma.
  - *Armadilha:* com rollback não use compressão; ajuste o sinal da diferença na direção certa ao ligar as raízes.

Combinações de estruturas (uma transformação + uma estrutura):

Problema	Transformação	Estrutura
Update/query em subárvore	Euler tour (tin, tout) → intervalo	Segtree / BIT (lazy se update em intervalo)
Update em vértice, query no caminho raiz–v	Euler tour com +x em tin[u] e −x em tout[u] + 1	BIT de prefixo
Update no caminho raiz–v, query em vértice	Ponto em v, query = soma na subárvore	Euler tour + BIT
Query em caminho u–v com updates	HLD → O(log) intervalos	Segtree (lazy)
Query em caminho sem updates, operação invertível	Prefixo da raiz: f(u) + f(v) − 2f(lca) em arestas; em vértices, − f(lca) − f(pai do lca)	LCA
Query em caminho, min/max estático	Binary lifting guardando o agregado	Sparse por saltos
LCA em O(1)	Euler tour completo (2n − 1)	Sparse table de mínimo
Distintos em [l, r]	Ordene queries por r, guarde última ocorrência	BIT
k-ésimo menor em [l, r]	Compressão de coordenadas, versões por prefixo	Segtree persistente / wavelet tree
Contar a_i ≤ x em [l, r]	Offline por x, ou merge sort tree	BIT / merge sort tree
Distintos ou cores em subárvore	Euler tour vira "distintos em intervalo"	BIT offline, ou small-to-large
Queries em caminho de árvore, add/remove O(1)	Euler tour de 2n posições	Mo em árvore
Dominância 2D (i < j, a_i < a_j)	Ordene por uma coordenada	BIT na outra
Dominância 3D	D&C na 1ª, ordene a 2ª	CDQ + BIT
Retângulos / pontos em retângulo	Sweep em x, eventos	Segtree em y (área: contagem de cobertura)
Conectividade com remoção de arestas	Intervalo de vida de cada aresta	Segtree no tempo + DSU com rollback
Conectividade só com remoções	Inverta o tempo	DSU
Max xor de a_i com x em [l, r]	Prefixo de versões	Trie persistente
Ocorrências de padrão em substring s[l..r]	Suffix array → intervalo de sufixos	Segtree persistente / BIT offline

Problema	Transformação	Estrutura
Ocorrências na subárvore do SAM/trie	Euler tour na árvore de links	BIT / small-to-large de sets
DP com transição min em intervalo de j	Índices de j como array	Segtree / deque monotônico
DP com transição sobre valores (LIS e variantes)	Indexe pelo valor comprimido	BIT de max
Distância em árvore para "vértice marcado mais próximo"	Centroid: dist(v, c) por ancestral	Centroid decomposition + min por centroide
Palíndromo/igualdade de substring com updates	Hash polinomial como soma	Segtree / BIT de hash

adicionado em 2026-11-10 por agentes; revisado

- **BIT range-update range-query.** Mantenha duas BITs B1, B2 sobre a diferença d: prefixo(x) = B1(x)·x − B2(x), com add(l, v): B1 += v em l, B2 += v·(l−1); add(r+1, −v) analogamente. Soma em [l, r] = prefixo(r) − prefixo(l−1).
  - *Gatilho:* somar v em [l, r] e consultar a soma de [l, r], com muitas operações.
  - *Por que funciona:* representando o array pela diferença d, a soma de prefixo é combinação linear de Σd e Σ(j−1)·d; duas BITs guardam esses dois coeficientes.
  - *Como fazer:* add(l, v): B1 += v, B2 += v·(l−1); add(r+1, −v): B1 −= v, B2 −= v·r; prefixo(x) = x·Σ_{i≤x}B1[i] − Σ_{i≤x}B2[i]. O(log n) por operação.
  - *Exemplo:* range add + range sum clássico.
  - *Armadilha:* os produtos x·v estouram 32 bits; use long long. A fórmula pressupõe 1-indexação.
- **BIT 2D e BIT de dimensão alta.** Update/query em O(log^d); memória n^d. Com coordenadas esparsas, comprima por offline (BIT de vetores ordenados) em vez de alocar n^2.
  - *Gatilho:* pontos (x, y, ...) com update de ponto e soma de caixa; d pequeno (2 ou 3).
  - *Por que funciona:* cada nível do BIT cobre uma faixa daquela coordenada; a consulta decompõe a caixa em O(log^d) blocos.
  - *Como fazer:* colete offline todas as coordenadas y que cada nó x visitará; ordene e faça binária no vetor de cada nó. Update $O(\log^2 n)$, query $O(\log^2 n)$, memória O(q log n).
  - *Exemplo:* contar pontos em [x1, x2] × [y1, y2] com inserções.
  - *Armadilha:* alocar n^2 é inviável para $n = 10^5$; a query de retângulo usa 4 prefixos por inclusão-exclusão.
- **Busca binária na BIT.** Para "menor i com prefixo ≥ k" (k-ésimo de multiconjunto, valores ≥ 0) suba pelas potências de 2 de cima para baixo: O(log n), sem $\log^2$ da binária com query.
  - *Gatilho:* achar o k-ésimo elemento de um multiconjunto, ou o primeiro prefixo que atinge k.
  - *Por que funciona:* os intervalos do BIT são potências de 2; descer do maior bit mantém a soma acumulada e localiza a posição sem busca binária externa.
  - *Como fazer:* pos = 0, acc = 0; para k de ⌊log n⌋ até 0, se pos + 2^k ≤ n e acc + t[pos + 2^k] < alvo, avance e some; resposta = pos + 1.
  - *Exemplo:* mediana dinâmica com BIT de contagens (alvo = ⌊(total + 1)/2⌋).
  - *Armadilha:* só vale com valores não negativos (prefixo monótono); se alvo > total, não existe.
- **BIT só serve a grupos (operação invertível) para query de intervalo.** Para max/min use BIT apenas se os updates só aumentam (max) e a query é de prefixo/sufixo; senão segtree.
  - *Gatilho:* quer usar BIT para max/min em intervalo.
  - *Por que funciona:* BIT responde intervalo por diferença de prefixos, o que exige inverso; max não tem inverso.
  - *Como fazer:* max com updates só crescentes: BIT de max sobre prefixo (índice invertido para sufixo); updates arbitrários: segtree.
  - *Exemplo:* LIS com BIT de max por valor (só insere).
  - *Armadilha:* um decremento invalida o BIT de max; remover um valor exige segtree/multiset.
- **BIT com construção em O(n).** Comece com t[i] = a[i] e, para i crescente, faça t[i + lowbit(i)] += t[i]; evita n·log n na leitura de arrays grandes.
  - *Gatilho:* array grande e build no caminho crítico.
  - *Por que funciona:* cada nó do BIT é a soma de um intervalo; propagar na ordem crescente soma cada bloco exatamente uma vez.
  - *Como fazer:* t = a (1-indexado); para i = 1..n, j = i + lowbit(i); se j ≤ n, t[j] += t[i]. O(n).
  - *Exemplo:* inicializar BIT de frequências de 10^6 posições.
  - *Armadilha:* a ordem i crescente é obrigatória; não use add em loop (n log n).

- **Segtree não comutativa.** Combine sempre esquerda·direita na ordem; em segtree iterativa (bottom-up) mantenha resl e resr separados. Exemplo: composição de funções afins ax+b, multiplicação de matrizes, concatenação de strings/hashes.
  - *Gatilho:* a operação não comuta (concatenação, matriz, função, hash).
  - *Por que funciona:* o intervalo é a composição na ordem; juntar fora de ordem produz outro resultado.
  - *Como fazer:* merge(left, right) sempre nessa ordem; na iterativa, resl = op(resl, tree[l]) e resr = op(tree[r], resr), e no fim op(resl, resr).
  - *Exemplo:* segtree de matrizes 2×2 para recorrência de Fibonacci.
  - *Armadilha:* o elemento neutro tem de funcionar dos dois lados; cuidado com a query vazia.
- **Nó como monóide de funções.** Se o update é "aplicar f em [l, r]" e a query é "soma", o lazy guarda a composição de f e o nó guarda (soma, tamanho) para saber aplicar f em O(1). Verifique que f∘g também pertence à classe (afim ∘ afim = afim).
  - *Gatilho:* update aplica uma função a cada elemento e a query é soma/contagem.
  - *Por que funciona:* se f é linear sobre o resumo, o nó inteiro é atualizado em O(1) sem descer; a composição das funções fica no lazy.
  - *Como fazer:* nó (soma, len); aplicar f(x) = a·x + b dá soma' = a·soma + b·len; compor tags na ordem: (a2, b2)∘(a1, b1) = (a2a1, a2b1 + b2).
  - *Exemplo:* range affine (ax + b) + range sum.
  - *Armadilha:* a tag nova age depois da antiga; com módulo, cuidado com b negativo.
- **Segtree de "autômato".** Cada nó guarda a função estado→estado para o bloco (tabela de k entradas, k pequeno); query = compor as tabelas. Serve para "processar string/sequência pelo autômato com updates pontuais" (ex.: contar subsequência "abc", parênteses com k tipos de estados).
  - *Gatilho:* a sequência é processada por um autômato pequeno e há updates pontuais.
  - *Por que funciona:* a ação de um bloco é uma função estado→estado e funções compõem associativamente.
  - *Como fazer:* tabela de k entradas por nó (estado inicial → final); merge C[s] = B[A[s]] (aplicar A, depois B); query compõe da esquerda para a direita. O(k log n) por operação.
  - *Exemplo:* contar subsequências "abc" com 4 estados; casar parênteses de k tipos.
  - *Armadilha:* a ordem da composição; se k cresce, o custo por nó domina.
- **Segtree de parênteses.** Nó guarda (soma, mínimo prefixo); sequência é balanceada ⇔ soma = 0 e mínimo ≥ 0. Merge: soma = a + b, min = min(a.min, a.soma + b.min). Mesmo nó resolve "primeiro prefixo negativo" descendo na árvore.
  - *Gatilho:* validar parênteses, "primeiro ponto em que o saldo fica negativo", updates de '(' ↔ ')'.
  - *Por que funciona:* (soma, mínimo prefixo) é associativo e resume completamente o bloco para essas perguntas.
  - *Como fazer:* nó (soma, min); merge soma = a.soma + b.soma, min = min(a.min, a.soma + b.min); desça para achar o primeiro prefixo negativo. O(log n).
  - *Exemplo:* validação de sequência de parênteses com updates pontuais.
  - *Armadilha:* o mínimo do prefixo vazio é 0; para k tipos de parênteses são necessários mais contadores.
- **Segtree: máximo subarray.** Nó = (soma, melhor prefixo, melhor sufixo, melhor). Base de dezenas de variantes; com update de atribuição/negação guarde também o par com sinal invertido (max e min) para o lazy de "negar".
  - *Gatilho:* soma máxima de subarray contíguo, com updates ou queries de intervalo.
  - *Por que funciona:* um subarray ou está inteiro num filho ou cruza o meio: sufixo do esquerdo + prefixo do direito.
  - *Como fazer:* merge: soma = a.soma + b.soma; pref = max(a.pref, a.soma + b.pref); suf = max(b.suf, b.soma + a.suf); best = max(a.best, b.best, a.suf + b.pref). O(log n).
  - *Exemplo:* GSS1/GSS3 (SPOJ) — max subarray estático e com updates.
  - *Armadilha:* com lazy de negação guarde também min, min pref, min suf e best negativo; negar troca max↔min.
- **Segtree beats (chmin/chmax).** Guarde max, segundo max e contagem do max; chmin(x) só toca nós onde segundo max < x < max. Amortizado O(log n) por op (O(log² n) se há add em intervalo).
  - *Gatilho:* chmin/chmax em intervalo combinado com soma, max ou min.
  - *Por que funciona:* se x está entre o 2º max e o max, só os máximos mudam; dá para atualizar o nó em O(1) sem descer.
  - *Como fazer:* nó (max1, max2, cnt1, soma); chmin(x): x ≥ max1 não faz nada; max2 < x < max1 → soma −= cnt1·(max1 − x), max1 = x; senão desça. Amortizado O(log n).
  - *Exemplo:* HDU 5306 "Gorgeous Sequence" (chmin + max + soma).
  - *Armadilha:* precisa dos dois sentidos (chmin e chmax) e das somas corretas; combinar com add exige atualizar também max2.

- **Segtree com descida em vez de binária+query.** "Primeiro i ≥ l com a_i ≥ x", "k-ésimo 1", "primeiro prefixo ≤ 0" → desça guardando max/soma no nó: O(log n) em vez de O(log² n).
  - *Gatilho:* achar a primeira posição que satisfaz uma condição monotônica no resumo.
  - *Por que funciona:* se o resumo do filho esquerdo não pode conter a resposta, o filho inteiro é descartado.
  - *Como fazer:* desça da raiz carregando o limite l e a soma acumulada; em cada nível decida esquerda/direita em O(1). O(log n).
  - *Exemplo:* primeiro elemento ≥ x à direita de l; k-ésimo 1.
  - *Armadilha:* tratar com cuidado os nós parcialmente à esquerda de l; a descida substitui a binária, não a query.
- **Merge sort tree.** Cada nó guarda o intervalo ordenado; "contar < x em [l, r]" em O(log² n), com fractional cascading em O(log n). Não suporta update (use BIT de BITs/ordered set por nó se precisar).
  - *Gatilho:* contar valores em faixa num intervalo de posições, sem updates.
  - *Por que funciona:* a query decompõe [l, r] em O(log n) nós; cada nó responde por busca binária no seu vetor ordenado.
  - *Como fazer:* build O(n log n) de tempo e memória; query com upper_bound em cada nó → O(log² n); fractional cascading reduz para O(log n).
  - *Exemplo:* contar elementos em [a, b] com posições em [l, r].
  - *Armadilha:* update não é suportado; com updates use BIT de vetores ordenados (offline) ou segtree de ordered sets.
- **Wavelet tree / wavelet matrix.** k-ésimo, contar < x, contar valores em [lo, hi] em [l, r], tudo em O(log σ) estático e com memória n log σ; mais leve que segtree persistente. Também dá quantil e "valor mais próximo de x" no intervalo.
  - *Gatilho:* muitas queries estáticas de ordem/faixa de valores, online.
  - *Por que funciona:* cada nível particiona os valores por um bit; as posições são guardadas por rank, então um nível responde metade da consulta.
  - *Como fazer:* O(log σ) níveis de bitvectors com rank; desça pelo bit do valor; memória n log σ bits; query O(log σ).
  - *Exemplo:* k-ésimo menor em [l, r] sem updates; quantil.
  - *Armadilha:* não suporta updates; comprima os valores antes e cuide do intervalo vazio.
- **Segtree persistente = versões por prefixo.** Query em [l, r] = (versão r) − (versão l−1) quando o nó é aditivo (contagens). Também resolve "k-ésimo em [l, r]", "contar distintos" (com a versão guardando a última ocorrência) e soma em retângulo online.
  - *Gatilho:* queries em [l, r] precisam do estado do prefixo r menos o do prefixo l−1; ou precisa de versões antigas.
  - *Por que funciona:* o nó é aditivo, então a diferença das duas versões isola exatamente os elementos de [l, r].
  - *Como fazer:* versão i = update da versão i−1 criando O(log n) nós; query desce comparando cnt(direita) − cnt(esquerda) entre as raízes.
  - *Exemplo:* POJ 2104 "K-th Number" (k-ésimo em [l, r]).
  - *Armadilha:* memória O(n log n) nós; para distintos, apague prev(i) e insira i em cada versão.
- **Persistência com lazy.** Não propague tags (marcação permanente / lazy sem push-down): o nó guarda a tag e a query soma as tags dos ancestrais; assim não precisa copiar filhos na leitura.
  - *Gatilho:* range update + range query com versões.
  - *Por que funciona:* a tag vale para todo o nó e é herdada; a query acumula as tags do caminho e aplica ao resumo, sem tocar nos filhos.
  - *Como fazer:* nó guarda resumo e tag; update cria nós no caminho e marca; query acumula as tags dos ancestrais e combina com o resumo do nó.
  - *Exemplo:* range add + range sum persistente.
  - *Armadilha:* o resumo do nó tem de incluir a própria tag; a ordem de acumulação importa.
- **Segtree dinâmica (esparsa).** Domínio até 10^9 ou 10^{18} → crie nós sob demanda; memória O(q log U). Útil para DP com CHT/Li Chao sobre x grande e para BIT-de-segtree online em 2D.
  - *Gatilho:* o domínio é enorme mas só poucos pontos são tocados.
  - *Por que funciona:* só existem nós nos caminhos visitados; o resto é implicitamente o elemento neutro.
  - *Como fazer:* nó com filhos nulos; crie ao descer; update/query O(log U); memória O(q log U).
  - *Exemplo:* Li Chao dinâmico; contar pontos em [1, 10^9].
  - *Armadilha:* o neutro tem de ser coerente para nós ausentes (0 para soma, +∞ para min); não use para varrer intervalos amplos.

- **Segtree de segtrees (2D online).** Update de ponto e query de retângulo com a camada externa em x e interna dinâmica em y: $O(\log^2)$ por op, memória $O(q \log^2)$. Para queries só de max sem update de decremento, a camada interna pode ser BIT.
  - *Gatilho:* 2D online com updates de ponto e queries de retângulo.
  - *Por que funciona:* a camada externa indexa x; cada nó externo tem uma estrutura em y com os pontos daquela faixa de x.
  - *Como fazer:* externa em segtree/BIT sobre x; interna em segtree dinâmica (ou BIT de max); update e query $O(\log^2)$; memória $O(q \log^2)$.
  - *Exemplo:* contar pontos em retângulo com inserções online.
  - *Armadilha:* constante alta; se as queries são offline, prefira sweep. Para max com updates só crescentes, a interna pode ser BIT.
- **Segtree 2D com lazy é doloroso.** Prefira offline: sweep em x + segtree 1D em y, ou transforme retângulo em 4 queries de prefixo 2D por inclusão-exclusão.
  - *Gatilho:* update de retângulo + query de retângulo.
  - *Por que funciona:* a tag 2D precisa cobrir combinações de faixas; a implementação fica cara e propensa a erro.
  - *Como fazer:* offline: eventos em x + segtree 1D em y; ou converta o retângulo em 4 prefixos 2D e use inclusão-exclusão.
  - *Exemplo:* soma em retângulos com updates de retângulo offline.
  - *Armadilha:* não tente lazy 2D ingênuo; memória e constante explodem.
- **Segtree kinetic (KTT) / kinetic segment tree.** Guarda retas $m_i \cdot t_i + c_i$ em cada nó para "max de $m_i \cdot t_i + c_i$"; cada t_i só cresce (update soma $\delta > 0$ a t_i no intervalo, o que equivale a $c_i \mathrel{+}= m_i \cdot \delta$); reconstrua o nó quando o tempo ultrapassa o ponto de troca. Amortizado polilog ($O(\log^3 n)$ na análise clássica) com $\delta > 0$.
  - *Gatilho:* manter o máximo de retas avaliadas em tempos que só crescem, com updates de "some δ ao tempo" num intervalo.
  - *Por que funciona:* somar δ a t_i é somar $m_i \cdot \delta$ a c_i; a reta vencedora de um nó só muda quando t passa do cruzamento, então a reconstrução é adiada.
  - *Como fazer:* nó guarda a reta vencedora e o próximo t de troca; se t não passou, só ajuste o offset; senão desça/reconstrua. Amortizado $O(\log^3 n)$ com $\delta > 0$.
  - *Exemplo:* range add ao tempo + query de max de $m \cdot t + c$.
  - *Armadilha:* δ negativo quebra a amortização; use __int128 nos cruzamentos.
- **Segtree com merge de nós via small-to-large / merge de segtrees.** Em DP de subárvore com estruturas indexadas por valor, funda segtrees dinâmicas dos filhos em $O(n \log n)$ total (merge destrutivo; o custo amortizado é o número de nós criados).
  - *Gatilho:* DP de subárvore em que cada nó tem um mapa/estrutura indexada por valor e é preciso juntar os filhos.
  - *Por que funciona:* fundir duas segtrees dinâmicas custa o número de nós sobrepostos; cada nó é destruído no máximo uma vez, então o total é proporcional aos nós criados.
  - *Como fazer:* merge(a, b) desce só onde ambos existem; se um é nulo, retorne o outro; atualize o resumo na volta. Destrutivo, $O(n \log n)$ no total.
  - *Exemplo:* contar cores distintas em cada subárvore; DP com mapas de valores.
  - *Armadilha:* não use a segtree fundida depois; memória $O(n \log n)$ nós.
- **Segtree iterativa bottom-up.** 2–3× mais rápida; funciona para operações não comutativas se tratar esquerda/direita, mas lazy exige a versão recursiva ou a variante "push do topo até a folha" antes da query.
  - *Gatilho:* performance e operação simples; muitas queries/updates.
  - *Por que funciona:* layout em array com folhas contíguas e sem recursão; melhor localidade de cache.
  - *Como fazer:* n = potência de 2; folhas em [n, 2n); query com l += n, r += n e resl/resr; com lazy, faça push do topo até as folhas antes e reconstrua os ancestrais depois.
  - *Exemplo:* soma/max com update de ponto e query de intervalo.
  - *Armadilha:* com lazy, esquecer o push/reconstrução dá resposta errada; operação não comutativa exige resl/resr.
- **Disjoint sparse table.** Query de qualquer operação associativa (não idempotente: soma, produto mod, xor, composição) em $O(1)$ com preprocessamento $n \log n$: para cada nível guarde prefixos/sufixos a partir do meio do bloco.
  - *Gatilho:* operação associativa mas não idempotente, array estático, muitas queries.
  - *Por que funciona:* cada nível divide em blocos de 2^(k+1) e guarda sufixos da metade esquerda e prefixos da direita; qualquer intervalo é um sufixo + um prefixo de um único nível.
  - *Como fazer:* pré-processe $O(n \log n)$; para $l < r$, $k = \lfloor \log(l \oplus r) \rfloor$ e resposta = op(suf[k][l], pref[k][r]); l = r devolve a[l].
  - *Exemplo:* soma de intervalos estática em $O(1)$.
  - *Armadilha:* memória $O(n \log n)$; para operações idempotentes a sparse table comum gasta menos.

- **Sparse table também resolve idempotentes (min, max, gcd, and, or) em O(1)** pois pode sobrepor dois blocos de 2^k. Para gcd, a query é O(1) sem log do gcd só pela sobreposição.
  - *Gatilho:* operação idempotente (aplicar o mesmo elemento duas vezes não muda o resultado).
  - *Por que funciona:* dois blocos de 2^k cobrem o intervalo e a sobreposição não altera op(x, x) = x.
  - *Como fazer:* k = ⌊log(len)⌋; resposta = op(st[k][l], st[k][r − 2^k + 1]). O(1).
  - *Exemplo:* RMQ; gcd de intervalo.
  - *Armadilha:* para soma/produto a sobreposição conta duas vezes; comprimento zero precisa de tratamento.
- **Sparse table de índices.** Guarde o índice do mínimo em vez do valor para devolver "onde"; comparação por (valor, índice) mantém desempate estável (mais à esquerda).
  - *Gatilho:* precisa da posição do mínimo/máximo, não só do valor.
  - *Por que funciona:* comparar (valor, índice) é uma ordem total; o desempate fica determinístico e estável.
  - *Como fazer:* guarde índices; compare a[i] e, no empate, o menor índice; a query devolve o índice.
  - *Exemplo:* LCA por Euler tour + sparse table de mínimo (posição da primeira ocorrência).
  - *Armadilha:* a convenção de desempate muda respostas de contagem; fixe esquerda ou direita.
- **Sqrt decomposition: escolha do bloco.** Se update custa O(1) e query O(B + n/B) ou ao contrário, B = √n; se o custo de reconstrução do bloco é O(B log B), ajuste B ≈ √(n / log n). Teste B por experimento: constante importa.
  - *Gatilho:* escolher o tamanho de bloco para balancear update e query.
  - *Por que funciona:* o custo por operação é a soma de um termo O(B) e um O(n/B); o ótimo equaliza os dois.
  - *Como fazer:* B = √n no caso simétrico; com reconstrução O(B log B), resolva B log B ≈ n/B e use B ≈ √(n / log n); valide medindo.
  - *Exemplo:* contar ≤ x com blocos ordenados.
  - *Armadilha:* o B ótimo depende da razão entre updates e queries; um B fixo pode ser 2× pior.
- **Sqrt em queries de tempo (rebuild a cada √q operações).** Mantenha estrutura estática reconstruída periodicamente + buffer das últimas ≤ √q mudanças verificadas à força bruta a cada query.
  - *Gatilho:* queries online com updates intercalados e uma estrutura estática fácil de reconstruir.
  - *Por que funciona:* entre reconstruções só há ≤ B mudanças; a query corrige a resposta estática olhando o buffer.
  - *Como fazer:* reconstrua a cada B = √q operações; guarde o buffer de mudanças; query = resposta estática ajustada pelas ≤ B mudanças. O((n + q)√q) se a query estática é O(1)/O(log).
  - *Exemplo:* conectividade dinâmica com poucos updates.
  - *Armadilha:* mudanças repetidas no mesmo ponto precisam ser consolidadas; reconstruir custa O(n).
- **Sqrt com bloco que guarda estrutura auxiliar.** Bloco com array ordenado (contar ≥ x), bitset, ou tabela de contagem por valor (blocos × valores) com prefix sums por bloco para "k-ésimo/contar em [l, r]" em O(√n) com O(1) por bloco.
  - *Gatilho:* contagens/k-ésimo em intervalo com updates.
  - *Por que funciona:* o bloco inteiro responde em O(1) pela tabela/prefixo; só as pontas custam O(B).
  - *Como fazer:* tabela blocos × valores com prefixo por bloco (memória O((n/B)·V)); ou vetor ordenado/bitset por bloco; query O(√n), update O(B) ou O(1) + recomputo.
  - *Exemplo:* k-ésimo em [l, r] por contagem de valores.
  - *Armadilha:* a tabela blocos × valores pode usar O(n√n) de memória; comprima os valores.
- **Sqrt no "pular" (Holes / jump).** Para cada i guarde onde cai após sair do bloco e quantos saltos deu; update reconstrói só o bloco (CF 13E "Holes").
  - *Gatilho:* sequência de saltos (i → i + a[i], ou i → pai) com updates pontuais.
  - *Por que funciona:* dentro do bloco o salto é local; guardando a saída e a contagem, atravessa o bloco em O(1).
  - *Como fazer:* nxt[i] = ponto de saída do bloco, cnt[i] = saltos até sair; update reconstrói o bloco de trás para frente; query O(√n).
  - *Exemplo:* CF 13E "Holes".
  - *Armadilha:* a direção do salto importa; reconstrua o bloco inteiro, não só a posição alterada.

- **Mo + rollback (sem remoção).** Quando só dá para adicionar (ex.: max, união sem desfazer), ordene por bloco de l e r crescente; para cada bloco refaça l a partir do fim do bloco e desfaça os adds de l após cada query ("Mo com rollback").
  - *Gatilho:* add é fácil e remove não existe ou é caro; queries de intervalo offline.
  - *Por que funciona:* o lado r só cresce dentro do bloco; o lado l é reconstruído do zero e desfeito por rollback, sem nunca remover de verdade.
  - *Como fazer:* ordene por bloco de l e r crescente; para cada bloco, comece com l no fim do bloco; adicione r; para cada query, adicione temporariamente os ≤ B elementos de l e desfaça. O((n + q)√n).
  - *Exemplo:* máximo em [l, r] mantendo só uma variável (sem multiset); união de conjuntos sem desfazer.
  - *Armadilha:* o rollback tem de restaurar exatamente o estado; não use quando a remoção é realmente necessária.
- **Mo com updates.** Terceira coordenada t; blocos de tamanho n^(2/3), ordem (⌊l/B⌋, ⌊r/B⌋, t); O(n^(5/3)). Update ao vivo: se a posição está dentro de [l, r], remova antigo e adicione novo.
  - *Gatilho:* o array sofre updates pontuais entre as queries, tudo offline.
  - *Por que funciona:* a dimensão tempo entra como terceira coordenada; com B = n^(2/3), o movimento total dos três ponteiros fica O(n^(5/3)).
  - *Como fazer:* ordene por (⌊l/B⌋, ⌊r/B⌋, t) com serpente; alterne t aplicando/desfazendo updates; se a posição está na janela, remova o valor antigo e adicione o novo.
  - *Exemplo:* "distintos em [l, r]" com point updates.
  - *Armadilha:* guarde o valor antigo para desfazer; alterar dentro da janela exige remover antes de adicionar.
- **Mo em árvore.** Euler tour de 2n posições; vértice aparece duas vezes; alterne add/remove a cada toggle; para query (u, v) com lca fora do segmento adicione o lca à parte. Ver "Combinações".
  - *Gatilho:* queries em caminho de árvore com add/remove O(1).
  - *Por que funciona:* o Euler de 2n posições transforma caminho em intervalo; cada vértice aparece duas vezes e o toggle liga/desliga.
  - *Como fazer:* tin/tout no Euler de 2n; para (u, v) com tin[u] > tin[v] troque; se lca(u,v) = u, use [tin[u], tin[v]]; senão [tout[u], tin[v]] e adicione o lca à parte.
  - *Exemplo:* contar valores distintos num caminho de árvore.
  - *Armadilha:* não inclua o lca duas vezes; a paridade do toggle tem de ser consistente.
- **Mo + bitset.** Quando o add/remove é O(1) mas a resposta é difícil, mantenha o bitset do conjunto atual e responda em n/64 (ex.: "existe soma x = a + b em [l, r]"? use bitset e shifts).
  - *Gatilho:* add/remove O(1), mas a pergunta é sobre existência de uma combinação no conjunto.
  - *Por que funciona:* operações de bitset custam n/64, muito menos que recomputar a resposta do zero.
  - *Como fazer:* mantenha o bitset S do conjunto; "existe diferença x" testa S & (S >> x); para soma x mantenha também o espelhado e teste S & (rev >> (M − x)). O(n/64) por query.
  - *Exemplo:* existe par com diferença x em [l, r] com Mo.
  - *Armadilha:* shifts com deslocamento ≥ largura; a soma exige o bitset espelhado, só o shift não basta.
- **Mo com ordenação de Hilbert.** Ordena queries pela curva de Hilbert em (l, r): mesma complexidade mas 1.5–2× menos movimento na prática; útil para Mo de 10^5–10^6 queries.
  - *Gatilho:* Mo com muitas queries e a constante importa.
  - *Por que funciona:* a curva de Hilbert preserva melhor a localidade 2D do que a ordem por blocos, reduzindo o movimento total dos ponteiros.
  - *Como fazer:* calcule a ordem de Hilbert de (l, r) e ordene por ela; mesma complexidade assintótica, menos movimento na prática.
  - *Exemplo:* Mo com 10^6 queries.
  - *Armadilha:* o cálculo da ordem tem custo; para poucas queries não compensa.
- **Mo e "ponteiros que andam"**: se add custa O(1) e remove custa O(log), prefira o Mo onde só a direção mais barata se repete mais.
  - *Gatilho:* add e remove com custos diferentes.
  - *Por que funciona:* o custo total é (#adds)·c_add + (#removes)·c_rem; a ordem e o tamanho do bloco mudam essa proporção.
  - *Como fazer:* se remove é caro, use rollback (só adds) ou aumente B para reduzir o movimento de l; a ordem serpente já reduz o movimento de r.
  - *Exemplo:* add O(1) e remove O(log n) — rollback ou bloco maior.
  - *Armadilha:* otimizar um ponteiro pode encarecer o outro; meça o custo total, não só o número de movimentos.

- **Mo 2D (queries em retângulos) é raro; prefira offline sweep.** Mo é para queries onde o estado é multiconjunto de elementos de um intervalo.
  - *Gatilho:* queries em retângulos 2D.
  - *Por que funciona:* Mo ganha por ordenar intervalos 1D; em 2D não há ordem barata que reduza o movimento como no 1D.
  - *Como fazer:* prefira sweep em uma dimensão + BIT/segtree na outra; Mo 2D só se o estado for mesmo um multiconjunto 2D.
  - *Exemplo:* contar pontos em retângulo → sweep em x + BIT em y.
  - *Armadilha:* Mo 2D costuma ser mais lento e mais complexo que o sweep.
- **CDQ para DP online.** dp[i] depende de dp[j], j < i, com condição 2D: resolva a metade esquerda, contribua da esquerda para a direita (ordenando por outra dimensão + BIT), depois resolva a direita. Vale para LIS ponderado 2D, CHT não monotônico, "dominância com DP".
  - *Gatilho:* DP em que dp[i] depende de dp[j] com j < i e uma condição 2D.
  - *Por que funciona:* a D&C no índice garante j < i entre metades; a contribuição da esquerda é aplicada antes de resolver a direita.
  - *Como fazer:* cdq(l, r); resolva a esquerda; ordene esquerda e direita pela 2ª dimensão e use BIT na 3ª para transferir; resolva a direita. O(n $\log^2$ n).
  - *Exemplo:* LIS ponderado 2D; DP com dominância.
  - *Armadilha:* dp[j] tem de estar finalizado antes de contribuir; a ordem das transferências importa.
- **CDQ para updates e queries no tempo.** Eventos em ordem temporal; D&C no tempo, a metade esquerda contribui para a direita, ordenada pelo espaço. Cuidado com ordem de desempate: update antes de query na mesma coordenada.
  - *Gatilho:* updates e queries offline com dependência espacial e temporal.
  - *Por que funciona:* a D&C no tempo faz cada evento da esquerda contribuir só para os da direita; dentro do nível, ordenar pelo espaço permite usar BIT.
  - *Como fazer:* eventos (tempo, posição, valor); cdq no tempo; na fusão, ordene por posição, aplique no BIT e desfaça. Desempate: update antes de query.
  - *Exemplo:* "quantos updates anteriores cobrem a posição x" (dominância 3D).
  - *Armadilha:* desempate errado perde ou duplica contribuições; limpe o BIT ao fim do nível.
- **Offline sweep de valor.** "Contar pares (i, j) com a_i + a_j ≤ x e outra condição" → ordene por valor, insira conforme o limite cresce, mantenha estrutura nas posições.
  - *Gatilho:* pares com soma/valor limitado e uma condição extra de posição.
  - *Por que funciona:* ordenar por valor faz o limite crescer monotonicamente; cada elemento entra uma vez na estrutura de posições.
  - *Como fazer:* ordene elementos e queries pelo limite; insira as posições dos a_i ≤ limite e responda com BIT/ordenação nas posições. O((n + q) log n).
  - *Exemplo:* contar pares (i, j) com a_i + a_j ≤ x e i < j.
  - *Armadilha:* contar cada par duas vezes; cuidado com i = j e com o desempate no limite.
- **Offline: ordene queries por limite e elementos por peso.** Padrão "quantos elementos de peso ≤ w têm propriedade P no intervalo/subárvore" = adicionar elementos em ordem de peso numa BIT.
  - *Gatilho:* a query tem um limite de peso e uma restrição de posição/intervalo/subárvore.
  - *Por que funciona:* inserir em ordem de peso transforma "peso ≤ w" em "já está na estrutura"; a posição responde o resto.
  - *Como fazer:* ordene elementos por peso e queries por w; insira na BIT na posição (Euler tour se for subárvore); some o intervalo. O((n + q) log n).
  - *Exemplo:* "quantos vértices com peso ≤ w na subárvore de v".
  - *Armadilha:* pesos repetidos e o limite ≤ (não <); a query tem de incluir todos os pesos iguais a w.
- **Distintos em intervalo, versão com soma.** Soma dos valores distintos em [l, r]: mesmo sweep por r, BIT guarda valor em prev(i) apagado e i ativo. Versão com multiplicidade (≥ 2 vezes) usa prev de segunda ordem.
  - *Gatilho:* somar os valores distintos (não contar) num intervalo, com queries offline.
  - *Por que funciona:* no sweep por r, cada valor deve contribuir apenas na sua última ocorrência; apagar a anterior mantém a soma dos distintos.
  - *Como fazer:* BIT de soma; ao processar r, subtraia a_r em prev(r) e some a_r em r; query = soma(l, r). Para "aparece ≥ 2", marque na segunda última ocorrência.
  - *Exemplo:* soma dos valores distintos em [l, r].
  - *Armadilha:* para a versão ≥ 2 a lógica muda (segunda última ocorrência); não reaproveite o mesmo BIT sem ajustar.

- **Moda em intervalo (range mode).** Sem update: frequência por valor + sqrt (candidatos = modas dos blocos mais valores avulsos, checando por vetor de posições + binária) em $O(n\sqrt{n} + q\sqrt{n} \log n)$; com tabela de contagens por bloco ($O(n\sqrt{n})$ de memória) a checagem é $O(1)$ e o total cai para $O((n+q)\sqrt{n})$; resposta online pelo "pesado/leve" por frequência.
  - *Gatilho:* valor mais frequente em [l, r], online, com ou sem updates.
  - *Por que funciona:* a moda de [l, r] é a moda do miolo de blocos inteiros ou um dos $O(B)$ valores das pontas.
  - *Como fazer:* pré-compute a moda de cada par de blocos em $O(n\sqrt{n})$; candidatos = moda do miolo + valores das pontas; conte cada candidato por posições + binária ($O(B \log n)$ por query) ou por tabela blocos × valores com prefixo ($O(1)$ por candidato).
  - *Exemplo:* modo de intervalo com n, q ≤ 10^5.
  - *Armadilha:* sem a tabela de contagens aparece um log; a tabela custa $O(n\sqrt{n})$ de memória.
- **Range kth online.** Wavelet matrix ou persistente $O(\log n)$; com updates → BIT de segtrees dinâmicas, ou sqrt; binária paralela é offline (todas as queries conhecidas), não responde online.
  - *Gatilho:* k-ésimo menor em [l, r], com ou sem updates, e as queries chegam online.
  - *Por que funciona:* a ordem estatística é respondida descendo uma estrutura de valores que respeita as posições.
  - *Como fazer:* estático: wavelet matrix ou segtree persistente $O(\log \sigma)$; com updates: BIT de segtrees dinâmicas $O(\log^2)$ ou decomposição sqrt; binária paralela só offline.
  - *Exemplo:* POJ 2104 sem updates; com updates, BIT de segtrees.
  - *Armadilha:* binária paralela não responde online; a memória do persistente é $O(n \log \sigma)$.
- **Contar inversões em intervalo.** Mo + BIT ($n\sqrt{n} \log n$); com blocos de contagem sobre valores, o update é $O(1)$ mas a query é $O(\sqrt{n})$, e como são $O(n\sqrt{n})$ movimentos isso leva a $O(n^2)$. Para $O(n\sqrt{n})$ use Mo secundário (2nd offline) calculando via prefixos; alternativa: decomposição sqrt estática.
  - *Gatilho:* muitas queries de "quantas inversões em [l, r]".
  - *Por que funciona:* Mo mantém a lista atual; cada add/remove conta quantos elementos são maiores/menores que o que entra ou sai.
  - *Como fazer:* Mo + BIT $O(n\sqrt{n} \log n)$; com blocos de valor o update é $O(1)$ mas a contagem por movimento é $O(\sqrt{n})$, então o total vira $O(n^2)$; para $O(n\sqrt{n})$, use um segundo Mo offline via prefixos ou sqrt estática.
  - *Exemplo:* inversões em subarrays para muitas queries.
  - *Armadilha:* o custo por movimento inclui a contagem, não só o update; não confunda os dois.
- **Inversões após swaps / updates pontuais.** Mantenha BIT de BITs, ou decomposição em blocos; ou CDQ (tempo, posição, valor), problema de dominância 3D.
  - *Gatilho:* o total de inversões muda após swaps ou updates pontuais.
  - *Por que funciona:* trocar dois elementos só altera os pares que envolvem as duas posições; dá para recalcular a diferença incrementalmente.
  - *Como fazer:* conte os elementos entre as posições com valores entre os dois; BIT de BITs $O(\log^2)$, blocos ordenados $O(\sqrt{n})$, ou CDQ 3D para o histórico.
  - *Exemplo:* contar inversões após cada swap.
  - *Armadilha:* swaps não adjacentes mexem em muitos pares; a fórmula incremental precisa dos dois lados.
- **MEX em intervalo.** Offline por r: para cada valor guarde a última ocorrência last[v] em segtree de mínimo; mex de [l, r] = menor v com last[v] < l (descida na segtree). Para mex com updates → contagem não indica presença em [l, r]: guarde as posições de cada valor (ordered set) e teste presença com estrutura 2D.
  - *Gatilho:* menor inteiro não negativo ausente em [l, r], com muitas queries.
  - *Por que funciona:* com r fixo, v está presente em [l, r] ⇔ last[v] ≥ l; o menor v com last[v] < l é o mex.
  - *Como fazer:* segtree de mínimo sobre last[0..n]; ao processar r, atualize last[a_r] = r; query = menor índice com valor < l (descida). $O(\log n)$.
  - *Exemplo:* mex de [l, r] com queries offline por r.
  - *Armadilha:* só valores ≤ n importam (mex ≤ n); com updates, frequência não indica presença, use posições + estrutura 2D.
- **Xor máximo com x em [l, r].** Trie persistente por prefixo com contagem; query compara contagem(r) − contagem(l−1) em cada nível. Alternativa: trie com "maior índice" em cada nó e query com l.
  - *Gatilho:* max $a_i \oplus x$ com $i \in [l, r]$, online.
  - *Por que funciona:* a diferença de contagens entre as versões diz se um ramo tem elemento dentro do intervalo.
  - *Como fazer:* trie persistente por prefixo; em cada bit prefira o oposto de x se cnt(r) − cnt(l−1) > 0; $O(\log A)$.
  - *Exemplo:* máximo xor com x num subarray.
  - *Armadilha:* a variante com "maior índice" guarda um índice por nó, não contagens; não misture as duas implementações.

- **Maior subarray de xor / par com xor máximo no intervalo.** Subarray: prefix xor + trie persistente (par de prefixos); base linear com "prefix linear basis" (mantém, para cada prefixo, base com timestamps mais recentes) responde "base de [l, r]" em O(log) por query. Par de elementos (a_i xor a_j, ambos em [l, r]): prefix xor não resolve; use trie sobre os valores com Mo + rollback (só adições) ou D&C offline.
  - *Gatilho:* "maior xor de subarray contíguo" ou "par de elementos com xor máximo" dentro de [l, r].
  - *Por que funciona:* xor de subarray = p[j] ⊕ p[i−1], então vira par de prefixos; xor de subconjunto é respondido pela base linear; par de elementos não tem redução a prefixos.
  - *Como fazer:* subarray: trie persistente sobre p; para cada extremidade j, pegue o melhor i em [l−1, j−1]; base linear com timestamps para a base de [l, r] em O(log A). Par: trie dos valores com Mo + rollback (só adiciona) ou D&C offline.
  - *Exemplo:* maior xor de subarray; maior xor de subconjunto em [l, r]; par com xor máximo.
  - *Armadilha:* a trie persistente responde "melhor par com um x fixo", não o máximo sobre todos os pares; prefix xor não resolve par arbitrário.
- **Prefix linear basis (timestamp).** Na inserção de x no índice i, em cada bit, se o ocupante tem índice menor, troque (guarde o mais recente); então a base de [l, r] usa só vetores com índice ≥ l. Resolve xor máximo em subarray.
  - *Gatilho:* xor máximo de um subconjunto de a[l..r], com queries online.
  - *Por que funciona:* trocar por vetores de índice maior mantém uma base válida e garante que a base do prefixo r restrita a índices ≥ l gere o mesmo espaço que a de [l, r].
  - *Como fazer:* insira x no índice i; para cada bit, se o novo tem índice maior que o ocupante, troque; guarde (vetor, índice); query percorre os pivôs com índice ≥ l. O(log A).
  - *Exemplo:* max xor de subconjunto de a[l..r] (CF 1100F "Ivan and Burgers").
  - *Armadilha:* trocar sem guardar o índice quebra a propriedade; a base é sobre bits, não sobre posições.
- **Maior retângulo em histograma/queries.** Estático: pilha monotônica. Com query de intervalo → Cartesian tree + sparse table (retângulo máximo que contém o mínimo) ou segtree com (min, melhor, prefixo/sufixo do nível).
  - *Gatilho:* maior área de retângulo num histograma, ou numa subarray [l, r] com muitas queries.
  - *Por que funciona:* cada barra i é o mínimo de um intervalo maximal [L_i, R_i]; a área é a_i·(R_i − L_i + 1). Numa query, os limites l e r cortam a expansão.
  - *Como fazer:* estático O(n) com pilha para L/R; para queries, resposta = max_{i∈[l,r]} a_i·(min(r,R_i) − max(l,L_i) + 1), com Cartesian tree + sparse table ou uma segtree especializada.
  - *Exemplo:* maior retângulo em histograma; maior retângulo em subarray.
  - *Armadilha:* o máximo global pré-computado não serve; o corte por l e r muda a área de cada barra.
- **Maior retângulo de 1s em matriz binária.** Histograma por linha + pilha: O(nm). Com updates → segtree por coluna é difícil; geralmente reduza.
  - *Gatilho:* maior retângulo de 1s numa matriz binária.
  - *Por que funciona:* cada linha define um histograma de alturas de 1s consecutivos; o retângulo ótimo termina em alguma linha.
  - *Como fazer:* para cada linha, h[j] = 0 se a célula é 0, senão h[j] + 1; rode a pilha monotônica em h. O(nm).
  - *Exemplo:* maximal rectangle em matriz binária.
  - *Armadilha:* com updates na matriz, manter por coluna é caro; reduza o problema ou use sqrt.
- **Queries de gcd em intervalo.** Sparse table (idempotente) ou segtree; a contagem de elementos iguais ao gcd é segtree de pares (gcd, contagem). Em Mo, gcd só funciona com rollback ou "duas pilhas".
  - *Gatilho:* gcd de [l, r]; contar quantos elementos são iguais ao gcd; muitas queries.
  - *Por que funciona:* gcd é idempotente, então sparse table responde em O(1); para a contagem, o nó (g, cnt) com cnt = #elementos iguais a g permite juntar: se o gcd de um filho difere do gcd do pai g, nenhum elemento desse filho vale g.
  - *Como fazer:* merge g = gcd(gL, gR); cnt = (gL == g ? cntL : 0) + (gR == g ? cntR : 0); a query junta os nós na ordem. Em Mo, como gcd não tem inverso, use rollback.
  - *Exemplo:* "quantos elementos de [l, r] são iguais ao gcd de [l, r]".
  - *Armadilha:* cnt conta iguais ao gcd do nó, não múltiplos; Mo normal não funciona sem inverso.

- **Duas pilhas (queue com agregado).** Fila com operação associativa sem inverso (gcd, min, matriz) via duas pilhas, cada uma guardando agregados; pop_front rebalanceia amortizado. Substitui deque monotônico quando a operação não é min/max. Janela deslizante com qualquer monóide.
  - *Gatilho:* janela deslizante com operação associativa sem inverso.
  - *Por que funciona:* cada pilha guarda o agregado dos seus elementos; a fila é saída + entrada, e o agregado é a composição dos topos.
  - *Como fazer:* push na entrada; pop da saída; se a saída está vazia, mova todos os elementos da entrada (amortizado); agregado = op(agg_saída, agg_entrada). O(1) amortizado.
  - *Exemplo:* janela deslizante de gcd ou de mínimo.
  - *Armadilha:* se a operação não é comutativa, o rebalanceamento inverte a ordem; a pilha tem de guardar o agregado na ordem correta.
- **Dividir blocos de k (janela deslizante de tamanho k).** Prefixos e sufixos por blocos de k dão qualquer monóide em janelas fixas em O(1) por query, sem inverso.
  - *Gatilho:* muitas janelas de tamanho fixo k e operação sem inverso.
  - *Por que funciona:* com blocos de tamanho k, uma janela de tamanho k cruza exatamente uma fronteira; é sufixo de um bloco + prefixo do próximo.
  - *Como fazer:* pré-compute sufixos dentro de cada bloco e prefixos do início de cada bloco; janela [i, i+k−1] = op(suf[i], pref[i+k−1]). O(n) de memória, O(1) por janela.
  - *Exemplo:* máximo de cada janela de tamanho k sem deque.
  - *Armadilha:* só vale para k fixo igual ao bloco; para k variável use duas pilhas.
- **Processo ao contrário.** Se a sequência de operações destrói (remove, une, quebra) e a pergunta é sobre o estado final ou intermediário, rode do fim para o início: remove → add; cortar aresta → unir; "máximo de componente após cada remoção" → DSU reverso.
  - *Gatilho:* operações que só destroem ou desfazem, e a pergunta é o estado após cada uma.
  - *Por que funciona:* a sequência inversa transforma remoção em adição; o que era difícil vira DSU/add.
  - *Como fazer:* leia todas as operações e processe do fim para o início aplicando o inverso; guarde as respostas e inverta a ordem no fim.
  - *Exemplo:* "máximo de componente após cada remoção de aresta" → DSU reverso.
  - *Armadilha:* operações sem inverso claro (atribuição que sobrescreve) exigem guardar o valor antigo.
- **DSU com rollback.** Union por tamanho sem compressão; guarde pilha de (raiz filho, tamanho antigo); desfaça em O(1). Base de segtree-no-tempo e Mo com rollback; custo O(log n) por find.
  - *Gatilho:* uniões que precisam ser desfeitas (segtree no tempo, Mo com rollback).
  - *Por que funciona:* sem compressão, a floresta tem altura O(log n) e cada união altera só a raiz e um tamanho; a pilha desfaz exatamente isso.
  - *Como fazer:* union por tamanho; empilhe (filho, raiz, tamanho antigo); rollback desempilha; find O(log n).
  - *Exemplo:* conectividade em intervalos de tempo; arestas ativas em $[t_1, t_2)$.
  - *Armadilha:* compressão de caminho impede o rollback; a pilha tem de guardar todas as alterações.
- **DSU com paridade / potencial.** Cada nó guarda a diferença (xor, soma, mod k) até o pai; find acumula. Detecta inconsistência de equações x_a − x_b = w e bipartição online.
  - *Gatilho:* equações entre variáveis, bipartição ou somas por componente com uniões.
  - *Por que funciona:* a diferença até o pai é aditiva; find acumula até a raiz e a diferença entre dois nós é a diferença dos potenciais.
  - *Como fazer:* pot[x] = valor de x − valor do pai; find comprime acumulando; union verifica consistência e ajusta a raiz. Quase O(α).
  - *Exemplo:* verificar se um grafo é bipartido online; sistema de equações x_a − x_b = w.
  - *Armadilha:* o sinal da diferença ao ligar as raízes; para "mod k" use aritmética modular.
- **DSU "próximo livre".** pai[i] = próximo índice não processado ≥ i; resolve "pintar intervalo, cada posição só pintada uma vez" e "atribuir valores distintos" em quase O(n α).
  - *Gatilho:* cada posição é usada no máximo uma vez e é preciso achar a próxima livre.
  - *Por que funciona:* pai[i] aponta para o próximo candidato; ao usar i, ligue i a i+1 e find salta todos os usados.
  - *Como fazer:* pai[i] = i; ao usar i, pai[i] = find(i+1); sentinela n+1. Total quase O(n α).
  - *Exemplo:* pintar intervalos sem repetir; atribuir valores distintos.
  - *Armadilha:* não serve se posições podem ser liberadas; precisa da sentinela.

- **DSU com merge de listas / small-to-large.** Mantenha em cada raiz um set/map e una pequeno no grande, O(n $\log^2$ n); para consultas de componente mais frequente ou menor elemento.
  - *Gatilho:* consultas por componente (mais frequente, menor elemento, k-ésimo) com uniões.
  - *Por que funciona:* cada elemento muda de conjunto O(log n) vezes; inserir no maior mantém o custo baixo.
  - *Como fazer:* cada raiz tem set/map; na união, itere o menor e insira no maior; O(n $\log^2$ n) com set, O(n log n) esperado com hash.
  - *Exemplo:* cor mais frequente em cada componente após uniões.
  - *Armadilha:* não mova o maior; com rollback, small-to-large não se desfaz facilmente.
- **DSU offline dinâmico (segtree no tempo).** Cada aresta vale em [t_1, t_2): insira na segtree sobre o tempo em O(log) nós; DFS aplicando uniões com rollback; responde conectividade e bipartição. MST com remoções (mesmo max-spanning com pesos) não sai do rollback — use LCT/dynamic MST.
  - *Gatilho:* arestas com intervalo de atividade e perguntas de conectividade/bipartição em cada tempo.
  - *Por que funciona:* cada aresta é inserida nos O(log T) nós que cobrem [t_1, t_2); a DFS aplica as uniões e desfaz na volta.
  - *Como fazer:* segtree sobre o tempo; insira a aresta em [t_1, t_2) nos nós; DFS com DSU rollback; responda nas folhas. O((m log T) log n).
  - *Exemplo:* conectividade dinâmica offline.
  - *Armadilha:* MST com remoções e pesos não sai do rollback; use LCT/dynamic MST. O intervalo é meio-aberto.
- **Treap/splay implícito.** Chave = posição (tamanho do subárvore); split/merge permitem inserir, remover, mover bloco, cortar-colar de substring e reverter intervalo com flag lazy de "reverse".
  - *Gatilho:* sequência com inserção/remoção no meio, mover bloco ou inverter intervalo.
  - *Por que funciona:* a chave é a posição implícita (tamanho da subárvore); split/merge localizam qualquer posição em O(log n).
  - *Como fazer:* split(root, k) devolve dois treaps; merge junta; reverse com flag lazy; mover bloco = split em 3 e remerge. O(log n) esperado.
  - *Exemplo:* editor de texto com cortar/colar; reverter substring.
  - *Armadilha:* atualize tamanhos e agregados na volta; prioridades aleatórias.
- **Treap com lazy de adição/atribuição e agregado.** Qualquer segtree com inserção/remoção no meio → treap implícito; custo ~5× mais lento, use só quando a operação precisa de estrutura mutável.
  - *Gatilho:* precisa de segtree com inserção/remoção no meio, ou split/merge.
  - *Por que funciona:* o treap implícito é uma segtree balanceada por prioridade; lazy e agregado funcionam igual.
  - *Como fazer:* nó guarda (tamanho, agregado, tag); push antes de descer; pull na volta. ~5× mais lento que a segtree.
  - *Exemplo:* range add com inserção no meio.
  - *Armadilha:* usar treap quando uma segtree basta custa constante; cuidado com a ordem do push.
- **Splay: amortizado e "acesso recente".** Sequência de n acessos custa O(n log n) + pior caso sobre as chaves; usado em link-cut e em problemas com operações "mover para frente".
  - *Gatilho:* acessos com localidade ("mover para frente") ou necessidade de link-cut.
  - *Por que funciona:* splay move o nó acessado para a raiz; a análise amortizada dá O(log n) por acesso.
  - *Como fazer:* rotações zig-zig/zig-zag até a raiz; operações por split/merge; base do LCT.
  - *Exemplo:* lista com "mover para frente"; LCT.
  - *Armadilha:* a garantia é amortizada, não por operação (uma operação pode custar O(n)); implementação propensa a erro.
- **Chtholly / ODT.** Mapa de intervalos com valor constante; só funciona se há atribuição aleatória em intervalo (custo esperado O((n+q) log n) sob assigns aleatórios) ou o número total de splits é amortizado O(n + q). Pior caso O(nq).
  - *Gatilho:* atribuição em intervalo com valores constantes e queries simples sobre os pedaços.
  - *Por que funciona:* a atribuição funde os intervalos tocados; sob assigns aleatórios o número de pedaços se mantém pequeno.
  - *Como fazer:* ver Set de intervalos (Chtholly / ODT) acima; o custo é esperado, não garantido.
  - *Exemplo:* "assign x em [l, r]" + "soma [l, r]".
  - *Armadilha:* adversarial derruba para O(nq); não use como estrutura geral.

- **Link-cut com informação de subárvore (virtual).** Para somas em subárvore mantenha "soma de filhos virtuais" além da soma da splay; permite update de ponto + query de componente.
  - *Gatilho:* árvore dinâmica (link/cut) e queries de subárvore/componente.
  - *Por que funciona:* a splay guarda o caminho preferido; os filhos virtuais são agregados à parte e atualizados no access.
  - *Como fazer:* guarde soma virtual e soma total; no access, ao trocar o filho preferido, mova a contribuição entre virtual e splay. Link/cut O(log n) amortizado.
  - *Exemplo:* soma de subárvore com link/cut.
  - *Armadilha:* atualizar os virtuais no access é o ponto de erro; distinguir subárvore de caminho.
- **Euler tour tree (ETT).** Floresta dinâmica com sequência de Euler guardada em treap/splay: link/cut/reroot em O(log n), query de subárvore como intervalo; alternativa ao LCT quando só precisa de subárvores.
  - *Gatilho:* floresta dinâmica e a pergunta é sobre subárvore (não caminho).
  - *Por que funciona:* a sequência de Euler em uma BST balanceada representa a DFS; a subárvore é um intervalo contíguo.
  - *Como fazer:* link/cut por split/merge da sequência; reroot gira a sequência; query de subárvore = soma no intervalo. O(log n).
  - *Exemplo:* soma de subárvore com link/cut.
  - *Armadilha:* reroot exige rotação da sequência; mais simples que LCT só para subárvores.
- **Ordered set trick (pbds / treap).** Rank e k-ésimo em O(log n) com multiconjuntos dinâmicos; para valores com duplicatas insira (valor, id). Inversões online e "mediana dinâmica" se resolvem assim.
  - *Gatilho:* multiconjunto dinâmico com rank, k-ésimo ou contagem de menores.
  - *Por que funciona:* a árvore balanceada mantém ordem e tamanho; rank e k-ésimo são O(log n).
  - *Como fazer:* pbds tree> ou treap; order_of_key e find_by_order; duplicatas com id.
  - *Exemplo:* mediana dinâmica; contar inversões inserindo da direita.
  - *Armadilha:* pbds não tem operações de intervalo; duplicatas sem id colapsam.
- **Mediana dinâmica com dois heaps.** Heap de max (metade inferior) + heap de min (metade superior), com remoção preguiçosa; contagens por lado separadas para soma de desvios absolutos.
  - *Gatilho:* mediana online com inserções/remoções; soma de desvios absolutos.
  - *Por que funciona:* a metade inferior e a superior ficam separadas; a mediana é o topo de um dos heaps.
  - *Como fazer:* max-heap inferior, min-heap superior; rebalanceie para $|dif| \leq 1$; remova com lazy; guarde as somas dos dois lados. O(log n).
  - *Exemplo:* mediana de um stream; soma $|x - mediana|$.
  - *Armadilha:* remoção preguiçosa exige limpar os topos; as somas têm de ser atualizadas na remoção.
- **Heap com remoção preguiçosa.** Dois heaps (ativos e deletados); ao topar, desempilhe enquanto os topos coincidem. Troque por set/multiset se precisar de iteração; por heap de pares (valor, versão) com array de versões se o item muda de valor.
  - *Gatilho:* fila de prioridade com remoções arbitrárias.
  - *Por que funciona:* adiar a remoção é mais barato que procurar no heap; o topo só é validado quando necessário.
  - *Como fazer:* heap de ativos + heap de deletados; ao topar, desempilhe enquanto os topos coincidem. Com iteração, use set/multiset; com valores mutáveis, (valor, versão) + array de versões.
  - *Exemplo:* Dijkstra com arestas que mudam; "remover item" em heap.
  - *Armadilha:* a memória cresce com as remoções; valide no topo dos dois heaps.
- **Heap mergeable (leftist / skew / pairing).** União de duas filas de prioridade em O(log n): permite "juntar heaps de filhos" em árvore (ex.: slope trick em árvore, "escolher k mínimos da subárvore") sem small-to-large.
  - *Gatilho:* juntar heaps de filhos numa árvore; slope trick; "k menores da subárvore".
  - *Por que funciona:* leftist/skew/pairing suportam merge sem mover elementos, em O(log n) (pairing amortizado).
  - *Como fazer:* cada nó é um heap; merge recursivo pela prioridade; no DP de árvore, funda os heaps dos filhos.
  - *Exemplo:* "somar os k menores de cada subárvore"; slope trick em árvore.
  - *Armadilha:* pairing tem garantia amortizada; skew é simples mas pode ser mais lento na prática.
- **Li Chao.** Insira reta em O(log) e segmento em $O(\log^2)$; consulte o mínimo em x em O(log); sem exigir monotonicidade das inclinações; dinâmico (domínio 10^9) funciona. Persistente em árvore ("CHT em árvore": DP no caminho raiz, rollback na volta).
  - *Gatilho:* inserir retas/segmentos e consultar min/max num x, sem monotonicidade.
  - *Por que funciona:* cada nó guarda a reta que vence no meio; a outra desce para o lado onde pode vencer.
  - *Como fazer:* insert(line) O(log C); insert de segmento $O(\log^2 C)$; query(x) percorre o caminho O(log C); domínio dinâmico com nós sob demanda.
  - *Exemplo:* CHT com queries arbitrárias; DP com transições lineares.
  - *Armadilha:* C grande exige domínio dinâmico; para max troque a comparação; segmento precisa respeitar [l, r].

- **CHT em árvore.** dp[v] = min sobre ancestrais u de dp[u] + b_u·a_v (queries no caminho) → Li Chao com rollback (guarde os nós modificados, O(log) por inserção) e DFS; ou Li Chao persistente.
  - *Gatilho:* dp[v] = min sobre ancestrais u de dp[u] + b_u·a_v.
  - *Por que funciona:* durante a DFS o conjunto de retas é exatamente o caminho atual; o rollback remove ao voltar.
  - *Como fazer:* insira a reta de v ao entrar, desfaça ao sair (guarde os nós modificados) ou use Li Chao persistente. O(n $\log^2$ C).
  - *Exemplo:* DP em árvore com transição linear no ancestral.
  - *Armadilha:* sem rollback/persistência as retas de outros ramos contaminam; a ordem de inserção importa.
- **CHT com inclinações monótonas e queries monótonas** → deque O(n); queries arbitrárias → busca binária no hull; inserções arbitrárias → Li Chao ou "CHT dinâmico" com set ordenado de retas.
  - *Gatilho:* escolher a variante de CHT conforme a monotonicidade das inclinações e das queries.
  - *Por que funciona:* com inclinações e queries monótonas, o ponteiro ótimo só anda para um lado; sem monotonicidade, é preciso busca.
  - *Como fazer:* ambos monótonos → deque O(n); queries arbitrárias → binária no hull; inserções arbitrárias → Li Chao ou set de retas.
  - *Exemplo:* DP de partição com custo linear; LIS com CHT.
  - *Armadilha:* assumir monotonicidade inexistente; remova as retas dominadas e trate empates de inclinação.
- **Logarithmic method (Bentley-Saxe).** Estrutura estática com merge O(n) + só inserções → mantenha ≤ log n estruturas de tamanhos 2^i e recombine como contador binário; query consulta todas. Torna estático insertion-only (hull, k-d tree, índice de SA) com inserção amortizada O(log n) e query O(log n · Q), onde Q é o custo da query estática (O($\log^2$) só se Q = O(log)).
  - *Gatilho:* estrutura estática com merge O(n) e apenas inserções.
  - *Por que funciona:* como no contador binário, cada elemento participa de O(log n) merges; as estruturas têm tamanhos 2^i.
  - *Como fazer:* ao inserir, crie uma estrutura de tamanho 1 e funda enquanto o bit estiver ocupado; query consulta as ≤ log n estruturas. Inserção amortizada O(log n), query O(log n · Q).
  - *Exemplo:* hull dinâmico só com inserções; índice de suffix array com append.
  - *Armadilha:* Q é o custo da query estática (se Q = O(log n), a query fica O($\log^2$ n)); remoções exigem outra técnica.
- **Remoção com Bentley-Saxe offline.** Se o tempo de vida de cada item é conhecido, segtree no tempo + estrutura com rollback/estática por nó.
  - *Gatilho:* inserções e remoções com tempo de vida conhecido.
  - *Por que funciona:* o intervalo de vida é inserido na segtree do tempo; a DFS mantém ativa a estrutura do caminho.
  - *Como fazer:* segtree sobre o tempo; insira o item nos O(log T) nós que cobrem [t_1, t_2); DFS aplicando/desfazendo; responda nas folhas.
  - *Exemplo:* hull com remoções offline; conectividade com arestas temporárias.
  - *Armadilha:* a estrutura por nó precisa suportar rollback ou ser reconstruída; memória O(n log T).
- **Contador binário e custo amortizado.** Incrementos custam O(1) amortizado (potencial = número de 1s); vale para "reconstruir quando metade foi mudado" (rebuild global em dobra de tamanho) e para analisar pilhas com undo.
  - *Gatilho:* analisar reconstruções/merges repetidos (hash table, Bentley-Saxe).
  - *Por que funciona:* um incremento custa o número de 1s que viram 0; cada bit vira 1 e 0 O(1) vezes por incremento.
  - *Como fazer:* Φ = #1s; custo real k + ΔΦ ≤ 2. Rebuild global em dobra de tamanho tem a mesma análise.
  - *Exemplo:* hash table com rebuild ao dobrar; Bentley-Saxe.
  - *Armadilha:* a garantia é amortizada; uma operação isolada pode custar O(n).
- **Potencial para amortizar.** Defina Φ = tamanho da pilha/#bits/#componentes; custo real + ΔΦ ≤ constante. "Pop while condition" é amortizado O(1) pelo potencial.
  - *Gatilho:* provar que "pop while" ou uma pilha com undo é barato no total.
  - *Por que funciona:* se cada elemento entra e sai no máximo uma vez, o trabalho total é proporcional ao número de inserções.
  - *Como fazer:* defina Φ (pilha, bits, componentes); mostre custo real + ΔΦ ≤ c; some os passos.
  - *Exemplo:* pilha monotônica O(n); DSU com rollback.
  - *Armadilha:* Φ tem de ser não negativo e limitado; não confunda com pior caso por operação.

- **Bitset como estrutura.** Matrizes booleanas, alcançabilidade em DAG com $n \leq 5 \cdot 10^4$ (n/64 palavras por vértice; $n^2/64$ no total, tempo O(mn/64)), mochila, soma de subconjuntos, filtro de vizinhos em grafos densos; use _Find_first/_Find_next para iterar bits ligados.
  - *Gatilho:* $n \leq 5 \cdot 10^4$ e operações booleanas em massa (alcançabilidade, subconjunto, vizinhança).
  - *Por que funciona:* 64 bits por palavra; união/interseção de conjuntos custa n/64 em vez de n.
  - *Como fazer:* bitset por vértice para alcançabilidade (ordem topológica: reach[v] |= reach[u]); mochila com dp |= dp << a; itere com _Find_first/_Find_next.
  - *Exemplo:* alcançabilidade em DAG; soma de subconjuntos.
  - *Armadilha:* memória $n^2/8$ bytes (≈ 312 MB para $n = 5 \cdot 10^4$); _Find_next custa O(n/64) por chamada.
- **Bitset + transições do tipo shift-or.** "Existe subconjunto/par com soma/diferença x" → dp |= dp << a; "x + a_i para todo i" → bitset de valores deslocado.
  - *Gatilho:* DP de somas pequenas ou existência de combinação por soma/diferença.
  - *Por que funciona:* deslocar o bitset equivale a somar a ao conjunto de somas; OR acumula as possibilidades.
  - *Como fazer:* dp |= dp << a para cada elemento (mochila booleana); para "x + a_i", mantenha os bitsets deslocados. O(n·S/64).
  - *Exemplo:* soma de subconjunto até $S \leq 10^5$.
  - *Armadilha:* shifts maiores que a largura; a mochila booleana guarda existência, não contagem.
- **K-d tree.** Contagem em retângulo em O(√n) em 2D; vizinho mais próximo é rápido na média, mas O(n) no pior caso; reconstrução parcial (scapegoat) para inserções. Escolha mediana alternando eixos; poda por distância à caixa.
  - *Gatilho:* pontos 2D/3D com queries de retângulo ou vizinho mais próximo.
  - *Por que funciona:* cada nó divide o espaço por um eixo; a poda pela caixa elimina subárvores inteiras.
  - *Como fazer:* mediana alternando eixos; range count O(√n + k); nearest com poda por distância à caixa; inserções com rebuild parcial.
  - *Exemplo:* contar pontos em retângulo; nearest neighbor.
  - *Armadilha:* pior caso O(n) para nearest e para count com pontos degenerados; não é para updates frequentes.
- **Quadtree/octree.** Para pontos muito agrupados com queries aproximadas ou colisão; no pior caso (pontos colineares) degenera; a k-d tree dá garantia.
  - *Gatilho:* pontos agrupados, colisão ou queries aproximadas.
  - *Por que funciona:* subdivide o espaço em quadrantes; pontos densos ficam juntos.
  - *Como fazer:* subdivida até uma capacidade; para colisão, compare só vizinhos da célula.
  - *Exemplo:* detecção de colisão em jogos; agrupamento.
  - *Armadilha:* pontos colineares degeneram; a k-d tree dá garantia melhor.
- **Fractional cascading.** Busca do mesmo valor em k listas ordenadas em O(log n + k) em vez de k·log n; usada para tirar um log em merge sort tree e interseção de camadas.
  - *Gatilho:* buscar o mesmo valor em k listas ordenadas (merge sort tree, camadas).
  - *Por que funciona:* cada lista guarda ponteiros para a posição correspondente na lista seguinte; a busca em uma localiza nas outras.
  - *Como fazer:* O(log n + k) com os ponteiros; no merge sort tree, tira um log da query.
  - *Exemplo:* contar < x em [l, r] em O(log n).
  - *Armadilha:* memória dos ponteiros; atualizações são caras (estrutura estática).
- **Timestamp trick (visitado sem limpar).** Em vez de zerar array por query, guarde a "época" da última escrita; reset vira incremento. Útil em BFS/DFS multi-teste e Mo.
  - *Gatilho:* muitos testes/BFS e arrays que precisam ser "zerados" por query.
  - *Por que funciona:* guardar a época da última escrita evita O(n) de limpeza; o reset é O(1).
  - *Como fazer:* vis[i] = época; teste vis[i] == cur; ao mudar de teste, cur++. Em Mo, o mesmo para marcar presença.
  - *Exemplo:* BFS multi-teste com n grande.
  - *Armadilha:* estouro de época; arrays de tamanho n por teste.
- **Versionamento / "timestamp" de atualização.** Para update "atribuir em tudo" (reset global) guarde (versão global, valor) por elemento e compare; evita O(n).
  - *Gatilho:* "atribua x a todos" (reset global) intercalado com leituras.
  - *Por que funciona:* em vez de escrever n posições, muda-se a versão global; cada elemento guarda (versão, valor).
  - *Como fazer:* glob_ver++; ao ler i, se ver[i] != glob_ver o valor é o default; ao escrever, ver[i] = glob_ver, val[i] = x.
  - *Exemplo:* reset global + point updates.
  - *Armadilha:* o default pode mudar; operações de soma em todo o array exigem mais que isso.

- **Parallel binary search (binária paralela).** Para q queries "primeiro tempo em que P(x) vale" com estrutura que só avança no tempo: todas as queries no mesmo nível mid são respondidas numa única passada de simulação, O(log n) passadas.
  - *Gatilho:* q queries do tipo "primeiro tempo em que P(x) vale", com uma estrutura que avança no tempo.
  - *Por que funciona:* todas as queries com o mesmo mid são respondidas na mesma simulação; O(log T) níveis cobrem todas.
  - *Como fazer:* mantenha lo/hi por query; agrupe por mid; simule os eventos até o mid uma vez por nível e atualize os limites. O((n + q) log T · custo).
  - *Exemplo:* "quando a aresta entra no MST"; "primeiro momento em que a soma ≥ k".
  - *Armadilha:* é offline; a estrutura tem de poder ser reiniciada ou avançar sem voltar.
- **Tirar log por offline.** Se O(log) vem de busca em estrutura dinâmica e as queries são ordenáveis, ordene tudo e use ponteiros/DSU/vetor (ex.: pares com soma ≤ x ordenados → two pointers em vez de BIT).
  - *Gatilho:* o log vem de uma estrutura dinâmica, mas as queries podem ser ordenadas.
  - *Por que funciona:* ordenar queries e elementos permite varrer com ponteiros/DSU/vetor, sem busca dinâmica.
  - *Como fazer:* ordene; use dois ponteiros, DSU ou prefixos; a estrutura dinâmica vira varredura. O(n log n) só na ordenação.
  - *Exemplo:* pares com soma ≤ x → ordenar + dois ponteiros.
  - *Armadilha:* a ordenação muda a ordem original; guarde os índices e confira a monotonicidade exigida.
- **Cache e constante.** Layout em SoA, loop interno em stride 1, blocos de 2^k na transposta de matriz, segtree iterativa com folhas contíguas, Eytzinger para busca binária; reduz 2–5× sem mudar a complexidade.
  - *Gatilho:* TLE com complexidade correta e n grande.
  - *Por que funciona:* menos cache misses e branches; o custo por operação cai sem mudar a assintótica.
  - *Como fazer:* SoA, loop interno em stride 1, blocos de 2^k na transposta, segtree iterativa com folhas contíguas, Eytzinger na binária.
  - *Exemplo:* transpor matriz $10^3 \times 10^3$ em blocos.
  - *Armadilha:* meça antes de otimizar; Eytzinger complica o código e o ganho depende do hardware.
- **Árvore de segmentos para "soma de min de subarrays com update".** Combine pilha monotônica offline (contribuição de cada a_i) com segtree por tempo; ou use sweep em r mantendo segtree de "soma de min(l..r)" com lazy de atribuição em intervalo (históricos).
  - *Gatilho:* soma dos mínimos de todos os subarrays, com queries de intervalo ou updates.
  - *Por que funciona:* ao varrer r, m[l] = min(a[l..r]) só muda quando entra um a_r menor; um range assign no intervalo (prevMenor(r), r] mantém m.
  - *Como fazer:* segtree sobre l com m[l] = min(a[l..r]); ao processar r, faça assign de a_r em (prevMenor(r), r]; a query soma m[l] para l num intervalo. Para todos os subarrays contidos em [L, R], use soma histórica (bullet seguinte).
  - *Exemplo:* soma dos mínimos dos subarrays que terminam em r.
  - *Armadilha:* a query de "todos os subarrays de [L, R]" não é a soma direta; updates pontuais quebram o sweep (reconstrua por blocos).
- **Segtree de soma histórica.** Para "soma dos valores de a_i em todos os tempos passados" guarde (valor atual, soma histórica, lazy de 'a' e de 'b') e componha tags 2D; clássico "historic sum" / sweep de r para "soma de max/min de todos os subarrays".
  - *Gatilho:* a resposta acumula valores ao longo do tempo ("soma dos valores em todos os tempos passados").
  - *Por que funciona:* o histórico é uma segunda dimensão; as tags sobre (valor, histórico) compõem como transformações afins.
  - *Como fazer:* nó (val, hist, tag_a, tag_b); ao aplicar uma tag, atualize val e hist na ordem correta; componha as tags como matrizes afins.
  - *Exemplo:* sweep de r para "soma de max de todos os subarrays".
  - *Armadilha:* a composição das tags 2D é o ponto de erro; use long long para os acumulados.
- **Sweep com segtree: máximo de cobertura.** Evento +1/−1 em intervalo, query = max global; "ponto coberto por mais retângulos", "máximo de sobreposição ponderada"; coordenada comprimida, lazy sem push-down.
  - *Gatilho:* máximo de intervalos sobrepostos ou ponto coberto por mais retângulos.
  - *Por que funciona:* a cobertura é a soma dos eventos ativos; o máximo global é o max da segtree.
  - *Como fazer:* comprima coordenadas; evento +1 em l, −1 em r+1; segtree com tag de cobertura e max; a query é o max da raiz. Lazy permanente (sem push-down).
  - *Exemplo:* ponto coberto por mais retângulos; união de áreas.
  - *Armadilha:* coordenadas contínuas exigem compressão e diferenças; eventos no mesmo ponto têm de ser somados antes de consultar.

- **Contagem de pares (i, j) com f(i) ≤ g(j) em estrutura de dois pontos.** Ordene f e g juntos num único conjunto, tipo de evento; BIT sobre o outro tipo (merge-sort-like). Evita D&C.
  - *Gatilho:* contar pares (i, j) com f(i) ≤ g(j), possivelmente com restrição de posição.
  - *Por que funciona:* ordenar os dois tipos juntos transforma a comparação em "quantos já foram inseridos"; um BIT resolve o resto.
  - *Como fazer:* eventos ordenados por valor; tipo inserção e tipo consulta; BIT sobre o índice/posição; desempate f antes de g para ≤ (depois para <).
  - *Exemplo:* pares com a_i + a_j ≤ x e i < j.
  - *Armadilha:* desempate; contar cada par uma vez (i < j).
- **Árvore de Fenwick para máximos de prefixo em "DP LIS com peso".** dp[v] = max dp[u] com u < v e a_u < a_v → BIT de max por valor comprimido; reconstrução guardando o índice.
  - *Gatilho:* DP com "melhor dp[u] entre valores menores", em ordem de posição.
  - *Por que funciona:* indexar pelo valor comprimido transforma a_u < a_v em prefixo; o BIT guarda o máximo.
  - *Como fazer:* processe v em ordem de posição; query max em [1, rank(a_v) − 1]; update em rank com dp; guarde o antecessor para reconstruir. O(n log n).
  - *Exemplo:* LIS; LIS com pesos.
  - *Armadilha:* BIT de max só com updates crescentes (aqui ok, pois dp só é inserido); empates usam < estrito.
- **Binary lifting guardando agregado e "salto com condição".** Para "primeiro ancestral com valor ≥ x" use binary lifting com max por salto; ou sparse por saltos; ou HLD + descida na segtree.
  - *Gatilho:* primeiro ancestral que satisfaz uma condição, ou "saltar enquanto a condição vale".
  - *Por que funciona:* up[k][v] e o max do salto permitem pular 2^k quando o agregado do salto não satisfaz a condição.
  - *Como fazer:* pré-processe up e mx; para o primeiro ≥ x, suba do maior k para o menor aceitando saltos com max < x; a resposta é o pai do nó final. O(log n).
  - *Exemplo:* primeiro ancestral com peso ≥ x; LCA.
  - *Armadilha:* a condição precisa ser resumível pelo agregado; saltos que passam do ponto não são aceitos.
- **Chunking temporal de queries (offline por blocos).** Quando as queries e updates são em √q lotes, as mudanças do lote são poucas → comprima a árvore/array nesses ≤ √q pontos relevantes e responda cada query em O(√q).
  - *Gatilho:* queries e updates em lotes, com poucos pontos alterados por lote.
  - *Por que funciona:* só ≤ B pontos mudam no lote; comprimir a árvore/array neles deixa cada query O(B).
  - *Como fazer:* processe em blocos de B = √q; colete os pontos tocados; construa a estrutura comprimida; responda as queries do lote ajustando pelos pontos. O((n + q)√q).
  - *Exemplo:* queries de conectividade com poucos vértices alterados por bloco.
  - *Armadilha:* reconstruir a estrutura por bloco custa O(n); escolha B com cuidado.
- **Persistência fat-node / path copy e rollback de memória.** Persistir uma estrutura é equivalente a árvore de versões; para "voltar a versão k" use DFS na árvore de versões com rollback quando os ops são desfazíveis (mais barato que persistência).
  - *Gatilho:* precisa voltar a versões antigas, não só consultá-las.
  - *Por que funciona:* versões formam uma árvore; se as operações são desfazíveis, uma DFS com rollback responde cada versão sem copiar nós.
  - *Como fazer:* construa a árvore de versões; DFS aplicando a operação da aresta e desfazendo na volta; responda nas versões. Alternativa: path copy/fat node.
  - *Exemplo:* "qual o estado na versão k" com updates desfazíveis.
  - *Armadilha:* operações não desfazíveis exigem persistência real; a DFS não responde queries online fora da ordem.
- **DSU com rollback em segtree + "Mo em tempo"**: para queries "conectividade no intervalo de arestas [l, r]" → ordene por bloco, rollback; ou offline: MST incremental com LCT.
  - *Gatilho:* conectividade no intervalo de arestas [l, r] para muitas queries.
  - *Por que funciona:* ordenar por bloco de l deixa poucas arestas fora do miolo; o rollback desfaz o lado esquerdo.
  - *Como fazer:* ordene por bloco de l e r crescente; adicione as arestas do miolo; para cada query, adicione as pontas, responda e desfaça. O((m + q)√m log n). Alternativa: segtree no tempo + rollback.
  - *Exemplo:* conectividade em subarray de arestas.
  - *Armadilha:* MST com pesos não funciona com rollback; use LCT.

- **Queries de "intervalo cujos elementos distintos estão todos em [lo, hi]".** Transforme em pontos (l, r) e use sweep: para cada elemento calcule prev e next; intervalo [l, r] contém todas as ocorrências de v ⇔ prev(l) < l e next(r) > r etc. Dualidade posição-intervalo.
  - *Gatilho:* perguntas sobre intervalos e o conjunto de valores distintos que eles contêm, com limites [lo, hi].
  - *Por que funciona:* cada intervalo [l, r] vira um ponto; condições do tipo "todas as ocorrências de v estão dentro" viram desigualdades com prev/next, isto é, dominância 2D.
  - *Como fazer:* para cada posição i, prev[i] e next[i] da mesma cor; expresse as condições em (l, r) e responda com sweep em uma coordenada + BIT na outra (dualidade posição-intervalo).
  - *Exemplo:* contar intervalos em que cada valor presente aparece por inteiro.
  - *Armadilha:* as bordas (primeira/última ocorrência) precisam de sentinelas; fixe a formulação antes de montar as desigualdades.

Repetidas de outras seções:

- **Queries com strings de Σ|s| ≤ S** → ≤ √(2S) tamanhos distintos; hash set por tamanho (strings).
- **Comparar substrings em query** → hash de prefixo, ou segtree de hash se houver updates (strings).
- **Elemento majoritário em [l, r]** → sorteie posições e conte com vetor de posições + binária (aleatorização).
- **Descida em BIT/segtree** substitui binária + query e tira um log (primeiro prefixo com soma ≥ x, k-ésimo livre).
- **Pilha monotônica + estrutura**: processe por r e mantenha a pilha de candidatos (ex.: "subarrays em que i é máximo" atualiza um segtree).
- **Base linear com prefixo** (mantendo o vetor mais recente por pivô) → max xor de subconjunto em [l, r] offline por r ou online.
- **Trie xor persistente** → max x⊕a_i em [l, r] por diferença de cnt entre versões.
- **Ocorrências de padrão em prefixo por AC + BIT na link tree** → +1 no nó visitado, query soma de subárvore.
- **Mo em árvore (Euler de 2n)**: v aparece duas vezes; alterne add/remove a cada ocorrência (o vértice está ativo quando o número de ocorrências no segmento é ímpar); query (u, v) com tin[u] < tin[v] → [tin[u], tin[v]] se u é ancestral de v (sem lca extra), senão [tout[u], tin[v]] mais o lca explicitamente.
- **Union-find com rollback e Mo** → Mo com rollback (ordena por bloco de l, r crescente, desfaz o lado esquerdo) quando a operação só tem "adicionar".
- **Árvore de Fenwick em Euler tour** para update de subárvore + query de ponto/subárvore com dois BITs (range-update/range-query).
- **Hash de multiconjunto em intervalo** → soma de valores aleatórios por prefixo: [l, r] é permutação de 1..k ⇔ hash(r) − hash(l−1) = hash(1..k) (aleatorização).
- **Distintos em intervalo / mode / majoritário** → majoritário por sorteio de posições + vetor de ocorrências e binária (aleatorização).

Construtivos e interativos

Construtivos:

- **Ache a condição de impossível primeiro** (paridade, soma, contagem); a construção costuma provar que é a única.
  - *Gatilho:* o enunciado pede "construa ou diga que é impossível"; aparecem restrições de paridade, soma total, coloração ou contagem.
  - *Por que funciona:* um invariante preservado por toda operação é obstrução necessária. Se ele falha, não há solução; se vale, a construção indutiva mostra que essa é a única obstrução.
  - *Como fazer:* derive o invariante (Σ mod 2, cor de xadrez, Σ fixa, contagem de inversões); teste em n pequeno; construa caso base + passo; se a construção não fechar, procure outro invariante.
  - *Exemplo:* tabuleiro de xadrez com número ímpar de casas não tem tour fechado de cavalo; dominó não cobre 8×8 com dois cantos opostos removidos (mesma cor); soma dos graus ímpar impede grafo.
  - *Armadilha:* condição necessária não é suficiente; não declare impossível sem provar a volta (construção ou teorema).
- **Resolva n pequeno à mão**, ache o padrão, e faça indução: de n para n + 2 ou n + 4.
  - *Gatilho:* "para todo n", sem fórmula aparente, e a resposta parece depender da paridade de n.
  - *Por que funciona:* a resposta costuma ser periódica ou indutiva; n pequeno revela a base e o passo, e o passo de 2 em 2 (ou 4 em 4) preserva a paridade.
  - *Como fazer:* brute force n = 1..8 (às vezes até 12), imprima as respostas; compare n com n + 1, n + 2; formule a indução e verifique separadamente base e passo.
  - *Exemplo:* preencher 3×n com dominós; construir sequências balanceadas; grafos com exatamente k arestas.
  - *Armadilha:* o padrão pode só valer a partir de n = 5; confira a base e o passo, não só a tabela.

- **Respostas típicas:** 1, 2, 3, n, ordenado, ordenado ao contrário, alternado, $i \mapsto n + 1 - i$, potências de 2.
  - *Gatilho:* tempo curto, checker, ou uma construção simples que precisa apenas ser válida.
  - *Por que funciona:* muitas condições são simétricas ou monotônicas e aceitam uma sequência canônica; testar candidatos custa pouco.
  - *Como fazer:* teste na ordem 1, 2, 3, n, crescente, decrescente, alternado, $i \mapsto n + 1 - i$, potências de 2; valide no checker ou no brute force.
  - *Exemplo:* "1 2 … n" para somas; "n … 1" para ordem; "1 2 1 2…" para alternância; "1 2 4 …" para potências.
  - *Armadilha:* 0-based vs 1-based; n = 1; a resposta pode exigir tamanho exato, não só os valores.
- **Operação reversível** → leve as duas configurações para uma forma canônica.
  - *Gatilho:* duas configurações e operações que podem ser desfeitas; "é possível transformar A em B?".
  - *Por que funciona:* operações reversíveis geram uma relação de equivalência; um invariante completo (a forma canônica) decide se A e B são equivalentes.
  - *Como fazer:* escolha uma forma canônica (ordenar, reduzir, normalizar); leve A e B a ela; compare. Se o invariante não for completo, refine a forma canônica.
  - *Exemplo:* ordenar por swaps; reduzir string por cancelamentos; contagem de inversões mod 2.
  - *Armadilha:* invariante incompleto (necessário mas não suficiente); prove que a forma canônica é alcançável de qualquer estado.
- **"No máximo n operações"** → uma operação fixa um elemento de cada vez; "no máximo 2n" → duas por elemento.
  - *Gatilho:* limite de operações n, 2n ou n + 1; construção com orçamento apertado.
  - *Por que funciona:* cada operação deve fixar ou consumir ao menos um elemento; n operações ⇒ uma por elemento; 2n ⇒ duas por elemento.
  - *Como fazer:* projete uma operação que fixe a posição i sem tocar nas anteriores; conte o pior caso; se precisar de duas por elemento, use duas fases.
  - *Exemplo:* selection sort com ≤ n − 1 swaps; reversões de prefixo com ≤ 2n; construir permutação em 2n movimentos.
  - *Armadilha:* a operação pode desfazer trabalho anterior; o limite é apertado (2n + 5), conte inclusive casos de borda.
- **Grid** → xadrez (paridade de i + j), cobrir linha por linha, cobra (zigue-zague).
  - *Gatilho:* tabuleiro, movimentos ortogonais, cobrir/colorir casas, caminho hamiltoniano.
  - *Por que funciona:* a coloração xadrez separa paridade (cada movimento ortogonal troca de cor); o zigue-zague cobre linhas sem deixar buracos.
  - *Como fazer:* numere (i + j) mod 2 para impossibilidade/pareamento; para construir, percorra linhas alternando o sentido (cobra) ou colunas; trate a dimensão ímpar separadamente.
  - *Exemplo:* dominós em tabuleiro com buraco; caminho hamiltoniano em grade com n·m par; preencher com padrão xadrez.
  - *Armadilha:* grade ímpar×ímpar quebra a simetria; o centro precisa de tratamento separado.

Interativos:

- **Limite de queries ≈ log n** → binária; ≈ 2 log n → binária com duas perguntas por passo; ≈ n/2 → pares.
  - *Gatilho:* "no máximo 60 perguntas" com N até 10^{18}; "≈ 2 log n"; "n/2".
  - *Por que funciona:* uma pergunta binária tem 2 respostas e reduz a incerteza à metade; o limite inferior é $\lceil \log_2 N \rceil$. Com duas perguntas por passo, o limite dobra.
  - *Como fazer:* binária clássica; para 2 log n, faça duas comparações por passo (ex.: a[m] e a[m+1]) ou duas perguntas independentes; para n/2, pergunte em pares.
  - *Exemplo:* adivinhar $1..10^{18}$ em 60; achar pico com ~2 log n; verificar pares de posições.
  - *Armadilha:* contar queries no pior caso, não na média; off-by-one no intervalo; repetir pergunta desperdiça orçamento.
- **Adaptativo?** Pense no adversário: cada resposta deve dividir as possibilidades ao meio.
  - *Gatilho:* interator adversarial; a resposta depende das perguntas anteriores.
  - *Por que funciona:* o adversário mantém o maior conjunto consistente; a melhor pergunta equilibra os ramos da árvore de decisão.
  - *Como fazer:* mantenha o conjunto de estados possíveis; escolha a pergunta que divide o pior caso o mais perto possível do meio; se o protocolo não for adaptativo, planeje a árvore inteira antes.
  - *Exemplo:* pesar moedas; adivinhar com respostas sim/não; achar o item defeituoso.
  - *Armadilha:* pergunta desequilibrada; respostas impossíveis devem ser descartadas; o adversário pode escolher empates.

- **Teste com um interator local** antes de submeter.
  - *Gatilho:* problema interativo; antes de submeter.
  - *Por que funciona:* reproduz o protocolo e expõe flush, leitura e contagem de queries errados.
  - *Como fazer:* escreva um juiz que gera o caso, responde e valida; rode contra brute force; teste limites e resposta -1.
  - *Exemplo:* script que simula o número secreto e conta as perguntas.
  - *Armadilha:* esquecer flush, ler depois do EOF, ou não abortar em resposta inválida.

---

adicionado em 2026-11-10 por agentes; revisado

- **Indução removendo extremos.** Remova dois elementos/vértices que se cancelam (maior e menor, folha e pai), resolva n − 2 e cole dos lados; checa-se bem com n = 1..8.
  - *Gatilho:* estrutura recursiva; existe um par de elementos que "se cancela" e a instância menor é do mesmo tipo.
  - *Por que funciona:* a remoção preserva a condição; se a instância menor tem solução, reinserir o par mantém a validade; a base cobre n = 1..2.
  - *Como fazer:* identifique o par removível (maior/menor, folha/pai); resolva n − 2; reinsira o par; verifique n = 1..8.
  - *Exemplo:* construir sequência balanceada removendo o maior e o menor; árvore removendo folha e pai.
  - *Armadilha:* o par pode alterar somas ou graus globais; escolha um par que preserve o invariante.
- **Pareamento de soma constante** (i, n + 1 − i) dá soma n + 1; use para permutações com |p_i − p_j| fixo, magia e "dividir em dois grupos de soma igual".
  - *Gatilho:* permutações com diferenças fixas, somas iguais, quadrado mágico, "dois grupos de mesma soma".
  - *Por que funciona:* (i, n + 1 − i) tem soma n + 1 e diferença controlada; a simetria permite distribuir um elemento de cada par por grupo.
  - *Como fazer:* emparelhe i com n + 1 − i; para dois grupos de soma igual, coloque um de cada par em cada grupo; se n for ímpar, trate o centro separadamente.
  - *Exemplo:* n = 6, pares (1,6), (2,5), (3,4), todos somam 7; distribua um de cada par por grupo.
  - *Armadilha:* n ímpar deixa o centro sem par; o pareamento não cobre todos os índices.
- **Sentinela/sacrifício.** Reserve um elemento para absorver o resíduo (ex.: último elemento = s − Σ resto, ou um vértice universal para consertar graus).
  - *Gatilho:* resíduo que não fecha; graus ímpares; soma fixa; "a última linha/coluna".
  - *Por que funciona:* um elemento absorve a inconsistência e o resto vira construção fácil; remova a sentinela no final.
  - *Como fazer:* reserve o último = s − Σ resto; ou um vértice fantasma ligado aos ímpares; construa o resto e ajuste.
  - *Exemplo:* array com soma s; grafo euleriano com vértice fantasma; matriz com última linha determinada.
  - *Armadilha:* a sentinela pode sair do intervalo permitido (negativo, grau inválido); valide antes de imprimir.
- **Construir de dentro para fora** (começa pelo centro/maior/único elemento forçado) funciona quando a condição é local em cada inserção.
  - *Gatilho:* condição local em cada inserção; existe um centro, um maior ou um único elemento forçado.
  - *Por que funciona:* inserir em ordem garante que a condição local se mantém; o núcleo é a base da indução.
  - *Como fazer:* identifique o núcleo (centro, diâmetro, maior); insira elementos um a um checando só a vizinhança; escolha a ordem (geralmente decrescente de grau).
  - *Exemplo:* árvore a partir do diâmetro; permutação a partir do meio; sequência balanceada a partir do centro.
  - *Armadilha:* inserção tardia pode violar grau ou limite; escolha a ordem e o núcleo corretos.
- **Dividir [1..n] em dois conjuntos de mesma soma.** Precisa de n(n+1)/2 par; guloso do maior para o menor pegando se couber. Qualquer soma entre 0 e n(n+1)/2 é alcançável com subconjunto de {1..n}, pelo mesmo guloso.
  - *Gatilho:* partição, total par, "dois grupos de soma igual".
  - *Por que funciona:* a paridade do total é necessária; {1..n} é uma sequência completa, então todo valor de 0 a n(n+1)/2 é atingível; o guloso do maior para o menor constrói.
  - *Como fazer:* se o total for ímpar, impossível; alvo = total/2; percorra n..1 e pegue se couber; O(n).
  - *Exemplo:* n = 7, total 28, alvo 14 → {7,6,1} e {5,4,3,2}.
  - *Armadilha:* o guloso vale para {1..n}, não para um conjunto arbitrário (partição é NP-difícil).

- **Prouhet–Thue–Morse.** Separar {0..2^{k+1} − 1} pela paridade do popcount dá dois conjuntos com mesma soma de potências até k (soma, soma de quadrados, ...).
  - *Gatilho:* duas partes com as mesmas somas de potências; partição equilibrada de {0..2^m − 1}.
  - *Por que funciona:* identidade de Prouhet; a paridade do popcount iguala os momentos até m − 1.
  - *Como fazer:* para i em [0, 2^{k+1}), coloque i no grupo popcount(i) mod 2; as somas de i^j são iguais para j ≤ k; O(2^k·k).
  - *Exemplo:* {0..7} → {0,3,5,6} e {1,2,4,7}: somas 14 e soma de quadrados 70.
  - *Armadilha:* com 2^{k+1} elementos as potências vão até k, não até k + 1.
- **Base b / sistemas numéricos.** Todo n é soma de potências distintas de 2 (poucas moedas); ternário balanceado (dígitos −1,0,1) pesa qualquer valor com pesos 3^i nos dois pratos; Zeckendorf (Fibonacci sem consecutivos) representa qualquer inteiro; fatorial base dá a ordem de permutação (Lehmer).
  - *Gatilho:* representar qualquer inteiro com poucos pesos; moedas; ordem de permutação; "pesar nos dois pratos".
  - *Por que funciona:* cada sistema tem um conjunto de dígitos que cobre todos os inteiros, com representação única ou gananciosa.
  - *Como fazer:* binário: potências de 2, dígitos 0/1; ternário balanceado: dígitos −1,0,1, pesos 3^i; Zeckendorf: Fibonacci não consecutivos, guloso do maior; fatorial: Lehmer.
  - *Exemplo:* 5 = 4 + 1; 5 = 9 − 3 − 1; 5 = 5; permutação [2,1,0] → Lehmer.
  - *Armadilha:* o número de termos; no ternário balanceado o dígito −1 é uma peça no outro prato; Zeckendorf não usa Fibonacci consecutivos.
- **Gray code.** g(i) = i ^ (i >> 1) passa por todas as máscaras mudando 1 bit por vez; usa-se para caminhar por subconjuntos, resolver Torre de Hanói/anéis chineses e passear no hipercubo (Hamiltoniano).
  - *Gatilho:* percorrer todas as máscaras mudando 1 bit; subconjuntos; Hamiltoniano no hipercubo.
  - *Por que funciona:* g(i) = i ^ (i >> 1) é uma bijeção e consecutivos diferem em exatamente 1 bit; o código também é cíclico.
  - *Como fazer:* gere g(i) para i = 0..2^n − 1; para percorrer subconjuntos, adicione/remova o bit que mudou; para Hanói, use a posição do bit.
  - *Exemplo:* n = 3 → 000, 001, 011, 010, 110, 111, 101, 100.
  - *Armadilha:* a inversa não é g(x); para voltar de x para i, inverta o prefixo de xor.
- **de Bruijn.** Sequência cíclica de tamanho k^n com todas as strings de tamanho n uma vez = ciclo Euleriano no grafo de k^(n−1) vértices (aresta = dígito); também sai da concatenação de palavras de Lyndon (FKM). Cofres/PINs e "menor string contendo todas as máscaras".
  - *Gatilho:* menor string cíclica contendo todas as strings de tamanho n; cofre/PIN; "contém todas as máscaras".
  - *Por que funciona:* é um circuito Euleriano no grafo de Bruijn de k^{n−1} vértices; cada aresta é um dígito e cada caminho de n arestas é uma string.
  - *Como fazer:* Hierholzer no grafo de Bruijn; colete os dígitos das arestas; O(k^n).
  - *Exemplo:* k = 2, n = 2 → 0011 (cíclico: 00, 01, 11, 10).
  - *Armadilha:* para uma string linear, acrescente os n − 1 primeiros dígitos; rotação não muda o ciclo.
- **Euler tour como ferramenta de construção.** Cobrir todas as arestas de K_n com um passeio (n ímpar), orientar arestas com |in − out| ≤ 1 (pareie os ímpares com um vértice fantasma e ache o circuito), concatenar uma palavra.
  - *Gatilho:* cobrir arestas exatamente uma vez; orientar grafo; concatenar peças.
  - *Por que funciona:* Euler existe sse o grafo é conexo e tem 0 ou 2 vértices de grau ímpar; o circuito dá uma ordem global.
  - *Como fazer:* K_n: n ímpar → todos os graus pares; n par → adicione um vértice fantasma e remova depois; orientação: pareie os ímpares com o fantasma, ache o circuito, remova; O(n + m).
  - *Exemplo:* dominó circular; "desenho sem levantar o lápis"; de Bruijn.
  - *Armadilha:* grafo com componentes com arestas; verifique a conexidade dos vértices de grau positivo.
- **Decompor arestas em caminhos de 2 arestas.** Grafo conexo com m par sempre decompõe; DFS, pareando de baixo para cima as arestas pendentes em cada vértice e passando a sobrante ao pai.
  - *Gatilho:* m par; decompor em caminhos de comprimento 2 (P3); parear arestas.
  - *Por que funciona:* teorema: grafo conexo com m par tem decomposição em caminhos de comprimento 2; o pareamento DFS de baixo para cima sempre fecha.
  - *Como fazer:* DFS; em cada vértice, pareie as arestas ainda não usadas; se sobrar uma, passe ao pai; O(n + m).
  - *Exemplo:* estrela com 4 arestas → dois P3; ciclo C4 → dois P3.
  - *Armadilha:* a aresta pai também conta; cada aresta pertence a exatamente um caminho; m ímpar é impossível.

- **Grafo com graus dados.** Erdős–Gallai testa; Havel–Hakimi constrói (ligue o de maior grau aos próximos maiores). Árvore com graus d_i: Σd = 2(n − 1) e d_i ≥ 1 bastam (Prüfer).
  - *Gatilho:* sequência de graus; construir grafo ou árvore.
  - *Por que funciona:* Erdős–Gallai caracteriza a sequência; Havel–Hakimi é o guloso correspondente; para árvore, Σd = 2(n − 1) e d_i ≥ 1 são suficientes via Prüfer.
  - *Como fazer:* HH: ordene decrescente; conecte o maior aos próximos d; repita; árvore: coloque d_i − 1 ocorrências de i na sequência de Prüfer.
  - *Exemplo:* (3,3,2,2,2) é válido; (3,3,3,1) não é.
  - *Armadilha:* laços/multiedges; soma ímpar; d_i = 0 em árvore com n ≥ 2.
- **Matriz 0/1 com somas de linhas e colunas dadas.** Gale–Ryser decide; construa por linha, ligando as colunas de maior soma restante. Com valores inteiros não negativos, canto noroeste: min(r_i, c_j) em cada passo, ≤ n + m − 1 entradas não nulas.
  - *Gatilho:* marginais de matriz binária; graus de um grafo bipartido; "matriz com somas de linhas e colunas".
  - *Por que funciona:* Gale–Ryser (dominância) decide a existência; o guloso por linha funciona porque escolher as colunas de maior soma restante é seguro.
  - *Como fazer:* 0/1: ordene as colunas por soma restante decrescente; na linha i, marque 1 nas r_i maiores; inteiros: canto noroeste min(r_i, c_j), ≤ n + m − 1 entradas não nulas.
  - *Exemplo:* r = (2,1), c = (1,1,1) → a primeira linha usa as duas colunas de maior soma restante.
  - *Armadilha:* somas totais diferentes; a condição de Gale–Ryser não é só a soma; a ordem das colunas importa.
- **Matriz com condição em linha/coluna (todas soma k, xor 0, etc.).** Preencha o bloco (n−1)×(m−1) livremente e determine a última linha/coluna; cheque a consistência do canto (soma total de linhas = soma total de colunas).
  - *Gatilho:* todas as linhas/colunas somam k, xor 0, ou têm a mesma paridade.
  - *Por que funciona:* a última linha/coluna é determinada pelo resto; só o canto precisa de consistência global.
  - *Como fazer:* preencha (n−1)×(m−1) livremente; última coluna = k − Σ linha; última linha = k − Σ coluna; o canto deve fechar; para xor, use ⊕.
  - *Exemplo:* 3×3 com todas as linhas/colunas somando 3; preencha o bloco 2×2 e determine o resto.
  - *Armadilha:* valores fora do intervalo permitido (negativos); soma total de linhas ≠ soma total de colunas.
- **Torneio com sequência de pontuação.** Landau decide: ordenada, Σ dos k menores ≥ C(k, 2) com igualdade no total; construa removendo o vértice de maior pontuação.
  - *Gatilho:* pontuações de um torneio; cada par joga uma vez.
  - *Por que funciona:* o teorema de Landau caracteriza a sequência; a construção gulosa remove o maior e ajusta os que ele venceu.
  - *Como fazer:* ordene; verifique $\Sigma_{i\le k}\, s_i \ge C(k,2)$ para todo k e igualdade em n; construa: o maior vence os s_n maiores restantes; decremente; recurse.
  - *Exemplo:* (1,1,1) → ciclo de 3; (2,1,0) → 1 vence 2 e 3, 2 vence 3.
  - *Armadilha:* ordem crescente vs decrescente; empates; a soma total tem de ser C(n,2).
- **Round-robin (K_n em n − 1 rodadas, n par)** → método do círculo: fixe um time e gire os outros. Coloração de arestas de bipartido com Δ cores sempre existe (König).
  - *Gatilho:* n par, n − 1 rodadas, cada par joga uma vez.
  - *Por que funciona:* é uma 1-fatoração de K_n; o método do círculo mantém um time fixo e gira os outros, gerando todos os pares.
  - *Como fazer:* fixe n; em cada rodada pareie i com n − i; gire 1..n−1; $O(n^2)$.
  - *Exemplo:* n = 4 → (1,4),(2,3); (1,3),(4,2); (1,2),(3,4).
  - *Armadilha:* n ímpar → adicione um bye; mandante/visitante alternado.
- **Quadrado latino** L[i][j] = (i + j) mod n; dois ortogonais existem para n ≠ 2, 6. Forma a base para escalas, turnos e "cada linha e coluna com todos".
  - *Gatilho:* cada linha/coluna deve conter todos os símbolos; escalas; turnos.
  - *Por que funciona:* L[i][j] = (i + j) mod n faz cada linha/coluna ser uma permutação cíclica.
  - *Como fazer:* para um par ortogonal, use L2[i][j] = (i + 2j) mod n quando n for ímpar; para n arbitrário (n ≠ 2,6), use corpos finitos/teorema de Bose–Shrikhande–Parker.
  - *Exemplo:* n = 3, L = [[0,1,2],[1,2,0],[2,0,1]].
  - *Armadilha:* n = 2 e n = 6 não têm par ortogonal; coeficientes não coprimos com n quebram a bijeção.

- **Strings sem padrão proibido.** 3 letras: "abcabc..." não tem palíndromo de tamanho ≥ 2; Thue–Morse não tem cubo nem overlap em 2 letras; existe sequência sem quadrados em 3 letras. Sem 'aaa'/'bbb' com contagens dadas: intercale grupos.
  - *Gatilho:* evitar palíndromo, quadrado, cubo ou overlap; contagens de letras dadas.
  - *Por que funciona:* sequências conhecidas evitam os padrões; para contagens, intercalar grupos controla adjacências.
  - *Como fazer:* "abcabc..." evita palíndromo ≥ 2; Thue–Morse evita cubo/overlap em 2 letras; sem quadrados use uma sequência ternária (ex.: ponto fixo de 0→012, 1→02, 2→1); com contagens, intercale os mais frequentes.
  - *Exemplo:* "abcabc..." sem palíndromo; Thue–Morse 01101001; "aabbb" com aaa/bbb proibidos → "babab".
  - *Armadilha:* com 2 letras não existe sequência infinita sem quadrados; o alfabeto mínimo muda o padrão.
- **Rearranjar sem vizinhos iguais.** Possível sse maior frequência ≤ ⌈n/2⌉; preencha posições 0, 2, 4, ... com os caracteres mais frequentes primeiro e depois 1, 3, 5, ...
  - *Gatilho:* reorganizar string/array sem adjacentes iguais.
  - *Por que funciona:* a condição maxfreq ≤ ⌈n/2⌉ é necessária e suficiente; colocar os mais frequentes nas posições pares separa-os.
  - *Como fazer:* preencha 0,2,4,... com os mais frequentes; depois 1,3,5,...; O(n).
  - *Exemplo:* "aaabb" → "ababa"; "aabb" → "abab".
  - *Armadilha:* n par vs ímpar; se maxfreq = (n+1)/2, comece na posição 0; a ordem dos empatados importa.
- **Conjunto de Sidon (somas de pares todas distintas).** $\{2p \cdot i + (i^2 \bmod p) : 0 \le i < p\}$ com p primo tem p elementos; também serve {2^i}. Útil para "subarrays/pares com somas distintas".
  - *Gatilho:* somas de pares distintas; "subarrays/pares com somas distintas".
  - *Por que funciona:* a construção $\{2p \cdot i + (i^2 \bmod p)\}$ é Sidon; a prova usa que diferenças iguais implicam i = j.
  - *Como fazer:* escolha p primo ≥ n; gere os p valores; O(p).
  - *Exemplo:* p = 5 → {0,11,24,39,56}; todas as somas de pares são distintas.
  - *Armadilha:* precisa p ≥ n; use i^2 mod p, não i^2; os valores chegam a $\sim 2p^2$.
- **Permutação sem ponto fixo** → rotação cíclica; sem p_i = i ± 1 ou com restrição de vizinhança → rotações e pares de blocos; teste n = 1..8 por brute force antes de fixar o padrão.
  - *Gatilho:* p_i ≠ i; p_i ≠ i ± 1; restrição de vizinhança.
  - *Por que funciona:* a rotação cíclica desloca todos; para restrições locais, rotações de blocos/pares evitam as posições proibidas.
  - *Como fazer:* p_i = (i mod n) + 1; para proibir i ± 1, use rotação de 2 ou pares de blocos; brute force n = 1..8.
  - *Exemplo:* n = 4 → [2,3,4,1]; n = 5 sem i ± 1 → [3,4,5,1,2].
  - *Armadilha:* n = 1 não existe; n = 2 só [2,1], que viola i ± 1; n = 3 tem casos especiais.
- **Ordenar com poucas operações.** Reverter prefixo → panqueca com ≤ 2n; reverter qualquer segmento → ≤ n − 1 (seleção); troca de qualquer par → n − ciclos; ordene em ≤ n operações fixando um elemento por vez.
  - *Gatilho:* limite de operações de ordenação; tipos de operação permitidos.
  - *Por que funciona:* cada operação pode fixar ao menos um elemento; swaps minimizam por ciclos.
  - *Como fazer:* prefixo (panqueca): leve o maior ao topo e depois à posição, ≤ 2n; segmento: seleção, ≤ n − 1; swaps: n − ciclos; fixe um por vez.
  - *Exemplo:* [3,1,2] com reversões de prefixo: reverse(2) → [1,3,2]; reverse(2) na cauda → [1,2,3].
  - *Armadilha:* limites exatos; panqueca 2n − 3; não conte operações identidade; segmento precisa de posições válidas.
- **Contar caminhos mínimos exatos N num grafo pequeno.** Cadeia de "losangos" (2 caminhos cada) constrói 2^k; N em binário usa k camadas e arestas de atalho por bit; ≈ 2 log N vértices.
  - *Gatilho:* grafo pequeno, exatamente N caminhos mínimos, N grande.
  - *Por que funciona:* um losango (duas rotas paralelas) dobra o número de caminhos; a representação binária de N soma potências de 2.
  - *Como fazer:* k losangos dão 2^k; para N, use camadas e atalhos por bit; ~2 log N vértices; mantenha todas as arestas no mesmo comprimento e o grafo em camadas.
  - *Exemplo:* N = 5 = 4 + 1 → dois losangos + um atalho.
  - *Armadilha:* atalhos criam caminhos extras; verifique que todos os caminhos têm o mesmo comprimento mínimo.

- **Grafo com exatamente k triângulos / arestas.** Clique de tamanho t dá C(t, 3); acrescente vértices ligados a pares do clique (cada um soma uma quantidade de triângulos); guloso do maior t.
  - *Gatilho:* número exato de triângulos ou arestas.
  - *Por que funciona:* K_t dá C(t,3) triângulos; um vértice ligado a um par do clique soma 1; o guloso escolhe o maior t.
  - *Como fazer:* maior t com C(t,3) ≤ k; adicione vértices ligados a pares para o resto; para arestas, use componentes completas e ajuste.
  - *Exemplo:* k = 5 → K_4 (4 triângulos) + um vértice ligado a uma aresta.
  - *Armadilha:* arestas extras entre novos vértices criam triângulos indesejados; isole as componentes.
- **Parênteses com profundidade/pares dados.** Aninhe a quantidade de profundidade necessária e concatene o resto como "()"; contagens de "()" vs "(())" dão controle fino.
  - *Gatilho:* sequência balanceada, profundidade máxima, número de pares.
  - *Por que funciona:* aninhar controla a profundidade; concatenar "()" adiciona pares sem aumentar a profundidade.
  - *Como fazer:* abra d e feche d para profundidade d; complete com "()" repetido; se p < d, impossível; combine.
  - *Exemplo:* d = 2, p = 4 → "(())()()".
  - *Armadilha:* profundidade exata vs ≤; não deixe prefixo negativo; p ≥ d.
- **Arranjo com soma de |diferenças| adjacentes alta** → intercalar menor e maior (ordem pico-vale); como candidato inicial para maximizar e depois testar com brute force.
  - *Gatilho:* maximizar Σ |a_i − a_{i+1}|; ordem pico-vale.
  - *Por que funciona:* alternar pequenos e grandes coloca cada elemento entre extremos opostos, aumentando as diferenças.
  - *Como fazer:* ordene; intercale menor, maior, 2º menor, 2º maior,...; teste variantes (rotações/reverso) e brute force n pequeno.
  - *Exemplo:* [1,2,3,4] → [2,4,1,3], soma 2 + 3 + 2 = 7.
  - *Armadilha:* a ordem pico-vale não é sempre ótima; empates e n ímpar mudam a melhor variante.
- **Local search com monovariante.** "Dividir vértices em dois grupos com cada um tendo ≥ metade dos vizinhos no outro": mova quem viola; o corte cresce a cada passo, então termina.
  - *Gatilho:* dividir em dois grupos com cada vértice tendo ≥ metade dos vizinhos no outro; max cut.
  - *Por que funciona:* se um vértice viola, movê-lo aumenta o corte em ≥ 1; o corte é ≤ m, então o processo termina.
  - *Como fazer:* comece com partição arbitrária; enquanto houver violação, mova; cada movimento é O(deg); total O(m) movimentos.
  - *Exemplo:* max cut ≥ m/2; coloração 2-aproximada.
  - *Armadilha:* ótimo local ≠ global; o monovariante garante terminação, não otimalidade.
- **Passeio do cavalo / Hamiltoniano em grid** → heurística de Warnsdorff + backtracking; com número ímpar de casas não existe tour fechado (a coloração xadrez prova).
  - *Gatilho:* visitar todas as casas; movimentos de cavalo; tour fechado.
  - *Por que funciona:* Warnsdorff escolhe o vizinho com menos continuações, evitando becos; a coloração xadrez prova a impossibilidade de tour fechado com número ímpar de casas.
  - *Como fazer:* DFS com ordem de Warnsdorff (grau futuro mínimo); backtracking; para fechado, cheque a paridade; O(8^{nm}) no pior caso, mas rápido na prática.
  - *Exemplo:* 8×8 tour; 5×5 aberto existe, fechado não.
  - *Armadilha:* grades pequenas/estreitas; Warnsdorff não garante sucesso sem backtracking.
- **Ladrilhar com L-triminós** um tabuleiro 2^k com um buraco → indução dividindo em 4 quadrantes e colocando um L no centro para criar mais buracos. Coloração (módulo 3, xadrez) prova as impossibilidades de dominós.
  - *Gatilho:* tabuleiro 2^k × 2^k com um buraco.
  - *Por que funciona:* indução: divida em 4 quadrantes; o buraco está em um; um L central cobre um canto de cada um dos outros três, criando um buraco em cada quadrante; recurse.
  - *Como fazer:* rec(x,y,size,hole); base size = 2; posicione o L central de acordo com o quadrante do buraco; O(4^k) peças.
  - *Exemplo:* 4×4 com buraco → 5 Ls.
  - *Armadilha:* coordenadas do L central; o buraco pode estar em qualquer quadrante.

- **Armadilha: construtivo com limite no tamanho da resposta.** Verifique o pior caso do número de operações/vértices na construção (ex.: 2n + 5) antes de implementar.
  - *Gatilho:* limite de operações/vértices/tamanho da resposta; "≤ 2n + 5".
  - *Por que funciona:* uma construção correta pode estourar o limite; o pior caso precisa ser contado, não estimado pela média.
  - *Como fazer:* conte operações por elemento e some; teste o pior caso; simplifique se passar; inclua folga para casos de borda.
  - *Exemplo:* 2 operações por elemento = 2n; se precisar de 3, estoura 2n + 5.
  - *Armadilha:* o checker rejeita por limite, não por incorreção; leia o limite com atenção.
- **Armadilha: múltiplas soluções.** Se há checker, não force a resposta de exemplo; se o exemplo parece "estranho", ele costuma mostrar a construção do autor.
  - *Gatilho:* checker; exemplo "estranho"; qualquer resposta válida.
  - *Por que funciona:* o exemplo pode mostrar a construção do autor, não a única; forçá-lo pode não generalizar.
  - *Como fazer:* implemente sua construção; use o exemplo para entender o formato; teste com checker/brute force.
  - *Exemplo:* "YES" + qualquer permutação válida; o exemplo pode ter uma ordem específica.
  - *Armadilha:* copiar o exemplo e falhar em casos gerais; não assuma que a resposta é única.
- **Busca binária interativa.** Cada pergunta de resposta binária resolve 1 bit: $\lceil \log_2 N \rceil$ queries; para $N = 10^{18}$ são 60. Perguntas de 3 respostas (<, =, >) dão $\log_3$.
  - *Gatilho:* resposta binária; N grande; limite log.
  - *Por que funciona:* cada pergunta binária reduz a incerteza à metade; $\lceil \log_2 N \rceil$; com 3 respostas, $\log_3$.
  - *Como fazer:* lo = 1, hi = N; pergunte o meio; ajuste; para 10^{18}, 60 queries; com 3 respostas, descarte um terço.
  - *Exemplo:* adivinhar número; achar limiar.
  - *Armadilha:* while(lo < hi) vs <=; respostas inválidas; contar queries.
- **Flush e protocolo.** Imprima com flush após cada query, leia a resposta (−1 ou erro = abortar), conte queries localmente e teste com interator caseiro; misturar cin e scanf sem sync causa deadlock.
  - *Gatilho:* implementação interativa.
  - *Por que funciona:* buffer não enviado = deadlock; leitura errada = WA.
  - *Como fazer:* use endl/cout.flush() ou fflush(stdout); leia a resposta; se -1/erro, encerre; conte queries; teste com interator.
  - *Exemplo:* cout << "? " << x << endl; cin >> r;
  - *Armadilha:* misturar cin/scanf sem sync; usar printf sem fflush; não abortar.
- **Achar máximo/pico com queries** → compare a[m] e a[m+1] (binária); posição do segundo maior: consulte o max de [1, n] e localize a posição p do máximo por binária (o lado cujo max é o global contém p); o segundo maior é o maior entre o max de [1, p−1] e o de [p+1, n], em O(log n) queries.
  - *Gatilho:* array escondido, queries de máximo em intervalo; achar máximo/segundo máximo.
  - *Por que funciona:* a binária em a[m] vs a[m+1] acha um pico; para o máximo global, compare os lados; para o segundo, localize p e consulte os dois lados.
  - *Como fazer:* pico: se a[m] < a[m+1], vá para a direita; senão, esquerda. Segundo: p = posição do máximo de [1,n] por binária; segundo = max(max[1,p−1], max[p+1,n]).
  - *Exemplo:* n = 8 → ~3 queries para p e 2 para o segundo.
  - *Armadilha:* array não unimodal; "pico" ≠ máximo global; duplicatas.
- **Min e max juntos** em $\lceil 3n/2 \rceil - 2$ comparações (pareie e compare vencedores com vencedores); **segundo maior** em $n + \lceil \log_2 n \rceil - 2$ com torneio; **ordenar com mínimo de comparações** → inserção binária (n log n) ou Ford–Johnson (merge insertion), ótimo para n ≤ 22 mas não em geral: há algoritmos com menos comparações em certos n (por exemplo, n = 47) e ele não alcança $\lceil \log_2 n! \rceil$ para todo n.
  - *Gatilho:* minimizar comparações; limites apertados; min/max simultâneos.
  - *Por que funciona:* parear evita comparações redundantes; o torneio guarda os perdedores do campeão, e o segundo maior está entre eles.
  - *Como fazer:* min/max: pareie e compare dentro do par; compare vencedores e perdedores; $3n/2 - 2$. Segundo maior: torneio de $n - 1$; compare os $\lceil \log_2 n \rceil$ perdedores do campeão.
  - *Exemplo:* n = 8 → 10 comparações para min/max; 9 para o segundo.
  - *Armadilha:* n ímpar; contagem exata; Ford–Johnson não é ótimo geral.

- **Reconstruir permutação com queries de soma/xor/and de pares.** Três queries de soma (a+b, b+c, a+c) determinam três números; com and não (bit com um único 1 é indistinguível) e com xor só se determinam diferenças (a menos de um xor global); para n posições fixe a_1 por 3 queries e depois n − 3 queries "a_1 + a_i".
  - *Gatilho:* queries de pares; reconstruir valores.
  - *Por que funciona:* a soma de três pares resolve três incógnitas; xor determina só diferenças; and não separa bits únicos.
  - *Como fazer:* 3 queries (a1+a2, a2+a3, a1+a3); depois n − 3 queries a1+ai; total n queries.
  - *Exemplo:* n = 5 → 5 queries.
  - *Armadilha:* overflow; and não funciona; xor tem liberdade global.
- **Permutação com queries p_i mod p_j.** Compare (i, j) nos dois sentidos: o maior dos dois resultados é min(p_i, p_j) (se p_i < p_j, p_i mod p_j = p_i); o menor é só o resto de p_j mod p_i; mantenha o candidato a maior e descubra todos os outros em ≤ 2n queries.
  - *Gatilho:* query modular; permutação 1..n.
  - *Por que funciona:* max(p_i mod p_j, p_j mod p_i) = min(p_i, p_j); 2 queries revelam o menor do par.
  - *Como fazer:* mantenha um candidato ao máximo; para cada outro, faça 2 queries; elimine o menor (valor conhecido); ao final o candidato é n; ≤ 2(n−1).
  - *Exemplo:* (5,2): 5 mod 2 = 1 e 2 mod 5 = 2 → max = 2 = min.
  - *Armadilha:* p_j mod 1 = 0; queries repetidas; conte 2 por comparação.
- **Permutação com queries "quantos antes de i são menores".** Reconstrua de trás para frente com k-ésimo livre num BIT (inverso da codificação de Lehmer).
  - *Gatilho:* posto/Lehmer; query de contagem à esquerda.
  - *Por que funciona:* é a codificação de Lehmer invertida: de trás para frente, o posto entre os livres determina o valor.
  - *Como fazer:* BIT/order-statistics com 1..n livres; para i = n..1, escolha o (c_i + 1)-ésimo livre; remova; O(n log n).
  - *Exemplo:* c = [0,1,2] → identidade; c = [0,1,1] → [1,3,2].
  - *Armadilha:* 0-based vs 1-based; "menores" estritos vs ≤.
- **Queries em árvore: achar vértice escondido.** Se a resposta diz de que lado do centroide está o alvo, $\lceil \log_2 n \rceil$ queries no total (uma por centroide, a cada nível). Com queries de distância, d(u, x), d(v, x) e d(u, v) dão a projeção de x no caminho u–v: dist de u = (d(u,v) + d(u,x) − d(v,x)) / 2.
  - *Gatilho:* árvore escondida; query de lado/centroide/distância.
  - *Por que funciona:* a centroid decomposition tem profundidade O(log n); a distância dá a projeção no caminho.
  - *Como fazer:* pergunte de que lado do centroide está o alvo; desça; $\lceil \log_2 n \rceil$. Distância: projeção de x em u–v = (d(u,v) + d(u,x) − d(v,x)) / 2.
  - *Exemplo:* árvore de 8 nós → 3 queries.
  - *Armadilha:* resposta "é o centroide" encerra; em árvore a projeção é inteira.
- **Reconstruir árvore escondida com distâncias.** d(r, ·) dá as camadas; para cada vértice, o pai é um vértice da camada anterior que está a distância 1 (query d(v, candidato)); busque por binária entre candidatos quando possível, ou use pares de folhas para reduzir.
  - *Gatilho:* queries d(u,v); reconstruir a árvore.
  - *Por que funciona:* d(r,·) dá as camadas; o pai de v está na camada anterior a distância 1.
  - *Como fazer:* escolha r; agrupe por distância; para cada v, procure pai entre candidatos da camada anterior com d(v,c) = 1; binária/pares de folhas para reduzir; O(n log n) queries.
  - *Exemplo:* estrela: centro na camada 0, folhas na camada 1.
  - *Armadilha:* muitos candidatos na mesma camada; não gaste $O(n^2)$ queries.
- **Mentiras limitadas.** Se o juiz mente até k vezes (ou em até uma de duas consecutivas), repita a query em janelas de k + 1 ou mantenha intervalo candidato com folga e reduza apenas quando houver consenso; calcule o número de queries com a folga.
  - *Gatilho:* o judge mente até k vezes; respostas inconsistentes.
  - *Por que funciona:* repetir k + 1 vezes garante maioria verdadeira; o consenso elimina mentiras; a folga no intervalo absorve erros.
  - *Como fazer:* repita cada pergunta k + 1 vezes e tome a maioria; ou mantenha intervalo com folga e reduza só com consenso; recalcule o orçamento.
  - *Exemplo:* k = 1 → pergunte 3 vezes.
  - *Armadilha:* mentiras adaptativas; orçamento de queries; não assuma que o erro é sempre na primeira.

- **Quente/frio.** Perguntas "mais perto de x que antes?" → compare dois palpites, o ponto médio entre eles divide o espaço; reduz a binária no ponto médio.
  - *Gatilho:* "mais perto que antes?"; comparação de palpites.
  - *Por que funciona:* o ponto médio entre dois palpites separa o espaço; a resposta diz de que lado está x.
  - *Como fazer:* mantenha dois pontos; pergunte qual está mais perto; mova o ponto médio; O(log range).
  - *Exemplo:* x = 7, palpites 0 e 16 → meio 8.
  - *Armadilha:* empate à mesma distância; coordenadas contínuas vs inteiras.
- **Interativo em jogo.** Decida qual jogador ser (primeiro ou segundo) se o enunciado permite; a estratégia de espelho/pareamento costuma ser a resposta e o programa só responde ao movimento do juiz.
  - *Gatilho:* jogar contra o juiz; escolher primeiro/segundo.
  - *Por que funciona:* se há estratégia de espelho/pareamento, o programa só responde; a escolha do papel decide.
  - *Como fazer:* analise a simetria; escolha o papel vencedor; implemente a resposta ao lance do juiz; teste com interator.
  - *Exemplo:* jogo de pares; o segundo espelha.
  - *Armadilha:* escolher o papel errado; protocolo de entrada/saída.

Repetidas de outras seções:

- **Simetria/espelho** → construção pelo centro; mesmo truque do jogo espelho (teoria dos jogos).
- **Pares (i, n+1−i), (x, x xor 1)** → construir em pares resolve muitos construtivos e jogos.
- **Construir string** → palíndromo, repetição de um período, alternância 0101 (strings).
- **Construir árvore com graus dados** → soma dos graus = 2(n − 1) e todos ≥ 1 é suficiente; construção por Prüfer ou ligando maior grau a folhas (ordem decrescente).
- **Construir grafo com sequência de graus dada (Erdős–Gallai / Havel–Hakimi)** → ordene decrescente, ligue o maior aos próximos d vértices de maior grau, repita; condição $\Sigma_{i\le k} d_i \le k(k-1) + \Sigma_{i>k} \min(d_i, k)\ \forall k$ e soma par.
- **Construir árvore com distâncias dadas / diâmetro dado** → cadeia do diâmetro + folhas pendentes a profundidade ≤ ⌊diâm/2⌋ ao meio; verifique soma e matriz de distâncias (aditiva/four-point).
- **Grafo bipartido com graus dados (Gale–Ryser)** → ordene um lado, condição de dominância por conjugado; construção gulosa por maior grau.
- **Circuito de Euler dá construção de orientação balanceada, "ordem de dominós/linhas", de Bruijn e "desenho sem levantar".**
- **Torneio com pontuações dadas (Landau)** → construção gulosa; também "torneio com mínimo de ciclos 3".

Teoria dos jogos

Primeiro passo sempre: brute force de vitória/derrota para estados pequenos e imprima a tabela. O padrão quase sempre aparece em n ≤ 20.

Heurísticas de estratégia:

- **Imite o adversário (espelho).** Tabuleiro ou configuração simétrica → o segundo jogador copia o lance do outro do lado oposto e nunca fica sem jogada. Se o primeiro pode criar a simetria com um lance (tirar o meio, dividir em dois iguais), ele ganha.
  - *Gatilho:* tabuleiro/configuração simétrica; segundo jogador; "copie o lance".
  - *Por que funciona:* cada lance tem uma imagem simétrica disjunta; se o primeiro jogou, a imagem está livre; o segundo nunca fica sem resposta e, por finitude, o primeiro esgota primeiro.
  - *Como fazer:* identifique a simetria (central, eixo, translação); responda com o lance espelhado; se o primeiro pode criar a simetria (tomar o centro), faça isso e vire "segundo".
  - *Exemplo:* duas pilhas iguais; tabuleiro simétrico com dominós; Nim (1,1).
  - *Armadilha:* simetria quebrada por um elemento central; trate o centro antes; o lance espelhado precisa ser legal e distinto.
- **A primeira escolha é a que mais importa.** Muitos jogos se decidem no lance 1 (escolher a ponta, a primeira letra, a cor, a raiz); depois disso a resposta segue um padrão fixo. Teste as n primeiras jogadas e veja se o resto é forçado.
  - *Gatilho:* muitas jogadas, mas a primeira parece decidir; "escolha a ponta/letra/raiz".
  - *Por que funciona:* depois do primeiro lance, a resposta ótima costuma ser forçada ou recursiva; o espaço de primeiros lances é pequeno.
  - *Como fazer:* teste as n primeiras jogadas com brute force; veja se o resto é forçado; escolha a vencedora.
  - *Exemplo:* escolher a ponta e depois espelhar; escolher a primeira letra em um jogo de string.
  - *Armadilha:* assumir que o resto é forçado sem provar; pode haver mais de um ponto de decisão.

- **Roubo de estratégia.** Se "passar a vez" ou um lance inofensivo existe, o primeiro jogador nunca perde (se o segundo tivesse estratégia, o primeiro roubaria).
  - *Gatilho:* existe passe ou lance inofensivo; o segundo teria estratégia.
  - *Por que funciona:* se o segundo tivesse estratégia vencedora, o primeiro faz o lance inofensivo e "rouba" a estratégia do segundo; logo o primeiro não perde.
  - *Como fazer:* mostre que o lance extra não pode prejudicar (passe); conclua vitória/empate do primeiro; para construtivo, combine com simetria.
  - *Exemplo:* Chomp, Hex; jogo com passe.
  - *Armadilha:* não é construtivo (só existência); se o lance inofensivo não existe, não vale.
- **Paridade do número de lances.** Se o jogo sempre dura o mesmo número de lances, só a paridade importa. Procure o que é fixo: total de peças, de casas, de inversões.
  - *Gatilho:* o jogo sempre dura o mesmo número de lances; só a paridade importa.
  - *Por que funciona:* se o total é fixo, o vencedor é quem faz o último; a paridade de peças/casas/inversões determina.
  - *Como fazer:* identifique o invariante que fixa o total; calcule a paridade; procure o que é fixo mesmo com escolhas.
  - *Exemplo:* remover uma peça por lance; n peças → n lances.
  - *Armadilha:* o número de lances varia; paridade não basta.
- **Quem controla o último lance** ganha; conte lances forçados e lances "livres".
  - *Gatilho:* o jogo termina quando não há lances; conte forçados/livres.
  - *Por que funciona:* o vencedor é quem faz o último lance; controlar a paridade dos forçados decide.
  - *Como fazer:* separe lances forçados dos livres; quem controla os livres ajusta a paridade; conte.
  - *Exemplo:* Nim com pilhas de 1; retirar 1..k.
  - *Armadilha:* "último a mover perde" (misère) inverte; confira a regra.
- **Pares (pairing strategy).** Agrupe os lances em pares (i, i+1), (i, n+1−i), (x, x xor 1): sempre responda com o par do lance do adversário.
  - *Gatilho:* lances vêm em pares naturais; simetria.
  - *Por que funciona:* agrupe os lances em pares disjuntos; quando o adversário usa um, use o par; ele nunca esgota sozinho.
  - *Como fazer:* defina o pareamento (i,i+1), (i,n+1−i), (x,x xor 1); responda com o par; verifique que está disponível.
  - *Exemplo:* peças em pares; matching em grafo; tabuleiro com dominós.
  - *Armadilha:* pares sobrepostos; o par pode ter sido usado; o pareamento deve ser perfeito.
- **Reduzir para um estado perdedor conhecido.** Ache uma família de estados P (perdedores) fechada: de P só se vai para N, e de N sempre existe lance para P.
  - *Gatilho:* jogo com estados P/N; quer provar a classificação.
  - *Por que funciona:* defina uma família P fechada: de P só se vai para N; de N sempre há lance para P; indução bem-fundada.
  - *Como fazer:* chute P por brute force pequeno; prove as duas propriedades; implemente a transição.
  - *Exemplo:* Nim xor = 0; Wythoff; subtração 1..k.
  - *Armadilha:* família incompleta; com ciclos a indução falha (use BFS retrógrada).
- **Lance que mantém o invariante** (xor = 0, soma ≡ 0 mod k+1, todas as pilhas pares).
  - *Gatilho:* P-positions descritas por invariante (xor, soma mod k+1, paridade).
  - *Por que funciona:* se o invariante é 0, qualquer lance o torna ≠ 0; de ≠ 0 há um lance que o zera.
  - *Como fazer:* identifique o invariante; mostre que sempre se restaura; implemente a restauração.
  - *Exemplo:* Nim xor = 0; subtração 1..k com mod k+1.
  - *Armadilha:* invariante necessário mas não suficiente; prove a volta.
- **Ganância falha em jogos.** O lance que maximiza agora raramente é o certo; pense no que sobra para o adversário.
  - *Gatilho:* jogo adversarial; valores imediatos.
  - *Por que funciona:* o valor imediato ignora a resposta do adversário; o ótimo é minimax.
  - *Como fazer:* pense no que sobra; use DP/minimax ou brute force; desconfie de sort ganancioso.
  - *Exemplo:* pegar das pontas; escolher a maior agora pode perder.
  - *Armadilha:* a ordem gananciosa às vezes é ótima (jogos de troca); prove antes.

Ferramentas:

- **Jogos independentes somados** → Sprague-Grundy: xor dos Grundy; Grundy de um estado = mex dos sucessores.
  - *Gatilho:* várias pilhas/tabuleiros independentes; a jogada afeta um só.
  - *Por que funciona:* Sprague–Grundy: cada componente é um nimber; a soma é o xor; o Grundy é o mex dos sucessores.
  - *Como fazer:* calcule g(estado) = mex{g(sucessores)}; combine por xor; verifique a independência.
  - *Exemplo:* Nim; Kayles; vários tokens.
  - *Armadilha:* componentes que interagem; misère não é xor; estado grande.
- **Nim** → xor ≠ 0 ganha. **Tirar 1..k** → n mod (k+1). **Misère Nim** → igual, exceto quando todas as pilhas são 1.
  - *Gatilho:* pilhas; retirar qualquer quantidade; retirar 1..k; misère.
  - *Por que funciona:* xor = 0 é P; retirar 1..k tem período k+1; misère é igual exceto todas as pilhas 1.
  - *Como fazer:* xor; para 1..k, n mod (k+1); misère: se todas 1, paridade; senão Nim normal.
  - *Exemplo:* (3,4,5) xor = 2 ≠ 0 → o primeiro ganha; (1,1) misère → o primeiro ganha e o segundo é obrigado a levar a última.
  - *Armadilha:* misère só muda o caso todas-1 no Nim; não generalize.
- **Staircase Nim** → só as posições ímpares importam. **Nim em árvore (Green Hackenbush)** → Grundy(v) = xor de (Grundy(filho) + 1).
  - *Gatilho:* pilhas em escada; mover de i para i−1; árvore com remoção de arestas.
  - *Por que funciona:* pares de escadas se cancelam; só as posições ímpares importam; na árvore, cada aresta é uma haste e a fórmula é xor(filho + 1).
  - *Como fazer:* xor das pilhas nas posições ímpares (1-based); na árvore, calcule de baixo para cima com xor(g(filho) + 1).
  - *Exemplo:* (1,2,3) → xor 1 ⊕ 3 = 2 ≠ 0.
  - *Armadilha:* indexação 0/1; a paridade correta depende de onde começa a escada.
- **Jogo em grafo com ciclos (empate possível)** → BFS retrógrada a partir dos estados finais, com contador de saídas.
  - *Gatilho:* grafo de estados com ciclos; empate possível.
  - *Por que funciona:* BFS retrógrada dos terminais marca P/N; o contador de saídas detecta empate.
  - *Como fazer:* fila com terminais (sem lances = P); para cada predecessor, se algum sucessor é P → N; se todos N → P; restantes empate.
  - *Exemplo:* token em grafo com ciclo.
  - *Armadilha:* não marcar empate; a ordem da BFS; estados não alcançáveis.
- **Jogos de soma de pontos (minimax)** → DP em intervalo (pegar das pontas), dp = melhor diferença para quem joga.
  - *Gatilho:* pegar das pontas; maximizar a soma.
  - *Por que funciona:* dp[l][r] = melhor diferença; subtrai o dp do adversário; o jogo é de soma zero.
  - *Como fazer:* dp[l][r] = max(a[l] − dp[l+1][r], a[r] − dp[l][r−1]); $O(n^2)$; 1D rolando.
  - *Exemplo:* [1,5,2,4] → dp.
  - *Armadilha:* guardar dois valores é desnecessário; n ímpar muda a paridade.
- **Dois jogadores escolhem itens alternadamente com valores a_i (Alice) e b_i (Bob)** → ordene por a_i + b_i em ordem decrescente e pegue nessa ordem.
  - *Gatilho:* Alice pega item e ganha a_i; Bob pega e ganha b_i; alternam.
  - *Por que funciona:* no jogo de diferença, tomar i vale a_i e nega b_i ao adversário; o swing é a_i + b_i; o exchange argument ordena por esse valor.
  - *Como fazer:* ordene decrescente por a_i + b_i; simule a escolha alternada (Alice 1ª); O(n log n).
  - *Exemplo:* itens (a,b) = (5,5),(6,0): ordem por soma 10,6; Alice leva o primeiro e a diferença é 5 − 0 = 5.
  - *Armadilha:* vale para o jogo de diferença (soma zero); se cada um maximiza só a própria soma, o critério não é esse; jogo truncado também quebra o guloso.
- **Grundy grande demais** → calcule pequenos, encontre periodicidade, prove depois.
  - *Gatilho:* n até 10^{18}; Grundy de uma pilha.
  - *Por que funciona:* sequências de jogos de subtração finitos são eventualmente periódicas; o período pode ser detectado com valores pequenos.
  - *Como fazer:* DP até 2–3 mil; detecte o período; prove/confie; aplique n mod período.
  - *Exemplo:* subtração {1,2} → período 3.
  - *Armadilha:* pré-período; S infinito (quadrados) não garante periodicidade.

Gatilhos por tipo:

- **Jogo em array** → pontas e paridade das posições; quando n é par, o primeiro pode garantir todas as pares ou todas as ímpares.
  - *Gatilho:* array; jogadores das pontas.
  - *Por que funciona:* a paridade das posições; com n par, o primeiro garante todas as pares ou todas as ímpares.
  - *Como fazer:* calcule as somas das duas paridades; escolha a maior e use a estratégia de espelho; para n ímpar, DP.
  - *Exemplo:* n = 4, [1,5,2,4] → pares (5,4) somam 9; ímpares (1,2) somam 3; o primeiro garante 9.
  - *Armadilha:* n ímpar quebra; regra diferente (pegar k) muda.
- **Jogo em string** → a primeira letra/escolha costuma decidir; palíndromo e simetria sugerem espelho.
  - *Gatilho:* string; jogadores escolhem letras.
  - *Por que funciona:* a primeira letra/extremo costuma decidir; palíndromo e simetria sugerem espelho.
  - *Como fazer:* teste os primeiros lances; se palíndromo, o segundo espelha; se não, construa o guloso dos extremos.
  - *Exemplo:* CF 794C; jogo de formar string.
  - *Armadilha:* empates; a ordem lexicográfica; o alfabeto.
- **Jogo em árvore** → paridade da profundidade, centro do diâmetro, número de folhas.
  - *Gatilho:* árvore; remover folhas/caminhar.
  - *Por que funciona:* a paridade da profundidade, o centro do diâmetro e o número de folhas capturam a estrutura; simetria no centro permite espelho.
  - *Como fazer:* enraíze no centro; se simétrico, espelhe; conte folhas/profundidade; use Grundy para ramos.
  - *Exemplo:* remover folhas alternadamente; Green Hackenbush.
  - *Armadilha:* árvore não simétrica; o centro pode ser uma aresta.
- **Jogo em grid** → paridade de i + j (xadrez), espelho pelo centro.
  - *Gatilho:* tabuleiro; movimentos.
  - *Por que funciona:* xadrez (i + j); espelho pelo centro.
  - *Como fazer:* pinte; use a paridade para impossibilidade; espelhe se simétrico; para jogos de colocação, use Grundy.
  - *Exemplo:* dominó, cavalo, Cram.
  - *Armadilha:* dimensões ímpares; o centro.
- **Jogo com números** → divisores, potências de 2, n mod k, representação binária.
  - *Gatilho:* números; divisores, potências.
  - *Por que funciona:* estrutura multiplicativa/binária; n mod k.
  - *Como fazer:* fatore; olhe a maior potência de 2; use a representação binária; teste.
  - *Exemplo:* dividir por um primo; subtrair potências de 2.
  - *Armadilha:* 1 é terminal; primos; o número de divisores.

---

adicionado em 2026-11-10 por agentes; revisado

- **Moore's Nim_k.** Pode tirar de até k pilhas por lance → escreva as pilhas em binário e some cada coluna de bit mod (k+1); posição perdedora ⇔ todas as colunas ≡ 0. Para k = 1 é o Nim comum.
  - *Gatilho:* retirar de até k pilhas.
  - *Por que funciona:* a soma binária por coluna mod k+1 generaliza o xor; zero é P.
  - *Como fazer:* para cada bit, conte as pilhas com 1; se todas as colunas ≡ 0 mod k+1 → P; senão procure o movimento que zera (para k = 1 é o xor).
  - *Exemplo:* k = 2, (1,1,1): a coluna 1 soma 3 ≡ 0 → P.
  - *Armadilha:* k ≥ número de pilhas; a mudança pode reduzir até k pilhas; verifique se o lance existe.
- **Fibonacci Nim.** Uma pilha; o 1º lance tira de 1 até n−1; depois cada um tira de 1 até 2× o que o outro acabou de tirar. Perdedor para quem começa ⇔ n é Fibonacci (Zeckendorf: tire o menor termo da representação).
  - *Gatilho:* retirada limitada a 2× a anterior; uma pilha.
  - *Por que funciona:* Zeckendorf; as P-positions são os Fibonacci; a estratégia tira o menor termo.
  - *Como fazer:* represente n em Fibonacci não consecutivos; se n é Fibonacci, perde; senão tire o menor termo; depois responda.
  - *Exemplo:* n = 6 = 5 + 1, tire 1.
  - *Armadilha:* o primeiro lance não tem limite (1..n−1); n = 1 perde; não confunda com Nim.

- **Nim com conjunto de subtração S.** Grundy de uma pilha = mex{g(n − s)}; é eventualmente periódico para S finito. Calcule 2–3 mil valores e detecte o período; "tirar 1..k" é o caso n mod (k+1).
  - *Gatilho:* retirar s ∈ S de uma pilha.
  - *Por que funciona:* g(n) = mex{g(n − s)}; S finito → eventualmente periódico.
  - *Como fazer:* DP até detectar o período; aplique; O(n|S|).
  - *Exemplo:* S = {1,3,4}.
  - *Armadilha:* pré-período longo; S infinito não garante.
- **Subtraia só quadrados / primos / potências.** Sem fórmula: g(n) por DP em O(n·|S|) com mex; se $n \le 10^6$, é só isso. Se $n \le 10^{18}$, não presuma período: periodicidade de Grundy só é garantida para S finito; com quadrados/primos/potências (S infinito) não há garantia — para quadrados é problema aberto.
  - *Gatilho:* S infinito; n grande.
  - *Por que funciona:* DP para n pequeno; não há periodicidade garantida.
  - *Como fazer:* se $n \le 10^6$ e |S| pequeno (quadrados |S| ≈ √n; potências |S| ≈ log n), DP O(n|S|); para primos |S| = π(n) ≈ n/log n, o DP é $O(n^2/\log n)$ e só serve para n bem menor; se $n \le 10^{18}$, não presuma período.
  - *Exemplo:* subtract-a-square; quadrados até n.
  - *Armadilha:* assumir período; para quadrados é aberto; o custo para primos não é pequeno.
- **Kayles e Dawson.** Kayles (derrubar 1 pino ou 2 adjacentes numa fileira; vira soma de duas fileiras) tem Grundy periódico de período 12 após exceções iniciais; Dawson's Kayles (só 2 adjacentes) tem período 34. Qualquer "remover bloco e partir a fileira em duas" → soma de jogos e Grundy por DP em n^2.
  - *Gatilho:* derrubar pinos; a fileira se divide.
  - *Por que funciona:* soma de subjogos; Grundy por DP; periodicidade.
  - *Como fazer:* g(n) = mex over movimentos; Kayles: remover 1 ou 2 adjacentes → g(a) ⊕ g(b); Dawson: só 2; use período 12/34 após o pré-período.
  - *Exemplo:* fileira de 5.
  - *Armadilha:* pré-período; bordas; a divisão em duas fileiras é soma.
- **Lasker's Nim.** Pilha: tirar qualquer quantidade OU dividir em duas não vazias → g(4k+1) = 4k+1, g(4k+2) = 4k+2, g(4k+3) = 4k+4, g(4k+4) = 4k+3. Mostra que "dividir" pode tornar o Grundy quase o tamanho.
  - *Gatilho:* tirar ou dividir a pilha.
  - *Por que funciona:* a fórmula g(4k+r) mostra que dividir pode aumentar o Grundy; a divisão vira xor.
  - *Como fazer:* use a fórmula direta; para múltiplas pilhas, xor.
  - *Exemplo:* g(3) = 4, g(4) = 3.
  - *Armadilha:* divisão em duas não vazias; zero; a fórmula depende de 4k.
- **Grundy limitado pelo grau.** g(v) ≤ grau de saída de v, e num grafo com m arestas g ≤ O(√m) (um vértice com g = x precisa de sucessores com 0..x−1, e eles de sucessores próprios). Vetor de "valores vistos" pode ser pequeno.
  - *Gatilho:* grafo grande; Grundy.
  - *Por que funciona:* g(v) ≤ outdeg(v); se g = x, precisa de sucessores com 0..x−1; a soma dos graus limita x = O(√m).
  - *Como fazer:* calcule por DP topológica; use um vetor de valores vistos; limite O(√m).
  - *Exemplo:* DAG esparso.
  - *Armadilha:* grafo com ciclos; outdeg no DAG; laços.
- **Vários tokens num DAG.** Cada token é um jogo independente: xor dos Grundy das posições; vence quem começa se ≠ 0. Se tokens podem ocupar o mesmo vértice, ainda vale (jogos somados, não interagem).
  - *Gatilho:* vários tokens; movem independentes.
  - *Por que funciona:* soma de jogos; xor dos Grundy das posições.
  - *Como fazer:* calcule g(v) no DAG; xor; se ≠ 0, o primeiro ganha.
  - *Exemplo:* dois tokens.
  - *Armadilha:* tokens que bloqueiam/interagem; aí não é soma.
- **Lances reversíveis são ignoráveis.** Se o jogo permite também aumentar uma pilha, o vencedor pode desfazer o aumento (remove o que foi somado), então a classificação P/N é a do jogo sem esse lance, desde que o jogo termine.
  - *Gatilho:* o jogo permite aumentar pilha/desfazer.
  - *Por que funciona:* o oponente pode desfazer o aumento; remover o lance reversível não muda P/N.
  - *Como fazer:* ignore lances que aumentam; classifique o jogo reduzido; prove terminação.
  - *Exemplo:* Nim com "adicionar" permitido.
  - *Armadilha:* jogo infinito; a reversão pode não estar sempre disponível.

- **Coin-turning games.** Virar moedas com regra "a moeda mais à direita vira de cara para coroa": o jogo é soma dos jogos de cada cara, g(posição) = xor de g(i) de cada cara na posição i. Turning Turtles (virar 1 ou 2 moedas) → g(i) = i (Nim). Ruler game (virar qualquer bloco contíguo) → g(i) = maior potência de 2 que divide i.
  - *Gatilho:* virar moedas; a mais à direita vira de cara para coroa.
  - *Por que funciona:* o jogo é a soma dos jogos de cada cara; g(posição) = xor dos g(i).
  - *Como fazer:* calcule g(i); Turning Turtles g(i) = i; Ruler g(i) = maior potência de 2 que divide i; xor das caras.
  - *Exemplo:* cara na posição 3 vale 3 (TT) ou 1 (Ruler).
  - *Armadilha:* a regra da direita; indexação 0/1; a posição 0.
- **Tartan (produto de jogos).** Coin-turning em 2D que é produto de dois jogos 1D → g(x, y) = nim-produto $g_1(x) \otimes g_2(y)$. Raro, mas explica Grundy de "virar retângulo de moedas" (usa nim-multiplicação, que se calcula recursivamente em potências de 2 de Fermat).
  - *Gatilho:* coin-turning 2D; produto de dois jogos 1D.
  - *Por que funciona:* o produto de jogos usa o nim-produto; g(x,y) = g1(x) ⊗ g2(y).
  - *Como fazer:* calcule g1,g2; nim-multiplique; para retângulos, use potências de 2 de Fermat.
  - *Exemplo:* virar um retângulo de moedas.
  - *Armadilha:* nim-produto ≠ produto comum; raro.
- **Green Hackenbush geral.** Ciclos colapsam: **fusion principle** — todo ciclo pode ser contraído num único vértice (laços são removidos) sem mudar o Grundy; depois é uma árvore. Chão = raiz.
  - *Gatilho:* Hackenbush verde com ciclos.
  - *Por que funciona:* fusion principle; contrai ciclos; vira árvore.
  - *Como fazer:* contraia cada ciclo; remova laços; calcule g na árvore (xor(filho + 1)).
  - *Exemplo:* triângulo no chão.
  - *Armadilha:* chão/raiz; ciclos que compartilham vértices.
- **Colon principle.** Vários ramos pendurados no mesmo vértice → só importa o Grundy de cada ramo (xor); pode substituir um ramo por uma haste Nim de mesmo Grundy. Dá a fórmula g(v) = xor(g(filho) + 1).
  - *Gatilho:* ramos no mesmo vértice.
  - *Por que funciona:* só importa o xor dos ramos; um ramo pode ser substituído por uma haste Nim equivalente.
  - *Como fazer:* g(v) = xor(g(filho) + 1); substitua.
  - *Exemplo:* estrela.
  - *Armadilha:* só vale para verde; partizan não.
- **Hackenbush azul-vermelho (partizan).** Hastes só azuis valem +n inteiros, só vermelhas −n; ao trocar de cor o valor binário acrescenta ±1/2, ±1/4... Soma dos valores > 0 ⇒ Esquerda ganha começando ou não; = 0 ⇒ quem começa perde; < 0 ⇒ Direita ganha.
  - *Gatilho:* hastes coloridas; jogadores com cores.
  - *Por que funciona:* o valor é dyadic; +n azul, −n vermelho; após a primeira troca de cor, cada aresta seguinte vale metade da anterior (sinal pela cor).
  - *Como fazer:* calcule o valor da haste da base para o topo; some; > 0 Esquerda, = 0 segundo, < 0 Direita.
  - *Exemplo:* B = 1, R = −1, B-R = 1/2, B-R-R = 1/4, B-R-B = 3/4.
  - *Armadilha:* não pare na primeira troca: depois dela, toda aresta seguinte cai pela metade; B-R-R não é 1 − 1/2 = 1/2.
- **Jogos partizan: compare com 0.** Quando cada jogador tem lances próprios, defina o valor do jogo (número dyadic, ou jogo quente) e some; o sinal do total decide. "Cada um só pode mexer nas suas peças" → valores numéricos, soma, sinal.
  - *Gatilho:* cada jogador tem lances próprios.
  - *Por que funciona:* o valor do jogo (número ou quente); a soma; para números, o sinal decide.
  - *Como fazer:* defina o valor recursivamente {L|R}; se número, some; se quente, use a teoria de jogos (não só o sinal).
  - *Exemplo:* Hackenbush; Domineering.
  - *Armadilha:* jogo quente não tem sinal; comparar com 0 é fuzzy, não "positivo/negativo".

- **Jogos quentes (switches).** {a | b} com a > b: quem joga primeiro ganha (a−b)/2 em relação à média; média (a+b)/2 e temperatura (a−b)/2. Em jogos de pontuação em que mover é bom, jogue primeiro o de maior temperatura (heurística exata nos casos simples; confirme por brute force).
  - *Gatilho:* {a|b}, a > b; pontuação.
  - *Por que funciona:* média (a+b)/2, temperatura (a−b)/2; o primeiro ganha (a−b)/2 sobre a média.
  - *Como fazer:* calcule média/temperatura; em soma, jogue o de maior temperatura primeiro; confirme com brute force.
  - *Exemplo:* {5|1}: média 3, temperatura 2; o primeiro ganha 2.
  - *Armadilha:* a heurística não é sempre exata; confirme.
- **Cram e Domineering.** Cram (colocar dominó sem sobrepor, ambos com os mesmos lances): as duas dimensões pares → o 2º espelha pelo centro; exatamente uma ímpar → o 1º coloca um dominó central simétrico e espelha; ímpar×ímpar não é coberto pela simetria central (3×3, 5×5, 1×5 são P; 3×5, 1×3 são N) → Grundy/brute. Em Domineering (partizan) use contagem de lances seguros de cada lado.
  - *Gatilho:* colocar dominós; Cram ambos, Domineering vertical/horizontal.
  - *Por que funciona:* simetria central; Cram: pares/ímpares; Domineering: contagem de lances seguros.
  - *Como fazer:* Cram: ambos pares → 2º espelha; exatamente um ímpar → 1º dominó central simétrico; ímpar×ímpar → Grundy/brute. Domineering: conte lances seguros.
  - *Exemplo:* 3×3 P, 5×5 P, 1×5 P, 3×5 N, 1×3 N.
  - *Armadilha:* ímpar×ímpar não tem regra uniforme; use Grundy.
- **Strategy stealing em Chomp.** Chomp com tabuleiro retangular m×n: para m·n ≥ 2 o 1º sempre ganha (se o lance de pegar o canto perdedor perdesse, o 2º teria uma resposta que o 1º poderia ter feito já); 1×1 é a exceção (o 1º é obrigado a comer a casa envenenada e perde). Construtivo em casos especiais: n×n → tire (2,2) e espelhe o L; 2×n → deixe a 1ª linha 1 maior que a 2ª.
  - *Gatilho:* Chomp; primeiro sempre ganha mas sem construção; casos n×n e 2×n explícitos.
  - *Por que funciona:* roubo de estratégia: se o segundo tivesse estratégia vencedora, o primeiro come um canto não envenenado e passa a imitar; o lance extra não atrapalha (comer mais só remove opções do adversário); 1×1 é a exceção, pois o primeiro é forçado a comer a casa envenenada.
  - *Como fazer:* n×n: coma (2,2), deixando um L simétrico, e espelhe o lance do adversário; 2×n: o lance vencedor deixa as linhas com diferença 1 (a linha da casa envenenada 1 maior) e, a cada lance do adversário, restaure a diferença 1; retângulo geral: use só a existência (strategy stealing).
  - *Exemplo:* 3×3: comer (2,2) deixa a linha e a coluna; espelhe; 2×3: deixe a diferença 1 e restaure.
  - *Armadilha:* o roubo não dá o lance vencedor; a construção só cobre formatos especiais; comer a casa envenenada perde na hora.
- **Hex lemma.** Tabuleiro cheio de Hex tem exatamente um vencedor (nunca empate) ⇒ por stealing o 1º ganha. "Bloquear o outro = conectar eu" também vale em jogos de conexão em grade/triangular.
  - *Gatilho:* Hex; conexão de lados; provar que não há empate; primeiro ganha.
  - *Por que funciona:* o lemma do hex: num tabuleiro cheio, exatamente um dos jogadores conecta seus dois lados (dualidade planar); logo não há empate; por strategy stealing o segundo não pode ter estratégia vencedora, então o primeiro ganha.
  - *Como fazer:* para existência, aplique o lemma; para implementar, BFS/union-find nas duas cores; para estratégia explícita, use simetria/pareamento em tabuleiros simétricos.
  - *Exemplo:* Hex 3×3; "bloquear o outro = conectar eu" em jogos de conexão.
  - *Armadilha:* o lemma é específico do Hex em grade hexagonal; em outras topologias pode haver empate; o stealing não constrói o lance.
- **m,n,k-games.** Em jogos de alinhar k (tic-tac-toe generalizado) o 2º nunca ganha por stealing; a pergunta real é empate vs vitória do 1º → brute force com poda.
  - *Gatilho:* alinhar k; tabuleiro m×n; tic-tac-toe generalizado.
  - *Por que funciona:* strategy stealing: o segundo não pode ter estratégia vencedora, pois o primeiro poderia jogar um lance extra inofensivo e roubá-la; logo o resultado é empate ou vitória do primeiro.
  - *Como fazer:* brute force com poda/alpha-beta e simetrias para tabuleiros pequenos; para faixas grandes, prove empate com pareamento; a pergunta é empate vs vitória do 1º.
  - *Exemplo:* 3×3, k=3 → empate com jogo ótimo; 4×4, k=4 → o primeiro ganha.
  - *Armadilha:* stealing só exclui a vitória do 2º; não decide; o custo cresce rápido; simetrias são essenciais.

- **Geography em grafo não-direcionado.** Token percorrendo arestas sem revisitar vértices: o jogador que move a partir de v ganha ⇔ v está em todo emparelhamento máximo (calcule matching máximo com e sem v). Em direcionado geral o problema é PSPACE-completo, então o enunciado tem estrutura (DAG, árvore, bipartido).
  - *Gatilho:* token em grafo não-direcionado; sem revisitar vértices; matching.
  - *Por que funciona:* Fraenkel–Scheinerman–Ullman: o primeiro vence a partir de v sse v está em todo emparelhamento máximo; se há um máximo que não cobre v, o segundo usa o matching para responder; se v é essencial, o primeiro força.
  - *Como fazer:* calcule o matching máximo M; v é essencial sse o matching máximo de G − v é menor que |M|; equivalentemente, procure caminho alternado a partir de v não coberto.
  - *Exemplo:* caminho 1–2–3: máximo 1; v=2 está em todo máximo → o primeiro ganha; v=1 não está em {(2,3)} → perde.
  - *Armadilha:* direcionado geral é PSPACE-completo; a estrutura do enunciado é a pista; a aresta é consumida, não só o vértice.
- **Euclid game.** (a, b) com a > b: tirar múltiplo de b de a. Se ⌊a/b⌋ ≥ 2 quem joga ganha (escolhe entre terminar em a mod b ou a mod b + b); senão o lance é forçado → simule Euclides e conte os passos forçados; a paridade decide, mas depende do terminal: quociente ≥ 2 (ganha quem move) ou b = 0 (perde quem move).
  - *Gatilho:* (a,b); subtrair múltiplo de b; "Euclid game".
  - *Por que funciona:* se ⌊a/b⌋ ≥ 2, o jogador pode escolher entre (b, a mod b) e (b, a mod b + b); um dos dois é P, então o jogador ganha; se ⌊a/b⌋ = 1, o único lance é forçado para (b, a−b).
  - *Como fazer:* rode Euclides; enquanto o quociente for 1, o lance é forçado; no primeiro quociente ≥ 2, quem move ganha (escolhe a continuação); a paridade dos passos forçados decide; pare em b = 0 (quem move perde).
  - *Exemplo:* (5,2): ⌊5/2⌋ = 2 → ganha quem move; (4,3): quociente 1 → (3,1), que tem quociente 3 → o adversário ganha, então (4,3) perde.
  - *Armadilha:* a paridade depende do terminal; quociente ≥ 2 é vitória imediata no estado; b = 0 é derrota de quem move; não conte o lance forçado errado.
- **Trie de palavras (jogo de prefixos).** Jogadores alternam acrescentando uma letra mantendo prefixo de alguma palavra; quem não pode perde. Calcule em cada nó win(v) e lose(v) (pode forçar perder?). Com k partidas em que o perdedor começa a próxima: se pode forçar ganhar e perder → 1º ganha; só ganhar → vence ⇔ k ímpar; nenhum → 2º.
  - *Gatilho:* acrescentar letra; prefixo de palavra; trie; k partidas com o perdedor começando a próxima.
  - *Por que funciona:* o estado é o nó; win(v) = existe filho com win = false; lose(v) = existe filho com win = true (o jogador pode escolher perder); no meta-jogo, o perdedor de uma partida começa a seguinte.
  - *Como fazer:* construa a trie; calcule win/lose de baixo para cima; no nó raiz: win e lose → o 1º ganha; só win → ganha sse k ímpar; nem win (só lose) → o 2º ganha.
  - *Exemplo:* trie de "a": a folha tem win = lose = false; a raiz tem win = true, lose = false → só ganhar; k ímpar.
  - *Armadilha:* win e lose não são complementares; "forçar perder" é uma escolha; a regra de quem começa a próxima partida é o que dá o meta-jogo.
- **Jogo com bitmask de itens usados.** "Escolher números distintos de 1..n até a soma passar T" → estado = máscara de usados (a vez vem do popcount, e a soma da própria máscara). n ≤ 20, dp[mask] ∈ {vence, perde}.
  - *Gatilho:* subconjunto de 1..n; soma até passar T; n ≤ 20.
  - *Por que funciona:* a máscara determina os usados; o popcount determina a vez; a soma é a soma dos bits; o jogo é acíclico.
  - *Como fazer:* dp[mask] = vence/perde; para cada i não usado tal que a soma ainda não passou de T (ou o lance que passa), transicione; a vez é popcount(mask) mod 2; O(2^n·n).
  - *Exemplo:* n = 3, T = 4: teste os subconjuntos e a paridade.
  - *Armadilha:* definir exatamente quando a soma "passa" e quem perde; a memória 2^n; pré-compute a soma.
- **Memoização com hash em jogos de busca.** Tabela de transposição (tabuleiro → resultado) e ordenar lances (melhor primeiro) fazem alpha-beta cortar a maior parte; ainda é exponencial, então só vale com poucos lances ou profundidade ≤ ~20.
  - *Gatilho:* estados repetidos; busca adversarial; tabuleiro.
  - *Por que funciona:* a tabela de transposição evita reexpandir o mesmo estado; ordenar lances melhora a poda.
  - *Como fazer:* hash Zobrist do tabuleiro; guarde o resultado (e a profundidade); ordene lances (melhor primeiro); ainda exponencial.
  - *Exemplo:* damas/xadrez pequeno; jogos de tabuleiro.
  - *Armadilha:* colisões; a profundidade faz parte da chave; só vale com poucos lances/profundidade ≤ ~20.

- **Alpha-beta e minimax.** minimax com profundidade fixa e retorno em "diferença de pontos"; alpha-beta reduz b^d para ≈ b^(d/2) com boa ordenação. Em problemas de ICPC o estado costuma ser pequeno o bastante para memo puro.
  - *Gatilho:* soma zero; árvore de busca; profundidade fixa.
  - *Por que funciona:* minimax; alpha-beta poda ramos que não podem mudar o resultado; com boa ordenação, o fator efetivo cai de b para ~√b, ou seja, b^d → ~b^(d/2).
  - *Como fazer:* dfs(alpha, beta); max/min; corte quando alpha ≥ beta; avaliação heurística na folha; ordenação.
  - *Exemplo:* b = 10, d = 6 → ~10^3 com ordenação perfeita.
  - *Armadilha:* a avaliação; a ordenação; em ICPC o estado costuma ser pequeno o bastante para memo puro.
- **Pegar das pontas com n par.** O 1º pode garantir todas as posições pares ou todas as ímpares → nunca perde (empata ou ganha); decida pelo maior somatório. Para n ímpar ou regra diferente → dp[l][r] em $O(n^2)$, com array 1D rolando se a memória for curta.
  - *Gatilho:* pegar das pontas; n par; garantir soma.
  - *Por que funciona:* com n par, as pontas têm paridades opostas; o primeiro escolhe a paridade que quer e, após cada lance do adversário, pega a outra ponta da mesma paridade, garantindo todas as posições pares ou todas as ímpares.
  - *Como fazer:* some as posições de índice par e as de índice ímpar; escolha a maior; pegue a ponta correspondente; responda sempre do lado que mantém a paridade; para n ímpar ou outra regra, dp[l][r].
  - *Exemplo:* [1,5,2,4]: índices 0,2 somam 3; 1,3 somam 9; pegue a direita (4) e garanta 9.
  - *Armadilha:* indexação 0/1; n ímpar quebra a garantia; "pegar k" muda o invariante.
- **Diferença como DP.** dp[l][r] = max(a[l] − dp[l+1][r], a[r] − dp[l][r−1]): a mesma recorrência vale para qualquer jogo de soma zero com dois jogadores alternados (subtraia o valor do adversário). Evite guardar dois valores.
  - *Gatilho:* jogo de soma zero; alternar; pegar das pontas.
  - *Por que funciona:* dp[l][r] = melhor diferença para quem joga; se pega a[l], o adversário fica com dp[l+1][r], então a diferença é a[l] − dp[l+1][r]; idem à direita; o máximo dos dois.
  - *Como fazer:* preencha por comprimento crescente; resposta dp[0][n−1]; $O(n^2)$; 1D rolando (dp[l] depende de l+1 e l).
  - *Exemplo:* [1,5,2,4]: dp[0][3] = max(1 − dp[1][3], 4 − dp[0][2]).
  - *Armadilha:* guardar dois valores é desnecessário; a diferença pode ser negativa; a paridade de n muda a estratégia mas não a recorrência.
- **Jogos de soma não zero / multi-jogador.** Sem Sprague-Grundy; cada estado vira vetor de payoffs e cada um maximiza a própria coordenada (backward induction). Empates de desempate entram na ordem da comparação, defina-os no enunciado.
  - *Gatilho:* mais de dois jogadores; payoffs não opostos.
  - *Por que funciona:* sem soma zero não há Grundy; o estado é um vetor de payoffs; backward induction: cada jogador maximiza a própria coordenada.
  - *Como fazer:* dp[estado] = vetor; no estado com jogador i, escolha o lance que maximiza a coordenada i; propague; defina os desempates.
  - *Exemplo:* 3 jogadores; votação/leilão.
  - *Armadilha:* desempates mudam o resultado; não há xor; o número de estados.
- **Jogos probabilísticos.** dp[estado] = probabilidade de vencer quem joga; em estados com ciclo e só acaso (sem escolhas) vira sistema linear (Gauss); com escolhas max/min a equação de Bellman não é linear → iteração de valores ou LP. Se sem ciclo, DP de trás pra frente. Ambos escolhem lances: expectiminimax.
  - *Gatilho:* dado/moeda; escolhas e acaso.
  - *Por que funciona:* dp = probabilidade de vencer; sem ciclo, DP; com ciclo e só acaso, o sistema linear das probabilidades; com max/min, Bellman não linear.
  - *Como fazer:* sem ciclo → trás para frente; ciclo só acaso → Gauss; com escolhas → iteração de valores/LP; expectiminimax (max, min, média).
  - *Exemplo:* dado; tabuleiro com casa de retorno.
  - *Armadilha:* convergência; precisão; a política ótima pode não ser única.

- **Loopy Grundy (Smith).** Com ciclos o valor de um estado é um nimber ou ∞ com conjunto de sucessores; para soma de jogos cíclicos: se algum componente é ∞ o resultado vira empate/ganho conforme os sucessores finitos. Geralmente basta BFS retrógrada e, em somas, evite o caso.
  - *Gatilho:* ciclos; soma de jogos cíclicos; empate.
  - *Por que funciona:* com ciclos, um estado pode não ter nimber; o valor é um nimber ou ∞ com conjunto de sucessores; na soma, se algum componente é ∞, o resultado é empate/ganho conforme os sucessores finitos.
  - *Como fazer:* BFS retrógrada; em somas, evite o caso ou use a teoria de Smith; para uma componente, o valor ∞ indica empate.
  - *Exemplo:* token em grafo com ciclo.
  - *Armadilha:* ∞ não entra no xor; empates; a soma de cíclicos é sutil.
- **Misère além de Nim.** Último a mover perde: a classificação difere da normal só perto do fim; teste por brute force e compare com a normal. Em Nim com pilhas todas 1, ganha quem tem número par de pilhas. Se o jogo é "tame" (comporta-se como Nim) o misère se resolve com a normal exceto quando todas as componentes são pequenas; Kayles é tame (quociente de misère de ordem 40, período 12), mas Dawson's Kayles não é conhecido como tame (comportamento wild; análise de misère em aberto): use brute force.
  - *Gatilho:* último a mover perde; misère; não é Nim.
  - *Por que funciona:* a classificação difere da normal só perto do fim; em Nim com todas as pilhas 1, a paridade decide; jogos tame têm quociente de misère finito e se resolvem como Nim exceto componentes pequenas.
  - *Como fazer:* brute force e compare com a normal; Nim all-1: par de pilhas ganha; Kayles: use o quociente de misère (ordem 40, período 12); Dawson: não é conhecido como tame, brute force.
  - *Exemplo:* Nim (1,1) misère → o primeiro ganha (o segundo leva a última).
  - *Armadilha:* a exceção all-1 é só do Nim; Kayles tame ≠ todos os jogos; Dawson wild; não generalize.
- **Jogo com passe ou lance livre.** Se alguém pode passar, ninguém pode ficar em zugzwang; vence quem consegue empatar a contagem de lances seguros → conte lances seguros por jogador.
  - *Gatilho:* passe permitido; lance livre; zugzwang.
  - *Por que funciona:* com passe, ninguém é obrigado a jogar em posição ruim; o jogo vira uma disputa de paridade de lances seguros; quem consegue igualar a contagem do adversário vence.
  - *Como fazer:* conte os lances seguros de cada jogador; se o adversário pode passar, o valor de cada lance seguro; empate a contagem; simule.
  - *Exemplo:* jogo de peças em que passar é permitido.
  - *Armadilha:* passe ilimitado pode dar empate; a regra exata do passe (uma vez? sempre?) muda tudo.
- **Jogo interativo adversarial.** O interator escolhe a resposta mais cruel consistente; mantenha o conjunto de candidatos possíveis, e a melhor pergunta é a que divide o pior caso ao meio (ou em três, se a resposta tem 3 valores). Prove limite inferior contando folhas da árvore de decisão: $\lceil \log_3 N \rceil$ ou $\lceil \log_2 N \rceil$.
  - *Gatilho:* interator adversarial; query; adivinhar; orçamento.
  - *Por que funciona:* o adversário escolhe a resposta mais cruel consistente com o conjunto; a melhor pergunta minimiza o maior subconjunto; o limite inferior vem das folhas da árvore de decisão.
  - *Como fazer:* mantenha o conjunto de candidatos; escolha a query que divide o pior caso ao meio (ou em três); atualize; prove $\lceil \log_2 N \rceil$ ou $\lceil \log_3 N \rceil$.
  - *Exemplo:* adivinhar número com <, =, >.
  - *Armadilha:* respostas inválidas; o adversário adaptativo; o orçamento inclui a verificação final.
- **Alice/Bob em strings.** Se cada um escolhe um caractere para formar a string final (um quer a menor, outro a maior), o ótimo é guloso dos extremos, com uma virada (CF 794C): enquanto o menor de Alice < o maior de Bob, cada um põe o seu melhor na primeira posição livre; quando deixa de valer, cada um passa a pôr o seu pior na última posição livre. Se há simetria entre posições → espelho.
  - *Gatilho:* cada um escolhe caractere; formar string; um minimiza, outro maximiza.
  - *Por que funciona:* enquanto o menor de Alice < o maior de Bob, vale a pena cada um colocar seu melhor na frente; quando não vale, colocar o pior no fim é melhor; a virada é ótima.
  - *Como fazer:* use só as $\lceil n/2 \rceil$ menores de Alice e as $\lfloor n/2 \rfloor$ maiores de Bob; ordene Alice crescente e Bob decrescente; dois ponteiros para a primeira e a última posição livre; no turno de Alice, se alice[0] < bob[0], ponha alice[0] na frente, senão ponha alice.back() no fim; no turno de Bob, simétrico (bob[0] > alice[0] → frente, senão bob.back() no fim).
  - *Exemplo:* CF 794C.
  - *Armadilha:* a virada; o número ímpar de letras; empates; a posição escolhida.

- **Nim com pilha inicial grande e k pequeno.** Tabule g(n) para n ≤ 200 e olhe a parte periódica; o que escapa do período vem do início (pré-período).
  - *Gatilho:* uma pilha grande; k pequeno; n enorme.
  - *Por que funciona:* para "tirar 1..k" o Grundy é n mod (k+1); para regras de subtração finitas, a sequência é eventualmente periódica; o pré-período vem do início.
  - *Como fazer:* tabule g(n) até 200; detecte o período; aplique n mod período respeitando o pré-período; para 1..k, responda n mod (k+1) ≠ 0.
  - *Exemplo:* k = 3 → período 4; n = 6 → 2 ≠ 0 → o primeiro ganha.
  - *Armadilha:* o pré-período; k diferente muda o período; não confunda com Moore (até k pilhas).

Repetidas de outras seções:

- **Estratégia de pareamento/espelho** em construtivos interativos: se você escolhe ser o segundo jogador, responda ao par do lance do juiz.
- **Wythoff** → posições perdedoras (⌊kφ⌋, ⌊kφ²⌋), φ = (1+√5)/2; Fibonacci Nim: retirada ≤ 2× a anterior, perdedores = Fibonacci (Zeckendorf).
- **Moore's Nim_k e Nim com limite.** Tirar até k de uma pilha: Grundy = a mod (k+1); tirar de até k pilhas: some cada bit mod (k+1), posição perdedora se todos 0.

Geometria

Use inteiros sempre que puder: produto vetorial e escalar resolvem quase tudo sem ponto flutuante.

- **Orientação, interseção, área** → produto vetorial (cross); área de polígono → shoelace.
  - *Gatilho:* "sentido horário/anti-horário", "os segmentos se cruzam", "área do polígono", "o ponto está dentro".
  - *Por que funciona:* cross(b − a, c − a) é duas vezes a área sinalizada do triângulo abc; o sinal não usa divisão e é exato em inteiros. Shoelace soma trapézios sinalizados e as partes externas se cancelam.
  - *Como fazer:* cross = (b.x − a.x)(c.y − a.y) − (b.y − a.y)(c.x − a.x); área2 = |Σ (x_i·y_{i+1} − x_{i+1}·y_i)|; guarde 2·área inteira.
  - *Exemplo:* (0,0), (4,0), (0,3) → cross = 12 → área 6; quadrado unitário → área2 = 2.
  - *Armadilha:* com coordenadas 10^9 o cross chega a $8 \cdot 10^{18}$ (quase o limite do int64); somas de vários cross pedem __int128. Colinear dá 0 e precisa de desempate explícito.
- **Pontos extremos, maior distância** → fecho convexo + rotating calipers.
  - *Gatilho:* "par mais distante", "diâmetro", "largura", "retângulo envolvente mínimo", "ponto mais extremo numa direção".
  - *Por que funciona:* o máximo de uma função linear sobre um conjunto finito é atingido num vértice do hull; os dois extremos do diâmetro são vértices. Os calipers aproveitam que o vértice antípoda gira monotonicamente.
  - *Como fazer:* Andrew monotone chain O(n log n), depois calipers O(h) percorrendo arestas e vértices antípodas. Ver **Rotating calipers**.
  - *Exemplo:* nuvem de 10^5 pontos → hull com dezenas de vértices → diâmetro em O(h).
  - *Armadilha:* hull de 1 ou 2 pontos quebra os calipers; compare distâncias ao quadrado para não usar sqrt.
- **Muitos pontos e uma reta/direção** → ordene por ângulo (sweep angular).
  - *Gatilho:* "quantos pares são separados por uma reta", "triângulos que contêm a origem", "gire uma direção".
  - *Por que funciona:* fixado um pivô, só a ordem angular importa; um semiplano vira um arco contíguo de comprimento π na ordem circular, e dois ponteiros contam sem reinserir pontos.
  - *Como fazer:* ordene os vetores por semiplano + cross; duplique a lista somando 2π; para cada i avance j enquanto o ângulo < θ_i + π; $O(n^2 \log n)$ sobre todos os pivôs.
  - *Exemplo:* máximo de pontos numa reta que passa por um ponto fixo; contagem de triângulos que contêm a origem.
  - *Armadilha:* pontos colineares com o pivô (mesmo ângulo) precisam ser agrupados ou desempatados por distância; decida se a fronteira do semiplano conta.
- **Manhattan** → rotação (x + y, x − y); **pontos em grid** → Pick (A = I + B/2 − 1).
  - *Gatilho:* distâncias L1, vizinho mais próximo/mais distante em Manhattan; polígono com vértices inteiros e contagem de pontos interiores.
  - *Por que funciona:* a rotação leva L1 em L∞, então |dx| + |dy| = max(|Δu|, |Δv|). Pick relaciona exatamente área, pontos de fronteira e pontos interiores de um polígono reticulado simples.
  - *Como fazer:* u = x + y, v = x − y; B = Σ gcd(|dx|, |dy|) sobre as arestas; I = A − B/2 + 1, usando 2A para não fracionar.
  - *Exemplo:* (0,0) e (3,4): L1 = 7; em (u,v) viram (0,0) e (7,−1) → max(7,1) = 7.
  - *Armadilha:* a rotação dobra distâncias e troca paridade (u e v têm a mesma paridade); Pick exige polígono simples e vértices inteiros; A pode ser meio-inteiro.

- **Pares mais próximos** → divide e conquista ou grid de hash.
  - *Gatilho:* "par mais próximo", "existe par a distância ≤ d", detecção de colisão.
  - *Por que funciona:* na faixa de largura 2d do D&C, cada retângulo d×2d contém no máximo 7 pontos, então cada ponto compara com poucos vizinhos; no grid de célula d, cada célula tem O(1) pontos.
  - *Como fazer:* ordene por x, recurse, ordene a faixa por y e compare os próximos ≤ 7; ou hash de células de lado d e reconstrua quando d diminui (amortizado com ordem aleatória). O(n log n) / O(n) esperado.
  - *Exemplo:* 10^5 pontos → par mais próximo.
  - *Armadilha:* compare d^2 em inteiros; pontos coincidentes dão distância 0; no grid, a reconstrução é o que garante o amortizado.
- **Interseções de muitos retângulos/segmentos** → sweep line com segtree.
  - *Gatilho:* união de área/perímetro, cobertura, contagem de cruzamentos.
  - *Por que funciona:* entre dois eventos de x o conjunto ativo não muda, então o comprimento de y coberto é constante no intervalo; a segtree mantém esse comprimento em O(log n) por evento.
  - *Como fazer:* eventos (x, tipo, y1, y2), comprima y, segtree com (cobertura, comprimento coberto); área += comprimento·Δx; O(n log n). Ver **União de retângulos (área)**.
  - *Exemplo:* união de 10^5 retângulos axis-alinhados.
  - *Armadilha:* processe todos os eventos do mesmo x juntos; intervalos half-open; área em long long.
- **Precisão:** compare com eps, evite sqrt e atan2 quando der, use long double.
  - *Gatilho:* coordenadas reais, ângulos, distâncias, ou resposta que depende de comparação geométrica.
  - *Por que funciona:* erro de arredondamento cresce com as operações e o sinal de uma expressão quase nula é instável; predicados inteiros (cross, dot) decidem sem erro.
  - *Como fazer:* use cross inteiro para orientação; eps relativo $|a - b| \le \text{eps}\cdot\max(1,|a|,|b|)$; deixe sqrt para a saída; long double para acumular; evite subtrair números grandes e próximos.
  - *Exemplo:* testar interseção de círculos comparando d^2 com $(r_1 + r_2)^2$, não d com $r_1 + r_2$.
  - *Armadilha:* eps absoluto falha em magnitudes grandes; eps grande demais funde pontos distintos; sqrt de negativo pequeno e acos fora de [−1,1] viram NaN.

adicionado em 2026-11-10 por agentes; revisado

- **Cross como orientação.** cross(b − a, c − a) > 0 ⇒ anti-horário; = 0 ⇒ colinear; o sinal decide tudo sem divisões. Com coordenadas até 10^9 as diferenças chegam a $2\cdot10^9$ e o cross a $2\cdot(2\cdot10^9)^2 \approx 8\cdot10^{18}$, que ainda cabe em int64 (perto do limite); com 10^{18} use __int128.
  - *Gatilho:* "à esquerda/direita", "sentido da curva", "os três pontos são colineares", "ponto dentro".
  - *Por que funciona:* o cross é duas vezes a área sinalizada do triângulo; o sinal é exato em inteiros e independe de escala positiva; zero é exatamente colinear.
  - *Como fazer:* calcule com as diferenças a partir de a; > 0 anti-horário (com y para cima), < 0 horário, = 0 colinear. Compare com 0, não com eps, se as coordenadas são inteiras.
  - *Exemplo:* (0,0), (1,0), (0,1) → cross = 1 > 0, anti-horário.
  - *Armadilha:* $8\cdot10^{18}$ é perto de $2^{63} - 1 \approx 9.22\cdot10^{18}$; a diferença de dois cross ou uma soma de vários estoura. O sinal depende da ordem dos três pontos.
- **Dot como ângulo / projeção.** dot > 0 agudo, = 0 reto, < 0 obtuso; projeção de p em reta ab devolve o parâmetro $t = \text{dot}(p - a, b - a)/|b - a|^2$; pé foot = a + t·(b − a); reflexão de p = 2·foot − p.
  - *Gatilho:* perpendicularidade, "ângulo menor que 90°", projeção, ponto mais próximo numa reta, reflexão.
  - *Por que funciona:* $\text{dot} = |u||v|\cos\theta$; o ponto da reta mais próximo de p é a projeção ortogonal, e a reflexão é o ponto simétrico em relação ao pé.
  - *Como fazer:* t é razão de dois escalares; para segmento, limite t em [0,1]; para comparar projeções sem dividir, compare numeradores com denominador positivo.
  - *Exemplo:* distância de (0,0) à reta por (1,0) e (1,2): t = 0, pé = (1,0), distância 1.
  - *Armadilha:* a = b deixa $|b - a|^2 = 0$; dot só distingue ≤ 90° (não dá o ângulo); estouro de int64 no dot com 10^9.

- **Cross e dot juntos dão ângulo orientado.** atan2(cross, dot) ∈ (−π, π]; evite atan2 como chave de ordenação: use meio-plano (y > 0 ou y = 0 e x > 0) + cross como comparador exato.
  - *Gatilho:* ângulo entre vetores, ordenar por ângulo, comparar dois ângulos sem trigonometria.
  - *Por que funciona:* (dot, cross) são cosseno e seno escalados por |u||v|; atan2 recupera o ângulo, mas a ordenação só precisa do sinal do cross dentro do mesmo meio-plano.
  - *Como fazer:* para ordenar, primeiro meio-plano, depois cross > 0; para o valor do ângulo, atan2 uma vez no fim.
  - *Exemplo:* ordenar vetores em torno do pivô sem calcular ângulos.
  - *Armadilha:* atan2 tem erro ~10^{-16} e a fronteira ±π é ambígua; colineares opostos caem em meio-planos diferentes; use cross inteiro quando as coordenadas são inteiras.
- **Ordenação por ângulo exata.** Divida em semiplanos e compare por cross > 0; pontos colineares na mesma direção: desempate por distância. Evita erro de atan2 em coordenadas grandes.
  - *Gatilho:* ordenação radial, sweep angular, "gire em torno do ponto".
  - *Por que funciona:* dentro de um semiplano de abertura π, a ordem angular é exatamente a ordem do sinal do cross; não há descontinuidade nem transcendental.
  - *Como fazer:* half(v) = 0 se (v.y > 0 ou (v.y = 0 e v.x > 0)), senão 1; se half difere, menor vem antes; senão cross(u,v) > 0; empate por $|v|^2$.
  - *Exemplo:* ordenar 10^5 pontos ao redor da origem com coordenadas 10^9.
  - *Armadilha:* vetor nulo não tem ângulo; colineares opostos ficam em semiplanos distintos; o comparador precisa ser strict weak ordering (devolva false no empate).
- **Interseção de segmentos (inteira).** Segmentos se cruzam se o1·o2 ≤ 0 e o3·o4 ≤ 0, onde o1, o2 são as orientações de (a, b, c) e (a, b, d) e o3, o4 as de (c, d, a) e (c, d, b); quando algum oi = 0 cheque onSegment (ponto sobre o outro segmento); caso colinear: cheque sobreposição das caixas (intervalo de projeção).
  - *Gatilho:* "os segmentos se intersectam", extremos compartilhados, toque em endpoint, casos degenerados.
  - *Por que funciona:* dois segmentos se cruzam exatamente quando cada um tem extremos dos dois lados (ou sobre) a reta do outro; o caso colinear exige checagem separada porque os quatro cross são zero.
  - *Como fazer:* calcule o1..o4; se o1·o2 < 0 e o3·o4 < 0, cruzamento próprio; se algum zero, teste se o ponto está na caixa do outro segmento; se todos zero, teste sobreposição 1D das projeções.
  - *Exemplo:* (0,0)-(2,2) e (0,2)-(2,0) se cruzam em (1,1); (0,0)-(1,0) e (2,0)-(3,0) não.
  - *Armadilha:* o produto o1·o2 pode estourar; compare sinais separadamente ou use __int128. Só testar lados opostos perde toque e colinearidade.
- **Ponto de interseção de retas** como a + t·d com t = cross(c − a, e)/cross(d, e); mantenha frações (num/den) em inteiros para comparar sem erro.
  - *Gatilho:* "onde as retas se cruzam", comparar a interseção com pontos/retas, interseção de diagonais.
  - *Por que funciona:* igualando a + t·d = c + s·e e aplicando cross com e, o termo s desaparece e t fica como razão de dois cross; a razão é exata.
  - *Como fazer:* num = cross(c − a, e), den = cross(d, e); normalize den > 0; guarde (num, den) e compare por multiplicação cruzada em __int128. Para o ponto, divida só na saída.
  - *Exemplo:* diagonais de um quadrilátero se cruzam no ponto médio de ambas quando é paralelogramo.
  - *Armadilha:* den = 0 (paralelas ou coincidentes); usar double perde exatidão ao comparar com coordenadas inteiras; den negativo inverte a ordem.
- **Ponto em polígono.** Convexo: busca binária por ângulo a partir de v_0 em O(log n); qualquer polígono: contagem de cruzamentos de raio (cuide de vértices no raio com a regra "y_a ≤ y < y_b") ou winding number.
  - *Gatilho:* "está dentro do polígono", muitas consultas, polígono convexo vs arbitrário.
  - *Por que funciona:* num convexo, o polígono é um leque de triângulos a partir de v_0; num polígono simples, um raio cruza a fronteira um número ímpar de vezes se e só se o ponto é interior (Jordan).
  - *Como fazer:* convexo: localize o setor por busca binária e teste os sinais de cross O(log n); geral: raio horizontal, conte arestas com y_a ≤ y < y_b e x de interseção > x; ímpar ⇒ dentro.
  - *Exemplo:* (1,1) no quadrado (0,0)-(2,2) → dentro.
  - *Armadilha:* a regra half-open evita contar duas vezes um vértice no raio; ponto na aresta precisa de teste explícito; interseção do raio com aresta é fração, não float.

- **Área e sinal do polígono.** Shoelace dá área sinalizada: positivo ⇒ anti-horário. Área de triângulo = |cross|/2; guarde 2·área em inteiros.
  - *Gatilho:* área de polígono, orientação dos vértices, soma de áreas com sinal.
  - *Por que funciona:* Σ cross(p_i, p_{i+1})/2 é a soma de trapézios sinalizados; para polígono simples, as contribuições externas se cancelam e sobra a área.
  - *Como fazer:* area2 = Σ (x_i·y_{i+1} − x_{i+1}·y_i), com p_n = p_0; positivo se os vértices estão em sentido anti-horário.
  - *Exemplo:* quadrado unitário → area2 = 2, área = 1; triângulo (0,0),(4,0),(0,3) → 6.
  - *Armadilha:* inverter a ordem inverte o sinal; somas com coordenadas 10^9 e n grande pedem __int128; esquecer de fechar o polígono.
- **Centroide de polígono.** C = Σ (p_i + p_{i+1})·cross(p_i, p_{i+1}) / (6·A); decomponha em triângulos com área sinalizada. Centroide de pontos com massas = média ponderada.
  - *Gatilho:* centro de massa de região poligonal, balanceamento, média ponderada por área.
  - *Por que funciona:* decompondo em triângulos a partir da origem, cada centroide é a média dos três vértices e o peso é a área sinalizada; partes fora do polígono se cancelam.
  - *Como fazer:* acumule numx += (x_i + x_{i+1})·cross_i e idem y; divida por 6A no fim; A é a área sinalizada.
  - *Exemplo:* triângulo → média aritmética dos três vértices.
  - *Armadilha:* A = 0 (degenerado); usar áreas absolutas em polígono não convexo conta em dobro; a saída é racional.
- **Teorema de Pick.** A = I + B/2 − 1, com B = Σ gcd(|dx|, |dy|) sobre as arestas; resolve "pontos interiores de polígono em grid" e "contar pontos inteiros num segmento" (gcd + 1).
  - *Gatilho:* polígono com vértices inteiros, "quantos pontos interiores", "quantos pontos na fronteira", pontos num segmento.
  - *Por que funciona:* para polígono reticulado simples, área, pontos interiores e de fronteira satisfazem uma relação linear exata; cada aresta contribui gcd(|dx|,|dy|) pontos de fronteira.
  - *Como fazer:* B = Σ gcd; I = A − B/2 + 1; use 2A inteiro para evitar meio.
  - *Exemplo:* triângulo (0,0),(4,0),(0,4): B = 4 + 4 + 4 = 12, A = 8, I = 8 − 6 + 1 = 3 (pontos (1,1),(1,2),(2,1)).
  - *Armadilha:* Pick exige polígono simples e vértices inteiros; autointerseção quebra; A meio-inteiro exige 2A.
- **Pontos de grid em triângulo/reta.** Σ ⌊(a·i + b)/c⌋ em O(log) (floor sum) conta pontos inteiros abaixo de uma reta; use para pontos em triângulo rectângulo/trapézio sem iterar.
  - *Gatilho:* "quantos pontos inteiros satisfazem ax + by ≤ c", triângulo/trapézio com coordenadas grandes.
  - *Por que funciona:* em cada coluna i há ⌊(a·i + b)/c⌋ + 1 pontos; somar floors é uma recorrência euclidiana que reduz (a,c) como no gcd.
  - *Como fazer:* floorsum(n, m, a, b) = Σ_{i=0}^{n−1} ⌊(a·i + b)/m⌋; reduza a e b por quociente/resto e troque os papéis de a e m; O(log).
  - *Exemplo:* pontos x,y ≥ 0 com 2x + 3y ≤ 10.
  - *Armadilha:* a ou b negativos exigem normalização; fronteira inclusiva vs estrita muda o resultado em 1; a·n pode estourar.
- **Fecho convexo (Andrew monotone chain).** Ordene por (x, y), construa inferior e superior com pop enquanto cross ≤ 0 (≤ para remover colineares, < para manter). Cuidado com todos os pontos colineares e duplicados.
  - *Gatilho:* fecho convexo, pontos extremos, reduzir o conjunto antes de calipers/queries.
  - *Por que funciona:* com os pontos ordenados por x, cada cadeia é construída mantendo apenas curvas consistentes; um ponto que faz uma volta à direita nunca pertence à cadeia; cada ponto entra e sai uma vez.
  - *Como fazer:* remova duplicados; para lower e upper, enquanto size ≥ 2 e cross(penúltimo, último, novo) ≤ 0, pop; concatene sem repetir as pontas; O(n log n) pelo sort.
  - *Exemplo:* quadrado com um ponto interior → 4 vértices no hull.
  - *Armadilha:* todos colineares → hull de 2 pontos; duplicados quebram a cadeia; ≤ vs < decide se colineares ficam no hull; a orientação final é anti-horária.
- **Fecho convexo é a estrutura do "máximo de função linear".** max a·x + b·y sobre pontos = vértice do hull; sempre reduza o conjunto aos vértices do hull. Queries de direção → busca binária no hull.
  - *Gatilho:* maximizar/minimizar combinação linear sobre pontos, "ponto mais extremo numa direção", support function.
  - *Por que funciona:* uma função linear não tem máximo interior; ao longo do hull os valores são unimodais, então a busca binária acha o vértice ótimo.
  - *Como fazer:* construa o hull e, para cada direção, busque o vértice que maximiza dot; ou use calipers para varrer todas as direções.
  - *Exemplo:* ponto do conjunto mais distante na direção (1,0) é o de maior x (empate no hull).
  - *Armadilha:* aresta inteira pode empatar (dois vértices com o mesmo valor); hull degenerado de 1–2 pontos; comparar valores em inteiros, não em double.

- **Rotating calipers.** Diâmetro (par mais distante), largura mínima, retângulo envolvente de área/perímetro mínimo, distância mínima entre dois polígonos convexos; dois ponteiros que giram monotonicamente em O(n).
  - *Gatilho:* medidas de um convexo: diâmetro, largura, caixa mínima, distância entre convexos.
  - *Por que funciona:* quando a direção de suporte gira, o vértice antípoda só avança; por isso os dois ponteiros dão uma volta completa cada e o total é linear.
  - *Como fazer:* para cada aresta, avance o vértice j enquanto a área do triângulo (i, i+1, j) cresce; avalie as distâncias/áreas candidatas; O(h).
  - *Exemplo:* diâmetro de um polígono convexo com 10^5 vértices.
  - *Armadilha:* vértices colineares no hull; h < 3; compare alturas/áreas com cross inteiro, não com float.
- **Diâmetro de nuvem de pontos = diâmetro do hull.** Só existem O(h) candidatos; com hull pequeno (pontos aleatórios) o hull tem O(log n)–O(n^(1/3)) vértices em média; vale a aproximação por brute force.
  - *Gatilho:* par mais distante em n grande, pontos aleatórios ou hull pequeno.
  - *Por que funciona:* os extremos do diâmetro são vértices do hull; se o hull tem poucos vértices, testar todos os pares do hull é barato.
  - *Como fazer:* hull O(n log n); se h é pequeno, $O(h^2)$ com distâncias ao quadrado; se h é grande, calipers O(h).
  - *Exemplo:* 10^5 pontos aleatórios num quadrado → h da ordem de dezenas → brute force no hull.
  - *Armadilha:* pior caso com todos os pontos numa circunferência → h = n, calipers obrigatórios; distância ao quadrado com coordenadas 10^9 chega a $8 \cdot 10^{18}$ (cabe, mas some com cuidado).
- **Soma de Minkowski de convexos.** Mescle as arestas ordenadas por ângulo começando pelos vértices mais baixo-esquerdos: O(n + m). Dois convexos intersectam ⇔ origem ∈ A ⊕ (−B); distância entre convexos idem.
  - *Gatilho:* soma de conjuntos convexos, colisão de formas por translação, alcance com vetores.
  - *Por que funciona:* a fronteira da soma é o merge das sequências de arestas ordenadas por ângulo; a interseção de A e B é equivalente a 0 ∈ A ⊕ (−B).
  - *Como fazer:* comece nos vértices de menor (y, x); em cada passo escolha a aresta de menor ângulo e avance; arestas de mesmo ângulo devem ser somadas; O(n + m).
  - *Exemplo:* dois quadrados → soma é um octógono (arestas paralelas se fundem).
  - *Armadilha:* ângulos iguais não podem virar duas arestas separadas; os dois polígonos precisam estar no mesmo sentido (CCW); segmentos degenerados.
- **Minkowski e "mover o objeto".** Colisão de polígono convexo em translação = ponto em Minkowski(A, −B); "alcançável com k passos de vetores" = soma de Minkowski de k segmentos (zonótopo: arestas ordenadas por ângulo).
  - *Gatilho:* "o objeto pode ser transladado sem colidir", "alcance com k passos", configurações de robô.
  - *Por que funciona:* o conjunto de translações que evitam B é o complemento de B ⊕ (−A) (ou A ⊕ (−B) com sinal conveniente); a soma de k segmentos é um zonótopo cuja fronteira tem 2k arestas ordenadas por ângulo.
  - *Como fazer:* construa a soma de Minkowski e teste pertinência do vetor de translação; para k segmentos, ordene as 2k direções e mescle; O(k log k).
  - *Exemplo:* robô poligonal desviando de obstáculos convexos.
  - *Armadilha:* só vale para convexos (não convexos exigem decomposição); toque na fronteira conta como colisão conforme o enunciado; confira o sinal da translação.
- **Interseção de semiplanos (HPI).** Ordene por ângulo, mantenha deque; O(n log n). Resolve "kernel de polígono estrelado", "feasibilidade de LP 2D", "região onde todas as condições valem"; trate regiões ilimitadas com uma caixa grande e vazias via checagem do deque.
  - *Gatilho:* região que satisfaz todas as desigualdades lineares, kernel, viabilidade de LP 2D.
  - *Por que funciona:* a interseção é convexa; ordenadas por ângulo, cada reta nova só pode cortar um sufixo da fronteira atual, então a deque mantém a região em tempo linear após o sort.
  - *Como fazer:* ordene as retas por ângulo (desempate pela mais restritiva), deque de retas e pontos de interseção; ao inserir, pop na frente e atrás enquanto o ponto estiver fora; verifique se sobrou região.
  - *Exemplo:* kernel de um polígono estrelado.
  - *Armadilha:* retas paralelas (mesmo ângulo) — mantenha só a mais restritiva; desigualdade estrita vs não estrita; região ilimitada precisa de caixa delimitadora; empates numéricos.

- **LP em 2D / 3D por randomização.** Seidel: O(n) esperado em dimensão fixa; ou HPI em 2D. Gatilho: "maximize ax + by sujeito a desigualdades lineares" com poucas variáveis.
  - *Gatilho:* objetivo linear e restrições lineares, dimensão 2 ou 3, n grande.
  - *Por que funciona:* na ordem aleatória, a i-ésima restrição viola o ótimo atual com probabilidade ≤ d/i; a recorrência dá O(d!·n) esperado.
  - *Como fazer:* embaralhe; resolva recursivamente; quando uma restrição é violada, ela fica na fronteira e reduz a dimensão; em 2D pode-se usar HPI.
  - *Exemplo:* maximizar ax + by sobre um polígono definido por semiplanos.
  - *Armadilha:* casos ilimitado/inviável precisam ser detectados; degenerescência (restrições concorrentes); aritmética exata é caro — eps ou frações.
- **Par de pontos mais próximo.** D&C por x e tira de largura 2d ordenada por y (cada ponto checa ≤ 7 vizinhos), O(n log n); ou sweep com set ordenado por y; ou grid de hash com tamanho d em O(n) esperado (reconstrói o grid quando d cai, amortizado com ordem aleatória).
  - *Gatilho:* "par mais próximo", "existe par a distância ≤ d".
  - *Por que funciona:* na faixa crítica, um retângulo d×2d contém no máximo 7 pontos, então cada ponto só precisa comparar com um número constante; no grid de célula d, cada célula tem O(1) pontos.
  - *Como fazer:* D&C com merge por y; ou hash de células de lado d, reconstruindo quando d diminui; compare d^2 inteiro.
  - *Exemplo:* 10^5 pontos no plano.
  - *Armadilha:* usar sqrt para comparar; pontos iguais; a análise do grid depende da ordem aleatória para o amortizado.
- **Círculo mínimo envolvente (Welzl).** Embaralhe os pontos; incremental com 3 laços aninhados "se p fora do círculo atual, refaça com p na fronteira": O(n) esperado. Círculo por 2 pontos = diâmetro; por 3 = circuncentro.
  - *Gatilho:* menor círculo que cobre todos os pontos, menor raio de cobertura.
  - *Por que funciona:* análise backward: o i-ésimo ponto está na fronteira do ótimo com probabilidade ≤ 3/i; somando, o custo esperado é linear.
  - *Como fazer:* shuffle; mantenha o círculo; se p está fora, reinicie com círculo (p); para cada q anterior fora, círculo = diâmetro(p,q); para cada r anterior fora, círculo = circunscrito(p,q,r).
  - *Exemplo:* 3 pontos não colineares → circuncentro; 2 pontos → diâmetro.
  - *Armadilha:* pontos colineares tornam o circunscrito indefinido (use o círculo de diâmetro); duplicados; centro em double — versão exata usa frações.
- **Circuncentro e incentro.** Circuncentro: resolver 2 equações lineares de mediatrizes (não bissetoras); use denominador D = 2·cross e mantenha numeradores; compare raios com frações para evitar ponto flutuante. Incentro = (a·A + b·B + c·C)/(a + b + c) com a, b, c = lados opostos (raízes), em geral não inteiro.
  - *Gatilho:* círculo por 3 pontos, centros do triângulo, comparar raios.
  - *Por que funciona:* o circuncentro é equidistante dos três vértices, logo está na interseção de duas mediatrizes (equações lineares); o incentro é a média dos vértices ponderada pelos lados opostos.
  - *Como fazer:* monte as duas mediatrizes e resolva o sistema 2×2; guarde num/den; para o incentro, a = |BC|, b = |CA|, c = |AB| (comprimentos, em geral irracionais).
  - *Exemplo:* (0,0),(1,0),(0,1) → circuncentro (0.5, 0.5).
  - *Armadilha:* triângulo colinear D = 0; o circuncentro pode ficar fora do triângulo; incentro não é inteiro em geral; compare raios ao quadrado.
- **Teste de ponto em círculo circunscrito (in-circle).** Determinante 4×4 (ou 3×3 após subtrair) dá sinal exato com inteiros; base de Delaunay e da checagem de "círculo vazio".
  - *Gatilho:* "d está dentro do círculo por a, b, c", Delaunay, círculo vazio.
  - *Por que funciona:* o determinante é o predicado de orientação no paraboloide levantado (x, y, $x^2 + y^2$); seu sinal é exato e independe de divisões.
  - *Como fazer:* subtraia d dos outros três pontos e monte a matriz 3×3 com $|p|^2$; o sinal é positivo/negativo conforme a orientação de abc; use __int128 (grau 4).
  - *Exemplo:* quatro pontos cocirculares → determinante 0.
  - *Armadilha:* o sinal depende da ordem de abc (inverta se necessário); com coordenadas 10^9 o determinante chega a ~10^{36}; cocirculares dão 0 e precisam de desempate simbólico.

- **Delaunay / Voronoi.** Maior círculo vazio, vizinho mais próximo para todos os pontos, MST euclidiano (MST está contido em Delaunay), O(n log n); em contestes normalmente se evita implementando a MST euclidiana via Boruvka+k-d tree; os 8 octantes são da MST Manhattan, não da euclidiana.
  - *Gatilho:* vizinho mais próximo de todos, maior círculo vazio, MST euclidiano, triangulação com círculo vazio.
  - *Por que funciona:* Delaunay é o dual de Voronoi; a propriedade do círculo vazio garante que cada triângulo tem circunferência sem pontos; toda aresta da MST euclidiana é aresta de Delaunay.
  - *Como fazer:* randomized incremental O(n log n) ou D&C; para MST euclidiano em contest, Boruvka com k-d tree costuma ser mais simples; para vizinho mais próximo, k-d tree ou sweep.
  - *Exemplo:* MST euclidiano de 10^5 pontos via Boruvka + k-d tree.
  - *Armadilha:* quatro pontos cocirculares (triangulação não única); implementação de Delaunay é longa e sujeita a degenerescência; a técnica dos 8 octantes é só para L1.
- **MST Manhattan.** Para cada ponto, em cada um dos 4 octantes só o vizinho mais próximo importa (sweep + BIT de min por x+y); 4n arestas candidatas + Kruskal.
  - *Gatilho:* MST com distância L1, n grande.
  - *Por que funciona:* dentro de um octante, a distância L1 é diferença de x+y; se um ponto tem outro mais próximo no octante, os demais não podem ser melhores que essa aresta pela desigualdade triangular.
  - *Como fazer:* faça 4 varreduras com transformações de sinal/troca de eixos; BIT sobre x+y guardando o mínimo; colete as arestas; Kruskal O(n log n).
  - *Exemplo:* 10^5 pontos → ~$4 \cdot 10^5$ arestas candidatas.
  - *Armadilha:* cobrir os 8 octantes com 4 transformações; empates em x; arestas duplicadas não atrapalham o Kruskal.
- **Manhattan e Chebyshev.** |dx| + |dy| = max(|dx + dy|, |dx − dy|); maior distância Manhattan entre pontos = max sobre as 4 combinações de sinais (x ± y). Em dimensões d use 2^(d−1) máscaras de sinais.
  - *Gatilho:* maior distância L1/L∞, diâmetro em Manhattan, redução de L1 para L∞.
  - *Por que funciona:* a identidade transforma L1 em L∞; para o máximo, a distância entre dois pontos é o máximo, sobre máscaras de sinais, da diferença de uma função linear, e o máximo global é max_máscara (max_i s·p_i − min_i s·p_i).
  - *Como fazer:* para cada uma das 2^(d−1) máscaras (a máscara toda-negativa é redundante), calcule max e min de Σ s_j·x_j; a resposta é a maior diferença.
  - *Exemplo:* pontos (0,0) e (3,4): máscaras dão max(7, 1) e (1, 7) → 7.
  - *Armadilha:* a identidade é para distância entre dois pontos, não para somas; coordenadas negativas exigem cuidado com os sinais; 2^d cresce rápido.
- **Distância Manhattan com sweep.** Para o ponto mais próximo/discos Manhattan, separe os 4 quadrantes por simetria (troca de sinal) e reduza para "max/min de (x + y) com restrição de ordem" → BIT/segtree. Discos Manhattan viram quadrados rotacionados.
  - *Gatilho:* vizinho mais próximo em L1, consultas de pontos num losango/disco L1.
  - *Por que funciona:* num quadrante fixo, |dx| + |dy| = ±(dx + dy); ordenando por x, a restrição "x' ≤ x" vira consulta de prefixo, e a distância vira diferença de x+y.
  - *Como fazer:* 4 varreduras com (x, y) transformados por sinais; BIT/segtree sobre x guardando max(x+y) ou min(−x−y); O(n log n).
  - *Exemplo:* par mais próximo em Manhattan entre 10^5 pontos.
  - *Armadilha:* cada par aparece em um quadrante só; empates de x precisam de ordem consistente; um disco L1 é um quadrado rotacionado, não um círculo.
- **Distância ponto-segmento.** Projete: t = dot(p − a, b − a)/|b − a|², limite t em [0, 1]; distância entre dois segmentos disjuntos = mínimo das 4 distâncias ponto-segmento (se não se cruzam).
  - *Gatilho:* distância de ponto a segmento/polilinha, distância entre segmentos.
  - *Por que funciona:* o ponto mais próximo do segmento é a projeção se ela cai dentro; senão é uma das extremidades; o clamp cobre os dois casos.
  - *Como fazer:* calcule t, limite, foot = a + t(b − a), distância |p − foot|; para dois segmentos, teste interseção primeiro e depois os 4 casos ponto-segmento.
  - *Exemplo:* (0,0) ao segmento (1,−1)-(1,1) → distância 1.
  - *Armadilha:* segmento degenerado a = b; comparar distâncias com sqrt; se os segmentos se cruzam a distância é 0.

- **Distância mínima entre segmentos e polígonos convexos.** Mínimo das 4 distâncias ponto-aresta é suficiente quando não se tocam; para convexos, calipers.
  - *Gatilho:* menor distância entre dois segmentos/polígonos que não se cruzam.
  - *Por que funciona:* se dois segmentos são disjuntos, o par mais próximo envolve pelo menos uma extremidade; para convexos, a distância é realizada entre arestas/vértices antípodas.
  - *Como fazer:* teste interseção; senão min das 4 distâncias ponto-segmento; polígonos convexos → rotating calipers O(n + m).
  - *Exemplo:* dois quadrados separados → distância entre um vértice e uma aresta.
  - *Armadilha:* toque/sobreposição dá 0; arestas paralelas; testar as 4 direções (não só uma).
- **Círculo ∩ reta / círculo ∩ círculo.** Reduza a distância centro-reta $h^2 = r^2 - d^2$; círculo-círculo: $a = (d^2 + r_1^2 - r_2^2)/(2d)$, $h = \sqrt{(r_1^2 - a^2)}$. Teste de interseção com inteiros comparando d^2 com $(r_1 \pm r_2)^2$.
  - *Gatilho:* interseção/tangência de círculos e retas, pontos de corte.
  - *Por que funciona:* a reta a distância d do centro corta uma corda cujo meio é o pé da perpendicular e cuja metade é $\sqrt{(r^2 - d^2)}$; no caso círculo-círculo, a reta dos centros reduz o problema a um triângulo retângulo.
  - *Como fazer:* calcule h^2; se < 0 não há interseção; os pontos são pé ± h·direção ortogonal. Para dois círculos, ache a e h e desloque perpendicularmente à linha dos centros.
  - *Exemplo:* círculos unitários em (0,0) e (1,0) se cortam em x = 0.5.
  - *Armadilha:* tangência h = 0 conta um ponto; d = 0 concêntricos; um círculo dentro do outro sem tocar; sqrt de negativo pequeno por float.
- **Tangentes a círculos.** Ponto externo: ângulo asin(r/d) em torno da direção; tangentes comuns de dois círculos: externas via mesmo raio deslocado, internas via soma de raios ($|r_1 \pm r_2|$).
  - *Gatilho:* reta tangente a um círculo, tangentes comuns, problemas de Apollonius.
  - *Por que funciona:* no ponto de tangência o raio é perpendicular à reta; o triângulo retângulo com hipotenusa d e cateto r dá o ângulo de desvio asin(r/d). Tangentes comuns reduzem-se a tangentes a um círculo deslocado.
  - *Como fazer:* do ponto externo, gire a direção ao centro por ±asin(r/d); para tangentes comuns, desloque um círculo por ±r (sinal conforme externa/interna) e resolva a tangente passando pelo centro do outro.
  - *Exemplo:* dois círculos iguais disjuntos → 2 tangentes externas e 2 internas.
  - *Armadilha:* ponto dentro → nenhuma; d = r → uma; círculos que se contêm → menos tangentes; raio zero degenera.
- **Área da união de círculos / de interseção círculo-polígono.** Teorema de Green: some a integral de cada arco da fronteira que não está coberto + termos cross; interseção círculo-triângulo com origem no centro como soma de setores/triângulos sinalizados.
  - *Gatilho:* união/interseção de áreas com círculos, cobertura por discos.
  - *Por que funciona:* Green converte área em integral de linha sobre a fronteira; só os arcos efetivamente expostos contribuem, e a integral de um arco é $(r^2/2)\cdot\Delta\theta$ mais o termo de cross com o centro.
  - *Como fazer:* para cada círculo, colete os intervalos angulares cobertos pelos outros; integre os arcos restantes; $O(n^2 \log n)$. Para círculo-polígono, decomponha em setores e triângulos com sinal.
  - *Exemplo:* união de dois círculos unitários com centros a distância 1.
  - *Armadilha:* círculo totalmente contido não contribui com fronteira; círculos duplicados; intervalos que cruzam o ângulo 0; eps na comparação de ângulos.
- **União de retângulos (área).** Sweep em x, segtree em y com (min, contagem de mínimo) ou (cobertura, comprimento coberto); O(n log n). Com perímetro: some também variação de comprimento coberto nas bordas.
  - *Gatilho:* área da união de retângulos, cobertura de área, "quantas células são cobertas".
  - *Por que funciona:* entre eventos de x o comprimento coberto em y é constante; a segtree mantém esse comprimento e a área é comprimento·Δx.
  - *Como fazer:* comprima os y, eventos (x, +1/−1, y1, y2), segtree com contagem de cobertura e comprimento coberto; processe todos os eventos do mesmo x antes de multiplicar por Δx.
  - *Exemplo:* dois retângulos sobrepostos → área da união.
  - *Armadilha:* ordem de eventos no mesmo x; intervalos half-open para não contar borda duas vezes; coordenadas grandes → área long long.

- **Perímetro da união de retângulos.** Sweep horizontal e vertical separadamente: perímetro = Σ |Δ comprimento coberto| ao longo do sweep em cada eixo; ou 2×(variações). Cuidado com a ordem de eventos (abre antes de fechar nas coordenadas iguais).
  - *Gatilho:* perímetro da união de retângulos, contorno de cobertura.
  - *Por que funciona:* cada mudança do comprimento coberto em y cria uma borda vertical desse tamanho; varrendo os dois eixos, todas as bordas são contadas uma vez.
  - *Como fazer:* sweep em x somando |Δ comprimento coberto| (bordas verticais); repita trocando os eixos para as horizontais; segtree com cobertura; O(n log n).
  - *Exemplo:* dois quadrados unitários disjuntos → perímetro 8.
  - *Armadilha:* eventos do mesmo x precisam ser agrupados para não gerar bordas espúrias; retângulos que compartilham aresta; some as variações de cada eixo.
- **Área da união de polígonos.** Para cada aresta, calcule o comprimento não coberto pelos outros polígonos (projeção em parâmetro t) e some cross das partes visíveis (Green); $O(n^2 \log n)$.
  - *Gatilho:* união de polígonos arbitrários, área coberta por formas.
  - *Por que funciona:* Green reduz a área à integral das partes visíveis da fronteira; uma aresta só contribui no trecho não coberto por outro polígono.
  - *Como fazer:* para cada aresta, colete os intervalos de t cobertos pelas interseções com as outras arestas/polígonos, ordene e integre os trechos livres; $O(n^2 \log n)$.
  - *Exemplo:* união de dois triângulos sobrepostos.
  - *Armadilha:* arestas colineares sobrepostas; orientação dos polígonos; parâmetros de interseção em double.
- **Sweep line + set (Shamos–Hoey).** Detectar se existe interseção entre n segmentos: mantenha set ordenado por y no x atual; eventos de abertura checam vizinhos; interseção só pode surgir entre vizinhos. Bentley–Ottmann reporta todas em O((n + k) log n).
  - *Gatilho:* "existe interseção entre segmentos", reportar todos os cruzamentos.
  - *Por que funciona:* no sweep, dois segmentos só podem se cruzar se forem adjacentes na ordem vertical no momento do cruzamento; qualquer evento altera apenas a vizinhança local.
  - *Como fazer:* eventos de início/fim/interseção; ao inserir ou trocar, teste os novos vizinhos; Bentley–Ottmann agenda interseções; O((n + k) log n).
  - *Exemplo:* detectar se um conjunto de segmentos tem qualquer cruzamento.
  - *Armadilha:* segmentos verticais; eventos no mesmo x; a ordem do set depende do x atual (comparador dinâmico); interseções múltiplas.
- **Comparador de segmentos no set.** Compare a y em x corrente (via fração), nunca guarde float; segmentos verticais tratados à parte; desempate por id para não apagar errado.
  - *Gatilho:* implementação de sweep line com set de segmentos.
  - *Por que funciona:* a y de um segmento em x é uma função racional; comparar por fração é exato, enquanto float acumula erro e pode violar a ordem.
  - *Como fazer:* y = y1 + (y2 − y1)(x − x1)/(x2 − x1), guarde numerador/denominador e compare por multiplicação cruzada em __int128; verticais em estrutura separada; desempate por id.
  - *Exemplo:* ordenar segmentos num sweep em x = 5/3.
  - *Armadilha:* denominador zero para verticais; estouro ao multiplicar em 64 bits; segmentos com mesma y no x atual precisam de desempate estável.
- **Sweep angular em torno de um ponto.** Ordene os outros pontos por ângulo e use dois ponteiros para contar "quantos em semiplano" ou "triângulos que contêm a origem" = C(n, 3) − Σ C(k_i, 2), onde k_i = pontos no semiplano à esquerda de i.
  - *Gatilho:* contagem de pontos num semiplano, triângulos que contêm um ponto, sweep circular.
  - *Por que funciona:* na ordem angular, os pontos de um semiplano aberto de abertura π formam um intervalo contíguo; dois ponteiros contam todos em O(n) após o sort.
  - *Como fazer:* ordene por ângulo; duplique somando 2π; para cada i avance j enquanto o ângulo for < θ_i + π; k_i = j − i − 1.
  - *Exemplo:* 3 pontos a 0°, 120°, 240° → todos os C(3,3) = 1 triângulo contém a origem.
  - *Armadilha:* pontos colineares com a origem (fronteira do semiplano); ângulos duplicados; o intervalo deve ser aberto para não contar pares antipodais.

- **Contar triângulos que contêm um ponto.** Complemento: triângulos que NÃO contêm = os 3 vértices num mesmo semiplano aberto; para cada ponto i some C(cnt_i, 2) onde cnt_i = pontos em (θ_i, θ_i + π). Cuidado com colinear com a origem.
  - *Gatilho:* "quantos triângulos contêm o centro", probabilidade de conter um ponto.
  - *Por que funciona:* um triângulo não contém o ponto exatamente quando seus três vértices cabem num semiplano aberto pelo ponto; ancorando no vértice de menor ângulo, cada triângulo ruim é contado uma vez.
  - *Como fazer:* para cada i, cnt_i = pontos no arco (θ_i, θ_i + π); subtraia Σ C(cnt_i, 2) de C(n, 3).
  - *Exemplo:* 3 pontos a 0°, 120°, 240° → cnt = 1 para cada, Σ C(1,2) = 0, resposta 1.
  - *Armadilha:* pontos colineares com o ponto (ângulo exatamente π); contar cada triângulo ruim uma única vez escolhendo a âncora certa; duplicados.
- **Contar triângulos de área não nula/ retângulos / paralelogramas.** Paralelogramas: conte pares de pontos por ponto médio (hash de soma de coordenadas); retângulos: hash por (ponto médio, comprimento da diagonal).
  - *Gatilho:* "quantos triângulos não degenerados", "quantos paralelogramos/retângulos com vértices nos pontos".
  - *Por que funciona:* três pontos são colineares se o cross é 0; quatro pontos formam paralelogramo se e só se as duas diagonais têm o mesmo ponto médio; um retângulo exige também diagonais de mesmo comprimento.
  - *Como fazer:* triângulos = C(n,3) − triplas colineares (conte por reta); paralelogramos: hash de (x1+x2, y1+y2); retângulos: hash de (média, $|\text{diagonal}|^2$).
  - *Exemplo:* quatro vértices de um quadrado → 1 retângulo; um paralelogramo → 1.
  - *Armadilha:* pares colineares com o mesmo ponto médio geram "paralelogramo" degenerado (área 0); colisões de hash; contar cada par duas vezes.
- **Quadrados / retângulos axis-alinhados em pontos.** Fixe duas colunas/linhas e conte: O(n√n) usando pesado/leve (colunas com muitos pontos vs poucos) ou O(Σ k^2).
  - *Gatilho:* contar retângulos/quadrados com lados paralelos aos eixos e vértices no conjunto.
  - *Por que funciona:* fixadas duas colunas x1 < x2, cada par de y presente em ambas forma um retângulo; para quadrados exige |x2 − x1| = |y2 − y1|.
  - *Como fazer:* agrupe por x e por y; para cada par de colunas, intersecte as listas de y (ou use bitset); pesado/leve para O(n√n); quadrados checam a diferença.
  - *Exemplo:* grade 3×3 → 9 retângulos (incluindo quadrados).
  - *Armadilha:* quadrados precisam da condição de lado; pontos duplicados; retângulo degenerado de área 0.
- **Sweep angular com rotação de eixo: pares com cross sinal.** Número de pares de pontos separados por uma reta que passa por dois pontos = contagem por ângulo; o "rotating sweep" (rotação contínua) mantém a ordem de projeção e troca vizinhos nos ângulos dos pares (n^2 eventos).
  - *Gatilho:* halving lines, pares separados por uma reta, contagem por semiplano que muda com a direção.
  - *Por que funciona:* ao girar a direção, a ordem só muda quando ela fica perpendicular à reta por dois pontos; nesse instante os dois são vizinhos na ordem de projeção.
  - *Como fazer:* gere os O(n^2) ângulos de pares, ordene, processe trocas de vizinhos e mantenha contadores; O(n^2 log n).
  - *Exemplo:* número de halving lines de um conjunto de pontos.
  - *Armadilha:* vários pares com o mesmo ângulo (paralelos) devem ser processados juntos; triplas colineares; compare ângulos por cross, não atan2.
- **Rotating sweep (ordem de projeção).** Ordene os n pontos pela projeção numa direção que gira; só muda a ordem quando a direção fica perpendicular a um par: processe os O(n^2) eventos por ângulo trocando posições vizinhas; resolve "maior triângulo/ área em k pontos" e "k-ésimo pelo qual passa" em O(n^2 log n).
  - *Gatilho:* consultas sobre a ordem de projeção em todas as direções, k-ésimo numa direção, eventos de troca.
  - *Por que funciona:* entre dois ângulos críticos a ordem é constante; no ângulo crítico, o par que define a perpendicular é adjacente e troca de posição.
  - *Como fazer:* projeção inicial, eventos (ângulo, par), ordene, troque os dois vizinhos e atualize a estrutura; O(n^2 log n) para ordenar os eventos e O(1) por troca.
  - *Exemplo:* k-ésimo ponto na ordem de projeção para alguma direção.
  - *Armadilha:* eventos simultâneos (direções paralelas) formam blocos que precisam ser revertidos juntos; pontos duplicados; ordenar ângulos por cross.

- **Maior triângulo num conjunto.** Está no hull; para cada vértice fixe e avance dois ponteiros (calipers) em $O(n^2)$ no hull; ou O(n) com técnicas mais sofisticadas.
  - *Gatilho:* "maior área de triângulo com vértices nos pontos".
  - *Por que funciona:* o triângulo de área máxima tem os três vértices no hull; fixado i, a área em função de j e k é unimodal ao longo do hull, então dois ponteiros avançam monotonicamente.
  - *Como fazer:* hull; para cada i, avance j e k enquanto a área aumentar; $O(h^2)$; use cross inteiro para a área.
  - *Exemplo:* triângulo máximo inscrito num pentágono convexo.
  - *Armadilha:* vértices colineares no hull; empates; h pequeno → $O(h^3)$ também passa.
- **Dualidade ponto-reta.** (a, b) ↔ y = a·x − b preserva "acima/abaixo" e incidência: o hull superior de pontos = envelope inferior (CHT) de retas; queries de "reta mais próxima/ponto mais alto numa direção" ↔ CHT/Li Chao.
  - *Gatilho:* relações acima/abaixo, interseção de retas, queries de máximo de retas, CHT.
  - *Por que funciona:* a dualidade troca incidência ponto-reta e inverte acima/abaixo; o hull superior de um conjunto de pontos vira o envelope inferior das retas duais.
  - *Como fazer:* transforme pontos em retas, responda queries com CHT/Li Chao e volte; confira a convenção de sinal.
  - *Exemplo:* "o ponto está acima da reta por i e j" vira ordem de interseção das retas duais.
  - *Armadilha:* convenções de sinal variam entre textos; retas verticais não têm representação; interseções comparadas em double.
- **Dualidade para arranjos de retas.** Pontos em posição geral ↔ retas; "quantos pontos acima da reta pelos pontos i e j" vira contagem de interseções e vértices do arranjo; k-nível de arranjo.
  - *Gatilho:* contagens acima/abaixo de retas determinadas por pares, k-level, arranjo.
  - *Por que funciona:* a dualidade preserva a ordem acima/abaixo, então a ordem dos pontos em relação a uma reta dual é a ordem das interseções no arranjo.
  - *Como fazer:* construa o arranjo combinatorialmente (ou conte interseções com BIT) e percorra os níveis; k-nível por sweep.
  - *Exemplo:* número de pontos acima da reta por i e j.
  - *Armadilha:* três pontos colineares viram três retas concorrentes; contagem dupla nas interseções; posição geral é hipótese.
- **Inversão de círculo.** Transforma círculos que passam pelo polo em retas e preserva tangência; útil para problemas de tangentes múltiplas (Apollonius, cadeia de Steiner) e para transformar interseção de círculos em retas (reduz a casos lineares).
  - *Gatilho:* círculos tangentes, cadeias de círculos, problemas de Apollonius, muitas circunferências.
  - *Por que funciona:* a inversão é uma transformação conforme: preserva ângulos e tangências, e leva círculos pelo polo em retas; o problema complicado vira linear.
  - *Como fazer:* escolha o polo e o raio; $p' = p/|p|^2$ (escalado); resolva o problema com retas/círculos mais simples e inverta de volta.
  - *Exemplo:* círculo tangente a três círculos dados (Apollonius).
  - *Armadilha:* o polo não tem imagem; círculo que passa pelo polo vira reta; orientação e raio da inversão; proximidade do polo causa instabilidade numérica.
- **Geometria 3D: produto misto.** Volume do tetraedro = |dot(a − d, cross(b − d, c − d))|/6; sinal dá orientação (lado do plano). Ponto em plano: dot(normal, p − p_0) = 0. Distância ponto-plano = |dot(n, p − p_0)|/|n|.
  - *Gatilho:* volume, coplanaridade, orientação 3D, distância a plano.
  - *Por que funciona:* o produto misto é o volume sinalizado do paralelepípedo; zero significa vetores coplanares; a normal do plano é o cross de dois vetores dele.
  - *Como fazer:* triple = dot(u, cross(v, w)); volume = |triple|/6; distância = |dot(n, p − p_0)|/|n|, com n = cross de duas arestas.
  - *Exemplo:* tetraedro unitário → volume 1/6.
  - *Armadilha:* coordenadas 10^9 → produtos ~10^{27}, não cabem em int64 (use __int128/long double); o sinal depende da ordem; n nulo se os vetores são colineares.
- **Fecho convexo 3D.** Incremental $O(n^2)$ (ou n log n com D&C/randomizado); conta faces via Euler V − E + F = 2 (F = 2V − 4 para triangulação). Distância mínima à origem entre poliedros também usa isso.
  - *Gatilho:* hull 3D, número de faces, pontos extremos no espaço.
  - *Por que funciona:* o hull é uma esfera topológica, então Euler vale; para uma triangulação, 3F = 2E e Euler dá F = 2V − 4.
  - *Como fazer:* inserção incremental com conflict graph e reparo local; ou $O(n^2)$ ingênuo; use orientação de faces para manter consistência.
  - *Exemplo:* n pontos em posição geral → F = 2n − 4 faces triangulares.
  - *Armadilha:* pontos coplanares geram faces não triangulares; degenerescência; orientação das faces precisa ser consistente.

- **Transformações lineares/afins.** Escala não uniforme preserva razão de áreas; rotação por matriz; "alongar eixo" transforma elipse em círculo e depois resolva como círculo (problemas de distância em elipse).
  - *Gatilho:* elipses, escalas por eixo, problemas que ficam simples após rotação/escala.
  - *Por que funciona:* transformações afins preservam colinearidade, razões ao longo de uma reta e razões de áreas (o determinante multiplica todas as áreas pelo mesmo fator); uma escala anisotrópica leva elipse em círculo.
  - *Como fazer:* aplique a matriz inversa, resolva o problema na forma redonda, aplique a transformação de volta; distâncias não são preservadas (só razões de áreas).
  - *Exemplo:* distância de um ponto a uma elipse → escale y e resolva para um círculo.
  - *Armadilha:* distâncias e ângulos mudam; determinante 0 (transformação singular); a resposta precisa ser convertida de volta.
- **Razões de áreas e cevianas.** Teorema de Ceva/Menelaus e massas pontuais em problemas de razões; coordenadas baricêntricas para ponto em triângulo e interpolação (λ_i = área sub-triângulo / área total).
  - *Gatilho:* razões de segmentos, concorrência de cevianas, colinearidade, interpolação em triângulo.
  - *Por que funciona:* Ceva e Menelaus dão condições exatas em produto de razões; baricêntricas expressam qualquer ponto como combinação convexa dos vértices com pesos iguais a razões de áreas.
  - *Como fazer:* Ceva: (BD/DC)(CE/EA)(AF/FB) = 1 para concorrência; Menelaus: produto = −1 para colinearidade (com sinal); λ_i = área oposta / área total.
  - *Exemplo:* medianas concorrem no baricentro, razões 1.
  - *Armadilha:* Menelaus usa razões sinalizadas; pontos externos ao triângulo; triângulo degenerado.
- **Ponto em triângulo.** Três sinais de cross com o mesmo sinal (ou zero na borda); ou baricêntricas. Para muitos pontos e muitos triângulos → sweep/estrutura offline (contagem por triângulo = diferença de contagens de 3 trapézios).
  - *Gatilho:* "está dentro do triângulo", muitas consultas, contagem por triângulo.
  - *Por que funciona:* o triângulo é a interseção de três semiplanos; os três cross têm o mesmo sinal exatamente para pontos internos; baricêntricas dão o mesmo critério com coordenadas.
  - *Como fazer:* calcule os três cross; dentro se todos ≥ 0 ou todos ≤ 0 (borda inclusa conforme o enunciado); para muitas consultas, decomponha em regiões trapezoidais e combine contagens offline.
  - *Exemplo:* (0.2,0.2) no triângulo unitário (0,0),(1,0),(0,1) → dentro.
  - *Armadilha:* a ordem dos vértices muda o sinal; ponto na aresta (cross 0) precisa de decisão explícita; coordenadas inteiras e cross exato.
- **Visibilidade de ponto entre obstáculos.** Sweep angular com set de segmentos ordenado pela distância no raio atual; ou grafo de visibilidade $O(n^2 \log n)$ (vértices dos obstáculos + sweep por vértice) e caminho mínimo euclidiano em obstáculos poligonais.
  - *Gatilho:* "o ponto vê o outro", linha de visão, obstáculos poligonais.
  - *Por que funciona:* a visibilidade só muda em eventos angulares definidos por vértices dos obstáculos; no raio atual, o segmento mais próximo é o que bloqueia.
  - *Como fazer:* sweep angular em torno do ponto, set ordenado por distância no raio; para todos os pares, repita de cada vértice $O(n^2 \log n)$; o grafo de visibilidade liga pares visíveis.
  - *Exemplo:* construir o grafo de visibilidade de um polígono com buracos.
  - *Armadilha:* segmentos que cruzam o raio; vértices colineares; obstáculos abertos vs fechados.
- **Caminho mais curto com obstáculos poligonais.** Só passa por vértices dos obstáculos: grafo de visibilidade + Dijkstra. Para um polígono simples, "funnel algorithm".
  - *Gatilho:* menor caminho euclidiano desviando de polígonos, robôs, labirinto geométrico.
  - *Por que funciona:* um caminho ótimo é poligonal e só dobra em vértices de obstáculos; qualquer outra dobra pode ser endireitada sem aumentar o comprimento.
  - *Como fazer:* construa o grafo de visibilidade (vértices + início + fim), rode Dijkstra; para polígono simples, o funnel algorithm resolve em $O(n)$.
  - *Exemplo:* caminho mais curto entre dois pontos de um polígono simples.
  - *Armadilha:* $O(n^2)$ arestas; obstáculos que se tocam; início/fim dentro de obstáculo.

- **Convex layers (camadas convexas).** Aplique o hull repetidamente (O(n log n) total com estrutura dinâmica ou $O(n^2)$ simples); a profundidade de um ponto = "mínimo de pontos a remover/disparos para isolá-lo". Gatilho: "retirar pontos extremos repetidamente".
  - *Gatilho:* onion peeling, profundidade de camada, remover hulls sucessivos.
  - *Por que funciona:* cada camada é o hull dos pontos restantes; remover a camada expõe a próxima, e a profundidade mede quantas remoções isolam o ponto.
  - *Como fazer:* repita hull e remova os vértices; $O(n^2)$ simples; com hull dinâmico, O(n log n) total.
  - *Exemplo:* pontos em quadrados concêntricos → uma camada por quadrado.
  - *Armadilha:* colineares no hull (fronteira vs vértice estrito); duplicados; definir se a camada inclui pontos colineares.
- **Pontos extremos repetidos e dominância.** Pontos que podem ser o maior para alguma função linear = vértices do hull; pontos "não dominados" em 2D = frente de Pareto (sweep + max corrente); em 3D → CDQ/sweep + set.
  - *Gatilho:* "ponto que pode ser máximo", frente de Pareto, dominância.
  - *Por que funciona:* um vértice do hull suporta alguma direção; um ponto dominado nunca maximiza função linear com coeficientes positivos. Em 2D, ordenar por x e manter o máximo de y encontra os não dominados.
  - *Como fazer:* hull para extremos; Pareto: ordene por x crescente e mantenha o maior y visto; 3D: CDQ divide e conquista com BIT.
  - *Exemplo:* (0,0) é vértice do hull de {(0,0),(1,1),(0,2)}, mas é dominado por (1,1).
  - *Armadilha:* dominância não é o mesmo que camada convexa; empates de coordenadas; dominância estrita vs fraca.
- **Point location.** Subdivisão planar → sweep persistente (set de segmentos persistente por x) ou trapezoidal map aleatorizado; em contests: offline com sweep e segtree/BIT por segmento. Para polígonos convexos: busca binária angular.
  - *Gatilho:* muitas consultas "em qual região está este ponto", subdivisão planar.
  - *Por que funciona:* a região que contém o ponto é determinada pela aresta imediatamente acima/abaixo dele no x da consulta; estruturas persistentes respondem isso em O(log n).
  - *Como fazer:* sweep com set persistente; trapezoidal map O(log n) esperado; polígono convexo O(log n) por ângulo.
  - *Exemplo:* localizar 10^5 pontos entre 10^5 polígonos disjuntos.
  - *Armadilha:* pontos sobre arestas; arestas verticais; custo de pré-processamento.
- **Maior retângulo vazio (maximum empty rectangle).** Pontos num retângulo: $O(n^2)$ por enumeração de pares de bordas com pilha, ou D&C/sweep em $O(n \log^2 n)$; canto inferior-esquerdo + histograma sobre colunas ordenadas. Gatilho: "maior retângulo sem pontos".
  - *Gatilho:* "maior retângulo vazio", área livre máxima, posicionamento.
  - *Por que funciona:* um retângulo vazio maximal tem pontos em três lados (ou na fronteira da região); enumerar essas bordas cobre o ótimo.
  - *Como fazer:* para cada par de limites em x, use pilha/histograma em y para achar o maior intervalo vazio; $O(n^2)$; D&C/sweep melhora para $O(n \log^2 n)$.
  - *Exemplo:* maior retângulo sem pontos dentro de uma caixa.
  - *Armadilha:* o retângulo deve ficar dentro da região; pontos na borda contam como bloqueio; área zero degenerada.
- **Maior círculo vazio.** Centro está num vértice de Voronoi (dentro do hull) ou na interseção da borda com arestas de Voronoi; vem do Delaunay.
  - *Gatilho:* maior círculo que não contém pontos, cobertura maximal.
  - *Por que funciona:* um círculo vazio maximal toca 2 ou 3 pontos; o centro é um vértice de Voronoi (ou a interseção de uma aresta de Voronoi com a fronteira do domínio).
  - *Como fazer:* construa Delaunay/Voronoi e teste os circuncentros e as interseções com a borda; O(n log n).
  - *Exemplo:* maior círculo evitando pontos dentro de um quadrado.
  - *Armadilha:* centro fora do hull (região ilimitada) → intersecte com o domínio; pontos cocirculares; círculo pode encostar na borda.
- **Número de interseções de segmentos, pares que se cruzam.** Pares de cordas de círculo cruzam ⇔ extremos se intercalam: contagem de inversões/BIT; "interseções entre diagonais" = pares com (a < c < b < d).
  - *Gatilho:* contar cruzamentos de cordas/diagonais, pares intercalados.
  - *Por que funciona:* duas cordas com extremos em ordem circular se cruzam exatamente quando os extremos alternam; isso é uma inversão numa ordem linear fixada.
  - *Como fazer:* numere os extremos na ordem circular; ordene por uma ponta e conte inversões da outra com BIT; O(n log n).
  - *Exemplo:* diagonais de um pentágono convexo → 5 interseções.
  - *Armadilha:* cordas que compartilham extremo não contam como cruzamento; sobreposição colinear; duplicados.

- **Cordas e pontos no círculo.** Ordem circular + intercalação; para geometria com ≥ 3 pontos num círculo, indexe por ângulo (ordem) e problemas viram parênteses/intervalos (Catalan para não cruzamento).
  - *Gatilho:* pontos numa circunferência, emparelhamentos não cruzados, diagonais.
  - *Por que funciona:* a ordem circular lineariza o problema; não cruzar é a mesma condição de parênteses bem formados, contada por Catalan.
  - *Como fazer:* escolha um corte e duplique a ordem para tratar o wrap; DP de intervalos para não cruzamento; Catalan(n) para emparelhamentos de 2n pontos.
  - *Exemplo:* número de emparelhamentos perfeitos não cruzados de 6 pontos = Catalan(3) = 5.
  - *Armadilha:* intervalos que dão a volta; pontos antipodais; rotações contam como o mesmo desenho.
- **Alturas/ângulos inteiros: tangente como fração.** Compare ângulos via cross em vez de atan2; ângulo entre direções ≤ 90° ⇔ dot ≥ 0; "ângulo do vetor em relação a outro" por cross/dot sem funções trigonométricas.
  - *Gatilho:* comparar ângulos, "ângulo ≤ 90°", ordenar por inclinação.
  - *Por que funciona:* dot e cross são cosseno e seno multiplicados por fatores positivos; o sinal de dot decide agudo/reto/obtuso e o de cross decide a orientação, sem transcendental.
  - *Como fazer:* para ≤ 90° use dot ≥ 0; para comparar dois ângulos, compare as direções por semiplano + cross; para inclinações, compare dy/dx por multiplicação cruzada.
  - *Exemplo:* saber se dois vetores formam ângulo agudo: dot > 0.
  - *Armadilha:* dot ≥ 0 só responde ≤ 90°, não dá o ângulo; inclinações verticais; estouro no produto.
- **Contagem de pontos dentro de círculos/ retângulos por consultas offline.** Retângulo = 4 prefixos 2D (inclusão-exclusão), com sweep + BIT; círculos de raio fixo → grade de células de lado r e examinar 3×3 vizinhanças.
  - *Gatilho:* muitas consultas de contagem em retângulos/círculos, pontos estáticos.
  - *Por que funciona:* contar num retângulo é incluir-excluir quatro quadrantes; com sweep em x e BIT em y, cada prefixo é O(log n). Para raio fixo r, um ponto só pode estar no círculo se estiver numa célula vizinha da célula da consulta.
  - *Como fazer:* ordene eventos por x, BIT por y comprimido; círculos: grid de lado r, examine as 3×3 células e teste a distância.
  - *Exemplo:* contar pontos em 10^5 retângulos.
  - *Armadilha:* bordas inclusivas/exclusivas; eventos no mesmo x; raio variável não serve para a grade fixa.
- **Geometria com float: epsilon relativo.** Compare $|a - b| \le \text{eps} \cdot \max(1, |a|, |b|)$; para ordens use comparador por cross inteiro quando as coordenadas são inteiras; evite acos fora de [−1, 1] (clamp) e sqrt de negativo pequeno (max(0, x)).
  - *Gatilho:* entrada com decimais, distâncias/ângulos calculados, comparações de igualdade.
  - *Por que funciona:* o erro absoluto de uma operação cresce com a magnitude dos operandos; normalizar pelo máximo torna o critério proporcional ao erro esperado.
  - *Como fazer:* eps ~10^{-9} com double (maior com long double), sempre relativo; clamp antes de acos e max(0, ·) antes de sqrt; prefira predicados inteiros.
  - *Exemplo:* comparar duas distâncias de ~10^6 com eps relativo em vez de 10^{-9} absoluto.
  - *Armadilha:* eps absoluto falha em magnitudes grandes; eps grande funde respostas distintas; encadear muitos eps acumula erro.
- **Geometria em inteiros: ordem de grandeza.** Coordenadas até 10^9: cross cabe em int64 ($\le 2 \cdot (2 \cdot 10^9)^2 \approx 8 \cdot 10^{18}$, perto do limite), mas produtos de cross (in-circle) exigem __int128 ou long double com cuidado; para interseção de retas em frações use __int128 em num/den.
  - *Gatilho:* coordenadas inteiras grandes, predicados de grau alto, interseção exata.
  - *Por que funciona:* o grau do polinômio determina o tamanho: cross é grau 2 nas coordenadas, in-circle é grau 4; o tipo precisa acomodar o pior caso.
  - *Como fazer:* estime o expoente antes de escolher o tipo; int64 para cross, __int128 para determinantes 3×3/4×4 e numeradores de frações; long double só se a precisão bastar.
  - *Exemplo:* in-circle com coordenadas 10^9 → termos ~10^{36}, que cabem em __int128 (~$1.7 \cdot 10^{38}$).
  - *Armadilha:* overflow silencioso em int64; misturar signed/unsigned; assumir que long double tem 80 bits em todo compilador.

- **Orientação robusta com long double.** Quando as coordenadas são float, o sinal de cross com |cross| < eps·|a||b| é duvidoso; reformule como inteiros escalados (multiplique por 10^k) se a entrada tem poucas casas decimais.
  - *Gatilho:* entrada decimal, pontos quase colineares, predicado de orientação.
  - *Por que funciona:* o erro relativo do cross é da ordem do eps da máquina vezes o produto dos comprimentos; abaixo disso o sinal é ruído. Escalar para inteiros elimina o erro se as casas decimais são finitas.
  - *Como fazer:* se a entrada tem k casas, multiplique por 10^k e use inteiros/__int128; senão use long double e um eps relativo; em último caso, predicados adaptativos.
  - *Exemplo:* três pontos com 3 casas decimais quase colineares.
  - *Armadilha:* 10^k pode estourar; long double no MSVC tem 64 bits de mantissa; o eps precisa ser proporcional a |a||b|.
- **Áreas e probabilidade geométrica.** "Escolha pontos aleatórios em região" → área da região favorável / total; integral com Simpson para áreas de interseção de formas curvas (adaptativo se não for suave).
  - *Gatilho:* probabilidade de um evento geométrico, pontos aleatórios uniformes.
  - *Por que funciona:* para distribuição uniforme por área, a probabilidade é a razão das áreas; para curvas, a área é uma integral que Simpson aproxima.
  - *Como fazer:* calcule a área favorável e a total com as ferramentas de área; para fronteiras curvas, integre o comprimento coberto com Simpson adaptativo.
  - *Exemplo:* probabilidade de um ponto aleatório no quadrado cair no círculo inscrito = π/4.
  - *Armadilha:* "aleatório" pode não ser uniforme por área (raio/ângulo uniformes não são); normalizar a distribuição.
- **Simpson / integração numérica.** Função convexa/unimodal em um intervalo → ternária; áreas de união de círculos/elipses como integral ao longo de x do comprimento coberto (Simpson adaptativo).
  - *Gatilho:* área sob curva, união de formas curvas, minimizar função de uma variável.
  - *Por que funciona:* Simpson é exato para polinômios cúbicos e converge rápido para funções suaves; a versão adaptativa refina onde a curvatura é alta. Para funções unimodais, a ternária descarta um terço a cada passo.
  - *Como fazer:* compare Simpson(l, m) + Simpson(m, r) com Simpson(l, r); se a diferença > eps, recurse; ~200 iterações para ternária em double.
  - *Exemplo:* área da união de dois círculos integrando o comprimento vertical coberto.
  - *Armadilha:* integrando não suave (quebras nos cruzamentos de círculos) — divida nos eventos; recursão sem limite; eps absoluto em área grande.
- **Ternária e geometria.** "Distância mínima de um ponto em movimento" ou "posição do ponto em reta que minimiza soma de distâncias" é convexo em t → ternária (ou ponto de Fermat); quando a função é dist linear por partes, vale busca de ponto ótimo nos vértices.
  - *Gatilho:* ponto que se move numa reta/curva, minimizar distância/soma convexa.
  - *Por que funciona:* uma função convexa tem um único mínimo; comparar $f(m_1)$ e $f(m_2)$ permite descartar um terço do intervalo sem derivada.
  - *Como fazer:* ternária com ~200 iterações ou até r − l < eps; para funções lineares por partes, avalie diretamente nos vértices/quebras.
  - *Exemplo:* posição numa reta que minimiza a soma das distâncias a pontos fixos.
  - *Armadilha:* função não unimodal (vários mínimos locais); platôs; precisão vs número de iterações.
- **Soma de distâncias mínima.** Ponto que minimiza $\Sigma|p - p_i|_1$ = mediana por eixo; $\Sigma|p - p_i|_2$ (Fermat/geometric median) → Weiszfeld ou ternária aninhada; Chebyshev (L∞) → centro da caixa envolvente alinhada (não rotacionada; a rotação de 45° apenas relaciona L1 e L∞).
  - *Gatilho:* "ponto que minimiza a soma das distâncias", mediana geométrica, cobertura.
  - *Por que funciona:* em L1 a soma separa por coordenada e a mediana minimiza $\Sigma|x - a_i|$; em L2 a soma é convexa; em L∞ o pior caso é meia largura da caixa.
  - *Como fazer:* L1: mediana independente por eixo; L2: iteração de Weiszfeld ou ternária aninhada; L∞: centro da caixa, valor max(largura, altura)/2.
  - *Exemplo:* 3 pontos na reta → mediana; triângulo → ponto de Fermat.
  - *Armadilha:* a mediana geométrica não tem forma fechada; a função não é diferenciável nos pontos dados; pesos mudam a solução.

- **Menor caixa/quadrado envolvendo pontos após rotação.** O ótimo ocorre quando um lado do retângulo é colinear com uma aresta do hull → calipers, O(n) após o hull.
  - *Gatilho:* retângulo envolvente de área/perímetro mínimo, caixa orientada.
  - *Por que funciona:* girar a caixa ótima até alinhar um lado com uma aresta do hull não aumenta a área; então só as direções das arestas do hull importam.
  - *Como fazer:* hull; para cada aresta, calcule largura e altura com ponteiros de suporte; pegue o mínimo; O(h).
  - *Exemplo:* nuvem de pontos em diagonal → caixa ótima girada.
  - *Armadilha:* hull colinear (h = 2); comparar áreas em inteiros ou com eps; "quadrado" exige lado igual.
- **Menor distância entre duas curvas/pontos convexos.** Funções convexas → ternária aninhada; polígonos → calipers; distância de polígonos ao ponto → min de pontos-segmento.
  - *Gatilho:* distância entre dois conjuntos convexos, curvas, polígonos.
  - *Por que funciona:* para conjuntos convexos a distância tem estrutura convexa; em polígonos, o par mais próximo é realizado entre arestas/vértices e os calipers percorrem candidatos monotonicamente.
  - *Como fazer:* ternária aninhada nos parâmetros; polígonos convexos → calipers O(n + m); ponto-polígono → mínimo de distâncias ponto-segmento.
  - *Exemplo:* distância entre dois quadrados convexos.
  - *Armadilha:* curvas não convexas podem ter vários mínimos locais; a ternária pressupõe unimodalidade; interseção dá 0.
- **Cobertura por intervalos angulares.** "Um ponto vê quantos objetos" / "o laser cobre todos?" → cada objeto vira um intervalo angular; ordene eventos e faça sweep circular (duplique a lista em 2π para wrap).
  - *Gatilho:* visão/câmera, cobertura angular, "quantos objetos são visíveis de um ponto".
  - *Por que funciona:* cada objeto subtende um intervalo de ângulos; a cobertura é uma união de intervalos num círculo, e duplicar a lista transforma o wrap num intervalo linear.
  - *Como fazer:* calcule $[\theta - \alpha, \theta + \alpha]$ para cada objeto; ordene eventos de início/fim; duplique somando 2π; sweep com contador.
  - *Exemplo:* número máximo de objetos visíveis de um ponto.
  - *Armadilha:* intervalos que cruzam o ângulo 0; tangência (fronteira aberta/fechada); objetos atrás do observador.
- **Pontos visíveis do lado de dentro de uma torre / skyline.** Pilha monotônica sobre inclinações ou hull superior do perfil; problema "quais prédios são visíveis" = inclinação crescente (cross) em varredura.
  - *Gatilho:* "quais prédios são visíveis", skyline, visão de dentro de uma torre.
  - *Por que funciona:* um prédio é visível se sua inclinação em relação ao observador supera todas as anteriores; a pilha mantém as inclinações estritamente crescentes e descarta as dominadas.
  - *Como fazer:* ordene por distância; mantenha pilha de candidatos; ao inserir, pop enquanto o cross indicar que o topo não pode mais ser visível; O(n log n).
  - *Exemplo:* prédios de alturas variadas vistos da esquerda.
  - *Armadilha:* alturas/inclinações iguais (visibilidade estrita vs não estrita); prédios no mesmo x; base do observador.
- **Skyline / envelope superior de segmentos.** Sweep com set ou D&C unindo duas envoltórias; para retas, hull superior das retas (CHT).
  - *Gatilho:* envelope superior/inferior de segmentos/retas, skyline.
  - *Por que funciona:* o envelope é a fronteira do máximo pontual; ao dividir, o envelope de cada metade é independente e o merge é linear no número de peças.
  - *Como fazer:* D&C: calcule os dois envelopes e mescle pela interseção; para retas, CHT/Li Chao; O(n log n).
  - *Exemplo:* contorno superior de retângulos (skyline).
  - *Armadilha:* segmentos verticais; sobreposição de intervalos; retas duplicadas.
- **Raio atravessa polígono convexo / segmento corta convexo.** Clipping em O(log n) por busca binária por tangente, ou O(n) clip por semiplanos; Cyrus–Beck para linha contra polígono convexo.
  - *Gatilho:* interseção de raio/segmento com convexo, clipping.
  - *Por que funciona:* a interseção de uma reta com um convexo é um único intervalo; as tangentes delimitam esse intervalo e podem ser achadas por busca binária.
  - *Como fazer:* ache as tangentes por busca binária O(log n) e teste o intervalo; ou recorte o segmento contra cada semiplano O(n); Cyrus–Beck parametriza a reta e mantém [tmin, tmax].
  - *Exemplo:* um raio partindo de um ponto cruza um hexágono convexo.
  - *Armadilha:* raio começando dentro; toque tangente conta ou não; arestas colineares com o raio.

- **Recorte de polígono (Sutherland–Hodgman).** Clipar polígono por semiplano: percorra arestas e emita interseções; vale para polígono convexo contra semiplano e HPI incremental.
  - *Gatilho:* recortar polígono por retângulo/semiplano, clipping.
  - *Por que funciona:* cada aresta é classificada em dentro/fora; quando cruza a fronteira, a interseção é emitida; o resultado é o polígono recortado.
  - *Como fazer:* para cada aresta (p, q), se p dentro emita p; se cruza, emita a interseção; repita para cada semiplano; O(n) por corte.
  - *Exemplo:* recortar um triângulo por um retângulo.
  - *Armadilha:* vértices duplicados/colineares no resultado; recortar não convexo por convexo pode gerar arestas degeneradas; orientação.
- **Triangulação de polígono (ear clipping / monótono).** Qualquer polígono simples tem triangulação com n − 2 triângulos; área = soma dos triângulos. Contagem de triangulações de polígono convexo = Catalan(n − 2).
  - *Gatilho:* triangular polígono, contar triangulações, área por decomposição.
  - *Por que funciona:* o teorema das duas orelhas garante que todo polígono simples tem uma orelha; remover uma orelha reduz n em 1, indutindo n − 2 triângulos.
  - *Como fazer:* ear clipping $O(n^2)$ testando orelhas; decomponha em monótono e triangule O(n); contagem no convexo por Catalan(n − 2).
  - *Exemplo:* quadrilátero → 2 triângulos; pentágono convexo → 5 triangulações.
  - *Armadilha:* polígono autointersectante; vértices colineares (orelhas de área zero); buracos.
- **Euler para planar e subdivisões.** V − E + F = 1 + C; contar regiões criadas por retas/segmentos: regiões = 1 + n + (nº de interseções) para retas em posição geral (R = 1 + n + I).
  - *Gatilho:* contar faces/regiões de um desenho planar, subdivisões.
  - *Por que funciona:* Euler vale para grafos planares com C componentes; cada novo segmento adiciona 1 + (interseções com os anteriores) regiões, pois é dividido em partes.
  - *Como fazer:* conte V, E e C no grafo planar e aplique V − E + F = 1 + C; para retas, conte I e use 1 + n + I.
  - *Exemplo:* 3 retas em posição geral → 7 regiões.
  - *Armadilha:* paralelas/concorrentes reduzem I; componentes desconexos; sobreposição de segmentos.
- **Retas e regiões.** n retas em posição geral dividem o plano em 1 + n + C(n, 2) regiões; n círculos em posição geral em $n^2 - n + 2$. Para cordas num círculo: 1 + C(n, 2) + C(n, 4) para n pontos em posição geral (diagonais).
  - *Gatilho:* número máximo de regiões por retas/círculos/cordas.
  - *Por que funciona:* cada nova curva adiciona tantas regiões quantas interseções tem com as anteriores, desde que esteja em posição geral; somando as progressões chega-se às fórmulas.
  - *Como fazer:* retas: 1 + n + C(n,2); círculos: 2 + Σ_{k=2}^{n} 2(k−1) = $n^2 - n + 2$; cordas: 1 + C(n,2) + C(n,4).
  - *Exemplo:* 4 retas → 11 regiões; 3 círculos → 8 regiões.
  - *Armadilha:* tangências, paralelismo e concorrência reduzem a contagem; a fórmula supõe posição geral.
- **Contagem de pontos de grid dentro de círculo (Gauss).** Σ $\lfloor\sqrt{(r^2 - x^2)}\rfloor$ em O(r); versão sublinear existe, mas a ideia padrão é varrer x.
  - *Gatilho:* contar pontos inteiros dentro de um círculo de raio r.
  - *Por que funciona:* para cada coluna x, os y válidos vão de $-\lfloor\sqrt{(r^2-x^2)}\rfloor$ a $+\lfloor\sqrt{(r^2-x^2)}\rfloor$; a simetria evita contar os quatro quadrantes separadamente.
  - *Como fazer:* para x de −r a r, calcule ymax com sqrt inteiro; some 2·ymax + 1; cuidado com os eixos para não contar duas vezes.
  - *Exemplo:* r = 1 → 5 pontos (centro e quatro vizinhos).
  - *Armadilha:* sqrt em double erra por 1 perto da fronteira (use isqrt); contar eixos em dobro; r grande torna O(r) inviável.
- **Mínimo de $x^2 + y^2$ sob restrição de grid / retas inteiras.** Reduza com Euclides estendido para achar solução paramétrica (x_0 + t·b/g, y_0 − t·a/g) e minimize função convexa em t.
  - *Gatilho:* minimizar forma quadrática com ax + by = c, pontos inteiros numa reta.
  - *Por que funciona:* as soluções inteiras de uma equação diofantina formam uma progressão aritmética; substituindo na quadrática, resta uma parábola convexa em t.
  - *Como fazer:* g = gcd(a,b); se g ł c não há solução; ache (x_0,y_0) por Euclides estendido; t = *−B/(2A) da quadrática; teste ⌊t⌋* e ⌈t*⌉; O(log).
  - *Exemplo:* minimizar $x^2 + y^2$ com 3x + 5y = 1.
  - *Armadilha:* sem solução quando g não divide c; overflow no t grande; t precisa ser inteiro.

- **Áreas de cruz de vetores e determinantes.** O sinal de determinante 2×2 testa dependência linear (paralelismo); em 3D o triplo produto = 0 ⇔ coplanares. Dimensão d: rank por Gauss.
  - *Gatilho:* paralelismo, coplanaridade, rank de um conjunto de vetores.
  - *Por que funciona:* o determinante é o volume sinalizado; zero exatamente quando os vetores são dependentes; o posto conta as direções independentes.
  - *Como fazer:* 2D: cross = 0; 3D: dot(u, cross(v,w)) = 0; rank: eliminação gaussiana sobre os reais ou módulo p.
  - *Exemplo:* três vetores coplanares → produto misto 0.
  - *Armadilha:* perto de zero em float; overflow em inteiros grandes; rank sobre corpo finito ≠ rank sobre os reais.

adicionado em 2026-11-11 a partir dos editoriais de CF Div. 2/3/4, A–D; triado contra o catálogo

- **Diagonais de grade.** Células com mesmo i + j (antidiagonal) ou mesmo i − j (diagonal) são n + m − 1 linhas cada; soma de "bispo" = soma da diagonal + antidiagonal − a célula. Ex.: CF 1676D.
  - *Gatilho:* movimentos de bispo, somas em diagonais, indexar diagonais.
  - *Por que funciona:* i + j é constante ao longo de uma antidiagonal e i − j ao longo da outra; há n + m − 1 valores possíveis de cada.
  - *Como fazer:* d1 = i + j, d2 = i − j + (m − 1) para ficar não negativo; acumule somas por diagonal; a soma das duas diagonais do bispo menos a célula evita contar o centro duas vezes.
  - *Exemplo:* CF 1676D (Bishop).
  - *Armadilha:* 0-based vs 1-based; i − j negativo exige offset; limites do grid.
- **Camadas concêntricas.** A camada de (i, j) num grid n×m (0-based) é min(i, j, n − 1 − i, m − 1 − j). Ex.: CF 1873C.
  - *Gatilho:* distância até a borda do grid, anéis, "camada k".
  - *Por que funciona:* a camada é a distância de Manhattan até a borda mais próxima; o mínimo das quatro distâncias dá o anel.
  - *Como fazer:* calcule o mínimo das quatro expressões; o número de camadas é ⌈min(n,m)/2⌉.
  - *Exemplo:* CF 1873C (Target Practice).
  - *Armadilha:* grid não quadrado; 1-based; anéis contados a partir de 0 ou 1.
- **Cortes que separam origem e destino.** Cada linha/corte que separa os dois pontos tem de ser cruzado: o nº de cortes é cota inferior; mostre que cruzar cada um uma vez basta. Ex.: CF 2148B.
  - *Gatilho:* grade com obstáculos/linhas, separar duas células, contar movimentos mínimos.
  - *Por que funciona:* qualquer caminho contínuo entre os pontos cruza toda curva que os separa; se os cortes são independentes, cruzar cada um uma vez é suficiente.
  - *Como fazer:* identifique as linhas/colunas/arestas que separam origem e destino; conte-as; construa um caminho que cruze cada uma exatamente uma vez.
  - *Exemplo:* CF 2148B.
  - *Armadilha:* cortes que compartilham células; se o caminho tem restrições, o número de cortes pode não ser suficiente.

Repetidas de outras seções:

- **Pontos de rede num segmento** → gcd(|dx|, |dy|) + 1; Pick com isso; círculo de Gauss O(r).
- **Dualidade ponto-reta ⇔ CHT / Li Chao** (hull superior de pontos = envelope inferior de retas); uso em DP (estruturas).
- **Dominância 2D/3D e frente de Pareto** → BIT/CDQ por coordenada (estruturas); frente de Pareto (não dominados) ≠ camadas convexas: um ponto do hull pode ser dominado (ex.: (0,0) com (1,1) presente).
- **Área da união de retângulos/segmentos** → sweep + segtree com (cobertura, comprimento coberto) (estruturas).

Aleatorização

Gatilho: "um elemento aparece em pelo menos metade" ou "a resposta é compartilhada por muitos" → sorteie e verifique.

- **Elemento frequente** (≥ n/2, ≥ n/k): escolha aleatória acerta com probabilidade ≥ 1/2 (≥ 1/k no caso n/k); repita 30–60 vezes (k vezes mais no caso n/k).
  - *Gatilho:* "há um elemento que aparece em mais da metade", "≥ n/k ocorrências", voto majoritário.
  - *Por que funciona:* um sorteio uniforme cai no conjunto frequente com probabilidade ≥ 1/2 (ou ≥ 1/k); cada tentativa é verificada em O(n), e tentativas independentes multiplicam a chance de falha.
  - *Como fazer:* sorteie índices e conte ocorrências do valor sorteado; 30 tentativas dão falha ≤ 2^{-30} no caso 1/2; para 1/k use ~k·ln(1/ε). Para maioria estrita, Boyer–Moore é determinístico.
  - *Exemplo:* vetor com 51% de um valor → 30 sorteios quase certamente o encontram.
  - *Armadilha:* "≥ n/2" vs "> n/2"; k perto de n torna a amostragem inútil; semente fixa pode ser hackeada.

- **gcd de pelo menos metade** → sorteie um a_i, teste cada divisor dele.
  - *Gatilho:* "existe d que divide pelo menos metade", "escolha um subconjunto com gcd grande".
  - *Por que funciona:* se g divide metade dos números, um a_i sorteado dessa metade é múltiplo de g com probabilidade ≥ 1/2; g está entre os divisores de a_i.
  - *Como fazer:* sorteie 20–30 a_i; fatore cada um; para cada divisor d, conte quantos a_j são múltiplos de d; guarde o maior d com contagem ≥ n/2.
  - *Exemplo:* 6 números, 4 múltiplos de 6 → sorteando um deles, 6 aparece entre os divisores.
  - *Armadilha:* fatorar cada a_i custa; número de divisores pode ser grande; a condição é "pelo menos metade" exata.
- **Hashing aleatório de conjuntos/multiconjuntos**: atribua um número aleatório de 64 bits a cada valor; soma identifica o multiconjunto; xor só identifica conjunto/paridade ({a, a} = ∅), não multiplicidade.
  - *Gatilho:* comparar multiconjuntos/conjuntos, igualdade de coleções, anagramas.
  - *Por que funciona:* a soma de hashes independentes é um fingerprint linear; se os multiconjuntos diferem, a diferença é uma combinação linear não trivial de valores aleatórios e colide com probabilidade $\sim 2^{-64}$.
  - *Como fazer:* gere h(v) com splitmix64 e semente aleatória; some para multiconjunto, xore para conjunto; use hash de prefixo para comparar intervalos.
  - *Exemplo:* duas strings são anagramas ⇔ somas de hashes iguais.
  - *Armadilha:* xor só vê paridade, então {a,a} colide com ∅; use soma para multiplicidade; valores com coeficientes pares reduzem a margem para $\sim 2^{-63}$.
- **XOR hashing**: "todo valor aparece um número par de vezes em [l, r]" → prefix xor dos hashes = 0.
  - *Gatilho:* "cada valor aparece um número par de vezes", paridade de frequências num intervalo.
  - *Por que funciona:* o xor é a soma em GF(2); todos os valores com frequência par se cancelam, então o prefixo xor é igual nas duas pontas do intervalo.
  - *Como fazer:* h[i] = h[i−1] xor hash(a[i]); o intervalo [l, r] tem todas as frequências pares ⇔ h[r] xor h[l−1] = 0.
  - *Exemplo:* [1,2,1,2] → prefixos iguais nas pontas.
  - *Armadilha:* só testa paridade, não contagens exatas; hash do valor 0; colisão com prob 2^{-64}.
- **Shuffle da entrada** para quebrar casos ruins (inserção em BST, incremental de círculo mínimo em O(n) esperado).
  - *Gatilho:* algoritmo incremental cuja ordem de entrada pode ser adversarial.
  - *Por que funciona:* a ordem aleatória torna o custo esperado independente da entrada; a análise backward mostra que cada elemento "atrás" muda a solução com probabilidade pequena.
  - *Como fazer:* embaralhe com mt19937_64 antes de processar; vale para BST, Welzl, Seidel, quicksort, closest pair em grade.
  - *Exemplo:* círculo mínimo O(n) esperado após shuffle.
  - *Armadilha:* embaralhar depois de construir a estrutura não ajuda; semente fixa é hackeável; o shuffle custa O(n).
- **Color coding**: caminho simples de k vértices → pinte com k cores aleatórias e faça DP em máscara; a chance de um conjunto fixo ficar colorido é k!/k^k ≈ e^{−k}, então repita Θ(e^k) vezes.
  - *Gatilho:* caminho/ciclo simples com k vértices, k pequeno, "existe caminho com exatamente k vértices".
  - *Por que funciona:* se o caminho for colorido com k cores distintas, o DP sobre o conjunto de cores o encontra; a probabilidade de isso acontecer para o caminho ótimo é k!/k^k.
  - *Como fazer:* repita Θ(e^k) colorações; DP[mask][v] = alcançável usando o conjunto de cores mask terminando em v; transições por aresta; complexidade O(e^k · 2^k · (n + m)).
  - *Exemplo:* caminho simples de 4 vértices em grafo esparso.
  - *Armadilha:* sem a máscara o DP repetiria vértices; k acima de ~10 fica caro; derandomização por perfect hash families.
- **Matriz/polinômio** → teste de Freivalds ou Schwartz-Zippel avaliando em ponto aleatório.
  - *Gatilho:* verificar produto de matrizes/polinômios mais rápido que recalcular; testar identidade.
  - *Por que funciona:* um polinômio não nulo de grau d se anula em ponto aleatório com probabilidade ≤ d/|F|; no caso de matrizes, um vetor aleatório detecta diferença com probabilidade ≥ 1/2.
  - *Como fazer:* Freivalds: sorteie r e compare A(Br) com Cr em $O(n^2)$; Schwartz–Zippel: avalie em x aleatório módulo primo grande; repita 20–30 vezes.
  - *Exemplo:* verificar C = A·B sem multiplicar A por B.
  - *Armadilha:* corpo pequeno demais; r = 0; erro por tentativa exige repetição; overflow na aritmética modular.

- **Anti-hack:** use mt19937_64 com semente de chrono e hash customizado no unordered_map.
  - *Gatilho:* contest com hacks, unordered_map, hashing determinístico.
  - *Por que funciona:* um hash determinístico pode ser atacado com chaves que colidem; a semente aleatória em tempo de execução torna o ataque impraticável.
  - *Como fazer:* semeie mt19937_64 com chrono::steady_clock::now().time_since_epoch().count() (e um pouco de entropia do ambiente); use splitmix64 no unordered_map.
  - *Exemplo:* unordered_map com chaves múltiplas do primo de bucket.
  - *Armadilha:* semente previsível (tempo fixo) ainda é hackeável; não use rand() da libc; reenvio com semente diferente muda o resultado.

adicionado em 2026-11-10 por agentes; revisado

- **Karger (corte mínimo global).** Contraia arestas aleatórias até restar 2 vértices; sucesso ≥ $2/(n(n-1))$; repita $O(n^2 \log n)$ vezes (ou Karger–Stein em $O(n^2 \log^3 n)$). Hoje Stoer–Wagner $O(nm)$ é determinístico e mais simples para $n \le 500$.
  - *Gatilho:* corte mínimo global de grafo não direcionado, "separar em duas partes com poucas arestas".
  - *Por que funciona:* um corte mínimo fixo sobrevive a uma contração aleatória com probabilidade ≥ $1 - 2/i$ em cada passo; o produto dá $2/(n(n-1))$, então repetir amplifica.
  - *Como fazer:* escolha aresta uniforme, contraia (multigrafo com pesos), repita até 2 vértices; repita $O(n^2 \log n)$ vezes e pegue o menor corte; Karger–Stein reduz o custo.
  - *Exemplo:* grafo com um corte de 1 aresta → contrações aleatórias o preservam com prob $\sim 2/n^2$.
  - *Armadilha:* arestas paralelas contam com peso; o produto supõe aresta uniforme por peso; dirigido exige outra formulação.
- **Treap.** Prioridade aleatória no BST dá altura $O(\log n)$ esperada independentemente da entrada; split/merge por prioridades. É a base de sequências implícitas.
  - *Gatilho:* ordered set com split/merge, sequência com inserções/remoções no meio, reversão de intervalo.
  - *Por que funciona:* as prioridades aleatórias fazem a árvore ter a mesma distribuição de uma BST construída por inserção aleatória; a altura esperada é $O(\log n)$ qualquer que seja a ordem das chaves.
  - *Como fazer:* nó com chave, prioridade aleatória e tamanho; split por chave, merge escolhendo a maior prioridade; treap implícito usa o tamanho do filho esquerdo como índice.
  - *Exemplo:* sequência com corte e colagem (split/merge em $O(\log n)$).
  - *Armadilha:* prioridades iguais; recursão profunda; semente previsível.
- **Hash polinomial seguro.** Módulo $2^{61} - 1$ com base aleatória em $[2, p-2]$; dupla base mod $10^9 + 7$ não basta (colisão por par $\sim (n/p)^2 \approx 10^{-8}$ para $n = 10^5$, e em n^2 comparações isso colide); probabilidade de colisão por par ≤ n/p por módulo (grau do polinômio sobre o corpo). Para n^2 comparações use $2^{61} - 1$ com duas bases (ou dois primos grandes).
  - *Gatilho:* igualdade de substrings, LCP, busca de padrão, comparações em massa.
  - *Por que funciona:* para base aleatória, a diferença entre dois polinômios de grau ≤ n tem no máximo n raízes; a probabilidade de a base ser uma raiz é ≤ n/p.
  - *Como fazer:* h = Σ s_i·b^{n−i} mod p; pré-compute potências e hashes de prefixo; use $p = 2^{61} - 1$ (multiplicação com __int128) e duas bases independentes.
  - *Exemplo:* comparar duas substrings de 10^5 em $O(1)$.
  - *Armadilha:* um único mod 10^9+7 tem colisão por par $\sim 10^{-4}$; em 10^{10} comparações isso vira ~100 colisões esperadas; base 0/1 ou fixa é atacável.
- **Anti-anti-hash.** Nunca use base fixa, mod 2^{64} natural (Thue–Morse quebra qualquer base), nem base pequena previsível; ler a base de chrono::steady_clock e de /dev/urandom/time. Em Codeforces sempre há hacks.
  - *Gatilho:* problemas com hashing em contests com hacks, testes adversariais.
  - *Por que funciona:* a sequência de Thue–Morse produz colisões sistemáticas no hash módulo 2^{64}; uma base aleatória grande e dois módulos tornam a colisão improvável e imprevisível.
  - *Como fazer:* base aleatória em runtime, $p = 2^{61} - 1$, duas bases; nunca mod 2^{64}; teste local com anti-hash tests.
  - *Exemplo:* hash mod 2^{64} com base 131 quebra com Thue–Morse.
  - *Armadilha:* base aleatória mas pequena; um só primo; ler a base de uma fonte previsível.

- **Hash de unordered_map customizado.** splitmix64 com semente aleatória: o padrão do GCC é atacável (primos de bucket); alternativamente use gp_hash_table com hash customizado ou map/sort.
  - *Gatilho:* TLE em unordered_map com chaves adversariais.
  - *Por que funciona:* o hash padrão para inteiros é a identidade e o número de buckets é primo; chaves múltiplas desse primo colidem todas. splitmix64 espalha os bits e a semente aleatória impede pré-computação.
  - *Como fazer:* struct Hash { size_t operator()(uint64_t x) const { static const uint64_t FIXED_RANDOM = chrono::steady_clock::now().time_since_epoch().count(); return splitmix64(x + FIXED_RANDOM); } }; use em unordered_map.
  - *Exemplo:* chaves 0, p, 2p, ... com p primo de bucket.
  - *Armadilha:* o hash precisa ser estável durante a execução; semente por elemento quebra a consistência; gp_hash_table também precisa de hash customizado.
- **Pollard rho.** Fatora $n \leq 10^{18}$ em O(n^(1/4)) esperado com ciclo de Floyd/Brent e gcd em lotes; combine com Miller–Rabin recursivo. Paradoxo do aniversário explica o $\sqrt{p}$.
  - *Gatilho:* fatorar números de 64 bits, muitos casos, encontrar um fator não trivial.
  - *Por que funciona:* iterando $f(x) = x^2 + c \bmod p$, a sequência entra em ciclo em $O(\sqrt{p})$ passos (aniversário); dois pontos com o mesmo valor mod p têm diferença múltipla de p, e o gcd com n revela p.
  - *Como fazer:* Miller–Rabin para primalidade; escolha c aleatório; Floyd/Brent para achar ciclo; gcd em lotes de produtos; divida e recursão.
  - *Exemplo:* fatorar um semiprima de 10^{18} com fatores $\sim 10^9$.
  - *Armadilha:* c = 0 ou −2 degenera o ciclo; gcd = 1 por muito tempo; use __int128 na multiplicação modular.
- **Freivalds.** Verificar A·B = C em $O(n^2)$: sorteie $r \in \{0,1\}^n$ (ou do corpo) e compare A(Br) = Cr; erro ≤ 1/2 por rodada. Também verifica identidade de matrizes inversas e produtos de polinômios.
  - *Gatilho:* verificar multiplicação de matrizes sem $O(n^3)$, checar C = A·B.
  - *Por que funciona:* se D = AB − C ≠ 0, então Dr ≠ 0 com probabilidade ≥ 1/2 para r aleatório em $\{0,1\}^n$; repetindo, o erro cai exponencialmente.
  - *Como fazer:* sorteie r, calcule Br e A(Br) (dois produtos matriz-vetor $O(n^2)$), compare com Cr; repita 20–30 vezes.
  - *Exemplo:* conferir a saída de uma multiplicação $O(n^3)$ em $O(n^2)$ por rodada.
  - *Armadilha:* r = 0; erro de 1/2 por rodada exige repetição; overflow na aritmética inteira (use módulo).
- **Schwartz–Zippel.** Polinômio não nulo de grau d em corpo F se anula em ponto aleatório com prob ≤ d/|F|. Testa identidade de polinômios, igualdade de multiconjuntos (Π(x − a_i)), determinante nulo (matching).
  - *Gatilho:* identidade de polinômios, igualdade de produtos, determinante nulo, teste de matching.
  - *Por que funciona:* um polinômio não nulo de grau d tem no máximo d raízes; a probabilidade de sortear uma delas é ≤ d/|F|.
  - *Como fazer:* avalie em x aleatório módulo um primo grande; repita; para multiconjuntos compare Π(x − a_i); para matching use a matriz de Tutte.
  - *Exemplo:* verificar se Π(x − a_i) = Π(x − b_i).
  - *Armadilha:* |F| pequeno em relação ao grau; polinômio identicamente nulo; avaliar em pontos não aleatórios.
- **Matching por determinante (Tutte/Edmonds).** Matriz de Tutte com variáveis aleatórias mod p: grafo tem matching perfeito ⇔ det ≠ 0 (com alta prob., n par); o rank da matriz de Tutte é 2ν(G), logo o matching máximo é ν = rank/2, em $O(n^3)$ para grafos gerais.
  - *Gatilho:* matching perfeito/máximo em grafo geral, n ≤ ~500.
  - *Por que funciona:* o determinante da matriz de Tutte é um polinômio que é não nulo exatamente quando existe matching perfeito; avaliar variáveis aleatórias módulo p detecta isso com alta probabilidade.
  - *Como fazer:* monte M[i][j] = x_ij para aresta ij; substitua por valores aleatórios mod p; eliminação gaussiana $O(n^3)$; rank/2 dá o matching máximo.
  - *Exemplo:* grafo ímpar não tem matching perfeito → determinante 0.
  - *Armadilha:* probabilidade de falso zero ≤ n/p; p precisa ser grande; para bipartido, Hopcroft–Karp é mais simples.
- **Reservoir sampling.** Escolha uniforme de 1 em stream de tamanho desconhecido: o i-ésimo substitui com prob 1/i; k itens: i-ésimo entra com prob k/i substituindo um aleatório do reservatório.
  - *Gatilho:* amostra uniforme de stream com tamanho desconhecido ou muito grande.
  - *Por que funciona:* por indução, após i itens cada um está no reservatório com probabilidade k/i; a troca com probabilidade k/i mantém a uniformidade.
  - *Como fazer:* guarde os k primeiros; para i > k, j = rand(0, i−1); se j < k, res[j] = a[i]; para k = 1, substitua com prob 1/i.
  - *Exemplo:* sortear uma linha uniforme de um arquivo enorme.
  - *Armadilha:* off-by-one no intervalo de rand; viés de módulo; k > n.

- **Amostragem uniforme em estruturas.** Elemento uniforme de conjunto com remoções → vetor + swap-pop; uniforme em segtree → desça escolhendo filho com prob proporcional à soma; sorteio ponderado → prefix sums + binária ou alias method.
  - *Gatilho:* sortear elemento de conjunto dinâmico, nó de árvore, distribuição ponderada.
  - *Por que funciona:* swap-pop mantém o vetor compacto e a remoção O(1); a descida na segtree escolhe o filho com probabilidade proporcional ao peso, o que induz a distribuição correta.
  - *Como fazer:* vetor + mapa posição; para segtree, r = rand(0, soma−1) e desça comparando com a soma do filho esquerdo; para pesos estáticos, alias method O(1) por consulta.
  - *Exemplo:* sortear vértice com probabilidade proporcional ao grau.
  - *Armadilha:* updates invalidam prefix sums; alias precisa de reconstrução; viés de módulo.
- **Las Vegas vs Monte Carlo.** Las Vegas: sempre certo, tempo aleatório (quicksort aleatório, Welzl, seleção); Monte Carlo: tempo fixo, erro com prob. pequena (Freivalds, Miller–Rabin). Amplifique Monte Carlo repetindo; converta para Las Vegas se há verificador barato.
  - *Gatilho:* classificar/decidir o tipo de algoritmo aleatório, escolher estratégia de repetição.
  - *Por que funciona:* Las Vegas troca tempo por certeza; Monte Carlo troca certeza por tempo; repetir um Monte Carlo com erro unilateral reduz a falha exponencialmente.
  - *Como fazer:* se há verificador O(poly), rode Monte Carlo e verifique (vira Las Vegas); senão repita e use a resposta majoritária.
  - *Exemplo:* quicksort aleatório é Las Vegas; Freivalds é Monte Carlo.
  - *Armadilha:* confundir tempo esperado com pior caso; erro unilateral vs bilateral; verificador caro.
- **Probabilidade de falha e repetições.** Se cada tentativa acerta com prob p, k = ⌈ln(1/ε)/p⌉ tentativas dão falha ≤ ε. Elemento que cobre ≥ 1/2: 30 tentativas ⇒ 2^{-30}; cobre ≥ 1/k: k·ln(1/ε) tentativas.
  - *Gatilho:* quantas repetições para garantir probabilidade de falha desejada.
  - *Por que funciona:* tentativas independentes falham todas com probabilidade (1−p)^k ≤ e^{−pk}; igualando a ε chega-se a k ≈ ln(1/ε)/p.
  - *Como fazer:* k = ceil(log(1/ε)/p) para p pequeno; exato: k = ceil(ln ε / ln(1−p)); para p = 1/2, ε = 10^{-9} → ~30; para 1/k → k·ln(1/ε).
  - *Exemplo:* maioria com 30 sorteios → falha $2^{-30} \approx 10^{-9}$.
  - *Armadilha:* tentativas não independentes; p desconhecido; arredondar para baixo; usar a fórmula aproximada com p grande.
- **Sorteio por pares (amostragem de pares).** "Existe par/triple com propriedade comum a muitos pares" → sorteie pares e teste; se a fração de pares bons é δ, ~1/δ amostras bastam. Ex.: reta com ≥ n/2 dos pontos → sorteie 2 pontos sem reposição: a reta por eles acerta com prob (n−2)/(4(n−1)) < 1/4 (com reposição, ≥ 1/4).
  - *Gatilho:* "há uma reta/par/tripla que contém muitos", propriedade compartilhada por uma fração dos pares.
  - *Por que funciona:* um par uniforme cai no conjunto bom com probabilidade δ; tentativas independentes dão falha (1−δ)^k.
  - *Como fazer:* sorteie pares (com ou sem reposição), teste a propriedade e amplifique; para a reta com metade dos pontos, a probabilidade é ≥ (n−2)/(4(n−1)).
  - *Exemplo:* metade dos pontos numa reta → ~4 sorteios de pares bastam para achá-la com prob alta.
  - *Armadilha:* pares do mesmo elemento; sem reposição muda o denominador; δ muito pequeno exige muitas amostras.
- **Maioria/mediana aproximada por amostragem.** A mediana de uma amostra de k elementos está dentro de ε·n do verdadeiro com alta prob. se $k = O(1/\varepsilon^2 \cdot \log(1/\delta))$; serve para quantis, pivôs de seleção e para estimar contagens.
  - *Gatilho:* mediana/quantil aproximado, pivô barato, estimativa de distribuição.
  - *Por que funciona:* a CDF empírica de uma amostra converge para a verdadeira (Dvoretzky–Kiefer–Wolfowitz); com k amostras, o quantil amostral está perto do real.
  - *Como fazer:* sorteie k elementos, ordene, use o quantil amostral; para pivô, use a mediana da amostra.
  - *Exemplo:* estimar a mediana de 10^9 elementos com $k = 10^4$.
  - *Armadilha:* quantis extremos precisam de mais amostras; amostragem enviesada; k depende de δ.
- **Shuffle em guloso e DP aleatório.** Quando a ordem de processamento arbitrária tem pior caso adversarial (ex.: inserções em BST, círculo mínimo, LP incremental), embaralhe a entrada: esperado O(n) ou O(n log n) pela análise de "elemento de trás" (probabilidade 3/i de mudar o ótimo).
  - *Gatilho:* algoritmo incremental cuja complexidade depende da ordem, entrada adversarial.
  - *Por que funciona:* na análise backward, o i-ésimo elemento muda a solução com probabilidade ≤ d/i (d = tamanho da base); o custo esperado é a soma dessa probabilidade vezes o custo de reparo.
  - *Como fazer:* embaralhe antes de processar; use a mesma ordem para todas as estruturas dependentes; no círculo mínimo d = 3.
  - *Exemplo:* Welzl sem shuffle pode ser $O(n^3)$; com shuffle é O(n) esperado.
  - *Armadilha:* embaralhar só uma parte; custo de reparo não constante; dependências de DP quebradas.

- **Análise "backward".** O i-ésimo ponto muda a solução com prob ≤ d/i (d = tamanho da base); o custo esperado total é Σ d/i · custo(refazer) — sustenta Welzl, Seidel, closest pair randomizado e quicksort.
  - *Gatilho:* provar complexidade esperada de um algoritmo incremental aleatorizado.
  - *Por que funciona:* condicionando nos primeiros i elementos, o i-ésimo só é relevante se pertencer à base do ótimo, o que tem probabilidade ≤ d/i; somando i = 1..n, o total é O(d·n·custo).
  - *Como fazer:* identifique a base (círculo: 3, LP em dimensão d: d, quicksort: pivô), multiplique a probabilidade pelo custo de reconstrução e some.
  - *Exemplo:* Welzl: Σ 3/i · O(i) = O(n).
  - *Armadilha:* d precisa ser constante; custo de reparo pode não ser linear; a análise vale para a ordem aleatória, não para qualquer ordem.
- **Randomized incremental.** Insira em ordem aleatória, repare quando o novo elemento viola: círculo mínimo, LP, hull 2D/3D (conflict graph), Delaunay, trapezoidal map, closest pair em grade, contagem de componentes.
  - *Gatilho:* construir estrutura geométrica/otimização incremental, ordem não importa para a resposta.
  - *Por que funciona:* a ordem aleatória garante que cada inserção viola a estrutura com probabilidade pequena; o reparo é local e o total esperado é O(n log n) ou O(n).
  - *Como fazer:* insira um a um; ao violar, reconstrua a parte afetada (ou use conflict graph); mantenha a estrutura consistente.
  - *Exemplo:* Delaunay por inserção aleatória com conflict graph.
  - *Armadilha:* manutenção do conflict graph é custosa; degenerescência; um adversário que conhece a semente.
- **Simulated annealing.** Soluções vizinhas por swap/perturbação; aceite pior com prob exp(−Δ/T), T decai geometricamente até o tempo limite; funciona em problemas de "maximize a pontuação" com TL 2–10 s e sem solução exata (AtCoder Heuristic, CF Marathon). Use avaliação incremental O(1) do delta.
  - *Gatilho:* otimização combinatória sem solução exata, pontuação, TL de segundos, maratona.
  - *Por que funciona:* aceitar pior com probabilidade decrescente permite escapar de mínimos locais; com T caindo, o processo converge para uma boa solução.
  - *Como fazer:* gere vizinho por swap/perturbação; calcule Δ em O(1) com estado incremental; aceite se Δ ≥ 0 ou com prob exp(−Δ/T); T *= 0.999 até o limite de tempo.
  - *Exemplo:* minimizar custo de atribuição com trocas aleatórias.
  - *Armadilha:* T inicial/escala; exp de Δ/T grande estoura; avaliação não incremental domina o tempo; semente.
- **Hill climbing com restarts.** Para otimização sem mínimos locais ruins: comece de ponto aleatório, melhore até estagnar, repita; guarde o melhor. Ótimo para problemas de geometria/continuo com função suave (centro de círculo mínimo, ponto ótimo).
  - *Gatilho:* função suave, ótimo contínuo, muitos mínimos locais.
  - *Por que funciona:* cada subida converge a um ótimo local; reinícios aleatórios exploram bacia diferentes e o melhor global aparece com mais tentativas.
  - *Como fazer:* ponto inicial aleatório; enquanto houver vizinho melhor, mova; quando estagnar, reinicie; guarde o melhor; ajuste o passo.
  - *Exemplo:* centro de círculo mínimo por subida com reinícios.
  - *Armadilha:* orçamento de avaliações; passo fixo grosseiro; função com platôs.
- **Heurística de "gradiente por perturbação".** Para otimizar em $\mathbb{R}^d$ sem derivada: passo de tamanho decrescente com tentativas aleatórias, redução do passo pela metade quando não melhora (útil p/ Fermat, círculo mínimo, ponto que minimiza o máximo).
  - *Gatilho:* otimização contínua sem derivada, d pequeno, função cara.
  - *Por que funciona:* uma perturbação aleatória tem componente na direção de descida com boa probabilidade; reduzir o passo quando não melhora dá convergência.
  - *Como fazer:* comece com passo s; tente x ± s·direção aleatória; aceite se melhorar; se não, s /= 2; pare quando s < eps.
  - *Exemplo:* ponto que minimiza a máxima distância a um conjunto.
  - *Armadilha:* d alto (maldição da dimensionalidade); passo inicial; mínimo local; orçamento.
- **Tempo como parâmetro.** Em heurísticas, itere "enquanto tempo < limite" com chrono; deixe 10% de folga. Pré-calcule tabelas de exp/log/rand para ganhar iterações, e use xorshift em vez de mt19937 no laço interno.
  - *Gatilho:* heurística com TL fixo, número de iterações imprevisível.
  - *Por que funciona:* o número de iterações que cabe no tempo varia com a máquina; medir o tempo usa o orçamento completo e evita TLE.
  - *Como fazer:* auto t0 = chrono::steady_clock::now(); while (elapsed < 0.9·TL) {...}; pré-compute exp/log em tabela; xorshift no laço.
  - *Exemplo:* SA rodando até 1.8 s num TL de 2 s.
  - *Armadilha:* chamar chrono a cada iteração custa; margem pequena; diferenças de TL entre judges.

- **Local search para cobertura/coloração.** Mudar um elemento por vez com delta de custo O(grau), tabu list pequena ou "kick" aleatório quando preso; solução inicial gulosa aleatória.
  - *Gatilho:* coloração de grafo, cobertura, atribuição com restrições.
  - *Por que funciona:* o custo muda localmente ao trocar um elemento; tabu/kick evitam ciclos e mínimos locais rasos.
  - *Como fazer:* inicialize com guloso aleatório; avalie trocas de um elemento com Δ O(grau); aceite a melhor não-tabu; se preso, perturbe (kick).
  - *Exemplo:* coloração com k cores minimizando conflitos.
  - *Armadilha:* erro no delta; tabu tenure; manter a factibilidade.
- **Beam search.** Quando o espaço é uma árvore de decisões com função de avaliação razoável: mantenha as K melhores configurações por nível (hash para deduplicar); típico em heurísticas de AtCoder.
  - *Gatilho:* construção sequencial com muitos estados, avaliação parcial confiável.
  - *Por que funciona:* em vez de explorar toda a árvore, mantém um feixe das K melhores parciais; deduplicar evita repetir estados.
  - *Como fazer:* expanda cada estado em filhos, pontue, ordene e mantenha K; use hash do estado para deduplicar; repita por nível.
  - *Exemplo:* planejamento de rotas com K = 1000.
  - *Armadilha:* K pequeno perde a solução; K grande estoura o tempo; avaliação parcial enganosa; duplicatas.
- **Greedy randomizado + melhor de N.** Execute guloso com desempates aleatórios N vezes e fique com o melhor; funciona para construtivos com TL alto e para aproximar problemas NP-difíceis pequenos.
  - *Gatilho:* construção gulosa com muitos empates, problema NP-difícil pequeno.
  - *Por que funciona:* desempates aleatórios geram construções diversas; o melhor de N tem valor esperado melhor que uma construção fixa.
  - *Como fazer:* rode o guloso N vezes com semente diferente, mantendo a melhor solução; ajuste N ao TL.
  - *Exemplo:* construção de uma cobertura de conjuntos.
  - *Armadilha:* N vs tempo; randomização não vence instâncias difíceis; semente fixa.
- **Polynomial identity para string/permutação.** Checar "a e b são permutações um do outro" → compare $\Pi(x - a_i)$ em x aleatório mod p; ou soma de f(a_i) com f aleatória (hash aditivo). "Sequência é permutação de 1..k em [l, r]" → soma de hash = soma esperada com hash de prefixos.
  - *Gatilho:* anagramas, permutação de um intervalo, comparação de multiconjuntos.
  - *Por que funciona:* o produto $\Pi(x - a_i)$ determina o multiconjunto; dois multiconjuntos distintos dão polinômios distintos, que coincidem num ponto aleatório com probabilidade ≤ grau/p.
  - *Como fazer:* avalie o produto em x aleatório mod primo grande (ou use soma de hashes aleatórios); para intervalo, compare a soma de hashes com a esperada para 1..k.
  - *Exemplo:* "a substring [l,r] é permutação de 1..(r−l+1)".
  - *Armadilha:* corpo pequeno; valores repetidos; colisão do hash aditivo; grau alto.
- **Hash de árvores (isomorfismo).** Hash de subárvore = f(multiconjunto de hashes dos filhos) com f aleatória (p. ex. $\Pi(x + h_{filho})$ mod p ou soma de φ(h)); duas árvores enraizadas iguais ⇔ hashes iguais com alta prob.; para não enraizadas, enraize no centro.
  - *Gatilho:* isomorfismo de árvores, subárvores iguais, hash de estrutura.
  - *Por que funciona:* o hash do multiconjunto de filhos identifica a estrutura; dois multiconjuntos diferentes colidem com probabilidade pequena; enraizar no centro remove a ambiguidade da raiz.
  - *Como fazer:* DFS pós-ordem; ordene os hashes dos filhos e combine com produto/soma aleatória; para árvore não enraizada, ache o centro (ou os dois centros) e compare os hashes.
  - *Exemplo:* checar se duas árvores de 10^5 nós são isomorfas.
  - *Armadilha:* filhos ordenados vs não ordenados; dois centros; colisões; recursão profunda.
- **Hash de grafos / conectividade por xor.** Cada aresta recebe valor aleatório; vértice guarda xor das arestas incidentes; um conjunto S é uma componente fechada / corte vazio ⇔ xor dos vértices = 0 (se é corte o xor é sempre 0; a recíproca vale com alta probabilidade, não deterministicamente). Detecta corte e conectividade com remoções de arestas por xor de cortes.
  - *Gatilho:* conectividade sob remoções, corte vazio, componente fechada.
  - *Por que funciona:* arestas internas a S contribuem duas vezes e se cancelam; se há aresta saindo de S, o xor inclui valores aleatórios independentes e é 0 só com probabilidade 2^{-64}.
  - *Como fazer:* atribua valores aleatórios às arestas; acumule xor por vértice; para um conjunto S, xor dos valores dos vértices; atualize remoções com xor.
  - *Exemplo:* verificar se remover um conjunto de arestas desconecta o grafo.
  - *Armadilha:* a recíproca é probabilística, não determinística; aresta com valor 0; multigrafos; dirigido exige adaptação.

- **Xor de valores aleatórios em árvore (cut hashing).** Dê a cada aresta que não está na árvore geradora um valor aleatório e a cada aresta da árvore o xor das de não-árvore que a cobrem; então arestas formam corte ⇔ xor = 0 com alta probabilidade (mincut com ≤ k arestas, pontes).
  - *Gatilho:* pontes, corte mínimo com poucas arestas, "quais arestas da árvore formam um corte".
  - *Por que funciona:* o valor de uma aresta da árvore é o xor das arestas não-árvore cujo ciclo fundamental a contém; o xor de um conjunto F de arestas da árvore é o xor das arestas não-árvore que cruzam o corte, que é 0 exatamente quando não há nenhuma (com alta prob.).
  - *Como fazer:* DFS, numere; para cada aresta não-árvore, faça xor nos extremos (diferença em árvore); o valor de cada aresta da árvore é o xor da subárvore; agrupe arestas por valor para achar cortes.
  - *Exemplo:* achar todas as pontes (valor 0).
  - *Armadilha:* probabilidade 2^{-64} por corte, mas muitos cortes aumentam a chance; precisa de 64 bits independentes; laços e multigrafos.
- **Contagem aproximada (Monte Carlo).** Estimar |S| amostrando e extrapolando; para contar elementos distintos em stream → HyperLogLog/k-mínimos (mantenha os k menores hashes; estimativa = k/h_k).
  - *Gatilho:* cardinalidade de stream, contagem de distintos, memória limitada.
  - *Por que funciona:* se os hashes são uniformes, o k-ésimo menor hash h_k é aproximadamente k/n; invertendo, n ≈ k/h_k.
  - *Como fazer:* mantenha os k menores hashes (heap) ou registradores do HLL; estime k/h_k; para HLL use a média harmônica dos registradores.
  - *Exemplo:* contar IPs distintos num fluxo de 10^9.
  - *Armadilha:* hash não uniforme; n pequeno (estimativa ruim); a estimativa k/h_k é levemente viesada (a versão (k−1)/h_k é comum).
- **Seleção randomizada (quickselect).** Pivô aleatório → O(n) esperado para k-ésimo; mediana das medianas dá O(n) determinístico mas é pesado. std::nth_element já é introselect.
  - *Gatilho:* k-ésimo menor/maior, mediana, ordem estatística.
  - *Por que funciona:* o pivô aleatório divide o array em partes equilibradas em expectativa; só um lado é recursado, e a soma geométrica das expectativas dá O(n).
  - *Como fazer:* particione em três vias (<, =, >); se k cai no igual, termine; senão recurse no lado certo; ou use nth_element.
  - *Exemplo:* achar a mediana de 10^6 números sem ordenar tudo.
  - *Armadilha:* muitos iguais (use partição 3-way); comparador adversarial; recursão profunda.
- **Sorteio com viés e "prefixo ponderado".** Para escolher índice com prob ∝ w_i com updates → BIT/segtree com descida; para o resultado só em O(1) amortizado → alias method com tabela reconstruída em lotes.
  - *Gatilho:* sorteio ponderado, pesos dinâmicos, simulações.
  - *Por que funciona:* a probabilidade acumulada define intervalos; sortear r uniforme em [0, Σw) e achar o intervalo por busca é proporcional ao peso; a descida na árvore faz isso em O(log n).
  - *Como fazer:* BIT com somas e descida por r; para pesos estáticos, alias method O(n) de construção e O(1) por consulta; reconstrua em lotes se mudar.
  - *Exemplo:* sortear evento com probabilidades 0.1, 0.2, 0.7.
  - *Armadilha:* pesos zero; ponto flutuante na soma; alias desatualizado após updates.
- **Aleatorização em grafos: coloração / separação.** Particionar vértices em dois lados aleatoriamente captura qualquer aresta/par fixo com prob 1/2 (ou 1/4 para dois); use para max-cut ≥ m/2, "MAX-3SAT ≥ 7/8" (MAX-2SAT ≥ 3/4; cláusula k: 1 − 2^{−k}), e para color coding com repetição.
  - *Gatilho:* max-cut, MAX-SAT, separação de vértices, prova de existência.
  - *Por que funciona:* uma atribuição aleatória satisfaz cada cláusula/aresta com probabilidade fixa; pela linearidade da esperança, existe atribuição pelo menos na média.
  - *Como fazer:* sorteie 0/1 para cada variável/vértice; calcule a expectativa; para obter o objeto, use esperanças condicionais (derandomização).
  - *Exemplo:* max-cut tem corte ≥ m/2; MAX-3SAT tem atribuição ≥ 7m/8.
  - *Armadilha:* esperança não garante alta probabilidade; eventos correlacionados; derandomização custa.

- **Probabilistic method (existência).** Se a esperança de uma contagem é ≥ k, existe configuração com ≥ k: "max-cut ≥ m/2", "tournament com caminho hamiltoniano", "sub-conjunto independente de tamanho ≥ n/(d + 1)". Desrandomize pelo método das esperanças condicionais.
  - *Gatilho:* provar que existe objeto com certa propriedade, sem construí-lo explicitamente.
  - *Por que funciona:* se toda configuração tivesse contagem < k, a esperança seria < k; logo alguma atinge ≥ k. Para o independente, o guloso em ordem aleatória sobrevive com prob ≥ 1/(d+1) por vértice.
  - *Como fazer:* defina a variável aleatória, calcule a esperança e conclua a existência; para construir, fixe variáveis mantendo a esperança.
  - *Exemplo:* max-cut ≥ m/2; independente ≥ n/(d+1) por guloso aleatório.
  - *Armadilha:* esperança ≥ k não dá alta probabilidade; a derandomização pode ser cara; a cota do independente usa grau máximo.
- **Desaleatorização por esperança condicional.** Fixe uma variável por vez escolhendo o valor que mantém a esperança ≥ alvo: transforma o argumento probabilístico em algoritmo guloso determinístico.
  - *Gatilho:* precisa construir o objeto cuja existência o método probabilístico garante.
  - *Por que funciona:* a esperança condicional é uma média ponderada das duas escolhas; pelo menos uma mantém a esperança ≥ alvo, então a escolha gulosa nunca perde a garantia.
  - *Como fazer:* para cada variável, calcule E[contagem | escolha 0] e E[contagem | escolha 1]; escolha a maior; prossiga.
  - *Exemplo:* derandomizar max-cut escolhendo o lado de cada vértice.
  - *Armadilha:* calcular as condicionais pode ser caro; empates; a garantia é sobre a esperança, não sobre o valor final além do alvo.
- **Estimar com poucas amostras: aproximação por tamanho.** Se a resposta desejada é "existe vértice com grau ≥ d" e há muitos candidatos, amostrar por aresta aleatória escolhe vértices proporcionalmente ao grau.
  - *Gatilho:* procurar vértice/objeto de tamanho grande, muitos candidatos.
  - *Por que funciona:* escolher uma aresta uniforme e um de seus extremos dá probabilidade proporcional ao grau; vértices de grau alto são muito mais prováveis que na amostragem uniforme de vértices.
  - *Como fazer:* sorteie aresta, escolha extremo; verifique a propriedade; repita; para grau ≥ d, a chance é ≥ d/(2m/n) em média.
  - *Exemplo:* achar vértice de grau alto em grafo enorme.
  - *Armadilha:* vértices isolados nunca são sorteados; viés por grau; a propriedade pode não ser monotônica no grau.
- **Aleatorização em strings: mínima rotação / igualdade.** Igualdade de substring por hash; compare duas strings gigantes comparando poucos índices aleatórios só serve se diferem em muitas posições (teste de proximidade); para "diferem em ≥ δ n posições" uma amostra de O(1/δ) acha uma diferença.
  - *Gatilho:* igualdade de substrings, rotação mínima, distância de Hamming grande.
  - *Por que funciona:* se as strings diferem em uma fração δ das posições, uma amostra uniforme acerta uma diferença com probabilidade δ por índice; O(1/δ) tentativas dão probabilidade constante.
  - *Como fazer:* rolling hash para igualdade exata; para proximidade, sorteie índices e compare; para rotação mínima, compare com hashes ou use Booth.
  - *Exemplo:* "as strings diferem em ≥ 10% das posições" → ~10 amostras acham uma diferença.
  - *Armadilha:* strings que diferem em poucas posições nunca são detectadas por amostragem; colisão de hash; periodicidade.
- **Probabilidade e antichain: erro de "hash por soma".** Hash aditivo de multiconjunto falha para somas coincidentes; use 64 bits e valores aleatórios independentes (não valor = índice), nunca $f(x) = x^2$ etc.
  - *Gatilho:* hash de multiconjunto, comparação de coleções, anagramas.
  - *Por que funciona:* se os valores são aleatórios independentes, uma combinação linear não trivial colide com probabilidade $\sim 2^{-64}$; funções determinísticas como x^2 criam colisões estruturais (somas iguais).
  - *Como fazer:* h(v) = splitmix64(v + semente), 64 bits; some para multiconjunto; nunca use o próprio valor ou potências fixas.
  - *Exemplo:* {1,4} e {2,3} têm a mesma soma; com hash aleatório as somas diferem.
  - *Armadilha:* usar o índice como hash; módulo pequeno; confundir soma com xor (multiplicidade).

- **Cuckoo / hash perfeito offline.** Para tabelas hash com acesso O(1) pior caso; em contest, use sort + binária, ou gp_hash_table; relevante só para constante.
  - *Gatilho:* dicionário estático com muitas consultas, necessidade de pior caso O(1).
  - *Por que funciona:* duas funções de hash e evicção tipo cuckoo dão O(1) esperado (e O(1) pior caso com hash perfeito offline de duas escolhas); a tabela é construída uma vez.
  - *Como fazer:* offline: duas funções, insira e ejete até estabilizar; se ciclar, reescolha; em contest, sort + busca binária costuma bastar.
  - *Exemplo:* dicionário de 10^6 strings consultado 10^6 vezes.
  - *Armadilha:* ciclos exigem reconstrução; hash perfeito é overkill; gp_hash_table pode ser atacado sem hash customizado.

Repetidas de outras seções:

- **Método probabilístico** → atribuição aleatória ≥ esperança; derandomize por esperanças condicionais.
- **Paradoxo do aniversário** → colisão com ~√N itens; Tutte matrix aleatória → emparelhamento máximo.
- **Treap e skip list** como estruturas de altura esperada log (estruturas).
- **Hash de subárvores/isomorfismo** → árvores (hash de multiconjunto de filhos).
- **Simulated annealing/hill climbing para ótimo contínuo** → geometria (círculo mínimo, Fermat, ponto que minimiza o máximo).

Álgebra, polinômios e FFT

adicionado em 2026-11-10 por agentes; revisado

Quando a contagem vira produto de polinômios, recorrência linear ou sistema linear, a ferramenta é álgebra: pense em coeficientes, não em objetos.

- **Convolução de somas.** "Quantos pares (i, j) com a_i + b_j = s para todo s" → polinômios A(x) = Σ x^{a_i}, B(x) = Σ x^{b_j}, A·B. Diferenças a_i − b_j → inverta B (x^{M−b_j}). Triplas/PAs de 3 termos: a_i + a_j = 2a_k → A^2 e olhe nos pontos 2a_k.
  - *Gatilho:* o enunciado pede a contagem para todo valor de soma, diferença ou termo médio, e os valores são limitados o bastante para indexar um vetor.
  - *Por que funciona:* o coeficiente de x^s em A·B soma exatamente os pares (i, j) com a_i + b_j = s; para diferenças, trocar b_j por M − b_j transforma a_i − b_j em a_i + (M − b_j) − M.
  - *Como fazer:* escolha M maior que o maior valor e crie vetores de coeficientes; multiplique por NTT/FFT; para diferenças, leia o coeficiente em s + M; para PA, calcule A^2 e consulte 2a_k, descontando os casos i = j quando necessário. Custo O(V log V), V = faixa de valores.
  - *Exemplo:* contar pares com soma exata s em dois arrays; contar progressões aritméticas de 3 termos em um conjunto.
  - *Armadilha:* valores negativos exigem deslocamento; em A^2 o termo i = j entra e precisa ser removido se a PA exige índices distintos; pares ordenados e não ordenados contam diferente.
- **NTT: primos amigáveis.** 998244353 = 119·2^{23} + 1 (g = 3); 167772161 = 5·2^{25} + 1 (g = 3); 469762049 = 7·2^{26} + 1 (g = 3); 1004535809 = 479·2^{21} + 1 (g = 3); 754974721 = 45·2^{24} + 1 (g = 11). Tamanho máximo = 2^k do primo; convolução de tamanho > 2^{23} não cabe em 998244353.
  - *Gatilho:* é preciso convolução exata módulo um primo e o resultado não pode ter erro de arredondamento.
  - *Por que funciona:* se p = c·2^k + 1, o grupo multiplicativo tem uma raiz primitiva 2^k-ésima; a NTT é uma DFT sobre essa raiz e todos os valores ficam em inteiros módulo p.
  - *Como fazer:* escolha um primo cujo 2^k seja ≥ n + m − 1; use a raiz primitiva g e w = g^{(p−1)/2^k}; faça as borboletas iterativas, multiplique ponto a ponto e aplique a inversa com w^{-1} e divisão por N. Custo O(N log N).
  - *Exemplo:* multiplicar dois polinômios de grau 10^6 com coeficientes módulo 998244353; a maior transformação utilizável é 2^{23}.
  - *Armadilha:* o limite é o 2^k que divide p − 1, não p; não esqueça a normalização da inversa; produtos de dois valores perto de 10^9 ainda cabem em `long long`, mas reduza antes de multiplicar se houver acúmulo.

- **Módulo arbitrário (MTT).** Três NTTs com primos distintos + CRT (Garner), pois o coeficiente real pode chegar a $n \cdot \text{mod}^2 \approx 10^{24}$ ($n = 10^6$), que cabe no produto de três primos $\approx 10^{26}$; ou FFT em double dividindo cada coeficiente em 15 bits (4 FFTs com o truque de empacotar parte real/imaginária).
  - *Gatilho:* a convolução é módulo $10^9 + 7$, $10^9 + 9$ ou outro módulo sem raiz 2^k grande.
  - *Por que funciona:* cada NTT calcula a convolução exata em um primo; se o produto dos primos ultrapassa o maior coeficiente inteiro possível, o CRT recupera o valor antes de reduzi-lo ao módulo pedido.
  - *Como fazer:* escolha três primos NTT cujo produto seja maior que $n \cdot (m-1)^2$; rode as três convoluções, combine por Garner sem construir inteiros gigantes e reduza módulo o alvo; a alternativa é dividir cada coeficiente em partes de 15 bits e usar FFT real.
  - *Exemplo:* multiplicar polinômios com coeficientes $< 10^9$ e $n = 10^6$ módulo $10^9 + 7$; o produto dos três primos usuais é $\approx 7{,}9 \cdot 10^{25}$.
  - *Armadilha:* o limitante precisa considerar o maior coeficiente possível, não o módulo final; Garner exige inversos dos primos; coeficientes negativos devem ser normalizados.
- **Precisão da FFT em double.** Regra: $n \cdot (\text{maior coeficiente})^2 \lesssim 10^{14}$–$10^{15}$ passa sem dividir (ex.: coeficientes $\le 10^4$ com $n \approx 10^6$, ou $\le 10^5$ com $n \approx 10^5$); coeficientes $\le 10^6$ com $n \approx 10^6$ já não cabem em double; coeficientes até 10^9 exigem split ou NTT. Use raízes calculadas por cos/sin diretos (ou long double), não por multiplicação acumulada.
  - *Gatilho:* FFT com coeficientes grandes ou muitos termos, em que o resultado precisa ser inteiro exato.
  - *Por que funciona:* o erro de arredondamento cresce com o número de operações e com a magnitude dos coeficientes; a regra $n \cdot C^2$ estima a ordem do erro absoluto acumulado.
  - *Como fazer:* estime $n \cdot C^2$ antes; se estiver perto do limite, use `long double`, divida os coeficientes em blocos de 15 bits ou troque por NTT; calcule raízes com `cos/sin` e normalize no final.
  - *Exemplo:* $C \le 10^4$ e $n = 10^6$ costuma ser seguro; $C = 10^6$ e $n = 10^6$ não é.
  - *Armadilha:* multiplicar raízes sucessivamente acumula erro; `double` não garante inteiros exatos; compare o resultado com uma versão módulo pequeno quando possível.
- **Convolução cíclica / tamanho não potência de 2.** Para mod $x^n - 1$ com n arbitrário: convolução linear e dobre; para DFT de tamanho primo ou arbitrário → Bluestein (chirp-z) usa ij = C(i+j,2) − C(i,2) − C(j,2).
  - *Gatilho:* o problema pede coeficientes módulo $x^n - 1$, somas cíclicas de índice, ou uma DFT de comprimento que não é potência de 2.
  - *Por que funciona:* a convolução cíclica é a convolução linear com os coeficientes de grau ≥ n somados de volta aos índices baixos; Bluestein reescreve o produto de índices como uma convolução.
  - *Como fazer:* para o cíclico, faça a convolução linear de tamanho ≥ 2n − 1 e, para i = n..2n−2, some o coeficiente i ao índice i − n; para Bluestein, monte as sequências chirp e convolua. Custo O(n log n).
  - *Exemplo:* contar subconjuntos com soma módulo n; DFT de comprimento primo.
  - *Armadilha:* dobrar antes de ter certeza de que a convolução linear foi calculada; para Bluestein, acertar o deslocamento dos índices negativos e a raiz usada.
- **Chirp-z.** Avaliar P em $a \cdot w^k$, k = 0..m−1 (progressão geométrica) em O(n log n) com uma convolução: usa w^{jk} = w^{C(j+k,2)}/(w^{C(j,2)}·w^{C(k,2)}). Aparece em "avalie o polinômio em todas as raízes da unidade de ordem n qualquer".
  - *Gatilho:* avaliação em pontos que formam progressão geométrica, ou em raízes n-ésimas com n arbitrário.
  - *Por que funciona:* a identidade jk = C(j+k,2) − C(j,2) − C(k,2) separa o expoente em uma parte que depende de j+k, uma de j e uma de k; a parte de j+k é uma convolução.
  - *Como fazer:* calcule u_j = a^j·P_j·w^{C(j,2)} e v_t = w^{−C(t,2)}; convolua u e v; o valor em k é w^{C(k+1,2)} vezes o coeficiente de índice k da convolução. Custo O((n+m) log(n+m)).
  - *Exemplo:* DFT de comprimento primo; avaliar P em todas as raízes de $x^n = 1$ sem exigir n potência de 2.
  - *Armadilha:* o sinal do chirp e o índice de deslocamento são a fonte usual de erro; trabalhe com expoentes reduzidos módulo a ordem da raiz.
- **Inverso de série.** B ← B·(2 − A·B) mod x^{2k} dobra a precisão (Newton); exige $a_0 \neq 0$. Custo total O(n log n) com constante ≈ 3 multiplicações (convolução de tamanho 2n).
  - *Gatilho:* é preciso dividir séries, inverter um polinômio módulo x^n ou implementar ln/exp.
  - *Por que funciona:* se A·B ≡ 1 mod x^k, então B·(2 − A·B) ≡ 1 mod x^{2k}; o erro atual é elevado ao quadrado e dobra o número de coeficientes corretos.
  - *Como fazer:* comece com $B = 1/a_0$; em cada passo dobre k, calcule A·B truncado e depois B·(2 − A·B); use NTT. O custo geométrico é O(n log n), com cerca de duas convoluções por nível.
  - *Exemplo:* inverso de $1 - x - x^2$ módulo x^n; base para divisão de polinômios.
  - *Armadilha:* $a_0 = 0$ não tem inverso; truncar no tamanho certo em cada passo; não usar o B antigo sem atualizar a precisão.

- **ln, exp, sqrt, potência.** ln A = ∫ A′/A ($a_0 = 1$); exp (exige A(0) = 0): B ← B·(1 − ln B + A); sqrt: B ← (B + A/B)/2 (a_0 quadrado, normalmente 1); A^k = exp(k·ln A) para $a_0 = 1$; senão fatore x^t e a_0^k (k grande: guarde k mod p e k mod (p−1) separados).
  - *Gatilho:* a fórmula envolve logaritmo, exponencial, raiz ou potência de uma série formal.
  - *Por que funciona:* a derivada do logaritmo é A′/A; a exponencial e a raiz usam Newton; potência vira composição de log e exp quando o termo constante é 1.
  - *Como fazer:* para ln, derive, multiplique pelo inverso, integre; para exp, itere B ← B·(1 − ln B + A) dobrando a precisão; para sqrt, itere B ← (B + A/B)/2; para A^k, faça exp(k·ln A). Tudo O(n log n) com inverso e NTT.
  - *Exemplo:* exp(x), sqrt(1 + x), (1 + x)^k módulo x^n.
  - *Armadilha:* ln exige $a_0 = 1$, exp exige A(0) = 0 e sqrt exige a_0 quadrado; em potência com $a_0 \neq 1$, fatore a_0^k e x^t; para k gigante, reduza k módulo p e módulo p − 1 conforme o coeficiente.
- **Divisão de polinômios.** rev(Q) = rev(A)·rev(B)$^{-1}$ mod x^{n−m+1}, depois R = A − QB. É o bloco básico de multipoint e de "x^N mod f(x)".
  - *Gatilho:* quociente e resto de A por B, redução módulo um polinômio, ou árvore de restos.
  - *Por que funciona:* inverter a ordem dos coeficientes transforma a divisão polinomial em uma divisão de séries; o quociente revertido é o produto do A revertido pelo inverso do B revertido.
  - *Como fazer:* n = deg A, m = deg B; tome rev(A) e rev(B), inverta rev(B) módulo x^{n−m+1}, multiplique e reverta para obter Q; calcule R = A − BQ. Custo O(n log n).
  - *Exemplo:* dividir $x^5 + 1$ por $x^2 + 1$; resto de P módulo x − a em multipoint.
  - *Armadilha:* o coeficiente líder de B precisa ser invertível; se deg A < deg B, Q = 0 e R = A; truncar no comprimento n − m + 1.
- **Produto de muitos polinômios.** ∏(x − r_i) ou ∏(1 + a_i x) por D&C com pares do mesmo tamanho: O(n $\log^2$ n). Em problemas de "escolher subconjunto com peso", o produto ∏(1 + x^{w_i}) agrupa por peso e usa potência binomial.
  - *Gatilho:* construir o polinômio com raízes dadas, ou a função geradora de todos os subconjuntos.
  - *Por que funciona:* multiplicar em pares mantém as alturas balanceadas; cada nível custa O(n log n) no total e há O(log n) níveis.
  - *Como fazer:* use uma fila de prioridade por grau ou divisão e conquista; multiplique com NTT; se muitos fatores são iguais, agrupe (1 + x^w)^{c} e expanda com binomiais. Custo O(n $\log^2$ n).
  - *Exemplo:* ∏(x − i) para i = 1..n; ∏(1 + x^{w_i}) para contagem de subconjuntos por soma.
  - *Armadilha:* multiplicar sequencialmente custa $O(n^2)$; a soma dos graus é n, então não confunda com o número de fatores; potências binomiais precisam de fatoriais.
- **Knapsack de contagem via ln/exp.** Subconjuntos de soma s (cada uma das cnt_w cópias entra 0/1): ∏(1 + x^w)^{cnt_w} = exp(Σ cnt_w·ln(1 + x^w)); mochila ilimitada (multiconjunto, cópias ilimitadas de cada peso): exp(Σ cnt_w Σ_k x^{wk}/k) = ∏(1 − x^w)^{−cnt_w}; limitada a cnt_w cópias por peso: ∏(1 − x^{w(cnt_w+1)})/(1 − x^w) — soma harmônica S log S para montar o argumento.
  - *Gatilho:* contar subconjuntos ou multiconjuntos por soma, com muitas cópias do mesmo peso.
  - *Por que funciona:* o produto de fatores independentes vira soma dentro do logaritmo; a exponencial da série resultante reconstrói a função geradora em O(S log S) quando o argumento é esparso.
  - *Como fazer:* monte C(x) = Σ_w cnt_w·ln(1 + x^w) para o caso 0/1; no ilimitado use Σ_w cnt_w Σ_{k≥1} x^{wk}/k; no limitado use a diferença dos dois produtos; calcule exp(C) com FPS. Percorrer múltiplos de cada w custa S log S.
  - *Exemplo:* contar formas de somar S com moedas de valores dados e quantidades limitadas.
  - *Armadilha:* 0/1, ilimitado e limitado têm fatores diferentes; o limite S precisa incluir w·cnt_w; a exponencial é módulo primo e exige C(0) = 0.
- **Partições.** p(n) para todos até N: 1/∏(1 − x^k) por pentagonal (Euler) em O(N√N): p(n) = Σ (−1)^{k+1}[p(n − k(3k−1)/2) + p(n − k(3k+1)/2)]. Partições em partes distintas = ∏(1 + x^k); em partes ≤ k: 1/∏_{i≤k}(1 − x^i). Valores: p(100) = 190 569 292.
  - *Gatilho:* contar partições de inteiros, com restrições de partes distintas ou limitadas.
  - *Por que funciona:* a função geradora 1/∏(1 − x^k) tem coeficientes p(n); o teorema pentagonal dá uma recorrência esparsa com O(√n) termos por n.
  - *Como fazer:* inicialize p[0] = 1; para cada n, some sobre k = 1, 2, ... com os dois pentagonais até passar de n e sinal alternado; para partes distintas use ∏(1 + x^k); para partes ≤ k, limite o produto. Custo O(N√N).
  - *Exemplo:* p(5) = 7; p(100) = 190 569 292; partições em partes ímpares têm a mesma contagem que em partes distintas.
  - *Armadilha:* índices negativos devem ser ignorados; p(0) = 1; o sinal é (−1)^{k+1} para os dois termos de cada k.

- **Transformada de Euler.** Contagem de estruturas "multiconjuntos de componentes conexas" (não rotuladas): exp(Σ c_k·Σ_j x^{kj}/j); para rotuladas (EGF): todas = exp(conexas), conexas = ln(todas). Ex.: grafos conexos rotulados = n!·[x^n] ln(Σ 2^{C(n,2)} x^n/n!).
  - *Gatilho:* construir todas as estruturas a partir de componentes conexas, com ou sem rótulos.
  - *Por que funciona:* para não rotuladas, cada componente de tamanho k pode aparecer c_k vezes e com multiplicidade j, gerando o fator (1 − x^k)^{−c_k}; para rotuladas, o produto de EGFs corresponde a distribuir rótulos entre as componentes.
  - *Como fazer:* no caso não rotulado, calcule a série logarítmica por múltiplos e aplique exp; no rotulado, calcule a EGF de todas as estruturas e tome ln para obter conexas. Custo O(N log N) com FPS.
  - *Exemplo:* grafos conexos rotulados a partir de todos os grafos; árvores não rotuladas a partir de componentes.
  - *Armadilha:* não misture OGF e EGF; no caso rotulado, multiplique pelo fatorial ao final; a estrutura vazia tem componente de tamanho 0 e precisa ser tratada à parte.
- **Funções geradoras como ferramenta.** Ordinária (OGF) para escolhas não ordenadas, exponencial (EGF) para arranjos rotulados (produto de EGFs = escolher quais rótulos vão em cada parte). Restrição "cada tipo aparece em {a,b,c} cópias" → polinômio por tipo e produto.
  - *Gatilho:* a contagem separa em escolhas independentes por tipo, ou em partes rotuladas/não rotuladas.
  - *Por que funciona:* o produto de OGFs combina somas de pesos sem ordenar; o produto de EGFs distribui rótulos entre as partes e já embute os fatoriais.
  - *Como fazer:* escreva um fator por tipo (por exemplo 1 + x^w + x^{2w}), multiplique todos e leia o coeficiente; para EGF, divida por n! ao montar e multiplique por n! ao extrair.
  - *Exemplo:* cada letra aparece 0, 1 ou 2 vezes; número de grafos com componentes de tipos dados.
  - *Armadilha:* OGF conta multiconjuntos, EGF conta estruturas rotuladas; a ordem dos fatores não importa, mas a ordem dentro da estrutura pode importar em EGFs.
- **Lagrange inversion.** Se f(g(x)) = x, [x^n]g = (1/n)x^{n−1}^{n}; mais geral: [x^n]H(g) = (1/n)[x^{n−1}] H′(x)(x/f)^n. Dá árvores enraizadas (n^{n-1}), Catalan; série inversa composicional: por coeficiente com (x/f)^n via ln/exp custa O(n^2 log n) no total — para O(n $\log^2$ n), use Newton + composição rápida.
  - *Gatilho:* a resposta é o coeficiente de uma função inversa composicional, ou uma família de árvores/Catalan.
  - *Por que funciona:* a fórmula de Lagrange extrai o coeficiente de g a partir de potências de x/f(x), sem calcular g inteira; a versão com H permite contar qualquer estatística aditiva.
  - *Como fazer:* identifique f com f(0) = 0 e f′(0) ≠ 0; calcule (x/f)^n e o coeficiente; para muitos coeficientes, use inversa composicional por Newton com composição rápida. Custo O(n^2 log n) coeficiente a coeficiente ou O(n $\log^2$ n) com composição rápida.
  - *Exemplo:* g = x·e^g dá g_n = n^{n−1}/n! para árvores enraizadas rotuladas; g = x + g^2 dá os números de Catalan.
  - *Armadilha:* f(0) = 0 e f′(0) ≠ 0 são necessários; o caso n = 0 é separado; a implementação rápida é bem mais delicada que a fórmula.
- **Composição e power projection.** f(g(x)) mod x^n em O(n $\log^2$ n) (Kinoshita–Li); a transposta é "power projection" [x^k] h·g^j para todo j. Gatilho: "aplique f repetidas vezes", "inversa composicional", "contagem com F(G(x))". Casos especiais são mais baratos: f(e^x), f(x + c) (shift de Taylor), f(a·x).
  - *Gatilho:* é preciso compor duas séries formais, iterar uma função ou inverter composicionalmente.
  - *Por que funciona:* a composição pode ser reduzida a uma power projection transposta; Kinoshita–Li obtêm O(M(n) log n), que com FFT é O(n $\log^2$ n).
  - *Como fazer:* implemente a rotina de composição rápida ou use os casos especiais; para f(e^x), trabalhe com a série exponencial; para x + c, use Taylor shift; para a·x, apenas escale os coeficientes. Custo O(n $\log^2$ n) no caso geral.
  - *Exemplo:* calcular exp(g(x)) quando g(0) = 0; aplicar P(x + 1) repetidamente; inversa composicional de uma série.
  - *Armadilha:* g(0) normalmente precisa ser 0 para a composição módulo x^n; se não for, fatore a constante; não confunda com multiplicação de séries.
- **Taylor shift.** P(x + c) em uma convolução: b_j = (1/j!)·Σ_i (a_i·i!)·c^{i−j}/(i−j)!. Usos: converter base de potências ↔ falling factorials, Stirling de primeira espécie por dobra (x(x+1)…(x+n−1) → duplicar n usando shift).
  - *Gatilho:* avaliar ou transladar um polinômio, converter bases, ou duplicar um produto de fatores lineares.
  - *Por que funciona:* expandir (x + c)^i pelo binômio separa i e j; após multiplicar por i! e dividir por j!, o que resta é uma convolução.
  - *Como fazer:* forme A_i = a_i·i! e C_t = c^t/t!; calcule a correlação Σ_{i≥j} A_i·C_{i−j} (inverta A para transformá-la em convolução) e divida por j! para obter b_j. Custo O(n log n).
  - *Exemplo:* P(x + 1) para um polinômio de grau 10^5; construir x(x+1)…(x+n−1) por dobra.
  - *Armadilha:* o índice da convolução é i − j e precisa do deslocamento correto; c pode ser negativo; lembre dos fatoriais inversos.

- **Multipoint evaluation.** Avaliar P (grau n) em n pontos: árvore de produtos $\prod(x - x_i)$, remainder tree descendo — $O(n \log^2 n)$ com divisão por série inversa. Gatilho: "avalie f em $x_1..x_m$" para $m \approx 10^5$.
  - *Gatilho:* avaliar um polinômio em muitos pontos, ou obter P(x_i) para todos os i.
  - *Por que funciona:* P mod $(x - x_i) = P(x_i)$; a árvore agrupa os pontos e reduz o grau pela metade a cada nível.
  - *Como fazer:* construa a árvore de produtos dos $(x - x_i)$; calcule P módulo a raiz; desça calculando o resto por cada filho com divisão rápida; nas folhas, o resto constante é o valor. Custo $O(n \log^2 n)$.
  - *Exemplo:* avaliar um polinômio de grau 10^5 em 10^5 pontos; base para interpolação.
  - *Armadilha:* pontos repetidos podem ser agrupados, mas a folha precisa de tratamento; se $m \gg n$, o custo é $O((n+m) \log^2)$; a divisão ingênua por cada ponto é O(nm).
- **Interpolação rápida.** Dados (x_i, y_i): $P = \Sigma\, y_i/M'(x_i) \cdot M(x)/(x - x_i)$; $M'(x_i)$ sai de multipoint em M'; a soma sobe pela árvore de produtos (D&C). $O(n \log^2 n)$. Em pontos 0..d é O(d) por Lagrange direto.
  - *Gatilho:* recuperar o polinômio de grau n a partir de n + 1 valores em pontos distintos.
  - *Por que funciona:* a forma de Lagrange com $M(x) = \prod(x - x_i)$ tem pesos $y_i/M'(x_i)$; os termos $M(x)/(x - x_i)$ são combinados pela árvore de produtos.
  - *Como fazer:* construa a árvore de M; avalie M' nos pontos por multipoint; calcule os pesos; suba a árvore somando numeradores e denominadores; no topo, obtenha P. Custo $O(n \log^2 n)$.
  - *Exemplo:* interpolar um polinômio de grau 10^5; recuperar uma sequência polinomial de valores tabelados.
  - *Armadilha:* pontos repetidos tornam $M'(x_i) = 0$; o módulo precisa ser primo e maior que o grau; em pontos consecutivos, use a fórmula O(d) de **Lagrange em pontos consecutivos**.
- **Lagrange em pontos consecutivos.** $P(n) = \Sigma_i\, y_i \cdot \prod_{j \neq i}(n - j)/(i - j)$ com denominador $i!(d-i)!(-1)^{d-i}$; prefixos/sufixos de $\prod(n - j)$ dão O(d) sem inverso por termo. Se $n \leq d$, devolva y_n direto (evita divisão por 0).
  - *Gatilho:* os pontos são 0, 1, ..., d e a consulta é um n enorme.
  - *Por que funciona:* o produto dos $(n - j)$ pode ser obtido por prefixos e sufixos; o denominador se fatora em fatoriais e um sinal.
  - *Como fazer:* precompute fatoriais e inversos; faça $prefix[i] = \prod_{j<i}(n - j)$; some $y_i \cdot prefix[i] \cdot sufix[i] \cdot invfact[i] \cdot invfact[d-i] \cdot (-1)^{d-i}$. Custo O(d) por consulta.
  - *Exemplo:* calcular $P(10^{18})$ para um polinômio de grau 10^5 dado em $0..10^5$.
  - *Armadilha:* se $n \leq d$, use y_n e não a fórmula; n módulo p pode zerar um fator; o sinal depende de $d - i$.
- **Grau d → d+1 pontos.** Uma sequência é polinomial de grau ≤ d iff a (d+1)-ésima diferença finita é 0. Se $f(n) = \Sigma_{i \leq n} g(i)$ com g polinomial de grau d, f tem grau d+1 → d+2 pontos. Gatilho: "$n \leq 10^{18}$ mas a resposta parece polinomial": calcule d+2 valores por brute force e interpole.
  - *Gatilho:* a resposta para n pequeno parece um polinômio e n é enorme.
  - *Por que funciona:* diferenças finitas baixam o grau em 1; a soma de um polinômio de grau d é um polinômio de grau d + 1.
  - *Como fazer:* calcule valores por força bruta para n = 0..D, estime D pela diferença de ordem D + 1, e interpole em n; use D + 1 pontos para grau ≤ D. Custo $O(D^2)$ para achar D e O(D) para interpolar.
  - *Exemplo:* somas de potências, contagens combinatórias de parâmetros pequenos.
  - *Armadilha:* a sequência só é polinomial a partir de certo ponto em alguns problemas; use pontos suficientes e confirme a diferença; módulo p limita o grau efetivo.
- **$\Sigma\, k^d$.** $\Sigma_{k=1}^{n} k^d$ é polinômio de grau d+1 em n → Lagrange com d+2 pontos em O(d) ($d \leq 10^6$, $n \leq 10^{18}$). Faulhaber: $\Sigma_{k=0}^{n-1} k^d = (1/(d+1)) \Sigma_j C(d+1, j) B_j n^{d+1-j}$ ($B_1 = -1/2$).
  - *Gatilho:* somar potências d-ésimas até n, com d moderado e n gigante.
  - *Por que funciona:* a soma é polinomial de grau d + 1; seus valores em d + 2 pontos determinam a resposta; Faulhaber é a fórmula fechada via Bernoulli.
  - *Como fazer:* calcule S(0), ..., S(d+1) por força bruta ou recorrência e interpole em n; alternativamente use Faulhaber com Bernoulli. Custo O(d log d) para preparar e O(d) por consulta.
  - *Exemplo:* $\Sigma_{k=1}^{n} k^2 = n(n+1)(2n+1)/6$; $\Sigma\, k^3 = (n(n+1)/2)^2$.
  - *Armadilha:* n e d podem precisar de redução módulo p; $d \geq p$ faz o grau efetivo mudar; B_1 tem duas convenções de sinal.

- **Bernoulli por série.** $\Sigma B_j x^j/j! = x/(e^x - 1)$ → inverso de série de $(e^x - 1)/x$; dá todos $B_0..B_n$ em $O(n \log n)$.
  - *Gatilho:* é preciso calcular números de Bernoulli, Faulhaber ou somas de potências em massa.
  - *Por que funciona:* a função geradora dos Bernoulli é o inverso da série (e^x − 1)/x, cujo termo constante é 1.
  - *Como fazer:* construa (e^x − 1)/x = Σ_{j≥0} x^j/(j+1)!; inverta a série módulo x^{n+1}; os coeficientes são B_j/j!. Custo O(n log n).
  - *Exemplo:* $B_0 = 1$, $B_1 = -1/2$, $B_2 = 1/6$, $B_3 = 0$, $B_4 = -1/30$.
  - *Armadilha:* a convenção $B_1 = +1/2$ também existe; não esqueça de multiplicar por j! ao ler os coeficientes; o inverso exige termo constante 1.
- **$\Sigma k^d \cdot r^k$.** Para $r \neq 1$: Σ_{k<n} P(k)r^k = r^n·Q(n) − Q(0) com deg Q = deg P; ache Q por Lagrange (d+1 pontos, Q(0) desconhecido via somas parciais). Aparece em "soma de $i \cdot x^i$" generalizada.
  - *Gatilho:* somas de polinômio vezes exponencial/geométrica, como Σ k·2^k.
  - *Por que funciona:* a antidiferença de P(k)r^k tem a forma Q(k)r^k com o mesmo grau de P; avaliar de 0 a n − 1 telescopa.
  - *Como fazer:* escreva Q com coeficientes incógnitos, imponha Q(k+1)r − Q(k) = P(k) ou determine Q pelos valores das somas parciais em d + 1 pontos; então use $r^n Q(n) - Q(0)$. Custo $O(d^2)$ direto ou O(d log d) com interpolação.
  - *Exemplo:* Σ_{k=0}^{n−1} k·2^k = (n−2)2^n + 2.
  - *Armadilha:* r = 1 é o caso polinomial separado; r = 0 também pede cuidado; a fórmula usa Q(0), não Q(1).
- **Potência de soma binomial.** Σ_i C(n,i)·i^k = Σ_j S(k,j)·n^{\underline j}·2^{n−j} (Stirling de segunda espécie): n enorme, $k \leq 10^5$ → S(k, ·) por convolução.
  - *Gatilho:* somar i^k ponderado por C(n,i), com n enorme e k moderado.
  - *Por que funciona:* i^k se expande em falling factorials via Stirling; a identidade Σ_i C(n,i)·i^{\underline j} = n^{\underline j} ·2^{n−j} elimina a soma.
  - *Como fazer:* calcule a linha S(k,j) para j ≤ k pela fórmula de inclusão-exclusão com convolução; some os termos com n^{\underline j} e 2^{n−j}. Custo O(k log k).
  - *Exemplo:* k = 2: $\Sigma C(n,i) i^2 = n(n+1)2^{n-2}$; k = 1: $n \cdot 2^{n-1}$.
  - *Armadilha:* n^{\underline j} zera para j > n; k pode ser maior que o módulo; não confunda Stirling de primeira e segunda espécie.
- **Stirling por FFT.** S(n,k) para um n e todo k: S(n,k) = (1/k!)Σ_j (−1)^{k−j}C(k,j) j^n → convolução de $j^n/j!$ com $(-1)^m/m!$. Primeira espécie (linha n): x(x+1)…(x+n−1) por D&C ou dobra com shift. Coluna k: S(n,k) = n$x^{nk}/k!$, via exp/ln.
  - *Gatilho:* é preciso uma linha ou coluna inteira de números de Stirling, ou potências de soma binomial.
  - *Por que funciona:* a inclusão-exclusão de S(n,k) é uma convolução; a primeira espécie é o produto dos fatores lineares; a coluna vem da potência da série (e^x − 1).
  - *Como fazer:* para a segunda espécie, convolua $j^n/j!$ com (−1)^m/m! e divida por k!; para a primeira, multiplique os fatores por D&C ou use dobra com Taylor shift; para a coluna, calcule (e^x − 1)^k/k! e leia o coeficiente. Custo O(n log n) ou $O(n \log^2 n)$.
  - *Exemplo:* a linha n = 5: S(5,k) para k = 0..5; primeira espécie de x(x+1)(x+2)(x+3)(x+4).
  - *Armadilha:* a fórmula da segunda espécie usa j^n com j = 0..k; $0^0 = 1$; a primeira e a segunda espécie têm sinais e interpretações diferentes.
- **Newton–Girard.** k·e_k = Σ_{i=1}^k (−1)^{i−1}e_{k−i}p_i; em série: Σ e_k x^k = exp(Σ (−1)^{k−1}p_k x^k/k). Converte somas de potências ↔ coeficientes do polinômio com raízes dadas; p_k de raízes de uma recorrência = traço de M^k.
  - *Gatilho:* são dadas somas de potências das raízes e pedem coeficientes, ou vice-versa.
  - *Por que funciona:* as identidades de Newton relacionam e_k e p_k; a forma de série é a exponencial da soma dos p_k, que é a derivada logarítmica do polinômio.
  - *Como fazer:* calcule e_k em $O(k^2)$ pela recorrência, ou todos de uma vez com exp em O(n log n); para recuperar p_k a partir de e_k, derive o logaritmo. Custo $O(n^2)$ direto ou O(n log n) com FPS.
  - *Exemplo:* raízes 1 e 2: $p_1 = 3$, $p_2 = 5$; $e_1 = 3$, $e_2 = 2$, polinômio $x^2 - 3x + 2$.
  - *Armadilha:* os sinais alternam; k precisa ser invertível módulo p; p_0 não entra.

- **Convolução online (relaxed).** f_n = Σ_{i<n} f_i·g_{n−i} (+ termo): CDQ — resolva [l, m), contribua [l, m) × g[0..r−l) em [m, r) — O(n log² n). Quando ambos f e g são online, contribua nos dois sentidos com cuidado no bloco inicial. Cobre exp, inverso e DPs "dp[n] = Σ dp[i]·w[n−i]".
  - *Gatilho:* uma recorrência em que f_n depende de todos os f_i anteriores e não dá para calcular em blocos independentes.
  - *Por que funciona:* quando a metade esquerda [l, m) já está resolvida, sua contribuição para a metade direita [m, r) é uma convolução com g; o CDQ aplica isso recursivamente.
  - *Como fazer:* resolva [l, m); convolua f[l..m) com g[0..r−l) e some no intervalo [m, r); resolva [m, r). Se g também é online, adicione a contribuição transposta com cuidado para não contar duas vezes. Custo O(n log² n).
  - *Exemplo:* f_n = Σ_{i<n} f_i·f_{n−1−i} (Catalan); exp de uma série; DP de partições.
  - *Armadilha:* o bloco inicial e o termo diagonal precisam de tratamento separado; ambos online podem contar a mesma contribuição duas vezes; use módulo.
- **Berlekamp–Massey.** De 2L termos de uma recorrência linear de ordem L (mod p primo) recupera os coeficientes em O(L²). Gatilho: "n-ésimo termo de algo definido por DP de estado pequeno" → calcule ~2·(estados) termos, BM, depois Kitamasa. Também recupera polinômio mínimo de matriz esparsa (Wiedemann).
  - *Gatilho:* uma sequência gerada por estado pequeno ou recorrência linear, com n gigante e sem fórmula explícita.
  - *Por que funciona:* 2L termos determinam a recorrência de ordem L; o BM acha a menor recorrência que os satisfaz e, com projeções aleatórias, ela costuma ser a recorrência do sistema.
  - *Como fazer:* gere 2L termos por DP/força bruta; rode BM para obter $c_1..c_L$; calcule o n-ésimo termo com Kitamasa/Bostan–Mori. Custo O(L² + L² log n).
  - *Exemplo:* Fibonacci, recorrência de ordem 2; número de caminhos em um grafo com estado pequeno.
  - *Armadilha:* são necessários pelo menos 2L termos; a sequência precisa ser linear sobre o corpo; para matriz esparsa, a projeção escalar pode não capturar todo o polinômio mínimo.
- **Kitamasa / Bostan–Mori.** [xⁿ]P(x)/Q(x) (exige Q(0) ≠ 0; fatore x^d antes): multiplique por Q(−x); denominador vira par, numerador seleciona paridade de n; O(k log k log n) com FFT (O(k² log n) ingênuo). Vence matriz k×k (k³ log n) quando k ≥ 50.
  - *Gatilho:* coeficiente distante de uma série racional ou termo n de recorrência linear de ordem k.
  - *Por que funciona:* Q(x)Q(−x) é par, então após substituir x² por y o denominador tem grau k; a paridade de n escolhe os coeficientes pares ou ímpares do numerador.
  - *Como fazer:* enquanto n > 0, faça Qm = Q(−x), S = P·Qm; se n é par, P ← coeficientes pares de S, senão ímpares; Q ← coeficientes pares de Q·Qm; n ← ⌊n/2⌋. No fim, resposta P[0]/Q[0]. Custo O(k log k log n).
  - *Exemplo:* Fibonacci como [xⁿ] x/(1 − x − x²); termo de ordem 1000 com n = 10^{18}.
  - *Armadilha:* Q(0) ≠ 0; se Q tem fator x^d, divida antes e ajuste o índice; na seleção de paridade, o coeficiente 2i ou 2i+1 deve ser lido corretamente.
- **Recorrência linear ↔ matriz.** Termo n de recorrência de ordem k: x^n mod $(x^k - c_1x^{k-1} - \dots - c_k)$, depois combine com os primeiros k termos. Ordem k pequena e várias queries n: precompute x^{2^j} mod f.
  - *Gatilho:* recorrência de ordem pequena, n enorme, ou muitas consultas de termos diferentes.
  - *Por que funciona:* a matriz companheira tem polinômio característico f; x^n mod f dá os coeficientes para escrever A^n como combinação de $A^0..A^{k-1}$, e os primeiros termos dão o vetor inicial.
  - *Como fazer:* reduza x^n módulo f por exponenciação binária; combine com a_k..a_{k−1}; para várias queries, precompute x^{2^j} mod f e combine os bits. Custo O(k² log n) por query; precomputar as potências evita reexponenciar, mas cada combinação ainda custa O(k² log n).
  - *Exemplo:* Fibonacci com f = x² − x − 1; recorrência de ordem 3.
  - *Armadilha:* os primeiros termos precisam ser compatíveis com a recorrência; multiplicação de polinômios módulo f custa O(k²) sem NTT; a ordem é o grau de f.
- **Polinômio característico.** det(xI − A) por Hessenberg em O(n³) mod p. Cayley–Hamilton: A^n = polinômio de grau < n em A; permite A^n·v com Kitamasa em O(n² log n) usando só produtos matriz-vetor (matriz esparsa).
  - *Gatilho:* determinante característico, potência de matriz, ou A^n aplicado a um vetor.
  - *Por que funciona:* reduzir A à forma de Hessenberg preserva o polinômio característico e torna o determinante O(n²); Cayley–Hamilton expressa potências altas em grau < n.
  - *Como fazer:* reduza por semelhança a Hessenberg em O(n³); calcule det(xI − H) pela recorrência de Hessenberg; para A^nv, use Kitamasa com o polinômio característico e multiplicações matriz-vetor. Custo O(n³) para o polinômio e O(n² log n) para A^nv.
  - *Exemplo:* matriz de adjacência esparsa; potência de matriz 500×500.
  - *Armadilha:* a redução precisa ser por semelhança, não apenas eliminação; o módulo deve ser primo para inverter pivôs; Cayley–Hamilton vale sobre qualquer corpo.

- **Matriz de adjacência e passeios.** $(A^k)_{uv}$ = nº de passeios de comprimento k; tr(A^k) = passeios fechados. Soma de comprimentos 0..k: matriz aumentada [[A, I],[0, I]] elevada a k+1. Contar ciclos de comprimento 3 = tr(A^3)/6; ≤ 4 por M^k com estado (anterior).
  - *Gatilho:* contar caminhos/passeios de comprimento fixo ou até k, ou triângulos.
  - *Por que funciona:* a entrada (u,v) de A^k soma os produtos de adjacências sobre todas as sequências de k arestas; o traço fecha o passeio no vértice inicial.
  - *Como fazer:* exponencie A por potenciação binária $O(n^3 \log k)$; para somar comprimentos, use o bloco aumentado; para triângulos em grafo simples não dirigido, calcule tr(A^3)/6. Para comprimentos ≤ 4, use estados com o vértice anterior.
  - *Exemplo:* número de passeios de tamanho 10 entre dois vértices; triângulos em um grafo.
  - *Armadilha:* em grafo dirigido, tr(A^3) conta cada triângulo dirigido 3 vezes, não 6; laços e arestas múltiplas mudam a contagem; A^k conta passeios, não caminhos simples.
- **Exponenciação min-plus / semi-anel.** Mesmo princípio em (min, +) ou (max, ×) para caminhos mínimos com exatamente k arestas; custo $n^3 \log k$. Se a matriz é circulante, o produto vira convolução (min,+) — não há FFT para esses semi-anéis; continue $O(n^2)$ por produto (ou explore estrutura especial).
  - *Gatilho:* caminho de exatamente k arestas, ou otimização em um semi-anel diferente de (+, ×).
  - *Por que funciona:* a potenciação de matriz usa apenas soma e produto do semi-anel; trocar (+, ×) por (min, +) conta caminhos mínimos com um número fixo de arestas.
  - *Como fazer:* use a identidade do semi-anel (0 na diagonal para min-plus, 1 para max-times) e potenciação binária; cada produto é $O(n^3)$. Se a matriz é circulante, cada produto é uma convolução min-plus $O(n^2)$.
  - *Exemplo:* voo mais barato com exatamente k trechos; maior produto de probabilidades com k passos.
  - *Armadilha:* não existe FFT conhecida para min-plus em geral; a identidade e os elementos infinitos precisam ser tratados; caminhos com menos de k arestas não são permitidos sem estado auxiliar.
- **Gauss mod p.** Sistema Ax = b, determinante, posto, inversa em $O(n^3)$ com pivô por busca de não-zero (mod primo sempre vale). Det = produto dos pivôs × (−1)^{trocas}. Mod composto: use Euclides entre linhas (como gcd).
  - *Gatilho:* resolver sistema linear, calcular posto, inversa ou determinante módulo primo.
  - *Por que funciona:* sobre um corpo, operações elementares preservam o conjunto solução e o determinante a menos do sinal das trocas; um pivô não nulo sempre existe se a coluna não for toda zero.
  - *Como fazer:* para cada coluna, ache uma linha com entrada não nula, troque, normalize ou elimine; acumule o determinante; no fim, posto = número de pivôs. Custo $O(n^3)$.
  - *Exemplo:* resolver Ax = b com n = 500; inversa módulo $10^9 + 7$.
  - *Armadilha:* módulo composto não é corpo; pivo não nulo pode não ter inverso; o sinal do determinante muda a cada troca de linhas.
- **Gauss em GF(2) com bitset.** n ≤ 2000–5000 variáveis: $O(n^3/64)$. Número de soluções = 2^{n − posto} (se consistente). Lights-Out, "xor de subconjunto = alvo", "quantos xors distintos" (2^posto) são o mesmo problema.
  - *Gatilho:* equações lineares sobre xor, com muitas variáveis e coeficientes 0/1.
  - *Por que funciona:* GF(2) é um corpo; cada linha é um bitset e a eliminação usa xor, que processa 64 coeficientes por operação.
  - *Como fazer:* represente cada equação como bitset; elimine pelo bit mais alto; se aparecer 0 = 1, é inconsistente; o número de soluções é 2^{n − posto}. Custo $O(n^3/64)$.
  - *Exemplo:* Lights-Out; contar subconjuntos com xor igual a um alvo; número de xors distintos gerados.
  - *Armadilha:* o bitset precisa incluir o termo independente; a consistência é checada antes de contar; 2^posto pode ser enorme, reduza módulo.
- **Sistema com módulo composto ou reais.** Reais: pivô parcial (maior |a|) e eps; módulo m composto: Gauss com troca por gcd; módulo 2^k: pivô de menor valuação 2-ádica.
  - *Gatilho:* o sistema não está sobre um corpo primo, ou vem de geometria com ponto flutuante.
  - *Por que funciona:* em reais, o pivô parcial controla a amplificação do erro; em anéis compostos, o gcd permite combinar linhas para obter um pivô invertível ou uma valuação mínima.
  - *Como fazer:* reais: escolha o maior |a| da coluna, elimine com eps e use `long double` se preciso; módulo composto: troque linhas por combinações com gcd; módulo 2^k: escolha o pivô com menor v_2 e trate os fatores 2. Custo $O(n^3)$.
  - *Exemplo:* sistema geométrico com doubles; congruências módulo 12; sistema módulo 2^{64}.
  - *Armadilha:* eps mal escolhido dá falso posto; módulo composto pode não ter solução única; a valuação 2-ádica precisa ser acompanhada até o fim.

- **Determinante como contagem.** Kirchhoff: nº de árvores geradoras = det de qualquer menor principal do Laplaciano (peso = produto de pesos; multigrafo ok). K_n: n^{n−2}; K_{a,b}: a^{b−1}b^{a−1}. Árvores contendo arestas fixas: contraia; arborescências: Laplaciano de entrada/saída.
  - *Gatilho:* contar árvores geradoras, árvores com pesos, ou arborescências em grafo dirigido.
  - *Por que funciona:* o teorema de Kirchhoff iguala o número de árvores a um determinante do Laplaciano; contrair arestas obrigatórias reduz o problema ao mesmo cálculo no grafo contraído.
  - *Como fazer:* monte o Laplaciano (grau na diagonal, −peso nas arestas), apague uma linha e a coluna correspondentes, calcule o determinante; para arestas fixas, contraia e multiplique pelos pesos; para arborescências, use o Laplaciano dirigido de entrada ou saída. Custo $O(n^3)$.
  - *Exemplo:* K_4 tem 4^{2} = 16 árvores; K_{2,3} tem $2^2 \cdot 3^1 = 12$.
  - *Armadilha:* menor principal, não qualquer menor; pesos negativos ou multigrafo precisam ser tratados no Laplaciano; a direção da arborescência muda o Laplaciano.
- **Emparelhamento via determinante.** Bipartido: perm(A) é #P-difícil mas det mod 2 = perm mod 2; existência de perfeito = det da matriz de Edmonds com variáveis aleatórias ≠ 0 (Schwartz-Zippel). Geral: posto da matriz de Tutte aleatória = 2·ν(G) mod p grande ($O(n^3)$, mais simples que blossom).
  - *Gatilho:* decidir se existe emparelhamento perfeito, ou calcular o tamanho do emparelhamento máximo, sem implementar blossom.
  - *Por que funciona:* o determinante é um polinômio que é identicamente nulo se não há emparelhamento e não nulo se há; uma avaliação aleatória detecta a diferença com alta probabilidade.
  - *Como fazer:* bipartido: matriz de Tutte/Edmonds com variáveis aleatórias, det ou posto; geral: matriz de Tutte skew-symmetric, rank = 2ν com alta probabilidade; use módulo primo grande. Custo $O(n^3)$.
  - *Exemplo:* perfeito em grafo bipartido; tamanho do emparelhamento máximo em grafo geral.
  - *Armadilha:* é probabilístico e pode errar com probabilidade pequena; em característica 2 a matriz de Tutte perde a propriedade; determinante não conta emparelhamentos, só decide existência.
- **Contagem de emparelhamentos perfeitos em planar.** FKT / Pfaffian com orientação Pfaffian — gatilho: "contar dominós / tilings em grid com buracos" → Pfaffian ou, em grid retangular, determinante com sinais; ou DP de perfil se a largura é pequena.
  - *Gatilho:* contar domino tilings ou emparelhamentos perfeitos em grafo planar.
  - *Por que funciona:* para uma orientação Pfaffian, o Pfaffian da matriz skew-symmetric conta os emparelhamentos perfeitos com sinal +1; em grafos planares tal orientação existe.
  - *Como fazer:* construa a matriz de Tutte/Pfaffian com pesos; encontre uma orientação Pfaffian; calcule o Pfaffian em $O(n^3)$ por eliminação; em grids retangulares, use a matriz com sinais ou DP de perfil. Custo $O(n^3)$ ou O(n·2^largura).
  - *Exemplo:* número de domino tilings de um grid com buracos; emparelhamentos perfeitos em grafo planar.
  - *Armadilha:* a orientação errada produz sinais trocados; o grafo precisa ser planar e a matriz skew-symmetric; para grids com largura pequena, o DP pode ser melhor.
- **Permanente.** Ryser: perm(A) = Σ_S (−1)^{n−|S|}∏*i* Σ{j∈S} a_ij em $O(2^n \cdot n)$ (com Gray code); n ≤ 20–25.
  - *Gatilho:* contar emparelhamentos perfeitos em grafo bipartido, ou permutações com restrições.
  - *Por que funciona:* a inclusão-exclusão sobre as colunas escolhidas remove as permutações que usam colunas fora de S; o produto interno conta as escolhas linha a linha.
  - *Como fazer:* percorra todos os subconjuntos S das colunas, mantenha as somas por linha e o sinal; atualize em Gray code para $O(2^n \cdot n)$. Custo $O(2^n \cdot n)$.
  - *Exemplo:* permutar 20 elementos com posições proibidas; contar matchings perfeitos bipartidos.
  - *Armadilha:* n = 25 já é pesado; o sinal é (−1)^{n−|S|}; zeros e módulos precisam ser tratados; permanente não tem o sinal do determinante.
- **Determinante com parâmetro.** det(A + xB) é polinômio de grau ≤ n em x → avalie em n+1 pontos e interpole; ou resolva generalizado (B inversível: det B · det(B^{-1}A + xI) = polinômio característico). Gatilho: "árvores geradoras com exatamente k arestas vermelhas".
  - *Gatilho:* contar objetos por um parâmetro k que aparece linearmente em pesos ou arestas.
  - *Por que funciona:* o determinante é multilinear nas entradas; com A + xB, cada termo escolhe x de algumas posições, então o resultado é polinômio de grau ≤ n.
  - *Como fazer:* avalie o determinante em n + 1 valores distintos de x e interpole o coeficiente de x^k; se B é inversível, reduza ao polinômio característico de B^{-1}A. Custo $O(n^4)$ avaliando n + 1 determinantes $O(n^3)$, ou menos com estrutura.
  - *Exemplo:* árvores geradoras de um grafo com arestas vermelhas e azuis, contadas por número de vermelhas.
  - *Armadilha:* o módulo precisa ter mais de n + 1 pontos; B singular impede a redução ao característico; grau pode ser menor que n.

- **Atualização de posto 1.** $\det(A + uv^T) = \det A \cdot (1 + v^T A^{-1} u)$; Sherman–Morrison: $(A + uv^T)^{-1} = A^{-1} - A^{-1}uv^T A^{-1}/(1 + v^T A^{-1} u)$. Troca de uma linha/coluna → $O(n^2)$ por atualização em vez de $O(n^3)$.
  - *Gatilho:* o determinante ou a inversa mudam por uma linha/coluna, ou por um rank-one, e há muitas atualizações.
  - *Por que funciona:* o lema do determinante e Sherman–Morrison expressam a nova matriz em termos da inversa antiga; a atualização custa dois produtos matriz-vetor.
  - *Como fazer:* mantenha A^{-1}; calcule $A^{-1}u$ e $v^T A^{-1}$; atualize det pelo fator $(1 + v^T A^{-1} u)$; atualize a inversa pela fórmula. Custo $O(n^2)$ por atualização.
  - *Exemplo:* trocar uma linha de uma matriz 500×500 mantendo o determinante; adicionar uma aresta em um Laplaciano.
  - *Armadilha:* se $1 + v^T A^{-1} u = 0$, a nova matriz é singular; a fórmula exige A inversível; erro numérico pode acumular em muitas atualizações.
- **Cauchy–Binet.** $\det(AB) = \Sigma_S \det(A_{\cdot,S}) \cdot \det(B_{S,\cdot})$ (A é n×m, B m×n, m ≥ n). Prova Kirchhoff e converte "soma sobre subconjuntos de colunas de determinantes" em um único determinante.
  - *Gatilho:* aparecem somas de produtos de determinantes sobre subconjuntos, ou uma prova de contagem por determinante.
  - *Por que funciona:* expandir o determinante do produto seleciona, para cada termo, um subconjunto S de n colunas de A e as mesmas linhas de B.
  - *Como fazer:* identifique A e B com o número de colunas m ≥ n; a soma percorre todos os S de tamanho n; se a soma tiver a forma de Cauchy–Binet, empacote em um único det(AB). Custo $O(n^3)$ após a fatoração.
  - *Exemplo:* prova do teorema de Kirchhoff; soma sobre todos os subconjuntos de arestas de um determinante.
  - *Armadilha:* a ordem das colunas/linhas de S importa para o sinal; m < n dá determinante 0; A e B precisam ter dimensões compatíveis.
- **Base de espaço vetorial (xor basis) generalizada.** Inserir vetores mod p numa base escalonada: dependência = reduz a 0; mantenha os "mais recentes" (base com timestamp) para consultas em janela [l, r] (prefix linear basis). Também vale para matroide linear.
  - *Gatilho:* máximo xor, dependência linear, ou consultas de base em intervalos.
  - *Por que funciona:* a eliminação gaussiana mantém uma base com pivôs distintos; reduzir um vetor pelos pivôs detecta dependência; manter o índice mais recente permite responder intervalos por prefixo.
  - *Como fazer:* insira cada vetor reduzindo pelos pivôs existentes; se sobrar 0, é dependente; para janelas, guarde em cada pivô o maior timestamp e troque quando o novo for mais recente; consulte a base do prefixo r filtrando timestamps ≥ l. Custo O(n log A).
  - *Exemplo:* maior xor de um subconjunto; quantos xors distintos; base de vetores em um intervalo.
  - *Armadilha:* módulo composto não é corpo; a troca por timestamp precisa manter a base válida; vetores nulos não entram.
- **Resultante.** Res(f,g) = a_n^m ∏ g(α_i) é 0 sse há raiz comum; discriminante = (−1)^{n(n−1)/2}·a_n^{−1}·Res(f,f'). Em competições: eliminar uma variável de duas equações polinomiais, número de raízes comuns, "f e g têm fator comum?" (gcd de polinômios) — gcd rápido por half-gcd $O(n \log^2 n)$; Euclides $O(n^2)$ basta para $n \le 5000$.
  - *Gatilho:* duas equações polinomiais, raiz comum, eliminação de variável, ou discriminação de raízes.
  - *Por que funciona:* a resultante é o determinante da matriz de Sylvester e se anula exatamente quando f e g têm fator comum; o discriminante detecta raiz repetida.
  - *Como fazer:* monte a matriz de Sylvester e calcule o determinante, ou use o algoritmo de Euclides entre polinômios; para graus grandes, half-gcd. Custo $O(n^3)$ pelo determinante, $O(n^2)$ por Euclides ou $O(n \log^2 n)$ por half-gcd.
  - *Exemplo:* eliminar y de $f(x,y) = g(x,y) = 0$; verificar se $x^2 - 2$ e $x^2 - 3$ têm raiz comum em Q.
  - *Armadilha:* coeficientes líderes nulos baixam o grau; a resultante pode se anular por fatores comuns em grau maior; o módulo precisa ser corpo.
- **Zeta/Möbius de subconjuntos.** Soma sobre subconjuntos (ou superconjuntos) em $O(2^n n)$; Möbius é a inversa (troca + por −). OR-convolução = zeta → produto ponto a ponto → Möbius; AND-convolução = zeta de superconjuntos.
  - *Gatilho:* somas sobre todos os subconjuntos/superconjuntos de uma máscara, ou convolução OR/AND.
  - *Por que funciona:* a transformada de zeta acumula cada valor em todos os superconjuntos (ou subconjuntos); a inversa de Möbius desfaz a acumulação; para OR, a transformada diagonaliza a convolução.
  - *Como fazer:* para cada bit, para cada máscara que contém o bit, some o valor da máscara sem o bit; a inversa subtrai; para OR, transforme as duas sequências, multiplique ponto a ponto e aplique a inversa. Custo $O(2^n n)$.
  - *Exemplo:* contar subconjuntos com união igual a uma máscara; OR-convolução de duas funções.
  - *Armadilha:* a direção (subconjunto vs superconjunto) muda a transformada; faça a inversa na ordem correta; módulo e sinais.

- **Walsh–Hadamard (XOR-convolução).** Transformada H de tamanho 2^n em $O(2^n n)$; produto ponto a ponto; inversa divide por 2^n. k-ésima potência (nº de maneiras de escolher k valores ordenados, com repetição, com xor x): eleve cada ponto à k (k grande). Mod p ímpar: divisão por 2^n ok.
  - *Gatilho:* convolução com xor, contagem de k-uplas com xor dado, ou potência de uma distribuição.
  - *Por que funciona:* a matriz de Hadamard diagonaliza a convolução XOR; no domínio transformado, convoluir vira multiplicar e a k-ésima potência vira potência ponto a ponto.
  - *Como fazer:* aplique a FWHT $O(2^n n)$; eleve cada coordenada a k; aplique a inversa e divida por 2^n. Custo $O(2^n n + 2^n \log k)$.
  - *Exemplo:* número de pares com xor x; número de k-uplas ordenadas com xor x.
  - *Armadilha:* o módulo 2 não permite dividir por 2^n; a convenção da transformada pode não ter fator 1/2 na ida; k = 0 e 0^0 precisam de cuidado.
- **Convolução de gcd/lcm e Dirichlet.** h(n) = Σ_{gcd(a,b)=n} f(a)g(b): zeta sobre múltiplos, produto, Möbius sobre múltiplos; lcm: sobre divisores. "Soma sobre divisores/múltiplos" em O(N log log N) fazendo uma passada por primo.
  - *Gatilho:* contar pares por gcd/lcm, ou somas sobre divisores/múltiplos.
  - *Por que funciona:* a transformada de zeta sobre múltiplos converte gcd em produto ponto a ponto; a inversa de Möbius recupera a função por gcd; para lcm, use a ordem dos divisores.
  - *Como fazer:* calcule F[d] = Σ_{d|a} f(a) e G[d] = Σ_{d|b} g(b); H[d] = F[d]G[d]; aplique Möbius sobre múltiplos para obter h; para lcm, use a transformada sobre divisores. Custo O(N log log N) por passada de primos.
  - *Exemplo:* contar pares coprimos por gcd = 1; soma de lcm sobre pares.
  - *Armadilha:* o índice 1 precisa ser tratado; o produto pode estourar; a transformada de lcm não é a mesma de gcd.
- **Coloração por IE de subconjuntos.** Cobertura de V por k conjuntos independentes = Σ_S (−1)^{n−|S|}·i(S)^k onde i(S) = nº de subconjuntos independentes de S (zeta de [independente]); a soma conta coberturas com sobreposição, não k-colorações próprias, mas χ(G) ≤ k iff o valor é > 0. Para #k-colorações próprias, use subset convolution de [independente] ou IE sobre arestas: Σ_{F⊆E} (−1)^{|F|}·k^{c(F)} (c(F) = nº de componentes de (V, F)). n ≤ 20–23.
  - *Gatilho:* decidir se o grafo é k-colorável, contar coberturas por conjuntos independentes, ou contar colorações próprias com n pequeno.
  - *Por que funciona:* a inclusão-exclusão sobre vértices não cobertos conta k-uplas de independentes cuja união é V; para colorações próprias, a IE sobre arestas usa o polinômio cromático.
  - *Como fazer:* compute i(S) para todo S por subset DP a partir dos independentes; some (−1)^{n−|S|}i(S)^k; para colorações próprias, use a IE sobre arestas e componentes. Custo $O(2^n n)$ para a cobertura e O(2^m) para a IE de arestas.
  - *Exemplo:* χ(G) ≤ 3; contar 3-colorações próprias de um grafo com 20 vértices.
  - *Armadilha:* cobertura por independentes não é o mesmo que coloração própria; a IE sobre arestas é exponencial no número de arestas; i(S)^k pode ser gigante.
- **Polinômio cromático e de grafos.** P(G, k) tem grau n; calcule em n+1 valores de k (DP exponencial ou IE) e interpole; contagem de orientações acíclicas = |P(G, −1)|. Polinômio de Tutte é #P-difícil, mas é fácil em largura de árvore pequena.
  - *Gatilho:* contar colorações próprias em função de k, ou orientações acíclicas.
  - *Por que funciona:* P(G,k) é polinomial de grau n em k; n + 1 valores determinam todos; a avaliação em −1 conta orientações acíclicas em valor absoluto.
  - *Como fazer:* calcule P(G,k) para k = 0..n por DP/IE e interpole; para árvore pequena, use DP de árvore; para orientações acíclicas, avalie em −1 e tome o módulo. Custo $O(n^2 2^n)$ ou $O(n^3)$ com estrutura.
  - *Exemplo:* triângulo: P(G,k) = k(k−1)(k−2); árvore: k(k−1)^{n−1}.
  - *Armadilha:* o grau pode cair se o grafo tiver poucas arestas? O grau é n para grafo com pelo menos uma aresta; a interpolação precisa de n + 1 pontos; P(G,−1) pode ser negativo.
- **Holonômico (P-recursivo).** Sequências com recorrência de coeficientes polinomiais (Catalan, desarranjos, Motzkin): "adivinhe" a recorrência com álgebra linear em ~poucos termos e rode em O(n); para $n \approx 10^{12}$ mod p use passos de √n (n! mod p em O(√p log p) por avaliação deslocada de ∏(x + i)).
  - *Gatilho:* uma sequência combinatória com recorrência de razão racional em n, ou fatorial/binomial generalizado.
  - *Por que funciona:* a recorrência P-recursiva permite calcular termos sucessivos em O(1) amortizado; para n gigante, métodos de avaliação de produtos deslocados evitam iterar até n.
  - *Como fazer:* gere termos suficientes, ajuste uma recorrência com coeficientes polinomiais por álgebra linear, valide em termos extras e use-a; para fatorial mod p, use o algoritmo de √n com polinômios deslocados. Custo O(n) para n moderado ou O(√n log n) para avaliação distante.
  - *Exemplo:* Catalan: (n+2)C_{n+1} = (4n+2)C_n; desarranjos; n! mod p.
  - *Armadilha:* a recorrência precisa ser válida para todos os n, não só nos dados; o módulo pode degenerar coeficientes; para n ≥ p, fatorial mod p é 0.

- **Hash de polinômios / verificação.** Dois polinômios de grau ≤ d iguais se coincidem em d+1 pontos; teste de identidade por avaliação em ponto aleatório mod p grande (erro ≤ d/p). Serve para "produto de matrizes está certo?" (Freivalds) e "duas expressões são iguais?".
  - *Gatilho:* comparar expressões/polinômios/matrizes sem expandir, ou verificar uma multiplicação.
  - *Por que funciona:* um polinômio não nulo de grau ≤ d tem no máximo d raízes; avaliar em um ponto aleatório de um corpo grande detecta a diferença com probabilidade alta.
  - *Como fazer:* escolha p grande e r aleatório; avalie os dois lados e compare; repita para reduzir o erro; para matrizes, use Freivalds com vetor aleatório. Custo O(avaliação + d) e $O(n^2)$ por verificação de matriz.
  - *Exemplo:* verificar se AB = C com um vetor aleatório; comparar dois produtos de polinômios.
  - *Armadilha:* p pequeno ou entrada adversária aumenta o erro; dois polinômios podem coincidir por azar; use mais de uma avaliação.
- **Soma em k-ésimas raízes.** Para contar subconjuntos com soma ≡ r (mod m) com m pequeno: polinômio em $Z[x]/(x^m - 1)$ elevado por produto de n termos (1 + x^{a_i}) — multiplicação cíclica $O(m^2)$ ou NTT quando m | p − 1.
  - *Gatilho:* contar subconjuntos por soma módulo m pequeno, ou coeficientes de uma potência cíclica.
  - *Por que funciona:* no anel Z[x]/(x^m − 1), multiplicar por (1 + x^{a_i}) acumula as somas módulo m; o coeficiente de x^r é a contagem desejada.
  - *Como fazer:* comece com o polinômio 1; para cada a_i, multiplique por (1 + x^{a_i}) com convolução cíclica; ou agrupe termos iguais e use exponenciação. Custo $O(n\, m^2)$ direto, O(n m log m) com NTT quando m divide p − 1.
  - *Exemplo:* contar subconjuntos de {1,2,3} com soma ≡ 0 mod 5.
  - *Armadilha:* a multiplicação é cíclica, não linear; m precisa dividir p − 1 para a NTT; o expoente da soma é reduzido módulo m.
- **Contagem de pares em disco/soma com limite.** Σ_{i<j} [a_i + a_j ≤ T] em valores pequenos: convolução do histograma e prefix sum; para valores grandes, ordene + two pointers (não precisa de FFT).
  - *Gatilho:* contar pares com soma limitada, ou pontos em um disco, com valores limitados.
  - *Por que funciona:* a convolução do histograma conta pares por soma; a soma de prefixos acumula todas as somas ≤ T; para valores grandes, a ordenação e dois ponteiros evitam a faixa.
  - *Como fazer:* para valores pequenos, convolua o histograma consigo mesmo, subtraia os pares i = i, divida por 2 e some o prefixo até T; para valores grandes, ordene e use dois ponteiros. Custo O(V log V) ou O(n log n).
  - *Exemplo:* contar pares com a_i + a_j ≤ 100; pontos de lattice em um disco de raio R.
  - *Armadilha:* a convolução conta pares ordenados e inclui i = j; a divisão por 2 exige subtrair os quadrados; valores negativos precisam de deslocamento.
- **Casamento com curinga e variantes.** $\Sigma_i (p_i - t_{s+i})^2 p_i\, t_{s+i} = 0$ (curinga = 0) usa 3 convoluções; casamento "≤ k erros por símbolo" faz uma convolução por símbolo (alfabeto pequeno) → |Σ| FFTs.
  - *Gatilho:* casamento de padrão com curingas, ou com até k erros por posição/símbolo.
  - *Por que funciona:* se os valores são não negativos e o curinga vale 0, cada termo é não negativo; a soma é 0 exatamente quando todas as posições casam; expandir o quadrado dá três convoluções.
  - *Como fazer:* atribua valores não negativos aos símbolos, curinga 0; calcule as convoluções de $p^3 \cdot t$, $p^2 \cdot t^2$ e $p \cdot t^3$ (ou as três combinações equivalentes) para todos os deslocamentos; para erros por símbolo, faça uma convolução por caractere. Custo O(3·n log n) ou O(|Σ|·n log n).
  - *Exemplo:* procurar "ab?d" em um texto; contar matches com no máximo 1 erro.
  - *Armadilha:* valores negativos podem cancelar; o curinga precisa ser 0 nas duas sequências; inteiros grandes exigem 64 bits ou módulo.
- **Convolução de funções em grid 2D / multidimensional.** Mapeie (i, j) → i·W' + j com W' ≥ 2W − 1 para convolução 2D com uma FFT 1D; vale para k dimensões com strides.
  - *Gatilho:* convolução de imagens/matrizes, ou correlação 2D.
  - *Por que funciona:* com um passo de linha suficientemente grande, as linhas não se misturam na convolução 1D; o produto de índices vira uma soma biunívoca.
  - *Como fazer:* escolha W' ≥ 2W − 1, mapeie cada célula para i·W' + j, faça a convolução 1D e desmapeie; para k dimensões, use strides análogos. Custo O(N log N), N = produto das dimensões.
  - *Exemplo:* contar ocorrências de um padrão 2D em uma imagem.
  - *Armadilha:* W' pequeno causa wraparound entre linhas; o padding precisa cobrir a soma dos tamanhos; memória do grid pode dominar.

- **Matriz de Vandermonde e interpolação.** Resolver Σ c_j x_i^j = y_i (sistema de Vandermonde) é interpolação: $O(n^2)$ ou $O(n \log^2 n)$; transposto (Σ c_j x_i^j dado c) é multipoint. Sistema "somas de potências p_1..p_k conhecidas → valores" = Newton + fatorar polinômio mod p.
  - *Gatilho:* sistema com potências dos pontos, recuperar coeficientes, ou obter valores a partir de coeficientes.
  - *Por que funciona:* a matriz de Vandermonde é invertível para pontos distintos; resolver o sistema é interpolar; a transposta avalia o polinômio nos pontos.
  - *Como fazer:* use interpolação $O(n^2)$ ou a árvore de produtos $O(n \log^2 n)$; para o transposto, multipoint; para somas de potências, aplique Newton–Girard e fatore. Custo $O(n^2)$ ou $O(n \log^2 n)$.
  - *Exemplo:* recuperar P de P(0),...,P(n); avaliar P nos pontos.
  - *Armadilha:* pontos repetidos tornam a matriz singular; o módulo precisa ter mais de n pontos; a interpolação clássica é $O(n^2)$.
- **Fatorar polinômio mod p.** Raízes em F_p: gcd(f, x^p − x) e Cantor–Zassenhaus aleatório (gcd com (x + a)^{(p−1)/2} − 1); gatilho: "ache x tal que f(x) ≡ 0 (mod p)" com f de grau pequeno, ou raiz quadrada/k-ésima mod p (Tonelli–Shanks, Cipolla).
  - *Gatilho:* encontrar raízes ou fatores de um polinômio módulo primo, ou extrair raiz quadrada/k-ésima.
  - *Por que funciona:* x^p − x é o produto de todos os lineares em F_p; o gcd isola as raízes; Cantor–Zassenhaus separa fatores de mesmo grau usando potências (p−1)/2.
  - *Como fazer:* calcule gcd(f, x^p − x) para as raízes; para fatores, use divisões aleatórias com (x + a)^{(p−1)/2} − 1; para raiz quadrada, Tonelli–Shanks ou Cipolla. Custo $O(\deg(f)^2 \log p)$ ou $O(\deg(f) \log^2 p)$ com polinômios rápidos.
  - *Exemplo:* resolver $x^3 + x + 1 \equiv 0 \bmod 10^9 + 7$; sqrt mod p.
  - *Armadilha:* p = 2 e p pequeno têm casos especiais; fatores repetidos precisam de derivada; Cantor–Zassenhaus é aleatório.
- **Logaritmo e raiz discreta.** BSGS para a^x ≡ b em O(√m); k-ésima raiz módulo primo: ache gerador g, resolva log, divida por gcd(k, p−1). Gatilho: "resolva x^k ≡ a mod p".
  - *Gatilho:* resolver congruência exponencial, ou raiz k-ésima em um corpo finito.
  - *Por que funciona:* no grupo cíclico gerado por g, a^x = b vira uma equação linear nos logaritmos; BSGS resolve o log em O(√m) por meet-in-the-middle.
  - *Como fazer:* encontre um gerador g; calcule log_g(b); resolva k·x ≡ log_g(b) mod (p−1); se gcd(k,p−1) não dividir o log, não há solução; caso contrário, há gcd soluções. Custo O(√p + log p).
  - *Exemplo:* $x^2 \equiv a \bmod p$; a^x ≡ b mod p.
  - *Armadilha:* m composto pode não ter gerador; a base precisa ser invertível; a raiz k-ésima pode ter múltiplas soluções.
- **Matriz diagonalizável e Fibonacci.** F_n por dobra: $F_{2k} = F_k(2F_{k+1} - F_k)$, $F_{2k+1} = F_{k+1}^2 + F_k^2$; gcd(F_a, F_b) = F_{gcd(a,b)}; Pisano $\pi(10^k) = 15 \cdot 10^{k-1}$ ($k \geq 3$).
  - *Gatilho:* Fibonacci com índice gigante, gcd de Fibonacci, ou período módulo 10^k.
  - *Por que funciona:* as identidades de dobra calculam F_n e F_{n+1} em O(log n); a propriedade do gcd vem da divisibilidade na sequência; o período de Pisano módulo 10^k é conhecido.
  - *Como fazer:* implemente fast doubling; para gcd, use a identidade; para período módulo 10^k, use 15·10^{k−1} quando k ≥ 3. Custo O(log n).
  - *Exemplo:* $F_{10^{18}} \bmod 10^9 + 7$; gcd(F_12, F_18) = F_6 = 8; período módulo 1000 = 1500.
  - *Armadilha:* a convenção F_0 = 0, F_1 = 1 precisa ser fixada; índices negativos usam F_{−n} = (−1)^{n+1}F_n; a fórmula do período não vale para k < 3.
- **Sparse + Wiedemann.** Det, posto ou solução de sistema esparso $n \approx 10^5$ com nnz pequeno: BM em $u \cdot A^i \cdot v$ (2n termos, cada um é um produto matriz-vetor de custo nnz) → O(n·nnz); BM escalar só dá o polinômio mínimo da projeção — det/posto exigem pré-condicionamento aleatório ou block Wiedemann.
  - *Gatilho:* matriz esparsa grande, determinante/posto/sistema sem preencher a matriz.
  - *Por que funciona:* a sequência de Krylov $u^T A^i v$ é gerada por multiplicações matriz-vetor; o BM recupera a recorrência e, com pré-condicionamento, o determinante ou posto.
  - *Como fazer:* escolha u, v aleatórios; gere 2n termos de $u^T A^i v$; rode BM; para determinante/posto, aplique pré-condicionamento diagonal aleatório ou use block Wiedemann. Custo $O(n \cdot nnz + n^2)$ para BM, com O(n) produtos matriz-vetor.
  - *Exemplo:* determinante de uma matriz esparsa $10^5 \times 10^5$ com poucas arestas por linha; resolver um sistema esparso.
  - *Armadilha:* BM escalar pode capturar só parte do polinômio mínimo; o método é probabilístico; block Wiedemann aumenta a garantia e o custo.

Matemática de olimpíada e math.stackexchange

adicionado em 2026-11-10 por agentes; revisado

Muita prova de olimpíada é, no fundo, um algoritmo ou uma garantia de existência: ache o argumento extremal, o invariante ou a desigualdade que corta a busca.

- **Princípio extremal.** Pegue o elemento máximo/mínimo/mais longo/mais à esquerda: ele é forçado e reduz o caso a n − 1. Ex.: torneio sempre tem caminho hamiltoniano (Rédei) — insira vértices por varredura linear na ordem (O(n) por inserção, $O(n^2)$); Sylvester–Gallai; "o pior vértice não pode piorar".
  - *Gatilho:* uma prova de existência ou uma construção em que escolher o objeto extremo simplifica as relações.
  - *Por que funciona:* o objeto extremal tem uma propriedade forçada; se ele não a tivesse, outro objeto mais extremo existiria, contradizendo a escolha.
  - *Como fazer:* escolha máximo/mínimo/mais longo/mais à esquerda; derive uma contradição ou uma redução para n − 1; no algoritmo, insira ou remova esse objeto e recorra. Custo depende da redução, muitas vezes $O(n^2)$.
  - *Exemplo:* caminho hamiltoniano em torneio por inserção de vértices; teorema de Sylvester–Gallai; argumento do vértice de maior grau.
  - *Armadilha:* o conjunto precisa ser finito e não vazio; empates exigem um desempate determinístico; o objeto extremo pode não ser único.
- **Menor contraexemplo.** Para provar P(n) para todo n, assuma o menor n que falha e derive contradição; algoritmicamente: o primeiro erro de um guloso é o contraexemplo mínimo, e vira uma checagem local.
  - *Gatilho:* prova por indução/contradição, ou busca de falha em um algoritmo guloso.
  - *Por que funciona:* o menor n que falha tem todos os menores válidos, então a estrutura de P(n − 1), n/2 etc. pode ser usada para construir uma solução para n.
  - *Como fazer:* suponha o menor n com falha; aplique a redução do problema para obter um caso menor; contradiga a minimalidade; no algoritmo, teste trocas locais para achar o primeiro contraexemplo. Custo O(tamanho da checagem).
  - *Exemplo:* menor contraexemplo para uma ordenação gulosa; indução forte em construções recursivas.
  - *Armadilha:* "menor" precisa ser bem ordenado; a redução deve preservar a hipótese; um contraexemplo local não prova a falha global.
- **Pombal (pigeonhole).** n+1 prefixos módulo n → algum bloco contíguo tem soma ≡ 0 (mod n); Erdős–Ginzburg–Ziv: em 2n − 1 inteiros há n com soma ≡ 0 (mod n). Gatilho: "sempre existe" → a resposta é só construir (DP em restos).
  - *Gatilho:* "sempre existe", muitas classes e poucos objetos, ou somas de prefixos.
  - *Por que funciona:* n + 1 prefixos em n classes de resto forçam uma repetição; a diferença entre dois prefixos é um bloco com soma ≡ 0; EGZ é a versão combinatória mais profunda.
  - *Como fazer:* calcule os prefixos módulo n, guarde a primeira ocorrência de cada resto e, ao repetir, devolva o bloco; para EGZ, use DP nos restos ou construa os n termos. Custo O(n) para o bloco, $O(n^2)$ ou O(n·n) para EGZ.
  - *Exemplo:* subarray com soma divisível por n; n números com soma ≡ 0.
  - *Armadilha:* o bloco não pode ser vazio; use prefixo 0; a repetição pode ocorrer no mesmo prefixo se você não inicializar corretamente.
- **Pombal por limiar.** Se há mais de k·m itens em m categorias, alguma tem > k; se Σ ≥ T então max ≥ T/m. Usado para limitar o número de iterações ("resposta ≤ √n" ou "≤ log n").
  - *Gatilho:* limitar o máximo a partir de uma soma, ou provar que uma resposta é pequena.
  - *Por que funciona:* se todos os m valores fossem ≤ T/m, a soma seria ≤ T; a contrapositiva dá o limite; a versão discreta usa > k em vez de ≥.
  - *Como fazer:* compare a soma total com o número de categorias; conclua que existe uma categoria grande; use isso para limitar laços, tamanhos de resposta ou número de iterações. Custo O(m) para verificar.
  - *Exemplo:* grau médio $\bar{d}$ implica vértice de grau ≥ $\bar{d}$; se a resposta tem soma S em n casos, algum caso tem ≥ S/n.
  - *Armadilha:* a média não garante qual categoria; desigualdades estritas mudam o limiar; categorias vazias contam no denominador.
- **Erdős–Szekeres.** Toda sequência de rs + 1 números distintos tem crescente de tamanho r + 1 ou decrescente s + 1; $n^2 + 1$ distintos → monótona de tamanho n + 1. Gatilho: "LIS ≥ k ou LDS ≥ k" é garantia, só construa.
  - *Gatilho:* uma sequência longa e a garantia de uma subsequência monótona longa.
  - *Por que funciona:* associe a cada posição o par (comprimento da LIS que termina ali, comprimento da LDS que termina ali); para valores distintos, esses pares são todos distintos; há no máximo r·s pares com componentes ≤ r e ≤ s.
  - *Como fazer:* compute LIS e LDS por paciência/segment tree; se o comprimento for rs + 1, um dos dois limites é ultrapassado; para construir, recupere a subsequência. Custo O(n log n).
  - *Exemplo:* 5 números distintos garantem crescente de 3 ou decrescente de 3; 10 números garantem crescente de 4 ou decrescente de 4.
  - *Armadilha:* a versão clássica exige valores distintos; com repetições, a definição de crescente/decrescente (estrita ou não) muda o enunciado.

- **Dilworth / Mirsky.** Min cadeias que cobrem o poset = max antichain; min antichains = maior cadeia. Cobertura mínima por cadeias em DAG transitivo = n − emparelhamento máximo; antichain máxima = n − emparelhamento máximo no fecho (König). Em permutações: cobrir por crescentes = LDS.
  - *Gatilho:* particionar um poset/DAG em cadeias ou antichains, ou achar a maior antichain.
  - *Por que funciona:* Dilworth e Mirsky são duais; no DAG, uma cadeia é um caminho e a cobertura mínima por caminhos é n − emparelhamento máximo na bipartição de vértices.
  - *Como fazer:* calcule o emparelhamento máximo no grafo bipartido de comparabilidades (fecho transitivo); a cobertura mínima é n − ν; a antichain máxima também é n − ν. Para permutações, use LDS. Custo $O(n^3)$ ou $O(E\sqrt{V})$.
  - *Exemplo:* cobrir uma permutação por subsequências crescentes; maior conjunto de elementos incomparáveis.
  - *Armadilha:* é preciso usar o fecho transitivo para cadeias, não apenas arestas originais; caminhos e cadeias coincidem no fecho; a cobertura pode ter caminhos de comprimento 0.
- **Invariantes clássicos.** Coloração de tabuleiro (xadrez, 3 cores em diagonais), soma mod k, xor total, paridade de inversões (puzzle de 15: solucionável iff paridade da permutação + linha do vazio batem), gcd, soma de pesos ponderados. Invariante completo = classificação em classes de equivalência.
  - *Gatilho:* uma operação preserva alguma quantidade e o alvo tem valor diferente, ou é preciso classificar estados alcançáveis.
  - *Por que funciona:* se a operação preserva I e o estado inicial e o final têm I diferentes, a transição é impossível; um invariante completo separa exatamente as classes alcançáveis.
  - *Como fazer:* escolha coloração, soma módulo k, xor, paridade ou gcd; calcule I no início e no fim; para classificação, procure um conjunto completo de invariantes. Custo O(n) por estado.
  - *Exemplo:* dominós cobrem números iguais de casas brancas e pretas; 15-puzzle usa paridade das inversões e a linha do espaço vazio.
  - *Armadilha:* um invariante necessário não é suficiente; no 15-puzzle a paridade da linha do vazio também importa; colorações podem depender da orientação.
- **Monovariantes.** Uma quantidade inteira limitada que muda em cada passo (soma de quadrados, número de inversões, número de pares ruins) prova término e limita o nº de passos: operação "troque se vale a pena" termina em ≤ C(n,2) trocas se cada troca reduz inversões.
  - *Gatilho:* provar que um processo termina, ou limitar o número de iterações de uma troca local.
  - *Por que funciona:* um inteiro não negativo que decresce estritamente a cada passo só pode decrescer um número finito de vezes; o valor inicial é uma cota.
  - *Como fazer:* escolha uma medida inteira (inversões, soma de quadrados, pares ruins); mostre que cada operação a diminui em pelo menos 1; conclua a cota pelo valor inicial. Custo O(valor inicial).
  - *Exemplo:* bubble sort reduz inversões e termina em ≤ C(n,2) trocas; soma de distâncias em uma troca de posições.
  - *Armadilha:* a medida precisa ser limitada inferiormente; empates podem causar laços; uma troca que não reduz estritamente não serve.
- **Descida infinita e Vieta jumping.** $a^2 + b^2 = k(ab + 1)$ com inteiros positivos ⇒ k é quadrado (IMO 1988/6): de uma solução (a, b) com a ≥ b, a outra raiz em a é $a' = kb − a = (b^2 − k)/a$, menor; descendo chega-se a $a' = 0$. Gatilho: equação quadrática simétrica em duas variáveis; algoritmicamente é Euclides.
  - *Gatilho:* equação diofantina quadrática simétrica, ou uma descida em soluções positivas.
  - *Por que funciona:* para a fixo, a equação é quadrática em b; a soma das raízes é ka e o produto é $a^2 − k$, então a outra raiz é inteira; a minimalidade força $a' = 0$, revelando k quadrado.
  - *Como fazer:* ordene a ≥ b, troque (a,b) pela outra raiz, mostre que ela é positiva e menor; repita até $a' = 0$; então $k = b^2$. Custo O(log max(a,b)) se implementado como Euclides.
  - *Exemplo:* IMO 1988/6; equações de Markov semelhantes.
  - *Armadilha:* a positividade da outra raiz precisa ser provada; sem a hipótese de minimalidade a descida não contradiz; k = 0 ou variáveis não positivas mudam o argumento.
- **Indução forte e estrutura recursiva.** Se o problema para n sai de n/2, n − 1 ou "tirar um vértice extremal", a construção/DP tem forma recursiva; ótimo para provar que um guloso ou construção funciona (e dá o algoritmo).
  - *Gatilho:* a solução de n usa uma solução menor do mesmo tipo, ou um objeto extremal pode ser removido.
  - *Por que funciona:* a hipótese de indução cobre todos os tamanhos menores; a redução mostra que a solução ótima de n contém uma solução ótima do subproblema.
  - *Como fazer:* identifique a redução (n − 1, n/2, remover vértice); prove que ela preserva a optimalidade; implemente recursão/DP com memoização ou iterativo. Custo depende do número de estados.
  - *Exemplo:* merge sort; construção de árvore por remoção de folha; divisão e conquista.
  - *Armadilha:* a redução pode precisar de mais de um parâmetro; casos base pequenos; remover um objeto pode destruir a estrutura que você quer manter.

- **AM-GM para otimizar.** Σ x_i fixa, ∏ x_i máximo quando todos iguais; inteiros: iguais ±1. Partir n em parcelas para maximizar produto → use 3s (com resto 4 = 2 + 2 ou 2): n mod 3 = 0: 3^{n/3}; 1: 4·3^{(n−4)/3}; 2: 2·3^{(n−2)/3}.
  - *Gatilho:* soma fixa e produto máximo, ou dividir um inteiro em parcelas com produto máximo.
  - *Por que funciona:* AM-GM dá igualdade quando todos os fatores são iguais; para inteiros, a desigualdade discreta força valores diferindo por no máximo 1; no problema das parcelas, o 3 é a melhor razão.
  - *Como fazer:* equalize os valores; para inteiros, distribua o resto; para parcelas, use a fórmula dos 3s e trate n pequeno à parte. Custo O(1).
  - *Exemplo:* n = 10 → 3·3·4 = 36; n = 8 → 3·3·2 = 18.
  - *Armadilha:* a fórmula clássica pressupõe n ≥ 5 e parcelas ≥ 2; para n = 2, 3, 4 os casos base são diferentes; módulo p exige exponenciação.
- **Cauchy–Schwarz e Engel.** $(\Sigma a_i b_i)^2 \le \Sigma a_i^2 \cdot \Sigma b_i^2$; $\Sigma a_i^2/b_i \ge (\Sigma a_i)^2/\Sigma b_i$. Minimizar Σ c_i/x_i com Σ x_i = S → x_i ∝ √c_i (igualdade da desigualdade); aloca recursos com retornos decrescentes.
  - *Gatilho:* somas de produtos, razões, ou alocação com soma fixa.
  - *Por que funciona:* Cauchy–Schwarz; Engel é a forma de Titu com denominadores; a igualdade ocorre quando os vetores são proporcionais.
  - *Como fazer:* escreva a soma como $\Sigma a_i^2/b_i$, aplique Engel e iguale a condição; para Σ c_i/x_i, tome $a_i = \sqrt{c_i}$, $b_i = x_i$ e obtenha $x_i \propto \sqrt{c_i}$. Custo O(n).
  - *Exemplo:* x + y = 10, minimizar 1/x + 4/y → x:y = 1:2, mínimo 9/10.
  - *Armadilha:* os denominadores precisam ser positivos; a igualdade exige proporcionalidade, não apenas soma fixa; sinais negativos invalidam a desigualdade.
- **Rearrangement e Chebyshev.** Σ a_i b_{σ(i)} é máximo com ordens iguais e mínimo com ordens opostas; para f convexa, Σ f(a_i − b_σ(i)) é mínimo com ordens iguais e máximo com ordens opostas (direção oposta à do produto). Gatilho: "emparelhe dois arrays minimizando Σ|a_i − b_σ(i)|" → ordene ambos.
  - *Gatilho:* emparelhar dois arrays para maximizar/minimizar uma soma de produtos ou de diferenças.
  - *Por que funciona:* uma troca de dois pares invertidos não piora a soma quando as ordens são iguais; para diferenças absolutas, a troca também favorece a ordem igual.
  - *Como fazer:* ordene ambos; para máximo de produtos, mesmo sentido; para mínimo, sentido oposto; para Σ|a_i − b_σ(i)|, mesmo sentido. Custo O(n log n).
  - *Exemplo:* minimizar Σ|a_i − b_σ(i)|; maximizar Σ a_i b_σ(i).
  - *Armadilha:* a afirmação para f convexa arbitrária exige simetria (como |x|^p); para f = e^x a direção pode mudar; compare com produto cruzado, não com divisão.
- **Jensen e distribuição igual.** f convexa: Σ f(x_i) com Σ x_i fixa é mínima quando todos iguais (inteiros: ⌊S/k⌋ ou ⌈S/k⌉). Ex.: minimizar pares Σ C(g_i, 2) ao dividir n em k grupos; Mantel/Turán por convexidade.
  - *Gatilho:* soma fixa e uma função convexa dos valores, ou balancear grupos para minimizar pares.
  - *Por que funciona:* Jensen diz que a média de f é ≥ f da média; a igualdade ocorre quando todos os x_i são iguais; para inteiros, a versão discreta distribui a diferença em 1.
  - *Como fazer:* calcule a média; distribua ⌊S/k⌋ e ⌈S/k⌉; para C(g,2), a convexidade mostra que grupos balanceados minimizam. Custo O(k).
  - *Exemplo:* dividir 10 em 3 grupos minimizando pares internos → tamanhos 3,3,4.
  - *Armadilha:* f precisa ser convexa; para concava o ótimo é o oposto; a restrição de inteiros pode impedir a igualdade exata.
- **Telescopia.** 1/(i(i+1)) = 1/i − 1/(i+1); Σ i·i! = (n+1)! − 1; $\Sigma (i^3 - (i-1)^3)$; produtos ∏(1 + 1/i) = n + 1. Gatilho: termo que "cancela com o vizinho": procure f(i+1) − f(i).
  - *Gatilho:* uma soma ou produto em que termos consecutivos compartilham fatores.
  - *Por que funciona:* se o termo é f(i+1) − f(i), a soma colapsa para f(n+1) − f(1); produtos similares telescopam por cancelamento.
  - *Como fazer:* procure uma primitiva discreta f; use frações parciais ou reescreva o termo; some os extremos. Custo O(n) para verificar ou O(1) com fórmula.
  - *Exemplo:* Σ_{i=1}^n 1/(i(i+1)) = 1 − 1/(n+1); Σ i·i! = (n+1)! − 1.
  - *Armadilha:* os limites do somatório precisam ser ajustados; sinais e índices podem deslocar a telescopia; nem toda soma tem primitiva simples.

- **Vieta e Newton.** Raízes de um polinômio mônico: soma = $-a_{n-1}$, produto = $(-1)^n a_0$; relações simétricas se escrevem por e_k; p_k satisfaz a recorrência linear do polinômio, logo p_k de raízes algébricas = tr(M^k) da matriz companheira.
  - *Gatilho:* relações entre raízes e coeficientes, somas de potências das raízes, ou recorrência satisfeita por p_k.
  - *Por que funciona:* Vieta dá e_k em função dos coeficientes; Newton–Girard liga e_k a p_k; a matriz companheira tem autovalores iguais às raízes, então p_k = tr(M^k).
  - *Como fazer:* leia os coeficientes; use Vieta para e_k; use Newton para p_k; para recorrência, monte a companheira e calcule traços ou use a recorrência. Custo $O(n^2)$ direto ou O(n log n) com FPS.
  - *Exemplo:* raízes 1,2,3: soma 6, produto 6; $p_2 = 1^2+2^2+3^2 = 14$.
  - *Armadilha:* o polinômio precisa ser mônico; os sinais alternam; multiplicidades contam nas somas de potências.
- **Identidades binomiais úteis.** Vandermonde Σ_i C(a,i)C(b,k−i) = C(a+b,k); hockey stick $\Sigma_{i=r}^{n} C(i,r) = C(n+1,r+1)$; Σ_k k·C(n,k) = n·2^{n−1}; Σ_k $C(n,k)^2 = C(2n,n)$; C(n,k)C(k,j) = C(n,j)C(n−j,k−j). Gatilho: "soma sobre subconjuntos" → reduza o somatório antes de implementar.
  - *Gatilho:* uma soma com binomiais, especialmente sobre todos os subconjuntos ou todos os k.
  - *Por que funciona:* cada identidade é uma contagem dupla ou uma manipulação algébrica; reduzem somas de O(n) termos a uma expressão fechada.
  - *Como fazer:* reconheça o padrão (Vandermonde, hockey stick, absorção); reescreva C(n,k) = n/k·C(n−1,k−1); simplifique antes de codar. Custo O(1) após a redução.
  - *Exemplo:* Σ_k $C(n,k)^2 = C(2n,n)$; $\Sigma_{k=0}^{n} k\,C(n,k) = n2^{n-1}$.
  - *Armadilha:* os limites do somatório precisam coincidir; binomiais com índices inválidos são 0; módulo p e n grande exigem Lucas.
- **Recorrências lineares: solução fechada.** Característica r^n: raízes r_i distintas → Σ $c_i\, r_i^n$; raiz dupla → $(c_1 + c_2 n)r^n$; a_n = c·a_{n−1} + d: desloque pelo ponto fixo. Útil para cortar um expoente: Fibonacci mod m tem período (Pisano), a_n mod m é periódico com período ≤ m^2.
  - *Gatilho:* recorrência linear de coeficientes constantes, ou uma sequência periódica módulo m.
  - *Por que funciona:* a solução é combinação de exponenciais r_i^n; raízes repetidas introduzem fatores polinomiais; o ponto fixo remove o termo constante.
  - *Como fazer:* monte o polinômio característico; ache raízes; ajuste constantes pelos termos iniciais; para período, use o par (a_n, a_{n+1}) e o fato de haver no máximo m^2 estados. Custo $O(k^3)$ para raízes ou $O(k^2 \log n)$ via matriz.
  - *Exemplo:* Fibonacci: $r^2 = r + 1$; a_n = c a_{n−1} + d com c ≠ 1 desloca para b_n = a_n − d/(1−c).
  - *Armadilha:* raízes repetidas exigem o termo $n \cdot r^n$; c = 1 no caso afim; o período pode ser bem menor que m^2.
- **Quadrado de Fibonacci e identidades.** Cassini $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$; F_{m+n} = F_{m+1}F_n + F_mF_{n−1}; Σ_{i≤n} F_i = F_{n+2} − 1; Zeckendorf: todo inteiro é soma única de Fibonacci não consecutivos (guloso). Gatilho: jogo de Fibonacci Nim.
  - *Gatilho:* identidades de Fibonacci, representação de inteiros, ou jogos de subtração.
  - *Por que funciona:* Cassini e a fórmula de adição vêm da matriz/identidades algébricas; Zeckendorf usa a greedy dos maiores Fibonacci e a unicidade dos não consecutivos.
  - *Como fazer:* use as identidades para reduzir somas; para Zeckendorf, escolha o maior F_i ≤ n, subtraia e pule F_{i−1}; para Fibonacci Nim, use a representação. Custo O(log n).
  - *Exemplo:* $\Sigma_{i=1}^{5} F_i = 1+1+2+3+5 = 12 = F_7 - 1 = 13 - 1$; 100 = 89 + 8 + 3.
  - *Armadilha:* a indexação F_0 = 0, F_1 = 1 precisa ser consistente; Zeckendorf proíbe consecutivos; a estratégia do jogo depende da maior potência de 2 na representação.
- **Floor sum.** $f(n,m,a,b) = \Sigma_{i=0}^{n-1} \lfloor (ai + b)/m \rfloor$ em O(log m) por Euclides: se a ≥ m ou b ≥ m, extraia (a/m)·n(n−1)/2 + (b/m)·n; senão troque papéis: $y_{max} = \lfloor (a(n-1) + b)/m \rfloor$, soma = (n−1)·y_max − f(y_max, a, m, m − b − 1). Versões com Σ i·⌊·⌋ e Σ $\lfloor \cdot \rfloor^2$ também em log.
  - *Gatilho:* somar pisos de uma função linear, contar pontos de rede sob uma reta, ou n gigante.
  - *Por que funciona:* o algoritmo de Euclides reduz (a,b,m) e troca os papéis de a e m; cada passo diminui o módulo, então há O(log m) passos.
  - *Como fazer:* reduza a e b módulo m acumulando as partes inteiras; use a recorrência padrão com y = a·n + b, atualize n = y/m, b = y%m, troque m e a; para versões ponderadas, carregue também as somas de i e i^2. Custo O(log m).
  - *Exemplo:* $\Sigma_{i=0}^{n-1} \lfloor (2i+1)/5 \rfloor$; pontos inteiros sob y = (ax+b)/m.
  - *Armadilha:* variantes com (n−1)·y_max e m−b−1 não são equivalentes à recorrência padrão; use y = a·n + b e confira em casos pequenos; valores não negativos evitam ajustes de piso.

- **Pontos de rede sob reta.** #{(x,y) inteiros: $1 \le x \le n$, $1 \le y \le ax/m + b$} = floor sum; área e polígonos de rede com Pick e floor sum fecham sem iterar. Gatilho: $n \approx 10^9$–10^{12} com "quantos pontos sob/sobre a reta".
  - *Gatilho:* contar pontos inteiros sob/sobre uma reta, ou área de um polígono de lattice.
  - *Por que funciona:* para cada x, o número de y é $\lfloor (ax+b)/m \rfloor$ (com os ajustes de início); a soma é exatamente uma floor sum; Pick converte área em pontos internos e de fronteira.
  - *Como fazer:* ajuste os limites para $x = 0..n-1$ e $y \ge 1$; aplique floor sum; para polígonos, use Pick com $B = \Sigma \gcd(|\Delta x|,|\Delta y|)$. Custo O(log m).
  - *Exemplo:* pontos sob $y = (3x+1)/7$; área de um polígono de lattice.
  - *Armadilha:* os limites $1 \le x$, $1 \le y$ mudam o deslocamento; "sob" pode ser estrito; coordenadas negativas exigem cuidado com o piso.
- **Hermite e floor úteis.** $\Sigma_{k=0}^{n-1} \lfloor x + k/n \rfloor = \lfloor nx \rfloor$; $\lfloor \lfloor x/a \rfloor / b \rfloor = \lfloor x/(ab) \rfloor$; $\lceil a/b \rceil = \lfloor (a + b - 1)/b \rfloor$; Legendre: $v_p(n!) = \Sigma_k \lfloor n/p^k \rfloor = (n - s_p(n))/(p - 1)$ com s_p = soma dos dígitos em base p.
  - *Gatilho:* simplificar pisos aninhados, contar fatores primos de fatorial, ou somas de pisos com passo uniforme.
  - *Por que funciona:* Hermite distribui os incrementos k/n e o piso de nx; Legendre conta quantos múltiplos de p, p^2 etc. aparecem em n!.
  - *Como fazer:* aplique as identidades para reduzir; para $v_p(n!)$, some n/p^k até zero; para o número de dígitos, use a fórmula com s_p. Custo O(log_p n).
  - *Exemplo:* $v_2(10!) = 5 + 2 + 1 = 8$; $\lfloor \lfloor x/2 \rfloor /3 \rfloor = \lfloor x/6 \rfloor$.
  - *Armadilha:* as identidades de piso valem para x real, mas sinais negativos pedem convenção; a fórmula de Legendre usa a soma dos dígitos, não o número de dígitos.
- **Beatty.** Se $1/\alpha + 1/\beta = 1$ com α irracional > 1, as sequências $\lfloor n\alpha \rfloor$ e $\lfloor n\beta \rfloor$ particionam os naturais. Posições P do Wythoff Nim: $(\lfloor n\varphi \rfloor, \lfloor n\varphi^2 \rfloor)$ com $\varphi = (1+\sqrt{5})/2$. Gatilho: duas progressões "quase aritméticas" que cobrem tudo sem sobreposição.
  - *Gatilho:* duas sequências de pisos que devem particionar os inteiros, ou Wythoff Nim.
  - *Por que funciona:* o teorema de Beatty garante a partição quando os recíprocos somam 1 e α é irracional; no Wythoff, as posições perdedoras são exatamente essas sequências.
  - *Como fazer:* identifique α e β; verifique a condição; para Wythoff, use φ e φ^2 e teste a posição; para gerar, calcule os pisos. Custo O(1) por termo.
  - *Exemplo:* $\alpha = \varphi$, $\beta = \varphi^2$; Wythoff (1,2), (3,5), (4,7), ...
  - *Armadilha:* α racional quebra a partição; a indexação começa em $n \ge 1$; erros de arredondamento em φ podem afetar o piso.
- **Sequências equidistribuídas.** $\{n\alpha\}$ é equidistribuída para α irracional; teorema dos três intervalos: os pontos $\{k\alpha\}$, $k < N$, dividem o círculo em intervalos de ≤ 3 comprimentos distintos. Aproximação de Dirichlet: $\exists\ q \le N$ com $|q\alpha - p| < 1/N$. Gatilho: "gire por α a cada passo" → frações contínuas.
  - *Gatilho:* rotações no círculo, distribuição de pontos, ou aproximação diofantina.
  - *Por que funciona:* a equidistribuição vem da irracionalidade; o teorema dos três intervalos descreve a estrutura dos gaps; Dirichlet usa pigeonhole nos múltiplos.
  - *Como fazer:* para gaps, use frações contínuas/convergentes; para aproximação, calcule convergentes com $q \le N$; para simular, mantenha a parte fracionária. Custo O(log N) por convergente.
  - *Exemplo:* gaps de $\{k\varphi\}$ para $k = 0..N-1$; aproximar α por p/q.
  - *Armadilha:* α racional tem comportamento periódico; precisão de double limita N; a cota de Dirichlet é estrita.
- **Frações contínuas / Stern–Brocot.** Melhor aproximação racional p/q com $q \le N$: convergentes e semiconvergentes; busca por p/q entre dois racionais com menor denominador: desça Stern–Brocot em O(log) com saltos. Gatilho: "menor fração em (a/b, c/d)".
  - *Gatilho:* aproximar um real por racionais, ou achar a fração de menor denominador em um intervalo.
  - *Por que funciona:* convergentes são as melhores aproximações; semiconvergentes cobrem os casos intermediários; a árvore de Stern–Brocot ordena todas as frações positivas e permite saltos por divisão.
  - *Como fazer:* calcule a fração contínua de α e teste convergentes; para o intervalo, desça a Stern–Brocot comparando com os extremos e acelere com quocientes. Custo O(log max).
  - *Exemplo:* menor denominador entre 1/3 e 1/2 é 2/5; aproximar π por 22/7.
  - *Armadilha:* os extremos podem ser inclusivos ou exclusivos; números gigantes exigem 128 bits; a árvore usa frações positivas.

- **Funções aritméticas.** $\Sigma_{d|n}\varphi(d) = n$; $\Sigma_{d|n}\mu(d) = [n = 1]$; $\Sigma_{i\le n}\gcd(i, n) = \Sigma_{d|n}\, d\cdot\varphi(n/d)$; $\Sigma_{i\le n}\tau(i) \approx n \ln n$; densidade de pares coprimos = $6/\pi^2 \approx 0{,}608$. Multiplicativa → crivo linear; Dirichlet: $(f*1)$ é soma sobre divisores.
  - *Gatilho:* somas sobre divisores, totientes, Möbius, ou contagem de coprimos.
  - *Por que funciona:* φ, μ e τ são multiplicativas; as identidades de Dirichlet convertem somas sobre divisores em convoluções; o crivo linear calcula todas em O(N).
  - *Como fazer:* use crivo linear para φ, μ, τ; para somas sobre divisores, itere d e seus múltiplos; para coprimos, use μ e prefixos. Custo O(N log log N) ou O(N).
  - *Exemplo:* $\Sigma_{d|12}\varphi(d) = 1+1+2+2+2+4 = 12$; número de pares coprimos até N.
  - *Armadilha:* 1 é coprimo de todos; $\mu(1) = 1$; a densidade $6/\pi^2$ é assintótica, não exata para N finito.
- **Soma de dois quadrados.** $n = a^2 + b^2$ iff todo primo $\equiv 3 \pmod 4$ tem expoente par; $r_2(n) = 4(d_1(n) - d_3(n))$ (divisores $\equiv 1$ e $3 \bmod 4$). Gaussianos inteiros: norma multiplicativa. Gatilho: "pontos de rede a distância $\sqrt{n}$ da origem".
  - *Gatilho:* representar n como soma de dois quadrados, contar pontos de lattice em um círculo, ou fatorar em Gaussianos.
  - *Por que funciona:* em Z[i], a norma é multiplicativa; primos $\equiv 3 \bmod 4$ permanecem primos e precisam de expoente par; primos $\equiv 1 \bmod 4$ se decompõem.
  - *Como fazer:* fatore n; verifique os expoentes dos primos 3 mod 4; para contar, calcule d_1 e d_3 e use $4(d_1 - d_3)$; para construir, combine as representações dos fatores. Custo O(√n) para fatorar ou O(n^{1/4}) com técnicas melhores.
  - *Exemplo:* $25 = 0^2 + 5^2 = 3^2 + 4^2$; $r_2(25) = 12$.
  - *Armadilha:* a fórmula conta sinais e ordens; n = 0 é especial; a condição é sobre expoentes pares, não sobre a ausência do primo.
- **Números complexos como rotações.** Multiplicar por e^{iθ} rotaciona; z^k/ potências de i para movimentos periódicos; vetores 2D como complexos simplificam rotação/escala. Trigonometria exata: use racionais (vetores inteiros), nunca graus.
  - *Gatilho:* rotações repetidas, movimentos periódicos, ou geometria 2D com ângulos.
  - *Por que funciona:* a multiplicação por e^{iθ} é uma rotação; potências de i ciclam; representar vetores como complexos ou pares inteiros evita funções trigonométricas.
  - *Como fazer:* use pares (x,y) e as fórmulas de rotação; para ângulos retos, use i; para exatidão, mantenha vetores inteiros e compare produtos. Custo O(1) por rotação.
  - *Exemplo:* girar (1,0) por 90° → (0,1); $i^4 = 1$.
  - *Armadilha:* `double` acumula erro; graus não são exatos; a orientação do ângulo depende da convenção.
- **Esperança por indicador e linearidade.** $E[X] = \Sigma\, P(\text{evento}_i)$, mesmo dependentes. Registros em permutação aleatória: H_n; ciclos de permutação aleatória: H_n; pontos fixos: 1; cupom: $n \cdot H_n$; profundidade média em BST aleatório $\approx 2 \ln n$ (altura $\approx 4{,}3 \ln n$); E[comprimento da LIS] $\approx 2\sqrt{n}$.
  - *Gatilho:* valor esperado de uma contagem em um processo aleatório.
  - *Por que funciona:* a linearidade da esperança não exige independência; basta escrever X como soma de indicadores e calcular cada probabilidade.
  - *Como fazer:* defina um indicador por posição/elemento; calcule P(evento); some; para permutações, use simetria. Custo O(n) ou O(1) por termo.
  - *Exemplo:* pontos fixos: $E = n \cdot (1/n) = 1$; cupom: nH_n.
  - *Armadilha:* dependência entre indicadores não afeta a esperança, mas afeta a variância; as aproximações de BST/LIS são assintóticas.
- **Cadeias e martingais.** Ache potencial Φ com $E[\Phi(\text{próximo}) - \Phi(\text{atual})] = -1$ por passo: então $E[\text{tempo}] = \Phi(\text{início}) - \Phi(\text{fim})$. Passeio aleatório em linha de 0..n: $E[\text{tempo}] = i(n - i)$; ruína do jogador: $P(\text{chegar a } n) = i/n$ (caso justo). Gatilho: "esperança de passos até terminar" com estrutura linear.
  - *Gatilho:* tempo esperado de absorção, passeio aleatório, ou ruína do jogador.
  - *Por que funciona:* se Φ tem drift esperado −1, o processo $\Phi(X_t) + t$ é um martingale; opcional stopping dá $E[T] = \Phi(\text{início}) - \Phi(\text{fim})$.
  - *Como fazer:* proponha Φ (quadrática na posição); imponha a equação de drift; resolva as condições de contorno; para probabilidade de ruína, use Φ linear. Custo O(n) ou O(1).
  - *Exemplo:* passeio em 0..n, tempo esperado $i(n-i)$; $P(\text{reach } n) = i/n$.
  - *Armadilha:* o tempo precisa ter esperança finita e as condições de parada valem; fronteiras absorventes; passeio enviesado muda as fórmulas.

- **Independência: Caro–Wei.** $\alpha(G) \geq \Sigma_v 1/(\deg(v) + 1)$ — guloso (tire o vértice de menor grau e seus vizinhos) alcança. Turán: $\alpha \geq n/(\bar{d} + 1)$. Gatilho: "achar conjunto independente de tamanho $\geq n/(d+1)$" ou "ache vértices sem arestas entre si".
  - *Gatilho:* garantir um conjunto independente grande em grafo esparso ou de grau limitado.
  - *Por que funciona:* em uma ordem aleatória, um vértice é escolhido se aparece antes de todos os vizinhos, com probabilidade $1/(d+1)$; a linearidade dá a cota; o guloso de menor grau atinge a mesma cota.
  - *Como fazer:* para a cota, calcule $\Sigma 1/(d+1)$; para construir, remova repetidamente o vértice de menor grau, adicione-o ao independente e remova seus vizinhos. Custo $O(n + m)$ com fila de graus.
  - *Exemplo:* triângulo: $3 \cdot 1/3 = 1$, $\alpha = 1$; ciclo C5: $5/3$, $\alpha = 2$.
  - *Armadilha:* laços e arestas múltiplas mudam o grau; a cota é sobre o grafo simples; o guloso de menor grau precisa remover vizinhos.
- **Mantel / Turán e limites de m.** Sem triângulo $\Rightarrow m \leq \lfloor n^2/4 \rfloor$; sem $K_{r+1} \Rightarrow m \leq (1 - 1/r)n^2/2$. Sem C_4 (ou: dois vértices têm ≤ 1 vizinho comum) $\Rightarrow m = O(n\sqrt{n})$ e $\Sigma_v C(\deg v, 2) \leq C(n, 2)$: iterar pares de vizinhos é $O(n^2)$. Gatilho: "grafo sem ciclo de comprimento 4".
  - *Gatilho:* limitar arestas sob ausência de um subgrafo, ou justificar uma enumeração de pares de vizinhos.
  - *Por que funciona:* Mantel/Turán são os teoremas extremos; sem C4, cada par de vértices tem no máximo um vizinho comum, então $\Sigma\, C(\deg v,2) \leq C(n,2)$.
  - *Como fazer:* use as cotas para limitar m; para sem C4, conte pares de vizinhos e conclua $m = O(n^{3/2})$. Custo $O(n + m)$.
  - *Exemplo:* grafo bipartido completo $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$ atinge $n^2/4$ sem triângulo.
  - *Armadilha:* as cotas são para grafos simples; sem C4 não significa livre de triângulos; o extremal de Turán é a união equilibrada de r partes.
- **Grafos planares.** $m \leq 3n - 6$ e há vértice de grau ≤ 5 (degenerescência ≤ 5): triângulos/K_4 em $O(n)$, orientação com grau de saída ≤ 5, coloração com 6 cores em $O(n)$. Euler: $V - E + F = 1 + C$. Fluxo s–t = corte mínimo; no planar, corte mínimo $\leftrightarrow$ ciclo mínimo no dual que separa s e t. Se s, t estão na mesma face, divida a face com uma aresta st e o corte vira o caminho mínimo no dual entre as duas faces novas (sem usar a aresta dual de st).
  - *Gatilho:* grafo planar, coloração, detecção de subgrafo, ou corte mínimo planar.
  - *Por que funciona:* a fórmula de Euler limita m; a degenerescência 5 permite remover vértices de grau baixo; o dual transforma cortes em ciclos e o corte s–t em caminho mínimo.
  - *Como fazer:* use $m \leq 3n - 6$; remova vértices de grau ≤ 5 para colorir/orientar; para corte, construa o dual, separe as faces e calcule o ciclo/caminho mínimo. Custo $O(n)$ para as propriedades e $O(n \log n)$ para o dual.
  - *Exemplo:* colorir um grafo planar com 6 cores; corte mínimo em um grid planar.
  - *Armadilha:* a cota $m \leq 3n - 6$ vale para $n \geq 3$ e grafo simples; desconexo e laços precisam de ajuste; a dualidade depende da imersão planar.
- **Euler em mapas.** $V - E + F = 1 + C$: contar componentes/buracos de uma figura em grid pelas células, arestas e vértices (ou pelo número de Euler local por 2×2) dispensa BFS; vale para qualquer desenho planar.
  - *Gatilho:* contar componentes, buracos ou regiões de uma figura em grid sem BFS.
  - *Por que funciona:* a característica de Euler é invariante por subdivisão; ao contar células, arestas e vértices da figura, a fórmula revela C ou o número de faces.
  - *Como fazer:* conte V, E e F (incluindo a face externa); use $C = V - E + F - 1$; ou some a contribuição local de cada bloco 2×2. Custo $O(nm)$.
  - *Exemplo:* número de buracos de um poliminó; componentes de uma máscara.
  - *Armadilha:* a face externa conta; conectividade 4 ou 8 muda a contagem de arestas/vértices; tocar na borda do grid afeta o complemento.
- **Ramsey.** $R(3,3) = 6$: em qualquer 2-coloração de K_6 há triângulo monocromático; $R(4,4) = 18$. Se n é $\geq \sim 6$ a resposta "existe" é sempre sim; também, um grafo ou seu complemento é conexo — gatilho de construção em duas cores.
  - *Gatilho:* colorir arestas com duas cores e garantir uma clique monocromática, ou um argumento de grafo/complemento.
  - *Por que funciona:* $R(3,3) = 6$ e $R(4,4) = 18$ são os números de Ramsey; para o complemento, se G é desconexo, o complemento liga componentes diferentes.
  - *Como fazer:* para $n \geq 6$ e triangle, invoque a garantia; para construir, use a coloração extremal para n abaixo do limite; para conectividade, verifique G e o complemento. Custo $O(1)$ para a garantia.
  - *Exemplo:* 6 pessoas, sempre há 3 amigos mútuos ou 3 estranhos mútuos; K_5 tem uma 2-coloração sem triângulo monocromático.
  - *Armadilha:* o limiar é exato; $R(4,4) = 18$, não 6; o grafo ou complemento conexo não garante ambos conexos.

- **Dirac / Ore.** Grau mínimo ≥ n/2 ⇒ hamiltoniano; Ore: deg u + deg v ≥ n para não adjacentes. Construção $O(n^2)$ por rotações de Pósa. Gatilho: "grafo denso, ache ciclo hamiltoniano" → sempre existe, só constrói.
  - *Gatilho:* grafo denso com condições de grau e a necessidade de um ciclo hamiltoniano.
  - *Por que funciona:* Dirac e Ore garantem a existência; as rotações de Pósa estendem um caminho e fecham um ciclo sem perder vértices.
  - *Como fazer:* verifique as condições; construa um caminho por inserção/rotações e feche o ciclo; se a condição de Ore falhar, o teorema não se aplica. Custo $O(n^2)$.
  - *Exemplo:* grafo completo K_n; grafo com grau mínimo n/2.
  - *Armadilha:* Dirac exige n ≥ 3; as condições são suficientes, não necessárias; o algoritmo de construção pode ser mais complexo que a verificação.
- **Hall e König.** Perfeito bipartido existe iff |L| = |R| e |N(S)| ≥ |S| para todo S ⊆ L (ou Hall nos dois lados); deficiência: ν = |L| − max_S(|S| − |N(S)|). König: cobertura mínima = emparelhamento máximo; indep. máximo = n − ν. Grafo bipartido k-regular decompõe-se em k emparelhamentos perfeitos; arestas colorem-se com Δ cores (König).
  - *Gatilho:* emparelhamento bipartido, cobertura de vértices, ou coloração de arestas.
  - *Por que funciona:* Hall é a condição necessária e suficiente; König iguala emparelhamento máximo e cobertura mínima; a decomposição regular segue por indução.
  - *Como fazer:* verifique Hall ou calcule o emparelhamento máximo; para a deficiência, maximize |S| − |N(S)|; para colorir arestas, decomponha em emparelhamentos. Custo $O(E\sqrt{V})$ ou $O(n^3)$.
  - *Exemplo:* assignment com vagas; cobertura mínima de um grid bipartido.
  - *Armadilha:* Hall precisa valer para todos os subconjuntos; no caso não balanceado, satura o lado menor; vértices isolados quebram o emparelhamento perfeito.
- **Tutte / Tutte–Berge.** Perfeito iff para todo S, componentes ímpares de G − S ≤ |S|; ν = (n − max_S(odd(G − S) − |S|))/2. Gatilhos: árvore tem perfeito iff, removido cada vértice, sobra exatamente uma componente de tamanho ímpar (greedy a partir das folhas); grafo conexo com número par de arestas decompõe-se em caminhos de 2 arestas.
  - *Gatilho:* emparelhamento perfeito em grafo geral, ou uma condição estrutural em árvores.
  - *Por que funciona:* Tutte caracteriza a existência; Tutte–Berge dá o tamanho máximo; em árvores, a condição se simplifica e o greedy das folhas constrói o emparelhamento.
  - *Como fazer:* para árvores, faça DP de folhas para cima ou verifique a condição de componentes ímpares; para grafos gerais, o algoritmo de blossom é o prático; a decomposição em P3 usa emparelhamento de arestas. Custo O(n) na árvore, $O(n^3)$ no geral.
  - *Exemplo:* estrela K_{1,3} não tem emparelhamento perfeito; caminho P4 tem.
  - *Armadilha:* verificar todos os S é exponencial; a condição da árvore é específica; a decomposição em P3 não é a mesma coisa que emparelhamento de vértices.
- **Gallai e coberturas.** α + τ = n (indep. + cobertura de vértices); ν + ρ = n (emparelhamento + cobertura de arestas, sem isolados). Cobertura de caminhos disjuntos em DAG = n − ν da bipartição entrada/saída.
  - *Gatilho:* relações entre independência, cobertura, emparelhamento e caminhos.
  - *Por que funciona:* o complemento de uma cobertura de vértices é independente; o complemento de um emparelhamento máximo em relação a uma cobertura de arestas dá a identidade de Gallai; a cobertura de caminhos usa a bipartição.
  - *Como fazer:* use α = n − τ; para caminhos, monte a bipartição (entrada/saída), calcule o emparelhamento máximo e devolva n − ν. Custo $O(E\sqrt{V})$.
  - *Exemplo:* cobertura mínima de caminhos em um DAG; cobertura de arestas de um grafo sem isolados.
  - *Armadilha:* a identidade ν + ρ = n exige que não haja vértices isolados; caminhos disjuntos permitem caminhos de comprimento 0; a bipartição precisa da versão correta.
- **Sequências gráficas.** Erdős–Gallai: $d_1 \geq \ldots \geq d_n$ é gráfica iff Σ par e $\Sigma_{i\leq k} d_i \leq k(k-1) + \Sigma_{i>k} \min(d_i, k)$; construção por Havel–Hakimi (ligue o maior aos próximos maiores). Torneios: Landau — ordenadas $s_1 \leq \ldots \leq s_n$, $\Sigma_{i\leq k} s_i \geq C(k,2)$ com igualdade em k = n. Bipartido: Gale–Ryser.
  - *Gatilho:* decidir se uma sequência de graus é realizável, ou construir um grafo/torneio com graus dados.
  - *Por que funciona:* Erdős–Gallai é a caracterização completa; Havel–Hakimi constrói removendo o maior grau; Landau é a versão para torneios; Gale–Ryser para bipartidos.
  - *Como fazer:* ordene; verifique a paridade e as desigualdades; para construir, conecte o maior vértice aos próximos maiores e recorra; para torneios, use Landau. Custo $O(n^2)$ ou O(n log n).
  - *Exemplo:* 3,3,2,2,2 é gráfica? Verifique as desigualdades; 3,3,3,3 é K4.
  - *Armadilha:* a sequência precisa ser simples (sem laços/multigrafo); a soma dos graus deve ser par; Landau usa a ordem não decrescente e a igualdade no fim.

- **Helly (1D e geral).** Intervalos que se intersectam dois a dois têm ponto comum: max(l_i) ≤ min(r_i). Em R^d convexos: basta d + 1. Gatilho: "todos os pares de intervalos se cruzam" ⇒ existe um ponto comum; verifique só max(l_i) ≤ min(r_i). No plano, segmentos que se cruzam dois a dois não têm ponto comum em geral.
  - *Gatilho:* interseção pairwise de intervalos/convexos, ou um ponto que serve para todos.
  - *Por que funciona:* em 1D, o maior início e o menor fim testemunham a interseção comum; em R^d, Helly garante que d + 1 conjuntos bastam.
  - *Como fazer:* para intervalos, calcule max(l) e min(r); para convexos, verifique todos os subconjuntos de tamanho d + 1 ou use a estrutura. Custo O(n) em 1D.
  - *Exemplo:* intervalos [1,4], [2,5], [3,6] têm ponto comum; três segmentos formando um triângulo se cruzam aos pares sem ponto comum.
  - *Armadilha:* Helly vale para convexos, não para segmentos arbitrários; a dimensão d + 1 é exata; intervalos abertos mudam o teste de igualdade.
- **Extremal / Sperner / EKR.** Antichain de subconjuntos de [n] tem ≤ C(n, ⌊n/2⌋) (Sperner); k-subconjuntos que se intersectam par a par têm ≤ C(n−1, k−1) (Erdős–Ko–Rado, n ≥ 2k). Gatilho: "família máxima sem inclusão / com interseção" → resposta fechada.
  - *Gatilho:* família de subconjuntos máxima sob ausência de inclusão ou com interseção pairwise.
  - *Por que funciona:* Sperner usa a cadeia simétrica/LYM; EKR usa o círculo ou emparelhamento e exige n ≥ 2k.
  - *Como fazer:* para antichain, a resposta é a camada do meio; para interseção, todos os k-subconjuntos que contêm um elemento fixo atingem a cota. Custo O(1) para a cota.
  - *Exemplo:* n = 4, maior antichain = C(4,2) = 6; k = 2, n = 4, maior família intersectante = C(3,1) = 3.
  - *Armadilha:* EKR falha para n < 2k; "interseção par a par" não é a mesma coisa que interseção total; a família pode ter tamanhos diferentes no caso Sperner.
- **Hipergrafos de intervalos e grafos de intervalo.** Cor mínima = clique máxima = máxima sobreposição (grafo de intervalos é perfeito); cordal e perfeito; ordem de eliminação simplicial. Escalonamento de k máquinas = sweep com heap.
  - *Gatilho:* intervalos com sobreposição, agendamento, ou coloração de intervalos.
  - *Por que funciona:* grafos de intervalos são perfeitos; a clique máxima é a maior sobreposição simultânea; a coloração gulosa por ordem de término usa o número de sobreposição.
  - *Como fazer:* faça um sweep dos endpoints com heap; a máxima sobreposição é a clique; para colorir, atribua a menor cor livre; para k máquinas, aloque o intervalo ao recurso que libera primeiro. Custo O(n log n).
  - *Exemplo:* salas de aula para intervalos de aulas; colorir intervalos com o máximo de sobreposição.
  - *Armadilha:* extremos que se tocam podem ou não sobrepor; intervalos abertos/fechados mudam a resposta; a ordem de eliminação importa.
- **Teorema de Pick e variantes.** A = I + B/2 − 1 com B = Σ gcd(|Δx|, |Δy|) sobre arestas do polígono. Triângulo de rede sem pontos internos nem de fronteira (além dos vértices) tem área 1/2; contagem de I a partir de A e B dispensa varredura.
  - *Gatilho:* área de polígono com vértices inteiros, ou contagem de pontos internos/de fronteira.
  - *Por que funciona:* Pick relaciona área, pontos interiores e pontos de fronteira para polígonos de lattice simples; o gcd conta os pontos de lattice em cada aresta.
  - *Como fazer:* calcule A por soma de produtos cruzados; calcule B somando gcd(|Δx|,|Δy|); obtenha I = A − B/2 + 1. Custo O(n).
  - *Exemplo:* triângulo (0,0),(1,0),(0,1): A = 1/2, B = 3, I = 0.
  - *Armadilha:* o polígono precisa ser simples (sem autointerseção); Pick não vale para furos diretamente; a orientação afeta o sinal da área.
- **Geometria analítica útil.** Reta ax + by = c com gcd(a, b) = 1; distância ponto–reta $|ax_0 + by_0 - c|/\sqrt{(a^2 + b^2)}$; pontos de rede em segmento = gcd(|Δx|, |Δy|) + 1; área = (1/2)|cross|; compare quadrados (inteiros) em vez de distâncias.
  - *Gatilho:* retas, distâncias, pontos de lattice, ou comparação de áreas/ângulos.
  - *Por que funciona:* normalizar a reta evita fatores comuns; o gcd conta os passos inteiros no segmento; o produto cruzado dá a área e a orientação.
  - *Como fazer:* normalize (a,b,c) por gcd e sinal; use distância com raiz apenas se necessário; compare d^2; para lattice, gcd + 1. Custo O(log C) por normalização.
  - *Exemplo:* distância de (0,0) a 3x + 4y = 10 é 2; segmento (0,0)–(6,9) tem gcd(6,9)+1 = 4 pontos.
  - *Armadilha:* overflow em produtos; normalizar o sinal; `sqrt` de inteiros pode perder precisão.

- **Desigualdade triangular como poda.** d(u, w) ≤ d(u, v) + d(v, w) ⇒ em árvore, o ponto mais distante de qualquer v é um dos extremos do diâmetro (exato); em métrica geral isso falha, mas ecc(v) ≤ 2·max(d(v, a), d(v, b)) para a, b extremos de um diâmetro (2-aprox). Em Manhattan, soma de distâncias é separável por coordenada; mediana minimiza.
  - *Gatilho:* diâmetro, ponto mais distante, poda por distância, ou soma de distâncias em Manhattan.
  - *Por que funciona:* em árvore, o caminho mais longo passa por um diâmetro; em métrica geral, a desigualdade triangular e o diâmetro limitam a excentricidade; Manhattan separa coordenadas.
  - *Como fazer:* faça dois BFS/DFS para achar um diâmetro; use os extremos para consultas; para Manhattan, resolva por coordenada ou use mediana. Custo O(n) na árvore, O(n log n) para mediana.
  - *Exemplo:* diâmetro de uma árvore; ponto que minimiza a soma de distâncias em Manhattan.
  - *Armadilha:* a propriedade exata é só para árvores; em grafos gerais, o endpoint do diâmetro não dá o mais distante; a 2-aprox é uma cota superior.
- **Schwartz–Zippel e grau ≤ d.** Polinômio não nulo de grau d em n variáveis tem raiz em S^n com probabilidade ≤ d/|S| (teste de identidade); em 1 variável, ≤ d raízes: dois polinômios de grau ≤ d concordantes em d + 1 pontos são iguais. Fundamento de hashing de polinômio e verificação de matrizes.
  - *Gatilho:* teste de identidade, comparação de expressões, ou verificação aleatória.
  - *Por que funciona:* um polinômio não nulo tem poucas raízes; escolher o ponto aleatoriamente em um conjunto grande torna a probabilidade de acertar uma raiz pequena.
  - *Como fazer:* escolha S grande; avalie em um ponto aleatório; repita para reduzir o erro; em 1D, use d + 1 pontos para certeza. Custo O(avaliação) por tentativa.
  - *Exemplo:* verificar AB = C por Freivalds; comparar dois polinômios de grau 10^5.
  - *Armadilha:* a cota é d/|S|, não zero; entradas adversárias podem escolher o ponto se ele for determinístico; em corpos pequenos, a probabilidade é alta.
- **Ordem de grandeza.** H_n ≈ ln n + 0,5772; n! ≈ $\sqrt{(2\pi n)}(n/e)^n$; $\log_2(n!) \approx n \log_2 n - 1{,}44n$ (cota inferior de comparações/queries); C(2n,n) ≈ $4^n/\sqrt{(\pi n)}$; Catalan ≈ $4^n/(n^{3/2}\sqrt{\pi})$; π(x) ≈ x/ln x (π(10^6) = 78 498, π(10^9) = 50 847 534); p(100) = 190 569 292; lcm(1..43) ≈ $9{,}42 \cdot 10^{18}$ (estoura int64 com sinal; cabe em uint64).
  - *Gatilho:* estimar complexidade, comparar ordens de grandeza, ou checar overflow.
  - *Por que funciona:* as aproximações de Stirling, harmônicos e o teorema dos números primos dão a ordem dominante; as constantes são úteis para limites práticos.
  - *Como fazer:* substitua n nas fórmulas; para fatorial, use Stirling; para lcm, considere as maiores potências de primos; use 128 bits ou módulo quando necessário. Custo O(1).
  - *Exemplo:* $\log_2(10^6!) \approx 10^6 \cdot 20 - 1{,}44 \cdot 10^6 \approx 1{,}86 \cdot 10^7$; lcm(1..43) não cabe em `long long` com sinal.
  - *Armadilha:* as aproximações têm termos de erro; int64 com sinal estoura em $9{,}22 \cdot 10^{18}$; π(x) ≈ x/ln x é assintótico, não exato.
- **Cota de informação.** Se há M respostas possíveis e cada query tem b respostas, são necessárias ≥ ⌈log_b M⌉ queries; em interativos o limite do enunciado é quase sempre isso — descubra a estratégia que divide M igualmente.
  - *Gatilho:* problema interativo com limite de queries, ou prova de cota inferior.
  - *Por que funciona:* cada query cria uma árvore de decisão com no máximo b ramos; para distinguir M folhas, a profundidade precisa ser pelo menos log_b M.
  - *Como fazer:* conte as respostas possíveis; calcule o limite; desenhe uma estratégia que divida o conjunto de possibilidades em partes quase iguais. Custo O(1) para a cota.
  - *Exemplo:* adivinhar um número em [1,10^6] com respostas sim/não requer ≥ 20 queries.
  - *Armadilha:* queries com mais de b resultados mudam a base; respostas adaptativas não furam a cota; o limite pode não ser atingível.
- **Limites de crescimento.** Σ_{i≤n} 1/i gera n·H_n ≈ n ln n para laços "para cada i, múltiplos de i"; soma de divisores σ(n) ≤ n·H_n; nº de divisores de n é O(n^{ε}); Σ τ(i) para i ≤ 10^7 ≈ $1{,}6 \cdot 10^8$.
  - *Gatilho:* laços harmônicos, soma de divisores, ou estimativa de crivo.
  - *Por que funciona:* o laço de múltiplos executa n/i vezes para cada i; a soma dos recíprocos é H_n; σ(n) soma divisores e é limitada por nH_n.
  - *Como fazer:* reconheça o padrão n/i; estime n ln n; para σ, fatore ou use crivo; para τ médio, use a soma harmônica. Custo O(n log n) no laço.
  - *Exemplo:* crivo de múltiplos até 10^7 executa ≈ $1{,}6 \cdot 10^8$ iterações; σ(12) = 28.
  - *Armadilha:* a constante de H_n importa; o limite O(n^ε) para τ é assintótico; a soma dos τ até N é ≈ N ln N + (2γ−1)N.

- **Ponto flutuante.** Evite subtrair números próximos (use fórmulas alternativas: hypot, atan2, log1p, expm1); compare por razão cruzada em inteiros (__int128 para até $10^{18} \times 10^{18}$); binária em reais: itere 60–100 vezes em vez de usar eps; √ inteira: s = sqrtl(x) e ajuste nos dois sentidos (s++ enquanto $(s+1)^2 \le x$; s−− enquanto $s^2 > x$), com s^2 em __int128; soma de muitos termos: Kahan ou ordene por magnitude.
  - *Gatilho:* geometria computacional, comparações de distâncias, somas longas, ou raízes inteiras.
  - *Por que funciona:* a subtração de valores próximos cancela dígitos significativos; comparar produtos cruzados evita divisão e arredondamento; Kahan reduz o erro da soma.
  - *Como fazer:* use `hypot`, `atan2`, `log1p`, `expm1`; compare d_1^2 vs d_2^2 em inteiros; faça busca binária com número fixo de iterações; ajuste a raiz com `__int128`; para somas, use Kahan. Custo O(1) por operação.
  - *Exemplo:* comparar |a−b| com |c−d| via quadrados; calcular sqrt de 10^{18} com ajuste.
  - *Armadilha:* `eps` fixo não escala; `double` tem 53 bits; `sqrtl` pode errar por 1; NaN e infinito precisam ser tratados.
- **Provar guloso por desigualdade.** Para checar uma ordem: escreva custo de trocar adjacentes i, j e reduza a uma comparação de pares (ex.: $a_i \cdot b_j$ vs $a_j \cdot b_i \rightarrow$ ordene por a/b; a+b vs b+a em strings). Se a comparação é transitiva e total, ordenar é correto.
  - *Gatilho:* "escolha a ordem", agendamento, concatenação, ou uma troca local que deve melhorar.
  - *Por que funciona:* se trocar dois vizinhos invertidos nunca piora, toda ordem ótima pode ser transformada na ordem ordenada por trocas adjacentes; a transitividade garante que a ordenação é bem definida.
  - *Como fazer:* escreva o custo com i antes de j e com j antes de i; cancele os termos comuns; obtenha o comparador; verifique transitividade; ordene. Custo O(n log n).
  - *Exemplo:* concatenar strings: i antes de j se i+j < j+i; tarefas com tempo/peso: ordenar por t/w.
  - *Armadilha:* o comparador precisa ser transitivo; compare produtos cruzados em inteiros, não divisões em double; empates precisam de critério consistente.
- **Truques de somatório.** Troque a ordem $\Sigma_i \Sigma_{j|i} = \Sigma_j \Sigma_{k \le n/j}$; use $\Sigma_{i<j} f = (1/2)(\Sigma_{i,j} - \Sigma_i f(i,i))$ para simetria; $(\Sigma x)^2 = \Sigma x^2 + 2\Sigma_{i<j} x_i x_j$ para tirar pares; expanda produtos e agrupe por potência $(\Sigma_i \Sigma_j (a_i - a_j)^2 = 2n \cdot \Sigma a^2 - 2(\Sigma a)^2)$.
  - *Gatilho:* somas duplas, somas sobre divisores, ou pares simétricos.
  - *Por que funciona:* trocar a ordem reordena os mesmos termos; a simetria permite contar cada par duas vezes e dividir; a expansão do quadrado separa termos individuais e pares.
  - *Como fazer:* desenhe a região de soma; troque os limites; para pares, use a identidade de simetria; expanda e agrupe. Custo O(n) ou O(n log n).
  - *Exemplo:* $\Sigma_i \Sigma_{j|i} = \Sigma_j \lfloor n/j \rfloor$; $(\Sigma a)^2$ para contar pares.
  - *Armadilha:* os limites após a troca; o termo diagonal i = j precisa ser subtraído; somas ordenadas vs não ordenadas.
- **Variância e pares.** $\Sigma_{i<j}(a_i - a_j)^2 = n \cdot \Sigma a_i^2 - (\Sigma a_i)^2$: soma de quadrados das diferenças em O(n); para absoluta $|a_i - a_j|$, ordene e use prefixos.
  - *Gatilho:* soma de diferenças ao quadrado entre todos os pares, ou variância.
  - *Por que funciona:* expandir $(a_i - a_j)^2$ e contar quantas vezes cada a_i^2 e cada produto aparece; o resultado simplifica para $n\Sigma a^2 - (\Sigma a)^2$.
  - *Como fazer:* calcule a soma e a soma dos quadrados; aplique a fórmula; para $|a_i - a_j|$, ordene e use prefixos para remover os valores menores. Custo O(n) ou O(n log n).
  - *Exemplo:* a = [1,2,3]: Σ pares de diferenças² = 1+4+1 = 6; 3·14 − 36 = 6.
  - *Armadilha:* a fórmula é para pares não ordenados; para variância, divida por n ou n−1 conforme a definição; overflow.
- **Convexidade e extremos.** Máximo de função convexa sobre politopo ocorre num vértice; mínimo de função côncava idem. Gatilho: "função convexa em intervalo fechado, ache o máx" → só extremos; minimizar côncava = testar vértices.
  - *Gatilho:* otimizar uma função convexa/côncava em um domínio poliedral.
  - *Por que funciona:* uma função convexa tem epígrafo convexo; o máximo em um compacto ocorre em um ponto extremo; para côncava, o mínimo também.
  - *Como fazer:* identifique a convexidade; teste os vértices; em 1D, teste os extremos do intervalo; se necessário, use ternary search para o mínimo de convexa. Custo O(número de vértices) ou O(log).
  - *Exemplo:* máximo de x^2 em [−1,2] é 4; mínimo de $-x^2$ em [−1,2] é −4.
  - *Armadilha:* a função precisa ser convexa no domínio inteiro; o máximo de uma convexa não é no ponto crítico; empates em arestas.

- **Subseqüências e somas de 2 classes.** Sempre que uma operação preserva "soma mod k" e o alvo exige outro resto → impossível; se a operação gera mod k × restos de uma base, procure classes de equivalência por gcd (ex.: valores alcançáveis = múltiplos de gcd dentro dos limites).
  - *Gatilho:* operações que somam/subtraem valores fixos e uma pergunta de alcançabilidade.
  - *Por que funciona:* o resto módulo k é invariante sob certas operações; o gcd dos passos determina as classes alcançáveis; os limites podem restringir ainda mais.
  - *Como fazer:* calcule o gcd dos incrementos; verifique o resto do alvo; se necessário, use BFS/DP nos restos para os limites. Custo O(k) ou O(k log k).
  - *Exemplo:* com moedas de 4 e 6, só são alcançáveis os múltiplos de gcd(4,6) = 2; passos de +3 e +5 alcançam todos os restos módulo 1.
  - *Armadilha:* o gcd é necessário, mas os limites podem impedir a alcançabilidade; operações não lineares quebram o invariante.
- **Moedas / Frobenius.** Com a, b coprimos, o maior valor não representável é ab − a − b e há (a − 1)(b − 1)/2 não representáveis. Muitas moedas: Dijkstra nos restos módulo a menor moeda (O(a_min·n log)); valores acima de max_r dist[r] (o maior dos menores representáveis por resíduo) são todos alcançáveis; com gcd > 1, só múltiplos do gcd.
  - *Gatilho:* representar valores com moedas, maior valor impossível, ou contagem de não representáveis.
  - *Por que funciona:* o semigrupo numérico tem periodicidade módulo a menor moeda; Dijkstra nos restos encontra o menor valor representável em cada classe; acima do máximo desses valores, todos são representáveis.
  - *Como fazer:* para duas moedas coprimas, use ab−a−b e (a−1)(b−1)/2; para muitas, rode Dijkstra nos restos módulo a_min; o limiar é max dist. Custo O(a_min·n log).
  - *Exemplo:* moedas 3 e 5: maior não representável = 15−3−5 = 7; não representáveis = 2·4/2 = 4.
  - *Armadilha:* a fórmula de duas moedas exige coprimalidade; com gcd > 1, divida tudo pelo gcd; o limiar pode ser grande.
- **Prova por colorir.** Dominós em tabuleiro: cobertura perfeita exige #brancas = #pretas; coloração 2×2 ou por linhas mostra impossibilidade de L-triominós/tetraminós. Gatilho: "cobrir com peças de forma fixa" → teste invariantes de coloração antes de procurar construção.
  - *Gatilho:* cobrir/particionar uma região com peças fixas e provar impossibilidade.
  - *Por que funciona:* cada peça cobre um número fixo de casas de cada cor; uma coloração bem escolhida dá uma equação que a região precisa satisfazer.
  - *Como fazer:* escolha uma coloração periódica; conte as casas de cada cor na região e o que cada peça cobre; se os totais não baterem, é impossível. Custo O(área).
  - *Exemplo:* tabuleiro de xadrez com dois cantos opostos removidos não é coberto por dominós; L-triominó em tabuleiro 2×2.
  - *Armadilha:* a coloração dá uma condição necessária, não suficiente; peças que cobrem cores variáveis precisam de mais de uma coloração; bordas e rotações.
- **Contagem dupla.** Conte pares (a, b) de duas formas: Σ_v deg(v) = 2m; Σ_v C(deg(v), 2) = #caminhos de comprimento 2; Σ_S |S ∩ T|. Útil para limites (existe v com grau ≥ média) e para identidades.
  - *Gatilho:* uma identidade combinatória, ou um limite que envolve graus/interseções.
  - *Por que funciona:* contar o mesmo conjunto de pares por duas decomposições dá uma igualdade; a média surge ao dividir pelo número de objetos.
  - *Como fazer:* escolha o conjunto de pares; conte por vértice e por aresta; iguale; use para limitar ou construir. Custo O(n + m).
  - *Exemplo:* handshake: Σ deg = 2m; Σ C(deg,2) conta caminhos de 2 arestas.
  - *Armadilha:* ordem dos pares; caminhos de comprimento 2 têm um vértice central; laços e arestas múltiplas mudam a contagem.
- **Pareamento de somas e simetria (Gauss).** Σ_{i=0}^{n−1} f(i) + f(n−1−i) em pares; função simétrica f(x) = f(n − x) → metade dos casos. Gatilho: P(x) com simetria → dobre e use combinatória central.
  - *Gatilho:* soma em um intervalo com simetria, ou polinômio/coeficientes simétricos.
  - *Por que funciona:* parear i com n−1−i cobre todos os termos; se f é simétrica, os dois lados são iguais e a soma dobra; o termo central é tratado à parte.
  - *Como fazer:* separe o termo central; some os pares; use a simetria para reduzir a casos; para coeficientes, relacione a_k e a_{n−k}. Custo O(n/2).
  - *Exemplo:* Σ_{i=1}^{n} i = n(n+1)/2 por pareamento; coeficientes de (1+x)^n são simétricos.
  - *Armadilha:* n ímpar tem termo central; a simetria precisa ser exata; índices deslocados.

- **Soma sobre permutações.** Média de um estatístico sobre permutações = E[X] por linearidade e simetria: P(i antes de j) = 1/2; P(i é máximo entre k posições) = 1/k; E[#pares invertidos] = n(n−1)/4.
  - *Gatilho:* valor esperado de uma estatística em uma permutação aleatória uniforme.
  - *Por que funciona:* por simetria, cada elemento tem a mesma chance de ser o primeiro/maior em um subconjunto; a linearidade soma as probabilidades.
  - *Como fazer:* escreva a estatística como soma de indicadores; calcule a probabilidade de cada um por simetria; some. Custo O(n) ou O(1).
  - *Exemplo:* número esperado de inversões = C(n,2)/2 = n(n−1)/4; posição esperada de um elemento.
  - *Armadilha:* eventos dependentes não afetam a esperança, mas afetam a variância; empates não existem em permutações; a simetria exige distribuição uniforme.
- **Parcial → global por escada.** "Escada" de desigualdades $a_{i+1} \geq a_i + \delta$ implica $a_n \geq a_1 + (n-1)\delta$; use para limitar tamanhos (Fibonacci-like: valores crescem exponencialmente, logo ≤ log passos).
  - *Gatilho:* uma desigualdade local entre termos consecutivos e uma cota global.
  - *Por que funciona:* somar as desigualdades telescopa os termos intermediários; no caso Fibonacci, a recorrência faz o crescimento ser exponencial, então o número de passos é logarítmico.
  - *Como fazer:* escreva a desigualdade para cada i; some; use a cota; para crescimento exponencial, compare com uma Progressão geométrica/Fibonacci. Custo O(n) para somar.
  - *Exemplo:* $a_{i+1} \geq a_i + 2$ com $a_1 = 3 \to a_n \geq 2n+1$; Fibonacci cresce como φ^n.
  - *Armadilha:* δ pode ser negativo; a base da indução; uma cota local não implica crescimento global sem somar corretamente.

Otimização, programação linear e matroides

adicionado em 2026-11-10 por agentes; revisado

Quando o problema pede "o melhor de uma família enorme", pergunte se é LP/fluxo, matroide, submodular ou convexo — cada um tem um algoritmo exato.

- **Dualidade em LP.** max c^Tx s.t. $Ax \leq b$, $x \geq 0$ ↔ min b^Ty s.t. $A^Ty \geq c$, $y \geq 0$; valores iguais no ótimo; folga complementar: $x_j > 0 \Rightarrow$ restrição dual j justa. Gatilho: provar otimalidade com um certificado (corte = fluxo, cobertura = emparelhamento, potenciais = Hungarian).
  - *Gatilho:* "prove que este valor é ótimo", "dê um certificado", ou o problema tem uma formulação de max/min com restrições lineares e você suspeita de um par dual (fluxo/corte, emparelhamento/cobertura, atribuição/potenciais).
  - *Por que funciona:* dualidade fraca: para x primal viável e y dual viável, $c^Tx \leq (Ax)^Ty \leq b^Ty$, pois $y \geq 0$. Dualidade forte: se ambos os problemas são viáveis, os ótimos coincidem. Folga complementar: $x_j > 0 \Rightarrow (A^Ty)_j = c_j$ e $y_i > 0 \Rightarrow (Ax)_i = b_i$; os produtos de folgas primal/dual se anulam.
  - *Como fazer:* no formato max c^Tx, $Ax \leq b$, $x \geq 0$, o dual é min b^Ty, $A^Ty \geq c$, $y \geq 0$; cada restrição primal vira uma variável dual e cada variável primal vira uma restrição dual. Se o primal tem igualdade, a variável dual é livre; se tem variável livre, a restrição dual é igualdade. Para certificar, exiba y viável com b^Ty igual ao valor primal; para recuperar x, use folga complementar.
  - *Exemplo:* max flow = min cut (as variáveis duais são os potenciais do corte); König: emparelhamento máximo = cobertura mínima de vértices no bipartido; no Hungarian, $u_i + v_j \leq c_{ij}$ é o dual e as arestas do matching são justas.
  - *Armadilha:* sinais errados ao converter ≥/=/variáveis livres; dualidade forte exige viabilidade (para inviabilidade use Farkas); dualidade é do LP — integralidade exige TU ou argumento combinatório.
- **Fluxo e corte como LP.** Max flow = min cut; emparelhamento máximo = cobertura mínima (bipartido); matrizes totalmente unimodulares (incidência de grafo bipartido, intervalos consecutivos, redes) fazem o LP ter ótimo inteiro — então relaxação resolve o problema inteiro.
  - *Gatilho:* modelar max flow/min cut/emparelhamento/cobertura como LP e perguntar se a relaxação já dá o ótimo inteiro; "por que o fluxo é inteiro?".
  - *Por que funciona:* o LP de fluxo tem restrições de capacidade e conservação; a matriz de incidência nó-arco de um grafo dirigido é TU, então com b inteiro todo vértice do poliedro é inteiro e o ótimo do LP é uma solução inteira. O dual do max flow é o min cut (potenciais 0/1); König e Hall são o mesmo fenômeno no bipartido.
  - *Como fazer:* escreva o IP; verifique TU: incidência nó-arco dirigida é TU; incidência de grafo bipartido é TU; matriz de intervalos consecutivos é TU. Se for TU, resolva o LP (ou o algoritmo combinatório) e use a solução inteira. Se não for — a incidência não orientada de um grafo geral com ciclo ímpar tem determinante 2 — acrescente restrições de blossom/odd-set ou mude a formulação.
  - *Exemplo:* matching bipartido: o LP relaxado é integral; matching geral no triângulo tem solução fracionária $x = 1/2$ em cada aresta (valor 1,5) sem as restrições de odd-set; min-cost flow com capacidades inteiras devolve fluxo inteiro.
  - *Armadilha:* TU é propriedade da matriz exata e b precisa ser inteiro; transpor ou acrescentar restrições redundantes pode quebrar TU; integralidade garante que existe ótimo inteiro, não que todo ótimo do LP seja inteiro.

- **Simplex.** Para variáveis/restrições $\lesssim$ 100–300 em double: use Bland ou maior coeficiente com eps 1e-9; problemas de mistura, dieta, jogos de soma zero, max flow pequeno. Cuidado com degenerescência (ciclo) e inviabilidade inicial (fase 1).
  - *Gatilho:* LP pequeno (variáveis/restrições ~100–300), coeficientes em double, precisa do ótimo exato; mistura/dieta/jogo de soma zero/max flow pequeno.
  - *Por que funciona:* o simplex caminha entre soluções básicas viáveis (vértices) ao longo de arestas; um ótimo de LP é atingido num vértice. Bland ou desempate lexicográfico evita ciclagem em degenerescência; a fase 1 acha uma BFS ou prova inviabilidade.
  - *Como fazer:* forme max $c^T x$, $Ax = b$, $x \geq 0$ com folgas; fase 1 com artificiais; escolha a coluna de entrada (maior custo reduzido ou Bland) e faça o teste de razão; eps 1e-9; se nenhuma razão é positiva, o LP é ilimitado; os custos reduzidos das folgas no tableau final dão o dual.
  - *Exemplo:* dieta (min custo sujeito a nutrientes $\geq$ exigência), transporte pequeno, valor de jogo de soma zero; um LP 2D pode ser conferido por interseção de semiplanos.
  - *Armadilha:* degenerescência/ciclagem (use Bland ou lexicográfico; maior coeficiente sozinho pode ciclar); escala e eps; casos inviável/ilimitado; pior caso exponencial — ok para instâncias pequenas, não para 1e5.
- **Jogo de soma zero.** Valor do jogo matricial (estratégias mistas) = LP: max v s.t. $\Sigma_i p_i A_{ij} \geq v \ \forall j$, $\Sigma p_i = 1$; minimax de von Neumann. Antes: elimine linhas/colunas dominadas; 2×2 tem fórmula fechada.
  - *Gatilho:* matriz de payoffs, "estratégias mistas", valor do jogo, minimax; cada jogador escolhe probabilidades.
  - *Por que funciona:* von Neumann: $\max_p \min_j \Sigma_i p_i A_{ij} = \min_q \max_i \Sigma_j A_{ij} q_j = v$. O LP do jogador das linhas é max v s.t. $\Sigma_i p_i A_{ij} \geq v \ \forall j$, $\Sigma p_i = 1$, $p \geq 0$; o dual dá a estratégia q e o mesmo v.
  - *Como fazer:* resolva o LP (simplex) ou o dual; elimine linhas/colunas dominadas antes; para 2×2, $v = (a11 \cdot a22 - a12 \cdot a21)/(a11 + a22 - a12 - a21)$ e $p1 = (a22 - a21)/(a11 + a22 - a12 - a21)$, quando o denominador é $\neq 0$; para k×k, simplex.
  - *Exemplo:* matching pennies [[1, −1], [−1, 1]] → p = (1/2, 1/2), v = 0; pedra-papel-tesoura → uniforme, v = 0.
  - *Armadilha:* v pode ser negativo (não imponha $v \geq 0$); denominador zero no 2×2 (jogo degenerado, pode haver estratégia pura); dominação mista é mais delicada que dominação pura.
- **LP em poucas variáveis.** 2 variáveis e n restrições → interseção de semiplanos O(n log n); d fixo → Seidel aleatório O(d!·n). Gatilho: "maximize ax + by sujeito a n desigualdades".
  - *Gatilho:* "maximize ax + by sujeito a n desigualdades"; 2 variáveis; d fixo e pequeno.
  - *Por que funciona:* a região viável é a interseção de semiplanos, um polígono convexo; o ótimo de uma função linear está num vértice (ou numa aresta); em dimensão fixa, Seidel aleatório resolve em O(d!·n) esperado.
  - *Como fazer:* 2D: interseção de semiplanos O(n log n) (ordene por ângulo, deque) ou teste os $O(n^2)$ pares de retas e avalie os vértices viáveis; d fixo: Seidel (incremental aleatório) ou enumere d restrições ativas; verifique ilimitado por direção de recessão.
  - *Exemplo:* max 3x + 2y s.t. x + y ≤ 4, x ≤ 2, y ≤ 3, x, y ≥ 0 → vértices (0,0), (2,0), (2,2), (1,3), (0,3); ótimo (2,2), valor 10.
  - *Armadilha:* retas paralelas/duplicadas e eps na interseção; LP ilimitado; Seidel não é prático se d não é fixo.
- **Lema de Farkas.** $Ax = b$, $x \geq 0$ tem solução OU $\exists y$: $A^T y \geq 0$, $b^T y < 0$ (exatamente um). Em grafos: restrições de diferença inviáveis ⇔ ciclo negativo (o certificado é o ciclo); Hall ⇔ Farkas para emparelhamento.
  - *Gatilho:* precisa de certificado de inviabilidade; "exatamente um dos dois"; restrições de diferença; condições tipo Hall.
  - *Por que funciona:* Farkas: exatamente um de (I) $Ax = b$, $x \geq 0$ e (II) $\exists y$, $A^T y \geq 0$, $b^T y < 0$. (II) é um hiperplano que separa b do cone gerado pelas colunas de A; se (I) é inviável, esse separador existe.
  - *Como fazer:* para provar inviabilidade, exiba y; para restrições de diferença $x_v - x_u \leq c$, a inviabilidade é um ciclo negativo (some as desigualdades e obtenha 0 ≤ negativo); Hall: matching que satura a esquerda iff $|N(S)| \geq |S|$ para todo S; a falha dá o certificado.
  - *Exemplo:* $x \leq y - 1$ e $y \leq x - 1 \Rightarrow$ ciclo de peso −2, inviável; o ciclo é o certificado combinatório do Farkas.
  - *Armadilha:* variantes de sinal ($A^T y \leq 0$, $b^T y > 0$); igualdades vs desigualdades; um y com $b^T y = 0$ não é contradição.

- **Problema de atribuição (Hungarian).** $n \le 500$ ($O(n^3)$): potenciais $u_i + v_j \le c_{ij}$ são exatamente a dual. Para "atribuir k tarefas a pessoas com custo" também min-cost flow. Para $n \le 20$ DP em máscara.
  - *Gatilho:* matriz n×n de custos, matching perfeito min/max, "atribuir tarefas", $n \le 500$, ou k atribuições.
  - *Por que funciona:* o LP de atribuição é integral (matriz de permutação/TU); o dual é max $\Sigma u_i + \Sigma v_j$ s.t. $u_i + v_j \le c_{ij}$; folga complementar: arestas do matching são justas. O Hungarian mantém viabilidade dual e aumenta o matching por caminhos alternados.
  - *Como fazer:* potenciais u, v; para cada linha, ache caminho aumentante no grafo de igualdade ($u_i + v_j = c_{ij}$); atualize potenciais pelo menor slack; $O(n^3)$; retangular: complete com zeros ou use min-cost flow com k unidades; maximização: negue os custos ou troque o sentido do dual.
  - *Exemplo:* designar n pessoas a n tarefas com custo total mínimo; para $n \le 20$, DP[mask]; para "k tarefas", min-cost flow com capacidade 1 nas arestas de tarefa.
  - *Armadilha:* sinal de max/min; custos negativos; índices 0/1; $n = 500$ é ~1,25e8 operações, cuidado com a constante; para $n > 2000$, use estrutura esparsa/min-cost flow.
- **Transporte em linha e círculo.** Mover unidades em linha com custo $|\Delta|$: custo = $\Sigma_i |P_i|$ com P = prefixo da (oferta − demanda). Em círculo: escolha constante c minimizando $\Sigma|P_i - c| \to c$ = mediana dos P_i. Gatilho: "balancear pessoas/moedas em fila ou mesa redonda".
  - *Gatilho:* "balancear pessoas/moedas em fila ou mesa redonda"; custo proporcional à distância; oferta e demanda.
  - *Por que funciona:* na linha, o fluxo em cada aresta é forçado pelo prefixo $P_i = \Sigma_{k \le i}(a_k - b_k)$; o custo total é $\Sigma|P_i|$. No círculo, os fluxos diferem por uma circulação constante c, então o custo é $\Sigma|P_i - c|$, minimizado na mediana dos P_i.
  - *Como fazer:* calcule P; linha: some $|P_i|$; círculo: ordene P e escolha a mediana (qualquer valor entre os dois medianos dá o mesmo mínimo) e avalie; use 64 bits; se o custo por unidade varia, use mediana ponderada.
  - *Exemplo:* linha a = [1,0,0], b = [0,0,1] → P = [1,1,0], custo 2; no círculo, c = 1 dá |0| + |0| + |−1| = 1 (mover 1 unidade pela aresta que fecha o círculo).
  - *Armadilha:* P_i é o fluxo entre i e i+1; oferta total = demanda total; mediana com n par e platôs; somas em 64 bits.
- **Fluxo com custos convexos.** Aresta com custo convexo c(f) → substitua por arestas paralelas de capacidade 1 com custos marginais c(k) − c(k−1) crescentes (ou capacidades maiores por trechos). Custo ótimo como função do fluxo total F é convexo: dá "resposta para todo k" pelas sucessivas augmentações.
  - *Gatilho:* custo de enviar f unidades numa aresta é convexo (cresce cada vez mais rápido); "custo marginal crescente"; quer respostas para todo fluxo k.
  - *Por que funciona:* para c convexo, os incrementos c(k) − c(k−1) são não decrescentes; substituir a aresta por arcos paralelos unitários com esses custos lineariza o problema, e as sucessivas augmentações usam os incrementos em ordem. O custo mínimo como função do fluxo total é convexo.
  - *Como fazer:* divida a aresta em arcos unitários (ou blocos) com custo marginal; rode min-cost flow com potenciais; registre o custo após cada augmentação para responder todo k; se os incrementos podem ser negativos, sature/os inicialize com Bellman-Ford.
  - *Exemplo:* $c(f) = f^2$ → arcos de custo 1, 3, 5, ...; enviar 3 unidades custa 9. Com duas rotas, a primeira unidade usa o arco mais barato.
  - *Armadilha:* côncavo é diferente (custo fixo vira NP-difícil); o número de arcos = Σ capacidades pode explodir — use blocos; custos marginais negativos e ciclos negativos.
- **Min-cost flow para escalonamento.** Intervalos em k máquinas; "escolha subconjunto de intervalos com peso máximo sem sobrepor mais de k": linha com arestas i→i+1 (cap k, custo 0) + arco por intervalo (cap 1, custo −w) — fluxo k mínimo custo. Também "k caminhos disjuntos de custo mínimo".
  - *Gatilho:* intervalos em k máquinas; "escolha subconjunto de intervalos com peso máximo sem sobrepor mais de k"; k caminhos disjuntos de custo mínimo.
  - *Por que funciona:* cada unidade de fluxo na linha é uma máquina; aresta i→i+1 com capacidade k; arco do intervalo l→r com capacidade 1 e custo −w deixa a máquina "pular" o intervalo e ganhar w; enviar k unidades minimiza o custo, que é −peso escolhido. A integralidade do min-cost flow garante seleção 0/1.
  - *Como fazer:* comprima coordenadas; nós 1..m; arestas i→i+1 cap k custo 0; para cada intervalo [l,r] adicione l→r cap 1 custo −w; envie k unidades de 1 a m (fonte/sink se preciso); resposta = −custo. Para k caminhos disjuntos, use cap 1 nas arestas de linha.
  - *Exemplo:* intervalos [1,3] w = 5 e [2,4] w = 6, k = 1 → escolhe o de peso 6; k = 2 → ambos (11). O fluxo ou usa o arco ou segue pela linha.
  - *Armadilha:* convenção [l,r] vs [l,r); a capacidade da linha deve ser k, não 1; custos negativos precisam de potenciais; k maior que a sobreposição máxima não muda a resposta; 64 bits.

- **Fluxo com limites inferiores.** Aresta com low ≤ f ≤ high: cap = high − low e excessos em nós; adicione S′, T′ e aresta T→S infinita; viável iff fluxo S′→T′ satura. Circulação viável e max-flow com lower bounds.
  - *Gatilho:* aresta com mínimo e máximo; "pelo menos X unidades"; demandas; circulação viável; max flow com lower bounds.
  - *Por que funciona:* substitua f = low + f′, 0 ≤ f′ ≤ high − low; o balanço de cada nó ganha b_v = Σ low_in − Σ low_out; as arestas do super fonte/sumidouro cobrem esse excesso; a viabilidade é equivalente a saturar todas as arestas de S′. Para s-t, acrescente T→S infinito e converta em circulação.
  - *Como fazer:* para u→v [low,high], adicione capacidade high − low e atualize demand[u] −= low, demand[v] += low (ou o inverso, conforme a convenção); se demand[v] > 0, S′→v cap demand[v]; se < 0, v→T′ cap −demand[v]; adicione T→S INF para s-t; rode max flow S′→T′; viável iff o fluxo satura Σ demandas positivas; fluxo real = low + residual (+ fluxo em T→S).
  - *Exemplo:* S→a [2,5], a→T [1,3], S→T [0,10]: há fluxo viável; os lower bounds forçam pelo menos 2 por a.
  - *Armadilha:* sinal de demand (in−out vs out−in); esquecer T→S; checar a soma de todas as arestas de S′; extrair o fluxo real; arestas múltiplas.
- **Custo negativo e ciclos.** Para min-cost com ciclos negativos: sature as arestas negativas (empurre fluxo, corrija demanda) ou use Bellman-Ford/SPFA com potenciais iniciais; no DAG basta DP para potenciais iniciais.
  - *Gatilho:* min-cost flow com custos negativos; ciclos negativos; rede em DAG; "o custo pode ser negativo".
  - *Por que funciona:* o SSP com potenciais exige custo reduzido ≥ 0; se as arestas iniciais têm custo negativo, os potenciais iniciais vêm de Bellman-Ford (ou DP topológica no DAG). Um ciclo negativo viável torna o custo ilimitado se circulações são permitidas; saturar as arestas negativas é uma redução padrão para eliminar isso.
  - *Como fazer:* DAG: potenciais por DP topológica. Geral: Bellman-Ford/SPFA uma vez para h; depois Dijkstra com c + h[u] − h[v] ≥ 0. Para ciclos negativos: sature cada aresta negativa, ajuste demandas/balanços e cancele ciclos negativos restantes; se o modelo não permite circulação, o problema pode ser inviável/ilimitado.
  - *Exemplo:* a→b custo −2 e b→a custo 1 formam ciclo −1; se circulação é permitida, o custo não tem mínimo; sature a→b ou proíba o ciclo.
  - *Armadilha:* não confunda "fluxo de valor fixo" com "circulação": ciclos negativos fora do caminho s-t ainda podem reduzir o custo se circulação for permitida; potenciais têm de ser viáveis em todas as arestas residuais; SPFA pode degradar.
- **Dinic em redes especiais.** Capacidades unitárias: O(E·min(V^{2/3}, E^{1/2})); bipartido unitário: Hopcroft–Karp O(E√V). Gatilho: "n = 10^5 emparelhamento bipartido" → HK, não Kuhn simples.
  - *Gatilho:* capacidades unitárias; matching bipartido com n = 1e5; caminhos disjuntos com cap 1; "rede unitária".
  - *Por que funciona:* Dinic trabalha em fases de grafo de níveis; em redes unitárias o número de fases é O(min(V^{2/3}, E^{1/2})), dando O(E·min(V^{2/3}, E^{1/2})); no bipartido unitário isso é exatamente Hopcroft–Karp O(E√V).
  - *Como fazer:* BFS de níveis; DFS de blocking flow com arco atual; no bipartido, HK: BFS dos livres à esquerda, DFS aumentante em camadas; pare quando o sink é inalcançável. Com cap 1, cada DFS satura a aresta.
  - *Exemplo:* 1e5 à esquerda, 1e5 à direita, 2e5 arestas: HK ~ 9e7; Kuhn O(VE) não passa.
  - *Armadilha:* capacidades > 1 mudam o bound; Dinic geral é O(V^2E); recursão profunda; resetar level/ptr por fase.
- **Fecho máximo (project selection).** max Σ w(S) com S fechado por dependências → fonte→v (w > 0), v→sink (−w), v→u de ∞; resposta = Σ positivos − min cut. Densidade máxima (Goldberg): binária em λ + fluxo.
  - *Gatilho:* projetos com lucro e pré-requisitos; "se escolher v, escolha u"; fecho máximo; densidade máxima.
  - *Por que funciona:* o fecho é uma implicação; o corte codifica penalidades: lado da fonte = escolhidos; aresta ∞ v→u força o fecho; pesos positivos fonte→v, negativos v→sumidouro; o custo do corte é Σ positivos − peso(S), então o min cut maximiza peso(S).
  - *Como fazer:* S→v cap w se w > 0; v→T cap −w se w < 0; v→u cap ∞ para cada dependência; resposta = Σ_{w>0} w − mincut; recupere S pelo lado da fonte. Densidade máxima: binária em λ, pesos w_e − λ, use fecho máximo/fluxo paramétrico (Goldberg).
  - *Exemplo:* projeto A lucro 5 exige B custo 3: escolher ambos dá 2, nenhum dá 0; grafo S→A 5, B→T 3, A→B ∞; min cut 3, resposta 5 − 3 = 2.
  - *Armadilha:* direção do ∞ (v→u para "v exige u"); resposta pode ser 0 (conjunto vazio); densidade precisa de denominador e precisão em λ; 64 bits.

- **Rotulagem convexa (Ishikawa / min-cost tension).** min Σ f_i(x_i) + Σ w_ij·g(x_i − x_j) com f_i convexas, g convexa, w_ij ≥ 0 e rótulos ordenados x_i ∈ {0, …, k−1} → corte mínimo em camadas (cada x_i em k níveis) ou dual de min-cost flow. Caso g = |·|: a dual é fluxo de custo mínimo (potenciais).
  - *Gatilho:* escolher rótulo x_i em {0, …, k−1} ordenado; custos unários convexos; custo de par w_ij·g(x_i − x_j) com g convexa e w ≥ 0; suavização/visão.
  - *Por que funciona:* g convexa torna o termo de par submodular na cadeia de rótulos; a construção de Ishikawa usa um nó por (i, nível) e arestas de capacidade infinita para representar o corte como limiar; um corte corresponde a uma rotulagem e sua capacidade é a energia. O dual é o min-cost tension (fluxo de custo mínimo com potenciais).
  - *Como fazer:* k−1 nós de limiar por variável; arestas para incrementos unários f_i(t) − f_i(t−1) (desloque se negativos); para cada par e nível, arestas que codificam os incrementos de g com capacidade w; min s-t cut. Para g = |·|, o dual é min-cost flow com potenciais.
  - *Exemplo:* k = 2 reduz ao caso binário submodular; k = 3 com f_i convexa e g(t) = |t| é um problema de suavização com 3 rótulos resolvido por um corte mínimo.
  - *Armadilha:* g precisa ser convexa; a ordem dos rótulos precisa ter significado; capacidades ∞ e 64 bits; grafo O(nk + mk); incrementos unários negativos exigem deslocamento.
- **Binário submodular.** Termo θ(x_i, x_j) binário é representável por corte iff θ(0,1) + θ(1,0) ≥ θ(0,0) + θ(1,1) (submodular); tudo o que não é, precisa inverter alguns rótulos (grafos bipartidos). Gatilho: "escolher 0/1 por vértice com custos por par".
  - *Gatilho:* variáveis binárias; custos por par θ_ij(0/1); "escolher 0/1 por vértice com custos por par"; minimizar energia.
  - *Por que funciona:* um termo binário de par é representável por corte s-t iff θ(0,1) + θ(1,0) ≥ θ(0,0) + θ(1,1) (submodular). O corte codifica cada atribuição e o min cut minimiza a energia. Se for supermodular, complemente uma variável (inverta o rótulo); isso é possível globalmente iff o grafo de termos supermodulares é bipartido.
  - *Como fazer:* para cada termo, adicione arestas S→i, i→T e i→j com capacidades derivadas da tabela θ (construção padrão); min cut; recupere os rótulos pelo lado da fonte. Para supermodular, inverta variáveis de um dos lados da bipartição; se houver ciclo ímpar de termos supermodulares, a energia não é representável por corte.
  - *Exemplo:* θ(0,0) = θ(1,1) = 1 e θ(0,1) = θ(1,0) = 0 (recompensa discordância): 0 ≥ 2 é falso → supermodular; inverta uma variável para obter um termo submodular. Penalidade de discordância θ(0,0) = 0, θ(0,1) = θ(1,0) = 1, θ(1,1) = 0: 2 ≥ 0 → submodular.
  - *Armadilha:* direção da desigualdade; termos constantes; multi-rótulo precisa de Ishikawa; inverter variável muda os custos unários.
- **Funções submodulares.** f(A) + f(B) ≥ f(A ∪ B) + f(A ∩ B) (retornos decrescentes). Exemplos: cobertura, cortes, rank de matroide, concavidade de |A|. Maximização monótona com cardinalidade k: guloso dá (1 − 1/e); minimização é polinomial para n arbitrário (Lovász/Fujishige); os algoritmos genéricos é que só são viáveis na prática para instâncias pequenas.
  - *Gatilho:* objetivo sobre conjuntos com retornos decrescentes; cobertura, cortes, rank; "escolher k conjuntos para maximizar cobertura".
  - *Por que funciona:* f(A) + f(B) ≥ f(A ∪ B) + f(A ∩ B); equivalentemente, o ganho marginal f(A ∪ {e}) − f(A) decresce com A. Para maximização monótona com cardinalidade k, o guloso dá (1 − 1/e)·OPT (Nemhauser–Wolsey). Minimização irrestrita é polinomial (Lovász/Fujishige–Wolfe), mas com algoritmos de grau alto; implementações genéricas só são práticas para instâncias pequenas.
  - *Como fazer:* maximização: guloso pelo maior ganho marginal, com avaliação preguiçosa (heap) para acelerar; minimização: extensão de Lovász/algoritmos de minimização, ou redução a min cut se f for função de corte; para n pequeno, brute force/ILP.
  - *Exemplo:* cobertura máxima: conjuntos cobrem elementos; o guloso escolhe o que cobre mais novos; garantia 1 − 1/e ≈ 0,632. A função corte f(S) = δ(S) é submodular.
  - *Armadilha:* guloso não é exato; a garantia é tight; não-monótona ou com restrição de matroide muda a garantia; minimização é polinomial mas não trivial — não use guloso e chame de ótimo.

- **Guloso em matroide.** Se os conjuntos viáveis formam matroide, ordenar por peso e adicionar quando mantém independência dá ótimo (inclusive "menor peso" e "maior peso"). Gatilho: "escolher itens com restrição de independência (sem ciclo, ≤ c por cor, linearmente independentes)".
  - *Gatilho:* escolher itens com restrição de independência (sem ciclo, ≤ c por cor, linearmente independentes); maximizar/minimizar peso.
  - *Por que funciona:* axioma de troca: se I, J independentes e |I| < |J|, existe e ∈ J\I com I ∪ {e} independente. Isso implica que o guloso (ordene por peso, adicione se independente) dá uma base de peso máximo (e, na ordem inversa, de peso mínimo); a prova é por troca.
  - *Como fazer:* ordene por peso (decrescente para máximo, crescente para mínimo); mantenha a independência com DSU (gráfico), base linear (linear), contadores por grupo (partição), oráculo de matching (transversal); adicione se independente; pare no rank. O(m log m · oráculo).
  - *Exemplo:* Kruskal = guloso no matroide gráfico; base xor máxima = guloso no matroide linear; jobs unitários com deadline = matroide de escalonamento.
  - *Armadilha:* os conjuntos viáveis precisam do axioma de troca, não só ser hereditários (matching não é matroide); para base de peso mínimo use ordem crescente; empates são livres; interseção de matroides não é guloso.
- **Matroides do dia a dia.** Gráfica (florestas) → Kruskal; linear (vetores independentes, base xor) → insira por peso decrescente numa base linear; uniforme (≤ k itens); de partição (≤ c_i do grupo i); transversal (emparelhável); de escalonamento (jobs unitários com deadline, via DSU/heap).
  - *Gatilho:* reconhecer uma das famílias padrão; "sem ciclo", "≤ c por cor", "linearmente independentes", "emparelhável", "jobs com deadline".
  - *Por que funciona:* cada família listada satisfaz os axiomas de matroide, então o guloso é exato; reconhecer a estrutura diz qual oráculo/DSU usar.
  - *Como fazer:* gráfico → DSU/Kruskal; linear → eliminação gaussiana/base xor, insira em ordem de peso; uniforme → pegue os k maiores; partição → contador por grupo com cap c_i; transversal → matching bipartido como oráculo; escalonamento → DSU "último slot livre" ou heap, ordenado por lucro/deadline. O teste de independência é o gargalo.
  - *Exemplo:* "máximo de arestas sem ciclo" = floresta geradora; "máximo de vetores xor-independentes" = base linear; "máximo de jobs unitários antes do deadline" = matroide de escalonamento.
  - *Armadilha:* "≤ c por cor" com grupos sobrepostos não é matroide de partição; arestas dirigidas pedem arborescência (interseção); o oráculo transversal é um matching por passo — pode ser lento.
- **Escalonamento com prazos.** Jobs unitários com deadline e lucro: ordene por lucro e coloque no último slot livre (DSU) — é matroide. Moore–Hodgson: minimizar nº de atrasados com tempos p_i e prazos d_i: ordene por prazo, mantenha heap de p, remova o maior quando estoura. Smith: min Σ w_iC_i → ordene por p_i/w_i; EDD: min lateness máxima.
  - *Gatilho:* jobs unitários com deadline e lucro; minimizar nº de atrasados; minimizar Σ w_iC_i; minimizar lateness máxima.
  - *Por que funciona:* jobs unitários com deadlines formam um matroide (independente = escalonável), então ordenar por lucro e colocar no último slot livre é ótimo (DSU/heap). Moore–Hodgson: EDD minimiza lateness máxima; quando o tempo total excede um deadline, remover o job mais longo é ótimo para minimizar o número de atrasados. Smith: troca dá p_i/w_i crescente para 1||ΣwC.
  - *Como fazer:* (a) lucro/deadline: ordene por lucro decrescente; DSU sobre slots acha o último livre ≤ d_i; se não há, pule. (b) Moore–Hodgson: ordene por d_i; max-heap de p_i; quando o tempo > d_i, remova o maior p; conte remoções. (c) Smith: ordene por p_i/w_i; EDD para L_max.
  - *Exemplo:* deadline 2 com 3 jobs unitários → no máximo 2 escolhidos; o DSU encontra os slots 1 e 2. Moore–Hodgson remove o mais longo quando estoura.
  - *Armadilha:* Moore–Hodgson minimiza o número de atrasados, não ΣwU (esse é NP-difícil); preempção muda o problema; release times; empates.

- **Interseção de matroides.** Máximo conjunto comum independente por caminhos aumentantes no grafo de trocas ($O(r^2n)$ consultas de oráculo; com pesos: Bellman-Ford mínimo em arestas). Reduz: emparelhamento bipartido (partição ∩ partição), árvore geradora arco-íris (gráfica ∩ partição por cor), arborescência (gráfica ∩ in-grau ≤ 1), "floresta + emparelhamento".
  - *Gatilho:* máximo conjunto independente em dois matroides; matching bipartido, árvore geradora arco-íris, arborescência, "floresta + emparelhamento".
  - *Por que funciona:* o grafo de trocas sobre o conjunto comum independente tem caminho aumentante iff o conjunto pode crescer; caminhos mínimos dão um máximo. Na versão com pesos, os pesos das trocas e um caminho mínimo (Bellman–Ford) mantêm a otimalidade.
  - *Como fazer:* mantenha I comum; construa o grafo de trocas com arcos válidos em M1/M2; BFS de elementos adicionáveis em M1 até adicionáveis em M2; aumente ao longo do caminho; repita ≤ r vezes. Com pesos: Bellman–Ford com custos de troca e potenciais. $O(r^2n)$ consultas de oráculo (sem pesos).
  - *Exemplo:* matching bipartido = partição ∩ partição; árvore arco-íris = gráfica ∩ partição (≤ 1 por cor); arborescência = gráfica ∩ in-degree ≤ 1.
  - *Armadilha:* o grafo de trocas precisa das propriedades dos dois matroides; o caminho pode ter várias trocas; a versão com pesos exige potenciais consistentes; consultas de oráculo dominam.
- **União de matroides.** Particionar arestas em k florestas (arboricidade) = união de k matroides gráficas = interseção via transformação; max conjunto que cabe em k florestas. Gatilho: "dividir em k grupos sem ciclo".
  - *Gatilho:* particionar arestas em k florestas (arboricidade); "dividir em k grupos sem ciclo"; máximo conjunto que cabe em k florestas.
  - *Por que funciona:* a união de k matroides é um matroide; um conjunto é independente iff pode ser particionado em k independentes, um por matroide. A união tem algoritmo de aumento polinomial (parecido com interseção) e reduz-se à interseção por uma transformação padrão.
  - *Como fazer:* construa k cópias do conjunto; ache o máximo particionável em k florestas por aumento de união de matroides (ou reduza à interseção com um matroide de partição nas cópias); Nash-Williams: arboricidade = $\max_{S,|S|\geq 2} \lceil |E(S)|/(|S|-1) \rceil$; para k pequeno, guloso/DP pode bastar.
  - *Exemplo:* grafo com arboricidade 2 se parte em 2 florestas; K_5 tem $|E| = 10$, $|V| = 5 \rightarrow \lceil 10/4 \rceil = 3$ florestas.
  - *Armadilha:* união de matroides é matroide, mas a união de duas gráficas não é gráfica; "k florestas" conta cada aresta uma vez; a fórmula da arboricidade é um máximo sobre subconjuntos, não só o grafo todo.
- **Árvore geradora com restrição de cor.** Exatamente k arestas vermelhas: o custo é convexo em k (matroide) → WQS (adicione λ ao peso das vermelhas) + Kruskal com desempate; ou matroid intersection / interpolação Kirchhoff para contagem.
  - *Gatilho:* árvore geradora com exatamente k arestas vermelhas; "≤ k"; peso mínimo/máximo; restrição de cor.
  - *Por que funciona:* o peso mínimo de uma árvore geradora com exatamente k arestas vermelhas é uma sequência convexa em k (troca de bases/matroide valuado). WQS: some λ às arestas vermelhas, rode Kruskal; conforme λ cresce, o número de vermelhas no ótimo decresce monotonicamente; a resposta em k é $g(\lambda) - \lambda k$. Alternativa: interseção de matroides (gráfica ∩ partição com contagem exata) ou interpolação de Kirchhoff para contagem.
  - *Como fazer:* binária em λ inteiro (ou real); Kruskal ordenando por w + λ para vermelhas e w para azuis, com desempate que favorece (ou evita) vermelhas para obter o mínimo/máximo de vermelhas; registre (custo, contagem); pare quando cruzar k; resposta = custo − λk. Para contar, use Matrix-Tree com variável por cor e interpole.
  - *Exemplo:* exatamente k arestas vermelhas na MST; λ = 0 dá a MST sem restrição; λ grande evita vermelhas; o casco convexo de (k, f(k)) é traçado.
  - *Armadilha:* f(k) pode não existir para algum k (nenhuma árvore com exatamente k vermelhas); platôs exigem guardar mínimo e máximo de contagem; desempate; faixa de λ e 64 bits.
- **Troca de base.** Em matroide, para bases B_1, B_2 e $x \in B_1 \setminus B_2$ existe $y \in B_2 \setminus B_1$ com $B_1 - x + y$ base. Gatilhos: segunda melhor MST (trocar uma aresta), "melhorar ótimo trocando 1 elemento", árvore de mínima mudança.
  - *Gatilho:* segunda melhor MST; "melhorar o ótimo trocando 1 elemento"; árvore de mínima mudança; análise de sensibilidade.
  - *Por que funciona:* axioma de troca de bases: para bases B1, B2 e $x \in B1\backslash B2$, existe $y \in B2\backslash B1$ com $B1 - x + y$ base. Assim, qualquer base é alcançável por trocas simples, e o melhor vizinho de uma troca é achado no circuito fundamental.
  - *Como fazer:* segunda melhor MST: para cada aresta não-árvore e = (u,v), a melhor aresta a remover é a de peso máximo no caminho u–v (binary lifting); candidato = MST − max + e; pegue o mínimo. Em matroides gerais, teste trocas com o oráculo; para "mínima mudança", maximize $|B \cap B0|$.
  - *Exemplo:* adicionar uma aresta não-árvore cria um ciclo; remover a aresta mais pesada desse ciclo dá a melhor árvore que contém e.
  - *Armadilha:* a segunda melhor pode ter o mesmo peso (empates); a aresta trocada precisa estar no circuito fundamental; em matroides, B1 − x + y pode exigir um y específico, não qualquer um.

- **Lagrangiana / WQS (Aliens).** Se f(k) é convexa, otimize f(k) + λk por λ real/inteiro; busca binária em λ, resposta = g(λ) − λ·k_alvo; desempate consistente (menor ou maior k) e cuidado com platôs: use λ inteiro se f inteiro e guarde "contagem mínima". Aplicações: k intervalos disjuntos, k árvores/ciclos, k grupos.
  - *Gatilho:* "exatamente k" grupos/intervalos/árvores; f(k) convexa (ou côncava); k até n; o DP teria uma dimensão k.
  - *Por que funciona:* se f(k) é convexa, o casco convexo inferior é traçado minimizando f(k) + λk; o k ótimo é monotônico em λ. O dual lagrangiano recupera f(k) no k alvo se ele está no casco; platôs significam vários k com o mesmo λ. A resposta é g(λ) − λk.
  - *Como fazer:* binária em λ (inteiro se f é inteiro); resolva o problema sem restrição com penalidade λ por grupo; guarde (valor, contagem) com desempate consistente (mínimo ou máximo de contagem); quando a contagem cruzar k, resposta = valor − λk. Para maximização com f côncava, use f(k) − λk e some λk de volta.
  - *Exemplo:* particionar o array em exatamente k subarrays minimizando a soma de custos; k intervalos disjuntos de peso máximo; k árvores num grafo.
  - *Armadilha:* f precisa ser convexa/côncava — verifique com brute force; platôs e desempate podem dar o k errado; λ real precisa de eps; o valor recuperado pode errar se o casco não é justo.
- **Convexidade comprovada.** f(k) é convexa se vem de fluxo de custo mínimo ou de matroide ponderado, ou se a matriz de custo é Monge; sem uma dessas provas, teste numericamente com brute force pequeno antes de apostar em Aliens.
  - *Gatilho:* vai aplicar Aliens/WQS ou merge convexo; precisa justificar a convexidade de f(k).
  - *Por que funciona:* o valor do min-cost flow como função do fluxo é convexo (custos marginais não decrescentes); o peso de uma base de matroide ponderado como função de $|B \cap S|$ é M-convexo, logo convexo; custos Monge dão opt monótono e f(k) convexa. Sem uma prova estrutural, um brute force pequeno pode falsificar a convexidade.
  - *Como fazer:* prove por uma das rotas estruturais; caso contrário, calcule f(k) para k = 1..n em instâncias pequenas aleatórias e cheque as segundas diferenças ≥ 0 (convexa) ou ≤ 0 (côncava); se falhar, não use Aliens — use DP ou outra abordagem.
  - *Exemplo:* custos de subgrafo k-conexo; k caminhos disjuntos em grafo planar; DP de partição com custo Monge.
  - *Armadilha:* convexidade em instâncias pequenas pode falhar em n grande; convexidade inteira vs real; empates podem esconder não-convexidade; convenções de "Monge" diferem.
- **Min-plus de convexas = merge.** $(f \oplus g)(k) = \min_{i+j=k} f(i) + g(j)$ com f, g convexas: mescle as diferenças ordenadas (soma de Minkowski). Mochila onde cada grupo tem custo convexo em itens → guloso por inclinação (heap).
  - *Gatilho:* combinar dois grupos/filhos independentes, escolher o total k, custo é o mínimo sobre a divisão; as duas sequências de custo são convexas.
  - *Por que funciona:* a convolução min-plus de duas sequências convexas é convexa, e sua sequência de diferenças discretas é a intercalação ordenada das diferenças das duas (soma de Minkowski dos epígrafos). Assim, mesclar as inclinações dá todos os k em tempo linear.
  - *Como fazer:* represente cada sequência convexa pelas diferenças $d_i = f(i) - f(i-1)$; mescle as duas listas ordenadas (ou use heap para muitas sequências); as somas prefixo dão a convolução. Para mochila com custo convexo por grupo, pegue os k menores ganhos marginais globais (heap).
  - *Exemplo:* f = [0,3,7] (diferenças 3,4), g = [0,2,5] (2,3); mescla [2,3,3,4] → h = [0,2,5,8,12]; conferindo h(2) = min(0+5, 3+2, 7+0) = 5.
  - *Armadilha:* as diferenças precisam estar ordenadas (convexidade); para min-plus de côncavas, mescle na outra ordem; offsets; empates de inclinação.
- **DP em árvore convexa.** dp[v][k] convexa em k para tarefas do tipo "escolher k vértices com distância" → merge de filhos por Minkowski com small-to-large (heap) ou long-path decomposition: O(n log n) em vez de $O(n^2)$.
  - *Gatilho:* DP de árvore com dp[v][k] = "escolher k na subárvore"; merge de filhos; k até o tamanho da subárvore; $O(n^2)$ ingênuo.
  - *Por que funciona:* se cada dp[v] é convexa em k, o merge de filhos é uma convolução min-plus de convexas, que é merge de inclinações; small-to-large ou long-path decomposition mantém o custo total O(n log n) (cada elemento/inclinação é movido O(log n) vezes ou segue o caminho pesado).
  - *Como fazer:* guarde a sequência convexa como heap de inclinações (slope trick) ou vetor de diferenças; mescle heaps dos filhos small-to-large; adicione custos unários; em long-path, reuse o vetor do filho pesado e mescle os leves; resposta na raiz.
  - *Exemplo:* escolher k vértices numa árvore com custo convexo de subárvore; k centros com custo convexo; mochila de árvore com custos convexos.
  - *Armadilha:* a convexidade precisa valer; heap merge com offsets preguiçosos; memória O(n); a condição de "distância" pode destruir a convexidade — verifique.

- **Minimização de função convexa.** $\Sigma|x - a_i| \rightarrow$ mediana; $\Sigma(x - a_i)^2 \rightarrow$ média; $\Sigma w|x - a_i| \rightarrow$ mediana ponderada; $\max|x - a_i| \rightarrow$ ponto médio do intervalo; max_i f_i(x) com f_i convexas é convexa → ternária ou binária no subgradiente.
  - *Gatilho:* minimizar soma de valores absolutos/quadrados/máximo de convexas; localização em 1D; "ponto que minimiza a soma das distâncias".
  - *Por que funciona:* $\Sigma|x - a_i|$ é convexa e minimizada na mediana; $\Sigma(x - a_i)^2$ na média; $\Sigma w|x - a_i|$ na mediana ponderada; $\max|x - a_i|$ no ponto médio entre mínimo e máximo; o máximo de convexas é convexo, então ternária ou subgradiente funciona.
  - *Como fazer:* mediana: ordene (ponderada: ache o prefixo ≥ metade do peso total); média para quadrados (long double se preciso); máximo: ponto médio; para convexa geral em 1D, ternária em inteiros com cuidado nos platôs, ou binária no sinal do subgradiente. Para max_i f_i, ternária funciona se cada f_i é convexa.
  - *Exemplo:* pontos 1, 2, 10 → mediana 2, custo 9 + 0 + 8 = 17; média 13/3 minimiza a soma dos quadrados; pesos (1,3), (10,1) → mediana ponderada 1.
  - *Armadilha:* inteiro vs real; ternária em platôs inteiros pode perder o ótimo — use binária nas diferenças; overflow nos quadrados; pesos em 64 bits.
- **Ternária 2D.** Minimizar convexa g(x, y): min_x min_y g é convexa (minimização parcial preserva convexidade) → ternária aninhada $O(\log^2)$ avaliações; ou descida coordenada/gradiente/Newton em R^d suave. Conferir convexidade (Hessiana semi-definida) antes.
  - *Gatilho:* minimizar convexa de duas variáveis numa caixa/grid; otimização aninhada.
  - *Por que funciona:* se g é conjuntamente convexa, o mínimo parcial h(x) = min_y g(x,y) é convexo; cada fatia g(x,·) é convexa, então ternária aninhada converge para o mínimo global. Descida coordenada também funciona para convexa suave, mas pode ser lenta em problemas mal condicionados.
  - *Como fazer:* ternária externa em x; ternária interna em y (ou binária na derivada); $O(\log^2)$ avaliações; em grid inteiro, ternária inteira + checagem dos vizinhos; em dimensão alta e suave, gradiente/Newton.
  - *Exemplo:* $(x-1)^2 + (y-2)^2 + xy$ tem Hessiana [[2,1],[1,2]] positivo-definida → convexa; ternária aninhada acha o mínimo. g(x,y) = xy não é conjuntamente convexa (Hessiana indefinida), apesar de fatias lineares.
  - *Armadilha:* ternária aninhada exige convexidade conjunta, não só fatias convexas; precisão e número de avaliações; arredondamento inteiro.
- **Gradiente e hill climbing.** Para convexa suave em poucas dimensões: passo decrescente ("anel": tente 4–8 direções, reduza o passo pela metade se não melhora) converge em ~100 iterações; funções não convexas → vários inícios aleatórios.
  - *Gatilho:* otimização convexa/suave em poucas dimensões; sem forma fechada; precisa de resposta numérica dentro de eps.
  - *Por que funciona:* para uma função convexa, busca local com passo decrescente converge para o mínimo global; o método do "anel" tenta várias direções e divide o passo quando não melhora, aproximando uma busca de linha. Para não convexas, múltiplos inícios aleatórios exploram bacias.
  - *Como fazer:* comece num ponto viável; tente 4–8 direções (eixos + diagonais) com passo t; se nenhuma melhora, t /= 2; pare quando t < eps; ~100 iterações; para restrições, projete de volta ou use penalidades.
  - *Exemplo:* maximizar uma função suave de 2–3 parâmetros num problema de geometria; ajustar um custo convexo; calibrar uma razão.
  - *Armadilha:* sem garantia para não convexas; cronograma do passo; escala das coordenadas; precisão; custo de avaliação.
- **Otimização inteira pequena.** ≤ 3–5 variáveis limitadas → enumere as duas menores e resolva o resto por fórmula; ax + by = c com minimização → Euclides estendido + deslocamentos; "melhor inteiro perto do ótimo real" → teste $\lfloor x \rfloor$ e $\lceil x \rceil$.
  - *Gatilho:* 2–5 variáveis inteiras com limites pequenos; equação diofantina linear; "melhor inteiro perto do ótimo real".
  - *Por que funciona:* enumere os menores domínios e resolva o resto por fórmula/LP; para ax + by = c, a solução geral é x = x0 + (b/g)t, y = y0 − (a/g)t, e o objetivo é linear em t, então o ótimo está num extremo. Arredondar o ótimo contínuo funciona para uma variável unimodal, não em geral.
  - *Como fazer:* ordene as variáveis por tamanho de domínio; enumere todas menos as duas maiores; para o resto, resolva o LP/diofantina; para ax + by = c use Euclides estendido, depois desloque t para satisfazer os limites e otimize; para "perto do ótimo", teste piso e teto (e uma vizinhança pequena).
  - *Exemplo:* max 3x + 5y s.t. 2x + 3y ≤ 12, x, y ≥ 0 inteiros → x = 0, y = 4 dá 20; x = 3, y = 2 dá 19; x = 6, y = 0 dá 18; ótimo 20.
  - *Armadilha:* arredondar cada variável independentemente pode violar restrições; coeficientes negativos; overflow; o ótimo pode estar longe do contínuo em várias dimensões.

- **Mochila: LP e poucos fracionários.** Relaxação linear da mochila 0/1 tem no máximo 1 item fracionário (guloso por v/w); o ótimo inteiro não é "quase" o guloso — o guloso puro pode ser arbitrariamente ruim e max(guloso, melhor item) é só 1/2-aproximação; use branch & bound ou DP só ao redor do ponto de corte (O(n·W_max)).
  - *Gatilho:* mochila 0/1 com W e V grandes; quer um bound ou um algoritmo exato que explore o LP; "quase guloso".
  - *Por que funciona:* a relaxação linear ordena por v/w e toma um prefixo mais no máximo um item fracionário; o valor é um limite superior. O ótimo inteiro pode estar longe do prefixo guloso: o guloso puro pode ser arbitrariamente ruim, e max(guloso, melhor item) é uma 1/2-aproximação. Branch & bound usa o bound do LP para podar; o teorema de proximidade diz que um ótimo inteiro difere do guloso por uma quantidade limitada (ligada ao maior peso de item), então DP numa janela ao redor do ponto de corte é exato.
  - *Como fazer:* ordene por v/w; ache o item de corte b; bound LP = valor do prefixo + (capacidade restante)·(v_b/w_b). Exato: branch & bound (inclui/exclui b, poda pelo LP); ou DP de pesos numa janela de tamanho w_max ao redor do peso do guloso, O(n·w_max); reconstrua.
  - *Exemplo:* capacidade 10; itens (w = 6, v = 6) e dois (w = 5, v = 5): com um desempate que pega o de 6 primeiro, o guloso fica com 6; o ótimo pega os dois de 5 (10). O LP pega 6 + 4/5 de um de 5 = 10.
  - *Armadilha:* o bound do LP não é o ótimo inteiro; o guloso puro pode ser arbitrariamente ruim; max(guloso, melhor item) é só 1/2; a janela de proximidade precisa do tamanho certo e só é boa se w_max é pequeno.
- **Mochila por valor pequeno / peso pequeno.** Se peso ≤ W: DP [peso]; se valor total ≤ V: DP [valor] = menor peso; ambos grandes → meet in the middle. Itens com muitos pesos iguais: agrupe e use mochila limitada por deque monotônico (mod w) ou binária.
  - *Gatilho:* W ou V total pequeno (≤ 1e5–1e6), n grande; muitos pesos iguais.
  - *Por que funciona:* o DP por peso guarda o valor máximo; o DP por valor guarda o peso mínimo; ambos são exatos porque o estado captura a restrição. Se ambos são grandes, meet in the middle divide n. Muitas cópias iguais podem ser tratadas por binary splitting ou deque monotônico por resíduo.
  - *Como fazer:* DP de peso: dp[w] = max valor, itens em ordem decrescente de w; DP de valor: dp[v] = min peso, resposta = maior v com dp[v] ≤ W; limitada: binary split c em 1,2,4,... ou deque nos resíduos mod w; MITM para n ≤ 40: enumere as metades, ordene uma, busque na outra.
  - *Exemplo:* n = 100, W = 1e5, w_i ≤ 1e5 → DP de peso 1e7; V total 1e5 → DP de valor; n = 40, W = 1e9 → MITM 2^20.
  - *Armadilha:* o DP de valor usa a soma V, não o maior v; deque limitada trata exatamente c cópias; binary splitting adiciona log c itens; valores em 64 bits; memória do MITM.
- **Mochila 2D / com conflitos.** Conflitos formam floresta/árvore → DP em árvore; conflitos bipartidos de peso → corte mínimo.
  - *Gatilho:* duas capacidades (peso e volume); itens em conflito (não escolher ambos); conflitos em floresta/árvore ou bipartidos.
  - *Por que funciona:* com a segunda capacidade, adicione uma dimensão ao DP; se os conflitos formam uma floresta, faça DP em árvore sobre o grafo de conflitos, com o estado da mochila em cada nó; se o grafo de conflitos é bipartido e não há capacidade (ou a capacidade é tratada à parte), o máximo conjunto independente de peso é total − corte mínimo. Grafos de conflito gerais tornam o problema NP-difícil.
  - *Como fazer:* árvore: enraíze cada componente, dp[v][escolhido?][w] mesclando filhos (exclua o filho se v escolhido); bipartido: min vertex cover por max flow, resposta = peso total − mincut; com capacidade e conflitos, combine a capacidade no DP ou resolva por ILP/branch & bound.
  - *Exemplo:* itens com conflitos em caminho: conjunto independente de peso máximo por DP de árvore; conflitos bipartidos: jobs/máquinas.
  - *Armadilha:* conflitos em grafo geral são NP-difíceis; o DP de árvore precisa lembrar se o pai foi escolhido; o corte mínimo resolve a versão sem capacidade — com capacidade e conflitos, não basta.
- **Monge e SMAWK.** Custo Monge: w(a,c) + w(b,d) ≤ w(a,d) + w(b,c) ⇒ opt monótono: D&C DP O(n log n); Knuth em intervalos O(n^2); SMAWK mínimo de linhas O(n); LARSCH para DP 1D/1D online. Monge côncavo ⇒ f(k) convexa ⇒ Aliens.
  - *Gatilho:* DP dp[i] = min_{j<i} dp[j] + w(j,i) com desigualdade quadrangle; opt monótono; partição em k segmentos; mínimo de linhas de matriz.
  - *Por que funciona:* a condição de Monge w(a,c) + w(b,d) ≤ w(a,d) + w(b,c) para a ≤ b ≤ c ≤ d implica que o argmin é monótono em i; D&C optimization calcula uma linha em O(n log n) (ou O(n) com SMAWK para matriz totalmente monótona). Knuth para DP de intervalos exige a mesma desigualdade mais monotonicidade, O(n^2). SMAWK acha todos os mínimos de linha de uma matriz n×m totalmente monótona em O(n+m). LARSCH trata o caso 1D/1D online. Na partição em k blocos, a variante de Monge usada dá a convexidade discreta de f(k) que o Aliens exige.
  - *Como fazer:* verifique Monge no custo (ou nas segundas diferenças); D&C: resolva mid, busque opt em [optL,optR], recursão; Knuth: opt[i][j−1] ≤ opt[i][j] ≤ opt[i+1][j]; SMAWK: redução recursiva das linhas ímpares; LARSCH para online.
  - *Exemplo:* árvore binária de busca ótima (Knuth); particionar array em k segmentos com custo = soma ao quadrado (Monge); transporte com custo |i−j|.
  - *Armadilha:* Monge é suficiente, não necessária; verifique numericamente em casos pequenos; D&C exige faixa de opt válida; Knuth precisa da estrutura de intervalos; a convenção de "Monge côncavo" é inconsistente.

- **Alocação ótima com retornos decrescentes.** Distribuir k recursos entre itens com ganho marginal decrescente → heap pelo ganho marginal (guloso exato). Se os ganhos não são decrescentes → DP/mochila de grupos.
  - *Gatilho:* distribuir k recursos idênticos entre itens; cada item tem ganho com retornos marginais decrescentes; maximizar o ganho total.
  - *Por que funciona:* se os ganhos marginais g_i(1) ≥ g_i(2) ≥ ... são não crescentes, o problema é escolher k ganhos marginais da união com a restrição de prefixo por item; os k maiores automaticamente formam um prefixo, pois um ganho posterior nunca é maior que um anterior. Logo, o guloso com heap é exato.
  - *Como fazer:* empilhe o primeiro ganho marginal de cada item; repita k vezes: pop o maior, some, empilhe o próximo ganho daquele item; O((n+k) log n). Se os ganhos são arbitrários, use DP de grupos/mochila.
  - *Exemplo:* investir k unidades em projetos com retornos decrescentes; escolher k tarefas com lucros decrescentes; alocar orçamento de propaganda.
  - *Armadilha:* os retornos decrescentes precisam valer (concavidade); com custo fixo ou ganhos não monótonos o guloso falha; k grande, mas os ganhos podem ser agrupados.
- **Binária + guloso, binária no λ.** Min-max de partição em k blocos → binária no limite + guloso; razão Σa/Σb → Dinkelbach (converge em poucas iterações, mais rápido que binária) ou binária; fração ótima de rótulo → λ_{k+1} = Σa(S_k)/Σb(S_k).
  - *Gatilho:* min-max de partição em k blocos; maximizar a razão Σa/Σb; "fração ótima"; a viabilidade é monótona num limiar.
  - *Por que funciona:* se uma partição com bloco máximo ≤ X é viável, ela é viável para X maior (monotonicidade), então a binária funciona; o check guloso varre da esquerda para a direita e corta quando a soma passaria de X, o que é ótimo para minimizar o número de blocos. Para razões, Dinkelbach itera λ ← Σa(S)/Σb(S) e converge superlinearmente; Aliens/binária em λ resolve "exatamente k".
  - *Como fazer:* binária em X inteiro/real; conte blocos com o guloso; compare com k. Razão: binária em λ, cheque max Σ(a − λb) ≥ 0; Dinkelbach atualiza λ para a razão obtida. Para "exatamente k", use WQS com penalidade λ.
  - *Exemplo:* dividir um array em k partes contíguas minimizando a maior soma; escolher um subarray/intervalo de média máxima; maximizar (soma a)/(soma b) sobre partições.
  - *Armadilha:* a contagem gulosa precisa ser exata; iterações da binária real (100 para 1e-9); Dinkelbach precisa de solução inicial e pode ciclar com empates; a monotonicidade pode falhar com valores negativos.
- **Programação por restrições: 2-SAT / difference constraints.** Sistemas x_j − x_i ≤ c são LP dual de menor caminho; se há função objetivo linear Σ w_i x_i → min-cost flow dual. Com somas, gatilho "x_i ≤ x_j + c e minimize Σ".
  - *Gatilho:* implicações x→y, "se A então B", escolher uma de duas; restrições x_j − x_i ≤ c; minimizar uma função linear das diferenças.
  - *Por que funciona:* 2-SAT: cada cláusula (a ∨ b) dá implicações ¬a→b e ¬b→a; é satisfatível iff nenhuma variável e sua negação estão na mesma SCC (Tarjan/Kosaraju). Restrições de diferença: viáveis iff o grafo de restrições não tem ciclo negativo; os menores caminhos dão uma solução (potenciais). O dual do caminho mínimo é exatamente o sistema de diferenças; um objetivo linear leva ao dual de min-cost flow.
  - *Como fazer:* 2-SAT: grafo de implicações com 2n nós, SCC, atribua pela ordem topológica (ou compare comp); diferenças: Bellman-Ford de um super fonte, x_v = dist[v]; para inteiros e desigualdades estritas, escale por 2; para min Σw_i x_i, monte o dual de min-cost flow ou use LP.
  - *Exemplo:* 2-SAT: (x1 ∨ x2) ∧ (¬x1 ∨ x3); diferenças: x2 ≤ x1 + 3, x3 ≤ x2 − 1, x1 ≤ x3 + 0 → soma 3 − 1 + 0 = 2, sem ciclo negativo; se fosse −1, inviável.
  - *Armadilha:* cláusulas com o mesmo literal; direção da atribuição na SCC; desigualdades estritas precisam de escala; ciclos negativos; nós inalcançáveis.
- **Problema de transporte entre classes.** Distribuir mercadorias de depósitos a clientes com custo por par e capacidades → min-cost flow; grande se custo = |p_i − q_j| em linha (sorted matching) → greedy.
  - *Gatilho:* depósitos para clientes, ofertas/demandas, custo por par, capacidades; custo = |p_i − q_j| na linha.
  - *Por que funciona:* o min-cost flow na rede bipartida (fonte→depósitos com oferta, depósito→cliente com custo, cliente→sumidouro com demanda) dá o ótimo exato; a integralidade do fluxo garante entregas inteiras. Se o custo é distância absoluta na linha, o matching ótimo é order-preserving (uncrossing): se p_i ≤ p_j e q_k ≤ q_l, casar i–l e j–k custa mais que i–k e j–l, então ordene e case em ordem.
  - *Como fazer:* min-cost flow por caminhos mínimos sucessivos ou network simplex; para custo na linha, ordene os dois lados e case em ordem, usando prefixos/two pointers para capacidades; para 1D com ofertas/demandas, use a fórmula de prefixo do transporte.
  - *Exemplo:* depósitos em 0 e 10 com 5 unidades cada, clientes em 3 e 7 com 5 cada → casar 0→3 e 10→7 custa 3 + 3 = 6; cruzado custaria 7 + 7 = 14.
  - *Armadilha:* oferta ≠ demanda (dummy); capacidades nas rotas; matriz de custo não métrica; 64 bits.

- **Cobertura / hitting set pequeno.** Cobertura mínima de vértices com parâmetro k pequeno → FPT: ramifique (escolha u ou todos os vizinhos), $O(2^k \cdot m)$; em bipartido, exato por fluxo; árvore: guloso nas folhas.
  - *Gatilho:* cobertura de vértices com k pequeno; hitting set com parâmetro pequeno; "remover ≤ k vértices para...".
  - *Por que funciona:* árvore de busca limitada: pegue uma aresta (u,v) não coberta; qualquer cobertura contém u ou v, então ramifique duas vezes e decremente k; profundidade k, ramificação 2 → O(2^k·m). Para hitting set com conjuntos de tamanho d, ramifique nos d elementos de um conjunto não coberto → O(d^k). Cobertura bipartida é polinomial via matching (König); árvore por guloso nas folhas.
  - *Como fazer:* ramificação recursiva com k; kernelize (remova isolados, regra de grau 1, regra de grau alto); para bipartido, max flow/matching + caminhos alternados; para árvore, enraíze e cubra arestas nas folhas.
  - *Exemplo:* triângulo com k = 2: escolha uma aresta, ramifique; as duas ramificações cobrem; k = 1 é impossível.
  - *Armadilha:* o parâmetro precisa ser o tamanho da solução; cobertura de vértices em grafo geral é NP-difícil para k grande; a regra de ramificação precisa cobrir todas as soluções; kernelização é necessária para velocidade.
- **Corte mínimo global.** Stoer–Wagner $O(n^3)$ / $O(nm + n^2 \log n)$; Karger aleatório; k-corte de arestas em grafo de poucos vértices; árvore de Gomory–Hu para todos os pares (n − 1 fluxos). Gatilho: "corte mínimo entre todos os pares" ou "mínimo corte global".
  - *Gatilho:* corte mínimo entre todos os pares; "dividir o grafo em duas partes minimizando o corte"; confiabilidade.
  - *Por que funciona:* Stoer–Wagner contrai repetidamente a aresta mais pesada para achar o corte mínimo global em grafos não dirigidos; um corte mínimo é um corte s-t mínimo para algum par, e o algoritmo mantém o corte da fase. Gomory–Hu constrói uma árvore que representa todos os cortes mínimos entre pares com n − 1 max flows. Karger contrai arestas aleatórias e acha um corte mínimo com probabilidade ≥ 1/C(n,2), repetindo.
  - *Como fazer:* Stoer–Wagner $O(n^3)$ com matriz de adjacência, ou $O(nm + n^2 \log n)$ com heaps; Gomory–Hu: para cada par na árvore, rode max flow e divida; Karger: contraia até 2 vértices, repita $O(n^2 \log n)$ vezes.
  - *Exemplo:* corte mínimo global de um grafo com dois cliques ligados por uma aresta → essa aresta; grafos dirigidos precisam de outro algoritmo (Stoer–Wagner é não dirigido).
  - *Armadilha:* dirigido vs não dirigido; grafo desconexo (corte 0); multigrafo; detalhes da árvore de Gomory–Hu; probabilidade de Karger.
- **LP inteiro e arredondamento.** Se o LP é totalmente unimodular, ótimo inteiro; se não, arredondamento aleatório dá aproximação (e.g., cobertura de vértices por 2-aproximação; 1 − 1/e em cobertura máxima). Em ICPC, só se o enunciado pede aproximação; caso contrário procure estrutura especial.
  - *Gatilho:* IP/ILP com relaxação LP natural; o enunciado pede aproximação; checar TU.
  - *Por que funciona:* se a matriz de restrições é TU e b é inteiro, os vértices do LP são inteiros, então resolver o LP resolve o IP. Caso contrário, arredondar a solução do LP pode dar aproximação: para cobertura de vértices, o limiar $x_v \geq 1/2$ é 2-aproximação e é viável; para cobertura máxima, o guloso (ou arredondamento aleatório com reparo) dá 1 − 1/e; Chernoff controla o custo. Em ICPC, só use aproximação se o enunciado pedir; caso contrário, procure estrutura especial/exponencial exato.
  - *Como fazer:* verifique TU (matriz de rede, matriz de intervalos, incidência bipartida); se não for, resolva o LP (simplex), arredonde e prove viabilidade/razão; para cobertura de vértices, pegue $x_v \geq 1/2$; para cobertura máxima, o guloso é mais simples e dá 1 − 1/e; para set cover, o guloso dá H_n.
  - *Exemplo:* matching bipartido LP é integral; arredondamento do LP de cobertura de vértices; guloso de cobertura máxima.
  - *Armadilha:* arredondar pode violar restrições; a garantia depende do arredondamento; TU é suficiente, não necessária; não submeta aproximação quando o juiz espera exato.

Amostras de problemas e padrões de ICPC

adicionado em 2026-11-10 por agentes; revisado

ICPC raramente cobra uma técnica sozinha: o problema médio combina 2–3 estruturas conhecidas, então aprenda os pares que se encaixam.

- **Binária + fluxo.** "Minimizar a carga máxima / maximizar o mínimo" em atribuição ou transporte → binária no limite e checagem por max flow; minimizar a carga máxima usa capacidade = limite, mas maximizar o mínimo exige lower bounds/demandas (circulação com limites inferiores); o grafo é sempre o mesmo, só mudam capacidades e limites.
  - *Gatilho:* "minimizar a carga máxima", "maximizar o mínimo", atribuição/transporte com limite; decidir se um limite X é viável.
  - *Por que funciona:* a viabilidade é monótona em X; fixado X, o problema vira fluxo com capacidades (para o máximo) ou com lower bounds (para o mínimo).
  - *Como fazer:* binária em X; fonte→trabalhador cap 1, trabalhador→tarefa cap 1, tarefa→sumidouro cap X (ou [X, ∞) no mínimo); rode Dinic; compare o fluxo com n. No mínimo, converta os lower bounds e cheque a saturação.
  - *Exemplo:* distribuir n tarefas em m máquinas; a menor carga máxima é o menor X com fluxo n.
  - *Armadilha:* maximizar o mínimo não é só trocar a capacidade por X; precisa de lower bounds/demandas; escolha os limites da binária (0..soma); trate a infeasibilidade.
- **Binária + 2-SAT.** Escolher 1 de 2 posições/cores por item, todo par escolhido deve estar a distância ≥ D → binária em D e 2-SAT onde cada par proibido vira cláusula ($O(n^2)$ cláusulas se n pequeno ou segtree de arestas se grande).
  - *Gatilho:* cada item tem 2 posições/cores; pares escolhidos não podem estar a distância < D; "escolha uma opção por item".
  - *Por que funciona:* fixado D, cada par de opções incompatíveis gera a cláusula (¬a ∨ ¬b); 2-SAT decide se há escolha consistente; D maior significa menos conflitos, então a viabilidade é monótona.
  - *Como fazer:* binária em D; uma variável booleana por item (opção 0/1); para cada par incompatível, adicione as implicações a→¬b e b→¬a; rode SCC. Para n grande, codifique faixas com segtree/arestas auxiliares.
  - *Exemplo:* duas torres por cidade com distância mínima entre torres escolhidas; duas cores com restrições de distância.
  - *Armadilha:* a cláusula de incompatibilidade é (¬a ∨ ¬b), não (a ∨ b); "exatamente uma opção" é representada por uma variável; distância < D vs ≤ D; compare distâncias ao quadrado para evitar double.
- **Binária + MST/Kruskal.** Caminho que minimiza o máximo → MST mínimo (bottleneck); com consultas, árvore de reconstrução de Kruskal + LCA; com condição extra (conjunto de vértices conectado) → binária no peso + DSU.
  - *Gatilho:* caminho que minimiza o máximo de aresta (bottleneck); conectividade com limite de peso; consultas de minimax.
  - *Por que funciona:* o valor minimax entre u e v é o maior peso na rota do MST; a árvore de reconstrução de Kruskal responde consultas; para "conectar com arestas ≤ X", o DSU processa só as arestas permitidas.
  - *Como fazer:* MST por Kruskal e binary lifting para o máximo no caminho; ou reconstruction tree (cada união cria um nó com o peso, LCA dá o bottleneck); para condição extra de conjunto conectado, binária em X + DSU.
  - *Exemplo:* "qual o menor peso máximo para ir de A a B"; rede de fibra com custo máximo; estradas com limite de altura.
  - *Armadilha:* minimax não é caminho mínimo em soma; empates nos pesos; grafo desconexo; a reconstruction tree precisa de LCA.
- **Binária + Dijkstra.** "Maximize o mínimo/limite de combustível/ruído em caminho" → binária no limite e BFS/Dijkstra só em arestas permitidas; se o limite é do vértice, remova vértices no check.
  - *Gatilho:* maximizar o mínimo de combustível/ruído/altura num caminho; arestas permitidas se peso ≤ X; limite no vértice.
  - *Por que funciona:* fixado X, restam só arestas (ou vértices) permitidos; a existência de caminho é monótona em X; BFS/Dijkstra decide.
  - *Como fazer:* binária em X sobre os valores; BFS se os pesos são 0/1, Dijkstra se é soma; para "maximizar o mínimo", cheque alcançabilidade com arestas ≥ X; se o limite é do vértice, remova vértices com valor > X (ou divida o vértice).
  - *Exemplo:* caminho de s a t com altura mínima máxima; voo com ruído máximo; capacidade de carga.
  - *Armadilha:* binária no valor errado (aresta vs vértice); precisão se real; 0-1 BFS vs Dijkstra; o caminho pode repetir vértices.
- **Binária + geometria.** Raio mínimo de cobertura, círculo com condição, margem máxima: binária no raio + interseção de discos/semiplanos; checar com varredura angular e eps; para o maior círculo vazio, Voronoi ou binária + hull.
  - *Gatilho:* raio mínimo de cobertura, círculo com condição, margem máxima; "existe ponto/raio que...".
  - *Por que funciona:* a viabilidade é monótona no raio/tamanho; fixado R, o problema vira interseção de discos/semiplanos ou cobertura por intervalos angulares, que é decidível com sweep.
  - *Como fazer:* binária em R; para cobertura de pontos por discos com centros candidatos, interseção de discos (ou teste de cobertura); para "círculo que contém todos", interseção de discos; para "maior círculo vazio", Voronoi ou binária + hull; use eps e varredura angular.
  - *Exemplo:* menor raio para cobrir pontos com k centros; círculo de raio R que intersecta todos os segmentos; margem entre polígonos.
  - *Armadilha:* eps e tangência; interseção de discos numericamente instável; contar cobertura no sweep (wrap-around); o ótimo pode estar na fronteira.

- **HLD + segtree + lazy.** Queries e updates de caminho em árvore com agregado não invertível (max, soma + atribuição, "primeiro vértice com valor ≥ x"): HLD vira O(log) intervalos, segtree guarda o agregado; ordem cuidadosa em caminhos não comutativos (esquerda/direita).
  - *Gatilho:* queries/updates de caminho em árvore; agregado não invertível (max, atribuição, "primeiro ≥ x"); soma com lazy.
  - *Por que funciona:* HLD decompõe o caminho em O(log n) intervalos contíguos na base; a segtree responde cada intervalo em O(log n); para agregados não comutativos, a ordem dos intervalos importa.
  - *Como fazer:* HLD (heavy child, pos, head); para caminho u–v, suba cabeças, acumule da esquerda para a direita e da direita para a esquerda separadamente; segtree com lazy (soma/assign/max); combine na ordem correta.
  - *Exemplo:* soma/máximo no caminho; atribuir valor a todos os vértices do caminho; achar o primeiro vértice com valor ≥ x.
  - *Armadilha:* ordem em agregados não comutativos; incluir/excluir o LCA; lazy de atribuição vs soma; head e profundidade.
- **HLD + estruturas de aresta.** Pesos em arestas → guarde no filho; query de caminho exclui o LCA. Com update de subárvore + caminho misto, use Euler tour para subárvore e HLD para caminho sobre a mesma segtree (tin da HLD).
  - *Gatilho:* peso nas arestas; update/query de caminho e subárvore; LCA.
  - *Por que funciona:* cada aresta é representada pelo filho; o caminho u–v exclui o LCA (a aresta do pai do LCA não pertence); o Euler tour da subárvore é contíguo no tin da HLD, então a mesma segtree serve para subárvore e caminho.
  - *Como fazer:* armazene o peso da aresta (pai→filho) no nó filho; query de caminho com HLD e exclua a posição do LCA; update de subárvore = intervalo [tin[v], tout[v]]; use a mesma segtree.
  - *Exemplo:* soma de arestas no caminho; atribuir peso a arestas de subárvore; consultas mistas.
  - *Armadilha:* a aresta raiz não existe; LCA incluído por engano; posições da HLD vs Euler; arestas de peso 0.
- **Link-cut / Euler tour tree.** Quando a floresta muda (link/cut) e o agregado é de caminho ou componente → LCT; quando só componentes e subárvore → ETT; offline, segtree no tempo + DSU rollback é mais simples.
  - *Gatilho:* floresta muda (link/cut); agregado de caminho ou componente; offline com rollback.
  - *Por que funciona:* LCT mantém caminhos preferenciais e permite link/cut/query em O(log n) amortizado; ETT mantém a sequência Euler e componentes/subárvores; para queries offline, segtree no tempo + DSU rollback evita a estrutura dinâmica.
  - *Como fazer:* LCT para caminho (makeroot, access, agregado de caminho); ETT para subárvore/componente (splay na sequência); offline: cada aresta ativa num intervalo de tempo, insira na segtree do tempo, DFS com rollback DSU.
  - *Exemplo:* conectar/remover arestas e perguntar o máximo no caminho; componentes dinâmicos; "a aresta existiu entre os tempos [l,r]?".
  - *Armadilha:* LCT não faz subárvore facilmente; ETT não faz caminho; rollback DSU sem path compression; constantes altas.
- **Bitset + SCC / DAG.** Alcançabilidade em DAG com $n \approx 5 \cdot 10^4$ e $m \approx 10^5$: condense SCC, processe em ordem topológica com bitsets de n/64 palavras em blocos (memória) → O(nm/64). Também "transitive closure" e "quantos vértices alcança cada v".
  - *Gatilho:* alcançabilidade em DAG, n ~ 5e4, m ~ 1e5; "quantos vértices cada um alcança"; fecho transitivo.
  - *Por que funciona:* condense SCCs (dentro do SCC todos se alcançam); processe em ordem topológica reversa, com OR dos bitsets dos sucessores; cada bitset tem n/64 palavras; O(mn/64) no total.
  - *Como fazer:* Tarjan/Kosaraju para SCC; DAG condensado; ordem topológica; reach[v] = {v} ∪ OR reach[u] para arestas v→u, processando de trás para frente; contar bits; para memória, processar em blocos de vértices.
  - *Exemplo:* "quantos vértices são alcançáveis a partir de cada v"; fecho transitivo.
  - *Armadilha:* memória $n^2/64$ bits (312 MB para n = 5e4); estourar cache; ordem topológica; SCC com auto-laços.
- **Bitset + BFS denso.** $n \le 3 \cdot 10^4$ com grafo denso: BFS mantendo bitset de não visitados e iterando por `_Find_next` → $O(n^2/64)$ em vez de O(m).
  - *Gatilho:* grafo denso, n ≤ 3e4, m ~ n^2; BFS repetido; vizinhança.
  - *Por que funciona:* mantenha o conjunto de não visitados como bitset; para cada vértice, os vizinhos são um bitset; a interseção com não visitados dá os novos em O(n/64) por vértice, total $O(n^2/64)$.
  - *Como fazer:* adj[i] bitset; unvisited bitset; fila; ao visitar v, novos = adj[v] & unvisited; iterar pelos bits com `_Find_first`/`_Find_next` ou `ctzll`, marcar e enfileirar; remover de unvisited.
  - *Exemplo:* BFS em grafo completo ou quase completo; distâncias em grid denso; componentes.
  - *Armadilha:* memória $n^2/64$; direção do grafo; bitset de adjacência é pior que lista em grafo esparso; `_Find_next` não é padrão.

- **Bitset + DP.** Mochila booleana, alcançabilidade em grid com obstáculos, LCS de strings $5 \cdot 10^4$ (Hyyrö, n·m/64), distância de edição em bits; sempre que o DP é "OR de deslocamentos".
  - *Gatilho:* DP booleano "OR de deslocamentos"; mochila booleana; alcançabilidade em grid; LCS/edit distance grandes.
  - *Por que funciona:* se a transição é dp |= dp << w, cada palavra de 64 bits processa 64 estados de uma vez; para LCS, o algoritmo de Hyyrö usa diferenças de bits.
  - *Como fazer:* represente o conjunto de somas alcançáveis como bitset; para cada item, dp |= dp << w; para LCS, bitset de matches por caractere + carries; para edit distance, bit-parallel.
  - *Exemplo:* subset sum com W ≤ 1e5 e n = 1e3; LCS de strings de 5e4; alcançabilidade em labirinto.
  - *Armadilha:* direção do shift; máscara para não passar de W; LCS bit-parallel é delicado (carry); memória.
- **Centroide + FFT.** Contar, para todo comprimento d, o número de pares de vértices a distância d: em cada centroide, polinômio das profundidades, A^2 menos soma dos A_filho^2 (remove pares do mesmo filho) → $O(n \log^2 n)$.
  - *Gatilho:* contar pares a cada distância na árvore; soma de caminhos; "para todo d, quantos pares".
  - *Por que funciona:* cada caminho tem um centroide mais alto; no centroide, o polinômio de profundidades conta pares; subtrair os pares do mesmo filho remove caminhos que não passam pelo centroide; a soma sobre a decomposição é $O(n \log^2 n)$.
  - *Como fazer:* decomposição por centroide; para cada centroide, contar profundidades (BFS/DFS limitada); A(x) = Σ count[d] x^d; contribuição = A^2 − Σ A_filho^2; coeficiente de x^d é o nº de pares ordenados; divida por 2 e remova (v,v); NTT/FFT.
  - *Exemplo:* número de pares a distância exatamente d; soma de distâncias; caminhos de comprimento k.
  - *Armadilha:* contar cada par uma vez (centroide mais alto); pares do mesmo filho; d = 0; precisão do FFT (use NTT ou long double); árvore de n = 1.
- **Centroide + estrutura.** Caminhos com condição em soma e comprimento (ex.: soma ≤ K): em cada centroide, ordene as distâncias e use two pointers com subtração do mesmo filho (ou BIT/ordenação por filho).
  - *Gatilho:* caminhos com condição em soma e comprimento (soma ≤ K); contar/otimizar caminhos que passam por um centroide; K grande.
  - *Por que funciona:* no centroide, as distâncias dos nós de cada filho são coletadas; o problema vira contar pares entre listas com soma ≤ K (two pointers após ordenar) e subtrair pares do mesmo filho; cada caminho é contado no seu centroide mais alto.
  - *Como fazer:* para cada centroide, colete (dist, depth) por filho; ordene; two pointers para contar pares com soma ≤ K; subtraia a contribuição de cada filho; para "exatamente K" ou máximo, use BIT/ordenação; para comprimento, inclua a dimensão.
  - *Exemplo:* contar caminhos com soma ≤ K; achar caminho de soma K; caminho com soma e comprimento limitados.
  - *Armadilha:* contagem dupla; pares do mesmo filho; a condição pode não ser monótona com a ordenação; K negativo; 64 bits.
- **Mo + DSU rollback.** Queries "o grafo com arestas l..r é bipartido/conexo/ #componentes": Mo sem remoções (Mo com rollback / blocos): adições ordenadas e desfazer; $O((n + q)\sqrt{m} \cdot \log)$.
  - *Gatilho:* queries de intervalo de arestas [l,r] sobre propriedade do grafo (bipartido, conexo, #componentes); sem updates; adições fáceis, remoções difíceis.
  - *Por que funciona:* DSU com rollback suporta união e desfazer em O(log n) sem path compression; Mo ordena as queries para amortizar as adições e só desfaz até o snapshot; a estrutura não precisa remover de verdade.
  - *Como fazer:* blocos de l; ordene por (bloco, r); para cada bloco, comece com DSU vazio e r crescente; para queries cujo l está no bloco, adicione arestas até o fim do bloco temporariamente, responda e faça rollback ao snapshot; $O((n+q)\sqrt{m} \log n)$.
  - *Exemplo:* "o subgrafo com arestas l..r é bipartido?"; "quantos componentes?"; "é conexo?".
  - *Armadilha:* path compression impede rollback; snapshot por query; arestas temporárias; ordenação e tamanho de bloco.
- **D&C no tempo + DSU rollback.** Arestas com tempo de vida, queries offline (conectividade, bipartição por paridade, nº de componentes): insira cada aresta em O(log) nós da segtree do tempo e faça DFS com rollback.
  - *Gatilho:* arestas com tempo de vida; queries offline em instantes; conectividade/bipartição/#componentes ao longo do tempo.
  - *Por que funciona:* cada aresta ativa em um intervalo de tempo é inserida em O(log T) nós da segtree do tempo; uma DFS na segtree aplica as arestas do nó com rollback ao sair; nas folhas, o DSU representa o estado naquele instante.
  - *Como fazer:* construa a segtree sobre o tempo; para cada aresta [l,r], insira nos nós que cobrem [l,r]; DFS: aplique as uniões do nó, responda as queries da folha, desfaça; DSU com paridade para bipartição.
  - *Exemplo:* arestas que aparecem e desaparecem; "o grafo é bipartido no tempo t"; componentes conexos por instante.
  - *Armadilha:* intervalos abertos/fechados; rollback; queries no nó vs na folha; memória O(m log T).

- **Mo em árvore / Mo com tempo.** Consulta de caminho com add/remove O(1): Euler de 2n posições (paridade de ocorrências); com updates: Mo 3D, n^{5/3}.
  - *Gatilho:* consultas de caminho em árvore com add/remove O(1); updates pontuais entre queries; "quantos valores distintos no caminho".
  - *Por que funciona:* no Euler tour de 2n posições, cada vértice aparece na entrada e na saída; no intervalo entre first[u] e first[v], os vértices que aparecem um número ímpar de vezes são exatamente os do caminho, exceto o LCA; com updates, o Mo 3D adiciona a dimensão tempo e usa bloco n^{2/3}.
  - *Como fazer:* DFS Euler (2n); query (u,v) vira [first[u], first[v]] com o LCA à parte; no add/remove, alterne o vértice (se estava, remove; se não, adiciona); para updates, ordene por (bloco l, bloco r, tempo) e aplique/desfaça updates.
  - *Exemplo:* valores distintos no caminho; soma de valores no caminho; caminho com updates de cor.
  - *Armadilha:* LCA incluído à parte; primeira ocorrência vs qualquer; updates alteram o array e a resposta; tamanho de bloco.
- **SAM + DP.** DP no DAG do autômato: nº de substrings distintas, k-ésima lexicográfica, contagem de ocorrências (soma nos links), substrings que ocorrem em exatamente k strings (generalizado) ou em todas.
  - *Gatilho:* substrings distintas, k-ésima lexicográfica, ocorrências, substrings comuns a k strings.
  - *Por que funciona:* o SAM é um DAG acíclico de estados que representa todas as substrings; caminhos da raiz ↔ substrings; o número de caminhos é o número de substrings distintas; endpos sizes contam ocorrências; a ordem por len é topológica.
  - *Como fazer:* construir SAM; contar caminhos no DAG (dp[state] = 1 + Σ dp[to]) para k-ésima; ocorrências: cnt[state] = 1 para prefixos, propague por suffix links em ordem decrescente de len; generalizado: construa SAM de várias strings e marque cores.
  - *Exemplo:* nº de substrings distintas de uma string; k-ésima substring; substrings que ocorrem em todas as k strings.
  - *Armadilha:* contar caminhos inclui a string vazia; ocorrências vs estados; ordem por len; generalizado com reset de last.
- **SAM / SA + segtree.** Ocorrências de padrão em substring s[l, r]: intervalo de sufixos no SA + estrutura 2D (BIT offline/persistente); no SAM, segtree merging em endpos (pergunta "existe ocorrência com início em [l, r]").
  - *Gatilho:* ocorrências de um padrão em s[l,r]; "existe ocorrência começando em [l,r]"; queries de substrings.
  - *Por que funciona:* no SA, as ocorrências de um padrão formam um intervalo de sufixos (busca binária); restringir o início em [l,r] vira uma query 2D; no SAM, o estado do padrão tem endpos, e a segtree merging responde se há fim/começo na faixa.
  - *Como fazer:* SA + LCP + busca binária para o intervalo; BIT persistente/offline para contar posições em [l,r]; SAM + segtree de endpos (merge bottom-up) para "existe ocorrência com início em [l,r]" (ajuste o comprimento).
  - *Exemplo:* contar ocorrências de P em s[l..r]; verificar se P ocorre em s[l..r]; padrão mais frequente.
  - *Armadilha:* início vs fim (endpos); o padrão precisa caber em [l,r] (fim ≤ r); SA com sentinela; queries offline.
- **SA + DSU por LCP.** Para todo k, nº de pares de sufixos com LCP ≥ k, ou maior grupo com LCP ≥ k: processe LCP em ordem decrescente fundindo vizinhos no SA (Kruskal sobre o array LCP) com DSU que guarda agregados.
  - *Gatilho:* para todo k, nº de pares de sufixos com LCP ≥ k; maior grupo com LCP ≥ k; agrupamentos por prefixo comum.
  - *Por que funciona:* no SA, o LCP de dois sufixos é o mínimo do array LCP entre eles; processar o LCP em ordem decrescente e unir vizinhos (Kruskal) cria componentes que são grupos de sufixos com LCP ≥ k; os agregados dão as respostas.
  - *Como fazer:* construir SA e LCP; ordenar as posições do LCP por valor decrescente; DSU une i e i+1; para cada k, o componente tem tamanho s e contribui C(s,2); acumule para responder todos os k.
  - *Exemplo:* nº de pares de substrings com prefixo comum de comprimento ≥ k; maior classe de equivalência; soma de LCP de todos os pares.
  - *Armadilha:* o LCP é entre sufixos adjacentes; o mínimo no intervalo; unir na ordem certa; C(s,2) em 64 bits.
- **Aho-Corasick + matriz.** Contar strings de comprimento ≤ 10^{18} sem padrões proibidos → matriz de transição do autômato (≤ 100–200 estados) elevada; menor string que contém todos os padrões → Aho + máscara de padrões (≤ 10).
  - *Gatilho:* contar strings de comprimento ≤ 1e18 que evitam/contêm padrões; autômato pequeno; "menor string que contém todos".
  - *Por que funciona:* o AC é um autômato finito determinístico de estados ≤ soma dos padrões; a matriz de transição M[i][j] conta transições; M^L conta caminhos de comprimento L; para "≤ L", some as potências ou aumente a matriz com um estado acumulador.
  - *Como fazer:* construir AC, remover/marcar estados que são padrões; matriz (S+1)×(S+1) com contagens; exponenciação rápida O(S^3 log L); para "≤ L", use um estado acumulador ou soma geométrica; para "todos os padrões", BFS/DP no produto (estado, máscara).
  - *Exemplo:* contar DNA de comprimento 1e18 sem "ACGT"; menor string com todos os padrões (k ≤ 10) via BFS no produto.
  - *Armadilha:* estados de padrão e suffix links; contar a string vazia; mod; matriz densa S^3; "≤ L" não é só M^L.

- **Aho-Corasick + digit DP.** Contar números ≤ N cuja representação contém/evita substrings → DP[pos][estado AC][tight]. Idem com DP de KMP para um padrão só.
  - *Gatilho:* contar números ≤ N que contêm/evitam substrings; representação decimal; N grande.
  - *Por que funciona:* o AC reconhece substrings proibidas; o digit DP percorre os dígitos com estado (posição, estado AC, tight); a transição é a aresta do autômato; a condição de conter/evitar é absorvente ou marcada.
  - *Como fazer:* construir AC dos padrões; dp[pos][state][tight][found?]; para cada dígito permitido, transite; tight controla o limite N; some no fim; para um padrão só, o estado pode ser o prefixo do KMP.
  - *Exemplo:* contar números ≤ 1e18 sem "13"; contar números com "42"; evitar vários padrões.
  - *Armadilha:* zeros à esquerda; tight; estado de "já encontrou"; mod; N com 1e18 (string).
- **Fail tree + BIT.** Queries "quantos padrões i ocorrem em prefixo/segmento do texto j": árvore de falhas do AC, Euler tour; cada posição do texto soma +1 no nó, query = soma na subárvore do padrão (BIT offline).
  - *Gatilho:* queries "quantos padrões i ocorrem em prefixo/segmento do texto j"; múltiplos padrões; offline.
  - *Por que funciona:* no AC, um padrão é sufixo de um prefixo do texto se o estado do prefixo está na subárvore do padrão na fail tree; a Euler tour transforma subárvore em intervalo; somar +1 no estado de cada posição e consultar a subárvore conta ocorrências.
  - *Como fazer:* construir AC e fail tree; Euler tour (tin/tout); para cada texto, percorra e some +1 no estado atual (BIT); para cada query, consulte a soma em [tin[padrão], tout[padrão]]; offline por texto/segmento.
  - *Exemplo:* quantas vezes cada padrão aparece num texto; ocorrências em um prefixo; padrões mais frequentes.
  - *Armadilha:* ocorrências sobrepostas contam; o estado do padrão pode falhar para outro padrão; segmento [l,r] precisa de diferença de prefixos; offline.
- **Hash + trie / ordenação.** Comparar caminhos de árvore (isomorfismo) → hash AHU canônico (ordenar filhos); isomorfismo não enraizado: enraíze nos centros (≤ 2) e compare. Subárvores iguais em árvore → contagem por hash.
  - *Gatilho:* comparar caminhos/árvores; isomorfismo; subárvores iguais; hashing de sequências.
  - *Por que funciona:* o hash AHU canônico de uma árvore enraizada é calculado ordenando os hashes dos filhos; árvores isomorfas têm o mesmo hash; para não enraizada, os centros (≤ 2) são canônicos; para subárvores iguais, conte hashes.
  - *Como fazer:* DFS, hash[v] = f(sorted(hash[filhos])); para isomorfismo, compare os hashes dos centros; para caminhos, hash de sequência com prefixos e potências; use par de mods ou 64 bits.
  - *Exemplo:* duas árvores são isomorfas?; contar subárvores distintas; comparar strings em O(1).
  - *Armadilha:* colisões (use dois hashes ou verificação); ordenar filhos; centros; hash de caminho com direção.
- **DP + CHT + persistência.** dp[v] = min_{u ancestral} dp[u] + b_u·a_v (DP em caminho raiz→folha): CHT persistente (Li Chao persistente ou pilha com rollback + busca binária no hull) para voltar ao pai; O(n log n).
  - *Gatilho:* dp[v] = min_{u ancestral} dp[u] + b_u·a_v; DP em caminho raiz→folha; voltar ao pai.
  - *Por que funciona:* cada ancestral u define uma reta y = b_u·x + dp[u]; a consulta em a_v é o mínimo das retas; como o conjunto muda ao descer/voltar, uma CHT persistente (Li Chao) ou uma pilha com rollback mantém as retas do caminho.
  - *Como fazer:* DFS; insira a reta do nó; consulte a_v; para filhos, passe a versão; ao voltar, desfaça; Li Chao persistente O(log C) por operação, ou hull com rollback se as inclinações são monotônicas; O(n log n).
  - *Exemplo:* custo de caminho com transição linear; DP de árvore com "melhor ancestral"; otimização de funções lineares.
  - *Armadilha:* inclinações não monotônicas (Li Chao); overflow (b·a ~1e18); memória da persistência; consulta antes/depois de inserir.
- **CDQ + CHT / DP 1D/1D online.** dp[i] depende de dp[j], j < i, mas o hull precisa de ordem: CDQ — resolva esquerda, construa o hull de [l, m], contribua para [m, r] ordenando queries; O(n $\log^2$ n) ou log com merge.
  - *Gatilho:* dp[i] depende de dp[j], j < i, e a transição é uma reta; o hull precisaria de ordem, mas as consultas chegam online.
  - *Por que funciona:* CDQ divide o índice; resolve a metade esquerda, constrói o hull das transições de [l,m], usa para atualizar [m,r], e resolve a direita; cada par (j,i) é considerado uma vez; o hull é construído offline.
  - *Como fazer:* cdq(l,r): se l == r, retorne; cdq(l,m); construa hull com j ∈ [l,m] (ordene por inclinação); ordene as queries i ∈ [m,r] por x; atualize dp; cdq(m+1,r); O(n $\log^2$ n), ou O(n log n) com merge.
  - *Exemplo:* dp[i] = min_j dp[j] + $(a_i - a_j)^2$; partição com custo linear; LIS com transição de reta.
  - *Armadilha:* ordem das consultas; inclinações iguais; o hull deve ser reconstruído por nível; overflow; a recorrência precisa ser acíclica.

- **DP + segtree + pilha monotônica.** dp[i] = min_j dp[j] + max(a[j+1..i]) (ou soma de máximos): a pilha monotônica diz quais j têm o mesmo máximo → atualizações de intervalo (+ constante) na segtree sobre dp + consulta de min em [l, i−1].
  - *Gatilho:* dp[i] = min_j dp[j] + max(a[j+1..i]) (ou soma de máximos); manter o máximo/soma dos máximos ao estender i.
  - *Por que funciona:* a pilha monotônica agrupa os j cujo máximo de [j+1..i] é o mesmo; quando entra a[i], os grupos com máximo menor são fundidos e o máximo deles vira a[i]; cada grupo contribui dp[j] + const, então a segtree sobre dp com lazy de +const mantém o mínimo.
  - *Como fazer:* pilha de pares (valor, intervalo de j); segtree sobre dp[j] + max; ao processar i, desempilhe valores ≤ a[i], aplique −valor + a[i] nos intervalos correspondentes, empilhe; consulte min em [0,i−1]; dp[i] = min + ...; O(n log n).
  - *Exemplo:* soma dos máximos de todos os subarrays; partição minimizando soma de máximos; dp com máximo no intervalo.
  - *Armadilha:* intervalos da pilha; lazy com valores negativos; consulta antes de inserir dp[i]; 64 bits.
- **DP + SOS + máscara.** DP[mask] com transição sobre submáscaras por soma/condição → SOS; "escolher k de n ≤ 20 com restrição de pares" → DP[mask] com popcount; n ≤ 40 → meet in the middle (item seguinte).
  - *Gatilho:* DP sobre máscaras com transição por submáscara; soma/condição em subconjuntos; n ≤ 20; "escolher k com restrição de pares".
  - *Por que funciona:* a soma sobre submáscaras (SOS) calcula f[mask] = Σ_{sub⊆mask} g[sub] em O(n2^n) em vez de O(3^n); para restrições de pares, o estado mask é suficiente e a transição verifica compatibilidade; para n ≤ 40, o estado inteiro não cabe e usa meet in the middle.
  - *Como fazer:* SOS: para cada bit, para cada mask com o bit, f[mask] += f[mask ^ bit]; DP de subconjunto: dp[mask] = melhor sobre último elemento/partição; para "k de n", popcount; para n ≤ 40, divida e combine.
  - *Exemplo:* contar subconjuntos com soma/xor; partição em grupos; independent set em grafo pequeno.
  - *Armadilha:* O(3^n) para enumerar submáscaras; memória 2^n; n = 20 ok, n = 25 pesado; ordem do SOS.
- **Meet in the middle + ordenação.** n ≤ 40 em subset sum / contar subconjuntos com soma ≤ T / maximizar soma ≤ T: duas metades, ordenar e two pointers; com restrição extra de xor ou paridade → mapa por classe.
  - *Gatilho:* n ≤ 40 em subset sum/contagem de subconjuntos; duas metades; maximizar soma ≤ T; xor/paridade.
  - *Por que funciona:* enumerar 2^{n/2} por metade e combinar; ordenar uma metade e usar two pointers/binária para a outra; cada subconjunto é representado por uma combinação única; a restrição extra vira uma classe/mapa.
  - *Como fazer:* divida em n1, n2; gere todas as somas (e classes de xor/paridade); ordene a segunda; para cada soma da primeira, busque o complemento com upper_bound; conte/maximize; para contagem, mapa de frequências.
  - *Exemplo:* contar subconjuntos com soma ≤ T; maior soma ≤ T; soma exata; xor igual.
  - *Armadilha:* 2^20 ~ 1e6; memória; duplicatas; 64 bits; não divida se n ≤ 25 (DP de máscara).
- **Persistent segtree + árvore.** k-ésimo na caminho u–v: versão em cada nó = pai + valor; resposta com versões ver[u] + ver[v] − ver[lca] − ver[parent(lca)] e descida simultânea.
  - *Gatilho:* k-ésimo no caminho u–v; versões por nó; queries de caminho com valores.
  - *Por que funciona:* a versão de cada nó é a do pai mais o valor do nó; a diferença entre versões conta valores no caminho; a identidade ver[u] + ver[v] − ver[lca] − ver[parent(lca)] isola o caminho; descer simultaneamente nas quatro versões dá o k-ésimo.
  - *Como fazer:* compressão de valores; DFS construindo raiz[u] = update(raiz[pai], valor); LCA; para k-ésimo, em cada nó da segtree conte left = cnt[left(u)] + cnt[left(v)] − cnt[left(lca)] − cnt[left(parent(lca))]; desça; O(log n).
  - *Exemplo:* mediana/k-ésimo no caminho; contar valores em faixa no caminho; modo.
  - *Armadilha:* versão do pai do LCA; compressão; memória O(n log n); LCA binário.
- **Wavelet / persistente + binária.** Contar valores em [x, y] no subarray [l, r], k-ésimo, mediana: wavelet tree em O(log σ) por query; com updates → BIT de segtrees ou sqrt; offline → BIT + varredura.
  - *Gatilho:* contar valores em [x,y] no subarray [l,r]; k-ésimo; mediana; queries 2D.
  - *Por que funciona:* a wavelet tree divide o domínio por valores e guarda prefixos de bits; em O(log σ) desce para o intervalo e conta; o persistente faz o mesmo com versões de prefixo; com updates, BIT de segtrees ou sqrt.
  - *Como fazer:* wavelet tree (ou merge sort tree) para count/k-ésimo; persistente: raiz[i] = update(raiz[i−1], a[i]); count = query(r) − query(l−1); k-ésimo descendo; offline: BIT + sweep por valor.
  - *Exemplo:* k-ésimo em subarray; contar valores em [x,y]; mediana de subarray.
  - *Armadilha:* compressão; índices l−1; updates; memória O(n log σ).

- **Offline varredura + BIT.** "Quantos pares (i, j) em [l, r] com a_i < a_j" é contagem de pares em subarray (inversões/não inversões): use Mo + BIT, D&C offline (CDQ) ou estrutura 2D/wavelet; já "soma de max/min em subarrays" usa pilha monotônica para definir intervalos de contribuição e varredura por r com a segtree histórica do item seguinte.
  - *Gatilho:* "quantos pares (i,j) em [l,r] com a_i < a_j"; inversões/não inversões em subarray; soma de max/min de subarrays.
  - *Por que funciona:* pares podem ser contados offline ordenando por um dos índices e usando BIT sobre o outro; para soma de max/min, a pilha monotônica define o intervalo de contribuição de cada a_i, e uma varredura por r com segtree histórica acumula.
  - *Como fazer:* para pares, Mo + BIT (O((n+q)√n log n)) ou CDQ/2D; para soma de max, para cada i encontre L_i, R_i (anterior/next maior) e some a_i·(i−L_i)·(R_i−i); para queries dentro de [L,R], varra r e mantenha a segtree histórica.
  - *Exemplo:* contar inversões em [l,r]; soma dos máximos de todos os subarrays; soma dos mínimos.
  - *Armadilha:* contar cada par uma vez; i < j; empates com < vs ≤; 64 bits; queries offline.
- **Segtree histórica.** Soma de f(l..r) para todos os subarrays dentro de [L, R], "versão do array a cada tempo": segtree com lazy de "soma acumulada ao longo do tempo" (histórico), usada com varredura por r e pilha monotônica.
  - *Gatilho:* soma de f(l..r) para todos os subarrays dentro de [L,R]; "versão do array a cada tempo"; soma acumulada ao longo do tempo.
  - *Por que funciona:* a segtree mantém, além do valor atual, a soma histórica; o lazy registra quanto tempo um valor ficou aplicado; ao somar um incremento, a contribuição histórica é incremento × tempo; isso permite acumular respostas de todos os r.
  - *Como fazer:* nós com sum, hist, lazy_add, lazy_hist; push combinando; em cada passo de r, atualize os intervalos da pilha monotônica e consulte a soma histórica; responda queries por R; O((n+q) log n).
  - *Exemplo:* soma dos máximos de todos os subarrays; queries de "soma de todos os subarrays"; soma de a_i ao longo do tempo.
  - *Armadilha:* composição de lazy (ordem); histórico de máximo vs soma; 64 bits; implementação delicada.
- **Sweep angular + BIT/two pointers.** Contar triângulos que contêm a origem ou que não contêm: ordene por ângulo e some C(k, 2) por ponto (complemento); fecho convexo + pontos dentro: binária no hull.
  - *Gatilho:* contar triângulos que contêm/não contêm a origem; pontos em semicírculos; cobertura angular.
  - *Por que funciona:* um triângulo não contém a origem se os três pontos estão num mesmo semicírculo; para cada ponto, conte os pontos num ângulo < π à frente e some C(k,2); subtraia de C(n,3). Para pontos dentro de um polígono, hull + binária.
  - *Como fazer:* ordene por ângulo (atan2 ou cross); duplique o array com +2π; two pointers para a janela < π; some C(k,2); cuidado com colineares; para "contém", complemente.
  - *Exemplo:* quantos triângulos contêm o centro; pontos num semicírculo; cobertura por setores.
  - *Armadilha:* origem sobre um lado; pontos colineares; wrap-around; precisão de atan2 (use cross).
- **Fecho convexo + binária/CHT.** Queries de reta extrema, tangente a partir de ponto, maior produto escalar com direção → hull + busca binária; muitas direções ofensivas → rotating calipers.
  - *Gatilho:* queries de reta extrema, tangente a partir de ponto, maior produto escalar com direção; muitas direções.
  - *Por que funciona:* o máximo de um produto escalar sobre um conjunto é atingido num vértice do hull; a função ao longo do hull é unimodal, então binária; para muitas direções, rotating calipers percorre o hull em O(n).
  - *Como fazer:* construa o hull (Andrew, cross ≤ 0); para query de direção d, binária no hull comparando vizinhos; para tangente de ponto, binária; para todas as direções ordenadas, rotating calipers.
  - *Exemplo:* ponto mais distante em cada direção; largura de um polígono; interseção de semiplanos.
  - *Armadilha:* hull com colineares; ordem horária/anti-horária; direção zero; precisão; pontos duplicados.
- **Geometria + sweep + segtree.** Área da união de retângulos (segtree com contagem de cobertura + comprimento coberto, ou min da cobertura + contagem do mínimo), pontos em círculos/retângulos (sweep + BIT), interseção de segmentos (sweep com set ordenado por y).
  - *Gatilho:* área da união de retângulos; pontos em círculos/retângulos; interseção de segmentos.
  - *Por que funciona:* a varredura processa eventos ordenados por x; a segtree mantém a cobertura no eixo y; para união de retângulos, a área entre eventos é (x_{i+1} − x_i) × (comprimento coberto); para segmentos, um set ordenado por y detecta interseções.
  - *Como fazer:* eventos (x, +1/−1, y1, y2); compressão de y; segtree com contagem de cobertura e comprimento coberto (ou min + count); para pontos em retângulos, sweep + BIT; para segmentos, Bentley–Ottmann (ou força bruta se n pequeno).
  - *Exemplo:* área da união de retângulos; perímetro; pontos cobertos por círculos (sweep angular).
  - *Armadilha:* coordenadas 64-bit; empates de x; retângulos degenerados; interseção de segmentos é delicada (colineares).

- **Geometria + DSU/flow.** Discos que se tocam → grafo de interseção + DSU; "cortar caminho com círculos" → caminho mínimo no dual ou componentes conexas; fluxo planar ↔ dual.
  - *Gatilho:* discos que se tocam; "cortar caminho com círculos"; separar regiões; fluxo planar.
  - *Por que funciona:* discos que se intersectam/tangenciam formam componentes via DSU; para bloquear um caminho, os obstáculos precisam formar uma cadeia que conecta as bordas; em grafo planar, o corte mínimo é o caminho mínimo no dual.
  - *Como fazer:* DSU nos discos (dist ≤ r1 + r2, com eps); adicione as bordas do retângulo como nós e veja se a cadeia conecta os lados; para fluxo planar, construa o dual e rode Dijkstra; para "cortar", min cut no primal ou shortest path no dual.
  - *Exemplo:* círculos bloqueiam a passagem de (0,0) a (W,H); menor número de discos para separar; fluxo em grid planar.
  - *Armadilha:* tangência (eps); bordas do domínio; o dual de grafo planar é trabalhoso; discos que só se tocam contam como bloqueio.
- **Dijkstra + segtree de arestas.** Vértice → intervalo de vértices com custo: segtree de grafo com duas árvores (in/out) e arestas dirigidas de custo 0 (uma para cima, outra para baixo) permite Dijkstra com O(n log n) arestas; uma única árvore com arestas 0 bidirecionais colapsa as distâncias.
  - *Gatilho:* vértice → intervalo de vértices com custo; muitas arestas implícitas; grafo de intervalos.
  - *Por que funciona:* a segtree de grafo representa "de v para todos em [l,r]" com O(log n) arestas; duas árvores (in/out) com arestas dirigidas de custo 0 permitem arestas direcionadas; uma única árvore com arestas bidirecionais 0 serve para arestas não direcionadas.
  - *Como fazer:* construa segtree out (pai→filhos) e in (filhos→pai); aresta v→[l,r] = v→nós out que cobrem [l,r]; [l,r]→v = nós in→v; rode Dijkstra no grafo expandido O((n+m) $\log^2$ n).
  - *Exemplo:* teletransporte para intervalo; arestas de custo uniforme; menor caminho com saltos.
  - *Armadilha:* direção das arestas; uma árvore só colapsa distâncias; memória O(n log n); pesos.
- **Dijkstra em camadas.** (vértice, k usos de bônus, paridade, tempo mod c) como estado; K ≤ 20 → n·K estados; muitas arestas com peso 0/1 → deque.
  - *Gatilho:* estado inclui k usos de bônus, paridade, tempo mod c; K ≤ 20; "usar no máximo k cupons".
  - *Por que funciona:* o grafo de estados (v,k) tem n·K vértices; cada transição atualiza o estado; Dijkstra resolve; se os pesos são 0/1, 0-1 BFS é O(nK).
  - *Como fazer:* expanda o grafo ou rode Dijkstra com estado no heap; para "até k", use camadas e arestas de bônus; para "exatamente k", responda na camada k; para paridade, 2 camadas.
  - *Exemplo:* menor caminho com até k arestas grátis; caminho de paridade par; tempo módulo c.
  - *Armadilha:* K grande (n·K explode); bônus com custo; 0-1 BFS vs Dijkstra; memória.
- **Floyd incremental / reverso.** Remoções de vértices em sequência, "soma de todas as distâncias após cada remoção" → inverta: adicione vértices e atualize O(n^2) cada: O(n^3) total.
  - *Gatilho:* remoções de vértices em sequência; "soma de todas as distâncias após cada remoção"; adições.
  - *Por que funciona:* remover é difícil; inverta a ordem: adicionar vértices um a um e usar o novo vértice como intermediário no Floyd; atualizar dist[i][j] = min(dist[i][j], dist[i][v] + dist[v][j]) para todos i,j é O(n^2) por adição, total O(n^3).
  - *Como fazer:* processe a ordem reversa; mantenha a matriz de distâncias; ao adicionar v, atualize todos os pares com v como intermediário; some as distâncias ativas (não removidas) após cada passo; O(n^3).
  - *Exemplo:* cidades destruídas uma a uma; soma das distâncias após cada remoção; conectividade dinâmica.
  - *Armadilha:* ordem reversa; pares com vértices ainda não adicionados; INF; 64 bits na soma.
- **Fluxo em grid (xadrez).** Paridade de i + j faz bipartição para dominós e para "escolher células sem vizinhas" → matching/max independent set (n − ν); com custos → min-cost; corte mínimo para cercar/separar. Torres que não se atacam → matching entre linhas e colunas (células proibidas viram arestas ausentes); rainhas com bloqueios é NP-difícil.
  - *Gatilho:* dominós; escolher células sem vizinhas; torres que não se atacam; cercar/separar; grid bipartido.
  - *Por que funciona:* o grid é bipartido por paridade de i+j; dominós = matching perfeito/máximo; máximo conjunto independente = n − min vertex cover = n − max matching; com custos, min-cost flow; cortar = min cut.
  - *Como fazer:* modele células como vértices; arestas entre vizinhas; matching máximo (HK/Dinic); para torres, matching entre linhas e colunas (célula = aresta); para cercar, min cut com fonte/lado.
  - *Exemplo:* máximo de dominós; máximo de células sem adjacência; torres em células bloqueadas.
  - *Armadilha:* paridade; n − ν para independent set; rainhas com bloqueios é NP-difícil; custos exigem min-cost.

- **Cobertura mínima por caminhos / cadeias.** Min path cover em DAG vértice-disjunto = n − ν; com sobreposição, feche transitivamente primeiro. Min antichains/ chains → Dilworth via matching; "mínimo de guardas".
  - *Gatilho:* min path cover em DAG; min cadeias/antichains; "mínimo de guardas"; Dilworth.
  - *Por que funciona:* path cover vértice-disjunto = n − matching máximo no grafo de compatibilidade (cada aresta do matching une dois vértices num caminho); Dilworth: min decomposição em cadeias = max antichain; com sobreposição, feche transitivamente.
  - *Como fazer:* construa bipartido (u_left, v_right) para cada aresta de compatibilidade; matching máximo; resposta n − ν; reconstrua caminhos seguindo o matching; para antichain, matching no grafo de comparabilidade.
  - *Exemplo:* mínimo de caminhos para cobrir um DAG; mínimo de guardas em DAG; máximo antichain.
  - *Armadilha:* vértice-disjunto vs aresta-disjunto; fecho transitivo $O(n^3)$; DAG vs grafo geral; reconstrução.
- **Euler tour + flow/matching.** Decompor arestas de multigrafo euleriano em caminhos/ciclos, ou orientar aresta a aresta: circuito euleriano; "pintar arestas com 2 cores balanceadas" → Euler tour alternando cores (grafo bipartido regular em potência de 2 → colorir por D&C).
  - *Gatilho:* decompor arestas de multigrafo euleriano em caminhos/ciclos; orientar arestas; colorir arestas com 2 cores balanceadas.
  - *Por que funciona:* um grafo euleriano (graus pares, conexo) tem circuito euleriano; o circuito alterna arestas e pode ser cortado em caminhos; para colorir, um tour euleriano alterna cores; grafos bipartidos regulares têm decomposição em matchings (König).
  - *Como fazer:* Hierholzer para o circuito; corte nos vértices desejados; para 2 cores balanceadas, percorra o tour alternando; para k cores em grafo regular, decomponha em matchings; para potência de 2, D&C.
  - *Exemplo:* decompor em k caminhos; colorir arestas de um grafo 4-regular com 2 cores; orientar arestas para balancear graus.
  - *Armadilha:* graus ímpares (adicione arestas); conectividade; multigrafo; a decomposição em matchings exige regularidade.
- **Postman chinês.** Percorrer toda aresta custo mínimo: adicione emparelhamento perfeito mínimo entre vértices de grau ímpar (≤ 20 → DP em máscara sobre distâncias de Floyd).
  - *Gatilho:* percorrer toda aresta com custo mínimo; voltar ao início ou não; grafo com graus ímpares.
  - *Por que funciona:* em um grafo euleriano, o tour percorre cada aresta uma vez; para graus ímpares, é preciso repetir arestas; o custo extra é o emparelhamento perfeito mínimo entre vértices de grau ímpar (nas distâncias de Floyd), pois repetir um caminho adiciona seu custo.
  - *Como fazer:* ache vértices ímpares; Floyd para distâncias; se ≤ 20 ímpares, DP em máscara para o matching mínimo; some ao custo total das arestas; para "não voltar", escolha o par de ímpares a deixar sem emparelhar (caminho aberto).
  - *Exemplo:* carteiro que percorre todas as ruas; inspecionar todas as arestas; rota de menor custo.
  - *Armadilha:* o número de ímpares é par; grafo desconexo; dirigido precisa de min-cost flow; "não voltar" muda o emparelhamento.
- **TSP e variantes.** n ≤ 20: DP[mask][last] $O(2^n n^2)$; caminho hamiltoniano contagem: DP ou IE (2^n n); n ≤ 40 só com estrutura especial (meet in the middle em ciclos de comprimento fixo); métrica e n grande → aproximação (MST × 2) só se pedido.
  - *Gatilho:* n ≤ 20; caminho/ciclo hamiltoniano; "visitar todos"; contagem.
  - *Por que funciona:* DP[mask][last] = menor custo para visitar mask e terminar em last; transição para um não visitado; O(2^n n^2); para contagem, conte caminhos; para n ≤ 40, só com estrutura especial/MITM.
  - *Como fazer:* inicialize dp[1<<i][i] = 0; para mask, last, transite; resposta min_last dp[full][last] (+ volta se ciclo); para contar, soma; para Hamiltoniano em DAG, DP topológica.
  - *Exemplo:* caixeiro viajante com n = 18; contagem de ciclos hamiltonianos; caminho de custo mínimo visitando todos.
  - *Armadilha:* fixar o início para não contar rotações; memória 2^n·n; overflow; n = 20 já é 20M estados.
- **Ciclo/ rho + fast-forward.** Simulação de 10^{18} passos de função determinística → detecte ciclo (Brent/Floyd ou mapa de visitados) e extrapole; com estado grande, hash do estado.
  - *Gatilho:* simular 1e18 passos de função determinística; estado grande; "depois de k operações".
  - *Por que funciona:* a sequência entra em ciclo; detecte o prefixo e o período (Floyd/Brent ou mapa de visitados); pule os passos com aritmética modular; se o estado é grande, hash.
  - *Como fazer:* Brent (O(1) memória) ou mapa (O(período)); calcule mu e lambda; para k, se k < mu responda direto, senão k' = mu + (k − mu) % lambda; simule k'.
  - *Exemplo:* f(x) = x^2 mod m; permutação aplicada k vezes; gerador pseudoaleatório.
  - *Armadilha:* mu e lambda; k antes do ciclo; colisões de hash; estado não determinístico.

- **Árvore de Steiner.** Poucos terminais (k ≤ 10): DP[mask][v] = custo mínimo para conectar o conjunto mask e terminar em v, combinando submáscaras e Dijkstra por máscara: O(3^k n + 2^k m log n).
  - *Gatilho:* conectar k terminais (k ≤ 10) em grafo com pesos; "custo mínimo para conectar todos"; terminais não são todos os vértices.
  - *Por que funciona:* DP[mask][v] = custo mínimo para conectar o conjunto mask e terminar em v; combine submáscaras em v e relaxe com Dijkstra; a árvore ótima pode ser decomposta em subárvores que se encontram em v.
  - *Como fazer:* inicialize dp[1<<i][t_i] = 0; para cada mask, combine submáscaras (fixe um bit para evitar duplicatas), depois Dijkstra/SPFA para propagar; resposta min_v dp[full][v]; O(3^k n + 2^k m log n).
  - *Exemplo:* conectar cidades com fibra; rede mínima com terminais; k = 5.
  - *Armadilha:* enumerar submáscaras duas vezes; terminais vs vértices de Steiner; k = 10 → 3^10 ~ 59049; Dijkstra por máscara.
- **Circular: corte e duplicação.** Estrutura em círculo → fixe um ponto de corte (teste as 2–3 candidatas), ou duplique o array e use janela de tamanho n; DP circular: fixe o primeiro estado e rode DP linear.
  - *Gatilho:* estrutura em círculo; "mesa redonda"; DP circular; janela de tamanho n.
  - *Por que funciona:* um círculo pode ser cortado em algum ponto; testar os 2–3 candidatos (ou todos, se n pequeno) reduz ao caso linear; duplicar o array e usar janela de tamanho n captura todos os cortes; para DP circular, fixe o estado inicial e rode o DP linear, fechando o ciclo.
  - *Como fazer:* duplique a[0..n−1] em a[0..2n−1]; janela deslizante de tamanho n; para DP, rode duas vezes (primeiro escolhido/não escolhido) e combine; para "corte ótimo", teste cada posição se n pequeno.
  - *Exemplo:* soma máxima de subarray circular; colorir mesa redonda; partição circular.
  - *Armadilha:* contar rotações; n = 1; o corte pode ser entre elementos; DP com primeiro estado.
- **Geradores de problemas "somas de produtos".** Σ_{i<j} a_i a_j f(i, j) com f polinomial/exponencial em i − j → convolução (FFT/NTT) ou prefixos com potências; f = gcd → Möbius + múltiplos.
  - *Gatilho:* Σ_{i<j} a_i a_j f(i,j) com f polinomial/exponencial em i−j; f = gcd; convolução.
  - *Por que funciona:* se f(i,j) depende de i−j, a soma vira uma convolução: Σ_i a_i Σ_j a_j f(i−j); FFT/NTT calcula em O(n log n); para f = gcd, use Möbius/totiente e múltiplos.
  - *Como fazer:* escreva a soma como convolução (inverta um dos arrays); use NTT mod 998244353 ou FFT; para gcd, Σ_d φ(d) (Σ_{d|a_i} 1)²/2 ou Möbius; para potências, prefixos.
  - *Exemplo:* Σ_{i<j} a_i a_j (i−j)²; Σ gcd(a_i,a_j); soma de produtos por distância.
  - *Armadilha:* i < j vs i ≤ j (dividir/ajustar); mod; f não é de diferença; valores negativos.
- **Contagem em árvores com FFT.** DP de árvore com polinômio por vértice (nº de maneiras como função do tamanho/grau): produto de filhos por D&C + NTT: O(n log² n); com peso por profundidade, long-path decomposition.
  - *Gatilho:* DP de árvore com polinômio por vértice; nº de maneiras como função do tamanho/grau; produto de filhos.
  - *Por que funciona:* o produto de polinômios dos filhos é uma convolução; fazê-lo por D&C + NTT custa O(n log² n) no total; para pesos por profundidade, long-path decomposition evita recomputar.
  - *Como fazer:* dpv = x·Π dpfilho (ou soma); combine filhos com D&C/NTT; para contagem de caminhos por profundidade, long-path; trunque no necessário; mod NTT.
  - *Exemplo:* contar subárvores de cada tamanho; número de caminhos por comprimento; árvore com pesos.
  - *Armadilha:* mod 998244353; produto de muitos polinômios; truncamento; árvore estrela (muitos filhos pequenos).
- **Enumerar caminhos curtos em árvores/grafos esparsos.** Caminhos de comprimento ≤ k em grafo com grau limitado: BFS limitado; ciclos de comprimento ≤ 6 em grafo planar usam a 5-degenerescência: oriente as arestas do menor para o maior rank para limitar o out-degree a 5 na enumeração — grau médio < 6 sozinho não limita Σ C(grau, 2), como na estrela.
  - *Gatilho:* caminhos de comprimento ≤ k; ciclos de comprimento ≤ 6 em grafo planar; grau limitado.
  - *Por que funciona:* BFS limitada em profundidade enumera caminhos; em grafo planar (5-degenerado), orientar arestas do menor para o maior rank limita o out-degree a 5, então enumerar wedges/ciclos é O(n); grau médio < 6 sozinho não basta (estrela).
  - *Como fazer:* para caminhos, DFS/BFS com profundidade k e poda; para ciclos, oriente por degeneração e enumere a partir do menor vértice; para planar, use a ordem de degeneração; conte cada ciclo no seu vértice canônico.
  - *Exemplo:* contar triângulos/ciclos de comprimento 4–6; caminhos de comprimento ≤ k; k pequeno.
  - *Armadilha:* contar o mesmo caminho/ciclo várias vezes; grafo com estrela; degeneração vs grau médio; k grande.

- **Jogos em ICPC.** Soma de jogos independentes → Grundy: calcule pequeno por brute force, ache período; jogos em grafo/árvore → Grundy com DP; "primeiro ou segundo vence" geralmente é uma condição local (xor ≠ 0 ou paridade de profundidade).
  - *Gatilho:* soma de jogos independentes; "primeiro/segundo vence"; movimentos em grafo/árvore; Grundy.
  - *Por que funciona:* o teorema de Sprague–Grundy: a soma de jogos tem Grundy = xor dos Grundies; um estado é perdedor iff Grundy 0; para jogos com período, calcule pequeno e detecte o ciclo.
  - *Como fazer:* calcule Grundy por brute force para estados pequenos; veja periodicidade; para árvore/grafo, DP de Grundy; para "primeiro vence", xor ≠ 0; para jogos de moeda, paridade.
  - *Exemplo:* Nim; soma de jogos de tabuleiro; jogo de retirar pedras.
  - *Armadilha:* misère (muda a regra); jogos com ciclos (não são DAGs); período do Grundy; soma não é xor se os jogos interagem.
- **Simulação com estruturas.** "Ordem de eventos com tempo contínuo" → heap de eventos com cancelamento preguiçoso; "fila de impressão/CPU" → heap + sort por tempo; "elevador" → simulação com map; não faça passo a passo se T é grande — pule para o próximo evento.
  - *Gatilho:* ordem de eventos com tempo contínuo; fila de impressão/CPU; elevador; T grande.
  - *Por que funciona:* uma heap de eventos processa só os instantes relevantes; o cancelamento preguiçoso evita remover eventos obsoletos; pular para o próximo evento é O(log n) em vez de O(T).
  - *Como fazer:* heap (tempo, tipo, id); ao processar, descarte eventos cancelados; para filas, heap + ordenação por tempo; para elevador, map de passageiros/chamadas; para tempo contínuo, use eventos e comparações exatas (frações) ou eps.
  - *Exemplo:* simulação de CPU com chegadas e saídas; elevador; impressão.
  - *Armadilha:* empates de tempo; eventos cancelados; precisão; ordem de eventos no mesmo instante.
- **Grafos funcionais + estrutura.** Cada vértice aponta para um só → ciclos com árvores penduradas: binary lifting para k passos, ciclo + prefixo de somas para aplicar f^k; "colorir a menor quantidade" → ciclos independentes.
  - *Gatilho:* cada vértice aponta para um só; ciclos com árvores penduradas; aplicar f^k; "colorir o mínimo".
  - *Por que funciona:* o grafo funcional é um conjunto de componentes, cada uma com um ciclo e árvores; binary lifting responde f^k em O(log k); somas de prefixo no ciclo respondem acumulados; componentes são independentes para colorir.
  - *Como fazer:* encontre ciclos (visited/estado), numere posições no ciclo, calcule prefixos; binary lifting para fora do ciclo; para k, pule até o ciclo e use módulo; para colorir, processe cada componente.
  - *Exemplo:* permutação aplicada k vezes; "quantos passos até repetir"; colorir funcional.
  - *Armadilha:* mu e lambda; k antes do ciclo; múltiplos componentes; 64 bits.
- **Árvores de sufixo e DSU.** Contagem de substrings distintas em intervalo [l, r] (queries) → SA + segtree/BIT offline, ou SAM com LCT (heavy) — se a questão é mais simples, prefira hash com restrição.
  - *Gatilho:* contagem de substrings distintas em intervalo [l,r] com queries; estruturas de sufixo; queries difíceis.
  - *Por que funciona:* SA + estruturas offline conta substrings distintas num intervalo; SAM com LCT mantém endpos dinamicamente; para casos mais simples, hash com restrição evita a estrutura pesada.
  - *Como fazer:* SA + segtree/BIT offline (processar queries por r, manter última ocorrência); SAM + LCT (heavy) para updates; se a query é só "existe", hash/busca binária; avalie o custo.
  - *Exemplo:* substrings distintas em s[l..r]; número de substrings novas; queries de repetição.
  - *Armadilha:* problema genuinamente difícil; não superengenheirar; SA com sentinela; LCT com constante alta.
- **Raiz quadrada de queries.** Rebuild a cada B updates: queries de menor caminho com poucas arestas modificadas (Dijkstra a cada √q) ou contagem de coisas com updates em lote; combine com "pesado/leve por grau".
  - *Gatilho:* rebuild a cada B updates; queries de menor caminho com poucas arestas modificadas; contagem com updates em lote; pesado/leve.
  - *Por que funciona:* separar updates em blocos; dentro do bloco, trate as mudanças como exceções e use a estrutura base; rebuild O(n) a cada B updates dá O((n+q)√q) ou similar; pesado/leve por grau reduz o custo.
  - *Como fazer:* escolha B ~ √q; para cada bloco, marque arestas modificadas; rode Dijkstra da base e ajuste com as exceções; ou reconstrua a estrutura; combine com heavy/light (grau > √m).
  - *Exemplo:* distâncias com arestas que mudam; contagem de cores com updates; consultas em grafo dinâmico.
  - *Armadilha:* escolha de B; custo de reconstrução; exceções; pior caso.

- **Reconstrução da resposta.** Em ICPC, conferir o formato: se o problema exige "uma solução qualquer", rode o DP/fluxo e extraia os escolhidos (reverso de predecessores / arestas saturadas); teste o checker próprio no brute force.
  - *Gatilho:* problema pede "uma solução qualquer"; DP/fluxo; formato de saída; checker.
  - *Por que funciona:* DP/fluxo constrói a resposta ótima; é preciso armazenar precursores ou olhar arestas saturadas; testar com brute force e checker próprio evita WA de formato.
  - *Como fazer:* guarde a escolha em cada estado (predecessor); no fluxo, arestas com fluxo > 0; reconstrua de trás para frente; imprima no formato exato; rode o checker no brute force.
  - *Exemplo:* imprimir o caminho do TSP; escolher projetos do fecho; matching.
  - *Armadilha:* múltiplas respostas válidas; formato (índices 0/1); precisão; não reconstruir a escolha.
- **Ler como grafo de dependências.** Pré-requisitos, "A deve vir antes de B" → ordenação topológica (lexicograficamente menor = heap; contagem de ordens = DP em máscara ou fórmula do gancho em floresta, no item seguinte).
  - *Gatilho:* pré-requisitos; "A deve vir antes de B"; ordem topológica; contagem de ordens.
  - *Por que funciona:* a relação "antes" é uma aresta; a ordem topológica existe iff não há ciclo; o menor lexicográfico usa heap; a contagem de ordens é DP em máscara (n pequeno) ou fórmula do gancho em floresta.
  - *Como fazer:* Kahn (indegree) com heap para lexicográfico; DFS para ciclo; para contagem, DP[mask] ou hook-length; para dependências com custos, caminho crítico.
  - *Exemplo:* ordem de disciplinas; build system; scheduling com pré-requisitos.
  - *Armadilha:* ciclo; múltiplos componentes; arestas duplicadas; contagem é #P-hard em DAG geral.
- **Contagem de linearizações de floresta.** Nº de ordens topológicas de uma floresta enraizada = n!/∏_v sz(v) (fórmula do gancho para árvores); com condições extras → DP de intercalação com C(a+b, a).
  - *Gatilho:* nº de ordens topológicas de uma floresta enraizada; "quantas ordens respeitam os pré-requisitos"; árvore.
  - *Por que funciona:* a fórmula do gancho para árvores: n!/∏ sz(v); cada subárvore se intercala com as outras, e o número de intercalações é multinomial; a raiz deve vir antes dos filhos.
  - *Como fazer:* calcule sz[v]; resposta = n! × Π inv(sz[v]) mod p; para condições extras (ordem entre irmãos), DP de intercalação com C(a+b,a); para floresta, adicione uma raiz virtual.
  - *Exemplo:* árvore de pré-requisitos; contagem de ordens de um heap; linearizações de uma floresta.
  - *Armadilha:* mod primo e n ≥ p (inverso inexistente); raiz virtual; condições extras; 64 bits.
- **Ranges + conjuntos de intervalos.** Atribuição em intervalo (range assign) e "conte valores distintos": ODT (set de intervalos) com amortização; já range add/consultas de soma podem ser O(nq) no ODT — use segtree com lazy (ou ODT só para decompor quando houver assigns frequentes).
  - *Gatilho:* range assign; "conte valores distintos"; intervalos; ODT.
  - *Por que funciona:* o ODT (set de intervalos) mantém intervalos com o mesmo valor; assign divide e substitui, com amortização O((n+q) log n) quando há assigns frequentes; range add/sum não tem essa amortização e pode ser O(nq).
  - *Como fazer:* set de (l,r,val); split(pos) divide; assign apaga intervalos cobertos e insere um; para contar distintos, mantenha um mapa/set; para add/sum, use segtree com lazy.
  - *Exemplo:* pintar intervalos; contar cores após assigns; consultas de soma.
  - *Armadilha:* dados adversariais sem assigns; iterador invalidado; amortização não vale para add; usar segtree quando não há assign.
- **Escaneamento duplo (two-pointer em estrutura).** Subarrays com "max − min ≤ k" ou "k distintos" → janela + deque/multiset; contagem de subarrays ótimos → número de janelas × saídas por início (soma n − L).
  - *Gatilho:* subarrays com "max − min ≤ k" ou "k distintos"; janela; contagem de subarrays ótimos.
  - *Por que funciona:* a validade é monótona no intervalo (se [l,r] é válido, subintervalos também); ao avançar r, l nunca volta; deques/multiset mantêm max−min ou contagem de distintos; para contar, cada l tem um r mínimo e contribui com n−r+1.
  - *Como fazer:* para cada r, adicione a[r]; enquanto inválido, remova a[l] e avance l; conte r−l+1 ou n−l+1; use duas deques para max/min, ou mapa de frequências para distintos.
  - *Exemplo:* contar subarrays com max−min ≤ K; no máximo k distintos; soma ≤ S (com positivos).
  - *Armadilha:* negativos quebram a monotonia da soma; max−min usa duas deques; "exatamente k" = f(≤k) − f(≤k−1); contar cada subarray uma vez.

- **Meet in the middle em grafo.** Caminhos de comprimento 2k → conecte metades em vértice do meio (k passos de cada lado), contagem por mapa; caminho de soma S em subconjunto.
  - *Gatilho:* caminhos de comprimento 2k; encontrar a metade; caminho de soma S em subconjunto; contagem.
  - *Por que funciona:* um caminho de comprimento 2k tem um vértice do meio; enumerar k passos de cada lado e combinar no vértice do meio reduz o expoente; para subconjuntos, divida os itens.
  - *Como fazer:* para cada vértice do meio, gere os caminhos de k passos de s e de t; combine por soma/hash; para "soma S", mapa de somas; para contagem, frequências; para evitar contar duas vezes, ordene/oriente.
  - *Exemplo:* caminho de comprimento 2k com soma alvo; contar caminhos; subconjunto de soma S.
  - *Armadilha:* contar o mesmo caminho duas vezes; vértices repetidos; memória; k grande.
- **Fluxo + DP.** Primeiro DP calcula quais pares são compatíveis (ex.: janelas de tempo), depois fluxo/matching sobre o grafo resultante; ou fluxo dá estrutura e DP otimiza custos (redes em camadas por tempo).
  - *Gatilho:* primeiro DP calcula compatibilidade (janelas de tempo); depois matching/fluxo; ou fluxo dá estrutura e DP otimiza custos.
  - *Por que funciona:* o DP reduz o problema a um grafo bipartido/network; o fluxo resolve a seleção/atribuição; em redes em camadas por tempo, o fluxo modela a dinâmica e o DP otimiza o custo.
  - *Como fazer:* DP para gerar arestas (i compatível com j); matching/fluxo; para custos, min-cost flow; para rede temporal, camadas (v,t).
  - *Exemplo:* escalonar jobs em janelas; atribuir tarefas com pré-processamento; evacuação.
  - *Armadilha:* grafo grande; compatibilidade $O(n^2)$; reconstrução; integralidade.
- **Expandir o grafo no tempo.** Tempo discreto pequeno ($T \leq 100$): vértice (v, t) com arestas de espera (v, t) → (v, t + 1); fluxo em rede temporal (evacuação) com capacidade por unidade de tempo.
  - *Gatilho:* tempo discreto pequeno ($T \leq 100$); esperar; evacuação; capacidade por unidade de tempo.
  - *Por que funciona:* o estado (v,t) captura o tempo; arestas de espera (v,t)→(v,t+1) e de movimento (v,t)→(u,t+1); o fluxo na rede temporal modela capacidade e evacuação; o tamanho é O((n+m)T).
  - *Como fazer:* construa camadas; fonte no tempo 0, sink no tempo T; arestas de espera com capacidade; arestas de movimento com capacidade/custo; max flow/min-cost; para "chegar até T", conecte todos os tempos ao sink.
  - *Exemplo:* evacuação com capacidade por aresta; movimento com espera; entrega com prazos.
  - *Armadilha:* T grande; capacidade na espera vs movimento; múltiplas fontes; reconstrução.
- **Interativo em ICPC.** Poucos problemas, mas padrões: busca binária com perguntas de comparação, "descobrir permutação com ≤ n log n queries" (merge sort/insertion com binária), localizar vértice em árvore por centroide ou por descida no caminho pesado.
  - *Gatilho:* problema interativo; queries de comparação; "descobrir permutação com ≤ n log n"; localizar vértice em árvore.
  - *Por que funciona:* cada query dá informação; binária reduz o espaço por 2; merge sort/insertion com binária descobre a permutação em O(n log n); centroide/HLD localiza em O(log n) por descida.
  - *Como fazer:* leia o limite de queries; use binária/merge sort; para árvore, centroide (metade) ou heavy path; adapte-se às respostas; faça flush.
  - *Exemplo:* adivinhar número; ordenar com comparações; achar vértice especial.
  - *Armadilha:* estourar queries; flush; queries adaptativas vs fixas; caso de empate.

Quando nada funciona

O brute force é a ferramenta mais subestimada: ele gera dados, e dados geram observações.

- **Imprima a resposta para n = 1..15** e procure padrão, fórmula fechada ou a sequência na OEIS.
  - *Gatilho:* n pequeno e a resposta parece irregular; você não tem ideia da recorrência.
  - *Por que funciona:* valores pequenos revelam periodicidade, diferenças constantes ou uma recorrência; a OEIS identifica a sequência pelo começo.
  - *Como fazer:* rode o brute para n = 1..15 (até o maior que couber), imprima valores e diferenças; procure recorrência ou período; consulte a OEIS se necessário.
  - *Exemplo:* 1, 2, 5, 14, 42 → Catalan; diferenças de ordem fixa → polinômio em n.
  - *Armadilha:* n pequeno demais ou casos degenerados enganam; confirme a fórmula com mais termos.

- **Imprima a estrutura ótima** (qual escolha o brute force fez), não só o valor: o guloso costuma aparecer ali.
  - *Gatilho:* você tem o valor ótimo do brute, mas não enxerga o critério da solução.
  - *Por que funciona:* a escolha vencedora mostra qual estado ou troca local a DP/guloso precisa representar.
  - *Como fazer:* faça o brute reconstruir pais/itens/cortes e imprima a estrutura ótima para n pequeno; compare com o que o guloso faria.
  - *Exemplo:* imprimir os intervalos escolhidos num scheduling; imprimir o caminho reconstruído da DP.
  - *Armadilha:* imprimir só o valor; empates na reconstrução podem esconder a regra.
- **Stress test** contra o brute force quando tiver WA: gere casos aleatórios pequenos e compare.
  - *Gatilho:* WA em teste escondido e nenhum contraexemplo à mão.
  - *Por que funciona:* casos pequenos aleatórios cobrem combinações que o sample não tem; comparar com um brute confiável isola a falha.
  - *Como fazer:* escreva gerador com seed, brute e solução; rode muitos casos pequenos; no primeiro erro, reduza o caso mantendo a falha.
  - *Exemplo:* 1000 grafos com n ≤ 7 contra enumeração; reduzir o contraexemplo a 3 vértices.
  - *Armadilha:* gerador viciado; brute também errado; casos grandes demais para atingir bordas.
- **Chute a resposta** entre as candidatas usuais (0, 1, n, n − 1, 2^n, C(n, 2), soma, max − min) e teste.
  - *Gatilho:* problema de contagem/otimização com resposta que parece sempre um valor especial.
  - *Por que funciona:* muitas respostas extremas são casos degenerados (0, 1, n, n − 1); testar candidatas elimina hipóteses rápido.
  - *Como fazer:* gere casos aleatórios, compare a resposta do brute com cada candidata e veja qual coincide; só então prove a fórmula.
  - *Exemplo:* "mínimo de operações" costuma ser 0, 1, 2 ou n − 1; "número de subarrays" costuma ser C(n, 2).
  - *Armadilha:* coincidência em poucos casos; validar apenas com n pequeno.
- **Releia o enunciado** procurando a frase que você ignorou; quase todo "impossível" vem de uma condição esquecida.
  - *Gatilho:* a solução parece impossível ou dá WA em casos que deveriam ser simples.
  - *Por que funciona:* condições como "distintos", "conexo", "ordenado" ou "cabe em 64 bits" mudam o problema e são a causa mais comum de leitura errada.
  - *Como fazer:* liste cada restrição do enunciado e marque onde ela entra na solução; procure a que você não usou.
  - *Exemplo:* "a_i distintos" permite indexar por posição; "grafo conexo" elimina componentes isoladas.
  - *Armadilha:* reconhecer o problema clássico e resolver esse em vez do enunciado.
- **Pergunte o que o autor queria testar:** se a solução óbvia é $O(n^2)$ e $n = 2 \cdot 10^5$, a ideia está em tirar exatamente essa dimensão.
  - *Gatilho:* a solução óbvia é $O(n^2)$ e n é grande; nenhuma técnica do catálogo encaixa.
  - *Por que funciona:* as constraints denunciam a técnica pretendida; a distância entre o óbvio e o alvo é a dimensão a otimizar.
  - *Como fazer:* compare a complexidade óbvia com o limite; identifique a dimensão que precisa sair (n, valor, queries) e escolha a técnica que remove exatamente essa dimensão.
  - *Exemplo:* $n = 2 \cdot 10^5$ com DP $O(n^2)$ → segtree/CHT; $a_i \leq 10^6$ → crivo ou contagem por valor.
  - *Armadilha:* otimizar a dimensão errada; assumir a técnica da moda sem olhar as constraints.

Sobre forçar ideias: os dois lados estão certos em momentos diferentes. Forçar técnica cedo atrapalha, porque você encaixa o problema numa ferramenta sem entendê-lo. Forçar depois de observar ajuda, porque cada pergunta do catálogo vira um experimento barato no seu exemplo pequeno. A ordem que funciona é observar primeiro, catálogo depois.

Depois de cada problema, escreva uma linha nova aqui: o gatilho que você deveria ter visto e a ideia que ele pedia. O catálogo que mais ajuda é o que cresce com os seus próprios erros.

---

adicionado em 2026-11-10 por agentes; revisado

- **Diagnóstico por veredito.** WA 1: leitura/formato/sample. WA em teste médio: caso de borda ou guloso errado. WA em grande: overflow, precisão ou lógica errada. TLE: complexidade alta (n×n em vez de n log n), pior caso (estrela, caminho, todos iguais), recursão profunda, constante. RE: índice, stack, divisão por 0. MLE: arrays em 2D de tamanho cheio.
  - *Gatilho:* primeiro veredito sem pista; WA, TLE, RE ou MLE que não se reproduz no sample.
  - *Por que funciona:* cada veredito aponta para uma classe de causa; o tamanho do teste que falha separa leitura/formato (sample), borda/guloso (médio) e overflow/lógica (grande); TLE separa complexidade de constante.
  - *Como fazer:* monte a tabela veredito→hipóteses; no WA, rode sample, um caso médio e um grande; no TLE, meça com n máximo e pior caso; no RE, cheque índices, stack e divisão por 0; no MLE, conte a memória.
  - *Exemplo:* WA só no teste grande costuma ser overflow; WA no sample costuma ser formato ou leitura.
  - *Armadilha:* fixar numa causa só; não reproduzir o veredito localmente; esquecer que o erro pode ser multiteste.

- **Casos de borda padrão.** n = 1, n = 2, todos iguais, já ordenado, ordenado ao contrário, valores máximos/mínimos, 0 e negativos, árvore caminho e estrela, grafo completo, componente isolada, resposta 0 / −1.
  - *Gatilho:* antes de submeter; checklist de casos que costumam quebrar a implementação.
  - *Por que funciona:* os casos degenerados exercitam ramos (laços vazios, divisões, inicializações) que o sample não cobre.
  - *Como fazer:* teste n = 1 e n = 2; todos iguais; ordenado e reverso; valores extremos; 0 e negativos; caminho e estrela; grafo completo e componente isolada; respostas 0/−1.
  - *Exemplo:* árvore estrela (muitos filhos) vs caminho (profundidade); array de zeros.
  - *Armadilha:* testar só um caso; n = 1 com laço que assume n ≥ 2; não testar a resposta −1.
- **Gerador de testes.** Escreva um gerador com seed e parâmetros (n, valor máx); 1000 testes pequenos contra o brute force acham 95% dos bugs; reduza o contraexemplo (menor n, menor valor) até caber à mão.
  - *Gatilho:* WA sem contraexemplo; validar guloso/DP contra brute force.
  - *Por que funciona:* casos aleatórios pequenos cobrem combinações que o sample não tem; a redução preserva a falha e isola a causa.
  - *Como fazer:* gerador com seed e parâmetros; rode 1000 casos pequenos contra o brute; ao achar falha, reduza n e os valores mantendo o erro; imprima o caso mínimo.
  - *Exemplo:* stress de um matching contra enumeração; redução de um grafo de 8 para 3 vértices.
  - *Armadilha:* gerador viciado (só casos fáceis); casos grandes aleatórios não acham bordas; redução que muda a falha.
- **Overflow.** a·b com a, b ≤ 10^9 em long long; produto mod p: multiplique em ll antes do %; INF perto de $9{\cdot}10^{18}$ estoura em INF + INF (use 1e18 ou 4e18 e cheque antes de somar); 1 << 40 precisa de 1LL; __int128 para produto de $10^{18} \times 10^{18}$ mod m.
  - *Gatilho:* produtos/somas com valores ≤ 10^9; INF; shifts; acumuladores.
  - *Por que funciona:* int estoura em ~$2{,}1{\cdot}10^9$; long long vai até ~$9{,}2{\cdot}10^{18}$; INF + INF estoura se INF for perto do limite.
  - *Como fazer:* use long long em a·b; 1LL << k; multiplicação modular com __int128 (ou método russo); INF = $4{\cdot}10^{18}$ e cheque antes de somar; compare distâncias sem somar.
  - *Exemplo:* a·b com a,b ≤ 10^9; 1 << 40; soma de n distâncias.
  - *Armadilha:* (a*b) % m com int; INF + dist; acumular muitos termos; conversão implícita.
- **Mod negativo.** (x % m + m) % m; subtração modular sempre soma m; cuidado com `-` e `unsigned` (size() − 1 quando vazio vira $2^{64} - 1$).
  - *Gatilho:* subtração modular; size() − 1; contas com unsigned.
  - *Por que funciona:* em C++, o % de um negativo é negativo ou zero; unsigned faz wrap-around para um valor enorme.
  - *Como fazer:* normalize com (x % m + m) % m; em subtração modular some m antes do %; use tipos com sinal; evite size_t em diferenças.
  - *Exemplo:* (a − b) % m com a < b; v.size() − 1 com vazio.
  - *Armadilha:* unsigned; % com negativo; m = 1; misturar int e size_t.
- **TLE vs constante.** Antes de otimizar assintótica, troque map por array/sort, vetor 2D por plano, endl por \n, sync_with_stdio(false) + cin.tie(0), evite divisões mod no laço interno, acesse memória em ordem, reserve vector.
  - *Gatilho:* complexidade correta mas TLE; laço interno pesado; estruturas caras.
  - *Por que funciona:* a constante e o cache dominam em n grande; map, endl e alocação multiplicam o custo; acesso sequencial melhora a cache.
  - *Como fazer:* map→array/sort; vetor 2D→array plano; \n; fast IO; tire % do laço; percorra em ordem; reserve; meça antes e depois.
  - *Exemplo:* 10^7 operações com endl vs \n; map vs unordered/array.
  - *Armadilha:* otimizar assintótica antes da constante; micro-otimizar sem medir; unordered_map com pior caso.
- **Zerar arrays por teste.** memset de MAXN com t = 10^5 testes é O(MAXN·t); zere só n usados. Estruturas globais devem ser limpas entre testes; esquecer isso é a causa nº 1 de WA em multitestes.
  - *Gatilho:* multitestes; arrays/estruturas globais; memset de MAXN por teste.
  - *Por que funciona:* memset de MAXN por teste custa O(MAXN·t); e estado residual do teste anterior contamina o próximo.
  - *Como fazer:* zere só as posições usadas; limpe vetores/estruturas; use timestamp/versão para visitados; cuidado com globais e DSU.
  - *Exemplo:* t = 10^5 com MAXN = $2{\cdot}10^5$; DSU não reinicializado.
  - *Armadilha:* memset em tudo; esquecer uma estrutura; limpar parcialmente.

- **Ordem de leitura.** Ao dar `return` cedo (n = 1, caso trivial), leia primeiro o resto do teste; senão os testes seguintes viram lixo.
  - *Gatilho:* return cedo; caso trivial; multitestes; getline.
  - *Por que funciona:* a entrada é um fluxo; retornar sem consumir o teste desalinha a leitura dos testes seguintes.
  - *Como fazer:* leia toda a entrada do teste antes de decidir; ou pule explicitamente o restante; cuidado ao misturar >> e getline.
  - *Exemplo:* n = 1 e return antes de ler as arestas; linha em branco após o inteiro.
  - *Armadilha:* return no meio da leitura; cin >> seguido de getline; quebra de linha pendente.
- **Recursão profunda.** DFS em caminho de $2{\cdot}10^5$ → stack overflow; faça iterativo ou aumente stack (ulimit/#pragma); BFS/ordem topológica quando possível.
  - *Gatilho:* DFS em caminho de $2{\cdot}10^5$; recursão em árvore profunda; segtree recursiva.
  - *Por que funciona:* a pilha do sistema tem alguns MB; cada frame ocupa dezenas/centenas de bytes; $2{\cdot}10^5$ frames podem estourar.
  - *Como fazer:* converta para iterativo com pilha explícita; use BFS/ordem topológica; aumente a stack (ulimit -s) ou #pragma quando o ambiente permitir.
  - *Exemplo:* caminho de $2{\cdot}10^5$; DFS recursiva em grafo.
  - *Armadilha:* aumentar a stack no juiz não funciona; recursão em segtree também é profunda; frames grandes por variáveis locais.
- **Comparador inválido.** `<=` em sort/set derruba o programa ou dá resultado errado; deve ser ordem fraca estrita. Em set/map de pares com ordem customizada, igualdade = nenhum menor que o outro.
  - *Gatilho:* sort/set/map com comparador customizado; pares; ordenação de structs.
  - *Por que funciona:* o algoritmo exige strict weak ordering; `<=` viola a irreflexividade e a transitividade, podendo causar comportamento indefinido.
  - *Como fazer:* use `<`; para pares, compare os campos em ordem; igualdade significa que nenhum é menor que o outro; teste com elementos iguais.
  - *Exemplo:* sort com return a <= b; comparador que usa subtração com overflow.
  - *Armadilha:* comparador que depende de estado global; set de pares; double NaN.
- **Armadilhas do C++.** map[x] insere; multiset.erase(valor) apaga todos (use erase(find)); iterador invalidado ao inserir em vector; ponto flutuante igual com `==`; char com sinal; endl misturado com printf; uso de variável não inicializada (use -fsanitize=memory ou Valgrind; ASan/UBSan/_GLIBCXX_DEBUG não detectam).
  - *Gatilho:* comportamento estranho; map/set/vector; comparação de double; char.
  - *Por que funciona:* map[] insere por padrão; erase(valor) apaga todas as cópias; inserir em vector pode realocar e invalidar iteradores; double tem erro de representação; char pode ser signed.
  - *Como fazer:* use find antes de erase; guarde índices ou reserve; compare double com eps; cast para unsigned char; não misture endl e printf; inicialize variáveis.
  - *Exemplo:* multiset.erase(3) apaga todas as cópias; iterador após push_back.
  - *Armadilha:* sanitizers não pegam tudo; variável não inicializada; ordem de avaliação.
- **Conferir leitura e saída.** Imprima o que leu; confira 0/1-indexado; confirme que a resposta é a pedida (índice? valor? quantidade?). Muitos WA são problema bem resolvido com a pergunta errada.
  - *Gatilho:* WA sem causa aparente; formato; índices; múltiplas respostas.
  - *Por que funciona:* muitos erros são de leitura/índice ou de responder algo diferente do pedido, não de algoritmo.
  - *Como fazer:* imprima o que leu; confira 0/1-indexado; releia o pedido (índice, valor ou quantidade); rode o sample e compare exatamente.
  - *Exemplo:* imprimir o valor em vez do índice; off-by-one.
  - *Armadilha:* múltiplas respostas válidas; espaços/newlines; não ler o enunciado até o fim.
- **Cheque a hipótese do guloso.** Escreva o contraexemplo de pelo menos 3 tentativas de ordem; se nenhuma falha, enumere exaustivamente instâncias pequenas (n ≤ 6–7, valores pequenos) — 1000 testes aleatórios não validam o guloso — e ainda assim exija um argumento de troca.
  - *Gatilho:* guloso plausível; WA em casos médios; "ordene por..." sem prova.
  - *Por que funciona:* o aleatório raramente encontra o contraexemplo estruturado; a enumeração pequena é exaustiva; a prova de troca é a única validação real.
  - *Como fazer:* teste 3 ordens candidatas; enumere n ≤ 6–7 com valores pequenos; procure o contraexemplo; exija argumento de troca; compare com brute force.
  - *Exemplo:* interval scheduling ordenando por início vs fim; mochila fracionária vs 0/1.
  - *Armadilha:* achar que 1000 testes provam; contraexemplo de 3 itens; confundir "funciona no sample".

- **Se o brute force é muito lento.** Faça brute force em duas camadas: um exponencial pequeno ($n \leq 8$) para testar WA e um polinomial lento para testar TLE de ideias.
  - *Gatilho:* brute exponencial não roda nem para n pequeno; quer testar TLE de uma ideia.
  - *Por que funciona:* as duas camadas separam correção (exponencial pequeno) de eficiência (polinomial lento); cada uma responde uma pergunta.
  - *Como fazer:* exponencial com poda para $n \leq 8$; polinomial $O(n^2)$ ou $O(n^3)$ para n médio; compare a ideia rápida com a lenta em n médio.
  - *Exemplo:* TSP brute $n \leq 10$; DP lenta vs rápida.
  - *Armadilha:* confundir as camadas; brute com poda que muda a resposta; a camada lenta ser rápida demais.
- **Se não acha a recorrência.** Calcule 2d termos por brute force e rode Berlekamp–Massey (ordem $\leq d$); funciona para recorrências lineares, e confirme com termos extras.
  - *Gatilho:* sequência sem fórmula; DP sem recorrência; termos pequenos.
  - *Por que funciona:* Berlekamp–Massey encontra a recorrência linear mínima de ordem $\leq d$ com 2d termos; se a sequência é linearmente recorrente, os termos extras confirmam.
  - *Como fazer:* calcule 2d termos por brute; rode BM; verifique os próximos termos; se bater, use a recorrência; cuidado com o módulo.
  - *Exemplo:* contar caminhos; sequência de Catalan; recorrência de ordem 2.
  - *Armadilha:* recorrência não linear; poucos termos; mod; sequência que só coincide nos primeiros termos.
- **Se a estrutura é grande demais.** Imprima o estado ótimo para $n = 5, 6, 7$ e desenhe; procure uma bijeção com algo contável ou um padrão de bits/paridade.
  - *Gatilho:* estado ótimo grande; não sabe como representar; $n = 5..7$.
  - *Por que funciona:* desenhar o ótimo revela simetrias, bijeções e padrões de paridade que a fórmula esconde.
  - *Como fazer:* imprima as escolhas do brute force; desenhe; procure bijeção com objetos contáveis; olhe bits/paridade; formule e teste.
  - *Exemplo:* matching ótimo; coloração; subconjunto ótimo.
  - *Armadilha:* padrão válido só para n pequeno; bijeção falsa; confundir coincidência com estrutura.
- **Estratégia de equipe (ICPC).** Leia todos os problemas nos 15–20 primeiros minutos; ordene por dificuldade; um computador por vez (quem pensa escreve no papel); depure no papel ou numa tela de outro, não no computador do time; penalidade de 20 min por WA ⇒ revise antes de submeter.
  - *Gatilho:* contest ICPC; três pessoas; um computador; muitos problemas.
  - *Por que funciona:* ler tudo antes evita travar num problema difícil; ordenar por dificuldade maximiza o número de soluções; pensar no papel libera o computador.
  - *Como fazer:* 15–20 min de leitura; ordenação; um no teclado, os outros no papel; depuração fora do computador do time; revisão antes de submeter (penalidade de 20 min).
  - *Exemplo:* começar pelos fáceis; não disputar o computador; dividir problemas.
  - *Armadilha:* todos no mesmo problema; submeter com dúvida; não ler todos os problemas.
- **Quando submeter.** Rode samples, os 3 casos de borda e um stress rápido se há brute force; submeter com leve dúvida custa 20 min, enquanto 2 minutos de verificação raramente custam.
  - *Gatilho:* dúvida se a solução está certa; pressão de tempo; penalidade.
  - *Por que funciona:* 20 min de penalidade por WA vs 2 min de verificação; a verificação tem custo baixo e evita penalidade alta.
  - *Como fazer:* rode samples; 3 casos de borda; stress rápido se houver brute; se ainda houver dúvida, submeta.
  - *Exemplo:* WA por formato evitado com o sample; borda $n = 1$.
  - *Armadilha:* esperar demais; submeter sem rodar o sample; ignorar a penalidade.
- **Treinamento.** Resolva problemas ~200–300 de rating acima do seu; passe 30–45 min tentando antes de ver o editorial; depois reimplemente sem olhar. Anote por problema o gatilho que faltou; refaça os que errou após 1–2 semanas.
  - *Gatilho:* evolução de rating; rotina de treino; problemas travados.
  - *Por que funciona:* dificuldade ligeiramente acima força aprendizado; tentar antes do editorial consolida; reimplementar sem olhar fixa a ideia; revisar depois combate o esquecimento.
  - *Como fazer:* +200–300 de rating; 30–45 min antes do editorial; reimplemente sem olhar; anote o gatilho; refaça os erros em 1–2 semanas.
  - *Exemplo:* upsolve de um problema de grafo; reimplementação de DP.
  - *Armadilha:* ver o editorial cedo; não reimplementar; volume sem revisão.

- **Pós-contest (checklist).** 1) Upsolve todos os problemas que tentou. 2) Classifique cada erro: leitura / ideia / implementação / tempo. 3) Escreva a linha do gatilho no catálogo. 4) Reimplemente o que errou limpo, com brute force. 5) Leia o código de 2–3 soluções melhores para ver o truque. 6) Registre quanto tempo gastou em cada problema e se a ordem foi boa.
  - *Gatilho:* fim do contest; aprendizado; erros recentes.
  - *Por que funciona:* upsolve e classificação convertem a experiência em catálogo; reimplementar e ler soluções melhores revelam o truque; registrar tempos melhora a estratégia.
  - *Como fazer:* siga os 6 passos na ordem; classifique o erro; anote o gatilho; reimplemente; leia 2–3 códigos; registre tempos e ordem.
  - *Exemplo:* classificar um WA como leitura vs ideia; reimplementar o problema errado.
  - *Armadilha:* só upsolve; não classificar; não revisar; esquecer de anotar.
- **Biblioteca de ICPC.** Um template que compila em 1 s, com macros mínimas, DSU, segtree, BIT, grafo, Dinic, matching, hash, NTT, fórmula de mod; depois do contest, adicione o que você ainda reimplementou na hora.
  - *Gatilho:* template; contest; retrabalho de estruturas.
  - *Por que funciona:* um template rápido e testado economiza tempo; adicionar após o contest o que faltou mantém a biblioteca útil.
  - *Como fazer:* template que compila em 1 s; macros mínimas; DSU, segtree, BIT, grafo, Dinic, matching, hash, NTT, mod; teste; versione.
  - *Exemplo:* Dinic pronto; DSU com rollback.
  - *Armadilha:* template gigante que não compila; código não testado; dependências desnecessárias.

Índice da lib (técnica → arquivo)

Índice da biblioteca

Gerado em 2026-11-11 a partir dos cabeçalhos de cada arquivo. Total: **206 trechos**. Status `compila` = `g++ -std=c++17 -Wall -fsyntax-only` com o prelude do template; `testado` = conferido contra brute force/casos conhecidos; `com binom_mod` = precisa colar `code/combinatorics/binom_mod.cpp` antes.

Pastas ainda vazias: `code/bits/`, `code/games/`, `code/primitives/` (Lote 1 e 2 de `docs/PLANO_LIB.md`).

Arquivo	Nome	Descrição	Seção do catálogo	Status
algebra/				
`code/algebra/berlekamp_massey.cpp`	Berlekamp-Massey + n-esimo termo de recorrencia linear (Kitamasa O(k^2 log n))	dado s[0..m) de uma recorrencia linear de ordem L (mod primo), acha os	Álgebra, polinômios e FFT	compila
`code/algebra/bostan_mori.cpp`	Bostan-Mori: [x^n] P(x)/Q(x) mod 998244353	n-esimo coeficiente de uma serie racional P/Q (Q[0] != 0)	Álgebra, polinômios e FFT	compila
`code/algebra/conv_mod.cpp`	Convolucao com modulo arbitrario por 3 NTT + CRT (Garner)	c = a*b mod M para M qualquer (ex.: 1e9+7), exato (sem erro de ponto flutuante)	Álgebra, polinômios e FFT	compila
`code/algebra/fft.cpp`	FFT complexa (double) e convolucao com modulo arbitrario por split	conv(a,b) convolui vetores de double	Álgebra, polinômios e FFT	compila
`code/algebra/lagrange.cpp`	Interpolacao de Lagrange em pontos consecutivos (0..d)	dado y[i] = P(i) para i = 0..d (P de grau <= d), calcula P(n) para n qualquer	Álgebra, polinômios e FFT	compila
`code/algebra/ntt.cpp`	NTT (transformada teorica dos numeros) e convolucao mod 998244353	multiplica dois polinomios com coeficientes mod 998244353 (g = 3)	Álgebra, polinômios e FFT	testado (brute, 2026-11-11)
`code/algebra/poly.cpp`	Polinomios e series formais mod 998244353 (inv, ln, exp, sqrt, pow, divmod, multipoint, interpolacao)	operacoes de series formais por Newton, divisao de polinomios, avaliacao em	Álgebra, polinômios e FFT	compila
`code/algebra/power_sum.cpp`	Soma de potencias sum_{k=1}^n k^d mod p (Faulhaber via Lagrange)	calcula S(n,d) = 1^d + 2^d +	Álgebra, polinômios e FFT	compila
combinatorics/				
`code/combinatorics/binom_general.cpp`	Binomial modulo composto qualquer (e potencia de primo)	C(n,k) mod m para m arbitrario, n,k ate 1e18	Combinatória, contagem e probabilidade	compila
`code/combinatorics/binom_mod.cpp`	Fatoriais e binomiais modulo primo	pre-calcula fact, ifact e inv_[1..N]	Combinatória, contagem e probabilidade	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/combinatorics/burnside.cpp	Burnside, colares, pulseiras, strings primitivas	(1) Burnside generico: dado um grupo de permutacoes (lista de	Combinatória, contagem e probabilidade	compila
code/combinatorics/catalan_family.cpp	Familia de Catalan e contagens de caminhos/selecoes classicas	Catalan, ballot (estrito e com empate), Narayana, Fuss-Catalan,	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/counting_misc.cpp	Contagens diversas: hook length, floresta, graficidade, torneios, torres, Ryser	# SYT por hook length	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/grid_paths.cpp	Caminhos em grid: obstaculos, faixa 0<=y<=h (reflexao alternada)	(1) # caminhos (direita/cima) de (0,0) a (X,Y) evitando k pontos	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/inclusion_exclusion.cpp	Inclusao-exclusao: utilitarios	(1) contar inteiros em [1,N] nao divisiveis por nenhum de uma lista	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/lgv.cpp	Lindstrom-Gessel-Viennot (caminhos disjuntos em vertices)	dado e[i][j] = # caminhos de a_i ate b_j em um DAG, devolve	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/lucas.cpp	Lucas e Kummer	C(n,k) mod p para p primo PEQUENO (p ate ~1e6) e n,k ate 1e18	Combinatória, contagem e probabilidade	compila
code/combinatorics/matrix_tree.cpp	Matrix-Tree (Kirchhoff), determinante mod p e mod composto	det de matriz mod primo (Gauss) e mod qualquer m (eliminacao tipo	Combinatória, contagem e probabilidade	compila
code/combinatorics/partitions.cpp	Particoes de inteiros (numeros pentagonais de Euler e DPs em sqrt)	p(0..N) por pentagonais	Combinatória, contagem e probabilidade	compila
code/combinatorics/perm_counts.cpp	Contagens de permutacoes especiais	desarranjos D_n, perms com exatamente k pontos fixos, involucoes,	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/prob_expect.cpp	Probabilidade e esperanca modulo p: utilitarios	cadeia de Markov absorvente (E[tempos] = (I-Q)^-1 1 por Gauss mod p),	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/stirling_bell.cpp	Stirling 1a e 2a especie, Bell, Eulerianos (tabelas O(n^2) e valor unico)	tabelas S2[n][k], S1[n][k] (sem sinal), Eulerianos A[n][k],	Combinatória, contagem e probabilidade	compila (com binom_mod)
code/combinatorics/tree_counting.cpp	Contagens de arvores rotuladas e estruturas classicas (Cayley, Prufer, componentes)	Cayley n^{n-2}	Combinatória, contagem e probabilidade	compila (com binom_mod)
dp/				
code/dp/bitset_subset_sum.cpp	Subset sum com bitset e multiplicidades	dado multiconjunto de pesos, diz quais somas 0..W sao alcancaveis	DP	compila
code/dp/broken_profile.cpp	Broken profile DP (plug DP simples): ladrilhar grade com dominos	conta (mod) o numero de ladrilhamentos de uma grade n x m com dominos 1x2, com celulas bloqueadas '#'	DP	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/dp/coins.cpp	Mochila de moedas: minimo de moedas para um valor	dp[i] = minimo de moedas (uso ilimitado) para somar i	DP	compila + testado
code/dp/comb.cpp	Fatoriais e binomiais modulo M (versao antiga e simples)	pre-calcula fact/invFact e responde C(n,k) modulo M	Combinatória, contagem e probabilidade	compila
code/dp/dc_opt.cpp	Divide and conquer optimization (D&C DP)	calcula dp[k][i] = min_{j<i} dp[k-1][j] + C(j,i) para k=1..K quando opt(i) e monotono	DP	compila
code/dp/digitdp.cpp	Digit DP (soma dos digitos de 0..k)	conta/otimiza sobre os digitos de 0..k com estado (posicao, soma, tight)	DP	compila + testado
code/dp/edit_distance.cpp	Edit distance (Levenshtein) com reconstrucao e versao em faixa	edit_dist: distancia minima (inserir/remover/trocar) entre a e b e uma sequencia de operacoes	DP	compila
code/dp/frog.cpp	Frog: custo minimo pulando de 1 a k posicoes	dp[i] = min_{1<=j<=k} dp[i-j] + |a[i]-a[i-j]|	DP	compila + testado
code/dp/frog3_cht.cpp	Frog 3 com CHT (exemplo completo, standalone)	resolve o DP dp[i] = c + h[i]^2 + min_j (dp[j] + h[j]^2 - 2*h[i]*h[j])	DP	compila (programa completo)
code/dp/interval_dp.cpp	DP em intervalo: exemplos classicos (baloes, impressora estranha, jogo em intervalo, circular)	moldes de dp[l][r] O(n^3)	DP	compila
code/dp/knapsack.cpp	Mochila 0/1	valor maximo com peso total <= w, cada item no maximo uma vez	DP	compila + testado
code/dp/knapsack_bounded.cpp	Mochila limitada (bounded knapsack): fila monotona e binary splitting	maximiza valor com itens (peso w_i, valor v_i, quantidade c_i), capacidade W	DP	compila
code/dp/knuth.cpp	Knuth optimization (DP em intervalo)	dp[l][r] = min_{l<=k<r} dp[l][k] + dp[k+1][r] + w(l,r) em O(n^2) usando opt[l][r-1] <= opt[l][r] <= opt[l+1][r]	DP	compila
code/dp/lcs.cpp	LCS classica O(n*m) (subsequencia comum maxima)	dp[i][j] = LCS dos prefixos a[0..i) e b[0..j)	DP	compila
code/dp/lcs_bitset.cpp	LCS com bitset (Hyyro / Allison-Dix)	comprimento da LCS de a e b em O(n*m/64), usando vetor de palavras de 64 bits com soma e carry	DP	compila
code/dp/lis.cpp	LIS em O(n log n)	tamanho da maior subsequencia estritamente crescente	DP	compila + testado
code/dp/slope_trick.cpp	Slope trick	mantem uma funcao convexa linear por partes f(x) com dois heaps (pontos de quebra a esquerda L e a direita R)	DP	compila
code/dp/sosdp.cpp	SOS DP recursivo (soma sobre submascaras)	gen_submask(b, mask) = soma de F[s] para todas as submascaras s de mask	Bits e XOR	compila
code/dp/subsetsum.cpp	Maior soma de subconjunto <= t (subset sum otimizado)	dado w (pesos inteiros >= 0) e limite t, devolve o maior valor <= t	DP	compila + testado

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/dp/tree_knapsack.cpp	Knapsack em arvore: merge O(n^2) e DP sobre ordem DFS O(nK)	escolher exatamente k vertices formando subarvore conexa que contem a raiz (se pega v, pega o pai),	Árvores	compila
code/dp/tsp_bitmask.cpp	DP em bitmask: TSP, atribuicao e particao em subconjuntos	(1) tsp: menor ciclo hamiltoniano, dp[mask][ultimo]	DP	compila
ds/				
code/ds/aliens.cpp	Aliens trick / WQS binary search (lambda optimization) generico	se f(k) = melhor custo usando EXATAMENTE k itens eh CONVEXA	Busca binária e otimização	compila
code/ds/bit.cpp	BIT (Fenwick) 1D + busca binaria na BIT + BIT de maximo	soma de prefixo/intervalo com update pontual	Estruturas de dados e queries	compila
code/ds/bit2d.cpp	BIT 2D	update pontual (soma) e soma em retangulo numa grade n x m	Estruturas de dados e queries	compila
code/ds/bit_rr.cpp	BIT range-update range-query	soma em intervalo de v (add em [l,r]) e soma de intervalo,	Estruturas de dados e queries	compila
code/ds/busca_binaria.cpp	Busca binaria: templates (inteira, real, exponencial, k-esimo, mediana)	colecao de templates de binaria na resposta	Busca binária e otimização	compila
code/ds/busca_binaria_paralela.cpp	Busca binaria paralela (por niveis)	para Q queries "primeiro tempo t em que pred(query) vale", com	Busca binária e otimização	compila
code/ds/busca_ternaria.cpp	Busca ternaria (real, inteira) e Dinkelbach	acha o minimo de f unimodal/convexa	Busca binária e otimização	compila
code/ds/cdq.cpp	CDQ (divide e conquista offline) - template com BIT: dominancia 3D / DP 2D	exemplo completo de CDQ: res[i] = val[i] + max(res[j]) para	Estruturas de dados e queries	compila
code/ds/cht_dinamico.cpp	Line container (CHT dinamico, estilo KACTL)	insere retas y=k*x+m em qualquer ordem e consulta o MAXIMO em x	Estruturas de dados e queries	compila
code/ds/cht_monotono.cpp	Convex Hull Trick monotono (deque)	mantem envoltoria inferior de retas y=m*x+b inseridas com	Estruturas de dados e queries	compila
code/ds/conectividade_dinamica.cpp	Conectividade dinamica offline (segtree no tempo + DSU com rollback)	cada aresta existe em um intervalo de tempo [l, r)	Estruturas de dados e queries	compila
code/ds/disjoint_sparse.cpp	Disjoint sparse table	query de qualquer operacao ASSOCIATIVA (soma, produto mod, xor,	Estruturas de dados e queries	compila
code/ds/distinct_range.cpp	Distintos em intervalo (offline, sweep por r + BIT)	para cada query [l,r] devolve (a) quantidade de valores	Estruturas de dados e queries	compila
code/ds/dsu_paridade.cpp	DSU com paridade (bipartido online / equacoes xor)	mantem, para cada no, a paridade (xor) da distancia ate a raiz	Estruturas de dados e queries	compila
code/ds/dsu_potencial.cpp	DSU com pesos / potencial (soma)	mantem w[x] = pot[x] - pot[raiz]	Estruturas de dados e queries	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/ds/dsu_proximo_livre.cpp	DSU "proximo livre"	nxt(i) = menor j >= i ainda nao removido	Estruturas de dados e queries	compila
code/ds/dsu_rollback.cpp	DSU com rollback	uniao-busca sem compressao de caminho, com desfazer (undo) das	Estruturas de dados e queries	compila
code/ds/fila_monotonica.cpp	Fila/janela monotonica + fila com agregado (duas pilhas)	(1) deque monotono: min de janela deslizante em O(n)	Estruturas de dados e queries	compila
code/ds/heap_lazy.cpp	Heap com remocao preguicosa	priority_queue que aceita erase(x) de um valor presente,	Estruturas de dados e queries	compila
code/ds/heap_mergeavel.cpp	Heap mergeavel (leftist) de minimo	fila de prioridade com uniao de dois heaps em O(log n)	Estruturas de dados e queries	compila
code/ds/kdtree.cpp	K-d tree 2D (estatica): vizinho mais proximo e contagem em retangulo	constroi arvore balanceada alternando eixos (mediana por	Estruturas de dados e queries	compila
code/ds/lichao.cpp	Li Chao tree (minimo de retas / segmentos em x)	insere retas y=a*x+b (ou segmentos) e consulta o minimo em x	Estruturas de dados e queries	testado (brute, 2026-11-11)
code/ds/merge_sort_tree.cpp	Merge sort tree	cada no guarda seu intervalo ordenado	Estruturas de dados e queries	compila
code/ds/odt.cpp	ODT / Chtholly (set de intervalos com valor constante)	mantem a particao de [0,n) em intervalos de valor constante	Estruturas de dados e queries	compila
code/ds/ordered_set.cpp	Ordered set / multiset (pb_ds): rank e k-esimo	set ordenado com order_of_key(x) (quantos < x) e	Estruturas de dados e queries	compila
code/ds/ps2d.cpp	Soma de prefixo 2D (prefix sum em matriz)	prefix[i][j] = soma do retangulo [0..i] x [0..j]	Estruturas de dados e queries	compila + testado
code/ds/seg2d.cpp	Segtree de segtrees (2D online): update de ponto, soma em retangulo	camada externa iterativa em x, cada no externo tem uma segtree	Estruturas de dados e queries	compila
code/ds/seg_beats.cpp	Segtree beats (chmin em intervalo + add em intervalo; soma e max)	a[i] = min(a[i], x) em [l,r), a[i] += v em [l,r), query soma e	Estruturas de dados e queries	compila
code/ds/seg_descent.cpp	Segtree com descida: primeiro i>=l com a[i]>=x, ultimo i<=r, k-esimo 1	segtree de max (point update) com first_ge/last_ge em O(log n),	Estruturas de dados e queries	compila
code/ds/seg_dyn.cpp	Segtree dinamica (esparsa) com soma, k-esimo e merge de segtrees	segtree sobre dominio [0, U) enorme (ate 1e18), nos criados sob	Estruturas de dados e queries	compila
code/ds/seg_iter.cpp	Segtree iterativa (bottom-up) com monoide generico (nao comutativo ok)	update pontual e query de intervalo com qualquer operacao	Estruturas de dados e queries	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/ds/seg_lazy.cpp`	Segtree com lazy generica (estilo ACL) + descida max_right	update em intervalo (aplica tag F) e query de intervalo (monoide S),	Estruturas de dados e queries	testado (brute, 2026-11-11)
`code/ds/seg_nodes.cpp`	Nos prontos para segtree (max subarray, parenteses, funcao afim)	structs de no + merge para usar com Seg (code/ds/seg_iter.cpp)	Estruturas de dados e queries	compila
`code/ds/seg_persist.cpp`	Segtree persistente (contagem) - k-esimo em [l,r] e contagem <= x	versao i = prefixo a[0..i] inserido numa segtree sobre os valores	Estruturas de dados e queries	testado (brute, 2026-11-11)
`code/ds/seg_t.cpp`	Segtree iterativa (ponto + consulta de intervalo)	arvore de soma com update pontual O(log n) e query de intervalo O(log n)	Estruturas de dados e queries	compila + testado
`code/ds/segtree.cpp`	Segtree lazy (soma em intervalo + soma em range)	arvore recursiva com lazy para somar val em [l, r] e consultar a soma	Estruturas de dados e queries	compila
`code/ds/sparse_min.cpp`	Sparse table para minimo em intervalo estatico	pre-processa um array e responde min(a[l..r]) em O(1)	Estruturas de dados e queries	compila + testado
`code/ds/sparse_table.cpp`	Sparse table (RMQ O(1) para operacoes idempotentes) com indice	min/max estatico em O(1)	Estruturas de dados e queries	compila
`code/ds/sqrt_decomp.cpp`	Sqrt decomposition (exemplo: add em intervalo + contar >= x em intervalo)	blocos de tamanho B	Estruturas de dados e queries	compila
`code/ds/treap_implicito.cpp`	Treap implicito (sequencia) com split/merge, reverse e soma/add lazy	sequencia dinamica	Estruturas de dados e queries	testado (brute, 2026-11-11)
`code/ds/wavelet.cpp`	Wavelet matrix	sobre array estatico de inteiros >= 0: k-esimo menor, contar < x,	Estruturas de dados e queries	testado (brute, 2026-11-11)
flow/				
`code/flow/blossom.cpp`	Blossom (Edmonds) - emparelhamento maximo em grafo geral	matching maximo em grafo nao bipartido, encolhendo ciclos impares (blossoms)	Grafos	compila
`code/flow/dag_path_cover.cpp`	Cobertura minima de caminhos em DAG (disjuntos em vertices, ou com repeticao)	minimo de caminhos que cobrem todos os vertices de um DAG	Grafos	compila
`code/flow/dinic.cpp`	Dinic (fluxo maximo) + recuperacao do corte minimo	fluxo maximo s->t	Grafos	testado (brute, 2026-11-11)
`code/flow/flow_lower_bounds.cpp`	Fluxo com limites inferiores: circulacao viavel e fluxo maximo s->t	arestas com low <= f <= high	Grafos	compila
`code/flow/gale_shapley.cpp`	Gale-Shapley - casamento estavel	n homens e n mulheres com listas de preferencia completas	Grafos	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/flow/gomory_hu.cpp`	Arvore de Gomory-Hu (Gusfield) - corte minimo entre todos os pares	dado grafo NAO direcionado com capacidades, constroi arvore com n-1 arestas	Grafos	compila
`code/flow/hopcroft_karp.cpp`	Hopcroft-Karp - emparelhamento bipartido maximo	matching maximo em bipartido L x R em fases (BFS de niveis + DFS)	Grafos	testado (brute, 2026-11-11)
`code/flow/hungarian.cpp`	Hungarian - problema de atribuicao de custo minimo	dada matriz de custos n x m (n <= m), atribui cada linha a uma coluna	Grafos	testado (brute, 2026-11-11)
`code/flow/konig.cpp`	Konig - cobertura minima de vertices e conjunto independente maximo em bipartido	a partir do matching maximo (Hopcroft-Karp embutido) constroi cobertura	Grafos	compila
`code/flow/kuhn.cpp`	Kuhn - emparelhamento bipartido maximo (com heuristicas)	matching maximo em bipartido L x R por caminhos aumentantes	Grafos	compila
`code/flow/matroid_intersection.cpp`	Interseccao de matroides (cardinalidade maxima e peso maximo)	maior conjunto S do universo {0..n-1} independente em DUAS matroides, via	Grafos	compila
`code/flow/mcmf.cpp`	Min-cost max-flow (Dijkstra com potenciais; custos negativos ok se sem ciclo negativo)	fluxo de custo minimo de s a t, com limite opcional de fluxo	Grafos	testado (brute, 2026-11-11)
`code/flow/mcmf_spfa.cpp`	Min-cost max-flow simples com SPFA (aceita custos negativos sem ciclo negativo)	igual a mcmf.cpp, porem mais curto (bom para colar rapido, grafos pequenos)	Grafos	compila
`code/flow/project_selection.cpp`	Fecho maximo / selecao de projetos (closure) via corte minimo	escolhe conjunto S de itens maximizando soma dos lucros p[i] (podem ser	Grafos	compila
`code/flow/push_relabel.cpp`	Push-relabel (highest label + gap) - fluxo maximo	fluxo maximo	Grafos	compila
`code/flow/simplex.cpp`	Simplex (programacao linear) - estilo KACTL	maximiza c.x sujeito a A x <= b, x >= 0	Grafos	compila
general/				
`code/general/annealing.cpp`	Simulated annealing e hill climbing	template de recozimento simulado (maximizar score) com T geometrico e limite de tempo	Aleatorização	compila
`code/general/buscaternaria.cpp`	Busca ternaria (minimo de funcao convexa/unimodal)	tern_double(l, r) acha o x que minimiza func em [l, r] (250 iteracoes)	Busca binária e otimização	compila
`code/general/compressao.cpp`	Compressao de coordenadas	mapeia valores arbitrarios para 0..k-1 preservando a ordem	Arrays e guloso	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/general/debruijn.cpp	Sequencia de de Bruijn	string ciclica de tamanho k^n contendo toda string de tamanho n sobre alfabeto {0..k-1} exatamente uma vez	Construtivos e interativos	compila
code/general/debug.cpp	Macro de debug e timer	dbg(x, y, ...) imprime nome e valor em cerr (so com -DLOCAL)	Aleatorização	compila
code/general/diff2d.cpp	Prefixos 2D, diferencas 2D e maior submatriz de soma maxima	soma de retangulo em O(1)	Estruturas de dados e queries	compila
code/general/fastio.cpp	Fast IO (leitura/escrita manual com buffer)	leitura de inteiros/strings via fread e escrita via buffer	Como usar quando travar	compila
code/general/gale_ryser.cpp	Matriz 0/1 com somas de linhas e colunas dadas (Gale-Ryser) e canto noroeste	constroi matriz 0/1 com somas r[i] (linhas) e c[j] (colunas), ou diz que nao existe	Grafos	compila
code/general/gen_exemplo.cpp	Gerador de testes (exemplo para stress.sh / stress.bat)	gera um caso aleatorio pequeno determinado pela seed (argv[1])	Amostras de problemas e padrões de ICPC	compila (programa completo)
code/general/grau_seq.cpp	Sequencias de graus: Erdos-Gallai, Havel-Hakimi, arvore com graus dados	testa se uma sequencia e grafica	Construtivos e interativos	compila
code/general/gray_code.cpp	Codigo de Gray	g(i) = i ^ (i>>1) percorre todas as mascaras mudando 1 bit por passo	Construtivos e interativos	compila
code/general/guloso_heap.cpp	Gulosos com heap e arrependimento	(1) jobs com prazo e lucro (1 unidade cada)	Arrays e guloso	compila + testado
code/general/hash_anti.cpp	Hash customizado anti-hack (unordered_map / gp_hash_table)	splitmix64 com semente aleatoria para unordered_map/unordered_set e pb_ds gp_hash_table	Aleatorização	compila
code/general/int128_io.cpp	IO de __int128	le e imprime __int128 (cin/cout nao suportam)	Como usar quando travar	compila
code/general/interativo.cpp	Template interativo + buscas binarias interativas + interator local	funcao de query com flush, busca binaria por query (valor ate 1e18 em ~60 queries),	Construtivos e interativos	compila (standalone)
code/general/intervalos.cpp	Intervalos: sweep de eventos, salas, furar, cobrir, k maquinas	maxima sobreposicao (com tratamento de empates), atribuicao de salas, minimo de pontos para furar todos,	Arrays e guloso	compila
code/general/inversoes.cpp	Contagem de inversoes (merge sort e BIT) e pares com condicao	numero de pares i a[j]	Arrays e guloso	compila
code/general/maioria.cpp	Elemento majoritario: Boyer-Moore, > n/k, e versao aleatorizada	acha candidatos a elemento com frequencia > n/k em O(1)·k de memoria	Arrays e guloso	compila
code/general/mediana_janela.cpp	Mediana dinamica em janela deslizante (dois multisets)	mantem mediana e soma |a_i - mediana| de um multiconjunto com inserir/remover	Arrays e guloso	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/general/meet_middle.cpp	Meet in the middle (subconjuntos, n <= 40)	gera somas das duas metades e combina com ordenacao + two pointers	Arrays e guloso	compila
code/general/perm_ciclos.cpp	Ciclos de permutacao, potencia, paridade, trocas minimas	decompoe permutacao em ciclos	Arrays e guloso	compila
code/general/permutacoes.cpp	Geracao de permutacoes (backtracking)	generatePermutations(nums) devolve todas as permutacoes dos n elementos,	Construtivos e interativos	compila
code/general/pilha_monotonica.cpp	Pilha monotonica (proximo maior/menor, contribuicao, histograma, lexicografico)	prox/ant maior e menor	Arrays e guloso	compila
code/general/rng.cpp	RNG para gerar testes e stress	random_a(n) gera n inteiros uniformes em [l, r]	Aleatorização	compila
code/general/subsets.cpp	Geracao de todos os subconjuntos (por mascara)	subsets(nums) devolve os 2^n subconjuntos de nums (cada um como vetor)	Construtivos e interativos	compila
code/general/template.cpp	Template base para colar no inicio do arquivo	includes, macros (FORI, FORJ, endl, pb, all, pii), solve() vazio e main	Como usar quando travar	compila (programa completo)
code/general/torneio_landau.cpp	Torneio com pontuacoes dadas (Landau)	decide se uma sequencia de pontuacoes e de um torneio e constroi o torneio	Construtivos e interativos	compila
code/general/two_pointers.cpp	Two pointers / janela deslizante (templates)	(1) conta subarrays "bons" quando a propriedade e monotonica (encolher nunca piora)	Arrays e guloso	compila
geometry/				
code/geometry/angle_sort.cpp	Sweep angular: contar pontos em semiplano e triangulos que contem a origem	ordena pontos por angulo em torno da origem (exato, com cross) e usa dois	Geometria	compila
code/geometry/calipers.cpp	Rotating calipers: diametro, largura minima, retangulo envolvente de menor area	sobre um fecho convexo CCW (de hull_andrew.cpp) acha o par mais distante,	Geometria	compila
code/geometry/circles.cpp	Circulos: interseções, tangentes, area de interseção com poligono, area da uniao	circulo-reta, circulo-circulo, tangentes de ponto e comuns, area de interseção	Geometria	compila
code/geometry/closest_pair.cpp	Par de pontos mais proximo (sweep + set)	menor distancia^2 entre dois pontos (e os indices)	Geometria	compila
code/geometry/convex_extreme.cpp	Vertice extremo de poligono convexo numa direcao (busca binaria O(log n))	dado poligono convexo CCW e direcao dir, devolve o indice do vertice que maximiza	Geometria	compila
code/geometry/convex_layers.cpp	Camadas convexas (convex layers / onion peeling) e frente de Pareto 2D	layer[i] = indice da camada do ponto i (0 = fecho externo)	Geometria	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/geometry/convexhull.cpp	Fecho convexo (monotone chain)	convexHull(pontos) devolve o hull em ordem anti-horaria, sem pontos	Geometria	compila
code/geometry/geometry3d.cpp	Geometria 3D basica + fecho convexo 3D (incremental O(n^2))	struct P3 (cross, dot), volume de tetraedro (produto misto), distancia ponto-plano,	Geometria	compila
code/geometry/half_plane.cpp	Interseção de semiplanos (HPI) O(n log n)	acha o poligono convexo (vertices CCW) = interseção de semiplanos "a esquerda de p->p+d"	Geometria	compila
code/geometry/hull_andrew.cpp	Fecho convexo (Andrew monotone chain) - versao correta	devolve os vertices do fecho convexo em ordem anti-horaria, comecando pelo	Geometria	compila
code/geometry/min_circle.cpp	Circulo minimo envolvente (Welzl iterativo) e circuncentro	menor circulo contendo todos os pontos	Geometria	compila
code/geometry/minkowski.cpp	Soma de Minkowski de poligonos convexos	A + B = {a+b}	Geometria	compila
code/geometry/point_dbl.cpp	Ponto/vetor em ponto flutuante (long double) + primitivas	struct Pd com operadores, rotacao, projecao, reflexao, interseção de retas,	Geometria	compila
code/geometry/point_int.cpp	Ponto/vetor inteiro + primitivas exatas	struct P (ll) com cross, dot, orientacao, ordenacao angular exata,	Geometria	compila
code/geometry/polygon.cpp	Poligonos: area, centroide, Pick, ponto em poligono (geral e convexo O(log n)), pontos de rede	area*2 sinalizada (shoelace), centroide, contagem de pontos de rede (Pick),	Geometria	compila
code/geometry/polygon_cut.cpp	Corte de poligono por semiplano (Sutherland-Hodgman)	devolve a parte do poligono que fica a esquerda (ou sobre) da reta orientada a->b	Geometria	compila
code/geometry/rect_union.cpp	Area e perimetro da uniao de retangulos (sweep + segtree)	area da uniao de retangulos alinhados aos eixos	Geometria	compila
code/geometry/seg_any_inter.cpp	Existe interseção entre n segmentos? (Shamos-Hoey, sweep + set)	acha um par de segmentos que se intersectam (inclui toque), ou {-1,-1}	Geometria	compila
graph/				
code/graph/bcc_block_cut.cpp	BCC (componentes biconexas por vertices) e Block-Cut Tree	separa o grafo em blocos (BCCs)	Grafos	compila
code/graph/bellman_ford.cpp	Bellman-Ford (ciclo negativo, -infinito, ciclo explicito)	distancias com pesos negativos	Grafos	compila
code/graph/bfs.cpp	BFS (busca em largura)	percorre o grafo nao ponderado a partir de start, marcando visitados e	Grafos	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
code/graph/bfs01.cpp	0-1 BFS e BFS com pesos pequenos (Dial)	menor distancia com pesos em {0,1} usando deque (bfs01), ou com pesos	Grafos	compila
code/graph/bipartite.cpp	Teste de bipartido / 2-coloracao com ciclo impar	colore o grafo com 2 cores por BFS	Grafos	compila
code/graph/bridges_2ecc.cpp	Pontes, articulacoes e componentes 2-aresta-conexas (bridge tree)	acha pontes e vertices de articulacao	Grafos	compila
code/graph/ciclos.cpp	Deteccao de ciclos: orientado, nao orientado, girth (menor ciclo nao ponderado)	find_cycle_dir devolve um ciclo orientado	Grafos	compila
code/graph/dijkstra.cpp	Dijkstra (menores caminhos de fonte unica, pesos nao negativos)	devolve dist[] com a menor distancia de start a todos os vertices	Grafos	compila
code/graph/dijkstra_estados.cpp	Dijkstra em grafo implicito / de estados (grid, k cupons, camadas)	Dijkstra generico onde os vizinhos sao gerados por uma lambda	Grafos	compila
code/graph/dijkstra_variantes.cpp	Dijkstra: variantes (denso O(n^2), gargalo/minimax, contagem de caminhos, arestas em	dij_dense (sem heap)	Grafos	compila
code/graph/dsu.cpp	DSU (union-find com path compression + union by size)	find(u) com compressao	Grafos	compila
code/graph/euler.cpp	Caminho/circuito euleriano (Hierholzer) - nao direcionado e direcionado	devolve a sequencia de vertices de um circuito/trilha que usa cada aresta	Grafos	compila
code/graph/floyd.cpp	Floyd-Warshall (distancias, ciclo negativo, caminho, ciclo minimo)	todos-os-pares em O(n^3)	Grafos	compila
code/graph/grafo_funcional.cpp	Grafo funcional (cada vertice tem exatamente uma aresta de saida)	acha os ciclos, e para cada vertice: se esta no ciclo, id do ciclo, posicao,	Grafos	compila
code/graph/johnson.cpp	Johnson (todos-os-pares com pesos negativos, esparso)	Bellman-Ford de super-fonte para achar potenciais h, reponderar	Grafos	compila
code/graph/kruskal.cpp	Kruskal: arvore geradora minima	calcula a MST de um grafo nao direcionado e diz se ele e conexo	Grafos	compila
code/graph/kth_walks.cpp	K menores caminhos (walks) de s a t	devolve os k menores comprimentos de walks (vertices/arestas podem repetir)	Grafos	compila
code/graph/range_graph.cpp	Grafo com arestas para intervalos (segtree de arestas) + Dijkstra	permite arestas u -> [l,r] e [l,r] -> v com custo w usando duas arvores	Grafos	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/graph/scc_kosaraju.cpp`	SCC (Kosaraju iterativo)	componentes fortemente conexas	Grafos	compila
`code/graph/scc_tarjan.cpp`	SCC (Tarjan iterativo) + condensacao	componentes fortemente conexas	Grafos	testado (brute, 2026-11-11)
`code/graph/spfa.cpp`	SPFA (Bellman-Ford com fila)	menor caminho com pesos negativos	Grafos	compila
`code/graph/toposort.cpp`	Ordenacao topologica (Kahn)	devolve uma ordem topologica de um DAG	Grafos	compila
`code/graph/two_sat.cpp`	2-SAT	decide satisfatibilidade de clausulas de 2 literais e devolve uma atribuicao	Grafos	testado (brute, 2026-11-11)
math/				
`code/math/bsgs.cpp`	Logaritmo discreto (BSGS) para modulo qualquer	menor x >= 0 com a^x = b (mod m), ou -1 se nao existe	Teoria dos números e divisores	compila
`code/math/crt.cpp`	CRT geral (modulos nao coprimos) e Garner	crt_merge combina x=a1 (m1) e x=a2 (m2) em x=r (lcm)	Teoria dos números e divisores	compila
`code/math/divisores.cpp`	Divisores: listar, formulas, altamente compostos, serie geometrica	divisores de n por sqrt	Teoria dos números e divisores	compila
`code/math/exgcd.cpp`	Euclides estendido, diofantina linear, contagem de solucoes em caixa	exgcd	Teoria dos números e divisores	compila
`code/math/floor_sum.cpp`	Floor sum (algoritmo tipo Euclides)	floor_sum = sum_{i=0}^{n-1} floor((a*i+b)/m) exato (cabe em ll se	Teoria dos números e divisores	compila
`code/math/inv_lote.cpp`	Inversos modulares em lote	inv[1..n] mod p linear	Teoria dos números e divisores	compila
`code/math/miller_rabin.cpp`	Miller-Rabin deterministico (64 bits)	testa primalidade de n < 2^64 de forma exata	Teoria dos números e divisores	compila
`code/math/mint.cpp`	mint: struct de inteiro modular	aritmetica mod MOD (primo) com operadores, pow e inverso	Teoria dos números e divisores	compila
`code/math/mod.cpp`	Aritmetica modular e utilidades de inteiros	gcd, lcm, soma/subtracao/multiplicacao mod MOD, exponenciacao e inverso	Teoria dos números e divisores	compila
`code/math/ntt.cpp`	NTT (Number Theoretic Transform) modulo 998244353	convolucao/multiplicacao de polinomios com raiz primitiva 3	Álgebra, polinômios e FFT	compila
`code/math/pollard_rho.cpp`	Pollard rho (Brent) + fatoracao completa	fatora n < 2^63 (~9e18	Teoria dos números e divisores	testado (brute, 2026-11-11)
`code/math/primitive_root.cpp`	phi, lambda de Carmichael, ordem multiplicativa, raiz primitiva	phi(n) e lambda(n) por trial division	Teoria dos números e divisores	compila
`code/math/sieve.cpp`	Crivo de Eratostenes	marca is_prime[0..MAXN] e guarda todos os primos em primes	Teoria dos números e divisores	compila

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/math/sieve_linear.cpp`	Crivo linear (menor fator primo, phi, mu, tau, sigma)	em O(N) calcula lp[i] (menor fator primo), lista de primos, phi, mu,	Teoria dos números e divisores	compila
`code/math/sieve_seg.cpp`	Crivo segmentado (primos e phi em [L, R])	primos em [L,R] e phi de cada numero do intervalo, usando so os	Teoria dos números e divisores	compila
`code/math/tonelli.cpp`	Raiz quadrada modular (Tonelli-Shanks) e simbolo de Jacobi	x com x^2 = a (mod p), p primo	Teoria dos números e divisores	compila
string/				
`code/string/aho_corasick.cpp`	Aho-Corasick	automato de varios padroes	Strings	compila
`code/string/bitset_match.cpp`	Matching e LCS com bitset	(1) ocorrencias de p em t por bitset (util quando ha variacoes tipo	Strings	compila + testado
`code/string/eertree.cpp`	Eertree (arvore de palindromos) + particao em palindromos (serie link)	no por palindromo distinto (<= n nos)	Strings	compila
`code/string/hash.cpp`	Hash polinomial de string (prefix hash)	StringHash(s) monta o hash dos prefixos	Strings	compila
`code/string/hash2d.cpp`	Hash 2D de submatrizes (mod 2^61-1)	hash de qualquer submatriz em O(1)	Strings	compila
`code/string/hash61.cpp`	Hash polinomial mod 2^61-1 (substrings, LCP, palindromo, concatenacao)	hash de substring em O(1) com base aleatoria	Strings	compila
`code/string/kmp.cpp`	Prefix function (KMP), automato KMP, bordas, periodos	pi[i] = maior borda propria de s[0..i]	Strings	testado (brute, 2026-11-11)
`code/string/lyndon.cpp`	Duval (fatoracao de Lyndon) e rotacao minima	duval(s) fatora s em palavras de Lyndon nao-crescentes	Strings	compila
`code/string/manacher.cpp`	Manacher (palindromos)	d1[i] = nro de palindromos de tamanho impar centrados em i (raio incl	Strings	testado (brute, 2026-11-11)
`code/string/suffix_array.cpp`	Suffix array + LCP + RMQ de LCP	sa (sufixos ordenados), rnk, lcp[i] = LCP(sa[i-1], sa[i]) (lcp[0]=0),	Strings	testado (brute, 2026-11-11)
`code/string/suffix_automaton.cpp`	Suffix automaton (SAM), incluindo generalizado	automato minimo dos sufixos	Strings	testado (brute, 2026-11-11)
`code/string/trie.cpp`	Trie (arvore de prefixos) para minusculas	insert(p) insere a palavra p	Strings	compila + testado
`code/string/zfunc.cpp`	Z-function	z[i] = LCP de s e s[i..]	Strings	testado (brute, 2026-11-11)
tree/				

Arquivo	Nome	Descrição	Seção do catálogo	Status
`code/tree/centroid.cpp`	Centroid decomposition (arvore de centroides + distancias por nivel)	constroi a arvore de centroides (altura <= log n)	Árvores	compila
`code/tree/centroid_paths.cpp`	Contar pares (u<v) com dist(u,v) <= K via centroide (arestas ponderadas >= 0)	para cada centroide junta as distancias da componente, conta pares com	Árvores	compila
`code/tree/hld.cpp`	Heavy-Light Decomposition + segtree sobre monoide (caminho e subarvore)	decompoe a arvore em cadeias pesadas	Árvores	testado (brute, 2026-11-11)
`code/tree/hld_bit.cpp`	HLD com BIT range-add / range-sum (soma e add em caminho e subarvore)	add x em todos os vertices de um caminho / subarvore e soma sobre	Árvores	compila
`code/tree/lca_lifting.cpp`	LCA com binary lifting (+ k-esimo ancestral, distancia, k-esimo no caminho)	pre-processa pai 2^k de cada vertice	Árvores	testado (brute, 2026-11-11)
`code/tree/lca_sparse.cpp`	LCA O(1) com sparse table sobre a ordem de DFS (n posicoes)	LCA em O(1) por query	Árvores	compila
`code/tree/lca_tarjan.cpp`	LCA offline de Tarjan (DFS + DSU)	responde um lote de queries (u,v) de LCA sem tabela de saltos	Árvores	compila